

Técnicas de cálculo para Sistemas de Ecuaciones, Programación Lineal y Programación Entera

SEGUNDA EDICIÓN

JOSÉ LUIS DE LA FUENTE O'CONNOR

EDITORIAL REVERTÉ, S.A.

Técnicas de cálculo para Sistemas de Ecuaciones, Programación Lineal y Programación Entera

Códigos en FORTRAN y C con Aplicaciones de Sistemas de Energía Eléctrica

SEGUNDA EDICIÓN

JOSÉ LUIS DE LA FUENTE O'CONNOR

Profesor Titular de la Universidad Politécnica de Madrid
Escuela Técnica Superior de Ingenieros Industriales



EDITORIAL REVERTÉ, S.A.

Barcelona - Bogotá - Buenos Aires - Caracas - México

Técnicas de Cálculo para Sistemas de Ecuaciones, Programación Lineal y Programación Entera

**Códigos en FORTRAN y C con Aplicaciones de Sistemas de
Energía Eléctrica**

José Luis de la Fuente O'Connor
Profesor Titular
Universidad Politécnica de Madrid
Escuela Técnica Superior de Ingenieros Industriales

A mi familia.

Índice General

Índice General	VII
Índice de Figuras	XXIII
Índice de Tablas	XXV
Prefacio	XXIX
I Sistemas de ecuaciones	1
Capítulo 1. MÉTODOS DIRECTOS DE SOLUCIÓN DE SISTEMAS DE ECUACIONES LINEALES	3
1.1 Planteamiento del problema a resolver	4
1.2 Eliminación de Gauss	7
1.2.1 Pivotación	14
1.2.2 Número de operaciones aritméticas del método	20
1.3 Método de Gauss-Jordan	23
1.4 Descomposición o factorización LU	24
1.4.1 Métodos directos para la obtención de factorizaciones LU	29
1.4.1.1 Método de Crout. Versión LU_1	29
1.4.1.2 Método de Crout. Versión L_1U	34
1.4.1.3 Método de Doolittle	36
1.5 Factorización de matrices simétricas	39
1.5.1 Factorización LDL^T	40
1.5.2 Factorización de Cholesky	41
1.5.3 Matrices simétricas semidefinidas positivas	46
1.5.3.1 Pivotación	48
1.5.4 Matrices simétricas indefinidas	48
1.5.4.1 El método de Parlett y Reid	50
1.5.4.2 El método de Aasen	53
1.5.4.3 Factorización de pivotación diagonal	59
1.5.4.3.1 El método de Bunch y Kaufman	60
1.6 Condicionamiento de sistemas de ecuaciones lineales	66
1.7 Mínimos cuadrados lineales	73
1.7.1 Fundamentos teóricos del problema	74
1.7.1.1 Descomposición en valores singulares	74
1.7.1.2 Sistemas incompatibles. Ecuaciones normales	79
1.7.1.3 Sistemas indeterminados	81

1.7.2	Resolución numérica del problema	81
1.7.2.1	Método de Gram-Schmidt	83
1.7.2.2	Factorización QR o triangularización ortogonal. Transfor- maciones ortogonales	88
1.7.2.2.1	Transformaciones de Householder	90
1.7.2.2.1.1	Resolución numérica de $A\mathbf{x} = \mathbf{b}$, $A^{m \times n}$, $m > n$ y rango completo	94
1.7.2.2.1.2	Resolución numérica de $A\mathbf{x} = \mathbf{b}$, $A^{m \times n}$, $n > m$ y rango completo	98
1.7.2.2.1.3	Resolución numérica de $A\mathbf{x} = \mathbf{b}$, $A^{m \times n}$, $m > n$ ó $m < n$ y rango incompleto	98
1.7.2.2.2	Transformaciones de Givens	105
1.7.2.2.3	Transformaciones rápidas de Givens	110
1.7.3	Descomposición numérica en valores singulares. Método de Golub-Reinsch	115
1.8	El problema generalizado de mínimos cuadrados	128
1.9	Mínimos cuadrados lineales con restricciones lineales	131
1.9.1	Resolución numérica del problema	132
1.9.1.1	Método de eliminación directa	132
1.9.1.2	Método de la base del subespacio núcleo de la matriz de restricciones	137
1.9.1.3	Método de la ponderación	138
Referencias		138
Ejercicios		139

Capítulo 2. MÉTODOS ITERATIVOS DE SOLUCIÓN DE SISTEMAS DE ECUACIONES LINEALES

143

2.1	Método de Jacobi	145
2.2	Método de Gauss-Seidel	149
2.3	Convergencia de los métodos de Jacobi y Gauss-Seidel	152
2.3.1	Matrices generales	152
2.3.2	Matriz de diagonal dominante	156
2.3.3	Matriz simétrica definida positiva	159
2.4	Métodos de relajación	163
2.4.1	Convergencia del método SOR	166
2.4.2	Método SSOR	168
2.5	Métodos de minimización	169
2.5.1	Direcciones de descenso	171
2.5.1.1	Relajación en una variable	171
2.5.1.2	Relajación SOR	172
2.5.1.3	Máxima pendiente	173
2.5.2	Direcciones de descenso conjugadas	177
2.5.2.1	Determinación de direcciones conjugadas	179
2.5.2.2	Determinación de direcciones conjugadas. Método de los gradientes conjugados	179
2.5.2.2.1	Convergencia	182
2.5.2.2.2	Interpretación geométrica del método de los gra- dientes conjugados	186

2.5.2.2.3	Implementación práctica del método de los gradientes conjugados	188
2.5.2.2.4	Método de los gradientes conjugados con preconditionamiento	190
2.6	Comparación numérica de los algoritmos	193
2.7	Mínimos cuadrados y métodos iterativos	194
2.7.1	Método de Jacobi	194
2.7.2	Método de Gauss-Seidel	194
2.7.3	Método de relajación SOR	195
2.7.4	Método de los gradientes conjugados	195
	Referencias	196
	Ejercicios	196

Capítulo 3. SISTEMAS DE ECUACIONES LINEALES DE MATRIZ DE COEFICIENTES

DISPERSA		201
3.1	Almacenamiento en ordenador de matrices dispersas	202
3.1.1	Almacenamiento por coordenadas	202
3.1.2	Almacenamiento por filas o columnas	203
3.1.3	Almacenamiento por perfil o envolvente	204
3.1.4	Almacenamiento por listas encadenadas	207
3.2	Operaciones algebraicas elementales con matrices dispersas	208
3.2.1	Producto interior de dos vectores	208
3.2.2	Multiplicación de matrices por vectores	210
3.2.2.1	Multiplicación de una matriz por un vector	210
3.2.2.2	Multiplicación de un vector por una matriz	210
3.2.3	Suma de matrices dispersas	211
3.2.3.1	Suma o resta simbólica	211
3.2.3.2	Suma o resta numérica	214
3.2.4	Multiplicación de matrices dispersas	215
3.2.4.1	Multiplicación $A^T A$	217
3.3	Solución de grandes sistemas lineales de matriz dispersa	219
3.3.1	Ordenación de las ecuaciones	219
3.3.2	Proceso de solución	225
3.4	Matrices dispersas simétricas y eliminación de Gauss	226
3.4.1	Nociones básicas sobre teoría de grafos	227
3.4.2	Interpretación grafo-teórica de la eliminación de Gauss de matrices dispersas de estructura simétrica	231
3.4.3	El algoritmo de grado mínimo	234
3.4.4	Reducción del ancho de banda de una matriz dispersa simétrica. El algoritmo de Cuthill-McKee	238
3.4.4.1	Selección del nudo inicial	240
3.4.5	Reducción de la envolvente de una matriz dispersa simétrica. El algoritmo inverso de Cuthill-McKee	241
3.4.6	Método de la disección anidada	242
3.4.7	Método de la disección en un sentido	245
3.5	Matrices dispersas no simétricas y eliminación de Gauss	246
3.5.1	Nociones básicas sobre grafos dirigidos	248

3.5.2	Interpretación grafo-teórica de la eliminación de Gauss de matrices dispersas no simétricas	250
3.5.3	Obtención de un transversal completo. Algoritmo de Hall	251
3.5.4	Permutaciones simétricas hacia una estructura triangular en bloques . . .	254
3.5.4.1	Algoritmo de Sargent y Westerberg	256
3.5.4.2	Algoritmo de Tarjan	257
3.5.5	Pivotación en matrices dispersas y eliminación de Gauss	261
3.5.6	Método de los frentes	263
3.6	Problemas de mínimos cuadrados dispersos	266
3.6.1	El método de las ecuaciones normales	267
3.6.1.1	Dispersidad parcial	268
3.6.2	Métodos basados en transformaciones ortogonales. Método de George-Heath	270
3.6.2.1	Ordenación de filas	271
3.6.3	Otros métodos	272
	Referencias	273
	Ejercicios	274

Capítulo 4. SOLUCIÓN DE SISTEMAS DE ECUACIONES NO LINEALES 279

4.1	Velocidad o rapidez de convergencia	281
4.2	Problemas de una variable	284
4.2.1	Método de la bisección	284
4.2.2	Método de Newton-Raphson	286
4.2.3	Convergencia del método de Newton para una variable	291
4.2.4	Variantes del método de Newton	293
4.2.4.1	Método de Newton por diferencias finitas	295
4.2.4.2	Método de Newton modificado	299
4.2.5	Método de la secante	300
4.2.6	Método de la falsa posición	302
4.2.7	Método de Müller	302
4.3	Sistemas de ecuaciones no lineales. Método de Newton-Raphson	306
4.3.1	Convergencia del método de Newton para sistemas de ecuaciones no lineales	310
4.3.2	Modificaciones del método de Newton para sistemas de ecuaciones no lineales	312
4.3.2.1	El método de Newton-Raphson por diferencias finitas para sistemas de ecuaciones no lineales	313
4.3.2.2	Newton modificado	316
4.3.2.3	Jacobi	316
4.3.2.4	Gauss-Seidel	318
4.3.2.5	Relajación SOR	318
4.4	Métodos cuasi Newton	320
4.4.1	Método de Broyden	321
4.4.1.1	Convergencia del método de Broyden	326
4.4.1.2	Implementación práctica del método de Broyden	329
4.5	Métodos globalmente convergentes para sistemas de ecuaciones no lineales	331
4.6	Mínimos cuadrados no lineales	335

4.6.1	Referencias teóricas del problema	342
4.6.2	Resolución numérica del problema	344
4.6.2.1	Método de Gauss-Newton	346
4.6.2.1.1	Convergencia del método de Gauss-Newton	349
4.6.2.2	Métodos de Gauss-Newton globalmente convergentes	351
4.6.2.3	Métodos de región de confianza. Método de Levenberg-Marquardt	352
4.6.2.4	Métodos tipo Newton	358
	Referencias	358
	Ejercicios	359

II Programación lineal 363

Capítulo 5. PROGRAMACIÓN LINEAL. FORMULACIÓN 365

5.1	Conceptos y definiciones generales	366
5.2	Ejemplos de problemas de programación lineal	368
	Referencias	371
	Ejercicios	372

Capítulo 6. TEORÍA BÁSICA DE LA PROGRAMACIÓN LINEAL 379

6.1	Consideraciones geométricas sobre la programación lineal	379
6.1.1	Representación geométrica del programa lineal en el subespacio de bienes	382
6.1.1.1	Factibilidad y condiciones de igualdad	384
6.1.1.2	Factibilidad y condiciones de desigualdad	386
6.1.1.3	Óptimo	386
6.2	Politopos	389
6.3	Puntos extremos y soluciones básicas factibles	391
6.3.1	Teorema de la representación	397
6.3.2	Teorema fundamental de la programación lineal	402
	Referencias	406
	Ejercicios	406

Capítulo 7. EL MÉTODO SIMPLEX 411

7.1	Mejora de una solución básica factible	412
7.2	Finalización. Solución óptima, solución no acotada y soluciones óptimas alternativas	417
7.3	El algoritmo simplex	418
7.3.1	Degeneración y ciclado	428
7.3.1.1	La regla lexicográfica	429
7.3.1.2	La regla de Bland	429
7.4	Solución básica factible inicial	429
7.4.1	Variables artificiales	431
7.4.2	Método de penalización o de la gran M	441
7.5	Implementaciones prácticas del método simplex	441
7.5.1	El método simplex en forma de tableau	442
7.5.2	Forma producto de la inversa de la base	444

7.5.3	Factorización LU de la base	447
7.6	El método simplex para variables acotadas	450
7.7	Complejidad computacional del método simplex	459
	Referencias	460
	Ejercicios	461
Capítulo 8. DUALIDAD Y ANÁLISIS DE SENSIBILIDAD		465
8.1	Dualidad y condiciones de óptimo	466
8.1.1	Condiciones de punto óptimo de Karush-Kuhn-Tucker	475
8.2	Interpretación económica de la dualidad	476
8.3	El algoritmo dual del simplex	479
8.3.1	El algoritmo dual del simplex para variables acotadas	482
8.4	El método primal-dual	486
8.5	Análisis de sensibilidad	492
	Referencias	494
	Ejercicios	494
Capítulo 9. PROGRAMAS LINEALES DE ESTRUCTURA ESPECIAL		499
9.1	Problemas de flujos en redes	499
9.1.1	Conceptos básicos de teoría de grafos	500
9.1.2	Problemas típicos de flujos en redes	502
9.1.3	El método simplex para problemas de flujos en redes	505
9.1.3.1	Implementación práctica del método simplex	512
9.1.3.1.1	Paso 1. Asignación de precios. Comprobación de condiciones de óptimo	512
9.1.3.1.2	Paso 2. Determinación de la columna de pivotación	515
9.1.3.1.3	Paso 3. Determinación de la fila de pivotación. Análisis de ratios	515
9.1.3.1.4	Paso 4. Pivotación. Actualización de las estructu- ras de datos	516
9.1.3.1.4.1	Actualización de $s(\cdot)$	517
9.1.3.1.4.2	Actualización de $p(\cdot)$ y $d(\cdot)$	520
9.1.3.2	Solución básica factible inicial	527
9.2	El principio de descomposición de Dantzig-Wolfe	527
9.2.1	Implementación práctica	535
9.2.2	Problemas con estructura en escalera	545
9.3	El problema del corte de materiales	546
	Referencias	550
	Ejercicios	551
Capítulo 10. MÉTODOS DE PUNTOS INTERIORES		557
10.1	Ideas básicas de los métodos de puntos interiores para programación lineal	558
10.2	El método del escalado proyectivo de Karmarkar	561
10.2.1	Transformación proyectiva en el simplex	562
10.2.2	Complejidad computacional	570
10.3	Variantes y extensiones del método de Karmarkar	571
10.4	El método primal de escalado afín	571

10.4.1	Transformación afín del octante positivo	572
10.4.2	Solución de partida del método	580
10.4.2.1	El método de la gran M	580
10.4.2.2	El método en dos fases	580
10.4.3	Reglas de parada del método	581
10.4.3.1	Factibilidad del programa primal	581
10.4.3.2	Factibilidad del programa dual	581
10.4.3.3	Complementariedad de holguras	582
10.4.4	Complejidad computacional del método primal de escalado afín	582
10.4.4.1	Método del empuje potencial	582
10.4.4.2	Método de función barrera logarítmica	585
10.5	El método dual de escalado afín	587
10.5.1	Ideas básicas del método dual de escalado afín	588
10.5.2	Solución de partida del método	593
10.5.2.1	El método de la gran M	593
10.5.2.2	Método de la condición artificial o del límite superior	593
10.5.3	Reglas de parada del método	594
10.5.4	Mejora de la complejidad computacional del método dual de escala- do afín	594
10.5.4.1	Método de función barrera logarítmica	594
10.6	El método primal-dual	595
10.6.1	Dirección y amplitud de movimiento	598
10.6.1.1	Amplitud de movimiento	600
10.6.2	Ajuste del parámetro de penalización y reglas de parada del método ...	600
10.6.2.1	Reglas de parada del método	601
10.6.3	Solución de partida del método	604
10.6.4	Complejidad computacional del método	606
	Referencias	607
	Ejercicios	608

III Programación entera

611

Capítulo 11. PROGRAMACIÓN LINEAL EN VARIABLES ENTERAS	613
11.1 Formulación y ejemplos de programas lineales en variables enteras	615
11.1.1 Problemas de estructura especial	620
11.1.2 Modelización con variables binarias	622
11.2 Resolución gráfica de programas enteros	623
11.3 Propiedades de la región factible de los programas enteros	624
11.4 Algunas relajaciones de la formulación de programas enteros	625
11.4.1 Relajación lineal	626
11.4.1.1 Generación de desigualdades	627
11.4.2 Relajación lagrangiana	632
11.4.3 Descomposición y separación de costes	633
11.4.4 Descomposición de Benders	633
Referencias	635
Ejercicios	635

Capítulo 12. ALGORITMOS GENERALES DE RELAJACIÓN	639
12.1 El algoritmo de los planos cortantes de Gomory	639
12.1.1 Extensión a programas enteros mixtos	645
12.2 Algoritmos de ramificación y acotamiento o branch and bound	645
12.2.1 Algoritmos de ramificación y acotamiento con relajación lineal	648
12.2.1.1 Criterios de poda o rechazo de ramas del árbol enumerativo . .	649
12.2.1.2 División	650
12.2.1.3 Selección del nudo a estudiar	652
12.2.1.4 Selección de la variable de ramificación	654
12.2.1.4.1 Selección basada en penalizaciones	654
Referencias	664
Ejercicios	664

IV Apéndices **669**

Apéndice A. REPASO DE MATEMÁTICAS: DEFINICIONES, NOTACIONES Y RELACIONES BÁSICAS	671
A.1 Conjuntos	671
A.2 Aplicaciones	672
A.3 Espacios vectoriales	673
A.3.1 Espacios normados	675
A.3.2 Espacios con producto interior	677
A.3.3 Aplicaciones lineales	678
A.4 Matrices	680
A.4.1 Normas de matrices	681
A.4.2 Matrices ortogonales, matrices de permutación y matrices de proyección	683
A.5 Autovalores, valores singulares y formas cuadráticas	685
A.5.1 Autovalores	685
A.5.2 Valores singulares	686
A.5.3 Formas cuadráticas	687
A.6 Topología	691
A.7 Teorema de la proyección	692
A.8 Funciones	693
A.8.1 Condiciones necesarias y suficientes de primer y segundo orden que ha de cumplir un punto mínimo	695
A.9 Conjuntos convexos. Existencia de los hiperplanos separador y soporte	696
Apéndice B. ERRORES DE REDONDEO Y ARITMÉTICA DE PRECISIÓN FINITA	699
B.1 Sistema de numeración en un ordenador de cálculo	699
B.2 Precisión de un ordenador. Errores de redondeo	703
B.3 Aritmética en un ordenador	706
B.3.1 Solución de una ecuación cuadrática	708
B.3.2 Más errores. Una suma de infinitos sumandos	709
Referencias	710

Apéndice C. REDES ELÉCTRICAS: FLUJOS POR SUS ELEMENTOS Y POTENCIAS

INYECTADAS EN SUS NUDOS	711
C.1 Línea	711
C.1.1 Potencias inyectadas en los nudos	712
C.1.2 Flujos de potencia entre los nudos	714
C.2 Transformador	716
C.2.1 Esquema equivalente con el regulador del transformador en el primario	716
C.2.2 Esquema equivalente con el regulador del transformador en el secundario	717
C.2.3 Potencias inyectadas en los nudos	719
C.2.4 Flujos de potencia entre los nudos	720
Referencias	721

Apéndice D. CASUÍSTICA DE PROGRAMACIÓN LINEAL **723**

D.1 Gestión financiera a corto plazo	723
D.1.1 Modelo del problema a optimizar	725
D.1.2 Análisis de sensibilidad	732
D.1.2.1 Cambio en las condiciones de la adquisición del pasivo no crediticio	737
D.1.3 Solución factible inicial	741
D.1.3.1 Análisis de los valores duales de las condiciones	743
D.2 Gestión operativa de una refinería de crudo de petróleo	744
D.2.1 Producción de vapor de agua y electricidad en una refinería de petróleo	745
D.2.2 Modelo del problema a optimizar	751
D.2.3 Formulación del problema	754
D.2.4 Análisis de sensibilidad	761
Referencias	764

Apéndice E. El PROGRAMA BBMI **765**

E.1 Datos del problema. Formato MPS	765
E.1.1 Clave NAME	766
E.1.2 Sección ROWS	767
E.1.3 Sección COLUMNS	768
E.1.3.1 Clave INT	768
E.1.4 Sección RHS	769
E.1.5 Sección RANGES	769
E.1.6 Sección BOUNDS	771
E.2 Parámetros y especificaciones de la resolución	772
E.3 Resultados	773
E.3.1 Programas lineales	773
E.3.2 Programas enteros puros	779
E.3.3 Programas enteros mixtos	783
E.4 Listado de BBMI	791

Apéndice F. EL PROGRAMA CCNET	813
F.1 Ejecución del programa	814
F.2 Datos del problema	814
F.3 Resultados	816
F.4 Listado de CCNET	819
Apéndice G. VERSIONES EN C y FORTRAN 90 DE LOS PROGRAMAS DEL TEXTO	
EN FORTRAN 77	831
G.1 Códigos en C	832
G.1.1 Códigos del capítulo 1	832
G.1.2 Códigos del capítulo 2	840
G.1.3 Códigos del capítulo 3	842
G.1.4 Códigos del capítulo 4	847
G.1.5 Códigos del apéndice B	856
G.1.6 Códigos del apéndice H	856
G.2 Códigos en Fortran 90	858
G.2.1 Códigos del capítulo 1	858
G.2.2 Códigos del capítulo 2	866
G.2.3 Códigos del capítulo 3	868
G.2.4 Códigos del capítulo 4	868
Apéndice H. ESTIMACIÓN DEL NÚMERO DE CONDICIÓN DE MATRICES CUADRA-	
DAS	879
H.1 El estimador de Cline, Moler, Stewart y Wilkinson	880
H.2 El algoritmo de Hager	886
Referencias	889
Apéndice I. SOFTWARE DISPONIBLE EN INTERNET	891
I.1 Software de pago	892
I.2 Software de dominio público	893
Apéndice J. EL SOFTWARE DEL LIBRO	895
Bibliografía	897
Índice de materias	913

Índice de Figuras

1.1	Casos posibles de sistemas de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$ dependiendo del tamaño y rango de la matriz A	5
1.2	Descripción geométrica en dos dimensiones de la resolución de un sistema de ecuaciones lineales	7
1.3	Representación geométrica en el subespacio $\text{Im}(A)$ de dos dimensiones de la resolución de un sistema de ecuaciones lineales	8
1.4	Permutaciones elementales en una matriz triangular inferior	25
1.5	Ilustración del proceso del algoritmo de Doolittle para la factorización LU por columnas de una matriz	39
1.6	Partes ya calculadas y por calcular de la factorización de Cholesky for filas (etapa i) y por columnas (etapa j) de una matriz A	47
1.7	Ilustración del buen y mal condicionamiento de dos sistemas de ecuaciones lineales	68
1.8	Ejemplo de problema de mínimos cuadrados: ajuste de una función a una nube de puntos	74
1.9	Ilustración en dos dimensiones de una transformación lineal de la esfera unidad . .	76
1.10	Descripción geométrica del problema $\min_{\mathbf{x} \in \mathbb{R}^2} \ A\mathbf{x} - \mathbf{b}\ _2$, $A \in \mathbb{R}^{3 \times 2}$	80
1.11	Interpretación geométrica en $\mathbb{R}^3$ del problema $\mathbf{x}^* = \min_{\mathbf{x} \in \mathbb{R}^3} \{\ \mathbf{x}\ _2 : A\mathbf{x} = \mathbf{b}\}$	82
1.12	Descripción geométrica del proceso de ortonormalización de Gram-Schmidt	84
1.13	Representación de la aplicación a $\mathbf{a}$ de la transformación de Householder definida por $\mathbf{w}$	91
1.14	Resultado de aplicar a $\mathbf{x}$ la transformación de Householder que define el vector $(\mathbf{x} - \mathbf{y})/\ \mathbf{x} - \mathbf{y}\ _2$	92
1.15	Factorización de una matriz 6×4 por transformaciones de Householder	92
1.16	Representación de cómo obtener las dos transformaciones de Householder posibles de un vector $\mathbf{a}$	93
1.17	Resultado de la factorización de una matriz $m \times n$ de rango r por transformaciones de Householder	100
1.18	Segundo proceso de transformaciones ortogonales para resolver un problema general de mínimos cuadrados	101
1.19	Segundo proceso de transformaciones ortogonales para resolver un problema general de mínimos cuadrados (continuación)	101
1.20	Ejemplo de una transformación de Givens en el espacio euclídeo tridimensional . .	106

1.21	Proceso de bidiagonalización de una matriz 6×4 mediante transformaciones de Householder	117
2.1	Movimiento a lo largo de un vector dirección de descenso	171
2.2	Minimización en la variable α	172
2.3	Relajación SOR	173
2.4	Proceso de convergencia del método de la máxima pendiente aplicado a una función cuadrática	175
2.5	Interpretación geométrica del método de los gradientes conjugados	187
3.1	Estructura simbólica (simétrica) de una matriz 14×14 antes de proceder a su factorización mediante eliminación de Gauss	220
3.2	Estructura simbólica de la matriz de la figura 3.1 después de proceder a su factorización mediante eliminación de Gauss	220
3.3	Estructura simbólica de la matriz de la figura 3.1 después de proceder a la reordenación de sus filas y columnas mediante el algoritmo de grado mínimo y a su posterior factorización mediante eliminación de Gauss	221
3.4	Matriz 35×35 , de estructura simbólica simétrica, antes y después de reordenar sus filas y columnas con el algoritmo de Cuthill-McKee	223
3.5	Matriz triangular inferior en bloques	223
3.6	Matriz 16×16 , de estructura simbólica no simétrica, antes de reordenar sus filas y columnas para reducirla a una de estructura triangular inferior en bloques	224
3.7	Matriz de la figura 3.6 después de reordenar sus filas y columnas para reducirla a una de estructura triangular inferior en bloques	224
3.8	Patrón de elementos distintos de cero de una matriz simétrica 480×480 y el de su factor L una vez efectuada la factorización LL^T	227
3.9	Patrón de elementos distintos de cero de una matriz simétrica 480×480 ordenada mediante el algoritmo de grado mínimo y el de su factor L una vez efectuada la factorización LL^T	228
3.10	Patrón de elementos distintos de cero de una matriz simétrica 480×480 ordenada mediante el algoritmo de Cuthill-McKee y el de su factor L una vez efectuada la factorización LL^T	228
3.11	Matriz 11×11 de estructura simbólica simétrica y su grafo numerado asociado	229
3.12	Grafo no dirigido de 20 nudos, su estructura de niveles y su correspondiente árbol cociente con numeración monótona	231
3.13	Árbol maximal del grafo de la figura 3.12	232
3.14	Tres primeras etapas de la eliminación de Gauss de una matriz simétrica 11×11 y sus correspondientes grafos de eliminación. Los elementos de relleno se indican mediante el símbolo $\otimes$	233
3.15	Resultado de la eliminación simbólica de Gauss en la matriz de la figura 3.11 mediante grafos de eliminación	234
3.16	Grafo asociado a una matriz 7×7 sobre el que se ilustra el algoritmo de grado mínimo	236
3.17	Matriz 7×7 y su grafo asociado con la numeración resultado del algoritmo de grado mínimo	236
3.18	Grafo donde la renumeración que resultaría de aplicarle el algoritmo de grado mínimo no es la óptima	236

3.19	Grafo de 10 nudos antes y después de aplicarle el algoritmo de Cuthill-McKee, comenzando la numeración en a	239
3.20	Grafo de 10 nudos de la figura 3.19 una vez aplicado el algoritmo de Cuthill-McKee, comenzando la numeración en e	240
3.21	Grafo de 10 nudos de la figura 3.19 al que se le aplica el algoritmo de la tabla 3.6 para determinar qué nudo ha de ser el de partida para el algoritmo de Cuthill-McKee	241
3.22	Ejemplo 3.2	242
3.23	Ejemplo de la adaptación del algoritmo de Cuthill-McKee al grafo de la figura 3.22	243
3.24	Resultado de la aplicación del algoritmo inverso de Cuthill-McKee al grafo de la figura 3.22	243
3.25	Resultado del algoritmo inverso de Cuthill-McKee aplicado el grafo de la figura 3.19	243
3.26	Método de la disección anidada	244
3.27	Método de la disección en un sentido	245
3.28	Matriz no simétrica y su digrafo asociado	249
3.29	Primera etapa de la eliminación de Gauss y su correspondiente digrafo de eliminación de la matriz de la figura 3.28. El elemento de relleno se indica mediante el símbolo $\otimes$	250
3.30	Resultado final de la eliminación de Gauss simbólica de la matriz de la figura 3.28	251
3.31	Algoritmo de Hall para la búsqueda de un transversal completo en una matriz 12×12	253
3.32	Digrafo con dos componentes fuertes	254
3.33	Digrafo de una matriz triangular	256
3.34	Digrafo sobre el que se aplica el algoritmo de Sargent y Westerberg	257
3.35	Digrafo en el que el algoritmo de Sargent y Westerberg presenta dificultades	257
3.36	Ejemplo de digrafo con dos componentes fuertes no triviales	259
3.37	Digrafo de la figura 3.36 una vez reenumerado con el algoritmo de Tarjan	260
3.38	Etapa $k = 3$ de la eliminación de Gauss de una matriz de orden 7	262
3.39	Pieza mecánica mallada para su análisis por elementos finitos	263
3.40	Matriz A después de ensamblados los primeros seis elementos de la figura 3.39	264
3.41	Malla 2×4 y primeras tres filas de la matriz a que da lugar al método de los frentes	266
3.42	Matriz A de un problema no de elementos finitos en el proceso de tratamiento por el método de los frentes	266
3.43	Procesamiento simbólico de la fila 9 de una matriz $A \in \mathbb{R}^{9 \times 8}$ por el algoritmo de George y Heath. Los símbolos $\otimes$ designan los elementos de R_8 involucrados en la eliminación de $\mathbf{a}_9^T$; $\oplus$ los que se crean en esa eliminación	271
4.1	Sistema eléctrico de generación y transporte de 3 nudos, 3 líneas y 2 generadores	282
4.2	Decisiones posibles en la primera iteración del método de la bisección	285
4.3	Proceso de obtención de la solución de $x \operatorname{sen}(x) - 1 = 0$ con el método de la bisección	287
4.4	<i>Telas de araña</i> de $g(x) = (\operatorname{sen}(x))^{1/3}$ y $g(x) = \operatorname{sen}(x)/x^2$	288
4.5	Aproximación lineal de $f(x)$ en $x = x_1$	289
4.6	Obtención de la solución de $x^3 - \operatorname{sen}(x) = 0$ con el método de Newton	290
4.7	Método de Newton aplicado a $f(x) = \arctan(x)$	294
4.8	Método de Newton con mecanismo de salvaguarda	295
4.9	Método de Newton modificado	299

4.10	Método de la secante	301
4.11	Método <i>Regula Falsi</i>	303
4.12	Ejemplo donde los métodos de la secante y <i>regula falsi</i> convergen muy lentamente	303
4.13	Primera aproximación parabólica del método de Muller	304
4.14	Método de Broyden en una variedad lineal	327
4.15	Criterio de Armijo	331
4.16	Red eléctrica IEEE de 30 Nudos	337
4.17	Conjunto típico de medidas para la estimación del estado de un sistema eléctrico	338
4.18	Geometría del ajuste de una función no lineal con un parámetro a dos puntos	345
5.1	Región factible del problema de programación lineal del ejemplo 5.1	367
5.2	Representación gráfica del problema del transporte	370
6.1	Resolución geométrica del problema de programación lineal del ejemplo 6.1	380
6.2	Solución óptima única finita: (a) región factible acotada; (b) región factible no acotada	381
6.3	Soluciones óptimas alternativas: (a) región factible acotada; (b) región factible no acotada	381
6.4	(a) Solución óptima no acotada. (b) Región factible vacía	382
6.5	Conjuntos convexo y no convexo; cono convexo	383
6.6	Interpretación geométrica de la factibilidad de un programa lineal: (a) región factible no vacía; (b) región factible vacía	385
6.7	Regiones factibles del ejemplo 6.2: (a) no vacía; (b) vacía	386
6.8	Programa lineal con condiciones de desigualdad: (a) región factible no vacía; (b) región factible vacía	387
6.9	Descripción geométrica del ejemplo 6.3	388
6.10	Geometría del ejemplo 6.4	388
6.11	Representación del hiperplano $-x_1 + 4x_2 = 11$, y los semiespacios que define	390
6.12	Soluciones básicas/soluciones básicas factibles	393
6.13	Soluciones básicas factibles degeneradas	396
6.14	Direcciones en el politopo del ejemplo 6.7	399
6.15	Puntos y direcciones extremos de un politopo P	399
6.16	Representación de un punto de un politopo (poliedro) como combinación convexa de puntos extremos	401
6.17	Representación del politopo del ejemplo 6.8	403
6.18	Direcciones extremas y óptimo: (a) solución óptima no acotada; (b) óptimo acotado	405
7.1	Solución básica degenerada y dirección no factible	414
7.2	Proceso de mejora de una solución básica factible del problema del ejemplo 7.1	415
7.3	Representación del proceso seguido hasta la solución en el problema del ejemplo 7.2	424
7.4	Problema con solución no acotada del ejemplo 7.3	425
7.5	El algoritmo <i>simplex</i> resolviendo un problema con soluciones óptimas alternativas	427
7.6	Trayectoria seguida en la resolución del ejemplo 7.8 empleando las <i>fases I y II</i> del método <i>simplex</i>	441
7.7	Búsqueda de la solución del problema de Klee y Minty para $n = 2$ y $n = 3$	460
8.1	Geometría de las condiciones de óptimo del ejemplo 8.2	468

8.2	Descripción geométrica de la existencia de un hiperplano separador	473
8.3	El sistema (I) del lema de Farkas no tiene solución. La tiene (II)	474
8.4	El sistema (II) del lema de Farkas no tiene solución. La tiene (I)	474
9.1	Grafo dirigido, o digrafo, de 4 nudos y 6 arcos	500
9.2	Algunas estructuras básicas de un grafo dirigido	501
9.3	Flujo máximo en una red y su formulación como problema de coste mínimo	504
9.4	El problema de la asignación en forma de grafo	505
9.5	Determinación del árbol maximal de una red	507
9.6	Árbol maximal del ejemplo 9.2 con nudo ficticio	509
9.7	Digrafo o grafo correspondiente a los datos de la tabla 9.3	513
9.8	Árbol maximal sobre el que se ilustra el proceso de adaptación del vector $s(\cdot)$ una vez efectuada una iteración del método simplex	518
9.9	Árbol maximal resultante del de la figura 9.8 una vez introducido el arco (3,20) en la base. Sale el (8,9)	519
9.10	Árbol maximal del ejemplo de la tabla 9.3 una vez introducido el arco (7,9) en la base y retirado el (1,8)	521
9.11	Grafo correspondiente al problema del ejemplo 9.4	522
9.12	Árbol maximal de la iteración 1 del ejemplo 9.4	523
9.13	Iteración 1 Paso 2: determinación del camino para encontrar la fila de pivotación	524
9.14	Árbol maximal de la iteración 2	525
9.15	Iteración 2 Paso 2: determinación del camino para encontrar la fila de pivotación	525
9.16	Árbol maximal de la iteración 3 del ejemplo 9.4	526
9.17	(a) Grafo de la figura 9.1 aumentado en el nudo artificial 5 para obtener una solución factible inicial. (b) Árbol maximal inicial	528
9.18	Estructura diagonal por bloques de la matriz del problema 9.6	529
9.19	Politopos X_1 y X_2 que define el problema 9.5	539
9.20	Evolución del valor de la función objetivo del problema del ejemplo 9.5 y del de su límite inferior calculado	545
9.21	Estructura en escalera de una matriz de condiciones	545
9.22	Digrafo del ejercicio 5	553
10.1	Itinerarios hacia el óptimo por el interior y exterior del poliedro que definen las condiciones de un problema	559
10.2	Máxima esfera (circunferencia en este caso) que se puede centrar en a dentro de la región factible $\mathbf{x} \geq \mathbf{0}$	560
10.3	Máximas elipses que se pueden inscribir en a y en b' en la región factible $\mathbf{x} \geq \mathbf{0}$	561
10.4	Esferas de radio mínimo y máximo que pueden circunscribir un simplex e inscribirse en él	562
10.5	Transformación proyectiva del ejemplo 10.1	564
10.6	a. Región factible original; b. Después de la primera transformación proyectiva $\bar{\mathbf{x}}$ se convierte en $\mathbf{e}/n$; c. Después de la segunda transformación	565
10.7	Región factible del problema del ejemplo 10.3	568
10.8	Descripción geométrica de la transformación afín en dos dimensiones	573
10.9	Obtención de la dirección en el subespacio núcleo de A_k	574
10.10	Representación de la idea en la que se basa el método de empuje potencial	583

10.11	Función barrera logarítmica del problema: minimizar $f(x) = 3 - x/2$ sujeta a $x \leq 2$	586
11.1	Función objetivo cóncava del problema de la localización de almacenes	618
11.2	Función de costes de un grupo de una central térmica	619
11.3	Bucles en el problema del representante de comercio	620
11.4	Región factible del problema del ejemplo 11.2	624
11.5	Generación de desigualdades por redondeo entero	628
11.6	Región factible del problema del ejemplo 11.3	629
11.7	Ilustración del ejemplo 11.4 sobre desigualdades disyuntivas	631
11.8	Funciones del ejemplo 11.5 para generar desigualdades válidas	632
12.1	Resolución del problema del ejemplo 12.1 mediante el algoritmo de los planos cortantes de Gomory	644
12.2	División recursiva de una región factible	646
12.3	División recursiva de una región factible de un problema en variables 0 ó 1	647
12.4	División recursiva de la región factible del problema en variables 0 ó 1 del ejemplo 12.2	648
12.5	División, por dicotomía de la variable x_j , en un árbol enumerativo	650
12.6	Dicotomía debida a la existencia de cotas superiores generalizadas	651
12.7	División del árbol enumerativo en tantas ramas como valores enteros puede tomar la variable x_j	651
12.8	Selección de los nudos de un árbol enumerativo de acuerdo con la regla LIFO	653
12.9	Árbol enumerativo del problema del ejemplo 12.3	661
12.10	Región factible y árbol enumerativo del problema del ejemplo 12.4	665
A.1	Representación gráfica de la regla del triángulo	675
A.2	Gráfica de una de las funciones de una sucesión de Cauchy	677
A.3	Efecto de una aplicación lineal sobre la bola unidad para diferentes normas	684
A.4	Representación en dos dimensiones de una transformación lineal de la esfera unidad	687
B.1	Conjunto F de números reales representables en un ordenador con $\beta = 2$, $t = 3$, $L = -1$ y $U = 2$	700
C.1	Esquema en Π de una línea entre dos nudos i y j	712
C.2	Transformador entre los nudos i y j	716
C.3	Esquema en Π del transformador entre i y j con el regulador conectado a i	718
C.4	Transformador entre i y j	718
C.5	Esquema en Π del transformador entre i y j con el regulador conectado a j	719
D.1	Proceso productivo simplificado de una refinería de crudo de petróleo	744
D.2	Esquema productivo de vapor de agua de una refinería de crudo de petróleo	746
D.3	Esquema productivo de las turbinas de vapor de la refinería	747
D.4	Fluidos que se consumen y producen en la unidad de producción número 11 y esquema de flujos energéticos en la refinería	750
E.1	Estructura de elementos distintos de cero de un programa entero mixto para prueba de BBMI	784

J.1	Representación de la disposición del <i>software</i> del libro que se incluye en el CD-ROM que se adjunta al mismo	896
-----	---	-----

Índice de Tablas

1.1	Algoritmo para la resolución de $A\mathbf{x} = \mathbf{b}$ mediante eliminación de Gauss con pivotación parcial	16
1.2	Algoritmo para la factorización LU_1 de una matriz $A^{n \times n}$ por el método de Crout .	30
1.3	Algoritmo de Crout con pivotación parcial para la factorización LU_1 de una matriz $A^{n \times n}$	33
1.4	Algoritmo para la factorización L_1U de una matriz $A^{n \times n}$ por el método de Crout .	36
1.5	Algoritmo para la factorización L_1U de una matriz $A^{n \times n}$ por el método de Doolittle. Los coeficientes de los factores se generan por columnas	37
1.6	Algoritmo para la factorización LDL^T de una matriz $A^{n \times n}$ simétrica	41
1.7	Algoritmo para la factorización G^TG de Cholesky por filas de una matriz $A^{n \times n}$ simétrica definida positiva	44
1.8	Algoritmo para la factorización G^TG de Cholesky por columnas de una matriz $A^{n \times n}$ simétrica definida positiva	46
1.9	Variante del algoritmo de Cholesky de la tabla 1.7 para matrices $A^{n \times n}$ simétricas semidefinidas positivas. Sin pivotación	48
1.10	Algoritmo para la factorización G^TG de Cholesky de una matriz $A^{n \times n}$ simétrica semidefinida positiva con pivotación	49
1.11	Algoritmo de Aasen sin pivotación para la factorización LTL^T de una matriz $A^{n \times n}$ simétrica indefinida	55
1.12	Algoritmo de Aasen con pivotación para la factorización LTL^T de una matriz $A^{n \times n}$ simétrica indefinida	56
1.13	Operaciones de la pivotación en el método de Bunch y Kaufman	62
1.14	Algoritmo para la factorización UBU^T de una matriz $A^{n \times n}$ simétrica indefinida por el método de Bunch y Kaufman con pivotación	63
1.15	Algoritmo clásico de Gram-Schmidt para la ortonormalización de los vectores columna de una matriz $A^{m \times n}$	84
1.16	Algoritmo modificado de Gram-Schmidt para la ortonormalización de los vectores columna de una matriz $A^{m \times n}$	86
1.17	Algoritmo modificado de Gram-Schmidt para la ortonormalización de los vectores columna de una matriz $A^{m \times n}$. Versión por filas	86
1.18	Algoritmo para la resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \ A\mathbf{x} - \mathbf{b}\ _2$ por transformaciones de Householder	95
1.19	Algoritmo para la resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \ A\mathbf{x} - \mathbf{b}\ _2$ mediante transformaciones de Givens	108

1.20	Cálculo de los elementos de las filas i y j de las matrices D y P en las transformaciones rápidas de Givens	112
1.21	Algoritmo para la resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \ \mathbf{Ax} - \mathbf{b}\ _2$ por transformaciones rápidas de Givens	113
1.22	Algoritmo de Golub-Kahan: etapa k del procedimiento de Golub-Reinsch para obtener los valores singulares de una matriz bidiagonal $B^{n \times n}$	121
1.23	Algoritmo de Golub-Reinsch para la obtención de los valores singulares de una matriz $A \in \mathbb{R}^{m \times n}$	122
1.24	Número de operaciones necesarias para efectuar las distintas variantes de una descomposición en valores singulares de una matriz $A \in \mathbb{R}^{m \times n}$	127
1.25	Número de operaciones necesarias para resolver el problema de mínimos cuadrados $\min_{\mathbf{x} \in \mathbb{R}^n} \ \mathbf{Ax} - \mathbf{b}\ _2$ por distintos métodos	129
2.1	Algoritmo de Jacobi para la resolución de $\mathbf{Ax} = \mathbf{b}$	147
2.2	Algoritmo de Gauss-Seidel para la resolución de $\mathbf{Ax} = \mathbf{b}$	150
2.3	Algoritmo de relajación SOR para la resolución de $\mathbf{Ax} = \mathbf{b}$	165
2.4	Algoritmo de la máxima pendiente para resolver $\mathbf{Ax} = \mathbf{b}$	176
2.5	Algoritmo de los gradientes conjugados para resolver $\mathbf{Ax} = \mathbf{b}$	188
2.6	Proceso de convergencia de la resolución de un sistema de ecuaciones lineales mediante el método de los gradientes conjugados	190
2.7	Algoritmo de los gradientes conjugados con preconditionamiento para resolver $\mathbf{Ax} = \mathbf{b}$	191
2.8	Resultados obtenidos por diversos métodos iterativos para resolver un problema lineal mal condicionado de 50 ecuaciones con 50 incógnitas	193
2.9	Algoritmo de los gradientes conjugados para resolver $A^T(\mathbf{b} - \mathbf{Ax})$	196
3.1	Algoritmo para resolver sistemas de ecuaciones lineales $\mathbf{Ax} = \mathbf{b}$, siendo A dispersa	221
3.2	Número de operaciones a realizar con diversas variantes de la matriz de la figura 3.1 para, utilizando eliminación de Gauss, resolver un sistema de ecuaciones lineales	222
3.3	Algoritmo de grado mínimo	235
3.4	Ejemplo de aplicación del algoritmo de grado mínimo	237
3.5	Algoritmo de Cuthill-McKee	238
3.6	Algoritmo para determinar un nudo pseudoperiférico en un grafo (para obtener el nudo de partida del algoritmo de Cuthill-McKee)	241
3.7	Pasos y camino trazado para reenumerar el digrafo de la figura 3.33	256
3.8	Pila correspondiente al digrafo de la figura 3.34	258
3.9	Pila correspondiente al digrafo de la figura 3.36	259
3.10	Algoritmo de Tarjan	260
3.11	Algoritmo para resolver mínimos cuadrados con matrices dispersas mediante las ecuaciones normales	269
3.12	Algoritmo de ortogonalización dispersa de George y Heath	272
4.1	Convergencia de diversas sucesiones escalares	283
4.2	Convergencia del método de la bisección aplicado a $x \operatorname{sen}(x) - 1 = 0$	286
4.3	Convergencia del método de Newton modificado aplicado a $x^3 - \operatorname{sen}(x) = 0$	300
4.4	Algoritmo de Newton-Raphson para sistemas de ecuaciones no lineales	307

4.5	Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton-Raphson	310
4.6	Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton-Raphson por diferencias finitas	316
4.7	Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton, variante Jacobi	318
4.8	Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton, variante SOR	320
4.9	Algoritmo cuasi Newton con la fórmula de Broyden para la solución de sistemas de ecuaciones no lineales	323
4.10	Proceso de convergencia a la solución del problema del ejemplo 4.5 con el método cuasi Newton basado en la fórmula de Broyden	326
4.11	Algoritmo de Newton para sistemas de ecuaciones no lineales con el criterio de salvaguarda de Armijo	333
4.12	Proceso de convergencia a la solución del sistema de ecuaciones no lineales del ejemplo 4.6 con el método de Newton y el criterio de Armijo	336
4.13	Parámetros del problema de la figura 4.17	340
4.14	Algoritmo de Gauss-Newton para resolver problemas no lineales de mínimos cuadrados	346
4.15	Método de Gauss-Newton. Proceso de convergencia a la solución del problema del ejemplo 4.8	349
4.16	Algoritmo de Levenberg-Marquart para resolver problemas no lineales de mínimos cuadrados	353
4.17	Datos del problema no lineal de mínimos cuadrados del ejemplo 4.9	354
4.18	Método de Levenberg-Marquardt. Proceso de convergencia a la solución del problema del ejemplo 4.9	357
5.1	Parámetros del problema de la planificación de la generación de energía de una empresa eléctrica	371
6.1	Bases y soluciones básicas del poliedro del ejemplo 6.5	394
7.1	El algoritmo <i>simplex</i> revisado (comienza a partir de una solución factible)	420
7.2	El método <i>simplex</i> en sus dos fases	432
7.3	Algoritmo <i>simplex</i> revisado en la forma producto de la inversa de la base	446
7.4	Algoritmo <i>simplex</i> revisado para variables acotadas	454
8.1	Combinaciones posibles primal-dual	471
8.2	Algoritmo dual del <i>simplex</i>	481
8.3	Algoritmo dual del <i>simplex</i> para variables acotadas	483
8.4	Algoritmo primal-dual	489
9.1	Algoritmo para la obtención de un árbol maximal de un grafo dirigido	507
9.2	Algoritmo para la triangularización de una base	510
9.3	Estructura de datos del grafo de la figura 9.7	513
9.4	Algoritmo para la obtención de los multiplicadores <i>simplex</i> en el algoritmo <i>simplex</i> para flujos en redes	514

9.5	Algoritmo para la actualización del vector $s(\cdot)$ en el método <i>simplex</i> especializado para optimización de flujos en redes	520
9.6	Estructura de datos del árbol de la figura 9.10	522
9.7	Algoritmo de descomposición de Dantzig-Wolfe	538
9.8	Resultado del problema del ejemplo 9.6	551
10.1	Algoritmo de Karmarkar	567
10.2	Algoritmo primal de escalado afín	578
10.3	Algoritmo dual de escalado afín	591
10.4	Proceso de convergencia del algoritmo dual de escalado afín aplicado al ejemplo 10.5	592
10.5	Algoritmo primal-dual de puntos interiores	602
10.6	Proceso de convergencia del algoritmo primal-dual de puntos interiores aplicado al ejemplo 10.6	604
12.1	Algoritmo general para programas enteros basado en relajaciones sucesivas	640
12.2	El algoritmo de los planos cortantes de Gomory	643
12.3	El algoritmo de ramificación y acotamiento o <i>branch and bound</i>	649
A.1	Forma de la bola unidad para diferentes normas en $\mathbb{R}^2$	676
B.1	Parámetros de la aritmética de precisión finita de diversas máquinas	701
D.1	Costes unitarios de la compra o venta de valores o productos financieros	725
D.2	Balance equilibrado a partir del cual se obtiene una solución factible inicial del problema de la gestión financiera a corto plazo	742
D.3	Producción/consumo horario de agua, vapor de agua y condensados de las diversas unidades de producción de la refinería	748
D.4	Requisitos horarios de energía eléctrica y combustibles en las distintas unidades de producción, y consumos de vapor y potencias de las turbinas	749
D.5	Entalpías en kcal/kg de los diversos fluidos de vapor de agua de la refinería	752
D.6	Soluciones óptimas de los diversos modelos del problema de la refinería	762
E.1	Especificaciones numéricas de un problema de dieta alimenticia como el introducido en el capítulo 5	775
F.1	Segundos de <i>c.p.u.</i> invertidos en una estación de trabajo HP APOLLO 9000 730 para resolver diversos problemas de optimización en redes	817
H.1	Algoritmo para la estimación del número de condición $\kappa_1(T)$ de una matriz triangular superior	881
H.2	El algoritmo de Hager para estimar el número de condición 1 de una matriz A	887

Prefacio

El contenido de este libro tiene que ver fundamentalmente con la tecnología hoy en día disponible de lo que en sentido amplio se conoce como análisis numérico o cálculo numérico. Por precisar un poco más, se refiere a aquellas técnicas y procedimientos de cómputo que abordan los problemas de resolver sistemas de ecuaciones lineales y no lineales, programas lineales (también denominados problemas de programación lineal) y programas enteros (programas lineales donde algunas o todas las variables están restringidas a tomar valores enteros). Constituye la tercera edición impresa de un esfuerzo tendente a estudiar los problemas mencionados con cierta profundidad y a transmitir a los lectores las experiencias que de ello se han derivado en los últimos tiempos. El precedente más cercano es el publicado en 1993 en esta misma editorial bajo el título *Tecnologías Computacionales para Sistemas de Ecuaciones, Optimización Lineal y Entera*, que mereció el honor de ser designado Premio José Morillo y Farfán por la *Fundación F²I²* del Ministerio de Industria y Energía y la Universidad Politécnica de Madrid.

Aun cuando los ejemplos y la casuística que se abordan en el libro con más énfasis son los que se suscitan en la modelización, simulación y optimización de sistemas de energía eléctrica de generación, transporte y distribución, los métodos, técnicas y algoritmos que se estudian son universalmente aplicables. Si se utilizan como banco de pruebas los problemas que se mencionan, es porque la experiencia profesional no académica del autor se ha desarrollado fundamentalmente en el sector energético-eléctrico (primero en Hidroeléctrica Española, después en Iberdrola), donde surgen con asiduidad.

El libro tiene un carácter esencialmente práctico. Antes de abordar un procedimiento o algoritmo de cálculo para resolver un problema, se estudian con rigor los fundamentos teóricos de lo que se va a proponer, el porqué es ventajoso hacerlo de una u otra manera y cuáles son los resultados que cabe esperar de las operaciones que hay que llevar a cabo. En la gran mayoría de los casos, a todo lo expuesto le acompaña un programa de ordenador, codificado en FORTRAN 77 ó 90 y C, el cual se incluye en un CD-ROM que se adjunta al libro, con el fin de que el lector pueda constatar la utilidad de lo expuesto y aplicarlo a algún problema concreto si es el caso. Cuando la complejidad del algoritmo no aconseja listar su codificación por ser excesivamente larga, se indican cuáles son las mejores librerías de *software* donde se pueden recabar versiones adecuadas o aquellas direcciones de INTERNET donde se distribuyen programas similares.

Los algoritmos que se listan en las tablas correspondientes utilizan como vehículo de expresión un lenguaje muy similar al del *software* MATLAB. Éste, ya en su versión 5.0, constituye sin duda uno de los instrumentos más usados y referenciados para ensayar, diseñar o incluso codificar profesionalmente procedimientos numéricos y algoritmos. Una recomendación que osamos hacer al lector interesado en los asuntos que trata este libro es que estudie en él los

fundamentos teóricos de los procedimientos que le interesen y su funcionamiento, y que si en el futuro necesita de su concurso para cualesquiera sean las oportunidades, utilice el *software* que se incluye en el libro o acuda a MATLAB, pues aquí encontrará tratados de forma compacta muchas de las posibilidades que hoy en día se ofrecen numéricamente para resolver problemas como los que aborda el libro. Una alternativa aceptable a MATLAB es MATHEMATICA. En cualquiera de los casos, si de lo que se trata es construir un programa que resuelva de forma robusta un problema de características profesionales, lo mejor siempre es diseñar su esqueleto trozo a trozo, con herramientas como las que propone el libro, o proporcionan MATLAB o MATHEMATICA, y luego codificar de forma óptima lo que se sabe que funciona en un lenguaje como FORTRAN 90 o C, ahorrándose el tratamiento de casuística no necesaria.

El libro se ha escrito con la intención de dirigirse a personas que deseen poner al día sus conocimientos sobre técnicas de cálculo en ordenador para resolver los problemas que habitualmente surgen de la modelización matemática de sistemas físicos, técnicos, económicos o sociales: concretamente cuando se obtienen de ellos sistemas de ecuaciones lineales y no lineales, de pequeño y gran tamaño, problemas de programación lineal y problemas de programación entera o mixtos lineales-enteros, también de cualquier dimensión. También está dirigido a alumnos de cursos avanzados de ingeniería, licenciatura, o incluso doctorado, como libro de texto. En la Escuela Técnica Superior de Ingenieros Industriales de Madrid este libro se utiliza como texto oficial de la asignatura *Matemáticas de la Especialidad Electricidad-Electrotecnia*, en cuarto curso de la carrera.

Cómo estudiar el libro como texto

La primera parte, *Sistemas de ecuaciones*, puede constituir gran parte del programa de un curso cuatrimestral sobre técnicas de cálculo numérico para resolver sistemas de ecuaciones lineales y no lineales. Además de los tradicionales y más novedosos procedimientos para llegar a la solución numérica de sistemas en los que la matriz de coeficientes, o la matriz Jacobiana correspondiente, se guarda y estudia en su totalidad, en esta primera parte también se estudian los algoritmos necesarios para resolver sistemas de matrices dispersas. En este sentido se abordan todos los problemas anejos que esto representa: la reordenación de las ecuaciones, las operaciones elementales con matrices dispersas, etc.

La segunda y tercera partes del libro, enfocadas dentro de lo que se conoce como técnicas de optimización y dedicadas a la programación lineal y a la programación entera, pueden conformar el programa idóneo de un curso cuatrimestral sobre técnicas básicas y avanzadas de métodos y algoritmos de optimización lineal (programación lineal y entera). Lo incluido en estas partes del libro son los procedimientos más modernos y fiables para resolver programas lineales y enteros, cualesquiera sean sus dimensiones. Además de todas las variantes más utilizadas del método *simplex*, se estudian en profundidad los algoritmos de puntos interiores más extendidos: el primal y el dual de escalado afín y los primal-dual. Estos últimos permiten, con una sustancial ventaja respecto del *simplex*, resolver problemas de programación lineal de muy grandes dimensiones en tiempos polinómicos.

Agradecimientos

El producto final que representa este libro ha sido posible gracias al apoyo consciente o inconsciente de varias instituciones y particulares. La experiencia profesional de más de 20 años en Iberdrola, qué duda cabe, es el hilo conductor que ha permitido plasmar en muchos apartados

de él conocimientos, técnicas y formación. Mi aportación docente a la Universidad Politécnica de Madrid, más concretamente a la Escuela Técnica Superior de Ingenieros Industriales, mi auténtica segunda casa, durante más de 10 años, me ha enriquecido científicamente muy notablemente a la vez que permitido conocer a un gran número de excelentes profesores de los que he aprendido mucho. También deseo agradecer a la Editorial Reverté la oportunidad que me brindó en su momento para poder publicar esta obra en su prestigiosa empresa.

Nada de lo que se puede leer en estas páginas hubiese sido posible sin la existencia de los modernos procesadores de texto científicos $\text{\LaTeX}$ y $\text{\TeX}$, al igual que la facilidad que en los últimos años ha supuesto poder acceder a los mismos y a otros muchos recursos a través de INTERNET. La comunidad científico-tecnológica está de enhorabuena con la amplia difusión que ésta está experimentando en todos los ámbitos de la sociedad.

José Luis de la Fuente O'Connor
Madrid, Junio de 1997.

Primera parte

Sistemas de ecuaciones

Capítulo 1

MÉTODOS DIRECTOS DE SOLUCIÓN DE SISTEMAS DE ECUACIONES LINEALES

ABORDAMOS EN ESTE capítulo uno de los problemas básicos del álgebra lineal numérica y de muchos procesos de la ingeniería y de la ciencia: la solución de sistemas de ecuaciones lineales. Muchos *algoritmos* —métodos o procedimientos numéricos esencialmente orientados a su implementación en un ordenador— que buscan dar solución numérica a un determinado *modelo matemático* —resultado de la representación formal del comportamiento de los elementos o procesos que definen o integran un proyecto, fenómeno o actividad—, deben resolver sistemas de ecuaciones lineales de mayor o menor tamaño. Ejemplos simples los constituyen la determinación de las tensiones en unos nudos de una red eléctrica de corriente continua mediante las leyes de Kirchhoff, o la evaluación de las tensiones mecánicas en las vigas que definen una estructura reticulada.

La resolución de un sistema de ecuaciones lineales aparece también con mucha frecuencia como un subproblema de un problema más complicado de análisis numérico; tal ocurre por ejemplo cuando se resuelve iterativamente un sistema de ecuaciones no lineales por el método de Newton-Raphson, donde en cada etapa de ese proceso iterativo se requiere resolver un sistema de ecuaciones lineales, o en procesos de optimización tanto lineales como no lineales.

Los sistemas de ecuaciones presentan con frecuencia una estructura muy especial que puede ser objeto de tratamiento particular. Por ejemplo, los problemas de interpolación polinomial, que conducen de manera natural a sistemas de ecuaciones con una matriz de coeficientes de Vandermonde, o los problemas derivados de la modelización de series temporales, que conducen a sistemas de ecuaciones en los que la matriz de coeficientes son del tipo de las denominadas de Toeplitz. Algunos problemas lineales de ajuste de parámetros por mínimos cuadrados también conducen a sistemas de ecuaciones lineales con matrices simétricas definidas positivas, etc.

La resolución de un sistema de ecuaciones lineales desde el punto de vista teórico no tiene ninguna dificultad conceptual; llevarlo a la práctica, sí. Esto es debido a que los sistemas a resolver son frecuentemente de un tamaño considerable y, esencialmente, al hecho de que en el entorno físico en que se resuelven la aritmética¹ opera con precisión finita, lo que introduce errores de redondeo en todas las operaciones efectuadas, amén de que cualquier singularidad puede acarrear, si no se prevé, consecuencias no deseadas.

En lo que sigue nos dedicamos a estudiar los *métodos directos* para dar solución numérica a los sistemas de ecuaciones lineales. Estos métodos proporcionan la respuesta al problema en un número fijo de pasos; la bondad de la solución que obtienen sólo se ve afectada por los errores de redondeo del sistema de numeración en coma flotante de la máquina u ordenador que lleva a efecto esos pasos.

1.1 Planteamiento del problema a resolver

El problema que se plantea es la solución de *sistemas de ecuaciones lineales* del tipo

$$\begin{array}{ccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \cdots & + & a_{2n}x_n & = & b_2 \\ \vdots & & \vdots & & & & \vdots & & \vdots \\ a_{m1}x_1 & + & a_{m2}x_2 & + & \cdots & + & a_{mn}x_n & = & b_m, \end{array}$$

lo que significa determinar los valores de las variables $x_1, \dots, x_n$ que hacen que se cumplan las igualdades. A los números a_{ij} se les denomina *coeficientes del sistema* y a los b_i *términos independientes*.

Si se introducen las matrices

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad \text{y} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix},$$

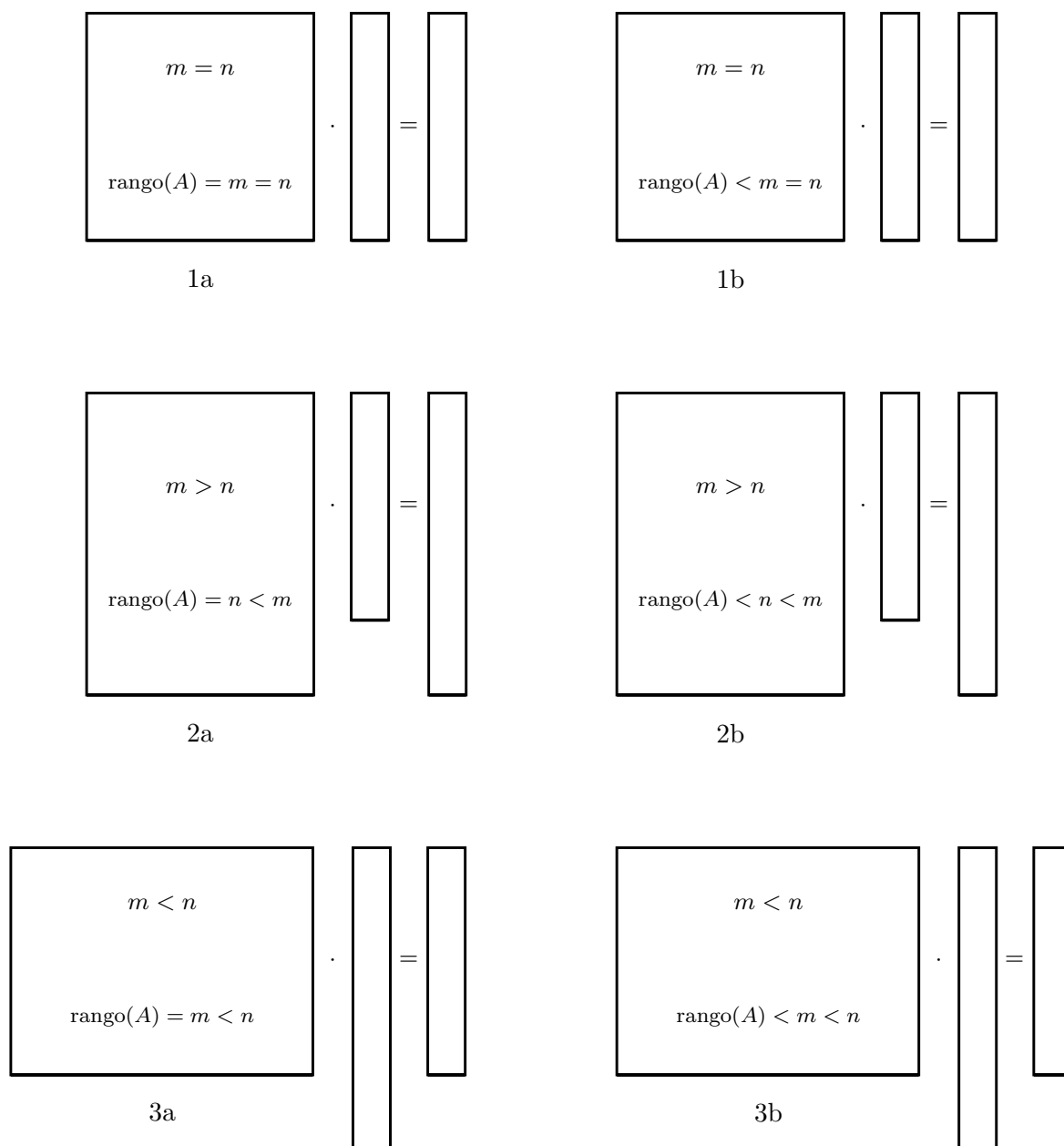
el sistema se puede representar de forma más compacta por

$$A\mathbf{x} = \mathbf{b}.$$

En general se supondrá que la *matriz de coeficientes* $A \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$ y $\mathbf{b} \in \mathbb{R}^m$. Los casos posibles que se pueden presentar con este planteamiento del problema, según las dimensiones y rango de la matriz A , son los de la figura 1.1. El caso 1a, la matriz A es cuadrada regular, es el que estudiaremos inmediatamente; los demás los abordaremos posteriormente dentro del contexto de los denominados problemas de mínimos cuadrados.

Antes de proseguir, recordemos brevemente algunos resultados importantes de álgebra lineal referidos a los sistemas objeto de nuestro interés.

¹En el apéndice B se estudia la aritmética con precisión finita de los ordenadores y sus consecuencias en los procesos de cálculo numérico.

**Figura 1.1**

Casos posibles de sistemas de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$ dependiendo del tamaño y rango de la matriz A

Teorema 1.1 (*Compatibilidad de un sistema de ecuaciones lineales*) La ecuación $A\mathbf{x} = \mathbf{b}$ admite solución si y sólo si

$$\text{rango}(A|\mathbf{b}) = \text{rango}(A).$$

Corolario 1.1 Si $A^{m \times n}$ tiene rango m , $A\mathbf{x} = \mathbf{b}$ siempre tiene solución.

Teorema 1.2 Si $\mathbf{x}_0$ es una solución de $A\mathbf{x} = \mathbf{b}$, el conjunto de soluciones de la ecuación está dado por $\mathbf{x}_0 + \ker(A)$.

Corolario 1.2 Una solución de $A\mathbf{x} = \mathbf{b}$ es única si y sólo si $\ker(A) = \emptyset$.

Teorema 1.3 La ecuación $A\mathbf{x} = \mathbf{0}$, $A^{m \times n}$, $n > m$, siempre tiene una solución no trivial.

Teorema 1.4 Si A es una matriz cuadrada de orden n , las siguientes condiciones son equivalentes:

1. $\text{rango}(A) = n$.
2. $\ker(A) = \emptyset$.
3. Los vectores columna de A son linealmente independientes.
4. Los vectores fila de A son linealmente independientes.
5. Existe una matriz de orden n , A^{-1} , tal que

$$A^{-1}A = AA^{-1} = I.$$

Interpretemos geoméricamente el problema de resolver en dos dimensiones un sistema de ecuaciones lineales cualquiera

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 &= b_1 \\ a_{21}x_1 + a_{22}x_2 &= b_2. \end{aligned}$$

Cada una de las ecuaciones que componen el sistema representa una recta del plano euclídeo según se describe en la figura 1.2. La resolución del sistema tiene como objetivo la búsqueda de las coordenadas del punto donde se cortan esas dos rectas.

Generalizando a $\mathbb{R}^n$, la resolución de un sistema de ecuaciones lineales se puede interpretar como la búsqueda de las coordenadas del punto(s) de intersección de los hiperplanos asociados a cada una de las ecuaciones.

También es posible interpretar geoméricamente el problema en términos de vectores en el subespacio $\text{Im}(A)$. Si escribimos el sistema anterior de dos ecuaciones con dos incógnitas de la forma

$$\begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} x_1 + \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix} x_2 = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix},$$

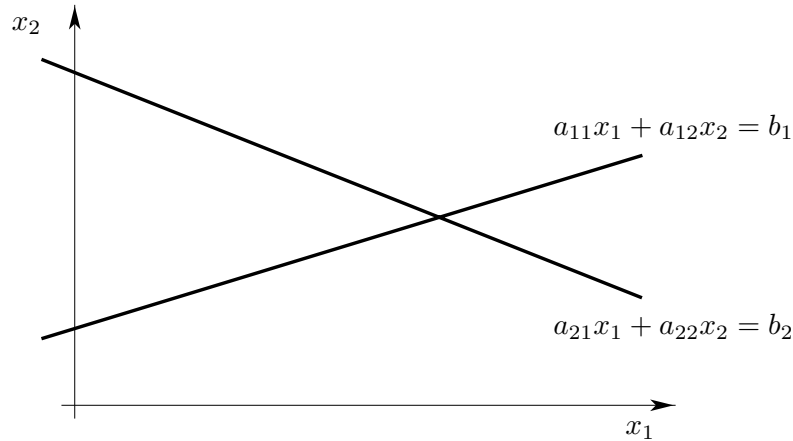


Figura 1.2

Descripción geométrica en dos dimensiones de la resolución de un sistema de ecuaciones lineales

el problema es el de descomponer linealmente el vector $\mathbf{b}$ en los vectores columna que definen las dos columnas de la matriz de coeficientes. En la figura 1.3 se representa esta situación. En $\mathbb{R}^n$ el problema se refiere a la búsqueda de la descomposición lineal de un vector de n componentes según n vectores dados.

1.2 Eliminación de Gauss

Comenzamos el estudio de los procedimientos numéricos directos para resolver el sistema $A\mathbf{x} = \mathbf{b}$, $A \in \mathbb{R}^{n \times n}$, $\mathbf{x} \in \mathbb{R}^n$ y $\mathbf{b} \in \mathbb{R}^n$, con el método por excelencia del álgebra lineal numérica: la eliminación de Gauss. Supondremos que la matriz A es de rango completo, por lo tanto invertible, y que si eventualmente no lo es el procedimiento deberá detectarlo.

El método, aunque varios autores anteriores (Lagrange, Leibniz, Bézout y otros) ya habían investigado sobre el mismo, se atribuye a Carl Friedrich Gauss (1777-1855) quien lo aplicó por primera vez en 1809 con motivo de unos estudios sobre órbitas de ciertos cuerpos celestes.

La idea en la que se basa el método es muy sencilla: aplicar al sistema

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ \vdots & \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

una serie de transformaciones lineales de tal forma que al final de n pasos se haya transformado en uno mucho más fácil de resolver: concretamente, en un *sistema lineal triangular superior* de la forma

$$\begin{aligned} u_{11}x_1 + u_{12}x_2 + \cdots + u_{1n}x_n &= b'_1 \\ u_{22}x_2 + \cdots + u_{2n}x_n &= b'_2 \\ &\vdots \\ u_{nn}x_n &= b'_n, \end{aligned}$$

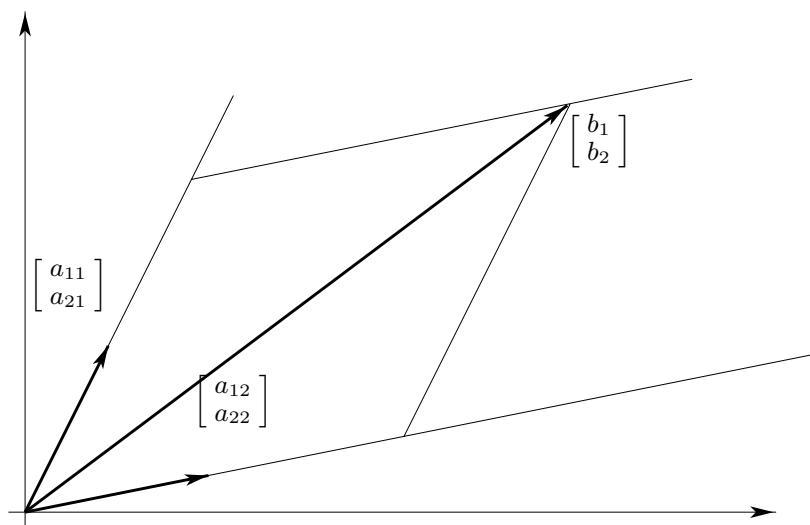


Figura 1.3

Representación geométrica en el subespacio $\text{Im}(A)$ de dos dimensiones de la resolución de un sistema de ecuaciones lineales

o, escrito en forma matricial,

$$U\mathbf{x} = \mathbf{b}'.$$

Todo ello tratando de evitar el cálculo de la inversa A^{-1} , lo que comporta, como veremos más adelante, un número de operaciones significativamente mayor.

Un sistema triangular superior, siempre y cuando se satisfagan las condiciones

$$u_{ii} \neq 0, \quad i = 1, \dots, n,$$

es fácilmente resoluble de manera recurrente mediante las fórmulas

$$x_k = \frac{1}{u_{kk}} \left(b'_k - \sum_{i=k+1}^n u_{ki} x_i \right), \quad k = 1, \dots, n.$$

Este proceso se conoce como *sustitución inversa* o hacia atrás.

La eliminación de Gauss convierte un sistema de ecuaciones lineales cualquiera en uno triangular superior equivalente mediante una sucesión de etapas, cada una de las cuales comporta las siguientes operaciones fundamentales:

- Multiplicación de una cualquiera de las ecuaciones del sistema por un número distinto de cero.
- Sustitución de una ecuación cualquiera del sistema por la que resulta de sumarle otra cualquiera.
- Permutación del orden en que aparecen en el sistema dos ecuaciones cualesquiera del mismo.

Comencemos la exposición de la mecánica del método mediante un ejemplo que nos servirá como introducción.

Se desea resolver el sistema de cuatro ecuaciones lineales con cuatro incógnitas

$$\begin{aligned} 2x_1 + x_2 + 4x_4 &= 2 \\ -4x_1 - 2x_2 + 3x_3 - 7x_4 &= -9 \\ 4x_1 + x_2 - 2x_3 + 8x_4 &= 2 \\ -3x_2 - 12x_3 - x_4 &= 2. \end{aligned} \tag{1.1}$$

Escrito en forma matricial, $A\mathbf{x} = \mathbf{b}$, los distintos componentes son

$$A = \begin{bmatrix} 2 & 1 & 0 & 4 \\ -4 & -2 & 3 & -7 \\ 4 & 1 & -2 & 8 \\ 0 & -3 & -12 & -1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ -9 \\ 2 \\ 2 \end{bmatrix} \quad \text{y} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Reconfiguremos inicialmente la matriz A aumentándola con la columna que define el término independiente $\mathbf{b}$ y llamemos a la nueva matriz resultante $\hat{A}$; es decir,

$$\hat{A} = [A|\mathbf{b}] = \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ -4 & -2 & 3 & -7 & -9 \\ 4 & 1 & -2 & 8 & 2 \\ 0 & -3 & -12 & -1 & 2 \end{array} \right].$$

Etapas 1

Comprobemos que el elemento $\hat{a}_{11}$ —denominado *elemento pivote*— no es cero. Si es distinto de cero, eliminemos los elementos de la primera columna por debajo de ese $\hat{a}_{11}$. Para ello, definamos para cada fila $2, \dots, n$ los factores o *multiplicadores*

$$r_i = \frac{\hat{a}_{i1}}{\hat{a}_{11}}, \quad i = 2, \dots, n.$$

A continuación, restemos de las filas $i = 2, \dots, n$, la primera multiplicada por r_i : todos los elementos debajo de la diagonal principal de la columna 1 se anularán. Los demás elementos de $\hat{A}$ debajo de la primera fila también se verán afectados de acuerdo con la siguiente expresión:

$$\hat{a}_{ij} \leftarrow \hat{a}_{ij} - r_i \cdot \hat{a}_{1j}, \quad i = 2, \dots, n; \quad j = 2, \dots, n+1.$$

En el ejemplo que venimos manejando, los multiplicadores son

$$\begin{aligned} r_2 &= \hat{a}_{21}/\hat{a}_{11} = -4/2 = -2 \\ r_3 &= \hat{a}_{31}/\hat{a}_{11} = 4/2 = 2 \\ r_4 &= \hat{a}_{41}/\hat{a}_{11} = 0/2 = 0. \end{aligned}$$

Los coeficientes de la matriz $\hat{A}$ que cambian de valor son:

$$\begin{aligned} \text{en la 2ª fila:} \quad & \hat{a}_{21} \leftarrow 0 \\ & \hat{a}_{22} \leftarrow \hat{a}_{22} - r_2 \cdot \hat{a}_{12} = -2 + 2 \cdot 1 = 0 \\ & \hat{a}_{23} \leftarrow \hat{a}_{23} - r_2 \cdot \hat{a}_{13} = 3 + 2 \cdot 0 = 3 \\ & \hat{a}_{24} \leftarrow \hat{a}_{24} - r_2 \cdot \hat{a}_{14} = -7 + 2 \cdot 4 = 1 \\ & \hat{a}_{25} \leftarrow \hat{a}_{25} - r_2 \cdot \hat{a}_{15} = -9 + 2 \cdot 2 = -5; \end{aligned}$$

$$\begin{array}{ll}
\text{en la 3ª fila:} & \begin{array}{l} \hat{a}_{31} \leftarrow 0 \\ \hat{a}_{32} \leftarrow \hat{a}_{32} - r_3 \cdot \hat{a}_{12} = 1 - 2 \cdot 1 = -1 \\ \hat{a}_{33} \leftarrow \hat{a}_{33} - r_3 \cdot \hat{a}_{13} = -2 - 2 \cdot 0 = -2 \\ \hat{a}_{34} \leftarrow \hat{a}_{34} - r_3 \cdot \hat{a}_{14} = 8 - 2 \cdot 4 = 0 \\ \hat{a}_{35} \leftarrow \hat{a}_{35} - r_3 \cdot \hat{a}_{15} = 2 - 2 \cdot 2 = -2; \end{array} \\
\text{en la 4ª fila:} & \begin{array}{l} \hat{a}_{41} \leftarrow 0 \\ \hat{a}_{42} \leftarrow \hat{a}_{42} - r_4 \cdot \hat{a}_{12} = -3 - 0 \cdot 1 = -3 \\ \hat{a}_{43} \leftarrow \hat{a}_{43} - r_4 \cdot \hat{a}_{13} = -12 - 0 \cdot 0 = -12 \\ \hat{a}_{44} \leftarrow \hat{a}_{44} - r_4 \cdot \hat{a}_{14} = -1 - 0 \cdot 4 = -1 \\ \hat{a}_{45} \leftarrow \hat{a}_{45} - r_4 \cdot \hat{a}_{15} = 2 - 0 \cdot 2 = 2. \end{array}
\end{array}$$

La nueva matriz $\hat{A}_1$, resultado de transformar $\hat{A}$, es:

$$\hat{A}_1 = \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ 0 & 0 & 3 & 1 & -5 \\ 0 & -1 & -2 & 0 & -2 \\ 0 & -3 & -12 & -1 & 2 \end{array} \right].$$

Obsérvese que se hubiese obtenido exactamente el mismo resultado o transformación de haber premultiplicado $\hat{A}$ por la denominada *transformación de Gauss* que define la matriz

$$L_1 = \left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right],$$

matriz *triangular inferior unitaria*, denominada a su vez *matriz de transformación de Gauss*, que también se puede escribir de la forma $L_1 = I - \alpha e_1^T$, donde

$$\alpha = \begin{bmatrix} 0 \\ -2 \\ 2 \\ 0 \end{bmatrix} \quad \text{y} \quad e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

En efecto,

$$L_1 \hat{A} = \left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ -4 & -2 & 3 & -7 & -9 \\ 4 & 1 & -2 & 8 & 2 \\ 0 & -3 & -12 & -1 & 2 \end{array} \right] = \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ 0 & 0 & 3 & 1 & -5 \\ 0 & -1 & -2 & 0 & -2 \\ 0 & -3 & -12 & -1 & 2 \end{array} \right].$$

En resumen, $\hat{A}_1 = L_1 \hat{A}$.

La inversa de la matriz L_1 , por otro lado, como se puede comprobar fácilmente, es

$$L_1^{-1} = \left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ 2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right].$$

Obsérvese que es idéntica a L_1 salvo por lo que respecta al signo de los coeficientes de la primera columna debajo de la diagonal principal.

Etapas 2

Hagamos cero los elementos debajo de la diagonal principal de la 2ª columna de $\hat{A}_1$.

Al intentar hacerlo observamos que el elemento pivote $\hat{a}_{22}^1$ es cero, lo que nos impide proceder como en la etapa anterior. Para solventar esta eventualidad, comprobemos si algún elemento de la columna 2 por debajo de $\hat{a}_{22}^1$ es distinto de cero: si no hay ninguno, como se puede demostrar, esta columna es combinación lineal de la primera y por tanto la matriz es singular; si existe alguno, escojamos el de mayor valor absoluto y procedamos a intercambiar la fila correspondiente con la segunda.

El elemento de mayor valor absoluto debajo de la diagonal principal en la segunda columna, -3 , se encuentra en la fila 4. Intercambiamos esa fila 4 con la 2. Se obtendrá

$$\hat{A}'_1 = \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ 0 & -3 & -12 & -1 & 2 \\ 0 & -1 & -2 & 0 & -2 \\ 0 & 0 & 3 & 1 & -5 \end{array} \right].$$

Este mismo resultado, como se puede comprobar de forma inmediata, se obtiene premultiplicando la matriz $\hat{A}_1$ por la matriz de permutación

$$P_1 = \left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{array} \right].$$

Recapitulemos, la matriz con la que vamos a operar a continuación, $\hat{A}'_1$, es:

$$\hat{A}'_1 = P_1 L_1 \hat{A}.$$

Apliquemos a continuación a la columna 2 la misma idea que a la columna 1 y hagamos cero sus elementos 3 a n . Los nuevos multiplicadores saldrán de la expresión

$$r_i = \frac{\hat{a}_{i2}^{1'}}{\hat{a}_{22}^{1'}}, \quad i = 3, 4.$$

Los nuevos valores de los elementos de la matriz $\hat{A}'_1$ por debajo de la segunda fila se obtendrán aplicando la expresión

$$\hat{a}_{ij}^{1'} \leftarrow \hat{a}_{ij}^{1'} - r_i \cdot \hat{a}_{2j}^{1'}, \quad i = 3, 4; \quad j = 3, \dots, 5.$$

Los valores numéricos que se obtienen en el ejemplo son, para el caso de los multiplicadores,

$$r_3 = \hat{a}_{32}^{1'}/\hat{a}_{22}^{1'} = 1/3 \quad \text{y}$$

$$r_4 = \hat{a}_{42}^{1'}/\hat{a}_{22}^{1'} = 0.$$

Los nuevos elementos de la matriz $\hat{A}'_1$ resultante:

$$\begin{aligned} \text{en la 3ª fila: } \hat{a}_{32}^{1'} &\leftarrow 0 \\ \hat{a}_{33}^{1'} &\leftarrow \hat{a}_{33}^{1'} - r_3 \cdot \hat{a}_{23}^{1'} = -2 + \frac{1}{3} \cdot 12 = 2 \\ \hat{a}_{34}^{1'} &\leftarrow \hat{a}_{34}^{1'} - r_3 \cdot \hat{a}_{24}^{1'} = 0 + \frac{1}{3} \cdot 1 = 1/3 \\ \hat{a}_{35}^{1'} &\leftarrow \hat{a}_{35}^{1'} - r_3 \cdot \hat{a}_{25}^{1'} = -2 - \frac{1}{3} \cdot 2 = -8/3; \end{aligned}$$

$$\begin{aligned}
\text{en la 4ª fila: } \hat{a}_{42}^{1'} &\leftarrow 0 \\
\hat{a}_{43}^{1'} &\leftarrow \hat{a}_{43}^{1'} - r_4 \cdot \hat{a}_{23}^{1'} = 3 - 0 \cdot 12 = 3 \\
\hat{a}_{44}^{1'} &\leftarrow \hat{a}_{44}^{1'} - r_4 \cdot \hat{a}_{24}^{1'} = 1 - 0 \cdot 1 = 1 \\
\hat{a}_{45}^{1'} &\leftarrow \hat{a}_{45}^{1'} - r_4 \cdot \hat{a}_{25}^{1'} = -5 - 0 \cdot 2 = -5.
\end{aligned}$$

Obsérvese que, al ser $r_4 = 0$, los cálculos para adaptar la cuarta fila podrían haberse evitado. La nueva matriz resultado de estas transformaciones es

$$\hat{A}_2 = \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ 0 & -3 & -12 & -1 & 2 \\ 0 & 0 & 2 & 1/3 & -8/3 \\ 0 & 0 & 3 & 1 & -5 \end{array} \right].$$

Razonando de forma similar a como lo hicimos en la etapa anterior, la última matriz $\hat{A}_2$ se expresa a partir de la inicial por

$$\hat{A}_2 = L_2 P_1 L_1 \hat{A},$$

donde

$$L_2 = \left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1/3 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right].$$

Etapas 3

Para conseguir transformar el sistema original en uno triangular superior sólo resta anular el elemento $\hat{a}_{43}^2$. El elemento de la diagonal principal $\hat{a}_{33}^2$ es distinto de cero, luego procedemos a calcular el multiplicador r_4 :

$$r_4 = \hat{a}_{43}^2 / \hat{a}_{33}^2 = 3/2.$$

Los nuevos valores de los elementos de la matriz $\hat{A}_2$ por debajo de la tercera fila se obtendrán aplicando la expresión

$$\hat{a}_{ij}^2 \leftarrow \hat{a}_{ij}^2 - r_i \cdot \hat{a}_{3j}^2, \quad i = 4; j = 4, 5.$$

En concreto, en la cuarta fila:

$$\begin{aligned}
\hat{a}_{43}^2 &\leftarrow 0 \\
\hat{a}_{44}^2 &\leftarrow \hat{a}_{44}^2 - r_4 \cdot \hat{a}_{34}^2 = 1 - \frac{3}{2} \cdot \frac{1}{3} = 1/2 \\
\hat{a}_{45}^2 &\leftarrow \hat{a}_{45}^2 - r_4 \cdot \hat{a}_{35}^2 = -5 + \frac{3}{2} \cdot \frac{8}{3} = -1.
\end{aligned}$$

La nueva matriz resultado de estas transformaciones es

$$\hat{A}_3 = \left[\begin{array}{cccc|c} 2 & 1 & 0 & 4 & 2 \\ 0 & -3 & -12 & -1 & 2 \\ 0 & 0 & 2 & 1/3 & -8/3 \\ 0 & 0 & 0 & 1/2 & -1 \end{array} \right].$$

A este resultado se ha llegado después de aplicar a la matriz inicial $\hat{A}$ una serie de transformaciones; concretamente:

$$\hat{A}_3 = L_3 L_2 P_1 L_1 \hat{A},$$

donde

$$L_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -3/2 & 1 \end{bmatrix}.$$

Tenemos, en conclusión, que la matriz original que definía el sistema, A , se puede transformar en la triangular superior U , tal como queríamos, aplicándole las mismas transformaciones que a $\hat{A}$. Es decir,

$$U = L_3 L_2 P_1 L_1 A.$$

Como al vector $\mathbf{b}$ también se le han efectuado las mismas transformaciones dando lugar a otro $\mathbf{b}'$, resolver el sistema de ecuaciones original es equivalente a resolver

$$U\mathbf{x} = \mathbf{b}'.$$

Es decir,

$$\begin{bmatrix} 2 & 1 & 0 & 4 \\ 0 & -3 & -12 & -1 \\ 0 & 0 & 2 & 1/3 \\ 0 & 0 & 0 & 1/2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ -8/3 \\ -1 \end{bmatrix}.$$

La solución de este sistema de ecuaciones se lleva a cabo muy fácilmente mediante sustitución inversa:

$$x_4 = -2,$$

sustituyendo en la tercera ecuación,

$$x_3 = \frac{-8/3 - (-2)(1/3)}{2} = -1,$$

y, a su vez, haciéndolo en la segunda,

$$x_2 = \frac{2 - (-1)(-2) - (-12)(-1)}{-3} = 4.$$

Por último, sustituyendo los valores de las variables ya calculados en la primera ecuación se obtiene

$$x_1 = \frac{2 - 4(-2) - 1(4)}{2} = 3.$$

La solución de nuestro ejemplo es pues

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ -1 \\ -2 \end{bmatrix}.$$

1.2.1 Pivotación

Ya conocemos la mecánica de la eliminación de Gauss. Ahora bien, veamos qué ocurre si resolvemos con el procedimiento esbozado el sistema,

$$\underbrace{\begin{bmatrix} 10^{-4} & 1 \\ 1 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}_b,$$

en una máquina con tres dígitos significativos que efectúe redondeos.

Aplicando la mecánica apuntada, en la primera etapa se obtendría la nueva matriz A_1 y el nuevo vector b_1 ; son:

$$A_1 = \begin{bmatrix} 10^{-4} & 1 \\ 0 & 1 - 10^4 \end{bmatrix} \quad \text{y} \quad b_1 = \begin{bmatrix} 1 \\ 2 - 10^4 \end{bmatrix}.$$

El número $1 - 10^4 = -9999$, la máquina lo redondearía a -10^4 ; de la misma forma procedería con $2 - 10^4$. La solución del sistema sería

$$\begin{aligned} x_2 &= \frac{-10^4}{-10^4} = 1 \\ x_1 &= \frac{1 - x_2}{10^{-4}} = 0. \end{aligned}$$

Desde un punto de vista meramente algebraico, es indiferente la elección que se haga de los pivotes, siempre y cuando sean distintos de cero. Esto es debido a que los sistemas que se van obteniendo son equivalentes y tienen todos la misma solución, sea cual sea el modo en que se van construyendo. Desde esta idea, la elección más simple consiste en escoger en cada etapa el primer elemento no nulo debajo de la diagonal principal de la columna correspondiente.

No ocurre lo mismo, como acabamos de ver, si se analiza la elección del pivote teniendo en cuenta los errores de redondeo que inevitablemente introduce el sistema de numeración en coma flotante de la máquina donde se efectúan los cálculos.

Para paliar estos errores, aplicaremos la estrategia que utilizamos cuando nos encontramos un elemento en la diagonal principal igual a cero, aunque éste no sea el caso. Es decir, antes de empezar cada etapa i que nos lleve a hacer cero los elementos por debajo de la diagonal principal en una determinada columna i , se realizará una búsqueda entre esos elementos y se escogerá aquel de mayor valor absoluto. A continuación, se deberán intercambiar las filas que determinan ese elemento y la i . Para el caso de este último ejemplo, en la primera etapa, en lugar de operar sobre la matriz anterior, como el valor absoluto del elemento a_{21} es mayor que el del a_{11} , se intercambiaría la fila 1 con la 2, obteniéndose

$$A' = \begin{bmatrix} 1 & 1 \\ 10^{-4} & 1 \end{bmatrix} \quad \text{y} \quad b' = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Continuando con el procedimiento normal, después de la primera etapa, se llegará a

$$A'_1 = \begin{bmatrix} 1 & 1 \\ 0 & 1 - 10^{-4} \end{bmatrix} \quad \text{y} \quad b'_1 = \begin{bmatrix} 2 \\ 1 - 2 \cdot 10^{-4} \end{bmatrix}.$$

Por redondeos internos, la máquina *vería*

$$A'_1 = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \quad \text{y} \quad \mathbf{b}'_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix},$$

siendo la solución del sistema de ecuaciones correspondiente

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

solución mucho mejor que la anterior pues la real es

$$x_2 = \frac{0,9998}{0,9999} = 0,99989999$$

$$x_1 = 1,00010001.$$

El proceso descrito se denomina *pivotación parcial*. Su motivación radica por un lado en la aritmética de precisión finita con la que trabajan todos los ordenadores y máquinas calculadoras, y que por lógica será en uno de estos soportes donde se implemente el método, y por otro en el hecho de que los factores o multiplicadores antes introducidos son inversamente proporcionales al elemento pivote, por lo que si éste es demasiado pequeño puede amplificar los errores de redondeo a lo largo del proceso de solución. Lo ideal desde el punto de vista numérico sería no escoger el elemento de mayor valor absoluto al que nos referíamos antes sino que el pivote no fuese demasiado pequeño (por ejemplo no menor que una décima parte del elemento de mayor valor absoluto).

En la tabla 1.1 se describe el algoritmo completo para resolver el sistema $A\mathbf{x} = \mathbf{b}$ mediante eliminación de Gauss. Parte de la matriz A ampliada en el término independiente.

Una segunda estrategia de pivotación es la conocida como *pivotación total*. La idea consiste en alterar convenientemente el orden natural de eliminación de las variables a base de buscar en una etapa i no sólo el elemento de mayor valor absoluto en la columna correspondiente, sino en todo lo que queda de la matriz; es decir, el $\max_{i \leq k \leq n, i \leq j \leq n} |a_{kj}|$. Desde el punto de vista de la minimización del efecto de los errores de redondeo, la pivotación total proporciona la estrategia óptima. Puede demostrarse de hecho que, desde un punto de vista teórico y en un cierto sentido que habría que precisar, la eliminación de Gauss con pivotación total es con carácter general un algoritmo numéricamente estable. La eliminación de Gauss con pivotación parcial, por el contrario, no tiene esa propiedad teórica. Pese a los pequeños inconvenientes apuntados, sin embargo, es raro que se recurra a la pivotación total en la práctica dada la gran cantidad de comparaciones que es necesario efectuar para llevarla a cabo y los buenos resultados y excelentes prestaciones numéricas que la parcial proporciona.

La versión en FORTRAN 77 del algoritmo de la tabla 1.1 para resolver el sistema del ejemplo que nos ha servido de introducción al método es la que sigue.

```

      PROGRAM Gauss
C
      parameter (n = 4)
      real a(n,n+1),x(n)
C
      data a/2.,-4.,4.,0.,1.,-2.,1.,-3.,0.,3.,-2.,-12.,4.,-7.,8.,-1.,2.,
+        -9.,2.,2./
C

```

Tabla 1.1

Algoritmo para la resolución de $A\mathbf{x} = \mathbf{b}$ mediante eliminación de Gauss con pivotación parcial

```

*
* TRANSFORMACIÓN DE LA MATRIZ AUMENTADA  $[A|\mathbf{b}]$ 
*
for  $i = 1$  to  $n - 1$ 
    Determinar índice  $p \in \{i, i + 1, \dots, n\}$  tal que  $|a(p, i)| = \max_{i \leq j \leq n} |a(j, i)|$ .
    Intercambiar filas  $p$  e  $i$ .
    for  $j = i + 1$  to  $n$ 
         $\eta = a(j, i)/a(i, i)$ 
        for  $k = i + 1$  to  $n + 1$ 
             $a(j, k) \leftarrow a(j, k) - \eta \cdot a(i, k)$ 
        end
    end
end
*
* SUSTITUCIÓN INVERSA.
*
for  $j = n$  to  $1$ 
    
$$x(j) \leftarrow \left( b(j) - \sum_{k=j+1}^n a(j, k) \cdot x(k) \right) / a(j, j)$$

end

```

```

C *** Eliminación de Gauss ***
C
C * Triangularización *
C
  do k = 1,n-1
    l = 0
    smax = abs(a(k,k))
    do i = k+1,n
      if (abs(a(i,k)).gt.smax) then
        l = i
        smax = abs(a(i,k))
      endif
    end do
    if (l.ne.0) then
      do i = k,n+1
        t = a(l,i)
        a(l,i) = a(k,i)
        a(k,i) = t
      end do
    endif
    do i = k+1,n
      r = a(i,k)/a(k,k)
      do j = k+1,n+1
        a(i,j) = a(i,j)-r*a(k,j)
      end do
    end do
  end do

```

```

        end do
C
C  * Sustitución inversa *
C
      x(n) = a(n,n+1)/a(n,n)
      do i = n-1,1,-1
        c = a(i,n+1)
        do j = i+1,n
          c = c-a(i,j)*x(j)
        end do
        x(i) = c/a(i,i)
      end do
C
      print *,x
C
      end

```

De esta codificación de la eliminación de Gauss con pivotación parcial conviene destacar dos aspectos negativos. El primero se refiere a que no es realmente necesario intercambiar las filas una vez elegido el elemento pivote de cada etapa (lo que cuando el sistema es de gran dimensión puede alargar en demasía la resolución); basta con tener constancia en cada momento de dónde está la fila que intercambiar. El segundo, a que tal como está estructurado el programa sólo se podría resolver un sistema —el definido por el $\mathbf{b}$ dado— y no, como es lo más habitual, por ejemplo, distintos sistemas con la misma matriz A y diversos términos independientes.

Estos dos inconvenientes se pueden paliar mediante unas sencillas modificaciones en la forma en que están dispuestos los cálculos y con la introducción de un elemento muy común en técnicas numéricas de todo tipo: un vector índice o puntero. Este vector índice, al que denominaremos IPIV, cuya dimensión es el número de ecuaciones del sistema, se inicializa de tal forma que cada uno de sus elementos indique la posición inicial en el sistema de cada una de las ecuaciones; es decir,

$$\text{IPIV} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ \vdots \\ n \end{bmatrix}.$$

Cuando haya que intercambiar dos filas en un etapa, no se hará intercambiando físicamente los elementos de esas dos filas, sino haciéndolo en las correspondientes de IPIV. Si por ejemplo, en la primera etapa hay que utilizar como pivote un elemento de la cuarta fila, una vez efectuado el cambio, el vector IPIV quedará:

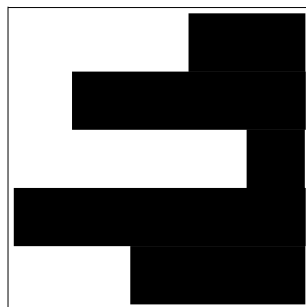
$$\text{IPIV} = \begin{bmatrix} 4 \\ 2 \\ 3 \\ 1 \\ \vdots \\ n \end{bmatrix}.$$

Si al final de un proceso de resolución de un sistema de cinco ecuaciones con cinco incógnitas,

el vector puntero resultase

$$\text{IPIV} = \begin{bmatrix} 4 \\ 2 \\ 5 \\ 1 \\ 3 \end{bmatrix},$$

la matriz A' que se obtendría no resultaría ser estrictamente triangular superior sino que tendría la forma que sigue.



Para resolver el sistema que define $Ax = b$ habría que aplicar esas mismas manipulaciones de filas al vector b o tenerlo en cuenta.

Para solventar el segundo inconveniente de los mencionados, lo más lógico parece, en lugar de operar sobre la matriz aumentada, hacerlo sobre A directamente. Lo que habrá que hacer, claro está, es guardar de alguna manera la información relativa a las manipulaciones efectuadas a A a fin de podérselas también hacer al vector b , cualquiera que sea éste. El vector índice IPIV, en este sentido, puede guardar la información relativa a las pivotaciones efectuadas. Respecto a las transformaciones que se aplican a la matriz A , la forma de guardarlas para aplicárselas a cada término independiente de interés se basa en recordar que esas transformaciones estaban perfectamente determinadas por unos multiplicadores —recordemos también las matrices elementales L_i — asociados a cada fila de cada etapa. Si guardamos los $i - 1$ multiplicadores que definen cada etapa i en algún sitio, todo el proceso de eliminación se podrá reconstruir fácilmente. Ahora bien, qué mejor sitio para guardar esos multiplicadores que los lugares vacíos —o mejor dicho, que se hacen cero— que provocan las transformaciones que definen: en la etapa i , debajo de la diagonal principal en la columna i .

Recordando el sistema (1.1) que nos servía para introducir la mecánica de la eliminación de Gauss, sólo considerando la matriz A , con esta forma de proceder, al final del proceso, esa matriz sería:

$$A = \begin{bmatrix} 2 & 1 & 0 & 4 \\ -2 & -3 & -12 & -1 \\ 2 & 1/3 & 2 & 1/3 \\ 0 & 0 & 3/2 & 1/2 \end{bmatrix}.$$

Los multiplicadores distintos de cero que se han calculado en todo el proceso son -2 , 2 , $1/3$ y $3/2$.

Un programa completo que implementa estas ideas y que permite resolver cualquier sistema $Ax = b$, con pivotación parcial, cualquiera que sea su término independiente b , se lista a continuación.

```

PROGRAM Gaussc
C
parameter (m = 10)
integer ipvt(m),pi
C
real a(m,m),b(m),x(m)
C
character*12 fil
C
*** Resolución de un sistema lineal regular cualquiera Ax=b
C     mediante eliminación de Gauss ***
C
print *, 'Dimensión de la Matriz A?'
read *, n
print *, 'Fichero de datos?'
read '(A)', fil
open (10, file=fil)
read (10, *) ((a(i,j), i=1,n), j=1,n)
read (10, *) (b(i), i=1,n)
do i = 1,n
    ipvt(i) = i
end do
C
C * Triangularización *
C
do k = 1,n-1
    l = 0
    smax = abs(a(ipvt(k),k))
    do i = k+1,n
        ip = ipvt(i)
        if (abs(a(ip,k)).gt.smax) then
            l = i
            smax = abs(a(ip,k))
        endif
    end do
    if (l.ne.0) then
       iaux = ipvt(k)
        ipvt(k) = ipvt(l)
        ipvt(l) = iaux
    endif
    pi = ipvt(k)
    r1 = 1.0/a(pi,k)
    do i = k+1,n
        ip = ipvt(i)
        r = a(ip,k)*r1
        do j = k+1,n
            a(ip,j) = a(ip,j)-r*a(pi,j)
        end do
        a(ip,k) = -r
    end do
end do
C
do k = 1,n-1
    ip = ipvt(k)
    do i = k+1,n
        pi = ipvt(i)
        b(pi) = b(pi)+a(pi,k)*b(ip)
    end do
end do

```

```

C
C * Sustitución inversa *
C
    x(n) = b(ipvt(n))/a(ipvt(n),n)
    do i = n-1,1,-1
        pi = ipvt(i)
        c = b(pi)
        do j = i+1,n
            c = c-a(pi,j)*x(j)
        end do
        x(i) = c/a(pi,i)
    end do
C
    print '('Solución:',(4f5.1))',(x(i),i=1,n)
C
end

```

1.2.2 Número de operaciones aritméticas del método

Para valorar las prestaciones de un algoritmo numérico se han de considerar diversos factores. Dos de los más importantes son sin duda su estabilidad numérica ante el efecto de los errores de redondeo y la cantidad de tiempo necesaria para completar los cálculos que conlleva. Ambos factores dependen del número de operaciones aritméticas necesarias para la aplicación del algoritmo.

Los tiempos necesarios para realizar en un ordenador la multiplicación y la división de dos números son aproximadamente iguales y considerablemente mayores, en términos relativos, que los requeridos para realizar la suma o diferencia, que también son muy semejantes entre sí. La relación entre el tiempo que requiere una multiplicación o división y una suma o resta varía de un ordenador a otro. En lo que resta del libro, al referirnos a las operaciones de multiplicación o división en un algoritmo lo haremos mediante la expresión *multiplicaciones/divisiones* y a las sumas o restas mediante *sumas/restas*. Cuando mencionemos sumas/restas, por ejemplo, nos estaremos refiriendo al número total de sumas más restas que el correspondiente algoritmo necesita para llevarse a efecto.

Determinemos el número de operaciones aritméticas que requiere el procedimiento de eliminación de Gauss para resolver un sistema de ecuaciones lineales. En la primera etapa, las operaciones que se realizan están simbólicamente representadas por el esquema que sigue.

$$\begin{array}{|c|} \hline \begin{array}{cccccc} \times & \times & \cdots & \times & \times & \times \\ \times & \times & \cdots & \times & \times & \times \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ \times & \times & \cdots & \times & \times & \times \\ \times & \times & \cdots & \times & \times & \times \end{array} \\ \hline \end{array} \rightarrow \begin{array}{|c|} \hline \begin{array}{cccccc} \times & \times & \cdots & \times & \times & \times \\ 0 & \square & \cdots & \square & \square & \square \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ 0 & \square & \cdots & \square & \square & \square \\ 0 & \square & \cdots & \square & \square & \square \end{array} \\ \hline \end{array}$$

El símbolo $\square$ designa los elementos de la matriz que se ven afectados en esa etapa y que, en principio, son distintos de cero.

Si en la etapa i se está transformando una matriz $n \times n$, las operaciones que en ella se

realizan son:

$n - i$	divisiones para el cálculo de los factores o multiplicadores;
$(n - i)(n - i + 1)$	multiplicaciones y restas para modificar los elementos de la matriz por debajo de la fila i que no están en la propia columna i .

Si como hemos indicado, las multiplicaciones/divisiones y las sumas/restas emplean aproximadamente el mismo tiempo de cálculo, podemos sumar sus números de operaciones con lo que se obtienen para cada etapa,

$$(n - i) + (n - i)(n - i + 1) = (n - i)(n - i + 2)$$

multiplicaciones/divisiones y

$$(n - i)(n - i + 1)$$

sumas/restas.

En $n - 1$ etapas de que consta el proceso, se harán

$$\begin{aligned} \sum_{i=1}^{n-1} (n - i)(n - i + 2) &= (n^2 + 2n) \sum_{i=1}^{n-1} 1 - 2(n + 1) \sum_{i=1}^{n-1} i + \sum_{i=1}^{n-1} i^2 \\ &= (n^2 + 2n)(n - 1) - 2(n + 1) \frac{(n - 1)n}{2} \\ &\quad + \frac{(n - 1)n(2n - 1)}{6} \\ &= \frac{2n^3 + 3n^2 - 5n}{6} \end{aligned}$$

multiplicaciones/divisiones y

$$\begin{aligned} \sum_{i=1}^{n-1} (n - i)(n - i + 1) &= (n^2 + 2n) \sum_{i=1}^{n-1} 1 - (2n + 1) \sum_{i=1}^{n-1} i + \sum_{i=1}^{n-1} i^2 \\ &= (n^2 + n)(n - 1) - (2n + 1) \frac{(n - 1)n}{2} \\ &\quad + \frac{(n - 1)n(2n - 1)}{6} \\ &= \frac{n^3 - n}{3} \end{aligned}$$

sumas/restas.

El comportamiento de estos valores para n grande es como

$$\frac{1}{3}n^3.$$

El proceso de sustitución inversa, por otro lado, requiere $(n-i)$ multiplicaciones y $(n-i-1)$ sumas, para cada término del sumatorio, y una resta y una división. El número total de operaciones de todo el proceso es

$$1 + \sum_{i=1}^{n-1} ((n-i) + 1) = \frac{n^2 + n}{2}$$

multiplicaciones/divisiones y

$$\sum_{i=1}^{n-1} ((n-i-1) + 1) = \frac{n^2 - n}{2}$$

sumas/restas. Contando la transformación de la matriz del sistema y la sustitución inversa, la eliminación de Gauss requiere

$$\frac{2n^3 + 3n^2 - 5n}{6} + \frac{n^2 + n}{2} = \frac{n^3 + n^2 - n}{3}$$

multiplicaciones/divisiones y

$$\frac{n^3 - n}{3} + \frac{n^2 - n}{2} = \frac{2n^3 + 3n^2 - 5n}{6}$$

sumas/restas.

El comportamiento de estos últimos números para valores grandes de n es como

$$\frac{1}{3}n^3$$

lo que da idea de la complejidad que puede suponer resolver un sistema de varios miles de ecuaciones mediante eliminación de Gauss.²

Aunque la cantidad $n^3/3$ puede parecer muy grande, recordemos las *fórmulas de Cramer* para la solución de un sistema de ecuaciones:

$$x_i = \frac{\det(B_i)}{\det(A)}, \quad \text{donde } B_i = \begin{bmatrix} a_{11} & \cdots & a_{1i-1} & b_1 & a_{1i+1} & \cdots & a_{1n} \\ a_{21} & \cdots & a_{2i-1} & b_2 & a_{2i+1} & \cdots & a_{2n} \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & \cdots & a_{ni-1} & b_n & a_{ni+1} & \cdots & a_{nn} \end{bmatrix}.$$

Mediante estas fórmulas se requieren:

$$\begin{cases} (n+1)! & \text{sumas,} \\ (n+2)! & \text{multiplicaciones y} \\ n & \text{divisiones.} \end{cases}$$

²En los cálculos del número de operaciones que necesita la eliminación de Gauss no se han tenido en cuenta las que se necesitan para realizar las pivotaciones: comparaciones y trueque de posiciones de memoria.

Para un sistema de diez ecuaciones con diez incógnitas se requerirán:

$$\begin{cases} 740 & \text{operaciones en total utilizando eliminación de Gauss, y} \\ 500.000.000 & \text{operaciones, aproximadamente, aplicando las fórmulas de Cramer.} \end{cases}$$

Evidentemente, las fórmulas de Cramer superan en mucho al número de operaciones equivalente que requiere la eliminación de Gauss.

1.3 Método de Gauss-Jordan

Una extensión natural de la eliminación de Gauss consiste en eliminar de cada columna de la matriz de coeficientes del sistema no sólo los elementos no nulos que están debajo de la diagonal sino también los que están encima. Al método que así procede se le conoce como método de Gauss-Jordan.

Si recordamos que la transformación que daba lugar a una etapa i en la eliminación de Gauss estaba definida por la matriz

$$L_i = I - \alpha_i e_i^T,$$

donde

$$\alpha_i = \begin{bmatrix} 0 \\ \vdots \\ a_{i+1,i}^i/a_{ii}^i \\ \vdots \\ a_{ni}^i/a_{ii}^i \end{bmatrix} \leftarrow \text{fila } i+1 \quad \text{y } e_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \leftarrow \text{fila } i,$$

ahora, en la etapa equivalente i , la *matriz de transformación de Gauss-Jordan*, está dada por

$$T_i = I - \alpha_i e_i^T,$$

donde, en este caso,

$$\alpha_i = \begin{bmatrix} a_{1i}^i/a_{ii}^i \\ \vdots \\ 1/a_{ii}^i \\ \vdots \\ a_{ni}^i/a_{ii}^i \end{bmatrix} \leftarrow \text{fila } i,$$

siendo el vector e_i igual que antes.

Si se tiene en cuenta que $A^0 = A$ y $A^n = I$, se tendrá que

$$T_{n-1} \cdots T_2 T_1 A = I,$$

por lo que el método obtiene directamente la matriz inversa de A en forma factorizada (producto de matrices elementales) sin más que observar que

$$A^{-1} = T_{n-1} \cdots T_2 T_1.$$

Si de la misma forma que se procedía en el caso de la eliminación de Gauss, en las posiciones que se hacen cero se guardan los valores de los multiplicadores correspondientes, al final del proceso del método de Gauss-Jordan, en la propia matriz A se tendrá su inversa.

El número de multiplicaciones y divisiones que se llevan a cabo en el método de Gauss-Jordan es $O(n^3/2)$. El de sumas y restas es el mismo. Para cuantificar totalmente las operaciones que realiza el método habría que añadir las comparaciones y el trueque de posiciones de memoria necesarias para llevar a cabo las pivotaciones.

1.4 Descomposición o factorización LU

La descomposición o factorización LU (también conocida en la literatura especializada como factorización triangular) busca expresar una matriz cuadrada regular como producto de una triangular inferior, L , y otra triangular superior, U . La denominación “LU” está motivada por los términos *lower* (inferior) y *upper* (superior) que adjetivan los factores triangulares.

Su utilidad inmediata, aparte de que bajo ciertas circunstancias almacenar una matriz, dispersa por ejemplo, en forma factorizada necesita menos posiciones de memoria que en forma compacta, radica en que para resolver un sistema de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$, si $A = LU$, el problema se reduce a resolver dos sistemas de ecuaciones triangulares:

$$L\mathbf{y} = \mathbf{b} \quad \text{y} \quad U\mathbf{x} = \mathbf{y}.$$

Este hecho tiene una importancia indudable cuando se requiere resolver sistemas de ecuaciones en los que la matriz A es siempre la misma y lo único que cambia es el término independiente.

Una forma de conseguir esta factorización LU la constituye la propia eliminación de Gauss que se acaba de estudiar. En efecto, si recordamos, el método procedía reduciendo la matriz original a una triangular superior mediante unas permutaciones y unas transformaciones definidas por matrices elementales triangulares inferiores, de la siguiente manera:

$$L_{n-1}P_{n-1} \cdots L_1P_1A = U.$$

De este proceso, haciendo

$$P = P_{n-1} \cdots P_1$$

y

$$L = P(L_{n-1}P_{n-1} \cdots L_2P_2L_1P_1)^{-1},$$

se obtiene la factorización

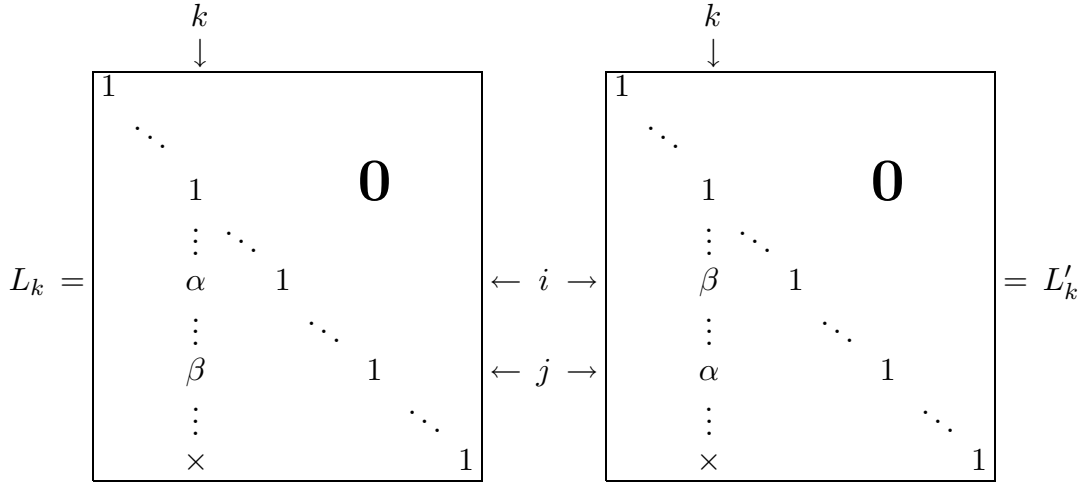
$$PA = LU.$$

Para demostrarlo, recordemos primero que las matrices de permutación que se definen en la eliminación de Gauss, P_i , permutan siempre dos filas i y j , $j > i$.

Lema 1.1 Sea P_i una matriz de permutación ($P_i = P_i^{-1}$) de índices i y j , $j > i$. Para un $k < i$, se tiene que

$$L_kP_i = P_iL'_k \quad \text{o, lo que es lo mismo, que} \quad P_iL_kP_i = L'_k,$$

donde la matriz triangular inferior unitaria L'_k se obtiene de la también triangular inferior unitaria L_k sin más que permutar los coeficientes de las filas i y j (ver figura 1.4).

**Figura 1.4**

Permutaciones elementales en una matriz triangular inferior

DEMOSTRACIÓN. Como $i > k$, la permutación P_i aplicada al vector $\mathbf{e}_k$ deja a éste inalterado: $P_i \mathbf{e}_k = \mathbf{e}_k$. Si $\mathbf{l}_k$ es el vector columna k -ésimo de la matriz elemental, L_k , $\mathbf{l}'_k = P_i \mathbf{l}_k$. En consecuencia,

$$L'_k = P_i(I + \mathbf{l}_k \mathbf{e}_k^T)P_i = P_i^2 + \mathbf{l}'_k \mathbf{e}_k^T = I + \mathbf{l}'_k \mathbf{e}_k^T,$$

donde el vector columna $\mathbf{l}'_k$ se obtiene a partir de $\mathbf{l}_k$ permutando las componentes i y j . ■

Lema 1.2 Sea $\{\mathbf{l}_k\}$, $1 \leq k \leq n-1$, una sucesión de $n-1$ vectores de dimensión n tales que los k primeros componentes de $\mathbf{l}_k$ son nulos. Se cumple que

$$(I + \mathbf{l}_1 \mathbf{e}_1^T)(I + \mathbf{l}_2 \mathbf{e}_2^T) \cdots (I + \mathbf{l}_{n-1} \mathbf{e}_{n-1}^T) = I + \mathbf{l}_1 \mathbf{e}_1^T + \cdots + \mathbf{l}_{n-1} \mathbf{e}_{n-1}^T.$$

DEMOSTRACIÓN. Para hacerlo es suficiente efectuar el producto de las matrices del término de la izquierda, teniendo en cuenta que los términos que contienen factores del tipo $\mathbf{l}_i \mathbf{e}_i^T \mathbf{l}_j \mathbf{e}_j^T$, con $i < j$, son nulos ya que $\mathbf{e}_i^T \mathbf{l}_j = l_{ji} = 0$. ■

Con estos resultados ya podemos formalizar lo antes dicho sobre la factorización triangular o LU que se obtiene mediante eliminación de Gauss.

Teorema 1.5 Sea A una matriz cuadrada regular de orden n . Existe una matriz de permutación P y dos matrices, una triangular inferior y otra triangular superior, L y U , respectivamente, tales que

$$PA = LU.$$

La matriz L tiene todos los elementos de la diagonal principal igual a 1 (triangular inferior unitaria).

DEMOSTRACIÓN. Del proceso de eliminación de Gauss se tiene que

$$L_{n-1}P_{n-1} \cdots L_1P_1A = U,$$

o, lo que es lo mismo, que

$$A = P_1L'_1P_2L'_2 \cdots P_{n-1}L'_{n-1}U, \quad (1.2)$$

donde las matrices L'_i son las correspondientes inversas de las L_i obtenidas de éstas sin más que cambiar el signo a los elementos de la columna i por debajo de la diagonal principal. Las matrices de permutación, recordemos, son iguales a sus inversas.

Hagamos $P = P_{n-1}P_{n-2} \cdots P_2P_1$ y premultipliquemos los dos miembros de la ecuación (1.2) por P :

$$PA = P_{n-1}P_{n-2} \cdots P_2P_1P_1L'_1P_2L'_2P_3L'_3 \cdots P_{n-2}L'_{n-2}P_{n-1}L'_{n-1}U.$$

De este producto de matrices se sabe que $P_1P_1 = I$, que el producto $P_2L'_1P_2$, según el lema 1.1, da como resultado una matriz L''_1 triangular inferior obtenida a partir de L'_1 sin más que permutar los elementos 2 y j , $j > 2$, de la columna 1; que, según el lema 1.2, $L''_1L'_2$ es otra matriz triangular inferior... En definitiva, que

$$P_{n-1}P_{n-2} \cdots P_2P_1P_1L'_1P_2L'_2P_3L'_3 \cdots P_{n-2}L'_{n-2}P_{n-1}L'_{n-1} = L.$$

De donde, como pretendíamos,

$$PA = LU.$$

■

En términos algebraicos, el proceso de eliminación de Gauss proporciona una factorización $PA = LU$. Para resolver la ecuación $Ax = b$ a partir de esta factorización habría que proceder en dos etapas:

1. Resolviendo el sistema $Ly = Pb$ mediante un proceso de sustitución directa.
2. Resolviendo $Ux = y$ mediante otro proceso de sustitución inversa.

De acuerdo con esto, ya se dispone de una pseudoforma de factorizar numéricamente la matriz A de un sistema de ecuaciones lineales en la forma LU . Estudiemos a continuación las condiciones en las que una matriz cuadrada A admite este tipo de factorización y no sean necesarias, en consecuencia, las permutaciones de filas para conseguir la triangularización de la matriz de coeficientes.

Para enunciar estas condiciones introduzcamos, a partir de la matriz A , las matrices

$$A_k = \begin{bmatrix} a_{11} & \cdots & a_{1k} \\ \vdots & & \vdots \\ a_{k1} & \cdots & a_{kk} \end{bmatrix}, \quad k = 1, \dots, n.$$

Cada matriz A_k es la submatriz principal de la matriz A obtenida con sus primeras k filas y columnas.

Lema 1.3 *La matriz A admite una factorización LU si y sólo si se cumple que*

$$\det(A_k) \neq 0, \quad k = 1, \dots, n.$$

DEMOSTRACIÓN. La necesidad es fácil de comprobar. Si la matriz admite la factorización LU , se cumplen las desigualdades

$$A_k = L_k U_k, \quad k = 1, \dots, n,$$

donde L_k y U_k tienen respecto de las matrices L y U el mismo significado que A_k respecto de A . Esto es debido a la especial estructura triangular de L y de U . De estas desigualdades se deduce que

$$\begin{aligned} \det(A_k) &= \det(L_k) \det(U_k) \\ &= \det(U_k) \\ &= u_{11} \cdots u_{kk}, \quad k = 1, \dots, n. \end{aligned}$$

En particular se cumple que $\det(A) = u_{11} \cdots u_{nn}$ por lo que, como estamos suponiendo que A es una matriz no singular, se sigue que todos los números u_{kk} son distintos de cero y por tanto que todas las matrices A_k son no singulares.

Comprobemos la suficiencia de lo enunciado. El razonamiento es por inducción sobre el orden, n , de la matriz.

Para $n = 1$, la prueba es trivial ya que si $A = [a_{11}]$, basta tomar $L = [1]$ y $U = [a_{11}]$.

Sea $\hat{A}$ una matriz de orden $n + 1$ estructurada en bloques de la forma

$$\hat{A} = \begin{bmatrix} A & \mathbf{p} \\ \mathbf{q}^T & r \end{bmatrix}.$$

En esta matriz, A es la matriz de orden n formada con la primeras n filas y columnas de $\hat{A}$ (con la notación anterior, $A = \hat{A}_n$), $\mathbf{p}$ y $\mathbf{q}$ designan dos vectores columna,

$$\mathbf{p} = \begin{bmatrix} a_{1\ n+1} \\ \vdots \\ a_{n\ n+1} \end{bmatrix} \quad \text{y} \quad \mathbf{q} = \begin{bmatrix} a_{n+1\ 1} \\ \vdots \\ a_{n+1\ n} \end{bmatrix},$$

y, por último, $r = a_{n+1\ n+1}$.

Con la hipótesis indicada de que la matriz A es regular, se puede escribir la identidad

$$\begin{bmatrix} I & \mathbf{0} \\ -\mathbf{q}^T A^{-1} & 1 \end{bmatrix} \begin{bmatrix} A & \mathbf{p} \\ \mathbf{q}^T & r \end{bmatrix} = \begin{bmatrix} A & \mathbf{p} \\ \mathbf{0}^T & r - \mathbf{q}^T A^{-1} \mathbf{p} \end{bmatrix}.$$

Esta identidad, que se comprueba directamente con facilidad, es una generalización de la interpretación matricial de la pivotación, respecto de un elemento, en el algoritmo de eliminación de Gauss. Podríamos decir que la fórmula es la interpretación matricial de la pivotación respecto de una submatriz principal de la matriz A . Es inmediato comprobar que

$$\begin{bmatrix} I & \mathbf{0} \\ -\mathbf{q}^T A^{-1} & 1 \end{bmatrix}^{-1} = \begin{bmatrix} I & \mathbf{0} \\ \mathbf{q}^T A^{-1} & 1 \end{bmatrix}.$$

Con esto y la anterior identidad podemos escribir que

$$\hat{A} = \begin{bmatrix} I & \mathbf{0} \\ \mathbf{q}^T A^{-1} & 1 \end{bmatrix} \begin{bmatrix} A & \mathbf{p} \\ \mathbf{0}^T & r - \mathbf{q}^T A^{-1} \mathbf{p} \end{bmatrix}.$$

Por la hipótesis inductiva podemos suponer que A admite la factorización triangular $A = LU$. Utilizando esta factorización en la representación que acabamos de obtener para $\hat{A}$, se tiene que

$$\begin{aligned}\hat{A} &= \begin{bmatrix} I & \mathbf{0} \\ \mathbf{q}^T U^{-1} L^{-1} & 1 \end{bmatrix} \begin{bmatrix} LU & \mathbf{p} \\ \mathbf{0}^T & r - \mathbf{q}^T U^{-1} L^{-1} \mathbf{p} \end{bmatrix} \\ &= \begin{bmatrix} I & \mathbf{0} \\ \mathbf{q}^T U^{-1} L^{-1} & 1 \end{bmatrix} \begin{bmatrix} L & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} L^{-1} & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} LU & \mathbf{p} \\ \mathbf{0}^T & r - \mathbf{q}^T U^{-1} L^{-1} \mathbf{p} \end{bmatrix} \\ &= \begin{bmatrix} L & \mathbf{0} \\ \mathbf{q}^T U^{-1} & 1 \end{bmatrix} \begin{bmatrix} U & L^{-1} \mathbf{p} \\ \mathbf{0}^T & r - \mathbf{q}^T U^{-1} L^{-1} \mathbf{p} \end{bmatrix}.\end{aligned}$$

Basta entonces tomar

$$\hat{L} = \begin{bmatrix} L & \mathbf{0} \\ \mathbf{q}^T U^{-1} & 1 \end{bmatrix} \quad \text{y} \quad \hat{U} = \begin{bmatrix} U & L^{-1} \mathbf{p} \\ \mathbf{0}^T & r - \mathbf{q}^T U^{-1} L^{-1} \mathbf{p} \end{bmatrix},$$

para con estas matrices poder escribir que

$$\hat{A} = \hat{L} \hat{U}$$

lo cual proporciona la factorización LU de $\hat{A}$. ■

Es interesante destacar de esta última demostración que si se conoce la factorización LU de A , basta resolver los dos sistemas de ecuaciones triangulares $L\mathbf{x} = \mathbf{p}$ y $U^T \mathbf{y} = \mathbf{q}$ para tener la factorización LU de $\hat{A}$. En efecto, si las soluciones de estos sistemas son los vectores columna $\boldsymbol{\xi}$ y $\boldsymbol{\eta}$, la factorización triangular de $\hat{A}$ se escribe:

$$\hat{A} = \begin{bmatrix} L & \mathbf{0} \\ \boldsymbol{\eta}^T & 1 \end{bmatrix} \begin{bmatrix} U & \boldsymbol{\xi} \\ \mathbf{0}^T & r - \boldsymbol{\eta}^T \boldsymbol{\xi} \end{bmatrix}.$$

Esta observación es la base de algunos procedimientos para la obtención directa de la factorización LU de una matriz.

Teorema 1.6 *Si una matriz regular A de orden n admite una factorización $A = LU$, donde L es una matriz triangular inferior de elementos diagonales 1 y U una triangular superior, esa factorización es única.*

DEMOSTRACIÓN. Razonemos por reducción al absurdo. Supongamos que existen dos descomposiciones triangulares de la forma $A = L_1 U_1$ y $A = L_2 U_2$, donde todas las matrices involucradas son regulares. Si $L_1 U_1 = L_2 U_2$, sea

$$X = L_2^{-1} L_1 = U_2 U_1^{-1}.$$

Como $X = L_2^{-1} L_1$, esta matriz es triangular inferior de elementos diagonales 1; como además $X = U_2 U_1^{-1}$, también es triangular superior. Para que se puedan cumplir estas dos condiciones simultáneamente, X debe ser I . Es decir, $L_2 = L_1$ y $U_2 = U_1$ por lo que la descomposición es única. ■

1.4.1 Métodos directos para la obtención de factorizaciones LU

Probado que una matriz admite la factorización triangular o LU , veamos otros métodos más directos para obtenerla que el basado en la eliminación de Gauss.

1.4.1.1 Método de Crout. Versión LU_1

Supongamos que se desea obtener la factorización triangular de una matriz A en la forma LU_1 , donde por U_1 se designa una matriz triangular superior en la que todos los elementos de la diagonal principal son 1. Si, por ejemplo, la matriz A es de orden 3 y se quiere factorizar de la forma

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} 1 & u_{12} & u_{13} \\ 0 & 1 & u_{23} \\ 0 & 0 & 1 \end{bmatrix},$$

efectuando el producto usando las reglas de multiplicación de matrices se obtendrá:

1ª columna de L :

$$\begin{aligned} l_{11} &= a_{11} \\ l_{21} &= a_{21} \\ l_{31} &= a_{31}; \end{aligned}$$

1ª fila de U :

$$\left. \begin{aligned} l_{11}u_{12} &= a_{12} \\ l_{11}u_{13} &= a_{13} \end{aligned} \right\} \longrightarrow u_{1j} = a_{1j}/l_{11}, \quad j = 2, 3;$$

2ª columna de L :

$$\left. \begin{aligned} l_{21}u_{12} + l_{22} &= a_{22} \\ l_{31}u_{12} + l_{32} &= a_{32} \end{aligned} \right\} \longrightarrow l_{i2} = a_{i2} - l_{i1}u_{12}, \quad i = 2, 3;$$

2ª fila de U :

$$l_{21}u_{13} + l_{22}u_{23} = a_{23} \longrightarrow u_{2j} = (a_{2j} - l_{21}u_{1j})/l_{22}, \quad j = 3;$$

3ª columna de L :

$$l_{31}u_{13} + l_{32}u_{23} + l_{33} = a_{33} \longrightarrow l_{i3} = a_{i3} - \sum_{j=1}^{i-1} l_{ij}u_{ji}, \quad i = 3.$$

En general, las fórmulas de recurrencia que se pueden deducir de este proceso, denominado factorización LU de Crout, son:

$$\begin{aligned} l_{i1} &= a_{i1}, & i &= 1, 2, \dots, n, \\ u_{1j} &= a_{1j}/l_{11}, & j &> 1, \\ l_{ik} &= a_{ik} - \sum_{p=1}^{k-1} l_{ip}u_{pk}, & i &\geq k, \\ u_{kj} &= \left(a_{kj} - \sum_{p=1}^{k-1} l_{kp}u_{pj} \right) / l_{kk}, & j &> k. \end{aligned}$$

El algoritmo de Crout para factorizar una matriz regular $A^{n \times n}$ en la forma LU_1 es el que describe la tabla 1.2. Al final del proceso las matrices L y U aparecen en las mismas posiciones de memoria que ocupaban los coeficientes de A .

Tabla 1.2

Algoritmo para la factorización LU_1 de una matriz $A^{n \times n}$ por el método de Crout

```

for  $k = 1$  to  $n$ 
  for  $i = k$  to  $n$ 
     $l(i, k) \leftarrow a(i, k) - \sum_{p=1}^{k-1} l(i, p)u(p, k)$ 
  end
  for  $i = k + 1$  to  $n$ 
     $u(k, i) \leftarrow \left( a(k, i) - \sum_{p=1}^{k-1} l(k, p)u(p, i) \right) / l(k, k)$ 
  end
end

```

La versión en FORTRAN 77 de este algoritmo para factorizar la matriz

$$\begin{bmatrix} 10 & 10 & 20 \\ 20 & 25 & 40 \\ 30 & 50 & 61 \end{bmatrix}$$

se lista a continuación. El resultado es

$$LU = \begin{bmatrix} 10 & & \\ 20 & 5 & \\ 30 & 20 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 2 \\ & 1 & 0 \\ & & 1 \end{bmatrix}.$$

```

PROGRAM Crout
C
  parameter (n = 3)
  real a(n,n)
  data a/10.,20.,30.,10.,25.,50.,20.,40.,61/
C
C *** Factorización LU1 por el método de Crout ***
C
  do k = 1,n
    do i = k,n
      suma = 0.0
      do l = 1,k-1
        suma = suma+a(i,l)*a(l,k)
      end do
      a(i,k) = a(i,k)-suma
    end do
    do i = k+1,n
      suma = 0.0
      do l = 1,k-1
        suma = suma+a(k,l)*a(l,i)
      end do
      a(k,i) = (a(k,i) - suma) / a(k,k)
    end do
  end do

```

```

        end do
        a(k,i) = (a(k,i)-suma)/a(k,k)
    end do
end do
C
    print 20,((a(i,j),j=1,n),i=1,n)
C
20 format(3f7.2)
C
end

```

La secuencia de operaciones asociadas al algoritmo de Crout son: determinar los elementos de la primera columna de la matriz L ; los de la primera fila de la matriz U ; los de la segunda columna de la matriz L ; los de la segunda fila de la matriz U , etc. Para una pequeña matriz 4×4 , el orden en que se calculan los coeficientes de las matrices L y U es el que se indica en el esquema que sigue.

1	5	6	7
2	8	11	12
3	9	13	15
4	10	14	16

El método de Crout y la eliminación de Gauss

Dado que consiguen el mismo objetivo, el algoritmo de Crout se puede comparar con el de eliminación de Gauss. En efecto, la ecuación de la eliminación de Gauss por la que se adaptaban los elementos de la matriz se puede escribir de la forma

$$a(j, k) \leftarrow a(j, k) - a(j, i)a(i, k)/a(i, i)$$

por lo que asociando

$$l(i, k) \equiv a(i, k) \quad \text{y} \quad u(j, i) \equiv a(j, i)/a(i, i),$$

es decir, dividiendo la fila de pivotación por el elemento pivote $a(i, i)$ en lugar de hacerlo en la columna de pivotación, los procedimientos son enteramente equivalentes. La única diferencia estriba, desde el punto de vista de cómo se realizan las operaciones, en que en el método de Crout los productos interiores $l(i, p)u(p, k)$ y $l(k, p)u(p, i)$ se pueden acumular en una operación evitando propagar errores de redondeo; en el de Gauss se calculan paso a paso.

Ejemplo 1.1 Se desea factorizar la matriz

$$A = \begin{bmatrix} 0,001 & 2,000 & 3,000 \\ -1,000 & 3,712 & 4,623 \\ -2,000 & 1,072 & 5,643 \end{bmatrix}$$

en una máquina u ordenador con cuatro dígitos significativos.

Las operaciones que se realizan en la máquina son:

$$l_{11} = 0,001;$$

$$l_{21} = -1,000;$$

$$\begin{aligned}l_{31} &= -2,000; \\u_{12} &= fl\left(\frac{2,000}{0,001}\right) = 2000; \\u_{13} &= fl\left(\frac{3,000}{0,001}\right) = 3000; \\l_{22} &= fl[3,712 + (1,000)(2000)] = 2004; \\l_{32} &= fl[1,072 + (2,000)(2000)] = 4001; \\u_{23} &= fl\left(\frac{4,623 + (1,000)(3000)}{2004}\right) = 1,500 \text{ y} \\l_{33} &= fl[5,643 + (2,000)(3,000) - (4,001)(1,500)] = 5,642.\end{aligned}$$

Obsérvese que el cálculo de l_{33} conlleva la pérdida de tres dígitos significativos: el valor que debería obtenerse es 5,922. ●

Pivotación

El último ejemplo pone de manifiesto que, aunque se sepa que una matriz no es singular y que su factorización LU existe, e independientemente de que se use un procedimiento algorítmicamente adecuado, los errores de redondeo que se pueden producir al calcularla pueden dar al traste con el resultado. En el procedimiento de Crout, en concreto, el efecto de esos errores de redondeo pueden paliarse en gran medida recurriendo, como en el caso de la eliminación de Gauss, a la pivotación parcial. El principal obstáculo para incorporar intercambios de filas en este algoritmo es que no se sabe que un l_{kk} es pequeño hasta que no se ha calculado. Una vez hecho, un intercambio de filas en la matriz A cambiaría su descomposición LU , por lo que habría que rehacerla. Afortunadamente existe una relación muy simple entre la descomposición LU obtenida con el algoritmo de Crout y la matriz que se obtendría intercambiando dos filas de esa factorización.

Para ver esta relación, supongamos que se tiene una matriz de orden 5 a la que se le intercambian la fila 3 y la 5; resultará una A' dada por

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} \\ a_{41} & a_{42} & a_{43} & a_{44} & a_{45} \\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix}.$$

Si a esta matriz se le aplica el algoritmo de Crout, parándose, cuando $k = 3$, antes de calcular los elementos de la matriz u_{34} y u_{35} , se llegará a

$$\begin{bmatrix} l_{11} & u_{12} & u_{13} & u_{14} & u_{15} \\ l_{21} & l_{22} & u_{23} & u_{24} & u_{25} \\ l_{51} & l_{52} & l_{53} & a_{54} & a_{55} \\ l_{41} & l_{42} & l_{43} & a_{44} & a_{45} \\ l_{31} & l_{32} & l_{33} & a_{34} & a_{35} \end{bmatrix}.$$

Es decir, la única diferencia es la que resulta del intercambio de las filas 3 y 5: la pivotación parcial por consiguiente no plantea ningún problema significativo en el método de Crout.

El algoritmo de Crout con pivotación parcial se describe en la tabla 1.3.

El efecto de las pivotaciones es que, igual que en el caso de la eliminación de Gauss, en lugar de A , realmente se factoriza una matriz PA . La versión en FORTRAN 77 de este nuevo algoritmo para factorizar otra vez la matriz

$$\begin{bmatrix} 10 & 10 & 20 \\ 20 & 25 & 40 \\ 30 & 50 & 61 \end{bmatrix},$$

se lista a continuación. Al final de este proceso, el vector $\text{IPVT}(\cdot)$, que indica las pivotaciones realizadas, es $[3, 2, 1]^T$. Esto quiere decir que la matriz PA factorizada es

$$\begin{bmatrix} 30 & 50 & 61 \\ 20 & 25 & 40 \\ 10 & 10 & 20 \end{bmatrix} = \begin{bmatrix} 30 & & \\ 20 & -8,3333 & \\ 10 & -6,6667 & 0,2 \end{bmatrix} \begin{bmatrix} 1 & 1,6667 & 2,0333 \\ & 1 & 0,0800 \\ & & 1 \end{bmatrix}.$$

```

PROGRAM Croutp
C
  parameter (n = 3)
  real a(n,n)
  integer ipvt(n)
C
  data a/10.,20.,30.,10.,25.,50.,20.,40.,61/
C
  do i = 1,n
    ipvt(i) = i
  end do
C
C *** Factorización LU1 con pivotación por el método de Crout ***

```

Tabla 1.3

Algoritmo de Crout con pivotación parcial para la factorización LU_1 de una matriz $A^{n \times n}$

```

for  $k = 1$  to  $n$ 
  for  $i = k$  to  $n$ 
    
$$l(i, k) \leftarrow a(i, k) - \sum_{p=1}^{k-1} l(i, p)u(p, k)$$

  end
  Determinar índice  $p \in \{k, k+1, \dots, n\}$  tal que  $|a(p, i)| = \max_{i \leq j \leq n} |a(j, i)|$ .
  Intercambiar filas  $p$  y  $k$ .
  for  $i = k+1$  to  $n$ 
    
$$u(k, i) \leftarrow \left( a(k, i) - \sum_{p=1}^{k-1} l(k, p)u(p, i) \right) / l(k, k)$$

  end
end

```



```
C
do k = 1,n
  l = 0
  smax = 0.0
  do i = k,n
    suma = 0.0
    do l = 1,k-1
      suma = suma+a(i,l)*a(l,k)
    end do
    a(i,k) = a(i,k)-suma
    if (abs(a(i,k)).gt.smax) then
      smax = abs(a(i,k))
      l = i
    endif
  end do
  if (l.ne.0) then
    do j = 1,n
      aux = a(l,j)
      a(l,j) = a(k,j)
      a(k,j) = aux
    end do
   iaux = ipvt(l)
    ipvt(l) = ipvt(k)
    ipvt(k) =iaux
  endif
  do i = k+1,n
    suma = 0.0
    do l = 1,k-1
      suma = suma+a(k,l)*a(l,i)
    end do
    a(k,i) = (a(k,i)-suma)/a(k,k)
  end do
end do

C
print *,(ipvt(j),j=1,n)
print *,((a(i,j),j=1,n),i=1,n)

C
end
```

El algoritmo de Crout también requiere para la factorización de la matriz $\boxed{O(n^3/3)}$ operaciones de multiplicación/división y suma/resta.

1.4.1.2 Método de Crout. Versión L_1U

Si en vez de querer obtener la factorización LU_1 de una determinada matriz A , se desea una L_1U por un método más directo que la eliminación de Gauss, una variante del método de Crout permite llegar a ella fácilmente.

Su desarrollo sigue las mismas ideas que en el caso LU_1 . Si se pretende conseguir la descomposición L_1U de una matriz de orden 3 de la forma

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix},$$

operando de acuerdo con las reglas de multiplicación matricial se obtendrá:

1ª fila de U :

$$u_{11} = a_{11}$$

$$u_{12} = a_{12}$$

$$u_{13} = a_{13};$$

1ª columna de L :

$$\left. \begin{array}{l} l_{21}u_{11} = a_{21} \\ l_{31}u_{11} = a_{31} \end{array} \right\} \longrightarrow l_{i1} = a_{i1}/u_{11}, \quad i = 2, 3;$$

2ª fila de U :

$$\left. \begin{array}{l} l_{21}u_{12} + u_{22} = a_{22} \\ l_{21}u_{13} + u_{23} = a_{23} \end{array} \right\} \longrightarrow u_{2j} = a_{2j} - l_{21}u_{1j}, \quad j = 2, 3;$$

2ª columna de L :

$$l_{31}u_{12} + l_{32}u_{22} = a_{32} \longrightarrow l_{i2} = (a_{i2} - l_{i1}u_{12})/u_{22}, \quad i = 3.$$

3ª fila de U :

$$l_{31}u_{13} + l_{32}u_{23} + u_{33} = a_{33} \longrightarrow u_{3j} = a_{3j} - \sum_{i=1}^{j-1} l_{3i}u_{ij}, \quad j = 3.$$

Las fórmulas de recurrencia que se pueden deducir de este proceso son:

$$\begin{aligned} u_{1j} &= a_{1j}, & j &= 1, 2, \dots, n, \\ l_{i1} &= a_{i1}/u_{11}, & j &> 1, \\ u_{kj} &= a_{kj} - \sum_{p=1}^{k-1} l_{kp}u_{pj}, & j &\geq k, \\ l_{ik} &= \left(a_{ik} - \sum_{p=1}^{k-1} l_{ip}u_{pk} \right) / u_{kk}, & i &> k. \end{aligned}$$

El algoritmo para factorizar una matriz regular $A^{n \times n}$ en la forma L_1U por el método de Crout se describe en la tabla 1.4. Como en la versión para factorizar una matriz en la forma LU_1 , las matrices L y U aparecen al final del proceso en las mismas posiciones de memoria que ocupaban los coeficientes de A .

La versión en FORTRAN 77 de este algoritmo para factorizar nuevamente la matriz

$$\begin{bmatrix} 10 & 10 & 20 \\ 20 & 25 & 40 \\ 30 & 50 & 61 \end{bmatrix},$$

es la que sigue a este párrafo. El resultado es

$$LU = \begin{bmatrix} 1 & & \\ 2 & 1 & \\ 3 & 4 & 1 \end{bmatrix} \begin{bmatrix} 10 & 10 & 20 \\ & 5 & 0 \\ & & 1 \end{bmatrix}.$$

Tabla 1.4Algoritmo para la factorización L_1U de una matriz $A^{n \times n}$ por el método de Crout

```
for k = 1 to n
  for j = k to n
    
$$u(k, j) \leftarrow a(k, j) - \sum_{p=1}^{k-1} l(k, p)u(p, j)$$

  end
  for i = k + 1 to n
    
$$l(i, k) \leftarrow \left( a(i, k) - \sum_{p=1}^{k-1} l(i, p)u(p, k) \right) / u(k, k)$$

  end
end
```

```
PROGRAM Crout11u
C
  parameter (n = 3)
  real a(n,n)
C
  data a/10.,20.,30.,10.,25.,50.,20.,40.,61/
C
C *** Factorización L1U por el método de Crout ***
C
  do k = 1,n
    do j = k,n
      sum = 0.0
      do l = 1,k-1
        sum = sum+a(k,l)*a(l,j)
      end do
      a(k,j) = a(k,j)-sum
    end do
    do i = k+1,n
      sum = 0.0
      do l = 1,k-1
        sum = sum+a(i,l)*a(l,k)
      end do
      a(i,k) = (a(i,k)-sum)/a(k,k)
    end do
  end do
C
  print 20,((a(i,j),j=1,n),i=1,n)
C
  20 format(3f7.2)
C
  end
```

1.4.1.3 Método de Doolittle

El método de Doolittle es una variante del de Crout que obtiene las matrices de la factorización, L y U , fila a fila o columna a columna. Resulta particularmente útil para matrices de grandes

dimensiones de las que sólo se guardan, fila a fila o columna a columna, los elementos distintos de cero, o para implementarse en ordenadores con arquitectura en paralelo.

Para la factorización de la matriz, el algoritmo de Doolittle también requiere $O(n^3/3)$ operaciones de multiplicación/división y suma/resta.

La versión de este algoritmo que obtiene una factorización L_1U , generándose L_1 y U columna a columna, es la que describe la tabla 1.5.

Tabla 1.5

Algoritmo para la factorización L_1U de una matriz $A^{n \times n}$ por el método de Doolittle. Los coeficientes de los factores se generan por columnas

```

for  $k = 1$  to  $n$ 
  for  $i = 1$  to  $k - 1$ 
     $u(i, k) \leftarrow a(i, k) - \sum_{p=1}^{i-1} l(i, p)u(p, k)$ 
  end
  for  $i = k$  to  $n$ 
     $l(i, k) \leftarrow \left( a(i, k) - \sum_{p=1}^{k-1} l(i, p)u(p, k) \right) / u(k, k)$ 
  end
end

```

Su codificación completa en FORTRAN 77, incluida la pivotación parcial, para factorizar directamente la matriz

$$\begin{bmatrix} 1 & -4 & 1 & 1 \\ 1 & 0 & 1 & 3 \\ -2 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

es la que sigue. La descomposición resultante es

$$LU = \begin{bmatrix} 1 & & & \\ -0,5 & 1 & & \\ 0 & 0 & 1 & \\ -0,5 & 0 & 0,5 & 1 \end{bmatrix} \begin{bmatrix} -2 & 0 & -1 & 0 \\ & -4 & 0,5 & 1 \\ & & 1 & 0 \\ & & & 3 \end{bmatrix}.$$

El vector $IPVT(\cdot)$ que indica las pivotaciones realizadas en el transcurso de esta factorización es $[3, 1, 4, 2]$; la matriz PA realmente factorizada es por lo tanto

$$\begin{bmatrix} -2 & 0 & -1 & 0 \\ 1 & -4 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 3 \end{bmatrix}.$$

```
PROGRAM Dool
C
parameter (n = 4)
integer ipvt(n)
real a(n,n)
C
data a/1.,1.,-2.,0.,-4.,0.,0.,0.,1.,1.,-1.,1.,1.,3.,0.,0./
C
do i = 1,n
    ipvt(i) = i
end do
C
C *** Factorización L1U con pivotación por el método de Doolittle ***
C
do j = 1,n
    do i = 1,j-1
        suma = a(i,j)
        do k = 1,i-1
            suma = suma-a(i,k)*a(k,j)
        end do
        a(i,j) = suma
    end do
    amax = 0.0
    do i = j,n
        suma = a(i,j)
        do k = 1,j-1
            suma = suma-a(i,k)*a(k,j)
        end do
        a(i,j) = suma
        if (abs(suma).ge.amax) then
            imax = i
            amax = abs(suma)
        endif
    end do
    if (j.ne.imax) then
        do k = 1,n
            dum = a(imax,k)
            a(imax,k) = a(j,k)
            a(j,k) = dum
        end do
       iaux = ipvt(imax)
        ipvt(imax) = ipvt(j)
        ipvt(j) =iaux
    endif
    if (j.ne.n) then
        if (a(j,j).eq.0.0) a(j,j) = 1.0e-20 ! Se divide la columna j
        dum = 1.0/a(j,j) ! por A(j,j)
        do i = j+1,n
            a(i,j) = a(i,j)*dum
        end do
    endif
end do
if (a(n,n).eq.0.0) a(n,n) = 1.0e-20
C
print *,ipvt
print 1,((a(i,j),j=1,n),i=1,n)
C
1 format(4f8.3)
C
```

end

Obsérvese lo poco que se complica el algoritmo, con respecto a los hasta ahora presentados, al calcular los elementos de la factorización por columnas e incluir la pivotación.

En la figura 1.5 se esquematiza la factorización de Doolittle por columnas: se indica cómo se van obteniendo los elementos de las matrices L y U y qué elementos de los ya calculados son utilizados para obtener uno nuevo.

1.5 Factorización de matrices simétricas

Como venimos insistiendo, uno de los principios básicos que debe presidir la resolución numérica de cualquier problema lineal o de optimización es sacar partido de la posible estructura especial de que disponga el problema a tratar.

En álgebra lineal numérica, y concretamente en la resolución de sistemas de ecuaciones lineales, este principio resulta si cabe más decisivo cuando alguna de las matrices a manipular es simétrica, definida positiva, dispersa, resultante de la representación de la estructura nudo-arco de un grafo, etc. El número de operaciones que resulte de tener en cuenta estos hechos debe resultar sustancialmente inferior al de no hacerlo.

En lo que sigue nos ocupamos de la factorización de matrices simétricas y, cuando ese sea el caso, definidas positivas. Casos más particulares como matrices en banda, tridiagonales, etc, no los estudiamos aunque ya se comprende la reducción potencial que puede acarrear el modificar la mecánica de los procedimientos vistos hasta ahora y los que vamos a ver para tenerlos en cuenta.

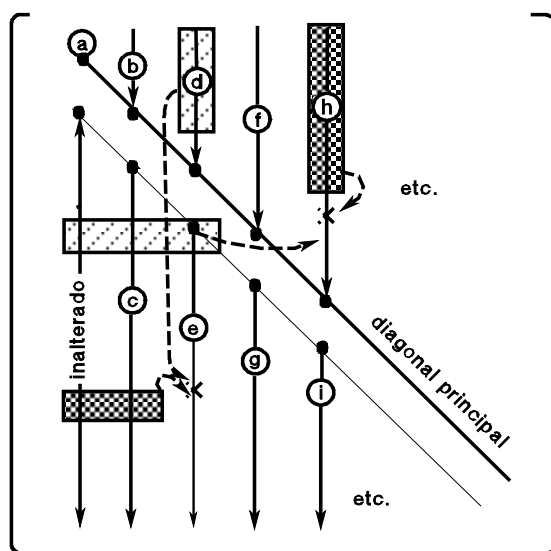


Figura 1.5

Ilustración del proceso del algoritmo de Doolittle para la factorización LU por columnas de una matriz

1.5.1 Factorización LDL^T

Particularicemos alguno de los resultados obtenidos para matrices generales al caso de matrices simétricas.

Lema 1.4 *Si todas las submatrices principales de una matriz $A \in \mathbb{R}^{n \times n}$ son regulares, existen dos matrices triangulares inferiores unitarias únicas, L y M , y otra diagonal también única, $D = \text{diag}(d_1, \dots, d_n)$, tales que $A = LDM^T$.*

DEMOSTRACIÓN. Según el resultado del lema 1.3, $A = LU$, donde L es una matriz triangular inferior unitaria y U una triangular superior. Sea $D = \text{diag}(d_1, \dots, d_n)$, donde $d_i = u_{ii}$, $i = 1, \dots, n$. Obsérvese que la matriz D es regular. Si se hace $M^T = D^{-1}U$, ésta es una matriz triangular superior unitaria. Ahora bien,

$$A = LU = LD(D^{-1}U) = LDM^T.$$

La unicidad de L , M y D se deriva de la de la factorización $A = LU$ según el teorema 1.6. ■

Teorema 1.7 *Si A admite una factorización LDM^T y es simétrica, $L = M$.*

DEMOSTRACIÓN. La matriz $M^{-1}AM^{-T} = M^{-1}LD$ es simétrica y triangular inferior, por consiguiente es diagonal. Como D es regular, esto implica que $M^{-1}L$ es también diagonal. Ahora bien, $M^{-1}L$ es triangular inferior unitaria, luego $M^{-1}L = I$.

También se puede demostrar teniendo en cuenta que según el teorema 1.6, si A admite la factorización LDM^T , ésta es única. Ahora bien, como es simétrica,

$$A = A^T = MDL^T = LDM^T \Rightarrow L = M.$$

■

La factorización LDL^T resulta de gran utilidad cuando la matriz es simétrica pero no se sabe con seguridad si es definida positiva o no. Para desarrollar un algoritmo para su obtención directa se puede proceder de la misma manera que cuando se estudió el algoritmo de Crout, es decir, estableciendo unas fórmulas de recurrencia del método a partir de un ejemplo simbólico de orden 3:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} d_{11} & & \\ & d_{22} & \\ & & d_{33} \end{bmatrix} \begin{bmatrix} 1 & l_{21} & l_{31} \\ 0 & 1 & l_{32} \\ 0 & 0 & 1 \end{bmatrix}.$$

Operando de acuerdo con las reglas de multiplicación matricial se obtiene:

$$\begin{array}{rcl} a_{11} & = & d_{11} \\ \hline a_{21} & = & l_{21}d_{11} \\ a_{31} & = & l_{31}d_{11} \\ \hline a_{22} & = & l_{21}^2d_{11} + d_{22} \\ a_{32} & = & l_{31}l_{21}d_{11} + l_{32}d_{22} \\ \hline a_{33} & = & l_{31}^2d_{11} + l_{32}^2d_{22} + d_{33}. \end{array}$$

Generalizando este proceso se obtiene el algoritmo que describe la tabla 1.6.

Este algoritmo requiere $\boxed{O(n^3/6)}$ operaciones de multiplicación/división y suma/resta.

Tabla 1.6

Algoritmo para la factorización LDL^T de una matriz $A^{n \times n}$ simétrica

```

for  $k = 1$  to  $n$ 
     $d(k) \leftarrow a(k, k) - \sum_{p=1}^{k-1} a^2(k, p)d(p)$ 
    if  $d(k) = 0$  then stop
    for  $i = k + 1$  to  $n$ 
         $a(i, k) \leftarrow \left( a(i, k) - \sum_{p=1}^{k-1} a(i, p)a(k, p)d(p) \right) / d(k)$ 
    end
end

```

1.5.2 Factorización de Cholesky

Los métodos expuestos hasta ahora pueden fallar si no se efectúan pivotaciones parciales o totales debido, por un lado, a la posible presencia de elementos pivote muy pequeños, o a la acumulación de errores de redondeo importantes, por otro. Existe una clase muy importante de matrices para las cuales no es necesario efectuar esas operaciones si se desean factorizar en forma triangular: nos referimos a las matrices simétricas definidas positivas. En este caso las matrices admiten una descomposición de la forma

$$A = G^T G,$$

donde G es una matriz triangular superior. Esta descomposición fue desarrollada por André Louis Cholesky (1875-1918), comandante del ejército francés de la época, durante la ocupación internacional de Creta entre 1906 y 1909. La utilizó por primera vez en unos trabajos sobre estudios geodésicos para calcular la solución de problemas de ajustes por mínimos cuadrados.

Las matrices simétricas definidas positivas se presentan habitualmente en problemas relacionados con el análisis de sistemas eléctricos de generación y transporte de energía, ajuste de funciones por mínimos cuadrados, análisis de estructuras mecánicas y en muchos procedimientos de optimización lineal y no lineal. En general aparecen en todas aquellas aplicaciones donde al modelizar un sistema, la expresión $\mathbf{x}^T A \mathbf{x}$ mide la energía presente o disponible en un entorno determinado: esta energía habitualmente es una cantidad positiva.

Recordemos que una matriz se dice *definida positiva* si para todo $\mathbf{x} \neq \mathbf{0}$ se cumple que

$$\mathbf{x}^T A \mathbf{x} > 0.$$

También recordemos que todos los autovalores de una matriz definida positiva son positivos.

Lema 1.5 *Las submatrices principales de una matriz definida positiva son definidas positivas.*

DEMOSTRACIÓN. Sea A' la submatriz principal de A formada por sus r primeras filas y columnas. Sea $\mathbf{x}' \neq \mathbf{0}$ un vector r -dimensional y $\mathbf{x}$ otro vector n -dimensional definido de la siguiente

manera:

$$\begin{aligned} x_k &= x'_k & k &= 1, 2, \dots, r, \\ x_j &= 0 & j &= r+1, \dots, n. \end{aligned}$$

De esta definición se deduce que $\mathbf{x} \neq \mathbf{0}$ y que $\mathbf{x}^T A \mathbf{x} = \mathbf{x}'^T A' \mathbf{x}'$. Como A es definida positiva, se tiene que

$$0 < \mathbf{x}^T A \mathbf{x} = \mathbf{x}'^T A' \mathbf{x}',$$

por lo que la submatriz principal A' es positiva definida. ■

Teorema 1.8 *Si A es una matriz definida positiva de orden n , tiene una descomposición de la forma LDM^T , siendo todos los elementos de la matriz diagonal D positivos.*

DEMOSTRACIÓN. Como todas las submatrices principales de una matriz definida positiva son definidas positivas y por tanto regulares, de acuerdo con el resultado del lema 1.4, existen dos matrices triangulares inferiores unitarias L y M y una diagonal $D = \text{diag}(d_1, \dots, d_n)$ tales que $A = LDM^T$. Como la matriz $S = L^{-1}AL^{-T} = DM^TL^{-T}$ es definida positiva (sus autovalores son los mismos de A por ser L triangular unitaria) y triangular superior con $s_{ii} = d_i$, los d_i han de ser positivos. ■

A continuación se demuestra, de forma muy similar a como se hizo en el caso de la descomposición LU , la existencia de la descomposición de Cholesky de una matriz simétrica definida positiva.

Teorema 1.9 *Si A es una matriz simétrica definida positiva de orden n , existe una única matriz triangular superior, G , con todos sus elementos diagonales positivos, tal que $A = G^T G$.*

DEMOSTRACIÓN. Procederemos por inducción respecto al orden de la matriz A . Si A es de orden 1 y definida positiva, la matriz G está definida de forma trivial por $g_{11} = \sqrt{a_{11}}$.

Supongamos que lo enunciado se cumple para matrices de orden $n-1$ y que A' es una matriz definida positiva de orden n . Como es simétrica, se puede estructurar de la forma

$$A' = \begin{bmatrix} A & \mathbf{a} \\ \mathbf{a}^T & \alpha \end{bmatrix}.$$

Según el lema 1.5, esta matriz A' es definida positiva.

Para encontrar una matriz G' tal que $A' = G'^T G'$, definamos esa G' como

$$G' = \begin{bmatrix} G & \mathbf{g} \\ \mathbf{0}^T & \lambda \end{bmatrix}.$$

Como $A = G^T G$, si desarrollamos el producto $G'^T G'$, se tiene que

$$G'^T G' = \begin{bmatrix} G^T & \mathbf{0} \\ \mathbf{g}^T & \lambda \end{bmatrix} \begin{bmatrix} G & \mathbf{g} \\ \mathbf{0} & \lambda \end{bmatrix} = \begin{bmatrix} G^T G & G^T \mathbf{g} \\ \mathbf{g}^T G & \mathbf{g}^T \mathbf{g} + \lambda^2 \end{bmatrix}.$$

Haciéndola igual a A' se tiene que

$$\begin{bmatrix} G^T G & G^T \mathbf{g} \\ \mathbf{g}^T G & \mathbf{g}^T \mathbf{g} + \lambda^2 \end{bmatrix} = \begin{bmatrix} A & \mathbf{a} \\ \mathbf{a}^T & \alpha \end{bmatrix}.$$

Es decir, se requiere que

$$G^T \mathbf{g} = \mathbf{a}, \quad (1.3)$$

$$\mathbf{g}^T G = \mathbf{a}^T \quad \text{y} \quad (1.4)$$

$$\mathbf{g}^T \mathbf{g} + \lambda^2 = \alpha.$$

Por la hipótesis de inducción, G es única. Como G es no singular, $\mathbf{g} = G^{-T} \mathbf{a}$ es el único vector que satisface (1.3) y (1.4). Por último, si $\alpha - \mathbf{g}^T \mathbf{g} > 0$, el valor de λ lo define de forma única la expresión $\sqrt{\alpha - \mathbf{g}^T \mathbf{g}}$.

Para comprobar que efectivamente $\alpha - \mathbf{g}^T \mathbf{g} > 0$, obsérvese en primer lugar que la regularidad de la matriz G implica la regularidad de A . Sin pérdida de generalidad, como A es regular, para cualquier vector $\mathbf{b}$, $\mathbf{a}$ se puede expresar de la forma $\mathbf{a} = A\mathbf{b}$; por lo tanto $\mathbf{b} = A^{-1}\mathbf{a}$. Como A es definida positiva, se tiene que

$$\begin{aligned} 0 &< [\mathbf{b}^T, -1] \begin{bmatrix} A & \mathbf{a} \\ \mathbf{a}^T & \alpha \end{bmatrix} \begin{bmatrix} \mathbf{b} \\ -1 \end{bmatrix} \\ &= \mathbf{b}^T A \mathbf{b} - 2\mathbf{b}^T \mathbf{a} + \alpha \\ &= \alpha - \mathbf{b}^T \mathbf{a} \\ &= \alpha - \mathbf{a}^T A^{-1} \mathbf{a} \\ &= \alpha - \mathbf{a}^T (G^T G)^{-1} \mathbf{a} \\ &= \alpha - \mathbf{a}^T G^{-1} G^{-T} \mathbf{a} \\ &= \alpha - \mathbf{g}^T \mathbf{g}. \end{aligned}$$

■

La demostración de este último teorema define implícitamente un método para obtener la descomposición de Cholesky de una matriz simétrica definida positiva calculando sucesivamente las descomposiciones de sus submatrices principales. Implícitamente contiene el algoritmo que se obtendrá a continuación.

Para determinar el algoritmo de descomposición de Cholesky de una forma intuitiva y directa, procedamos como lo hemos hecho en factorizaciones anteriores simulando la descomposición simbólica de una matriz 3×3 . Es decir, si se desea obtener la factorización

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix} = \begin{bmatrix} g_{11} & 0 & 0 \\ g_{12} & g_{22} & 0 \\ g_{13} & g_{23} & g_{33} \end{bmatrix} \begin{bmatrix} g_{11} & g_{12} & g_{13} \\ 0 & g_{22} & g_{23} \\ 0 & 0 & g_{33} \end{bmatrix},$$

operando de acuerdo con las reglas de multiplicación matricial se obtiene que:

$$\begin{array}{rcl} a_{11} & = & g_{11}^2 \\ a_{12} & = & g_{11}g_{12} \\ a_{13} & = & g_{11}g_{13} \\ \hline a_{22} & = & g_{12}^2 + g_{22}^2 \\ a_{23} & = & g_{12}g_{13} + g_{22}g_{23} \\ \hline a_{33} & = & g_{13}^2 + g_{23}^2 + g_{33}^2. \end{array}$$

Generalizando este proceso se obtiene el algoritmo que describe la tabla 1.7.

Tabla 1.7

Algoritmo para la factorización $G^T G$ de Cholesky por filas de una matriz $A^{n \times n}$ simétrica definida positiva

```

for  $i = 1$  to  $n$ 
   $g(i, i) \leftarrow \sqrt{a(i, i) - \sum_{k=1}^{i-1} g^2(k, i)}$ 
  for  $j = i + 1$  to  $n$ 
     $g(i, j) \leftarrow \left( a(i, j) - \sum_{k=1}^{i-1} g(k, i)g(k, j) \right) / g(i, i)$ 
  end
end

```

La codificación completa de este algoritmo en FORTRAN 77, incluida la resolución del sistema

$$G^T G \mathbf{x} = \mathbf{b},$$

para resolver

$$\begin{bmatrix} 5 & 1 & -2 & 0 \\ 1 & 2 & 0 & 0 \\ -2 & 0 & 4 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \\ 14 \\ 15 \end{bmatrix},$$

es la que sigue. La parte triangular superior de la matriz original, A , se sustituye por el factor G . La factorización que se obtiene es

$$G = \begin{bmatrix} 2,2361 & 0,4472 & -0,8944 & 0 \\ & 1,3416 & 0,2981 & 0 \\ & & 1,7638 & 0,5669 \\ & & & 1,6366 \end{bmatrix}.$$

La solución del problema es $[1, 2, 3, 4]^T$.

```

      PROGRAM Chol
C
      parameter (n = 4)
      real a(n,n),b(n)
      integer i,j,k
C
      data a/5.,1.,-2.,0.,1.,2.,0.,0.,-2.,0.,4.,1.,0.,0.,1.,3./
      data b/1.,5.,14.,15./
C
C                                     T
C *** Factorización de Cholesky G G ***
C
      do i = 1,n
        suma = a(i,i)
        do k = 1,i-1
          suma = suma-a(k,i)**2
        end do
        a(i,i) = sqrt(suma)
        do j = i+1,n
          suma = a(i,j)
          do k = 1,i-1
            suma = suma-a(k,i)*a(k,j)
          end do
          a(i,j) = suma/a(i,i)
        end do
      end do
C
C *** Sustitución directa
C
      do i = 1,n
        do j = 1,i-1
          b(i) = b(i)-a(j,i)*b(j)
        end do
        b(i) = b(i)/a(i,i)
      end do
C
C *** Sustitución inversa
C
      b(n) = b(n)/a(n,n)
      do i = n-1,1,-1
        do j = i+1,n
          b(i) = b(i)-a(i,j)*b(j)
        end do
        b(i) = b(i)/a(i,i)
      end do
C
      print 1,((a(i,j),j=1,n),i=1,n)
      print 1,b
C
      1 format(4f9.4)
C
      end

```

Recordemos que para resolver un sistema $G^T G \mathbf{x} = \mathbf{b}$, primero se resuelve $G^T \mathbf{y} = \mathbf{b}$ por sustitución directa y luego $G \mathbf{x} = \mathbf{y}$ por sustitución inversa. En el programa presentado los vectores $\mathbf{y}$ y $\mathbf{x}$ ocupan las mismas posiciones de memoria que el término independiente $\mathbf{b}$; la matriz G ocupará la parte triangular superior de la matriz original A .

El algoritmo descrito en la tabla 1.7 es la versión fila a fila de la factorización de Cholesky. Es posible también modificar la secuencia de operaciones que se realizan de tal forma que se

obtenga columna a columna. En efecto, volviendo a la descomposición simbólica,

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{bmatrix} = \begin{bmatrix} g_{11} & 0 & 0 \\ g_{12} & g_{22} & 0 \\ g_{13} & g_{23} & g_{33} \end{bmatrix} \begin{bmatrix} g_{11} & g_{12} & g_{13} \\ 0 & g_{22} & g_{23} \\ 0 & 0 & g_{33} \end{bmatrix},$$

operando para obtener columna a columna G se obtiene lo siguiente:

$$\begin{array}{rcl} g_{11} & = & \sqrt{a_{11}} \\ g_{12} & = & a_{12}/g_{11} \\ g_{22} & = & \sqrt{a_{22} - g_{12}^2} \\ g_{13} & = & a_{13}/g_{11} \\ g_{23} & = & (a_{23} - g_{12}g_{13})/g_{22} \\ g_{33} & = & \sqrt{a_{33} - g_{13}^2 - g_{23}^2}. \end{array}$$

La generalización de este proceso se describe en la tabla 1.8.

Tabla 1.8

Algoritmo para la factorización $G^T G$ de Cholesky por columnas de una matriz $A^{n \times n}$ simétrica definida positiva

```

for  $j = 1$  to  $n$ 
  for  $i = 1$  to  $j - 1$ 
     $g(i, j) \leftarrow \left( a(i, j) - \sum_{k=1}^{i-1} g(k, i)g(k, j) \right) / g(i, i)$ 
  end
   $g(j, j) \leftarrow \sqrt{a(j, j) - \sum_{k=1}^{j-1} g^2(k, j)}$ 
end

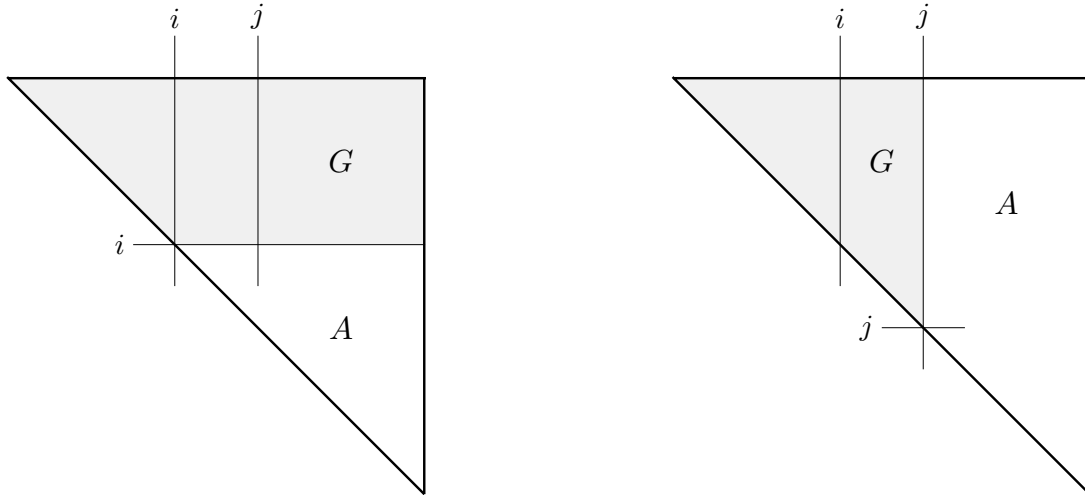
```

La secuencia de las diferentes operaciones del algoritmo de Cholesky por filas y por columnas se describen en la figura 1.6.

El algoritmo para descomponer una matriz simétrica definida positiva en la forma de Cholesky requiere $\boxed{O(n^3/6)}$ operaciones de multiplicación/división y de suma/resta.

1.5.3 Matrices simétricas semidefinidas positivas

Recordemos que una matriz A se dice *semidefinida positiva* si para todo vector $\mathbf{x} \neq \mathbf{0}$, $\mathbf{x}^T A \mathbf{x} \geq 0$.

**Figura 1.6**

Partes ya calculadas y por calcular de la factorización de Cholesky for filas (etapa i) y por columnas (etapa j) de una matriz A

Teorema 1.10 Si $A \in \mathbb{R}^{n \times n}$ es simétrica semidefinida positiva, se cumple que

$$|a_{ij}| \leq (a_{ii} + a_{jj})/2 \quad (1.5)$$

$$|a_{ij}| \leq \sqrt{a_{ii}a_{jj}} \quad (i \neq j) \quad (1.6)$$

$$\max_{i,j} |a_{ij}| = \max_i a_{ii} \quad (1.7)$$

$$a_{ii} = 0 \Rightarrow a_{ij} = a_{ji} = 0, \quad j = 1, \dots, n. \quad (1.8)$$

DEMOSTRACIÓN. Si $\mathbf{x} = \mathbf{e}_i + \mathbf{e}_j$ entonces $0 \leq \mathbf{x}^T A \mathbf{x} = a_{ii} + a_{jj} + 2a_{ij}$. Si por otro lado $\mathbf{x} = \mathbf{e}_i - \mathbf{e}_j$, entonces $0 \leq \mathbf{x}^T A \mathbf{x} = a_{ii} + a_{jj} - 2a_{ij}$. La desigualdad (1.5) se deduce de estos dos últimos resultados. La ecuación (1.7), que expresa el hecho de que el coeficiente de mayor valor absoluto de la matriz está en la diagonal principal, es consecuencia inmediata de (1.5).

Para demostrar la desigualdad (1.6), supongamos sin pérdida de generalidad que $i = 1$ y $j = 2$ y consideremos la desigualdad

$$0 \leq [x, y] \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = a_{11}x^2 + 2a_{12}xy + a_{22}y^2,$$

la cual se cumple dado que A es semidefinida positiva. Para asegurar que esta ecuación cuadrática se cumple, descomponiéndola de la forma

$$a_{11} \left(x + \frac{a_{12}}{a_{11}} y \right)^2 + \left(a_{22} - \frac{a_{12}^2}{a_{11}} \right) y^2,$$

dado que $a_{11} \geq 0$ por ser A semidefinida positiva, basta que $a_{11}a_{22} - a_{12}^2$ sea positivo; es decir, se ha de cumplir que $|a_{12}| \leq \sqrt{a_{11}a_{22}}$.

La implicación de (1.8) se deduce de (1.6). ■

Si el algoritmo de Cholesky de la tabla 1.7 se aplica a una matriz semidefinida positiva, encontrándose a lo largo del proceso que un a_{kk} es cero, del teorema anterior se deduce que $a_{jk} = 0$, $j = k, \dots, n$, por lo que no habría nada más que hacer en la columna k . Un algoritmo parecido al de la tabla 1.7 que tuviese en cuenta esta eventualidad, se podría reescribir de forma inmediata de la forma que describe la tabla 1.9. En la práctica, sin embargo, los errores de redondeo imposibilitan la obtención de ceros exactos por lo que se recurre a incorporar la pivotación para evitar problemas.

Tabla 1.9

Variante del algoritmo de Cholesky de la tabla 1.7 para matrices $A^{n \times n}$ simétricas semidefinidas positivas. Sin pivotación

```

for  $i = 1$  to  $n$ 
  if  $a(i, i) > 0$ 
     $g(i, i) \leftarrow \sqrt{a(i, i) - \sum_{k=1}^{i-1} g^2(k, i)}$ 
    for  $j = i + 1$  to  $n$ 
       $g(i, j) \leftarrow \left( a(i, j) - \sum_{k=1}^{i-1} g(k, i)g(k, j) \right) / g(i, i)$ 
    end
  end
end

```

1.5.3.1 Pivotación

Si se desea llevar a cabo pivotaciones en una matriz simétrica y mantener la simetría, es necesario que esas pivotaciones se hagan simétricamente: si se intercambian dos filas, también hay que intercambiar las correspondientes columnas. Hay que recordar que una transformación de la matriz A de la forma $A \leftarrow PAP^T$ se denomina *permutación simétrica*.

Si en una etapa k del proceso que conduce a la factorización de Cholesky se determina el elemento de mayor valor de la diagonal principal, $\max_{k \leq j \leq n} a_{jj}$, y se intercambia con el a_{kk} , si el resultante $a_{kk} = 0$, el resto de la matriz a factorizar sería nula y no sería necesario realizar más operaciones. En la tabla 1.10 se describe el algoritmo de Cholesky para matrices semidefinidas positivas con pivotación.

1.5.4 Matrices simétricas indefinidas

Recordemos que una matriz A se dice *indefinida* si para algún vector $\mathbf{x} \neq \mathbf{0}$ la forma cuadrática $\mathbf{x}^T A \mathbf{x}$ es positiva y para otros negativa. Aunque una matriz simétrica indefinida puede factorizarse de la forma LDL^T , los elementos de L y D pueden tomar valores arbitrarios. En efecto,

Tabla 1.10

Algoritmo para la factorización $G^T G$ de Cholesky de una matriz $A^{n \times n}$ simétrica semidefinida positiva con pivotación

```

for  $i = 1$  to  $n$ 
  Determinar índice  $p \in \{i, i+1, n\}$  tal que  $|a(p, p)| = \max_{i \leq j \leq n} \{|a(j, j)|\}$ 
  if  $a(p, p) > 0$ 
    Intercambiar filas/columnas  $p$  y  $i$ .
     $g(i, i) \leftarrow \sqrt{a(i, i) - \sum_{k=1}^{i-1} g^2(k, i)}$ 
    for  $j = i+1$  to  $n$ 
       $g(i, j) \leftarrow \left( a(i, j) - \sum_{k=1}^{i-1} g(k, i)g(k, j) \right) / g(i, i)$ 
    end
  end
end

```

supóngase la matriz

$$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 0 \end{bmatrix}$$

y su factorización LDL^T (de acuerdo con el algoritmo de la tabla 1.6 de la página 41):

$$\begin{bmatrix} 1 & 0 \\ 1/\varepsilon & 1 \end{bmatrix} \begin{bmatrix} \varepsilon & 0 \\ 0 & -1/\varepsilon \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1/\varepsilon & 1 \end{bmatrix}^T,$$

donde $1 \gg \varepsilon > 0$. Dependiendo de la precisión de la máquina en la que se implemente este método, el resultado puede llegar a ser cualquier cosa.

Para evitar estos problemas se puede recurrir, como venimos haciendo, a algún tipo de pivotación. Ahora bien, las pivotaciones normales destruirían la simetría de la matriz y, en consecuencia, la velocidad $O(n^3/6)$ obtenible con ella. Un tipo de pivotación que podría utilizarse en este caso, como hemos indicado anteriormente, sería la simétrica; es decir, la definida por una matriz P tal que $A' \leftarrow PAP^T$ siga siendo simétrica. Esta forma de proceder, sin embargo, tampoco garantiza una estabilidad numérica completa en el cálculo de LDL^T . Por ejemplo, si los valores de ε_1 y ε_2 son muy pequeños, cualquiera que sea P , la matriz

$$\tilde{A} = P \begin{bmatrix} \varepsilon_1 & 1 \\ 1 & \varepsilon_2 \end{bmatrix} P^T,$$

seguirá teniendo en la diagonal principal elementos muy pequeños. Como los elementos pivote siempre se escogen de esa diagonal principal, sus valores serán muy pequeños si se comparan con aquellos que no están en la diagonal principal que se han de hacer cero. La factorización LDL^T con pivotaciones simétricas tampoco es pues del todo deseable desde el punto de vista numérico.

La idea de los dos métodos que estudiaremos a continuación consiste en tener también en cuenta los elementos de la matriz a factorizar que no están en la diagonal principal y a la vez conservar la simetría, no penalizando así la velocidad obtenible $O(n^3/6)$.

Los dos métodos calculan una factorización

$$PAP^T = LTL^T, \quad (1.9)$$

donde L , de coeficientes l_{ij} , es una matriz triangular inferior con $l_{ii} = 1$, P representa una permutación tal que $|l_{ij}| \leq 1$ y T es una matriz tridiagonal de la forma

$$T = \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \ddots & & \mathbf{0} \\ & \ddots & \ddots & \ddots & \\ & \mathbf{0} & \ddots & \ddots & \beta_{n-1} \\ & & & \beta_{n-1} & \alpha_n \end{bmatrix}.$$

Mediante una factorización como esta, la resolución del sistema $A\mathbf{x} = \mathbf{b}$ constaría de las siguientes etapas:

1. $L\mathbf{z} = P\mathbf{b}$;
2. $T\mathbf{w} = \mathbf{z}$;
3. $L^T\mathbf{y} = \mathbf{w}$ y
4. $\mathbf{x} = P^T\mathbf{y}$.

Para resolver $T\mathbf{w} = \mathbf{z}$ se utiliza la eliminación de Gauss en su variante para matrices tridiagonales, proceso que requiere n operaciones de multiplicación/división y suma/resta.

1.5.4.1 El método de Parlett y Reid

Este método —Parlett y Reid [1970]— se basa en la utilización de transformaciones de Gauss. Para analizar su mecánica, apliquémoslo a una matriz $A^{5 \times 5}$, suponiendo que estamos en la etapa $k = 2$.

Al comienzo de esta etapa, la matriz A tiene la forma

$$A^{(1)} = M_1 P_1 A P_1^T M_1^T = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & 0 & 0 \\ \beta_1 & \alpha_2 & v_3 & v_4 & v_5 \\ 0 & v_3 & \times & \times & \times \\ 0 & v_4 & \times & \times & \times \\ 0 & v_5 & \times & \times & \times \end{bmatrix},$$

donde P representa una permutación tal que los módulos de los elementos de la transformación o eliminación de Gauss M_1 están acotados superiormente por la unidad. En esta etapa $k = 2$ se busca el elemento del vector $[v_3, v_4, v_5]^T$ de mayor valor absoluto y se determina una permutación, que representaremos por $\tilde{P}_2$, tal que

$$\tilde{P}_2 \begin{bmatrix} v_3 \\ v_4 \\ v_5 \end{bmatrix} = \begin{bmatrix} \tilde{v}_3 \\ \tilde{v}_4 \\ \tilde{v}_5 \end{bmatrix}, \quad \text{donde } |\tilde{v}_3| = \max\{|v_3|, |v_4|, |v_5|\}.$$

Si $\tilde{v}_3$ es cero, se hace $M_2 = P_2 = I$ y se pasa a la etapa $k = 3$. Si no, se hace $P_2 = \text{diag}(I_2, \tilde{P}_2)$, es decir una matriz diagonal en dos bloques (el primero I_2 y el segundo $\tilde{P}_2$), y $M_2 = I_5 - \alpha_2 \mathbf{e}_3^T$, donde,

$$\alpha_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \tilde{v}_4/\tilde{v}_3 \\ \tilde{v}_5/\tilde{v}_3 \end{bmatrix}.$$

El resultado de esta etapa $k = 2$ será una matriz $A^{(2)}$ de la forma

$$A^{(2)} = M_2 P_2 A^{(1)} P_2^T M_2^T = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & 0 & 0 \\ \beta_1 & \alpha_2 & \tilde{v}_3 & 0 & 0 \\ 0 & \tilde{v}_3 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \\ 0 & 0 & \times & \times & \times \end{bmatrix}.$$

Este proceso se completa en $n-2$ etapas al final de las cuales se obtiene la matriz tridiagonal que se deseaba:

$$T = A^{(n-2)} = (M_{n-2} P_{n-2} \cdots M_1 P_1) A (M_{n-2} P_{n-2} \cdots M_1 P_1)^T.$$

Si se hace $P = P_{n-2} \cdots P_1$ y $L = (M_{n-2} P_{n-2} \cdots M_1 P_1 P^T)^{-1}$, mediante un razonamiento similar al del apartado 1.4, se puede comprobar que

$$P A P^T = L T L^T.$$

La primera columna de L es $\mathbf{e}_1$; las restantes k ($k > 1$) las forman los multiplicadores de M_{k-1} .

Ejemplo 1.2 Aplicar el método de Parlett y Reid a

$$A = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 2 & 2 & 2 \\ 2 & 2 & 3 & 3 \\ 3 & 2 & 3 & 4 \end{bmatrix}.$$

En la primera etapa se tiene que:

$$\begin{aligned} P_1 &= [\mathbf{e}_1, \mathbf{e}_4, \mathbf{e}_3, \mathbf{e}_2] \\ M_1 &= I_4 - \begin{bmatrix} 0 \\ 0 \\ 2/3 \\ 1/3 \end{bmatrix} [0, 1, 0, 0] \\ A^{(1)} &= M_1 P_1 A P_1^T M_1^T = \begin{bmatrix} 0 & 3 & 0 & 0 \\ 3 & 4 & 1/3 & 2/3 \\ 0 & 1/3 & 7/9 & 5/9 \\ 0 & 2/3 & 5/9 & 10/9 \end{bmatrix}. \end{aligned}$$

En la segunda:

$$P_2 = [\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_4, \mathbf{e}_3]$$

$$M_2 = I_4 - \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1/2 \end{bmatrix} [0, 0, 1, 0]$$

$$A^{(2)} = M_2 P_2 A^{(1)} P_2^T M_2^T = \begin{bmatrix} 0 & 3 & 0 & 0 \\ 3 & 4 & 2/3 & 0 \\ 0 & 2/3 & 10/9 & 0 \\ 0 & 0 & 0 & 1/2 \end{bmatrix}.$$

En resumen, $PAP^T = LTL^T$, donde:

$$P = P_2 P_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$L = (M_2 P_2 M_1 P_1 P^T)^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1/3 & 1 & 0 \\ 0 & 2/3 & 1/2 & 1 \end{bmatrix} \quad \text{y}$$

$$T = \begin{bmatrix} 0 & 3 & 0 & 0 \\ 3 & 4 & 2/3 & 0 \\ 0 & 2/3 & 10/9 & 0 \\ 0 & 0 & 0 & 1/2 \end{bmatrix}.$$

•

Para implementar de forma eficaz este método en ordenador hay que tener cuidado al calcular

$$A^{(k)} = M_k \left(P_k A^{(k-1)} P_k^T \right) M_k^T. \quad (1.10)$$

Para apreciar las operaciones que implica esta fórmula, supongamos que $B = B^T$ es una matriz de orden $n - k$ y que se desea obtener

$$B_+ = \left(I - \mathbf{w} \mathbf{e}_1^T \right) B \left(I - \mathbf{w} \mathbf{e}_1^T \right)^T$$

(operación clave en (1.10)), donde $\mathbf{w} \in \Re^{n-k}$ y $\mathbf{e}_1$ es la primera columna de I_{n-k} . Si se hace

$$\mathbf{u} = B \mathbf{e}_1 - \frac{b_{11}}{2} \mathbf{w},$$

la matriz simétrica B_+ es igual a $B - \mathbf{w} \mathbf{u}^T - \mathbf{u} \mathbf{w}^T$, la cual puede obtenerse realizando $(n - k)^2$ operaciones. Si esto se repite variando k de 1 a $n - 2$, como es el caso del método de Parlett y Reid, el número total de operaciones que requiere el método es $\boxed{O(n^3/3)}$ multiplicaciones/divisiones y sumas/restas: dos veces más que las deseadas en principio.

1.5.4.2 El método de Aasen

Este método —Aasen [1971]— calcula una factorización

$$PAP^T = LTL^T$$

igual que el método de Parlett y Reid, pero mediante un proceso que requiere $\boxed{O(n^3/6)}$ multiplicaciones/divisiones y sumas/restas. Para estudiarlo, partamos del de Parlett y Reid y reconsideremos el cálculo de las transformaciones de Gauss $M_1, \dots, M_{n-2}$. Ignoremos de momento la pivotación.

Supongamos que estamos en la etapa j y que ya se han calculado unas transformaciones $M_1, \dots, M_{j-1}$ tales que

$$(M_{j-1} \cdots M_1)A(M_{j-1} \cdots M_1)^T = \begin{bmatrix} T_{11} & \begin{bmatrix} 0 \\ \mathbf{v}^T \end{bmatrix} \\ 0 & \begin{bmatrix} \mathbf{v} & T_{22} \end{bmatrix} \end{bmatrix} \begin{matrix} j-1 \\ 1 \\ n-j \end{matrix},$$

donde

$$T_{11} = \begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \ddots & & \mathbf{0} \\ & \ddots & \ddots & \ddots & \\ & \mathbf{0} & \ddots & \ddots & \beta_{j-1} \\ & & & \beta_{j-1} & \alpha_j \end{bmatrix}$$

y que conocemos todos los elementos de T_{11} excepto α_j . El objetivo de la etapa j del método de Aasen es el cálculo de M_j , α_j y β_j (estos dos parámetros forman la columna j -ésima de T).

Obsérvese que

$$M_1^{-1} \cdots M_{j-1}^{-1} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{matrix} j \\ n-j \end{matrix}$$

es una matriz triangular inferior unitaria, cuyos coeficientes designaremos mediante l_{ij} , y cuya primera columna es $\mathbf{e}_1$ pues

$$M_i = I - \underbrace{[0, \dots, 0, \times, \dots, \times]}_{i+1}^T \mathbf{e}_{i+1}^T.$$

Como

$$A = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} H_{11} & H_{12} \\ 0 & \mathbf{v} & H_{22} \end{bmatrix}, \quad (1.11)$$

donde

$$\begin{bmatrix} T_{11} & \begin{bmatrix} 0 \\ \mathbf{v}^T \end{bmatrix} \\ 0 & \begin{bmatrix} \mathbf{v} & T_{22} \end{bmatrix} \end{bmatrix} \begin{bmatrix} L_{11}^T & L_{21}^T \\ 0 & I \end{bmatrix} = \begin{bmatrix} H_{11} & H_{12} \\ 0 & \mathbf{v} & H_{22} \end{bmatrix} \begin{matrix} j \\ n-j \end{matrix}, \quad (1.12)$$

se tiene que

$$\mathbf{v} = \begin{bmatrix} v_{j+1} \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} a_{j+1j} \\ \vdots \\ a_{nj} \end{bmatrix} - L_{21}H_{11}\mathbf{e}_j.$$

En consecuencia, una vez que se conoce el vector

$$H_{11}\mathbf{e}_j = \begin{bmatrix} h_1 \\ \vdots \\ h_j \end{bmatrix},$$

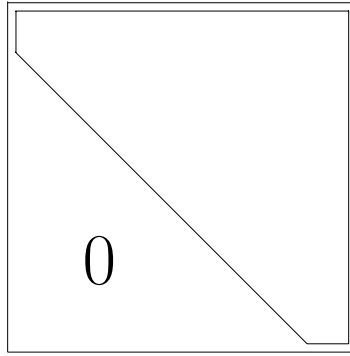
se pueden calcular

$$v_i = a_{ij} - \sum_{k=1}^j l_{ik}h_k, \quad i = j+1, \dots, n \quad (1.13)$$

y, a continuación,

$$M_j = I - \frac{1}{v_{j+1}}[0 \dots, 0, v_{j+2} \dots, v_n]^T \mathbf{e}_{j+1}^T.$$

La idea clave del método de Aasen estriba en darse cuenta que la matriz $H_{11} = T_{11}L_{11}^T$ de la expresión (1.12) es una matriz superior de Hessemberg; es decir, tiene la forma que se ilustra a continuación.



De acuerdo con esto, desarrollando ese producto matricial, se tiene que:

$$h_1 = \beta_1 l_{j2}; \quad (1.14)$$

$$h_i = \beta_{i-1} l_{ji-1} + \alpha_i l_{ji} + \beta_i l_{ji+1}, \quad i = 2, \dots, j-1 \text{ y} \quad (1.15)$$

$$h_j = \beta_{j-1} l_{jj-1} + \alpha_j.$$

El problema con la última de estas fórmulas es que α_j es desconocida. Para paliarlo se usa la siguiente ecuación, deducible fácilmente de (1.11):

$$h_j = a_{jj} - \sum_{i=2}^{j-1} l_{ji} h_i. \quad (1.16)$$

Esta fórmula junto con (1.14), (1.15) y (1.13) sirve para calcular la transformación M_j .

Para finalizar la etapa j se hace

$$\beta_j = v_{j+1} \quad \text{y} \quad \alpha_j = \begin{cases} a_{11} & \text{si } j = 1 \\ h_j - \beta_{j-1} l_{jj-1} & \text{si } j > 1. \end{cases}$$

El algoritmo que se describe en la tabla 1.11 implementa el método de Aasen sin pivotación. La matriz T de la factorización $LT L^T$ que se obtiene queda almacenada en $\alpha_1, \dots, \alpha_n$ y $\beta_1, \dots, \beta_{n-1}$.

El método de Aasen, como ya indicábamos, requiere $\boxed{O(n^3/6)}$ multiplicaciones/divisiones y sumas/restas.

Tabla 1.11

Algoritmo de Aasen sin pivotación para la factorización LTL^T de una matriz $A^{n \times n}$ simétrica indefinida

```

for  $j = 1$  to  $n$ 
  if  $j = 1$ 
     $h(1) = a(1, 1)$ 
  else if  $j = 2$ 
     $h(1) = \beta(1); h(2) = a(2, 2)$ 
  else
     $\ell(0) = 0; \ell(1) = 0; \ell(2:j-1) = l(j, 2:j-1); \ell(j) = 1$ 
     $h(j) = a(j, j)$ 
    for  $k = 1$  to  $j-1$ 
       $h(k) = \beta(k-1)\ell(k-1) + \alpha(k)\ell(k) + \beta(k)\ell(k+1)$ 
       $h(j) = h(j) - \ell(k)h(k)$ 
    end
  end
  if  $j = 1$  or  $j = 2$ 
     $\alpha(j) = h(j)$ 
  else
     $\alpha(j) = h(j) - \beta(j-1)l(j, j-1)$ 
  end
  if  $j \leq n-1$ 
     $v(j+1:n) = a(j+1:n, j) - l(j+1:n, 1:j)h(1:j)$ 
     $\beta(j) = v(j+1)$ 
  end
  if  $j \leq n-2$ 
     $l(j+2:n, j+1) = v(j+2:n)/v(j+1)$ 
  end
end

```

Pivotación

Los vectores columna de la matriz L son múltiplos de los vectores $\mathbf{v}$. Si alguno de esos múltiplos es grande (v_{j+1} es pequeño), puede haber problemas de estabilidad numérica. Para soslayarlos, se permuta el mayor componente v_i , $i = j+1, \dots, n$, con el v_{j+1} . Esta permutación, por supuesto, habrá de hacerse también en la parte de la matriz A que todavía no se haya tratado.

El algoritmo completo de Aasen con pivotación es el que describe la tabla 1.12. Con este algoritmo se obtiene una matriz de permutación P —vector $\text{PIV}(\cdot)$ —, una matriz triangular inferior L , tal que $|l_{ij}| \leq 1$, y una matriz tridiagonal T , definida por α_i , $i = 1, \dots, n$ y β_j , $j = 1, \dots, n-1$, tales que

$$PAP^T = LTL^T.$$

Sólo se calculan los elementos l_{ij} , $i = 2, \dots, n$, $j = 2, \dots, n$. La matriz $P = P_1 \cdots P_{n-2}$, donde P_j es la identidad con las filas $\text{PIV}(j)$ y $j+1$ intercambiadas.

Tabla 1.12

Algoritmo de Aasen con pivotación para la factorización $LT L^T$ de una matriz $A^{n \times n}$ simétrica indefinida

```
for  $j = 1$  to  $n$ 
  if  $j = 1$ 
     $h(1) = a(1, 1)$ 
  else if  $j = 2$ 
     $h(1) = \beta(1)$ ;  $h(2) = a(2, 2)$ 
  else
     $\ell(0) = 0$ ;  $\ell(1) = 0$ ;  $\ell(2:j-1) = l(j, 2:j-1)$ ;  $\ell(j) = 1$ 
     $h(j) = a(j, j)$ 
    for  $k = 1$  to  $j-1$ 
       $h(k) = \beta(k-1)\ell(k-1) + \alpha(k)\ell(k) + \beta(k)\ell(k+1)$ 
       $h(j) = h(j) - \ell(k)h(k)$ 
    end
  end
  if  $j = 1$  or  $j = 2$ 
     $\alpha(j) = h(j)$ 
  else
     $\alpha(j) = h(j) - \beta(j-1)l(j, j-1)$ 
  end
  if  $j \leq n-1$ 
     $v(j+1:n) = A(j+1:n, j) - l(j+1:n, 1:j)h(1:j)$ 
    Determinar  $q$  tal que  $|v(q)| = \|v(j+1:n)\|_\infty$ .
     $piv(j) = q$ ;  $v(j+1) \leftrightarrow v(q)$ ;  $l(j+1, 2:j) \leftrightarrow l(q, 2:j)$ 
     $a(j+1, j+1:n) \leftrightarrow a(q, j+1:n)$ 
     $a(j+1:n, j+1) \leftrightarrow a(j+1:n, q)$ 
     $\beta(j) = v(j+1)$ 
  end
  if  $j \leq n-2$ 
     $l(j+2:n, j+1) = v(j+2:n)$ 
    if  $v(j+1) \neq 0$ 
       $l(j+2:n, j+1) = l(j+2:n, j+1)/v(j+1)$ 
    end
  end
end
end
```

La codificación completa de este algoritmo en FORTRAN 77 para factorizar la matriz

$$A = \begin{bmatrix} 1 & 10 & 20 \\ 10 & 1 & 30 \\ 20 & 30 & 1 \end{bmatrix}$$

es la que sigue.

```

      PROGRAM Aasen
C
      parameter (n = 3)
      real a(n,n),alfa(n),beta(n),l(n,n),h(n),v(n),lo(0:n)
      integer ipvt(n)
C
      data a/1.,10.,20.,10.,1.,30.,20.,30.,1./
C
      do i = 1,n
         ipvt(i) = i
      end do
C
C                                     T
C *** FACTORIZACION LTL ***
C
      do j = 1,n
         if (j.eq.1) then
            h(j) = a(1,1)
         else if (j.eq.2) then
            h(1) = beta(1)
            h(2) = a(2,2)
         else
            lo(0) = 0.
            lo(1) = 0.
            do k = 2,j-1
               lo(k) = l(j,k)
            end do
            lo(j) = 1
            h(j) = a(j,j)
            do k = 1,j-1
               h(k) = beta(k-1)*lo(k-1)+alfa(k)*lo(k)+beta(k)*lo(k+1)
               h(j) = h(j)-lo(k)*h(k)
            end do
         endif
C
         if (j.eq.1.or.j.eq.2) then
            alfa(j) = h(j)
         else
            alfa(j) = h(j)-beta(j-1)*l(j,j-1)
         endif
C
         if (j.le.n-1) then
            smax = 0.
            iq = j
            do k = j+1,n
               suma = 0.
               do k1 = 1,j
                  suma = suma-l(k,k1)*h(k1)
               end do
               v(k) = a(k,j)+suma
               if (abs(v(k)).gt.smax) then
                  smax = abs(v(k))
               end if
            end do
         end if
      end do

```



```
        iq    = k
      endif
    end do
    aux      = v(j+1)
    v(j+1) = v(iq)
    v(iq)   = aux
    do k = 2,j
      aux      = l(j+1,k)
      l(j+1,k) = l(iq,k)
      l(iq,k)  = aux
    end do
   iaux      = ipvt(j+1)
    ipvt(j+1) = ipvt(iq)
    ipvt(iq)  = iaux
    do k = j+1,n
      aux      = a(j+1,k)
      a(j+1,k) = a(iq,k)
      a(iq,k)  = aux
    end do
    do k = j+1,n
      aux      = a(k,j+1)
      a(k,j+1) = a(k,iq)
      a(k,iq)  = aux
    end do
    beta(j) = v(j+1)
  endif
if (j.le.n-2) then
  do k = j+2,n
    l(k,j+1) = v(k)
  end do
  if (v(j+1).ne.0.) then
    do k = j+2,n
      l(k,j+1) = l(k,j+1)/v(j+1)
    end do
  endif
endif
end do
C
print *,alfa
print *,(beta(i),i=1,n-1)
print *,((l(j,i),j=i+1,n),i=1,n-1)
print *,ipvt
C
end
```

La permutación resultante es $P = [e_1, e_3, e_2]$, por lo que

$$PAP^T = \begin{bmatrix} 1 & 20 & 10 \\ 20 & 1 & 30 \\ 10 & 30 & 1 \end{bmatrix}.$$

La factorización final es

$$PAP^T = LTL^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0,5 & 1 \end{bmatrix} \begin{bmatrix} 1 & 20 & 0 \\ 20 & 1 & 29,5 \\ 0 & 29,5 & -28,75 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0,5 & 1 \end{bmatrix}^T.$$

1.5.4.3 Factorización de pivotación diagonal

La idea que preside estos métodos consiste en descomponer la matriz simétrica A de la forma

$$PAP^T = LBL^T,$$

donde, como siempre, P es una matriz de permutación y B una matriz diagonal en bloques, el tamaño de los cuales no es superior a 2×2 , y hacerlo de manera que se realicen el menor número de operaciones y comparaciones posible. La idea de utilizar pivotes 2×2 es tratar de evitar las dificultades que se pueden presentar al factorizar una matriz simétrica indefinida con un proceso que sólo tenga en cuenta pivotes individuales y encontrarse que estos son cero o muy pequeños.

Para exponer las características de estos métodos y su mecánica, supongamos que

$$P_1AP_1^T = \begin{bmatrix} B & C^T \\ C & D \end{bmatrix} \begin{matrix} s \\ n-s \end{matrix}$$

donde P_1 es una matriz de permutación y $s = 1$ ó 2 . Si A no tiene todos sus elementos nulos siempre es posible escoger los parámetros s y P_1 de tal forma que B sea no singular, pudiéndose entonces escribir

$$P_1AP_1^T = \begin{bmatrix} I_s & 0 \\ CB^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} B & 0 \\ 0 & D - CB^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & B^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}.$$

A los efectos de conseguir una buena estabilidad numérica en el proceso de factorización, el *pivote* $s \times s$, B , se escoge de tal manera que los elementos de la matriz

$$D - CB^{-1}C^T,$$

que denominaremos $\tilde{A}$, estén acotados convenientemente. En este sentido, si se tiene un $\alpha \in (0, 1)$ y se definen

$$\mu_0 = \max_{i,j} |a_{ij}| \quad \text{y} \quad \mu_1 = \max_i |a_{ii}|,$$

la estrategia de pivotación que usa uno de los métodos que realizan pivotación diagonal, concretamente el de Bunch y Parlett [1971], es la siguiente (en una hipotética etapa k del proceso):

```

if  $\mu_1 \geq \alpha\mu_0$  then
   $s = 1$ 
  Determinar una permutación  $P$  de tal manera que  $|b_{11}| = \mu_1$ 
else
   $s = 2$ 
  Determinar una permutación  $P$  de tal manera que  $|b_{21}| = \mu_0$ 
end

```

Se puede comprobar fácilmente de la definición de $\tilde{A}$, que si $s = 1$,

$$|\tilde{a}_{ij}| \leq (1 + \alpha^{-1}) \mu_0,$$

mientras que si $s = 2$,

$$|\tilde{a}_{ij}| \leq \frac{3 - \alpha}{1 - \alpha} \mu_0.$$

Igualando $(1 + \alpha^{-1})^2$, factor de crecimiento asociado a dos pivotaciones sucesivas $s = 1$, a $(3 - \alpha)/(1 - \alpha)$, asociado a una pivotación $s = 2$, se obtendría el valor óptimo de α que minimiza el límite de crecimiento de los elementos que se obtienen en la factorización. Ese óptimo se puede demostrar que es

$$\alpha = \frac{1 + \sqrt{17}}{8}.$$

Efectuadas las permutaciones convenientes de cada etapa mediante un proceso de pivotación diagonal, estos procedimientos a continuación hacen cero las submatrices C y C^T de

$$\begin{bmatrix} B & C^T \\ C & D \end{bmatrix}$$

mediante transformaciones caracterizadas por matrices del tipo

$$\begin{bmatrix} I_s & 0 \\ CB^{-1} & I_{n-s} \end{bmatrix}.$$

El proceso comenzaría otra vez en una etapa ulterior permutando convenientemente la matriz A_{n-s} , y así sucesivamente.

El método de Bunch y Parlett requiere $O(n^3/6)$ multiplicaciones/divisiones y sumas/restas y entre $O(n^3/12)$ y $O(n^3/6)$ comparaciones. Su estabilidad numérica es comparable a la eliminación de Gauss con pivotación total.

1.5.4.3.1 El método de Bunch y Kaufman

Este método de pivotación diagonal —Bunch y Kaufman [1977]— es el más eficaz de los conocidos hasta la fecha para factorizar matrices simétricas indefinidas. Está implementado por casi todas las librerías de rutinas comercialmente disponibles, tanto para tratar matrices dispersas como densas. Es una mejora del de Bunch y Parlett que reduce mucho las comparaciones necesarias inherentes a las pivotaciones; las multiplicaciones/divisiones y sumas/restas son las mismas: $O(n^3/6)$.

En cada etapa k de este método (se procede desde la última columna a la primera con objeto de conservar la orientación de las columnas), sólo se analizan dos columnas de la submatriz A_k que queda por factorizar de

$$\begin{bmatrix} A_k & 0 \\ 0 & B_k \end{bmatrix},$$

donde A_k es una matriz $k \times k$ y B_k una diagonal en bloques $(n - k) \times (n - k)$ con bloques 1×1 ó 2×2 .

En esa etapa k , analizando los elementos de la última columna, $\mathbf{a}_k$, de A_k no en la diagonal principal con respecto al de la diagonal principal, y los de aquella fila de A_k donde se registra

en $\mathbf{a}_k$ el máximo elemento, se determinan una matriz P_k y otra B'_k (1×1 ó 2×2), tales que

$$\tilde{A}_k = P_k A_k P_k = \left[\begin{array}{cc|c} D_k & C_k & 0 \\ C_k^T & B'_k & 0 \\ \hline 0 & & B_k \end{array} \right].$$

Posteriormente se eliminan los C_k y C_k^T mediante unas transformaciones U_k de tal manera que

$$\begin{aligned} U_k \tilde{A}_k U_k^T &= \left[\begin{array}{cc|c} I_{k-s} & M_k & 0 \\ 0 & I_s & 0 \\ \hline 0 & & I_{n-k} \end{array} \right] \left[\begin{array}{cc|c} D_k & C_k & 0 \\ C_k^T & B'_k & 0 \\ \hline 0 & & B_k \end{array} \right] \left[\begin{array}{cc|c} I_{k-s} & 0 & 0 \\ M_k^T & I_s & 0 \\ \hline 0 & & I_{n-k} \end{array} \right] \\ &= \left[\begin{array}{cc|c} D_k - M_k B'_k M_k^T & 0 & 0 \\ 0 & B'_k & 0 \\ \hline 0 & & B_k \end{array} \right]. \end{aligned}$$

Para obtenerlas se resuelve $M_k B'_k = -C_k$. Si B'_k es 2×2 , se hace $P_{k-1} = I$ y $U_{k-1} = I$. Si B'_k resulta ser un bloque 2×2 , resolver $M_k B'_k = -C_k$ representa resolver $k-2$ sistemas de ecuaciones lineales 2×2 donde cada uno de los pares de incógnitas de cada sistema son los dos elementos de cada fila de la matriz M_k . Estas incógnitas se obtienen fácilmente teniendo en cuenta que

$$M_k = -C_k B_k'^{-1},$$

donde

$$B_k'^{-1} = \frac{1}{b'_{k_{11}} b'_{k_{22}} - b'^2_{k_{21}}} \begin{bmatrix} b'_{k_{22}} & -b'_{k_{21}} \\ -b'_{k_{21}} & b'_{k_{11}} \end{bmatrix}.$$

Del proceso de factorización en su totalidad se llega a que

$$U_1 P_1 \cdots U_n P_n A P_n U_n^T \cdots P_1 U_1^T = B$$

o que

$$A = U B U^T,$$

donde

$$U = P_n U_n^{-1} \cdots P_1 U_1^{-1}$$

y

$$U_k^{-1} = \left[\begin{array}{cc|c} I_{k-s} & -M_k & 0 \\ 0 & I_s & 0 \\ \hline 0 & 0 & I_{n-k} \end{array} \right],$$

con s , como siempre, 1 ó 2.

Pivotación

Aun cuando el criterio que se sigue para llevar a cabo las pivotaciones en el método de Bunch y Kaufman es parecido al de Bunch y Parlett, ambos métodos difieren en cuanto al número de comparaciones necesarias: el de Bunch y Kaufman es sensiblemente inferior. Si A expresa, para simplificar la notación, la submatriz A_k que resta por factorizar en la etapa k , el proceso de

Tabla 1.13

Operaciones de la pivotación en el método de Bunch y Kaufman

```

Determinar  $\lambda = |a_{rk}| = \max\{|a_{1k}|, \dots, |a_{k-1k}|\}$ 
if  $\lambda > 0$ 
  if  $|a_{kk}| \geq \alpha\lambda$  then
     $P = I$ ;  $s = 1$ 
  else
    Determinar  $\sigma = \max\{|a_{1k}|, \dots, |a_{r-1r}|, |a_{r+1r}|, \dots, |a_{kr}|\}$ 
    if  $\sigma|a_{kk}| \geq \alpha\lambda^2$  then
       $P = I$ ;  $s = 1$ 
    else if  $|a_{rr}| > \alpha\sigma$  then
       $s = 1$ ; determinar  $P$  tal que  $(P^T AP)_{kk} = a_{rr}$ 
    else
       $s = 2$ ; determinar  $P$  tal que  $(P^T AP)_{k-1k} = a_{rk}$ 
    end
  end
end

```

pivotación parcial de este método, en el que sólo se examinan los elementos de dos columnas de A , es el que se describe en la tabla 1.13. El número de comparaciones que se efectúan de esta manera es $n^2 - 1$.

El algoritmo completo de Bunch y Kaufman se representa en la tabla 1.14. Requiere de $O(n^3/6)$ multiplicaciones/divisiones y sumas/restas y $n^2 - 1$ comparaciones. Como en el algoritmo de Bunch y Parlett,

$$\alpha = \frac{1 + \sqrt{17}}{8}.$$

La codificación de este algoritmo en forma de rutina en FORTRAN 77 es la que sigue a continuación. Al final del procedimiento numérico, la parte triangular superior de la matriz A recoge los multiplicadores o matrices elementales U_k .

```

subroutine bunchkauf(a,n,ipvt)
C
  integer ipvt(n)
  real a(n,n),mulk,mulkm1
  logical swap
C
  a(-,-) Al final de la factorización contiene la matriz
C          diagonal en bloques y los multiplicadores necesarios
C          para obtenerla. T
C          Se puede escribir de la forma a=u*b*u , donde u es el
C          producto de matrices de permutación y matrices trian-
C          gular superior.
C
  ipvt() Indicador de la pivotaciones realizadas.
C
  alpha = (1.0e0+sqrt(17.0e0))/8.0e0
  k = n
  do while (k.gt.1)

```

Tabla 1.14

Algoritmo para la factorización UBU^T de una matriz $A^{n \times n}$ simétrica indefinida por el método de Bunch y Kaufman con pivotación

```

while  $k > 1$  do
  Determinar  $\lambda_k = |a_{rk}^{(k)}| = \max \left\{ |a_{1k}^{(k)}|, \dots, |a_{k-1,k}^{(k)}| \right\}$ 
  if  $|a_{kk}^{(k)}| \geq \alpha \cdot \lambda_k$  then
     $P_k = I$ ;  $s = 1$ 
  else
    Determinar  $\sigma = \max \left\{ \max_{1 \leq j \leq r-1} |a_{jr}^{(k)}|, \max_{r+1 \leq j \leq k} |a_{rj}^{(k)}| \right\}$ 
    if  $\sigma |a_{kk}^{(k)}| \geq \alpha \lambda_k^2$  then
       $P_k = I$ ;  $s = 1$ 
    else if  $|a_{rr}^{(k)}| \geq \alpha \sigma$  then
       $s = 1$ ; determinar  $P_k$  tal que  $(P_k A_k P_k)_{kk} = a_{rr}^{(k)}$ 
      for  $j = k - 1$  to 1
         $\eta = -a(j, k) / a(k, k)$ 
         $a(j, k) = \eta$ 
        for  $i = 1$  to  $j$ 
           $a(i, j) \leftarrow a(i, j) - \eta \cdot a(i, k)$ 
        end
      end
    else
       $s = 2$ ; determinar  $P_k$  tal que  $(P_k A_k P_k)_{k-1,k} = a_{rk}^{(k)}$ 
      for  $j = k - 2$  to 1
         $\eta1 = \frac{a(k-1, k-1) \cdot a(j, k) - a(k-1, k) \cdot a(j, k-1)}{a(k-1, k) \cdot a(k-1, k) - a(k, k) \cdot a(k-1, k-1)}$ 
         $\eta2 = \frac{a(k, k) \cdot a(j, k-1) - a(k-1, k) \cdot a(j, k)}{a(k-1, k) \cdot a(k-1, k) - a(k, k) \cdot a(k-1, k-1)}$ 
         $a(j, k) = \eta1$ 
         $a(j, k-1) = \eta2$ 
        for  $i = 1$  to  $j$ 
           $a(i, j) \leftarrow a(i, j) - \eta1 \cdot a(i, k)$ 
           $a(i, j) \leftarrow a(i, j) - \eta2 \cdot a(i, k-1)$ 
        end
      end
    end
  end
end

```

```
C
C *** Determinar pivotación diagonal.
C     kstep indica el tamaño del bloque;
C     swap si se realizan intercambios de filas y columnas.
C
    km1    = k-1                                ! Se determina el
    absakk = abs(a(k,k))                        ! mayor elemento no
    imax   = isamax(k-1,a(1,k))                 ! en la diagonal de
    colmax = abs(a(imax,k))                     ! la columna k.
    if (absakk.ge.alpha*colmax) then
        kstep = 1
        swap  = .false.
    else
        ! Mayor ele-
        rowmax = 0.0e0                          ! mento no
        do j = imax+1,k                         ! en la
            rowmax = amax1(rowmax,abs(a(imax,j)))! diagonal en
        end do                                  ! fila imax.
        if (imax.ne.1) then
            jmax = isamax(imax-1,a(1,imax))
            rowmax = amax1(rowmax,abs(a(jmax,imax)))
        endif
        if (abs(a(imax,imax)).ge.alpha*rowmax) then
            kstep = 1
            swap  = .true.
        else if (absakk.ge.alpha*colmax*(colmax/rowmax)) then
            kstep = 1
            swap  = .false.
        else
            kstep = 2
            swap  = imax.ne.km1
        endif
    endif
    if (amax1(absakk,colmax).eq.0.0e0) then
        ipvt(k) = k                                ! La columna k es cero;
        cycle                                     ! seguir a otra.
    endif
    if (kstep.eq.1) then
C
C * Bloque pivote 1 x 1 *
C
        ipvt(k) = k
        if (swap) then
            call sswap (imax,a(1,imax),a(1,k))! Intercambiar filas
            do j = k,imax,-1                    ! y columnas.
                t      = a(j,k)
                a(j,k)  = a(imax,j)
                a(imax,j) = t
            end do
            ipvt(k) = imax
        endif
C
        do j = k-1,1,-1                        ! Eliminación.
            mulk = -a(j,k)/a(k,k)
            call saxpy (j,mulk,a(1,k),a(1,j))
            a(j,k) = mulk
        end do
    else
        ! KSTEP=2
C
C * Bloque pivote 2 x 2 *
```

```

C
    ipvt(k) = 1-k
    ipvt(k-1) = ipvt(k)
    if (swap) then
        call sswap (imax,a(1,imax),a(1,k-1)) ! Intercambiar
        do j = k-1,imax,-1                    ! filas y
            t = a(j,k-1)                      ! columnas.
            a(j,k-1) = a(imax,j)
            a(imax,j) = t
        end do
        t = a(k-1,k)
        a(k-1,k) = a(imax,k)
        a(imax,k) = t
        ipvt(k) = -imax
    endif
C
    if (k-2.ne.0) then                        ! Eliminación
        ak = a(k,k)/a(k-1,k)
        akm1 = a(k-1,k-1)/a(k-1,k)
        deno = 1.0e0-ak*akm1
        do j = k-2,1,-1
            bk = a(j,k)/a(k-1,k)
            bkm1 = a(j,k-1)/a(k-1,k)
            mulk = (akm1*bk-bkm1)/deno
            mulkm1 = (ak*bkm1-bk)/deno
            call saxpy (j,mulk,a(1,k),a(1,j))
            call saxpy (j,mulkm1,a(1,k-1),a(1,j))
            a(j,k) = mulk
            a(j,k-1) = mulkm1
        end do
    endif
    k = k-kstep
end do
ipvt(1) = 1
C
return
end

subroutine sswap(n,a,b)
real a(n),b(n)
do i = 1,n
    aux = a(i)
    a(i) = b(i)
    b(i) = aux
end do
return
end

integer function isamax (n,a)
real a(n)
isamax = 1
dmax = abs(a(1))
do i = 2,n
    if (abs(a(i)).gt.dmax) then
        isamax = i
        dmax = abs(a(i))
    endif
end do
end do

```



```
return
end

subroutine saxpy(j,t,a,b)
real a(1),b(1)
do i = 1,j
    b(i) = b(i)+t*a(i)
end do
return
end
```

Ejemplo 1.3 Si el algoritmo de Bunch y Kaufman se aplica a la matriz

$$A = \begin{bmatrix} 1 & 10 & 20 \\ 10 & 1 & 30 \\ 20 & 30 & 1 \end{bmatrix},$$

la matriz factorizada que se obtiene es

$$A = \begin{bmatrix} 1 & 0,6562848 & 0,3114572 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -11,79199 & 0 & 0 \\ 0 & 1 & 30 \\ 0 & 30 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0,6562848 & 1 & 0 \\ 0,3114572 & 0 & 1 \end{bmatrix}.$$

●

Si se quiere usar la rutina anterior para factorizar la matriz A de este último ejemplo, un programa que se podría utilizar el que se lista a continuación.

```
PROGRAM Bunch
C
parameter (n = 3)
real a(n,n)
integer ipvt(n)
C
data a/1.,10.,20.,10.,1.,30.,20.,30.,1./
C
call bunchkauf (a,n,ipvt)
print *,((a(i,j),j=1,n),i=1,n)
print *,ipvt
C
end
```

1.6 Condicionamiento de sistemas de ecuaciones lineales

El concepto de *condicionamiento de un problema* es algo a lo que se recurre a menudo aun cuando su sentido sea tal vez difuso o vago. En un sentido general, se dice que un problema está bien condicionado si pequeños cambios en los parámetros que lo definen producen pequeños cambios en los resultados. Para decidir si tal o cual problema está bien o mal condicionado habría que determinar su sensibilidad a cada uno de sus parámetros. Como ejemplo de condicionamiento podríamos considerar el de una carga sujeta a una superficie firme mediante un cable o una barra de hierro. Aumentando la carga en pequeñas cantidades, el cable o barra sufre unos pequeños estiramientos proporcionales a los incrementos de esa carga. Alcanzado el umbral que define la zona denominada de fluencia, incrementos muy pequeños de la carga

suponen, proporcionalmente, grandes estiramientos del cable. Antes de este umbral, el problema estiramiento/carga se puede decir que está bien condicionado; en la zona de fluencia, por el contrario, el problema está mal condicionado.

Un sistema de ecuaciones lineales, representado por $A\mathbf{x} = \mathbf{b}$, como modelo matemático de un determinado problema físico, social, mecánico, etc, también puede estar bien o mal condicionado. Su condicionamiento lo caracterizará la sensibilidad del vector solución $\mathbf{x}$ a pequeños cambios, tanto en el término de la derecha $\mathbf{b}$, como en los coeficientes que definen la matriz A .

La cuestión del condicionamiento es particularmente interesante en un sistema de ecuaciones lineales y en el proceso que conlleva su resolución pues, como hemos visto, el ordenador o máquina que lo ha de llevar a cabo, al no trabajar más que con una precisión determinada, no resolverá el sistema

$$A\mathbf{x} = \mathbf{b}$$

como tal, sino una aproximación

$$(A + \Delta A)\mathbf{x} = \mathbf{b} + \Delta \mathbf{b}.$$

Si el algoritmo utilizado es estable y el sistema también, cabe esperar que el resultado obtenido sea muy parecido al real. Sin embargo, si el sistema está mal condicionado, o el algoritmo no es numéricamente estable, la solución puede diferir sustancialmente de la real.

De estas consideraciones se desprende la utilidad de cuantificar el condicionamiento de un sistema de ecuaciones. Esto se consigue mediante el denominado número de condición de una matriz que veremos inmediatamente.

Antes, consideremos los dos sistemas de ecuaciones siguientes:

$$A\mathbf{x} = \mathbf{b} \rightarrow \begin{bmatrix} 8 & -5 \\ 4 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 14 \end{bmatrix} \quad \text{y} \quad \hat{A}\hat{\mathbf{x}} = \hat{\mathbf{b}} \rightarrow \begin{bmatrix} 0,66 & 3,34 \\ 1,99 & 10,01 \end{bmatrix} \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 12 \end{bmatrix}.$$

La solución de ambos es el vector $[1, 1]^T$.

Si introducimos un $\Delta \mathbf{b} = [-0,04, -0,06]^T$ en el término independiente del primer sistema, su solución pasará a ser $[0,993, 0,9968]^T$. El cambio relativo en la norma euclídea del vector $\mathbf{b}$ es

$$\frac{\|\Delta \mathbf{b}\|_2}{\|\mathbf{b}\|_2} = \frac{\sqrt{0,04^2 + 0,06^2}}{\sqrt{3^2 + 14^2}} \approx 0,0050.$$

Por lo que respecta al vector solución, ese cambio relativo en la norma euclídea es

$$\frac{\|\Delta \mathbf{x}\|_2}{\|\mathbf{x}\|_2} = \frac{\sqrt{(1 - 0,993)^2 + (1 - 0,9968)^2}}{\sqrt{1^2 + 1^2}} \approx 0,0054.$$

Como se puede ver, un pequeño cambio en el vector $\mathbf{b}$ induce un cambio pequeño en el vector solución.

Introduciendo el mismo cambio, $\Delta \hat{\mathbf{b}} = [-0,04, -0,06]^T$, en el vector en el término independiente del segundo sistema, $\hat{\mathbf{b}}$, su solución pasa a ser $[6, 0]^T$. Es decir, un cambio relativo en la norma euclídea de $\hat{\mathbf{b}}$ igual a

$$\frac{\sqrt{0,04^2 + 0,06^2}}{\sqrt{4^2 + 12^2}} = 0,0057,$$

produce un cambio en el vector solución igual a:

$$\frac{\sqrt{5^2 + 1^2}}{\sqrt{1^2 + 1^2}} = 3,6055.$$

Evidentemente, el segundo sistema es mucho más sensible a cambios en el término independiente que el primero.

En la figura 1.7 se representan geoméricamente estos dos sistemas de ecuaciones: en (a) el primero; en (b) el segundo. Como se puede ver, las dos rectas que representan las ecuaciones del primer sistema se cortan nítidamente en el punto $[1, 1]^T$. En el caso del segundo sistema, aun usando una resolución gráfica mayor, apenas se diferencian las dos rectas y mucho menos dónde se cortan.

Estudiemos a continuación la forma de cuantificar la sensibilidad de un sistema de ecuaciones lineales a pequeñas modificaciones tanto en el término independiente como en los elementos de la matriz de coeficientes.

Analicemos en primer lugar el caso de una modificación $\Delta \mathbf{b}$ del término independiente. Veamos cómo se relaciona la solución de

$$A(\mathbf{x} + \Delta \mathbf{x}) = \mathbf{b} + \Delta \mathbf{b}$$

con la de

$$A\mathbf{x} = \mathbf{b}.$$

Designemos por $\|\cdot\|$ cualquier norma vectorial y su correspondiente norma matricial consistente. De las igualdades

$$A(\mathbf{x} + \Delta \mathbf{x}) = \mathbf{b} + \Delta \mathbf{b} \quad \text{y} \quad A\mathbf{x} = \mathbf{b},$$

se obtiene, restando y despejando $\Delta \mathbf{x}$, que

$$\Delta \mathbf{x} = A^{-1} \Delta \mathbf{b}.$$

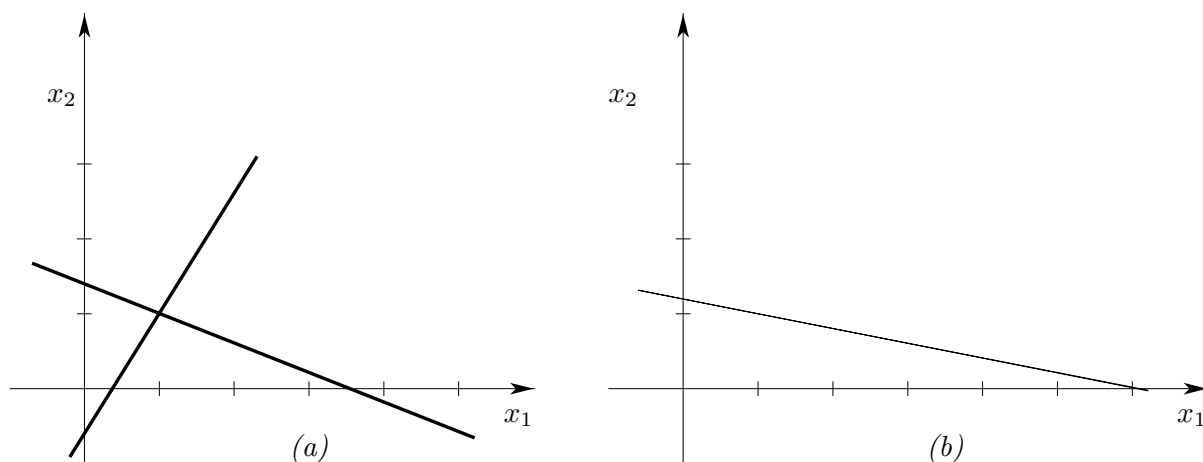


Figura 1.7

Ilustración del buen y mal condicionamiento de dos sistemas de ecuaciones lineales

De la definición de norma matricial consistente con una norma vectorial (ver apéndice A) se tiene que

$$\|\Delta \mathbf{x}\| \leq \|A^{-1}\| \|\Delta \mathbf{b}\| \quad (1.17)$$

y que $\|\mathbf{b}\| \leq \|A\| \|\mathbf{x}\|$ o, lo que es lo mismo, que

$$\frac{1}{\|\mathbf{x}\|} \leq \frac{\|A\|}{\|\mathbf{b}\|}. \quad (1.18)$$

Combinando (1.17) y (1.18) se deduce que el error relativo, $\|\Delta \mathbf{x}\|/\|\mathbf{x}\|$, de la solución del sistema $A\mathbf{x} = \mathbf{b}$ al modificar el término independiente de $\mathbf{b}$ a $\mathbf{b} + \Delta \mathbf{b}$ es

$$\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \|A\| \|A^{-1}\| \frac{\|\Delta \mathbf{b}\|}{\|\mathbf{b}\|}.$$

Definición 1.1 Sea $\|\cdot\|$ una norma matricial consistente con una norma vectorial. Asociado a esa norma, el *número de condición de una matriz* invertible A , $\kappa(A)$, es:

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

El concepto de número de condición de una matriz se generaliza a cualquier matriz A (no necesariamente cuadrada) de rango completo mediante la expresión

$$\kappa(A) = \|A\| \|A^\dagger\|,$$

donde $A^\dagger$ es la *matriz pseudoinversa* de la matriz A .

El número de condición de una matriz A es un indicador del error de amplificación que produce en un vector $\mathbf{x}$ el someterlo a la transformación que define dicha matriz A . Concretamente, si la esfera unidad se somete a esa transformación, el número de condición será igual al cociente de las longitudes de los semiejes mayor y menor del hiperelipsoide resultante de esa transformación.

De la misma forma que hemos analizado la sensibilidad de un sistema de ecuaciones a pequeños cambios en el término independiente, se pueden estudiar cambios en los elementos de la matriz de coeficientes. En efecto, comparemos la solución de

$$A\mathbf{x} = \mathbf{b} \quad \text{y} \quad (A + \Delta A)(\mathbf{x} + \Delta \mathbf{x}) = \mathbf{b}.$$

De la segunda igualdad, como $A\mathbf{x} = \mathbf{b}$, haciendo $\Delta \mathbf{x} = -A^{-1}\Delta A(\mathbf{x} + \Delta \mathbf{x})$ resulta, despreciando el producto $\Delta A \cdot \Delta \mathbf{x}$, que

$$\|\Delta \mathbf{x}\| \leq \|A^{-1}\| \|\Delta A\| \|\mathbf{x}\|.$$

Expresión que también se puede escribir como

$$\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \|A^{-1}\| \|A\| \frac{\|\Delta A\|}{\|A\|}.$$

Así pues, el error relativo que resulta de perturbar ligeramente los coeficientes de la matriz del sistema $A\mathbf{x} = \mathbf{b}$ está acotado en términos del número de condición de la matriz A .

Teorema 1.11 *Para toda matriz A de rango completo:*

1. Su número de condición $\kappa(A) \geq 1$.
2. $\kappa(A) = \kappa(A^\dagger)$.
3. $\kappa(\alpha A) = \kappa(A)$ para todo escalar $\alpha \neq 0$.
4. $\kappa_2(A) = \frac{\sigma_n(A)}{\sigma_1(A)}$, donde σ_n y σ_1 son, respectivamente, los valores singulares mayor y menor de la matriz A .
5. $\kappa_2(A) = \frac{\max_i |\lambda_i(A)|}{\min_i |\lambda_i(A)|}$, si A es simétrica.
6. $\kappa_2(A^T A) = \kappa_2^2(A)$.
7. Su número $\kappa_2(A) = 1$ si la matriz es la identidad o se trata de una matriz ortogonal.
8. Su número de condición $\kappa_2(A)$ es invariante frente a transformaciones ortogonales.

DEMOSTRACIÓN.

1. $AA^\dagger = I \Rightarrow 1 = \|I\| \leq \|A\| \|A^\dagger\|$.
2. $\kappa(A) = \|A\| \|A^\dagger\| = \|A^\dagger\| \|A\| = \kappa(A^\dagger)$.
3. $\kappa(\alpha A) = \|\alpha A\| \|(\alpha A)^{-1}\| = |\alpha| \|A\| |\alpha^{-1}| \|A^{-1}\| = \|A\| \|A^{-1}\| = \kappa(A)$.
4. Recordemos que si σ_i , $1 \leq i \leq n$, son los valores singulares de la matriz A y λ_i , $1 \leq i \leq n$, sus valores propios,

$$\|A\|_2^2 = \max_{1 \leq i \leq n} \lambda_i(A^T A) = \max_i \sigma_i^2 = \sigma_n^2$$

y

$$\|A^\dagger\|_2^2 = \max_{1 \leq i \leq n} \lambda_i\left((A^T A)^{-1}\right) = \max_i \frac{1}{\sigma_i^2} = \frac{1}{\min_i \sigma_i^2} = \frac{1}{\sigma_1^2}.$$

Por lo tanto

$$\kappa_2(A) = \frac{\sigma_n(A)}{\sigma_1(A)}.$$

5. Cuando A es simétrica $\sigma_i(A) = \left| \sqrt{\lambda_i(A^T A)} \right| = \left| \sqrt{\lambda_i^2(A)} \right| = |\lambda_i(A)|$. En consecuencia,

$$\kappa_2(A) = \frac{\max_i |\lambda_i(A)|}{\min_i |\lambda_i(A)|}.$$

6. De la definición de valor singular, de acuerdo con el punto 4 de este teorema,

$$\kappa_2(A) = \sqrt{\frac{\max_i \lambda_i(A^T A)}{\min_i \lambda_i(A^T A)}}.$$

Como $A^T A$ es definida positiva, si $B = A^T A$, $B^T B = B^2$ y $\lambda(B^2) = \lambda^2(B)$. Se tiene entonces que

$$\kappa_2(A^T A) = \sqrt{\frac{\max_i \lambda_i(B^2)}{\min_i \lambda_i(B^2)}} = \sqrt{\frac{\max_i \lambda_i^2(B)}{\min_i \lambda_i^2(B)}} = \frac{\max_i \lambda_i(B)}{\min_i \lambda_i(B)} = \kappa_2^2(A).$$

7. Si la matriz es ortogonal o la identidad, $\|A\|_2^2 = \max_{1 \leq i \leq n} \lambda_i(A^T A) = 1$, lo que implica que su número de condición κ_2 es 1.

8. Esta última propiedad se deduce inmediatamente recordando que $QQ^T = I$ y que, por tanto, $\|QA\|_2 = \|AQ\|_2 = \|A\|_2$. ■

El número de condición de una matriz indica también lo cerca que esa matriz está de la singularidad.

Volvamos al ejemplo que utilizábamos para introducir estos conceptos. La matriz

$$A = \begin{bmatrix} 8 & -5 \\ 4 & 10 \end{bmatrix},$$

cuya inversa es

$$A^{-1} = \begin{bmatrix} 0,10 & 0,05 \\ -0,04 & 0,08 \end{bmatrix},$$

tiene un número de condición $\kappa_1(A) = \|A\|_1 \|A^{-1}\|_1 = 15 \cdot 0,14 = 2,1$. El de

$$\hat{A} = \begin{bmatrix} 0,66 & 3,34 \\ 1,99 & 10,01 \end{bmatrix},$$

cuya inversa es

$$\hat{A}^{-1} = \begin{bmatrix} 250,25 & 83,5 \\ 49,75 & -16,5 \end{bmatrix},$$

es $\kappa_1(\hat{A}) = \|\hat{A}\|_1 \|\hat{A}^{-1}\|_1 = 13,35 \cdot 300 = 4005$: tres órdenes de magnitud superior.

Un error que se comete con frecuencia es asimilar el concepto de número de condición de una matriz con el de su determinante y que, en ese sentido, a mayor determinante, mayor número de condición; nada más lejos de la realidad.

Ejemplo 1.4 Sea A una matriz diagonal de orden 100 definida por

$$a_{11} = 1; \quad a_{ii} = 0,1 \quad 2 \leq i \leq 100.$$

De esta matriz, $\|A\|_2 = 1$ y $\|A^{-1}\|_2 = 10$. El número de condición $\kappa_2(A) = 10$. Por el contrario, su determinante es $\det(A) = 1 \cdot (0,1)^{99} = 10^{-99}$. ●

Ejemplo 1.5 Sea A una matriz bidiagonal de la forma

$$\begin{bmatrix} 1 & 2 & & & \\ & 1 & 2 & & \\ & & 1 & 2 & \\ & & & \ddots & \ddots \\ & & & & 1 & 2 \\ & & & & & 1 \end{bmatrix}.$$

Su inversa es

$$\begin{bmatrix} 1 & -2 & 4 & \cdots & (-2)^{n-1} \\ & 1 & -2 & & (-2)^{n-2} \\ & & 1 & & \vdots \\ & & & \ddots & \vdots \\ & & & & 1 \end{bmatrix}.$$

Se tiene que

$$\|A\|_{\infty} = \|A\|_1 = 3 \quad \text{y} \quad \|A^{-1}\|_{\infty} = \|A^{-1}\|_1 = 1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1.$$

Los números de condición de A son:

$$\kappa_{\infty}(A) = \kappa_1(A) \approx 3 \cdot 2^n.$$

Su determinante en cambio es 1. ●

Los distintos números de condición de una matriz $A \in \mathbb{R}^{n \times n}$ asociados con las normas matriciales más habituales cumplen que:

$$\begin{aligned} \kappa_2(A)/n &\leq \kappa_1(A) \leq n \kappa_2(A); \\ \kappa_{\infty}(A)/n &\leq \kappa_2(A) \leq n \kappa_{\infty}(A); \\ \kappa_1(A)/n^2 &\leq \kappa_{\infty}(A) \leq n^2 \kappa_1(A). \end{aligned}$$

Las matrices con números de condición pequeños, próximos a la unidad, se dicen bien condicionadas; las que tienen números de condición altos, mal condicionadas.

Dado que muchos de los algoritmos para la resolución directa de sistemas de ecuaciones lineales basan su eficacia en la adecuada factorización de la matriz correspondiente y que la representación de esa matriz en el sistema de numeración en coma flotante de un ordenador sufre errores de redondeo, el número de condición de la matriz es también determinante, desde un punto de vista cualitativo, a la hora de valorar la eficacia del algoritmo al factorizar y resolver el sistema. Existen diversos algoritmos para estimar el número de condición de una matriz sin necesidad de invertirla. Aunque calcularlo o estimarlo es un proceso bastante complejo, por lo que se refiere al número de operaciones a efectuar, a aquellos lectores interesados en cómo obtener el número de condición de una matriz, les sugerimos que consulten la bibliografía sobre álgebra lineal numérica indicada al final del capítulo.

1.7 Mínimos cuadrados lineales

Hasta ahora nos hemos ocupado de la resolución de

$$A\mathbf{x} = \mathbf{b},$$

con $A \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, mediante métodos directos: el caso 1a de la figura 1.1 de la página 5. En lo que sigue de este capítulo nos centraremos en la resolución, también por métodos directos, de los demás casos representados en esa figura.

Estudiaremos problemas sin solución, debido a que $\text{rango}(A|\mathbf{b}) \neq \text{rango}(A)$, a los que sin embargo se les puede encontrar una *pseudosolución* siguiendo el criterio de minimizar la norma $\|A\mathbf{x} - \mathbf{b}\|_2$, o problemas con muchas soluciones, de las que se escoge aquella $\mathbf{x}$ cuya norma euclídea, $\|\mathbf{x}\|_2$, es mínima.

El hecho de que para dar solución a los problemas referidos se utilice el criterio de minimizar la norma euclídea de una manera u otra es lo que engloba y da nombre a los procedimientos para resolver esos problemas: mínimos cuadrados.

El *problema lineal de mínimos cuadrados* se plantea formalmente en los siguientes términos:

Dada una matriz $A \in \mathbb{R}^{m \times n}$, de rango $k \leq \min(m, n)$, y un vector $\mathbf{b} \in \mathbb{R}^m$, encontrar un vector $\mathbf{x} \in \mathbb{R}^n$ que minimice $\|A\mathbf{x} - \mathbf{b}\|_2$.

Así enunciado, éste es un problema de optimización no lineal sin condiciones, pudiéndose resolver con cualquiera de las técnicas que la programación no lineal proporciona. En este apartado nos centraremos en métodos específicos que lo resuelven directamente.

La aplicación más generalizada de las técnicas que describiremos surge de forma natural en todas las ramas de la ciencia y de la ingeniería en que se trate de estimar parámetros de funciones cuando el número de datos disponibles es muy superior al de incógnitas a estimar.

El ejemplo paradigmático lo constituye el tratar de ajustar o aproximar a un conjunto de m pares de puntos (t_i, b_i) —pudiendo representar unas mediciones, b_i , obtenidas en unos tiempos, t_i — una función $f(\mathbf{x}, t)$ de n parámetros independientes $x_1, x_2, \dots, x_n$. Si la función es lineal en $x_1, \dots, x_n$ se tiene un problema de mínimos cuadrados lineales en el que si los n parámetros se disponen como los componentes de un vector n -dimensional $\mathbf{x}$ y los datos obtenidos en otro vector m -dimensional $\mathbf{b}$ (usualmente $m \geq n$), se llega a una relación de la forma $A\mathbf{x} = \mathbf{b}$, donde los coeficientes de la matriz A quedan determinados por las observaciones concretas a las que se refiere el experimento o situación física estudiada.

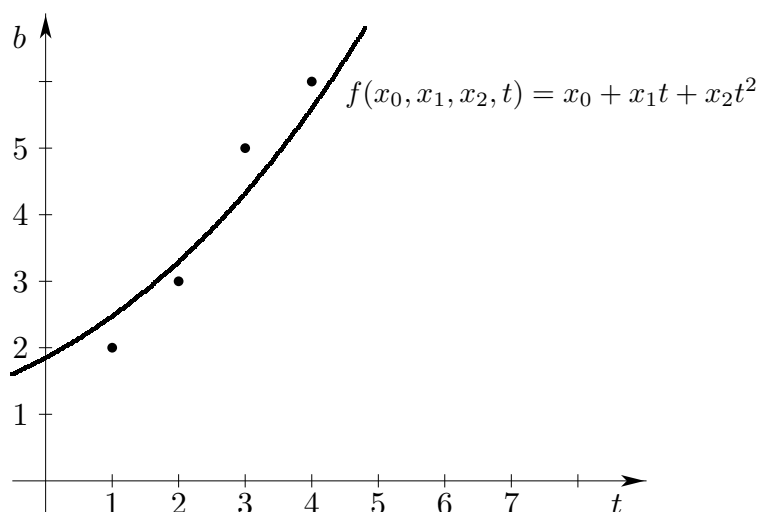
Como ejemplo ilustrativo supongamos que queremos ajustar al conjunto de pares de puntos $\{(t_i, b_i)\} = \{(1, 2), (2, 3), (3, 5), (4, 6)\}$ la función

$$f(x_0, x_1, x_2, t) = x_0 + x_1 t + x_2 t^2,$$

según representa la figura 1.8.

Para los datos y parámetros de este ejemplo el sistema $A\mathbf{x} = \mathbf{b}$ tiene la forma siguiente:

$$\underbrace{\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 2 \\ 3 \\ 5 \\ 6 \end{bmatrix}}_{\mathbf{b}}.$$

**Figura 1.8**

Ejemplo de problema de mínimos cuadrados: ajuste de una función a una nube de puntos

Este sistema de ecuaciones,³ así planteado, no tiene solución; sí es posible, sin embargo, determinar una pseudosolución que mejor cumpla un criterio determinado, por ejemplo, minimizar una norma $\|A\mathbf{x} - \mathbf{b}\|$. Si la norma que se emplea es la más estándar en un espacio vectorial de dimensión finita, es decir, la norma euclídea, el problema que se plantea es el que se conoce habitualmente como el de estimación por mínimos cuadrados.

Las aplicaciones de esta técnica en las ciencias y en la ingeniería son muchísimas y de muy diversa índole —entre las más modernas quizás estén las de determinar la posición de un vehículo espacial en un momento dado o la de definir su trayectoria—. En el capítulo relativo a sistemas de ecuaciones no lineales se describe una aplicación muy interesante sobre la que se profundiza en sus aspectos teóricos y prácticos: la de analizar sistemas eléctricos de generación y transporte de energía, el problema a que da lugar y la forma de resolverlo mediante una sucesión de subproblemas lineales de mínimos cuadrados como los que a continuación pasamos a estudiar.

1.7.1 Fundamentos teóricos del problema

1.7.1.1 Descomposición en valores singulares

La descomposición en valores singulares de una matriz constituye una gran ayuda para el estudio teórico y práctico de problemas de mínimos cuadrados.

³La matriz de este ejemplo es del tipo Vandermonde.

Teorema 1.12 (Descomposición en valores singulares) Si $A \in \mathbb{R}^{m \times n}$ es una matriz de rango r , existen matrices ortogonales $U \in \mathbb{R}^{m \times m}$ y $V \in \mathbb{R}^{n \times n}$ tales que

$$A = U \Sigma V^T, \quad (1.19)$$

donde

$$\Sigma = \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix},$$

$\Sigma \in \mathbb{R}^{m \times n}$ y $\Sigma_r = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$, con

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0.$$

Si las matrices U y V se escriben como

$$U = [\mathbf{u}_1, \dots, \mathbf{u}_m] \quad y \quad V = [\mathbf{v}_1, \dots, \mathbf{v}_n],$$

los $\mathbf{u}_i$ y $\mathbf{v}_i$ son los vectores singulares izquierdos y derechos, respectivamente, correspondientes a los valores singulares σ_i , $i = 1, \dots, r$.

DEMOSTRACIÓN. Sean $\mathbf{x} \in \mathbb{R}^n$ e $\mathbf{y} \in \mathbb{R}^m$ dos vectores tales que

$$\|\mathbf{x}\|_2 = \|\mathbf{y}\|_2 = 1 \quad y \quad A\mathbf{x} = \sigma\mathbf{y}, \quad \text{con} \quad \sigma = \|A\|_2.$$

$\|A\|_2$ es la norma espectral o norma matricial 2 inducida por la norma euclídea $\|\cdot\|_2$. La existencia de estos vectores $\mathbf{x}$ e $\mathbf{y}$ está garantizada por la definición de $\|A\|_2$.

Sean las dos matrices ortogonales

$$V = [\mathbf{x}, V_1] \in \mathbb{R}^{n \times n} \quad y \quad U = [\mathbf{y}, U_1] \in \mathbb{R}^{m \times m}$$

(recuérdese que siempre es posible ampliar un conjunto de vectores ortogonales hasta formar una base ortonormal de $\mathbb{R}^n$). Como $U_1^T A\mathbf{x} = \sigma U_1^T \mathbf{y} = \mathbf{0}$, la matriz $U^T A V$ tiene la siguiente estructura:

$$A_1 = U^T A V = \begin{bmatrix} \mathbf{y}^T \\ U_1^T \end{bmatrix} A [\mathbf{x}, V_1] = \begin{bmatrix} \sigma & \mathbf{w}^T \\ 0 & B \end{bmatrix},$$

donde $B = U_1^T A V_1 \in \mathbb{R}^{(m-1) \times (n-1)}$ y $\mathbf{w}^T = \mathbf{y}^T A V_1$. Dado que

$$\left\| A_1 \begin{bmatrix} \sigma \\ \mathbf{w} \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} \sigma^2 + \mathbf{w}^T \mathbf{w} \\ B\mathbf{w} \end{bmatrix} \right\|_2 \geq \sigma^2 + \mathbf{w}^T \mathbf{w},$$

como

$$\left\| A_1 \begin{bmatrix} \sigma \\ \mathbf{w} \end{bmatrix} \right\|_2 \leq \|A_1\|_2 \left\| \begin{bmatrix} \sigma \\ \mathbf{w} \end{bmatrix} \right\|_2 = \|A_1\|_2 \sqrt{(\sigma^2 + \mathbf{w}^T \mathbf{w})^2},$$

se cumple que $\|A_1\|_2 \geq (\sigma^2 + \mathbf{w}^T \mathbf{w})^{1/2}$. Como las matrices U y V son ortogonales, $\|A_1\|_2 = \|A\|_2 = \sigma$ y, por consiguiente, $\mathbf{w} = \mathbf{0}$. La argumentación de la demostración se completa por inducción. ■

De (1.19) se tiene que

$$A^T A = V \Sigma \Sigma^T V^T \quad \text{y que} \quad A A^T = U \Sigma \Sigma^T U^T.$$

En consecuencia, los σ_i^2 , $i = 1, \dots, r$, son los valores propios no nulos de las matrices simétricas semidefinidas positivas $A^T A$ y $A A^T$; los $\mathbf{v}_i$ y $\mathbf{u}_i$ los correspondientes vectores propios.

Los *valores singulares* de A son las longitudes de los semiejes del hiperelipsoide E definido por:

$$E = \{\mathbf{y} : \mathbf{y} = A\mathbf{x}, \|\mathbf{x}\|_2 = 1\}.$$

Es decir, las longitudes de los semiejes del hiperelipsoide imagen de la esfera unidad resultante de la aplicación que caracteriza la matriz $A \in \mathbb{R}^{m \times n}$. En la figura 1.9 se representa el caso en que $m = n = 2$.

Los valores singulares de una matriz son únicos. Un vector singular $\mathbf{v}_j$, $j \leq r$, es único sólo cuando σ_j^2 es un autovalor único de $A^T A$. Los vectores singulares correspondientes a valores singulares múltiples se pueden utilizar como base ortonormal del subespacio que generan o determinan. Elegidos los vectores singulares $\mathbf{v}_j$, $1 \leq j \leq r$, los $\mathbf{u}_j$, $1 \leq j \leq r$, quedan determinados de forma única de acuerdo con la relación

$$A\mathbf{v}_j = \sigma_j \mathbf{u}_j, \quad j = 1, \dots, r.$$

De forma análoga, dados los vectores singulares $\mathbf{u}_j$, $1 \leq j \leq r$, los $\mathbf{v}_j$, $1 \leq j \leq r$, quedan determinados de forma única de acuerdo con

$$A^T \mathbf{u}_j = \sigma_j \mathbf{v}_j, \quad j = 1, \dots, r.$$

Del teorema anterior se tiene que

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T = U_r \Sigma_r V_r^T, \quad (1.20)$$

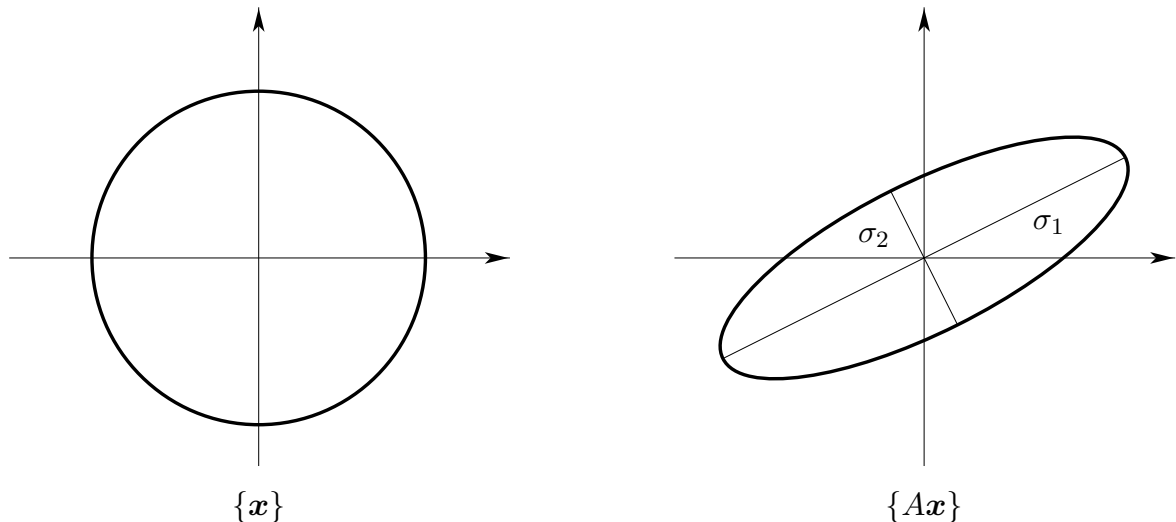


Figura 1.9

Ilustración en dos dimensiones de una transformación lineal de la esfera unidad

donde

$$U_r = [\mathbf{u}_1, \dots, \mathbf{u}_r] \quad \text{y} \quad V_r = [\mathbf{v}_1, \dots, \mathbf{v}_r].$$

A esta descomposición se la denomina habitualmente *descomposición en valores singulares de rango completo*. La expresión (1.20) pone de manifiesto una descomposición de la matriz A , de rango r , en suma de r matrices de rango 1.

Teorema 1.13 *El vector*

$$\mathbf{x} = V \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix} U^T \mathbf{b}$$

es la solución del problema

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \|A\mathbf{x} - \mathbf{b}\|_2,$$

donde $A \in \mathbb{R}^{m \times n}$ y $\text{rango}(A) = r \leq \min(m, n)$, que hace mínima $\|\mathbf{x}\|_2$.

DEMOSTRACIÓN. Sean

$$\mathbf{z} = V^T \mathbf{x} = \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \end{bmatrix} \quad \text{y} \quad \mathbf{c} = U^T \mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix},$$

con $\mathbf{z}_1, \mathbf{c}_1 \in \mathbb{R}^r$. Entonces, teniendo en cuenta la ortogonalidad de U y V , y que $U^T A V = \Sigma$,

$$\begin{aligned} \|\mathbf{b} - A\mathbf{x}\|_2 &= \|U^T(\mathbf{b} - A V V^T \mathbf{x})\|_2 \\ &= \left\| \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix} - \begin{bmatrix} \Sigma_r & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \end{bmatrix} \right\|_2 = \left\| \begin{bmatrix} \mathbf{c}_1 - \Sigma_r \mathbf{z}_1 \\ \mathbf{c}_2 \end{bmatrix} \right\|_2. \end{aligned}$$

La norma euclídea del vector de residuos $\mathbf{b} - A\mathbf{x}$ será mínima cuando

$$\mathbf{z}_1 = \Sigma_r^{-1} \mathbf{c}_1,$$

para cualquier $\mathbf{z}_2$. Es evidente que haciendo $\mathbf{z}_2 = \mathbf{0}$ se minimizará $\|\mathbf{z}\|_2$ y, por tanto, también $\|\mathbf{x}\|_2 = \|V\mathbf{z}\|_2 = \|\mathbf{z}\|_2$. ■

Definición 1.2 A la matriz

$$A^\dagger = V \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix} U^T \in \mathbb{R}^{n \times m}$$

se la denomina *matriz pseudoinversa* o *inversa generalizada Moore-Penrose* de A .

La definición de $A^\dagger$ no depende de las U y V que se escogen en la descomposición de valores singulares de A . Obsérvese que $(A^T)^\dagger = (A^\dagger)^T$; en general, $(AB)^\dagger \neq B^\dagger A^\dagger$.

Se puede comprobar fácilmente que la matriz pseudoinversa satisface las denominadas condiciones de Penrose:

$$\begin{aligned} AA^\dagger A &= A \\ A^\dagger A A^\dagger &= A^\dagger \\ (AA^\dagger)^T &= AA^\dagger \\ (A^\dagger A)^T &= A^\dagger A. \end{aligned}$$

La solución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ es, de acuerdo con esta última definición, $\mathbf{x} = A^\dagger \mathbf{b}$. Además, cumple que

$$\mathbf{x} \perp \ker(A) \quad \text{y que} \quad \mathbf{A}\mathbf{x} = P_{\text{Im}(A)} \mathbf{b},$$

donde $P_{\text{Im}(A)}$ es la matriz de proyección ortogonal sobre $\text{Im}(A)$ paralelamente a $\ker(A^T)$. El vector $P_{\text{Im}(A)} \mathbf{b}$ es la *proyección ortogonal* de $\mathbf{b}$ sobre $\text{Im}(A)$.

Dos casos de matriz pseudoinversa son de particular interés:

a) Si $A \in \mathbb{R}^{m \times n}$, $m \geq n$ y $\text{rango}(A) = n$,

$$A^\dagger = (A^T A)^{-1} A^T.$$

b) Si $A \in \mathbb{R}^{m \times n}$, $m \leq n$ y $\text{rango}(A) = m$,

$$A^\dagger = A^T (A A^T)^{-1}.$$

El caso a) representa el problema de mínimos cuadrados resultante de un sistema de ecuaciones incompatible con matriz de rango completo. El b) el de un sistema de ecuaciones compatible indeterminado con, también, matriz de rango completo. Este caso b) resuelve

$$\underset{\mathbf{x} \in S}{\text{minimizar}} \|\mathbf{x}\|_2, \quad \text{donde} \quad S = \{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{b}\}.$$

Si $S \subset \mathbb{R}^n$ es un subespacio, $P_S \in \mathbb{R}^{n \times n}$ es la *matriz de proyección ortogonal* sobre S si

$$\text{Im}(P_S) = S, \quad P_S^2 = P_S \quad \text{y} \quad P_S^T = P_S.$$

Si $\mathbf{x} \in \mathbb{R}^n$, se puede descomponer en dos componentes ortogonales de la forma

$$\mathbf{x} = \mathbf{x}_1 + \mathbf{x}_2 = P_S \mathbf{x} + (I - P_S) \mathbf{x},$$

donde $\mathbf{x}_1 \in S$ y $\mathbf{x}_2 \perp \mathbf{x}_1$.

La matriz pseudoinversa proporciona unas fórmulas para la obtención de las matrices de proyección ortogonal sobre los cuatro subespacios fundamentales de A :

$P_{\text{Im}(A)}$	$= AA^\dagger$
$P_{\ker(A^T)}$	$= I - AA^\dagger$
$P_{\text{Im}(A^T)}$	$= A^\dagger A$
$P_{\ker(A)}$	$= I - A^\dagger A$

Si un subespacio S está generado por los vectores ortogonales $\mathbf{u}_1, \dots, \mathbf{u}_k$, se deduce inmediatamente que

$$P_S = U U^T,$$

donde $U = [\mathbf{u}_1, \dots, \mathbf{u}_k]$.

1.7.1.2 Sistemas incompatibles. Ecuaciones normales

Si se tiene una ecuación $A\mathbf{x} = \mathbf{b}$, con $A \in \mathbb{R}^{m \times n}$, y no existe solución a la misma si $\mathbf{b} \notin \text{Im}(A)$ o $\text{rango}(A|\mathbf{b}) \neq \text{rango}(A)$, el problema se resolverá, como adelantábamos en la introducción de este capítulo, buscando una pseudosolución, $\mathbf{x}$, que acerque $A\mathbf{x}$ lo más posible a $\mathbf{b}$ en el sentido de la $\|\cdot\|_2$, es decir, $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$. El siguiente resultado garantiza que el problema de encontrar ese mínimo es equivalente a resolver un sistema lineal de ecuaciones.

Teorema 1.14 Sean X e Y dos espacios vectoriales de dimensiones finitas n y m sobre el cuerpo $\mathbb{R}$ y $\mathcal{A}$ una transformación lineal representada en dos bases de X e Y por la matriz A . Para un vector dado $\mathbf{b} \in Y$, el vector $\mathbf{x} \in X$ minimiza $\|A\mathbf{x} - \mathbf{b}\|_2$ si y sólo si $A^T A\mathbf{x} = A^T \mathbf{b}$.

DEMOSTRACIÓN. Sean $\text{Im}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\}$ y $\ker(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$. El complemento ortogonal del conjunto $\text{Im}(A)$ será:

$$(\text{Im}(A))^\perp = \{\mathbf{r} : \mathbf{r}^T \mathbf{z} = 0, \forall \mathbf{z} \in \text{Im}(A)\} = \{\mathbf{r} : \mathbf{r}^T A = \mathbf{0}^T\} = \{\mathbf{r} : A^T \mathbf{r} = \mathbf{0}\} = \ker(A^T).$$

El problema planteado es obviamente equivalente a minimizar $\|\mathbf{b} - \hat{\mathbf{b}}\|_2$, donde $\hat{\mathbf{b}} \in \text{Im}(A)$. Por el teorema de la proyección (ver apéndice A), $\hat{\mathbf{b}}$ es un vector que minimiza la norma anterior si y sólo si $\mathbf{b} - \hat{\mathbf{b}} \in (\text{Im}(A))^\perp$; es decir, si $\mathbf{b} - \hat{\mathbf{b}} \in \ker(A^T)$, o de forma equivalente, $\mathbf{0} = A^T(\mathbf{b} - \hat{\mathbf{b}}) = A^T \mathbf{b} - A^T A\mathbf{x}$. ■

Al sistema de ecuaciones que define la relación $A^T A\mathbf{x} = A^T \mathbf{b}$ se le denomina *ecuaciones normales*.

El vector solución $\mathbf{x}$ es único si $A^T A$ es invertible (si y sólo si la transformación lineal $\mathcal{A}$ es inyectiva: $\text{rango}(A) = n$); en este caso

$$\mathbf{x} = (A^T A)^{-1} A^T \mathbf{b}.$$

La representación geométrica del problema en tres dimensiones es la de la figura 1.10. Como el vector de residuos, $\mathbf{r} = \mathbf{b} - A\mathbf{x}$, es ortogonal al subespacio (en este caso es un plano) $\text{Im}(A)$, lo es a los vectores que lo definen: $\mathbf{a}_1$ y $\mathbf{a}_2$; es decir, $A^T(A\mathbf{x} - \mathbf{b}) = \mathbf{0}$. Si la matriz $A^T A$ es invertible,

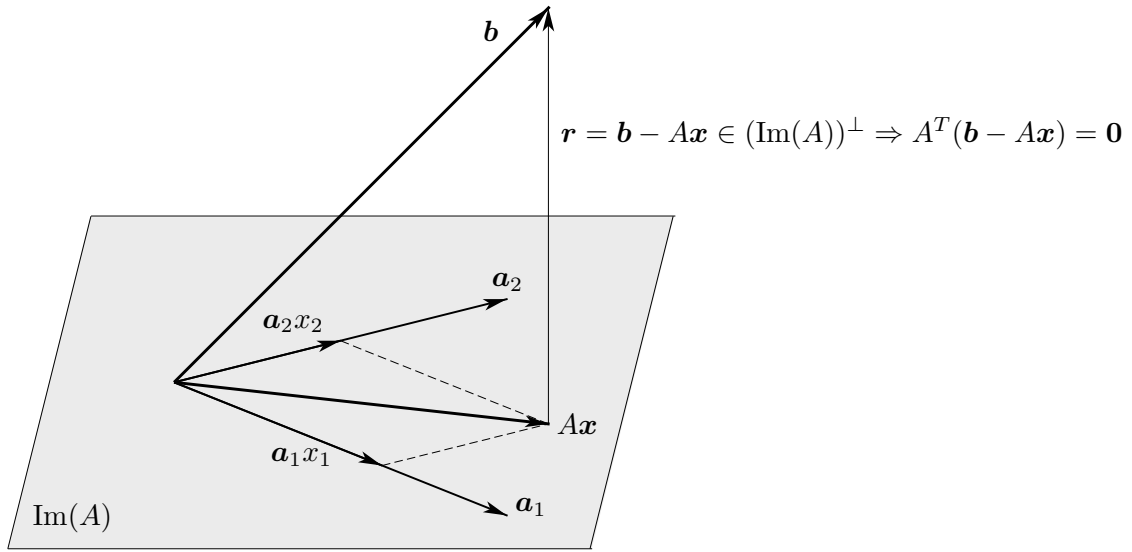
$$\mathbf{r} = \mathbf{b} - A\mathbf{x} = (I - P_{\text{Im}(A)})\mathbf{b},$$

donde $P_{\text{Im}(A)} = A(A^T A)^{-1} A^T$ es la matriz de proyección ortogonal sobre $\text{Im}(A)$ paralelamente a $\ker(A^T)$.

Las ecuaciones normales y el vector de residuos del problema se pueden combinar y formar un sistema de ecuaciones ampliado, $(m+n) \times (m+n)$:

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{r} \\ \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix}.$$

Este sistema posee una matriz simétrica e indefinida (a menos que $A = 0$). Se suele utilizar para mejorar iterativamente la solución numérica del problema original y en métodos donde A es dispersa.

**Figura 1.10**

Descripción geométrica del problema $\min_{\mathbf{x} \in \mathbb{R}^2} \|A\mathbf{x} - \mathbf{b}\|_2$, $A \in \mathbb{R}^{3 \times 2}$

Antes de proseguir con la profundización teórica del problema, hagamos una breve incursión en el terreno de la estadística que es donde más frecuentemente se presenta la resolución de problemas de mínimos cuadrados.

Si los componentes del vector de datos $\mathbf{b}$ que venimos manejando, $b_i, i = 1, \dots, m$, son variables aleatorias independientes, normalmente distribuidas, de media μ_i e igual varianza σ^2 , se tendrá que

$$E[b_i] = \mu_i$$

$$E[(b_i - \mu_i)(b_k - \mu_k)] = \begin{cases} \sigma^2 & \text{para } i = k \\ 0 & \text{para los demás} \end{cases}$$

Haciendo $\boldsymbol{\mu} = [\mu_1, \dots, \mu_m]^T$, las expresiones anteriores resultan:

$$E[\mathbf{b}] = \boldsymbol{\mu}; \quad E[(\mathbf{b} - \boldsymbol{\mu})(\mathbf{b} - \boldsymbol{\mu})^T] = \sigma^2 I.$$

La matriz de covarianzas del vector aleatorio $\mathbf{b}$ es pues $\sigma^2 I$. El valor óptimo antes obtenido, $\mathbf{x} = (A^T A)^{-1} A^T \mathbf{b}$, es también un vector aleatorio de media

$$\begin{aligned} E[\mathbf{x}] &= E[(A^T A)^{-1} A^T \mathbf{b}] \\ &= (A^T A)^{-1} A^T E[\mathbf{b}] \\ &= (A^T A)^{-1} A^T \boldsymbol{\mu} \end{aligned}$$

y de matriz de covarianzas

$$E[(\mathbf{x} - E(\mathbf{x}))(\mathbf{x} - E(\mathbf{x}))^T] = E[(A^T A)^{-1} A^T (\mathbf{b} - \boldsymbol{\mu})(\mathbf{b} - \boldsymbol{\mu})^T A (A^T A)^{-1}]$$

$$\begin{aligned}
&= (A^T A)^{-1} A^T E \left[(\mathbf{b} - \boldsymbol{\mu})(\mathbf{b} - \boldsymbol{\mu})^T \right] A (A^T A)^{-1} \\
&= \sigma^2 (A^T A)^{-1}.
\end{aligned}$$

1.7.1.3 Sistemas indeterminados

Si la ecuación —sistema de ecuaciones— tiene más de una solución, como ocurre cuando se pretende ajustar una función a un conjunto de puntos y el número de éstos es menor que el de parámetros de la función, o en problemas de optimización con condiciones lineales donde el número de condiciones es menor que el de variables del problema y se pretende encontrar una buena solución inicial factible, siempre se puede calcular aquella solución que tiene menor norma euclídea.

Teorema 1.15 Sean X e Y dos espacios vectoriales de dimensiones finitas n y m sobre el cuerpo $\mathbb{R}$ y A una transformación lineal representada en dos bases de X e Y por la matriz A . El vector $\mathbf{x}$ de norma euclídea mínima que satisface la ecuación $A\mathbf{x} = \mathbf{b}$ es el dado por $\mathbf{x} = A^T \mathbf{z}$, donde $\mathbf{z}$ es una solución de la ecuación $AA^T \mathbf{z} = \mathbf{b}$.

DEMOSTRACIÓN. Si $\mathbf{x}_1$ es una solución de la ecuación $A\mathbf{x} = \mathbf{b}$, cualquier solución de la misma se puede expresar como $\mathbf{x} = \mathbf{x}_1 + \mathbf{u}$, donde $\mathbf{u} \in \ker(A)$; es decir, estará en la variedad lineal $\mathbf{x}_1 + \ker(A)$. El teorema de la proyección garantiza la existencia en esta variedad lineal de un único $\mathbf{x}$ tal que su norma $\|\mathbf{x}\|_2$ es mínima y además pertenece a $(\ker(A))^\perp$.

Como $\mathbf{x} \in (\ker(A))^\perp$, pertenecerá a $\text{Im}(A^T)$, es decir, se podrá expresar como $\mathbf{x} = A^T \mathbf{z}$ para algún $\mathbf{z} \in Y$. Como $A\mathbf{x} = \mathbf{b}$, entonces

$$AA^T \mathbf{z} = \mathbf{b}.$$

■

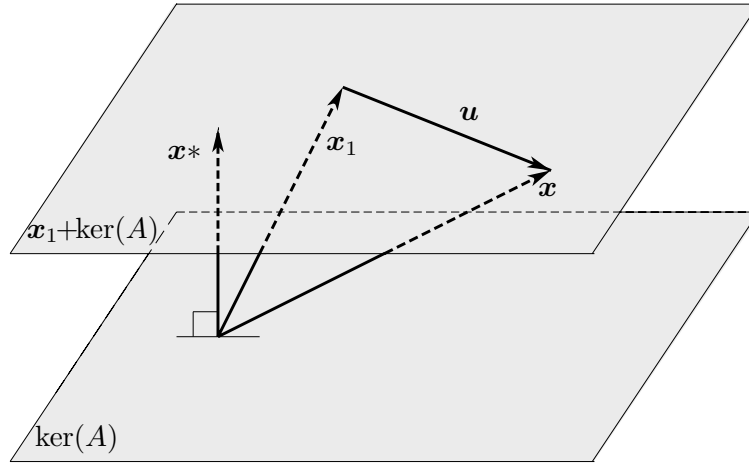
Cuando la matriz AA^T es invertible, la solución óptima es

$$\mathbf{x} = A^T (AA^T)^{-1} \mathbf{b}.$$

La interpretación geométrica de este resultado en $\mathbb{R}^3$ se esquematiza en la figura 1.11.

1.7.2 Resolución numérica del problema

Como hemos visto en estos dos últimos apartados, para resolver el problema de mínimos cuadrados lineales podría utilizarse cualquiera de los métodos que se estudian para resolver sistemas en los que la matriz es cuadrada y simétrica —tanto las matrices AA^T como $A^T A$ lo son— y aplicarlos a las ecuaciones normales $A^T A \mathbf{x} = A^T \mathbf{b}$, en el caso de que el sistema fuese incompatible, o al sistema $AA^T \mathbf{z} = \mathbf{b}$, cuando se estuviese frente a un sistema indeterminado. Como el número de condición κ_2 de AA^T y $A^T A$ es el cuadrado del de la matriz A (ver el teorema 1.11 de la página 70), puede ocurrir que si el problema originalmente no está bien condicionado, las dificultades numéricas resulten insalvables al resolver el sistema correspondiente,

**Figura 1.11**

Interpretación geométrica en $\mathbb{R}^3$ del problema $\mathbf{x}^* = \min_{\mathbf{x} \in \mathbb{R}^3} \{\|\mathbf{x}\|_2 : A\mathbf{x} = \mathbf{b}\}$

por el método de Cholesky, por ejemplo, en una máquina donde la precisión de los cálculos no sea la adecuada. Como ejemplo de esta desfavorable circunstancia, consideremos la matriz

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ \varepsilon & & & & \\ & \varepsilon & & & \\ & & \varepsilon & & \\ & & & \varepsilon & \\ & & & & \varepsilon \end{bmatrix}.$$

El rango de esta matriz es 5, para $\varepsilon \neq 0$. La matriz

$$A^T A = \begin{bmatrix} 1 + \varepsilon^2 & 1 & 1 & 1 & 1 \\ 1 & 1 + \varepsilon^2 & 1 & 1 & 1 \\ 1 & 1 & 1 + \varepsilon^2 & 1 & 1 \\ 1 & 1 & 1 & 1 + \varepsilon^2 & 1 \\ 1 & 1 & 1 & 1 & 1 + \varepsilon^2 \end{bmatrix}$$

también es de rango 5, para $\varepsilon \neq 0$. El número de condición $\kappa_2(A)^2 = \kappa_2(A^T A) = (5 + \varepsilon^2)/\varepsilon^2$. Si ε es superior a la precisión de la máquina pero ε^2 no (por ejemplo, si $\varepsilon = 0,5 \times 10^{-5}$, $\varepsilon^2 = 0,25 \times 10^{-10}$ y la precisión de la máquina $\epsilon = 1,0 \times 10^{-10}$), la representación interna de la matriz $A^T A$ será

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

por lo que, a efectos numéricos en esa máquina, esta matriz será singular de rango 1: las ecuaciones normales no podrían ser resueltas.

Otro aspecto importante que hace poco aconsejable en general la utilización de las ecuaciones basadas en el cálculo de $A^T A$ ó AA^T , nace del hecho de que aun cuando la matriz original A sea muy dispersa, puede que aquellos productos no lo sean tanto e incluso pueden ser totalmente densas. Un ejemplo sería

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & & & \\ & 0 & & \\ & & 0 & \\ & & & 0 \end{bmatrix}, \quad y \quad A^T A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

Veremos a continuación cómo para resolver problemas de mínimos cuadrados se pueden utilizar unas técnicas que evitan estos problemas transformando directamente la matriz A y reduciéndola a otra más fácil de manipular.

1.7.2.1 Método de Gram-Schmidt

El procedimiento clásico de Gram-Schmidt obtiene una base ortonormalizada del subespacio $\text{Im}(A)$. Comienza normalizando el primer vector columna de A : $\mathbf{e}_1 = \mathbf{a}_1 / \|\mathbf{a}_1\|_2$. A continuación se sustrae del vector $\mathbf{a}_2$ su componente en la dirección de $\mathbf{e}_1$, $\langle \mathbf{a}_2 | \mathbf{e}_1 \rangle \mathbf{e}_1$, resultando un vector ortogonal a $\mathbf{e}_1$, el cual a su vez se normaliza... El proceso continúa con los demás vectores columna de A . Los diversos vectores ortonormales que forman la base se van obteniendo de la siguiente forma:

$$\begin{aligned} \mathbf{e}_1 &= \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|_2}; \\ \mathbf{e}_2 &= \frac{\mathbf{a}_2 - \langle \mathbf{a}_2 | \mathbf{e}_1 \rangle \mathbf{e}_1}{\|\mathbf{a}_2 - \langle \mathbf{a}_2 | \mathbf{e}_1 \rangle \mathbf{e}_1\|_2}; \\ \mathbf{e}_3 &= \frac{\mathbf{a}_3 - \langle \mathbf{a}_3 | \mathbf{e}_1 \rangle \mathbf{e}_1 - \langle \mathbf{a}_3 | \mathbf{e}_2 \rangle \mathbf{e}_2}{\|\mathbf{a}_3 - \langle \mathbf{a}_3 | \mathbf{e}_1 \rangle \mathbf{e}_1 - \langle \mathbf{a}_3 | \mathbf{e}_2 \rangle \mathbf{e}_2\|_2}; \\ &\vdots \end{aligned}$$

En la figura 1.12 se describe esquemáticamente el procedimiento de Gram-Schmidt en la fase de obtención de $\mathbf{e}_3$.

El algoritmo de la tabla 1.15 describe, para una matriz general $A^{m \times n}$, los pasos del procedimiento descrito.

Para la resolución del problema de mínimos cuadrados que nos hemos propuesto, sin usar las ecuaciones normales, el método de Gram-Schmidt, en principio, podría ser útil. En efecto, en el procedimiento podemos ver que si se hace

$$\mathbf{p}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} \langle \mathbf{e}_i | \mathbf{a}_j \rangle \mathbf{e}_i,$$

cualquier vector $\mathbf{e}_j = \mathbf{p}_j / \|\mathbf{p}_j\|_2 = \mathbf{p}_j / u_{jj}$. Definiendo $u_{ij} = \langle \mathbf{e}_i | \mathbf{a}_j \rangle$,

$$\mathbf{p}_j = u_{jj} \mathbf{e}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} u_{ij} \mathbf{e}_i.$$

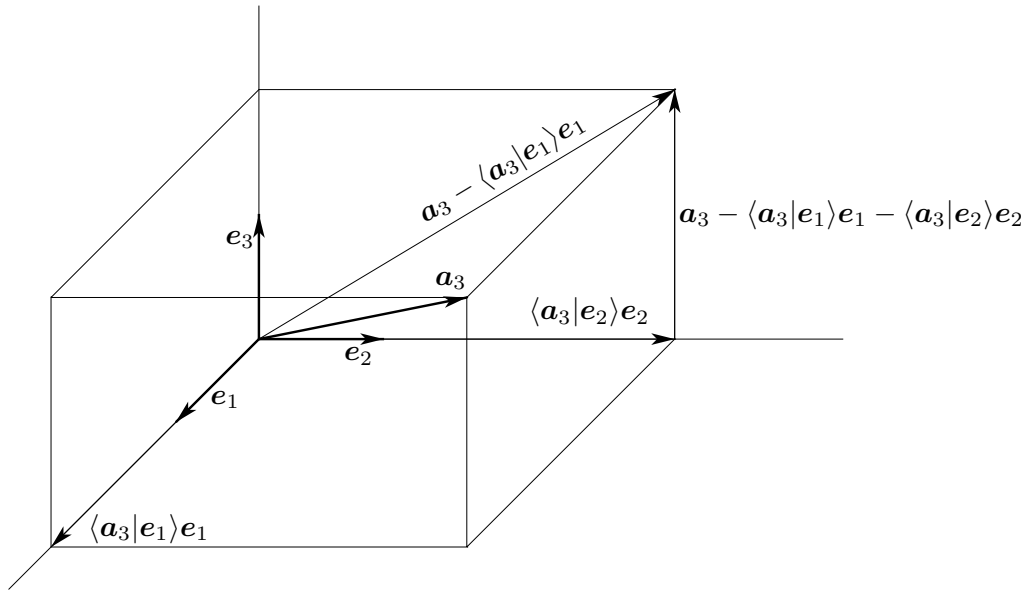


Figura 1.12

Descripción geométrica del proceso de ortonormalización de Gram-Schmidt

Tabla 1.15

Algoritmo clásico de Gram-Schmidt para la ortonormalización de los vectores columna de una matriz $A^{m \times n}$

```

for  $j = 1$  to  $n$ 
  for  $i = 1$  to  $j - 1$ 
     $u(i, j) \leftarrow e(1 : m, i)^T \cdot a(1 : m, j)$ 
     $e(1 : m, j) \leftarrow a(1 : m, j) - u(i, j) \cdot e(1 : m, i)$ 
  end
   $u(j, j) \leftarrow \sqrt{\sum_{k=1}^m e(k, j)^2}$ 
   $e(1 : m, j) \leftarrow e(1 : m, j) / u(j, j)$ 
end

```

Si se despeja $\mathbf{a}_j$,

$$\mathbf{a}_j = \sum_{i=1}^j \mathbf{e}_i u_{ij}.$$

En notación matricial esta expresión es $A = EU$,⁴ donde $E^{m \times n}$ es la matriz de columnas $\mathbf{e}_i$ y $U^{n \times n}$ la matriz triangular superior formada por los u_{ij} . Las ecuaciones normales $A^T A \mathbf{x} = A^T \mathbf{b}$, sustituyendo A por EU , se transforman en

$$U^T E^T E U \mathbf{x} = U^T E^T \mathbf{b}$$

y, en definitiva, en

$$U \mathbf{x} = E^T \mathbf{b}. \quad (1.21)$$

Resolver pues $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ es equivalente, después de ortonormalizar los vectores columna de A , a resolver (1.21) por sustitución inversa.

En cada etapa j del algoritmo clásico de Gram-Schmidt se calculan la columnas j de E y U . La explicación geométrica de este proceso es la siguiente: primero se calcula una base ortonormal del subespacio $\text{Im}(A)$; luego se proyecta en esta base el vector $\mathbf{b}$ y, por último, se refiere a la base que definen los vectores columna de A la proyección resultante.

El método clásico de Gram-Schmidt puede resultar numéricamente inestable pudiendo incurrirse en errores de cancelación importantes si alguno de los vectores columna $\mathbf{a}_j$ está próximo al subespacio generado por los vectores $\mathbf{e}_1, \dots, \mathbf{e}_{j-1}$. En este caso los componentes del vector $\mathbf{a}_j - \sum_{i=1}^{j-1} \langle \mathbf{a}_j | \mathbf{e}_i \rangle \mathbf{e}_i$ pueden llegar a ser muy pequeños y su error relativo grande por lo que, al dividirlo por su norma, se amplificarán errores, propagándose al resto de los cálculos desfavorablemente.

Método de Gram-Schmidt modificado

Para solventar estos problemas se ha desarrollado, Rice [1966], el denominado *método de Gram-Schmidt modificado*. Resulta del clásico al reordenar determinados cálculos. La diferencia estriba en que en vez de sustraer del vector $\mathbf{a}_k$ sus componentes sobre los $k-1$ vectores $\mathbf{e}_i$ calculados en las etapas anteriores, el vector $\mathbf{e}_k$, que al principio de la etapa k se hace igual a $\mathbf{a}_k$, se descompone paso a paso en cada uno de los vectores $\mathbf{e}_i$, $i = 1, \dots, k-1$, reactualizándose pues su valor $k-1$ veces una vez sustraído su componente en cada uno de esos $\mathbf{e}_i$. El algoritmo que resulta es el que describe la tabla 1.16. Obsérvese que para $n = 2$ los algoritmos de Gram-Schmidt clásico y modificado coinciden.

Para tratar problemas de rango incompleto se introduce la pivotación de columnas. Con este fin es conveniente intercambiar el orden de las operaciones del algoritmo al objeto de calcular U fila a fila. El resultado, según se describe en la tabla 1.17, es que en vez de calcular en cada etapa j la columnas j de E y U , se calculan la columna j de E y la fila j de U .

La codificación en FORTRAN 77 del algoritmo de Gram-Schmidt modificado de la tabla 1.16

⁴Como E es ortogonal y U triangular superior, el método de Gram-Schmidt define una forma de conseguir una factorización $A = QR$, que definiremos inmediateamente.

Tabla 1.16

Algoritmo modificado de Gram-Schmidt para la ortonormalización de los vectores columna de una matriz $A^{m \times n}$

```
for  $j = 1$  to  $n$ 
   $e(1 : m, j) \leftarrow a(1 : m, j)$ 
  for  $i = 1$  to  $j - 1$ 
     $u(i, j) \leftarrow e(1 : m, i)^T \cdot e(1 : m, j)$ 
     $e(1 : m, j) \leftarrow e(1 : m, j) - u(i, j) \cdot e(1 : m, i)$ 
  end
   $u(j, j) \leftarrow \sqrt{\sum_{k=1}^m e(k, j)^2}$ 
   $e(1 : m, j) \leftarrow e(1 : m, j) / u(j, j)$ 
end
```

Tabla 1.17

Algoritmo modificado de Gram-Schmidt para la ortonormalización de los vectores columna de una matriz $A^{m \times n}$. Versión por filas

```
for  $i = 1$  to  $n$ 
   $e(1 : m, i) \leftarrow a(1 : m, i)$ 
   $u(i, i) \leftarrow \sqrt{\sum_{k=1}^m e(k, i)^2}$ 
   $e(1 : m, i) \leftarrow e(1 : m, i) / u(i, i)$ 
  for  $j = i + 1$  to  $n$ 
     $u(i, j) \leftarrow e(1 : m, i)^T \cdot e(1 : m, j)$ 
     $e(1 : m, j) \leftarrow e(1 : m, j) - u(i, j) \cdot e(1 : m, i)$ 
  end
end
```

para resolver el problema

$$\underbrace{\begin{bmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_b,$$

es la que sigue a continuación. Su solución es

$$x = \begin{bmatrix} 0,3333 \\ 0,3333 \\ 0,3333 \end{bmatrix}.$$

La suma de residuos al cuadrado es 0. Los componentes del vector de residuos son todos cero.

```

PROGRAM Grmsch
C
  implicit double precision (a-h,o-z)
C
  parameter (m=4,n=3)
  dimension a(m,n),u(n,n),x(n),b(m),res(m)
C
  data a/1.d0,0.,0.,0.,1.d0,0.,0.,0.,1.d0,0.,0.,0./
  data b/1.d0,0.,0.,0./
C
C *** Ortonormalizar columnas de A ***
C
  epsi = dsqrt(epsilon(1.d0))*10
  a(2,1) = epsi
  a(3,2) = epsi
  a(4,3) = epsi
  dmax = 0.d0
  do j = 1,n
    do i = 1,j-1
      u(i,j) = prod(m,a(1,i),a(1,j))
      do k = 1,m
        a(k,j) = a(k,j)-u(i,j)*a(k,i)
      end do
    end do
    temp = dsqrt(prod(m,a(1,j),a(1,j)))
    u(j,j) = temp
    do k = 1,m
      a(k,j) = a(k,j)/temp
    end do
  end do
C
C * Comprobar dependencia lineal de los vectores columna *
C
  dmax = dmax1(temp,dmax)
  if (dmax+temp.eq.dmax) then
    print *, 'Stop: dependencia lineal de columna ',k
    stop
  endif
end do
C
C                                     T
C *** Resolver Ux=E b ***
C
  x(n) = prod(m,a(1,n),b)/u(n,n)

```

```
do i = n-1,1,-1
    temp = prod(m,a(1,i),b)
    do j = i+1,n
        temp = temp-u(i,j)*x(j)
    end do
    x(i) = temp/u(i,i)
end do
C
C *** Residuos: sustraer del vector b sus componentes en la base
C      ortonormal que define E ***
C
    res = b
    do j = 1,n
        temp = prod(m,a(1,j),res)
        do i = 1,m
            res(i) = res(i)-temp*a(i,j)
        end do
    end do
C
    print *,x,res
C
end

double precision function prod (n,x,y)
double precision x(1),y(1),suma
C
    suma = 0.d0
    if (n.eq.0) then
        prod = 0.d0
    else
        do i = 1,n
            suma = suma+x(i)*y(i)
        end do
        prod = suma
    endif
C
    return
end
```

El número de operaciones que realizan tanto el método clásico como el modificado es

$O(mn^2)$ sumas/restas y multiplicaciones/divisiones y $O(n)$ raíces cuadradas.

1.7.2.2 Factorización QR o triangularización ortogonal. Transformaciones ortogonales

Recordemos que una propiedad importante de las transformaciones lineales ortogonales es que conservan la norma euclídea; esto es, si $Q^{n \times n}$ es una matriz ortogonal y $\mathbf{x}$ un vector n -dimensional, se cumple que

$$\|Q\mathbf{x}\|_2 = \|\mathbf{x}\|_2.$$

En efecto:

$$\|Q\mathbf{x}\|_2 = \sqrt{\langle Q\mathbf{x}|Q\mathbf{x} \rangle} = \sqrt{\mathbf{x}^T Q^T Q \mathbf{x}} = \sqrt{\mathbf{x}^T \mathbf{x}} = \|\mathbf{x}\|_2.$$

Según esto, si Q es una tal matriz ortogonal, al aplicar su transformación correspondiente

a la norma $\|A\mathbf{x} - \mathbf{b}\|_2$, resulta que

$$\|Q(A\mathbf{x} - \mathbf{b})\|_2 = \|QA\mathbf{x} - Q\mathbf{b}\|_2 = \|A\mathbf{x} - \mathbf{b}\|_2.$$

Es decir, el resultado del problema que se quiere resolver, $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$, no se verá afectado si se realizan una serie de transformaciones ortogonales a la ecuación $A\mathbf{x} = \mathbf{b}$.

Lo que pretendemos en este apartado es, definiendo convenientemente unas transformaciones ortogonales, reducir el problema de mínimos cuadrados a otro más sencillo de resolver desde el punto de vista numérico. En concreto, en el caso que nos ocupa, si $A \in \mathbb{R}^{m \times n}$, $m > n$, $\mathbf{b} \in \mathbb{R}^m$, $\text{rango}(A) = n$ y suponemos que se ha calculado una matriz ortogonal $Q \in \mathbb{R}^{m \times m}$ de tal manera que la matriz

$$QA = R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \begin{matrix} n \\ m - n \end{matrix}$$

es triangular superior, si hacemos

$$Q\mathbf{b} = \begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix} \begin{matrix} n \\ m - n \end{matrix},$$

entonces

$$\|A\mathbf{x} - \mathbf{b}\|_2 = \|QA\mathbf{x} - Q\mathbf{b}\|_2 = \left\| \begin{bmatrix} R_1\mathbf{x} - \mathbf{c} \\ \mathbf{d} \end{bmatrix} \right\|_2 = \sqrt{\|R_1\mathbf{x} - \mathbf{c}\|_2^2 + \|\mathbf{d}\|_2^2},$$

para cualquier $\mathbf{x} \in \mathbb{R}^n$. La solución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ será aquella que haga mínimo $\|R_1\mathbf{x} - \mathbf{c}\|_2^2 + \|\mathbf{d}\|_2^2$. Como $\|\mathbf{d}\|_2^2$ es constante, la solución resultará de resolver el sistema $R_1\mathbf{x} = \mathbf{c}$ por sustitución inversa. La suma de residuos al cuadrado vendrá dada por el valor de la expresión $\|\mathbf{d}\|_2^2$ y el vector de residuos, $\mathbf{r} = A\mathbf{x} - \mathbf{b}$, por

$$\mathbf{r} = Q^T \begin{bmatrix} \mathbf{0} \\ \mathbf{d} \end{bmatrix}.$$

El proceso de reducción de A a R descrito se denomina *factorización QR* o *triangularización ortogonal*. Como hemos indicado anteriormente, el método de Gram-Schmidt obtiene una factorización de este tipo.

Teorema 1.16 Sea la matriz $A \in \mathbb{R}^{m \times n}$ de rango n . El factor R_1 de A tiene todos los elementos de su diagonal principal positivos y es igual al que resulta de la factorización de Cholesky, $G^T G$, de $A^T A$.

DEMOSTRACIÓN. Si $\text{rango}(A) = n$, de acuerdo con el teorema 1.9 de la página 42, la factorización de Cholesky de $A^T A$ es única. Por otro lado,

$$A^T A = \begin{bmatrix} R_1^T & 0 \end{bmatrix} Q Q^T \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = R_1^T R_1.$$

Con esto se concluye la demostración. ■

1.7.2.2.1 Transformaciones de Householder

Las transformaciones de Householder son las primeras que introducimos para reducir la matriz A a una triangular superior R .

Definición 1.3 Se denomina *transformación de Householder* a una transformación lineal de $\mathbb{R}^n$ en $\mathbb{R}^n$ caracterizada por una matriz $H^{n \times n}$ —matriz de Householder— de la forma

$$H = I - 2\mathbf{w}\mathbf{w}^T,$$

donde $\mathbf{w}$ es un vector de $\mathbb{R}^n$ de norma euclídea 1 denominado *vector de Householder*.

Las transformaciones de Householder también se suelen denominar *reflexiones de Householder*.

Teorema 1.17 Toda transformación de Householder es simétrica y ortogonal.

DEMOSTRACIÓN. Por definición

$$H^T = I - 2(\mathbf{w}\mathbf{w}^T)^T = I - 2(\mathbf{w}^T)^T \mathbf{w}^T = I - 2\mathbf{w}\mathbf{w}^T = H.$$

Como además $\mathbf{w}^T \mathbf{w} = \|\mathbf{w}\|_2^2 = 1$,

$$H^T H = H^2 = (I - 2\mathbf{w}\mathbf{w}^T)(I - 2\mathbf{w}\mathbf{w}^T) = I - 4\mathbf{w}\mathbf{w}^T + 4\mathbf{w}(\mathbf{w}^T \mathbf{w})\mathbf{w}^T = I.$$

■

Aplicar una transformación de Householder a un vector cualquiera equivale a *reflejarlo* en el subespacio $(\text{Im}(\mathbf{w}))^\perp$, según representa la figura 1.13. En efecto

$$H\mathbf{a} = (I - 2\mathbf{w}\mathbf{w}^T)\mathbf{a} = \mathbf{a} - 2\mathbf{w}\mathbf{w}^T \mathbf{a} = \mathbf{a} - 2(\mathbf{w}^T \mathbf{a})\mathbf{w}.$$

El vector $(\mathbf{w}^T \mathbf{a})\mathbf{w}$ es la proyección de $\mathbf{a}$ sobre $\mathbf{w}$; es decir, $H\mathbf{a}$ es igual al vector $\mathbf{a}$ menos dos veces su proyección sobre $\mathbf{w}$.

La importancia fundamental de estas transformaciones radica en su capacidad para modificar ortogonalmente —eventualmente hacer cero— determinados componentes de un vector dado. En efecto, si $\mathbf{x}$ e $\mathbf{y}$ son dos vectores no nulos de igual norma euclídea y $\mathbf{w}$ se define como

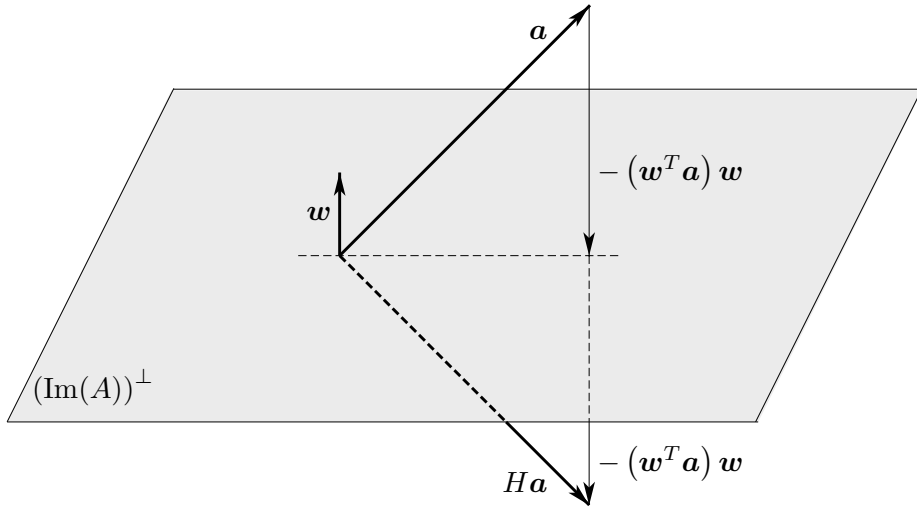
$$\mathbf{w} = \frac{1}{\|\mathbf{x} - \mathbf{y}\|_2}(\mathbf{x} - \mathbf{y}),$$

entonces

$$(I - 2\mathbf{w}\mathbf{w}^T)\mathbf{x} = \mathbf{y}.$$

En efecto:

$$\begin{aligned} \mathbf{x} - 2 \left(\frac{(\mathbf{x} - \mathbf{y})^T \mathbf{x}}{\sqrt{(\mathbf{x} - \mathbf{y})^T (\mathbf{x} - \mathbf{y})}} \right) \frac{\mathbf{x} - \mathbf{y}}{\sqrt{(\mathbf{x} - \mathbf{y})^T (\mathbf{x} - \mathbf{y})}} &= \mathbf{x} - 2 \left(\frac{\mathbf{x}^T \mathbf{x} - \mathbf{y}^T \mathbf{x}}{(\mathbf{x} - \mathbf{y})^T (\mathbf{x} - \mathbf{y})} \right) (\mathbf{x} - \mathbf{y}) \\ &= \mathbf{x} - 2 \left(\frac{\mathbf{x}^T \mathbf{x} - \mathbf{y}^T \mathbf{x}}{2(\mathbf{x}^T \mathbf{x} - \mathbf{y}^T \mathbf{x})} \right) (\mathbf{x} - \mathbf{y}) = \mathbf{y}. \end{aligned}$$

**Figura 1.13**

Representación de la aplicación a $\mathbf{a}$ de la transformación de Householder definida por $\mathbf{w}$

Esto es así pues, al tener $\mathbf{x}$ e $\mathbf{y}$ la misma norma euclídea,

$$\begin{aligned} (\mathbf{x} - \mathbf{y})^T (\mathbf{x} - \mathbf{y}) &= \mathbf{x}^T \mathbf{x} - \mathbf{y}^T \mathbf{x} - \mathbf{x}^T \mathbf{y} + \mathbf{y}^T \mathbf{y} \\ &= 2(\mathbf{x}^T \mathbf{x} - \mathbf{y}^T \mathbf{x}), \end{aligned}$$

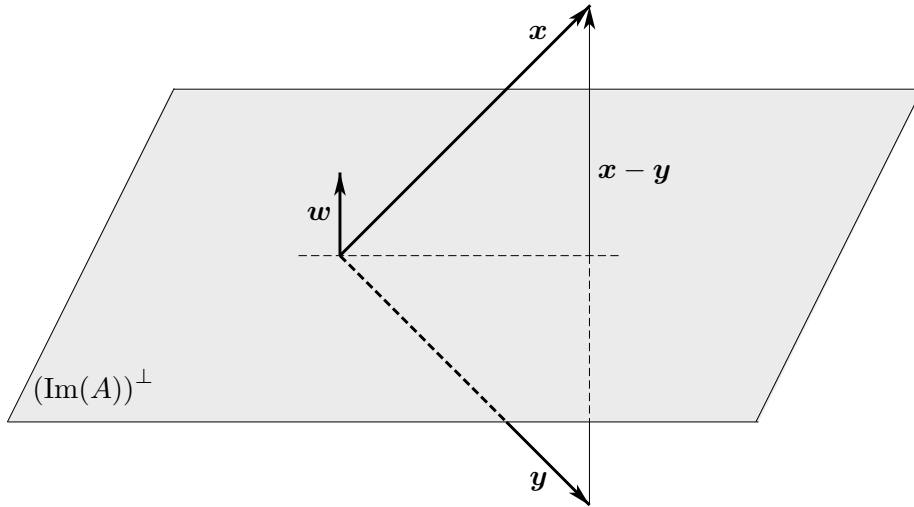
pues $\mathbf{x}^T \mathbf{x} = \mathbf{y}^T \mathbf{y}$ y $\mathbf{y}^T \mathbf{x} = \mathbf{x}^T \mathbf{y}$. Este resultado, geoméricamente, se deduce inmediatamente de la reflexión antes mencionada. El vector $\mathbf{w}$ es colineal con el vector $\mathbf{x} - \mathbf{y}$. Como $\mathbf{x}$ e $\mathbf{y}$ tienen la misma longitud, la reflexión de $\mathbf{x}$ respecto a $(\text{Im}(\mathbf{w}))^\perp$ es $\mathbf{y}$. Ver figura 1.14.

De acuerdo con estas consideraciones, eligiendo el vector $\mathbf{w}$ adecuadamente, se puede construir una transformación de Householder que anule los componentes que se deseen de un vector $\mathbf{x}$ cualquiera dejando los demás como estaban. Por ejemplo, la figura 1.15 representa los cuatro pasos del proceso que se llevaría a cabo para reducir una matriz $A^{6 \times 4}$ a una triangular superior $R^{6 \times 4}$. La matriz A_1 resultaría de la transformación $H_1 A_0$; la A_2 sería $H_2 A_1 = H_2 H_1 A_0$; y así sucesivamente hasta completar los cuatro pasos.

En un problema general, de cara a transformar la matriz original $A^{m \times n}$ de un problema de mínimos cuadrados que se quiere resolver en una triangular superior, interesa aplicarle una sucesión de n transformaciones de Householder que hagan cero los componentes $k+1, \dots, m$, para $k = 1, \dots, n$. Se pretende que la transformación k -ésima haga:

$$H_k \mathbf{a}_k = \begin{cases} a_{ik} & \text{para } i = 1, 2, \dots, k-1 \\ 0 & \text{para } i = k+1, \dots, m. \end{cases}$$

Con este fin, refiriéndonos a la construcción geométrica anterior, los componentes del vector $\mathbf{y}$

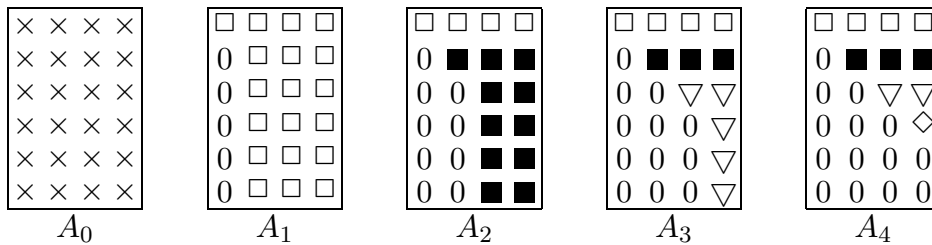

Figura 1.14

Resultado de aplicar a $\mathbf{x}$ la transformación de Householder que define el vector $(\mathbf{x} - \mathbf{y})/\|\mathbf{x} - \mathbf{y}\|_2$

correspondiente a la transformación k serán:

$$\begin{aligned} y_1 &= a_{1k} \\ y_2 &= a_{2k} \\ &\vdots \\ y_k &= \pm \sqrt{a_{kk}^2 + a_{k+1k}^2 + \cdots + a_{mk}^2} \\ y_{k+1} &= 0 \\ &\vdots \\ y_m &= 0. \end{aligned}$$

Como $y_1^2 + y_2^2 + \cdots + y_m^2 = a_{1k}^2 + a_{2k}^2 + \cdots + a_{mk}^2$, los vectores $\mathbf{y}$ y $\mathbf{a}_k$ tienen la misma longitud, por lo que si se construye una transformación de Householder basada en el vector $\mathbf{w} = (\mathbf{a}_k - \mathbf{y})/\|\mathbf{a}_k - \mathbf{y}\|_2$ se tendrá que $H_k \mathbf{a}_k = (I - 2\mathbf{w}\mathbf{w}^T)\mathbf{a}_k = \mathbf{y}$: los primeros $k - 1$ componentes del vector $\mathbf{y}$ serán los mismos de $\mathbf{a}_k$ y los $k + 1$ a m todos cero.


Figura 1.15

Factorización de una matriz 6×4 por transformaciones de Householder

Obsérvese que el componente k -ésimo del vector $\mathbf{y}$ puede adoptar dos signos. Como el cálculo de $\mathbf{w}$ conlleva restar $\mathbf{y}$ de $\mathbf{a}_k$, el error relativo que se puede cometer en esa operación será menor si los componentes k -ésimos de $\mathbf{a}_k$ e $\mathbf{y}$ tienen signo opuesto (recordemos que restar dos números casi iguales en una máquina que trabaja con precisión limitada es *peligroso* pues pueden presentarse errores de cancelación), lo que dará como resultado un número de valor absoluto mayor que si tienen el mismo signo.

En la figura 1.16 se representa un caso en dos dimensiones. Se pretende transformar el vector $\mathbf{a}$ en otro en el eje x_1 : con coordenada por tanto nula en x_2 . Como el vector $\mathbf{a}$ tiene coordenada positiva en el eje x_1 , siguiendo lo apuntado anteriormente, se deberá coger aquel $\mathbf{y}$ cuyo componente y_1 (en este caso un vector en el eje x_1 de igual norma que $\mathbf{a}$) sea positivo. El primer componente del vector $\mathbf{w}$ será $a_1 - (-y_1) = a_1 + y_1$; el segundo, el de $\mathbf{a}$ en x_2 . La transformación de Householder que define este $\mathbf{w}$ transforma el vector $\mathbf{x} = \mathbf{a}$ en $\mathbf{y}'$; la alternativa, que define $\mathbf{w}_{alt}$, en $\mathbf{y}$.

En definitiva, en la transformación de Householder k -ésima que se aplica a la matriz A , como se puede deducir fácilmente, el vector $\mathbf{w}$ es:

$$\mathbf{w} = \frac{1}{\sqrt{2s(s + |a_{kk}|)}} \begin{bmatrix} 0 \\ \vdots \\ a_{kk} + s \cdot \text{signo}(a_{kk}) \\ a_{k+1k} \\ \vdots \\ a_{mk} \end{bmatrix},$$

donde $s = \sqrt{a_{kk}^2 + a_{k+1k}^2 + \cdots + a_{mk}^2}$.

Si la matriz a transformar, A , es de rango completo y el procedimiento está en su etapa j ,

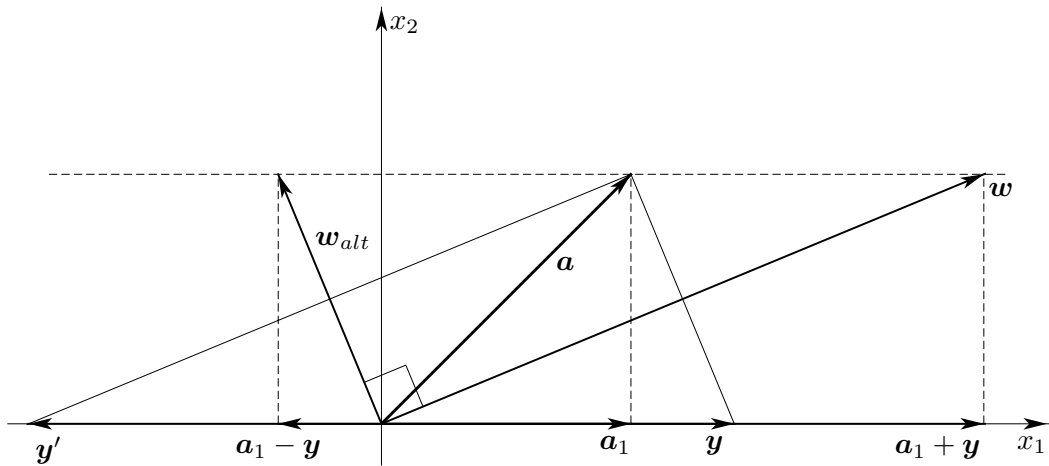


Figura 1.16

Representación de cómo obtener las dos transformaciones de Householder posibles de un vector $\mathbf{a}$

la matriz de Householder que hay que utilizar se puede expresar como

$$H_j = I - \beta \begin{bmatrix} \mathbf{0} \\ \hat{\mathbf{w}} \end{bmatrix} \begin{bmatrix} \mathbf{0}^T, \hat{\mathbf{w}}^T \end{bmatrix} = \begin{bmatrix} \tilde{I} & 0 \\ 0 & \hat{H}_j \end{bmatrix},$$

donde $\hat{\mathbf{w}}$ es un vector de orden $m - j + 1$, $\beta = 2/\|\mathbf{w}\|_2^2$, $\tilde{I}$ es la matriz identidad de orden $j - 1$ y $\hat{H}_j$ es una matriz simétrica de orden $m - j + 1$ dada por

$$\hat{H}_j = \hat{I} - \beta \hat{\mathbf{w}} \hat{\mathbf{w}}^T.$$

Una vez evaluada H_j , supongamos que se aplica a un vector $\mathbf{q}$ (puede representar cualquiera de las columnas $j + 1, \dots, n$ de la matriz A_{j-1}). Dividamos el vector $\mathbf{q}$ en dos: $\tilde{\mathbf{q}}$, de orden $j - 1$ y $\hat{\mathbf{q}}$, de orden $m - j + 1$. Transformemos $\mathbf{q}$ con H_j ; entonces

$$\mathbf{q}' = H_j \mathbf{q} = \begin{bmatrix} \tilde{I} & 0 \\ 0 & \hat{H}_j \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{q}} \\ \hat{\mathbf{q}} \end{bmatrix} = \begin{bmatrix} \tilde{\mathbf{q}} \\ \hat{H}_j \hat{\mathbf{q}} \end{bmatrix}.$$

Es decir, como sabíamos, los primeros $j - 1$ componentes permanecen inalterados. El vector $\hat{H}_j \hat{\mathbf{q}}$ es

$$\hat{H}_j \hat{\mathbf{q}} = (\hat{I} - \beta \hat{\mathbf{w}} \hat{\mathbf{w}}^T) \hat{\mathbf{q}} = \mathbf{z},$$

donde las componentes del vector $\mathbf{z}$ son

$$z_k = q_k - \beta \hat{w}_k \sum_{i=j}^m \hat{w}_i q_i,$$

para $k = 1, \dots, m - j + 1$.

1.7.2.2.1.1 Resolución numérica de $A\mathbf{x} = \mathbf{b}$, $A^{m \times n}$, $m > n$ y rango completo

Como mencionábamos en el apartado 1.7.2.2, mediante transformaciones de Householder se reduce la matriz A a una triangular superior $\begin{bmatrix} R_1 \\ 0 \end{bmatrix}$ y el vector $\mathbf{b}$ a otro $\begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix}$. La solución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ se obtendrá del sistema $R_1 \mathbf{x} = \mathbf{c}$ por sustitución inversa. La suma de residuos al cuadrado será $\|\mathbf{d}\|_2^2$.

El algoritmo completo para resolver el problema es el que describe la tabla 1.18. La versión FORTRAN 77 de este algoritmo para resolver el problema del apartado 1.7 es la que sigue a este párrafo. Su solución es

$$\mathbf{x} = \begin{bmatrix} 0,5 \\ 1,4 \\ 0,0 \end{bmatrix}.$$

La suma de residuos al cuadrado es 0,2. El vector de residuos,

$$\mathbf{r} = \begin{bmatrix} 0,1 \\ -0,3 \\ 0,3 \\ -0,1 \end{bmatrix}.$$

Tabla 1.18

Algoritmo para la resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ por transformaciones de Householder

```

*
* TRANSFORMACIÓN DE LA MATRIZ  $A^{m \times n}$ , COLUMNAS 1 A  $n$ 
*
for  $j = 1$  to  $n$ 
  if  $\max\{|a(j, j)|, \dots, |a(m, j)|\} = 0$  then stop
   $\sigma = \left( \sqrt{\sum_{k=j}^m a(k, j)^2} \right) \cdot \text{signo}(a(j, j))$ 
   $w(j : m) \leftarrow a(j : m, j)$ 
   $w(j) \leftarrow w(j) + \sigma$ 
   $\beta = 2 / \sum_{k=j}^m w^2(k)$ 
   $a(j, j) \leftarrow -\sigma$ 
  for  $l = j + 1$  to  $n$ 
     $s = w^T(j : m) \cdot a(j : m, l)$ 
     $a(j : m, l) \leftarrow a(j : m, l) - w(j : m) \cdot s \cdot \beta$ 
  end
** TRANSFORMACIÓN DEL VECTOR  $b$ .
   $s = w^T(j : m) \cdot b(j : m)$ 
   $b(j : m) \leftarrow b(j : m) - w(j : m) \cdot s \cdot \beta$ 
end
*
* RESOLUCIÓN DEL SISTEMA  $Rx = b$ .
*
for  $j = n$  to  $1$ 
   $x(j) \leftarrow \left( b(j) - \sum_{k=j+1}^n a(j, k) \cdot x(k) \right) / a(j, j)$ 
end
*
* RESIDUOS AL CUADRADO.
*
 $rescua \leftarrow \sum_{k=n+1}^m b^2(k)$ 

```

Obsérvese que, a efectos prácticos, en la etapa j , el vector $w(\cdot)$ se almacena en las posiciones de memoria j a m del vector columna $a(\cdot, j)$. Los valores de $a(1,1)$, $a(2,2), \dots, a(N,N)$ se almacenan en el vector $d(\cdot)$. La idea de estas manipulaciones es ahorrar el máximo número posible de posiciones de memoria.

```
PROGRAM Qrdes
C
  parameter (m=4,n=3)
  dimension a(m,n),b(m),d(n),x(n),betas(n)
C
  data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16./
  data b/2.,3.,5.,6./
C
C *** Reducción QA=R ***
C
  do j = 1,n
    rmm = 0.0
    do i = j,m
      rmm = amax1(rmm,abs(a(i,j)))
    end do
    if (rmm.eq.0.0) stop 'Matriz A de rango incompleto'
    beta = 0.0
    do i = j+1,m
      beta = beta+a(i,j)**2
    end do
    wj      = a(j,j)
    sigma   = sign(sqrt(beta+wj*wj),wj)
    wj      = wj+sigma
    beta    = 2.0/(beta+wj*wj)
    a(j,j)  = wj
    d(j)    = -sigma
    betas(j) = beta
    do l = j+1,n
      s = 0.0
      do k = j,m
        s = s+a(k,j)*a(k,l)
      end do
      s = beta*s
      do k = j,m
        a(k,l) = a(k,l)-a(k,j)*s
      end do
    end do
    s = 0.0
    do k = j,m
      s = s+a(k,j)*b(k)
    end do
    s = beta*s
    do k = j,m
      b(k) = b(k)-a(k,j)*s
    end do
  end do
C
C *** Resolución Rx = b
C
  x(n) = b(n)/d(n)
  do i = n-1,1,-1
    suma = 0.0
    do k = i+1,n
      suma = suma+a(i,k)*x(k)
```

```

        end do
        x(i) = (b(i)-suma)/d(i)
    end do
C
C *** Suma de residuos al cuadrado
C
    s1 = 0.0
    do i = n+1,m
        s1 = s1+b(i)**2
    end do
C
C *** Vector de residuos
C
    do i = n,1,-1
        s = 0.0
        do k = i+1,m
            s = s+a(k,i)*b(k)
        end do
        s = s*betas(i)
        b(i) = -a(i,i)*s
        do k = i+1,m
            b(k) = b(k)-a(k,i)*s
        end do
    end do
C
    print 50,x
    print 60,s1
    print 70,b
C
50 format('x=(',f6.4,',',f6.4,',',f6.4,')')
60 format('Suma de residuos al cuadrado=',f9.6)
70 format('Vector de residuos',4f8.4)
C
end

```

El vector de residuos se obtiene, si el algoritmo ha transformado $\mathbf{b}$ en $\begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix}$, sin más que hacer las siguientes operaciones.

$$\begin{aligned}
 & \mathbf{r} \leftarrow \begin{bmatrix} \mathbf{0} \\ \mathbf{d} \end{bmatrix} \\
 & \textbf{for } k = n \textbf{ to } 1 \\
 & \quad \mathbf{r} \leftarrow H_k \mathbf{r} \\
 & \textbf{end}
 \end{aligned}$$

El número de operaciones que este algoritmo requiere para transformar la matriz A en R es $\boxed{O(mn^2 - n^3/3)}$ sumas/restas y multiplicaciones/divisiones y $\boxed{n}$ raíces cuadradas, más $\boxed{O(n^2/2)}$ multiplicaciones/divisiones y sumas/restas para efectuar la sustitución inversa.

Del proceso de obtención $H_n \cdots H_1 A = R$ se puede llegar también fácilmente a una factorización $A = QR$, caso de requerirse por cualquier circunstancia, sin más que hacer lo que sigue.

$$\begin{aligned}
 & Q \leftarrow I \\
 & \textbf{for } k = n \textbf{ to } 1 \\
 & \quad Q \leftarrow H_k Q
 \end{aligned}$$

end

1.7.2.2.1.2 Resolución numérica de $Ax = b$, $A^{m \times n}$, $n > m$ y rango completo

Si el problema tiene solución, es indeterminado; es decir, tiene muchas soluciones. La de menor norma euclídea se puede calcular procediendo de acuerdo con los pasos siguientes:

Paso 1. Se aplica el algoritmo descrito hasta ahora a la matriz A^T en vez de a A . Resultará

$$Q^T A^T = \begin{bmatrix} R \\ 0 \end{bmatrix},$$

o, lo que es lo mismo, $A^T = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$, donde Q es una matriz ortogonal $n \times n$ y R una triangular superior $m \times m$.

Paso 2. La matriz original A será

$$A = [R^T, 0^T] Q^T.$$

Si se sustituye en la ecuación $Ax = b$, se tendrá que

$$[R^T, 0^T] Q^T x = b.$$

Si se hace el cambio de variable $z = Q^T x$, la última ecuación queda

$$[R^T, 0^T] z = b.$$

Como $z^T z = (Q^T x)^T (Q^T x) = x^T Q^T Q x = x^T x$, las normas euclídeas de x y z serán iguales. Por consiguiente, estructurando el vector z en $\begin{bmatrix} z_R \\ z_0 \end{bmatrix}$ y b de igual manera, la solución de $[R^T, 0^T] z = b$ saldrá de resolver

$$R^T z_R = b_R,$$

siendo los demás componentes del vector z , z_0 , nulos.

Paso 3. El vector solución x que se busca resultará de deshacer el cambio de variable introducido:

$$x = Q \begin{bmatrix} z_R \\ 0 \end{bmatrix}.$$

1.7.2.2.1.3 Resolución numérica de $Ax = b$, $A^{m \times n}$, $m > n$ ó $m < n$ y rango incompleto

Los procedimientos presentados en los dos apartados anteriores dan solución al problema cuando la matriz A es de rango completo. Es conveniente disponer de un procedimiento robusto que resuelva los casos anteriores y que contemple la posibilidad de que la matriz no sea de rango completo e incluso que determine o informe sobre lo cerca que la matriz está de ser de rango incompleto.

Teorema 1.18 *Dada una matriz $A \in \mathbb{R}^{m \times n}$ de rango r , $r \leq \min(m, n)$, existen una permutación, representada por la matriz P , y una matriz ortogonal $Q \in \mathbb{R}^{m \times m}$, tales que*

$$QAP = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \begin{matrix} r \\ m-r \end{matrix}, \quad (1.22)$$

donde $R_{11} \in \mathbb{R}^{r \times r}$ es una matriz triangular superior en la que todos sus elementos de la diagonal principal son positivos (obsérvese que no se requiere que $m \geq n$).

DEMOSTRACIÓN. Como el rango de A es r , siempre será posible elegir unas permutaciones, que se representarán por una matriz P , tales que

$$AP = [A_1, A_2],$$

donde los vectores columna de la matriz $A_1 \in \mathbb{R}^{m \times r}$ son linealmente independientes. Sea

$$QA_1 = \begin{bmatrix} R_{11} \\ 0 \end{bmatrix}$$

la descomposición QR de la matriz A_1 , donde $Q = [Q_1, Q_2]$. De acuerdo con el teorema 1.16 de la página 89, las matrices Q y R_{11} están definidas de forma única y R_{11} tiene todos sus elementos en la diagonal principal positivos. Ahora bien,

$$QAP = [QA_1, QA_2] = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix}.$$

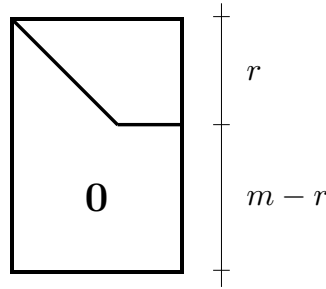
Como $\text{rango}(QAP) = \text{rango}(A) = r$, la matriz $R_{22} = 0$ pues, de lo contrario, QAP tendría más de r filas linealmente independientes, lo cual es imposible. La estructura de QAP tendrá pues la forma de (1.22). ■

Para llevar a cabo las operaciones numéricas necesarias para resolver el problema planteado en este apartado, se procederá siguiendo los tres pasos siguientes.

Paso 1. Transformar la matriz A mediante transformaciones de Householder efectuando además permutaciones de columnas. El procedimiento para llevar esto adelante consiste en evaluar en la etapa k del proceso la norma euclídea de los vectores columna $k, \dots, n$ de la matriz A , limitándose a sus componentes *activos*: $k, \dots, m$. La columna que dé un mayor valor de esa norma se intercambiará con la que se iba a procesar, k , y se continúa el procedimiento en esa columna k . Si, eventualmente, la matriz es de rango r , el resultado de aplicar el proceso de factorización descrito sería el esquematizado en la figura 1.17.

La justificación de esta forma de actuar radica en el proceso de ortonormalización inherente a la factorización $A = QR$ que se realiza. En efecto, en una etapa k , al elegir aquella columna $\mathbf{a}_i, i = k+1, \dots, n$, que tiene una mayor norma euclídea en sus componentes k a m , se está eligiendo aquel vector más distante del subespacio $\text{Im}(\mathbf{a}_1, \dots, \mathbf{a}_{k-1})$ y, por ende, de $\text{Im}(\mathbf{q}_1, \dots, \mathbf{q}_{k-1})$, donde $\mathbf{q}_i$ son los $k-1$ primeros vectores columna de Q ($A = QR$). Esta forma de actuar nos garantiza que cuando un $\mathbf{a}_\ell$ esté en $\text{Im}(\mathbf{a}_1, \dots, \mathbf{a}_{\ell-1})$, los demás $\mathbf{a}_{\ell+1}, \dots, \mathbf{a}_n$ también estarán en ese subespacio.

Desde el punto de vista práctico, el procedimiento de triangularización con pivotación se detendrá no cuando un a_{kk} sea cero sino cuando sea menor que una tolerancia ϵ_1 que se elija al efecto (función de la precisión de la máquina, ϵ , donde se trabaje).

**Figura 1.17**

Resultado de la factorización de una matriz $m \times n$ de rango r por transformaciones de Householder

Paso 2. Del paso anterior se llegará a

$$QAP = R = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix} \begin{matrix} r \\ m - r \\ r & n - r \end{matrix}$$

en la que $\|R_{22}\|_2 \leq \epsilon_1 \|A\|_2$. Debido a los errores de redondeo asociados a los cálculos de matrices ortogonales, R será ortogonalmente equivalente a una matriz $A + E_k$, donde

$$\|E_k\|_2 \leq \epsilon_2 \|A\|_2,$$

siendo ϵ_2 del mismo orden de magnitud que la precisión de la máquina, ϵ , en donde se trabaje. En función de esta precisión también se elige, como se apuntaba antes, ϵ_1 .

Aplicando las mismas transformaciones al vector $\mathbf{b}$ quedará como sigue:

$$Q\mathbf{b} = \begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix} \begin{matrix} r \\ m - r \end{matrix}.$$

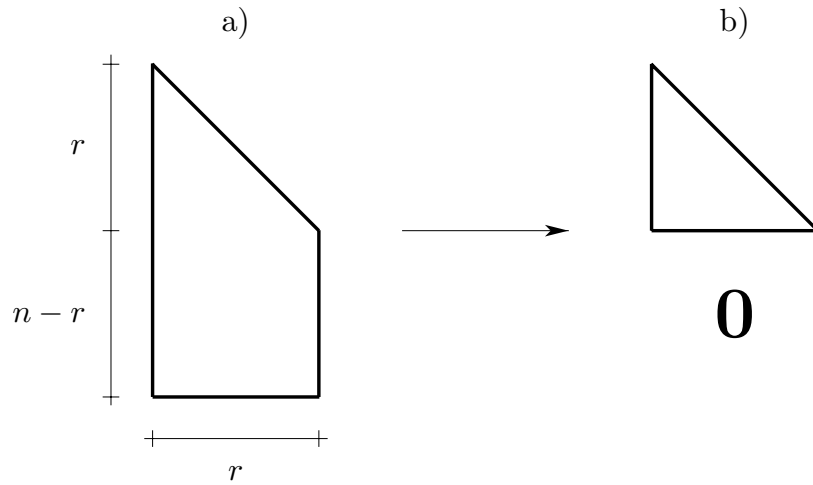
A partir de aquí se pueden presentar dos casos:

- Que $r = n$. La solución del problema saldrá entonces de resolver el sistema triangular superior $R_{11}\mathbf{x} = \mathbf{c}$.
- Que $r < n$. Se deberán construir unas transformaciones ortogonales tales que

$$[R_{11}, R_{12}] Q_1 = [W, 0],$$

donde $Q_1^{n \times n}$ es una matriz ortogonal y $W^{r \times r}$ una triangular superior.

La aplicación de estas últimas transformaciones requiere alguna aclaración. Si la matriz $[R_{11}, R_{12}]$ se denomina W_1 , el proceso a llevar a cabo busca convertir esta matriz, cuya forma traspuesta es la de la figura 1.18-a), en otra de la forma de la figura 1.18-b). Para ello, la idea es proceder en r etapas en cada una de las cuales, k , se construye una transformación de Householder que haga cero los elementos $r + 1$ a n de la columna k y que deje inalterados, de esa columna k , los elementos 1 a $k - 1$ y $k + 1$ a r . En

**Figura 1.18**

Segundo proceso de transformaciones ortogonales para resolver un problema general de mínimos cuadrados

la figura 1.19 se esquematiza cómo se procedería en cuatro etapas para transformar una matriz $W_1^{6 \times 4}$. El elemento que se utiliza para definir cada transformación de Householder y los elementos a hacer cero se indican, respectivamente, con una elipse y el signo $\otimes$.

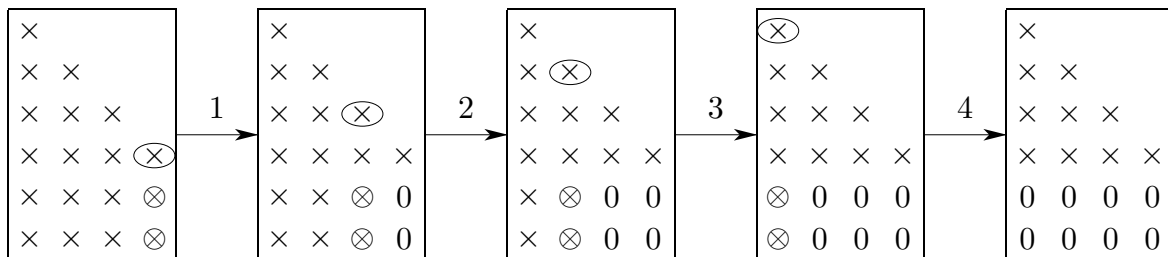
Paso 3. De los pasos anteriores se tendrá que

$$\|A\mathbf{x} - \mathbf{b}\|_2 = \|(QAP)(P^T\mathbf{x}) - Q\mathbf{b}\|_2.$$

Ahora bien, $(QAP)P^T\mathbf{x}$ se puede escribir $(QAP)Q_1Q_1^TP^T\mathbf{x}$ y también,

$$\begin{bmatrix} W & 0 \\ 0 & 0 \end{bmatrix} Q_1^TP^T\mathbf{x} = \begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix}.$$

Si se hace $Q_1^TP^T\mathbf{x} = \mathbf{y}$ y se resuelve $W\mathbf{y}_1 = \mathbf{c}$, el vector solución que se busca, $\mathbf{x}$,

**Figura 1.19**

Segundo proceso de transformaciones ortogonales para resolver un problema general de mínimos cuadrados (continuación)

resultará de

$$x = PQ_1 \begin{bmatrix} y_1 \\ 0 \end{bmatrix}.$$

La codificación en FORTRAN 77 de las ideas expuestas las recoge el programa que se lista a continuación. Este código resuelve el problema del ejemplo del apartado 1.7 de la página 73; se ha añadido una cuarta columna, idénticamente igual a la tercera, a fin de hacer la matriz A de rango incompleto y poder comprobar el buen funcionamiento y las prestaciones del algoritmo.

```

PROGRAM Mincuat
C
  implicit double precision (a-h,o-z)
C
  parameter (m=4,n=4)
  dimension a(m,n),a1(n,m),b(m),w(m),w1(n,m),x(n),beta1(m),
+          ipiv(n)
  data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16.,1.,4.,9.,16./
  data b/2.,3.,5.,6./
  data tau/0.000001/
C
C *** Reducción QAP=
C          |R   R |
C          |11 12| ***
C          |0   0 |
  ira = min0(m,n)
  do i = 1,ira

    imax = i
    rmax = 0.0
    do j = i,n
      h = 0.0
      do k = i,m
        h = h+a(k,j)**2
      end do
      if (h.gt.rmax) then
        rmax = h
        imax = j
      endif
    end do
    ipiv(i) = imax
    if (imax.ne.i) then
      do j = 1,m
        tmp = a(j,i)
        a(j,i) = a(j,imax)
        a(j,imax) = tmp
      end do
    endif
    if (i+1.le.m) then
      call h1 (beta,i,i+1,m,w,a(1,i)) ! Aplicar trans.
      do j = i+1,n
        call h2 (beta,i,i+1,m,w,a(1,j))! a columnas i a n.
      end do
      call h2 (beta,i,i+1,m,w,b)      ! Aplicar trans. a b.
    endif
  end do
C
  k = ira
  do j = 1,ira
    if (dabs(a(j,j)).le.tau) then
      k = j-1
    end if
  end do

```

```

        exit
    endif
end do
kp1 = k+1
C
s1 = 0.0                                ! Suma de residuos
do i = kp1,m                            ! al cuadrado.
    s1 = s1+b(i)**2
end do
C
do i = 1,k                                ! Trasponer A.
    do j = 1,n
        a1(j,i) = a(i,j)
    end do
end do
C
if (k.ne.n) then
C
C    Reducir R    a cero y R    a T.
C           12          11
C
    do i = k,1,-1
        call h1 (beta1(i),i,kp1,n,w1(1,i),a1(1,i))
        do j = i-1,1,-1
            call h2 (beta1(i),i,kp1,n,w1(1,i),a1(1,j))
        end do
    end do
endif
C
x(k) = b(k)/a1(k,k)                      ! Resolución de Tx=Qb.
do i = k-1,1,-1
    sum = 0.0
    do k1 = i+1,k
        sum = sum+a1(k1,i)*x(k1)
    end do
    x(i) = (b(i)-sum)/a1(i,i)
end do
C
if (k.ne.n) then
    do j = kp1,n
        x(j) = 0.0
    end do
    do i = 1,k
        call h2 (beta1(i),i,kp1,n,w1(1,i),x)
    end do
endif
do j = ira,1,-1
    if (ipiv(j).ne.j) then
        l = ipiv(j)
        tmp = x(l)
        x(l) = x(j)
        x(j) = tmp
    endif
end do
C
print '(\'\' Rango de A:\',i3)',k
print '(\'\' Solución:\',6f8.4)',x
print '(\'\' Suma de residuos al cuadrado:\',f9.6)',s1
C

```

```
end

subroutine h1 (beta,i,j,m,w,x)
C                                     ! Construir transforma.
double precision beta,w(m),x(m)      ! de Householder.
C
beta = 0.0
do k = j,m
    w(k) = x(k)
    beta = beta+w(k)*w(k)
end do
w(i) = x(i)
sigma = sign(sqrt(beta+w(i)*w(i)),x(i))
w(i) = w(i)+sigma
beta = 2.0/(beta+w(i)*w(i))
x(i) = -sigma
C
return
end

subroutine h2 (beta,i,j,m,w,x)      ! Aplicar transforma.
C                                     ! de Householder.
double precision beta,w(m),x(m),s
C
s = w(i)*x(i)
do k = j,m
    s = s+w(k)*x(k)
end do
s = s*beta
x(i) = x(i)-w(i)*s
do k = j,m
    x(k) = x(k)-w(k)*s
end do
C
return
end
```

1.7.2.2.2 Transformaciones de Givens

Definición 1.4 Se denomina *transformación de Givens* a una transformación lineal de $\mathbb{R}^n$ en $\mathbb{R}^n$ caracterizada por una matriz $G(i, j)^{n \times n}$ —matriz de Givens— de la forma:

$$G(i, j) = \begin{bmatrix} 1 & & & & & & & & \\ & \ddots & & & & & & & \\ & & 1 & & & & & & \\ & & & c & & & s & & \\ & & & & 1 & & & & \\ & & & & & \ddots & & & \\ & & & & & & 1 & & \\ & & -s & & & & c & & \\ & & & & & & & 1 & \\ & & & & & & & & \ddots & \\ & & & & & & & & & 1 \end{bmatrix} \begin{matrix} \leftarrow i \\ \\ \leftarrow j \end{matrix},$$

donde $c^2 + s^2 = 1$.

Las transformaciones de Givens también se denominan *rotaciones de Givens*. Es inmediato demostrar que las transformaciones de Givens son ortogonales.

Si se tiene una transformación de Givens de $\mathbb{R}^n$ en $\mathbb{R}^n$ representada por una matriz $G(i, j)$ de la forma antes definida, con $c = \cos \theta$ y $s = \sin \theta$, al aplicarla a un vector $\mathbf{x} \in \mathbb{R}^n$ se producirá el siguiente efecto:

$$G(i, j)\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_{i-1} \\ x_i \cos \theta + x_j \sin \theta \\ x_{i+1} \\ \vdots \\ x_{j-1} \\ -x_i \sin \theta + x_j \cos \theta \\ x_{j+1} \\ \vdots \\ x_n \end{bmatrix} \begin{matrix} \leftarrow i \\ \\ \leftarrow j \end{matrix}$$

Es decir, se rota el vector $\mathbf{x}$ un ángulo θ en el subespacio que generan los vectores $\mathbf{e}_i$ y $\mathbf{e}_j$ de $\mathbb{R}^n$.

Si se desea hacer cero alguno de los componentes i ó j de un vector $\mathbf{x}$, concretamente el j , se deberá escoger un θ tal que $-x_i \sin \theta + x_j \cos \theta = 0$, es decir, habrá que hacer

$$\tan \theta = \frac{x_j}{x_i},$$

o, lo que es equivalente,

$$c = \cos \theta = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}$$

y

$$s = \sin \theta = \frac{x_j}{\sqrt{x_i^2 + x_j^2}}.$$

Ejemplo 1.6 En la figura 1.20 se describe, en el espacio euclídeo tridimensional, la rotación del vector

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

en el plano $z - y$ para anular el tercer componente. El ángulo a rotar $\mathbf{x}$ es 45° ; la matriz de Givens que hay que utilizar es, por tanto,

$$G(2,3) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & -\sqrt{2}/2 & \sqrt{2}/2 \end{bmatrix}.$$

El nuevo vector será

$$\mathbf{x}' = \begin{bmatrix} 1 \\ \sqrt{2} \\ 0 \end{bmatrix}.$$

La norma euclídea de éste y del original es $\sqrt{3}$.

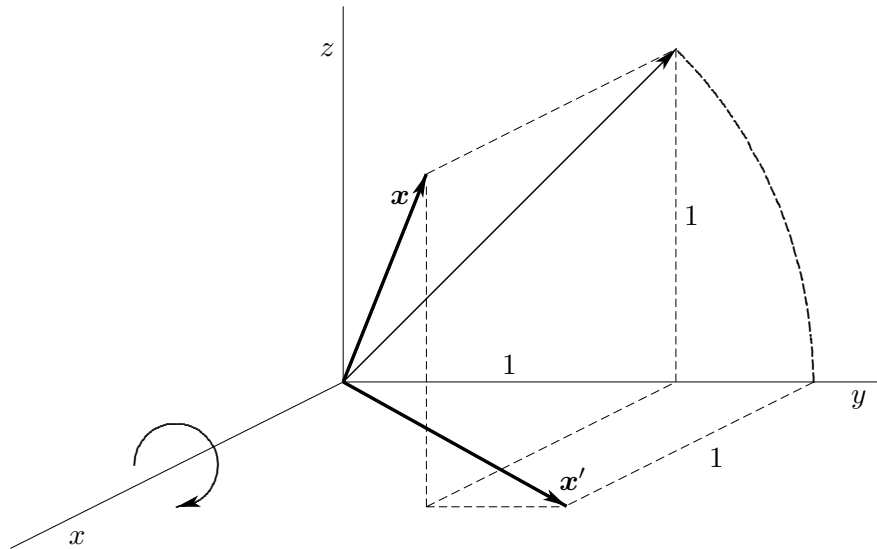


Figura 1.20

Ejemplo de una transformación de Givens en el espacio euclídeo tridimensional

Las transformaciones de Givens se pueden utilizar en un problema de mínimos cuadrados para transformar la matriz A en n etapas en una triangular superior R . En cada una de esas etapas, j , se han de hacer uno a uno cero los componentes $j+1$ a m .

Así, por ejemplo, las operaciones necesarias para transformar la matriz

$$A = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \quad \text{a} \quad R = \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \\ 0 & 0 & 0 \end{bmatrix},$$

son las siguientes:

$$\begin{aligned} A_1 &= \begin{bmatrix} \square & \square & \square \\ 0 & \square & \square \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} = G(1,2)A; & A_2 &= \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ 0 & \square & \square \\ 0 & \square & \square \\ \times & \times & \times \end{bmatrix} = G(1,3)A_1; \\ A_3 &= \begin{bmatrix} \nabla & \nabla & \nabla \\ 0 & \square & \square \\ 0 & \square & \square \\ 0 & \square & \square \end{bmatrix} = G(1,4)A_2; & A_4 &= \begin{bmatrix} \nabla & \nabla & \nabla \\ 0 & \blacksquare & \blacksquare \\ 0 & 0 & \blacksquare \\ 0 & \square & \square \end{bmatrix} = G(2,3)A_3; \\ A_5 &= \begin{bmatrix} \nabla & \nabla & \nabla \\ 0 & \nabla & \nabla \\ 0 & 0 & \blacksquare \\ 0 & 0 & \blacksquare \end{bmatrix} = G(2,4)A_4 & \text{y} & A_6 &= \begin{bmatrix} \nabla & \nabla & \nabla \\ 0 & \nabla & \nabla \\ 0 & 0 & \nabla \\ 0 & 0 & 0 \end{bmatrix} = G(3,4)A_5. \end{aligned}$$

Los símbolos $\square$, $\blacksquare$ y ∇ indican que el elemento al que se hace referencia ha experimentado 1, 2 ó 3 transformaciones desde su valor inicial $\times$.

El algoritmo completo para resolver $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ mediante transformaciones de Givens es el que representa la tabla 1.19. La versión en FORTRAN 77 de este algoritmo para resolver el ejemplo de la figura 1.8, de la página 74, es la que sigue a continuación.

```

PROGRAM Givens
C
parameter (m=4,n=3)
dimension a(m,n),b(m),x(n)
data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16./
data b/2.,3.,5.,6./
C
C *** Reducción QA=R ***
C
do i = 1,n
do k = i+1,m
if (1.0+abs(a(k,i)).ne.1.0) then
if (abs(a(k,i)).ge.abs(a(i,i))) then
t = a(i,i)/a(k,i)
s = 1.0/sqrt(1.0+t*t)
c = s*t
else
t = a(k,i)/a(i,i)
c = 1.0/sqrt(1.0+t*t)
s = c*t

```

Tabla 1.19Algoritmo para la resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$ mediante transformaciones de Givens

```
*
* TRANSFORMACIÓN DE LA MATRIZ  $A^{m \times n}$ 
*
for  $i = 1$  to  $n$ 
    for  $k = i + 1$  to  $m$ 
        ** HACER NULO EL ELEMENTO  $(k, i)$ .
        if  $a(k, i) \neq 0$  then
            if  $|a(k, i)| \geq |a(i, i)|$  then
                 $t = a(i, i)/a(k, i)$ ;  $s = 1/\sqrt{1+t^2}$ ;  $c = s \cdot t$ 
            else
                 $t = a(k, i)/a(i, i)$ ;  $c = 1/\sqrt{1+t^2}$ ;  $s = c \cdot t$ 
            end
             $a(i, i) \leftarrow c \cdot a(i, i) + s \cdot a(k, i)$ 
            for  $j = i + 1$  to  $n$ 
                 $aux = c \cdot a(i, j) + s \cdot a(k, j)$ 
                 $a(k, j) \leftarrow -s \cdot a(i, j) + c \cdot a(k, j)$ 
                 $a(i, j) \leftarrow aux$ 
            end
        ** TRANSFORMACIÓN DEL VECTOR  $b$ .
         $aux = c \cdot b(i) + s \cdot b(k)$ 
         $b(k) \leftarrow -s \cdot b(i) + c \cdot b(k)$ 
         $a(i) \leftarrow aux$ 
    end
end
*
* RESOLUCIÓN DEL SISTEMA  $Rx = b$ .
*
for  $j = n$  to  $1$ 
    
$$x(j) \leftarrow \left( b(j) - \sum_{k=j+1}^n a(j, k) \cdot x(k) \right) / a(j, j)$$

end
*
* RESIDUOS AL CUADRADO.
*

$$rescua \leftarrow \sum_{k=n+1}^m b^2(k)$$

```

```

endif
a(i,i) = c*a(i,i)+s*a(k,i)
do j = i+1,n
    q = c*a(i,j)+s*a(k,j)
    a(k,j) = (-s*a(i,j))+c*a(k,j)
    a(i,j) = q
end do
q = c*b(i)+s*b(k)
b(k) = (-s*b(i))+c*b(k)
b(i) = q
endif
end do
end do
C
C *** Resolución Rx = b ***
C
x(n) = b(n)/a(n,n)
do i = n-1,1,-1
    suma = 0.0
    do k = i+1,n
        suma = suma+a(i,k)*x(k)
    end do
    x(i) = (b(i)-suma)/a(i,i)
end do
C
C *** Suma de residuos al cuadrado
C
s = 0.0
do i = n+1,m
    s = s+b(i)*b(i)
end do
C
print 50,x
print 60,s
C
50 format(' X=(',f6.4,',',f6.4,',',f6.4,')''')
60 format(' Suma de residuos al cuadrado=',f9.6)
end

```

El número de operaciones que requiere este algoritmo es $O(2mn^2 - 2n^3/3)$ sumas/restas y multiplicaciones/divisiones y $O(mn/2)$ raíces cuadradas para transformar la matriz A , más $O(n^2/2)$ sumas/restas y multiplicaciones/divisiones para efectuar la sustitución inversa.

Comparando este último número de operaciones con las que requería el algoritmo basado en transformaciones de Householder, asumiendo que la precisión de los resultados es semejante, el método de Givens resulta ser el doble de *caro* que el de Householder. La pregunta que surge entonces es: ¿por qué utilizar Givens y no Householder?

La respuesta a esta pregunta se basa en considerar la estructura de la matriz A del problema: si ésta es densa, es decir, muchos de sus coeficientes son distintos de cero, el método de Householder es sin duda el más aconsejable; si, por el contrario, la estructura de A es dispersa, convendrá centrarse en hacer cero sólo aquellos elementos no nulos en las columnas correspondientes, por lo que, a priori, si hay pocos de éstos, el método de Givens deberá ser más ventajoso.

1.7.2.2.3 Transformaciones rápidas de Givens

Un método que palia la desventaja del de Givens respecto al de Householder, por lo que respecta a la velocidad de cálculo, y que incorpora la facilidad de hacer ceros determinados elementos, es el denominado de transformaciones rápidas de Givens. Se basa en reordenar los cálculos del algoritmo de la tabla 1.19 para conseguir una velocidad próxima a la del algoritmo de Householder.

Antes de entrar en la descripción de estas transformaciones, esbochemos la idea básica que las preside. Para ello, obsérvese una vez más que al aplicar a una matriz A una transformación de Givens, definida por una matriz $G(i, j)$, los únicos elementos que se ven modificados son los de las filas i y j , de tal forma que

$$\begin{aligned} a_{ik} &\leftarrow c \cdot a_{ik} + s \cdot a_{jk} \\ a_{jk} &\leftarrow -s \cdot a_{ik} + c \cdot a_{jk}, \end{aligned} \quad (1.23)$$

para $k = 1, \dots, n$. Estas modificaciones conllevan, para cada k , cuatro multiplicaciones ($4n$ en total). Ahora bien, si alguno de los c o s se pudiese reemplazar por un 1, el número de multiplicaciones se reduciría a la mitad. Este es el objetivo de las transformaciones rápidas de Givens.

Para resolver el problema de mínimos cuadrados con el método de las transformaciones de Givens, a la matriz $A^{m \times n}$ se le aplican un conjunto de tales transformaciones para convertirla, como se viene haciendo, en una matriz más simple de manipular: una matriz triangular superior R . Con las transformaciones rápidas de Givens, en lugar de premultiplicar la matriz A por unas $G(i, j)$, se multiplica por unas matrices más simples, resultado de premultiplicar a su vez esas $G(i, j)$ por unos escalares a fin de hacer uno el elemento s o c ; esos escalares se almacenan en unos vectores de tal forma que una vez que se haya conseguido transformar la matriz A a una triangular superior, se multiplica cada fila de ésta por su correspondiente escalar obteniéndose la matriz R . Más concretamente, el procedimiento basado en transformaciones rápidas de Givens almacena la matriz resultado de las transformaciones como producto DP , donde D es una matriz diagonal que contiene los escalares mencionados. En cada una de las n etapas se readaptan D y P . Al final del proceso, P es triangular superior y R resulta del producto DP .

Para estudiar el proceso detalladamente, supongamos que estamos en la etapa j y que se tiene que hacer cero el elemento i de la columna j ($i > j$) de DP . Dado que premultiplicar la matriz P por una matriz diagonal equivale a multiplicar cada fila de P por el correspondiente elemento en la diagonal, el elemento de la columna j en la fila i de la matriz DP que se quiere hacer cero será $d_i p_{ij}$, donde d_i es el elemento i -ésimo de D . Como los elementos c y s que definen la matriz $G(i, j)$ de una transformación de Givens, recordemos, están dados por

$$c = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}$$

y

$$s = \frac{x_j}{\sqrt{x_i^2 + x_j^2}},$$

para hacer cero $d_i p_{ij}$ con una transformación de este tipo c y s deberán ser:

$$c = \frac{d_j p_{jj}}{\sqrt{(d_j p_{jj})^2 + (d_i p_{ij})^2}} \quad \text{y} \quad s = \frac{d_i p_{ij}}{\sqrt{(d_j p_{jj})^2 + (d_i p_{ij})^2}}. \quad (1.24)$$

Definida la transformación de Givens, en lugar de calcular el producto $G(i, j) \cdot DP$, se calcula una nueva matriz D' y otra P' , de tal manera que

$$D'P' = G(i, j)DP.$$

A tal efecto, obsérvese que

$$G(i, j)DP = DD^{-1}G(i, j)DP = D \left(D^{-1}G(i, j)D \right) P.$$

Como se comprueba de forma inmediata, $D^{-1}G(i, j)D$ es idéntica a la matriz $G(i, j)$ excepto en el elemento que ocupa la fila i y la columna j y en el que ocupa la fila j y la columna i . En particular, la submatriz 2×2 de $D^{-1}G(i, j)D$ formada por la intersección de la fila i y la fila j con la columna i y la columna j es

$$\begin{bmatrix} c & s_1 \\ s_2 & c \end{bmatrix}, \quad (1.25)$$

donde $s_1 = sd_i/d_j$ y $s_2 = -sd_j/d_i$.

Si $|c| \geq 1/\sqrt{2}$, a cada fila de esta matriz se la multiplica por $1/c$ obteniéndose

$$\begin{bmatrix} 1 & \frac{s_1}{c} \\ \frac{s_2}{c} & 1 \end{bmatrix}; \quad (1.26)$$

si $|s| > 1/\sqrt{2}$, a la primera fila se la multiplica por $1/s_1$ y a la segunda por $1/s_2$, obteniéndose

$$\begin{bmatrix} \frac{c}{s_1} & 1 \\ 1 & \frac{c}{s_2} \end{bmatrix}. \quad (1.27)$$

En cualquiera de los dos casos se consigue hacer la unidad dos de los cuatro elementos, por lo que cualquier multiplicación que afecte a (1.26) o (1.27) requerirá la mitad de multiplicaciones que hacerlo con (1.25). Las matrices de la forma (1.26) y (1.27) representan *transformaciones rápidas de Givens* 2×2 .

Multiplicar por $1/c$, $1/s_1$ ó $1/s_2$ equivale a multiplicar la correspondiente matriz por una diagonal. Designemos como E la matriz diagonal cuyos elementos $e_1, e_2, \dots, e_n$ son todos iguales a uno excepto el i y el j que son: $e_i = 1/c = e_j$, si $|c| \geq 1/\sqrt{2}$; y $e_i = 1/s_1$ y $e_j = 1/s_2$, si $|s| > 1/\sqrt{2}$. Como $EE^{-1} = I$, se tendrá que

$$D'P' = DEE^{-1} \left(D^{-1}G(i, j)D \right) P = (DE) \left(E^{-1}D^{-1}G(i, j)D \right) P.$$

Como el producto de dos matrices diagonales es otra matriz diagonal, el producto DE es la nueva D . Por lo que respecta a la matriz $E^{-1}D^{-1}G(i, j)D$, la submatriz 2×2 formada por la intersección de la fila i y la fila j con la columna i y la columna j , es de la forma (1.26) o (1.27), por lo que el producto $(E^{-1}D^{-1}G(i, j)D)P$ es la nueva matriz P . En resumen:

$$D' = DE \quad \text{y} \quad P' = E^{-1}D^{-1}G(i, j)DP.$$

Tabla 1.20

Cálculo de los elementos de las filas i y j de las matrices D y P en las transformaciones rápidas de Givens

para $ c \geq 1/\sqrt{2}$	para $ s > 1/\sqrt{2}$
$r(1) \leftarrow (s \cdot d(i))/(c \cdot d(j))$ $r(2) \leftarrow (s \cdot d(j))/(c \cdot d(i))$ $d(j) \leftarrow c \cdot d(j); \quad d(i) \leftarrow c \cdot d(i)$ for $k = j$ to n $t \leftarrow p(j, k) + r(1) \cdot p(i, k)$ $p(i, k) \leftarrow p(i, k) - r(2) \cdot p(j, k)$ $p(j, k) \leftarrow t$ end	$r(1) \leftarrow (c \cdot d(j))/(s \cdot d(i))$ $r(2) \leftarrow (c \cdot d(i))/(s \cdot d(j))$ $d(i) \leftrightarrow d(j); \quad d(j) \leftarrow s \cdot d(j); \quad d(i) \leftarrow s \cdot d(i)$ for $k = j$ to n $t \leftarrow p(i, k) + r(1) \cdot p(j, k)$ $p(i, k) \leftarrow p(j, k) - r(2) \cdot p(i, k)$ $p(j, k) \leftarrow t$ end

De acuerdo con la definición de las matrices (1.26), (1.27) y E , los elementos de las filas i y j de las nuevas D y P se calculan como se describe en la tabla 1.20.

El proceso se inicia haciendo $D = I$ y $P = A$. El algoritmo completo para resolver el problema $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ mediante transformaciones rápidas de Givens es el que describe la tabla 1.21. Como se puede comprobar fácilmente, los parámetros r_1 y r_2 sólo dependen de d_i^2 y d_j^2 por lo que en el algoritmo se almacena el cuadrado de los elementos diagonales. Este algoritmo requiere $O(n^2(m - n/3))$ multiplicaciones/divisiones y sumas/restas para transformar la matriz del problema A y $O(n^2/2)$ multiplicaciones/divisiones y sumas/restas para realizar la sustitución inversa.

Cada una de las etapas del algoritmo de la tabla 1.21 multiplica dos componentes de $\mathbf{d}(\cdot)$ por un factor de magnitud c^2 o s^2 dependiendo cual es mayor. Dado que

$$\frac{1}{2} \leq \max\{c^2, s^2\} \leq 1,$$

al avanzar el algoritmo los componentes de $\mathbf{d}(\cdot)$ tienden a cero a medida que los elementos de la matriz A tienden a $\pm\infty$. Para evitar operaciones con *overflow* o *underflow*, tanto la matriz A como $\mathbf{d}(\cdot)$ se normalizan de vez en cuando. En particular, cuando haya peligro de *overflow* en la fila k , se puede llevar a cabo la siguiente normalización al comienzo del bucle que afecta a i :

$$a(k, l : n) \leftarrow d(k)^{1/2} \cdot a(k, l : n); \quad d(k) \leftarrow 1.$$

La versión en FORTRAN 77 de este algoritmo para resolver el ejemplo de la figura 1.8 es la que sigue a continuación.

```

PROGRAM Fastgivens
C
  parameter (m = 4, n = 3)
  dimension a(m,n), b(m), x(n), d(m)
C
  data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16./
  data b/2.,3.,5.,6./
C
C *** Reducción QA=R ***
C
```

Tabla 1.21

Algoritmo para la resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ por transformaciones rápidas de Givens

```

 $d(i : m) = 1$ 
*
* TRANSFORMACIÓN DE LA MATRIZ  $A^{m \times n}$ 
*
for  $j = 1$  to  $n$ 
  for  $i = j + 1$  to  $m$ 
    if  $a(i, j) \neq 0$  then
       $c = d(j) \cdot a(j, j)^2$ ;  $s = d(i) \cdot a(i, j)^2$ 
      if  $s \leq c$  then
         $r(2) \leftarrow a(i, j)/a(j, j)$ ;  $r(1) \leftarrow d(i) \cdot r(2)/d(j)$ ;  $c \leftarrow c/(c + s)$ 
         $d(j) \leftarrow c \cdot d(j)$ ;  $d(i) \leftarrow c \cdot d(i)$ 
        for  $k = j$  to  $n$ 
           $t \leftarrow a(j, k) + r(1) \cdot a(i, k)$ ;  $a(i, k) \leftarrow a(i, k) - r(2) \cdot a(j, k)$ ;  $a(j, k) \leftarrow t$ 
        end
         $t \leftarrow b(j) + r(1) \cdot b(i)$ ;  $b(i) \leftarrow b(i) - r(2) \cdot b(j)$ ;  $b(j) \leftarrow t$ 
      else
         $r(2) \leftarrow a(j, j)/a(i, j)$ ;  $r(1) \leftarrow d(j) \cdot r(2)/d(i)$ ;  $s \leftarrow s/(c + s)$ 
         $d(i) \leftrightarrow d(j)$ ;  $d(j) \leftarrow s \cdot d(j)$ ;  $d(i) \leftarrow s \cdot d(i)$ 
        for  $k = j$  to  $n$ 
           $t \leftarrow a(i, k) + r(1) \cdot a(j, k)$ ;  $a(j, k) \leftarrow a(j, k) - r(2) \cdot a(i, k)$ ;  $a(i, k) \leftarrow t$ 
        end
         $t \leftarrow b(i) + r(1) \cdot b(j)$ ;  $b(i) \leftarrow b(j) - r(2) \cdot b(i)$ ;  $b(j) \leftarrow t$ 
      end
    end
  end
end
for  $i = 1$  to  $m$ 
  for  $j = i$  to  $n$ 
     $a(i, j) \leftarrow d(i)^{1/2} \cdot a(i, j)$ 
  end
   $b(i) \leftarrow d(i)^{1/2} \cdot b(i)$ 
end
*
* RESOLUCIÓN DEL SISTEMA  $R\mathbf{x} = \mathbf{b}$ 
*
for  $j = n$  to  $1$ 
  
$$x(j) \leftarrow \left( b(j) - \sum_{k=j+1}^n a(j, k) \cdot x(k) \right) / a(j, j)$$

end

```



```
do i = 1,m
  d(i) = 1.0
end do
do j = 1,n
  do i = j+1,m
    if (1.0+abs(a(i,j)).ne.1.0) then
      c = d(j)*a(j,j)**2
      s = d(i)*a(i,j)**2
      if (s.le.c) then
        r2 = a(i,j)/a(j,j)
        r1 = d(i)*r2/d(j)
        c = c/(s+c)
        d(j) = c*d(j)
        d(i) = c*d(i)
        do k = j,n
          t = a(j,k)+r1*a(i,k)
          a(i,k) = a(i,k)-r2*a(j,k)
          a(j,k) = t
        end do
        t = b(j)+r1*b(i)
        b(i) = b(i)-r2*b(j)
        b(j) = t
      else
        r2 = a(j,j)/a(i,j)
        r1 = d(j)*r2/d(i)
        s = s/(s+c)
        t = d(j)
        d(j) = d(i)
        d(i) = t
        d(j) = s*d(j)
        d(i) = s*d(i)
        do k = j,n
          t = a(i,k)+r1*a(j,k)
          a(i,k) = a(j,k)-r2*a(i,k)
          a(j,k) = t
        end do
        t = b(i)+r1*b(j)
        b(i) = b(j)-r2*b(i)
        b(j) = t
      endif
    endif
  end do
end do
C
do i = 1,m
  sqrd = sqrt(d(i))
  do j = i,n
    a(i,j) = sqrd*a(i,j)
  end do
  b(i) = sqrd*b(i)
end do
C
C *** Resolución Rx = b ***
C
x(n) = b(n)/a(n,n)
do i = n-1,1,-1
  suma = 0.0
  do k = i+1,n
```

```

        suma = suma+a(i,k)*x(k)
    end do
    x(i) = (b(i)-suma)/a(i,i)
end do
C
C *** Suma de residuos al cuadrado ***
C
    s = 0.0
    do i = n+1,m
        s = s+b(i)*b(i)
    end do
C
    print 50,x
    print 60,s
C
50 format(' x=(',f6.4,',',f6.4,',',f6.4,')''')
60 format(' Suma de residuos al cuadrado=',f9.6)
C
end

```

1.7.3 Descomposición numérica en valores singulares. Método de Golub-Reinsch

Como ya indicábamos en el apartado teórico dedicado a la descomposición en valores singulares, ésta tiene gran importancia para analizar un problema lineal de mínimos cuadrados e incluso para resolverlo. La evaluación numérica de la descomposición en valores singulares de la matriz A que caracteriza un problema de mínimos cuadrados puede resultar muy útil. En este apartado nos centraremos en el algoritmo de Golub y Reinsch [1970] para llevarla a cabo.

Teorema 1.19 (*Descomposición Bidiagonal de una matriz*) Sea la matriz $A \in \mathbb{R}^{m \times n}$, $m \geq n$. Existen matrices ortogonales $Q \in \mathbb{R}^{m \times m}$ y $P \in \mathbb{R}^{n \times n}$ tales que

$$Q^T A P = \begin{bmatrix} B_1 \\ 0 \end{bmatrix}, \quad (1.28)$$

donde B_1 es una matriz triangular superior bidiagonal en la que todos los elementos de la diagonal principal son positivos o cero (no negativos).

DEMOSTRACIÓN. Procederemos por inducción. Para $m = 1$, se puede hacer $Q = \pm 1$ y $P = 1$.

Para $m > 1$, sea $A = [\mathbf{a}_1, A_2]$, con $\mathbf{a}_1 \in \mathbb{R}^m$, y $U = [\mathbf{y}, U_1]$ una matriz ortogonal tal que:

$$\mathbf{y} = \begin{cases} \mathbf{a}_1 / \|\mathbf{a}_1\|_2 & \text{si } \mathbf{a}_1 \neq \mathbf{0} \\ \mathbf{e}_1 & \text{si } \mathbf{a}_1 = \mathbf{0}. \end{cases}$$

Como $U_1^T \mathbf{y} = \mathbf{0}$, entonces

$$U^T A = \begin{bmatrix} \rho & \mathbf{r}^T \\ 0 & B \end{bmatrix},$$

donde $\rho = \|\mathbf{a}_1\|_2 > 0$, $\mathbf{r} = A_2^T \mathbf{y}$ y $B = U_1^T A_2 \in \mathbb{R}^{(m-1) \times (n-1)}$.

Sea ahora la matriz ortogonal $\bar{V} = [\mathbf{z}, V_1]$ con:

$$\mathbf{z} = \begin{cases} \mathbf{r}/\|\mathbf{r}\|_2 & \text{si } \mathbf{r} \neq \mathbf{0} \\ \mathbf{e}_1 & \text{si } \mathbf{r} = \mathbf{0}. \end{cases}$$

Como $\mathbf{r}^T V_1 = 0$, entonces

$$U^T A V = \begin{bmatrix} \rho & \mathbf{c}^T \\ 0 & C \end{bmatrix},$$

donde $\mathbf{c}^T = [\sigma, \mathbf{0}^T]$, $V = \text{diag}(1, \bar{V})$, $\sigma = \|\mathbf{r}\|_2$ y $C = B\bar{V} \in \Re^{(m-1) \times (n-1)}$.

Por la hipótesis de inducción, existirán matrices ortogonales $\bar{Q}$ y $\bar{P}$ que reduzcan la matriz C a una bidiagonal. La igualdad (1.28) se cumplirá si se escogen

$$Q = U \begin{bmatrix} 1 & 0 \\ 0 & \bar{Q} \end{bmatrix} \quad \text{y} \quad P = V \begin{bmatrix} 1 & 0 \\ 0 & \bar{P} \end{bmatrix}.$$

■

Algoritmo de Golub y Reinsch. Primera fase

Consiste en reducir la matriz A a una triangular superior bidiagonal mediante transformaciones ortogonales de Householder de acuerdo con el esquema usado para demostrar el teorema anterior. Es decir, hacer

$$Q_B^T A \Pi_B = B = \begin{bmatrix} B_1 \\ 0 \end{bmatrix},$$

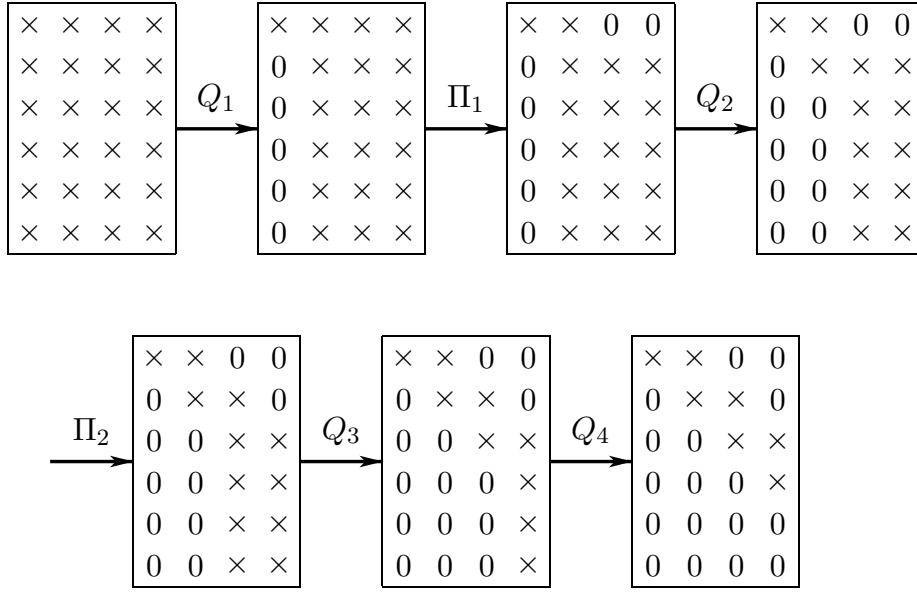
donde

$$B = \begin{bmatrix} d_1 & f_2 & & & \\ & d_2 & f_3 & & \\ & & \ddots & \ddots & \\ & & & \ddots & f_n \\ & & & & d_n \\ \hline & & & & & 0 \end{bmatrix}, \quad (1.29)$$

$Q_B = Q_1 \cdots Q_n \in \Re^{m \times m}$ y $\Pi_B = \Pi_1 \cdots \Pi_{n-2} \in \Re^{n \times n}$. Estas matrices Q_B y Π_B son el producto de matrices elementales que caracterizan las mencionadas transformaciones de Householder. La transformación que define Q_k se escoge de tal manera que haga cero los elementos $k+1$ a m de la columna k . La transformación que define Π_k , por otro lado, hará cero los elementos $k+2$ a n de la fila k . El esquema que se seguiría con una matriz $A^{6 \times 4}$ será el de la figura 1.21.

Un algoritmo que implemente esta forma de actuación requerirá $O(2mn^2 - \frac{2}{3}n^3)$ operaciones. La forma de proceder, por lo que respecta al almacenamiento de datos, es similar a la que se viene utilizando.

Un procedimiento para bidiagonalizar la matriz A más rápido que el propuesto por Golub y Reinsch, cuando $m \gg n$, es el debido a Hanson y Lawson [1974] y [1995], implementado por

**Figura 1.21**

Proceso de bidiagonalización de una matriz 6×4 mediante transformaciones de Householder

Chan [1982]. Se basa en reducir inicialmente A de la forma

$$Q_1^T A = \begin{bmatrix} R \\ 0 \end{bmatrix},$$

donde $R \in \mathfrak{R}^{n \times n}$ es una matriz triangular superior, y luego bidiagonalizar R . Es decir,

$$Q_2^T R \Pi_B = B_1,$$

donde $Q_2, \Pi_B \in \mathfrak{R}^{n \times n}$ son dos matrices ortogonales y $B_1 \in \mathfrak{R}^{n \times n}$ es triangular superior bidiagonal. Si se define la matriz $U_B = Q_1 \text{diag}(Q_2, I_{m-n})$, se verifica que

$$U_B^T A \Pi_B = \begin{bmatrix} B_1 \\ 0 \end{bmatrix} = B.$$

Este procedimiento requiere $\boxed{O(mn^2 + n^3)}$ operaciones. Cuando $m \geq \frac{5}{3}n$, es más rápido que el de Golub y Reinsch.

Segunda fase

Una vez bidiagonalizada la matriz A , en una segunda fase se hacen cero los elementos que no están en la diagonal principal mediante un algoritmo que obtenga

$$Q_S^T B_1 \Pi_S = \Sigma = \text{diag}(\sigma_1, \dots, \sigma_n),$$

donde $Q_S \in \mathbb{R}^{n \times n}$ y $\Pi_S \in \mathbb{R}^{n \times n}$ son matrices ortogonales. La descomposición en valores singulares de la matriz A será

$$A = U \begin{bmatrix} \Sigma \\ 0 \end{bmatrix} V^T,$$

donde $U = Q_B \text{diag}(Q_S, I_{m-n})$ y $V = \Pi_B \Pi_S$.

Antes de seguir con la explicación de esta segunda fase, demostremos un resultado muy útil para lo que expondremos.

Teorema 1.20 (*Teorema de la Q implícita*) Sean dos matrices ortogonales $Q = [\mathbf{q}_1, \dots, \mathbf{q}_n]$ y $V = [\mathbf{v}_1, \dots, \mathbf{v}_n]$ con la propiedad de que $H = Q^T A Q$ y $G = V^T A V$ son matrices de Hessenberg. Sea k el menor número entero positivo tal que $h_{k+1,k} = 0$ (se adopta el convenio de que $k = n$ si todos los elementos de la subdiagonal de H son distintos de cero). Si $\mathbf{v}_1 = \mathbf{q}_1$, entonces $\mathbf{v}_i = \pm \mathbf{q}_i$ y $|h_{i,i-1}| = |g_{i,i-1}|$, para $i = 2, \dots, k$. Además, si $k < n$, $g_{k+1,k} = 0$.

DEMOSTRACIÓN. Definamos la matriz ortogonal $W = [\mathbf{w}_1, \dots, \mathbf{w}_n] = V^T Q$; obsérvese que $GW = WH$. Para $i = 2, \dots, k$ se tendrá que

$$h_{i,i-1} \mathbf{w}_i = G \mathbf{w}_{i-1} - \sum_{j=1}^{i-1} h_{j,i-1} \mathbf{w}_j.$$

Como $\mathbf{w}_1 = \mathbf{e}_1$, $[\mathbf{w}_1, \dots, \mathbf{w}_k]$ es una matriz triangular superior y, por lo tanto, $\mathbf{w}_i = \pm \mathbf{e}_i$ para $i = 2, \dots, k$. Como $\mathbf{w}_i = V^T \mathbf{q}_i$ y $h_{i,i-1} = \mathbf{w}_i^T G \mathbf{w}_{i-1}$, entonces $\mathbf{v}_i = \pm \mathbf{q}_i$ y $|h_{i,i-1}| = |g_{i,i-1}|$ para $i = 2, \dots, k$. Si $h_{k+1,k} = 0$, ignorando los signos, se tiene que

$$\begin{aligned} g_{k+1,k} &= \mathbf{e}_{k+1}^T G \mathbf{e}_k = \mathbf{e}_{k+1}^T G W \mathbf{e}_k = (\mathbf{e}_{k+1}^T W)(H \mathbf{e}_k) \\ &= \mathbf{e}_{k+1}^T \sum_{i=1}^k h_{ik} W \mathbf{e}_i = \sum_{i=1}^k h_{ik} \mathbf{e}_{k+1}^T \mathbf{e}_i = 0. \end{aligned}$$

■

La idea esencial del algoritmo de la Q implícita es que si $Q^T A Q = H$ y $Z^T A Z = G$ son matrices de Hessenberg sin elementos nulos en la subdiagonal debajo de la diagonal principal y Q y Z tienen todos los elementos de la primera columna iguales, G y H son esencialmente iguales en el sentido de que $G = D^{-1} H D$, donde $D = \text{diag}(\pm 1, \dots, \pm 1)$. Veremos más adelante la utilidad de este resultado.

El algoritmo para obtener los valores singulares de la matriz bidiagonal B_1 procede iterativamente haciendo

$$B_{k+1} = U_k^T B_k V_k, \quad k = 1, 2, \dots,$$

donde U_k y V_k son matrices ortogonales, de tal forma que

$$\Sigma = \lim_{k \rightarrow \infty} B_k.$$

Para llegar a obtener los valores singulares de B se podría proceder de forma análoga a como lo hace el denominado *Algoritmo QR con Desplazamiento Implícito* —ver el apartado *referencias* de este mismo capítulo—, para obtener los valores propios de una matriz simétrica, y

manipular la matriz $B^T B$ pues, recordemos, los valores singulares de B son las raíces cuadradas positivas de los valores propios de $B^T B$. Ahora bien, debido a los errores numéricos de redondeo que se pueden producir en los valores singulares menores al actuar sobre $B^T B$ y no sobre B y a que el método sólo obtendría los vectores propios de $B^T B$, es decir, sólo los vectores singulares derechos de B , no es aconsejable seguir ese procedimiento aunque sí su principio de actuación y las propiedades de su convergencia.

En cada etapa k del proceso de obtención de los valores singulares de la matriz bidiagonal B , siguiendo la idea del método QR con desplazamiento implícito en el sentido de hacer lo más pequeños posibles los elementos que no están en la diagonal principal de B_k , se determinan los valores propios λ_1 y λ_2 de la submatriz 2×2 que forman los elementos $(n-1, n-1)$, $(n-1, n)$, $(n, n-1)$ y (n, n) de $B_k^T B_k$. Siguiendo la notación de (1.29), esa submatriz es

$$\begin{bmatrix} d_{n-1}^2 + f_{n-1}^2 & d_{n-1}f_n \\ d_{n-1}f_n & d_n^2 + f_n^2 \end{bmatrix}.$$

Posteriormente se obtiene un σ_k cuyo valor es el del valor propio más próximo al valor del elemento (n, n) de $B_k^T B_k$. Este parámetro⁵ es igual a

$$d_n^2 + f_n \left(f_n - \frac{d_{n-1}}{t} \right),$$

donde

$$t = \begin{cases} -f - (1 + f^2)^{1/2} & \text{si } f \geq 0 \\ -f + (1 + f^2)^{1/2} & \text{si } f < 0 \end{cases}$$

y

$$f = \frac{d_n^2 - d_{n-1}^2 + f_n^2 - f_{n-1}^2}{2f_n d_{n-1}}.$$

A continuación se determina una transformación de Givens, $G(1, 2)$, que haga el elemento $(2, 1)$ de $B_k^T B_k - \sigma_k I_n$ cero; es decir, tal que

$$G(1, 2) \begin{bmatrix} d_1^2 - \sigma_k \\ d_1 f_2 \end{bmatrix} = \begin{bmatrix} \times \\ 0 \end{bmatrix}.$$

Esta transformación de Givens se aplica posteriormente a B_k creándose en ésta un elemento no nulo en la posición $(2, 1)$. Para anularlo se le aplica otra transformación de Givens, simbolizada por U_1 , la cual a su vez crea otro elemento no nulo en la posición $(1, 3)$ de B_k . Para anularlo se aplica otra transformación de Givens, V_2 , la cual a su vez crea otro elemento distinto de cero ... Así se procede sucesivamente hasta restaurar la condición inicial de B_k de ser bidiagonal.

Si, por ejemplo, se parte de

$$B_k G(1, 2) = B'_k = \begin{bmatrix} \times & \times & & & \\ + & \times & \times & & \\ & & \times & \times & \\ & & & \times & \times \\ & & & & \times & \times \\ & & & & & \times \end{bmatrix},$$

⁵Ver su deducción en Lawson y Hanson [1974] y [1995].

los sucesivos pasos de esta etapa k para conseguir que B_k siga siendo bidiagonal, serían:

$$\begin{aligned}
 U_1^T B'_k &= \begin{bmatrix} \times & \times & + & & & \\ & \times & \times & & & \\ & & \times & \times & & \\ & & & \times & \times & \\ & & & & \times & \times \\ & & & & & \times \end{bmatrix}; & U_1^T B'_k V_2 &= \begin{bmatrix} \times & \times & & & & \\ & \times & \times & & & \\ & + & \times & \times & & \\ & & & \times & \times & \\ & & & & \times & \times \\ & & & & & \times \end{bmatrix}; \\
 U_2^T U_1^T B'_k V_2 &= \begin{bmatrix} \times & \times & & & & \\ & \times & \times & + & & \\ & & \times & \times & & \\ & & & \times & \times & \\ & & & & \times & \times \\ & & & & & \times \end{bmatrix}; & U_2^T U_1^T B'_k V_2 V_3 &= \begin{bmatrix} \times & \times & & & & \\ & \times & \times & & & \\ & & \times & \times & & \\ & & & \times & \times & \\ & & & + & \times & \times \\ & & & & \times & \times \\ & & & & & \times \end{bmatrix};
 \end{aligned}$$

y así sucesivamente hasta conseguir que

$$U_5^T U_4^T U_3^T U_2^T U_1^T B_k G(1, 2) V_2 V_3 V_4 V_5$$

sea bidiagonal.

En general, en cada etapa k , designando la transformación de Givens inicial $G(1, 2)$ mediante G , se obtendrá una matriz $\bar{B}_k$ relacionada con la inicial de la etapa, B_k , mediante la expresión

$$\bar{B}_k = (U_{n-1}^T \cdots U_1^T) B_k (G V_2 \cdots V_{n-1}) = \bar{U}^T B_k \bar{V}.$$

Como cada transformación V_i es del tipo $G(i, i+1)$, donde $i = 2, \dots, n-1$, se puede comprobar fácilmente que $V e_1 = Q e_1$, donde Q representa el conjunto de transformaciones de Givens que se aplican en una etapa k del algoritmo QR con desplazamiento implícito para restaurar la condición de matriz tridiagonal en $B_k^T B_k$ una vez aplicado el desplazamiento σ_k antes descrito. Es decir, tal que $Q^T (B_k^T B_k - \sigma_k I_n) Q$ sea tridiagonal. De acuerdo con el teorema de la Q implícita se puede concluir que las matrices V y Q son esencialmente iguales por lo que el efecto, con vistas a conseguir los valores singulares de B , de una etapa del algoritmo QR con desplazamiento implícito sobre $B^T B$ es el mismo manipulando solamente B .

En la tabla 1.22 se describen los pasos de cada una de estas etapas k que acabamos de describir. El algoritmo a que da lugar —denominado de Golub-Kahan—, dada una matriz bidiagonal $B \in \mathbb{R}^{m \times n}$ —obsérvese que hemos suprimido el subíndice k para especificar la etapa a la que hacíamos referencia anteriormente— sin ningún elemento nulo ni en la diagonal principal ni en la sobrediagonal inmediata a esa diagonal principal, calcula una $\bar{B}$, que reemplaza a B , tal que $\bar{B} = \bar{U}^T B \bar{V}$, donde $\bar{U}$ y $\bar{V}$ son matrices ortogonales siendo $\bar{V}$ esencialmente la que se obtendría al aplicar el algoritmo QR con desplazamiento implícito a $T = B^T B$ para calcular sus autovalores. La convergencia del método QR con desplazamiento implícito garantiza que el valor de los elementos f_i de la matriz B convergerán rápidamente a cero.

Todo lo expuesto hasta ahora para obtener los valores singulares de la matriz bidiagonal B en etapas sucesivas presupone que todos los elementos de la diagonal principal y de la sobrediagonal más próxima a esa diagonal principal son distintos de cero. Si algún $f_{k+1} = 0$, entonces

$$B = \begin{bmatrix} B_1 & 0 \\ 0 & B_2 \end{bmatrix} \begin{matrix} k \\ n - k \end{matrix}$$

Tabla 1.22

Algoritmo de Golub-Kahan: etapa k del procedimiento de Golub-Reinsch para obtener los valores singulares de una matriz bidiagonal $B^{n \times n}$

Paso 1 – Determinar el autovalor σ de la submatriz 2×2 de $T = B^T B$ que forman $t_{n-1\ n-1}$, $t_{n-1\ n}$, $t_{n\ n-1}$ y t_{nn} más próximo en valor a t_{nn} .
Hacer $y = t_{11} - \sigma$
 $z = t_{12}$.

Paso 2 – Para $k = 1, \dots, n-1$:
Determinar los parámetros de $G(k, k+1)$, $c = \cos \theta$ y $s = \sin \theta$, tales que

$$\begin{bmatrix} y & z \end{bmatrix} \begin{bmatrix} c & s \\ -s & c \end{bmatrix} = \begin{bmatrix} \times & 0 \end{bmatrix}.$$

Hacer $B = BG(k, k+1)$
 $y = b_{kk}$
 $z = b_{k+1\ k}$.

Determinar los parámetros de $G(k, k+1)$, $c = \cos \theta$ y $s = \sin \theta$, tales que

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix}^T \begin{bmatrix} y \\ z \end{bmatrix} = \begin{bmatrix} \times \\ 0 \end{bmatrix}.$$

Hacer $B = G(k, k+1)^T B$.
Si $k < n-1$, hacer $y = b_{k\ k+1}$ y $z = b_{k\ k+2}$.

y el procedimiento para calcular los valores singulares de B se podría descomponer en dos subproblemas: uno para calcular los de B_1 y otro para hacerlo de los de B_2 .

Si algún d_k fuese 0, la matriz se podría premultiplicar por una transformación de Givens a fin de hacer cero el correspondiente f_{k+1} próximo, buscando, como antes, subdividir el problema en subproblemas. Si, en este sentido, por ejemplo, se supone

$$B = \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & \times & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix},$$

(en este caso $n = 6$ y $k = 3$), construyendo unas transformaciones de Givens en los planos (3,4), (3,5) y (3,6), se producirá el siguiente efecto:

$$\xrightarrow{G(3,4)} \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \times & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{G(3,5)} \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \times \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix} \xrightarrow{G(3,6)} \begin{bmatrix} \times & \times & 0 & 0 & 0 & 0 \\ 0 & \times & \times & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \times & \times & 0 \\ 0 & 0 & 0 & 0 & \times & \times \\ 0 & 0 & 0 & 0 & 0 & \times \end{bmatrix}.$$

El esquema completo del algoritmo que hemos descrito para obtener numéricamente los valores singulares de una matriz A es el que describe la tabla 1.23. Esta versión parte de una matriz $A \in \mathbb{R}^{m \times n}$ ($m \geq n$). La matriz A se reemplaza por $U(D + E)V^T$, donde $U \in \mathbb{R}^{m \times n}$, $V \in \mathbb{R}^{n \times n}$, $D \in \mathbb{R}^{m \times n}$ y E cumple que $\|E\|_2 \approx \epsilon \|A\|_2$. En la memoria de la máquina donde se trabaje U reemplazará a A .

A continuación se lista un programa en FORTRAN 77 que codifica todos los pasos de este algoritmo tal cual se describen en la tabla 1.23.

```

      subroutine dcmsvd (a,m,n,sv,v)
      C
      dimension a(m,n),sv(n),v(m,n),rv1(20)
      C
      g      = 0.0
      anorm = 0.0
      C
      C *** Reducir matriz A a matriz bidiagonal.
      C
      do i = 1,n
         l      = i+1
         rv1(i) = g

```

Tabla 1.23

Algoritmo de Golub-Reinsch para la obtención de los valores singulares de una matriz $A \in \mathbb{R}^{m \times n}$

Paso 1 – Bidiagonalizar la matriz A mediante transformaciones de Householder. Hacer

$$B \leftarrow (U_1 \cdots U_n)^T A (V_1 \cdots V_{n-2}).$$

Paso 2 – Realizar las siguientes operaciones:

- Hacer $a_{ii+1} = 0$ si $|a_{ii+1}| \leq \epsilon(|a_{ii}| + |a_{i+1,i+1}|)$ para todo $i = 1, \dots, n-1$.
- Determinar el mayor q y el menor p tales que si

$$A = \begin{bmatrix} B_{11} & 0 & 0 \\ 0 & B_{22} & 0 \\ 0 & 0 & B_{33} \end{bmatrix} \begin{matrix} p \\ n-p-q \\ q \end{matrix},$$

B_{33} es diagonal y B_{22} tiene todos sus elementos de la sobrediagonal próxima a la diagonal principal distintos de cero.

- Si $q = n$, pararse; el procedimiento ha concluido.
- Si cualquier elemento en la diagonal de B_{22} es cero, anular el elemento en la sobrediagonal de las misma fila i y comenzar de nuevo en a).
- Aplicar el algoritmo de la tabla 1.22 a B_{22} . Hacer

$$B = \text{diag}(I_p, \bar{U}, I_{q+m-n})^T B \text{diag}(I_p, \bar{V}, I_q).$$

Comenzar de nuevo en a).

```

g      = 0.0
s      = 0.0
if (i.le.m) then
  rmax = 0.0
  do k = i,m
    rmax = amax1(rmax,abs(a(k,i)))
  end do
  if (rmax.ne.0.0) then
    do k = i,m
      s = s+a(k,i)**2
    end do
    f      = a(i,i)
    g      = -sign(sqrt(s),f)
    h      = f*g-s
    a(i,i) = f-g
    do j = 1,n
      s = 0.0
      do k = i,m
        s = s+a(k,i)*a(k,j)
      end do
      f = s/h
      do k = i,m
        a(k,j) = a(k,j)+f*a(k,i)
      end do
    end do
  endif
endif
sv(i) = g
g      = 0.0
s      = 0.0
if (i.le.m.and.i.ne.n) then
  rmax = 0.0
  do k = 1,n
    rmax = amax1(rmax,abs(a(i,k)))
  end do
  if (rmax.ne.0.0) then
    do k = 1,n
      s = s+a(i,k)**2
    end do
    f      = a(i,1)
    g      = -sign(sqrt(s),f)
    h      = f*g-s
    a(i,1) = f-g
    do k = 1,n
      rv1(k) = a(i,k)/h
    end do
    do j = 1,m
      s = 0.0
      do k = 1,n
        s = s+a(j,k)*a(i,k)
      end do
      do k = 1,n
        a(j,k) = a(j,k)+s*rv1(k)
      end do
    end do
  endif
endif
anorm = amax1(anorm,abs(sv(i))+abs(rv1(i)))
end do

```

```
C
C *** Acumular en la matriz V las transformaciones
C     por la derecha hechas a A. ***
C
  do i = n,1,-1
    if (i.lt.n) then
      if (g.ne.0.0) then
        do j = 1,n
          v(j,i) = (a(i,j)/a(i,1))/g
        end do
        do j = 1,n
          s = 0.0
          do k = 1,n
            s = s+a(i,k)*v(k,j)
          end do
          do k = 1,n
            v(k,j) = v(k,j)+s*v(k,i)
          end do
        end do
      endif
      do j = 1,n
        v(i,j) = 0.0
        v(j,i) = 0.0
      end do
    endif
    v(i,i) = 1.0
    g = rv1(i)
    l = i
  end do

C
C *** Acumular en la matriz A las transformaciones
C     por la izquierda hechas a A.
C
  do i = min(m,n),1,-1
    l = i+1
    g = sv(i)
    do j = l,n
      a(i,j) = 0.0
    end do
    if (g.ne.0.0) then
      g = 1.0/g
      do j = l,n
        s = 0.0
        do k = l,m
          s = s+a(k,i)*a(k,j)
        end do
        f = (s/a(i,i))*g
        do k = i,m
          a(k,j) = a(k,j)+f*a(k,i)
        end do
      end do
      do j = i,m
        a(j,i) = a(j,i)*g
      end do
    else
      do j = i,m
        a(j,i) = 0.0
      end do
    endif
  end do
```

```

        a(i,i) = a(i,i)+1.0
    end do
C
C *** Diagonalizar la matriz bidiagonal almacenada en sv(.) y en
C     rv1(.). Sólo se realizan 30 iteraciones como máximo.
C
    do k = n,1,-1
        do its = 1,30
            do l = k,1,-1
                nm = l-1
                if (abs(rv1(l))+anorm.eq.anorm) exit
                if (abs(sv(nm))+anorm.eq.anorm) then
                    c = 0.0
                    s = 1.0
                    do i = l,k
                        f = s*rv1(i)
                        rv1(i) = c*rv1(i)
                        if (abs(f)+anorm.eq.anorm) exit
                        g = sv(i)
                        h = sqrt(f*f+g*g)
                        sv(i) = h
                        c = g/h
                        s = -f/h
                        do j = 1,m
                            y = a(j,nm)
                            z = a(j,i)
                            a(j,nm) = y*c+z*s
                            a(j,i) = (-y*s)+z*c
                        end do
                    end do
                    exit
                endif
            end do
            z = sv(k)
            if (l.eq.k) then
                if (z.lt.0.0) then
                    sv(k) = -z
                    do j = 1,n
                        v(j,k) = -v(j,k)
                    end do
                endif
            endif
            exit
        endif
        if (its.eq.30) stop 'No hay convergencia'
        x = sv(l)
        nm = k-1
        y = sv(nm)
        g = rv1(nm)
        h = rv1(k)
        f = ((y-z)*(y+z)+(g-h)*(g+h))/(2.0*h*y)
        g = sqrt(f*f+1.0)
        f = ((x-z)*(x+z)+h*(y/(f+sign(g,f))-h))/x
        c = 1.0
        s = 1.0
        do j = 1,nm
            i = j+1
            g = rv1(i)
            y = sv(i)
            h = s*g

```

```

g      = c*g
z      = sqrt(f*f+h*h)
rv1(j) = z
c      = f/z
s      = h/z
f      = x*c+g*s
g      = (-x*s)+g*c
h      = y*s
y      = y*c
do jj = 1,n
  x      = v(jj,j)
  z      = v(jj,i)
  v(jj,j) = x*c+z*s
  v(jj,i) = (-x*s)+z*c
end do
z      = sqrt(f*f+h*h)
sv(j) = z
if (z.ne.0.0) then
  c = f/z
  s = h/z
endif
f = c*g+s*y
x = (-s*g)+c*y
do jj = 1,m
  y      = a(jj,j)
  z      = a(jj,i)
  a(jj,j) = y*c+z*s
  a(jj,i) = (-y*s)+z*c
end do
end do
rv1(1) = 0.0
rv1(k) = f
sv(k) = x
end do
end do
return
end

```

El número de operaciones que requiere el algoritmo de la tabla 1.23 depende de la cantidad de información que se pretenda obtener de la descomposición en valores singulares

$$A = U\Sigma V^T$$

de la matriz A . Si lo que se pretende es simplemente resolver un problema de mínimos cuadrados $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$, cuya solución es

$$\mathbf{x} = V \begin{bmatrix} \Sigma_r^{-1} & 0 \\ 0 & 0 \end{bmatrix} U^T \mathbf{b},$$

no será necesario calcular la matriz U^T sino simplemente aplicar las transformaciones que la originan a $\mathbf{b}$. En otras ocasiones sólo interesará calcular $U_1^{m \times n}$, donde

$$U = \begin{bmatrix} U_1 & U_2 \\ n & m - n \end{bmatrix}.$$

En cualquier caso, existen seis posibilidades distintas de obtener información relativa a una descomposición en valores singulares (SVD). El trabajo requerido para llevar a cabo éstas según

se utilice el método de Golub-Reinsch completo o su variante de Chan, es el que se indica en la tabla 1.24.

Para la resolución del problema de mínimos cuadrados hay que tener cuidado con los errores de redondeo inherentes al proceso, sobre todo si los valores singulares son muy pequeños. Si se elige un parámetro cualquiera $\delta = \epsilon \|A\|_\infty$, por ejemplo, donde como siempre ϵ es la precisión de la máquina donde se trabaja, y los valores singulares verifican que

$$\sigma_1 \geq \cdots \geq \sigma_r > \delta \geq \sigma_{r+1} \geq \cdots \geq \sigma_n,$$

la solución del problema de mínimos cuadrados con norma euclídea mínima será

$$\mathbf{x} = \sum_{i=1}^r \frac{\mathbf{u}_i^T \mathbf{b}}{\sigma_i} \mathbf{v}_i. \quad (1.30)$$

Los componentes $r + 1$ a n serán cero.

A continuación se lista un programa en FORTRAN 77 para resolver el problema de mínimos cuadrados $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$, en concreto,

$$\underbrace{\begin{bmatrix} 1 & 6 & 11 \\ 2 & 7 & 12 \\ 3 & 8 & 13 \\ 4 & 9 & 14 \\ 5 & 10 & 15 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 5 \\ 5 \\ 5 \\ 5 \\ 5 \end{bmatrix}}_{\mathbf{b}},$$

utilizando la rutina DCMSVD presentada antes. La solución del problema es

$$\mathbf{x} = \begin{bmatrix} -0,5 \\ 0,0 \\ 0,5 \end{bmatrix}.$$

Los valores singulares de la matriz A son $\sigma_1 = 35,12723$, $\sigma_2 = 2,465397$ y $\sigma_3 = 0$. El programa también proporciona el rango de la matriz A .

Tabla 1.24

Número de operaciones necesarias para efectuar las distintas variantes de una descomposición en valores singulares de una matriz $A \in \mathbb{R}^{m \times n}$

Obtener	Método de Golub-Reinsch	Método de Golub-Reinsch-Chan
Σ	$2mn^2 - \frac{2}{3}n^3$	$mn^2 + n^3$
Σ, V	$2mn^2 + 4n^3$	$mn^2 + \frac{17}{3}n^3$
Σ, U	$2m^2n + 4mn^2$	$2m^2n + \frac{19}{3}n^3$
Σ, U_1	$7mn^2 - n^3$	$3mn^2 + \frac{16}{3}n^3$
Σ, U, V	$2m^2n + 4mn^2 + \frac{14}{3}n^3$	$2m^2n + 11n^3$
Σ, U_1, V	$7mn^2 + \frac{11}{3}n^3$	$3mn^2 + 10n^3$

```
PROGRAM Svdre
C
parameter (m=5,n=3)
dimension a(m,n),sv(n),v(m,n),b(m),x(n),tmp(m)
C
data a/1.,2.,3.,4.,5.,6.,7.,8.,9.,10.,11.,12.,13.,14.,15./
data b/5.,5.,5.,5.,5./
C
call dcmsvd (a,m,n,sv,v)
C
ns = 0
sm = 0.0
do i = 1,n
    sm = amax1(sm,sv(i))
end do
sp = sm*1.0e-6
do j = 1,n
    s = 0.0
    if (sv(j).gt.sp) then
        ns = ns+1
        do i = 1,m
            s = s+a(i,j)*b(i)
        end do
        s = s/sv(j)
    else
        sv(j) = 0.0
    endif
    tmp(j) = s
end do
do j = 1,n
    s = 0.0
    do jj = 1,n
        s = s+v(j,jj)*tmp(jj)
    end do
    x(j) = s
end do
C
print '( Rango de A:',i3)',ns
print '( Solución:',3(f11.7:',','))',x
print '( Valores singulares de A:',3(f11.7:',','))',sv
C
end
```

Para finalizar este apartado, a modo de resumen, en la tabla 1.25, se comparan, por lo que respecta al número de operaciones involucradas en sus procesos, todos los métodos para resolver el problema de mínimos cuadrados que hemos presentado en este capítulo.

1.8 El problema generalizado de mínimos cuadrados

El denominado en la literatura *problema generalizado de mínimos cuadrados* consiste en determinar un vector $\mathbf{x} \in \mathbb{R}^n$ que resuelva

$$\underset{\mathbf{x}}{\text{minimizar}} \ (\mathbf{Ax} - \mathbf{b})^T \mathbf{W}^{-1} (\mathbf{Ax} - \mathbf{b}), \quad (1.31)$$

Tabla 1.25

Número de operaciones necesarias para resolver el problema de mínimos cuadrados $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ por distintos métodos

Método	Operaciones
Ecuaciones Normales	$\frac{mn^2}{2} + \frac{n^3}{6}$
Transformaciones de Householder	$mn^2 - \frac{n^3}{3}$
Transformaciones de Givens	$2mn^2 - \frac{2}{3}n^3$
Método de Gram Schmidt	mn^2
Método de Gram Schmidt Modificado	mn^2
Método de Golub-Reinsch (SVD)	$2mn^2 + 4n^3$
Método de Golub-Reinsch-Chan (SVD)	$mn^2 + \frac{17}{3}n^3$

donde $\mathbf{b} \in \mathbb{R}^m$ es un vector dado, $A \in \mathbb{R}^{m \times n}$ es una matriz conocida y $W \in \mathbb{R}^{m \times m}$ es una matriz simétrica definida positiva, también conocida. Al problema así planteado también se le denomina de *ponderado de mínimos cuadrados*.

Este problema surge frecuentemente al tratar de encontrar el estimador de mínimos cuadrados de un vector $\mathbf{x}$ relacionado con un conjunto de observaciones o muestra $\mathbf{b}$ mediante un modelo lineal de la forma

$$\mathbf{A}\mathbf{x} = \mathbf{b} + \boldsymbol{\varepsilon},$$

donde $\boldsymbol{\varepsilon}$ es un vector aleatorio desconocido de media cero y matriz de covarianzas $\sigma^2 W$. Si $W = I$, cosa que ocurre cuando las componentes del vector $\boldsymbol{\varepsilon}$ no están relacionadas, el estimador de mínimos cuadrados coincide con el denominado de máxima verosimilitud. Si $W = B^T B$, $B \in \mathbb{R}^{m \times m}$, el problema (1.31) es equivalente a

$$\underset{\mathbf{x}}{\text{minimizar}} \|B^{-1}(\mathbf{A}\mathbf{x} - \mathbf{b})\|_2, \quad (1.32)$$

problema que, a su vez, se puede escribir en el formato tradicional de un problema de mínimos cuadrados lineal; es decir, de la forma

$$\underset{\mathbf{x}}{\text{minimizar}} \|\bar{\mathbf{A}}\mathbf{x} - \bar{\mathbf{b}}\|_2,$$

donde $\bar{\mathbf{A}} = B^{-1}A$ y $\bar{\mathbf{b}} = B^{-1}\mathbf{b}$. Desafortunadamente, en la mayoría de los problemas de este tipo, $\bar{\mathbf{A}}$ suele estar muy mal condicionada por lo que conviene tener cuidado con el método que se utiliza para resolverlo numéricamente; los basados en las ecuaciones normales o el de Gram-Schmidt clásico, probablemente, no serán los más adecuados.

Existe un método muy eficaz, diseñado ex profeso para este caso, propuesto por Paige [1979 (las dos)]. Está basado en la idea de que (1.32) es equivalente a

$$\begin{aligned} &\underset{\mathbf{x}, \mathbf{v}}{\text{minimizar}} && \mathbf{v}^T \mathbf{v} \\ &\text{sujeta a} && \mathbf{A}\mathbf{x} = \mathbf{b} + B\mathbf{v}, \end{aligned} \quad (1.33)$$

problema que está definido incluso cuando las matrices A y B no son de rango completo. En lo sucesivo, con vistas a facilitar la exposición de la mecánica del método, supondremos que ambas matrices sí son de rango completo. El procedimiento a seguir en el caso de que A y B no tuviesen rango completo varía muy poco.

Paige propone resolver (1.33) en tres pasos:

Paso 1. Transformar la matriz A mediante transformaciones ortogonales obteniendo:

$$QA = \begin{bmatrix} R_1 \\ 0 \end{bmatrix},$$

con $R_1 \in \mathbb{R}^{n \times n}$.

Aplicar las transformaciones ortogonales obtenidas a $\mathbf{b}$ y a B :

$$Q\mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix} \begin{matrix} n \\ m-n \end{matrix}, \quad QB = \begin{bmatrix} C_1 \\ C_2 \end{bmatrix} \begin{matrix} n \\ m-n \end{matrix}.$$

La condición de (1.33) de que $A\mathbf{x} = \mathbf{b} + B\mathbf{v}$ se puede entonces desdoblar y escribir de la forma

$$R_1\mathbf{x} = \mathbf{c}_1 + C_1\mathbf{v} \tag{1.34}$$

$$\mathbf{0} = \mathbf{c}_2 + C_2\mathbf{v}. \tag{1.35}$$

Por lo que respecta a la primera de estas ecuaciones, para cualquier vector $\mathbf{v} \in \mathbb{R}^m$, se puede determinar uno, $\mathbf{x}$, que la satisfaga.

Paso 2. Determinar una matriz ortogonal $P \in \mathbb{R}^{m \times m}$ tal que

$$PC_2^T = \begin{bmatrix} 0 \\ S^T \end{bmatrix} \begin{matrix} n \\ m-n \end{matrix}, \tag{1.36}$$

donde $S \in \mathbb{R}^{(m-n) \times (m-n)}$ es una matriz triangular superior. Si B no es singular, S tampoco lo será.

Expresando la ecuación (1.35) de la forma

$$\mathbf{0} = \mathbf{c}_2 + C_2 P^T P\mathbf{v},$$

haciendo

$$P\mathbf{v} = \mathbf{u} = \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \end{bmatrix},$$

se obtiene la ecuación

$$\mathbf{0} = \mathbf{c}_2 + S\mathbf{u}_2.$$

Como P es ortogonal, $\|\mathbf{v}\|_2 = \|\mathbf{u}\|_2$. El mínimo de (1.33) se obtendrá haciendo

$$\begin{aligned} \mathbf{u}_1 &= \mathbf{0} \\ \mathbf{u}_2 &= -S^{-1}\mathbf{c}_2 \quad \text{y} \\ \mathbf{v} &= P_2^T \mathbf{u}_2, \end{aligned}$$

donde

$$P = \begin{bmatrix} P_1 \\ P_2 \end{bmatrix} \begin{matrix} n \\ m-n \end{matrix}.$$

Paso 3. Resolver

$$R_1 \mathbf{x} = \mathbf{c}_1 - C_1 P_2^T S^{-1} \mathbf{c}_2$$

obteniéndose así el vector buscado, $\mathbf{x}$.

Un algoritmo que lleve a cabo estas ideas necesita $O(\frac{2}{3}m^3 - m^2n - mn^2 + \frac{2}{3}n^3)$ operaciones. Según demuestra Paige, el procedimiento es numéricamente estable.

1.9 Mínimos cuadrados lineales con restricciones lineales

Un problema lineal de mínimos cuadrados con restricciones se puede presentar de diversas formas:

MCI (Mínimos Cuadrados con Restricciones de Igualdad). Dadas dos matrices, $A \in \mathbb{R}^{m \times n}$ y $B \in \mathbb{R}^{p \times n}$, y dos vectores, $\mathbf{b} \in \mathbb{R}^m$ y $\mathbf{d} \in \mathbb{R}^p$, determinar un vector $\mathbf{x}$ que resuelva

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimizar}} \quad \|A\mathbf{x} - \mathbf{b}\|_2 \\ & \text{sujeta a} \quad B\mathbf{x} = \mathbf{d}. \end{aligned}$$

MCDQ (Mínimos Cuadrados con Restricciones Cuadráticas de Desigualdad). Dadas dos matrices, $A \in \mathbb{R}^{m \times n}$ y $B \in \mathbb{R}^{p \times n}$, y dos vectores, $\mathbf{b} \in \mathbb{R}^m$ y $\mathbf{d} \in \mathbb{R}^p$, determinar un vector $\mathbf{x}$ que resuelva

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimizar}} \quad \|A\mathbf{x} - \mathbf{b}\|_2 \\ & \text{sujeta a} \quad \|B\mathbf{x} - \mathbf{d}\|_2 \leq \gamma, \end{aligned}$$

donde $\gamma > 0$.

MCLD (Mínimos Cuadrados con Restricciones Lineales de Desigualdad). Dadas dos matrices, $A \in \mathbb{R}^{m \times n}$ y $B \in \mathbb{R}^{p \times n}$, y un vector $\mathbf{b} \in \mathbb{R}^m$, determinar un vector $\mathbf{x}$ que resuelva

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimizar}} \quad \|A\mathbf{x} - \mathbf{b}\|_2 \\ & \text{sujeta a} \quad \mathbf{l} \leq B\mathbf{x} \leq \mathbf{u}, \end{aligned}$$

donde $\mathbf{l} \in \mathbb{R}^p$ y $\mathbf{u} \in \mathbb{R}^p$.

En este apartado nos centraremos en el más frecuente de todos ellos: el de mínimos cuadrados con restricciones lineales de igualdad, MCI.

MCI tiene solución si y sólo si la ecuación $B\mathbf{x} = \mathbf{d}$ es compatible. Si $\text{rango}(B) = p$, recordemos, $B\mathbf{x} = \mathbf{d}$ es compatible para cualquier $\mathbf{d}$.

De existir solución de MCI, ésta es única si y sólo si la intersección de los conjuntos $\ker(A)$ y $\ker(B)$ es el conjunto vacío; es decir, si

$$\ker(A) \cap \ker(B) = \emptyset, \quad (1.37)$$

o, lo que es lo mismo, que

$$\text{rango} \begin{pmatrix} A \\ B \end{pmatrix} = n.$$

Si no se cumple (1.37), existirá un vector $\mathbf{z} \neq \mathbf{0}$ tal que $A\mathbf{z} = B\mathbf{z} = \mathbf{0}$, por lo que, si $\mathbf{x}$ es una solución de MCI, también lo será $\mathbf{x} + \mathbf{z}$. En este caso, como venimos observando cuando existe la posibilidad de que haya muchas soluciones, siempre es posible obtener una de norma euclídea mínima.

Cualquier algoritmo que resuelva el problema MCI con un mínimo de robustez deberá comprobar la compatibilidad de $B\mathbf{x} = \mathbf{d}$. Si no se sabe a priori si esto es así, MCI se puede reformular de la siguiente manera:

$$\begin{aligned} & \underset{\mathbf{x} \in S}{\text{minimizar}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2 \\ & S = \{\mathbf{x} : \|\mathbf{B}\mathbf{x} - \mathbf{d}\|_2 = \min\}. \end{aligned} \quad (1.38)$$

Este problema siempre tendrá una solución de norma euclídea mínima.

1.9.1 Resolución numérica del problema

Expondremos tres métodos: dos de ellos basados en la idea de transformar el problema original en otro sin restricciones, y el último en la conversión a un problema generalizado de mínimos cuadrados como el descrito anteriormente.

1.9.1.1 Método de eliminación directa

Comienza reduciendo la matriz B mediante transformaciones ortogonales a otra triangular superior o trapezoidal (ver el teorema 1.18 de la página 99) de la forma

$$Q_B B P_B = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \begin{matrix} r \\ p-r \end{matrix}, \quad (1.39)$$

donde $r = \text{rango}(B) \leq p$ y $R_{11} \in \mathbb{R}^{r \times r}$ es una matriz triangular superior no singular.

Aplicando las transformaciones que representa la matriz Q_B al vector $\mathbf{d}$, se tiene que

$$Q_B \mathbf{d} = \bar{\mathbf{d}} = \begin{bmatrix} \bar{\mathbf{d}}_1 \\ \bar{\mathbf{d}}_2 \end{bmatrix}$$

y, por lo tanto, la restricción del problema, $B\mathbf{x} = \mathbf{d}$, se puede escribir

$$[R_{11}, R_{12}]\bar{\mathbf{x}} = \bar{\mathbf{d}}_1, \quad (1.40)$$

donde $\bar{\mathbf{x}} = P_B^T \mathbf{x}$, siendo $\bar{\mathbf{d}}_2 = \mathbf{0}$ si y sólo si la restricción es compatible.

Si la permutación que representa P_B se aplica también a las columnas de la matriz A y se reordena el resultado conforme a (1.39), se tiene que

$$\mathbf{A}\mathbf{x} - \mathbf{b} = \bar{\mathbf{A}}\bar{\mathbf{x}} - \mathbf{b} = [\bar{\mathbf{A}}_1, \bar{\mathbf{A}}_2] \begin{bmatrix} \bar{\mathbf{x}}_1 \\ \bar{\mathbf{x}}_2 \end{bmatrix} - \mathbf{b},$$

donde $\bar{A} = AP_B$. Eliminando la variable $\bar{x}_1$ de esta última expresión y utilizando (1.40), es decir, que $\bar{x}_1 = R_{11}^{-1}(\bar{d}_1 - R_{12}\bar{x}_2)$, se obtiene que

$$A\mathbf{x} - \mathbf{b} = \hat{A}_2\bar{x}_2 - \hat{\mathbf{b}},$$

donde

$$\begin{aligned}\hat{A}_2 &= \bar{A}_2 - \bar{A}_1 R_{11}^{-1} R_{12} \quad \text{y} \\ \hat{\mathbf{b}} &= \mathbf{b} - \bar{A}_1 R_{11}^{-1} \bar{\mathbf{d}}_1.\end{aligned}$$

De acuerdo con todo este razonamiento, MCI es equivalente al siguiente problema:

$$\underset{\bar{x}_2}{\text{minimizar}} \quad \|\hat{A}_2\bar{x}_2 - \hat{\mathbf{b}}\|_2, \quad (1.41)$$

donde $\hat{A}_2 \in \Re^{m \times (n-r)}$.

Si se cumple la condición (1.37), el rango de la matriz $\hat{A}_2$ es $n-r$ y el problema (1.41) tiene una única solución. En efecto, si $\text{rango}(\hat{A}_2) < n-r$ existirá entonces un vector $\mathbf{v} \neq \mathbf{0}$ tal que

$$\hat{A}_2\mathbf{v} = \bar{A}_2\mathbf{v} - \bar{A}_1 R_{11}^{-1} R_{22}\mathbf{v} = \mathbf{0}.$$

Si se hace $\mathbf{u} = -R_{11}^{-1} R_{12}\mathbf{v}$, resulta que

$$\begin{aligned}R_{11}\mathbf{u} + R_{12}\mathbf{v} &= \mathbf{0} \quad \text{y} \\ \bar{A}_1\mathbf{u} + \bar{A}_2\mathbf{v} &= \mathbf{0}.\end{aligned}$$

De acuerdo con esto, el vector

$$\mathbf{w} = P_B \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} \neq \mathbf{0}$$

pertenece al núcleo de A y B por lo que no se cumple (1.37).

Si se cumple (1.37), se puede efectuar la descomposición QR de la matriz $\hat{A}_2$ obteniéndose

$$Q_A \hat{A}_2 = \begin{bmatrix} R_{22} \\ 0 \end{bmatrix} \quad \text{y} \quad Q_A \mathbf{b} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix},$$

donde $R_{22} \in \Re^{(n-r) \times (n-r)}$ es una matriz triangular superior regular.

Calculando después $\bar{x}$, resolviendo el sistema triangular

$$\begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix} \bar{\mathbf{x}} = \begin{bmatrix} \bar{\mathbf{d}}_1 \\ \mathbf{c}_1 \end{bmatrix},$$

se llega, finalmente, a la solución del problema MCI, $\mathbf{x}$, haciendo $\mathbf{x} = P_B \bar{\mathbf{x}}$.

El conjunto de vectores $\mathbf{x} = P_B \bar{\mathbf{x}}$ que satisface (1.40) es el mismo que minimiza $\|B\mathbf{x} - \mathbf{d}\|_2$. La forma de proceder que hemos descrito, por tanto, no sólo resuelve MCI sino también su forma equivalente (1.38).

Si no se cumple la condición (1.37), la solución del problema (1.41) no es única; para conseguir una de norma euclídea mínima habría que efectuar permutaciones de columnas al factorizar en la forma QR la matriz $\hat{A}_2$ y luego proceder de la misma forma que se hacía en

el apartado 1.7.2.2.1.3 de la página 98 al exponer cómo resolver $A\mathbf{x} = \mathbf{b}$ con una matriz A de rango incompleto.

El procedimiento que se ha descrito se puede codificar fácilmente en un programa en FORTRAN 77 como el que sigue. Concretamente, el código que se lista resuelve

$$\begin{aligned} & \underset{\mathbf{x} \in \mathbb{R}^3}{\text{minimizar}} \left\| \begin{bmatrix} 0,2113 & 0,4524 & 0,6538 \\ 0,0824 & 0,8075 & 0,4899 \\ 0,7599 & 0,4832 & 0,7741 \\ 0,0087 & 0,6135 & 0,9626 \end{bmatrix} \mathbf{x} - \begin{bmatrix} 3,0775 \\ 3,1671 \\ 4,0485 \\ 4,1237 \end{bmatrix} \right\|_2 \\ & \text{sujeta a} \quad \begin{bmatrix} 0,8096 & 0,2749 & 0,9933 \\ 0,8474 & 0,8807 & 0,8360 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 4,3393 \\ 5,1169 \end{bmatrix}. \end{aligned}$$

La solución de este problema es

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

```

PROGRAM Mci
C
  implicit double precision (a-h,o-z)
  parameter (m1=4,m2=2,n=3)
  dimension a(m1,n),b(m1),c(m2,n),d(m2),w(m1),x(n),ipiv(n)
C
  data a/0.2113,0.0824,0.7599,0.0087,0.4524,0.8075,0.4832,
+      0.6135,0.6538,0.4899,0.7741,0.9626/
  data b/3.0775,3.1671,4.0485,4.1237/
  data c/0.8096,0.8474,0.2749,0.8807,0.9933,0.8360/
  data d/4.3393,5.1169/
  data tau/0.000001/
C
  |R   R   |
C *** Reducción  QCP=| 11 12| ***
C                    |0   0   |
  ira = min0(m2,n)
  do i = 1,ira
    imax = i
    rmax = 0.0
    do j = i,n
      h = 0.0
      do k = i,m2
        h = h+c(k,j)**2
      end do
      if (h.gt.rmax) then
        rmax = h
        imax = j
      endif
    end do
    ipiv(i) = imax
    if (imax.ne.i) then
      do j = 1,m2
        tmp = c(j,i)
        c(j,i) = c(j,imax)
        c(j,imax) = tmp
      end do
      do j = 1,m1
        tmp = a(j,i)

```

```

        a(j,i)    = a(j,imax)    !      en matriz A.
        a(j,imax) = tmp          !
    end do          !
endif
if (i+1.le.m2) then
    call h1 (beta,i,i+1,m2,w,c(1,i)) ! Aplicar transformación
    do j = i+1,n                      ! de Householder a columnas
        call h2 (beta,i,i+1,m2,w,c(1,j)) ! i a n de la matriz C.
    end do
    call h2 (beta,i,i+1,m2,w,d)      ! Aplicar trans. a vector d.
endif
end do
C
k = ira                                ! Determinar rango de C.
do j = 1,ira
    if (dabs(c(j,j)).le.tau) then
        k = j-1
        exit
    endif
end do
C
do i = 1,m1                            !      ^      ^
    a(i,1) = a(i,1)/c(1,1)             ! Determinar A y B
    do j = 2,ira                       !      2
        s = 0.0
        do k = 1,j-1
            s = s+a(i,k)*c(k,j)
        end do
        a(i,j) = (a(i,j)-s)/c(j,j)
    end do
    do j = ira+1,n
        s = 0.0
        do k = 1,ira
            s = s+a(i,k)*c(k,j)
        end do
        a(i,j) = a(i,j)-s
    end do
    s = 0.0
    do k = 1,ira
        s = s+a(i,k)*d(k)
    end do
    b(i) = b(i)-s
end do
C
do i = ira+1,n                        ! Aplicar transformación
    k = i-ira                         ! de Householder a columnas
    call h1 (beta,k,k+1,m1,w,a(1,i)) ! IRA+1 a N de matriz A; es
    do j = i+1,n                      ! decir a ^
        call h2 (beta,k,k+1,m1,w,a(1,j)) !      A
    end do                            !      2
    call h2 (beta,k,k+1,m1,w,b)      ! Aplicar trans. a vector B.
end do
C
n1 = n-ira                            ! Resolver el sistema
x(n) = b(n1)/a(n1,n)                  !
do i = n1-1,1,-1                      !      |R   R   || | |D |
    s = 0.0                           !      | 11 12||x|=| 1|
    do j = i+1,n1                     !      |0   R   || | |C |
        s = s+a(i,j+ira)*x(j+ira)     !      |      22|| | | 1|
    end do
end do

```

```
        end do
        x(i+ira) = (b(i)-s)/a(i,i+ira)
    end do
    do i = ira,1,-1
        s = 0.0
        do j = i+1,n
            s = s+c(i,j)*x(j)
        end do
        x(i) = (d(i)-s)/c(i,i)
    end do
C
    do j = ira,1,-1
        if (ipiv(j).ne.j) then          ! Deshacer permutación intro-
            l = ipiv(j)                ! ducida por pivotaciones.
            tmp = x(l)
            x(l) = x(j)
            x(j) = tmp
        endif
    end do
C
    print '(\'\' Rango de C:\',i3)',k
    print '(\'\' Solución:\', 6(f8.4:\',\'))',x
C
end

subroutine h1(beta,i,j,m,w,x)
C
double precision beta,w(m),x(m)
C
    beta = 0.0
    do k = j,m
        w(k) = x(k)
        beta = beta+w(k)*w(k)
    end do
    w(i) = x(i)
    sigma = sign(sqrt(beta+w(i)*w(i)),x(i))
    w(i) = w(i)+sigma
    beta = 2.0/(beta+w(i)*w(i))
    x(i) = -sigma
C
    return
end

subroutine h2(beta,i,j,m,w,x)
C
double precision beta,w(m),x(m),s
C
    s = w(i)*x(i)
    do k = j,m
        s = s+w(k)*x(k)
    end do
    s = s*beta
    x(i) = x(i)-w(i)*s
    do k = j,m
        x(k) = x(k)-w(k)*s
    end do
C
    return
end
```

1.9.1.2 Método de la base del subespacio núcleo de la matriz de restricciones

En este apartado supondremos que $\text{rango}(B) = p$. El método inicialmente calcula una sucesión de transformaciones ortogonales, representadas por la matriz $Q_B \in \mathbb{R}^{n \times n}$, tales que

$$Q_B^T B^T = \begin{bmatrix} R_B \\ 0 \end{bmatrix},$$

donde $R_B \in \mathbb{R}^{p \times p}$ es una matriz triangular superior regular. Si se hace

$$Q_B = [Q_1, Q_2], \quad \text{con } Q_1 \in \mathbb{R}^{n \times p} \text{ y } Q_2 \in \mathbb{R}^{n \times (n-p)},$$

entonces, $\ker(B) = \text{Im}(Q_2)$; es decir, los vectores columna de Q_2 forman una base del subespacio núcleo de B . Cualquier vector $\mathbf{x} \in \mathbb{R}^n$ que satisfaga la restricción $B\mathbf{x} = \mathbf{d}$ se puede representar como

$$\mathbf{x} = \mathbf{x}_1 + Q_2 \mathbf{y}_2, \quad (1.42)$$

donde $\mathbf{x}_1 = B^\dagger \mathbf{d} = Q_1 R_B^{-T} \mathbf{d}$. En consecuencia,

$$A\mathbf{x} - \mathbf{b} = A\mathbf{x}_1 + AQ_2 \mathbf{y}_2 - \mathbf{b},$$

donde $\mathbf{y}_2 \in \mathbb{R}^{n-p}$, por lo que resolver MCI es equivalente a

$$\underset{\mathbf{y}_2}{\text{minimizar}} \|(AQ_2)\mathbf{y}_2 - (\mathbf{b} - A\mathbf{x}_1)\|_2. \quad (1.43)$$

Sea $\mathbf{y}_2$ la solución de este último problema de norma euclídea mínima; es decir,

$$\mathbf{y}_2 = (AQ_2)^\dagger (\mathbf{b} - A\mathbf{x}_1),$$

y sea $\mathbf{x}$ un vector de la forma (1.42). Como $\mathbf{x}_1 \perp Q_2 \mathbf{y}_2$, entonces

$$\|\mathbf{x}\|_2^2 = \|\mathbf{x}_1\|_2^2 + \|Q_2 \mathbf{y}_2\|_2^2 = \|\mathbf{x}_1\|_2^2 + \|\mathbf{y}_2\|_2^2$$

siendo $\mathbf{x}$ precisamente el vector solución de MCI de norma euclídea mínima.

Supongamos ahora que se cumple la condición (1.37). La matriz

$$C = \begin{bmatrix} B \\ A \end{bmatrix} Q_B = \begin{bmatrix} R_B & 0 \\ AQ_1 & AQ_2 \end{bmatrix}$$

debe tener rango n . Si esto es así, todas las columnas de C son linealmente independientes por lo que $\text{rango}(AQ_2) = n - p$. Se puede entonces calcular la descomposición QR ,

$$Q_A^T (AQ_2) = \begin{bmatrix} R_A \\ 0 \end{bmatrix},$$

donde R_A es una matriz triangular superior regular. La única solución de (1.43) se puede calcular de

$$R_A \mathbf{y}_2 = \mathbf{c}_1,$$

donde

$$\mathbf{c} = \begin{bmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{bmatrix} = Q_A^T (\mathbf{b} - A\mathbf{x}_1),$$

obteniéndose, finalmente, $\mathbf{x} = \mathbf{x}_1 + Q_2 \mathbf{y}_2$, la única solución de MCI.

1.9.1.3 Método de la ponderación

Se basa en una idea muy simple. Supóngase que se desea que en un problema de mínimos cuadrados se satisfagan exactamente alguna de las ecuaciones. Una forma de conseguirlo consiste en asignar un peso, γ , a esas ecuaciones mucho mayor que al resto y resolver el problema resultante sin condiciones mediante alguno de los métodos estudiados. Es decir, para resolver MCI, calcular la solución de

$$\underset{\mathbf{x}(\gamma)}{\text{minimizar}} \left\| \begin{bmatrix} \gamma B \\ A \end{bmatrix} \mathbf{x} - \begin{bmatrix} \gamma \mathbf{d} \\ \mathbf{b} \end{bmatrix} \right\|_2. \quad (1.44)$$

Obsérvese que si se cumple la condición (1.37), éste es un problema de mínimos cuadrados de rango completo.

Para resolver (1.44) se puede aplicar el método sugerido en el apartado relativo al problema generalizado de mínimos cuadrados, puesto que este problema es como el introducido entonces.

Referencias

Todo el material incluido en este capítulo es bastante estándar en la literatura de análisis numérico, álgebra lineal numérica e incluso de álgebra lineal. Las referencias básicas esenciales en las que se puede encontrar casi todo lo expuesto son: Ciarlet [1988]; Forsythe, Malcolm y Moler [1977]; Golub y Van Loan [1983] y [1989]; Lascaux y Théodor [1986]; Stewart [1973]; Stoer y Bulirsch [1980]; Strang [1976] y Wilkinson [1965]. Más especializado sólo en mínimos cuadrados pero también esencial, Lawson y Hanson [1974] y [1995]. Muy recientemente, Gill, Murray y Wright [1991] y Björk [1996].

Todos los programas de ordenador son del autor; están basados en los correspondientes algoritmos. El que plasma el de Bunch y Kaufman sigue el programa SSIFA de Dongarra, Bunch, Moler y Stewart [1979]. El relativo a la descomposición en valores singulares de una matriz está basado en el de Forsythe, Malcolm y Moler [1977] y en el de Press, Flannery, Teukolsky y Vetterling [1986]. Otros libros donde se pueden encontrar programas parecidos son: Atkinson, Harley y Hudson [1989]; Forsythe, Malcolm y Moler [1977]; Hager [1988]; Lascaux y Théodor [1986]; Lawson y Hanson [1974] y [1995]; Longley [1984] y Press, Flannery, Teukolsky y Vetterling [1986]. En PASCAL en Phillips y Cornelius [1986]. Una buena revisión del *software* existente de este tipo hasta la fecha de su publicación puede encontrarse en Rice [1983] y [1993], y Moré y Wright [1993]. Paquetes como LINPACK, EISPACK y LAPACK incluyen variantes de todos los algoritmos presentados.

El apartado 1.5 sigue enteramente a Golub y Van Loan [1983] y [1989], con algunas consideraciones de Stewart [1973], Bunch y Kaufman [1977] y Dongarra, Bunch, Moler y Stewart [1979].

El material del apartado 1.6 es también estándar. Está basado en este caso en Dennis y Schnabel [1983], Forsythe, Malcolm y Moler [1977] y Lascaux y Théodor [1986].

Los resultados teóricos del apartado 1.7 se pueden encontrar en Lawson y Hanson [1974] y [1995], Luenberger [1969], Golub y Van Loan [1983] y [1989] y Björk [1990] y [1996]. Algunas de las consideraciones son de Stoer y Bulirsch [1980]. Como hemos dicho, Golub y Van Loan [1983] y [1989] son esenciales para estudiar las cuestiones numéricas relativas a los problemas lineales de mínimos cuadrados. Una buena descripción del problema se puede encontrar en Hager [1988]. Los métodos numéricos del problema de rango incompleto o indeterminado son

de Hager [1988] y Lascaux y Théodor [1986]. El método de las transformaciones rápidas de Givens se ha obtenido de Hager [1988].

Referencias de los comienzos históricos de los asuntos estudiados en este capítulo se pueden encontrar en Longley [1984] y Schrijver [1986].

Ejercicios

- 1.1. Demostrar que la matriz $A \in \Re^{n \times n}$ es regular si y sólo si la ecuación $A\mathbf{x} = \mathbf{0}$ tiene solución distinta de $\mathbf{x} = \mathbf{0}$.
- 1.2. Demostrar que:
 - a) Si A tiene rango completo, $A^T A$ es regular.
 - b) Si A tiene vectores columna linealmente dependientes, $A^T A$ es singular.
 - c) El sistema $A^T A\mathbf{x} = A^T \mathbf{b}$ es siempre compatible para cualquier $\mathbf{b}$ (de dimensión adecuada); incluso si $A^T A$ es singular.
- 1.3. Una matriz cuadrada, T , es triangular en bloques si se puede reordenar de la forma

$$T = \begin{bmatrix} T_{11} & T_{12} & \cdots & T_{1m} \\ 0 & T_{22} & \cdots & T_{2m} \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & T_{mm} \end{bmatrix},$$

donde cada bloque T_{ii} es cuadrado. Demostrar que T es regular si y sólo si sus bloques diagonales T_{ii} son regulares, y que su inversa también es triangular en bloques con la misma partición de columnas que T .

- 1.4. Deducir una forma eficaz del algoritmo de eliminación de Gauss para reducir una matriz de Hessenberg.
- 1.5. Igual que en el ejercicio anterior para el caso de una matriz tridiagonal.
- 1.6. Igual que los dos ejercicios anteriores para el caso de una matriz en banda con ancho de banda igual a $2k + 1$, donde k indica la fila.
- 1.7. Sea A una matriz simétrica con $a_{11} \neq 0$. Después de la primera etapa de la eliminación de Gauss, A tiene la forma

$$\begin{bmatrix} a_{11} & \mathbf{a}_1^T \\ \mathbf{0} & A_2 \end{bmatrix}.$$

Demostrar que A_2 es simétrica.

- 1.8. Sea A una matriz de diagonal dominante. Después de la primera etapa de la eliminación de Gauss, A tiene la forma

$$\begin{bmatrix} a_{11} & \mathbf{a}_1^T \\ \mathbf{0} & A_2 \end{bmatrix}.$$

Demostrar que A_2 es diagonal dominante.

- 1.9. Demostrar que si las matrices B y C son regulares, $\kappa(BC) \leq \kappa(B) \cdot \kappa(C)$.
- 1.10. Demostrar que $\kappa_2(A^T) = \kappa_2(A)$. ¿Se cumple este resultado con la condición uno e infinito?
- 1.11. Dada la matriz

$$A = \begin{bmatrix} 0,550 & 0,423 \\ 0,484 & 0,372 \end{bmatrix}:$$

- a) Encontrar unos vectores $\mathbf{b}$ y $\delta\mathbf{b}$ tales que la solución de

$$A(\mathbf{x} + \delta\mathbf{x}_b) = \mathbf{b} + \delta\mathbf{b} \quad (1.45)$$

satisfaga

$$\frac{\|\delta\mathbf{x}_b\|}{\|\mathbf{x}\|} \approx \kappa(A) \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}.$$

- b) Encontrar unos vectores $\mathbf{b}$ y $\delta\mathbf{b}$ tales que la solución de (1.45) satisfaga

$$\frac{\|\delta\mathbf{x}_b\|}{\|\mathbf{x}\|} \ll \kappa(A) \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|}.$$

- c) ¿Se puede encontrar una δA tal que la solución exacta de

$$(A + \delta A)(\mathbf{x} + \delta\mathbf{x}_A) = \mathbf{b}$$

satisfaga

$$\frac{\|\delta\mathbf{x}_A\|}{\|\mathbf{x} + \delta\mathbf{x}_A\|} \approx \kappa(A) \frac{\|\delta A\|}{\|A\|}?$$

- 1.12. Sea la siguiente matriz triangular superior con un *pico* en la primera columna,

$$V = \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & & \times & \times & \times \\ \times & & & \times & \times \\ \times & & & & \times \end{bmatrix}.$$

- a) Describir cómo se llevaría a cabo, mediante rotaciones elementales, la transformación de esta matriz en otra que tuviese la primera columna situada en la última.
b) Sea A una matriz simétrica definida positiva cuya factorización de Cholesky es $G^T G$ y sea P la matriz de permutación que refleja un único intercambio de filas o columnas. Basándose en el resultado del punto anterior, describir un método que calcule la factorización de Cholesky de $P^T A P$ a partir de G .

- 1.13. Resolver el sistema lineal

$$\begin{aligned} 2x_1 + x_2 - 2x_3 &= 4 \\ 4x_1 + x_2 - x_3 &= 7 \\ 6x_1 - x_2 + 6x_3 &= 8. \end{aligned}$$

Después, cambiar el elemento 8 del término independiente por un 7 y volverlo a resolver. ¿Qué pasa?

- 1.14. Resolver el sistema lineal

$$\begin{aligned} 5x_1 + 3x_2 + 10x_3 &= 5 \\ 2x_1 + 0,2x_2 + 3x_3 &= -1 \\ x_1 - 0,4x_2 + 3x_3 &= 6. \end{aligned}$$

Resolverlo después usando pivotación total.

- 1.15. Descomponer la matriz

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 8 & 7 \\ 2 & 16 & 12 \end{bmatrix}$$

en la forma $L_1 U$. Obtener la solución del sistema $A\mathbf{x} = \mathbf{b}$, donde

$$\mathbf{b} = \begin{bmatrix} 2 \\ 7 \\ 10 \end{bmatrix}.$$

1.16. Considérese la matriz de Hessenberg

$$A = \begin{bmatrix} 1,2113 & 0,8096 & 0,4832 & 0,6538 \\ 0,0824 & 1,7424 & 0,6135 & 0,4899 \\ & 0,6524 & 1,2749 & 0,7741 \\ & & 0,8807 & 1,9626 \end{bmatrix}.$$

Encontrar una matriz triangular superior U y matrices elementales L_1 , L_2 y L_3 tales que

$$A = L_1 L_2 L_3 U.$$

1.17. Considérese la matriz

$$\psi = \begin{bmatrix} 0 & 1 \\ 1 & -\mu \end{bmatrix} :$$

- Calcular explícitamente la inversa de ψ .
- Deducir una fórmula de $\kappa(\psi)$ respecto a cualquier norma.
- Supóngase que $|\mu| \leq 1$ y considérese el producto Π_k de k ($k > 1$) matrices de la forma de ψ . Dar un límite superior de la condición de Π_k .

1.18. Resolver los siguientes sistemas lineales:

- $$\begin{aligned} 1/4x_1 + 1/5x_2 + 1/6x_3 &= 9 \\ 1/3x_1 + 1/4x_2 + 1/5x_3 &= 8 \\ 1/2x_1 + x_2 + 2x_3 &= 8. \end{aligned}$$
- $$\begin{aligned} 3,333x_1 + 15920x_2 - 10,333x_3 &= 15913 \\ 2,222x_1 + 16,71x_2 + 9,612x_3 &= 28,544 \\ 1,5611x_1 + 5,1791x_2 + 1,6852x_3 &= 8,4254. \end{aligned}$$
- $$\begin{aligned} 4,01x_1 + 1,23x_2 + 1,43x_3 - 0,73x_4 &= 5,94 \\ 1,23x_1 + 7,41x_2 + 2,412x_3 + 3,02x_4 &= 14,07 \\ 1,43x_1 + 2,41x_2 + 5,792x_3 - 1,11x_4 &= 8,52 \\ -0,73x_1 + 3,02x_2 - 1,11x_3 + 6,41x_4 &= 7,59. \end{aligned}$$
- $$\begin{aligned} x_1 + 1/2x_2 + 1/3x_3 + 1/4x_4 &= 1/6 \\ 1/2x_1 + 1/3x_2 + 1/4x_3 + 1/5x_4 &= 1/7 \\ 1/3x_1 + 1/4x_2 + 1/5x_3 + 1/6x_4 &= 1/8 \\ 1/4x_1 + 1/5x_2 + 1/6x_3 + 1/7x_4 &= 1/9. \end{aligned}$$

1.19. Repetir el ejercicio anterior usando el método de Gauss-Jordan.

1.20. Comparar las operaciones necesarias para resolver un sistema lineal de ecuaciones 10.000×10.000 por los métodos de eliminación de Gauss, Gauss-Jordan y Cholesky (supuesto que se pueda, claro).

1.21. Considérese la matriz

$$A = \begin{bmatrix} 2 & 4 \\ 1 & 2 \\ 1 & 2 \end{bmatrix} \quad \text{y el vector} \quad \mathbf{b} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} :$$

- ¿Cuál es el rango de la matriz A ? Obtener una expresión general de los vectores del subespacio $\text{Im}(A)$.
- Demostrar que la dimensión del subespacio $\ker(A^T)$ es 2. Obtener dos vectores linealmente independientes de este último subespacio. Deducir una expresión general de dicho subespacio.
- Encontrar dos vectores $\mathbf{b}_I \in \text{Im}(A)$ y $\mathbf{b}_K \in \ker(A^T)$ tales que $\mathbf{b} = \mathbf{b}_I + \mathbf{b}_K$.

1.22. Resolver el sistema

$$\begin{aligned}x_1 + 5x_2 &= 36 \\ 2x_1 - x_2 &= 45 \\ -3x_1 + x_2 &= 0.\end{aligned}$$

1.23. Resolver el sistema

$$\begin{aligned}x_1 + 2x_2 - 3x_3 &= 42 \\ 5x_1 - x_2 + 3x_3 &= 54.\end{aligned}$$

1.24. Resolver el siguiente problema de optimización:

$$\begin{aligned}\text{minimizar } & 4x_1^2 + 9x_2^2 \\ \text{s. a } & 8x_1 + 9x_2 = 15.\end{aligned}$$

1.25. ¿Cuál es el punto del plano $y = 2x_1 + x_2 - 12$ más cerca del origen?

1.26. Deducir la transformación de Householder que anula el segundo elemento del vector $[5, 12]^T$.

1.27. Calcular la factorización QR de la matriz

$$A = \begin{bmatrix} 5 & -13 \\ 12 & 26 \end{bmatrix}.$$

1.28. Igual que el ejercicio anterior usando transformaciones de Givens.

1.29. Demostrar que cualquier matriz de Householder 2×2 tiene la forma

$$H = \begin{bmatrix} a & b \\ b & -a \end{bmatrix},$$

donde $a^2 + b^2 = 1$.

1.30. Dada la matriz

$$A = \begin{bmatrix} 0,6 & 0,8 & 0 \\ 0,8 & -0,6 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 0 & 0 \end{bmatrix} :$$

a) Determinar la solución del sistema $A\mathbf{x} = \mathbf{b}$, donde $\mathbf{b} = [10, 20, 10]^T$.

b) Determinar la solución del sistema $A^T\mathbf{x} = \mathbf{b}$, donde $\mathbf{b} = [20, 40]^T$.

Capítulo 2

MÉTODOS ITERATIVOS DE SOLUCIÓN DE SISTEMAS DE ECUACIONES LINEALES

EN EL CAPÍTULO dedicado a los métodos directos de solución de sistemas de ecuaciones lineales dejábamos entrever las dificultades con que se puede encontrar un usuario de esos métodos a la hora de resolver problemas grandes o muy grandes (miles, decenas o cientos de miles de ecuaciones e incógnitas). Como ejemplo, si se desea modelizar la temperatura en las distintas partes de un cuerpo tridimensional con forma de paralelepípedo, suponiendo que la temperatura de una partícula de ese cuerpo depende de su posición, su valor se puede aproximar discretizando cada una de las tres dimensiones del cuerpo en unos intervalos determinados y considerando cada uno de los pequeños trocitos de la malla que se obtiene. Si cada lado del paralelepípedo se divide en 100 intervalos la malla resultante tendrá $100 \times 100 \times 100 = 1.000.000$ de elementos o pequeños cuerpos. A pesar de que en este caso existe un solo parámetro físico a considerar, la temperatura, el modelo adoptado involucra cálculos con un millón de variables o incógnitas: la temperatura en cada elemento.

Tal como se han expuesto, los métodos directos no se deben aplicar a sistemas de muy grandes dimensiones, como el del ejemplo anterior, a no ser que se disponga de grandes y potentes ordenadores o que la matriz del sistema presente una estructura que permita, utilizando o desarrollando técnicas ad hoc, su reducción o transformación a otra sencilla de manipular. Afortunadamente, muchos de los grandes sistemas de ecuaciones lineales que se plantean habitualmente en la industria y en la técnica presentan unas matrices de coeficientes en las que los elementos distintos de cero son muy pocos. Tal es el caso, por ejemplo, de los que surgen en el análisis y planificación de sistemas eléctricos de generación y transporte de energía, en problemas de contorno para ecuaciones en derivadas parciales, en análisis de estructuras mecánicas mediante elementos finitos, en problemas de transmisión de calor, y en muchos otros. En el

caso del ejemplo anterior se puede llegar a una tal matriz sin más que asumir que la temperatura en cada uno de los elementos definidos sólo está íntimamente relacionada con la de los más próximos a él. El límite más o menos amplio de esa proximidad definirá una matriz de coeficientes más o menos llena de elementos.

En los últimos veinticinco años se han venido desarrollado técnicas especiales, sencillas al principio y cada día más sofisticadas, para manipular matrices con pocos elementos no nulos —*matrices dispersas*— y poder aplicar en ellas los procedimientos numéricos directos a los que nos hemos referido.

Una forma clásica de resolver ciertos sistemas de ecuaciones lineales de grandes dimensiones, sin utilizar los métodos directos, la conforman los métodos iterativos. A ellos y a las condiciones en las cuales se pueden aplicar se dedica este capítulo.

La idea básica de los métodos iterativos consiste en llegar a la solución del problema mediante una sucesión de soluciones que converja a aquella. Estos métodos no proporcionan, teóricamente, la solución exacta aunque sí permiten, cuando funcionan bien, acercarse a ella tanto como se desee.

Si consideramos que el problema a resolver es el de determinar un vector $\mathbf{x}$ tal que $A\mathbf{x} = \mathbf{b}$, la idea común de todos los métodos iterativos estriba en descomponer la matriz de coeficientes A de la forma

$$A = R - S, \quad (2.1)$$

de tal manera que R sea invertible y con inversa fácilmente calculable (lo cual ocurre, por ejemplo, si R es diagonal o triangular). Con esta descomposición la ecuación $A\mathbf{x} = \mathbf{b}$ se puede escribir

$$R\mathbf{x} = S\mathbf{x} + \mathbf{b},$$

o

$$\begin{aligned} \mathbf{x} &= R^{-1}(R - A)\mathbf{x} + R^{-1}\mathbf{b} \\ &= (I - R^{-1}A)\mathbf{x} + R^{-1}\mathbf{b} \\ &= M\mathbf{x} + \mathbf{c}, \end{aligned} \quad (2.2)$$

donde $M = I - R^{-1}A$ y $\mathbf{c} = R^{-1}\mathbf{b}$.

La ecuación $\mathbf{x} = M\mathbf{x} + \mathbf{c}$ sugiere la definición de un *esquema iterativo del tipo*

$$\boxed{\mathbf{x}^{(k+1)} = M\mathbf{x}^{(k)} + \mathbf{c}} \quad (2.3)$$

con el que, partiendo de un vector inicial arbitrario, $\mathbf{x}^{(0)}$, se obtenga una sucesión de vectores que converja a la solución de la ecuación $A\mathbf{x} = \mathbf{b}$. El método iterativo será convergente si

$$\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}.$$

Un caso particular de esquema iterativo es el que define el denominado de Richardson,

$$\boxed{\mathbf{x}^{(k+1)} = (I - A)\mathbf{x}^{(k)} + \mathbf{b}},$$

al que se llega escribiendo la ecuación $A\mathbf{x} = \mathbf{b}$ como $\mathbf{x} = (I - A)\mathbf{x} + \mathbf{b}$.

Los métodos basados en esquemas iterativos como los expuestos se denominan *métodos iterativos estacionarios* pues la transición del punto $\mathbf{x}^{(k)}$ a $\mathbf{x}^{(k+1)}$ no depende de lo que ocurre en iteraciones precedentes. Las técnicas iterativas rara vez se emplean para resolver sistemas de ecuaciones lineales de pequeña dimensión pues el tiempo y número de iteraciones requerido para lograr una precisión suficiente en la solución exceden a los de los métodos directos.

2.1 Método de Jacobi

El primero de los métodos que consideramos es el que Carl Gustav Jacobi (1804-1851) desarrolló en 1845. Su mecánica es muy simple: supongamos que se desea resolver el sistema de tres ecuaciones lineales con tres incógnitas

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3.\end{aligned}$$

Admitiendo que los coeficientes a_{11} , a_{22} y a_{33} son distintos de cero, se puede despejar de la primera ecuación la incógnita x_1 , de la segunda x_2 y x_3 de la tercera, resultando

$$\begin{aligned}x_1 &= \frac{1}{a_{11}}(b_1 - a_{12}x_2 - a_{13}x_3) \\x_2 &= \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3) \\x_3 &= \frac{1}{a_{33}}(b_3 - a_{31}x_1 - a_{32}x_2).\end{aligned}$$

Estas expresiones y la ecuación general (2.3) sugieren emplear como método iterativo el que definen las siguientes relaciones de recurrencia:

$$\begin{aligned}x_1^{(k+1)} &= \frac{1}{a_{11}} \left(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} \right) \\x_2^{(k+1)} &= \frac{1}{a_{22}} \left(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} \right) \\x_3^{(k+1)} &= \frac{1}{a_{33}} \left(b_3 - a_{31}x_1^{(k)} - a_{32}x_2^{(k)} \right).\end{aligned}$$

La generalización de esta idea es la base del método iterativo de Jacobi. La relación general de recurrencia para un sistema $n \times n$ es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j^{(k)} \right); \quad i = 1, \dots, n. \quad (2.4)$$

Razonando tal y como se hacía al principio de este capítulo, si se descompone la matriz de coeficientes del sistema, A , de la forma sugerida en (2.1) haciendo

$$A = D - (D - A),$$

donde D es la matriz diagonal formada con los elementos de la diagonal principal de A , es

decir,

$$D = \begin{bmatrix} a_{11} & 0 & \cdots & 0 & 0 \\ 0 & a_{22} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & a_{n-1\ n-1} & 0 \\ 0 & 0 & \cdots & 0 & a_{nn} \end{bmatrix}, \quad (2.5)$$

el *esquema iterativo del método de Jacobi* escrito en forma matricial, a partir de las expresiones (2.2) y (2.3), resulta

$$\mathbf{x}^{(k+1)} = (I - D^{-1}A) \mathbf{x}^{(k)} + D^{-1}\mathbf{b}.$$

Si todos los elementos a_{ii} , $i = 1, \dots, n$, son no nulos, la matriz D es invertible. A la matriz

$$J = I - D^{-1}A$$

que caracteriza el método de Jacobi se la denomina *matriz de Jacobi*.

Ejemplo 2.1 Resolvamos el sistema de ecuaciones lineales

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6 \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25 \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11 \\ 3x_2 - x_3 + 8x_4 &= 15. \end{aligned}$$

Aplicando la relación general de recurrencia (2.4) a este caso, partiendo del punto inicial $\mathbf{x}^{(0)} = [0, 0, 0, 0]^T$, se tendrá:

$$\begin{aligned} x_1^{(1)} &= \frac{1}{10}x_2^{(0)} - \frac{1}{5}x_3^{(0)} + \frac{3}{5} = 0,6000 \\ x_2^{(1)} &= \frac{1}{11}x_1^{(0)} + \frac{1}{11}x_3^{(0)} - \frac{3}{11}x_4^{(0)} + \frac{25}{11} = 2,2727 \\ x_3^{(1)} &= -\frac{1}{5}x_1^{(0)} + \frac{1}{10}x_2^{(0)} + \frac{1}{10}x_4^{(0)} - \frac{11}{10} = -1,1000 \\ x_4^{(1)} &= -\frac{3}{8}x_2^{(0)} + \frac{1}{8}x_3^{(0)} + \frac{15}{8} = 1,8750. \end{aligned}$$

Las iteraciones que siguen se generan de forma similar obteniéndose los resultados de la siguiente tabla.

k	0	1	2	3	4	$\dots$	9
$x_1^{(k)}$	0,0000	0,6000	1,0473	0,9326	1,0152		0,9997
$x_2^{(k)}$	0,0000	2,2727	1,7159	2,0533	1,9537		2,0004
$x_3^{(k)}$	0,0000	-1,1000	-0,8052	-1,0493	-0,9681		-1,0009
$x_4^{(k)}$	0,0000	1,8750	0,8852	1,1309	0,9739		1,0006

La decisión de parar el proceso iterativo se puede basar en cualquier criterio que se estime adecuado. En este caso hemos forzado a que la parada se produzca cuando

$$\frac{\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_{\infty}}{\|\mathbf{x}^{(k)}\|_{\infty}} < 10^{-3}.$$

En $k = 9$ se cumple que

$$\frac{\|\mathbf{x}^{(9)} - \mathbf{x}^{(8)}\|_{\infty}}{\|\mathbf{x}^{(9)}\|_{\infty}} = \frac{8,0 \times 10^{-4}}{2,0004} = 0,0003999 < 10^{-3}.$$

La cantidad 10^{-3} se ha considerado suficiente como aproximación a la solución de este ejemplo. ●

El algoritmo que representa el procedimiento iterativo de Jacobi para resolver la ecuación $A\mathbf{x} = \mathbf{b}$, partiendo de un punto inicial $\mathbf{x}^{(0)}$ dado, es el que se describe en la tabla 2.1. Obsérvese que para llegar a la solución sólo es necesario efectuar el producto de una matriz por un vector y restar al resultado otro vector.

Tabla 2.1

Algoritmo de Jacobi para la resolución de $A\mathbf{x} = \mathbf{b}$

```

while  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty} / \|\mathbf{x}^{(k+1)}\|_{\infty} > Tol$  do
  for  $i = 1$  to  $n$ 
    
$$x(i) \leftarrow \frac{1}{a(i,i)} \left( b(i) - \sum_{\substack{j=1 \\ j \neq i}}^n a(i,j)x(j) \right)$$

  end
end
end

```

La codificación en FORTRAN 77 de ese algoritmo para resolver el ejemplo 2.1 es la que sigue.

```

PROGRAM Jacobi
C
  parameter (n = 4)
  real a(n,n),b(n),x(n),y(n),s1,su,sm
C
  data a/10.,-1.,2.,0.,-1.,11.,-1.,3.,2.,-1.,10.,-1.,0.,3.,-1.,8./
  data b/6.,25.,-11.,15./
  data sm/1.0/
C
  x = 0.
C
C *** Proceso iterativo ***
C
  do while (sm.ge.0.001)
    s1 = 0.0
    do i = 1,n
      su = b(i)

```

```

do j = 1,i-1
    su = su-a(i,j)*x(j)
end do
do j = i+1,n
    su = su-a(i,j)*x(j)
end do
y(i) = su/a(i,i)
s1 = amax1(s1,abs(y(i)))
end do
sm = 0.0
do i = 1,n
    sm = amax1(abs(x(i)-y(i))/s1,sm)
    x(i) = y(i)
end do
print *,x                                ! Salida de resultados
end do
C
end

```

Ejemplo 2.2 Resolvamos con el método de Jacobi el sistema

$$\begin{aligned} 10x_1 + x_2 &= 11 \\ 2x_1 + 10x_2 &= 12 \end{aligned}$$

partiendo del punto $\mathbf{x}^{(0)} = [0, 0]^T$. Los puntos que se generan en el proceso iterativo son los de la tabla que sigue.

k	0	1	2	3	4	5
$x_1^{(k)}$	0,0000	1,1000	0,9800	1,0020	0,9996	1,00004
$x_2^{(k)}$	0,0000	1,2000	0,9800	1,0040	0,9996	1,00008

La solución exacta es $[1, 1]^T$.

Resolvamos ahora el sistema

$$\begin{aligned} x_1 + 10x_2 &= 11 \\ 10x_1 + 2x_2 &= 12 \end{aligned}$$

cuya solución es también $[1, 1]^T$. Partiendo de $\mathbf{x}^{(0)} = [0, 0]^T$, los cinco primeros puntos que se generan utilizando el esquema iterativo de Jacobi son esta vez los que recoge la tabla que sigue.

k	0	1	2	3	4	5
$x_1^{(k)}$	0,0000	11	-49	501	-2499	25001
$x_2^{(k)}$	0,0000	6	-49	251	-2499	12501

Los dos sencillos sistemas de este ejemplo nos permiten constatar que la sucesión de puntos que genera el método de Jacobi puede converger a la solución o diverger de ésta. Para poderlo aplicar con garantía, por tanto, se hace necesario definir en qué condiciones converge y se puede aplicar. Volveremos sobre esta cuestión más adelante.

2.2 Método de Gauss-Seidel

En el método de Jacobi cada una de las componentes del vector solución en la iteración $k+1$ se determina a partir de las de la iteración k . En el de Carl Friedrich Gauss (1777-1855) y Phillip Ludwig Seidel (1874) se modifica el de Jacobi utilizando en el cálculo de cada componente de la solución en una iteración el valor de aquellos ya calculados en esa misma iteración. Volviendo al sistema de tres ecuaciones que considerábamos para introducir el método de Jacobi,

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3,\end{aligned}$$

suponiendo una vez más que a_{11} , a_{22} y a_{33} son distintos de cero, el esquema iterativo del método de Gauss-Seidel es el siguiente:

$$\begin{aligned}x_1^{(k+1)} &= \frac{1}{a_{11}} \left(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} \right) \\x_2^{(k+1)} &= \frac{1}{a_{22}} \left(b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)} \right) \\x_3^{(k+1)} &= \frac{1}{a_{33}} \left(b_3 - a_{31}x_1^{(k+1)} - a_{32}x_2^{(k+1)} \right).\end{aligned}$$

Si en el método de Jacobi las relaciones de recurrencia que conformaban su esquema iterativo se obtenían de despejar cada variable en su correspondiente ecuación, en el método de Gauss-Seidel esas relaciones surgen de hacer esto mismo pero de una forma que podríamos denominar escalonada. En efecto, no es difícil comprobar que el método de Gauss-Seidel tiene una relación directa con escribir el sistema original en la forma

$$\begin{aligned}a_{11}x_1 &= b_1 - a_{12}x_2 - a_{13}x_3 \\a_{21}x_1 + a_{22}x_2 &= b_2 - a_{23}x_3 \\a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3.\end{aligned}$$

La relación de recurrencia general para un sistema $n \times n$ es la siguiente:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right); \quad i = 1, \dots, n. \quad (2.6)$$

Si se introducen las matrices

$$E = - \begin{bmatrix} 0 & 0 & \cdots & 0 & 0 \\ a_{21} & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ a_{n-1,1} & a_{n-1,2} & \cdots & 0 & 0 \\ a_{n1} & a_{n2} & \cdots & a_{nn-1} & 0 \end{bmatrix} \quad \text{y} \quad F = - \begin{bmatrix} 0 & a_{12} & \cdots & a_{1,n-1} & a_{1n} \\ 0 & 0 & \cdots & a_{2,n-1} & a_{2n} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & a_{n-1,n} \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix}$$

y volvemos a considerar la descomposición de la matriz del sistema A según el esquema general (2.1), el método iterativo de Gauss-Seidel descompone la matriz A de la forma

$$A = (D - E) - F,$$

donde D es la misma matriz diagonal (2.5) que en el caso del método de Jacobi.

Recordando las expresiones (2.2) y (2.3), el *esquema iterativo del método de Gauss-Seidel* escrito en forma matricial resulta

$$\mathbf{x}^{(k+1)} = (I - (D - E)^{-1}A)\mathbf{x}^{(k)} + (D - E)^{-1}\mathbf{b}. \quad (2.7)$$

La matriz que caracteriza al método es en este caso

$$I - (D - E)^{-1}A.$$

Como $A = (D - E) - F$, la expresión (2.7) también se puede representar de la siguiente forma

$$\begin{aligned} \mathbf{x}^{(k+1)} &= (D - E)^{-1} [(D - E) - A] \mathbf{x}^{(k)} + (D - E)^{-1}\mathbf{b} \\ &= (D - E)^{-1}F\mathbf{x}^{(k)} + (D - E)^{-1}\mathbf{b}. \end{aligned}$$

A la matriz

$$G = (D - E)^{-1}F$$

se la denomina habitualmente *matriz de Gauss-Seidel*.

El algoritmo para resolver la ecuación $A\mathbf{x} = \mathbf{b}$ con el método de Gauss-Seidel, suponiendo que se dan las condiciones para poderlo aplicar que veremos más adelante, es el que se describe en la tabla 2.2.

Tabla 2.2

Algoritmo de Gauss-Seidel para la resolución de $A\mathbf{x} = \mathbf{b}$

```

while  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty} / \|\mathbf{x}^{(k+1)}\|_{\infty} > Tol$  do
  for  $i = 1$  to  $n$ 
    
$$x(i) \leftarrow \frac{1}{a(i,i)} \left( b(i) - \sum_{j=1}^{i-1} a(i,j)x(j) - \sum_{j=i+1}^n a(i,j)x(j) \right)$$

  end
end

```

Ejemplo 2.3 Resolvamos por el método de Gauss-Seidel el sistema lineal de ecuaciones del ejemplo 2.1:

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6 \\ -x_1 + 11x_2 - x_3 + 3x_4 &= 25 \\ 2x_1 - x_2 + 10x_3 - x_4 &= -11 \\ 3x_2 - x_3 + 8x_4 &= 15. \end{aligned}$$

Aplicando la relación general de recurrencia (2.6) a este caso, partiendo del punto inicial

$\mathbf{x}^{(0)} = [0, 0, 0, 0]^T$, se tiene

$$\begin{aligned} x_1^{(1)} &= \frac{1}{10}x_2^{(0)} - \frac{1}{5}x_3^{(0)} + \frac{3}{5} = 0,6000 \\ x_2^{(1)} &= \frac{1}{11}x_1^{(1)} + \frac{1}{11}x_3^{(0)} - \frac{3}{11}x_4^{(0)} + \frac{25}{11} = 2,3273 \\ x_3^{(1)} &= -\frac{1}{5}x_1^{(1)} + \frac{1}{10}x_2^{(1)} + \frac{1}{10}x_4^{(0)} - \frac{11}{10} = -0,9873 \\ x_4^{(1)} &= -\frac{3}{8}x_2^{(1)} + \frac{1}{8}x_3^{(1)} + \frac{15}{8} = 0,8789. \end{aligned}$$

Las iteraciones que siguen se generan de forma similar obteniéndose los resultados de la siguiente tabla.

k	0	1	2	3	4	5
$x_1^{(k)}$	0,0000	0,6000	1,0302	1,0066	1,0009	1,0001
$x_2^{(k)}$	0,0000	2,3273	2,0369	2,0036	2,0003	2,0000
$x_3^{(k)}$	0,0000	-0,9873	-1,0145	-1,0025	-1,0003	-1,0000
$x_4^{(k)}$	0,0000	0,8789	0,9843	0,9984	0,9998	0,9999

El criterio para parar el proceso de este ejemplo es el mismo que el del ejemplo 2.1. Obsérvese que con este método el problema converge a su solución en 5 iteraciones; el de Jacobi lo hacía en 9. ●

La codificación en FORTRAN 77 del algoritmo de la tabla 2.2 para resolver este último ejemplo es la que sigue.

```

PROGRAM GaussSeidel
C
parameter (n = 4)
real a(n,n),b(n),x(n),s1,su,sm,xi
C
data a/10.,-1.,2.,0.,-1.,11.,-1.,3.,2.,-1.,10.,-1.,0.,3.,-1.,8./
data b/6.,25.,-11.,15./
data sm/1.0/
C
x = 0.
C
*** Proceso iterativo ***
C
do while (sm.ge.0.001)
s1 = 0.
sm = 0.
do i = 1,n
su = b(i)
do j = 1,n
su = su-a(i,j)*x(j)
end do
xi = x(i)+su/a(i,i)
sm = amax1(abs(x(i)-xi),sm)
x(i) = xi
s1 = amax1(s1,abs(x(i)))
end do
sm = sm/s1
print *,x
! Salida de resultados

```

```
        end do
c
    end
```

Como se puede observar de la aplicación directa del algoritmo de la tabla 2.2, el método de Gauss-Seidel requiere menos posiciones de memoria que el de Jacobi: en éste existía un vector $y(*)$; aquí, no.

Como es lógico pensar, al tener inmediatamente en cuenta los nuevos componentes del vector solución una vez calculados, el proceso iterativo del método de Gauss-Seidel convergerá más rápidamente que el de Jacobi. La realidad es que esto ocurre así generalmente aunque no siempre.

2.3 Convergencia de los métodos de Jacobi y Gauss-Seidel

2.3.1 Matrices generales

Para que un esquema iterativo que genere una sucesión de vectores construidos mediante una fórmula de recurrencia del tipo

$$\mathbf{x}^{(k+1)} = M\mathbf{x}^{(k)} + \mathbf{c},$$

comenzando con un determinado vector $\mathbf{x}^{(0)}$, sea eficaz, es necesario que la sucesión generada converja a la solución de la ecuación que se desea resolver, cualquiera que sea ese vector inicial $\mathbf{x}^{(0)}$.

Diremos que un esquema como el anterior es convergente si para cualquier vector inicial, $\mathbf{x}^{(0)}$, la sucesión obtenida converge a un mismo límite.

En este apartado estudiaremos las condiciones generales de convergencia para los esquemas iterativos que definen los métodos de Jacobi y Gauss-Seidel. Los resultados son extensibles a cualquier otro esquema. Los teoremas obtenidos dependen de algunos resultados básicos sobre series de matrices que expondremos como paso previo al enunciado y demostración de esos teoremas.

Antes de entrar en ello, recordemos algunos conceptos y resultados de álgebra que nos ayudarán en ese análisis.

El *espectro* de una matriz A , $\Lambda(A)$, lo constituyen el conjunto de soluciones de su ecuación característica, es decir:

$$\Lambda(A) = \{z \in \mathbf{C} : \det(zI - A) = 0\}.$$

El *radio espectral*, $\rho(A)$, de una matriz A de orden n se define como el valor máximo de los módulos de los valores propios de la matriz. En otros términos:

$$\rho(A) = \max_{\lambda_i \in \Lambda(A)} |\lambda_i|.$$

Nótese que el significado geométrico del radio espectral de una matriz, que justifica la denominación, es el de proporcionar el radio del menor círculo del plano complejo centrado en el origen que contiene a todos los valores propios de la matriz A . La noción de radio espectral de una matriz es muy importante en el estudio de la convergencia de los métodos iterativos para la resolución de sistemas de ecuaciones lineales.

Supongamos que consideramos una norma matricial consistente con una cierta norma vectorial. Utilizaremos el mismo signo para ambas normas.

De la definición de valor propio λ de una matriz A , para $\mathbf{x} \neq \mathbf{0}$, se tiene que

$$A\mathbf{x} = \lambda\mathbf{x}.$$

Por consiguiente

$$\|\lambda\mathbf{x}\| = |\lambda|\|\mathbf{x}\| = \|A\mathbf{x}\| \leq \|A\|\|\mathbf{x}\|.$$

De aquí que

$$|\lambda| \leq \|A\|.$$

De este resultado se sigue inmediatamente que

$$\rho(A) \leq \|A\|.$$

Dicho en otros términos, cualquier norma matricial proporciona una cota superior del radio espectral. Basta por tanto que una cierta norma matricial, por ejemplo la inducida por una norma vectorial, sea menor que 1 para que el radio espectral sea necesariamente menor que 1. Veremos la importancia de esta conclusión más adelante.

Las relaciones entre radio espectral y normas matriciales son aún más profundas. De hecho, puede probarse que el radio espectral de una matriz es el ínfimo de los valores que pueden tomar las normas de esa matriz.

Lema 2.1 *Sea T una matriz no singular, la norma vectorial $\|\mathbf{x}\|_T = \|T\mathbf{x}\|_\infty$ y $\|A\|_T = \sup_{\mathbf{x} \neq \mathbf{0}} (\|A\mathbf{x}\|_T / \|\mathbf{x}\|_T)$ su correspondiente norma matricial inducida. Se cumple que:*

a) $\|A\|_T = \|TAT^{-1}\|_\infty.$

b) *Para todo $\epsilon > 0$ y toda matriz A , existe una matriz no singular T tal que*

$$\|A\|_T = \rho(A) + \epsilon.$$

El lema anterior pone de manifiesto que existen normas matriciales arbitrariamente próximas al radio espectral de una matriz pero no implica la existencia de una norma matricial cuyo valor sea justamente el del radio espectral. Así, por ejemplo, la matriz cuadrada de orden 2,

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix},$$

tiene claramente radio espectral nulo ya que sus dos valores propios son iguales a 0. Sin embargo, al ser una matriz no nula, es imposible que una norma matricial cualquiera tome el valor 0 en esa matriz.

La nueva norma matricial introducida toma en A el valor de la norma inicial en la matriz semejante a A , tomando como matriz de semejanza la matriz T . Con esta definición se tiene el siguiente resultado.

Teorema 2.1 *Si el radio espectral de una matriz A es menor que 1, existe una matriz invertible T tal que para la norma euclídea se cumple que*

$$\|A\|_T < 1.$$

Dicho en otras palabras, el teorema afirma que si el radio espectral de una matriz es menor que 1, existe una matriz semejante cuya norma espectral es también menor que 1.

Dado un número complejo z , es sabido que la sucesión formada con sus potencias z^k converge a 0 si y sólo si el módulo de z es estrictamente menor que 1. Un hecho íntimamente ligado con éste es que *la serie geométrica*

$$1 + z + z^2 + \dots$$

es convergente si y sólo si $|z| < 1$. Además, en caso de convergencia, se cumple que

$$1 + z + z^2 + \dots = \frac{1}{1 - z}$$

A continuación generalizaremos estos resultados para la sucesión formada por las potencias de una matriz cuadrada.

Proposición 2.1 *Sea M una matriz cuadrada de números complejos. La sucesión $X^{(k)} = M^k$ de potencias de M converge a la matriz nula si y sólo si el radio espectral de M es estrictamente menor que 1.*

DEMOSTRACIÓN. Probemos primero la necesidad de la condición. Si el radio espectral de M fuese mayor que 1 existiría un valor propio λ de módulo mayor o igual que 1. Sea $\mathbf{v}$ su vector propio asociado. De la igualdad

$$M\mathbf{v} = \lambda\mathbf{v}$$

se deduce que

$$M^k\mathbf{v} = \lambda^k\mathbf{v}.$$

Se tendría así un vector para el que

$$\lim_{k \rightarrow \infty} M^k\mathbf{v} \neq \mathbf{0},$$

lo cual contradice que

$$\lim_{k \rightarrow \infty} M^k = 0.$$

Comprobemos la suficiencia. Según hemos visto con anterioridad, si el radio espectral de M es menor que 1, existe una norma matricial para la que la norma de la matriz es también menor que 1. Como para toda norma matricial se cumple que

$$\|M^k\| \leq \|M\|^k,$$

se sigue que

$$\lim_{k \rightarrow \infty} \|M^k\| \leq \lim_{k \rightarrow \infty} \|M\|^k = 0,$$

lo cual fuerza a que

$$\lim_{k \rightarrow \infty} M^k = 0$$

■

El siguiente resultado es una generalización al caso matricial de la fórmula para la suma de una serie geométrica.

Proposición 2.2 *La serie matricial*

$$I + M + M^2 + \dots$$

es convergente si y sólo si el radio espectral de la matriz M es menor que 1. En este supuesto, su suma es la matriz

$$(I - M)^{-1}.$$

DEMOSTRACIÓN. Para la convergencia de la serie es necesario que

$$\lim_{k \rightarrow \infty} M^k = 0,$$

lo que sólo puede ocurrir si el radio espectral es menor que 1, según acabamos de demostrar. Esto prueba la necesidad de la condición.

Para probar la suficiencia comencemos verificando que si el radio espectral de M es menor que 1, la matriz $I - M$ es invertible. Esto es así porque los valores propios de $I - M$ vienen dados por los números de la forma $1 - \lambda$, donde λ es un valor propio de M . Como al ser el radio espectral de M menor que 1 ninguno de estos números puede ser cero, la matriz es forzosamente invertible. De la identidad matricial

$$(I - M)(I + M + M^2 + \dots + M^n) = I - M^{n+1}$$

se deduce entonces la igualdad

$$(I + M + M^2 + \dots + M^n) = (I - M)^{-1} (I - M^{n+1}).$$

Pasando al límite

$$I + M + M^2 + \dots = (I - M)^{-1},$$

como queríamos demostrar. ■

La proposición anterior permite afrontar el estudio de las condiciones de convergencia de un determinado esquema iterativo para la solución de un sistema de ecuaciones lineales. El resultado fundamental es el que sigue.

Teorema 2.2 *El esquema iterativo*

$$\mathbf{x}^{(k+1)} = M\mathbf{x}^{(k)} + \mathbf{c}$$

es convergente si y sólo si el radio espectral de la matriz M es menor que 1. En ese caso la sucesión $\mathbf{x}^{(k)}$ converge a la solución de la ecuación

$$\mathbf{x} = M\mathbf{x} + \mathbf{c}.$$

DEMOSTRACIÓN. Para que el esquema iterativo converja a un $\mathbf{x}$ que cumpla $\mathbf{x} = M\mathbf{x} + \mathbf{c}$, es necesario y suficiente que el error en cada iteración, $\mathbf{x}^{(k)} - \mathbf{x}$, converja a cero. Ahora bien, recordemos de (2.2) que

$$\mathbf{x}^{(k+1)} = (I - R^{-1}A)\mathbf{x}^{(k)} + R^{-1}\mathbf{b}.$$

El error $\mathbf{x}^{(k+1)} - \mathbf{x}$, $\mathbf{e}^{(k+1)}$, es:

$$\begin{aligned}\mathbf{e}^{(k+1)} &= (I - R^{-1}A)\mathbf{x}^{(k)} + R^{-1}\mathbf{b} - (I - R^{-1}A)\mathbf{x} - R^{-1}\mathbf{b} \\ &= (I - R^{-1}A)(\mathbf{x}^{(k)} - \mathbf{x}) \\ &= M\mathbf{e}^{(k)}.\end{aligned}$$

De aquí que, en general,

$$\mathbf{e}^{(k)} = M^k \mathbf{e}^{(0)}.$$

Para que el error converja a cero pues, de acuerdo con la proposición 2.1, es necesario y suficiente que $\rho(M) < 1$. ■

2.3.2 Matriz de diagonal dominante

Recordemos que se dice que una matriz compleja, $A = (a_{ij})$, cuadrada y de orden n , es de *diagonal estrictamente dominante por filas*, o simplemente de diagonal dominante, cuando cumple que

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, \dots, n.$$

Análogamente se define una matriz de diagonal dominante por columnas. En lo que sigue entenderemos por matriz de diagonal dominante una matriz de diagonal dominante por filas.

Una importante propiedad de las matrices de diagonal dominante la establece el siguiente teorema.

Teorema 2.3 *Toda matriz de diagonal dominante es una matriz invertible.*

DEMOSTRACIÓN. Si la matriz A no fuese invertible, la ecuación $A\mathbf{x} = \mathbf{b}$ admitiría una solución no nula. Si

$$\mathbf{x} = [x_1, \dots, x_n]^T$$

es una solución, podemos suponer sin pérdida de generalidad que

$$\max_{1 \leq i \leq n} |x_i| = 1.$$

Sea r un índice para el que es $|x_r| = 1$. Tomando módulos en la ecuación

$$a_{r1}x_1 + \dots + a_{rr}x_r + \dots + a_{rn}x_n = 0$$

se concluye que

$$\begin{aligned}|a_{rr}| &\leq \sum_{i \neq r} |a_{ri}| |x_i| \\ &\leq \sum_{i \neq r} |a_{ri}|,\end{aligned}$$

desigualdad que contradice la hipótesis de que la matriz A tiene diagonal estrictamente dominante. Esto permite dar por demostrado el teorema. ■

A continuación probaremos que cuando se tiene una ecuación $A\mathbf{x} = \mathbf{b}$ en la que la matriz de coeficientes A es de diagonal estrictamente dominante, al aplicar los métodos de Jacobi y Gauss-Seidel para obtener una solución, éstos convergen.

Teorema 2.4 *Si la matriz A es de diagonal dominante, el método de Jacobi para resolver $A\mathbf{x} = \mathbf{b}$ converge a su solución.*

DEMOSTRACIÓN. Recordemos que si expresamos la matriz de coeficientes del sistema de la forma

$$A = (D - E) - F,$$

la matriz de iteración de Jacobi es

$$J = I - D^{-1}A = D^{-1}(E + F) = D^{-1}E + D^{-1}F$$

y la de Gauss-Seidel,

$$G = (D - E)^{-1}F = (I - D^{-1}E)^{-1}D^{-1}F.$$

Si hacemos

$$L = D^{-1}E \quad \text{y} \quad U = D^{-1}F,$$

se puede escribir que

$$J = L + U \quad \text{y} \quad G = (I - L)^{-1}U.$$

Nótese ahora que el hecho de que la matriz A sea de diagonal dominante supone¹ que

$$\|L + U\|_{\infty} < 1$$

Esto se sigue de la definición de matriz de diagonal dominante la cual se puede escribir

$$\sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|} < 1; \quad i = 1, \dots, n.$$

En notación matricial esto equivale a

$$\|D^{-1}(E + F)\|_{\infty} < 1.$$

La matriz de iteración de Jacobi tiene por tanto norma infinito inferior a 1. Puede entonces asegurarse que la matriz J tiene radio espectral menor que 1 ya que una de sus normas es menor que 1. El método iterativo de Jacobi es por consiguiente convergente en el caso de matriz de coeficientes de diagonal dominante. ■

Proposición 2.3 *Si la matriz A es de diagonal dominante entonces se cumple que*

$$\|G\|_{\infty} \leq \|J\|_{\infty}.$$

¹Recordemos además que la norma matricial $\|\cdot\|_{\infty}$ de una matriz viene dada por el máximo de las sumas de los valores absolutos de los elementos de cada fila de la matriz.

DEMOSTRACIÓN. Si M es una matriz compleja arbitraria, introduzcamos la notación

$$|M|$$

para designar la matriz que tiene por elementos los módulos de los elementos de M . Siendo M y N dos matrices de las mismas dimensiones, la notación

$$|M| < |N|$$

se entenderá como que cada elemento de la matriz M es en módulo menor que el correspondiente elemento de la matriz N .

Es fácil comprobar que si $M = M_1 M_2$,

$$|M| \leq |M_1| |M_2|.$$

Usando este resultado, de la definición de la matriz G , se tiene que

$$|G| \leq |(I - L)^{-1}| |U|.$$

Dado que, como se comprueba fácilmente,

$$L^n = 0,$$

se cumple que

$$(I - L)^{-1} = I + L + L^2 + \cdots + L^{n-1}.$$

Tomando módulos resulta la siguiente desigualdad:

$$|(I - L)^{-1}| \leq I + |L| + |L|^2 + \cdots + |L|^{n-1} = (I - |L|)^{-1}. \quad (2.8)$$

Por otro lado, es claro que

$$|U| = |J| - |L|.$$

Utilizando estos dos resultados podemos escribir:

$$\begin{aligned} |D| &\leq (I - |L|)^{-1} (|J| - |L|) \\ &= (I - |L|)^{-1} [(I - |L|) - (I - |J|)] \\ &= I - (I - |L|)^{-1} (I - |J|). \end{aligned}$$

Hasta ahora no hemos utilizado la propiedad de que A es de diagonal dominante, que sabemos implica que $\|J\|_\infty < 1$. Si introducimos el vector

$$\mathbf{u} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix},$$

podemos reflejar esta propiedad en la desigualdad

$$(I - |J|)\mathbf{u} > 0.$$

La fórmula (2.8) antes obtenida permite escribir que

$$(I - |L|)^{-1} \geq I.$$

Podemos entonces afirmar que

$$(I - |L|)^{-1}(I - |J|)\mathbf{u} \geq (I - |J|)\mathbf{u}$$

y de ahí que

$$\begin{aligned} |G|\mathbf{u} &= \mathbf{u} - (I - |L|)^{-1}(I - |J|)\mathbf{u} \\ &\leq \mathbf{u} - (I - |J|)\mathbf{u} \\ &= |J|\mathbf{u}. \end{aligned}$$

Obviamente esto implica que, en el caso de tratar con una matriz A de diagonal dominante, se cumple lo enunciado: $\|G\|_\infty \leq \|J\|_\infty$. ■

Teorema 2.5 *El método de Gauss-Seidel para resolver $A\mathbf{x} = \mathbf{b}$ converge a su solución para el caso de una matriz de coeficientes de diagonal dominante.*

DEMOSTRACIÓN. Como antes se vio, bajo la hipótesis de que A es de diagonal dominante,

$$\|J\|_\infty < 1$$

y por tanto también se cumple que

$$\|G\|_\infty < 1$$

por lo que el radio espectral de G será también menor que 1. ■

2.3.3 Matriz simétrica definida positiva

En muchos casos de interés se plantea el problema de resolver un sistema de ecuaciones en el que la matriz de coeficientes es una matriz simétrica y definida positiva. En este apartado se estudia la convergencia de los métodos iterativos de Jacobi y de Gauss-Seidel en este supuesto.

Comencemos por el método de Jacobi. La matriz de iteración se puede escribir

$$J = D^{-1}(D - A) = I - D^{-1}A,$$

donde D es la matriz diagonal formada con los elementos de la diagonal principal de la matriz A . Dado que estamos suponiendo que la matriz A es definida positiva, la matriz D también es definida positiva. Designemos con la notación $D^{1/2}$ la matriz diagonal cuyos elementos son las raíces cuadradas de los elementos de la matriz D . Con este convenio podemos escribir que la matriz

$$\begin{aligned} J &= I - D^{-1/2} \left(D^{-1/2} A D^{-1/2} \right) D^{1/2} \\ &= D^{-1/2} \left(I - D^{-1/2} A D^{-1/2} \right) D^{1/2}. \end{aligned}$$

Se aprecia entonces que J es una matriz semejante a la matriz simétrica

$$I - D^{-1/2} A D^{-1/2}$$

y por tanto sus valores propios, que coinciden con los de esta matriz, son números reales. Para que su radio espectral sea menor que 1, esos valores propios han de pertenecer al intervalo abierto $(-1, 1)$. Esto es equivalente a afirmar que las dos matrices

$$I \pm \left(I - D^{-1/2} A D^{-1/2} \right)$$

han de ser definidas positivas. Si tomamos el signo negativo resulta la matriz

$$D^{-1/2}AD^{-1/2}$$

que es congruente con la matriz A , y por tanto también definida positiva. Si se toma el signo positivo, resulta la matriz

$$2I - D^{-1/2}AD^{-1/2} = D^{-1/2}(2D - A)D^{-1/2}$$

que, como se deduce de la expresión del segundo miembro de la igualdad anterior, es congruente con la matriz

$$2D - A$$

y será definida positiva si y sólo si esta matriz lo es. Hemos obtenido así un resultado para caracterizar la convergencia del método de Jacobi para una matriz definida positiva.

Teorema 2.6 *Sea A una matriz simétrica y definida positiva. El método iterativo de Jacobi para un sistema de ecuaciones con matriz de coeficientes A es convergente si y sólo si la matriz*

$$2D - A$$

es una matriz definida positiva.

A diferencia de lo que ocurre con el método de Jacobi, el método de Gauss-Seidel va a ser siempre convergente en el caso de un sistema con matriz definida positiva. La demostración de este resultado es bastante más laboriosa debido a que los valores propios de la matriz G no son, en el caso de matriz simétrica, necesariamente reales, propiedad que sí tienen en las mismas circunstancias los valores propios de la matriz de Jacobi.

Para analizar la convergencia del método de Gauss-Seidel en este caso resulta muy útil el siguiente resultado.

Teorema 2.7 (Stein) *Una matriz compleja M tiene radio espectral menor que 1 si y sólo si existe una matriz hermítica² Q definida positiva tal que la matriz³ dada por*

$$P = Q - M^*QM$$

es también una matriz definida positiva.

DEMOSTRACIÓN. Comencemos probando que la existencia de una tal matriz Q es suficiente para que el radio espectral sea menor que 1. Sea λ un valor propio de M y $\mathbf{v}$ su vector propio asociado. De la igualdad

$$M\mathbf{v} = \lambda\mathbf{v}$$

se sigue que

$$\mathbf{v}^* M^* Q M \mathbf{v} = |\lambda|^2 \mathbf{v}^* Q \mathbf{v}$$

y que

$$\mathbf{v}^* (Q - M^* Q M) \mathbf{v} = (1 - |\lambda|^2) \mathbf{v}^* Q \mathbf{v},$$

introduciendo en ambos miembros $\mathbf{v}^*Q\mathbf{v}$. Como tanto Q como $Q - M^*QM$ son matrices definidas positivas y $\mathbf{v}$ un vector no nulo,

$$(1 - |\lambda|^2) > 0$$

y, por lo tanto,

$$|\lambda| < 1.$$

Así pues, todos los valores propios tienen módulo inferior a 1 y el radio espectral es, por tanto, también menor que 1.

Probemos que la existencia de Q es necesaria. Según sabemos, si el radio espectral de la matriz M es menor que 1, existe una matriz $\hat{M}$, semejante a M , cuya norma espectral es menor que 1. Dicho de otro modo, existe $\hat{M}$ tal que la matriz

$$\hat{P} = I - \hat{M}^*\hat{M},$$

es definida positiva. Si las matrices M y $\hat{M}$ se relacionan de la forma

$$\hat{M} = TMT^{-1}$$

se tiene que

$$\hat{P} = I - (T^*)^{-1}M^*T^*TMT^{-1} = (T^{-1})^*[T^*T - M^*T^*TM]T^{-1}.$$

La matriz

$$P = T^*\hat{P}T = T^*T - M^*T^*TM,$$

por consiguiente, es congruente con una matriz definida positiva, y por tanto, es ella misma definida positiva. Basta tomar

$$Q = T^*T$$

para tener una matriz definida positiva que satisface la condición. ■

Puede suceder que una matriz M tenga radio espectral menor que 1 y que la matriz

$$P = Q - M^*QM$$

no sea definida positiva para una cierta matriz definida positiva Q . Un ejemplo lo proporcionan las matrices

$$M = \begin{bmatrix} 0 & 2 \\ 0 & 0 \end{bmatrix} \quad \text{y} \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Es claro que $\rho(M) = 0$, en tanto que

$$Q - M^*QM = \begin{bmatrix} 1 & 0 \\ 0 & -3 \end{bmatrix},$$

no es una matriz definida positiva. Comprobemos, en cambio, siempre en el supuesto de que el radio espectral de M es menor que 1, que si la matriz P es definida positiva, la matriz Q también ha de ser definida positiva. En efecto, de la relación entre P y Q se obtiene fácilmente que

$$Q - (M^*)^{k+1}QM^{k+1} = P + M^*PM + \cdots + (M^*)^kPM^k.$$

Como el radio espectral de M es menor que 1, las potencias de M convergen a la matriz nula, por lo que al pasar al límite

$$Q = P + M^*PM + (M^*)^2PM^2 + \dots.$$

Siendo P definida positiva, es claro que la suma de esta serie es definida positiva. Por tanto Q resulta ser definida positiva.

Apliquemos el teorema anterior a la siguiente cuestión. Siendo A una matriz real, simétrica y definida positiva, queremos obtener condiciones suficientes para que la descomposición

$$A = R - S$$

conduzca a un esquema iterativo convergente. La matriz R es una matriz invertible no necesariamente simétrica. Según ya sabemos, que el esquema sea convergente equivale a que el radio espectral de la matriz M dada por

$$M = R^{-1}S = I - R^{-1}A,$$

sea menor que 1.

La idea es aplicar el teorema de Stein tomando como matriz hermítica Q la propia matriz A . Con esta motivación, consideremos las igualdades

$$\begin{aligned} A - M^*AM &= A - (I - M^* - I)A(I - M - I) \\ &= A - [(I - M^*)A(I - M) - A(I - M) - (I - M^*)A + A] \\ &= A(I - M) + (I - M^*)A - (I - M^*)A(I - M). \end{aligned}$$

Si utilizamos que

$$I - M = R^{-1}A,$$

podemos afirmar que la matriz $I - M$ es una matriz invertible cuya inversa es

$$(I - M)^{-1} = A^{-1}R.$$

De donde deducimos además que

$$A(I - M)^{-1} = R.$$

De acuerdo con esto, podemos escribir que

$$\begin{aligned} A - M^*AM &= (I - M^*) \left[(I - M^*)^{-1}A + A(I - M)^{-1} - A \right] (I - M) \\ &= (I - M^*) (R^* + R - A) (I - M). \end{aligned}$$

El cálculo anterior hace ver que las matrices

$$A - M^*AM \quad \text{y} \quad R^* + R - A$$

son congruentes. Aplicando el teorema de Stein será suficiente que la segunda de estas matrices sea definida positiva para que el radio espectral de la matriz M sea menor que 1.

Teorema 2.8 Sea A una matriz real, simétrica y definida positiva, y sea R una matriz invertible tal que la matriz

$$R^* + R - A$$

sea definida positiva. El radio espectral de la matriz

$$M = I - R^{-1}A$$

es entonces menor que 1.

Este teorema puede aplicarse para desarrollar una nueva demostración de la condición de convergencia del método de Jacobi con matriz definida positiva.

A continuación pasamos a aplicarlo para demostrar la convergencia del método de Gauss-Seidel.

Teorema 2.9 El método iterativo de Gauss-Seidel es convergente para todo sistema de ecuaciones cuya matriz de coeficientes es simétrica definida positiva.

DEMOSTRACIÓN. Con el teorema anterior la demostración es muy sencilla. Si A es una matriz simétrica que se descompone según

$$A = D - E - E^T$$

y la matriz de iteración del método de Gauss-Seidel es

$$G = (D - E)^{-1}E^T,$$

aplicando el teorema anterior, tomando

$$R = D - E,$$

se tiene que

$$R^T + R - A = (D - E)^T + (D - E) - (D - E - E^T) = D.$$

Como la matriz A es definida positiva, todos los elementos de su diagonal principal son positivos, lo que equivale a decir que la matriz diagonal D es definida positiva. ■

2.4 Métodos de relajación

Los dos métodos iterativos que hemos estudiado hasta ahora, Jacobi y Gauss-Seidel, se pueden generalizar. En efecto, las relaciones de recurrencia de estos dos métodos se pueden escribir de la forma

$$x_i^{(k+1)} = x_i^{(k)} + r_i^{(k)}, \quad i = 1, \dots, n.$$

En el caso del método de Jacobi,

$$r_i^{(k)} = \frac{b_i - \sum_{j=1}^n a_{ij}x_j^{(k)}}{a_{ii}};$$

en el de Gauss-Seidel,

$$r_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i}^n a_{ij}x_j^{(k)}}{a_{ii}}.$$

Visto así, estos dos procedimientos iterativos llegan a la solución a través de un número de pasos, en cada uno de los cuales se avanza una cantidad $\mathbf{r}^{(k)}$.

La idea de los métodos de relajación consiste, en cada iteración, en aplicar la relación de recurrencia,

$$x_i^{(k+1)} = x_i^{(k)} + \omega r_i^{(k)}, \quad i = 1, \dots, n,$$

de tal forma que se mejoren las prestaciones del procedimiento avanzando un paso más amplio, $\omega > 1$, o más corto, $\omega < 1$. Al parámetro ω se le conoce como *parámetro de relajación*.

El método de relajación más conocido es el SOR, *Successive Overrelaxation*: resulta de aplicar esta idea sobre la base del método de Gauss-Seidel. Su relación de recurrencia es:

$$x_i^{(k+1)} = \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) + (1 - \omega)x_i^{(k)}, \quad i = 1, \dots, n.$$

Una elección adecuada del valor de ω puede mejorar la convergencia del método. La idea que debe dirigir esa elección es que si la corrección que introduce cada iteración en la solución es excesiva, se puede disminuir con un factor $\omega < 1$. Por el contrario, si la corrección tiende a quedarse corta, se puede aumentar con un factor $\omega > 1$.

La elección del parámetro ω plantea dos problemas: en primer lugar, que ha de estudiarse el conjunto de valores del parámetro que hacen que el método SOR converja; en segundo, que hay que determinar el valor del parámetro que haga que la convergencia sea lo más rápida posible.

Si la matriz de coeficientes del sistema de ecuaciones lineales que hay que resolver se representa como antes de la forma

$$A = D - E - F,$$

el *esquema iterativo del método SOR* en forma matricial es el que sigue:

$$\mathbf{x}^{(k+1)} = (D - \omega E)^{-1} ((1 - \omega)D + \omega F) \mathbf{x}^{(k)} + \omega (D - \omega E)^{-1} \mathbf{b}.$$

La matriz que caracteriza la iteración del método y, por tanto, su convergencia es:

$$G(\omega) = (D - \omega E)^{-1} ((1 - \omega)D + \omega F).$$

Si introducimos las matrices

$$L = D^{-1}E \quad \text{y} \quad U = D^{-1}F,$$

aún podemos escribir que esa matriz característica es

$$G(\omega) = (I - \omega L)^{-1} [(1 - \omega)I + \omega U].$$

Tabla 2.3Algoritmo de relajación SOR para la resolución de $A\mathbf{x} = \mathbf{b}$

```

while  $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_\infty / \|\mathbf{x}^{(k+1)}\|_\infty > Tol$  do
  for  $i = 1$  to  $n$ 
     $x(i) \leftarrow \frac{\omega}{a(i,i)} \left( b(i) - \sum_{j=1}^{i-1} a(i,j)x(j) - \sum_{j=i+1}^n a(i,j)x(j) \right) + (1 - \omega)x(i)$ 
  end
end

```

El algoritmo para resolver la ecuación $A\mathbf{x} = \mathbf{b}$ con el método SOR es el que se describe en la tabla 2.3.

Ejemplo 2.4 Resolvamos por el método SOR el sistema de ecuaciones lineales

$$\begin{aligned} 4x_1 + 3x_2 &= 24 \\ 3x_1 + 4x_2 - x_3 &= 30 \\ -x_2 + 4x_3 &= -24. \end{aligned}$$

Aplicando la relación general de recurrencia del método, partiendo del punto inicial $\mathbf{x}^{(0)} = [1, 1, 1]^T$, con $\omega = 1,25$, se obtienen los resultados de la siguiente tabla.

k	0	1	2	3	4	5	6	7
$x_1^{(k)}$	1,0000	6,3125	2,6223	3,1333	2,9570	3,0037	2,9963	3,0000
$x_2^{(k)}$	1,0000	3,5195	3,9585	4,0102	4,0074	4,0029	4,0009	4,0002
$x_3^{(k)}$	1,0000	-6,6501	-4,6004	-5,0966	-4,9734	-5,0057	-4,9982	-5,0003

Por el contrario, partiendo del mismo punto pero esta vez con $\omega = 2,25$, los resultados que se obtienen son los de la siguiente tabla.

k	0	1	2	3	4	5	6	7
$x_1^{(k)}$	1,0000	10,5625	3,0588	1,3328	-10,8367	12,2136	10,9919	18,5938
$x_2^{(k)}$	1,0000	-1,6367	4,9442	13,4344	8,7895	-7,5608	-11,1607	11,9961
$x_3^{(k)}$	1,0000	-15,6706	8,8695	-17,0300	12,7316	-33,6674	22,3064	-34,6352

Como se puede ver, en este caso la solución diverge. ●

La codificación en FORTRAN 77 del algoritmo de la tabla 2.3 para resolver este último ejemplo es la que sigue.

```

PROGRAM Sor
C
  parameter (n = 3)
  real a(n,n),b(n),x(n),s1,su,sm,xi,w
C

```

```

data a/4.,3.,0.,3.,4.,-1.,0.,-1.,4./
data b/24.,30.,-24./
data sm/1.0/
C
  x = 1.
  w = 1.25
C
C *** Proceso iterativo ***
C
  do while (sm.ge.0.001)
    s1 = 0.
    sm = 0.
    do i = 1,n
      su = b(i)
      do j = 1,i-1
        su = su-a(i,j)*x(j)
      end do
      do j = i+1,n
        su = su-a(i,j)*x(j)
      end do
      xi = (1-w)*x(i)+w*su/a(i,i)
      sm = amax1(abs(x(i)-xi),sm)
      x(i) = xi
      s1 = amax1(s1,abs(x(i)))
    end do
    sm = sm/s1
    print *,x
  end do
C
end

```

2.4.1 Convergencia del método SOR

Para estudiar la convergencia del método hay que analizar, como ya sabemos, el radio espectral de la matriz $G(\omega)$.

Teorema 2.10 (Kahan) *Para toda matriz A , el radio espectral de la matriz $G(\omega)$ del método de relajación SOR satisface la desigualdad*

$$\rho(G(\omega)) \geq |\omega - 1|.$$

DEMOSTRACIÓN. Utilizaremos que el determinante de una matriz es el producto de los valores propios de la matriz. Para la matriz $G(\omega)$ se tiene que

$$\begin{aligned} \det(G(\omega)) &= \det\left((I - \omega L)^{-1}[(1 - \omega)I + \omega U]\right) \\ &= \det(I - \omega L)^{-1} \det[(1 - \omega)I + \omega U]. \end{aligned}$$

Siendo L una matriz triangular inferior con ceros en la diagonal principal,

$$\det(I - \omega L) = 1$$

y por tanto también será 1 el determinante de la matriz inversa de $I - \omega L$. La matriz $(1 - \omega)I + \omega U$ es a su vez una matriz triangular superior y los elementos de su diagonal principal son todos iguales a $1 - \omega$. Así pues,

$$\det [(1 - \omega)I + \omega U] = (1 - \omega)^n.$$

Se tiene por consiguiente, para el producto de los valores propios de la matriz $G(\omega)$, la expresión

$$\prod_{k=1}^n \lambda_k = (1 - \omega)^n.$$

De la definición de radio espectral de una matriz se tiene obviamente que

$$\rho(G(\omega)) \geq |\lambda_k|, \quad k = 1, \dots, n,$$

de manera que tomando módulos en la fórmula anterior se llega a la desigualdad

$$\rho(G(\omega))^n \geq \prod_{k=1}^n |\lambda_k| = |\omega - 1|.$$

Sin más que tomar raíces n -ésimas resulta la desigualdad dada en el enunciado del teorema. ■

Corolario 2.1 *Una condición necesaria para la convergencia del método de relajación SOR es que el parámetro de relajación ω cumpla las desigualdades*

$$0 < \omega < 2.$$

DEMOSTRACIÓN. La prueba es una aplicación directa del teorema anterior. En efecto, si el método es convergente se cumple que $\rho(G(\omega)) < 1$. Utilizando la desigualdad que se establece en el teorema, el valor de ω debe cumplir que

$$|\omega - 1| < 1,$$

siendo ω un parámetro real. Esto es equivalente a decir que

$$-1 < \omega - 1 < 1$$

o bien que $\omega \in (0, 2)$. ■

Pasemos a estudiar condiciones suficientes de convergencia. Mencionaremos sin demostración el resultado de que si la matriz de coeficientes del sistema es de diagonal dominante, el método de relajación SOR converge si $\omega \in (0, 1)$. Esta afirmación prolonga el resultado anteriormente obtenido para el método de Gauss-Seidel con matriz de diagonal dominante; su demostración se puede hacer del mismo modo que se hizo entonces.

Mayor interés tiene el hecho de que cuando se tiene una matriz simétrica definida positiva, el método de relajación SOR converge para todos los valores permisibles del parámetro ω , es decir para $\omega \in (0, 2)$. En este caso la condición que antes se dio como necesaria se convierte en suficiente.

Teorema 2.11 (*Ostrowski-Reich*) Para un sistema de ecuaciones con matriz simétrica y definida positiva, el método iterativo de relajación SOR converge si y sólo si el parámetro de relajación cumple que $0 < \omega < 2$.

DEMOSTRACIÓN. Nótese que el teorema extiende aquel que garantizaba la convergencia de Gauss-Seidel en este mismo caso de matriz definida positiva. Su demostración se basa también en el teorema de Stein. Si

$$A = D - E - E^T,$$

la matriz de iteración $G(\omega)$ corresponde, como sabemos, a la descomposición

$$A = \left(\frac{1}{\omega} D - E \right) + \left(\left(1 - \frac{1}{\omega} \right) D - E^T \right).$$

Tomando

$$R(\omega) = \frac{1}{\omega} D - E$$

se tiene que

$$\begin{aligned} R + R^T - A &= \frac{2}{\omega} D - (E + E^T) - A \\ &= \left(\frac{2}{\omega} - 1 \right) D. \end{aligned}$$

Para $\omega \in (0, 2)$ se cumple que

$$\frac{2}{\omega} - 1 > 0$$

y al ser todos los elementos de la diagonal de una matriz definida positiva números positivos, es de aplicación el teorema de Stein. ■

Queda por estudiar la cuestión de la selección del valor del parámetro de relajación de forma que la velocidad de convergencia sea máxima. Esta cuestión es en general difícil y no existen teoremas generales aplicables para la determinación del parámetro óptimo. En ciertos casos particulares —como es el caso que habitualmente se analiza en la literatura referido a estos métodos para matrices tridiagonales— se dispone de resultados parciales que lo facilitan.

2.4.2 Método SSOR

Una modificación interesante de los métodos de relajación la constituye el *método simétrico SOR*. La idea del mismo consiste en realizar cada una de las iteraciones del SOR en dos pasos: uno siguiendo el mismo orden de obtención de los componentes del vector $\mathbf{x}$ y otro calculándolos en sentido inverso. Analicemos qué ocurre si se aplica esta idea directamente al método de Gauss-Seidel. El primer paso de cada iteración lo define la fórmula de recurrencia

$$\mathbf{x}^{(k+1/2)} = (D - E)^{-1} F \mathbf{x}^{(k)} + (D - E)^{-1} \mathbf{b}.$$

El segundo paso determina el valor de $\mathbf{x}^{(k+1)}$ de acuerdo con la siguiente expresión:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij} x_j^{(k+1)} - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1/2)} \right); \quad i = n, n-1, \dots, 1.$$

Utilizando las matrices D , E y F , esta última expresión se puede escribir

$$D\mathbf{x}^{(k+1)} = F\mathbf{x}^{(k+1)} + E\mathbf{x}^{(k+1/2)} + \mathbf{b}$$

o

$$\mathbf{x}^{(k+1)} = (D - F)^{-1}E\mathbf{x}^{(k+1/2)} + (D - F)^{-1}\mathbf{b}.$$

Es decir, en este paso el papel de las matrices E y F se invierte. Una iteración del método SGS (*Symmetric Gauss-Seidel*) combinaría los dos pasos descritos. El esquema iterativo del método en forma matricial es

$$\mathbf{x}^{(k+1)} = (D - F)^{-1}E(D - E)^{-1}F\mathbf{x}^{(k)} + \hat{\mathbf{d}}$$

donde

$$\hat{\mathbf{d}} = (D - F)^{-1}E(D - E)^{-1}\mathbf{b} + (D - F)^{-1}\mathbf{b}.$$

El método SSOR simplemente inserta el parámetro de relajación ω en los dos pasos de cada iteración. Su *esquema iterativo* en forma matricial es

$$\mathbf{x}^{(k+1)} = (D - \omega F)^{-1}[(1 - \omega)D + \omega E](D - \omega E)^{-1}[(1 - \omega)D + \omega F]\mathbf{x}^{(k)} + \hat{\mathbf{d}}$$

donde ahora,

$$\hat{\mathbf{d}} = \omega(D - \omega F)^{-1}\{[(1 - \omega)D + \omega E](D - \omega E)^{-1} + I\}\mathbf{b}.$$

Esta última expresión sólo pretende representar en forma matricial una iteración del método: no quiere decir que haya que calcular su resultado en cada iteración. Desde el punto de vista de los cálculos, el método SSOR se lleva a efecto mediante fórmulas de recurrencia análogas a las vistas con anterioridad.

Una variante del teorema 2.11 de Ostrowski-Reich permite concluir que el método SSOR, si A es simétrica definida positiva y $\omega \in (0, 2)$, converge para cualquier $\mathbf{x}^{(0)}$.

Debido a que el número de iteraciones requeridas en cada iteración del método SSOR es el doble de las que necesita el método SOR, su utilización como tal método iterativo no está muy extendida. Su ventaja fundamental radica en que los autovalores de su matriz de iteración son reales por lo que se utiliza en combinación con otros métodos para acelerar la convergencia.

2.5 Métodos de minimización

La idea clave de estos métodos para resolver el problema $A\mathbf{x} = \mathbf{b}$ se basa en el siguiente resultado.

Teorema 2.12 Sea A una matriz simétrica definida positiva. La solución $\bar{\mathbf{x}}$ de la ecuación $A\mathbf{x} = \mathbf{b}$ es el vector para el cual la forma cuadrática

$$Q(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T \mathbf{x} \quad (2.9)$$

alcanza su mínimo. Este mínimo es:

$$-\frac{1}{2}\mathbf{b}^T A^{-1}\mathbf{b}.$$

DEMOSTRACIÓN. Por el teorema de Taylor, ver apéndice A en la página 695, desarrollando la expresión (2.9), se tiene que

$$Q(\bar{\mathbf{x}} + \mathbf{d}) - Q(\bar{\mathbf{x}}) = \frac{1}{2}\mathbf{d}^T A\mathbf{d} + O(\|\mathbf{d}\|^2).$$

Como la matriz A es definida positiva, para cualquier $\mathbf{d} \neq \mathbf{0}$, $1/2\mathbf{d}^T A\mathbf{d} > 0$. Como también, para pequeñas $\|\mathbf{d}\|$, el segundo término de la derecha es positivo, $Q(\bar{\mathbf{x}} + \mathbf{d}) - Q(\bar{\mathbf{x}}) > 0$. Es decir, el mínimo $\bar{\mathbf{x}}$ es único.

Por las condiciones de mínimo de una función como $Q(\mathbf{x})$, ver apéndice A, página 695, $\bar{\mathbf{x}}$ debe cumplir que

$$\nabla Q(\bar{\mathbf{x}}) = \mathbf{g}(\bar{\mathbf{x}}) = A\bar{\mathbf{x}} - \mathbf{b} = \mathbf{0}.$$

Es decir, será la solución de $A\mathbf{x} = \mathbf{b}$.

El óptimo (mínimo) de $Q(\mathbf{x})$ es

$$Q(\bar{\mathbf{x}}) = \frac{1}{2}\bar{\mathbf{x}}^T A\bar{\mathbf{x}} - \bar{\mathbf{x}}^T A^T \bar{\mathbf{x}} = \frac{1}{2}\bar{\mathbf{x}}^T A\bar{\mathbf{x}} - \bar{\mathbf{x}}^T A\bar{\mathbf{x}} = -\frac{1}{2}\bar{\mathbf{x}}^T A\bar{\mathbf{x}} = -\frac{1}{2}\mathbf{b}^T A^{-1}\mathbf{b}.$$

■

De acuerdo con este resultado, es posible utilizar cualquiera de los métodos que existen para minimizar funciones como $Q(\mathbf{x})$ y resolver así $A\mathbf{x} = \mathbf{b}$. Muchos de esos métodos se basan en un *esquema iterativo de descenso* del tipo

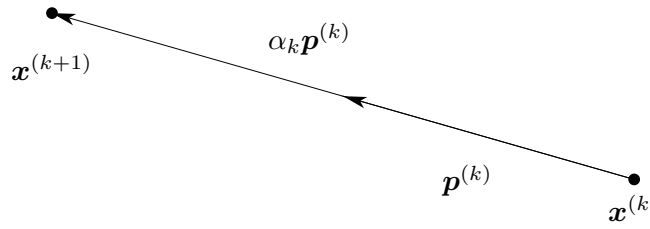
$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}, \quad k = 0, 1, \dots \quad (2.10)$$

donde $\mathbf{p}^{(k)}$ es un vector *dirección de descenso* y el escalar α_k (*factor de avance*) determina la distancia que se desplaza el punto a lo largo de $\mathbf{p}^{(k)}$ (ver figura 2.1).

Existen gran variedad de métodos de descenso; se diferencian unos de otros en la forma de calcular la dirección $\mathbf{p}^{(k)}$.

Por lo que respecta al factor α_k , la forma quizás más natural de calcularlo es aquella que minimiza $Q(\mathbf{x})$ a lo largo de $\mathbf{p}^{(k)}$. Es decir

$$Q(\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}) = \min_{\alpha} Q(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}). \quad (2.11)$$

**Figura 2.1**

Movimiento a lo largo de un vector dirección de descenso

Para unos $\mathbf{x}^{(k)}$ y $\mathbf{p}^{(k)}$ fijos, (2.11) es un problema de optimización en una sola variable, denominado *cálculo de la amplitud de paso*,⁴ pudiendo resolverse explícitamente. Si para facilitar la notación suprimimos los superíndices, se tiene que

$$\begin{aligned} q(\alpha) &= Q(\mathbf{x} + \alpha \mathbf{p}) = \frac{1}{2}(\mathbf{x} + \alpha \mathbf{p})^T A(\mathbf{x} + \alpha \mathbf{p}) - \mathbf{b}^T(\mathbf{x} + \alpha \mathbf{p}) \\ &= \frac{1}{2}\mathbf{x}^T A\mathbf{x} + \alpha \mathbf{p}^T A\mathbf{x} + \frac{1}{2}\alpha^2 \mathbf{p}^T A\mathbf{p} - \alpha \mathbf{p}^T \mathbf{b} - \mathbf{b}^T \mathbf{x} \\ &= \frac{1}{2}\mathbf{p}^T A\mathbf{p}\alpha^2 + \mathbf{p}^T (A\mathbf{x} - \mathbf{b})\alpha + \frac{1}{2}\mathbf{x}^T (A\mathbf{x} - 2\mathbf{b}). \end{aligned} \quad (2.12)$$

Como hemos supuesto que la matriz A es definida positiva, $\mathbf{p}^T A\mathbf{p} > 0$. De acuerdo con las condiciones de existencia de un mínimo, la forma cuadrática $q(\alpha)$ se minimiza cuando $q'(\alpha) = 0$. Es decir, en términos de $\mathbf{x}^{(k)}$ y $\mathbf{p}^{(k)}$, cuando

$$\alpha_k = \frac{(\mathbf{p}^{(k)})^T (\mathbf{b} - A\mathbf{x}^{(k)})}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}}.$$

El problema del cálculo de la amplitud de paso (a veces también denominado determinación del factor de avance) se ilustra en la figura 2.2.

2.5.1 Direcciones de descenso

2.5.1.1 Relajación en una variable

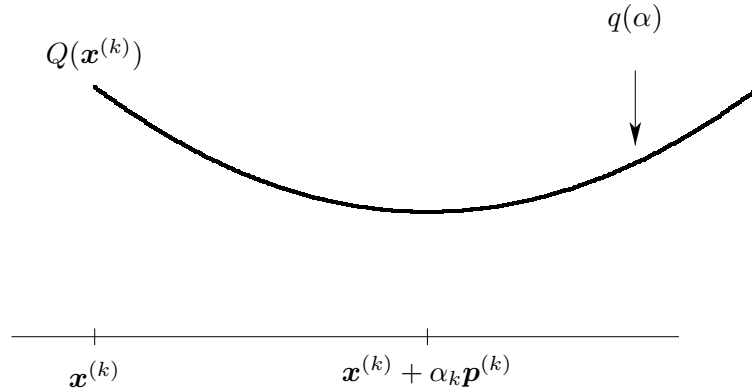
Sea el vector unitario $\mathbf{e}_i$ (cuyo componente i es uno y todos los demás cero). Uno de los procedimientos de minimización más sencillos de búsqueda a lo largo de una dirección es el que hace que las sucesivas direcciones sean los vectores $\mathbf{e}_1, \dots, \mathbf{e}_n, \mathbf{e}_1, \dots$. Es decir,

$$\mathbf{p}^{(0)} = \mathbf{e}_1, \quad \mathbf{p}^{(1)} = \mathbf{e}_2, \quad \dots, \quad \mathbf{p}^{(n-1)} = \mathbf{e}_n, \quad \mathbf{p}^{(n)} = \mathbf{e}_1, \dots$$

Obsérvese que en este caso $\mathbf{e}_i^T A\mathbf{e}_i = a_{ii}$ y que

$$\mathbf{e}_i^T (A\mathbf{x} - \mathbf{b}) = \sum_{j=1}^n a_{ij}x_j - b_i.$$

⁴También de búsqueda lineal o *linesearch*.

**Figura 2.2**Minimización en la variable α

Si en una iteración k , $\mathbf{p}^{(k)} = \mathbf{e}_i$ y α_k se elige con el criterio de minimización en una variable de (2.12), el siguiente punto del proceso iterativo, $\mathbf{x}^{(k+1)}$, estará dado por

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{e}_i = \mathbf{x}^{(k)} - \frac{1}{a_{ii}} \left(\sum_{j=1}^n a_{ij} x_j^{(k)} - b_i \right) \mathbf{e}_i. \quad (2.13)$$

En esta última expresión los vectores $\mathbf{x}^{(k+1)}$ y $\mathbf{x}^{(k)}$ difieren solamente en su componente i . En efecto, (2.13) es equivalente a minimizar la función Q de (2.9) sólo en la componente i -ésima del vector $\mathbf{x}^{(k)}$, permaneciendo las demás en los valores que tomaban en $\mathbf{x}^{(k)}$.

Consideremos ahora los n primeros pasos de este proceso y escribamos sólo los componentes que cambian (obsérvese que $x_j^{(k)} = x_j^{(0)}$ hasta que varía el componente j -ésimo):

$$\begin{aligned} x_i^{(i)} &= x_i^{(0)} + \alpha_i = x_i^{(0)} - \frac{1}{a_{ii}} \left(\sum_{j=1}^{i-1} a_{ij} x_j^{(j)} + \sum_{j=i}^n a_{ij} x_j^{(0)} - b_i \right) \\ &= \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(j)} - \sum_{j=i+1}^n a_{ij} x_j^{(0)} \right); \quad i = 1, \dots, n. \end{aligned}$$

Esta expresión es la misma que la de recurrencia del método de Gauss-Seidel. El método de Gauss-Seidel, por tanto, es equivalente a combinar n pasos sucesivos de relajación univariable, llamando al producto de esos pasos el siguiente punto del proceso iterativo de Gauss-Seidel.

2.5.1.2 Relajación SOR

A cualquier método de dirección de descenso del tipo $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}$ se le puede añadir un parámetro de relajación ω de tal forma que se escoja como amplitud de paso

$$\alpha_k = \omega \hat{\alpha}_k,$$

donde $\hat{\alpha}_k$ es el valor de α que minimiza la función cuadrática Q de (2.9) a lo largo de la dirección $\mathbf{p}^{(k)}$.

El valor de la función $Q(\mathbf{x}^{(k)} + \omega \hat{\alpha}_k \mathbf{p}^{(k)})$ es menor que $Q(\mathbf{x}^{(k)})$, siempre y cuando $\omega > 0$, hasta que se alcanza un determinado ω para el cual $Q(\mathbf{x}^{(k)} + \omega \hat{\alpha}_k \mathbf{p}^{(k)}) = Q(\mathbf{x}^{(k)})$. Por la simetría existente en una función cuadrática en una variable respecto al eje que pasa por su mínimo, este valor de ω es 2 (ver figura 2.3). En consecuencia, $Q(\mathbf{x}^{(k+1)}) < Q(\mathbf{x}^{(k)})$ si $0 < \omega < 2$; en cualquier otro caso, $Q(\mathbf{x}^{(k+1)}) \geq Q(\mathbf{x}^{(k)})$. Recordemos en este punto el teorema 2.11 de Ostrowski-Reich, base de la demostración de la convergencia del método de relajación SOR.

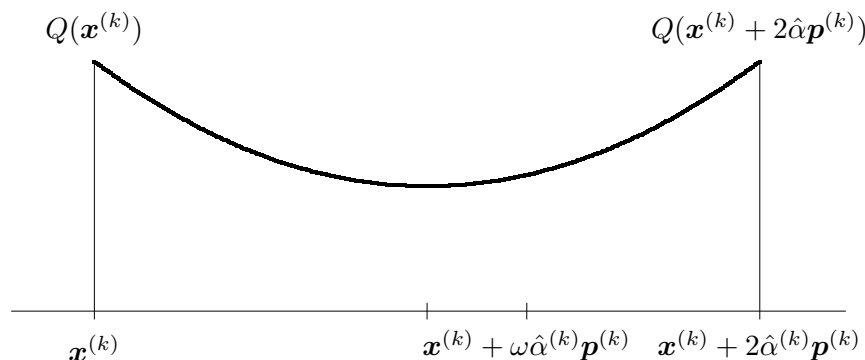


Figura 2.3
Relajación SOR

2.5.1.3 Máxima pendiente

Sea f una función, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, continua y diferenciable en algún conjunto abierto de $\mathbb{R}^n$. Consideremos la aproximación de esta función en un punto $\mathbf{x}^{(k)}$ que se deriva del desarrollo en serie de Taylor truncado en primer orden,

$$f(\mathbf{x}^{(k)} + \mathbf{p}) \approx f(\mathbf{x}^{(k)}) + (\mathbf{g}^{(k)})^T \mathbf{p},$$

donde $\mathbf{g}^{(k)}$ es el gradiente de la función en $\mathbf{x}^{(k)}$: ∇f . Si de lo que se trata, alrededor de $\mathbf{x}^{(k)}$, es determinar una dirección $\mathbf{p}^{(k)}$ a lo largo de la cual la función decrece lo más posible, la idea es hacer el término

$$(\mathbf{g}^{(k)})^T \mathbf{p} \tag{2.14}$$

lo más grande y negativo posible. A tal efecto es obvio que habrá que normalizar $\mathbf{p}$ de alguna manera pues si no, para cualquier $\bar{\mathbf{p}}$ tal que

$$(\mathbf{g}^{(k)})^T \bar{\mathbf{p}} < 0,$$

se podría escoger $\mathbf{p}$ igual a un múltiplo muy grande de $\bar{\mathbf{p}}$ y bastaría. Se trata, sin embargo, de encontrar la $\mathbf{p}^{(k)}$ que cualitativamente más minimiza (2.14). Es decir, dada cualquier norma

$\|\cdot\|$, $\mathbf{p}^{(k)}$ será la solución al problema:

$$\underset{\mathbf{p} \in \mathbb{R}^n}{\text{minimizar}} \frac{\mathbf{g}^T \mathbf{p}}{\|\mathbf{p}\|}.$$

La solución de este problema depende de la norma que se elija. Si se define por una matriz A simétrica definida positiva, es decir,

$$\|\mathbf{p}\|_A = \sqrt{\langle \mathbf{p} | A \mathbf{p} \rangle},$$

la solución al problema de minimización es:

$$\mathbf{p}^{(k)} = -A^{-1} \mathbf{g}^{(k)}.$$

Si se determina a partir de la norma euclídea, $\|\mathbf{p}\| = \sqrt{\langle \mathbf{p} | \mathbf{p} \rangle}$, apoyándose en la desigualdad de Cauchy-Schwarz, que para cualesquiera dos vectores $\mathbf{x}$ e $\mathbf{y}$ obedece a la expresión

$$\|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \leq \mathbf{x}^T \mathbf{y} \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2,$$

y que cuando $\mathbf{y} = -\mathbf{x}$ se verifica la igualdad en el límite inferior, la solución es entonces el negativo del gradiente:

$$\mathbf{p}^{(k)} = -\mathbf{g}^{(k)}.$$

A esta dirección se la conoce como la de la *máxima pendiente*.

Volviendo al problema de búsqueda de direcciones que minimicen la función cuadrática $Q(\mathbf{x}^{(k)})$, la dirección de máxima pendiente en $\mathbf{x}^{(k)}$ es

$$\mathbf{p}^{(k)} = -\nabla Q(\mathbf{x}^{(k)}) = -\left(A\mathbf{x}^{(k)} - \mathbf{b}\right). \quad (2.15)$$

El procedimiento iterativo que en cada nuevo punto del proceso utiliza esta dirección de búsqueda se denomina *método de la máxima pendiente* o *método de Cauchy*, pues fue el ingeniero francés Augustin Louis Cauchy (1789-1857) quien lo introdujo en 1847.

En el caso de la función cuadrática $Q(\mathbf{x})$, la fórmula de recurrencia que se obtiene aplicando este método es

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha_k \left(A\mathbf{x}^{(k)} - \mathbf{b}\right). \quad (2.16)$$

El valor de α_k que minimiza $Q(\mathbf{x})$ a lo largo de $-\nabla Q(\mathbf{x}^{(k)})$ es

$$\alpha_k = \frac{\left(\mathbf{g}^{(k)}\right)^T \mathbf{g}^{(k)}}{\left(\mathbf{g}^{(k)}\right)^T A \mathbf{g}^{(k)}}.$$

El gran inconveniente del método de Cauchy es su lenta convergencia en cierto tipo de funciones. En efecto, como en general,

$$Q'_{\alpha_k} \left(\mathbf{x}^{(k)} + \alpha_k \mathbf{g}^{(k)}\right) = \left(\mathbf{g}^{(k)}\right)^T \nabla Q \left(\mathbf{x}^{(k)} + \alpha_k \mathbf{g}^{(k)}\right) = \left(\mathbf{g}^{(k)}\right)^T \nabla Q \left(\mathbf{x}^{(k+1)}\right) = 0,$$

de donde se deduce que

$$\left(\mathbf{p}^{(k)}\right)^T \mathbf{p}^{(k+1)} = 0,$$

el número de iteraciones que serán necesarias para minimizar una $Q(\mathbf{x})$ con una matriz A mal condicionada será elevado. Como ejemplo, en la figura 2.4 se describe el proceso iterativo que este método seguiría para alcanzar el mínimo de una función cuadrática relativamente mal condicionada. Como se puede ver, el procedimiento atraviesa el valle que define la función de un lado al otro del mismo, en vez de ir directamente por el valle como sería lo ideal. Este hecho lo confirma el estudio de la velocidad de convergencia del método. Está definida por la desigualdad

$$Q\left(\mathbf{x}^{(k)}\right) + \frac{1}{2}\mathbf{b}^T A \mathbf{b} \leq \left(1 - \frac{1}{\kappa_2(A)}\right) \left(Q\left(\mathbf{x}^{(k-1)}\right) + \frac{1}{2}\mathbf{b}^T A \mathbf{b}\right).$$

Cuanto más grande sea $\kappa_2(A)$, más lenta será la convergencia del método.

En la tabla 2.4 se describe el algoritmo de la máxima pendiente para resolver el sistema de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$.

Para probar la efectividad del método se ha construido artificialmente un problema forzando a que la matriz, además de ser simétrica y definida positiva, esté mal condicionada. La matriz A proviene de multiplicar una matriz de Hilbert 50×50 por su traspuesta. La respuesta del problema se ha forzado que sea $\mathbf{x} = \mathbf{1}$. Para evitar que el número de condición de A sea muy malo, se ha mejorado sumando a algunos elementos de la diagonal principal un uno. El número de condición resultante es aproximadamente 5×10^4 .

La codificación del método en FORTRAN 77 para resolver este problema es la que sigue a continuación. Para conseguir llegar a la solución el programa necesita 112 iteraciones.

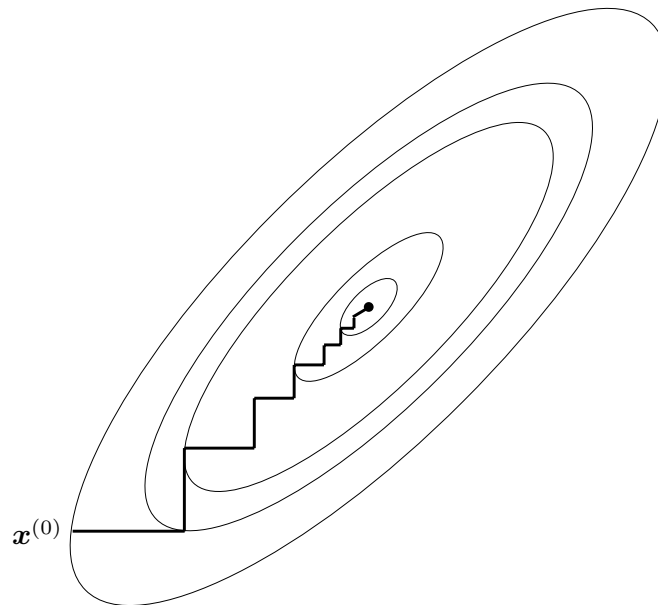


Figura 2.4

Proceso de convergencia del método de la máxima pendiente aplicado a una función cuadrática

Tabla 2.4

Algoritmo de la máxima pendiente para resolver $A\mathbf{x} = \mathbf{b}$

```

 $\mathbf{x}^{(0)} \leftarrow \mathbf{0}; \mathbf{r}^{(0)} \leftarrow \mathbf{b}; k = 1$ 
while  $\|\mathbf{r}^{(k)}\|_2 > \varepsilon \|\mathbf{b}\|_2$  do
     $\alpha_k \leftarrow (\mathbf{r}^{(k-1)})^T \mathbf{r}^{(k-1)} / (\mathbf{r}^{(k-1)})^T A \mathbf{r}^{(k-1)}$ 
     $\mathbf{x}^{(k)} \leftarrow \mathbf{x}^{(k-1)} + \alpha_k \mathbf{r}^{(k-1)}$ 
     $\mathbf{r}^{(k)} \leftarrow \mathbf{b} - A\mathbf{x}^{(k)}$ 
     $k \leftarrow k + 1$ 
end

```

```

PROGRAM Steep
C
parameter (n=50)
double precision a(n,n),b(n),x(n),r(n),prod,ro1,xnormb,rar
C
open(10,file='stp.dat')
C
read (10,*) a,b
C
x      = 0.0
r      = b
ro1    = prod(r,r,n)
C
C *** Proceso iterativo ***
C
xnormb = epsilon(1.0)*dsqrt(ro1)*5
k      = 1
do while (dsqrt(ro1).gt.xnormb)
    rar = 0.0
    do i = 1,n
        rar = rar+r(i)*prod(a(1,i),r,n)
    end do
    x = x+(ro1/rar)*r
    do i = 1,n
        r(i) = b(i)-prod(a(1,i),x,n)
    end do
    ro1 = prod(r,r,n)
    k = k+1
    print *,k,ro1                                ! Resultados de iteraciones
end do
C
print '(10f8.5)',x                                ! Solución
end

double precision function prod (x,y,n)
C
double precision x(n),y(n)
C
prod = 0.0
do i = 1,n

```

```

        prod = prod+x(i)*y(i)
    end do
C
    return
end

```

Obsérvese que si $\alpha_k = 1$, el esquema iterativo del método de la máxima pendiente coincide con el de Richardson. También, que si A es diagonal con todos sus elementos iguales a uno y $\alpha_k = 1$, la fórmula (2.16) define una iteración del método de Jacobi.

2.5.2 Direcciones de descenso conjugadas

Una importante clase de métodos de minimización es aquella en que se consideran como direcciones de descenso vectores $\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(n-1)}$ que cumplen

$$\left(\mathbf{p}^{(i)}\right)^T A \mathbf{p}^{(j)} = 0, \quad i \neq j. \quad (2.17)$$

Tales vectores son ortogonales con respecto al producto interior $\langle \mathbf{x} | A \mathbf{y} \rangle = \mathbf{x}^T A \mathbf{y}$, definido por la matriz A , diciéndose de ellos que son *A ortogonales*. También se dicen *conjugados* respecto a A .

Proposición 2.4 Si A es una matriz simétrica definida positiva y el conjunto de vectores $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(k)}$ son *A ortogonales*, o *conjugados* con respecto a A , esos vectores son linealmente independientes.

DEMOSTRACIÓN. Supongamos que existen constantes α_i , $i = 0, 1, 2, \dots, k$, tales que

$$\alpha_0 \mathbf{p}^{(0)} + \dots + \alpha_k \mathbf{p}^{(k)} = \mathbf{0}.$$

Multiplicando por A y efectuando el producto escalar por $\mathbf{p}^{(i)}$,

$$\alpha_i \left(\mathbf{p}^{(i)}\right)^T A \mathbf{p}^{(i)} = 0.$$

Ahora bien, dado que $\left(\mathbf{p}^{(i)}\right)^T A \mathbf{p}^{(i)} > 0$, pues A es definida positiva, se debe cumplir que $\alpha_i = 0$. ■

Teorema 2.13 (*Teorema de las Direcciones Conjugadas*) Si A es una matriz simétrica definida positiva y $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(n-1)}$ son conjugados con respecto a A , para todo $\mathbf{x}^{(0)} \in \mathbb{R}^n$, la sucesión de vectores que define la expresión

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}, \quad (2.18)$$

donde

$$\alpha_k = \frac{(\mathbf{b} - A\mathbf{x}^{(k)})^T \mathbf{p}^{(k)}}{(\mathbf{p}^{(k)})^T A \mathbf{p}^{(k)}},$$

obtenida según el principio de minimización, converge a la solución exacta de $A\mathbf{x} = \mathbf{b}$ en no más de n iteraciones.

DEMOSTRACIÓN. Como los $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(n-1)}$ son linealmente independientes, cualquier vector en $\mathbb{R}^n$ se puede expresar como combinación lineal de ellos. En particular, se puede escribir que

$$\hat{\mathbf{x}} - \mathbf{x}^{(0)} = \alpha_0 \mathbf{p}^{(0)} + \alpha_1 \mathbf{p}^{(1)} + \dots + \alpha_{n-1} \mathbf{p}^{(n-1)},$$

para un conjunto $\alpha_0, \alpha_1, \dots$ ($\hat{\mathbf{x}}$ designa la solución exacta de $A\mathbf{x} = \mathbf{b}$). Si se multiplica por A y efectúa el producto escalar por $\mathbf{p}^{(k)}$, se obtiene que

$$\alpha_k = \frac{(\mathbf{p}^{(k)})^T A (\hat{\mathbf{x}} - \mathbf{x}^{(0)})}{(\mathbf{p}^{(k)})^T A \mathbf{p}^{(k)}}. \quad (2.19)$$

Siguiendo el proceso iterativo (2.18) desde $\mathbf{x}^{(0)}$ hasta $\mathbf{x}^{(k)}$, se llegará a que

$$\mathbf{x}^{(k)} - \mathbf{x}^{(0)} = \alpha_0 \mathbf{p}^{(0)} + \alpha_1 \mathbf{p}^{(1)} + \dots + \alpha_{k-1} \mathbf{p}^{(k-1)}.$$

Al ser los vectores $\mathbf{p}$ conjugados con respecto a A , se cumple que

$$(\mathbf{p}^{(k)})^T A (\mathbf{x}^{(k)} - \mathbf{x}^{(0)}) = 0.$$

Sustituyendo el valor de $(\mathbf{p}^{(k)})^T A \mathbf{x}^{(0)}$ que resulta de esta última expresión en (2.19) se obtiene la expresión de α_k :

$$\alpha_k = \frac{(\mathbf{p}^{(k)})^T A (\hat{\mathbf{x}} - \mathbf{x}^{(k)})}{(\mathbf{p}^{(k)})^T A \mathbf{p}^{(k)}} = \frac{(\mathbf{p}^{(k)})^T A \hat{\mathbf{x}} - (\mathbf{p}^{(k)})^T A \mathbf{x}^{(k)}}{(\mathbf{p}^{(k)})^T A \mathbf{p}^{(k)}} = \frac{(\mathbf{b} - A\mathbf{x}^{(k)})^T \mathbf{p}^{(k)}}{(\mathbf{p}^{(k)})^T A \mathbf{p}^{(k)}}.$$

■

Este teorema garantiza no sólo que las iteraciones convergen sino que, en ausencia de errores de redondeo, lo hacen en un número finito de ellas. Es decir, teóricamente, los métodos de minimización basados en el cálculo de direcciones conjugadas son métodos directos aunque se usen como iterativos.

Si la matriz A es diagonal con todos sus elementos positivos, las direcciones $\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n$ son direcciones conjugadas con respecto a esa matriz por lo que, en este caso, la relajación univariable es equivalente al método de minimización basado en el cálculo de direcciones conjugadas.

Si P es la matriz cuyas columnas son los vectores $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(n-1)}$, la propiedad (2.17) es equivalente a $P^T A P = D$, donde D es una matriz diagonal. Haciendo el cambio de variable $\mathbf{x} = P\mathbf{y}$, la función cuadrática $Q(\mathbf{x})$ de (2.9) queda

$$\frac{1}{2}(P\mathbf{y})^T A P\mathbf{y} - \mathbf{b}^T P\mathbf{y} = \frac{1}{2}\mathbf{y}^T D\mathbf{y} - (P^T \mathbf{b})^T \mathbf{y}$$

por lo que el método basado en el cálculo de direcciones conjugadas en las variables originales $\mathbf{x}$ es equivalente a una relajación univariable en las variables $\mathbf{y}$. La función $Q(\mathbf{x})$ alcanzará su mínimo en, como máximo, n pasos.

El teorema y las consideraciones anteriores permiten concluir que el método basado en el cálculo de direcciones conjugadas puede ser adecuado para obtener la solución del problema en un número de pasos conveniente. Ahora bien, ¿cómo se pueden determinar esas direcciones conjugadas?

2.5.2.1 Determinación de direcciones conjugadas

Un método inmediato sería elegir los vectores propios de la matriz A . Si $\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^n$ son esos vectores propios, con valores propios correspondientes $\lambda_1, \lambda_2, \dots, \lambda_n$, se cumple que

$$(\mathbf{x}^i)^T A \mathbf{x}^j = \lambda_j (\mathbf{x}^i)^T \mathbf{x}^j = 0, \quad i \neq j.$$

El problema está en que encontrar los vectores propios de A es una labor incluso más complicada que resolver el sistema $A\mathbf{x} = \mathbf{b}$.

Otra posibilidad sería ortogonalizar un conjunto de vectores linealmente independientes $\mathbf{y}^1, \mathbf{y}^2, \dots, \mathbf{y}^n$ con respecto al producto interior $\langle \mathbf{x} | A\mathbf{y} \rangle = \mathbf{x}^T A\mathbf{y}$. Esta también es una tarea complicada y que implica muchas operaciones.

2.5.2.2 Determinación de direcciones conjugadas. Método de los gradientes conjugados

La forma más eficaz de obtener un conjunto de direcciones conjugadas para resolver la ecuación $A\mathbf{x} = \mathbf{b}$ la constituye el *método de los gradientes conjugados*. Éste genera una sucesión de direcciones tratando de que sean próximas a la de máxima pendiente en cada punto del proceso, que de acuerdo con (2.15) es el negativo del gradiente, $-\mathbf{g} = -(A\mathbf{x} - \mathbf{b}) = \mathbf{b} - A\mathbf{x} = \mathbf{r}$, y que se cumpla la condición de ser conjugadas. Si se elige $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$, y un punto cualquiera como $\mathbf{x}^{(0)}$, a continuación las direcciones, $\mathbf{p}^{(k)}$, serán las que definan una combinación de $\mathbf{r}^{(k)}$ y direcciones previas $\mathbf{p}^{(k-1)}$, es decir $\mathbf{p}^{(k)} = \mathbf{r}^{(k)} + \beta_k \mathbf{p}^{(k-1)}$, de tal forma que se cumpla la

condición de que $\mathbf{p}^{(k)T} A \mathbf{p}^{(k-1)} = 0$. Para que esto sea así, el parámetro β_k ha de ser:

$$\beta_k = - \left(\mathbf{p}^{(k-1)} \right)^T A \mathbf{r}^{(k)} / \left(\mathbf{p}^{(k-1)} \right)^T A \mathbf{p}^{(k-1)}.$$

Además, $\mathbf{r}^{(k)} = \mathbf{b} - A \mathbf{x}^{(k)} = \mathbf{b} - A(\mathbf{x}^{(k-1)} - \alpha_k \mathbf{p}^{(k-1)}) = \mathbf{r}^{(k-1)} + \alpha_k A \mathbf{p}^{(k-1)}$. Ordenando los cálculos, el esquema básico de recurrencia de cada punto del proceso es el que sigue.

Escoger un $\mathbf{x}^{(0)}$. Hacer $\mathbf{p}^{(0)} = \mathbf{r}^{(0)} = \mathbf{b} - A \mathbf{x}^{(0)}$
for $k = 0, 1, \dots$
 $\alpha_k = \left(\mathbf{r}^{(k)} \right)^T \mathbf{p}^{(k)} / \left(\mathbf{p}^{(k)} \right)^T A \mathbf{p}^{(k)}$
 $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}$
 $\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} + \alpha_k A \mathbf{p}^{(k)}$
 $\beta_k = - \left(\mathbf{p}^{(k)} \right)^T A \mathbf{r}^{(k+1)} / \left(\mathbf{p}^{(k)} \right)^T A \mathbf{p}^{(k)}$
 $\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}$
end

(2.20)

En el algoritmo que se deriva de este esquema, el primer paso será idéntico al de máxima pendiente. Como se puede observar, las únicas operaciones que requiere este esquema son productos de matrices por vectores y de vectores entre sí.

Para verificar que este procedimiento es un algoritmo de direcciones conjugadas, verifiquemos que los vectores $\mathbf{p}^{(k)}$ que se generan en el proceso son A ortogonales.

Teorema 2.14 *Sea A una matriz $n \times n$ simétrica definida positiva y $\hat{\mathbf{x}}$ la solución de la ecuación $A \mathbf{x} = \mathbf{b}$. Los vectores $\mathbf{p}^{(k)}$ generados por el algoritmo de los gradientes conjugados (2.20) satisfacen:*

$$\left(\mathbf{p}^{(k)} \right)^T A \mathbf{p}^{(j)} = 0, \quad 0 \leq j < k, \quad k = 1, \dots, n-1, \quad (2.21)$$

siendo $\mathbf{p}^{(k)} \neq \mathbf{0}$ a menos que sea $\mathbf{x}^{(k)} = \hat{\mathbf{x}}$. De esta manera, $\mathbf{x}^{(m)} = \hat{\mathbf{x}}$ para algún $m \leq n$.

DEMOSTRACIÓN. De las definiciones de α_k , β_k , $\mathbf{r}^{(k+1)}$ y $\mathbf{p}^{(k+1)}$ en (2.20) se tiene que

$$\left(\mathbf{p}^{(j)} \right)^T \mathbf{r}^{(j+1)} = \left(\mathbf{p}^{(j)} \right)^T \mathbf{r}^{(j)} + \alpha_j \left(\mathbf{p}^{(j)} \right)^T A \mathbf{p}^{(j)} = 0, \quad j = 0, 1, \dots \quad (2.22a)$$

y que

$$\left(\mathbf{p}^{(j)} \right)^T A \mathbf{p}^{(j+1)} = \left(\mathbf{p}^{(j)} \right)^T A \mathbf{r}^{(j+1)} + \beta_j \left(\mathbf{p}^{(j)} \right)^T A \mathbf{p}^{(j)} = 0, \quad j = 0, 1, \dots \quad (2.22b)$$

Además de la relación de la expresión (2.21), probaremos que los vectores residuo $\mathbf{r}^{(j)} = \mathbf{b} - A \mathbf{x}^{(j)}$ satisfacen

$$\left(\mathbf{r}^{(k)} \right)^T \mathbf{r}^{(j)} = 0, \quad 0 \leq j < k, \quad k = 1, \dots, n-1. \quad (2.23)$$

Es decir, son ortogonales.

Supongamos por inducción que las igualdades de las expresiones (2.21) y (2.23) se cumplen para algún $k < n - 1$. Probemos que se cumplen para $k + 1$.

Como $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$, se cumplen para $k = 1$. Para todo $j < k$, de las definiciones de $\mathbf{r}^{(k+1)}$ y $\mathbf{p}^{(k+1)}$ de (2.20), se cumple que

$$\begin{aligned} \left(\mathbf{r}^{(j)}\right)^T \mathbf{r}^{(k+1)} &= \left(\mathbf{r}^{(j)}\right)^T \left(\mathbf{r}^{(k)} + \alpha_k A \mathbf{p}^{(k)}\right) = \left(\mathbf{r}^{(j)}\right)^T \mathbf{r}^{(k)} + \alpha_k \left(\mathbf{p}^{(k)}\right)^T A \mathbf{r}^{(j)} \\ &= \left(\mathbf{r}^{(j)}\right)^T \mathbf{r}^{(k)} + \alpha_k \left(\mathbf{p}^{(k)}\right)^T A \left(\mathbf{p}^{(j)} - \beta_{j-1} \mathbf{p}^{(j-1)}\right) = 0 \end{aligned}$$

pues, por la hipótesis de inducción, los tres sumandos son cero. Más aun, usando la definición de $\mathbf{p}^{(k+1)}$, α_k y $\mathbf{r}^{(k+1)}$ de (2.20),

$$\begin{aligned} \left(\mathbf{r}^{(k)}\right)^T \mathbf{r}^{(k+1)} &= \left(\mathbf{p}^{(k)} - \beta_{k-1} \mathbf{p}^{(k-1)}\right)^T \mathbf{r}^{(k+1)} \\ &= -\beta_{k-1} \left(\mathbf{p}^{(k-1)}\right)^T \mathbf{r}^{(k+1)} = -\beta_{k-1} \left(\mathbf{p}^{(k-1)}\right)^T \left(\mathbf{r}^{(k)} + \alpha_k A \mathbf{p}^{(k)}\right) = 0 \end{aligned}$$

pues los últimos dos sumandos son cero de acuerdo con (2.22). Con esto hemos probado que (2.23) se cumple para $k + 1$.

Para todo $j < k$, de la definición de $\mathbf{p}^{(k+1)}$ y $\mathbf{r}^{(k+1)}$ de (2.20), por la hipótesis de inducción y de (2.21) se tiene que

$$\begin{aligned} \left(\mathbf{p}^{(j)}\right)^T A \mathbf{p}^{(k+1)} &= \left(\mathbf{p}^{(j)}\right)^T A \left(\mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}\right) = \left(\mathbf{p}^{(j)}\right)^T A \mathbf{r}^{(k+1)} \\ &= \alpha_j^{-1} \left(\mathbf{r}^{(j+1)} - \mathbf{r}^{(j)}\right)^T \mathbf{r}^{(k+1)} = 0. \end{aligned}$$

Obsérvese que hemos supuesto que $\alpha_j \neq 0$; volveremos sobre este asunto más adelante.

Como por (2.22b) se tiene que

$$\left(\mathbf{p}^{(k)}\right)^T A \mathbf{p}^{(k+1)} = 0,$$

se concluye que la expresión (2.21) también se cumple para $k + 1$ completándose así el razonamiento inductivo.

Probemos a continuación que los vectores $\mathbf{p}^{(k)}$ son distintos de cero a no ser que se haya llegado a la solución. Para ello, supongamos que $\mathbf{p}^{(m)} = 0$ para algún $m < n$. De la definición de $\mathbf{p}^{(k+1)}$ de (2.20) se tiene que

$$\begin{aligned} 0 &= \left(\mathbf{p}^{(m)}\right)^T \mathbf{p}^{(m)} = \left(\mathbf{r}^{(m)} + \beta_{m-1} \mathbf{p}^{(m-1)}\right)^T \left(\mathbf{r}^{(m)} + \beta_{m-1} \mathbf{p}^{(m-1)}\right) \\ &= \left(\mathbf{r}^{(m)}\right)^T \mathbf{r}^{(m)} + 2\beta_{m-1} \left(\mathbf{r}^{(m)}\right)^T \mathbf{p}^{(m-1)} + \beta_{m-1}^2 \left(\mathbf{p}^{(m-1)}\right)^T \mathbf{p}^{(m-1)} \geq \left(\mathbf{r}^{(m)}\right)^T \mathbf{r}^{(m)} \end{aligned}$$

dado que

$$\left(\mathbf{r}^{(m)}\right)^T \mathbf{p}^{(m-1)} = 0,$$

de acuerdo con (2.22a). De aquí que $\mathbf{r}^{(m)} = \mathbf{b} - A\mathbf{x}^{(m)} = 0$, por lo que $\mathbf{x}^{(m)} = \hat{\mathbf{x}}$. Por otro lado, si los vectores $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(n-1)}$ son todos distintos del cero, por el teorema 2.13 de las direcciones conjugadas, se cumple que $\mathbf{x}^{(n)} = \hat{\mathbf{x}}$.

Por último, volvamos sobre el supuesto mencionado anteriormente de que $\alpha_j \neq 0$. De la definición de $\mathbf{p}^{(k+1)}$ en (2.20) se tiene que

$$\left(\mathbf{r}^{(j)}\right)^T \mathbf{p}^{(j)} = \left(\mathbf{r}^{(j)}\right)^T \left(\mathbf{r}^{(j)} + \beta_{j-1} \mathbf{p}^{(j-1)}\right) = \left(\mathbf{r}^{(j)}\right)^T \mathbf{r}^{(j)}. \quad (2.24)$$

De aquí y de acuerdo con la definición de α_j en (2.20), ésta se puede expresar como

$$\alpha_j = \frac{\left(\mathbf{r}^{(j)}\right)^T \mathbf{r}^{(j)}}{\left(\mathbf{p}^{(j)}\right)^T A \mathbf{p}^{(j)}}. \quad (2.25)$$

Por consiguiente, si $\alpha_j = 0$, $\mathbf{r}^{(j)} = 0$ y, como antes, $\mathbf{x}^{(j)} = \hat{\mathbf{x}}$, por lo que el proceso se pararía en $\mathbf{x}_j$. ■

De las definiciones de $\mathbf{x}^{(k+1)}$, $\mathbf{r}^{(k+1)}$ y $\mathbf{p}^{(k+1)}$ en (2.20) y de (2.23) se cumple que

$$\begin{aligned} \left(\mathbf{r}^{(k+1)}\right)^T \mathbf{p}^{(k+1)} &= \left(\mathbf{r}^{(k)} + \alpha_k A \mathbf{p}^{(k)}\right)^T \mathbf{p}^{(k+1)} = \left(\mathbf{r}^{(k)}\right)^T \mathbf{p}^{(k+1)} \\ &= \left(\mathbf{r}^{(k)}\right)^T \left(\mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}\right) = \beta_k \left(\mathbf{r}^{(k)}\right)^T \mathbf{p}^{(k)}. \end{aligned}$$

De esta manera, usando (2.24), β_k se puede expresar de esta otra forma:

$$\beta_k = \frac{\left(\mathbf{r}^{(k+1)}\right)^T \mathbf{p}^{(k+1)}}{\left(\mathbf{r}^{(k)}\right)^T \mathbf{p}^{(k)}} = \frac{\left(\mathbf{r}^{(k+1)}\right)^T \mathbf{r}^{(k+1)}}{\left(\mathbf{r}^{(k)}\right)^T \mathbf{r}^{(k)}}. \quad (2.26)$$

Las nuevas definiciones de α y β de las expresiones (2.25) y (2.26) son las que se usan habitualmente en los cálculos de los algoritmos que implementan el método de los gradientes conjugados. Las de (2.20) se emplean con fines teóricos.

2.5.2.2.1 Convergencia

El siguiente resultado es fundamental para determinar la velocidad de convergencia del método de los gradientes conjugados. Para formularlo, introducimos la notación $[\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(k)}]$ para designar el subespacio generado por los vectores $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(k)}$.

Teorema 2.15 Sean $\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(n-1)}$ los vectores de dirección que genera el algoritmo de los gradientes conjugados y $\mathbf{r}^{(0)}, \mathbf{r}^{(1)}, \dots, \mathbf{r}^{(n-1)}$ los vectores residuo. Se cumple que

$$A\mathbf{p}^{(i)} \in [\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(i+1)}], \quad i = 0, \dots, n-2; \quad (2.27)$$

$$\mathbf{r}^{(i)} \in [\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(i)}], \quad i = 0, \dots, n-1; \quad (2.28)$$

$$\begin{aligned} [\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(i)}] &= [\mathbf{p}^{(0)}, A\mathbf{p}^{(0)}, \dots, A^i\mathbf{p}^{(0)}] \\ &= [\mathbf{r}^{(0)}, A\mathbf{r}^{(0)}, \dots, A^i\mathbf{r}^{(0)}], \quad i = 0, \dots, n-1. \end{aligned} \quad (2.29)$$

DEMOSTRACIÓN. Probaremos las expresiones (2.27) y (2.28) por inducción. De la definición de $\mathbf{r}^{(k+1)}$ y $\mathbf{p}^{(k+1)}$ en (2.20) se tiene que

$$\mathbf{p}^{(1)} = \mathbf{r}^{(1)} + \beta_0\mathbf{p}^{(0)} = \mathbf{r}^{(0)} + \alpha_0 A\mathbf{p}^{(0)} + \beta_0\mathbf{p}^{(0)}.$$

De aquí que, como $\mathbf{r}^{(0)} = \mathbf{p}^{(0)}$,

$$A\mathbf{p}^{(0)} = \alpha_0^{-1} (\mathbf{p}^{(1)} - \mathbf{p}^{(0)} - \beta_0\mathbf{p}^{(0)}),$$

por lo que se cumple (2.27) para $i = 0$. Como $\mathbf{r}^{(0)} = \mathbf{p}^{(0)}$, la expresión (2.28) también se cumple para $i = 0$.

Supongamos ahora que lo expresado en (2.27) y (2.28) se cumple para $i = 0, \dots, k < n-2$. De la definición de $\mathbf{r}^{(k+1)}$ de (2.20) y de la hipótesis de inducción,

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} + \alpha_k A\mathbf{p}^{(k)} = \sum_{j=0}^k \nu_j \mathbf{p}^{(j)} + \alpha_k \sum_{j=0}^{k+1} \eta_j \mathbf{p}^{(j)}.$$

De aquí que $\mathbf{r}^{(k+1)} \in [\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k+1)}]$. De la definición de $\mathbf{r}^{(k+1)}$ y $\mathbf{p}^{(k+1)}$ en (2.20) se llega a que

$$\mathbf{p}^{(k+2)} = \mathbf{r}^{(k+2)} + \beta_{k+1}\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \alpha_{k+1} A\mathbf{p}^{(k+1)} + \beta_{k+1}\mathbf{p}^{(k+1)}.$$

Como $\mathbf{r}^{(k+1)} \in [\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k+1)}]$, entonces $A\mathbf{p}^{(k+1)} \in [\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k+2)}]$. Con esto se completa la inducción salvo en lo que respecta a que (2.28) también se cumple para $\mathbf{r}^{(n-1)}$; ésto, no obstante, se deduce de la misma manera.

Para probar la expresión (2.29) usamos inducción una vez más. Es evidente que se cumple para $i = 0$ puesto que $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$. Supongamos que se cumple para $k < n-1$. Por la hipótesis de inducción y la expresión (2.27) se tiene que

$$A^{k+1}\mathbf{p}^{(0)} = A(A^k\mathbf{p}^{(0)}) = A\left(\sum_{j=0}^k \nu_j \mathbf{p}^{(j)}\right) = \sum_{j=0}^k \nu_j A\mathbf{p}^{(j)} \in [\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k+1)}].$$

Para cualquier η_j se verifica que

$$\sum_{j=0}^{k+1} \eta_j A^j \mathbf{p}^{(0)} = \eta_{k+1} A^{k+1} \mathbf{p}^{(0)} + \sum_{j=0}^k \eta_j A^j \mathbf{p}^{(0)}.$$

El primer sumando de la derecha de esta última expresión pertenece a $[\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k+1)}]$, de acuerdo con la ecuación anterior; el segundo también por la hipótesis de inducción. De esta manera hemos probado que

$$[\mathbf{p}^{(0)}, A\mathbf{p}^{(0)}, \dots, A^{k+1}\mathbf{p}^{(0)}] \subset [\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(k+1)}].$$

Para probar la inclusión opuesta primero escribamos

$$\sum_{j=0}^{k+1} \eta_j \mathbf{p}^{(j)} = \eta_{k+1} \mathbf{p}^{(k+1)} + \sum_{j=0}^k \eta_j \mathbf{p}^{(j)}.$$

El segundo término de esta expresión pertenece a $[\mathbf{p}^{(0)}, \dots, A^k \mathbf{p}^{(0)}]$ por la hipótesis de inducción; el primero, de acuerdo con la definición de $\mathbf{r}^{(k+1)}$ y $\mathbf{p}^{(k+1)}$ en (2.20), se puede escribir como

$$\eta_{k+1} \mathbf{p}^{(k+1)} = \eta_{k+1} (\mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}) = \eta_{k+1} (\mathbf{r}^{(k)} + \alpha_k A\mathbf{p}^{(k)} + \beta_k \mathbf{p}^{(k)}).$$

Los términos primero y tercero de la expresión entre paréntesis del miembro de la derecha de esta ecuación pertenecen a $[\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k)}]$, de acuerdo con (2.28), y por lo tanto a $[\mathbf{p}^{(0)}, \dots, A^k \mathbf{p}^{(k)}]$, por la hipótesis de inducción. Usando (2.27) y la hipótesis de inducción, $A\mathbf{p}^{(k)}$ se puede escribir

$$A\mathbf{p}^{(k)} = A \sum_{j=1}^k \nu_j A^j \mathbf{p}^{(0)} = \sum_{j=1}^k \nu_j A^{j+1} \mathbf{p}^{(0)},$$

por lo que pertenece a $[\mathbf{p}^{(0)}, \dots, A^{k+1} \mathbf{p}^{(0)}]$. De esta manera hemos probado que

$$[\mathbf{p}^{(0)}, \dots, \mathbf{p}^{(k+1)}] \subset [\mathbf{p}^{(0)}, \dots, A^{k+1} \mathbf{p}^{(0)}]$$

y, por consiguiente, la primera igualdad de (2.29). La segunda es trivial puesto que $\mathbf{r}^{(0)} = \mathbf{p}^{(0)}$. ■

Un subespacio de la forma $[\mathbf{p}^{(0)}, \dots, A^i \mathbf{p}^{(0)}]$ se denomina *subespacio de Krylov*.

Teorema 2.16 *El punto obtenido en la iteración k del método de los gradientes conjugados cumple que*

$$\|\hat{\mathbf{x}} - \mathbf{x}^{(k)}\|_A \leq 2\alpha^k \|\hat{\mathbf{x}} - \mathbf{x}^{(0)}\|_A \quad (2.30)$$

y

$$\|\hat{\mathbf{x}} - \mathbf{x}^{(k)}\|_2 \leq 2\sqrt{\kappa_2} \alpha^k \|\hat{\mathbf{x}} - \mathbf{x}^{(0)}\|_2,$$

donde $\|\mathbf{x}\|_A = \mathbf{x}^T A \mathbf{x}$, $\alpha = (\sqrt{\kappa_2} - 1)/(\sqrt{\kappa_2} + 1)$ y κ_2 es el número de condición (número de condición 2) de A asociado a $\|\cdot\|_2$ de A .

Este resultado es una consecuencia del siguiente teorema.

Teorema 2.17 *Los puntos obtenidos en las iteraciones del método de los gradientes conjugados cumplen*

$$\|\hat{\mathbf{x}} - \mathbf{x}^{(k)}\|_2 < \|\hat{\mathbf{x}} - \mathbf{x}^{(k-1)}\|_2$$

a menos que $\mathbf{x}^{(k-1)} = \hat{\mathbf{x}}$.

DEMOSTRACIÓN. Observemos primero que

$$\begin{aligned} \|\hat{\mathbf{x}} - \mathbf{x}^{(k-1)}\|_2^2 &= (\hat{\mathbf{x}} - \mathbf{x}^{(k)} + \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)})^T (\hat{\mathbf{x}} - \mathbf{x}^{(k)} + \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}) \\ &= \langle \hat{\mathbf{x}} - \mathbf{x}^{(k)} | \hat{\mathbf{x}} - \mathbf{x}^{(k)} \rangle + 2 \langle \hat{\mathbf{x}} - \mathbf{x}^{(k)} | \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)} \rangle \\ &\quad + \langle \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)} | \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)} \rangle. \end{aligned}$$

o que

$$\|\hat{\mathbf{x}} - \mathbf{x}^{(k-1)}\|_2^2 = \|\hat{\mathbf{x}} - \mathbf{x}^{(k)}\|_2^2 + 2 \langle \hat{\mathbf{x}} - \mathbf{x}^{(k)} | \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)} \rangle + \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_2^2. \quad (2.31)$$

La última cantidad del término de la derecha de esta ecuación es positiva a no ser que $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)}$. Como venimos diciendo, el hecho de que $\mathbf{x}^{(k)}$ sea igual a $\mathbf{x}^{(k-1)}$ implica que $\mathbf{x}^{(k-1)}$ es la solución del problema, $\hat{\mathbf{x}}$. De aquí que, si $\mathbf{x}^{(k)} \neq \mathbf{x}^{(k-1)}$, es suficiente probar que el segundo término del miembro de la derecha de (2.31) es no negativo.

Sea $\mathbf{x}^{(m)} = \hat{\mathbf{x}}$. Como

$$\mathbf{x}^{(m)} - \mathbf{x}^{(k)} = \mathbf{x}^{(m)} - \mathbf{x}^{(m-1)} + \mathbf{x}^{(m-1)} - \dots - \mathbf{x}^{(k+1)} + \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$$

se tiene, usando la definición de $\mathbf{x}^{(k+1)}$ en (2.20), que

$$(\hat{\mathbf{x}} - \mathbf{x}^{(k)})^T (\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}) = \left(\alpha_{m-1} (\mathbf{p}^{(m-1)})^T \mathbf{p}^{(k-1)} + \dots + \alpha_k (\mathbf{p}^{(k)})^T \mathbf{p}^{(k-1)} \right) \alpha_{k-1}.$$

De la redefinición de α_j de (2.25) se sabe que todos los α_i son no positivos por lo que es suficiente probar que

$$(\mathbf{p}^{(j)})^T \mathbf{p}^{(k-1)} \geq 0, \quad j \geq k.$$

Aplicando repetidamente la definición de $\mathbf{p}^{(k+1)}$ de (2.20) se tiene que

$$\mathbf{p}^{(j)} = \mathbf{r}^{(j)} + \beta_{j-1} \mathbf{r}^{(j-1)} + \dots + (\beta_{j-1} \dots \beta_k) \mathbf{r}^{(k)} + (\beta_{j-1} \dots \beta_{k-1}) \mathbf{p}^{(k-1)}. \quad (2.32)$$

En particular, para $k = 1$, la expresión (2.32) indica que $\mathbf{p}^{(j)} \in [\mathbf{r}^{(0)}, \dots, \mathbf{r}^{(j)}]$, por lo que, junto con la ortogonalidad de $\mathbf{r}^{(i)}$ —recordemos (2.23), página 180—, se tiene que

$$(\mathbf{r}^{(j)})^T \mathbf{p}^{(k-1)} = 0, \quad j \geq k.$$

En consecuencia, de (2.32) y de la redefinición de β_k de (2.26), se obtiene que

$$\begin{aligned} \left(\mathbf{p}^{(j)}\right)^T \mathbf{p}^{(k-1)} &= \beta_{j-1} \cdots \beta_{k-1} \left(\mathbf{p}^{(k-1)}\right)^T \mathbf{p}^{(k-1)} \\ &= \left(\left(\mathbf{r}^{(j)}\right)^T \mathbf{r}^{(j)}\right) \left(\left(\mathbf{p}^{(k-1)}\right)^T \mathbf{p}^{(k-1)}\right) / \left(\left(\mathbf{r}^{(k-1)}\right)^T \mathbf{r}^{(k-1)}\right) \geq 0. \end{aligned}$$

■

Los resultados anteriores no tienen en cuenta ninguna propiedad de la matriz A . El siguiente sí.

Teorema 2.18 *Si A tiene sólo m valores propios distintos, el método de los gradientes conjugados converge en un máximo de m iteraciones.*

2.5.2.2.2 Interpretación geométrica del método de los gradientes conjugados

En dos dimensiones, el método de los gradientes conjugados se puede interpretar geométricamente con la ayuda del problema que se representa en la figura 2.5 en el que se resuelve $A\mathbf{x} = \mathbf{b}$, donde

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \quad \text{y} \quad \mathbf{b} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}.$$

En dos dimensiones, la ecuación $f(x_1, x_2) = z$ representa un paraboloide elíptico en $\mathbb{R}^3$. Las curvas de nivel $f(x_1, x_2) = \text{cte.}$ son elipses centradas en el mismo punto.

El método de los gradientes conjugados procede de la siguiente manera:

1. Desde el punto $\mathbf{x}^{(0)} = \mathbf{1}$ se determina la curva de nivel de la función

$$f(\mathbf{x}^{(0)}) = \frac{1}{2} \left(\mathbf{x}^{(0)}\right)^T A \mathbf{x}^{(0)} - \mathbf{b}^T \mathbf{x}^{(0)} = \left(x_1^{(0)}\right)^2 + \frac{3}{2} \left(x_2^{(0)}\right)^2 - 4x_1^{(0)} - x_2^{(0)} = \text{cte.}$$

La constante será $-2,5$.

2. Se traza la tangente en $\mathbf{x}^{(0)}$ a esta elipse y su normal en este punto $\mathbf{p}^{(0)}$:

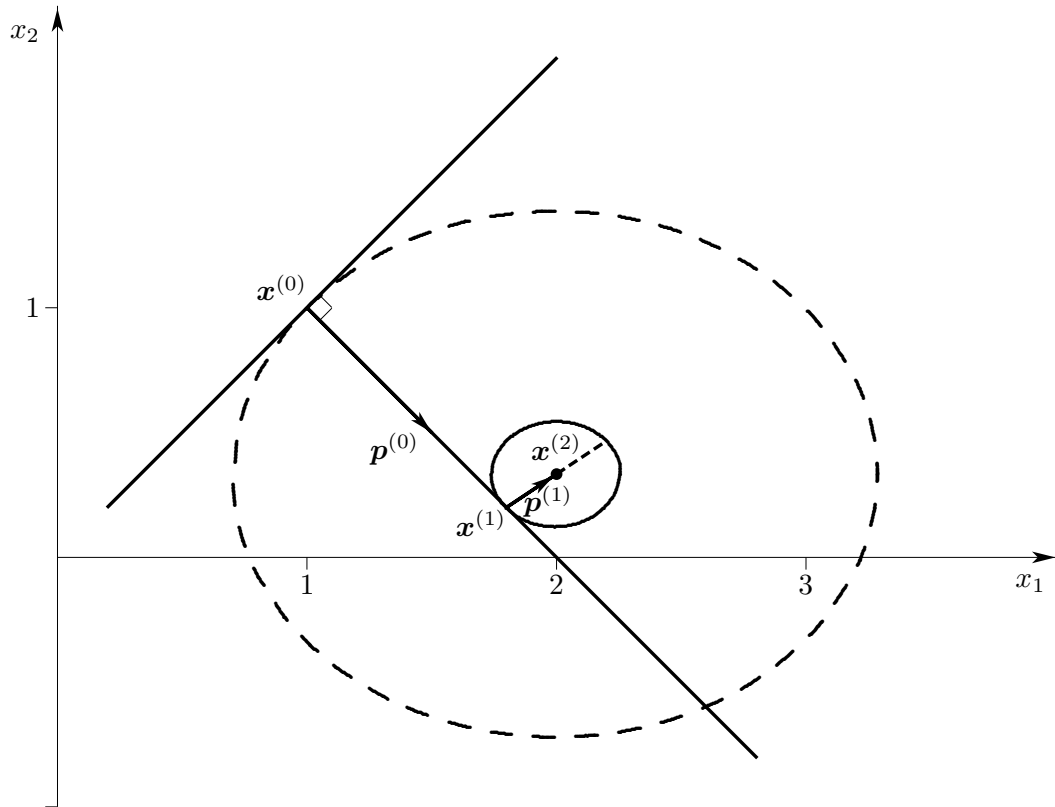
$$\mathbf{p}^{(0)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

3. Se determina el punto medio, $\mathbf{x}^{(1)}$, del segmento de recta que va desde $\mathbf{x}^{(0)}$ hasta el punto donde $\mathbf{p}^{(0)}$ corta a esa elipse. Ese punto medio es

$$\mathbf{x}^{(1)} = \begin{bmatrix} 1,8 \\ 0,2 \end{bmatrix}.$$

4. Desde $\mathbf{x}^{(1)}$ se determina la nueva elipse:

$$f(\mathbf{x}^{(1)}) = \left(x_1^{(1)}\right)^2 + \frac{3}{2} \left(x_2^{(1)}\right)^2 - 4x_1^{(1)} - x_2^{(1)} = -4,1.$$

**Figura 2.5**

Interpretación geométrica del método de los gradientes conjugados

5. Desde $\mathbf{x}^{(1)}$ se determina la dirección $\mathbf{p}^{(1)}$ que es A conjugada con respecto a $\mathbf{p}^{(0)}$. Esta dirección pasará por el centro de la nueva elipse uniendo el punto $\mathbf{x}^{(1)}$ con el diametralmente opuesto por el que se podría trazar una tangente a esta elipse que fuese paralela a $\mathbf{p}^{(0)}$. La nueva dirección es

$$\mathbf{p}^{(1)} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

Comprobar que $(\mathbf{p}^{(1)})^T A \mathbf{p}^{(0)} = 0$ es inmediato:

$$[3, 2] \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 0.$$

6. Se determina el punto medio, $\mathbf{x}^{(2)}$, de ese diámetro. La solución del problema es este nuevo punto:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 2 \\ 1/3 \end{bmatrix}.$$

2.5.2.2.3 Implementación práctica del método de los gradientes conjugados

Los errores de redondeo inherentes a la implementación en cualquier ordenador del método de los gradientes conjugados tienden a hacer perder la ortogonalidad (2.23) de los vectores residuo. Como consecuencia de ello la propiedad de que el método converge en un número fijo de pasos no se cumple en la práctica. Este hecho hace que el método de los gradientes conjugados se considere más un método iterativo que uno directo.

Todas las implementaciones prácticas del método de los gradientes conjugados siguen un esquema como el de la tabla 2.5. Dado un $\mathbf{b} \in \mathbb{R}^n$, una matriz simétrica definida positiva $A \in \mathbb{R}^{n \times n}$ y una tolerancia $\varepsilon \geq \epsilon$ (la precisión de la máquina donde se trabaje), ese algoritmo calcula un vector $\mathbf{x} \in \mathbb{R}^n$ tal que $\|\mathbf{b} - A\mathbf{x}\|_2 \cong \varepsilon \|\mathbf{b}\|_2$. Obsérvese que esta implementación parte de $\mathbf{x}^{(0)} = \mathbf{0}$; si se comenzase desde otro punto habría que hacer al principio $\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}$.

La codificación en FORTRAN 77 de este algoritmo para resolver el problema

$$\begin{array}{rrrrrrr} 4x_1 & - & x_2 & & - & x_4 & & = & 0 \\ -x_1 & + & 4x_2 & - & x_3 & & - & x_5 & = & 5 \\ & & - & x_2 & + & 4x_3 & & - & x_6 & = & 0 \\ -x_1 & & & & & + & 4x_4 & - & x_5 & & = & 6 \\ & & - & x_2 & & & - & x_4 & + & 4x_5 & - & x_6 & = & -2 \\ & & & & - & x_3 & & & - & x_5 & + & 4x_6 & = & 6 \end{array}$$

es la que sigue.

```
PROGRAM Cg
C
```

Tabla 2.5

Algoritmo de los gradientes conjugados para resolver $A\mathbf{x} = \mathbf{b}$

```

 $\mathbf{x}^{(0)} \leftarrow \mathbf{0}; \mathbf{r}^{(0)} \leftarrow \mathbf{b}; \rho_0 \leftarrow \|\mathbf{r}^{(0)}\|_2^2; k = 1$ 
while  $\sqrt{\rho_{k-1}} > \varepsilon \|\mathbf{b}\|_2$  do
  if  $k = 1$  then
     $\mathbf{p}^{(1)} \leftarrow \mathbf{r}^{(0)}$ 
  else
     $\beta_k \leftarrow \rho_{k-1} / \rho_{k-2}$ 
     $\mathbf{p}^{(k)} \leftarrow \mathbf{r}^{(k-1)} + \beta_k \mathbf{p}^{(k-1)}$ 
  end
   $\mathbf{w} \leftarrow A\mathbf{p}^{(k)}$ 
   $\alpha_k \leftarrow \rho_{k-1} / (\mathbf{p}^{(k)})^T \mathbf{w}$ 
   $\mathbf{x}^{(k)} \leftarrow \mathbf{x}^{(k-1)} + \alpha_k \mathbf{p}^{(k)}$ 
   $\mathbf{r}^{(k)} \leftarrow \mathbf{r}^{(k-1)} - \alpha_k \mathbf{w}$ 
   $\rho_k \leftarrow \|\mathbf{r}^{(k)}\|_2^2$ 
   $k \leftarrow k + 1$ 
end
```

```

parameter (n = 6)
real a(n,n),b(n),x(n),r(n),p(n),w(n)
C
  data a/4.,-1.,0.,-1.,0.,0.,-1.,4.,-1.,0.,-1.,0.,0.,-1.,4.,0.,0.,
+    -1.,-1.,0.,0.,4.,-1.,0.,0.,-1.,0.,-1.,4.,-1.,0.,0.,-1.,0.,-1.,
+    4./
  data b/0.,5.,0.,6.,-2.,6./
C
  x      = 0.
  r      = b
  ro0    = prod(r,r,n)
  ro1    = ro0
C
C *** Proceso iterativo ***
C
  xnormb = epsilon(1.0)*sqrt(ro0)*5
  k      = 0
  do while (sqrt(ro1).gt.xnormb)
    betak = ro1/ro0
    if (k.eq.0) betak = 0
    p = r+betak*p
    do i = 1,n
      w(i) = prod(a(1,i),p,n)
    end do
    alfak = ro1/prod(p,w,n)
    x      = x+alfak*p
    r      = r-alfak*w
    ro0    = ro1
    ro1    = prod(r,r,n)
    k      = k+1
    print *,k,x
  end do
C
  end

  real function prod (x,y,n)
  real x(n),y(n)
C
  prod = 0.0
  do i = 1,n
    prod = prod+x(i)*y(i)
  end do
C
  return
end

```

Los puntos del proceso iterativo que se obtienen con este código son los de la tabla 2.6.

Si este mismo código se utiliza para resolver el problema al que aludíamos en el apartado dedicado al método de la máxima pendiente, aquel en el que la matriz A era una matriz de Hilbert 50×50 , el número de iteraciones que necesita es sólo 7.

Si también se resuelve el problema que ha permitido ilustrar geoméricamente el método de los gradientes conjugados con el código anterior, al que se le han de efectuar las modificaciones obvias pues no se parte de $\mathbf{x}^{(0)} = \mathbf{0}$ sino de $\mathbf{x}^{(0)} = \mathbf{1}$, el resultado que se obtiene es el que sigue.

1	1.800000	2.000000E-01	2.000000	-2.000000
2	2.000000	3.333333E-01	4.800000E-01	3.200000E-01

Las dos últimas columnas indican los componentes del vector $\mathbf{p}^{(k)}$. Como se puede comprobar

Tabla 2.6

Proceso de convergencia de la resolución de un sistema de ecuaciones lineales mediante el método de los gradientes conjugados

k	0	1	2	3	4
$x_1^{(k)}$	0,000000	0,000000	1,022376	0,990783	1,000000
$x_2^{(k)}$	0,000000	1,069915	1,686452	1,991635	2,000000
$x_3^{(k)}$	0,000000	0,000000	1,022376	0,990783	1,000000
$x_4^{(k)}$	0,000000	1,283898	2,060920	2,005324	2,000000
$x_5^{(k)}$	0,000000	-0,427966	0,831099	1,011825	1,000000
$x_6^{(k)}$	0,000000	1,283898	2,060920	2,005324	2,000000

fácilmente haciendo las simplificaciones oportunas, $\mathbf{p}^{(1)}$ y $\mathbf{p}^{(2)}$ coinciden con las que se vieron con anterioridad:

$$\mathbf{p}^{(0)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}; \quad \mathbf{p}^{(1)} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

Los pasos que sigue la resolución del problema son pues idénticos a los que se estudiaron geoméricamente.

2.5.2.2.4 Método de los gradientes conjugados con preconditionamiento

Ya vimos al estudiar la velocidad de convergencia del método de los gradientes conjugados que ésta la definía la expresión

$$\|\hat{\mathbf{x}} - \mathbf{x}^{(k)}\|_2 \leq 2\sqrt{\kappa_2}\alpha^k \|\hat{\mathbf{x}} - \mathbf{x}^{(0)}\|_2,$$

donde $\hat{\mathbf{x}}$ representa la solución exacta, $\alpha = (\sqrt{\kappa_2} - 1)/(\sqrt{\kappa_2} + 1)$ y κ_2 es el número de condición 2 de la matriz A . Como ésta es simétrica y positiva definida, $\kappa_2 = \lambda_n/\lambda_1$, donde λ_n indica el mayor valor propio de A . De acuerdo con esta expresión, cuanto más grande sea κ_2 más lentamente convergerá el proceso.

El estudio de estas últimas consideraciones ha llevado a considerar el que se conoce en la literatura como el método de los gradientes conjugados con preconditionamiento. La idea que subyace en él es muy sencilla: sustituir la búsqueda de la solución de la ecuación $A\mathbf{x} = \mathbf{b}$ por la de $\tilde{A}\tilde{\mathbf{x}} = \tilde{\mathbf{b}}$ de tal forma que la condición de la nueva matriz $\tilde{A}$ sea mejor que la de A . Esto se suele hacer encontrando una matriz no singular simétrica C tal que $\tilde{A} = C^{-1}AC^{-1}$. En este caso, una vez resuelto $\tilde{A}\tilde{\mathbf{x}} = \tilde{\mathbf{b}}$, la solución original se obtendría sin más que hacer $\mathbf{x} = C^{-1}\tilde{\mathbf{x}}$.

Obsérvese que la mejor matriz C posible es $A^{1/2}$, pues en ese caso $\tilde{A} = I$.

Haciendo

$$\begin{aligned} M &= C^2 \\ \mathbf{p} &= C^{-1}\tilde{\mathbf{p}} \\ \mathbf{x} &= C^{-1}\tilde{\mathbf{x}} \\ \mathbf{z} &= C^{-1}\tilde{\mathbf{r}} \\ \mathbf{r} &= C\tilde{\mathbf{r}} = \mathbf{b} - A\mathbf{x} \end{aligned}$$

y sustituyendo estas expresiones en el algoritmo de la tabla 2.5, se tiene el *algoritmo de los gradientes conjugados con preconditionamiento* que describe la tabla 2.7.

Se puede comprobar que los residuos y las direcciones verifican que

$$\left(\mathbf{r}^{(j)}\right)^T M^{-1} \mathbf{r}^{(i)} = 0, \quad i \neq j,$$

y que

$$\left(\mathbf{p}^{(j)}\right)^T \left(C^{-1} A C^{-1}\right) \mathbf{p}^{(i)} = 0, \quad i \neq j.$$

Para que el algoritmo sea eficaz es necesario resolver rápidamente el sistema $M \mathbf{z}^{(k-1)} = \mathbf{r}^{(k-1)}$ pues de lo contrario otros métodos resultarían más ventajosos. Ahora bien, resolver eficazmente ese sistema está íntimamente relacionado con la elección de un buen *precondicionador*, M .

Existen muchos preconditionadores en la literatura especializada. Van desde los sencillos,

$$M = \text{diag} \left(d_1^{1/2}, \dots, d_n^{1/2} \right), \quad d_j = \|\mathbf{a}_j\|_2^2, \quad (2.33)$$

o un poco más complicados,

$$M = (D + \omega L) D^{-1} (D + \omega L)^T,$$

también llamado SSOR (recordemos el método SSOR en sí mismo), a los más sofisticados basados en factorizaciones incompletas de Cholesky de la matriz A . Como se puede intuir,

Tabla 2.7

Algoritmo de los gradientes conjugados con preconditionamiento para resolver $A\mathbf{x} = \mathbf{b}$

```

 $\mathbf{x}^{(0)} \leftarrow \mathbf{0}; \mathbf{r}^{(0)} \leftarrow \mathbf{b}; \rho_0 \leftarrow \|\mathbf{r}^{(0)}\|_2^2; k = 1$ 
while  $\sqrt{\rho_{k-1}} > \varepsilon \|\mathbf{b}\|_2$  do
  if  $k = 1$  then
     $\mathbf{p}^{(1)} \leftarrow \mathbf{r}^{(0)}$ 
  else
    Resolver  $M \mathbf{z}^{(k-1)} = \mathbf{r}^{(k-1)}$ 
     $\beta_k \leftarrow \left( \mathbf{z}^{(k-1)} \right)^T \mathbf{r}^{(k-1)} \Big/ \left( \mathbf{z}^{(k-2)} \right)^T \mathbf{r}^{(k-2)}$ 
     $\mathbf{p}^{(k)} \leftarrow \mathbf{z}^{(k-1)} + \beta_k \mathbf{p}^{(k-1)}$ 
  end
   $\mathbf{w} \leftarrow A \mathbf{p}^{(k)}$ 
   $\alpha_k \leftarrow \left( \mathbf{z}^{(k-1)} \right)^T \mathbf{r}^{(k-1)} \Big/ \left( \mathbf{p}^{(k)} \right)^T \mathbf{w}$ 
   $\mathbf{x}^{(k)} \leftarrow \mathbf{x}^{(k-1)} + \alpha_k \mathbf{p}^{(k)}$ 
   $\mathbf{r}^{(k)} \leftarrow \mathbf{r}^{(k-1)} - \alpha_k \mathbf{w}$ 
   $\rho_k \leftarrow \|\mathbf{r}^{(k)}\|_2^2$ 
   $k \leftarrow k + 1$ 
end

```

muchos preconditionadores están diseñados para resolver sistemas especiales en los que la matriz surge de algún tipo de problema concreto o posee una estructura determinada.

En lo que sigue se ilustra con un programa codificado en FORTRAN 77 el algoritmo de los gradientes conjugados con preconditionamiento para resolver un problema que se genera con números aleatorios $[0, 1]$. El preconditionador que se utiliza es el de la expresión (2.33).

```

PROGRAM Cgp
C
  implicit double precision (a-h,o-z)
C
  parameter (n=40)
  real ra
  double precision a(n,n),b(n),x(n),r(n),p(n),w(n),m(n),z(n),zm2(n),
+      rm2(n),aux(n,n),baux(n)
C
  do i = 1,n
    do j = 1,n
      call random (ra)
      aux(i,j) = dble(ra)
    end do
    baux(i) = dble(i)
  end do
  do i = 1,n
    do j = 1,n
      a(i,j) = prod(aux(1,i),aux(1,j),n)
    end do
  end do
  do i = 1,n
    b(i) = prod(a(1,i),baux,n)
  end do
C
  do i = 1,n
    m(i) = dsqrt(prod(a(1,i),a(1,i),n))
  end do
C
C *** Proceso iterativo ***
C
  xnormb = epsilon(1.D0)*1000.*sqrt(prod(b,b,n))
  x      = 0.
  r      = b
  k      = 1
  ro1    = prod(b,b,n)
  do while (dsqrt(ro1).gt.xnormb)
    z      = r/m
    betak  = prod(z,r,n)/prod(zm2,rm2,n)
    if (k.eq.1) betak = 0
    p      = z+betak*p
    do i = 1,n
      w(i) = prod(a(1,i),p,n)
    end do
    alfak  = prod(z,r,n)/prod(p,w,n)
    x      = x+alfak*p
    rm2    = r
    r      = r-alfak*w
    zm2    = z
    ro1    = prod(r,r,n)
    k      = k+1
    print *,k,x,ro1
  end do
  ! Salida de resultados

```

```

C      end do
      end

      double precision function prod (x,y,n)
      double precision x(n),y(n)
      prod = 0.0D0
      do i = 1,n
        prod = prod+x(i)*y(i)
      end do
      return
      end

```

El proceso converge en 65 iteraciones. Si para resolver este problema no se utiliza preconditionador, el proceso también converge en un número de iteraciones muy semejante, aunque mayor.

2.6 Comparación numérica de los algoritmos

Para poder comparar las prestaciones numéricas de los algoritmos iterativos que hemos presentado para resolver un sistema de ecuaciones lineales, $A\mathbf{x} = \mathbf{b}$, hemos utilizado el problema al que nos hemos referido anteriormente: el generado a partir de una matriz de Hilbert 50×50 . La matriz del sistema A proviene de multiplicar esa matriz de Hilbert por su traspuesta. La respuesta del problema se fuerza que sea $\mathbf{x} = \mathbf{1}$. Para evitar que el número de condición de A sea muy malo, se mejora sumando a algunos elementos de la diagonal principal el número 1. El número de condición resultante es aproximadamente 5×10^4 .

Los resultados que se obtienen con los métodos de Gauss-Seidel, SOR, máxima pendiente y gradientes conjugados, partiendo del punto $\mathbf{x} = \mathbf{0}$, se describen en la tabla 2.8. El método de Jacobi no puede resolver este problema pues diverge. Para que los resultados sean coherentes, el criterio de finalización para todos los procedimientos es conseguir que $\|\mathbf{b} - A\mathbf{x}\|_2 < 10^{-5}$.

Tabla 2.8

Resultados obtenidos por diversos métodos iterativos para resolver un problema lineal mal condicionado de 50 ecuaciones con 50 incógnitas

Método		Número de iteraciones
Gauss-Seidel		2673
SOR	$w = 1,5$	1450
	$w = 1,75$	827
	$w = 1,863$	344
Máxima pendiente		112
Gradientes conjugados		7

2.7 Mínimos cuadrados y métodos iterativos

Cuando los problemas de mínimos cuadrados son de grandes dimensiones o dispersos, una buena alternativa a la de utilizar métodos directos como los estudiados en el capítulo 1, o los que se estudian en el capítulo 3, consiste en utilizar los métodos iterativos. La idea consiste en resolver iterativamente las ecuaciones normales,

$$A^T(\mathbf{b} - A\mathbf{x}) = \mathbf{0}, \quad (2.34)$$

sin tener que obtener explícitamente el producto $A^T A$ pues, como sabemos, puede presentar dificultades numéricas.

2.7.1 Método de Jacobi

El método de Jacobi calcula una sucesión $\mathbf{x}^{(k)}$, $k = 1, 2, \dots$, cuya relación de recurrencia está dada por la expresión

$$x_i^{(k+1)} = x_i^{(k)} + \mathbf{a}_i^T (\mathbf{b} - A\mathbf{x}^{(k)}) / d_i, \quad i = 1, \dots, n,$$

donde $A = [\mathbf{a}_1, \dots, \mathbf{a}_n] \in \Re^{m \times n}$ y $d_i = \|\mathbf{a}_i\|_2^2$. Designando en este caso como D la matriz diagonal $\text{diag}(d_1, \dots, d_n)$, el método de Jacobi para problemas de mínimos cuadrados tiene como esquema iterativo en forma matricial el siguiente:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + D^{-1} A^T (\mathbf{b} - A\mathbf{x}^{(k)}).$$

Como se puede observar, no es necesario calcular el producto $A^T A$ explícitamente.

2.7.2 Método de Gauss-Seidel

El método de Gauss-Seidel calcula una sucesión $\mathbf{x}^{(k)}$, $k = 1, 2, \dots$, cuyo esquema iterativo en forma matricial es

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + (A^T \mathbf{b} - L\mathbf{x}^{(k+1)} - (D + L^T)\mathbf{x}^{(k)}),$$

donde $A^T A = L + D + L^T$. En Björk y Elfving [1979] se expone una forma práctica de utilizar el método de Gauss-Seidel para resolver (2.34) trabajando sólo con la matriz A . En él, cada iteración se divide en n pasos o iteraciones menores: haciendo $\mathbf{z}^{(1)} = \mathbf{x}^{(1)}$, se calcula

$$\mathbf{z}^{(j+1)} = \mathbf{z}^{(j)} + \mathbf{e}_j \mathbf{a}_j^T (\mathbf{b} - A\mathbf{z}^{(j)}) / d_j; \quad j = 1, 2, \dots, n,$$

donde $\mathbf{e}_j$, como siempre, es el vector unitario cuyo componente j es uno y d_j es el mismo que antes. De esta manera se tiene que $\mathbf{x}^{(k+1)} = \mathbf{z}^{(k+1)}$. Obsérvese que en cada iteración menor, j , sólo se modifica $z_j^{(j)}$.

El vector de residuos, $\mathbf{r}^{(j)} = \mathbf{b} - A\mathbf{z}^{(j)}$, se puede obtener fácilmente de iteración en iteración. En efecto, si $\mathbf{r}^{(1)} = \mathbf{b} - A\mathbf{x}^{(1)}$,

$$\mathbf{z}^{(j+1)} = \mathbf{z}^{(j)} + \delta_j \mathbf{e}_j, \quad \text{donde} \quad \delta_j = \mathbf{a}_j^T \mathbf{r}^{(j)} / d_j. \quad (2.35)$$

por lo que

$$\begin{aligned} \mathbf{r}^{(j+1)} &= \mathbf{b} - A\mathbf{z}^{(j+1)} = \mathbf{b} - A(\mathbf{z}^{(j)} + \delta_j \mathbf{e}_j) \\ &= \mathbf{b} - A\mathbf{z}^{(j)} - A\delta_j \mathbf{e}_j = \mathbf{r}^{(j)} - \delta_j \mathbf{a}_j. \end{aligned}$$

En cada iteración menor j , de esta forma, sólo es necesario acceder a la columna j de la matriz A .

2.7.3 Método de relajación SOR

Este método se obtiene de (2.35) sin más que hacer

$$\delta_j = \omega \mathbf{a}_j^T \mathbf{r}^{(j)} / d_j, \quad 0 < \omega < 2.$$

Björk [1990] explica cómo generalizar la aplicación de los métodos iterativos para resolver problemas de mínimos cuadrados al caso de matrices no cuadradas. Para ello se descompone la matriz A de la forma $A = M - N$ de tal forma que los subespacios imagen y núcleo de A y M sean iguales. El esquema iterativo en forma matricial que se obtiene es

$$\mathbf{x}^{(k+1)} = M^\dagger (N\mathbf{x}^{(k)} + \mathbf{b}).$$

El proceso converge a $\mathbf{x} = A^\dagger \mathbf{b}$ para todo $\mathbf{x}^{(0)}$ si y sólo si $\rho(M^\dagger N) < 1$.

2.7.4 Método de los gradientes conjugados

El método de los gradientes conjugados aplicado a las ecuaciones normales es el que se describe en la tabla 2.9. También se puede aplicar cuando A no tiene rango completo. Si $\mathbf{x}^{(0)} \in \text{Im}(A^T)$, lo que ocurre cuando $\mathbf{x}^{(0)} = \mathbf{0}$, $\mathbf{x}^{(k)}$ converge a $A^T \mathbf{b}$. El algoritmo requiere almacenar dos vectores de dimensión n , $\mathbf{x}$ y $\mathbf{p}$, y dos vectores de dimensión m , $\mathbf{r}$ y $\mathbf{q}$. Cada iteración requiere alrededor de $2nz(A) + 3n + 2m$ sumas/restas y multiplicaciones ($nz(A)$ designa el número de elementos distintos de cero de A).

Como en el caso general —y en este de los mínimos cuadrados con mayor razón pues la condición de $A^T A$ suele ser mala—, para mejorar las prestaciones del método de los gradientes conjugados se recurre al preconditionamiento. Si el preconditionador, M , se hace igual a R , el factor de Cholesky de $A^T A$, la condición $\kappa(AM^{-1}) = \kappa(Q) = 1$: el preconditionador ideal es, por tanto, R . Cualquier preconditionador debe tratar de conseguir aproximar bien R realizando el menor número de operaciones posible.

Existen muchos preconditionadores en la literatura. El más utilizado por su eficacia, aunque el más complicado de calcular, es

$$M = UP_c^T,$$

Tabla 2.9

Algoritmo de los gradientes conjugados para resolver $A^T(\mathbf{b} - A\mathbf{x})$

```

 $\mathbf{r}^{(0)} \leftarrow \mathbf{b} - A\mathbf{x}^{(0)}; \mathbf{p}^{(0)} = \mathbf{s}^{(0)} = A^T \mathbf{r}^{(0)}; \gamma_0 = \|\mathbf{s}^{(0)}\|_2^2$ 
while ( $\sqrt{\gamma_k} > \varepsilon \|A^T \mathbf{b}\|_2$ ) do
     $\mathbf{q}^{(k)} \leftarrow A\mathbf{p}^{(k)}$ 
     $\alpha_k \leftarrow \gamma_k / \|\mathbf{q}^{(k)}\|_2^2$ 
     $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}$ 
     $\mathbf{r}^{(k+1)} \leftarrow \mathbf{r}^{(k)} - \alpha_k \mathbf{q}^{(k)}$ 
     $\mathbf{s}^{(k+1)} \leftarrow A^T \mathbf{r}^{(k+1)}$ 
     $\gamma_{k+1} \leftarrow \|\mathbf{s}^{(k+1)}\|_2^2$ 
     $\beta_k \leftarrow \gamma_{k+1} / \gamma_k$ 
     $\mathbf{p}^{(k+1)} \leftarrow \mathbf{s}^{(k+1)} + \beta_k \mathbf{p}^{(k)}$ 
end

```

donde U resulta de la factorización

$$P_f A P_c = LU.$$

Con la notación P_f se indica un conjunto de permutaciones de filas; P_c indica unas permutaciones de columnas. El lector interesado en estos aspectos puede consultar Björk [1990].

Referencias

Todo lo que se expone en este capítulo relativo a los métodos de Jacobi, Gauss-Seidel y Relajación se puede encontrar en Axelsson [1996], Varga [1962], Young [1971], Lascaux y Théodor [1987], Stoer y Bulirsch [1980] y Ciarlet [1988]. El método SSOR está basado en Ortega [1988].

Todos los programas de ordenador del capítulo son del autor. Parecidos, a excepción de los relativos a gradientes conjugados, se pueden encontrar en Hager [1988], Atkinson, Harley y Hudson [1989] y Lascaux y Théodor [1987].

Según se expone, el método de los gradientes conjugados y su teoría es de Ortega [1988]. También se pueden encontrar enfoques muy similares en Stoer y Bulirsch [1980], Lascaux y Théodor [1987], Hestenes [1980], Golub y Van Loan [1983] y [1989] y Ortega y Rheinboldt [1970]; lo relativo a optimización en Gill, Murray y Wright [1981] y Luenberger [1984]; lo relativo a preconditionamiento en Golub y Van Loan [1989]. La interpretación geométrica es de Engeln-Müllges y Uhlig [1996].

Todo lo que hace referencia a procesos iterativos y mínimos cuadrados se puede estudiar en Björk [1990] y [1996].

Ejercicios

- 2.1. ¿Qué se puede decir de un método iterativo para resolver un sistema lineal si la matriz de ese método tiene un radio espectral igual a cero?
- 2.2. El objetivo de este ejercicio es demostrar que, en general, dos métodos iterativos son difícilmente comparables.

a) Sea la matriz

$$A = \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix}.$$

Demostrar que $\rho(J) < 1 < \rho(G)$, donde J es la matriz del método de Jacobi y G la del de Gauss-Seidel que poder aplicar en este caso.

b) Sea ahora

$$A = \begin{bmatrix} 2 & -1 & 1 \\ 2 & 2 & 2 \\ -1 & -1 & 2 \end{bmatrix}.$$

Demostrar que $\rho(G) < 1 < \rho(J)$.

2.3. Sea el sistema lineal

$$\underbrace{\begin{bmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 2 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 19 \\ 19 \\ -3 \\ -12 \end{bmatrix}}_b.$$

a) Calcular la solución exacta de este sistema mediante eliminación de Gauss.

b) Calcular los vectores $\mathbf{x}^{(k)} = [x_1^{(k)}, x_2^{(k)}, x_3^{(k)}, x_4^{(k)}]$, $k \leq 6$, que se obtienen aplicando los métodos iterativos de Jacobi, Gauss-Seidel y SOR para $\omega = 1.1, 1.2, \dots, 1.9$, partiendo del punto inicial $\mathbf{x}^{(0)} = \mathbf{0}$.

2.4. Se considera la matriz

$$A = \begin{bmatrix} 2 + \alpha_1 & -1 & & & \\ -1 & 2 + \alpha_2 & & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 + \alpha_{n-1} & -1 \\ & & & -1 & 2 + \alpha_n \end{bmatrix}, \quad \alpha_i \geq 0, \quad 1 \leq i \leq n,$$

y la descomposición $A = M_\beta - N_\beta$, donde

$$N_\beta = \text{diag}(\beta - \alpha_i),$$

siendo β un parámetro ≥ 0 . Estudiar la convergencia del método iterativo asociado a esta descomposición según el valor del parámetro β (existencia de un intervalo $I \subset \mathbb{R}$ tal que $\rho(M_\beta^{-1}N_\beta) < 1$, para $\beta \in I$ y existencia de un parámetro óptimo).

2.5. Considérese el sistema lineal de ecuaciones

$$\underbrace{\begin{bmatrix} 10 & -25 \\ -1 & 10 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 5 \\ 7 \end{bmatrix}}_b.$$

La matriz de Jacobi $J = I - D^{-1}A$ correspondiente a este sistema es

$$J = \begin{bmatrix} 0 & 2,5 \\ 0,1 & 0 \end{bmatrix}.$$

- a) Como $\|J\|_1 = 2,5$ ¿nos dice la desigualdad $\|e^{(k)}\|_1 \leq \|J\|_1^k \|e^{(0)}\|_1$ algo respecto a la convergencia del método de Jacobi en este caso?
- b) Comprobar que $\|e^{(k)}\|_1 \leq 5 \times 0,5^k \|e^{(0)}\|_1$. Las iteraciones del método de Jacobi aplicado a este problema, ¿convergerán o divergerán?

2.6. Considérese la matriz

$$A = \begin{bmatrix} 3 & -1 & 0 & 0 & 0 & -1 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 3 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & 0 \\ 0 & -1 & 0 & -1 & 3 & -1 \\ -1 & 0 & 0 & 0 & -1 & 3 \end{bmatrix}.$$

- a) Comprobar que los métodos de Jacobi, Gauss-Seidel y SOR convergerían al aplicarlos a un sistema lineal que definiese esta matriz.
- b) Si hubiese que decidirse por aplicar uno de los tres, ¿cuál se debería elegir?
- c) Escribir una iteración completa del método elegido para un $\mathbf{b}$ cualquiera.

2.7. Considérese la matriz

$$A = \begin{bmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{bmatrix}, \quad \text{con } a \in \mathbb{R}.$$

- a) ¿Para qué valores de a , A es definida positiva?
- b) ¿Para qué valores de a converge el método de Gauss-Seidel?
- c) Escribir la matriz J de la iteración del método de Jacobi.
- d) ¿Para qué valores de a converge el método de Jacobi?
- e) Escribir la matriz G de la iteración del método de Gauss-Seidel.
- f) Calcular el radio espectral de G .
- g) ¿Para qué valores de a el método de Gauss-Seidel converge más rápidamente que el de Jacobi?

2.8. Sean la matriz

$$B = \begin{bmatrix} 5 & 1 \\ 0 & \frac{1}{2} \end{bmatrix} \quad \text{y el vector } \mathbf{b} = \begin{bmatrix} 5 \\ -\frac{1}{2} \end{bmatrix}.$$

Considérese el método iterativo definido por

$$\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} - \mathbf{b}.$$

Comprobar que $\bar{\mathbf{x}} = [1, 1]^T$ es la solución de la ecuación $\mathbf{x} = B\mathbf{x} - \mathbf{b}$. Si se escoge como punto para iniciar el proceso iterativo $\mathbf{x}^{(0)} = [1 + \varepsilon, 1]^T$, calcular $\mathbf{x}^{(k)}$ en función de ε . ¿Converge la sucesión $\{\mathbf{x}^{(k)}\}$ a $\bar{\mathbf{x}}$.

Si se escoge ahora como punto para iniciar el proceso iterativo $\mathbf{x}^{(0)} = [1 + 2\varepsilon, 1 - 9\varepsilon]^T$, calcular $\mathbf{x}^{(k)}$ en función de ε . ¿Converge la sucesión $\{\mathbf{x}^{(k)}\}$ a $\bar{\mathbf{x}}$? ¿Por qué?

2.9. Sea A la matriz definida por $D - E - F$, donde

$$E = \begin{bmatrix} 0 & 2 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad \text{y} \quad F = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}.$$

Considérese la matriz de la iteración del método SOR, es decir

$$G(\omega) = (I - \omega L)^{-1} [(1 - \omega)I + \omega U].$$

- a) Calcular, en función de ω , los autovalores de la matriz $G(\omega)$; deducir posteriormente el radio espectral $\rho(G(\omega))$.
- b) ¿Cuáles son los valores de ω para los que el método de relajación converge?
- c) Determinar el valor de ω para el cual $\rho(G(\omega))$ alcanza su mínimo.

2.10. Sea

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

una matriz de coeficientes reales y positivos. Probar que el radio espectral $\rho(A)$ es una función creciente de cada uno de los coeficientes de A .

2.11. Sea B una matriz cuadrada de orden n . Se supone que $\|B\| \leq \alpha < 1$.

I. Considérese la sucesión

$$\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + c, \quad \mathbf{x}^{(0)} \in \mathbf{C}^n.$$

- a) Comprobar que esta sucesión es convergente. ¿Qué relación verifica el límite $\bar{\mathbf{x}}$?
- b) Se define el vector residuo por $\mathbf{r}^{(k)} = c - (I - B)\mathbf{x}^{(k)}$. Comprobar que $\mathbf{r}^{(k)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ y que $\mathbf{r}^{(k+1)} = B\mathbf{r}^{(k)}$. Deducir que $\mathbf{x}^{(k+1)} = \mathbf{x}^{(0)} + (I + B + \dots + B^k)\mathbf{r}^{(0)}$.
- c) Probar que $\bar{\mathbf{x}} = \mathbf{x}^{(0)} + (I - B)^{-1}\mathbf{r}^{(0)}$ y que $\bar{\mathbf{x}} = \mathbf{x}^{(k)} + B^k(I - B)^{-1}\mathbf{r}^{(0)}$.
- d) Probar que

$$\|(I - B)^{-1}\| \leq \frac{1}{1 - \|B\|}$$

y que

$$\frac{\|\bar{\mathbf{x}} - \mathbf{x}^{(k)}\|}{1 - \|B\|} \leq \|B\|^k \|\mathbf{r}^{(0)}\|.$$

e) Si

$$B = \frac{1}{10} \begin{bmatrix} 2 & 1 & 0 & 0 \\ 3 & 2 & 0 & 1 \\ 0 & -1 & 2 & 1 \\ 2 & 1 & 1 & 1 \end{bmatrix} :$$

- (a) Escoger una norma matricial cualquiera para B .
- (b) Hacer una estimación de α tal que $\|B\| \leq \alpha < 1$ y deducir una cota del número n de iteraciones para que $\|\bar{\mathbf{x}} - \mathbf{x}^{(n)}\| \leq \varepsilon$, con ε dado.

II. Considérese ahora la sucesión

$$\begin{aligned} \mathbf{y}^{(1)} &= B\mathbf{y}^{(0)} + c \\ \mathbf{y}^{(n+1)} &= \omega(B\mathbf{y}^{(n)} + c - \mathbf{y}^{(n-1)}) + \mathbf{y}^{(n-1)} \quad \text{para } n \geq 1 \text{ y } \omega \neq 0. \end{aligned}$$

El punto inicial $\mathbf{y}^{(0)}$ es dado.

- a) Demostrar que esta sucesión converge.
- b) Determinar la constante ω para la cual la velocidad de convergencia de esta sucesión es la mayor posible.

2.12. Sea $A = I - L - L^T$ una matriz simétrica definida positiva. Escribir la matriz $S(\omega)$ de la iteración del método SSOR aplicado a $A\mathbf{x} = \mathbf{b}$.

2.13. Demostrar el teorema 2.18.

2.14. Sea A una matriz simétrica y definida positiva. Considérese la descomposición $A = M - N$ con M simétrica y definida positiva. Sea $B = M^{-1}N$, suponiéndose que $\rho(B) < 1$.

a) Comprobar que

$$\kappa_2(M^{-1}A) \leq \frac{1 + \rho(B)}{1 - \rho(B)}.$$

b) Probar que el número de iteraciones k_1 para obtener

$$\frac{\|\mathbf{x}^{(k)} - \bar{\mathbf{x}}\|}{\|\mathbf{x}^{(0)} - \bar{\mathbf{x}}\|} < \varepsilon$$

por el método iterativo $\mathbf{x}^{(k+1)} = B\mathbf{x}^{(k)} + c$ (cumpliéndose $\bar{\mathbf{x}} = B\bar{\mathbf{x}} + c$) y el número de iteraciones k_2 para obtener la misma precisión mediante el método de los gradientes conjugados con preconditionado con $C = M$, con el mismo $\mathbf{x}^{(0)}$, verifican que

$$\frac{k_1}{k_2} \approx \frac{\ln(2/\varepsilon)}{\ln(1/\varepsilon)} \frac{\ln(1/k_2(M^{-1}A))}{2 \ln(1/\rho(B))}.$$

2.15. Sea A una matriz $n \times n$ simétrica y $\mathbf{d}^{(0)}, \mathbf{d}^{(1)}, \dots, \mathbf{d}^{(n-1)}$ vectores o direcciones A conjugadas. Encontrar una matriz E tal que $E^T A E$ sea diagonal.

Capítulo 3

SISTEMAS DE ECUACIONES LINEALES DE MATRIZ DE COEFICIENTES DISPERSA

LA UTILIZACIÓN masiva de los ordenadores en los últimos años y el aumento de su potencia y capacidad de cálculo, han permitido que la ciencia, técnica e ingeniería utilicen cada vez más modelos matemáticos para comprender, simular y optimizar fenómenos de diversa complejidad, y que esos modelos crezcan extraordinariamente en magnitud y exactitud. Muchos de estos modelos conllevan enfrentarse con matrices de un tamaño tal que hace sólo unos pocos años era casi inimaginable que se pudiesen tratar (cientos de miles de filas y columnas). Campos de la ciencia y de la tecnología donde surgen habitualmente matrices de grandes dimensiones son:

- Optimización lineal y no lineal.
- Análisis de sistemas eléctricos de generación y transporte de energía.
- Estudios geodésicos.
- Fotometría.
- Análisis estructural de moléculas.
- Análisis de campos gravitatorios.
- Tomografía.
- Prospecciones petrolíferas.
- Cálculo y análisis de estructuras mecánicas.

- Ajuste de superficies, etc.

Las *matrices dispersas* son aquellas matrices de gran tamaño en las que *muchos* de los elementos que las configuran son cero. Aunque el término *muchos* confiere ambigüedad a la definición y los autores especializados en esta cuestión lo cuantifican de forma dispar, cabe aceptar —Alvarado [1979]— que una matriz $n \times n$ es dispersa si el número de elementos no nulos es $n^{\gamma+1}$, donde $\gamma < 1$. Valores típicos de este parámetro suelen ser: 0,2 para problemas de análisis de sistemas eléctricos de generación y transporte de energía; 0,5 para matrices en banda asociadas a problemas de análisis de estructuras, etc.

A pesar de estas apreciaciones, si aplicamos el más estricto sentido común a la cuestión, una matriz se debe considerar en general dispersa, sea cual sea su número de elementos distintos de cero, si merece la pena manipularla mediante procedimientos *ad hoc* diseñados con ese propósito.

En este capítulo nos proponemos estudiar las matrices dispersas y cómo resolver por métodos directos sistemas de ecuaciones lineales de grandes dimensiones cuyas matrices de coeficientes son dispersas. Este estudio, grosso modo, se divide en tres grandes áreas:

- Almacenamiento de las matrices dispersas en un ordenador.
- Ordenación de las ecuaciones e incógnitas del sistema a resolver con el fin de reducir el número de elementos no nulos que se crean al factorizar la matriz.
- Adaptación e implementación eficaz de los métodos numéricos directos tradicionales con el fin de resolver sistemas de ecuaciones lineales con matriz de coeficientes dispersa.

3.1 Almacenamiento en ordenador de matrices dispersas

La efectividad del trabajo con matrices dispersas se mide no sólo en términos de la de los algoritmos que las manipulan sino también por la forma en que el ordenador se integra dentro del proceso que generan esos algoritmos. En este sentido, cuanto más eficaz es el esquema según el cual se almacenan las matrices dispersas en un ordenador y cuanto más ágilmente se pueda recuperar la información relativa a las mismas, mejores serán las prestaciones de los algoritmos que las manipulan.

Las estructuras de datos y los fragmentos de programas de ordenador que se presentan en este apartado y el siguiente están pensados para su codificación en FORTRAN 77. En el apéndice G, apartado G.1.3, se estudian los equivalentes para su codificación en C.

Los esquemas de almacenamiento que se presentan a continuación no son los únicos, aunque sí los más usados. Una buena revisión de éstos y muchos otros se pueden estudiar en Saad [1994].

3.1.1 Almacenamiento por coordenadas

La forma más intuitiva de almacenar en un ordenador los elementos no nulos de una matriz dispersa es haciéndolo mediante un conjunto ordenado o desordenado de triples (a_{ij}, i, j) , donde $a_{ij} \neq 0$.

Por ejemplo, si se quiere almacenar la matriz

$$A = \begin{bmatrix} 1 & 0 & 0 & -1 & 0 \\ 2 & 0 & -2 & 0 & 3 \\ 0 & -3 & 0 & 0 & 0 \\ 0 & 4 & 0 & -4 & 0 \\ 5 & 0 & -5 & 0 & 6 \end{bmatrix}$$

según este esquema, en FORTRAN 77 se podría hacer mediante la definición de tres vectores, **ifi**, **ico** y **val**: los dos primeros basta que sean **integer**; **val** debe ser **real**. En la siguiente tabla se pueden ver los valores de estos tres vectores para definir por filas, completa y ordenadamente, la matriz *A*.

Vector	Elementos										
	1	2	3	4	5	6	7	8	9	10	11
ifi	1	1	2	2	2	3	4	4	5	5	5
ico	1	4	1	3	5	2	2	4	1	3	5
val	1	-1	2	-2	3	-3	4	-4	5	-5	6

Esta forma de almacenamiento, aunque en su variante desordenada todavía se usa en alguno de los paquetes de *software* para manejo de matrices dispersas, presenta un inconveniente muy importante: la dificultad de recuperar fácilmente un vector columna o fila de la matriz. En su forma ordenada, el conjunto de instrucciones en FORTRAN 77 para recuperar en **vec**(*.*) el vector fila *i* podría ser el que sigue.

```

vec = 0
do ii=1,nelem
  if (ifi(ii).eq.i) then
    ia = ii
    do while (ifi(ia).eq.i)
      vec(ico(ia)) = val(ia)
      ia = ia+1
    end do
  exit
endif
end do

```

El índice *ii* que define el bucle principal de este programa podría también variar de *n* a 1, o en el rango que se considerase oportuno, buscando optimizar el tiempo de búsqueda del comienzo de la fila *i*.

En el caso de hacerse un almacenamiento ordenado, una dificultad añadida surge cuando se quiere hacer distinto de cero un valor que originalmente es cero.

3.1.2 Almacenamiento por filas o columnas

Esta forma de almacenar matrices dispersas es una de las más usadas para matrices sin ninguna estructura particular. Si nos referimos a la variante de almacenamiento por filas (por columnas sería igual con los cambios obvios), consiste en definir tres vectores: el primero, por ejemplo **val**, debe contener todos los elementos distintos de cero de la matriz, agrupados por filas; el segundo, **ico** por ejemplo, de la misma dimensión de **val**, los subíndices columna de los elementos de **val**; el tercero, un vector de punteros, **ia** por ejemplo, de dimensión *n* + 1, las

posiciones en **val** y **ico** del primer elemento no nulo de las filas que se corresponden con el orden de los elementos de **ia**.

Si consideramos de nuevo la matriz A definida en el apartado anterior, sus elementos, según este nuevo esquema, se almacenarían como indica la tabla siguiente.

Vector	Elementos										
	1	2	3	4	5	6	7	8	9	10	11
ia	1	3	6	7	9	12					
ico	1	4	1	3	5	2	2	4	1	3	5
val	1	-1	2	-2	3	-3	4	-4	5	-5	6

Obsérvese que, como hemos indicado, la dimensión de **ia** debe ser $n + 1$, pues es necesario definir el número de elementos no nulos de la última fila n .

Los valores de los elementos de cada fila da igual guardarlos en orden o en desorden. En general, la información relativa a la fila r de una matriz A estará en las posiciones **ia**(r) a **ia**($r+1$)-1 de **ico** y **val**, excepto cuando **ia**($r+1$)=**ia**(r), en cuyo caso la fila r estaría vacía.

La parte de un programa en FORTRAN 77 que recuperase en **vec**($\cdot$) la fila i de una matriz dispersa definida por los vectores **ia**, **ico** y **val** podría ser como la que sigue.

```
vec = 0
in = ia(i)
if = ia(i+1)-1
do ii=in,if
    vec(ico(ii)) = val(ii)
end do
```

Obsérvese lo sencillo que resulta recuperar una fila con este esquema si se compara con el del apartado anterior.

Un programa para recuperar la columna k sería un poco más complicado.

```
vec = 0
do j=1,m
    do ii=ia(j),ia(j+1)-1
        if (ico(ii).gt.k) exit
        if (ico(ii).lt.k) cycle
        vec(j) = val(ii)
    end do
end do
```

Por lo que respecta al almacenamiento de un nuevo elemento no nulo que se cree a lo largo de un proceso de manipulación de una matriz dispersa, las dificultades son grandes: habría que redefinir toda la estructura.

3.1.3 Almacenamiento por perfil o envolvente

Uno de los tipos de matrices dispersas más habituales lo constituye el denominado *matrices en banda*. Son matrices cuyos elementos están contenidos en una estrecha banda, normalmente alrededor de la diagonal principal de la matriz.

Definición 3.1 Una matriz $A \in \mathfrak{R}^{m \times n}$ se dice tiene un *ancho de banda* de filas w si

$$w = \max_{1 \leq i \leq m} w_i, \quad w_i = (l_i - f_i + 1),$$

donde w_i es el ancho de banda de la fila i , $f_i = \min\{j : a_{ij} \neq 0\}$ y $l_i = \max\{j : a_{ij} \neq 0\}$.

Para que sea de interés tener en cuenta que una matriz dispersa dispone de esta estructura, se ha de cumplir que $w \ll n$. Matrices de estas características surgen muy frecuentemente en modelos que plasman situaciones físicas donde sólo se influyen las variables que representan magnitudes cercanas en el espacio, en el tiempo, etc.

El almacenamiento de perfil o envolvente está diseñado para sacar partido de esa estructura en banda. De cada fila i se almacenan todos los elementos de subíndice ij tales que $f_i \leq j \leq l_i$.

Definición 3.2 El conjunto de elementos que forman la *envolvente de una matriz* A , $\text{Env}(A)$, es

$$\text{Env}(A) = \{(i, j) : f_i \leq j \leq l_i, 1 \leq i \leq n\}.$$

La envolvente de la matriz

$$A = \begin{array}{c} \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix} \end{array} \left[\begin{array}{ccccccc} \times & \times & \times & \times & & & \\ \times & \times & 0 & \times & & & \\ & & \times & 0 & \times & \times & \\ & \times & 0 & \times & & & \\ & & & \times & \times & \times & \times \\ & \times & \times & 0 & 0 & \times & \times \\ & & & & & & \times \end{array} \right]$$

es la que forman los elementos inscritos en el polígono, es decir,

$$\begin{aligned} \text{Env}(A) = \{ & (1, 1), (1, 2), (1, 3), (1, 4), (2, 1), (2, 2), (2, 3), (2, 4), \\ & (3, 3), (3, 4), (3, 5), (3, 6), (4, 2), (4, 3), (4, 4), \\ & (5, 4), (5, 5), (5, 6), (5, 7), (6, 2), (6, 3), (6, 4), (6, 5), (6, 6), (6, 7), (7, 7) \}. \end{aligned}$$

El esquema de almacenamiento por perfil guarda todos los elementos de la envolvente mediante tres vectores: el primero, **val**, contiene todos los elementos de la envolvente; el segundo, **ifa**, los índices f_i de cada fila i ; el tercero, un vector de punteros, **ia**, de dimensión $m + 1$ si $A \in \mathfrak{R}^{m \times n}$, las posiciones en **val** del primer elemento no nulo de las filas que se corresponden con el orden de los elementos de **ia**.

Por ejemplo, si se quiere almacenar según su perfil o envolvente la matriz

$$A = \begin{bmatrix} 1 & 0 & -2 & 0 & 0 \\ 2 & 3 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 & 0 \\ 0 & -4 & 0 & 4 & 0 \\ 0 & 0 & 0 & 3 & 1 \\ 0 & 0 & 0 & 0 & 6 \end{bmatrix},$$

los vectores que habría que definir son los de la siguiente tabla.

Vector	Elementos											
	1	2	3	4	5	6	7	8	9	10	11	12
ia	1	4	6	7	10	12						
ifa	1	1	3	2	4	5						
val	1	0	-2	2	3	6	-4	0	4	3	1	6

Obsérvese que, con respecto al esquema por filas o columnas, se ha reducido el número de posiciones de memoria necesarias, pues la dimensión del vector **ifa** debe ser sensiblemente inferior a la de **ico** de aquél.

La parte de un programa en FORTRAN 77 que recuperase una fila de la matriz A almacenada según este esquema podría ser como la que sigue.

```

vec = 0
in  = ia(i)
if  = ia(i+1)-1
j   = 0
do ii=in,if
    vec(ifa(i)+j) = val(ii)
    j = j+1
end do

```

Si la matriz dispersa que hay que guardar con el esquema envolvente es simétrica, sólo será necesario guardar la parte triangular inferior o superior (incluida la diagonal principal).

Definición 3.3 El ancho de banda (o de semibanda) de una matriz simétrica $B \in \Re^{n \times n}$, β , se define como

$$\beta = \max_{1 \leq i \leq n} \beta_i, \quad \beta_i = i - f_i,$$

donde β_i es el ancho de banda de la fila i (o, simplemente, el ancho de banda i -ésimo) de B .

De forma similar a como lo hacíamos anteriormente, la envolvente de una matriz simétrica B , $\text{Env}(B)$, se define como

$$\text{Env}(B) = \{(i, j) : f_i \leq j \leq i, 1 \leq i \leq n\}.$$

Para almacenar en un ordenador una matriz simétrica según el esquema de perfil o envolvente no sería necesario el vector **ifa**. Por ejemplo, la matriz

$$B = \begin{bmatrix} 10 & 2 & 3 & 0 & 0 \\ 2 & 4 & 0 & 3 & 0 \\ 3 & 0 & 6 & 0 & 0 \\ 0 & 3 & 0 & 1 & 8 \\ 0 & 0 & 0 & 8 & 3 \end{bmatrix}$$

se podría almacenar mediante los vectores de la tabla que sigue.

Vector	Elementos									
	1	2	3	4	5	6	7	8	9	10
ib	1	4	7	8	10					
val	10	2	3	4	0	3	6	1	8	3

Obsérvese que en este caso la dimensión del vector **ib** es n .

Si los anchos de banda, β_i , de las filas de una matriz simétrica son iguales, el vector **ib**, incluso, no sería necesario: sólo **val** y el número β_i .

3.1.4 Almacenamiento por listas encadenadas

La idea básica de esta forma de almacenamiento radica en que a cada elemento no nulo de una fila o columna se le asocia un puntero o eslabón que indica donde está el siguiente elemento no nulo de esa fila o columna. Si este puntero es cero, el mencionado elemento es el último no nulo de la fila o columna.

Para almacenar una matriz dispersa con este esquema se utilizan varios vectores (hasta siete distintos según el autor). Con carácter general podrían ser: **val**, en donde se almacenan, en cualquier orden, los valores de los elementos no nulos de la matriz; **ico**, de la misma dimensión de **val**, donde se guardan los subíndices columna de los elementos de **val**; **ifi** donde se guardan las posiciones en **ico** y **val** de los primeros elementos no nulos de cada fila o columna de la matriz; por último, **link**, de dimensión igual a **ico** y **val**, donde se almacenan los punteros antes mencionados, es decir, la posición en **val** e **ico** del siguiente elemento no nulo de la misma fila, si es que existe; si no es así, el puntero se hace cero.

Si se quiere guardar la matriz

$$B = \begin{bmatrix} -1 & 6 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 2 & 0 & 3 & 0 & 0 & 1 \\ 3 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

siguiendo el esquema de la listas encadenadas, unos posibles valores de los vectores antes indicados serían los de la siguiente tabla.

Vector	Elementos								
	1	2	3	4	5	6	7	8	9
val	6	2	2	-1	3	1	3	1	1
ico	2	1	4	1	1	6	3	4	6
ifi	4	6	2	5					
link	3	7	0	1	8	0	9	0	0

Como se puede observar, el número de posiciones de memoria que este esquema necesita es bastante superior al requerido por los presentados en apartados anteriores.

Recuperar en **vec**($\cdot$) la fila i de una matriz dispersa almacenada según este esquema mediante un programa en FORTRAN 77 sería tan sencillo como sigue.

```
vec = 0
ii = ifi(i)
do while (ii.ne.0)
```

```
      vec(ico(ii)) = val(ii)
      ii = link(ii)
end do
```

La gran ventaja de las listas encadenadas radica en lo relativamente fácil que resulta introducir en sus estructuras de datos elementos distintos de cero en posiciones de la matriz que previamente ocupaban ceros. En efecto, para insertar un nuevo elemento no nulo en la posición (i, j) habría que llevar a cabo las siguientes sencillas operaciones:

1. Añadir ese elemento al final de `val`.
2. Añadir el subíndice j al final de `ico`.
3. Comprobar si ese elemento va a ser el primero de la fila i : si es así, hacer `ifi(i)` igual a la posición en `val` que ocupará el nuevo elemento; si no, dejar `ifi` como estaba.
4. Si el nuevo elemento va a ser el último de la fila, cambiar el valor de `link` del último elemento anterior por el del que ocupará el nuevo elemento en `val` y asignar el correspondiente `link` a cero; si no, cambiar el valor de `link` del anterior elemento distinto de cero por el de la posición que ocupará el nuevo elemento en `val` y asignar al `link` de éste el del siguiente elemento en la fila distinto de cero.

Estas operaciones son fácilmente implementables en cualquier lenguaje de programación.

3.2 Operaciones algebraicas elementales con matrices dispersas

En este apartado vamos a estudiar cómo se pueden realizar operaciones algebraicas elementales entre matrices y vectores tratados como dispersos. Nos interesaremos sólo por aquellas operaciones que realizan habitualmente los algoritmos que estudiamos en este libro: producto interior de dos vectores, multiplicación de matrices por vectores y suma, trasposición y multiplicación de matrices ($A^T A$ como caso particular).

3.2.1 Producto interior de dos vectores

Supongamos que se desea calcular el producto interior,

$$h = \sum_{i=1}^n a_i b_i,$$

de dos vectores $\mathbf{a}$ y $\mathbf{b}$ almacenados como dispersos según el esquema de filas del apartado 3.1.2. Al tratarse de vectores (una sola fila), el vector `ia` que allí se definía no sería necesario; sí lo serían `ico` y `val`. Habrá que conocer también el número de componentes no nulos de $\mathbf{a}$ y $\mathbf{b}$.

Una forma directa de llevar este producto interior a cabo sería comprobando, para cada componente no nulo de $\mathbf{a}$, si el correspondiente componente de $\mathbf{b}$ es cero, y caso de no serlo, multiplicando esos componentes y acumulando el resultado en h . Llevar esto a efecto en FORTRAN 77, suponiendo que el almacenamiento es ordenado, sería como sigue.

```

h = 0.0
do i=1,na
  do j=1,nb
    if (icob(j).gt.icoa(i)) exit
    if (icob(j).lt.icoa(i)) cycle
    h = h+vala(icoa(i))*valb(icob(j))
  exit
end do
end do

```

Proceder de esta forma es sumamente ineficaz pues hay que inspeccionar el vector $\mathbf{a}$ o el $\mathbf{b}$ un número de veces proporcional al producto de elementos no nulos de $\mathbf{a}$ y de $\mathbf{b}$.

Una forma mucho más eficaz de hacer ese producto interior, válida incluso cuando el almacenamiento es desordenado, consiste en definir un nuevo vector $\mathbf{ip}$, de dimensión n , en el que se guarden los punteros de los elementos no nulos de, según se desee, $\mathbf{vala}$ o $\mathbf{valb}$. Por ejemplo, si el vector $\mathbf{a}$ está definido por

Vector	Elementos			
	1	2	3	4
icoa	10	3	7	4
vala	0,2	0,3	0,4	-0,5

el vector $\mathbf{ip}$, una vez almacenados los punteros, quedaría

Vector	Elementos											
	1	2	3	4	5	6	7	8	9	10	11	...
ip	0	0	2	4	0	0	3	0	0	1	0	...

lo cual quiere decir que a_3 está en la posición 2 de $\mathbf{vala}$, a_4 en la posición 4, a_7 en la 3, etc. A continuación, conocidos los elementos no nulos de $\mathbf{b}$, se usa $\mathbf{ip}$ y, si ha lugar, se multiplican los componentes acumulando el resultado en h . Si, por ejemplo, el vector $\mathbf{b}$ está definido por

Vector	Elementos		
	1	2	3
icob	5	4	10
valb	0,6	0,7	0,5

el primer componente no nulo de $\mathbf{b}$ es $b_5 = 0,6$. Ahora bien, $\mathbf{ip}(5)=0$, por lo que $a_5b_5 = 0$, no siendo necesario efectuar esta última operación ... Estas ideas expresadas en FORTRAN 77 darían lugar a un conjunto de instrucciones como el que sigue.

```

ip = 0
do i=1,na
  ip((icoa(i))) = i
end do
h = 0.0
do i=1,nb
  if (ip(icob(i)).ne.0) h=h+vala(ip(icob(i)))*valb(i)
end do

```

Es importante tener en cuenta que hacer $\mathbf{ip}(\dots)=0$ es caro: es necesario realizar muchas operaciones, aunque triviales, si n es grande. Si un vector se multiplica por otros muchos (caso por ejemplo de productos de matrices), evidentemente, sólo es necesario inicializar $\mathbf{ip}$ a cero una vez.

3.2.2 Multiplicación de matrices por vectores

En este apartado supondremos que el vector por el que se ha de multiplicar la matriz dispersa está almacenado en toda su extensión. Para llegar a tal vector se pueden aplicar las ideas apuntadas para recuperar un vector al introducir los distintos modos de almacenar matrices dispersas. En cualquier caso, lo que se expone a continuación se puede extender sin ninguna dificultad al caso en que el vector por el que se multiplica la matriz esté almacenado también en forma dispersa. Casos particulares, como el que se presenta cuando la matriz por la que multiplicar es simétrica, facilitarían las operaciones; nos ocuparemos, sin embargo, del caso más general posible.

3.2.2.1 Multiplicación de una matriz por un vector

Nos interesamos en primer lugar por la operación

$$\mathbf{c} = A\mathbf{b},$$

donde $A \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^n$ y $\mathbf{c} \in \mathbb{R}^m$.

Si suponemos que el vector $\mathbf{c}$ se guarda en toda su extensión y la matriz A según el esquema por filas, para realizar la operación basta saber qué elementos son no nulos en cada fila de la matriz A , multiplicarlos por el correspondiente de $\mathbf{b}$ y acumular el resultado. En FORTRAN 77 esto sería tan sencillo como lo que se lista a continuación.

```
do i=1,m
  s = 0.
  do ii=ia(i),ia(ii+1)-1
    s = s+val(ii)*b(ico(ii))
  end do
  c(i) = s
end do
```

3.2.2.2 Multiplicación de un vector por una matriz

En este caso queremos efectuar la operación

$$\mathbf{c}^T = \mathbf{b}^T A,$$

donde $A \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$ y $\mathbf{c} \in \mathbb{R}^n$.

Consideremos el siguiente ejemplo simbólico de este producto,

$$[c_1 \ c_2 \ c_3] = [b_1 \ b_2] \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}.$$

Efectuando las operaciones del producto de un vector por una matriz, se tiene que,

$$\begin{aligned} c_1 &= b_1 a_{11} + b_2 a_{21} \\ c_2 &= b_1 a_{12} + b_2 a_{22} \\ c_3 &= b_1 a_{13} + b_2 a_{23}. \end{aligned}$$

Usar estas ecuaciones resultaría altamente ineficaz si la matriz está almacenada por filas. Reescribamos las ecuaciones de la siguiente manera,

$$\begin{aligned}c_1 &\leftarrow b_1 a_{11} \\c_2 &\leftarrow b_1 a_{12} \\c_3 &\leftarrow b_1 a_{13} \\c_1 &\leftarrow c_1 + b_2 a_{21} \\c_2 &\leftarrow c_2 + b_2 a_{22} \\c_3 &\leftarrow c_3 + b_2 a_{23}.\end{aligned}$$

En este caso se puede acceder a los elementos secuencialmente por filas e ir acumulando los resultados de las operaciones en los propios elementos de $\mathbf{c}$

En FORTRAN 77, teniendo en cuenta que la matriz A se almacena por filas, este producto se haría como sigue.

```
c = 0
do i=1,n
  bi = b(i)
  do ii=ia(i),ia(i+1)-1
    j = ico(ii)
    c(j) = c(j)+val(ii)*bi
  end do
end do
```

3.2.3 Suma de matrices dispersas

La suma o resta de matrices dispersas es una de las operaciones algebraicas más utilizada. Para efectuar la suma vamos a utilizar un procedimiento basado en dos etapas: la *etapa simbólica* y la *numérica*. En la primera se determinará la estructura de la matriz resultante, en este caso la matriz suma, y en la segunda, una vez reservada la memoria correspondiente en la etapa anterior para almacenar el resultado, los valores de los elementos no cero. Aunque evidentemente estas operaciones se pueden realizar de una sola vez, se gana poco procediendo así mientras que haciéndolo en dos etapas, si sólo cambian los datos numéricos y se conserva la estructura (como por ejemplo en un proceso iterativo o cuando hay que ordenar las posiciones de los elementos no nulos independientemente del valor que tengan), solamente es necesario realizar la segunda etapa, reduciéndose en gran medida el número global de operaciones a llevar a cabo.

En lo que sigue vamos a suponer que las dos matrices a sumar o restar, $A \in \mathbb{R}^{m \times n}$ y $B \in \mathbb{R}^{m \times n}$, están almacenadas por filas.

3.2.3.1 Suma o resta simbólica

Para llevar a cabo la etapa simbólica se suele utilizar, como se hizo para efectuar el producto interior de dos vectores, un vector auxiliar, $\mathbf{ip}$, de dimensión n , inicializado a cero, en el que se reflejan qué elementos de cada fila en la matriz resultante C van a ser distintos de cero.

Una forma de proceder, fila por fila, sería la siguiente: 1. examinar los componentes del vector $\mathbf{icoa}$ de la matriz A , fila i ; 2. hacer igual a i los componentes de $\mathbf{ip}$ que indiquen los $\mathbf{icoa}$ anteriores; 3. examinar los componentes del vector $\mathbf{icob}$ de la matriz B , fila i ; 4. hacer igual a i , si no lo están ya por haberlos hecho en 2., los componentes de $\mathbf{ip}$ que indiquen los $\mathbf{icob}$ anteriores; 5. definir, a partir de los componentes de $\mathbf{ip}$ que sean igual a i , el vector $\mathbf{icoc}$

y, por último, 6. definir $\text{ic}(\text{i}+1)$ de acuerdo con el número de componentes que van a ser no nulos en la matriz C .

Ejemplo 3.1 Sumemos las dos matrices

$$A = \begin{bmatrix} 0 & 0 & 2 & 0 & -1 & 0 \\ 4 & 0 & 3 & 3 & 7 & 0 \\ -2 & 0 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 1 & 0 & 0 \end{bmatrix} \quad \text{y} \quad B = \begin{bmatrix} 1 & 0 & -1 & 0 & 0 & 5 \\ 0 & 0 & 0 & 0 & -2 & 0 \\ 4 & 6 & 0 & 2 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 & 0 \end{bmatrix}.$$

Almacenadas por filas, las matrices A y B están representadas por los vectores de las dos tablas siguientes.

Matriz A										
ia	1	3	7	9	11					
icoa	3	5	1	3	4	5	1	6	2	4
vala	2	-1	4	3	3	7	-2	-1	1	1

Matriz B										
ib	1	4	5	8	10					
icob	1	3	6	5	1	2	4	2	3	
valb	1	-1	5	-2	4	6	2	-1	1	

Efectuemos la suma de A y B en un número de pasos o etapas igual al número de filas de ambas matrices: 4.

Paso 1

Examinemos los componentes de **icoa** correspondientes a la fila 1 y hagamos igual a 1 dichos componentes de **ip**. Resultará lo que sigue.

Vector	Elementos					
	1	2	3	4	5	6
ip	0	0	1	0	1	0

Examinemos los componentes de **icob** correspondientes a la fila 1 y hagamos, igual a 1, si no lo están ya, dichos componentes de **ip**.

Vector	Elementos					
	1	2	3	4	5	6
ip	1	0	1	0	1	1

Después de analizadas la fila 1 de las dos matrices A y B , los vectores **ic** y **icoc** de la matriz resultante C serán los que se indican en la siguiente tabla.

Matriz C				
ic	1	5		
icoc	1	3	5	6

Paso 2

Examinemos los componentes de **icoa** correspondientes a la fila 2 y hagamos igual a 2 dichos componentes de **ip**.

Vector	Elementos					
	1	2	3	4	5	6
ip	2	0	2	2	2	1

Repasemos los componentes de **icob** correspondientes a la fila 2 y hagamos igual a 2, si no lo están ya, dichos componentes de **ip**. Resultará lo que sigue.

Vector	Elementos					
	1	2	3	4	5	6
ip	2	0	2	2	2	1

Obsérvese que los valores de **ip** iguales a uno que no se han modificado permanecen inalterados al no reinicializarse en cada paso el vector **ip**.

Después de analizadas la fila 2 de las dos matrices *A* y *B*, los vectores **ic** y **icoc** de la matriz resultante *C* serán los que se indican en la siguiente tabla.

Matriz <i>C</i>								
ic	1	5	9					
icoc	1	3	5	6	1	3	4	5

Paso 3

Examinemos los componentes de **icoa** correspondientes a la fila 3 y hagamos igual a 3 dichos componentes de **ip**.

Vector	Elementos					
	1	2	3	4	5	6
ip	3	0	2	2	2	3

Examinemos luego los componentes de **icob** correspondientes a la fila 3 y hagamos igual a 3, si no lo están ya, dichos componentes de **ip**.

Vector	Elementos					
	1	2	3	4	5	6
ip	3	3	2	3	2	3

Después de analizadas la fila 3 de las dos matrices *A* y *B*, los vectores **ic** y **icoc** de la matriz resultante *C* serán los siguientes.

Matriz <i>C</i>												
ic	1	5	9	13								
icoc	1	3	5	6	1	3	4	5	1	2	4	6

Paso 4

Examinemos los componentes de **icoa** correspondientes a la fila 4 y hagamos igual a 4 dichos componentes de **ip**.

Vector	Elementos					
	1	2	3	4	5	6
ip	3	4	2	4	2	3

Hagamos lo mismo con **icob**.

Vector	Elementos					
	1	2	3	4	5	6
ip	3	4	4	4	2	3

Después de analizadas la fila 4 de las dos matrices *A* y *B*, los vectores **ic** y **icoc** de la matriz resultante *C* serán los que se indican a continuación.

Matriz C															
ic	1	5	9	13	16										
icoc	1	3	5	6	1	3	4	5	1	2	4	6	2	3	4

Obsérvese que el componente 2 de la última fila resultará cero, aunque según lo expuesto figure como distinto de cero. ●

Un programa en FORTRAN 77 que lleve a cabo las operaciones descritas para efectuar la suma simbólica de dos matrices dispersas puede ser como el que sigue.

```

ip = 0
nu = 1
do i=1,m
  ic(i) = nu
  do ii=ia(i),ia(i+1)-1
    j = icoa(ii)
    icoc(nu) = j
    nu = nu+1
    ip(j) = i
  end do
  do ii=ib(i),ib(i+1)-1
    j = icob(ii)
    if (ip(j).eq.i) cycle
    icoc(nu) = j
    nu = nu+1
  end do
end do
ic(m+1) = nu

```

3.2.3.2 Suma o resta numérica

Efectuada la suma o resta simbólica, la parte numérica no representa mayor dificultad. Se utiliza un vector auxiliar **x**, de dimensión igual al número de filas de las matrices a sumar o restar, *m*, en el que se acumulan la suma de los elementos no nulos. Las posiciones de este

vector que designan los valores de los elementos de `icoc` se inicializan a cero antes de realizar esa acumulación.

Una implementación en FORTRAN 77 de esta suma o resta numérica es la que sigue a continuación.

```

do i=1,m
  do ii=ic(i),ic(i+1)-1
    x(icoc(ii)) = 0.0
  end do
  do ii=ia(i),ia(i+1)-1
    x(icoa(ii)) = vala(ii)
  end do
  do ii=ib(i),ib(i+1)-1
    j = icob(ii)
    x(j) = x(j)+valb(ii)
  end do
  do ii=ic(i),ic(i+1)-1
    valc(ii) = x(icoc(ii))
  end do
end do

```

3.2.4 Multiplicación de matrices dispersas

Analicemos de qué forma se puede calcular eficazmente el producto de dos matrices dispersas cualesquiera, $A \in \mathbb{R}^{m \times p}$ y $B \in \mathbb{R}^{p \times n}$. Los elementos de la matriz producto, $C \in \mathbb{R}^{m \times n}$, son, como es sabido,

$$c_{ij} = \sum_{k=1}^p a_{ik} b_{kj}, \quad \text{para } i = 1, \dots, m; \quad j = 1, \dots, n.$$

Esta fórmula indica que cualquier elemento de la matriz resultante, c_{ik} , es el producto interior de un vector fila de la matriz A , fila i , por un vector columna de la matriz B , columna k . Si, como venimos considerando, las matrices A y B se almacenan por filas, llevar a cabo ese producto interior, según la fórmula, presenta ciertas ineficacias numéricas. Para abordar este producto utilizaremos las mismas consideraciones del apartado 3.2.2.2, cuando estudiábamos el producto de un vector por una matriz. Analicemos el producto simbólico:

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}.$$

Efectuando las operaciones de acuerdo con las reglas de producto de matrices, se tiene que,

$$\begin{aligned} c_{11} &= a_{11}b_{11} + a_{12}b_{21} \\ c_{12} &= a_{11}b_{12} + a_{12}b_{22} \\ c_{21} &= a_{21}b_{11} + a_{22}b_{21} \\ c_{22} &= a_{21}b_{12} + a_{22}b_{22}. \end{aligned}$$

Con vistas a su implementación en ordenador, reordenemos los cálculos de la siguiente manera,

$$\begin{aligned}x_1 &\leftarrow a_{11}b_{11} \\x_2 &\leftarrow a_{11}b_{12} \\x_1 &\leftarrow x_1 + a_{12}b_{21} \\x_2 &\leftarrow x_2 + a_{12}b_{22} \\c_{11} &\leftarrow x_1 \\c_{12} &\leftarrow x_2 \\x_1 &\leftarrow a_{21}b_{11} \\x_2 &\leftarrow a_{21}b_{12} \\x_1 &\leftarrow x_1 + a_{22}b_{21} \\x_2 &\leftarrow x_2 + a_{22}b_{22} \\c_{21} &\leftarrow x_1 \\c_{22} &\leftarrow x_2.\end{aligned}$$

Obsérvese que con esta forma de proceder cada elemento de la matriz A se multiplica secuencialmente por todos los elementos de un vector fila de la matriz B ; éstos últimos son fácilmente accesibles de acuerdo con el esquema de almacenamiento que se utiliza.

Para llevar a efecto el producto de matrices también se utiliza la estrategia introducida en el párrafo anterior, es decir, hacerlo en dos etapas: una simbólica y otra numérica. La parte simbólica obtendrá la estructura de la futura matriz C , de la que se servirá con posterioridad la parte numérica.

Un programa en FORTRAN 77 que permite obtener el producto simbólico de dos matrices $A \in \mathbb{R}^{m \times p}$ y $B \in \mathbb{R}^{p \times n}$, definidas por los vectores $\mathbf{ia}$, $\mathbf{icoa}$ y $\mathbf{vala}$ y $\mathbf{ib}$, $\mathbf{icob}$ y $\mathbf{valb}$, podría contar con instrucciones como las que se listan a continuación.

```
ip = 0
nu = 1
do i=1,m
  ic(i) = nu
  do ii=ia(i),ia(i+1)-1
    j = icoa(ii)
    do iii=ip(j),ib(j+1)-1
      k = icob(iii)
      if (ip(k).eq.i) cycle
      icoc(nu) = k
      nu      = nu+1
      ip(k)    = i
    end do
  end do
end do
ic(m+1) = nu
```

La dimensión del vector auxiliar $\mathbf{ip}$ es p , el número de columnas de B .

Otro programa para efectuar posteriormente el producto numérico es el siguiente.

```
do i=1,m
  do ii=ic(i),ic(i+1)-1
    x(icoc(ii)) = 0.0
  end do
  do ii=ia(i),ia(i+1)-1
    j = icoa(ii)
    a = vala(ii)
```

```

do iii=ib(j),ib(j+1)-1
  k = icob(iii)
  x(k) = x(k)+a*valb(iii)
end do
end do
do ii=ic(i),ic(i+1)-1
  valc(ii) = x(icoc(ii))
end do
end do

```

Otra alternativa para efectuar el producto de dos matrices dispersas es la siguiente: si la matriz A se escribe como un conjunto de p vectores columna, esto es,

$$A = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_p],$$

y la B como otro conjunto de p vectores fila, es decir,

$$B = \begin{bmatrix} \mathbf{b}_1^T \\ \mathbf{b}_2^T \\ \vdots \\ \mathbf{b}_p^T \end{bmatrix},$$

el producto $C = AB$ se puede expresar como la suma de p matrices de rango uno, de la forma

$$C = AB = \sum_{i=1}^p \mathbf{a}_i \mathbf{b}_i^T.$$

Para llevar este producto a la práctica en ordenador, supuestas A y B almacenadas por filas, habría que extraer cada vector columna de A y multiplicarlo por todos los vectores fila de B , llevando constancia de qué resultados son cero.

3.2.4.1 Multiplicación $A^T A$

Un caso particular de producto de dos matrices dispersas lo constituye $A^T A$. Para llevarlo a efecto eficazmente se puede proceder de varias formas. La más extendida es aquella que utiliza las ideas apuntadas al final del párrafo anterior. Es decir, servirse de matrices de rango uno, haciendo

$$A^T A = \sum_{i=1}^m \mathbf{a}_i \mathbf{a}_i^T,$$

donde $\mathbf{a}_i$ es el vector fila i -ésimo de la matriz $A \in \mathbb{R}^{m \times n}$.

En este apartado, sin embargo, utilizaremos un enfoque distinto: trasponer la matriz A mediante un algoritmo general y multiplicar el resultado por la propia matriz A utilizando los algoritmos anteriores.

Trasposición de A

La sencilla operación de trasponer una matriz A en el caso general, se complica cuando se trata de una matriz dispersa. En lo que sigue, supondremos que la matriz A está almacenada por filas.

Para llevar a cabo la trasposición de A se definen tantas listas, o vectores lista, como columnas tiene A , apuntándose en ellas secuencialmente los índices de las columnas que ocupan los elementos distintos de cero. Por ejemplo, si

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & \times & 0 & \times & \times \\ \times & 0 & 0 & \times & 0 & 0 \\ 0 & 0 & \times & \times & 0 & 0 \\ \times & 0 & \times & \times & 0 & 0 \\ 0 & \times & 0 & 0 & \times & \times \end{bmatrix} \end{matrix}$$

estas listas, después de analizados qué elementos de la primera fila son distintos de cero, contendrán

1 :
2 :
3 : 1
4 :
5 : 1
6 : 1.

Después de analizada la segunda fila,

1 : 2
2 :
3 : 1
4 : 2
5 : 1
6 : 1.

Al final serán

1 : 2 4
2 : 5
3 : 1 3 4
4 : 2 3 4
5 : 1 5
6 : 1 5

de donde se deduciría inmediatamente un vector `icoat` con los índices columna de los elementos no nulos de cada fila de la matriz A^T .

En la práctica, estas listas se definen directamente en los vectores `icoat` y `valat`; en el vector `iat` se guardan los punteros del primer elemento de cada lista.

El algoritmo completo en FORTRAN 77 para llevar a cabo esta trasposición se lista a continuación.

```
1 -      iat = 0
2 -      do i=1,ia(m+1)-1                ! Determinación del número de
3 -          j = icoa(i)+2                !     elementos en cada lista.
4 -          if (j.le.n+1) iat(j)=iat(j)+1 !
5 -      end do
6 -      iat(1) = 1
7 -      iat(2) = 1
8 -      do i=3,n+1                      ! Determinación para I donde comienza
9 -          iat(i) = iat(i)+iat(i-1)      !     en JAT y valat la lista I-1.
```

```

10-      end do
11-      do i=1,m                                ! Incorporar elementos a cada lista
12-          do ii=ia(i),ia(i+1)-1                !
13-              j      = icoa(ii)+1              !
14-              k      = iat(j)                  !
15-              icoat(k) = i                      !
16-              valat(k) = vala(k)               !
17-              iat(j)  = k+1                    !
18-          end do
19-      end do

```

En las líneas 8 a 10 se determinan los punteros; `iat(i)` define la posición en `jat` y `valat` donde comienza la lista `i-1`, desde 1 a `n`. De esta forma, cuando se añaden elementos a la lista `i-1` y se incrementa su correspondiente puntero `iat(i)`, automáticamente `iat(i)` se convierte en el puntero del primer elemento de la lista `i`, la cual sigue a la `i-1` en los vectores `icoat` y `valat`.

El bucle que comienza en la línea 11 añade elementos a cada lista. A partir de la línea 12, en la línea 13 se encuentra el elemento de la matriz de la fila `i`, columna `j-1`. En la línea siguiente, `k` apunta a la posición de `icoat` y `valat` correspondiente a la primera posición libre de la lista `j-1`. En las dos líneas siguientes, 15 y 16, el índice de fila `i` y el valor `vala(ii)` se añaden a las listas `icoat` y `valat`, respectivamente.

3.3 Solución de grandes sistemas lineales de matriz dispersa

Como vimos al comienzo del capítulo, resolver sistemas de ecuaciones lineales de matriz de coeficientes dispersa lleva aparejadas tres tareas esenciales: el almacenamiento eficaz de la matriz, la ordenación de la ecuaciones para mantener la estructura de dispersidad al factorizar la matriz y la implementación eficaz del proceso de solución.

En los apartados anteriores nos hemos ocupado del almacenamiento en ordenador y de las operaciones algebraicas numéricas que se realizan en los algoritmos de forma que se tenga en cuenta el carácter disperso de las matrices afectadas. Como hemos podido comprobar, cualquiera de esas operaciones se puede adaptar con más o menos dificultad para tener en cuenta ese hecho.

En general, si las tres tareas apuntadas se estructuran convenientemente en un algoritmo, éste constaría de los pasos que se indican en la tabla 3.1.

3.3.1 Ordenación de las ecuaciones

Al resolver un sistema de ecuaciones lineales en el que la matriz de coeficientes que lo define es dispersa, el orden en que se disponen las filas o las columnas tiene una importancia fundamental. Por ejemplo, si consideramos la matriz simétrica A de un sistema de ecuaciones lineales cuyo patrón de elementos distintos de cero es el de la figura 3.1, y se utiliza la eliminación de Gauss para resolver dicho sistema, en el transcurso de dicha factorización se harán distintos de cero 46 elementos —*elementos de relleno* (*fill-in* en la literatura anglosajona)—: los que en la figura 3.2 aparecen sombreados.

Si las filas y las columnas del mismo sistema se reordenan de acuerdo con un criterio que veremos más adelante —algoritmo de grado mínimo—, obteniéndose un patrón de elementos

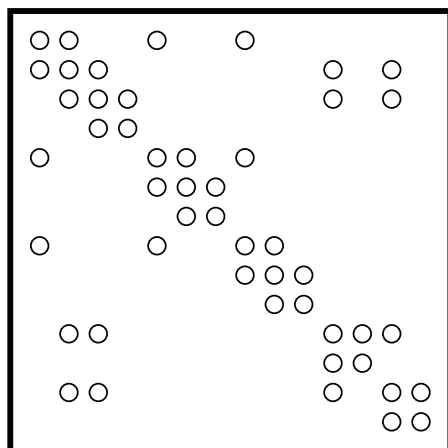


Figura 3.1

Estructura simbólica (simétrica) de una matriz 14×14 antes de proceder a su factorización mediante eliminación de Gauss

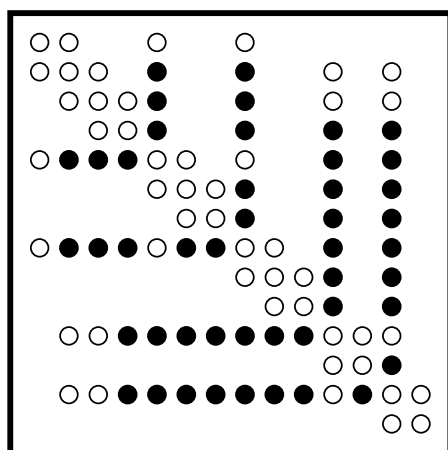


Figura 3.2

Estructura simbólica de la matriz de la figura 3.1 después de proceder a su factorización mediante eliminación de Gauss

Tabla 3.1

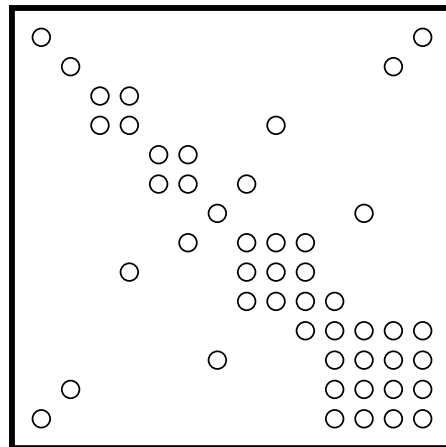
Algoritmo para resolver sistemas de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$, siendo A dispersa

- Paso 1** – Determinar la estructura simbólica de A .
- Paso 2** – Determinar unas permutaciones P y Q tales que PAQ tenga una estructura dispersa ventajosa en relación con el tipo de sistema a resolver.
- Paso 2'** – Factorizar simbólicamente la matriz PAQ y generar las estructuras de datos y memoria necesarias para L y U .
- Paso 3** – Obtener numéricamente $LU = PAQ$ y $\mathbf{c} = P\mathbf{b}$.
- Paso 4** – Resolver $L\mathbf{z} = \mathbf{c}$, $U\mathbf{y} = \mathbf{z}$ y, por fin, $\mathbf{x} = Q\mathbf{y}$.

distintos de cero como el de la figura 3.3, y esta matriz se factoriza también mediante eliminación de Gauss, el número de elementos cero que se hacen distintos de cero en este caso es cero. El vector que define las permutaciones que hay que efectuar simultáneamente en las filas y columnas de la matriz original para llegar a la de la figura 3.3 se puede ver en la siguiente tabla.

Vector	Elementos													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Permutación	14	12	10	9	7	6	4	5	8	1	2	3	11	13

Compruébese cómo la fila 1 original, fila 10 en la matriz reordenada, sigue teniendo 4 elementos no nulos, la fila 2 original, fila 11 en la reordenada, 5 elementos, etc. A las permutaciones que

**Figura 3.3**

Estructura simbólica de la matriz de la figura 3.1 después de proceder a la reordenación de sus filas y columnas mediante el algoritmo de grado mínimo y a su posterior factorización mediante eliminación de Gauss

actúan simultáneamente sobre filas y columnas se las denomina, como es sabido, *permutaciones simétricas*.

Trabajar con matrices reordenadas en las que el número de elementos de relleno se reduce considerablemente presenta tres ventajas fundamentales:

- Una disminución del número de posiciones de memoria que se han de reservar para los nuevos elementos que se harán distintos de cero en un proceso de factorización.
- Una disminución del número de operaciones a realizar y, por lo tanto, del tiempo total de cálculo para factorizar la matriz y resolver el correspondiente sistema.
- Una mejora de la estabilidad numérica del proceso global de resolución del sistema al disminuir el número de elementos a considerar y por tanto disminuir la probabilidad de encontrar grandes diferencias entre ellos, errores de cancelación, etc.

En la tabla 3.2 se resume en números las operaciones que habría que realizar para transformar la matriz presentada mediante eliminación de Gauss y, posteriormente, resolver el sistema lineal correspondiente, si se operase con esta matriz como si fuese densa, como si fuese dispersa pero sin reordenar filas y columnas y como si fuese dispersa reordenando filas y columnas, tal cual hemos visto.

Para definir una ordenación óptima es necesario tener en cuenta la estructura de la matriz, el esquema que define cómo se almacena la matriz y el tipo de operaciones que con ella se van a realizar. Si, como suele ser muy habitual, las matrices son simétricas y se almacenan según un esquema de perfil o envolvente (por dar lugar una vez ordenadas a matrices con estructura en banda), también interesa poder disponer de un procedimiento de ordenación que compacte los elementos precisamente cerca de la diagonal principal de la matriz. Este es el caso del algoritmo de Cuthill-McKee que veremos más adelante. El resultado de aplicar a una matriz simétrica 35×35 este algoritmo se ilustra en la figura 3.4. La solución de sistemas con este tipo de matrices dispersas en banda es extremadamente eficaz.

También existen diversas formas de ordenar matrices dispersas de estructura simbólica no simétrica¹ para resolver rápida y eficazmente sistemas lineales de ecuaciones en los que estén presentes. Para tratar matrices generales sin ninguna característica particular, entre los métodos más usados están los que buscan ordenar los elementos de la matriz A de tal forma que se consiga una estructura triangular inferior en bloques del tipo que se indica en la figura 3.5. Si de acuerdo con esta estructura se dividen de la misma forma los vectores $\mathbf{x}$ y $\mathbf{b}$ de $A\mathbf{x} = \mathbf{b}$,

¹Pueden existir matrices no simétricas numéricamente hablando aunque de estructura simbólica simétrica por lo que se refiere a la posición de los elementos distintos de cero. La ordenación casi siempre se centra en la estructura simbólica.

Tabla 3.2

Número de operaciones a realizar con diversas variantes de la matriz de la figura 3.1 para, utilizando eliminación de Gauss, resolver un sistema de ecuaciones lineales

Operación	Matriz	Matriz	Matriz
	Densa	Dispersa sin Ordenar	Dispersa Ordenada
Factorización	1911	408	105
Sustitución Inversa	196	94	48

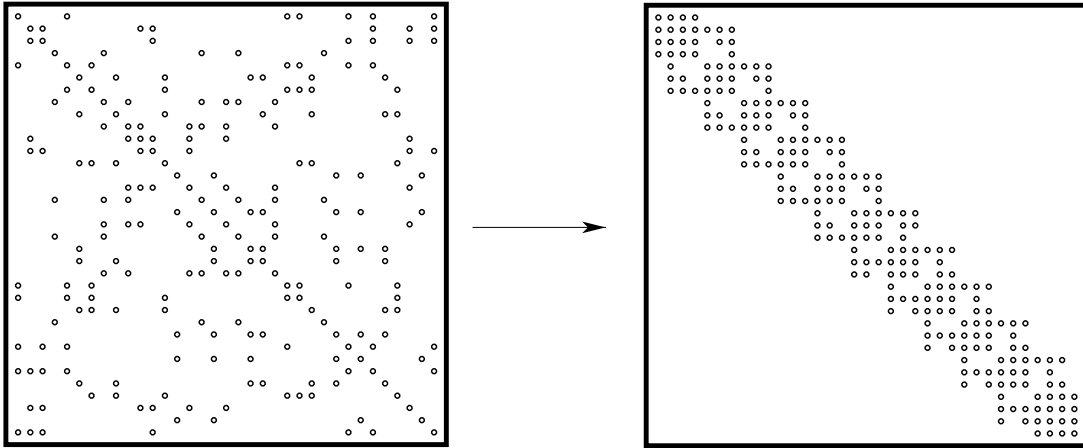


Figura 3.4

Matriz 35×35 , de estructura simbólica simétrica, antes y después de reordenar sus filas y columnas con el algoritmo de Cuthill-McKee

el sistema se resolverá mucho más eficazmente resolviendo los subsistemas

$$A_{ii}\mathbf{x}_i = \mathbf{b}_i - \sum_{j=1}^{i-1} A_{ij}\mathbf{x}_j, \quad i = 1, 2, 3.$$

Esta forma de manipular la matriz A hace que sólo sea necesario factorizar las submatrices A_{ii} (los bloques o submatrices que no están en la diagonal principal, A_{ij} , $i > j$, sólo se han de multiplicar por los subvectores $\mathbf{x}_j$), y, por tanto, que cualquier nuevo elemento distinto de cero sólo se pueda crear en esas submatrices. Cualquier otra reordenación que sea necesaria para garantizar la dispersidad y la estabilidad numérica del proceso, habrá de efectuarse únicamente a los bloques A_{ii} .

Si como ejemplo consideramos la matriz simétrica 16×16 de la figura 3.6, la reordenación triangular inferior en bloques que de ella se puede obtener es la de la figura 3.7.

$$A = \begin{bmatrix} A_{11} & & \\ A_{21} & A_{22} & \\ A_{31} & A_{32} & A_{33} \end{bmatrix}$$

Figura 3.5

Matriz triangular inferior en bloques

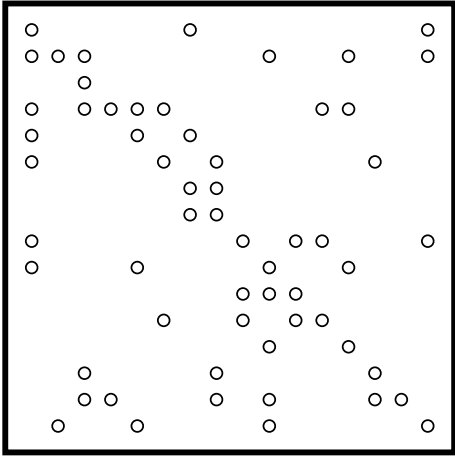


Figura 3.6

Matriz 16×16 , de estructura simbólica no simétrica, antes de reordenar sus filas y columnas para reducirla a una de estructura triangular inferior en bloques

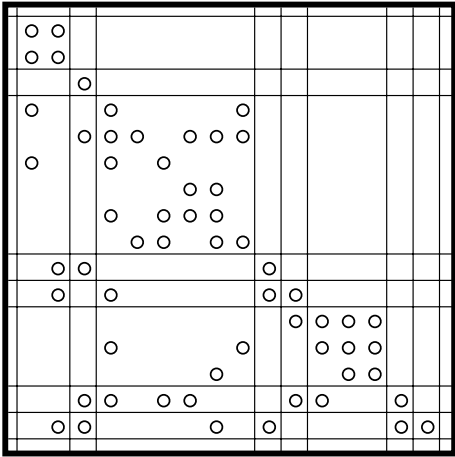


Figura 3.7

Matriz de la figura 3.6 después de reordenar sus filas y columnas para reducirla a una de estructura triangular inferior en bloques

3.3.2 Proceso de solución

La implementación eficaz del proceso de solución de un sistema de ecuaciones lineales de gran dimensión depende, como es obvio, de bastantes factores. Los más importantes, y en los que la literatura especializada centra fundamentalmente las metodologías y algoritmos, se refieren a si la matriz es definida positiva, semidefinida positiva o indefinida y a si la estructura simbólica y numérica que presenta es simétrica o no.

Si la matriz es de estructura simétrica y definida positiva, la mejor forma de proceder para obtener la solución del sistema correspondiente consiste en ordenar primero la matriz mediante, por ejemplo, el algoritmo de grado mínimo, o cualquier otro para estructuras particulares, y luego, una vez sabido qué elementos se harán distintos de cero, utilizar la factorización de Cholesky. Llevar esto a la práctica es relativamente sencillo y muy eficaz desde los puntos de vista de las operaciones a realizar y el tiempo de cálculo invertido. Casi todos los paquetes de rutinas matemáticas especializados en matrices dispersas poseen procedimientos dedicados a este caso. En concreto, el paquete de *software* SPARSPAK de la Universidad de Waterloo, Canadá, se centra esencialmente en este tipo de sistemas. Otros paquetes que también resuelven estos problemas son YSMP de la Universidad de Yale, EE.UU., Harwell Subroutine Library, Reino Unido: rutinas MA28, MA17, etc. y NAG, Numerical Algorithms Group, Reino Unido. Matlab también dispone de procedimientos especializados en sistemas de este tipo.

Si la matriz es de estructura simétrica y casi definida positiva, se puede proceder separándola según

$$A = M - N,$$

donde M es de estructura simétrica y definida positiva y N simétrica, y resolver $A\mathbf{x} = \mathbf{b}$ mediante un proceso iterativo del tipo

$$M\mathbf{x}^{(k+1)} = N\mathbf{x}^{(k)} + \mathbf{b}.$$

Este sistema lineal se puede resolver por Cholesky o cualquier otro método. Otra forma de actuar consiste en olvidarse de la simetría y utilizar la eliminación de Gauss, o triangularizar por bloques la matriz resolviendo según indicábamos en el apartado 3.3.1.

Si la matriz es de estructura simétrica e indefinida y se desea conservar la simetría, la mejor forma de proceder es utilizar el método de pivotación diagonal por bloques, que describíamos en el capítulo de métodos directos para matrices densas, especializándolo para matrices dispersas. La rutina MA32 del paquete Harwell Subroutine Library procede de esa manera. Matlab también puede resolver estos sistemas sin ninguna dificultad.

Si la matriz no es simétrica se puede proceder de dos formas: utilizando eliminación de Gauss con pivotación de acuerdo con el criterio de Markowitz que veremos más adelante, o triangularizando por bloques tal como hemos mencionado con anterioridad. En el paquete de Harwell se pueden encontrar diversas rutinas para hacer frente a este problema operando de las dos formas indicadas.

En lo que resta de capítulo nos centraremos en la forma más general de resolver un sistema lineal de ecuaciones, y a la que mayor cantidad de *software* dedican los paquetes ya mencionados y casi todos los existentes de matrices dispersas: la eliminación de Gauss. En torno a él, analizaremos los métodos más utilizados para la reordenación previa de la estructura de la matriz, tanto si es simétrica como si no, así como las estrategias de pivotación de los métodos que proceden directamente a factorizar la matriz.

3.4 Matrices dispersas simétricas y eliminación de Gauss

En este apartado estudiamos diversas ordenaciones para matrices dispersas de estructura simétrica simétrica, y cómo afectan a la eliminación de Gauss en ellas.

La factorización que se obtiene como consecuencia del proceso de eliminación de Gauss de una matriz A simétrica es, como sabemos,

$$A = L_1 D L_1^T,$$

donde L_1 es una matriz triangular inferior con todos los elementos de su diagonal principal igual a uno y D una matriz diagonal. Esta factorización también se puede escribir,

$$A = L L^T.$$

El patrón de elementos distintos de cero de L es idéntico al de L_1 .

Como sabemos, en el transcurso de la eliminación de Gauss en una matriz se pueden crear elementos de relleno; si son muchos, no sólo pueden destruir cualquier estructura de dispersidad que poseyese la matriz, sino también dar al traste con la consideración hecha a priori de que era dispersa, pues el número de operaciones que requieren los métodos que tratan tales matrices es sensiblemente superior, a igualdad de número de elementos no nulos, al de los tradicionales para matrices densas. Para evitar esto se recurre a efectuar una reordenación de filas y columnas, materializadas por un conjunto de permutaciones, de tal forma que al factorizar la matriz resultante aparezcan muchos menos elementos de relleno que en la original.

Si el sistema que hay que resolver es

$$Ax = b$$

y se le aplican a A un conjunto de permutaciones elementales, representadas por la matriz de permutación P , el sistema se puede reescribir,

$$P A P^T P x = P b,$$

pues $P^T P = I$. Haciendo $y = P x$ y $c = P b$, se tiene que

$$B y = c,$$

donde $B = P A P^T$ es la matriz A reordenada. La matriz B es también dispersa y simétrica. Si A es también definida positiva, lo mismo ocurre con B .

La idea alrededor de la que centraremos este apartado es encontrar una P adecuada que produzca el menor relleno posible al factorizar B . Si la matriz A es de orden n , el número posible de ordenaciones es $n!$: evidentemente, resulta imposible analizar todas. Aun cuando de esas $n!$ una al menos será óptima, no existe ningún algoritmo que garantice su obtención. Existen, no obstante, bastantes algoritmos heurísticos para tratar de llegar a un resultado óptimo o cercano a él. A continuación describiremos alguno de ellos.

Para ilustrar la diferencia que supone utilizar una ordenación u otra, o incluso ninguna, en las figuras 3.8, 3.9 y 3.10 se representan los patrones de elementos distintos de cero de una matriz 480×480 antes y después de, ordenándola de determinadas maneras, factorizarla de la forma LL^T . La figura 3.8 representa la matriz sin reordenar y el resultado que produciría factorizarla de la forma LL^T . La 3.9 describe el mismo caso cuando se reordena la matriz

original mediante el algoritmo denominado de grado mínimo, que veremos más adelante en este apartado. Por último, en 3.10 se representa la matriz reordenada mediante el algoritmo de Cuthill-McKee y el factor L correspondiente. Obsérvese que L tiene en el primer caso 30.366 elementos distintos de cero, 9.196 en el segundo y 24.226 en el tercero. Elegir un buen método de reordenación de la matriz, como se puede apreciar, es esencial.

3.4.1 Nociones básicas sobre teoría de grafos

La teoría de matrices dispersas y la de grafos son dos disciplinas con vínculos y resultados comúnmente aplicables. El patrón de elementos distintos de cero de una matriz dispersa cuadrada se puede representar mediante un grafo; en consecuencia, muchos resultados de la teoría de grafos pueden aplicarse para estudiar y obtener mejoras en las prestaciones numéricas de las matrices dispersas. De la teoría de grafos vamos a introducir, tanto en este apartado como en el que dedicaremos a matrices dispersas no simétricas, aquellos conceptos y resultados que nos serán útiles para agilizar la comprensión de los procedimientos que explicaremos o mejorar la visualización de los mismos.

Un *grafo*, $G = (V, E)$, es un par formado por un conjunto finito, V , de elementos denominados *vértices* o *nodos* del grafo, y por otro también finito, E , de *arcos* o *aristas*. Un arco es un par de nodos. Si los arcos de un grafo son ordenados, el grafo se denomina *digrafo* o *grafo dirigido*; si no, *grafo a secas* o *grafo no dirigido*. Un grafo no dirigido se puede ver como un digrafo en el que si el arco $e = (u, v) \in E$, también $e' = (v, u) \in E$. Si $e = (i, j) \in E$, este arco une un nudo de *origen* o *cola* $i = t(e)$ con otro de *destino*, *final* o *cabeza* $j = h(e)$. El número de elementos, o cardinal de V o E , se designa $|V|$ o $|E|$. Un grafo $G = (V, E)$ se dice *numerado* si existe una biyección $\alpha : \{1, 2, \dots, |N|\} \rightarrow V$. En lo sucesivo, cuando hablemos de un grafo,

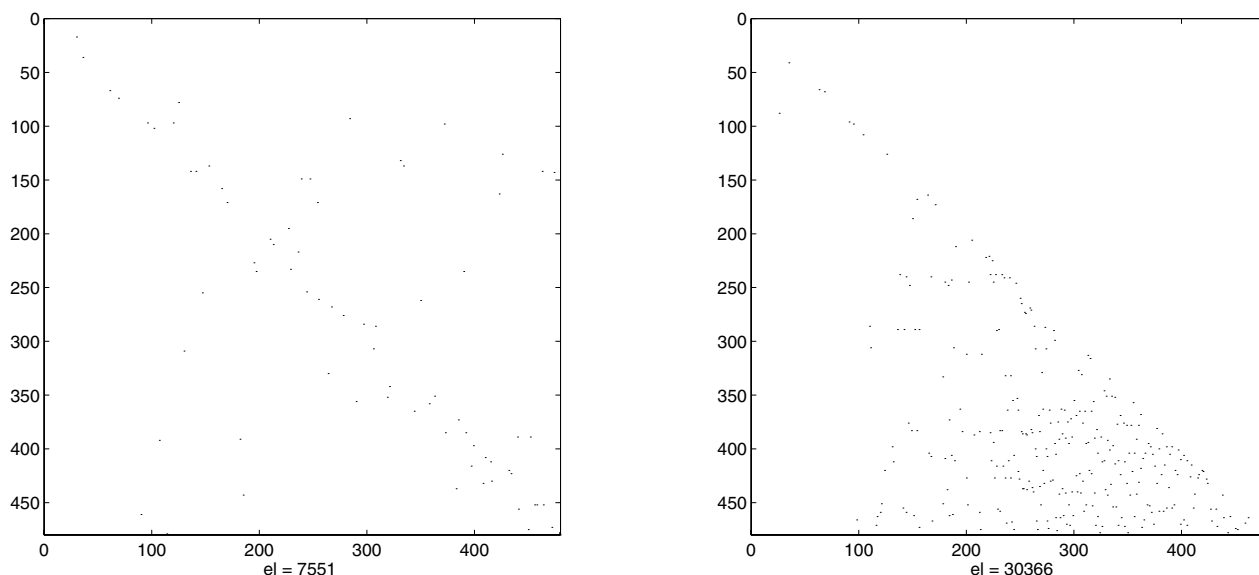
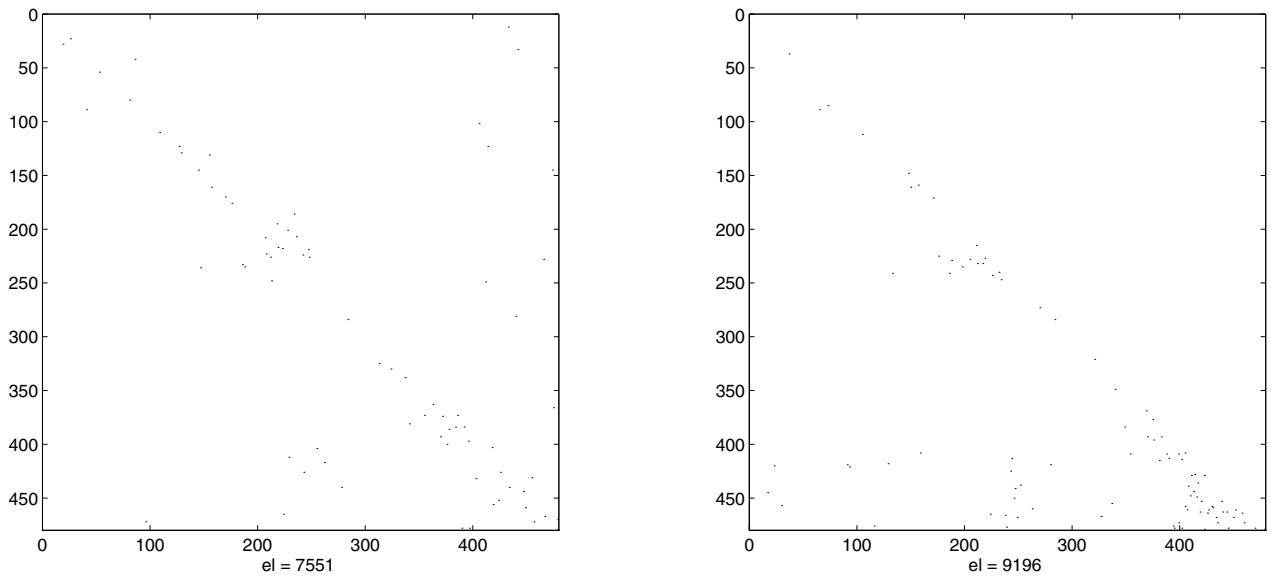
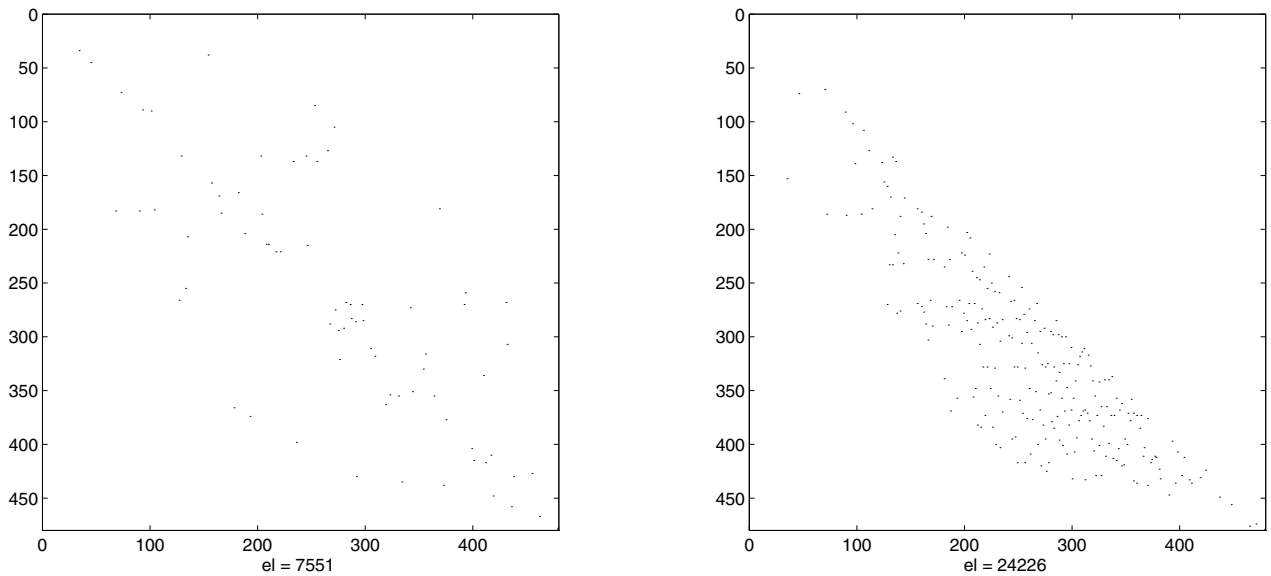


Figura 3.8

Patrón de elementos distintos de cero de una matriz simétrica 480×480 y el de su factor L una vez efectuada la factorización LL^T

**Figura 3.9**

Patrón de elementos distintos de cero de una matriz simétrica 480×480 ordenada mediante el algoritmo de grado mínimo y el de su factor L una vez efectuada la factorización LL^T

**Figura 3.10**

Patrón de elementos distintos de cero de una matriz simétrica 480×480 ordenada mediante el algoritmo de Cuthill-McKee y el de su factor L una vez efectuada la factorización LL^T

lo supondremos numerado.²

Un grafo se puede asociar a cualquier matriz A . Si A es cuadrada de orden n , de estructura simbólica simétrica, con todos sus elementos diagonales distintos de cero, se define el *grafo asociado a A* , $G^A = (V^A, E^A)$, como el grafo no dirigido numerado de nudos $V^A = \{v_1, v_2, \dots, v_n\}$ y arcos o aristas E^A definidas de tal forma que

$$(v_i, v_j) \in E^A \Leftrightarrow a_{ij} \neq 0, a_{ji} \neq 0.$$

La suposición de que los elementos diagonales son distintos de cero hace que no sea necesario representar los bucles que unen cada nudo consigo mismo. En la figura 3.11 se puede ver una matriz 11×11 de estructura simbólica simétrica y su grafo numerado asociado.

Al igual que se define un grafo no dirigido para matrices simétricas, para matrices no simétricas se define un digrafo; volveremos sobre esta cuestión al hablar de matrices no simétricas.

El grafo asociado a una matriz simétrica permanece invariable, salvo la numeración de sus nudos, al aplicarle a dicha matriz una *permutación simétrica* (se la pre y postmultiplica por una misma matriz de permutación P). Esta es una de las propiedades que hacen de los grafos un instrumento muy útil para estudiar matrices dispersas. Si $B = PAP^T$, los grafos asociados a B y a A son idénticos salvo en lo que respecta a su numeración.

Un *subgrafo* $G' = (V', E')$ de un grafo $G = (V, E)$ es un grafo formado por algunos o todos los nudos y por algunos de los arcos del grafo G : $V' \subseteq V$, $E' \subset E$. Un subgrafo se dice *subgrafo sección* cuando V' contiene sólo algunos nudos de G y E' todos los arcos (u, v) de G tales que u y v pertenecen a V' ; es decir: $V' \subset V$ y $E' = \{(u, v) \in E : u \in V' \text{ y } v \in V'\}$. En el grafo de la figura 3.11, los nudos 3, 5, 7, 8 y 11 junto con los arcos $(3,5)$, $(5,8)$, $(8,11)$, $(11,7)$, $(3,7)$ y $(3,11)$ constituyen un subgrafo sección.

Si (u, v) es un arco de un grafo, los nudos u y v se dicen *adyacentes*. El *grado* de un nudo es el número de arcos que tienen uno de sus extremos en ese nudo. Si W es un subconjunto de los nudos de un grafo G , el *conjunto adyacente* de W , $\text{Adj}(W)$, es el conjunto formado por los nudos de G que no pertenecen a W y son adyacentes a nudos de W . Es decir, $\text{Adj}(W) =$

²En algunas referencias bibliográficas un grafo numerado se designa por $G_\alpha = (V, E, \alpha)$.

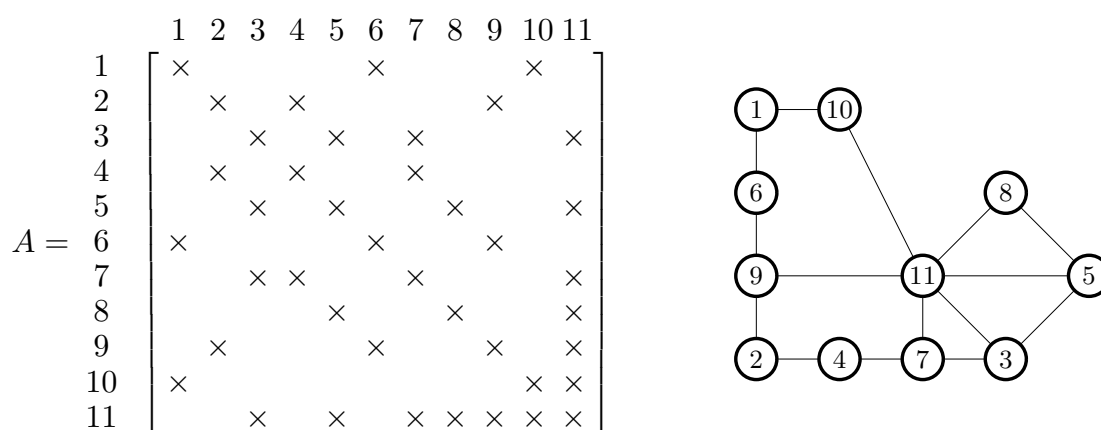


Figura 3.11

Matriz 11×11 de estructura simbólica simétrica y su grafo numerado asociado

$\{u \in V - W : (u, v) \in E \text{ para algún } v \in W\}$. El grado de un nudo es por consiguiente el número de elementos (cardinal) del conjunto adyacente de ese nudo. En el caso de la matriz 11×11 y de su grafo asociado de la figura 3.11, los nudos 1 y 6 son adyacentes; ambos de grado 2. Si W fuese el conjunto formado por los nudos 1 y 6, su conjunto adyacente sería el formado por los nudos 9 y 10.

Un *camino* de un nudo u_1 a otro u_{m+1} , es un conjunto ordenado de nudos $\{u_1, u_2, \dots, u_{m+1}\}$ tal que u_i y u_{i+1} son adyacentes para $i = 1, 2, \dots, m$. La *longitud* de ese camino es m . El nudo inicial de un camino se suele denominar de *partida*; el final, de *llegada*. Un camino también se puede definir como un conjunto ordenado de m arcos $(u_1, u_2), (u_2, u_3), \dots, (u_m, u_{m+1})$. Dos nudos dados, u y v , se dicen *unidos por un camino*, si existe un camino de u a v . Un camino es un *ciclo* cuando $u_1 = u_{m+1}$. La *distancia*, $d(u, v)$, entre dos nudos, u y v , es la longitud del camino más corto entre ambos nudos. Dado un nudo u , a la mayor distancia entre ese nudo y cualquier otro del grafo se la denomina *excentricidad*, $e(u)$, del nudo u . La mayor excentricidad de un grafo se denomina *diámetro* del grafo. Un *nudo periférico* de un grafo es aquel cuya excentricidad es igual al diámetro del grafo. Volviendo a nuestra matriz 11×11 y su grafo asociado de la figura 3.11, los nudos 1 y 3 están unidos por los caminos $\{1, 10, 11, 3\}$, de longitud 3, y $\{1, 6, 9, 11, 3\}$, de longitud 4. La distancia entre los nudos 1 y 3 es, por consiguiente, 3. El camino $\{5, 8, 11, 3, 5\}$ es un ciclo. El diámetro de este grafo es 4. Los nudos periféricos: 1, 2, 4, 5 y 6; su excentricidad es igual a 4.

Un grafo se dice *conexo* si cada par de nudos distintos se puede unir por un camino; *inconexo* en cualquier otro caso. Un grafo inconexo está formado por varios *componentes conexos*. Un *separador* o *conjunto separador* es un conjunto de nudos tal que quitando los nudos que pertenecen a él y los arcos a ellos unidos en un grafo conexo o componente conexo, resulta un grafo no conexo. Un *separador* es *mínimo* si cualquier subconjunto de él no es un separador. El grafo de la figura 3.11 es conexo. El conjunto de nudos de este grafo formado por el 7 y el 11 es un separador mínimo: al quitar esos nudos del grafo resultan los componentes conexos $\{3, 5, 8\}$ y $\{10, 1, 6, 9, 2, 4\}$.

Dado un grafo y un subconjunto S de sus nudos, si u y v son dos nudos distintos que no pertenecen a S , se dice que v es *accesible desde u a través de S* cuando u y v están unidos por un camino de longitud igual a 1 (u y v son adyacentes) o ese camino está formado enteramente por nudos pertenecientes a S (excepto, por supuesto, u y v). Dado ese subconjunto S y $u \notin S$, el *conjunto accesible*, $\text{Acc}(u, S)$, de u a través de S , es el conjunto de todos los nudos accesibles desde u a través de S . Obsérvese que cuando S es el vacío o u no pertenece a $\text{Adj}(S)$, $\text{Acc}(u, S) = \text{Adj}(u)$. En el grafo de la figura 3.11, si se escoge $S = \{7, 3\}$, entonces $\text{Acc}(5, S) = \{8, 11, 4\}$ y $\text{Acc}(8, S) = \text{Adj}(8) = \{5, 11\}$. Obsérvese que el propio u no pertenece a $\text{Acc}(u, S)$.

Los grafos se pueden dividir de acuerdo con diversos criterios. Cuando los nudos se agrupan en subconjuntos disjuntos $S_0, S_1, \dots, S_m$, se obtiene una *partición*. Cuando un grafo se divide de acuerdo con los niveles de los nudos se obtiene una *partición por niveles*, o *estructura de niveles*.

Un grafo conexo que no tiene ciclos se denomina *árbol*. Los árboles juegan un papel muy importante en el contexto de las matrices dispersas pues una matriz cuyo grafo asociado es un árbol se puede reordenar de tal forma que, al factorizarla mediante eliminación de Gauss, no experimente ningún relleno. En un árbol sólo existe un camino entre cualquier par de nudos. Un árbol se dice *enraizado* cuando uno de sus nudos se designa como nudo *raíz*. El camino único que existe entre ese nudo raíz y cualquier nudo u del árbol define las relaciones *ascendente/descendente* entre nudos: si u y v pertenecen a un camino y la distancia de v al nudo

raíz es menor que la de u , v se dice es un *ascendiente* de u y u un *descendiente* de v . Si u y v son adyacentes, v es el *padre* de u y u el *hijo* de v . Como en el caso del grafo, un árbol lo supondremos numerado. La numeración se dice *monótona* si cada nudo se numera antes que su padre.

La partición de un grafo que no es un árbol se puede usar para generar un grafo cociente. Cuando un grafo cociente es un árbol, éste se denomina *árbol cociente*; a la partición correspondiente, *árbol partición*.

En la figura 3.12 se puede ver un grafo de 20 nudos, su estructura de niveles y su correspondiente árbol cociente. En este árbol también se indica una numeración monótona.

Dado un grafo conexo $G = (V, E)$, un *árbol maximal* es un subgrafo de G que contiene todos los nudos de G y es además un árbol. En la figura 3.13 se puede ver un árbol maximal del grafo de la figura 3.12.

3.4.2 Interpretación grafo-teórica de la eliminación de Gauss de matrices dispersas de estructura simétrica

Profundizando en el objetivo de determinar algoritmos para ordenar la numeración del grafo asociado a una matriz dispersa de tal forma que al factorizarla mediante eliminación de Gauss se produzca el menor número posible de rellenos, estudiemos el efecto que esa eliminación produce en la matriz a través de su grafo asociado.

Al comienzo de una etapa k de un proceso de eliminación de Gauss, todos los elementos distintos de cero debajo de la diagonal principal en las columnas $1, 2, \dots, k-1$ ya se han hecho cero. En esta etapa k , como es sabido, se determinan unos multiplicadores y se restan, de las

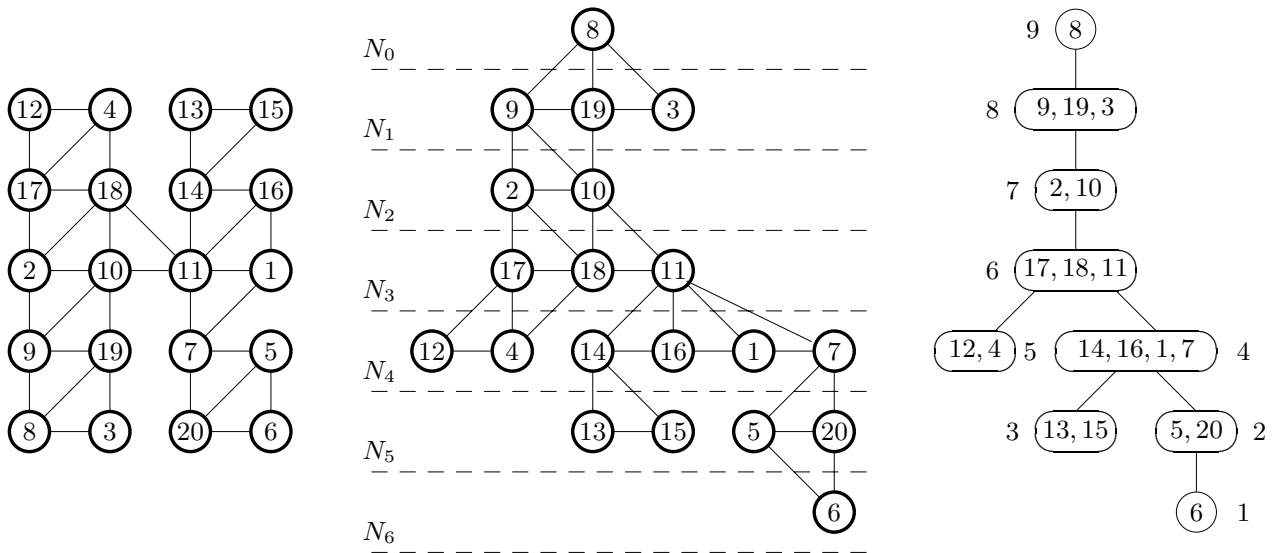


Figura 3.12

Grafo no dirigido de 20 nudos, su estructura de niveles y su correspondiente árbol cociente con numeración monótona

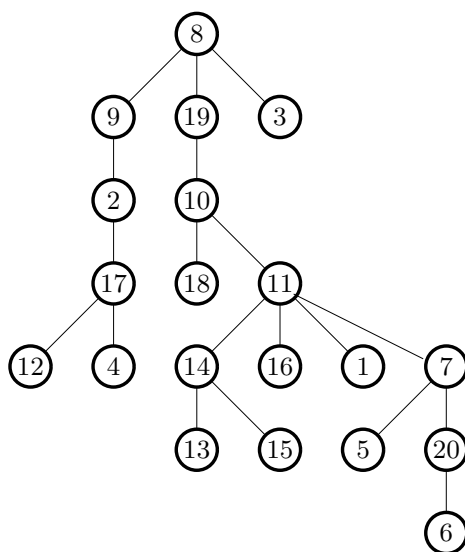


Figura 3.13
Árbol maximal del grafo de la figura 3.12

filas que tienen un elemento distinto de cero en la columna k debajo de la diagonal principal, la fila k multiplicada por el multiplicador correspondiente. Al hacer esto se pueden crear nuevos elementos distintos de cero en la submatriz que definen las filas y columnas $k + 1, \dots, n$.

Consideremos la submatriz activa en la etapa k^3 sin tener en cuenta que se pueden producir cancelaciones, como resultado de las cuales elementos que se supone se van a hacer distintos de cero siguen siendo cero. Sea G^k el grafo asociado a esa submatriz activa —a este grafo se le denomina *grafo de eliminación*—. Los nudos de este grafo son los $n - k + 1$ últimos del grafo asociado a la matriz original A , G^A . El grafo G^k contiene todos los arcos que unen esos nudos, y estaban presentes en G^A , más unos arcos adicionales correspondientes a los rellenos producidos en las $k - 1$ etapas anteriores del proceso. La sucesión de grafos $G^1 = G^A, G^2, \dots$ se obtiene aplicando la siguiente regla:

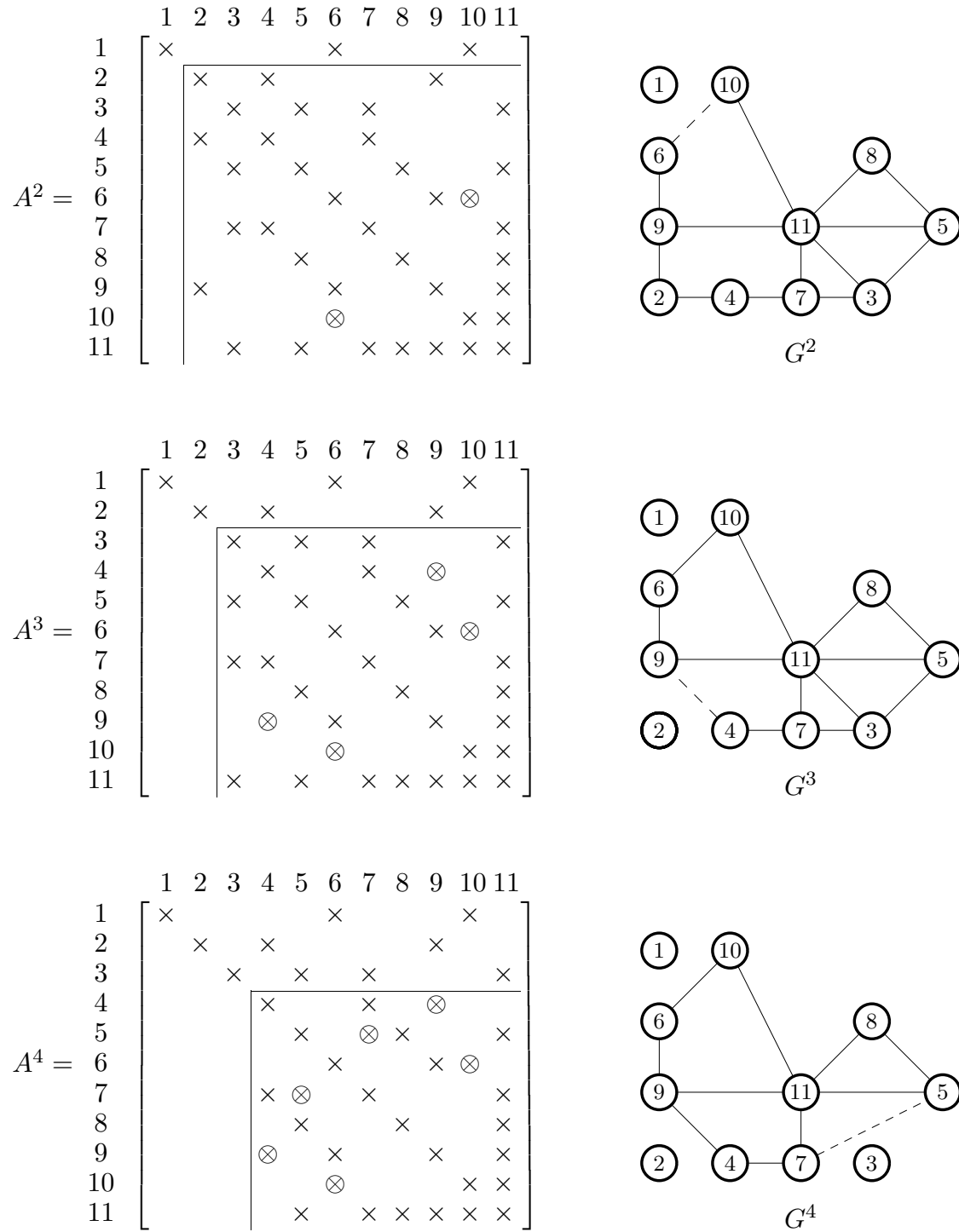
Para obtener G^{k+1} a partir de G^k , borrar en éste el nudo k y añadir todos los posibles nuevos arcos entre nudos que sean adyacentes al nudo k de G^k .

Los arcos que se añaden determinan qué elementos de relleno se producirán en la matriz como consecuencia del proceso de eliminación de Gauss. Como ejemplo de aplicación de estas ideas, en la figura 3.14 se ilustra su adaptación a la matriz 11×11 de la figura 3.11 y a su grafo asociado.

Al final del proceso la matriz simbólica que indica qué elementos serán distintos de cero una vez completada la factorización tendrá la forma de la figura 3.15.

Mediante los grafos de eliminación, introducidos por Parter [1961], se puede realizar, independientemente de los valores numéricos que adopten los elementos de la matriz, una eliminación de Gauss simbólica, pudiéndose determinar a partir de ella qué nuevos elementos distintos

³Esa tal submatriz activa contiene los elementos a_{ij}^k , donde $i, j \geq k$.


Figura 3.14

Tres primeras etapas de la eliminación de Gauss de una matriz simétrica 11×11 y sus correspondientes grafos de eliminación. Los elementos de relleno se indican mediante el símbolo $\otimes$

	1	2	3	4	5	6	7	8	9	10	11
1	×					×				×	
2		×		×					×		
3			×		×		×				×
4		×		×			×		⊗		
5			×		×		⊗	×			×
6	×					×			×	⊗	
7			×	×	⊗		×	⊗	⊗		×
8					×		⊗	×	⊗		×
9		×		⊗		×	⊗	⊗	×	⊗	×
10	×					⊗			⊗	×	×
11			×		×		×	×	×	×	×

Figura 3.15

Resultado de la eliminación simbólica de Gauss en la matriz de la figura 3.11 mediante grafos de eliminación

de cero se van a crear en el proceso numérico. Reservando posiciones de memoria para esos nuevos elementos, se puede pasar a realizar la eliminación o factorización numérica.

Llevar el procedimiento de los grafos de eliminación a la práctica en un ordenador es muy sencillo; la única dificultad reside en cómo está almacenada la matriz. Para hacerlo, en cualquier caso, es interesante tener en cuenta el siguiente resultado.

Teorema 3.1 Sea u un nudo del grafo de eliminación G^k y Q_k el conjunto de nudos $v_1, v_2, \dots, v_{k-1}$ del grafo original ya eliminados. El conjunto de nudos adyacentes a u en G^k es el conjunto $\text{Acc}(u, Q_k)$ en el grafo original G^A .

En el ejemplo de la figura 3.14, grafo G^4 , $Q_4 = \{1, 2, 3\}$. Del grafo original se obtiene: $\text{Acc}(6, Q_4) = \{9, 10\}$, conjunto de los nudos adyacentes al 6 en G^4 .

3.4.3 El algoritmo de grado mínimo

Este algoritmo (Tinney y Walker [1967]) es el de reordenación de uso más extendido para reducir el número de rellenos que produce la eliminación de Gauss —o la factorización de Cholesky— en una matriz dispersa de estructura simbólica simétrica. Es el que presenta unas características de eficacia, sencillez y facilidad de implementación en ordenador más destacadas.

El algoritmo de grado mínimo es la versión para matrices de estructura simbólica simétrica del de Markowitz [1957] para matrices no simétricas; comentaremos éste en detalle más adelante.

La idea en que se basa es muy sencilla. Como en cada etapa k del proceso de eliminación de Gauss, debido a las manipulaciones inherentes al proceso que se efectúa en la submatriz activa, si en la fila k hay elementos no cero a la derecha del elemento de la diagonal principal, al sumar un múltiplo de esta fila a cualquiera de las filas $k + 1$ a n donde se quiera hacer cero un elemento de la columna k por debajo de la diagonal principal, se pueden producir elementos no nulos en esas filas. Si se examina qué fila de la submatriz activa, ℓ , tiene el menor número

de elementos distintos de cero y se intercambian las filas ℓ y k y las columnas ℓ y k , en esa submatriz activa se crearán el mínimo de elementos de relleno posible.

El nombre de grado mínimo viene de que al realizar ese intercambio de filas y columnas en la etapa k , el elemento de la diagonal principal en la fila k representará el nudo que está unido al menor número —grado mínimo— de otros en el grafo de eliminación G^k .

El algoritmo completo de grado mínimo es el de la tabla 3.3 Obsérvese que esta implementación, al trabajar con los grafos de eliminación, permite también conocer al final del proceso qué nuevos elementos se harán distintos de cero al efectuar la correspondiente factorización.

Para ilustrar el proceso del algoritmo, consideremos el grafo de la figura 3.16 asociado a una matriz simétrica 7×7 .

En la tabla 3.4 se describen las 7 etapas de que consta la aplicación del algoritmo de grado mínimo a este grafo.

La matriz simbólica que indica qué elementos serán distintos de cero una vez completada la reordenación/factorización simbólica que lleva a cabo el algoritmo de grado mínimo, y el grafo con la numeración óptima, se indican en la figura 3.17.

La estrategia que sigue el algoritmo de grado mínimo produce en general muy buenos resultados prácticos. Como ya apuntábamos anteriormente, cuando el grafo que se quiere reordenar es un árbol, el resultado de aplicarlo no producirá elementos de relleno al efectuar la eliminación de Gauss o la factorización ulterior correspondiente. No obstante, no siempre da lugar a una ordenación que produzca el menor número posible de elementos de relleno. En efecto, si se aplica al grafo de la figura 3.18, el algoritmo elegirá el nudo número 5 como el inicial, lo que traerá como consecuencia que se produzca un relleno posterior en las posiciones (4, 6) y (6, 4). Utilizando por el contrario la numeración de la figura no se produciría ningún elemento de relleno.

La implementación en ordenador de este algoritmo es muy sencilla. Normalmente es necesario incluir, además de las estructuras de datos de la matriz a ordenar, un vector, **ng** por ejemplo, en el que inicialmente se indican el grado⁴ de cada uno de los nudos del grafo asociado a la matriz, G^A . En la etapa k , el nudo de grado mínimo se selecciona inspeccionando las posiciones k a n de **ng**. Luego, al construir el grafo de eliminación, G^k , sólo se deben modificar las posiciones de **ng** correspondientes a los nudos en $\text{Acc}(v_k, Q_k)$. Si $u \in \text{Acc}(v_k, Q_k)$, su nuevo grado será $|\text{Acc}(u, Q_{k+1})|$, donde $Q_{k+1} = Q_k \cup \{v_k\} = \{v_1, \dots, v_k\}$.

⁴Si la matriz se almacena por filas, por ejemplo, este grado lo determinará simplemente el número de elementos en la fila correspondiente menos 1.

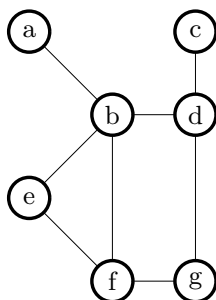
Tabla 3.3
Algoritmo de grado mínimo

Paso 1 – *Inicialización.* Hacer $i \leftarrow 1$.

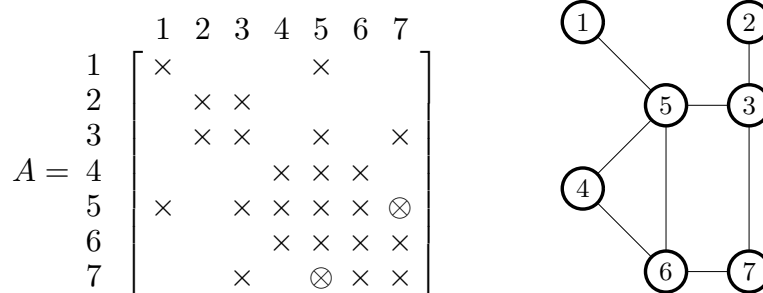
Paso 2 – *Selección del nudo de grado mínimo.* Seleccionar en el grafo de eliminación $G^{k-1} = (V^{k-1}, E^{k-1})$ aquel nudo v_k de grado mínimo.

Paso 3 – *Transformación.* Formar el nuevo grafo de eliminación $G^k = (V^k, E^k)$ eliminando v_k de G^{k-1} .

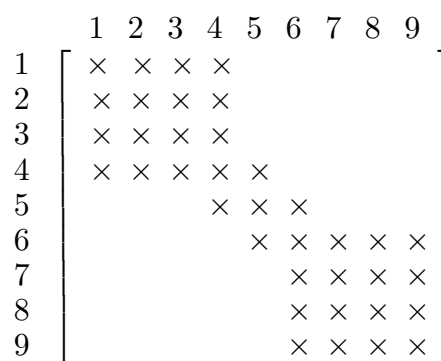
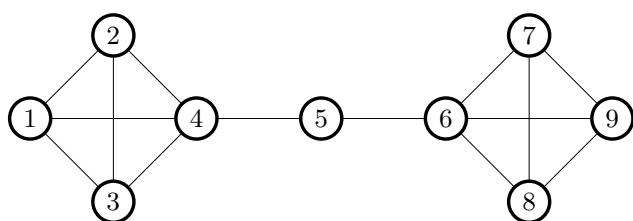
Paso 4 – *Bucle.* Hacer $i \leftarrow i + 1$. Si $i > |V|$, parar. Si no, ir al paso 2.


Figura 3.16

Grafo asociado a una matriz 7×7 sobre el que se ilustra el algoritmo de grado mínimo

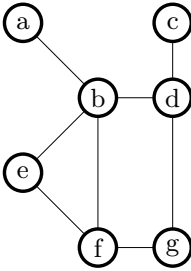
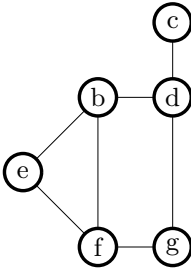
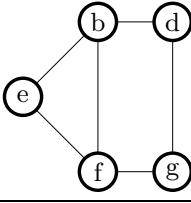
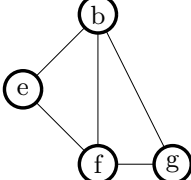
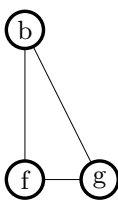
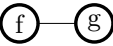

Figura 3.17

Matriz 7×7 y su grafo asociado con la numeración resultado del algoritmo de grado mínimo


Figura 3.18

Grafo donde la renumeración que resultaría de aplicarle el algoritmo de grado mínimo no es la óptima

Tabla 3.4
Ejemplo de aplicación del algoritmo de grado mínimo

Etapa k	Grafo de Eliminación G^{k-1}	Nudo Seleccionado	Grado
1		a	1
2		c	1
3		d	2
4		e	2
5		b	2
6		f	1
7		g	0

Existen diversas variantes del algoritmo de grado mínimo para tratar de mejorar alguna de sus prestaciones, concretamente en lo que se refiere a cómo actuar en el caso de que los posibles nudos iniciales sean varios. También se han desarrollado otros procedimientos distintos para también determinar la ordenación que produce el mínimo número de rellenos posible en una eliminación de Gauss. Al lector interesado en todo ello le remitimos a cualquiera de las referencias que sobre matrices dispersas se mencionan en la bibliografía al final del capítulo.

3.4.4 Reducción del ancho de banda de una matriz dispersa simétrica. El algoritmo de Cuthill-McKee

Recordemos las definiciones relativas al ancho de banda de una matriz simétrica, definiciones 3.1 y 3.3 de las páginas 205 y 206.

Como hemos venido diciendo, en muchos problemas con matrices dispersas la estructura de la matriz que los caracteriza hace aconsejable su almacenamiento mediante el esquema de perfil o envolvente, pues se sabe que todos los elementos no nulos pueden estar próximos a la diagonal principal. La existencia de tales problemas y la importancia que los mismos tienen en la ciencia y en la ingeniería, ha motivado que durante los últimos años se hayan dedicado muchos esfuerzos al objetivo de desarrollar algoritmos de reordenación para conseguir que en la matriz con la que se va a operar, los elementos distintos de cero estén lo más cerca posible de la diagonal principal.

De los algoritmos dedicados a este fin, el de utilización más extendida es el de Cuthill-McKee [1969]. La idea en la que basa su estrategia es muy sencilla: como de lo que se trata es de que los elementos distintos de cero estén lo más cerca posible de la diagonal principal, una vez numerado un nudo k , si se numeran inmediatamente después los que están unidos a él que no han sido numerados previamente, se conseguirá que en la fila k se cumpla ese objetivo.

El algoritmo que plasma esta idea es el de la tabla 3.5.

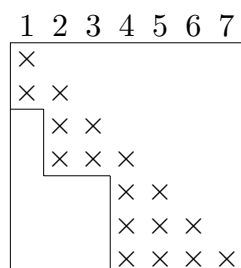
Definición 3.4 Se dice que una matriz simétrica tiene un *perfil monótono* si para todo k y ℓ , donde $k < \ell$, $l_k \leq l_\ell$.

A continuación se presentan dos matrices con perfiles monótono y no monótono, respectivamente.

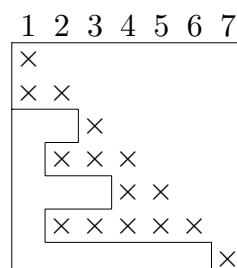
Tabla 3.5
Algoritmo de Cuthill-McKee

Paso 1 – *Inicialización.* Seleccionar un nudo inicial r . Hacer $v_i \leftarrow r$.

Paso 2 – *Bucle.* Para $i = 1, \dots, n$, determinar todos los nudos adyacentes al v_i no numerados y numerarlos en orden creciente de grado (de menor a mayor).



Perfil Monótono



Perfil No Monótono

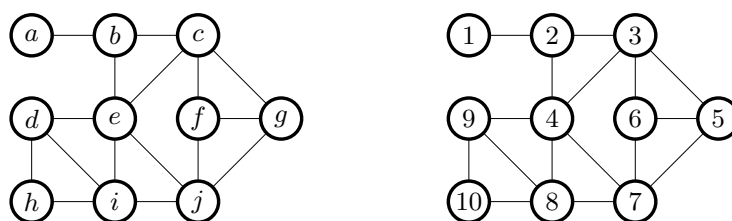
Teorema 3.2 *La numeración dada por el algoritmo de Cuthill-McKee conduce a un perfil monótono.*

A modo de ejemplo, apliquemos el algoritmo de Cuthill-McKee al grafo de la figura 3.19. Empezando a numerar por el nudo superior izquierdo, a , en la misma figura se puede ver la numeración final que se obtiene con el algoritmo. Con esta numeración, la distribución simbólica de elementos distintos de cero y ceros en la matriz asociada que habrá que incluir en el esquema de almacenamiento de envolvente, considerando sólo la parte triangular inferior, será

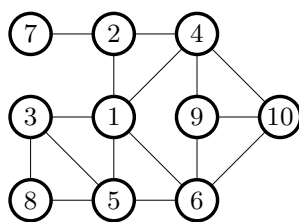
$$\begin{array}{c}
 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10
 \end{array}
 \begin{bmatrix}
 \times & & & & & & & & & \\
 \times & \times & & & & & & & & \\
 & \times & \times & & & & & & & \\
 & & \times & \times & \times & & & & & \\
 & & & \times & 0 & \times & & & & \\
 & & & & \times & 0 & \times & \times & & \\
 & & & & & \times & \times & \times & \times & \\
 & & & & & & \times & 0 & 0 & \times & \times \\
 & & & & & & & \times & 0 & 0 & 0 & \times & \times \\
 & & & & & & & & & \times & \times & \times & \times
 \end{bmatrix}
 .$$

El ancho de banda de esta matriz es 5. El número de elementos cero en la envolvente, 7. El número de elementos de la envolvente, 33.

Ahora bien, si se comienza a numerar el grafo por el nudo e , el resultado de aplicar el algoritmo de Cuthill-McKee es el de la figura 3.20. La distribución simbólica de elementos

**Figura 3.19**

Grafo de 10 nudos antes y después de aplicarle el algoritmo de Cuthill-McKee, comenzando la numeración en a

**Figura 3.20**

Grafo de 10 nudos de la figura 3.19 una vez aplicado el algoritmo de Cuthill-McKee, comenzando la numeración en e

distintos de cero y ceros de la parte triangular inferior de la matriz sería en este caso,

$$\begin{array}{c}
 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10 \\
 \begin{array}{c}
 1 \\
 2 \\
 3 \\
 4 \\
 5 \\
 6 \\
 7 \\
 8 \\
 9 \\
 10
 \end{array}
 \left[\begin{array}{cccccc}
 \times & & & & & \\
 \times & \times & & & & \\
 \times & 0 & \times & & & \\
 \times & \times & 0 & \times & & \\
 \times & 0 & \times & 0 & \times & \\
 \times & 0 & 0 & 0 & \times & \times \\
 & \times & 0 & 0 & 0 & 0 & \times \\
 & & \times & 0 & \times & 0 & 0 & \times \\
 & & & \times & 0 & \times & 0 & 0 & \times \\
 & & & \times & 0 & \times & 0 & 0 & \times & \times
 \end{array} \right].
 \end{array}$$

El ancho de banda de esta matriz es 6. El número de elementos cero en la envolvente, 20. El número total de elementos en la envolvente, 46

Como se puede observar, la elección del nudo de partida para comenzar la numeración es una cuestión crítica para el resultado del algoritmo.

3.4.4.1 Selección del nudo inicial

Para abordar el problema de determinar con qué nudo se ha de comenzar el algoritmo de Cuthill-McKee, recordemos los conceptos de excentricidad, diámetro y nudo periférico de un grafo, introducidos en el apartado 3.4.1.

La idea del procedimiento para determinar el mejor nudo de partida se basa en el hecho de que tal nudo es casi siempre uno periférico. Para obtener un nudo periférico o pseudoperiférico⁵ se han desarrollado diversos procedimientos. En la tabla 3.6 se puede ver una modificación de George y Liu [1979] de un algoritmo originalmente debido a Gibbs, Poole y Stockmeyer [1976] para determinar un nudo pseudoperiférico en un grafo no dirigido. Tal como se describe es el más usado y referenciado en la literatura especializada.

Apliquemos este procedimiento para determinar qué nudo se debe usar para iniciar el algoritmo de Cuthill-McKee en el grafo de la figura 3.19. Las tres etapas de que consta se esquematizan en la figura 3.21. Los números al lado de los nudos del grafo indican su excentricidad tomando como raíz el que se indica como 0. El algoritmo comienza en cualquier nudo,

⁵Un *nudo pseudoperiférico* se define por la condición de que si v es un nudo para el cual $d(u, v) = e(u)$, entonces $e(u) = e(v)$.

Tabla 3.6

Algoritmo para determinar un nudo pseudoperiférico en un grafo (para obtener el nudo de partida del algoritmo de Cuthill-McKee)

Paso 1 – *Inicialización.* Seleccionar un nudo arbitrario, r , del grafo.

Paso 2 – *Generar estructura de niveles.* Construir la estructura de niveles del grafo tomando como nudo raíz el nudo r : $\mathcal{L}(r) = \{L_0(r), L_1(r), \dots, L_{\ell(r)}(r)\}$.

Paso 3 – *Bucle.* Escoger un nudo v en $L_{\ell(r)}(r)$ todavía no tratado de grado mínimo:

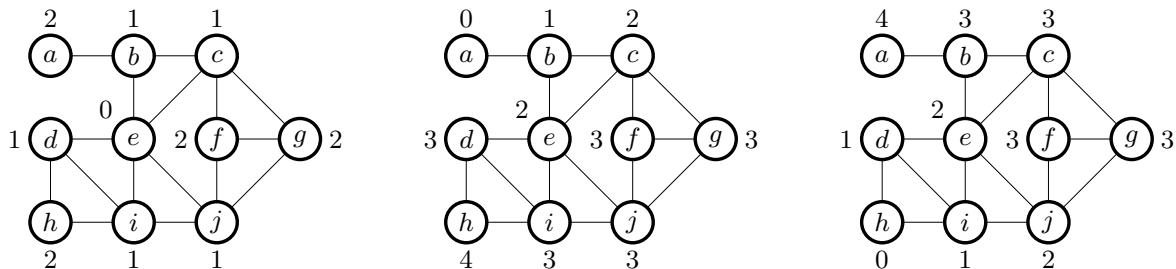
- a) Si $e(v) > e(r)$, hacer $r \leftarrow v$ e ir al Paso 2.
- b) Si $e(v) \leq e(r)$, escoger otro nudo de $L_{\ell(r)}(r)$ y volver al paso 3; si no hay más nudos, parar.

por ejemplo, en e . Obsérvese que del resultado del algoritmo se desprende que tanto el nudo a como el h podrían utilizarse como nudos de partida pues tienen la misma excentricidad: 4.

3.4.5 Reducción de la envolvente de una matriz dispersa simétrica. El algoritmo inverso de Cuthill-McKee

George [1971] descubrió que, utilizando el algoritmo de Cuthill-McKee para ordenar la numeración del grafo asociado a una matriz dispersa de estructura simbólica simétrica, pero invirtiendo el orden de la numeración final, es decir, asignando el número $1 + n - i$ al nudo numerado como i , se conseguía una matriz con el mismo ancho de banda pero con una envolvente con un número de elementos menor o igual.

Teorema 3.3 Sea A una matriz cuyo perfil es monótono. El número de elementos de $\text{Env}(A)$, numerando el grafo asociado a A de acuerdo con el resultado obtenido de aplicar el algoritmo inverso de Cuthill-McKee, es a lo sumo el mismo que el de la matriz asociada al grafo numerado de acuerdo con el resultado del algoritmo ordinario de Cuthill-McKee.

**Figura 3.21**

Grafo de 10 nudos de la figura 3.19 al que se le aplica el algoritmo de la tabla 3.6 para determinar qué nudo ha de ser el de partida para el algoritmo de Cuthill-McKee

Ejemplo 3.2 Considérese el grafo asociado a una matriz de la figura 3.22. Si se reordena de acuerdo con el algoritmo de Cuthill-McKee, el patrón de elementos distintos de cero y ceros que habría que guardar y la numeración que se obtendría son los que describe la figura 3.23.

Por el contrario, utilizando el algoritmo de Cuthill-McKee inverso se conseguiría el efecto que representa la figura 3.24.

Como se puede observar, desaparecen todos los ceros que aparecían antes, ahorrándose las correspondientes posiciones de memoria para guardarlos. ●

Si al grafo que se utilizaba para introducir el algoritmo de Cuthill-McKee, figura 3.19, se le aplicada el algoritmo inverso, el resultado que se obtiene es el que describe la figura 3.25. El ancho de banda sigue siendo el mismo que el que obtenía el algoritmo de Cuthill-McKee, 5, pero el número de elementos de la $\text{Env}(A)$ ha disminuido de 33 a 28.

3.4.6 Método de la disección anidada

Este método, conocido en la literatura especializada como *Nested Dissection*, tiene sus raíces en las técnicas de elementos finitos. Se basa en establecer unas particiones, usando separadores, para dividir sistemáticamente el grafo asociado a una matriz de estructura simbólica simétrica. Cuando se encuentra uno de estos separadores, se numeran sus nudos y se retiran del grafo, dejándolo dividido en dos o más componentes (si se retiran del grafo de la figura 3.26-a los nudos 11 al 15, resultan dos subgrafos sin conexiones comunes). En los componentes que resultan de la primera transformación también se buscan separadores, continuando el proceso hasta que se numeren todos los nudos del grafo. La numeración obtenida posee propiedades muy interesantes en lo que se refiere tanto a las operaciones necesarias que habrá que realizar para factorizar mediante eliminación de Gauss la matriz asociada como a los elementos de relleno que produce esa factorización.

Suponiendo que el conjunto de los nudos del grafo se puede representar por un rectángulo R_0 , en él se escoge un separador, S_0 , formado por un subconjunto de nudos de R_0 . Al retirar éstos del grafo, R_0 queda dividido en dos subgrafos o componentes, R_1^1 y R_1^2 . Los nudos de R_1^1 se numeran primero y luego los de R_1^2 y S_0 . El patrón de elementos distintos de cero al que da lugar esta numeración se representa mediante sectores sombreados en la figura 3.26-b. Si esta matriz se factorizase mediante eliminación de Gauss, los posibles rellenos sólo se producirían en las zonas destacadas.

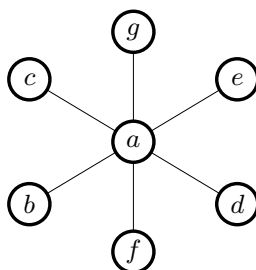
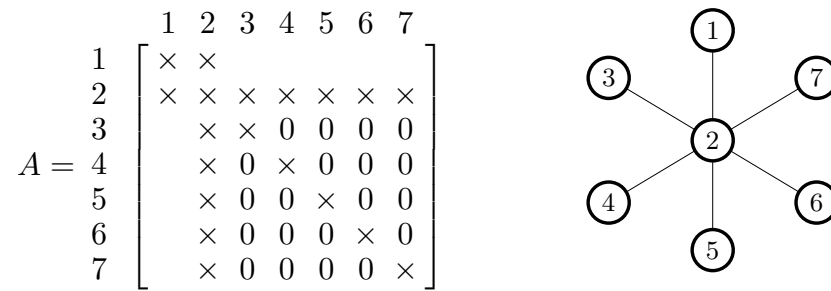
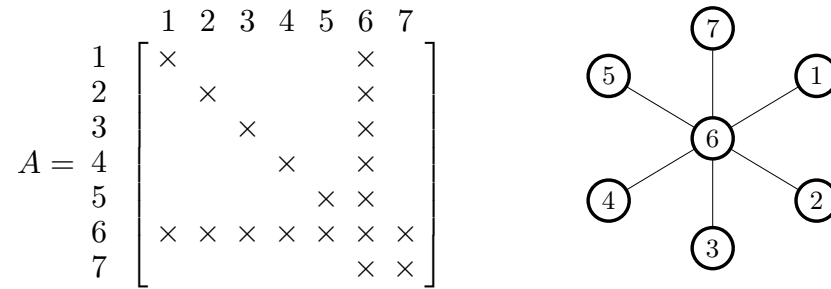


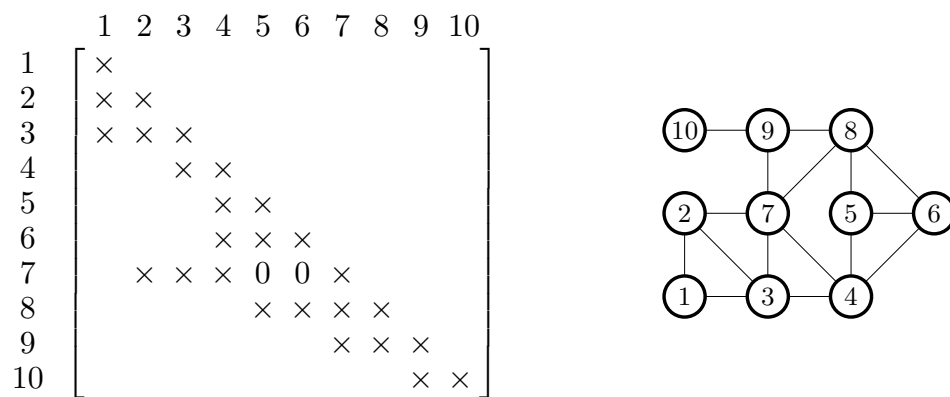
Figura 3.22
Ejemplo 3.2


Figura 3.23

Ejemplo de la adaptación del algoritmo de Cuthill-McKee al grafo de la figura 3.22


Figura 3.24

Resultado de la aplicación del algoritmo inverso de Cuthill-McKee al grafo de la figura 3.22


Figura 3.25

Resultado del algoritmo inverso de Cuthill-McKee aplicado el grafo de la figura 3.19

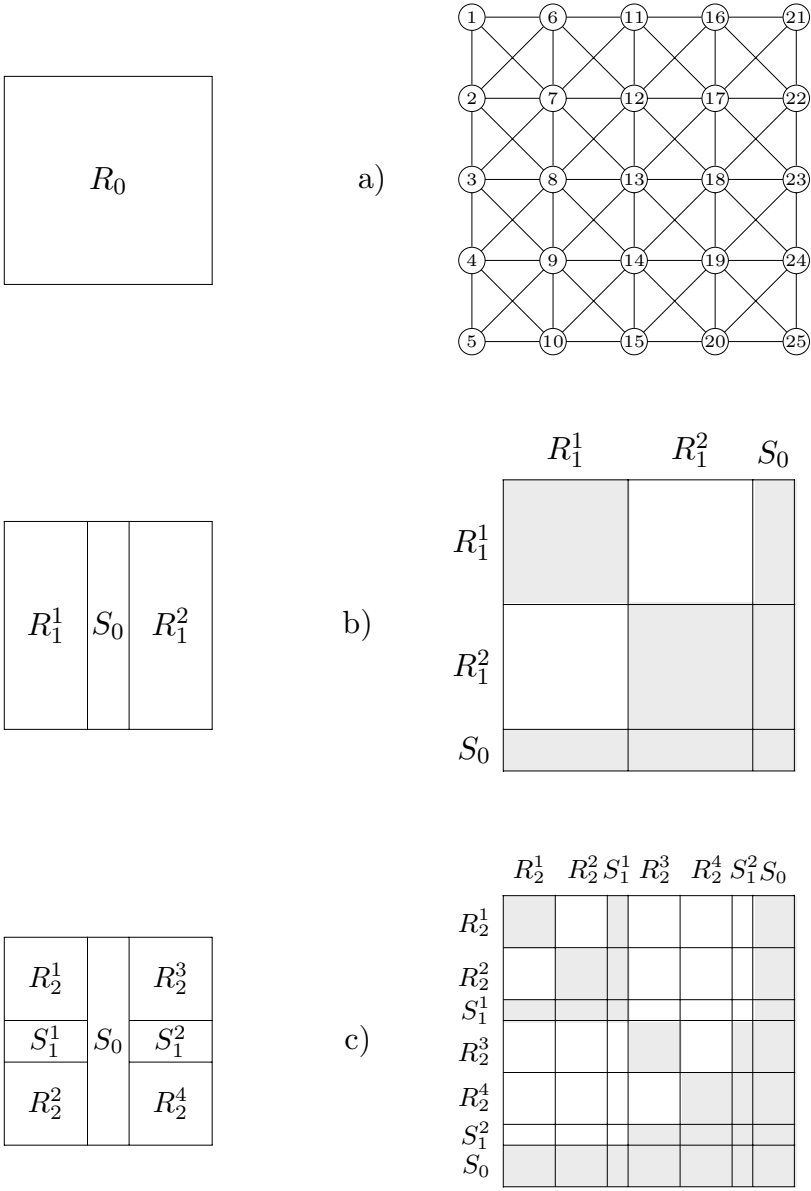


Figura 3.26
Método de la disección anidada

Si se sigue el proceso y se divide el componente R_1^1 , mediante un separador S_1^1 , en R_2^1 y R_2^2 , y R_1^2 , mediante otro S_1^2 , en R_2^3 y R_2^4 , numerando primero R_2^1 , luego R_2^2 , S_1^1 , R_2^3 , R_2^4 , S_1^2 y, por último, S_0 , la nueva numeración produciría un patrón de elementos cero y distintos de cero según se representa en la figura 3.26-c por las zonas en blanco y sombreadas, respectivamente.

El procedimiento continuaría hasta que no se pudiesen encontrar separadores en los subgrafos R .

3.4.7 Método de la disección en un sentido

Conocido en la literatura especializada como *One Way Dissection*, este método fue diseñado por George [1980] para problemas de elementos finitos de dos dimensiones. Posteriormente se ha aplicado también a problemas generales.

La idea en la que se basa se ilustra en la figura 3.27. El rectángulo esquematiza un grafo asociado a un problema de elementos finitos en dos dimensiones como, por ejemplo, el de la figura 3.26-a. Si se toman σ separadores ($\sigma = 3$ en la figura) y se disecciona el grafo en $\sigma + 1$ bloques $R_1, R_2, \dots$ de parecido tamaño, considerando que los separadores forman un único bloque, se obtiene un árbol partición como el árbol cociente que muestra la figura 3.27-b. Si posteriormente se numeran los nudos de cada bloque R comenzando por los de la última fila de izquierda a derecha, luego la penúltima, etc, y a continuación los nudos de los separadores

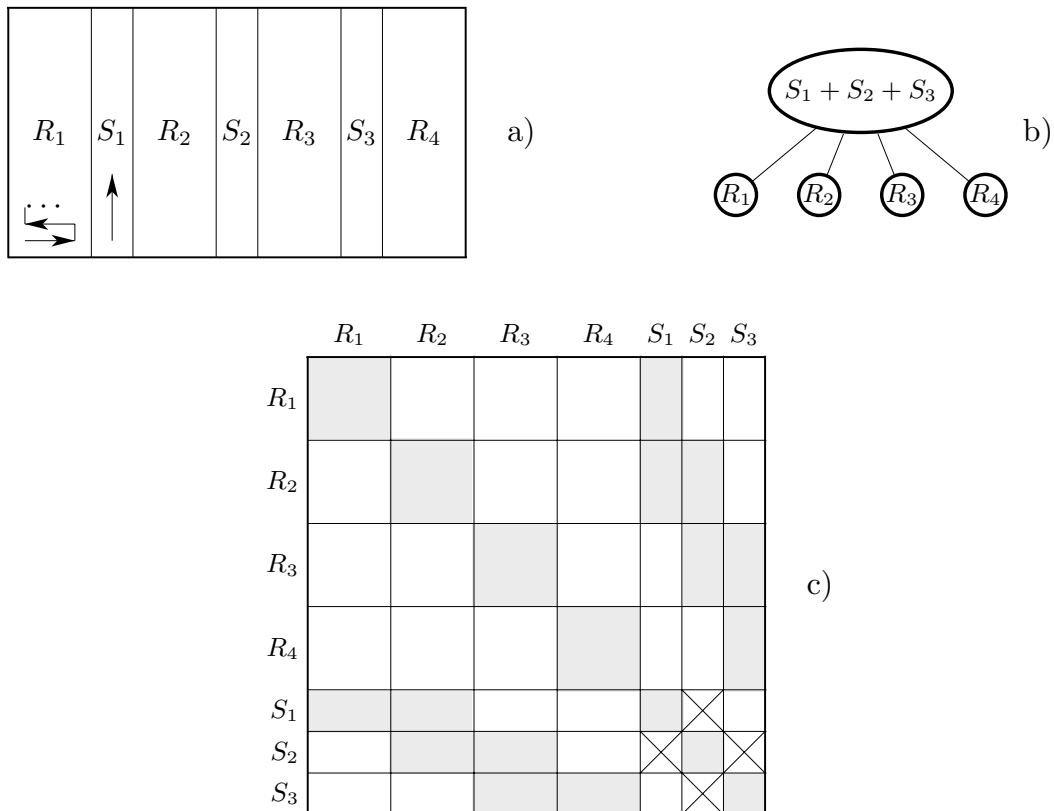


Figura 3.27
Método de la disección en un sentido

S , el patrón de elementos cero/distintos de cero que se obtendría en la matriz asociada al grafo sería el de la figura 3.27-c. Si esta matriz se factorizase mediante eliminación de Gauss, los elementos de relleno se darían sólo en las zonas reticuladas o en las cruzadas. Las zonas reticuladas, en cualquier caso, no suelen ser totalmente densas sino con forma de banda.

3.5 Matrices dispersas no simétricas y eliminación de Gauss

En este apartado estudiamos diversas ordenaciones y cómo factorizar de forma adecuada mediante eliminación de Gauss matrices dispersas de estructura general.

La factorización que se obtiene como consecuencia del proceso de eliminación de una matriz general, A , es,

$$A = L_1 D U_1,$$

donde L_1 es una matriz triangular inferior con todos los elementos de su diagonal principal iguales a 1, D una matriz diagonal y U_1 una matriz triangular superior, también con todos los elementos de su diagonal principal iguales a 1. El producto $L_1 D$ también se suele englobar en una sola matriz triangular inferior, L , cuyo patrón de elementos distintos de cero es idéntico al de L_1 .

Como se indicó en el caso de matrices simétricas, en el transcurso de la eliminación o factorización de la matriz se pueden crear elementos de relleno; si son muchos, no sólo pueden destruir cualquier estructura de dispersidad sino que también pueden dar al traste con la consideración hecha a priori de que la matriz era dispersa, pues se puede aumentar en exceso el número de operaciones a realizar. Para evitar esto, si es posible, se efectúa una reordenación de filas y columnas, materializadas por un conjunto de permutaciones, de tal forma que en la matriz resultante, al factorizarla, la estructura de elementos cero/distintos de cero sea mucho más fácil de manipular, o aparezcan muchos menos elementos de relleno que al factorizar la original.

Si el sistema a resolver es

$$A\mathbf{x} = \mathbf{b}$$

y se le aplican a A un conjunto de permutaciones elementales a la izquierda y a la derecha representadas por las matrices de permutación P y Q , respectivamente, el sistema se puede reescribir,

$$P A Q Q^T \mathbf{x} = P \mathbf{b},$$

pues $Q^T Q = I$. Haciendo $\mathbf{y} = Q^T \mathbf{x}$ y $\mathbf{c} = P \mathbf{b}$, se tiene que,

$$B \mathbf{y} = \mathbf{c},$$

donde $B = P A Q$ es la matriz A reordenada. En general $Q \neq P^T$ por lo que B se obtiene de A mediante un conjunto de permutaciones no simétricas de sus filas y de sus columnas. El objetivo de manipulaciones como éstas, como decíamos, es conseguir que B tenga una estructura de dispersidad más fácil de tratar que A , que su factorización sea lo más numéricamente estable posible y que el número de rellenos sea el menor posible.

Comenzaremos estudiando (cuando la matriz A no posea ninguna propiedad particular) la

forma de conseguir en A una estructura *triangular inferior en bloques*:

$$\begin{bmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ \vdots & & \ddots & \\ A_{n1} & A_{n2} & \cdots & A_{nn} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}_n \end{bmatrix}, \quad (3.1)$$

donde los coeficientes A_{ij} designan matrices, siendo las A_{ii} matrices cuadradas de orden n_i ; evidentemente, $\sum_{i=1}^n n_i = n$. Como indicamos en el apartado 3.3.1, el sistema (3.1) se resuelve mediante una sucesión de n subproblemas más pequeños: el subproblema i será de orden n_i y su matriz de coeficientes A_{ii} . Para resolver esos subproblemas será necesario factorizar sólo las A_{ii} por lo que el relleno de elementos cero sólo ocurrirá dentro de esas A_{ii} . El procedimiento de esta resolución lo describen los siguientes pasos:

- a) Resolver el primer subsistema en n_1 incógnitas, $A_{11}\mathbf{x}_1 = \mathbf{b}_1$, con A_{11} como matriz de coeficientes: se obtendrá $\mathbf{x}_1$.
- b) Restar los vectores $A_{j1}\mathbf{x}_1$ del término independiente, $\mathbf{b}_j$, para $j = 2, \dots, n$, obteniéndose una matriz triangular inferior en bloques de orden $n - n_1$. Repetir a) y b) hasta completar la solución.

Para llevar a cabo este proceso, evidentemente, se supone que los bloques de la diagonal principal son regulares.

Las siguientes consideraciones son fundamentales para la consecución de la triangularización en bloques de la matriz A .

Definición 3.5 Una matriz se dice que tiene un *transversal completo* cuando todos los elementos de su diagonal principal son distintos de cero.

Cualquier matriz regular se puede reordenar mediante permutaciones no simétricas P y Q , de tal forma que PAQ tenga un transversal completo. Si la matriz es singular esto puede no cumplirse.

Si una matriz tiene un transversal completo, puede reordenarse de tal forma que se consiga una estructura triangular inferior en bloques como la indicada en (3.1). Esta reordenación se consigue mediante permutaciones simétricas de la forma PAP^T . Si esa estructura de bloques existe, se dice que A es una *matriz reducible*. Si una matriz no tiene transversal completo pero puede reordenarse de tal forma que entonces sí lo tenga, y así reordenada es reducible, se dice *birreducible*.

Para conseguir triangularizar por bloques una matriz A dispersa cualquiera se procede, pues, en dos fases:

- Fase 1. Encontrando un *transversal completo* de esa matriz.
- Fase 2. Reordenando el resultado de la fase 1 mediante permutaciones simétricas.

Estas dos fases se materializan mediante sendos algoritmos. Para explicarlos recurrimos una vez más al concurso de los grafos; esta vez en su faceta de grafos dirigidos.

3.5.1 Nociones básicas sobre grafos dirigidos

Como ya hemos apuntado, la teoría de matrices dispersas y la de grafos son dos disciplinas con muchos vínculos y resultados comúnmente aplicables. El patrón de elementos no nulos de una matriz dispersa no simétrica cuadrada también se puede representar mediante un grafo.

Algunos de los conceptos que introducimos en este apartado ya se presentaron en el correspondiente a grafos no dirigidos; si se vuelve a hacer es para facilitar el seguimiento de lo referente a grafos dirigidos.

Un *grafo*, $G = (V, E)$, es un par formado por un conjunto finito, V , de elementos denominados *vértices* o *nudos* del grafo, y por otro también finito, E , de *arcos* o *aristas*. Un arco es un par de nudos. Si los arcos de un grafo son ordenados, el grafo se denomina *digrafo* o *grafo dirigido*. Si $e = (i, j) \in E$, este arco une un nudo de *origen* o *cola* $i = t(e)$ con otro de *destino*, *final* o *cabeza* $j = h(e)$. El número de elementos o cardinal de V o E se designa $|V|$ o $|E|$. Un grafo $G = (V, E)$ se dice *numerado* si existe una biyección $\alpha : \{1, 2, \dots, |N|\} \rightarrow V$. En lo sucesivo, cuando hablemos de un grafo dirigido o digrafo, lo supondremos numerado.

A cualquier matriz general cuadrada, A , se le puede asociar un digrafo. El digrafo asociado a una matriz A de orden n , $G^A = (V^A, E^A)$, es un grafo numerado dirigido de nudos $V^A = \{v_1, v_2, \dots, v_n\}$ y arcos E^A definidos de tal forma que

$$(v_i, v_j) \in E^A \Leftrightarrow a_{ij} \neq 0.$$

Los arcos de un digrafo también se designan por $(u \rightarrow v)$. A cualquier elemento diagonal $a_{ii} \neq 0$ también le corresponde un arco (bucle) que parte y llega a v_i . Cuando todos los elementos de la diagonal principal de la matriz asociada al digrafo son distintos de cero, estos arcos o bucles no se suelen representar. Al conjunto de elementos de la diagonal principal de una matriz que no son cero se le denomina *transversal*.

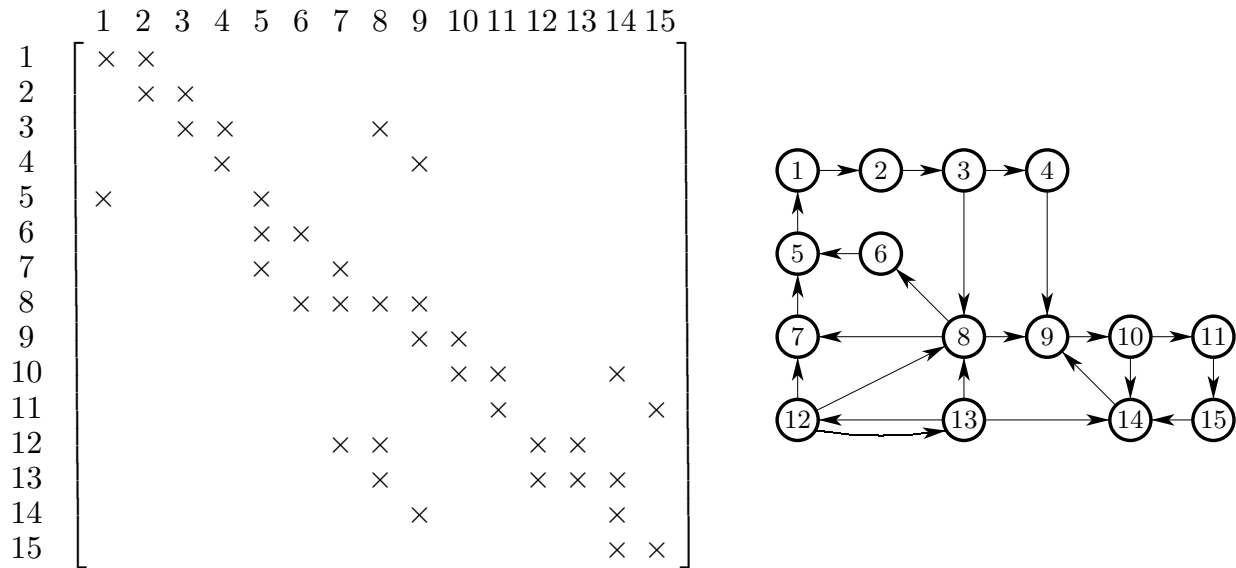
En la figura 3.28 se describe la estructura simbólica de una matriz no simétrica 15×15 y su digrafo asociado.

Si a una matriz A se le efectúan una serie de permutaciones simétricas, su digrafo asociado permanece inalterado: sólo se modifica la numeración de sus nudos.

Un arco de un grafo dirigido (u, v) ó $(u \rightarrow v)$ se dice que *sale* o *parte* del nudo u y *llega* o *entra* a/en el nudo v . También se dice que el arco $(u \rightarrow v)$ lleva del nudo u al nudo v . El *grado de entrada* o *llegada* de un nudo es el número de nudos que a él llegan; el *grado de salida*, el número de nudos que de él salen. Un nudo de un digrafo se dice de *oferta* cuando tiene un grado de entrada cero y un grado de salida positivo. Se dice de *demanda*, si tiene grado de entrada positivo y cero de salida.

Si $(u \rightarrow v)$ es un arco de un digrafo, el nudo v se dice *adyacente* al u . Si W es un subconjunto del de nudos del digrafo G , el *conjunto adyacente* de W , $\text{Adj}(W)$, es el conjunto de todos los nudos, no en W , adyacentes a los nudos de W . Es decir, $\text{Adj}(W) = \{v \in V - W : (u \rightarrow v) \in E \text{ para todo } u \in W\}$.

Un *camino dirigido* o camino de un digrafo, es un conjunto ordenado de nudos $\{u_1, u_2, \dots, u_{m+1}\}$ tal que u_{i+1} es adyacente a u_i para $i = 1, 2, \dots, m$. La longitud de ese camino es m . Cuando existe un camino de un nudo u a otro v , se dice que v es *accesible* desde u . La *matriz de accesibilidad* de un digrafo, $\mathbf{A}$, es una matriz Booleana definida de la siguiente manera: $\mathbf{a}_{ij} = 1$ si el nudo v_j es accesible desde v_i ; si no, $\mathbf{a}_{ij} = 0$. Un ciclo de un digrafo o *ciclo dirigido* es un camino, con al menos dos arcos, que sale y llega al mismo nudo. Si el nudo v es accesible desde u en un digrafo, la *distancia* desde u a v es la longitud del camino más corto de u a v (la distancia desde v a u puede ser distinta o incluso indefinida por no existir camino de v a u).

**Figura 3.28**

Matriz no simétrica y su digrafo asociado

Un digrafo se dice *conexo* si lo es el grafo no dirigido que se obtiene al suprimir las direcciones en los arcos del digrafo.

Resumiendo estos conceptos en el digrafo de la figura 3.28, éste es conexo; los nudos 6, 7 y 9 son adyacentes al nudo 8; si $W = \{8, 13\}$, $\text{Adj}(W) = \{6, 7, 9, 12, 14\}$; el nudo 5 tiene un grado de entrada de 2 y de salida de 1; el nudo 4 es accesible desde el 8; no hay nudos oferta ni nudos demanda.

Dado un digrafo conexo, $G = (V, E)$, el digrafo se dice *fuertemente conexo* si para cada par de nudos $u, w \in V$ existe un camino de u a w y otro de w a u , es decir, u y w son mutuamente accesibles. Como un camino de u a w seguido de otro de w a u constituye un ciclo, un digrafo se dice fuertemente conexo, por consiguiente, si para cada par de nudos existe un ciclo al que pertenecen. La matriz de accesibilidad de un digrafo fuertemente conexo es totalmente llena. El digrafo de la figura 3.28 no es fuertemente conexo.

Un *subgrafo sección*⁶ fuertemente conexo de un grafo G se denomina *componente fuertemente conexo* o *componente fuerte*. De la definición de subgrafo sección y de la de componente fuerte se deriva que cualquier ciclo del grafo G deberá estar compuesto en su totalidad por nudos del componente fuerte o por ninguno del componente fuerte, pues si existiese un ciclo que contuviese un nudo u del componente fuerte y otro w no en ese componente fuerte, se podría añadir w al componente fuerte sin perder su carácter, lo cual contradiría la hipótesis.

Debido a estas propiedades, un grafo conexo se puede dividir en un conjunto de componentes fuertes disjuntos $C_1, C_2, \dots, C_s$. Si G es fuertemente conexo, $s = 1$. Un arco $(v \rightarrow w)$ se dice que *sale* de un componente fuerte $C = (V_c, E_c)$, si $v \in V_c$ y $w \notin V_c$. El arco $(v \rightarrow w)$ *entra* en el componente fuerte $C = (V_c, E_c)$, si $v \notin V_c$ y $w \in V_c$. Como un componente fuerte es

⁶Recordemos la definición de subgrafo sección del apartado 3.4.1 (válida también para grafos dirigidos): Un subgrafo $G' = (V', E')$ de un grafo $G = (V, E)$ se dice *subgrafo sección* cuando V' contiene sólo algunos nudos de G y E' todos los arcos (u, v) de G tales que u y v pertenecen a V' ; es decir: $V' \subset V$ y $E' = \{(u, v) \in E : u \in V' \text{ y } v \in V'\}$.

un subgrafo sección, los arcos de entrada y salida no pertenecen evidentemente al componente fuerte.

Cuando un grafo G es divisible en sus componentes fuertes, es fácil ver que al menos uno de ellos debe salir de otro sin tener a su vez salida pues si cada componente la tuviese sería posible trazar un camino de un componente a otro hasta, eventualmente, llegar a uno de los ya visitados, lo que contradiría la definición de componente fuerte. En general, en un digrafo puede haber varios componentes fuertes sin salida. Con las ideas de entradas y salidas se puede construir la denominada *estructura de niveles de conexión* de un digrafo.

3.5.2 Interpretación grafo-teórica de la eliminación de Gauss de matrices dispersas no simétricas

En digrafos también existe el concepto de grafo de eliminación. En una etapa k de la eliminación de Gauss, el *digrafo de eliminación* (o, indistintamente, también, grafo de eliminación) correspondiente, relativo a la submatriz activa, se forma eliminando del de la fase $k - 1$ el nudo v_k , todos los arcos que a él llegan o de él parten, y añadiendo un arco $(u \rightarrow w)$ cuando existe un camino dirigido $\{u, v, w\}$. Por ejemplo, en el digrafo de la figura 3.28, en la primera etapa, el grafo de eliminación se obtiene quitando del original el nudo 1 y añadiendo el arco $(5 \rightarrow 2)$, según se representa en la figura 3.29. Al final del proceso, la matriz simbólica que indica qué elementos serán distintos de cero una vez completada la factorización tendrá la forma que se describe en la figura 3.30.

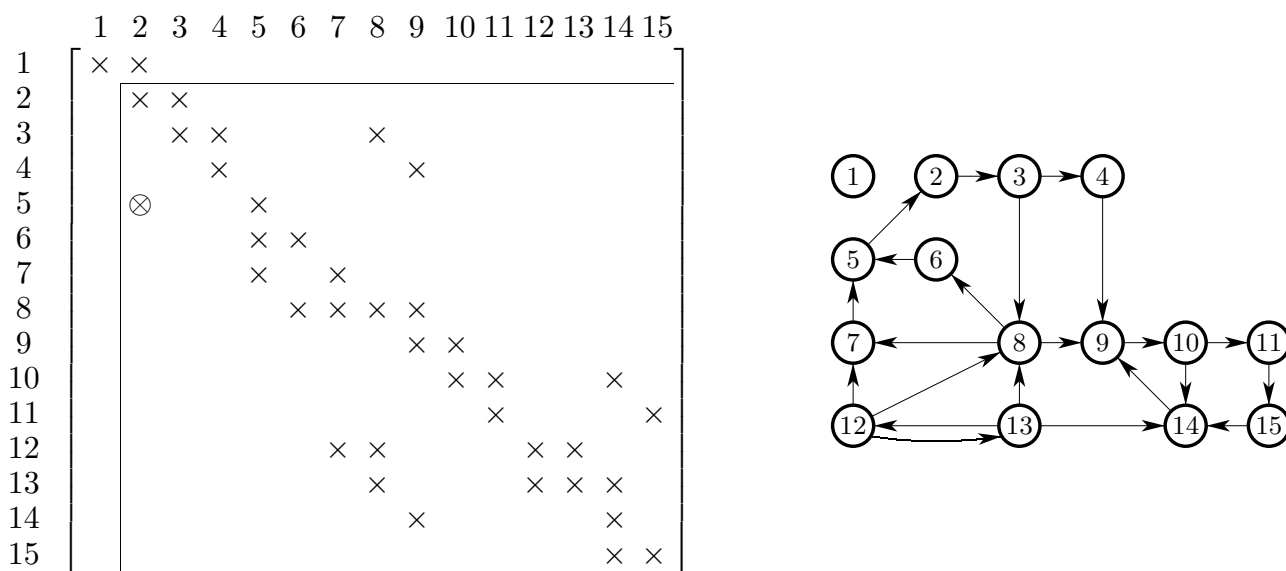


Figura 3.29

Primera etapa de la eliminación de Gauss y su correspondiente digrafo de eliminación de la matriz de la figura 3.28. El elemento de relleno se indica mediante el símbolo $\otimes$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	×	×													
2		×	×												
3				×	×				×						
4					×					×					
5	×	⊗	⊗	⊗	×			⊗	⊗						
6					×	×		⊗	⊗						
7					×		×	⊗	⊗						
8						×	×	×	×						
9									×	×					
10										×	×			×	
11											×				×
12							×	×	⊗	⊗	⊗	×	×	⊗	⊗
13								×	⊗	⊗	⊗	×	×	×	⊗
14									×	⊗	⊗			×	⊗
15														×	×

Figura 3.30

Resultado final de la eliminación de Gauss simbólica de la matriz de la figura 3.28

3.5.3 Obtención de un transversal completo. Algoritmo de Hall

Siguiendo el objetivo de conseguir en $Ax = b$ una estructura de la matriz de coeficientes triangular en bloques, de acuerdo con el esquema en dos etapas apuntado en la página 247, la primera tarea a realizar consiste en reordenar la matriz de tal forma que, si no lo tiene ya, se consiga un transversal completo; es decir, que todos los elementos de la diagonal principal de la matriz de coeficientes reordenada sean distintos de cero.

Si la matriz es regular siempre es posible conseguir un transversal completo; si es singular, no siempre. Si no se puede dar ese transversal completo, la matriz se dice *simbólicamente singular*. Si el máximo transversal conseguible es $k < n$, a k se le denomina *rango simbólico*.

El algoritmo que describimos a continuación para conseguir un transversal completo es una modificación del de Hall [1956]. Requiere n etapas; el objetivo de cada una, k , es colocar un elemento distinto de cero en la posición k de la diagonal principal.

Funciona de la siguiente manera. Supongamos que se han realizado k etapas del algoritmo y que los k primeros elementos de la diagonal principal son distintos de cero. En la etapa $k + 1$ podrá ocurrir:

- Que $a_{k+1, k+1} \neq 0$ con lo que se finaliza la etapa $k + 1$.
- Que $a_{k+1, k+1} = 0$ pero que exista en la submatriz activa, es decir la submatriz de índices de filas y columnas $k + 1$ a n , un elemento distinto de cero. En este caso, realizando los intercambios de filas y columnas necesarios, se puede llevar ese elemento distinto de cero a la posición $(k + 1, k + 1)$. La submatriz que forman los elementos de subíndices 1 a k no se verá afectada por estos intercambios por lo que los k primeros elementos de la diagonal principal seguirán siendo distintos de cero.
- Que sólo existan elementos cero en la submatriz activa. En este caso también puede ser posible conseguir colocar un elemento distinto de cero en la posición $k + 1$ de la diagonal

principal. Para ello se recurre al trazado de un denominado *camino creciente* a través de los elementos de la matriz. Si no se puede concluir con éxito, la matriz será singular y no se podrá conseguir un transversal completo.

El camino creciente comienza (si suponemos que estamos en la etapa $k + 1$) en la posición $(k + 1, k + 1)$ de la matriz A ; continúa por la fila $k + 1$ hasta que se encuentra con un elemento distinto de cero en una columna, por ejemplo, la ℓ (tal columna debe existir pues de lo contrario todos los elementos de la fila $k + 1$ serán cero y la matriz, por tanto, singular); de aquí al elemento (ℓ, ℓ) a lo largo de la columna ℓ ; a continuación, por la fila ℓ hasta encontrar un elemento distinto de cero, por ejemplo en la columna m , etc. El camino va por tanto alternativamente de un elemento diagonal a otro distinto de cero no en la diagonal. Este camino no puede atravesar una misma fila y columna más de una vez y, en consecuencia, tampoco un mismo elemento de la diagonal principal. Terminará en un elemento distinto de cero en la submatriz que definen las filas de índices 1 a k y las columnas de índices $k + 1$ a n .

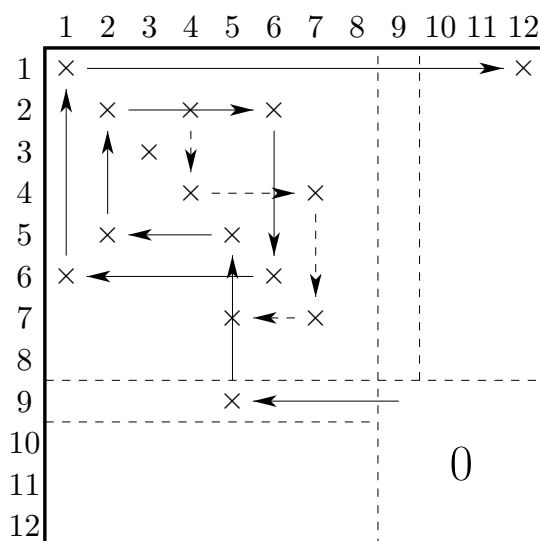
Si en el trazado del camino a lo largo de una fila no es posible encontrar un elemento distinto de cero no diagonal en una columna no visitada previamente, se borra la fila del camino (no de la lista de posiciones visitadas) y se vuelve a la fila anterior. Si en un momento dado, habiéndose visitado r posiciones entre las filas/columnas 1 a k , el camino no puede visitar ningún nuevo elemento no visitado previamente, la matriz A es singular. En este caso $r + 1$ filas (las visitadas y la fila $k + 1$) sólo tienen elementos distintos de cero en r columnas.

Una vez trazado el camino, por ejemplo $k + 1, \ell_1, \ell_2, \dots, \ell_r$, donde $\ell_r > k$, se intercambian $r + 1$ filas y dos columnas a fin de llevar el último elemento distinto de cero encontrado en el camino a la posición $(k + 1, k + 1)$ de la matriz A . Estos intercambios de filas se hacen de la siguiente manera:

$$\begin{array}{ll} \text{la fila } k + 1 & \text{se intercambia con la } \ell_1 \\ \text{la fila } \ell_1 & \text{se intercambia con la } \ell_2 \\ \vdots & \vdots \\ \text{la fila } \ell_{r-1} & \text{se intercambia con la } k + 1. \end{array}$$

Como las filas se seleccionan de tal manera que la fila ℓ_1 tiene un elemento distinto de cero en la posición (ℓ_1, ℓ_1) , mediante estos intercambios ese elemento distinto de cero se trasladará a la posición diagonal (ℓ_1, ℓ_1) cuando la fila ℓ_1 reemplace a la fila $k + 1$. En consecuencia, la estructura de elementos distintos de cero en la diagonal principal en las posiciones 1 a k no se verá afectada por esos intercambios de filas. Además, el último elemento distinto de cero encontrado en el camino, una vez efectuados los intercambios, acabará en la posición $(k + 1, \ell_r)$; un último intercambio de las columnas $k + 1$ y ℓ_r lo llevará a la posición deseada, $(k + 1, k + 1)$. Obviamente, si $\ell_r = k + 1$, este último intercambio de columnas no será necesario.

Para aclarar el procedimiento, consideremos una matriz 12×12 y el procedimiento descrito es su etapa 9 —ver figura 3.31—. Esta etapa 9 comienza en la posición $(9, 9)$. La submatriz que forman los elementos de índices de filas y columnas 9 a 12 son todos cero por lo que se trata de trazar un camino creciente. Éste comienza en la posición $(9, 9)$, sigue por la fila 9 hasta encontrar el elemento distinto de cero de la columna 5; continúa por la columna 5 hasta encontrar el correspondiente elemento distinto de cero de la diagonal principal; sigue por la fila 5 hasta encontrar el elemento distinto de cero de la columna 2; continúa por la columna 2 hasta encontrar el elemento distinto de cero de la diagonal principal en la fila 2; sigue por la fila 2 hasta encontrar el elemento distinto de cero de la columna 4; continúa por la columna

**Figura 3.31**

Algoritmo de Hall para la búsqueda de un transversal completo en una matriz 12×12

4 hasta alcanzar el elemento distinto de cero de la diagonal principal en la fila 4; sigue por la fila 4 hasta encontrar el elemento distinto de cero de la columna 7; continúa por la columna 7 hasta alcanzar el elemento distinto de cero de la diagonal principal en la fila 7; sigue por la fila 7 encontrándose que el único elemento distinto de cero de esta fila está en la columna 5 que ya se ha visitado. Esto obliga a borrar las filas 7 y 4 del camino (no de la lista de posiciones ya visitadas) y reemprender la marcha allí donde se abandonó en la fila 2. Por la fila 2 se llega hasta la columna 6 donde está el siguiente elemento distinto de cero; continúa en la columna 6 hasta la diagonal principal en la fila 6; sigue por esa fila 6 hasta llegar al elemento distinto de cero de la columna 1; continúa por la columna 1 hasta alcanzar el elemento distinto de cero de la diagonal principal en la fila 1; sigue por la fila 1, parándose al llegar al primer elemento distinto de cero que encuentra en la columna 12 por estar en la submatriz que nos interesa. El camino es pues

$$\{9, 5, 2, 6, 1, 12\}.$$

Una vez encontrado el camino, se efectúan los siguientes intercambios de filas:

- la fila 9 con la 5;
- la fila 5 con la 2;
- la fila 2 con la 6;
- la fila 6 con la 1;
- la fila 1 con la 12 y
- la fila 12 con la 9.

Estos intercambios trasladan el elemento distinto de cero (1, 12) a la posición (9, 12). Por último se intercambian las columnas 9 y 12.

Si el elemento $(6, 1)$ fuese cero, hubiésemos encontrado que las filas 9, 5, 2, 4, 7 y 6 tienen elementos distintos de cero sólo en las columnas 5, 2, 4, 7 y 6, lo que significaría que la matriz es singular.

La implementación de este algoritmo en ordenador es relativamente fácil; el camino es muy sencillo de construir en memoria pues sólo es necesario almacenar en un vector los índices de los elementos diagonales en el orden en que se visitan. Como no se pueden visitar más de una vez las posiciones de la diagonal principal y puede ocurrir que se supriman del camino, también es necesario guardar en otro vector las posiciones ya visitadas.

3.5.4 Permutaciones simétricas hacia una estructura triangular en bloques

Una vez obtenida una permutación P_1 de la matriz A de manera que P_1A tenga un transversal completo, la siguiente fase de un proceso de triangularización en bloques de esa matriz consiste en encontrar otra permutación, esta vez simétrica, Q , de tal manera que al aplicarla a P_1A se consiga la deseada estructura triangular en bloques, es decir,

$$Q^T(P_1A)Q = \begin{bmatrix} B_{11} & & & \\ B_{21} & B_{22} & & \\ \vdots & & \ddots & \\ B_{n1} & B_{n2} & \cdots & B_{nn} \end{bmatrix},$$

donde cada bloque diagonal, B_{ii} , no se pueda reducir a una forma triangular inferior.

Para conseguir la matriz Q nos apoyaremos una vez más en la teoría de grafos dirigidos (concretamente en el digrafo asociado a P_1A). En concreto, en la parte que hace referencia a los componentes fuertes de un digrafo. Recordemos una vez más que al aplicar permutaciones simétricas a un digrafo lo único que varía es la numeración asociada a los nudos.

Los dos algoritmos que vamos a estudiar se basan en encontrar en el digrafo asociado a una matriz los ciclos que definen sus componentes fuertes. Si, por ejemplo, existiesen dos de estos componentes fuertes y se reordenase la numeración de los nudos de tal forma que los del primero fuesen los nudos 1 al k y los del segundo del $k + 1$ al n , se obtendría una matriz triangular inferior en dos bloques, el primero con k columnas y el segundo con $n - k$. En el grafo dirigido de la figura 3.32 existen dos componentes fuertes, el formado por los nudos 1 y

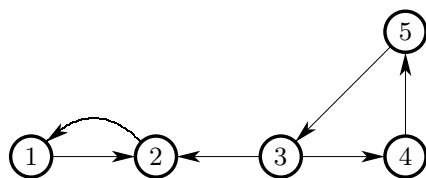


Figura 3.32

Digrafo con dos componentes fuertes

2 y el que definen 3, 4 y 5. La estructura simbólica de su matriz asociada es

×	×		
×	×		
	×	×	×
			×
		×	×

Estructura triangular inferior en 2 bloques.

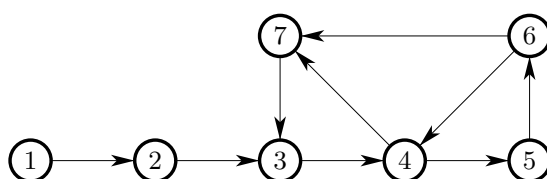
Una matriz triangular inferior se puede ver como el caso límite de la triangular inferior en bloques cuando cada bloque diagonal tiene un tamaño 1×1 . Recíprocamente, la triangular inferior en bloques se puede ver como una generalización de la triangular inferior en la que cada componente fuerte hace las veces de un supernudo o nudo generalizado.

Los algoritmos para reducir una matriz A cualquiera a una estructura triangular inferior se basan en la siguiente observación: si se pudiese reordenar A y transformarla en triangular inferior, debería haber un nudo en su digrafo asociado del cual no partiese ningún camino, pues a ese nudo sólo llegarían arcos en virtud de la estructura de la matriz. Si existiese un tal nudo debería numerarse el primero en el digrafo reenumerado y efectuarse las correspondientes permutaciones de fila y columna en la matriz a fin de llevarlo a la primera posición. Una vez hecho esto, eliminando ese nudo y todos los arcos que a él llegasen se conseguiría un subgrafo en el cual, en virtud de nuevo de la estructura de la matriz, existiría otro nudo al cual sólo llegarían arcos. Continuando con esta forma de actuación se llegaría a conseguir una permutación simétrica que daría como resultado una matriz triangular inferior.

Para llevar esta forma de proceder a la práctica, se puede comenzar desde cualquier nudo del digrafo asociado correspondiente y trazar un camino hasta que se encuentre un nudo desde el que no parta ningún otro. Esto es fácil de implementar pues, dado que sabemos que la matriz es triangularizable, no existen ciclos: cualquier camino sólo puede tener una longitud como máximo igual a $n - 1$, donde n es el orden de la matriz. Una vez encontrado ese nudo, se numera con el número 1 y se elimina del digrafo inicial así como todos los arcos que a él llegan. Del camino trazado se coge el nudo inmediatamente anterior al ya eliminado, u otro cualquiera, hasta encontrar uno del que como antes no parta ningún arco. Repitiendo esta forma de proceder hasta agotar los nudos se conseguía la estructura triangular inferior esperada.

El digrafo de la figura 3.33 y la tabla 3.7 ilustran este proceso. Los nudos que se van seleccionando para ser numerados se indican en negrita de izquierda a derecha. El digrafo reenumerado resultará de atribuir, al nudo 3 el número 1, al 5 el 2, al 4 el 3, al 2 el 4, al 1 el 5, al 7 el 6 y al 6 el 7. Obsérvese cómo en el paso 5 no existe camino alguno desde el nudo 5 puesto que el nudo 3 ya había sido eliminado. En el paso 9 hay que trazar un nuevo camino pues el anterior se había agotado de nudos. Las matrices original y reordenada de este ejemplo son

$$\begin{array}{c}
 \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{array}
 \begin{bmatrix}
 1 & 2 & 3 & 4 & 5 & 6 & 7 \\
 \times & \times & \times & & & & \\
 & \times & \times & \times & & & \\
 & & \times & & & & \\
 & & & \times & \times & & \\
 & & \times & & \times & & \\
 & & & \times & & \times & \times \\
 & & & & \times & & \times
 \end{bmatrix}
 \end{array}
 \quad y \quad
 \begin{array}{c}
 \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{array}
 \begin{bmatrix}
 1 & 2 & 3 & 4 & 5 & 6 & 7 \\
 \times & & & & & & \\
 \times & \times & & & & & \\
 & \times & \times & & & & \\
 \times & & \times & \times & & & \\
 & \times & & & \times & \times & \\
 & & \times & & & \times & \times \\
 & & & \times & & & \times
 \end{bmatrix}
 \end{array}$$

**Figura 3.34**

Digrafo sobre el que se aplica el algoritmo de Sargent y Westerberg

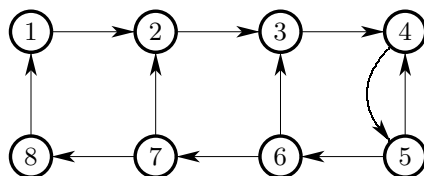
4'. El camino continúa desde este nudo 4' al 7 y de nuevo al 3. Se identifica la existencia del ciclo $\{3, 4', 7, 3\}$, renumerándose el conjunto que forman estos tres nudos como el nudo 3'. Como el nudo (nudo compuesto) 3' finaliza el camino, se numera como el nudo 1 en el digrafo renumerado y se elimina del digrafo original. El nudo 2 se numera también como 2 para el futuro digrafo renumerado y, por último, el nudo 1 como nuevo 3. Las matrices originales y reordenadas que corresponden a este proceso son las siguientes:

$$\begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{array} \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ \times & \times & & & & & \\ & \times & \times & & & & \\ & & \times & \times & & & \\ & & & \times & \times & & \times \\ & & & & \times & \times & \\ & & & \times & & \times & \times \\ & & \times & & & & \times \end{bmatrix} \quad y \quad \begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{array} \left[\begin{array}{cccccc|cc} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ \times & \times & & & & & \\ & \times & \times & & \times & & \\ & & \times & \times & & & \\ & \times & & \times & \times & & \\ & & & \times & \times & & \\ \times & & & & \times & & \\ \hline \times & & & & & \times & \\ \hline & & & & & \times & \times \end{array} \right].$$

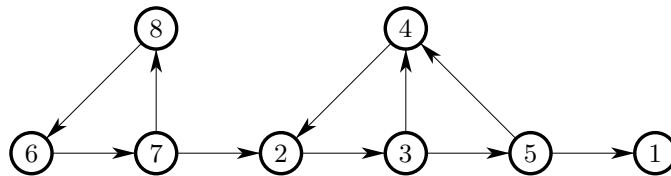
3.5.4.2 Algoritmo de Tarjan

El algoritmo de Tarjan [1972] se basa en la misma idea que el de Sargent y Westerberg: trazar caminos en el digrafo asociado a la matriz e identificar los componentes fuertes. La ventaja fundamental de éste frente al de Sargent y Westerberg, radica en que evita la gran cantidad de reenumeraciones que puede llegar a ser necesario hacer en aquel. Por ejemplo, al aplicar el algoritmo de Sargent y Westerberg al digrafo de la figura 3.35, los nudos compuestos que se van creando son $\{4, 5\}$, $\{3, 4', 6\}$, $\{2, 3', 7\}$ y $\{1, 2', 8\}$; en general, para un digrafo de estas características, con n nudos, se producirán $2 + 4 + 6 + \dots + n = n^2/4 + n/2$ reasignaciones de números de nudos.

El algoritmo propuesto por Tarjan evita esas reenumeraciones constantes mediante el ingenioso uso de unas denominadas *pilas* de números parecidas a las usadas en la tabla 3.7.

**Figura 3.35**

Digrafo en el que el algoritmo de Sargent y Westerberg presenta dificultades

**Figura 3.37**

Digrafo de la figura 3.36 una vez renumerado con el algoritmo de Tarjan

Tabla 3.10

Algoritmo de Tarjan

Paso 0 – *Inicialización.* Hacer $A_e \leftarrow \emptyset$, $V_v \leftarrow \emptyset$ y $i \leftarrow 0$. Ir al paso 1.

Paso 1 – *Selección del nudo de partida.* Seleccionar cualquier nudo $v \notin V_v$. Si no hay ninguno, PARAR.

Paso 2 – *Visitar un nudo.* Añadir el nudo v a la pila y al camino. Hacer:

$$\begin{aligned} V_v &\leftarrow V_v \cup \{v\} \\ i &\leftarrow i + 1 \\ num(v) &\leftarrow i \\ lowlink(v) &\leftarrow i. \end{aligned}$$

Paso 3 – *Explorar arcos.* Mirar entre los arcos que salen de v si hay $(v \rightarrow w) \notin A_e$:

- si hay un tal arco y el nudo $w \notin V_v$, hacer $A_e \leftarrow A_e \cup (v \rightarrow w)$, $v \leftarrow w$ e ir al paso 2.
- si hay un tal arco y el nudo $w \in V_v$, hacer $A_e \leftarrow A_e \cup (v \rightarrow w)$ e ir al paso 4 para ajustar $lowlink(v)$.
- si no hay un tal arco y $lowlink(v) < num(v)$, ir al paso 5.
- si no hay un tal arco y $lowlink(v) = num(v)$, ir al paso 6 para formar el componente fuerte correspondiente.

Paso 4 – *Ajustar el vector lowlink.* Si $num(w) < num(v)$ y w está en la pila, hacer $lowlink(v) \leftarrow \min\{lowlink(v), lowlink(w)\}$ e ir al paso 3. Si no, ir al paso 3 directamente.

Paso 5 – *Retroceder.* Retirar v del camino. Hacer:

$$\begin{aligned} w &\leftarrow \text{nudo al final del camino} \\ lowlink(u) &\leftarrow \min.\{lowlink(u), lowlink(v)\} \\ v &\leftarrow u. \end{aligned}$$

Ir al paso 3.

Paso 6 – *Definir un componente fuerte.* Retirar v y todos los nudos por encima de éste en la pila y formar con ellos un componente fuerte. Retirar v del camino. Si el camino está vacío, ir al paso 1. Si no, hacer $v \leftarrow$ el último nudo del camino e ir al paso 3.

$num(\cdot)$ indica el número asignado a un nudo una vez reenumerado. El conjunto A_e contiene los arcos ya explorados; V_v es el conjunto de nudos ya visitados.

El algoritmo consiste esencialmente en una serie de pasos principales cada uno de los cuales tiene otros varios secundarios. Uno de esos pasos principales comienza colocando en la pila y en el camino un nudo de los todavía no estudiados en pasos principales previos. A continuación se llevan a cabo los pasos secundarios, cada uno de los cuales consiste en ampliar o reducir en un nudo el camino que se está trazando. El paso principal termina cuando la pila y el camino se agotan.

Un paso secundario comienza con la búsqueda de, entre los arcos que no han sido estudiados, aquellos que salen del nudo v de final de camino. Si uno de esos arcos lleva a un nudo w cuyo vínculo/puntero indica un nudo más abajo en la pila que el del propio v , el de éste se hace igual al de w . Esta estrategia continúa hasta que:

1. Se encuentre un arco que llegue a un nudo que no está en la pila; en este caso se añade ese nudo a la pila y se aumenta el camino añadiendo ese nudo al mismo.
2. La lista de nudos que salen del de final del camino se vacíe; en este caso puede ocurrir que:
 - (a) El vínculo de ese último nudo apunte al propio nudo v del final del camino. En este caso al nudo v se le designa como raíz de un bloque formado por él mismo y por todos los que están por encima en la pila. Este bloque se elimina en su totalidad de la pila, y de cualquier ulterior consideración, numerando sus nudos a continuación. El paso secundario se completa volviendo al nudo anterior al nudo v en el camino, a menos que el camino y la pila ya estén vacíos.
 - (b) El vínculo indique un nudo más abajo en la pila que el nudo v del final del camino. En este caso se completa el paso volviendo al nudo w anterior al v en el camino. El vínculo de w se hace igual al de v si el de w indica uno más abajo en la pila que el de v .

Si se han reenumerado todos los nudos del camino se comienza un nuevo paso principal.

La implementación de este algoritmo en ordenador es muy sencilla. En las referencias bibliográficas se pueden encontrar breves y eficaces programas en FORTRAN 77 (alguno con menos de 70 instrucciones) que lo llevan a efecto.

3.5.5 Pivotación en matrices dispersas y eliminación de Gauss

Si el sistema de ecuaciones lineales que se quiere resolver no presenta ninguna estructura en la matriz de coeficientes digna de ser tenida en cuenta desde el punto de vista de su dispersidad, y ni siquiera interesa triangularizarla por bloques, la forma de abordar su resolución consiste en utilizar directamente la eliminación de Gauss; eso sí, teniendo en cuenta que esa matriz es dispersa y que por tanto conviene proceder con cierta cautela para beneficiarse en lo posible de esta circunstancia.

La forma más universalmente aceptada de llevar esto a cabo consiste en hacerlo siguiendo el *criterio de Markowitz* [1957]. La idea de este autor consiste en factorizar la matriz mediante eliminación de Gauss con pivotación, escogiendo como elemento pivote en una etapa k aquel a_{ij}^k de A que sea numéricamente aceptable y que minimice el producto

$$(r_i^k - 1)(c_j^k - 1),$$

donde r_i^k es el número de elementos distintos de cero en la fila i de la submatriz activa, A^k , y c_j^k el número de elementos distintos de cero en la columna j de esa submatriz activa. Obsérvese que es más conveniente usar $(r_i^k - 1)(c_j^k - 1)$ que $r_i^k c_j^k$ pues de esta manera se fuerza a que se elija un coeficiente a_{ij}^k tal que en su fila o columna sólo exista él como elemento distinto de cero. El criterio numérico de aceptabilidad requiere que

$$|a_{ij}^k| \geq u \max_{l \geq k} |a_{il}^k|$$

o que

$$|a_{ij}^k| \geq u \max_{l \geq k} |a_{lj}^k|,$$

donde u es un parámetro, $0 < u \leq 1$, que se fija previamente.

Para comprender más fácilmente el alcance del criterio de Markowitz, supongamos que estamos en la etapa k y que, como siempre en estos casos, la submatriz activa, A^k , es la que determinan las columnas k a n y las filas k a n de A . En la figura 3.38 se ilustra la situación para el caso en que $n = 7$ y $k = 3$. Para facilitar la notación, sin pérdida de generalidad, suponemos que los elementos de la diagonal principal se van haciendo 1. Los vectores $\mathbf{r}$ y $\mathbf{c}$ son de orden $n - k$. Si consideramos la matriz cuadrada de rango 1 y orden $n - k$, $\mathbf{c}\mathbf{r}^T$, la etapa k de la eliminación de Gauss consiste en restar la matriz $\mathbf{c}\mathbf{r}^T$ de la que determinan las filas y columnas $k + 1$ a n de A . El vector $\mathbf{c}$ se convierte en la subcolumna k de L y $[1, \mathbf{r}^T]$ en la subfila k de U . El criterio de Markowitz consiste en elegir un a_{ij}^k y llevarlo a la posición (k, k) , por medio de los intercambios de filas y columnas oportunos, de tal forma que el producto del número de elementos del vector $\mathbf{c}$ menos 1, $c_j^k - 1$, por el del vector $\mathbf{r}$ menos 1, $r_i^k - 1$, sea mínimo.

El criterio (heurístico) de Markowitz, combinado con algún criterio como el sugerido que garantice la estabilidad numérica del proceso de factorización de la matriz, produce excelentes resultados: muchas veces mejor que otros más sofisticados.

El objetivo que persigue el criterio de Markowitz consiste en encontrar aquel elemento pivote que modifique el menor número posible de coeficientes en la submatriz que resta por factorizar. También se puede ver como una forma de satisfacer el criterio de minimizar el número de multiplicaciones, $r_i^k(c_j^k - 1)$, a efectuar en la etapa k de la factorización y como una forma de producir el menor número de elementos de relleno en la etapa k : en el peor de los casos este número de rellenos será precisamente $(r_i^k - 1)(c_j^k - 1)$.

Para llevar a la práctica la eliminación de Gauss con el criterio de Markowitz hay que tener cierto cuidado pues, por ejemplo, si en una etapa k la submatriz activa es de orden 10.000 y

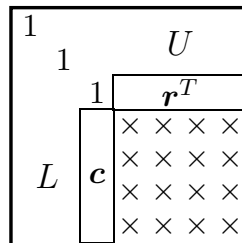


Figura 3.38

Etapla $k = 3$ de la eliminación de Gauss de una matriz de orden 7

existen en ella 40.000 elementos distintos de cero, si el pivote elegido cumple que $r_i = c_j = 3$, se habrán efectuado 40.000 comprobaciones para llevar a cabo una etapa que sólo comporta 10 operaciones aritméticas. Para evitar esto se suele utilizar dos vectores, **nr** y **nc**, inicializados con el número de elementos no nulos no en la diagonal principal en cada fila y en cada columna; sus valores se adaptan según evoluciona la factorización.

En la literatura especializada en matrices dispersas, y en las referencias bibliográficas del final de este capítulo, se pueden encontrar diversas formas y estrategias para llevar a efecto la eliminación de Gauss con el criterio de Markowitz. Los paquetes de rutinas matemáticas de Harwell (MA28), SPARSPAK, YSMP y SMMS (FACTORNS), resuelven sistemas lineales de matrices dispersas de estructura no simétrica de la forma explicada en este apartado.

3.5.6 Método de los frentes

Los esquemas de frentes para resolver sistemas de ecuaciones lineales con matriz dispersa tienen su origen en la solución de problemas de elementos finitos para análisis de estructuras mecánicas. En estos problemas, las matrices con las que se opera son simétricas y definidas positivas. En los últimos años se han adaptado de forma generalizada para tratar problemas muy diversos de grandísimas dimensiones debido a la poca memoria de ordenador que necesitan. La rutina MA32 del AERE Harwell es quizás la más conocida y usada de las que implementan esta técnica.

Para describir el procedimiento que se sigue y las ideas que subyacen en ellos nos referiremos a un problema de elementos finitos. En la figura 3.39 se presenta un ejemplo. En cada triángulo se consideran 7 variables: tensiones en los vértices, en los puntos medios de los lados y en el centroide. La matriz que resulta del problema se va configurando en el mismo orden que define la numeración de la figura 3.39. En este sentido, la matriz A será

$$A = \sum_{\ell} A^{[\ell]},$$

donde cada $A^{[\ell]}$ tiene sólo elementos distintos de cero en las submatrices donde están presentes

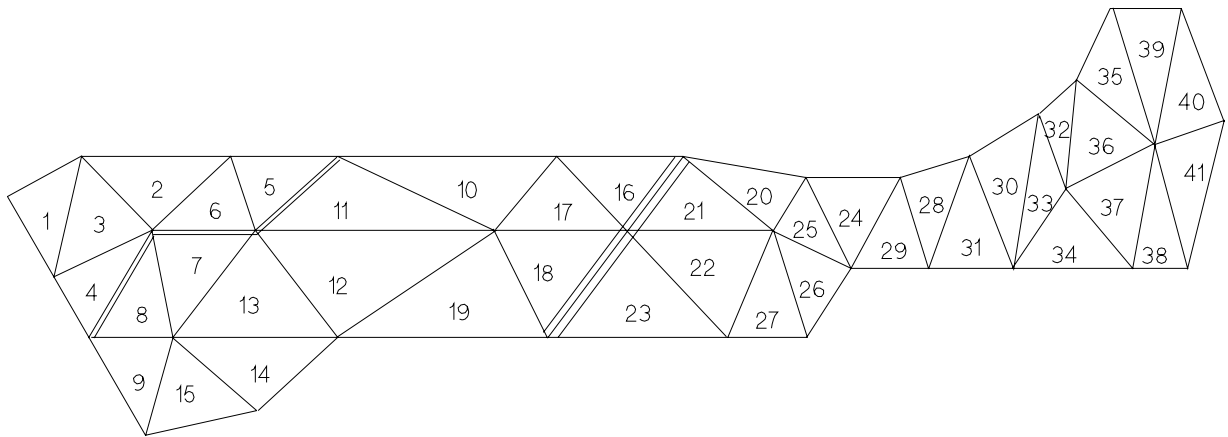


Figura 3.39

Pieza mecánica mallada para su análisis por elementos finitos

las variables que tienen que ver con el elemento ℓ . El proceso que tiene lugar al efectuar la suma se denomina *ensamblaje* de los elementos. Después de ensamblado el elemento número 6 de la figura 3.39, la matriz A tiene la forma de la figura 3.40. Lo más importante que resaltar de ésta son los dos bloques cero situados en la parte superior derecha e inferior izquierda de la matriz. Su situación quiere decir que si se eligen los elementos pivote para la eliminación de Gauss de entre los del bloque superior izquierdo, la parte de la matriz no ensamblada todavía (parte reticulada) no se verá afectada por esa eliminación; sí se modificarán, por supuesto, la parte superior izquierda, las zonas sombreadas y el denominado *frente*, de acuerdo con las fórmulas de la eliminación de Gauss:

$$a_{ij} \leftarrow a_{ij} - a_{ik}(a_{kk})^{-1}a_{kj}, \quad (3.2)$$

donde el elemento pivote es el que se designa por a_{kk} .

Una vez efectuadas estas operaciones, las variables involucradas en las zonas sombreadas y el bloque superior izquierdo estarán ya totalmente ensambladas; las involucradas en el frente se verán afectadas por la incorporación de los elementos 7 a 13. La contribución de esos elementos será de la forma

$$a_{ij} \leftarrow a_{ij} + E_{ij}^{[\ell]}, \quad (3.3)$$

donde $E_{ij}^{[\ell]}$ indica el valor del componente ij del elemento ℓ , por lo que da igual el orden en que se efectúen las operaciones (3.2) y (3.3) siempre que los valores de a_{ik} , a_{kk} y a_{kj} se hayan sumado en su totalidad y tengan un valor definitivo antes de efectuar (3.2).

Si no es necesario realizar pivotación (por ejemplo cuando la matriz ensamblada es definida positiva), el bloque superior izquierdo se puede eliminar de la memoria principal del ordenador en el que se lleva a cabo el proceso y guardarse donde se considere oportuno para una posterior manipulación. Como la parte reticulada de la figura 3.40 todavía no se ha ensamblado, una vez hecho esto, sólo sería necesario guardar momentáneamente las variables involucradas en el frente. En el caso de la figura 3.39, las variables en el frente, una vez ensamblados los seis primeros elementos, serán la siete variables en los triángulos cuyos lados se indican con doble raya.

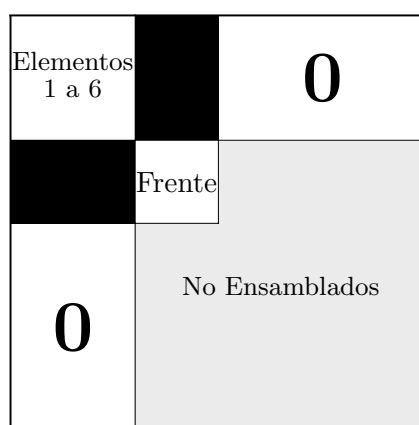


Figura 3.40

Matriz A después de ensamblados los primeros seis elementos de la figura 3.39

El tamaño del frente juega un papel primordial en el procedimiento y varía según progresa el ensamblaje de los distintos elementos. En el ejemplo, después de ensamblado el elemento 19 y efectuada la subsiguiente eliminación de Gauss, el frente estará formado por las variables de los triángulos cuyos cuatro lados se indican con triple raya. Para un orden dado de los elementos existe un tamaño máximo de frente; en el caso de la figura 3.39, después de ensamblado el elemento número 10, el frente estará formado por once variables. Es evidente que una ordenación adecuada puede reducir de forma apreciable el tamaño máximo de los frentes.

El método de los frentes evita, si se guarda la matriz que define el frente como una matriz totalmente llena, los problemas inherentes a utilizar matrices dispersas: operar por filas o columnas (no con ambas a la vez) según el esquema de almacenamiento elegido, manejos continuos de subíndices, etc.

El método de los frentes se puede aplicar también a sistemas cuya matriz de coeficientes no es definida positiva, requiriendo en este caso pivotaciones. Para escoger el pivote se puede recurrir a cualquiera de los elementos situados en el bloque superior izquierdo de la matriz ya ensamblada, requiriéndosele además que cumpla, por ejemplo, que

$$|a_{lk}| \geq u \max_i |a_{ik}|, \quad (3.4)$$

donde, $0 < u < 1$, es un umbral adecuado al problema. Obsérvese que si se efectúan intercambios no simétricos, la lista de índices de filas en el frente diferirá de la de columnas y se requerirá memoria adicional. En el caso de que la matriz sea simétrica se puede seguir conservando esa simetría eligiendo como pivote un elemento diagonal que cumpla (3.4) o realizar pivotaciones en bloques 2×2 como las estudiadas al analizar el método de Bunch y Kaufman en el apartado 1.5.4.3.1 de la página 60; es decir, del tipo

$$E = \begin{bmatrix} a_{ii} & a_{ij} \\ a_{ji} & a_{jj} \end{bmatrix},$$

donde todos los elementos de E deben pertenecer al bloque superior izquierdo ya ensamblado y cumplirse que

$$\|E^{-1}\|_1^{-1} \geq u \max \left(\max_{l \neq i, j} |a_{li}|, \max_{l \neq i, j} |a_{lj}| \right).$$

Si el pivote, o bloque pivote, no puede elegirse de entre los elementos del bloque superior izquierdo, se pueden realizar uno o más nuevos ensamblajes. La única penalización que esto puede traer consigo es el aumento del tamaño del frente.

Problemas generales

La técnica de los frentes no es exclusiva de las tecnologías aplicables a elementos finitos: se puede generalizar. Para ello, en lugar de ensamblar bloques, se hace fila a fila como si se tratase de bloques o matrices no simétricas. Esta forma de proceder se ilustra en la figura 3.41, donde se presenta la matriz una vez ensamblada la tercera ecuación de una discretización en cinco puntos del operador de Laplace en una malla 2×4 . Hecho este último ensamblaje, en este caso, ninguna otra ecuación producirá nuevos elementos en la columna 1 por lo que una vez factorizada se puede suponer que los cálculos con ella se han completado y, si así se desea, eliminarla.

La versión de la figura 3.40 para el caso en que se traten problemas que no son de elementos finitos es la 3.42. La generalización del método de los frentes capaz de tener en cuenta esquemas

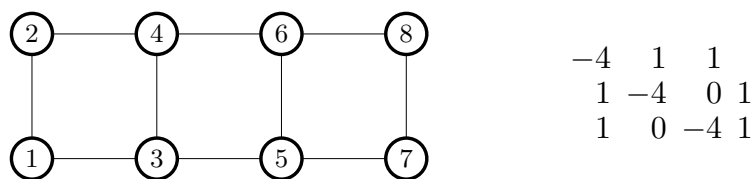


Figura 3.41

Malla 2×4 y primeras tres filas de la matriz a que da lugar el método de los frentes

de ordenación (basados en la estructura de la matriz) como los de grado mínimo o disección anidada se denomina *método multifrentes*. Una implementación comercial muy aceptada para el tratamiento de sistemas de muy grandes dimensiones con matriz simétrica o casi simétrica la constituye la rutina MA37 del paquete de *software* de matrices dispersas del AERE Harwell.

3.6 Problemas de mínimos cuadrados dispersos

Con este nombre se designan aquellos problemas de mínimos cuadrados lineales,

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \|\mathbf{Ax} - \mathbf{b}\|_2, \quad (3.5)$$

en los que la matriz A es de grandes dimensiones y dispersa. Los métodos que vamos a estudiar en este apartado son especializaciones de los vistos en el capítulo 1 para cuando la matriz A presenta unas características de dispersidad tales que hacen aconsejable su explotación por procedimientos *ad hoc*.

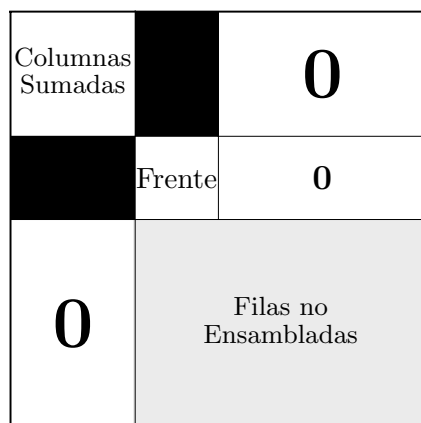


Figura 3.42

Matriz A de un problema no de elementos finitos en el proceso de tratamiento por el método de los frentes

3.6.1 El método de las ecuaciones normales

Como se recordará, si $S = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{Ax} - \mathbf{b}\|_2 = \min\}$,

$$\mathbf{x} \in S \Leftrightarrow A^T(\mathbf{Ax} - \mathbf{b}) = \mathbf{0}.$$

El segundo miembro de esta última expresión define las *ecuaciones normales*. Como es sabido, para resolver (3.5) se pueden utilizar las ecuaciones normales. Éstas no son sino un sistema lineal de ecuaciones en el que si A es de rango completo,⁷ cosa que supondremos en lo que sigue, la matriz $A^T A$ es simétrica y definida positiva.

En el caso que nos ocupa, cuando A es dispersa, si se quiere utilizar las ecuaciones normales, hay que tener en cuenta que, de la misma forma que al factorizarla, al formar la matriz $A^T A$ se pueden crear elementos de relleno.

Si $\mathbf{a}_i$ designa el vector fila i -ésimo de la matriz $A \in \mathbb{R}^{m \times n}$, entonces,

$$A^T A = \sum_{i=1}^m \mathbf{a}_i \mathbf{a}_i^T. \quad (3.6)$$

Esto expresa la matriz $A^T A$ como suma de m matrices de rango 1. Si suponemos que en (3.6) no se producen *errores numéricos de cancelación*, esto es, al sumar o restar dos cantidades distintas de cero el resultado es distinto de cero, la estructura de dispersidad de $A^T A$ es la suma de las estructuras de $\mathbf{a}_i \mathbf{a}_i^T$, $i = 1, 2, \dots, m$.

Teorema 3.4 *Supóngase que no se producen errores numéricos de cancelación en el cálculo de $A^T A$. Entonces,*

$$(A^T A)_{jk} \neq 0 \Leftrightarrow a_{ij} \neq 0 \quad y \quad a_{ik} \neq 0$$

para al menos una fila $i = 1, 2, \dots, m$.

Este teorema permite determinar muy fácilmente la posición de los elementos distintos de cero de $A^T A$, a partir de los de A , sin necesidad de calcularlos numéricamente. Si el supuesto de no cancelación numérica no fuese cierto, el número de elementos que se estimase para $A^T A$ podría ser mucho mayor que el real. Por ejemplo, si A es ortogonal, $A^T A = I$, por lo que $A^T A$ es dispersa aun cuando A fuese muy densa o totalmente llena.

Del teorema 3.4 se desprende que si A tiene una sola fila completamente ocupada, aunque el resto de las filas fuesen dispersas, $A^T A$ será totalmente densa. Por ejemplo, si

$$A = \begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & & & \\ & & \times & & \\ & & & \times & \\ & & & & \times \end{bmatrix}, \quad (3.7)$$

$A^T A$ será totalmente densa.

⁷Si A no es de rango completo, $A^T A$ es simétrica pero semidefinida positiva

También se presentarán dificultades cuando siendo A dispersa o muy dispersa, $A^T A$ está prácticamente llena de elementos distintos de cero. Tal es el caso, por ejemplo, cuando cada elemento a_{ij} es una variable aleatoria independiente y $\text{Prob}\{a_{ij} \neq 0\} = p \ll 1$. En este caso,

$$\text{Prob}\{a_{ij}a_{ik} = 0, j \neq k\} = 1 - p^2.$$

Como

$$(A^T A)_{jk} = \sum_{i=1}^m a_{ij}a_{ik},$$

entonces

$$\text{Prob}\left\{(A^T A)_{jk} \neq 0\right\} = 1 - (1 - p^2)^m \approx e^{-mp^2}.$$

Si el valor esperado del número de elementos distintos de cero en una columna cualquiera es $m^{1/2}$, entonces $p = m^{-1/2}$ y $mp^2 = 1$, por lo que $A^T A$ estará prácticamente llena de elementos distintos de cero. Estos problemas suelen presentarse habitualmente en reconstrucción de imágenes, prospecciones petrolíferas, etc. Se resuelven mediante procesos iterativos como los estudiados en el capítulo 2.

Si P y Q son matrices de permutación $m \times m$ y $n \times n$ (de filas y columnas), se tiene que

$$(PAQ)^T(PAQ) = Q^T A^T P^T PAQ = Q^T A^T AQ,$$

es decir, una reordenación de las filas de A no afecta a la matriz $A^T A$ (este resultado se deduce inmediatamente de (3.6)). La reordenación de las columnas de A , por el contrario, equivale a una permutación simétrica de $A^T A$.

Partiendo de estas consideraciones, un algoritmo que utilizase las ecuaciones normales para resolver el problema de mínimos cuadrados dispersos, basado en el de la tabla 3.1 de la página 221, sería el de la tabla 3.11. El algoritmo a utilizar para determinar la permutación Q que requiere el paso 2 puede ser el de grado mínimo. Si este es el caso, usando las ideas apuntadas en el apartado 3.4.3 y los grafos de eliminación correspondientes, los pasos 2 y 2' se podrían refundir en uno.

Aparte de las dificultades numéricas apuntadas en el capítulo 1 al exponer los problemas generales de mínimos cuadrados, y que hay que tener en cuenta en cualquier caso, si la matriz del problema está relativamente bien condicionada, para resolver un problema de mínimos cuadrados en el que esa matriz es dispersa siguiendo el esquema de la tabla 3.11, se puede usar cualquiera de los paquetes de *software* de matrices dispersas que existen en el mercado. En este sentido volvemos a citar la librería de rutinas matemáticas de AERE de Harwell y los paquetes SPARSPAK (George y Liu [1981]), YSMP (Eisentat, Schultz y Sherman [1981]) SMMS (Alvarado [1990]), NAG [1992 y 1993] y Matlab.

3.6.1.1 Dispersidad parcial

Si la matriz A es dispersa en casi su totalidad salvo algunas pocas filas llenas, consideremos el problema

$$\underset{\mathbf{x}}{\text{minimizar}} \left\| \begin{bmatrix} A_s \\ A_d \end{bmatrix} \mathbf{x} - \begin{bmatrix} \mathbf{b}_s \\ \mathbf{b}_d \end{bmatrix} \right\|_2, \quad (3.8)$$

Tabla 3.11

Algoritmo para resolver mínimos cuadrados con matrices dispersas mediante las ecuaciones normales

- Paso 1** – Determinar la estructura simbólica de $A^T A$.
- Paso 2** – Determinar una permutación de columnas Q tal que $Q^T A^T A Q$ tenga una estructura dispersa ventajosa en relación con el tipo de sistema a resolver: es decir, que su factor de Cholesky, G , sea disperso.
- Paso 2'** – Factorizar simbólicamente por Cholesky la matriz $Q^T A^T A Q$ y generar las estructuras de datos y memoria necesarias para G .
- Paso 3** – Calcular numéricamente $B = Q^T A^T A Q$ y $\mathbf{c} = Q^T A^T \mathbf{b}$; almacenar B en la estructura de datos correspondiente a G .
- Paso 4** – Calcular numéricamente la factorización de Cholesky, $G^T G$, de B . Resolver $G^T \mathbf{z} = \mathbf{c}$, $G\mathbf{y} = \mathbf{z}$ y, por fin, $\mathbf{x} = Q\mathbf{y}$.

donde $A_s \in \mathbb{R}^{m_1 \times n}$ es la parte dispersa y $A_d \in \mathbb{R}^{m_2 \times n}$, $m_2 \ll n$, la parte densa o llena. Supondremos que $\text{rango}(A_s) = n$. Sea $\mathbf{x}_s$ la solución del problema disperso

$$\underset{\mathbf{x}}{\text{minimizar}} \|A_s \mathbf{x} - \mathbf{b}_s\|_2$$

y G_s el factor de Cholesky de $A_s^T A_s$. Los vectores de residuos de (3.8) correspondientes a $\mathbf{x}_s$ son $\mathbf{r}_s(\mathbf{x}_s) = \mathbf{b}_s - A_s \mathbf{x}_s$ y $\mathbf{r}_d(\mathbf{x}_s) = \mathbf{b}_d - A_d \mathbf{x}_s$. La solución del problema completo, $\mathbf{x} = \mathbf{x}_s + \mathbf{z}$, será aquella que minimice

$$\|\mathbf{r}_s(\mathbf{x})\|_2^2 + \|\mathbf{r}_d(\mathbf{x})\|_2^2, \quad (3.9)$$

donde $\mathbf{r}_s(\mathbf{x}) = \mathbf{r}_s(\mathbf{x}_s) - A_s \mathbf{z}$ y $\mathbf{r}_d(\mathbf{x}) = \mathbf{r}_d(\mathbf{x}_s) - A_d \mathbf{z}$. Como se cumple que $A_s^T \mathbf{r}_s(\mathbf{x}_s) = \mathbf{0}$ y $\mathbf{r}_s(\mathbf{x}_s)$ es constante, (3.9) es equivalente a

$$\underset{\mathbf{z}}{\text{minimizar}} \left\{ \|A_s \mathbf{z}\|_2^2 + \|A_d \mathbf{z} - \mathbf{r}_d(\mathbf{x}_s)\|_2^2 \right\}. \quad (3.10)$$

Haciendo $\mathbf{u} = G_s \mathbf{z}$ y $B_d = A_d G_s^{-1}$ se tiene que $\|A_s \mathbf{z}\|_2 = \|A_s G_s^{-1} \mathbf{u}\|_2 = \|Q^T G_s G_s^{-1} \mathbf{u}\|_2 = \|\mathbf{u}\|_2$ por lo que (3.10) se reduce a

$$\underset{\mathbf{u}}{\text{minimizar}} \left\{ \|\mathbf{u}\|_2^2 + \|B_d \mathbf{u} - \mathbf{r}_d(\mathbf{x}_s)\|_2^2 \right\}.$$

Si se hace $\mathbf{v} = \mathbf{r}_d(\mathbf{x}_s) - B_d \mathbf{u}$, $C = [B_d, I_{m_2}]$ y $\mathbf{w} = [\mathbf{u}^T, \mathbf{v}^T]^T$, la expresión anterior se puede escribir de la siguiente manera:

$$\begin{aligned} &\text{minimizar } \|\mathbf{w}\|_2 \\ &\text{s. a } C\mathbf{w} = \mathbf{r}_d(\mathbf{x}_s). \end{aligned} \quad (3.11)$$

Como C tiene rango completo, su pseudoinversa es $C^T (CC^T)^{-1}$. La solución de (3.11) es pues

$$\mathbf{w} = C^T (CC^T)^{-1} \mathbf{r}_d(\mathbf{x}_s) = C^T (G_d^T G_d)^{-1} \mathbf{r}_d(\mathbf{x}_s),$$

donde G_d es el factor de Cholesky de CC^T . Una vez calculado el vector $\mathbf{w}$ y de él $\mathbf{u}$, $\mathbf{z}$ se obtiene resolviendo $G_s \mathbf{z} = \mathbf{u}$. Con $\mathbf{z}$ se llega luego a la solución $\mathbf{x} = \mathbf{x}_s + \mathbf{z}$.

3.6.2 Métodos basados en transformaciones ortogonales. Método de George-Heath

Como se recordará del capítulo 1, apartado 1.7.2.2, página 88, los métodos basados en transformaciones ortogonales evitan los problemas que surgen del posible mal condicionamiento de la matriz $A^T A$ de las ecuaciones normales. Estas transformaciones utilizan una matriz ortogonal $Q \in \mathbb{R}^{m \times m}$ para reducir $A \in \mathbb{R}^{m \times n}$ (que supondremos de rango n) y $\mathbf{b} \in \mathbb{R}^m$ de la forma

$$QA = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \quad \text{y} \quad Q\mathbf{b} = \begin{bmatrix} \mathbf{c} \\ \mathbf{d} \end{bmatrix},$$

donde $R_1 \in \mathbb{R}^{n \times n}$ es una matriz triangular superior y $\mathbf{c} \in \mathbb{R}^n$. La solución del problema de mínimos cuadrados se obtiene resolviendo el sistema $R_1 \mathbf{x} = \mathbf{c}$; la suma de residuos al cuadrado es $\|\mathbf{d}\|_2^2$.

De acuerdo con el teorema 1.16 de la página 89, la matriz R_1 es la misma que la que resulta de la factorización de Cholesky, $G^T G$, de $A^T A$. Como esta factorización es única, su estructura de elementos cero/distintos de cero es también única. Siguiendo la estrategia de actuación con matrices dispersas que venimos propiciando a lo largo de todo este capítulo, esto es, prever en cada procedimiento qué elementos distintos de cero se van a crear al manipular una matriz, en este caso habría que conocer, antes de calcularlos numéricamente, los de la matriz R_1 . Una forma de hacerlo —de hecho la más extendida— es usar los pasos 2 y 2' del algoritmo de la tabla 3.11 para determinar una buena reordenación de columnas, Q' ,⁸ de tal forma que el factor R_1 de AQ' sea lo más disperso posible. Esta forma de actuar, sin embargo, no está exenta de peculiaridades poco satisfactorias como es, por ejemplo, que puede resultar excesivamente generosa al reservar espacios para futuros elementos de relleno. En efecto, si A tiene la forma que veíamos en (3.7), el factor R_1 será igual a A ; ahora bien, $A^T A$ es totalmente llena por lo que la forma de actuar apuntada reservaría muchas más posiciones de memoria para elementos de relleno que las necesarias.

Otra forma de actuar consiste en llevar a cabo simbólicamente los algoritmos basados en transformaciones de Givens o de Householder que estudiábamos en el capítulo 1. En este sentido, George y Ng [1985] demuestran el siguiente resultado.

Teorema 3.5 *La estructura de R_1 que predice la factorización simbólica de $A^T A$ por Cholesky incluye la de R_1 que predice el método simbólico basado en transformaciones de Givens.*

También Manneback [1985] demuestra lo mismo para el caso en que se apliquen transformaciones de Householder.

El algoritmo que proponen George y Heath [1980], al que denominan *ortogonalización secuencial de filas*, procesa las filas de A secuencialmente. Si R_{i-1} designa la matriz triangular superior que se obtiene después de procesar las filas $\mathbf{a}_1^T, \dots, \mathbf{a}_{i-1}^T$, al procesar la fila $\mathbf{a}_i^T = [a_{i1}, a_{i2}, \dots, a_{in}]$ se buscan de izquierda a derecha los elementos distintos de cero; para cada $a_{ij} \neq 0$, se define una rotación o transformación de Givens simbólica que involucre a la fila j de R_{i-1} y anule a_{ij} . Procediendo así se pueden crear nuevos elementos distintos de cero tanto en R_{i-1} como en la fila $\mathbf{a}_i^T$. El proceso continúa hasta conseguir R_i .

Si en el transcurso del tratamiento de la fila i , al llegar al elemento j , éste, r_{jj} , querrá decir que la fila j en R_{i-1} todavía no se ha visto afectada por ninguna rotación y la totalidad de la

⁸No confundir esta matriz de permutación de columnas con la matriz ortogonal Q .

fila j debe ser cero. Cuando esto ocurre se intercambian la fila j con la i . En la figura 3.43 se ilustra este proceso al actuar sobre los elementos de la fila 9 de una matriz 9×8 . Obsérvese que los tres últimos elementos de esta fila 9, una vez anulados del 1 al 5, se intercambiarían con la fila 6.

Una vez efectuada la factorización simbólica y determinada por tanto la estructura de elementos distintos de cero de R_1 , se procede con la factorización numérica.

3.6.2.1 Ordenación de filas

A pesar de que la matriz R_1 que se obtiene es independiente del orden en que se tratan las filas de A , lo cierto es que el número de elementos de relleno en los pasos intermedios, y por tanto el número global de operaciones que necesita el algoritmo de George y Heath, depende mucho de ese orden. Por ejemplo, las matrices

$$A = \left[\begin{array}{cccccc} \times & \times & \times & \times & \times & \\ \times & & & & & \\ \vdots & & & & & \\ \times & & & & & \\ \times & & & & & \\ & \times & & & & \\ & & \times & & & \\ & & & \times & & \\ & & & & \times & \\ & & & & & \times \end{array} \right] \quad \left. \begin{array}{l} \left. \begin{array}{l} \vdots \\ \vdots \\ \vdots \end{array} \right\} m \\ \left. \begin{array}{l} \vdots \\ \vdots \\ \vdots \end{array} \right\} n \end{array} \right\} \quad \text{y} \quad PA = \left[\begin{array}{cccccc} \times & & & & & \\ \times & & & & & \\ \vdots & & & & & \\ \times & & & & & \\ \times & \times & \times & \times & \times & \times \\ & \times & & & & \\ & & \times & & & \\ & & & \times & & \\ & & & & \times & \\ & & & & & \times \end{array} \right] \quad \left. \begin{array}{l} \left. \begin{array}{l} \vdots \\ \vdots \\ \vdots \end{array} \right\} m \\ \left. \begin{array}{l} \vdots \\ \vdots \\ \vdots \end{array} \right\} n \end{array} \right\}$$

requieren, respectivamente, $O(mn^2)$ y $O(n^2)$ operaciones para reducirlas.

Supuesto que los vectores fila de A no tienen normas muy distintas, el orden en que se disponen no afecta a la estabilidad numérica del procedimiento y puede escogerse el que defina un patrón de dispersidad mejor. En cualquier caso, teniendo en cuenta esto, es deseable obtener una ordenación de las filas de la matriz por transformar que reduzca al mínimo los rellenos intermedios. Una regla que se puede aplicar con este objetivo es la siguiente:

$$\begin{array}{cccccccc} \times & 0 & \times & 0 & 0 & \times & 0 & 0 \\ & \otimes & 0 & \oplus & \otimes & 0 & 0 & \oplus \\ & & \times & 0 & \times & 0 & 0 & \times \\ & & & \otimes & \otimes & 0 & 0 & \oplus \\ & & & & \otimes & \otimes & 0 & \oplus \\ & & & & & \oplus & 0 & \oplus \\ & & & & & & \times & 0 \\ & & & & & & & \times \\ 0 & \times & 0 & \times & \oplus & \oplus & 0 & \times \end{array}$$

Figura 3.43

Procesamiento simbólico de la fila 9 de una matriz $A \in \mathbb{R}^{9 \times 8}$ por el algoritmo de George y Heath. Los símbolos $\otimes$ designan los elementos de R_8 involucrados en la eliminación de $\mathbf{a}_9^T$; $\oplus$ los que se crean en esa eliminación

Si el índice de columna del último elemento distinto de cero de la fila $\mathbf{a}_i^T$ es ℓ_i y el primero f_i , ordenar primero las filas de tal forma que los índices f_i , $i = 1, 2, \dots$, cumplan que $f_i \leq f_k$ si $i < k$ y luego, para cada grupo de filas tales que $f_i = k$, $k = 1, \dots, \max_i f_i$, ordenar las filas según ℓ_i crecientes.

Si se aplica la regla anterior a la matriz A representada más arriba, se obtendría la reordenación que se esquematizaba como PA . Esta regla no produce una única reordenación. Para resolver las situaciones donde se presentan varias posibilidades por existir empate se puede considerar el *coste* de rotar simbólicamente el vector fila $\mathbf{a}_i^T$ en todas las filas donde hay un elemento no nulo en la columna ℓ_i . Este coste sería el número de elementos distintos de cero que se crearían. Las columnas se ordenarían en orden creciente de ese coste. De acuerdo con este criterio, las filas $1, \dots, f_i - 1$ de R_i no se verían afectadas cuando se procesen las filas restantes.

Otra forma de ordenar las filas que suele dar también buenos resultados consiste en simplemente hacerlo de acuerdo con los valores crecientes de ℓ_i . Con esta regla, al procesar la fila $\mathbf{a}_i^T$ sólo se verían afectados las columnas f_i a ℓ_i de R_{i-1} , pues las filas anteriores sólo tendrían elementos distintos de cero hasta como mucho la columna ℓ_i . La matriz R_{i-1} tendría ceros en las columnas $\ell_{i+1}, \dots, n$ y no se producirían rellenos en esas columnas de la fila $\mathbf{a}_i^T$.

El procedimiento de George y Heath en la práctica resulta más ventajoso si la determinación de la estructura de R_1 se hace como en el caso de las ecuaciones normales: esto es, mediante los pasos 2 y 2' de la tabla 3.11. El algoritmo completo de George y Heath se describe en la tabla 3.12.

3.6.3 Otros métodos

Un método más para resolver problemas de mínimos cuadrados dispersos, relacionado con los vistos hasta ahora para sistemas cuadrados generales, consiste en reordenar la matriz A

Tabla 3.12
Algoritmo de ortogonalización dispersa de George y Heath

- Paso 1** – Determinar la estructura simbólica de $A^T A$.
- Paso 2** – Determinar una permutación de columnas Q tal que $Q^T A^T A Q$ tenga una estructura dispersa ventajosa en relación con el tipo de sistema a resolver: que su factor de Cholesky, G , sea disperso.
- Paso 2'** – Factorizar simbólicamente por Cholesky, $G^T G$, la matriz $Q^T A^T A Q$ y generar las estructuras de datos y memoria necesarias para G .
- Paso 3** – Determinar una permutación de filas P tal que las filas de PAQ tengan unos ℓ_i crecientes.
- Paso 4** – Calcular numéricamente R_1 y $\mathbf{c}$ procesando las filas de $[PAQ, P\mathbf{b}]$ mediante transformaciones de Givens.
- Paso 5** – Resolver $R\mathbf{y} = \mathbf{c}$. Hacer $\mathbf{x} = Q\mathbf{y}$.

mediante permutaciones P y Q de tal forma que se obtenga

$$PAQ = \begin{bmatrix} M_1 & U_{12} & \cdots & U_{1k} & U_{1k+1} \\ & M_2 & \cdots & U_{2k} & U_{2k+1} \\ & & \ddots & \vdots & \vdots \\ & & & M_k & U_{kk+1} \\ & & & & M_{k+1} \end{bmatrix}, \quad (3.12)$$

donde los bloques M_i , $i = 1, 2, \dots, k$, son cuadrados.

Definición 3.6 Sea $A \in \mathbb{R}^{m \times n}$, $m \geq n$. Si para todos los subconjuntos formados por k columnas de A , $k = 1, 2, \dots, n$, las correspondientes submatrices tienen elementos distintos de cero en al menos $k + 1$ filas, se dice que la matriz A posee la *propiedad fuerte de Hall*.

De las dos matrices siguientes,

$$A = \begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & & & \\ & & \times & & \\ & & & \times & \\ & & & & \times \end{bmatrix} \quad \text{y} \quad A' = \begin{bmatrix} \times & \times & \times & \times \\ \times & & & \\ & \times & & \\ & & \times & \\ & & & \times \end{bmatrix},$$

A' posee la propiedad fuerte de Hall y A no.

Coleman, Edenbrand y Gilbert [1986] prueban que si la matriz A de

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \|A\mathbf{x} - \mathbf{b}\|_2 \quad (3.13)$$

tiene la propiedad fuerte de Hall, se puede reordenar según (3.12). En este caso, los bloques M_i , $i = 1, 2, \dots, k + 1$, también tienen esa propiedad. La reordenación que conduce a la forma triangular en bloques de (3.12) se puede lograr mediante una sencilla variante del algoritmo de Tarjan estudiado en el apartado 3.5.4.2. El sistema reordenado correspondiente conduce a la siguiente formulación del problema 3.13:

$$\underset{\tilde{\mathbf{x}}_{k+1}}{\text{minimizar}} \|M_{k+1}\tilde{\mathbf{x}}_{k+1} - \tilde{\mathbf{b}}_{k+1}\|_2,$$

donde $\tilde{\mathbf{x}} = Q^T \mathbf{x}$ y $\tilde{\mathbf{b}} = P\mathbf{b}$ se dividen de la misma forma que PAQ en (3.12). Si $\text{rango}(A) = n$, los bloques M_i , $i = 1, 2, \dots, k$, son regulares y $\tilde{\mathbf{x}}_k, \dots, \tilde{\mathbf{x}}_1$ se pueden obtener mediante la siguiente sustitución inversa en bloques:

$$M_i \tilde{\mathbf{x}}_i = \tilde{\mathbf{b}}_i - \sum_{j=i+1}^{k+1} U_{ij} \tilde{\mathbf{x}}_j, \quad i = k, \dots, 2, 1.$$

Referencias

Existen pocas referencias bibliográficas recientes sobre matrices dispersas y los últimos adelantos de sus métodos y procedimientos (que son muchos).

Todo lo expuesto relativo a formas de almacenar en ordenador matrices dispersas trata de reflejar las últimas tendencias y, por supuesto, las más usadas; en este sentido, se pueden consultar los siguientes libros: Duff, Erisman y Reid [1986]; George y Liu [1981], Lascaux y Théodor [1986]; Pissanetzky [1984]; Saad [1994]; Schendel [1989] y Tewarson [1973].

Para la elaboración del apartado sobre operaciones algebraicas elementales en matrices dispersas, se ha seguido a Duff, Erisman y Reid [1986] y Pissanetzky [1984]; los programas de ordenador son del autor y se basan en los que propone Pissanetzky [1984].

Muchos de los gráficos y figuras de matrices dispersas del texto se han confeccionado con el paquete de *software* SMMS de Alvarado [1990]. Otro paquete muy interesante similar, más moderno y completo, es el de Youcef Saad [1994], SPARSKIT. Las direcciones de INTERNET y FTP donde se puede encontrar toda la información relativa a este último son las siguientes:

<http://www.cs.umn.edu/Research/arpa/SPARSKIT/sparskit.html>

 $\bar{y}$

<ftp://ftp.cs.umn.edu/dept/sparse/>

Existe un banco de datos de matrices dispersas denominado Harwell/Boeing, Duff y otros [1989]. En éste, con un formato universal, se pueden encontrar multitud de matrices dispersas de características, tamaño y dificultad de tratamiento diversos.

Las nociones sobre grafos, tanto dirigidos como no dirigidos, están basadas en las propuestas por Pissanetzky [1984] pero adaptadas y completadas para ser coherentes con las que se exponen en las partes del texto relativas a programación lineal y entera.

La interpretación mediante grafos de la eliminación de Gauss en matrices simétricas dispersas sigue a Pissanetzky [1984]. El algoritmo de grado mínimo para reordenar ese tipo de matrices está basado en George y Liu [1981]; el de Cuthill-McKee en George y Liu [1981] y Lascaux y Théodor [1986]; el de Cuthill-McKee inverso utiliza la descripción de Duff, Erisman y Reid [1986]. Los métodos de disección aparecen muy bien descritos en Duff, Erisman y Reid [1986] y en Pissanetzky [1984]; en la exposición presentada se ha hecho un resumen de la de éste último.

El algoritmo de Hall se puede encontrar en varias de las referencias mencionadas; la exposición hecha en el texto se ha basado parcialmente en la de Pissanetzky [1984]. La descripción de los algoritmos de Sargent y Westerberg y Tarjan sigue a Duff, Erisman y Reid [1986] y a Duff y Reid [1978].

Lo relativo al método de los frentes se ha basado en Duff [1981].

Lo que hace referencia a mínimos cuadrados dispersos se puede encontrar muy bien tratado en Björk [1990] y [1996], George y Heath [1980] y George y Ng [1985].

Ejercicios

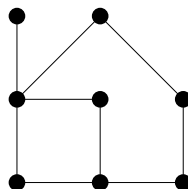
3.1. Escribir los vectores necesarios para almacenar la matriz

[illegible]

en un ordenador:

- a) Por coordenadas.
- b) Por filas.
- c) Por listas encadenadas.

3.2. Numerar el grafo



y escribir la matriz dispersa correspondiente.

3.3. Determinar el grafo asociado a la matriz A cuya estructura de elementos distintos de cero es la siguiente:

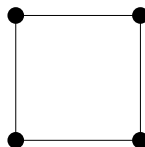
$$A = \begin{bmatrix} \times & \times & \times & \times & \times & \times & \times & \times & \times & \times \\ \times & \times & & & & & & & & \times \\ \times & & \times & & & & & & & \times \\ \times & & & \times & & & & & & \times \\ \times & & & & \times & & & & & \times \\ \times & & & & & \times & & & & \times \\ \times & & & & & & \times & & & \times \\ \times & & & & & & & \times & \times & \\ \times & \times & \times & \times & \times & \times & \times & \times & \times & \times \end{bmatrix}.$$

- 3.4. ¿Cuál es el ancho de banda de la matriz del ejercicio anterior? ¿Y su envolvente? ¿Cuántos elementos tiene la envolvente?
- 3.5. ¿Cómo habría que reordenar la matriz

$$A = \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & & & \\ \times & & \times & & \\ \times & & & \times & \\ \times & & & & \times \end{bmatrix}$$

para conseguir el menor número de elementos de relleno al factorizarla por Cholesky o Gauss?

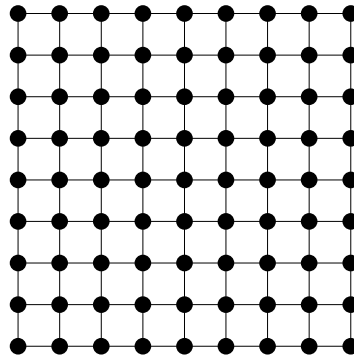
3.6. ¿Qué numeración se debe dar al grafo



para que al factorizar la matriz asociada mediante eliminación de Gauss no se produzcan elementos de relleno?

- 3.7. ¿Qué algoritmo de los estudiados para reordenar la numeración de los nudos del grafo asociado a una matriz dispersa habría que utilizar antes de factorizar la matriz del ejercicio 3? ¿Por qué?
- 3.8. ¿Cuál es la permutación P tal que PAP^T produce el efecto indicado en la matriz $A^{14 \times 14}$ de la página 220?

- 3.9. Probar que el ancho de banda o de semibanda de las filas, β_i , que produce la numeración directa de Cuthill-McKee cumple que $\beta_i \geq 1$, para $i \geq 2$, si la matriz no es reducible.
- 3.10. Estudiar cuál debe ser la numeración idónea del grafo

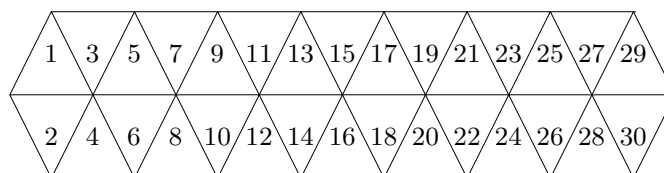


y analizar su perfil, envolvente y los elementos de relleno.

- 3.11. Obtener un transversal completo de la matriz cuya estructura de elementos distintos de cero es la siguiente

$$\begin{bmatrix} \times & & & & \times & \times \\ & \times & \times & & & \\ & \times & \times & & & \\ \times & & & \times & \times & \\ & & & & \times & \\ & & \times & \times & \times & \times \\ & \times & & \times & & \end{bmatrix}.$$

- 3.12. Dada una matriz triangular inferior en bloques, determinar unas permutaciones P y Q tales que PAQ sea triangular superior en bloques.
- 3.13. Aplicar el algoritmo de Tarjan al digrafo de la figura 3.34 comenzando por el nudo 7.
- 3.14. Construir una matriz dispersa para la cual la factorización mediante eliminación de Gauss no produce ningún elemento de relleno y sí muchos la factorización QR mediante transformaciones ortogonales de Givens.
- 3.15. Estudiar la aplicación del método de los frentes a la siguiente estructura.



y determinar el tamaño máximo del frente indicando en qué paso(s) se produce.

- 3.16. Indicar qué trabajo y qué cantidad de memoria son necesarios para aplicar el método de los frentes si éste es uniformemente igual a lo largo de todo el proceso y de tamaño $d \times d$.

3.17. Describir una sucesión de operaciones a realizar en la matriz

$$A = \begin{bmatrix} \times & \times & \times & & & & & & & & \times & \times & \times & \times \\ \times & \times & \times & & & & & & & & & & \times \\ \times & \times & \times & \times & & & & & & & & & \times \\ & & & \times & \times & \times & \times & & & & & & \times \\ & & & & \times & \times & \times & & & & & & \times \\ & & & & & \times & \times & \times & & & & & \times \\ & & & & & & \times & \times & \times & \times & & & \times \\ & & & & & & & \times & \times & \times & & & \times \\ & & & & & & & & \times & \times & \times & \times & \times \\ \times & & & & & & & & & \times & \times & \times & \times & \times \\ \times & & & & & & & & & & \times & \times & \times & \times \\ \times & & & & & & & & & & & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times & \times & \times & \times & \times & \times & \times & \times & \times \end{bmatrix}$$

de tal forma que la resolución del sistema lineal correspondiente sea óptima en cuanto a número de operaciones a realizar y posiciones de memoria a emplear.

Capítulo 4

SOLUCIÓN DE SISTEMAS DE ECUACIONES NO LINEALES

ESTE CAPÍTULO ESTÁ dedicado al estudio de los métodos de solución de sistemas de ecuaciones no lineales. Es decir, a dar respuesta al problema:

dada $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, hallar un x^* tal que $f(x^*) = 0$.

La función vectorial f se supone continua y diferenciable en algún conjunto abierto de $\mathbb{R}^n$, con derivadas parciales continuas en ese abierto.

Los métodos que estudiaremos se basan en procedimientos iterativos en los que en cada una de sus etapas se resuelve un sistema de ecuaciones lineales, resultante de una aproximación del no lineal original en el entorno del punto que define el comienzo de una nueva etapa. Como el problema que planteamos está íntimamente relacionado con problemas de optimización, los métodos que se expondrán en el capítulo son una especialización de otros más generales para minimizar funciones en $\mathbb{R}^n$.

Estudios de cargas en sistemas eléctricos de generación y transporte de energía

Los estudios de cargas en un sistema eléctrico de generación y transporte de energía se refieren a la determinación de, a partir de un patrón definido de demanda y generación de potencia en cada uno de los nudos que configuran ese sistema, las tensiones en módulo y argumento en los nudos, los flujos de potencia activa y reactiva por todos los elementos del sistema y cualquier otra magnitud de interés relativa al estado estacionario de ese sistema: intensidad por las líneas, pérdidas en éstas, etc. Como tal, un estudio de cargas no consiste en determinar con qué generación de potencia activa se puede satisfacer una demanda dada, sino hallar el estado en que ha de funcionar el sistema y qué parámetros lo caracterizan. Como *estudio de cargas* o *flujo de cargas* también se designa el programa de ordenador que resuelve este problema.

Constituye una de las herramientas de análisis de sistemas eléctricos más potentes de entre las que disponen los ingenieros para estudiar aspectos relativos a la explotación, planificación o mantenimiento del sistema. En cualquier empresa u organismo encargado del control de un sistema eléctrico son decenas las veces que se efectúan automática o manualmente estudios de cargas en un día normal de trabajo.

Si se supone que los parámetros físicos de un sistema eléctrico permanecen constantes, existen cuatro variables asociadas a cada nudo i de ese sistema: la tensión, en módulo, V_i , y argumento, θ_i ; la potencia activa inyectada, P_i , y la potencia reactiva inyectada, Q_i . Las potencias inyectadas dependen de la tensión en el propio nudo i y en los a él unidos. Las expresiones¹ que las relacionan, si no hay transformadores conectados al nudo i , están definidas por

$$P_i = |V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n (G_{p_{ij}} + G_{s_{ij}}) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| [G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j)]$$

$$Q_i = -|V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n (B_{p_{ij}} + B_{s_{ij}}) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| [G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j)]$$

donde: V_i es el módulo de la tensión en el nudo i ;

θ_i el argumento de la tensión en el nudo i ;

$G_{s_{ij}}$ la conductancia serie (constante) de la línea que une el nudo i con el nudo j ;

$G_{p_{ij}}$ la conductancia a tierra (constante) de la línea que une el nudo i con el j ;

$B_{s_{ij}}$ la susceptancia serie (constante) de la línea que une el nudo i con el nudo j ; y

$B_{p_{ij}}$ la susceptancia a tierra (constante) de la línea que une el nudo i con el j .

Si el nudo tiene algún condensador o reactancia conectado, $B_{p_{ij}}$ deberá englobar la del condensador/reactancia y las de tierra de las líneas conectadas a ese nudo.

Como las tensiones se miden con respecto a una determinada referencia, ésta se elige en un nudo cualquiera —siempre y cuando sus características físicas así lo aconsejen— asignando a la tensión en él el valor de referencia 1 para el módulo y 0 para el argumento. Al nudo de referencia se le suele denominar *nudo holgura*. Al suponer $V_1 = 1$ y $\theta_1 = 0$, para caracterizar un sistema eléctrico de n nudos se necesitarán conocer $2n - 2$ variables.

En un sistema eléctrico de generación y transporte de energía se pueden dar distintos tipos de nudos. Cuáles son éstos y qué variables e incógnitas definirían cada uno de ellos se recogen en la siguiente tabla.

Tipo de nudo	Variables dadas	Incógnitas
Carga o PQ	P, Q	V, θ
Generación o PV	P, V	Q, θ
Holgura	V, θ	P, Q

Si de un nudo, por ejemplo, se conoce el módulo de la tensión y la potencia activa inyectada, para caracterizarlo totalmente habrá que calcular la potencia reactiva inyectada en él y el argumento de su tensión.

¹La deducción de estas expresiones se puede ver en el apéndice C.

Para caracterizar un sistema general habrá que resolver un sistema de $2n - 2$ ecuaciones no lineales de la forma

$$\begin{aligned} f_1(x_1, x_2, \dots, x_{2n-2}) &= b_1 \\ f_2(x_1, x_2, \dots, x_{2n-2}) &= b_2 \\ &\vdots \\ f_{2n-2}(x_1, x_2, \dots, x_{2n-2}) &= b_{2n-2}. \end{aligned} \quad (4.1)$$

Las potencias activa y reactiva inyectadas en el nudo de referencia se calculan una vez resuelto el sistema, pues hasta entonces no se saben las pérdidas reales en el sistema y por lo tanto el balance global de potencia generada/demandada.

Consideremos como ejemplo el pequeño sistema eléctrico de la figura 4.1. Si se elige como nudo de holgura el 1, el 2 es PV y el 3 PQ. La función vectorial $\mathbf{f}(\mathbf{x})$ que definiría el sistema no lineal de ecuaciones con el que determinar el estado de funcionamiento de ese sistema es la siguiente:

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} V_2^2 \sum_{j=1,3} (G_{p_{2j}} + G_{s_{2j}}) - V_2 \sum_{j=1,3} V_j (G_{2j} \cos(\theta_2 - \theta_j) + B_{2j} \sin(\theta_2 - \theta_j)) \\ V_3^2 \sum_{j=1,2} (G_{p_{3j}} + G_{s_{3j}}) - V_3 \sum_{j=1,2} V_j (G_{3j} \cos(\theta_3 - \theta_j) + B_{3j} \sin(\theta_3 - \theta_j)) \\ -V_3^2 \sum_{j=1,2} (B_C + B_{p_{3j}} + B_{s_{3j}}) - V_3 \sum_{j=1,2} V_j (G_{3j} \sin(\theta_3 - \theta_j) - B_{3j} \cos(\theta_3 - \theta_j)) \end{bmatrix}.$$

La susceptancia del condensador conectado al nudo 3 es B_C . El término independiente del sistema de ecuaciones, el $\mathbf{b}$ de (4.1), lo constituirán los valores de V_2 , P_2 , P_3 y Q_3 , datos del problema.

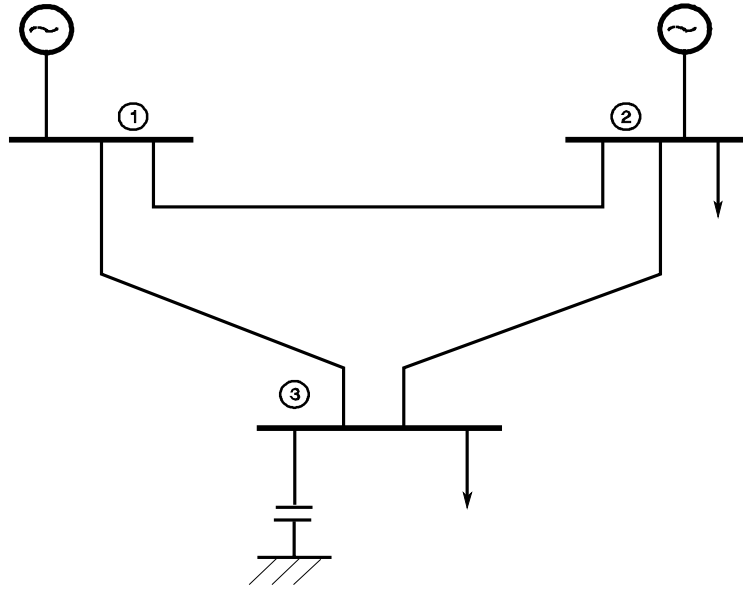
4.1 Velocidad o rapidez de convergencia

Muchos de los métodos para resolver sistemas de ecuaciones lineales y no lineales, y la mayoría de los relativos a procesos de optimización, son *iterativos*, esto es, generan una sucesión de puntos para aproximar tanto como se desee o sea posible el valor de la solución. El estudio de la velocidad o rapidez con que esa sucesión converge a la solución es de gran importancia desde los puntos de vista teórico y práctico. En los sucesivos supondremos que la sucesión converge a la solución y que ésta se designa $\mathbf{x}^*$.

Para facilitar la notación, en lo que sigue de capítulo designaremos con un subíndice el número de la iteración, de tal forma que $\mathbf{x}_k$ designará el valor de $\mathbf{x}$ en la iteración k .

Definición 4.1 Sea una sucesión $\{\mathbf{x}_k\}$, $\mathbf{x}_k \in \mathbb{R}^n$, convergente a $\mathbf{x}^*$. Se define el *orden de convergencia* de $\{\mathbf{x}_k\}$ como el máximo de los números no negativos r que satisface

$$0 \leq \lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^r} < \infty.$$

**Figura 4.1**

Sistema eléctrico de generación y transporte de 3 nudos, 3 líneas y 2 generadores

Si $r = 1$, la sucesión se dice que *converge linealmente*; si $r = 2$, se dice que lo hace *cuadráticamente*.

Si la sucesión $\{\mathbf{x}_k\}$ tiene una convergencia de orden r , el valor γ que satisface

$$\gamma = \lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^r}.$$

se denomina *tasa de convergencia* o *relación de convergencia*. Cuando $r = 1$, para que exista convergencia, γ debe ser estrictamente menor que 1.

Si $\gamma = 0$ y $r = 1$, la sucesión $\{\mathbf{x}_k\}$ se dice que *converge superlinealmente*. Obsérvese que un orden de convergencia mayor que 1 implica convergencia superlineal pues si

$$\gamma = \lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^r} < \infty$$

entonces

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|} = \lim_{k \rightarrow \infty} \frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^r} \|\mathbf{x}_k - \mathbf{x}^*\|^{r-1} = \gamma \cdot \lim_{k \rightarrow \infty} \|\mathbf{x}_k - \mathbf{x}^*\|^{r-1} = 0.$$

Los conceptos de orden de convergencia y velocidad de convergencia se suelen utilizar indistintamente. En opinión del autor es más exacto orden de convergencia, pues velocidad abarca algo más que lo expresado en la definición de orden. En cualquier caso, velocidad es un término generalmente más usado.

Para ilustrar los conceptos introducidos, analicemos varios ejemplos sencillos. En primer lugar, consideremos la sucesión escalar definida por

$$x_k = c^{2^k},$$

donde la constante c cumple $0 \leq c < 1$. La sucesión converge a cero. Calculemos su orden de convergencia:

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1} - 0|}{|x_k - 0|^2} = \lim_{k \rightarrow \infty} \frac{c^{2^{k+1}}}{c^{2^{k+1}}} = 1.$$

Es decir, converge cuadráticamente a 0. La convergencia cuadrática quiere decir, grosso modo, que en las proximidades del límite o solución el número de dígitos significativos que aporta cada paso del proceso al valor de ese límite o solución es el doble que el anterior. En la columna 2 de la tabla 4.1 se pueden ver los distintos puntos de la sucesión del primer ejemplo analizado para $c=0,99$.

Tabla 4.1

Convergencia de diversas sucesiones escalares

k	c^{2^k} ($c = 0,99$)	$c^{2^{-k}}$ ($c = 2,2$)	$1/k^k$
0	0,9900000000000000	2,2000000000000000	1,0000000000000000
1	0,9801000000000000	1,483239697419133	0,2500000000000000
2	0,9605960099999999	1,217883285630907	0,037037037037037
3	0,9227446944279201	1,103577494166543	0,0039062500000000
4	0,8514577710948755	1,050512967157732	0,0003200000000000
5	0,7249803359578534	1,024945348376065	0,000021433470507
6	0,5255964875255620	1,012395845692812	0,000001214265678
7	0,2762516676992083	1,006178833852518	0,000000059604644
8	0,0763149839065938	1,003084659364561	0,000000002581174
9	0,0058239767686636	1,001541142122759	0,1000000000000000E-10
10	0,0000339187054019	1,000770274400054	0,350493899481392E-12
11	0,1150478576143195E-08	1,000385063063246	0,112156654784615E-13
12	0,1323600954164474E-17	1,000192513000995	0,330169095523011E-15
13	0,1751919485865107E-35	1,000096251868287	0,899927452978128E-17
14	0,3069221884953861E-71	1,000048124776146	0,228365826052116E-18
15	0,9420122979079730E-143	1,000024062098581	0,542101086242752E-20
16	0,8873871694098596E-286	1,000012030976918	0,120883864830239E-21

Consideremos ahora la sucesión que define

$$x_k = c^{2^{-k}},$$

donde $c \geq 0$. Esta sucesión converge a 1. Analicemos su orden de convergencia:

$$\begin{aligned} \lim_{k \rightarrow \infty} \frac{|x_{k+1} - 1|}{|x_k - 1|} &= \lim_{k \rightarrow \infty} \frac{c^{2^{-(k+1)}} - 1}{c^{2^{-k}} - 1} \\ &= \lim_{k \rightarrow \infty} \frac{c^{2^{-(k+1)}} - 1}{(c^{2^{-(k+1)}} - 1)(c^{2^{-(k+1)}} + 1)} = \lim_{k \rightarrow \infty} \frac{1}{c^{2^{-(k+1)}} + 1} = \frac{1}{2}. \end{aligned}$$

La sucesión presenta una convergencia lineal. En la columna 3 de la tabla 4.1 se representan sus dieciséis primeros puntos.

Analicemos por último la sucesión que define

$$x_k = \frac{1}{k^k}.$$

Converge a cero. Estudiemos su orden de convergencia:

$$\lim_{k \rightarrow \infty} \frac{|x_{k+1}|}{|x_k|} = \lim_{k \rightarrow \infty} \frac{\frac{1}{(k+1)^{k+1}}}{\frac{1}{k^k}} = \lim_{k \rightarrow \infty} \frac{1}{k \left(1 + \frac{1}{k}\right)^{k+1}} = 0.$$

Es decir, converge superlinealmente a cero. En la columna 4 de la tabla 4.1 se pueden ver los primeros puntos de esta sucesión.

4.2 Problemas de una variable

Comencemos el estudio de los métodos para resolver sistemas de ecuaciones no lineales con el caso en que sólo se tiene una ecuación y una incógnita.

4.2.1 Método de la bisección

Para estudiar este método, atribuido a Bernhard Bolzano (1781-1848), recordemos previamente el teorema del valor intermedio.

Teorema 4.1 Si $f : \mathbb{R} \rightarrow \mathbb{R}$ es una función continua en $[a, b]$ y $f(a) \leq x \leq f(b)$ o $f(b) \leq x \leq f(a)$, existe un punto c tal que $a \leq c \leq b$ en el cual $f(c) = x$.

La idea en la que se basa el método de la bisección es muy simple: si la función $f : \mathbb{R} \rightarrow \mathbb{R}$ es continua en un intervalo $[a, b]$ y $f(a)f(b) < 0$, existirá un $a < c < b$ en el que $f(c) = 0$. Reduciendo convenientemente el intervalo $[a, b]$ se llegará a acotar el valor de c tanto como se desee.

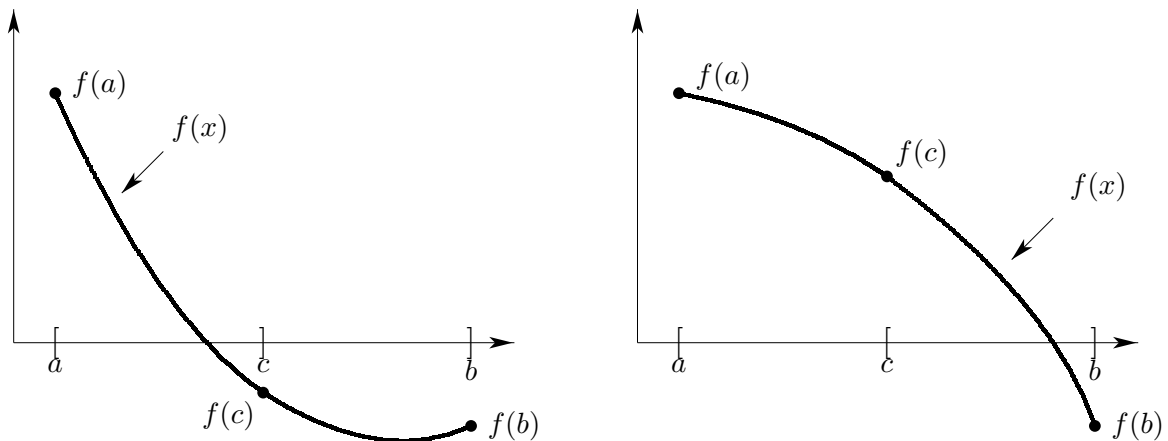
El procedimiento parte de $[a, b]$ y, sucesivamente, va dividiendo el intervalo en dos mitades, quedándose para la siguiente iteración con aquella en cuyos extremos la función toma valores de signo contrario. Si al comienzo $u = f(a)$ y $v = f(b)$, tales que $uv < 0$, se determina $c = \frac{1}{2}(a+b)$ y se calcula $w = f(c)$. Si $f(c) = 0$ se ha llegado a la solución buscada; si no, se cumplirá que $wu < 0$ o $wv < 0$. Si $wu < 0$, la solución estará en $[a, c]$; si $wv < 0$, en $[c, b]$. De acuerdo con el resultado obtenido se comienza a estudiar el nuevo intervalo, procediéndose iterativamente de la misma forma hasta que se estreche lo que se desea el intervalo que contenga el valor de la solución. En la figura 4.2 se representan los dos casos que se pueden presentar en la primera iteración del método.

Si el intervalo con que se empieza el proceso iterativo, $[a_0, b_0]$, contiene una solución r , usando como estimación de ésta $c_0 = (a_0 + b_0)/2$, se tendrá que

$$|r - c_0| \leq \frac{b_0 - a_0}{2}.$$

En cualquier iteración, razonando de forma similar,

$$|r - c_i| \leq \frac{b_i - a_i}{2}, \quad i = 0, 1, 2, \dots$$

**Figura 4.2**

Decisiones posibles en la primera iteración del método de la bisección

Teorema 4.2 Si se aplica el método de la bisección a una función $f : \mathbb{R} \rightarrow \mathbb{R}$ continua en un intervalo $[a, b]$, donde $f(a)f(b) < 0$, después de n iteraciones se habrá obtenido un valor de la solución c_n tal que

$$|r - c_n| \leq \frac{b - a}{2^{n+1}},$$

donde r es el valor real de la solución.

A continuación se lista un código en FORTRAN 77 que calcula mediante este método la solución de $x \sin(x) - 1 = 0$ en el intervalo $[1, 2]$.

```

PROGRAM Bisec
C
C *** Resolución de la ecuación x*sin(x)-1=0 ***
C
      data a/1.0/,b/2.0/
C
      fa = fx(a)
      fb = fx(b)
      if (fa*fb.gt.0) stop 'El intervalo [a,b] no contiene la solución'
C
      tol = epsilon(1.0)*10
      do while (abs(a-b).gt.tol)
        c = (a+b)/2.
        fc = fx(c)
        if (fc.eq.0) then
          a = c
          b = c
        else if (fb*fc.gt.0) then
          b = c
          fb = fc
        else
          a = c
          fa = fc
        endif
      endwhile
      print '(2f10.7)',a,b

```

```

      end do
c
      end

      real function fx (x)
      fx = x*sin(x)-1
      return
      end

```

Los valores de los extremos del intervalo $[a, b]$ que se obtienen con este código en las distintas iteraciones son los que describe la tabla 4.2. En la figura 4.3 se representa cómo procede el método para llegar a la solución.

Tabla 4.2

Convergencia del método de la bisección aplicado a $x \sin(x) - 1 = 0$

k	a	b	k	a	b
1	1,000000	1,500000	11	1,113770	1,114258
2	1,000000	1,250000	12	1,114014	1,114258
3	1,000000	1,125000	13	1,114136	1,114258
4	1,062500	1,125000	14	1,114136	1,114197
5	1,093750	1,125000	15	1,114136	1,114166
6	1,109375	1,125000	16	1,114151	1,114166
7	1,109375	1,117188	17	1,114151	1,114159
8	1,113281	1,117188	18	1,114155	1,114159
9	1,113281	1,115234	19	1,114157	1,114159
10	1,113281	1,114258	20	1,114157	1,114158

4.2.2 Método de Newton-Raphson

Consideremos la ecuación

$$x^3 - \sin(x) = 0.$$

Al ser una función senoidal, serán varias sus raíces o puntos para los cuales $f(x) = 0$. Calcularemos el más próximo a $x = 1$.

Escribamos la ecuación en la forma $x = g(x)$ y procedamos a aplicar un procedimiento iterativo para resolver el problema basándonos en la relación de recurrencia $x_{k+1} = g(x_k)$. La primera forma $x = g(x)$ que podemos utilizar es

$$x = \sqrt[3]{\sin(x)}.$$

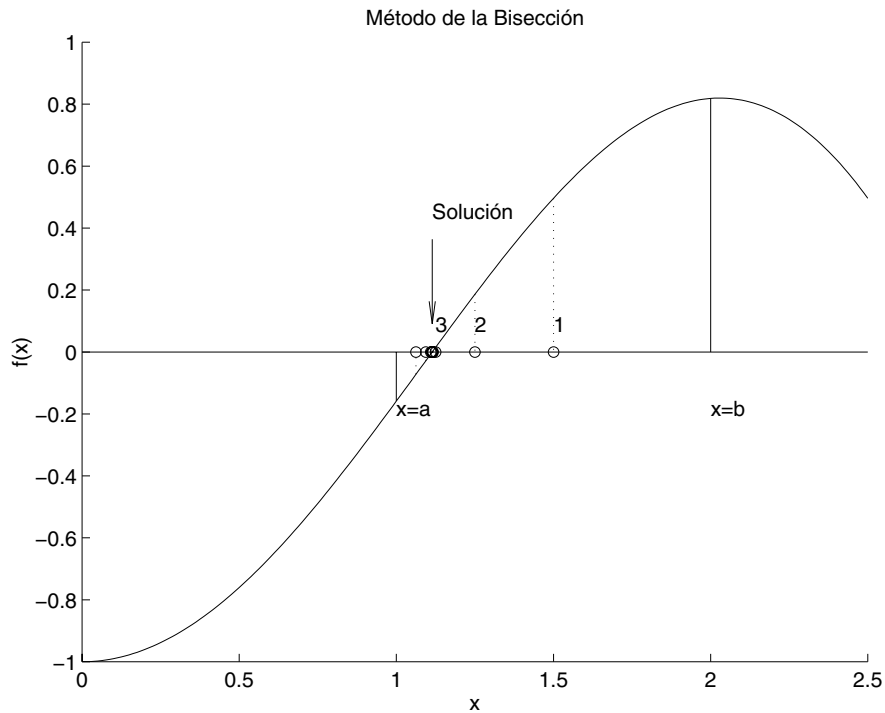
La relación de recurrencia será, por tanto,

$$x_{k+1} = \sqrt[3]{\sin(x_k)}.$$

Si comenzamos el proceso iterativo desde² $x_0 = 1$, se tendrá que:

$$x_1 = \sqrt[3]{\sin(x_0)} = 0,944; \quad x_2 = \sqrt[3]{\sin(0,944)} = 0,932; \quad x_3 = \sqrt[3]{\sin(0,932)} = 0,929;$$

²Radianes.

**Figura 4.3**

Proceso de obtención de la solución de $x \sin(x) - 1 = 0$ con el método de la bisección

y así sucesivamente. La solución converge a $x^* = 0,92862$.

Si en lugar de utilizar $x = \sqrt[3]{\sin(x)}$, hubiésemos hecho

$$x = \frac{\sin(x)}{x^2},$$

utilizando, por tanto, la relación de recurrencia

$$x_{k+1} = \frac{\sin(x_k)}{x_k^2},$$

partiendo de $x_0 = 1$, se hubiesen obtenido los puntos de la siguiente tabla.

k	x_k
0	1,000
1	0,841
2	1,053
3	0,783
4	1,149
$\vdots$	$\vdots$

El proceso diverge.

Analícemos gráficamente qué ocurre en estos dos procesos iterativos. El que define la relación de recurrencia $x_{k+1} = g(x_k)$ genera lo que se ha dado en llamar una *tela de araña* entre la recta

$y = x$ y la curva $y = g(x)$. En la figura 4.4 se pueden ver las dos *telas de araña* que generan los dos procesos iterativos anteriores.

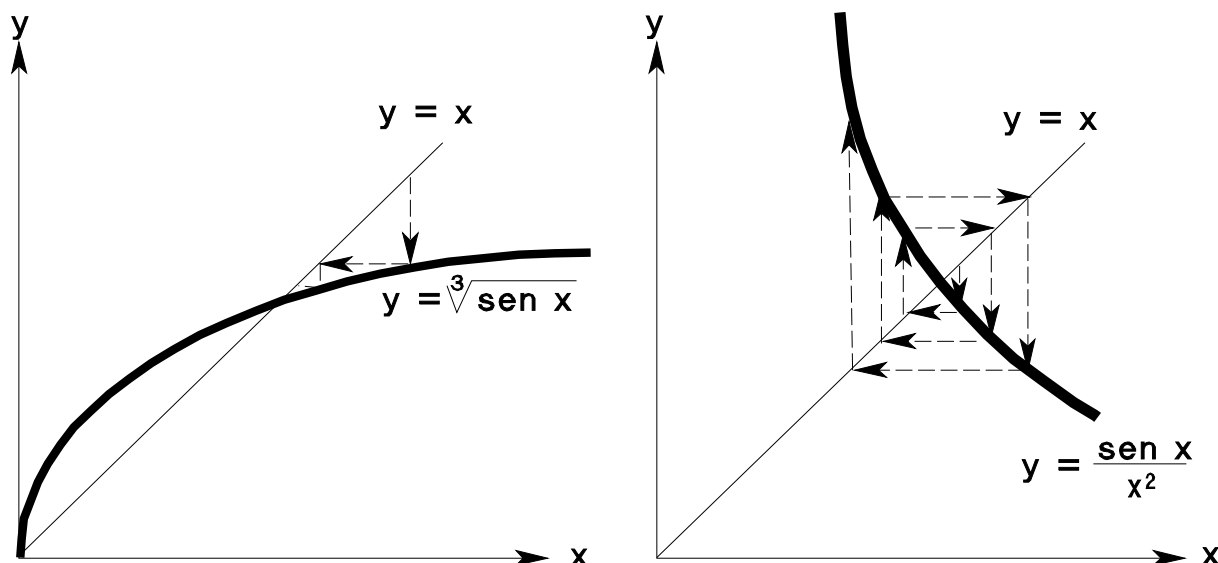


Figura 4.4

Telas de araña de $g(x) = (\text{sen}(x))^{1/3}$ y $g(x) = \text{sen}(x)/x^2$

Si se experimenta con diversas relaciones de recurrencia se puede observar que el comportamiento del proceso iterativo que generan está íntimamente relacionado con las pendientes de g en las proximidades de la raíz x^* . Si $|g'(x^*)| < 1$ y el punto de partida está *cerca* de x^* , el proceso converge. Si $|g'(x^*)| > 1$, diverge.

En el caso de $x_{k+1} = \sqrt[3]{\text{sen}(x_k)}$,

$$g'(x) = \frac{(\text{sen}(x))^{-2/3}}{3} \cos(x);$$

en $x^* \approx 0,929$, $g'(0,929) \approx 0,23$. Por el contrario, en el caso de $x_{k+1} = \text{sen}(x_k)/x_k^2$,

$$g'(x) = \frac{\cos(x)}{x^2} - 2\frac{\text{sen}(x)}{x^3};$$

en $x^* \approx 0,929$, $g'(0,929) \approx -1,23$.

Estas consideraciones nos sirven para constatar una necesidad: disponer de una vía sistemática y fiable de construir un modelo $x = g(x)$, caso de utilizar esta forma de resolver el problema, para, comenzando desde cualquier x_0 , hallar la solución de la ecuación $f(x) = 0$.

Isaac Newton (1642-1727) fue el primero que ideó esa vía y la forma de llevarla a la práctica sencillamente. Expresado en términos simples e intuitivos, su método consiste en reemplazar la función $f(x)$ en cada punto del proceso iterativo por el modelo de ella que define la recta tangente a $f(x)$ en ese punto —lo que se ha dado en llamar linealizar la función en un punto—. En un punto dado, $x = x_1$, la ecuación de la recta tangente a una función $f(x)$ es

$$y = f(x_1) + f'(x_1)(x - x_1). \quad (4.2)$$

En $x = x_1$, $y = f(x_1)$ por lo que la ordenada de (4.2) en x_1 es la misma que la de f . La pendiente de f en x_1 es la misma que la de y : $f'(x_1)$.

El siguiente punto del proceso iterativo que define el método de Newton lo determina la solución de $y(x) = 0$, es decir, dónde esa recta tangente corta al eje x :

$$0 = f(x_1) + f'(x_1)(x - x_1). \quad (4.3)$$

La solución de (4.3) es mucho más fácil de calcular que directamente la de $f(x)$. En concreto, esa solución es

$$x = x_1 - \frac{f(x_1)}{f'(x_1)}.$$

En la figura 4.5 se describe gráficamente este paso del proceso iterativo de Newton.

La relación general de recurrencia que define el método de Newton —también denominado Newton-Raphson— para encontrar una solución de la ecuación $f(x) = 0$ es, por consiguiente,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \quad (4.4)$$

Si aplicamos el método de Newton al problema anterior,

$$x^3 - \operatorname{sen}(x) = 0,$$

la relación de recurrencia es

$$x_{k+1} = x_k - \frac{x_k^3 - \operatorname{sen}(x_k)}{3x_k^2 - \cos(x_k)}.$$

En los términos en que nos expresábamos al comienzo de este apartado (recordemos la relación $x = g(x)$), el método de Newton hace

$$g_N(x) = x - \frac{x^3 - \operatorname{sen}(x)}{3x^2 - \cos(x)}.$$

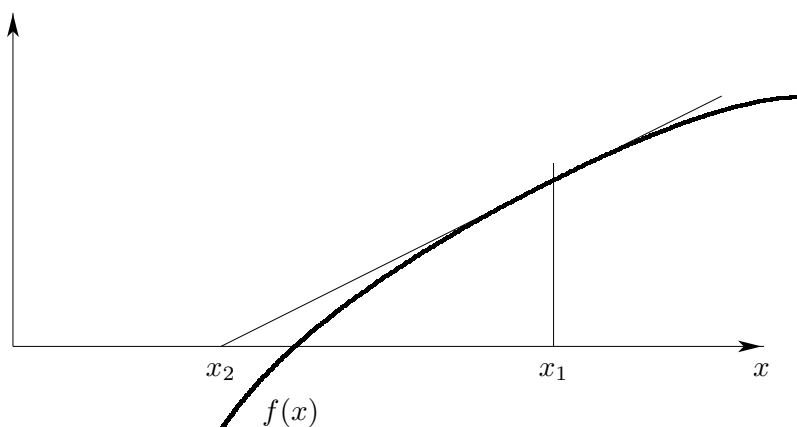


Figura 4.5
Aproximación lineal de $f(x)$ en $x = x_1$

A continuación se lista un pequeño código en FORTRAN 77 que implementa el método de Newton para resolver $x^3 - \sin(x) = 0$ partiendo de $x_0 = 1,4$.

```
PROGRAM Newt
tol = epsilon(1.0)
x0 = 0
x = 1.4
do while (abs(x-x0).gt.tol)
  x0 = x
  x = x0-(x0**3-sin(x0))/(3*x0*x0-cos(x0))
  print '(f10.7)',x          ! Salida de resultados
end do
end
```

Los puntos que se obtienen con el código son los de la tabla que sigue.

k	x_k
1	1,0920240
2	0,9589750
3	0,9299978
4	0,9286293
5	0,9286263

La representación gráfica del proceso que lleva a la solución se describe en la figura 4.6.

Tradicionalmente, el método de Newton-Raphson se explica partiendo del modelo lineal de la función $f(x)$ que resulta de su aproximación alrededor de un punto x_k mediante el *desarrollo*

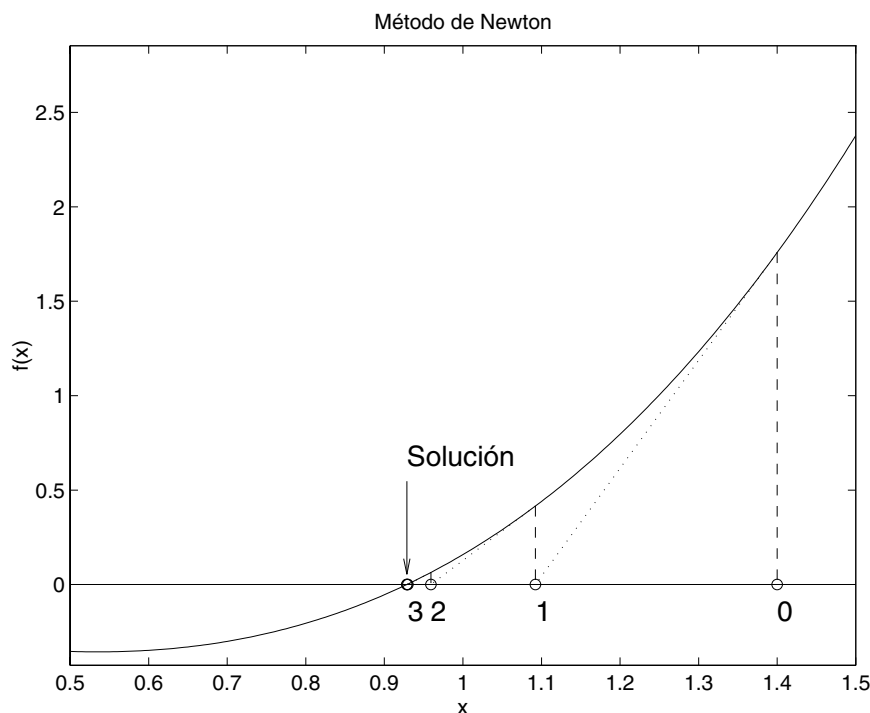


Figura 4.6

Obtención de la solución de $x^3 - \sin(x) = 0$ con el método de Newton

en serie de Taylor,³

$$\begin{aligned} f(x) = & f(x_k) + f'(x_k)(x - x_k) + f''(x_k) \frac{(x - x_k)^2}{2!} + \dots \\ & + f^r(x_k + \theta(x - x_k)) \frac{(x - x_k)^r}{r!}, \quad 0 \leq \theta \leq 1, \end{aligned} \quad (4.5)$$

siempre y cuando $f \in C^r$, truncando ese desarrollo a partir de los términos de segundo orden. Es decir, construir un modelo de la forma

$$M_k(x) = f(x_k) + f'(x_k)(x - x_k), \quad (4.6)$$

y utilizarlo en una iteración k en lugar de la propia función $f(x)$.

Otros enfoques prefieren basar su desarrollo en el *teorema de Newton*:

$$f(x) = f(x_k) + \int_{x_k}^x f'(z) dz,$$

y aproximar la integral de la forma

$$\int_{x_k}^x f'(z) dz \cong f'(x_k)(x - x_k),$$

obteniéndose la misma aproximación de $f(x)$ de (4.6).

4.2.3 Convergencia del método de Newton para una variable

En lo que sigue se usará el concepto de continuidad de Lipschitz que se define en el apéndice A, página 693.

Lema 4.1 Sea la función $f : D \rightarrow \mathfrak{R}$ con dominio de definición en un intervalo abierto D cumpliéndose que $f' \in \text{Lip}_\gamma(D)$. Para todo $x, y \in D$,

$$|f(y) - f(x) - f'(x)(y - x)| \leq \frac{\gamma|y - x|^2}{2}. \quad (4.7)$$

DEMOSTRACIÓN. De cálculo, $f(y) - f(x) = \int_x^y f'(z) dz$, o, de forma equivalente,

$$f(y) - f(x) - f'(x)(y - x) = \int_x^y [f'(z) - f'(x)] dz. \quad (4.8)$$

Haciendo el cambio de variable

$$z = x + t(y - x), \quad dz = dt(y - x),$$

³Ver el teorema de Taylor en el apéndice A.

la expresión (4.8) queda,

$$f(y) - f(x) - f'(x)(y - x) = \int_0^1 [f'(x + t(y - x)) - f'(x)] (y - x) dt.$$

Por la regla del triángulo aplicada a la integral y la continuidad Lipschitz de f' se tiene que

$$|f(y) - f(x) - f'(x)(y - x)| \leq |y - x| \int_0^1 \gamma |t(y - x)| dt = \gamma |y - x|^2 / 2.$$

■

Obsérvese que la expresión (4.7) se asemeja al error de la aproximación de f por desarrollo en serie de Taylor, (4.5), truncando ese desarrollo en $r = 2$, con la constante γ en lugar de $f''(\xi)$, $\xi \in D$. La ventaja de usar la continuidad Lipschitz es que no se necesita tener en cuenta los términos que involucran las derivadas segundas. Como veremos posteriormente, esto es particularmente útil en más de dos dimensiones.

Hechas estas consideraciones, probemos a continuación la convergencia del método de Newton-Raphson para ecuaciones no lineales en una variable.

Teorema 4.3 *Sea la función $f : D \rightarrow \Re$ con dominio de definición en un intervalo abierto D cumpliéndose que $f' \in \text{Lip}_\gamma(D)$. Supóngase que para algún $\rho > 0$, $|f'(x)| \geq \rho$ para todo $x \in D$. Si $f(x) = 0$ tiene solución, x^* , existe algún $\eta > 0$ tal que si $|x_0 - x^*| < \eta$ la sucesión $\{x_k\}$ que genera la relación de recurrencia*

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}; \quad k = 0, 1, 2, \dots$$

converge a x^ . Además,*

$$|x_{k+1} - x^*| \leq \frac{\gamma}{2\rho} |x_k - x^*|^2; \quad k = 0, 1, 2, \dots \quad (4.9)$$

DEMOSTRACIÓN. Sea $\tau \in (0, 1)$ y $\hat{\eta}$ el radio del intervalo más grande alrededor de x^* contenido en D ; hagamos $\eta = \min\{\hat{\eta}, \tau(2\rho/\gamma)\}$. Para $k = 0$,

$$\begin{aligned} x_1 - x^* &= x_0 - x^* - \frac{f(x_0)}{f'(x_0)} = x_0 - x^* - \frac{f(x_0) - f(x^*)}{f'(x_0)} \\ &= \frac{1}{f'(x_0)} [f(x^*) - f(x_0) - f'(x_0)(x^* - x_0)]. \end{aligned}$$

El término entre corchetes es $f(x^*) - M_0(x^*)$,⁴ el error en x^* del valor de $M_k(x)$ en $x = x^*$. De acuerdo con el lema anterior,

$$|x_1 - x^*| \leq \frac{\gamma}{2|f'(x_0)|} |x_0 - x^*|^2.$$

⁴Recordemos la expresión (4.6).

De los supuestos atribuidos a $f'(x)$,

$$|x_1 - x^*| \leq \frac{\gamma}{2\rho} |x_0 - x^*|^2.$$

Como $|x_0 - x^*| \leq \eta \leq \tau(2\rho/\gamma)$, entonces $|x_1 - x^*| \leq \tau|x_0 - x^*| < \eta$. Procediendo de igual forma, por inducción para $k > 0$, quedaría probado el teorema. ■

La condición expuesta en el teorema de que $f'(x)$ esté acotada inferiormente en D , significa que $f'(x^*)$ debe ser distinta de cero para que el método de Newton converja cuadráticamente a ese valor x^* . Si $f'(x^*) = 0$, entonces x^* es una raíz múltiple y el método sólo convergería linealmente. Para apreciar la diferencia, si aplicamos el método de Newton para resolver $f_1(x) = x^2 - 1 = 0$ y $f_2(x) = x^2 - 2x + 1 = 0$, partiendo de $x_0 = 2$, los primeros puntos que resultan son los de la tabla que sigue.

$f_1(x) = x^2 - 1 = 0$		$f_2(x) = x^2 - 2x + 1 = 0$
2	x_0	2
1,25	x_1	1,5
1,025	x_2	1,25
1,0003048780488	x_3	1,125
1,0000000464611	x_4	1,0625
1,0	x_5	1,03125

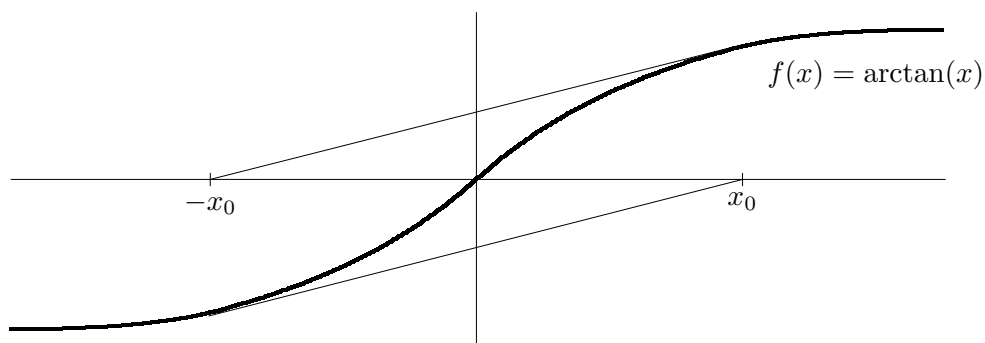
Es también interesante analizar la constante $\gamma/2\rho$ de la expresión (4.9). El numerador γ , la constante Lipschitz de f' en D , se puede considerar como el parámetro que mide de alguna manera el grado de no linealidad de la función f . Ahora bien, γ es una magnitud que depende de la escala adoptada; multiplicando f o cambiando las unidades de x por una constante, f' se verá multiplicada por una constante sin ver afectada su no linealidad. Una magnitud independiente de esa escala sería el índice relativo de variación de $f'(x)$ y podría obtenerse dividiendo γ por $f'(x)$. De esta manera, como ρ es un límite inferior de los valores que puede tomar $f(x)$, para $x \in D$, γ/ρ es un límite superior de la no linealidad relativa de $f(x)$. El teorema 4.3 dice, en este sentido, que cuanto más pequeña sea esa cantidad, más rápido convergerá el método de Newton. Si f es lineal, $\gamma = 0$ y $x_1 = x^*$.

El teorema 4.3 garantiza la convergencia del método de Newton sólo si se inicia el proceso desde un punto x_0 aceptable. Es fácilmente comprobable que el método puede no funcionar si $|x_0 - x^*|$ es grande. Por ejemplo, considérese el problema clásico de hallar la solución de $\arctan(x) = 0$. Partiendo de cualquier punto del intervalo $[1,39, 1,40]$, el método cicla obteniéndose $x_1 = -x_0$, $x_2 = x_0$, $x_3 = -x_0$, ... Si $x_0 < 1,39$, el procedimiento converge; si $x_0 > 1,40$, diverge. En la figura 4.7 se representan estas circunstancias.

Se puede concluir pues que el método de Newton es útil para resolver ecuaciones no lineales de una variable siempre y cuando se tengan en cuenta ciertas precauciones y se escoja adecuadamente el punto de partida.

4.2.4 Variantes del método de Newton

Una de las primeras variantes del método de Newton para calcular la raíz de una ecuación que se puede sugerir es aquella que resulta de incorporar algún mecanismo que impida que ocurran los problemas mencionados en el apartado anterior. A tal efecto conviene recordar que la resolución de la ecuación de Newton (4.4) en cada paso del proceso iterativo del método no

**Figura 4.7**Método de Newton aplicado a $f(x) = \arctan(x)$

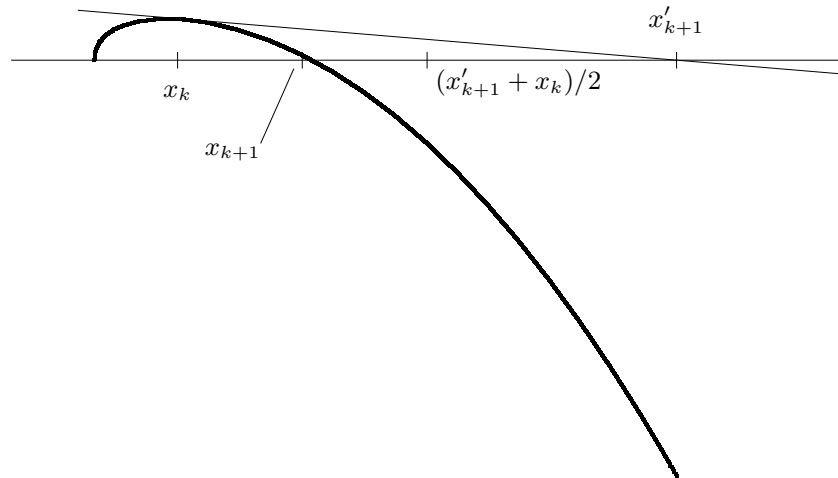
sólo define el nuevo punto del proceso iterativo, x_{k+1} , sino una dirección, $f'(x_k)$, a lo largo de la cual se da un *paso* (determinado por $x_{k+1} - x_k$). A pesar de que es probable que ese paso sea el adecuado y la función $f(x)$ en el nuevo punto adquiriera un valor menor que el que tenía en x_k , puede ocurrir lo contrario, como veíamos anteriormente, y el proceso diverja, siendo en cualquier caso buena la dirección calculada, pues a lo largo de ella la función decrece en ciertos puntos. Una idea muy intuitiva que se suele aplicar es la siguiente: si la dirección que determina la solución de la ecuación de Newton *promete* un descenso del valor de la función a lo largo de ella, incorporemos un *mecanismo de salvaguarda* que permita, a lo largo de esa dirección, moviéndose un paso adecuado, disminuir siempre el valor de la función; si el paso completo $x_{k+1} - x_k$ produce un aumento, disminuirlo hasta que $|f(x_{k+1})| < |f(x_k)|$. Ese posible mecanismo lo plasma el algoritmo que sigue.

$$\begin{array}{l}
 x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \\
 \textbf{while } (|f(x_{k+1})| \geq |f(x_k)|) \textbf{ do} \\
 \quad x_{k+1} \leftarrow \frac{x_{k+1} + x_k}{2} \\
 \textbf{end}
 \end{array} \tag{4.10}$$

En la figura 4.8 se ilustra un caso en el que ocurre lo que acabamos de indicar y cómo el mecanismo apuntado salva las dificultades que surgirían de aplicar el procedimiento de Newton sin él. El punto x'_{k+1} , que sería el que determinaría el paso de Newton, no valdría. Tampoco $(x'_{k+1} + x_k)/2$. Sí, por fin,

$$x_{k+1} = \frac{x_k + (x'_{k+1} + x_k)/2}{2}.$$

Esta forma de proceder, como veremos más adelante, es particularmente útil en problemas de más de una variable.

**Figura 4.8**

Método de Newton con mecanismo de salvaguarda

4.2.4.1 Método de Newton por diferencias finitas

Hasta ahora hemos supuesto que se conoce la expresión de la derivada de la función $f(x)$, y por tanto, es fácilmente evaluable en un determinado punto. Ahora bien, en muchas aplicaciones prácticas esto no es así, bien porque su determinación analítica es muy complicada —la función $f(x)$ surge de un procedimiento experimental, por ejemplo—, o, sencillamente, porque el usuario del método no desea obtenerla.

Previendo estas eventualidades, es conveniente modificar el método de tal forma que se soslaye esta dificultad. La primera modificación que se puede emplear surge de la aplicación inmediata de la definición de derivada de una función $f(x)$ en un punto: esto es,

$$f'(x_k) = \lim_{h \rightarrow 0} \frac{f(x_k + h) - f(x_k)}{h}.$$

En lugar de tener que saber la derivada de la función, se calcula su valor en un punto mediante la fórmula anterior utilizando un parámetro h adecuado. A la variante del método que surge de esta idea se la conoce como *método de Newton por diferencias finitas*.

Dos preguntas surgen al plantearse aplicar esta idea: ¿funciona?, ¿cómo se escoge el parámetro h ? La respuesta a la primera está relacionada con la de la segunda: el método de Newton por diferencias finitas, escogido h adecuadamente, insistimos, debe funcionar tan bien como el propio método de Newton.

La elección de h es crítica para el buen funcionamiento del procedimiento. Si se hace

$$a_k = (f(x_k + h) - f(x_k))/h,$$

la relación de recurrencia del método de Newton queda

$$x_{k+1} = x_k - \frac{f(x_k)}{a_k}.$$

De aquí que,

$$\begin{aligned}
 x_{k+1} - x^* &= x_k - x^* - \frac{f(x_k)}{a_k} \\
 &= a_k^{-1} [f(x^*) - f(x_k) - a_k(x^* - x_k)] \\
 &= a_k^{-1} [f(x^*) - M_k(x^*)] \\
 &= a_k^{-1} \{f(x^*) - f(x_k) - f'(x_k)(x^* - x_k) + [f'(x_k) - a_k](x^* - x_k)\} \\
 &= a_k^{-1} \left\{ \int_{x_k}^{x^*} [f'(z) - f'(x_k)] dz + [f'(x_k) - a_k](x^* - x_k) \right\}.
 \end{aligned}$$

Si definimos $e_k = |x_k - x^*|$ y $e_{k+1} = |x_{k+1} - x^*|$, teniendo en cuenta el mismo supuesto de continuidad Lipschitz de f' que hacíamos en el lema 4.1, se tiene que

$$e_{k+1} \leq |a_k^{-1}| \left(\frac{\gamma}{2} e_k^2 + |f'(x_k) - a_k| e_k \right). \quad (4.11)$$

Esta última expresión (que relaciona los errores entre dos iteraciones) es muy similar a la que se obtenía en el teorema 4.3: $|a_k^{-1}|$ reemplaza a $|f'(x_k)^{-1}|$ y se incorpora un término que expresa la diferencia entre $f'(x_k)$ y su aproximación por a_k .

Corolario 4.1 *Sea la función $f : D \rightarrow \mathbb{R}$ con dominio de definición en un intervalo abierto D siendo $f' \in \text{Lip}_\gamma(D)$. Si $x_k, x_{k+1} + h \in D$, y $a_k = (f(x_k + h) - f(x_k))/h$, entonces*

$$|a_k - f'(x_k)| \leq \frac{\gamma|h|}{2}. \quad (4.12)$$

DEMOSTRACIÓN. De acuerdo con el lema 4.1,

$$|f(x_k + h) - f(x_k) - hf'(x_k)| \leq \frac{\gamma|h|^2}{2}.$$

Dividiendo ambos miembros por $|h|$ se obtiene el resultado enunciado. ■

Sustituyendo (4.12) en (4.11) se tiene que

$$e_{k+1} \leq \frac{\gamma}{2|a_k|} (e_k + |h|) e_k.$$

En el supuesto de que $|f'(x)| \geq \rho > 0$ en un entorno de x , es fácilmente comprobable que, para un $|h|$ suficientemente pequeño y para un $x_k \in D$, se tiene que $|a_k|^{-1} \leq 2\rho^{-1}$. En definitiva,

$$e_{k+1} \leq \frac{\gamma}{\rho} (e_k + |h|) e_k.$$

Con estos resultados queda prácticamente probado el siguiente teorema.

Teorema 4.4 Sea la función $f : D \rightarrow \mathbb{R}$ con dominio de definición en un intervalo abierto D siendo $f' \in \text{Lip}_\gamma(D)$. Supóngase que, para todo $x \in D$, $|f'(x)| \geq \rho$ para algún $\rho > 0$. Si $f(x) = 0$ tiene solución $x^* \in D$, existen unas constantes positivas η y η' tales que si $\{h_k\}$ es una sucesión de números reales tales que $0 < |h_k| \leq \eta'$ y si $|x_0 - x^*| < \eta$, la sucesión $\{x_k\}$ que define

$$x_{k+1} = x_k - \frac{f(x_k)}{a_k}, \quad \text{con } a_k = \frac{f(x_k + h_k) - f(x_k)}{h_k}, \quad k = 0, 1, \dots,$$

converge linealmente a x^* . Si $\lim_{k \rightarrow \infty} h_k = 0$, la convergencia es superlineal. Si existe alguna constante c_1 tal que

$$|h_k| \leq c_1 |x_k - x^*|,$$

o, de forma equivalente, una constante c_2 tal que

$$|h_k| \leq c_2 |f(x_k)|,$$

la convergencia es cuadrática. Si existe alguna constante c_3 tal que

$$|h_k| \leq c_3 |x_k - x_{k-1}|,$$

la convergencia es al menos cuadrática cada dos pasos.

Desde un punto de vista práctico, la convergencia del método de Newton por diferencias finitas es muy buena. Si se tienen en cuenta las consideraciones del apéndice B correspondientes a errores de redondeo y aritmética en un ordenador, el parámetro h se debe escoger no muy pequeño de tal manera que no ocurra que $fl(x_k + h) = fl(x_k)$, o que, aun siendo $fl(x_k + h) \neq fl(x_k)$, dado que f es continua y su derivada también, al evaluar la función en dos puntos muy próximos, que $fl(f(x_k + h)) = fl(f(x_k))$. Si $|h|$ es pequeño, bastantes de los dígitos más significativos de $f(x_k + h)$ y de $f(x_k)$ serán iguales. Supongamos por ejemplo que estuviésemos trabajando en una máquina⁵ con $\beta = 10$ y $t = 5$. Si $f(x_k) = 1,0001$ y $f(x_k + h) = 1,0010$, con $h = 0,0001$, $f(x_k + h) - f(x_k)$ sería igual a 9×10^{-4} , por lo que a_k sería 9: se habrían perdido casi todos los dígitos significativos al calcular la diferencia de $f(x_k + h)$ y $f(x_k)$.

La forma más obvia de calcular a_k lo más precisamente posible consiste en escoger un $|h|$ lo suficientemente grande como para que los dígitos más significativos de $f(x_k + h)$ y de $f(x_k)$ no sean iguales. Ahora bien, existe un límite en la magnitud atribuible a ese $|h|$ dado que el objetivo de usar a_k es utilizarla en lugar de $f'(x_k)$; ese límite es el definido en la expresión (4.12). Un compromiso consiste en intentar ponderar el error introducido en la aproximación de la no linealidad de la función eligiendo un $|h|$ grande y el resultante de la evaluación de las funciones al elegir un $|h|$ muy pequeño.

Una regla sencilla que se suele usar habitualmente en casi todos los códigos comerciales que implementan estas técnicas consiste en elegir

$$|h| = \sqrt{\epsilon} \max\{\text{tip } x, |x_k|\},$$

donde $\text{tip } x$ indica la magnitud típica de x y ϵ es la precisión de la máquina en la que se utiliza

⁵Recordemos: base de numeración 10; número de dígitos significativos, 5.

el correspondiente código (ver apéndice B). Para un problema bien escalado bastaría hacer

$$|h| = \sqrt{\epsilon}.$$

Cuando existen problemas de precisión, también es posible recurrir a la aproximación de $f'(x_k)$ dada por

$$a_k = \frac{f(x_k + h) - f(x_k - h)}{2h}.$$

En este caso conviene tener en cuenta que el número de veces que se evalúa la función se duplica con respecto al anterior.

Ejemplo 4.1 Calculemos la solución de $f(x) = x^2 - 1$, partiendo de $x = 2$, mediante los métodos de Newton y Newton por diferencias finitas.

Los códigos en FORTRAN 77 para calcularlos son los que siguen.

<pre> C PROGRAM Newton C double precision fx,derfx,eps, + x1/2./,x0/0./ parameter (eps=epsilon(1.d0)) C do while (dabs(fx(x1)).gt.eps) x0 = x1 x1 = x0-fx(x0)/derfx(x0) print *,x1 end do C end double precision function fx(x) double precision x fx = x**2-1. return end double precision function derfx(x) double precision x derfx = 2.0*x return end </pre>	<pre> C PROGRAM Newtondf C double precision fx,derfx,eps,h, + x1/2./,x0/0./ parameter (eps=epsilon(1.d0)) C h = dsqrt(eps) do while (dabs(fx(x1)).gt.eps) x0 = x1 x1 = x0-fx(x0)/derfx(x0,h) print *,x1 end do C end double precision function fx(x) double precision x fx = x**2-1. return end double precision function derfx(x,h) double precision fx,x,h derfx = (fx(x+h)-fx(x))/h return end </pre>
--	--

Los resultados obtenidos con uno y otro código son los de la siguiente tabla.

Newton		Newton Dif. Fin.
1,2500000000000000	x_0	1,2500000000000000
1,0250000000000000	x_1	1,025000001341104
1,000304878048780	x_2	1,000304878259699
1,000000046461147	x_3	1,000000046463482
1,0000000000000001	x_4	1,0000000000000001
1,0000000000000000	x_5	1,0000000000000000

Como se puede observar, son prácticamente los mismos. ●

Conviene resaltar que, aunque en la práctica el método de Newton y el de Newton por diferencias finitas pueden funcionar casi igual, si se dispone de la expresión de la derivada, es mejor utilizar el método tradicional pues de la otra forma el número de evaluaciones de la función se duplica, pudiendo ello hacer que el tiempo de convergencia sea sustancialmente peor.

4.2.4.2 Método de Newton modificado

Este método utiliza como dirección de búsqueda no aquella que determina $f'(x_k)$ en cada iteración k , sino siempre la misma: $f'(x_0)$. La figura 4.9 ilustra la mecánica de esta variante.

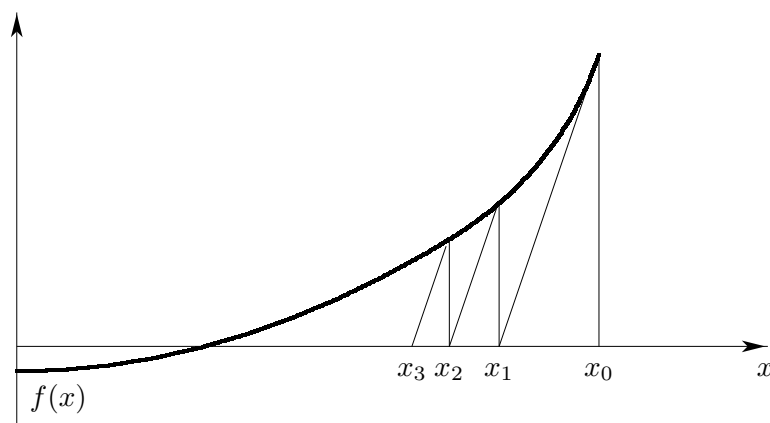


Figura 4.9
Método de Newton modificado

Como se puede intuir, si la pendiente de f en x_0 difiere de una forma apreciable de la de f en la solución, la convergencia puede ser muy lenta o no existir. Para evitar esta dificultad, la derivada de la función se puede reevaluar con una periodicidad fija de iteraciones.

Si se utiliza el método de Newton modificado para resolver $x^3 - \sin(x) = 0$, partiendo de $x_0 = 1$, los puntos del proceso que se obtienen son los de la tabla 4.3.

La convergencia en este caso es razonablemente rápida, aunque un poco peor que la del método de Newton. Éste llegaba a una solución con 6 dígitos significativos en 6 iteraciones, el modificado lo hace en 7.

El código en FORTRAN 77 que se ha utilizado para obtener la solución es el que se lista a continuación. Obsérvese que, a diferencia de la implementación del método de Newton, aquí se ha utilizado precisión doble.

```

PROGRAM Newtonmod
C
  implicit double precision (a-h,o-z)
  parameter (eps=epsilon(1.d0))
C
C *** Resolución de la ecuación x**3-sen(x)=0 ***
C
  x1 = 1.
  dx = 3.0*x1*x1-dcos(x1)
  x2 = x1-fx(x1)/dx
C

```

Tabla 4.3Convergencia del método de Newton modificado aplicado a $x^3 - \sin(x) = 0$

k	x_k
0	1,0000000000000000
1	0,9355493906546692
2	0,9298914189436776
3	0,9288667910316998
4	0,9286723408941727
5	0,9286351316207878
6	0,9286280002218482
7	0,9286266330332931
8	0,9286263709091434
9	0,9286263206528520
10	0,9286263110173409
11	0,9286263091699481
12	0,9286263088157520
13	0,9286263087478429
14	0,9286263087348229
15	0,9286263087323265
16	0,9286263087318479
17	0,9286263087317562
18	0,9286263087317386
19	0,9286263087317352

```

do while (dabs(fx(x2)).gt.eps)
    print *,x1                                ! Salida de resultados
    x1 = x2
    x2 = x1-fx(x1)/dx
end do
C
end

double precision function fx(x)
double precision x
fx = x**3-dsin(x)
return
end

```

4.2.5 Método de la secante

Este nuevo método se basa en utilizar como dirección de búsqueda, en vez de la tangente que define el método de Newton, la que determina una recta secante a la función en dos puntos sucesivos del proceso iterativo. Es decir, si en una iteración k del proceso la ecuación de Newton es

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

la idea es emplear, en vez de $f'(x_k)$,

$$\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

La relación de recurrencia del proceso iterativo queda

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} f(x_k).$$

La figura 4.10 ilustra esta aproximación. Las consideraciones hechas anteriormente en el caso del método por diferencias finitas son válidas para esta aproximación puesto que este método es un caso particular de aquel.

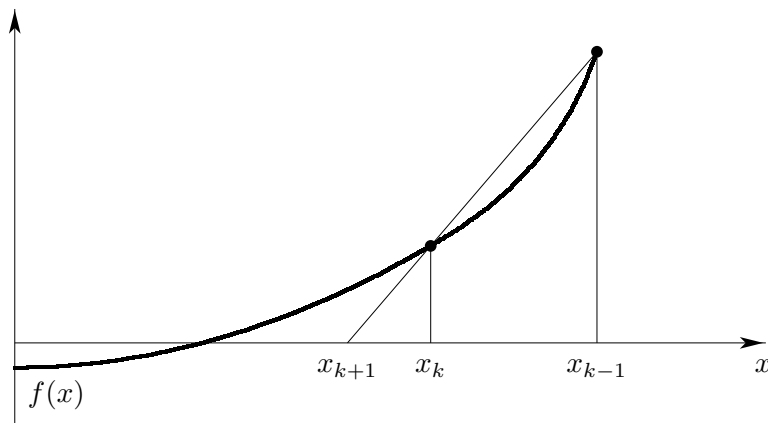


Figura 4.10
Método de la secante

El método de la secante converge superlinealmente siendo el orden $(1 + \sqrt{5})/1 = 1,618$: la denominada *razón áurea*.

Ejemplo 4.2 Resolvamos $x^3 - \sin(x) = 0$ mediante el método de la secante.

El código en FORTRAN 77 que implementa la resolución para este caso del método de la secante es el que sigue.

```

PROGRAM Newtonsecante
C
  implicit double precision (a-h,o-z)
  parameter (eps=epsilon(1.d0))
C
C *** Resolución de la ecuación x**3-sen(x)=0 ***
C
  x0 = 1.1
  x1 = 1.0
  x2 = x1-fx(x1)/secfx(x0,x1)
C
  do while (dabs(fx(x2)).gt.eps)
    x0 = x1
    x1 = x2
    x2 = x1-fx(x1)/secfx(x0,x1)
    print *,x2                                ! Salida de resultados
  end do
C
end

```

```

double precision function fx (x)
double precision x
fx = x**3-dsin(x)
return
end

double precision function secfx (x0,x1)
double precision x0, x1, fx
secfx = (fx(x1)-fx(x0))/(x1-x0)
return
end

```

El proceso de convergencia partiendo de $x_0 = 1,1$ y $x_1 = 1,0$ es el que describe la siguiente tabla.

k	x_k
1	0,9301746079136645
2	0,9286627955397819
3	0,9286263990904702
4	0,9286263087370180
5	0,9286263087317345

4.2.6 Método de la falsa posición

Este método, conocido como *Regula Falsi*, al igual que el de la secante, en vez de la tangente que define el método de Newton, utiliza como dirección de búsqueda una recta secante a la función en dos puntos sucesivos del proceso iterativo. Lo que le diferencia del de la secante es que esa dirección de búsqueda, siguiendo una estrategia similar a la del método de la bisección, la deben determinar los dos últimos puntos del proceso iterativo en los que la función toma valores de signo opuesto.

La figura 4.11 describe esta forma de abordar el problema. La convergencia de este método es también superlineal de orden 1,618: la razón áurea.

En determinadas circunstancias desfavorables, tanto el método de la secante como el de la falsa posición pueden presentar problemas de convergencia. En la figura 4.12 se representa un caso de convergencia lenta.

4.2.7 Método de Müller

Este método, presentado por primera vez por D.E. Müller en 1956, es una generalización del método de la secante. Utiliza una interpolación cuadrática de tres puntos del proceso iterativo que busca la solución para, a partir de las raíces de esa interpolación, definir un nuevo punto del proceso.

La figura 4.13 describe el proceso que sigue el método de Müller en una iteración genérica. Si se consideran los puntos x_0 , x_1 y x_2 , el procedimiento aproxima a estos puntos el polinomio cuadrático

$$p(x) = a(x - x_2)^2 + b(x - x_2) + c$$

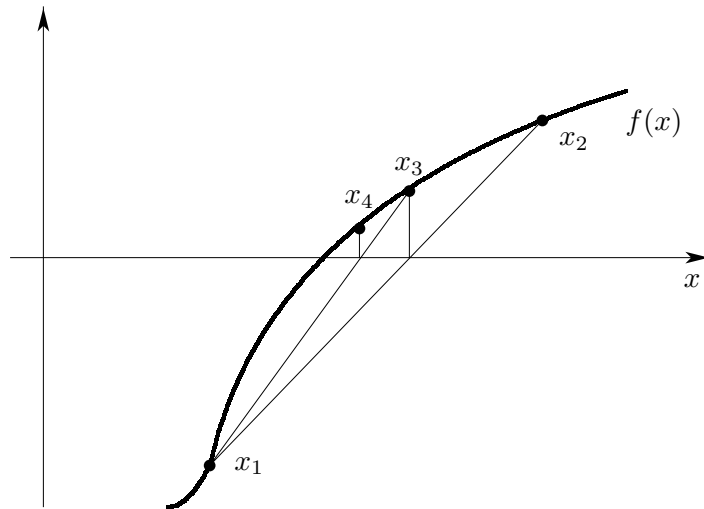


Figura 4.11
Método *Regula Falsi*

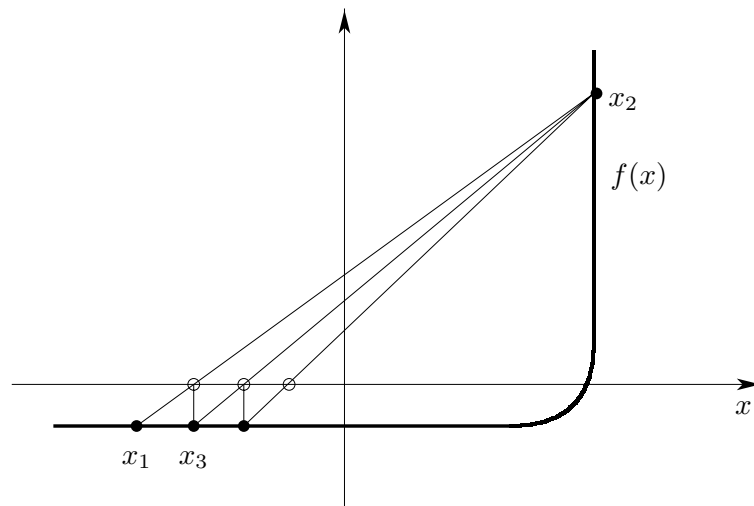
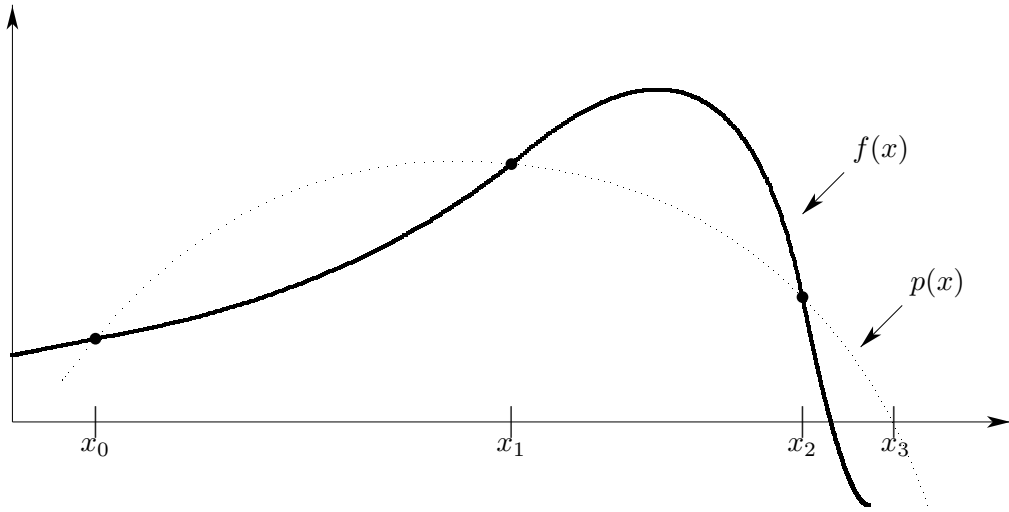


Figura 4.12
Ejemplo donde los métodos de la secante y *regula falsi* convergen muy lentamente

**Figura 4.13**

Primera aproximación parabólica del método de Muller

que pasa por $(x_0, f(x_0))$, $(x_1, f(x_1))$ y $(x_2, f(x_2))$. Los parámetros a , b y c de ese polinomio se determinan a partir de las siguientes condiciones:

$$f(x_0) = a(x_0 - x_2)^2 + b(x_0 - x_2) + c,$$

$$f(x_1) = a(x_1 - x_2)^2 + b(x_1 - x_2) + c$$

y

$$f(x_2) = c.$$

Resolviendo el sistema de 2 ecuaciones con 2 incógnitas

$$\begin{bmatrix} (x_0 - x_2)^2 & (x_0 - x_2) \\ (x_1 - x_2)^2 & (x_1 - x_2) \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} f(x_0) - f(x_2) \\ f(x_1) - f(x_2) \end{bmatrix}$$

se obtiene la expresión de los parámetros a y b . Es

$$\begin{bmatrix} a \\ b \end{bmatrix} = \frac{1}{(x_0 - x_2)(x_1 - x_2)(x_0 - x_1)} \begin{bmatrix} (x_1 - x_2) & -(x_0 - x_2) \\ -(x_1 - x_2)^2 & (x_0 - x_2)^2 \end{bmatrix} \begin{bmatrix} f(x_0) - f(x_2) \\ f(x_1) - f(x_2) \end{bmatrix}.$$

Para determinar el nuevo punto del proceso iterativo, x_3 , se aplica la fórmula cuadrática

$$z = \frac{-2c}{b \pm \sqrt{b^2 - 4ac}}, \quad (4.13)$$

con el fin de calcular las raíces de $p(x)$, escogiéndose para garantizar la estabilidad numérica del método, de las dos posibles, aquella que tiene un menor valor absoluto. Para ello, si $b > 0$ se usa el signo positivo en la expresión (4.13); si $b < 0$, el negativo. El nuevo punto x_3 será entonces

$$x_3 = x_2 + z.$$

Una vez obtenido este punto, el procedimiento se reinicia utilizando como nuevos tres puntos x_3 y, de entre x_0 , x_1 y x_2 , los dos más próximos a él.

Evidentemente, cuando sea necesario, el método deberá aproximar raíces complejas.

A continuación se lista un código en FORTRAN 77 que describe el método de Müller para resolver $x^3 - \sin(x) = 0$. La versión programada sólo calcula raíces reales.

```

      PROGRAM Muller
C
C *** Resolución de la ecuación x**3-sen(x)=0 ***
C
      implicit double precision (a-h,o-z)
      double precision x0/1.5/,x1/1.2/,x2/1.0/
C
      fx0 = fx(x0)
      fx1 = fx(x1)
      fx2 = fx(x2)
C
      eps = epsilon(1.0d0)
      do while (dabs(fx2).gt.eps)
        c = fx2
        d0 = x0-x2
        d1 = x1-x2
        det = d0*d1*(x0-x1)
        b = (d0*d0*(fx1-fx2)-d1*d1*(fx0-fx2))/det
        a = (d1*(fx0-fx2)-d0*(fx1-fx2))/det
        di = 0.
        if (b*b-4*a*c.gt.0) di = dsqrt(b*b-4*a*c)
        z = (-2)*c/(b+dsign(1.0,b)*di)
        x3 = x2+z
        if (dabs(x3-x1).lt.dabs(x3-x0)) then ! Escoger como nuevos
          u = x1 ! x0, x1 y x2 los
          x1 = x0 ! más próximos a
          x0 = u ! x3.
          u = fx1
          fx1 = fx0
          fx0 = u
        endif
        if (dabs(x3-x2).lt.dabs(x3-x1)) then
          u = x2
          x1 = u
          u = fx2
          fx1 = u
        endif
        x2 = x3
        fx2 = fx(x2)
        print *,x2,fx2
      end do
C
      end

      double precision function fx (x)
      double precision x
      fx = x**3-dsin(x)
      return
      end

```

El proceso de convergencia de la resolución del problema partiendo de $x_0 = 1,5$, $x_1 = 1,2$ y $x_2 = 1,0$ es el que describe la siguiente tabla. Obsérvese que el número de iteraciones, para la

precisión que se obtiene, decrece apreciablemente comparándolo con el de otros métodos.

k	x_k	$f(x_k)$
1	0,9218014997385994	-1,342038198649549E-002
2	0,9286993319308728	1,451948415762639E-004
3	0,9286263283651378	3,903328569669932E-008
4	0,9286263087317398	1,066665202345551E-014
5	0,9286263087317345	7,199102425303749E-017

4.3 Sistemas de ecuaciones no lineales. Método de Newton-Raphson

El método de Newton-Raphson para sistemas de ecuaciones no lineales es una generalización del analizado para el caso de una variable. Estudiaremos funciones vectoriales $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ cuando $n = m$.

Recurriendo a la forma tradicional de introducirlo, si se supone que $\mathbf{f} \in C^1$ y en un punto $\mathbf{x}_k$ de un proceso iterativo tendente a resolver $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ se aproxima la función mediante el modelo, $M_k(\mathbf{x}_k)$, que define el desarrollo en serie de Taylor alrededor de ese punto, truncándolo a partir de los términos de segundo orden, se tiene que

$$M_k(\mathbf{x}_k) = \mathbf{f}(\mathbf{x}_k) + J(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k),$$

donde $J(\mathbf{x}_k)$ es la matriz Jacobiana del sistema en $\mathbf{x}_k$:

$$J(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_n(\mathbf{x})}{\partial x_n} \end{bmatrix}_{\mathbf{x}=\mathbf{x}_k}.$$

Si se utiliza esa aproximación lineal de la función y se resuelve el sistema de ecuaciones lineales que define

$$\mathbf{f}(\mathbf{x}_k) + J(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k) = \mathbf{0},$$

su solución

$$\mathbf{x} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k)$$

determinará un nuevo punto del proceso iterativo.

La relación de recurrencia del método de Newton-Raphson para sistemas de ecuaciones no lineales es pues

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k). \quad (4.14)$$

El *paso* de Newton es $\mathbf{x}_{k+1} - \mathbf{x}_k$: una aproximación de $\mathbf{x}^* - \mathbf{x}_k$.

Volviendo a considerar las ideas que se expusieron al analizar el caso de una sola variable, en el método de Newton-Raphson para sistemas de ecuaciones no lineales cada ecuación, $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$, se reemplaza o aproxima por el hiperplano tangente en $\mathbf{x}_k$ a la curva que define esa

f_i . La solución del sistema de ecuaciones lineales de la expresión (4.14) determina el punto de intersección de todos los hiperplanos resultantes.

El algoritmo de Newton-Raphson para resolver sistemas de ecuaciones no lineales es el que describe la tabla 4.4. El paso 1 de este algoritmo comporta la resolución de un sistema de ecuaciones lineales $n \times n$. Ni que decir tiene que todas las consideraciones que hacíamos al hablar de los métodos para resolver sistemas lineales de ecuaciones referentes a estabilidad numérica, condicionamiento, etc, tienen, si cabe, una mayor trascendencia aquí puesto que de su buen tratamiento o toma en consideración depende que el procedimiento funcione adecuadamente.

Tabla 4.4

Algoritmo de Newton-Raphson para sistemas de ecuaciones no lineales

Paso 0 – Definir un $\mathbf{x}_0 \in \mathbb{R}^n$; hacer $k = 1$ y $\mathbf{x}_k \leftarrow \mathbf{x}_0$.
Paso 1 – Determinar la solución de $J(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k) = -\mathbf{f}(\mathbf{x}_k)$.
Paso 2 – Si $\|\mathbf{f}(\mathbf{x}_{k+1})\|_2 < Tol$, parar: el problema está resuelto.
 Si no, hacer $k = k + 1$, $\mathbf{x}_k = \mathbf{x}_{k+1}$ e ir al paso 1.

Ejemplo 4.3 Resolvamos, utilizando el método de Newton y partiendo del punto $[1, 1, 1]^T$, el sistema de ecuaciones no lineales

$$\begin{aligned} 3x_1 - \cos(x_2x_3) - \frac{1}{2} &= 0 \\ x_1^2 - 81\left(x_2 + \frac{1}{10}\right)^2 + \sin(x_3) + 1,06 &= 0 \\ e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0. \end{aligned}$$

A continuación se lista un código en FORTRAN 77 que implementa el método de Newton que acabamos de presentar, particularizado para este problema. La solución de los sistemas de ecuaciones lineales de cada iteración se realiza mediante la eliminación de Gauss de acuerdo con el algoritmo propuesto en la página 832.

```

      PROGRAM Newtrp
C
      parameter (n=3)
      double precision f(n),j(n,n),x(n),x1(n),s(n),tol,dnor,dnr
C
      tol = dsqrt(epsilon(1.0d0))
      x  = 1.0
      call fx (f,x,n)
      dnr = dnor(f,n)
C
C *** Proceso iterativo ***
C
      do while (dnr.ge.tol)
         call derfx (j,x,n)
         call gauss (j,f,s,n)

```

```
        x1 = x-s
        call fx (f,x1,n)
        dnr = dnor(f,n)
        print *,x1,dnr          ! Salida de resultados
        x = x1
    end do
C
    end

    subroutine fx (f,x,n)
    double precision f(n),x(n)
C
    f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
    f(2) = x(1)**2-81*(x(2)+0.1)**2+dsin(x(3))+1.06
    f(3) = dexp((-x(1)*x(2)))+20*x(3)+(10*dacos(-1.0d0)-3)/3
C
    return
    end

    subroutine derfx (j,x,n)
    double precision j(n,n),x(n)
C
    j(1,1) = 3.0
    j(1,2) = dsin(x(2)*x(3))*x(3)
    j(1,3) = dsin(x(2)*x(3))*x(2)
    j(2,1) = 2.0*x(1)
    j(2,2) = -162.0*(x(2)+0.1)
    j(2,3) = dcos(x(3))
    j(3,1) = -dexp((-x(1)*x(2)))*x(2)
    j(3,2) = -dexp((-x(1)*x(2)))*x(1)
    j(3,3) = 20.0
C
    return
    end
c
    double precision function dnor (x,n)
    double precision x(n)
C
    dnor = 0.d0
    do i = 1,n
        dnor = dnor+x(i)**2
    end do
C
    dnor = dsqrt(dnor)
    return
    end

    subroutine gauss(a,b,x,n)
C
C *** Resolución del sistema lineal mediante eliminación de Gauss
C
    integer ipvt(10),pi
    double precision a(n,n),b(n),x(n),smax,r,r1,c
C
    do i = 1,n
        ipvt(i) = i
    end do
C
C *** Triangularización ***
```

```

C
do k = 1,n-1
  l = 0
  smax = dabs(a(ipvt(k),k))
  do i = k+1,n
    ip = ipvt(i)
    if (dabs(a(ip,k)).gt.smax) then
      l = i
      smax = dabs(a(ip,k))
    endif
  end do
  if (l.ne.0) then
   iaux = ipvt(k)
    ipvt(k) = ipvt(l)
    ipvt(l) = iaux
  endif
  pi = ipvt(k)
  r1 = 1.0/a(pi,k)
  do i = k+1,n
    ip = ipvt(i)
    r = a(ip,k)*r1
    do j = k+1,n
      a(ip,j) = a(ip,j)-r*a(pi,j)
    end do
    a(ip,k) = -r
  end do
end do

C
do k = 1,n-1
  ip = ipvt(k)
  do i = k+1,n
    pi = ipvt(i)
    b(pi) = b(pi)+a(pi,k)*b(ip)
  end do
end do

C
C *** Sustitución inversa ***
C
x(n) = b(ipvt(n))/a(ipvt(n),n)
do i = n-1,1,-1
  pi = ipvt(i)
  c = b(pi)
  do j = i+1,n
    c = c-a(pi,j)*x(j)
  end do
  x(i) = c/a(pi,i)
end do

C
return
end

```

El proceso de convergencia es el que se describe en la tabla 4.5.



Tabla 4.5

Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton-Raphson

k	x_1	x_2	x_3	$\ f(x_k)\ _2$
1	0,919687213390398	0,46082245461787	-0,503387635514082	24,087256
2	0,501000485375849	0,18743347575308	-0,520869233062834	5,878800
3	0,500542935515392	6,11534507694258E-2	-0,522000964266343	1,291680
4	0,500104436279313	1,16171016280123E-2	-0,523295146222630	1,987617E-01
5	0,500005510372763	6,05610802953016E-4	-0,523582936446523	9,821480E-03
6	0,500000016655606	1,82133920264085E-6	-0,523598727952539	2,952947E-05
7	0,5000000000000152	-5,01225376366101E-9	-0,523598775723585	2,701800E-10

4.3.1 Convergencia del método de Newton para sistemas de ecuaciones no lineales

La forma de probar la convergencia del método para sistemas de ecuaciones es muy similar a la empleada para hacerlo en el caso de una sola variable.

Lema 4.2 Sea la función $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$, continua y diferenciable en un conjunto convexo abierto $D \subset \mathbb{R}^n$. Para todo x y $x + p \in D$,

$$f(x + p) - f(x) = \int_0^1 J(x + tp)p \, dt \equiv \int_x^{x+p} f'(z) \, dz.$$

Lema 4.3 Sea la función $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$, continua y diferenciable en un conjunto convexo abierto $D \subset \mathbb{R}^n$, $x \in D$ y $J \in \text{Lip}_\gamma(S(x, r))$, usando una norma vectorial y una norma matricial inducida y la constante γ . Entonces, para todo $x + p \in D$,

$$\|f(x + p) - f(x) - J(x)p\| \leq \frac{\gamma}{2} \|p\|^2.$$

DEMOSTRACIÓN. De acuerdo con el lema anterior,

$$\begin{aligned} f(x + p) - f(x) - J(x)p &= \int_0^1 J(x + tp)p \, dt - J(x)p \\ &= \int_0^1 (J(x + tp) - J(x))p \, dt. \end{aligned}$$

Usando la definición de norma matricial inducida y la continuidad Lipschitz de J en $S(\mathbf{x}, r)$, se tiene que

$$\begin{aligned}\|\mathbf{f}(\mathbf{x} + \mathbf{p}) - \mathbf{f}(\mathbf{x}) - J(\mathbf{x})\mathbf{p}\| &\leq \int_0^1 \|J(\mathbf{x} + t\mathbf{p}) - J(\mathbf{x})\| \|\mathbf{p}\| dt \\ &\leq \int_0^1 \gamma \|t\mathbf{p}\| \|\mathbf{p}\| dt \\ &= \gamma \|\mathbf{p}\|^2 \int_0^1 t dt \\ &= \frac{\gamma}{2} \|\mathbf{p}\|^2.\end{aligned}$$

■

Teorema 4.5 Sea la función $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, continua y diferenciable en un conjunto convexo abierto $D \subset \mathbb{R}^n$. Supóngase que existe un $\mathbf{x}^* \in \mathbb{R}^n$ y $r, \beta > 0$ tales que la bola abierta $S(\mathbf{x}^*, r) \subset D$, que $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ y que $J(\mathbf{x}^*)^{-1}$ existe con $\|J(\mathbf{x}^*)^{-1}\| \leq \beta$ y $J \in \text{Lip}_\gamma(S(\mathbf{x}^*, r))$. Existe entonces un $\varepsilon > 0$ tal que para todo $\mathbf{x}_0 \in S(\mathbf{x}^*, r)$, la sucesión $\mathbf{x}_1, \mathbf{x}_2, \dots$ generada por

$$\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1} \mathbf{f}(\mathbf{x}_k), \quad k = 0, 1, \dots$$

converge a $\mathbf{x}^*$ verificándose que

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq \beta\gamma \|\mathbf{x}_k - \mathbf{x}^*\|^2, \quad k = 0, 1, \dots \quad (4.15)$$

DEMOSTRACIÓN. Escojamos un ε de tal manera que la matriz J sea regular para todo $\mathbf{x} \in S(\mathbf{x}^*, r)$ y probemos entonces que, dado que el error que produce el modelo lineal

$$M_k(\mathbf{x}_k) = \mathbf{f}(\mathbf{x}_k) + J(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k)$$

es $O(\|\mathbf{x}_k - \mathbf{x}^*\|^2)$, la convergencia es cuadrática.

Sea

$$\varepsilon = \min \left\{ r, \frac{1}{2\beta\gamma} \right\}. \quad (4.16)$$

Esboceemos la prueba, por inducción en k , de que se cumple (4.15) y, también, que

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq \frac{1}{2} \|\mathbf{x}_k - \mathbf{x}^*\|$$

por lo que

$$\mathbf{x}_{k+1} \in S(\mathbf{x}^*, \varepsilon).$$

Primero comprobemos que $J(\mathbf{x}_0)$ es regular. De $\|\mathbf{x}_0 - \mathbf{x}^*\| \leq \varepsilon$, la continuidad Lipschitz de J en $\mathbf{x}^*$ y (4.16), se tiene que

$$\begin{aligned}\|J(\mathbf{x}^*)^{-1} (J(\mathbf{x}_0) - J(\mathbf{x}^*))\| &\leq \|J(\mathbf{x}^*)^{-1}\| \|J(\mathbf{x}_0) - J(\mathbf{x}^*)\| \\ &\leq \beta\gamma \|\mathbf{x}_0 - \mathbf{x}^*\| \leq \beta\gamma\varepsilon \leq \frac{1}{2}.\end{aligned}$$

De aquí y de la continuidad de la norma de matrices se deduce que $J(\mathbf{x}_0)$ es regular, pudiendo deducirse que la norma de su inversa cumple que

$$\begin{aligned}\|J(\mathbf{x}_0)^{-1}\| &\leq \frac{\|J(\mathbf{x}^*)^{-1}\|}{1 - \|J(\mathbf{x}^*)^{-1}(J(\mathbf{x}_0) - J(\mathbf{x}^*))\|} \\ &\leq 2\|J(\mathbf{x}^*)^{-1}\| \\ &\leq 2\beta.\end{aligned}\tag{4.17}$$

Es decir, $\mathbf{x}_1$ está definido, cumpliéndose que

$$\begin{aligned}\mathbf{x}_1 - \mathbf{x}^* &= \mathbf{x}_0 - \mathbf{x}^* - J(\mathbf{x}_0)^{-1}\mathbf{f}(\mathbf{x}_0) \\ &= \mathbf{x}_0 - \mathbf{x}^* - J(\mathbf{x}_0)^{-1}(\mathbf{f}(\mathbf{x}_0) - \mathbf{f}(\mathbf{x}^*)) \\ &= J(\mathbf{x}_0)^{-1}[\mathbf{f}(\mathbf{x}^*) - \mathbf{f}(\mathbf{x}_0) - J(\mathbf{x}_0)(\mathbf{x}^* - \mathbf{x}_0)].\end{aligned}$$

Obsérvese que el término entre corchetes es la diferencia entre $\mathbf{f}(\mathbf{x}^*)$ y el modelo $M_0(\mathbf{x}^*)$. En consecuencia, de acuerdo con el lema 4.3 y la ecuación (4.17), se tiene que

$$\begin{aligned}\|\mathbf{x}_1 - \mathbf{x}^*\| &\leq \|J(\mathbf{x}_0)^{-1}\| \|\mathbf{f}(\mathbf{x}^*) - \mathbf{f}(\mathbf{x}_0) - J(\mathbf{x}_0)(\mathbf{x}^* - \mathbf{x}_0)\| \\ &\leq 2\beta\frac{\gamma}{2}\|\mathbf{x}_0 - \mathbf{x}^*\|^2 \\ &= \beta\gamma\|\mathbf{x}_0 - \mathbf{x}^*\|^2.\end{aligned}$$

Lo que prueba (4.15). Como $\|\mathbf{x}_0 - \mathbf{x}^*\| \leq 1/2\beta\gamma$, entonces $\|\mathbf{x}_1 - \mathbf{x}^*\| \leq 1/2\|\mathbf{x}_0 - \mathbf{x}^*\|$, lo que prueba que $\mathbf{x}_1 \in S(\mathbf{x}^*, \epsilon)$, completándose el caso de $k = 0$. Para probar los demás pasos de la inducción se procede de forma idéntica. ■

Las constantes γ y β se pueden combinar en una sola $\gamma_{rel} = \gamma\beta$, siendo ésta entonces una constante de Lipschitz que mide la no linealidad relativa de $\mathbf{f}$ en $\mathbf{x}^*$, pues

$$\begin{aligned}\|J(\mathbf{x}^*)^{-1}(J(\mathbf{x}) - J(\mathbf{x}^*))\| &\leq \|J(\mathbf{x}^*)^{-1}\| \|J(\mathbf{x}) - J(\mathbf{x}^*)\| \\ &\leq \beta\gamma\|\mathbf{x} - \mathbf{x}^*\| \\ &= \gamma_{rel}\|\mathbf{x} - \mathbf{x}^*\|,\end{aligned}$$

para $\mathbf{x} \in S(\mathbf{x}^*, r)$. El último teorema viene a decir que el orden de convergencia del método de Newton es inversamente proporcional a la no linealidad relativa de $\mathbf{f}$ en $\mathbf{x}^*$.

4.3.2 Modificaciones del método de Newton para sistemas de ecuaciones no lineales

Existen diversas variantes del método de Newton; difieren unas de otras en la forma de resolver el sistema de ecuaciones lineales inherente al mismo. El objetivo de todas ellas es reducir al máximo la duración de esa fase del algoritmo simplificando la factorización de la matriz Jacobiana o el proceso de eliminación de Gauss correspondiente.

4.3.2.1 El método de Newton-Raphson por diferencias finitas para sistemas de ecuaciones no lineales

Es ésta una variante del método de Newton para cuando no se conoce, o no se desea calcular, la expresión analítica de la matriz Jacobiana del sistema. Ésta se reemplaza por su aproximación en diferencias finitas, siguiendo el mismo principio que veíamos en el apartado 4.2.4.1.

Teorema 4.6 Sea la función $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, continua y diferenciable en un conjunto convexo abierto $D \subset \mathbb{R}^n$. Supóngase que existe un $\mathbf{x}^* \in \mathbb{R}^n$ tal que $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ y unos $\varepsilon > 0$ y $h > 0$ tales que si $\{h_k\}$ es una sucesión de números reales $0 < |h_k| \leq h$ y $\mathbf{x}_0 \in S(\mathbf{x}^*, \varepsilon)$, la sucesión generada por

$$(\mathbf{a}_k)_j = \frac{\mathbf{f}(\mathbf{x}_k + h_k \mathbf{e}_j) - \mathbf{f}(\mathbf{x}_k)}{h_k}, \quad j = 1, \dots, n,$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - A_k^{-1} \mathbf{f}(\mathbf{x}_k), \quad k = 0, 1, \dots,$$

converge linealmente a $\mathbf{x}^*$. Si

$$\lim_{k \rightarrow \infty} h_k = 0,$$

la convergencia es superlineal. Si existe una constante c_1 tal que

$$|h_k| \leq c_1 \|\mathbf{x}_k - \mathbf{x}^*\|,$$

o, equivalentemente, una constante c_2 tal que

$$|h_k| \leq c_2 \|\mathbf{f}(\mathbf{x}_k)\|,$$

entonces la convergencia es cuadrática.

DEMOSTRACIÓN. Se efectúa de una forma muy similar a como se hizo la del teorema 4.5 combinándola con la del teorema 4.4. ■

Para escoger el parámetro h se pueden seguir las directrices dictadas en el apartado 4.2.4.1. Una forma razonable de hacerlo, cuando $\mathbf{f}(\mathbf{x})$ se puede calcular con t dígitos correctos, consiste en tomar un h tal que $\mathbf{f}(\mathbf{x} + h\mathbf{e}_j)$ difiera de $\mathbf{f}(\mathbf{x})$ en la mitad menos significativa de esos t dígitos. Más concretamente, si el error relativo del cálculo de $\mathbf{f}(\mathbf{x})$ es η , habría que conseguir que

$$\frac{\|\mathbf{f}(\mathbf{x} + h\mathbf{e}_j) - \mathbf{f}(\mathbf{x})\|}{\|\mathbf{f}(\mathbf{x})\|} \leq \sqrt{\eta}, \quad j = 1, \dots, n.$$

La forma más adecuada de conseguir este objetivo sería modificar cada componente x_j del vector $\mathbf{x}$ por separado, de acuerdo con la fórmula

$$h_j = \sqrt{\eta} x_j,$$

y luego calcular cada columna $\mathbf{a}_j$ de la matriz Jacobiana aproximada mediante la expresión

$$\mathbf{a}_j = \frac{\mathbf{f}(\mathbf{x} + h_j \mathbf{e}_j) - \mathbf{f}(\mathbf{x})}{h_j}.$$

Cuando el problema está bien escalado, el parámetro h puede elegirse igual a $\sqrt{\epsilon}$ para todos los x_j . Es importante tener siempre en cuenta que si el valor de los componentes del vector $\mathbf{x}$ difieren mucho unos de otros, si se elige un mismo h para todos, el resultado puede ser un desastre.

Ejemplo 4.4 Partiendo desde el punto $[1, 1, 1]^T$, resolvamos mediante el método de Newton-Raphson por diferencias finitas el problema del ejemplo 4.3.

La versión en FORTRAN 77 de un programa para implementar el procedimiento particularizándolo para este ejemplo es la que sigue.

```

PROGRAM Newtrpdf
C
  parameter (n = 3)
  integer ipvt(n)
  double precision f(n), j(n,n), x(n), f1(n), s(n)
  double precision tol, dnor, dnr, h
C
  tol = dsqrt(epsilon(1.0d0))
  h = tol
  x = 1.0
  call fx (f,x,n)
  dnr = dnor(f,n)
C
C *** Proceso iterativo ***
C
  do while (dnr.gt.tol)
    call derfxdf (j,x,n,f,f1,h)
    call gauss (j,f,s,ipvt,n)
    x = x-s
    call fx (f,x,n)
    dnr = dnor(f,n)
    print *,x,dnr                                ! Salida de resultados
  end do
C
  end

  subroutine fx (f,x,n)
  double precision f(n), x(n)
C
  f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
  f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
  f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+(10.0*dacos(-1.0D0)-3.0)/3.0
C
  return
  end

  subroutine derfxdf (j,x,n,f,f1,h)
  double precision j(n,n), x(n), f(n), f1(n), h
C
  do i = 1,n
    x(i) = x(i)+h
    call fx (f1,x,n)
    do k = 1,n
      j(k,i) = (f1(k)-f(k))/h
    end do
    x(i) = x(i)-h
  end do
C

```

```

        return
    end

    double precision function dnor (x,n)
    double precision x(n)
C
    dnor = 0.d0
    do i = 1,n
        dnor = dnor+x(i)*x(i)
    end do
C
    dnor = dsqrt(dnor)
    return
    end

    subroutine gauss (a,b,x,ipvt,n)
C
C *** Resolución del sistema lineal mediante eliminación de Gauss ***
C
    integer ipvt(n),pi,i,k,l,ip,iaux
    double precision a(n,n),b(n),x(n),smax,r,r1,c
C
    do i = 1,n
        ipvt(i) = i
    end do
C
C *** Triangularización ***
C
    do k = 1,n-1
        l = 0
        smax = dabs(a(ipvt(k),k))
        do i = k+1,n
            ip = ipvt(i)
            if (dabs(a(ip,k)).gt.smax) then
                l = i
                smax = dabs(a(ip,k))
            endif
        end do
        if (l.ne.0) then
            iaux = ipvt(k)
            ipvt(k) = ipvt(l)
            ipvt(l) = iaux
        endif
        pi = ipvt(k)
        r1 = 1.0/a(pi,k)
        do i = k+1,n
            ip = ipvt(i)
            r = a(ip,k)*r1
            do j = k+1,n
                a(ip,j) = a(ip,j)-r*a(pi,j)
            end do
            a(ip,k) = -r
        end do
    end do
C
    do k = 1,n-1
        ip = ipvt(k)
        do i = k+1,n
            pi = ipvt(i)

```

```

        b(pi) = b(pi)+a(pi,k)*b(ip)
    end do
end do
C
C *** Sustitución inversa ***
C
    x(n) = b(ipvt(n))/a(ipvt(n),n)
    do i = n-1,1,-1
        pi = ipvt(i)
        c = b(pi)
        do j = i+1,n
            c = c-a(pi,j)*x(j)
        end do
        x(i) = c/a(pi,i)
    end do
C
    return
end

```

El proceso de convergencia que resulta de la ejecución de este código es el que describe la tabla 4.6. ●

Tabla 4.6

Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton-Raphson por diferencias finitas

k	x_1	x_2	x_3	$\ f(x_k)\ _2$
1	9,196872128331276E-1	4,608224593269510E-1	-5,033876338748984E-1	24,087257011001890
2	5,010004854055197E-1	1,874334808918119E-1	-5,208692328288063E-1	5,878800956736615
3	5,005429355625435E-1	6,115345636598238E-2	-5,220009641201370E-1	1,291680877269570
4	5,001044363153647E-1	1,161710564942230E-2	-5,232951461173050E-1	1,987617768874427E-1
5	5,000055103830903E-1	6,056119366203792E-4	-5,235829364168181E-1	9,821499305243949E-3
6	5,000000166560779E-1	1,821390815254557E-6	-5,235987279511862E-1	2,953031350981317E-5
7	5,000000000001537E-1	-5,012116500463980E-9	-5,235987757235823E-1	2,723994307877944E-10

4.3.2.2 Newton modificado

Esta variante resulta de considerar la misma matriz Jacobiana, $J(x_0)$, durante todo el proceso iterativo o, al menos, durante un número fijo de iteraciones. Se la conoce, como en el caso de una variable, como *método de Newton modificado*.

4.3.2.3 Jacobi

Esta otra variante surge de aproximar la matriz Jacobiana sólo por los elementos de su diagonal principal.

El esquema iterativo lo define la relación de recurrencia

$$x_{k+1} = x_k - D_k^{-1} f(x_k), \quad (4.18)$$

donde $d_{ii}^k = J_{ii}^k$. A esta forma de aproximar el procedimiento de Newton se la conoce como *esquema Jacobi*, debido a la semejanza de (4.18) con la relación de recurrencia del método de Jacobi para la resolución iterativa de sistemas de ecuaciones lineales.

Si los elementos que no están en la diagonal principal de la matriz J son pequeños comparados con los de la diagonal principal esta estrategia puede resultar muy buena.

El siguiente código en FORTRAN 77 resuelve el problema del ejemplo 4.3 con esta variante del método de Newton-Raphson.

```

PROGRAM Newjac
C
  parameter (n=3)
  double precision f(n),j(n),x(n),x1(n),tol,dnor,dnr
C
  tol = dsqrt(epsilon(1.0d0))
  x = 1.0
  call fx (f,x,n)
  dnr = dnor(f,n)
C
  do while (dnr.gt.tol)
    call derfx (j,x,n)
    x1 = x-f/j
    call fx (f,x1,n)
    dnr = dnor(f,n)
    print *,x1,dnr
    x = x1
  end do
C
  end

  subroutine fx (f,x,n)
  double precision f(n),x(n)
C
  f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
  f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
  f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+(10.0*dacos(-1.0D0)-3.0)/3.0
C
  return
  end

  subroutine derfx (j,x,n)
  double precision j(n),x(n)
C
  j(1) = 3.0
  j(2) = -162.0*(x(2)+0.1)
  j(3) = 20.0
C
  return
  end

  double precision function dnor (x,n)
  double precision x(n)
C
  dnor = 0.D0
  do i = 1,n
    dnor = dnor+x(i)**2
  end do
C
  dnor = dsqrt(dnor)

```

```

return
end

```

El proceso de convergencia que resulta de la ejecución de este código es el que describe la tabla 4.7.

Tabla 4.7

Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton, variante Jacobi

k	x_1	x_2	x_3	$\ \mathbf{f}(\mathbf{x}_k)\ _2$
1	3,4676743528937E-1	4,6628210320363E-1	-4,9199274765687E-1	25,275175
2	4,9126710684223E-1	1,9085722828697E-1	-5,1613395807175E-1	6,044943
3	4,9838400645741E-1	6,2572952480534E-2	-5,1912375290199E-1	1,329867
4	4,9982415676754E-1	1,2127977333769E-2	-5,2206357004560E-1	2,086958E-1
5	4,9999331854835E-1	7,1943457186973E-4	-5,2329659959512E-1	1,277583E-2
6	4,999997637742E-1	1,8194817561808E-5	-5,2358079320878E-1	4,482211E-4
7	4,999999998487E-1	9,5631482748073E-7	-5,2359832072995E-1	1,745616E-5
8	4,999999999995E-1	1,9297729277228E-8	-5,2359875169043E-1	5,990302E-7
9	5,000000000000E-1	-3,7441679487420E-9	-5,2359877511585E-1	2,333013E-8
10	5,000000000000E-1	-4,9964526465087E-9	-5,2359877569190E-1	8,005792E-10

4.3.2.4 Gauss-Seidel

Esta variante aproxima la matriz Jacobiana mediante la que resulta de tener en cuenta sólo los elementos de su parte triangular inferior (incluidos los elementos de la diagonal principal).

El esquema iterativo queda definido por la relación de recurrencia

$$\mathbf{x}_{k+1} = \mathbf{x}_k - L_k^{-1} \mathbf{f}(\mathbf{x}_k), \quad (4.19)$$

donde $L_{ij}^k = J_{ij}^k$, $i \geq j$. A esta variante se la conoce como *esquema Gauss-Seidel*. Como se puede observar, para resolver la ecuación (4.19) sólo es necesario realizar una sustitución directa; muchas menos operaciones, por tanto, que las propias de la factorización y posterior sustitución inversa del método de Newton tradicional.

4.3.2.5 Relajación SOR

El esquema iterativo en forma matricial que se utiliza en este caso es

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (\rho D_k + L_k)^{-1} \mathbf{f}(\mathbf{x}_k).$$

El parámetro de relajación es $\omega = 1/(\rho + 1)$. La convergencia de este esquema iterativo y la del de Gauss-Seidel dependen de diversos factores. A aquel lector interesado en su estudio le aconsejamos consultar las referencias citadas al final del capítulo; en particular las de Hager [1988] y Ortega y Rheinboldt [1970].

A continuación se lista la implementación en FORTRAN 77 del método de relajación para resolver el ejemplo anterior. El parámetro ω se puede variar a voluntad.

```

PROGRAM Newsor
C
parameter (n=3)
double precision f(n),j(n,n),x(n),x1(n),s(n),tol,dnor,dnr,omega,ro
C
tol = dsqrt(epsilon(1.0d0))
x = 1.0
print '(A\)', ' Valor de OMEGA --->'
read '(bn,f9.0)', omega
ro = (1-omega)/omega
call fx (f,x,n)
dnr = dnor(f,n)
C
do while (dnr.gt.tol)
    call derfx (j,x,n,ro)
    call sustdi (j,f,s,n)
    x1 = x-s
    call fx (f,x1,n)
    dnr = dnor(f,n)
    print *,x1,dnr
    x = x1
end do
C
end

subroutine fx (f,x,n)
double precision f(n),x(n)
C
f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+(10.0*dacos(-1.0D0)-3.0)/3.0
C
return
end

subroutine derfx (j,x,n,ro)
double precision j(n,n),x(n),ro
C
j(1,1) = 3.0*(1.0+ro)
j(2,1) = 2.0*x(1)
j(2,2) = -162.0*(x(2)+0.1)*(1.0+ro)
j(3,1) = -dexp((-x(1)*x(2)))*x(2)
j(3,2) = -dexp((-x(1)*x(2)))*x(1)
j(3,3) = 20.0*(1.0+ro)
C
return
end

double precision function dnor (x,n)
double precision x(n)
C
dnor = 0.D0
do i = 1,n
    dnor = dnor+x(i)**2
end do
C
dnor = dsqrt(dnor)
return
end

```

```

      subroutine sustdi (a,b,x,n)
      double precision a(n,n),b(n),x(n),c
C
C *** Sustitución directa ***
C
      x(1) = b(1)/a(1,1)
      do i = 2,n
        c = b(i)
        do j = 1,i-1
          c = c-a(i,j)*x(j)
        end do
        x(i) = c/a(i,i)
      end do
C
      return
      end

```

El proceso de convergencia que resulta de la ejecución de este código con $\omega = 1,001$ es el que describe la tabla 4.8.

Tabla 4.8

Proceso de convergencia del problema del ejemplo 4.3 mediante el método de Newton, variante SOR

k	x_1	x_2	x_3	$\ f(x_k)\ _2$
1	3,461142027246693E-1	4,584022609276468E-1	-5,154964208593540E-1	24,573867038246120
2	4,908811949155542E-1	1,876309366498080E-1	-5,174321885631722E-1	5,895091340346604
3	4,984378248196516E-1	6,115261671271569E-2	-5,219702905590197E-1	1,293741548952080
4	4,998315939999299E-1	1,160081481603464E-2	-5,232927679798938E-1	1,987384037123210E-1
5	4,999940202065573E-1	6,062869047825676E-4	-5,235830701687643E-1	9,844104464408518E-3
6	4,999999891681415E-1	2,052083782429779E-6	-5,235987374862628E-1	3,330434571219442E-5
7	5,000000000106393E-1	-5,018110351151697E-9	-5,235987757619615E-1	7,900883617860966E-10

4.4 Métodos cuasi Newton

El objetivo de los métodos que se agrupan bajo esta denominación consiste en aproximar la matriz Jacobiana de cada iteración mediante fórmulas de recurrencia que la relacionen con el valor que toma en iteraciones precedentes. Se pueden interpretar, en cierta medida, como la extensión de la idea del método de la secante a n dimensiones.

Para explicarlos de forma simple, supongamos que $f(x) = 0$ es un sistema de ecuaciones lineales como, por ejemplo, $Ax - b = 0$. Si se restan los valores de $f(x)$ en dos puntos sucesivos del proceso iterativo, $k-1$ y k , se tiene que

$$f(x_k) - f(x_{k-1}) = A(x_k - x_{k-1}).$$

En el caso no lineal esta igualdad no se cumple aunque puede hacerse, eligiendo A_k adecuadamente, que

$$A_k(x_k - x_{k-1}) \approx f(x_k) - f(x_{k-1}).^6 \quad (4.20)$$

⁶Recordemos la aproximación en el de una variable de $f'(x_k)$ por $\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$.

Cuando la dimensión del sistema, n , es mayor que 1, la matriz A_k no está determinada: la expresión (4.20), en el caso de hacerse igualdad, es un sistema de n ecuaciones con n^2 incógnitas.

Los métodos cuasi Newton construyen una sucesión $\{A_k\}$ de tal forma que A_k aproxime lo mejor posible la matriz Jacobiana $J(\mathbf{x}_k)$.

4.4.1 Método de Broyden

Broyden [1965] utilizó una idea muy simple para obtener una aproximación satisfactoria de $J(\mathbf{x}_k)$: escogerla de tal forma que se minimice el valor de la función que se obtendría en un mismo punto mediante las dos aproximaciones A_k y A_{k-1} y que se cumpla a la vez que $A_k(\mathbf{x}_k - \mathbf{x}_{k-1}) = \mathbf{f}(\mathbf{x}_k) - \mathbf{f}(\mathbf{x}_{k-1})$.

Con la aproximación mencionada, partiendo de $\mathbf{x}_k$ y $\mathbf{x}_{k-1}$, la diferencia de los valores de la función en un punto $\mathbf{x} \in \mathbb{R}^n$ que habría que minimizar sería

$$\mathbf{f}(\mathbf{x}_k) + A_k(\mathbf{x} - \mathbf{x}_k) - \mathbf{f}(\mathbf{x}_{k-1}) - A_{k-1}(\mathbf{x} - \mathbf{x}_{k-1}).$$

Desarrollándola queda

$$\mathbf{f}(\mathbf{x}_k) - \mathbf{f}(\mathbf{x}_{k-1}) - A_k(\mathbf{x}_k - \mathbf{x}_{k-1}) + (A_k - A_{k-1})(\mathbf{x} - \mathbf{x}_{k-1}).$$

Sustituyendo (4.20) en ésta última expresión, la diferencia a minimizar resulta

$$(A_k - A_{k-1})(\mathbf{x} - \mathbf{x}_{k-1}). \quad (4.21)$$

Si para todo $\mathbf{x} \in \mathbb{R}^n$, la diferencia $\mathbf{x} - \mathbf{x}_{k-1}$ la expresamos como

$$\mathbf{x} - \mathbf{x}_{k-1} = \alpha \mathbf{s}_{k-1} + \mathbf{t},$$

donde $\mathbf{s}_{k-1}$ designa la diferencia $\mathbf{x}_k - \mathbf{x}_{k-1}$ y se cumple que $\mathbf{t}^T \mathbf{s}_{k-1} = 0$, la expresión a minimizar (4.21) queda

$$\alpha(A_k - A_{k-1})\mathbf{s}_{k-1} + (A_k - A_{k-1})\mathbf{t}.$$

Sobre el primer término de esta expresión no se puede actuar puesto que, según (4.20), $(A_k - A_{k-1})\mathbf{s}_{k-1} = \mathbf{y}_{k-1} - A_{k-1}\mathbf{s}_{k-1}$, donde $\mathbf{y}_{k-1} = \mathbf{f}(\mathbf{x}_k) - \mathbf{f}(\mathbf{x}_{k-1})$. El segundo término se puede hacer cero para todo $\mathbf{x} \in \mathbb{R}^n$ escogiéndolo A_k de tal manera que $(A_k - A_{k-1})\mathbf{t} = \mathbf{0}$, para todo $\mathbf{t}$ ortogonal a $\mathbf{s}_{k-1}$. Esto requiere que la matriz $A_k - A_{k-1}$ sea de rango uno, es decir, de la forma $\mathbf{u}\mathbf{s}_{k-1}^T$, con $\mathbf{u} \in \mathbb{R}^n$. Ahora bien, para que se cumpla que $A_k(\mathbf{x}_k - \mathbf{x}_{k-1}) = \mathbf{f}(\mathbf{x}_k) - \mathbf{f}(\mathbf{x}_{k-1})$, lo que equivale como acabamos de ver a que $(A_k - A_{k-1})\mathbf{s}_{k-1} = \mathbf{y}_{k-1} - A_{k-1}\mathbf{s}_{k-1}$, el vector $\mathbf{u}$ debe ser igual a $(\mathbf{y}_{k-1} - A_{k-1}\mathbf{s}_{k-1})/\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}$. La matriz

$$A_k = A_{k-1} + \frac{(\mathbf{y}_{k-1} - A_{k-1}\mathbf{s}_{k-1})\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} \quad (4.22)$$

es, por consiguiente, la que cumple ese propósito de minimizar la diferencia indicada, verificándose además que $A_k\mathbf{s}_{k-1} = \mathbf{y}_{k-1}$.

La *fórmula de Broyden*, (4.22), es la que propuso este autor para aproximar la matriz Jacobiana en cada iteración del método de Newton.

En el lema siguiente se demuestra que la fórmula de Broyden es la que introduce un menor cambio en A_{k-1} , en un sentido que veremos inmediatamente, siendo consistente con que $A_k \mathbf{s}_{k-1} = \mathbf{y}_{k-1}$. Antes sin embargo, recordemos la norma de Frobenius de una matriz A : $\|A\|_F = \sqrt{\sum_{1 \leq i, j \leq n} a_{ij}^2}$. También, que se cumple que

$$\|AB\| \leq \|A\| \|B\| \quad (4.23)$$

$$\|AB\|_F \leq \min \{ \|A\|_2 \|B\|_F, \|A\|_F \|B\|_2 \} \quad (4.24)$$

y que

$$\|\mathbf{v}\mathbf{w}^T\|_F = \|\mathbf{v}\mathbf{w}^T\|_2 = \|\mathbf{v}\|_2 \|\mathbf{w}\|_2. \quad (4.25)$$

También introducimos la notación $Q(\mathbf{y}, \mathbf{s})$ para designar:

$$Q(\mathbf{y}, \mathbf{s}) = \{B \in \mathbb{R}^{n \times n} : B\mathbf{s} = \mathbf{y}\}.$$

Lema 4.4 Sean la matriz $A \in \mathbb{R}^{n \times n}$ y los vectores $\mathbf{s}, \mathbf{y} \in \mathbb{R}^n$, $\mathbf{s} \neq \mathbf{0}$. Para cualesquiera normas matriciales $\|\cdot\|$ y $\|\cdot\|$ tales que

$$\|AB\| \leq \|A\| \|B\| \quad (4.26)$$

y

$$\left\| \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T \mathbf{v}} \right\| = 1, \quad (4.27)$$

la solución del problema

$$\underset{B \in Q(\mathbf{y}, \mathbf{s})}{\text{minimizar}} \|B - A\| \quad (4.28)$$

es

$$A_+ = A + \frac{(\mathbf{y} - A\mathbf{s})\mathbf{s}^T}{\mathbf{s}^T \mathbf{s}}. \quad (4.29)$$

En particular, (4.29) es la solución de (4.28) cuando $\|\cdot\|$ es la norma espectral, siendo la solución única cuando la norma $\|\cdot\|$ es la de Frobenius.

DEMOSTRACIÓN. Sea $B \in Q(\mathbf{y}, \mathbf{s})$; se tiene entonces que

$$\begin{aligned} \|A_+ - A\| &= \left\| \frac{(\mathbf{y} - A\mathbf{s})\mathbf{s}^T}{\mathbf{s}^T \mathbf{s}} \right\| = \left\| \frac{(B - A)\mathbf{s}\mathbf{s}^T}{\mathbf{s}^T \mathbf{s}} \right\| \\ &= \|B - A\| \left\| \frac{\mathbf{s}\mathbf{s}^T}{\mathbf{s}^T \mathbf{s}} \right\| \leq \|B - A\|. \end{aligned}$$

Si $\|\cdot\|$ y $\|\cdot\|$ son ambas la norma espectral, las expresiones (4.26) y (4.27) se deducen inmediatamente de (4.23) y (4.25). Si $\|\cdot\|$ y $\|\cdot\|$ son, respectivamente, las normas de Frobenius y la espectral, las expresiones (4.26) y (4.27) se deducen de (4.24) y (4.25). Para probar que (4.29) es la única solución de (4.28), recordemos que la norma de Frobenius es estrictamente convexa puesto que es la norma euclídea de una matriz escrita como un vector de dimensión n^2 . Como $Q(\mathbf{y}, \mathbf{s})$ es un subconjunto convexo de $\mathbb{R}^{n \times n}$ (es una variedad lineal), la solución de (4.28) es única. ■

El usar en este lema la norma de Frobenius parece razonable ya que esa norma mide el cambio en cada componente de la aproximación de la matriz Jacobiana. La norma espectral sería menos precisa.

El algoritmo para resolver un sistema no lineal de ecuaciones, $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, mediante el método cuasi Newton que se deriva de utilizar la fórmula de Broyden, partiendo de un punto $\mathbf{x}_0$, es el que describe la tabla 4.9.

Tabla 4.9

Algoritmo cuasi Newton con la fórmula de Broyden para la solución de sistemas de ecuaciones no lineales

Paso 0 – Definir un $\mathbf{x}_0 \in \mathbb{R}^n$ y una $A_0 \in \mathbb{R}^{n \times n}$; hacer $k = 1$ y $\mathbf{x}_k \leftarrow \mathbf{x}_0$.

Paso 1 – Determinar la solución de $A_k \mathbf{s}_k = -\mathbf{f}(\mathbf{x}_k)$.

Paso 2 – Si $\|\mathbf{f}(\mathbf{x}_k)\|_2 < Tol$, PARAR: el problema está resuelto.

Si $> Tol$, hacer: $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \mathbf{s}_k$

$\mathbf{y}_k \leftarrow \mathbf{f}(\mathbf{x}_{k+1}) - \mathbf{f}(\mathbf{x}_k)$

$A_{k+1} \leftarrow A_k + \frac{(\mathbf{y}_k - A_k \mathbf{s}_k) \mathbf{s}_k^T}{\mathbf{s}_k^T \mathbf{s}_k}$

$k \leftarrow k + 1$

y volver al paso 1.

Para determinar la A_0 que se cita en el algoritmo se puede emplear cualquier aproximación, por ejemplo, diferencias finitas.

Ejemplo 4.5 Resolvamos una vez más, partiendo del punto $[1, 1, 1]^T$, esta vez utilizando el método de Newton con la fórmula de Broyden, el siguiente sistema de ecuaciones no lineales:

$$\begin{aligned} 3x_1 - \cos(x_2 x_3) - \frac{1}{2} &= 0 \\ x_1^2 - 81 \left(x_2 + \frac{1}{10} \right)^2 + \sin(x_3) + 1,06 &= 0 \\ e^{-x_1 x_2} + 20x_3 + \frac{10\pi - 3}{3} &= 0. \end{aligned}$$

El código en FORTRAN 77 que implementa el método cuasi Newton basado en la fórmula de Broyden es el que sigue a continuación. Como matriz inicial A_0 se utiliza la que tiene como únicos elementos distintos de cero los de la diagonal de la Jacobiana en el punto de partida.

```

PROGRAM Broyden
C
  parameter (n=3)
  integer ip(n)
  double precision f(n), j(n,n), ja(n,n), x(n), x1(n), f1(n), y(n), s(n),
+               tol, dnor, dnr
C
```

```

      tol    = dsqrt(epsilon(1.0d0))
      x      = 1.d0
      j(1,1) = 3.d0
      j(2,2) = -178.2d0
      j(3,3) = 20.d0
      call fx (f,x,n)
      dnr = dnor(f,n)
C
C *** Proceso iterativo ***
C
      do while (dnr.gt.tol)
        f1 = f
        ja = j
        call gauss (ja,f,s,ip,n)
        x1 = x-s
        call fx (f,x1,n)
        dnr = dnor(f,n)
        print *,x1,dnr                ! Salida de resultados
        y = f-f1
        call broyd (j,y,s,n)
        x = x1
      end do
C
      end

      subroutine fx (f,x,n)
      double precision f(n),x(n)
C
      f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
      f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
      f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+(10.0*dacos(-1.0d0)-3.0)/3.0
C
      return
      end

      double precision function dnor (x,n)
      double precision x(n)
C
      dnor = 0.d0
      do i = 1,n
        dnor = dnor+x(i)**2
      end do
C
      dnor = dsqrt(dnor)
      return
      end

      subroutine broyd (a,y,s,n)
      integer i,j,n
      double precision a(n,n),y(n),s(n),sum,prod
C
      prod = 0.d0
      do i = 1,n
        prod = prod+s(i)**2
      end do
C
      do i = 1,n
        sum = 0.d0
        do j = 1,n
```

```

        sum = sum+a(i,j)*s(j)
    end do
    y(i) = (y(i)+sum)/prod
end do
C
do i = 1,n
    do j = 1,n
        a(i,j) = a(i,j)-y(i)*s(j)
    end do
end do
C
return
end

subroutine gauss (a,b,x,ipvt,n)
C
C *** Resolución del sistema lineal de ecuaciones mediante eliminación
C   de Gauss
C
    integer ipvt(n),ip,pi,l,i,j,k,n,iaux
    double precision a(n,n),b(n),x(n),smax,r,r1,c
C
    do i = 1,n
        ipvt(i) = i
    end do
C
C *** Triangularización ***
C
    do k = 1,n-1
        l = 0
        smax = dabs(a(ipvt(k),k))
        do i = k+1,n
            ip = ipvt(i)
            if (dabs(a(ip,k)).gt.smax) then
                l = i
                smax = dabs(a(ip,k))
            endif
        end do
        if (l.ne.0) then
            iaux = ipvt(k)
            ipvt(k) = ipvt(l)
            ipvt(l) = iaux
        endif
        pi = ipvt(k)
        r1 = 1.0/a(pi,k)
        do i = k+1,n
            ip = ipvt(i)
            r = a(ip,k)*r1
            do j = k+1,n
                a(ip,j) = a(ip,j)-r*a(pi,j)
            end do
            b(ip) = b(ip)-r*b(pi)
        end do
    end do
C
C *** Sustitución inversa ***
C
    x(n) = b(ipvt(n))/a(ipvt(n),n)
    do i = n-1,1,-1

```

```

        pi = ipvt(i)
        c = b(pi)
        do j = i+1,n
            c = c-a(pi,j)*x(j)
        end do
        x(i) = c/a(pi,i)
    end do
C
    return
end

```

El proceso de convergencia hasta la solución que se registra con este código es el de la tabla 4.10. Compárense estos resultados con los obtenidos en el ejemplo 4.3, página 307. ●

Tabla 4.10

Proceso de convergencia a la solución del problema del ejemplo 4.5 con el método cuasi Newton basado en la fórmula de Broyden

k	x_1	x_2	x_3	$\ f(x_k)\ _2$
1	3,467674352893799E-1	4,662821024806326E-01	-4,919927476568708E-1	25,275175252053120
2	4,921232306763561E-1	3,236527849976335E-01	-5,162769886149683E-1	13,729480399369230
3	4,993486752210201E-1	2,131483731754155E-01	-5,166714279059975E-1	7,127754268893964
4	5,011649166344201E-1	9,690341763001632E-02	-5,210585843043458E-1	2,327087146316625
5	5,003080441638231E-1	4,279330810076126E-02	-5,224749127576461E-1	8,403043972608411E-01
6	5,001066711564305E-1	1,172102964534057E-02	-5,232913081815899E-1	2,006362866054586E-01
7	5,000183047478762E-1	2,032314047074978E-03	-5,235457794542374E-1	3,319399780484372E-02
8	5,000009846717887E-1	1,115674463108231E-04	-5,235958520108367E-1	1,804970096278442E-03
9	5,000000108711760E-1	1,033227841870006E-06	-5,235987485509558E-1	1,678549255880026E-05
10	5,000000000415024E-1	6,118437770431628E-10	-5,235987755897009E-1	9,122344458875651E-08
11	4,99999999986228E-1	-5,059011531347285E-09	-5,235987757245316E-1	4,849895176081806E-10

4.4.1.1 Convergencia del método de Broyden

Para estudiar la convergencia del método se utiliza una estrategia muy similar a la utilizada para estudiar la del método de Newton para sistemas de ecuaciones no lineales.

Si $f(x^*) = 0$, de la ecuación de una iteración,

$$x_{k+1} = x_k - A_k^{-1} f(x_k),$$

se tiene que

$$x_{k+1} - x^* = x_k - x^* - A_k^{-1} [f(x_k) - f(x^*)],$$

o, también, que

$$A_k(x_{k+1} - x^*) = A_k(x_k - x^*) - f(x_k) + f(x^*).$$

Si se define el vector $e_k = x_k - x^*$ y se suma y resta el vector $J(x^*)e_k$ al segundo miembro de la ecuación anterior, se tiene que

$$A_k e_{k+1} = -f(x_k) + f(x^*) + J(x^*)e_k + (A_k - J(x^*))e_k.$$

De acuerdo con las hipótesis que venimos adoptando,

$$\| -f(x_k) + f(x^*) + J(x^*)e_k \| = O\|e_k\|^2,$$

por lo que la clave del análisis de la convergencia del método de Broyden estará en el término $(A_k - J(\mathbf{x}^*))\mathbf{e}_k$.

Enunciaremos un teorema que prueba la convergencia, al menos lineal, de la sucesión $\{\mathbf{e}_k\}$ a cero, viendo como la sucesión $\{\|A_k - J(\mathbf{x}^*)\|\}$ permanece acotada por una constante. También enunciaremos otro teorema mediante el que se prueba, aun cuando puede que no sea cierto que

$$\lim_{k \rightarrow \infty} \|A_k - J(\mathbf{x}^*)\| = 0,$$

la convergencia superlineal de la sucesión viendo que

$$\lim_{k \rightarrow \infty} \frac{\|(A_k - J(\mathbf{x}^*))\mathbf{e}_k\|}{\|\mathbf{e}_k\|} = 0.$$

Estos resultados garantizarán que el paso que se obtiene aplicando el método de Broyden, esto es, $-A_k^{-1}\mathbf{f}(\mathbf{x}_k)$, converge al paso de Newton, $-J(\mathbf{x}_k)^{-1}\mathbf{f}(\mathbf{x}_k)$, en magnitud y dirección.

Comencemos preguntándonos cuan adecuadamente la fórmula de Broyden aproxima la matriz Jacobiana $J(\mathbf{x}^*)$. Si $\mathbf{f}(\mathbf{x})$ es una variedad lineal⁷, con matriz Jacobiana igual a J , esa matriz satisfará la *ecuación secante*,

$$A_k(\mathbf{x}_k - \mathbf{x}_{k-1}) = \mathbf{f}(\mathbf{x}_k) - \mathbf{f}(\mathbf{x}_{k-1}),$$

es decir, $J \in Q(\mathbf{y}_k, \mathbf{s}_k)$. Como A_k es el elemento más próximo en $Q(\mathbf{y}_k, \mathbf{s}_k)$ a A_{k-1} , en el sentido de la norma de Frobenius, del teorema de Pitágoras se tiene que

$$\|A_k - J\|_F^2 + \|A_k - A_{k-1}\|_F^2 = \|A_{k-1} - J\|_F^2,$$

es decir, que $\|A_k - J\|_F \leq \|A_{k-1} - J\|_F$ (ver figura 4.14). De aquí que la fórmula de Broyden

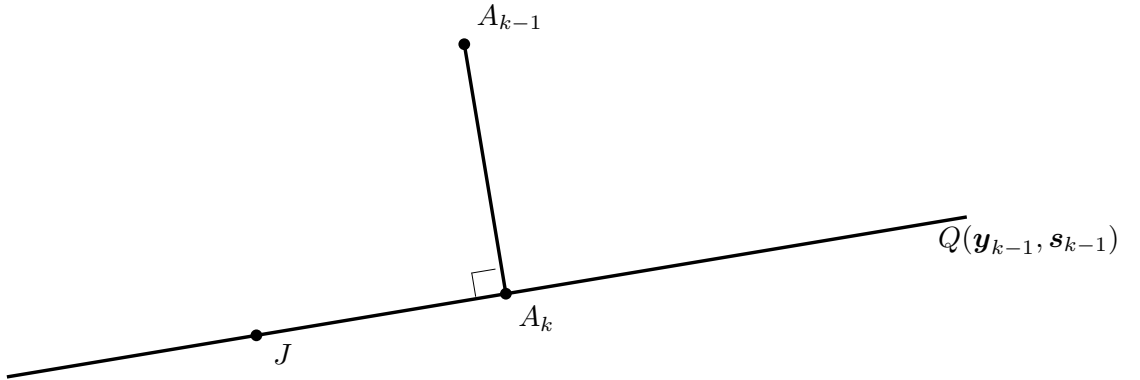


Figura 4.14

Método de Broyden en una variedad lineal

no puede hacer que la norma de Frobenius del error en la aproximación de la matriz jacobiana empeore. Este hecho, desafortunadamente, puede no ser necesariamente cierto para el caso no

⁷La función será en este caso $\mathbf{f}(\mathbf{x}) = J\mathbf{x} + \mathbf{b}$.

lineal de $\mathbf{f}(\mathbf{x})$. Por ejemplo, podría darse que $A_{k-1} = J(\mathbf{x}^*)$ pero ocurrir que $A_{k-1}\mathbf{s}_{k-1} \neq \mathbf{y}_{k-1}$, lo que conllevaría que $\|A_k - J(\mathbf{x}^*)\| > \|A_{k-1} - J(\mathbf{x}^*)\|$.

En el siguiente lema se prueba que si la aproximación de la matriz Jacobiana es cada vez peor, su deterioro es tan lento que no impide probar la convergencia de $\{\mathbf{x}_k\}$ a $\mathbf{x}^*$.

Lema 4.5 Sea $D \subset \mathbb{R}^n$ un conjunto convexo abierto que contiene a $\mathbf{x}_{k-1}$ y $\mathbf{x}_k$, $\mathbf{x}_{k-1} \neq \mathbf{x}_k$. Sea $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $J(\mathbf{x}) \in \text{Lip}_\gamma(D)$, $A_{k-1} \in \mathbb{R}^{n \times n}$ y

$$A_k = A_{k-1} + \frac{(\mathbf{y}_{k-1} - A_{k-1}\mathbf{s}_{k-1})\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}}.$$

Entonces, para las normas de Frobenius o la espectral,

$$\|A_k - J(\mathbf{x}_k)\| \leq \|A_{k-1} - J(\mathbf{x}_{k-1})\| + \frac{3\gamma}{2}\|\mathbf{x}_k - \mathbf{x}_{k-1}\|_2. \quad (4.30)$$

Además, si $\mathbf{x}^* \in D$ y $J(\mathbf{x})$ cumple la condición de Lipschitz,

$$\|J(\mathbf{x}) - J(\mathbf{x}^*)\| \leq \gamma\|\mathbf{x} - \mathbf{x}^*\| \quad \text{para todo } \mathbf{x} \in D,$$

entonces

$$\|A_k - J(\mathbf{x}^*)\| \leq \|A_{k-1} - J(\mathbf{x}^*)\| + \frac{\gamma}{2}(\|\mathbf{x}_k - \mathbf{x}^*\|_2 + \|\mathbf{x}_{k-1} - \mathbf{x}^*\|_2). \quad (4.31)$$

DEMOSTRACIÓN. Probemos primero la expresión (4.31). Sea $J_* = J(\mathbf{x}^*)$. Restando J_* de ambos miembros de la expresión que define A_k , se tiene que

$$\begin{aligned} A_k - J_* &= A_{k-1} - J_* + \frac{(\mathbf{y}_{k-1} - A_{k-1}\mathbf{s}_{k-1})\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} \\ &= A_{k-1} - J_* + \frac{(J_*\mathbf{s}_{k-1} - A_{k-1}\mathbf{s}_{k-1})\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} + \frac{(\mathbf{y}_{k-1} - J_*\mathbf{s}_{k-1})\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} \\ &= (A_{k-1} - J_*) \left[I - \frac{\mathbf{s}_{k-1}\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} \right] + \frac{(\mathbf{y}_{k-1} - J_*\mathbf{s}_{k-1})\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}}. \end{aligned}$$

Para las norma de Frobenius o la espectral, de (4.24) o (4.23) y de (4.25), se tiene que

$$\|A_k - J_*\| \leq \|A_{k-1} - J_*\| \left\| I - \frac{\mathbf{s}_{k-1}\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} \right\|_2 + \frac{\|\mathbf{y}_{k-1} - J_*\mathbf{s}_{k-1}\|_2}{\|\mathbf{s}_{k-1}\|_2}.$$

Usando el hecho de que

$$\left\| I - \frac{\mathbf{s}_{k-1}\mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{s}_{k-1}} \right\|_2 = 1$$

pues $I - \left(\mathbf{s}_{k-1} \mathbf{s}_{k-1}^T \right) / \left(\mathbf{s}_{k-1}^T \mathbf{s}_{k-1} \right)$ es una matriz de proyección, y que

$$\|\mathbf{y}_{k-1} - J_* \mathbf{s}_{k-1}\|_2 \leq \frac{\gamma}{2} (\|\mathbf{x}_k - \mathbf{x}^*\|_2 + \|\mathbf{x}_{k-1} - \mathbf{x}^*\|_2) \|\mathbf{s}_{k-1}\|_2,$$

resultado inmediato del lema 4.3, se concluye la demostración de (4.31). La prueba de (4.30) es muy similar. ■

Para el siguiente teorema supondremos que $\mathbf{x}_{k+1} \neq \mathbf{x}_k$, $k = 0, 1, \dots$. Como además, según se comprueba a continuación, bajo unos ciertos supuestos, A_k es regular, $k = 0, 1, \dots$ y dado que $\mathbf{x}_{k+1} - \mathbf{x}_k = A_k^{-1} \mathbf{f}(\mathbf{x}_k)$, el supuesto de que $\mathbf{x}_{k+1} \neq \mathbf{x}_k$ es equivalente a suponer que $\mathbf{f}(\mathbf{x}_k) \neq \mathbf{0}$, $k = 0, 1, \dots$. De esta forma se evita el caso simple en el que el algoritmo encuentra la solución del sistema en un número finito de pasos.

Teorema 4.7 *Sea la función $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, continua y diferenciable en un conjunto convexo abierto $D \subset \mathbb{R}^n$. Supóngase que existe un $\mathbf{x}^* \in \mathbb{R}^n$ y $r, \beta > 0$ tales que la bola abierta $S(\mathbf{x}^*, r) \subset D$, que $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ y que $J^{-1}(\mathbf{x}^*)$ existe con $\|J^{-1}(\mathbf{x}^*)\| \leq \beta$ y $J \in \text{Lip}_\gamma(S(\mathbf{x}^*, r))$. Existen entonces unas constantes positivas ε y δ tales que si $\|\mathbf{x}_0 - \mathbf{x}^*\| \leq \varepsilon$ y $\|A_0 - J(\mathbf{x}^*)\|_2 \leq \delta$, cumpliéndose (4.31), la sucesión $\{\mathbf{x}_k\}$ generada de la aplicación recursiva de la fórmula de Broyden converge a $\mathbf{x}^*$ al menos linealmente.*

Teorema 4.8 (Dennis-Moré) *Sea un conjunto convexo abierto $D \subseteq \mathbb{R}^n$, una función $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $J \in \text{Lip}_\gamma(D)$, $\mathbf{x}^* \in D$, siendo $J(\mathbf{x}^*)$ regular. Sea $\{A_k\}$ una sucesión de matrices regulares en $\mathbb{R}^{n \times n}$ y supóngase que para un $\mathbf{x}_0 \in D$, la sucesión de puntos generada por la fórmula de recurrencia*

$$\mathbf{x}_{k+1} = \mathbf{x}_k - A_k^{-1} \mathbf{f}(\mathbf{x}_k)$$

permanece en D satisfaciendo que $\mathbf{x}_k \neq \mathbf{x}^$ para todo k y que $\lim_{k \rightarrow \infty} \mathbf{x}_k = \mathbf{x}^*$. Entonces, $\{\mathbf{x}_k\}$ converge superlinealmente a $\mathbf{x}^*$ en alguna norma $\|\cdot\|$ y $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$, si y sólo si*

$$\lim_{k \rightarrow \infty} \frac{\|(A_k - J(\mathbf{x}^*))\mathbf{s}_k\|}{\|\mathbf{s}_k\|} = 0,$$

donde $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$.

La demostración detallada de estos dos teoremas se puede seguir en Dennis y Schnabel [1983] y [1996].

4.4.1.2 Implementación práctica del método de Broyden

Para llevar numéricamente a la práctica el método de Broyden son varios los aspectos importantes que hay que considerar. El primero consiste en determinar una buena aproximación inicial de A_0 . Para ello, lo que se suele hacer (así actúan de hecho la mayor parte de los códigos comerciales) es utilizar la aproximación por diferencias finitas de $J(\mathbf{x}_0)$.

Otro se refiere a que, dado que las modificaciones de la matriz A de una iteración a otra son de rango uno, en lugar de proceder a modificar la matriz A con la fórmula de Broyden y luego

factorizar toda ella de nuevo a fin de resolver el sistema lineal de ecuaciones correspondiente, se puede proceder directamente a modificar la factorización de la A en curso mediante las operaciones más adecuadas de acuerdo con las fórmulas estándar conocidas.

Otra cuestión importante a tener en cuenta nace del hecho de que el método de Broyden adapta de iteración en iteración la matriz A y no la A^{-1} , con la que se operaría mucho más eficazmente. En este sentido y sin tener en cuenta otras consideraciones, ¿por qué no partir de una A_0^{-1} y readaptar A^{-1} ? Se reducirían el número global de operaciones necesarias para resolver el sistema de ecuaciones. A tal efecto se utiliza el siguiente resultado.

Lema 4.6 (Fórmula de modificación de Sherman-Morrison-Woodbury) (a) Si A es una matriz regular $n \times n$ y $\mathbf{u}$ y $\mathbf{v}$ dos vectores cualesquiera de $\mathbb{R}^n$, $A + \mathbf{u}\mathbf{v}^T$ es regular si y sólo si $w = 1 + \mathbf{v}^T A^{-1} \mathbf{u} \neq 0$. (b) En este caso, además,

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \left(\frac{1}{w}\right) A^{-1} \mathbf{u}\mathbf{v}^T A^{-1}.$$

DEMOSTRACIÓN. Como $A + \mathbf{u}\mathbf{v}^T = (I + \mathbf{u}\mathbf{v}^T A^{-1})A$ y $\mathbf{u}\mathbf{v}^T$ es una matriz de rango 1, el punto (a) resulta del hecho de que la matriz $I + \mathbf{u}\mathbf{v}^T A^{-1}$ tiene $n - 1$ valores propios iguales a la unidad y el restante es $1 + \mathbf{v}^T A^{-1} \mathbf{u}$.

La fórmula del punto (b) se comprueba fácilmente sin más que multiplicar el primer miembro por $A + \mathbf{u}\mathbf{v}^T$. En efecto,

$$\begin{aligned} (A + \mathbf{u}\mathbf{v}^T) (A + \mathbf{u}\mathbf{v}^T)^{-1} &= AA^{-1} - \left(\frac{1}{w}\right) AA^{-1} \mathbf{u}\mathbf{v}^T A^{-1} + \mathbf{u}\mathbf{v}^T A^{-1} - \\ &\quad \left(\frac{1}{w}\right) \mathbf{u} \underbrace{\mathbf{v}^T A^{-1} \mathbf{u}}_{w-1} \mathbf{v}^T A^{-1} \\ &= I - \left(\frac{1}{w}\right) \mathbf{u}\mathbf{v}^T A^{-1} + \mathbf{u}\mathbf{v}^T A^{-1} - \left(\frac{w-1}{w}\right) \mathbf{u}\mathbf{v}^T A^{-1} \\ &= I. \end{aligned}$$

■

La aplicación inmediata de este lema lleva a deducir la fórmula de adaptación de Broyden para las sucesivas matrices A_k^{-1} . Es la siguiente:

$$A_{k+1}^{-1} = A_k^{-1} + \frac{(\mathbf{s}_k - A_k^{-1} \mathbf{y}_k) \mathbf{s}_k^T A_k^{-1}}{\mathbf{s}_k^T A_k^{-1} \mathbf{y}_k}.$$

Hasta hace relativamente poco tiempo en que se han empezado a utilizar en códigos comerciales factorizaciones de A muy eficaces (del tipo QR por ejemplo), a las que resulta fácil aplicar modificaciones de rango uno, esta última expresión era la más usada por los métodos cuasi Newton, y ello a pesar de ciertas dificultades numéricas que se pueden presentar, como por ejemplo, la de no detectar el mal condicionamiento de A_k .

4.5 Métodos globalmente convergentes para sistemas de ecuaciones no lineales

Una cuestión práctica importante de los métodos para resolver sistemas de ecuaciones no lineales es su robustez: su facilidad para obtener la solución de problemas de diversa índole y llegar a ellas partiendo de cualquier punto. Para conseguir esto, puesto que como ya hemos visto los métodos presentados sólo convergen a la solución deseada si el punto de partida es adecuado, se recurre a algún mecanismo de salvaguarda que asegure que en cada iteración el proceso mejora la solución, es decir, que las iteraciones son monótonamente convergentes.

Con tal objetivo, si el método particular que se utiliza establece una determinada dirección de movimiento $\mathbf{d}$ (el método de Newton $J^{-1}\mathbf{f}(\mathbf{x})$, los cuasi Newton $A^{-1}\mathbf{f}(\mathbf{x})$), la idea es, desde un $\mathbf{x}_k$, moverse a lo largo de esa dirección un paso que no sea estrictamente 1 sino un factor α de éste, de tal forma que siempre se mejore el valor de la función de acuerdo con algún criterio. Si a partir de este α se define la función

$$y(\alpha) = \mathbf{x}_k + \alpha \mathbf{d}_k,$$

donde $\mathbf{d}_k = J(\mathbf{x}_k)^{-1}\mathbf{f}(\mathbf{x}_k)$ o $A_k^{-1}\mathbf{f}(\mathbf{x}_k)$. Para $\alpha = 0$, evidentemente, $y(0) = \mathbf{x}_k$; para $\alpha = 1$, $y(1) = \mathbf{x}_{k+1}$. La función norma de $\mathbf{f}(y(\alpha))$, que designaremos por $r(\alpha)$, suele tener la forma de la figura 4.15. En las proximidades de una solución de $\mathbf{f}(\mathbf{x}) = \mathbf{0}$, el mínimo de $r(\alpha)$ se alcanza próximo a $\alpha = 1$. Es decir, ese paso hace que $\|\mathbf{f}(\mathbf{x}_{k+1})\| < \|\mathbf{f}(\mathbf{x}_k)\|$. Como $\mathbf{f}(\mathbf{x})$ se hace cero en la solución, $\|\mathbf{f}(\mathbf{x}_k)\|$ se aproxima a cero monótonamente cuando el punto de partida está suficientemente próximo a la solución.

Cuando el punto $\mathbf{x}_k$ está lejos de la solución, el mínimo de $r(\alpha)$ se alcanza en un valor de α menor que 1, por lo que es concebible que $\|\mathbf{f}(y(1))\| > \|\mathbf{f}(y(0))\|$, o que $\|\mathbf{f}(\mathbf{x}_{k+1})\| > \|\mathbf{f}(\mathbf{x}_k)\|$. Conseguir que $\|\mathbf{f}(\mathbf{x}_{k+1})\| < \|\mathbf{f}(\mathbf{x}_k)\|$ conllevará una reducción de α .

Para determinar cómo reducir α de forma adecuada se han estudiado diversos procedimientos. Uno de los más usados es el conocido como *regla de Armijo*. Establece que se evalúe

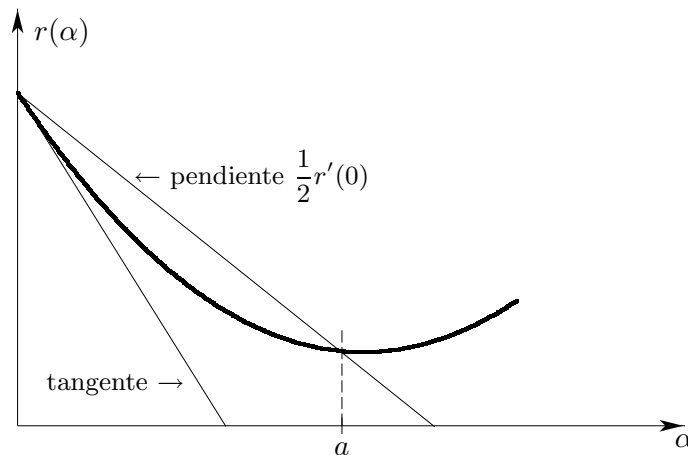


Figura 4.15
Criterio de Armijo

$\|\mathbf{f}(\mathbf{y}(\alpha))\|$ en $\alpha = 1, \frac{1}{2}, \frac{1}{4}, \dots$ hasta que

$$\|\mathbf{f}(\mathbf{y}(\alpha))\| \leq \left(1 - \frac{\alpha}{2}\right) \|\mathbf{f}(\mathbf{x}_k)\|. \quad (4.32)$$

Cuando se cumpla esta condición, el nuevo punto del proceso iterativo será

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \mathbf{d}_k. \quad (4.33)$$

El criterio de Armijo surge de la teoría de optimización. La norma que se utiliza en (4.32) es la euclídea.

Para estudiar, con el apoyo de la figura 4.15, qué representa la regla de Armijo, consideraremos la norma euclídea:

$$r(\alpha) = \|\mathbf{f}(\mathbf{y}(\alpha))\|_2 = \sqrt{\sum_{i=1}^n (f_i(\mathbf{y}(\alpha)))^2}.$$

La derivada de $r(\alpha)$ en $\alpha = 0$ es:

$$\left. \frac{dr}{d\alpha} \right|_{\alpha=0} = -\|\mathbf{f}(\mathbf{x}_k)\|_2. \quad (4.34)$$

Esto quiere decir que la pendiente de $r(\alpha)$ en $\alpha = 0$ es negativa. Si se rehace la expresión (4.32), teniendo en cuenta (4.34), y el hecho de que

$$r(0) = \|\mathbf{f}(\mathbf{y}(0))\|_2 = \|\mathbf{f}(\mathbf{x}_k)\|_2 = -r'(0),$$

se tiene que

$$\frac{r(\alpha) - r(0)}{\alpha} \leq -\frac{1}{2} \|\mathbf{f}(\mathbf{x}_k)\| \leq \frac{1}{2} r'(0). \quad (4.35)$$

Como el término de la izquierda de esta última expresión es la pendiente de una secante de la función $r(\alpha)$ que pasa por 0 y por α , el criterio de Armijo es equivalente a decir que la pendiente de esa secante es a lo sumo la mitad de la de la pendiente en $\alpha = 0$. Como se ve en la figura, aquellas α que satisfacen la expresión (4.35) pertenecen al intervalo $[0, a]$, donde la pendiente de la secante es como mucho $r'(0)/2$. Utilizando el criterio de Armijo, se va reduciendo α hasta que esté en $[0, a]$. En las proximidades de la solución se suele verificar que un $\alpha = 1$ hace que se cumpla esto último, por lo que el paso de la expresión (4.33) es el de Newton, supuesta $\mathbf{d}_k = -J(\mathbf{x}_k)^{-1}$. En cambio, lejos de la solución, el valor de α es menor que 1.

El algoritmo para resolver sistemas de ecuaciones no lineales mediante el método de Newton con el criterio de Armijo es el de la tabla 4.11.

Ejemplo 4.6 Resolvamos el siguiente sistema de tres ecuaciones no lineales con tres incógnitas:

$$\begin{aligned} 6 \arctan(x_1 - 10) - 2e^{-x_2} - 2e^{-x_3} + 2x_2 + 2x_3 - 9 &= 0 \\ 2 \arctan(x_1 - 10) - 4e^{-x_2} - e^{-x_3} + 7x_2 - 2x_3 - 3 &= 0 \\ 2 \arctan(x_1 - 10) - e^{-x_2} - 3e^{-x_3} - x_2 + 5x_3 - 3 &= 0. \end{aligned}$$

Apliquemos el método de Newton-Raphson con el criterio de Armijo. Partamos del punto $[0, 0, 0]^T$.

El código en FORTRAN 77 que implementa el algoritmo correspondiente para resolver este problema es el siguiente.

Tabla 4.11

Algoritmo de Newton para sistemas de ecuaciones no lineales con el criterio de salvaguarda de Armijo

Paso 0 – Definir un $\mathbf{x}_0 \in \Re^n$. Hacer $k = 1$ y $\mathbf{x}_k \leftarrow \mathbf{x}_0$.

Paso 1 – Determinar la solución de $J(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k) = -\mathbf{f}(\mathbf{x}_k)$.

Paso 2 – Si $\|\mathbf{f}(\mathbf{x}_{k+1})\|_2 < Tol$, PARAR: el problema está resuelto.

Si $> Tol$, hacer: $\alpha = 1$

while ($\|\mathbf{f}(\mathbf{x}_{k+1})\| > (1 - \alpha/2)\|\mathbf{f}(\mathbf{x}_k)\|$) **do**

$\alpha \leftarrow \alpha/2$

$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha \mathbf{s}_k$

end

$k \leftarrow k + 1$.

Ir al paso 1.

```

PROGRAM Newtarmijo
C
parameter (n=3)
double precision f(n),j(n,n),x(n),x1(n),s(n),tol,dnr,dnx,alfa,dnor
C
tol = dsqrt(epsilon(1.0d0))
x = 0.d0
call fx (f,x,n)
dnx = dnor(f,n)
C
do while (dnx.gt.tol)
  call derfx (j,x,n)
  call gauss (j,f,s,n)
  x1 = x-s
  call fx (f,x1,n)
  dnr = dnor(f,n)
  alfa = 1.d0
  do while(dnr.gt.(1.d0-alfa/2.)*dnx)
    alfa = alfa/2.
    x1 = x-alfa*s
    call fx (f,x1,n)
    dnr = dnor(f,n)
  end do
  print *,x1,alfa,dnr          ! Salida de resultados
  x = x1
  dnx = dnr
end do
C
end

subroutine fx (f,x,n)
double precision f(n),x(n)
C
f(1) = 6*datan(x(1)-10)-2*dexp(-x(2))-2*dexp(-x(3))+2*x(2)+

```

```
+      2*x(3)-9
f(2) = 2*datan(x(1)-10)-4*dexp(-x(2))-dexp(-x(3))+7*x(2)-
+      2*x(3)-3
f(3) = 2*datan(x(1)-10)-dexp(-x(2))-3*dexp(-x(3))-x(2)+5*x(3)-3
C
return
end

subroutine derfx (j,x,n)
double precision j(n,n),x(n)
C
j(1,1) = 6/(1+(x(1)-10)**2)
j(1,2) = 2*dexp(-x(2))+2
j(1,3) = 2*dexp(-x(3))+2
j(2,1) = 2/(1+(x(1)-10)**2)
j(2,2) = 4*dexp(-x(2))+7
j(2,3) = dexp(-x(3))-2
j(3,1) = 2/(1+(x(1)-10)**2)
j(3,2) = dexp(-x(2))-1
j(3,3) = 3*dexp(-x(3))+5
C
return
end

double precision function dnor (x,n)
double precision x(n)
C
dnor = 0.d0
do i = 1,n
    dnor = dnor+x(i)**2
end do
dnor = dsqrt(dnor)
C
return
end

subroutine gauss (a,b,x,n)
C
C ** Resolución del sistema lineal de ecuaciones mediante eliminación
C   de Gauss
C
integer ipvt(10),pi
double precision a(n,n),b(n),x(n),smax,r,r1,c
C
do i = 1,n
    ipvt(i) = i
end do
C
C *** Triangularización ***
C
do k = 1,n-1
    l = 0
    smax = dabs(a(ipvt(k),k))
    do i = k+1,n
        ip = ipvt(i)
        if (dabs(a(ip,k)).gt.smax) then
            l = i
            smax = dabs(a(ip,k))
        endif
    enddo
```

```

end do
if (l.ne.0) then
 iaux = ipvt(k)
ipvt(k) = ipvt(l)
ipvt(l) = iaux
endif
pi = ipvt(k)
r1 = 1.0/a(pi,k)
do i = k+1,n
  ip = ipvt(i)
  r = a(ip,k)*r1
  do j = k+1,n
    a(ip,j) = a(ip,j)-r*a(pi,j)
  end do
  a(ip,k) = -r
end do
end do
C
do k = 1,n-1
  ip = ipvt(k)
  do i = k+1,n
    pi = ipvt(i)
    b(pi) = b(pi)+a(pi,k)*b(ip)
  end do
end do
C
C *** Sustitución inversa ***
C
x(n) = b(ipvt(n))/a(ipvt(n),n)
do i = n-1,1,-1
  pi = ipvt(i)
  c = b(pi)
  do j = i+1,n
    c = c-a(pi,j)*x(j)
  end do
  x(i) = c/a(pi,i)
end do
C
return
end

```

El proceso iterativo que conlleva la resolución del problema, y los sucesivos pasos α son los que describe la tabla 4.12. ●

4.6 Mínimos cuadrados no lineales

El problema no lineal de mínimos cuadrados consiste en encontrar el mínimo global de la suma de los cuadrados de m funciones no lineales; es decir,

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} f(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^m r_i^2(\mathbf{x}) = \frac{1}{2} \|\mathbf{r}(\mathbf{x})\|_2^2, \quad (4.36)$$

donde $\mathbf{r}(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^m = [r_1(\mathbf{x}), \dots, r_m(\mathbf{x})]^T$ es el *vector de residuos* y cada $r_i(\mathbf{x})$, $i = 1, \dots, m$, $m \geq n$, es una función no lineal de $\mathbb{R}^n$ en $\mathbb{R}$. Este problema surge de la imposibilidad de

Tabla 4.12

Proceso de convergencia a la solución del sistema de ecuaciones no lineales del ejemplo 4.6 con el método de Newton y el criterio de Armijo

k	x_1	x_2	x_3	α	$\ f(x_k)\ _2$
1	300,083895104677200	5,000000000000001E-1	5,000000000000001E-1	1,0000	4,937E-1
2	211,530753313613000	5,010361094249567E-1	5,010361094249567E-1	1,5625E-2	4,874E-1
3	127,971241379208900	5,030767540185261E-1	5,030767540185261E-1	3,1250E-2	4,768E-1
4	73,759896400087980	5,070335953663429E-1	5,070335953663429E-1	6,2500E-2	4,580E-1
5	45,746086585685560	5,144640981917619E-1	5,144640981917619E-1	1,2500E-1	4,205E-1
6	32,053990425823070	5,275063585640013E-1	5,275063585640013E-1	2,5000E-1	3,431E-1
7	25,843790295333550	5,471809743227869E-1	5,471809743227869E-1	5,0000E-1	1,958E-1
8	23,887131421604770	5,670707461673926E-1	5,670707461673926E-1	1,0000	7,955E-3
9	24,098180028741330	5,671432894574930E-1	5,671432894574930E-1	1,0000	1,075E-4
10	24,101419206498890	5,671432904097838E-1	5,671432904097838E-1	1,0000	2,458E-8
11	24,101419947171680	5,671432904097838E-1	5,671432904097838E-1	1,0000	1,599E-15

encontrar la solución al sistema de ecuaciones no lineales $\mathbf{r}(\mathbf{x}) = \mathbf{0}$ y, en consecuencia, tratar de encontrar una pseudosolución que mejor la aproxime de acuerdo con la norma euclídea.⁸ Si $m = n$ se tiene un caso especial de sistemas de ecuaciones no lineales como los que acabamos de ver en apartados anteriores de este capítulo.

El campo donde más aplicaciones encuentran las técnicas que describimos a continuación para resolver estos problemas es el del ajuste de funciones a datos diversos. Se trata de aproximar a unos datos dados, definidos por ejemplo por un par y_i (valor) y t_i (tiempo), (y_i, t_i) , $i = 1, \dots, m$, una función o modelo $f(\mathbf{x}, t)$. Si $r_i(\mathbf{x})$ representa el error en la predicción que hace el modelo de la observación i ,

$$r_i(\mathbf{x}) = y_i - f(\mathbf{x}, t_i), \quad i = 1, \dots, m,$$

y se quiere minimizar la suma de los cuadrados de las desviaciones entre los valores reales y los predichos con el modelo, se llega a un problema del tipo (4.36).

Estimación del estado de sistemas eléctricos

La *estimación del estado* es, en sentido abstracto, el proceso por el cual se determina el valor del vector de variables de estado de un sistema, basándose en unas medidas efectuadas al mismo conforme a criterios diversos. Estas medidas no se realizan, usualmente, con precisión absoluta, debido a la imperfección operativa de los aparatos encargados de llevarlas a efecto, si bien suelen tener un grado de redundancia apreciable por lo que el aludido proceso de estimación se basa en maximizar o minimizar unos criterios estadísticos determinados. El más usado por científicos y técnicos es sin duda el de minimizar la suma de los cuadrados de las desviaciones entre los valores reales —medidas— y los estimados.

En el análisis, la operación y planificación de sistemas eléctricos de energía (un esquema muy sencillo de uno de estos sistemas se puede ver en la figura 4.16), uno de los problemas de más relevancia técnica y económica y que con mayor frecuencia se estudia en los departamentos de explotación y centros de control de empresas eléctricas, es el de la estimación del estado de funcionamiento del sistema de generación y transporte. Conocido este estado, es

⁸Recordemos las ideas introducidas en este sentido al hablar de mínimos cuadrados lineales

posible analizar multitud de parámetros sobre si su funcionamiento es correcto o no, técnica o económicamente, si conviene efectuar alguna maniobra para mejorarlo o para planificar su evolución a tenor de cambios que se avecinen o presuman en el corto o medio plazo, etc.

Para estimar el estado del sistema se instalan unos aparatos de medida que proporcionan el valor de una serie de magnitudes físicas relativas al mismo: tensiones en diversos puntos, flujos de potencia activa y reactiva por elementos de transporte, potencias activa y reactiva inyectadas por generadores, etc. Si todas estas medidas fuesen correctas con precisión total y dado que el sistema funciona, las relaciones matemáticas que plasman las leyes físicas que rigen su funcionamiento permitirían determinar la solución a ese estado de funcionamiento de forma única. Los errores aleatorios que incorporan los aparatos de medida en éstas, sin embargo, introducen una incompatibilidad matemática en aquellas relaciones, por lo que el cálculo de la solución no es posible teniendo que sustituirse por una estimación. El número de medidas que se efectúa suele ser varias veces superior al estrictamente necesario para determinar el estado con el fin de aumentar la bondad de la estimación así como poder identificar mediciones erróneas.

En la figura 4.17 se ha aumentado el detalle de la 4.16, en lo que hace referencia a tres de sus nudos, a fin de ilustrar una disposición típica de aparatos de medida en un sistema eléctrico

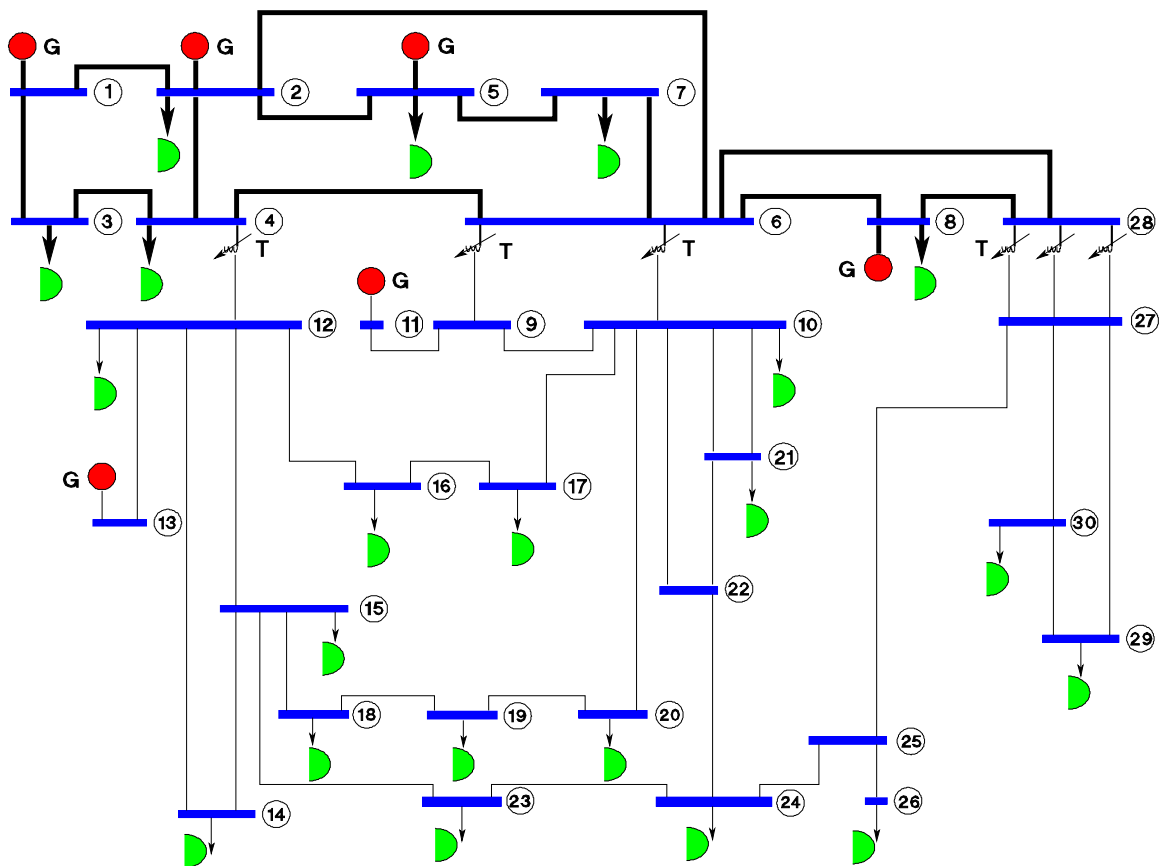


Figura 4.16
Red eléctrica IEEE de 30 Nudos

de generación y transporte con objeto de estimar su estado de funcionamiento.

Como apuntábamos al hablar de los estudios de cargas en sistemas eléctricos de generación y transporte de energía, si se supone que los parámetros físicos del sistema eléctrico permanecen constantes, existen cuatro variables asociadas a cada nudo i del referido sistema eléctrico: la tensión, en módulo, V_i , y argumento θ_i ; la potencia activa inyectada, P_i , y la potencia reactiva inyectada, Q_i . Las potencias inyectadas dependen de la tensión en el propio nudo i y en los a él unidos. Las expresiones —ver apéndice C— que las relacionan, en ausencia de transformadores conectados al nudo i , están definidas por:

$$\begin{aligned} P_i &= |V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n \left(G_{p_{ij}} + G_{s_{ij}} \right) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| \left[G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j) \right] \\ Q_i &= -|V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n \left(B_{p_{ij}} + B_{s_{ij}} \right) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| \left[G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j) \right] \end{aligned}$$

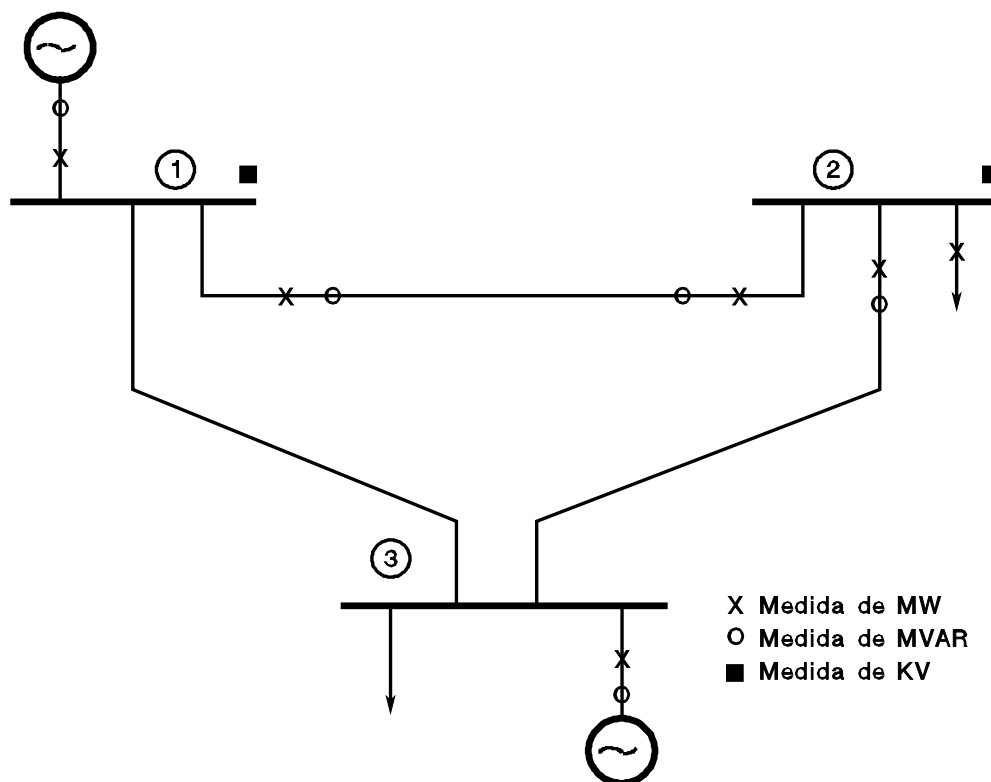


Figura 4.17

Conjunto típico de medidas para la estimación del estado de un sistema eléctrico

donde: V_i es el módulo de la tensión en el nudo i ;
 θ_i el argumento de la tensión en el nudo i ;
 $G_{s_{ij}}$ la conductancia serie (constante) de la línea que une el nudo i con el nudo j ;
 $G_{p_{ij}}$ la conductancia a tierra (constante) de la línea que une el nudo i con el j ;
 $B_{s_{ij}}$ la susceptancia serie (constante) de la línea que une el nudo i con el nudo j ; y
 $B_{p_{ij}}$ la susceptancia a tierra (constante) de la línea que une el nudo i con el j .

Si un nudo tiene algún condensador o reactancia conectado a él, $B_{p_{ij}}$ deberá englobar la del condensador/reactancia y las de tierra de las líneas conectadas a ese nudo.

El resto de las variables del sistema se pueden expresar en función de las ya mencionadas. Así, por ejemplo, entre los nudos i y j de una red, los flujos de potencias activa y reactiva están dados por:

$$\begin{aligned} P_{ij} &= |V_i|^2 G_{s_{ij}} - |V_i||V_j| G_{s_{ij}} \cos(\theta_i - \theta_j) - |V_i||V_j| B_{s_{ij}} \sin(\theta_i - \theta_j) + |V_i|^2 G_{p_{ij}} \\ Q_{ij} &= -|V_i|^2 B_{s_{ij}} - |V_i||V_j| G_{s_{ij}} \sin(\theta_i - \theta_j) + |V_i||V_j| B_{s_{ij}} \cos(\theta_i - \theta_j) - |V_i|^2 B_{p_{ij}}. \end{aligned}$$

Estas expresiones, al igual que las de las potencias inyectadas, se complican, según se puede ver en el apéndice C, al considerar transformadores.

En términos matemáticos, si se tiene una muestra $b_1, b_2, \dots, b_m$ determinada por las medidas de los aparatos, el sistema de ecuaciones que relaciona estas mediciones con las variables de estado $x_1, x_2, \dots, x_n$, se puede expresar como

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &= b_1 \\ f_2(x_1, x_2, \dots, x_n) &= b_2 \\ &\vdots \\ f_m(x_1, x_2, \dots, x_n) &= b_m, \end{aligned} \tag{4.37}$$

donde $m \gg n$. Se supone que los componentes de la función vectorial $\mathbf{f}(\mathbf{x})$ son exactos. Este sistema, debido a la imprecisión de los aparatos de medida antes comentada, es matemáticamente incompatible, aunque esta incompatibilidad suele ser muy pequeña, pues los errores en condiciones normales son pequeños.

Para el ejemplo de la figura 4.17, tomando $\theta_1 = 0$ como referencia de ángulos, los parámetros que definen el sistema (4.37) son los de la tabla 4.13.

Al no poder calcular la solución exacta de (4.37), por no existir, es necesario definir un criterio, métrica o *estimador* en $\mathbb{R}^n$ que permita encontrar otra solución —en lo sucesivo la denominaremos *pseudosolución*— lo más próxima a aquella. Los estimadores más usados son el de mínimos cuadrados y el de máxima verosimilitud.

El *estimador de mínimos cuadrados* elige como criterio de aproximación de la solución

$$\Phi(x_1, x_2, \dots, x_n) = \sum_{i=1}^m (b_i - f_i(x_1, x_2, \dots, x_n))^2$$

por ser ésta una función continua, diferenciable y de estructura matemática rica. El estimador de los parámetros que se elige es aquel que hace mínima la función $\Phi(x_1, x_2, \dots, x_n)$; es decir,

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \Phi(\mathbf{x}).$$

Tabla 4.13
Parámetros del problema de la figura 4.17

b	x	$f(x)$
V_1	V_1 V_2 V_3 θ_1 θ_2 θ_3	V_1
V_2		V_2
P_1		$V_1^2 \sum_{j=2,3} (G_{p1j} + G_{s1j}) - V_1 \sum_{j=2,3} V_j (G_{1j} \cos(\theta_1 - \theta_j) + B_{1j} \sin(\theta_1 - \theta_j))$
Q_1		$-V_1^2 \sum_{j=2,3} (B_{p1j} + B_{s1j}) - V_1 \sum_{j=2,3} V_j (G_{1j} \sin(\theta_1 - \theta_j) - B_{1j} \cos(\theta_1 - \theta_j))$
P_2		$V_2^2 \sum_{j=1,3} (G_{p2j} + G_{s2j}) - V_2 \sum_{j=1,3} V_j (G_{2j} \cos(\theta_2 - \theta_j) + B_{2j} \sin(\theta_2 - \theta_j))$
P_3		$V_3^2 \sum_{j=1,2} (G_{p3j} + G_{s3j}) - V_3 \sum_{j=1,2} V_j (G_{3j} \cos(\theta_3 - \theta_j) + B_{3j} \sin(\theta_3 - \theta_j))$
Q_3		$-V_3^2 \sum_{j=1,2} (B_{p3j} + B_{s3j}) - V_3 \sum_{j=1,2} V_j (G_{3j} \sin(\theta_3 - \theta_j) - B_{3j} \cos(\theta_3 - \theta_j))$
P_{12}		$V_1^2 G_{s12} - V_1 V_2 (G_{s12} \cos(\theta_1 - \theta_2) + B_{s12} \sin(\theta_1 - \theta_2)) + V_1^2 G_{p12}$
Q_{12}		$-V_1^2 B_{s12} - V_1 V_2 (G_{s12} \sin(\theta_1 - \theta_2) - B_{s12} \cos(\theta_1 - \theta_2)) - V_1^2 B_{p12}$
P_{23}		$V_2^2 G_{s23} - V_2 V_3 (G_{s23} \cos(\theta_2 - \theta_3) + B_{s23} \sin(\theta_2 - \theta_3)) + V_2^2 G_{p23}$
Q_{23}		$-V_2^2 B_{s23} - V_2 V_3 (G_{s23} \sin(\theta_2 - \theta_3) - B_{s23} \cos(\theta_2 - \theta_3)) - V_2^2 B_{p23}$

En la estimación del estado de sistemas eléctricos, el estimador más usado es el de máxima verosimilitud. Este estimador es idéntico al de mínimos cuadrados cuando los errores que afectan a las mediciones tienen una distribución de probabilidad $N(0, \sigma)$ —ambos convergen en probabilidad a x , son asintóticamente normales y consistentes para $m \rightarrow \infty$ —.

Si un determinado aparato suministra la medida b , siendo b^{real} la que debería dar si la precisión de la medición fuese total, se tendrá que

$$b = b^{real} + \eta,$$

donde η es el error aleatorio propio del aparato de medida. Si η no está sesgado, la función de densidad de probabilidad que se puede utilizar para describirlo es la de la distribución normal de media cero y desviación típica σ , es decir,

$$FDP(\eta) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{\eta^2}{2\sigma^2}}.$$

Como la media de η se supone cero, la media de la realización b es b^{real} . La función de densidad de probabilidad de b es

$$FDP(b) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(b - b^{real})^2}{2\sigma^2}}.$$

Si se tiene un vector de m medidas $\mathbf{b}$, cada componente del cual presenta una función de densidad de probabilidad semejante a la descrita, la función de densidad de probabilidad conjunta de la muestra aleatoria $b_1, \dots, b_m$, supuestas todas las medidas independientes unas de otras, es

$$FDP(b_1, \dots, b_m) = FDP(b_1) \cdot FDP(b_2) \cdots FDP(b_m) = \prod_{i=1}^m FDP(b_i).$$

A esta función de densidad de probabilidad conjunta se la denomina *verosimilitud* de los parámetros (los b_i^{real}) y se designa por $L(b_1^{real}, \dots, b_m^{real}) = L(\mathbf{b}^{real})$. De lo que se trata es de hacer máxima la probabilidad (verosimilitud) de que se obtenga la muestra que realmente se ha obtenido: $\mathbf{b}$. Es decir, hacer máxima

$$L(\mathbf{b}^{real}) = \prod_{i=1}^m \left(\frac{1}{\sigma_i \sqrt{2\pi}} \right) e^{-\sum_{i=1}^m \frac{(b_i - b_i^{real})^2}{2\sigma_i^2}},$$

o, lo que es lo mismo, $\ln L(\mathbf{b}^{real})$.

Ahora bien, maximizar la función $\ln L(\mathbf{b}^{real})$ es lo mismo que

$$\text{maximizar} \left[-\sum_{i=1}^m \ln(\sigma_i \sqrt{2\pi}) - \sum_{i=1}^m \frac{(b_i - b_i^{real})^2}{2\sigma_i^2} \right].$$

Como $-\sum_{i=1}^m \ln(\sigma_i \sqrt{2\pi})$ es constante, este último problema equivale a

$$\text{minimizar} \left[\sum_{i=1}^m \frac{(b_i - b_i^{real})^2}{2\sigma_i^2} \right].$$

Los parámetros $\mathbf{b}^{real}$ no son independientes entre sí; están relacionados a través de las variables de estado por la función no lineal antes mencionada

$$\mathbf{b}^{real} = \mathbf{f}(\mathbf{x}),$$

donde $\mathbf{x}$ es el vector de variables de estado —recordemos, tensiones en los nudos de la red y tomas de los transformadores con regulación—.

El problema expresado en forma matricial resulta

$$\text{minimizar}_{\mathbf{x} \in \mathbb{R}^n} [\mathbf{b} - \mathbf{f}(\mathbf{x})]^T \Theta^{-1} [\mathbf{b} - \mathbf{f}(\mathbf{x})],$$

donde la matriz

$$\Theta = \begin{bmatrix} \sigma_1^2 & & \\ & \ddots & \\ & & \sigma_m^2 \end{bmatrix}$$

es la *matriz de covarianzas* de las mediciones. Como es definida positiva su inversa se puede expresar de la forma $\Theta^{-1} = W^T W$, dando lugar a un planteamiento del problema de la forma

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \|W(\mathbf{b} - \mathbf{f}(\mathbf{x}))\|_2^2,$$

idéntico en estructura al que planteábamos con el estimador de mínimos cuadrados.

4.6.1 Referencias teóricas del problema

Volviendo al problema no lineal de mínimos cuadrados en sí, los métodos más importantes para resolverlo requieren la información que proporcionan las derivadas de los componentes $r_i(\mathbf{x})$ del vector $\mathbf{r}(\mathbf{x})$.⁹ En lo sucesivo supondremos que esas derivadas existen, como mínimo hasta segundo orden, siendo además continuas. La matriz Jacobiana de $\mathbf{r}(\mathbf{x})$ es $J(\mathbf{x}) \in \mathbb{R}^{m \times n}$ y la Hessiana de cada componente $r_i(\mathbf{x})$, $G_i(\mathbf{x}) \in \mathbb{R}^{n \times n}$, donde

$$G_i(\mathbf{x})_{jk} = \frac{\partial^2 r_i(\mathbf{x})}{\partial x_j \partial x_k}, \quad i = 1, \dots, m.$$

Las derivadas primera y segunda de $f(\mathbf{x})$ son fácilmente obtenibles:

$$\nabla f(\mathbf{x}) = \sum_{i=1}^m r_i(\mathbf{x}) \nabla r_i(\mathbf{x}) = J(\mathbf{x})^T \mathbf{r}(\mathbf{x})$$

y

$$\nabla^2 f(\mathbf{x}) = \sum_{i=1}^m \left(\nabla r_i(\mathbf{x}) \nabla r_i(\mathbf{x})^T + r_i(\mathbf{x}) \nabla^2 r_i(\mathbf{x}) \right) = J(\mathbf{x})^T J(\mathbf{x}) + Q(\mathbf{x}),$$

donde,

$$Q(\mathbf{x}) = \sum_{i=1}^m r_i(\mathbf{x}) G_i(\mathbf{x}).$$

El hecho de que $\nabla f(\mathbf{x})$ y $\nabla^2 f(\mathbf{x})$ posean una estructura especial y conocida favorece su explotación por los métodos que veremos posteriormente.

Ejemplo 4.7 Supongamos que se desea ajustar a unos datos (t_i, y_i) , $i = 1, \dots, 4$, la función $f(\mathbf{x}, t) = e^{t x_1} + e^{t x_2}$ mediante mínimos cuadrados.

La función residuo es $\mathbf{r}(\mathbf{x}) : \mathbb{R}^2 \rightarrow \mathbb{R}^4$, con $r_i(\mathbf{x}) = e^{t_i x_1} + e^{t_i x_2} - y_i$, $i = 1, \dots, 4$. El problema consiste en minimizar $f(\mathbf{x}) = \frac{1}{2} \mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x})$.

La matriz Jacobiana de $\mathbf{r}(\mathbf{x})$, $J(\mathbf{x}) \in \mathbb{R}^{4 \times 2}$, es

$$J(\mathbf{x}) = \begin{bmatrix} t_1 e^{t_1 x_1} & t_1 e^{t_1 x_2} \\ t_2 e^{t_2 x_1} & t_2 e^{t_2 x_2} \\ t_3 e^{t_3 x_1} & t_3 e^{t_3 x_2} \\ t_4 e^{t_4 x_1} & t_4 e^{t_4 x_2} \end{bmatrix}.$$

⁹ $\mathbf{b} - \mathbf{f}(\mathbf{x})$ con el planteamiento anterior.

El vector gradiente de $\frac{1}{2}\mathbf{r}(\mathbf{x})^T\mathbf{r}(\mathbf{x})$, $\nabla f(\mathbf{x}) \in \mathbb{R}^2$:

$$\nabla f(\mathbf{x}) = J(\mathbf{x})^T \mathbf{r}(\mathbf{x}) = \begin{bmatrix} \sum_{i=1}^4 r_i(\mathbf{x}) t_i e^{t_i x_1} \\ \sum_{i=1}^4 r_i(\mathbf{x}) t_i e^{t_i x_2} \end{bmatrix}.$$

La matriz Hessiana $\nabla^2 f(\mathbf{x}) \in \mathbb{R}^{2 \times 2}$, usando el hecho de que

$$\nabla^2 r_i(\mathbf{x}) = \begin{bmatrix} t_i^2 e^{t_i x_1} & 0 \\ 0 & t_i^2 e^{t_i x_2} \end{bmatrix},$$

es,

$$\begin{aligned} \nabla^2 f(\mathbf{x}) &= J(\mathbf{x})^T J(\mathbf{x}) + Q(\mathbf{x}) \\ &= \begin{bmatrix} \sum_{i=1}^4 t_i^2 e^{t_i x_1} (r_i(\mathbf{x}) + e^{t_i x_1}) & \sum_{i=1}^4 t_i^2 e^{t_i (x_1 + x_2)} \\ \sum_{i=1}^4 t_i^2 e^{t_i (x_1 + x_2)} & \sum_{i=1}^4 t_i^2 e^{t_i x_2} (r_i(\mathbf{x}) + e^{t_i x_2}) \end{bmatrix}. \end{aligned}$$

Como se puede observar, el cálculo analítico de las derivadas de este sencillo problema no es trivial. ●

Como el problema no lineal de mínimos cuadrados es un problema de búsqueda de un mínimo, la condición necesaria de primer orden (ver apéndice A, página 695) para que un punto $\mathbf{x}^*$ sea el óptimo (mínimo o máximo) del problema es que se satisfaga que

$$\nabla f(\mathbf{x}^*) = J(\mathbf{x}^*)^T \mathbf{r}(\mathbf{x}^*) = 0.$$

Cualquier punto que satisface esta condición se denomina *punto crítico*.

Definamos a continuación una condición necesaria y suficiente para que un punto crítico $\mathbf{x}^*$ sea un mínimo local de $f(\mathbf{x})$. Recurramos para ello al siguiente enfoque geométrico de Björk [1996] suponiendo que el problema de minimizar $f(\mathbf{x})$ es el de encontrar un punto en la superficie n -dimensional $\mathbf{z} = \mathbf{r}(\mathbf{x})$ de $\mathbb{R}^m$ más cerca del origen.

Supongamos primero que la matriz Jacobiana en $\mathbf{x}^*$ es de rango completo, n . Se cumplirá que $J(\mathbf{x}^*)^\dagger J(\mathbf{x}^*) = I_n$, donde $J(\mathbf{x}^*)^\dagger$ es la matriz pseudoinversa de la Jacobiana en $\mathbf{x}^*$. Reescribiendo la expresión de la matriz Hessiana de $f(\mathbf{x})$,

$$\nabla^2 f(\mathbf{x}) = J^T J - G_w = J^T \left(I - \gamma (J^\dagger)^T G_w J^\dagger \right) J,$$

donde,

$$G_w = \sum_{i=1}^m w_i Q_i, \quad \mathbf{w} = -\frac{\mathbf{r}}{\|\mathbf{r}\|_2}, \quad \text{y} \quad \gamma = \|\mathbf{r}\|_2.$$

La matriz simétrica

$$K = (J^\dagger)^T G_w J^\dagger$$

es la *matriz de curvatura* normal de la superficie n -dimensional $\mathbf{z} = \mathbf{r}(\mathbf{x})$ en $\mathbb{R}^m$ con respecto al vector normal $\mathbf{w}$.

Si los valores propios de K son

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n,$$

las cantidades $\rho_i = 1/\lambda_i$, $\lambda_i \neq 0$, representan los radios de curvatura de la superficie con respecto al vector normal $\mathbf{w}$.

Si $J(\mathbf{x}^*)$ es de rango completo, la matriz Hessiana $\nabla^2 f(\mathbf{x}^*) = J^T(I - \gamma K)J$ es definida positiva y el punto $\mathbf{x}^*$ un mínimo local del problema si y sólo si $I - \gamma K$ es definida positiva en $\mathbf{x}^*$; esto es así cuando se cumple que

$$1 - \gamma\lambda_1 > 0$$

en $\mathbf{x}^*$. Si $1 - \gamma\lambda_1 \leq 0$, el problema de mínimos cuadrados posee lo que se denomina un *punto de silla* en $\mathbf{x}^*$; si $1 - \gamma\lambda_1 < 0$ $f(\mathbf{x})$ incluso puede tener un máximo en $\mathbf{x}^*$.

La interpretación geométrica del problema no lineal de mínimos cuadrados es la siguiente: se trata de encontrar un punto $\mathbf{x}^* \in \mathbb{R}^m$ en la superficie $\mathbf{z} = \mathbf{r}(\mathbf{x})$ tal que su distancia al origen sea mínima. Cuando, como es el caso de la aproximación de datos, $r_i(\mathbf{x}) = y_i - f(\mathbf{x}, t_i)$, $i = 1, \dots, m$, la superficie es

$$\mathbf{z} = [f(\mathbf{x}, t_1), \dots, f(\mathbf{x}, t_m)]^T \in \mathbb{R}^m.$$

El problema consiste en este caso en encontrar el punto de esta superficie más próximo a $\mathbf{y} \in \mathbb{R}^m$. En el caso de que el número de observaciones sea $m = 2$, existiendo un único parámetro x , el problema se ilustra en la figura 4.18. El radio de curvatura de $\mathbf{z}$ en $\mathbf{x}^*$ es $\rho > \|\mathbf{r}\|_2 = \gamma$; por consiguiente, $1 - \gamma\lambda = 1 - \gamma/\rho > 0$. Se cumple de esta manera la condición necesaria y suficiente de mínimo.

4.6.2 Resolución numérica del problema

La forma de abordar la resolución numérica del problema no lineal de mínimos cuadrados responde a dos enfoques distintos. Uno consiste en contemplar el problema como un sistema de m ecuaciones y n incógnitas ($m > n$),

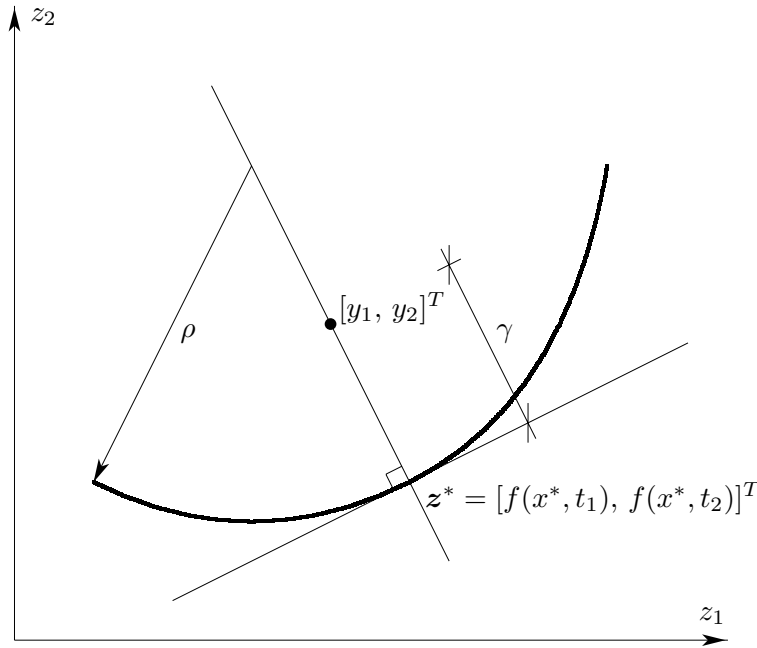
$$\mathbf{r}(\mathbf{x}) = \mathbf{0},$$

en general incompatible. Como se hizo en el caso en que $m = n$, es natural aproximar $\mathbf{r}(\mathbf{x})$ por el modelo lineal que define el desarrollo en serie de Taylor alrededor de un punto $\mathbf{x}_k$ truncándolo a partir de los términos de segundo orden; es decir,

$$M_k(\mathbf{x}) = \mathbf{r}(\mathbf{x}_k) + J(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k).$$

El sistema lineal $M_k(\mathbf{x}) = \mathbf{0}$ será en general incompatible. La solución del problema lineal de mínimos cuadrados que determina $\min_{\mathbf{x} \in \mathbb{R}^n} \|M_k(\mathbf{x})\|_2$ se puede usar para, iterativamente, aproximarse más y más a la solución del problema no lineal (4.36). Este enfoque aboca al método conocido como *Gauss-Newton* y a una variante denominada *Levenberg-Marquardt*.

El segundo enfoque consiste en contemplar el problema como uno de optimización en $\mathbb{R}^n$. Para resolverlo se aproxima $f(\mathbf{x})$ por el modelo cuadrático que resulta de su desarrollo en serie

**Figura 4.18**

Geometría del ajuste de una función no lineal con un parámetro a dos puntos

de Taylor alrededor de un punto truncándolo a partir de los términos de terceras derivadas; es decir,

$$\tilde{M}_k(\mathbf{x}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_k)^T \nabla^2 f(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k).$$

El mínimo de $\tilde{M}_k(\mathbf{x})$ se alcanza en un punto dado por la expresión

$$\mathbf{x}_N = \mathbf{x}_k - \left(J(\mathbf{x}_k)^T J(\mathbf{x}_k) + Q(\mathbf{x}_k) \right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k). \quad (4.38)$$

Esta solución se puede usar para aproximarse paso a paso a la de (4.36). Este enfoque es equivalente al método de Newton aplicado directamente a (4.36).

El método de Gauss-Newton se puede también considerar como un caso particular del de Newton en el que se desprecia $Q(\mathbf{x}_k)$, pues si $J(\mathbf{x}_k)$ es de rango completo, $\mathbf{x}_N$ es la misma solución que se obtendría resolviendo el problema lineal de mínimos cuadrados $\min \|M_k(\mathbf{x}_k)\|_2$. En cualquier caso, $Q(\mathbf{x}_k)$ se podrá despreciar si las $r_i(\mathbf{x})$ son sólo ligeramente no lineales o los residuos $r_i(\mathbf{x}_k)$, $i = 1, \dots, m$, son pequeños; en estos casos el comportamiento del método de Gauss-Newton debe ser muy similar al de Newton. En particular, para un problema compatible en el que $\mathbf{r}(\mathbf{x}^*) = \mathbf{0}$, la convergencia de los dos métodos es la misma.

Para problemas con residuos relativamente grandes o grandes, la convergencia local del método de Gauss-Newton puede ser muy inferior a la del método de Newton. El coste, sin embargo, de calcular las mn^2 derivadas necesarias para llevar a la práctica este último y determinar $Q(\mathbf{x}_k)$, puede hacerlo extremadamente lento.

En el caso de ajuste de funciones a puntos, los $r_i(\mathbf{x}) = y_i - f(\mathbf{x}, t_i)$ y sus derivadas pueden obtenerse con relativa facilidad (sobre todo si están compuestas de sumas de funciones

elementales), por lo que utilizar el método de Newton, al menos a priori, puede resultar más conveniente.

4.6.2.1 Método de Gauss-Newton

El método de Gauss-Newton para resolver problemas no lineales de mínimos cuadrados está basado en la resolución de una sucesión de aproximaciones lineales de $\mathbf{r}(\mathbf{x})$ de acuerdo con el algoritmo de la tabla 4.14.

Tabla 4.14

Algoritmo de Gauss-Newton para resolver problemas no lineales de mínimos cuadrados

Paso 0 – Definir un $\mathbf{x}_0$; hacer $k = 1$ y $\mathbf{x}_k \leftarrow \mathbf{x}_0$.
Paso 1 – Determinar $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{r}(\mathbf{x}_k) + J(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k)\|_2$.
Paso 2 – Si $\mathbf{x} - \mathbf{x}_k < Tol$, parar: el problema está resuelto;
 si no, hacer $k = k + 1$, $\mathbf{x}_k = \mathbf{x}$ e ir al paso 1.

Cada uno de los subproblemas del paso 1 es un problema lineal de mínimos cuadrados del tipo

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2.$$

La solución de esos subproblemas, una dirección de descenso como sabemos, si $J(\mathbf{x}_k)$ es de rango completo, está dada por¹⁰

$$\mathbf{p}_k = \mathbf{x} - \mathbf{x}_k = -\left(J(\mathbf{x}_k)^T J(\mathbf{x}_k)\right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k).$$

Para su resolución numérica, como también sabemos por lo explicado en el capítulo 1 al exponer lo relativo a mínimos cuadrados lineales, no conviene utilizar las ecuaciones normales, por su posible mal condicionamiento numérico, sino, si está disponible, algún método basado en transformaciones ortogonales que factorice la matriz $J(\mathbf{x}_k)$ en la forma QR . La solución, una vez llevada a cabo esa factorización, se obtendría de la resolución de un sistema triangular superior con R .

Si $J(\mathbf{x}_k)$ no tiene rango completo, $\mathbf{p}_k$ debe ser aquel de entre todas las soluciones existentes que tenga norma mínima:

$$\mathbf{p}_k = -J(\mathbf{x}_k)^\dagger \mathbf{r}(\mathbf{x}_k).$$

Ejemplo 4.8 Se desea utilizar el método Gauss-Newton para determinar los parámetros x_1 y x_2 de la función $e^{x_1 + tx_2}$ que mejor se ajuste a los pares de puntos

$$\{(t_i, y_i)\} = \{(-2, 1/2), (-1, 1), (0, 2), (1, 4)\}.$$

¹⁰Recordemos las ecuaciones normales de un problema lineal de mínimos cuadrados

La función $\mathbf{r}(\mathbf{x})$ es en este caso de $\Re^2$ en $\Re^4$. Su matriz Jacobiana es

$$J(\mathbf{x}) = \begin{bmatrix} e^{x_1-2x_2} & -2e^{x_1-2x_2} \\ e^{x_1-x_2} & -e^{x_1-x_2} \\ e^{x_1} & 0 \\ e^{x_1+x_2} & e^{x_1+x_2} \end{bmatrix}.$$

Si se parte de $\mathbf{x}_0 = [1, 1]^T$, el código en FORTRAN 77 que lo resuelve, utilizando como rutina para resolver el subproblema lineal de mínimos cuadrados QRDES de la página 96, es el que sigue. La solución es

$$\mathbf{x} = \begin{bmatrix} \ln 2 \\ \ln 2 \end{bmatrix}.$$

```

PROGRAM Gausnewt
C
  parameter (m=4,n=2)
  double precision f(m),j(m,n),x(n),p(n),tol,dmax,dnor,s
C
  tol = dsqrt(epsilon(1.0d0))
  x   = 1.0
C
C *** Proceso iterativo ***
C
  do i = 1,100
    call fx (f,x,m,n)
    call derfx (j,x,m,n)
    call qrdes (j,f,p,m,n,s)
    x   = x-p
    dnor = dmax(p,n)/dmax(x,n)
    print *,x,s,dnor                                ! Salida de resultados
    if (dnor.lt.tol) exit
  end do
C
  end

  subroutine fx (f,x,m,n)                            ! Cálculo de residuos
  double precision f(m),x(n)
C
  f(1) = dexp(x(1)-2.0*x(2))-0.5
  f(2) = dexp(x(1)-1.0*x(2))-1.0
  f(3) = dexp(x(1))-2.0
  f(4) = dexp(x(1)+x(2))-4.0
C
  return
  end

  subroutine derfx (j,x,m,n)                          ! Evaluación de la matriz
  double precision j(m,n),x(n)                        ! Jacobiana
C
  j(1,1) = dexp(x(1)-2.0*x(2))
  j(1,2) = -2.0*dexp(x(1)-2.0*x(2))
  j(2,1) = dexp(x(1)-x(2))
  j(2,2) = -dexp(x(1)-x(2))
  j(3,1) = dexp(x(1))
  j(3,2) = 0.0
  j(4,1) = dexp(x(1)+x(2))
  j(4,2) = dexp(x(1)+x(2))

```

```
C
    return
end

double precision function dmax (x,n)      ! Función auxiliar
double precision x(n)
C
    dmax = 0.d0
    do i = 1,n
        dmax = dmax1(dabs(x(i)),dmax)
    end do
C
    return
end

subroutine qrdes (a,b,x,m,n,s1)
C
C *** Resolución del problema lineal de mínimos cuadrados mediante
C   factorización QR por transformaciones de Householder.
C
    double precision a(m,n),b(m),d(20),x(n),rmax,beta,sigma,s1
C
C *** Reducción QA=R y vector b a b' ***
C
    do j = 1,n
        rmax = 0.0d0
        do i = j,m
            rmax = dmax1(rmax,dabs(a(i,j)))
        end do
        if (rmax.eq.0.0) stop 'Matriz A de rango incompleto'
        beta = 0.0
        do i = j+1,m
            beta = beta+a(i,j)**2
        end do
        wj      = a(j,j)
        sigma   = sign(dsqrt(beta+wj*wj),wj)
        wj      = wj+sigma
        beta    = 2.0/(beta+wj*wj)
        a(j,j)  = wj
        d(j)    = -sigma
        do l = j+1,n
            s = 0.0
            do k = j,m
                s = s+a(k,j)*a(k,l)
            end do
            s = beta*s
            do k = j,m
                a(k,l) = a(k,l)-a(k,j)*s
            end do
        end do
        s = 0.0
        do k = j,m
            s = s+a(k,j)*b(k)
        end do
        s = beta*s
        do k = j,m
            b(k) = b(k)-a(k,j)*s
        end do
    end do
```

```

C
C *** Resolución Rx = b' ***
C
      x(n) = b(n)/d(n)
      do i = n-1,1,-1
        suma = 0.0
        do k = i+1,n
          suma = suma+a(i,k)*x(k)
        end do
        x(i) = (b(i)-suma)/d(i)
      end do
C
C *** Suma de residuos al cuadrado ***
C
      s1 = 0.0
      do i = n+1,m
        s1 = s1+b(i)**2
      end do
C
      return
      end

```

Los puntos del proceso iterativo que se obtienen son los de la tabla 4.15

Tabla 4.15

Método de Gauss-Newton. Proceso de convergencia a la solución del problema del ejemplo 4.8

k	x_1	x_2	$\ \mathbf{r}\ _2^2$	$\ \mathbf{x}_k - \mathbf{x}_{k-1}\ _\infty / \ \mathbf{x}_k\ _\infty$
1	7,5406407460540E-1	7,8581936682613E-1	7,8266574774450E-3	3,1296747849328E-1
2	6,9782818348320E-1	6,9931189327404E-1	9,1871427399292E-5	1,2370367756599E-1
3	6,9317290130691E-1	6,9317774998543E-1	2,0369290094835E-9	8,8493314314808E-3
4	6,9314718125839E-1	6,9314718138758E-1	1,2555027835829E-18	4,4100591676052E-5
5	6,9314718055994E-1	6,9314718055994E-1	2,1270825699749E-31	1,1933987648446E-9

4.6.2.1.1 Convergencia del método de Gauss-Newton

Como el método de Gauss-Newton se puede considerar resultante del de Newton suprimiendo el término $Q(\mathbf{x}_k)$ —ver ecuación (4.38)—, y la convergencia de éste, bajo los supuestos estándar, es cuadrática, no parece desafortunado suponer que la buena o mala convergencia del método de Gauss-Newton dependerá de la importancia del término suprimido.

Teorema 4.9 Sea la función vectorial $\mathbf{r}(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^m$ y la función $f(\mathbf{x}) = \frac{1}{2}\mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x})$ continua y derivable en un conjunto abierto $D \subset \mathbb{R}^n$. Supóngase que $J(\mathbf{x}) \in \text{Lip}_\gamma(D)$ con $\|J(\mathbf{x})\|_2 \leq \alpha$ para todo $\mathbf{x} \in D$ y que existe un $\mathbf{x}^* \in D$ y unos escalares $\lambda, \sigma \geq 0$ tales que $J(\mathbf{x}^*)^T \mathbf{r}(\mathbf{x}) = \mathbf{0}$, λ es el menor autovalor de $J(\mathbf{x}^*)^T J(\mathbf{x}^*)$ y

$$\|(J(\mathbf{x}) - J(\mathbf{x}^*))^T \mathbf{r}(\mathbf{x}^*)\|_2 \leq \sigma \|\mathbf{x} - \mathbf{x}^*\|_2 \quad (4.39)$$

para todo $\mathbf{x} \in D$. Si $\sigma < \lambda$, entonces para todo $c \in (1, \lambda/\sigma)$, existe un $\varepsilon > 0$ tal que para todo $\mathbf{x}_0 \in S(\mathbf{x}^*, \varepsilon)$, la sucesión de puntos que genera el método de Gauss-Newton,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \left(J(\mathbf{x}_k)^T J(\mathbf{x}_k) \right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k)$$

converge a $\mathbf{x}^*$ cumpliéndose que

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\|_2 \leq \frac{c\sigma}{\lambda} \|\mathbf{x}_k - \mathbf{x}^*\|_2 + \frac{c\alpha\gamma}{2\lambda} \|\mathbf{x}_k - \mathbf{x}^*\|_2^2 \quad (4.40)$$

y

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\|_2 \leq \frac{c\sigma + \lambda}{2\lambda} \|\mathbf{x}_k - \mathbf{x}^*\|_2 < \|\mathbf{x}_k - \mathbf{x}^*\|_2. \quad (4.41)$$

DEMOSTRACIÓN. Por inducción. Supongamos que $\lambda > \sigma \geq 0$ puesto que las conclusiones del teorema sólo serán aplicables a este caso. Sea c una constante en $(1, \lambda/\sigma)$. Para abreviar la notación, hagamos $J(\mathbf{x}_0) = J_0$, $\mathbf{r}(\mathbf{x}_0) = \mathbf{r}_0$ y $\mathbf{r}(\mathbf{x}^*) = \mathbf{r}^*$. Designemos por $\|\cdot\|$ las normas euclídea y espectral.

Con los supuestos mencionados, existe un $\varepsilon_1 > 0$ tal que $J_0^T J_0$ es regular, cumpliéndose que

$$\left\| \left(J_0^T J_0 \right)^{-1} \right\| \leq \frac{c}{\lambda} \quad \text{para } \mathbf{x}_0 \in S(\mathbf{x}^*, \varepsilon). \quad (4.42)$$

Sea

$$\varepsilon = \min \left\{ \varepsilon_1, \frac{\lambda - c\sigma}{c\alpha\gamma} \right\}. \quad (4.43)$$

Se tiene que

$$\begin{aligned} \mathbf{x}_1 - \mathbf{x}^* &= \mathbf{x}_0 - \mathbf{x}^* - \left(J_0^T J_0 \right)^{-1} J_0^T \mathbf{r}_0 \\ &= - \left(J_0^T J_0 \right)^{-1} \left[J_0^T \mathbf{r}_0 + J_0^T J_0 (\mathbf{x}^* - \mathbf{x}_0) \right] \\ &= - \left(J_0^T J_0 \right)^{-1} \left[J_0^T \mathbf{r}^* - J_0^T (\mathbf{r}^* - \mathbf{r}_0 - J_0 (\mathbf{x}^* - \mathbf{x}_0)) \right]. \end{aligned} \quad (4.44)$$

De acuerdo con el lema 4.3,

$$\|\mathbf{r}^* - \mathbf{r}_0 - J_0 (\mathbf{x}^* - \mathbf{x}_0)\| \leq \frac{\gamma}{2} \|\mathbf{x}_0 - \mathbf{x}^*\|^2. \quad (4.45)$$

De la expresión (4.39), recordando que $J(\mathbf{x}^*)^T \mathbf{r}(\mathbf{x}^*) = \mathbf{0}$,

$$\|J_0^T \mathbf{r}^*\| \leq \sigma \|\mathbf{x} - \mathbf{x}^*\|. \quad (4.46)$$

Combinando (4.44), (4.42), (4.45) y (4.46) y que $\|J_0\| \leq \alpha$, se tiene que

$$\begin{aligned} \|\mathbf{x}_1 - \mathbf{x}^*\| &\leq \|(J_0^T J_0)^{-1}\| \left[\|J_0^T \mathbf{r}^*\| + \|J_0\| \|\mathbf{r}^* - \mathbf{r}_0 - J_0(\mathbf{x}^* - \mathbf{x}_0)\| \right] \\ &\leq \frac{c}{\gamma} \left[\sigma \|\mathbf{x}_0 - \mathbf{x}^*\| + \frac{\alpha\gamma}{2} \|\mathbf{x}_0 - \mathbf{x}^*\|^2 \right], \end{aligned}$$

lo que prueba (4.40) en el caso $k = 0$. De (4.43) y la última desigualdad,

$$\begin{aligned} \|\mathbf{x}_1 - \mathbf{x}^*\| &\leq \|\mathbf{x}_0 - \mathbf{x}^*\| \left[\frac{c\alpha}{\lambda} + \frac{c\alpha\gamma}{2\lambda} \|\mathbf{x}_0 - \mathbf{x}^*\| \right] \\ &\leq \|\mathbf{x}_0 - \mathbf{x}^*\| \left[\frac{c\gamma}{\lambda} + \frac{\lambda - c\sigma}{2\lambda} \right] \\ &= \frac{c\sigma + \lambda}{2\lambda} \|\mathbf{x}_0 - \mathbf{x}^*\| \\ &< \|\mathbf{x}_0 - \mathbf{x}^*\|, \end{aligned}$$

lo que prueba (4.41) en el caso en que $k = 0$. Para probar los demás pasos de la inducción se procedería de forma idéntica. ■

Corolario 4.2 *Con los mismos supuestos del teorema anterior, si $\mathbf{r}(\mathbf{x}^*) = \mathbf{0}$, existe un $\varepsilon > 0$ tal que para todo $\mathbf{x}_0 \in S(\mathbf{x}^*, \varepsilon)$, la sucesión de puntos que genera el método de Gauss-Newton converge cuadráticamente a $\mathbf{x}^*$.*

DEMOSTRACIÓN. Si $\mathbf{r}(\mathbf{x}^*) = \mathbf{0}$, σ se puede escoger igual a cero en la expresión (4.39) por lo que, de acuerdo con (4.41) y (4.40), la sucesión de puntos converge a $\mathbf{x}^*$ cuadráticamente. ■

La constante σ juega un papel muy importante en estos últimos teorema y corolario. En cierta medida, se puede ver como el valor de $\|Q(\mathbf{x}^*)\|_2$ pues, para $\mathbf{x}$ suficientemente próximos a $\mathbf{x}^*$, se comprueba fácilmente que

$$(J(\mathbf{x}) - J(\mathbf{x}^*))^T \mathbf{r}(\mathbf{x}^*) \cong Q(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*).$$

Con esta interpretación, o directamente de (4.39), se ve que σ representa de forma combinada el grado de no linealidad del problema y el tamaño de sus residuos; si $\mathbf{r}(\mathbf{x}^*) = \mathbf{0}$ o $\mathbf{r}(\mathbf{x})$ es lineal, $\sigma = 0$. De acuerdo con esto, el teorema 4.9 dice que el orden de convergencia del método de Gauss-Newton decrece a medida que crece la no linealidad del problema y el tamaño de los residuos.

4.6.2.2 Métodos de Gauss-Newton globalmente convergentes

Estos métodos, siguiendo los mismos principios expuestos en el apartado 4.5, de la página 331, son modificaciones del de Gauss-Newton que persiguen resolver de forma robusta, partiendo

de cualquier punto, el problema no lineal de mínimos cuadrados. A tal efecto, calculada la dirección de acercamiento a la solución, $\mathbf{p}_k = \mathbf{x} - \mathbf{x}_k$, resultado de

$$\underset{\mathbf{p} \in \mathbb{R}^n}{\text{minimizar}} \|\mathbf{r}(\mathbf{x}_k) + J(\mathbf{x}_k)\mathbf{p}\|_2,$$

en lugar de tomar un paso en esa dirección igual a 1, cosa que hace el método de Gauss-Newton, se multiplica por un factor α_k de tal manera que se mejore la solución obtenida hasta ese momento.

La amplitud del paso α_k , o *factor de amortiguación*, se toma de acuerdo con uno de estos criterios:

- a) Escoger el mayor α_k de la sucesión $1, \frac{1}{2}, \frac{1}{4}, \dots$, para el cual

$$\|\mathbf{r}(\mathbf{x}_k)\|_2^2 - \|\mathbf{r}(\mathbf{x}_k + \alpha_k \mathbf{p}_k)\|_2^2 \geq \frac{1}{2} \alpha_k \|J(\mathbf{x}_k) \mathbf{p}_k\|_2^2.$$

Este criterio es esencialmente la regla de Armijo ya expuesta en el apartado 4.5.

- b) Escoger el α_k solución de

$$\underset{\alpha \in \mathbb{R}}{\text{minimizar}} \|\mathbf{r}(\mathbf{x}_k + \alpha \mathbf{p}_k)\|_2^2.$$

Como los dos criterios funcionan adecuadamente, en la literatura especializada se aconseja indistintamente utilizar uno u otro. Incluso se pueden combinar los dos en algún caso según progresa el procedimiento correspondiente. El nombre que se le da a α_k es el que designa este tipo de métodos: de Gauss-Newton amortiguado.

4.6.2.3 Métodos de región de confianza. Método de Levenberg-Marquardt

Estos métodos surgieron para evitar las dificultades que el método de Gauss-Newton experimenta cuando a lo largo del proceso iterativo, en algún punto, la matriz Jacobiana no tiene rango completo. Para evitar esta dificultad, Levenberg [1944] y Marquardt [1963] sugirieron calcular la dirección, $\mathbf{p}_k = \mathbf{x} - \mathbf{x}_k$, mediante la resolución del subproblema

$$\underset{\mathbf{p}_k \in \mathbb{R}^n}{\text{minimizar}} \left\{ \|\mathbf{r}(\mathbf{x}_k) + J(\mathbf{x}_k) \mathbf{p}_k\|_2^2 = \mu_k \|\mathbf{p}_k\|_2^2 \right\},$$

donde el parámetro μ_k controla y limita el tamaño de $\mathbf{p}_k$. Obsérvese que $\mathbf{p}_k$ está definido aun cuando $J(\mathbf{x}_k)$ no tenga rango completo. Conforme $\mu_k \rightarrow \infty$, $\|\mathbf{p}_k\|_2 \rightarrow 0$ y $\mathbf{p}_k$ se hace paralela a la dirección de máxima pendiente.

Así definido, ese subproblema se puede comprobar que es equivalente al problema de optimización con condición cuadrática

$$\begin{aligned} &\underset{\mathbf{p} \in \mathbb{R}^n}{\text{minimizar}} \quad \|\mathbf{r}(\mathbf{x}_k) + J(\mathbf{x}_k) \mathbf{p}\|_2 \\ &\text{sujeta a} \quad \|\mathbf{p}\|_2 \leq \delta_k, \end{aligned} \tag{4.47}$$

donde $\mu_k = 0$ si en la solución las condiciones no se cumplen exactamente (no están activas) y $\mu_k > 0$, si están activas. El conjunto de vectores $\mathbf{p}$ factibles, $\|\mathbf{p}\|_2 \leq \delta_k$, se puede interpretar

como la *región de confianza* del modelo lineal $\mathbf{r}(\mathbf{x}) \cong \mathbf{r}(\mathbf{x}_k) + J(\mathbf{x}_k)\mathbf{p}$, $\mathbf{p} = \mathbf{x} - \mathbf{x}_k$, dentro de la cual se limita la búsqueda del óptimo del problema.

En Dennis y Schnabel [1983] y [1996] se puede encontrar la demostración de que la solución del problema (4.47) es

$$\mathbf{p} = - \left(J(\mathbf{x}_k)^T J(\mathbf{x}_k) + \mu_k I \right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k), \quad (4.48)$$

donde $\mu_k = 0$ si

$$\delta_k \geq \left\| \left(J(\mathbf{x}_k)^T J(\mathbf{x}_k) \right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k) \right\|_2$$

y $\mu_k > 0$ en cualquier otro caso. Las propiedades de la convergencia local del método Levenberg-Marquardt son muy similares a las de método de Gauss-Newton.

El algoritmo de Levenberg-Marquardt es el que describe la tabla 4.16.

Ejemplo 4.9 Utilizando el método de Levenberg-Marquardt, ajustar a la función

$$f(x) = \frac{b_1}{1 + b_2 e^{tb_3}}$$

el conjunto de puntos de la tabla 4.17.

Si se parte del punto $\mathbf{x}_0 = [200, 30, -0.4]^T$, el código en FORTRAN 77 que lo resuelve, utilizando GAUSS como rutina para resolver el subproblema (4.48), es el que sigue.

```
PROGRAM Levmar
C
```

Tabla 4.16

Algoritmo de Levenberg-Marquardt para resolver problemas no lineales de mínimos cuadrados

```
Paso 0 - Definir un  $\mathbf{x}_0 \in \mathbb{R}^n$ ; hacer  $\mu = 0.1$ ,  $k = 1$  y  $\mathbf{x}_k \leftarrow \mathbf{x}_0$ .
Paso 1 - Calcular  $\mathbf{p}_k = - \left( J(\mathbf{x}_k)^T J(\mathbf{x}_k) + \mu I \right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k)$ .
Paso 2 - if ( $\|\mathbf{r}(\mathbf{x}_k + \mathbf{p}_k)\|_2^2 < \|\mathbf{r}(\mathbf{x}_k)\|_2^2$ ) then
    si  $\mathbf{p}_k \leq Tol$ , parar: problema resuelto.
    si  $\mathbf{p}_k > Tol$ , hacer:
         $\mu \leftarrow \mu/10$ 
         $k \leftarrow k + 1$ 
         $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \mathbf{p}$ 
    y volver al paso 1.
else
     $\mu \leftarrow 10\mu$ 
    Volver al paso 1 sin tener que calcular  $J(\mathbf{x}_k)$ .
end
```


Tabla 4.17

Datos del problema no lineal de mínimos cuadrados del ejemplo 4.9

t_i	y_i	t_i	y_i
1	5,308	7	31,443
2	7,24	8	38,558
3	9,638	9	50,156
4	12,866	10	62,948
5	17,069	11	75,995
6	23,192	12	91,972

```

parameter (m=12,n=3)
double precision f(m),j(m,n),jtj(n,n),a(n,n),x(n),s(n),tol,dmax,
+          dnor,mu,res,b(n),prod,f1(m),res1
C
tol = dsqrt(epsilon(1.0d0))
x(1) = 200.                                ! Valores de partida
x(2) = 30.                                  !   de los parámetros
x(3) = -0.4                                 !   a estimar
mu   = 0.1d0
C
do k = 1,100
  call fx (f,x,m,n)
  call derfx (j,x,m,n)
  do i = 1,n
    do l = 1,n
      jtj(i,l) = prod(j(1,i),j(1,l),m)
    end do
  end do
  res = prod(f,f,m)
  do kk=1,100
    do i = 1,n
      b(i) = prod(j(1,i),f,m)
    end do
    a = jtj
    do i = 1,n
      a(i,i) = jtj(i,i)+mu
    end do
    call gauss (a,b,s,n)
    b = x-s
    call fx (f1,b,m,n)
    res1 = prod(f1,f1,m)
    if (res1.lt.res) then
      x = b
      f = f1
      dnor = dmax(s,n)/dmax(x,n)
      print *,x,res1,mu,dnor
      if (dnor.le.tol) stop
      mu = mu/10.d0
      exit
    else
      mu = mu*10.d0
      cycle
    endif
  end do
end do

```

```

C
    end

    subroutine fx (f,x,m,n)
    double precision f(m),x(n)
C
    f(1) = x(1)/(1+x(2)*dexp(x(3)))-5.308
    f(2) = x(1)/(1+x(2)*dexp(2*x(3)))-7.24
    f(3) = x(1)/(1+x(2)*dexp(3*x(3)))-9.638
    f(4) = x(1)/(1+x(2)*dexp(4*x(3)))-12.866
    f(5) = x(1)/(1+x(2)*dexp(5*x(3)))-17.069
    f(6) = x(1)/(1+x(2)*dexp(6*x(3)))-23.192
    f(7) = x(1)/(1+x(2)*dexp(7*x(3)))-31.443
    f(8) = x(1)/(1+x(2)*dexp(8*x(3)))-38.558
    f(9) = x(1)/(1+x(2)*dexp(9*x(3)))-50.156
    f(10) = x(1)/(1+x(2)*dexp(10*x(3)))-62.948
    f(11) = x(1)/(1+x(2)*dexp(11*x(3)))-75.995
    f(12) = x(1)/(1+x(2)*dexp(12*x(3)))-91.972
C
    return
    end

    subroutine derfx (j,x,m,n)
    double precision j(m,n),x(n)
C
    j(1,1) = 1/(1+x(2)*dexp(x(3)))
    j(1,2) = -x(1)*dexp(x(3))/(1+x(2)*dexp(x(3)))**2
    j(1,3) = -x(1)*x(2)*dexp(x(3))/(1+x(2)*dexp(x(3)))**2
    j(2,1) = 1/(1+x(2)*dexp(2*x(3)))
    j(2,2) = -x(1)*dexp(2*x(3))/(1+x(2)*dexp(2*x(3)))**2
    j(2,3) = -x(1)*x(2)*dexp(2*x(3))*2/(1+x(2)*dexp(2*x(3)))**2
    j(3,1) = 1/(1+x(2)*dexp(3*x(3)))
    j(3,2) = -x(1)*dexp(3*x(3))/(1+x(2)*dexp(3*x(3)))**2
    j(3,3) = -x(1)*x(2)*dexp(3*x(3))*3/(1+x(2)*dexp(3*x(3)))**2
    j(4,1) = 1/(1+x(2)*dexp(4*x(3)))
    j(4,2) = -x(1)*dexp(4*x(3))/(1+x(2)*dexp(4*x(3)))**2
    j(4,3) = -x(1)*x(2)*dexp(4*x(3))*4/(1+x(2)*dexp(4*x(3)))**2
    j(5,1) = 1/(1+x(2)*dexp(5*x(3)))
    j(5,2) = -x(1)*dexp(5*x(3))/(1+x(2)*dexp(5*x(3)))**2
    j(5,3) = -x(1)*x(2)*dexp(5*x(3))*5/(1+x(2)*dexp(5*x(3)))**2
    j(6,1) = 1/(1+x(2)*dexp(6*x(3)))
    j(6,2) = -x(1)*dexp(6*x(3))/(1+x(2)*dexp(6*x(3)))**2
    j(6,3) = -x(1)*x(2)*dexp(6*x(3))*6/(1+x(2)*dexp(6*x(3)))**2
    j(7,1) = 1/(1+x(2)*dexp(7*x(3)))
    j(7,2) = -x(1)*dexp(7*x(3))/(1+x(2)*dexp(7*x(3)))**2
    j(7,3) = -x(1)*x(2)*dexp(7*x(3))*7/(1+x(2)*dexp(7*x(3)))**2
    j(8,1) = 1/(1+x(2)*dexp(8*x(3)))
    j(8,2) = -x(1)*dexp(8*x(3))/(1+x(2)*dexp(8*x(3)))**2
    j(8,3) = -x(1)*x(2)*dexp(8*x(3))*8/(1+x(2)*dexp(8*x(3)))**2
    j(9,1) = 1/(1+x(2)*dexp(9*x(3)))
    j(9,2) = -x(1)*dexp(9*x(3))/(1+x(2)*dexp(9*x(3)))**2
    j(9,3) = -x(1)*x(2)*dexp(9*x(3))*9/(1+x(2)*dexp(9*x(3)))**2
    j(10,1) = 1/(1+x(2)*dexp(10*x(3)))
    j(10,2) = -x(1)*dexp(10*x(3))/(1+x(2)*dexp(10*x(3)))**2
    j(10,3) = -x(1)*x(2)*dexp(10*x(3))*10/(1+x(2)*dexp(10*x(3)))**2
    j(11,1) = 1/(1+x(2)*dexp(11*x(3)))
    j(11,2) = -x(1)*dexp(11*x(3))/(1+x(2)*dexp(11*x(3)))**2
    j(11,3) = -x(1)*x(2)*dexp(11*x(3))*11/(1+x(2)*dexp(11*x(3)))**2
    j(12,1) = 1/(1+x(2)*dexp(12*x(3)))

```

```
j(12,2) = -x(1)*dexp(12*x(3))/(1+x(2)*dexp(12*x(3)))**2
j(12,3) = -x(1)*x(2)*dexp(12*x(3))*12/(1+x(2)*dexp(12*x(3)))**2
C
return
end

double precision function dmax (x,n)
double precision x(n)
C
dmax = 0.d0
do i = 1,n
    dmax = dmax1(dabs(x(i)),dmax)
end do
C
return
end

double precision function prod (x,y,n)
double precision x(n),y(n)
C
prod = 0.d0
do i = 1,n
    prod = prod+x(i)*y(i)
end do
C
return
end

subroutine gauss (a,b,x,n)
C
C *** Resolución del sistema lineal de ecuaciones mediante eliminación
C   de Gauss
C
integer ipvt(10),pi
c
double precision a(n,n),b(n),x(n),smax,r,r1,c
C
do i = 1,n
    ipvt(i) = i
end do
C
C *** Triangularización ***
C
do k = 1,n-1
    l = 0
    smax = dabs(a(ipvt(k),k))
    do i = k+1,n
        ip = ipvt(i)
        if (dabs(a(ip,k)).gt.smax) then
            l = i
            smax = dabs(a(ip,k))
        endif
    end do
    if (l.ne.0) then
       iaux = ipvt(k)
        ipvt(k) = ipvt(l)
        ipvt(l) = iaux
    endif
    pi = ipvt(k)
```

```

    r1 = 1.0/a(pi,k)
    do i = k+1,n
        ip = ipvt(i)
        r = a(ip,k)*r1
        do j = k+1,n
            a(ip,j) = a(ip,j)-r*a(pi,j)
        end do
        a(ip,k) = -r
    end do
end do
C
do k = 1,n-1
    ip = ipvt(k)
    do i = k+1,n
        pi = ipvt(i)
        b(pi) = b(pi)+a(pi,k)*b(ip)
    end do
end do
C
C *** Sustitución inversa ***
C
x(n) = b(ipvt(n))/a(ipvt(n),n)
do i = n-1,1,-1
    pi = ipvt(i)
    c = b(pi)
    do j = i+1,n
        c = c-a(pi,j)*x(j)
    end do
    x(i) = c/a(pi,i)
end do
C
return
end

```

Para compactar la exposición, se ha calculado la matriz jacobiana analíticamente. Cuando la complejidad del cálculo de esta matriz es mayor, lo usual es calcularla mediante su aproximación por diferencias finitas.

Los puntos del proceso iterativo que se obtienen son los de la tabla 4.18. ●

Tabla 4.18

Método de Levenberg-Marquardt. Proceso de convergencia a la solución del problema del ejemplo 4.9

k	x_1	x_2	x_3	$\ \mathbf{r}\ _2^2$	μ	$\ \mathbf{x}_k - \mathbf{x}_{k-1}\ _\infty / \ \mathbf{x}_k\ _\infty$
1	172,054602169	27,576875139	-2,852439035E-1	225,726827415	1,0E-1	1,624216E-1
2	180,427478579	40,906816931	-3,173500543E-1	85,897325602	1,0E-2	7,387977E-2
3	190,598767569	47,354495934	-3,150142518E-1	3,211830744	1,0E-3	5,336492E-2
4	195,701854540	48,994138800	-3,137199088E-1	2,589465629	1,0E-4	2,607582E-2
5	196,177702377	49,090471766	-3,135736444E-1	2,587278602	1,0E-5	2,425595E-3
6	196,186188183	49,091630737	-3,135697714E-1	2,587278212	1,0E-6	4,325384E-5
7	196,186260992	49,091641855	-3,135697367E-1	2,587278212	1,0E-7	3,711204E-7
8	196,186261646	49,091641954	-3,135697364E-1	2,587278212	1,0E-8	3,337921E-9

4.6.2.4 Métodos tipo Newton

Los métodos tipo Newton determinan una dirección de mejora de la solución, desde un punto $\mathbf{x}_k$, cuya expresión es la siguiente:

$$\mathbf{p}_k = - \left(J(\mathbf{x}_k)^T J(\mathbf{x}_k) + Q(\mathbf{x}_k) \right)^{-1} J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k).$$

Llevar esto a la práctica conlleva el cálculo de la complicada matriz

$$Q(\mathbf{x}_k) = \sum_{i=1}^m r_i(\mathbf{x}_k) \nabla^2 r_i(\mathbf{x}_k).$$

El método de este tipo más referenciado en la literatura especializada, y que usan alguno de los paquetes de *software* comercialmente disponibles para el tratamiento de problemas generales de mínimos cuadrados no lineales, es el debido a Dennis, Gay y Welsch [1981]. En lugar de calcular en cada iteración la matriz $\nabla^2 \phi(\mathbf{x}_k) = J(\mathbf{x}_k)^T J(\mathbf{x}_k) + Q(\mathbf{x}_k)$, se la aproxima por $\nabla^2 \phi(\mathbf{x}_k) = J(\mathbf{x}_k)^T J(\mathbf{x}_k) + B_k$, donde B_k es una aproximación cuasi Newton de la matriz $Q(\mathbf{x}_k)$, con $B_0 = 0$, que se requiere sea simétrica. Esta aproximación debe satisfacer unas denominadas *relaciones cuasi Newton*:

$$B_k(\mathbf{x}_k - \mathbf{x}_{k-1}) = \mathbf{z}_k, \quad \mathbf{z}_k = J(\mathbf{x}_k)^T \mathbf{r}(\mathbf{x}_k) - J(\mathbf{x}_{k-1})^T \mathbf{r}(\mathbf{x}_{k-1}). \quad (4.49)$$

Si se hace $\mathbf{s}_k = \mathbf{x}_k - \mathbf{x}_{k-1}$, en Dennis y Schnabel [1983] y [1996] se demuestra que la solución de (4.49) que minimiza el cambio que tiene lugar desde B_{k-1} es:

$$B_k = B_{k-1} + \frac{(\mathbf{z}_k - B_{k-1} \mathbf{s}_k) \mathbf{y}_k^T + \mathbf{y}_k (\mathbf{z}_k - B_{k-1} \mathbf{s}_k)^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{(\mathbf{z}_k - B_{k-1} \mathbf{s}_k)^T \mathbf{s}_k \mathbf{y}_k \mathbf{y}_k^T}{(\mathbf{y}_k^T \mathbf{s}_k)^2}.$$

Esta fórmula garantiza que el cambio de la norma de Frobenius de la matriz B_{k-1} es mínimo.

Referencias

La referencia esencial en la que están basados los resultados más importantes de este capítulo y la mayor parte de lo expuesto es Dennis y Schnabel [1983] y [1996]; algunos ejemplos y apuntes son de Hager [1988] y Ortega y Rheinboldt [1970].

Los ejemplos y análisis de los diversos aspectos de problemas que se plantean en sistemas eléctricos de generación y transporte de energía eléctrica son del autor. Para profundizar más en ellos se pueden consultar Bergen [1986], Elgerd [1983] y Grainger y Stevenson [1994].

El apartado relativo a rapidez y velocidad de convergencia se puede también estudiar en Gill, Murray y Wright [1981] y Luenberger [1984]. Está muy bien tratado en Nash y Sofer [1996].

Todos los programas de ordenador son del autor; alternativas a ellos se pueden encontrar en Atkinson, Harley y Hudson [1989] y Press y otros [1986], [1992] y [1996].

El apartado relativo a mínimos cuadrados no lineales, además de en Dennis y Schnabel [1983] y [1996], está muy bien tratado en Björk [1990] y [1996]. Lo que hace referencia al estimador de máxima verosimilitud es estándar y se puede encontrar en cualquier buen libro de estadística.

Ejercicios

- 4.1. Estudiar la convergencia de la sucesión $\{x_k\}$ definida por la relación

$$x_k = 1 + 2^{-2^k}.$$

- 4.2. Estudiar la convergencia de la sucesión $\{x_k\}$ definida por

$$x_k = 1 + 2^{-k}.$$

- 4.3. Estudiar la convergencia de la sucesión $\{x_k\}$ definida por

$$x_k = 1 + \frac{1}{k!}.$$

¿Converge linealmente?

- 4.4. Considérese la función no lineal,

$$f(x) = x^4 - 12x^3 + 47x^2 - 60x.$$

Efectuar una iteración del método de Newton partiendo de $x_0 = 2$. ¿A qué punto convergerá?
¿Qué pasa si parte de $x_0 = 1$?

- 4.5. Determinar el orden de convergencia del método de Newton al aplicarlo para resolver las ecuaciones $x^2 = 0$, $x^3 = 0$, $x + x^3 = 0$ y $x + x^4 = 0$.
4.6. Obtener el orden de convergencia, utilizando Newton y partiendo de los puntos indicados, en cada uno de los problemas siguientes:

- a) $\cos(x) = 0$, $x = \pi/2$.
- b) $1 + \cos(2x) = 0$, $x = \pi/2$.
- c) $x^2 - 4 = 0$, $x = 2$.

- 4.7. Sea la función

$$f(x) = \begin{cases} x \operatorname{sen}\left(\frac{1}{x}\right), & x \neq 0 \\ 0, & x = 0. \end{cases}$$

Demostrar que $f(x)$ es Lipschitz continua pero no diferenciable.

- 4.8. Considérese la ecuación $x^2 = 2$.

- a) Analizar gráficamente cómo se comporta el método de Newton para resolverla partiendo de un punto cercano a cero.
- b) Analizar gráficamente el comportamiento del método de la secante si $x_0 \approx -0,3$ y $x_1 \approx 0,3$.
- c) Analizar gráficamente el comportamiento del método de Newton modificado.
- d) Si el método de la secante converge a una raíz de $f(x)$, debido a los errores de redondeo, es posible que $f(x_k) \approx 0$. ¿Cuál es en este caso x_{k+1} ? Al programar el método, ¿qué precauciones habría que tomar para tener en cuenta el caso en que $f(x_k) = f(x_{k+1}) \approx 0$?

- 4.9. Dada $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, donde

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} 3x_1^2 - 2x_2 \\ x_2^3 - \frac{1}{x_1} \end{bmatrix},$$

calcular $J(\mathbf{x})$ en $\mathbf{x} = [1, 1]^T$. Calcularla también por diferencias finitas con $h = 0,001$.

4.10. Escribir una iteración del método de Newton para resolver el sistema de ecuaciones

$$\begin{array}{rcl} x_1^2 + 2x_1x_2 + \operatorname{sen}(x_3) & = & 0 \\ x_1 + x_2 + x_3 & = & 0 \\ \cos(x_1) + e^{x_3} & = & 0, \end{array}$$

partiendo del punto $x_1 = x_2 = x_3 = 0$. ¿Se puede realmente aplicar el método de Newton partiendo de ese punto?

4.11. Considérese la aplicación del método de Newton para resolver $\mathbf{f}(\mathbf{x}) = \mathbf{0}$, donde

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2^2 + x_2 \\ e^{x_3} - 1 \end{bmatrix}.$$

- ¿Cuál es $f'_1(\mathbf{0})$?
- Considérese cada componente $f_i(\mathbf{x})$ como una función univariable. ¿Cuál es la constante Lipschitz de $f'(x)$ en el intervalo $[-a, a]$ (cuál es la cota de $|(f'(x) - f'(0))/x|$ en ese intervalo)?
- ¿Cuál es la matriz $J(\mathbf{x})$ en el punto $\mathbf{x} = \mathbf{0}$?
- ¿Cuál es la constante de Lipschitz de $J(\mathbf{x})$ en un intervalo de radio a alrededor de $\mathbf{x} = \mathbf{0}$?
- ¿Cuál es la región de convergencia del método de Newton al aplicarlo para resolver $\mathbf{f}(\mathbf{x}) = \mathbf{0}$?
- ¿Cuál sería esa región de convergencia del método de Newton si $x_3^0 = 0$? ¿Y si $x_2^0 = x_3^0 = 0$?

4.12. Dado el sistema no lineal de ecuaciones

$$\begin{array}{rcl} 6 \arctan(x_1 - 10) - 2e^{-x_2} - 2e^{-x_3} + 2x_2 + 2x_3 - 9 & = & 0 \\ 2 \arctan(x_1 - 10) - 4e^{-x_2} - e^{-x_3} + 7x_2 - 2x_3 - 3 & = & 0 \\ 2 \arctan(x_1 - 10) - e^{-x_2} - 3e^{-x_3} - x_2 + 5x_3 - 3 & = & 0. \end{array}$$

- Utilizar el método de Newton para resolverlo comenzando desde el punto $\mathbf{x} = \mathbf{0}$.
- Incorporar el criterio de Armijo y resolverlo otra vez. ¿Qué pasa? ¿Por qué?

4.13. Utilizar el método de Newton para resolver el sistema no lineal

$$\begin{array}{rcl} 3x_1 - \cos(x_2x_3) - 0,5 & = & 0 \\ x_1^2 - 625x_2^2 & = & 0 \\ e^{-x_1x_2} + 20x_3 + 9 & = & 0. \end{array}$$

Hay que tener cuidado pues el sistema admite más de una solución.

4.14. Utilizar el método de Newton para resolver los sistemas:

$$\begin{array}{l} \text{a)} \quad 3x_1 - \cos(x_2x_3) - 0,5 = 0 \\ \quad \quad x_1^2 - 625x_2^2 = 0 \\ \quad \quad e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} = 0. \end{array}$$

$$\begin{array}{l} \text{b)} \quad x_1 - 10x_2 + 9 = 0 \\ \quad \quad \sqrt{3}(x_3 - x_4) = 0 \\ \quad \quad (x_2 - 2x_3 + 1)^2 = 0 \\ \quad \quad \sqrt{2}(x_1 - x_4)^2 = 0. \end{array}$$

¿Qué ocurre con la matriz Jacobiana en la solución? ¿Cómo se afecta la convergencia por ello?

4.15. Sea $\mathbf{f} : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, con

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}.$$

Sea $\mathbf{x}_k = [10^7, 10^{-7}]^T$ y supóngase que estamos usando un ordenador de base 14 y 10 dígitos significativos. ¿Qué pasa si se aproxima la matriz Jacobiana en ese punto por diferencias finitas con $h = 1$? ¿Y con 10^{-14} ? ¿Existe algún h óptimo?

4.16. Efectuar dos iteraciones del método de Broyden para resolver el sistema de ecuaciones

$$\begin{aligned} x_1 + x_2 - 3 &= 0 \\ x_1^2 + x_2^2 - 9 &= 0, \end{aligned}$$

partiendo del punto $\mathbf{x}_0 = [2, 7]^T$ y tomando $A_0 = J(\mathbf{x}_0)$.

4.17. Sea $\mathbf{r}(\mathbf{x}) : \mathbb{R}^4 \rightarrow \mathbb{R}^{20}$, $r_i(\mathbf{x}) = x_1 + x_2 e^{-(t_i + x_3)^2/x_4} - y_i$, $i = 1, \dots, 20$. El problema consiste en minimizar $f(\mathbf{x}) = \frac{1}{2} \mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x})$. ¿Cuál es la matriz $J(\mathbf{x})$? ¿Y $Q(\mathbf{x})$? ¿Y $\nabla^2 f(\mathbf{x})$?

4.18. Sea $\mathbf{r}(\mathbf{x}) : \mathbb{R}^2 \rightarrow \mathbb{R}^4$, $r_i(\mathbf{x}) = e^{x_1 + t_i x_2} - y_i$, $i = 1, \dots, 4$. El problema consiste en minimizar $f(\mathbf{x}) = \frac{1}{2} \mathbf{r}(\mathbf{x})^T \mathbf{r}(\mathbf{x})$. Supóngase que $t_1 = -2$, $t_2 = -1$, $t_3 = 0$, $t_4 = 1$, $y_1 = 0.5$, $y_2 = 1$, $y_3 = 2$ y $y_4 = 4$ ($f(\mathbf{x}) = 0$ en $\mathbf{x}^* = [\ln 2, \ln 2]^T$). Efectuar una iteración del método de Gauss-Newton y otra del de Newton partiendo de $\mathbf{x}_0 = [1, 1]^T$. ¿Qué ocurre si los valores de y_1 y y_4 se cambian a 5 y -4, respectivamente?

4.19. Dada $R \in \mathbb{R}^m$ y $J \in \mathbb{R}^{m \times n}$, probar que $\mathbf{s} = -(J^T J + \mu I)^{-1} J^T R$ es la solución del problema de mínimos cuadrados

$$\underset{\mathbf{s} \in \mathbb{R}^n}{\text{minimizar}} \|\mathbf{A}\mathbf{s} + \mathbf{b}\|_2,$$

donde $A \in \mathbb{R}^{(m+n) \times n}$, $\mathbf{b} \in \mathbb{R}^{m+n}$ y

$$A = \begin{bmatrix} J \\ \mu^{1/2} I \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}.$$

4.20. Un experimento biológico consiste en la determinación de la temperatura máxima del agua, X_M , a la cual pueden sobrevivir varias especies de hidra sin que su tiempo de vida esperado disminuya. Un enfoque para resolver este problema consiste en usar un ajuste de mínimos cuadrados ponderado de la forma $f(\mathbf{x}) = y = a/(\mathbf{x} - a)^c$ a una colección de datos experimentales. Los valores de los datos, $\mathbf{x}$, se refieren a las temperaturas del agua por encima de X_M y los valores y al promedio de tiempo de vida a esa temperatura. La constante b es una asíntota a la gráfica de f y como tal es una aproximación a X_M .

a) Demuéstrese que escoger a , b y c para minimizar

$$\sum_{i=1}^n \left[w_i y_i - \frac{a}{(x_i - b)^c} \right]^2,$$

equivale a resolver el sistema no lineal

$$\begin{aligned} a &= \left(\sum_{i=1}^n \frac{y_i w_i}{(x_i - b)^c} \right) / \left(\sum_{i=1}^n \frac{y_i w_i}{(x_i - b)^{2c}} \right) \\ 0 &= \sum_{i=1}^n \frac{y_i w_i}{(x_i - b)^c} \sum_{i=1}^n \frac{y_i w_i}{(x_i - b)^{2c+1}} - \sum_{i=1}^n \frac{y_i w_i}{(x_i - b)^{c+1}} \sum_{i=1}^n \frac{w_i}{(x_i - b)^{2c}} \\ 0 &= \sum_{i=1}^n \frac{y_i w_i}{(x_i - b)^c} \sum_{i=1}^n \frac{w_i \ln(x_i - b)}{(x_i - b)^{2c}} - \sum_{i=1}^n \frac{w_i y_i \ln(x_i - b)}{(x_i - b)^c} \sum_{i=1}^n \frac{w_i}{(x_i - b)^{2c}}. \end{aligned}$$

- b) Resolver el sistema no lineal precedente para aquella especie de hidra cuyos datos son los de la tabla que sigue (usar como pesos $w_i = \ln y_i$).

i	1	2	3	4
y_i	2,4	3,8	4,75	21,6
x_i	31,8	31,5	31,2	30,2

Segunda parte

Programación lineal

Capítulo 5

PROGRAMACIÓN LINEAL. FORMULACIÓN

LA PROGRAMACIÓN LINEAL es la técnica de *programación matemática*, u *optimización*, que busca encontrar aquella solución o alternativa de entre las muchas posibles de un problema que mejor identifica un determinado criterio lineal atendiendo a diversas condiciones también lineales. La programación lineal surge como la forma más natural de abordar muchos problemas de la ciencia, la técnica o la economía donde se trata de asignar o compartir determinados recursos sólo disponibles en cantidades limitadas.

La formidable extensión de la programación lineal y el papel tan importante que juega hoy en día en todos aquellos entornos donde se utiliza para la asignación de recursos de cualquier tipo, se debe fundamentalmente a dos hechos: la aparición en 1947 del denominado método *simplex*, que permite la resolución de problemas¹ de programación lineal de grandes dimensiones muy eficazmente, y al enorme desarrollo que los ordenadores, su utilización e implantación han experimentado desde aquella fecha. La programación lineal juega un papel fundamental no sólo en optimización y en economía, sino también en planificación estratégica, análisis de algoritmos, problemas combinatorios, criptografía, y en muchos otros campos dispares unos de otros.

La programación lineal tiene en el análisis, planificación y control operativo de sistemas eléctricos de generación y transporte de energía, uno de sus campos de actuación más destacado. Los problemas que a diario se resuelven con esta técnica cubren casi todas las facetas involucradas en las tareas que los ingenieros y técnicos encargados de esos sistemas han de llevar a cabo; así, cuestiones como, generación de energía a mínimo coste, control de stock de combustibles, mantenimiento de equipos de generación, transporte y distribución, abastecimientos de combustibles a centrales de generación, optimización del transporte de energía en alta tensión, planificación de nuevos equipamientos generadores, control de inversiones, etc, por sólo citar unos pocos, utilizan la programación lineal dando respuesta a problemas con

¹En lo sucesivo emplearemos indistintamente los términos programa lineal, problema de programación lineal o, incluso, problema lineal

muchos miles de variables y cientos, cuando no también miles, de condiciones.

Existen dos formas tradicionales de abordar y enseñar la programación lineal: una, quizás la más extendida, aquella que la estudia como una disciplina aislada y completamente separada de las demás; otra, la que la aborda como un caso particular de procesos de optimización más amplios. Lo que aquí pretendemos es combinar esos dos enfoques. Nuestra intención es enfatizar lo más posible el hecho de que la programación lineal tiene mucho que ver con la optimización de problemas más generales a la vez que con el álgebra lineal numérica, en la que basa gran parte de la mecánica de sus procedimientos.

5.1 Conceptos y definiciones generales

La programación lineal trata de la búsqueda de la solución del siguiente *programa lineal*:

$$\begin{array}{ll}
 \text{minimizar} & c_1x_1 + c_2x_2 + \cdots + c_nx_n \\
 \text{sujeta a} & a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \geq b_1 \\
 & a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \geq b_2 \\
 & \vdots \\
 & a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \geq b_m \\
 \text{y} & x_1, x_2, \dots, x_n \geq 0.
 \end{array}$$

A la función $c_1x_1 + \cdots + c_nx_n$ se la denomina *función objetivo*; a las funciones $a_{i1}x_1 + \cdots + a_{in}x_n$, $1 \leq i \leq m$, *restricciones* o *condiciones*. Las variables del problema $x_1, \dots, x_n$ se denominan *variables de decisión*. Las constantes $c_1, \dots, c_n$, *coeficientes de coste*. La matriz de coeficientes de las condiciones del problema, A , es

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

A $\mathbf{b}^T = [b_1, b_2, \dots, b_n]$ se le denomina en algunas referencias bibliográficas *vector término de la derecha*. Un vector $\mathbf{x}^T = [x_1, x_2, \dots, x_n]$ que satisface todas las condiciones se denomina *factible*. El conjunto F de todos los vectores factibles constituye la *región factible*.

En términos más compactos de notación vectorial, el problema de programación lineal se plantea de la siguiente manera:

$$\begin{array}{ll}
 \text{min.} & \mathbf{c}^T \mathbf{x} \\
 \text{s. a} & A\mathbf{x} \geq \mathbf{b} \\
 & \mathbf{x} \geq \mathbf{0}.
 \end{array}$$

La región factible es

$$F = \{\mathbf{x} \in \Re^n : A\mathbf{x} \geq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}.$$

Ejemplo 5.1 Considérese el problema

$$\begin{aligned} \min. \quad & 2x_1 + 5x_2 \\ \text{s. a} \quad & x_1 + x_2 \geq 6 \\ & -x_1 - 2x_2 \geq -18 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Las variables de decisión son x_1 y x_2 . La función objetivo $2x_1 + 5x_2$. Las restricciones, y la región factible que delimitan, se describen en la figura 5.1. El problema consiste en encontrar aquel punto de la región factible que haga mínima la función objetivo. ●

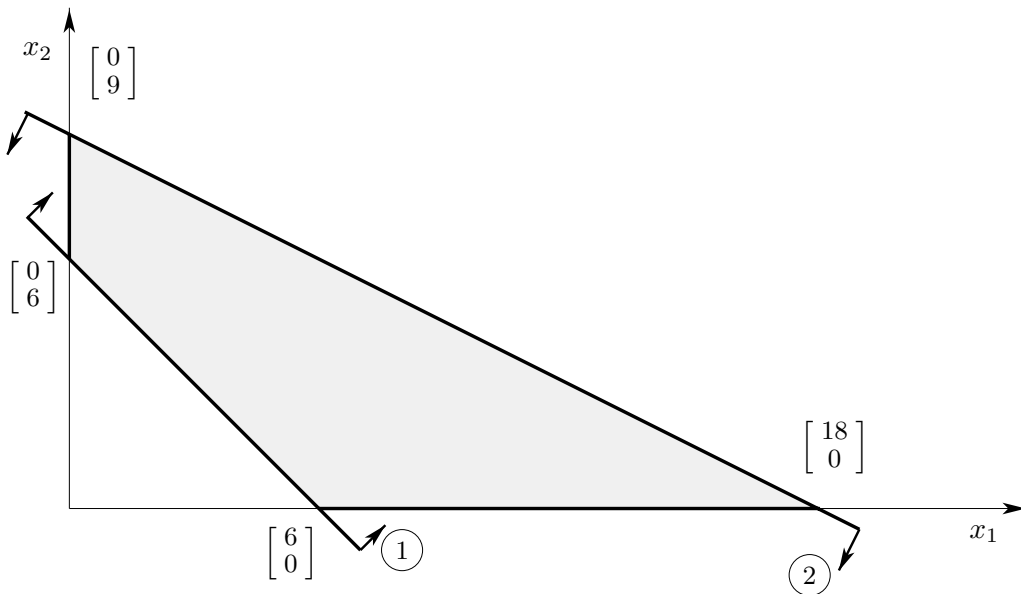


Figura 5.1

Región factible del problema de programación lineal del ejemplo 5.1

Un problema de programación lineal se puede expresar de diversas formas sin más que manipular convenientemente la función objetivo o las condiciones. Así, el problema,

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & \mathbf{Ax} \geq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

se puede transformar en otro en la denominada *forma estándar* (sólo con condiciones de igualdad),

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & \mathbf{Ax} - \mathbf{y} = \mathbf{b} \\ & \mathbf{x}, \mathbf{y} \geq \mathbf{0}, \end{aligned}$$

sin más que sustraer un vector $\mathbf{y}$, denominado de *variables de holgura*. De igual manera, si las condiciones fuesen $\mathbf{Ax} \leq \mathbf{b}$, añadiendo el vector $\mathbf{y}$, se llegaría a la forma estándar.

Si alguna de las variables x_i no está restringida a tomar valores no negativos, se puede reemplazar por otras dos, x'_i y x''_i , tales que

$$x_i = x'_i - x''_i,$$

donde $x'_i \geq 0$ y $x''_i \geq 0$, con lo que el problema pasa a tener todas sus variables restringidas a tomar valores no negativos.

De manera análoga a la anterior se puede transformar en la forma estándar un problema en el que una variable tiene como límite inferior un valor distinto de cero o incluso límite superior. Sobre estas variantes volveremos más adelante al estudiar el procedimiento concreto para tenerlas en cuenta implícitamente en el algoritmo general de resolución de problemas de programación lineal.

Si en un problema se trata de maximizar una función objetivo, se puede transformar en uno que la minimice teniendo en cuenta que

$$\text{maximizar } \mathbf{c}^T \mathbf{x} = - \text{minimizar } -\mathbf{c}^T \mathbf{x}.$$

5.2 Ejemplos de problemas de programación lineal

A continuación describimos un conjunto de problemas clásicos cuya modelización matemática da lugar a la formulación de problemas de programación lineal. Éstos, junto con los que se enuncian en los ejercicios del capítulo, representan sólo una pequeña parte de la gran variedad de problemas que se pueden plantear en la vida cotidiana bajo la forma de programas lineales.

Ejemplo 5.2 *El problema de la dieta alimenticia.* Es el problema más clásico y sobre el que se empezaron a ensayar los primeros procedimientos de cálculo de soluciones de problemas de programación lineal. Se trata de elaborar una dieta diaria para un colectivo de personas de tal forma que se suministre a cada individuo de ese colectivo una cantidad mínima de varios ingredientes nutritivos. Supongamos que existen en el mercado n alimentos distintos, a unos costes unitarios $c_1, \dots, c_n$, y que se quiere programar una dieta que contenga al menos $b_1, \dots, b_m$ unidades de m ingredientes nutritivos. Si el alimento j contiene a_{ij} unidades del ingrediente i , y el problema que se plantea consiste en programar el vector dieta $\mathbf{x}^T = [x_1, x_2, \dots, x_n]$ que fije las cantidades a comprar cada día de cada alimento de tal forma que el coste total sea mínimo, su formulación es:

$$\begin{array}{ll} \text{minimizar} & c_1x_1 + c_2x_2 + \cdots + c_nx_n \\ \text{sujeta a} & a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \geq b_1 \\ & a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \geq b_2 \\ & \vdots \\ & a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \geq b_m \\ & x_1, x_2, \dots, x_n \geq 0. \end{array}$$

Si se quisiese reformular este problema en la forma estándar habría que efectuar algunas de las operaciones descritas en el apartado anterior. ●

Ejemplo 5.3 *El problema de la emisión de deuda.* El ayuntamiento de una capital de provincia tiene comprometido gastar en determinados proyectos de infraestructura en cuatro años, 2.000, 4.000, 8.000 y 5.000 millones de pesetas, respectivamente. Se supone que todo ese dinero tiene que estar disponible el día 1 de Enero del año en que se va a gastar.

Para financiar estos gastos el ayuntamiento planea emitir unos bonos a 20 años con un interés remunerativo del 7% para la deuda emitida el primer año, del 6% para la emitida el segundo año, 6,5% para la del tercer año y del 7,5% para la emitida el cuarto año. Los intereses se empiezan a pagar inmediatamente. Si parte del dinero recaudado se depositase en cuentas a plazo fijo, el ayuntamiento sería capaz de obtener el 6% de interés el segundo año, el 5,5% el tercer año y el 4,5% el cuarto año. El problema que se plantea el ayuntamiento es el de determinar la estrategia o plan óptimo de financiación.

Si designamos por x_1, x_2, x_3 y x_4 las cantidades de deuda en miles de millones de pesetas que tiene que emitir cada uno de los cuatro años, y por y_1, y_2 e y_3 el dinero a depositar el segundo, tercer y cuarto año en cuentas a plazo fijo, el problema se puede formular de la siguiente manera:

$$\begin{aligned} \text{min. } & 20(0,07)x_1 + 20(0,06)x_2 + 20(0,065)x_3 + 20(0,075)x_4 \\ \text{s. a } & x_1 - y_1 = 2 \\ & x_2 + 1,06y_1 - y_2 = 4 \\ & x_3 + 1,055y_2 - y_3 = 8 \\ & x_4 + 1,045y_3 = 5 \\ & x_1, x_2, x_3, x_4, y_1, y_2, y_3 \geq 0. \end{aligned}$$

●

Ejemplo 5.4 *El problema del plan de fabricación.* En un taller se fabrican n tipos de piezas distintos, mecanizándose en m máquinas herramientas. Las piezas se venden a $c_1, c_2, \dots, c_n$ pesetas la unidad. La pieza tipo j requiere a_{ij} minutos de mecanización en la máquina i . Si la máquina i está disponible b_i minutos a la semana y se trata de maximizar el beneficio obtenible con la producción de piezas de una semana, el problema se puede formular como programa lineal de la siguiente manera:

$$\begin{aligned} \text{maximizar } & c_1x_1 + c_2x_2 + \dots + c_nx_n \\ \text{sujeta a } & a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1 \\ & a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_2 \\ & \vdots \\ & a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_m \\ & x_1, x_2, \dots, x_n \geq 0. \end{aligned}$$

●

Ejemplo 5.5 *El problema del transporte.* Una empresa dispone de m fábricas capaces de fabricar mensualmente $a_1, a_2, \dots, a_m$ cantidades de un producto. Este producto ha de ser enviado en cantidades $b_1, b_2, \dots, b_n$ a n almacenes. Si el coste de enviar una unidad de producto de la fábrica i al almacén j es c_{ij} , se trata de determinar las cantidades x_{ij} que habrá que enviar de cada fábrica a cada almacén —ver figura 5.2— de tal forma que el coste del transporte sea

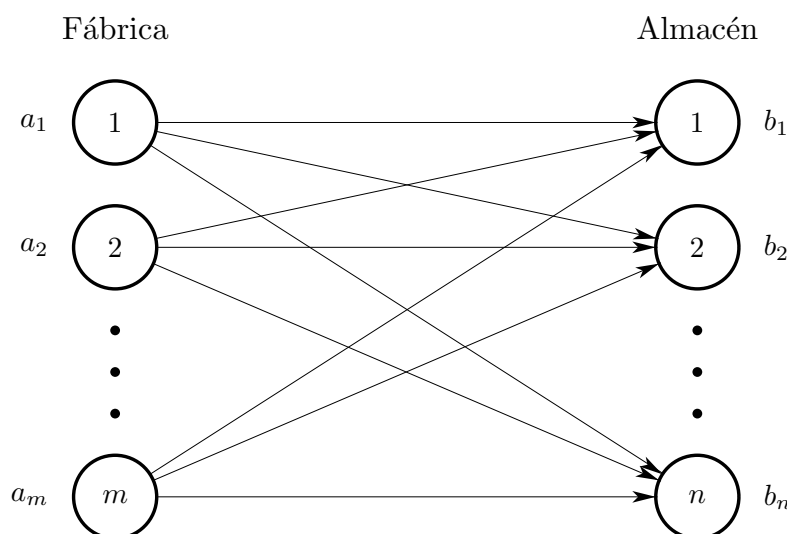


Figura 5.2
Representación gráfica del problema del transporte

mínimo y se satisfagan los requerimientos de envíos a realizar. Es decir,

$$\begin{aligned}
 &\text{minimizar } \sum_{ij} c_{ij}x_{ij} \\
 &\text{sujeta a } \sum_{j=1}^n x_{ij} = a_i, \quad \text{para } i = 1, \dots, m \\
 &\quad \sum_{i=1}^m x_{ij} = b_j, \quad \text{para } j = 1, \dots, n \\
 &\quad x_{ij} \geq 0, \quad \text{para } i = 1, \dots, m \\
 &\quad \quad \quad j = 1, \dots, n.
 \end{aligned}$$

Es evidente que al formular el problema se tendrá que cumplir que $\sum_{i=1}^m a_i = \sum_{j=1}^n b_j$, para que el total de las cantidades producidas sean igual al de las que llegan a los almacenes. ●

Ejemplo 5.6 *El Problema de la planificación de la generación de energía de una empresa eléctrica.* Una empresa que se dedica a producir, transportar y distribuir energía eléctrica está estudiando la evolución de su demanda y cómo hacerle frente en el futuro. Para ello dispone de cuatro formas posibles de generar energía eléctrica: centrales termoeléctricas con gas natural como combustible, centrales hidroeléctricas, molinos de viento y centrales de carbón.

La demanda eléctrica de esta compañía se caracteriza por tres parámetros esenciales: el consumo anual de energía, estimado en 1.750 TWh² para el conjunto de los diez años del estudio; la demanda máxima de potencia, estimada en 3.000 GW³ para el año número 10; y la potencia diaria media demandada en un día de invierno, estimada en 2.000 GW para el año número 10.

²1 TWh=10⁹ kWh

³1 GW=10⁶ kW

Los parámetros de las distintas centrales contempladas en los planes son las de la tabla 5.1. La compañía desea elaborar el plan óptimo de equipamiento para esos diez años, en el que se

Tabla 5.1

Parámetros del problema de la planificación de la generación de energía de una empresa eléctrica

Centrales para 1000 GWh de Producción Anual				
Tipo de Central	Potencia garantizada 10 ⁶ kW	Potencia máxima 10 ⁶ kW	Coste de inversión 10 ⁶ ptas.	Coste total actualizado 10 ⁶ ptas.
Gas	0,15	0,20	61	65
Hidroeléctricas	0,10	0,10	40	42
Carbón	0,80	0,90	100	110
Molinos de viento	0,10	0,40	60	64

detalle qué centrales son necesarias para hacer frente a las demandas especificadas y se minimice el coste total actualizado necesario para abordar dicho plan. Existe además la restricción de no poder gastar más de 350.000 millones de pesetas en este período.

Designando por x_1, x_2, x_3, x_4 el número de unidades de cada tipo de generación posible que instalar, el problema se puede formular de la siguiente manera:

$$\begin{aligned}
 \text{min.} \quad & 65x_1 + 42x_2 + 64x_3 + 110x_4 \\
 \text{s. a} \quad & 0,15x_1 + 0,1x_2 + 0,1x_3 + 0,8x_4 \geq 2.000 \\
 & 0,2x_1 + 0,1x_2 + 0,4x_3 + 0,9x_4 \geq 3.000 \\
 & 10x_1 + 10x_2 + 10x_3 + 10x_4 \geq 1.750 \\
 & 61x_1 + 40x_2 + 60x_3 + 100x_4 \leq 350.000 \\
 & x_1, x_2, x_3, x_4 \geq 0.
 \end{aligned}$$

●

Referencias

La introducción que se hace en este capítulo a la programación lineal es la tradicional. Unas buenas referencias para estudiar cómo se plantean los problemas de programación lineal, los orígenes de ésta, tanto técnicos como económicos, y su evolución en los últimos años son: Bazaraa y Jarvis [1977]; Bazaraa, Jarvis y Sherali [1990]; Chvátal [1983]; Cook y Russell [1977]; Dantzig [1963] y [1987]; Dorfman, Samuelson y Solow [1958]; Gill, Murray y Wright [1991]; Luenberger [1984]; Murty [1983]; Orchard-Hays [1988]; Pfaffenberger y Walker [1976]; Reklaitis, Ravindran y Ragsdell [1983]; Schrijver [1986]; Simonnard [1972] y [1973] y Sordet [1970].

Ejercicios

- 5.1. Un fabricante desea producir una aleación de metales compuesta en peso por un 30% de un metal A y por el restante 70% de otro metal B . Para ello dispone de cinco aleaciones cuya composición en metales A y B y precios por kilo es la de la tabla siguiente.

Aleación	1	2	3	4	5
% A	10	25	50	75	95
% B	90	75	50	25	5
Ptas./kilo	500	400	300	200	150

La aleación deseada se producirá mezclando alguna de las aleaciones de la tabla. El fabricante desea encontrar qué cantidades debe mezclar de cada aleación de tal forma que el coste que ello requiera sea mínimo. Formular este problema como un problema de programación lineal.

- 5.2. Una refinería de petróleo se abastece de dos tipos de crudo: uno ligero, cuyo coste por barril es de 35 dólares, y otro pesado, a 30 dólares por barril. La refinería produce gasolina para coches, *fuel-oil* para calefacción y queroseno para aviación, en las cantidades por barril de crudo que indica la siguiente tabla.

	Gasolina	<i>Fuel-oil</i>	Queroseno
Crudo ligero	0,3	0,2	0,3
Crudo pesado	0,3	0,4	0,2

La refinería ha contratado suministrar 900.000 barriles de gasolina, 800.000 de *fuel-oil* de calefacción y 500.000 de queroseno. Los responsables de su gestión desean encontrar qué cantidades de crudo deben comprar con el fin de acometer sus compromisos al mínimo coste. Formular este problema como un problema de programación lineal.

- 5.3. El director del departamento de atención a los pasajeros de la compañía aérea Satz Air Lines tiene que decidir cuántas auxiliares de vuelo debe contratar y entrenar en los próximos seis meses. Delante de él tiene la siguiente tabla que le informa de cuáles son los requisitos de esos seis meses en horas de vuelo de azafata.

Mes	Horas necesarias
Enero	8.000
Febrero	7.000
Marzo	8.000
Abril	10.000
Mayo	9.000
Junio	12.000

Dos factores complican el problema:

- El entrenamiento completo de una azafata necesita un mes; la contratación, por tanto, debe hacerse un mes antes de que se necesite su concurso.
- El entrenamiento de una azafata nueva requiere la dedicación a ese menester de un cierto tiempo del de una ya entrenada; en concreto, 100 horas.

Al director del mencionado departamento no le preocupa Enero dado que tiene una plantilla de 60 azafatas disponibles para esa fecha. Los acuerdos negociados en convenio colectivo en esa compañía impiden que una azafata trabaje más de 150 horas al mes. En Enero, por tanto, tiene disponibles 9.000 horas de azafata: 1.000 más de las que necesita.

Los registros y archivos de la compañía aseguran que, cada mes, el 10 por ciento de las azafatas abandonan el trabajo por matrimonio u otras razones.

El coste real mensual de una azafata para la compañía Satz es de 500.000 ptas., todo incluido (salario, S.S., beneficios, dietas, etc.), independientemente de cuanto trabaje —por supuesto que no puede trabajar más de 150 horas—. Entrenar una azafata nueva le cuesta 250.000 ptas.

Formular el problema que quita el sueño a nuestro valeroso directivo como un programa de programación lineal que trate de minimizar los costes a la compañía Satz.

- 5.4. Una pequeña empresa productora de piezas para automóviles fabrica cinco tipos diferentes de productos. Cada una de las piezas se obtiene por fundición de hierro, realizándose posteriormente su mecanizado-acabado donde se le efectúan una serie de taladros, torneados y pulidos. Los requerimientos en horas-hombre (por cada cien unidades) de los distintos tipos de piezas, se indican en la tabla siguiente.

Pieza	1	2	3	4	5
Fundición	2	1	3	3	1
Acabado	3	2	2	1	1

El beneficio que obtiene la empresa de la venta de cien unidades de cada una de estas piezas es 3.000, 2.000, 4.000, 2.500 y 1.000 pesetas, respectivamente. La capacidad en los próximos dos meses de las unidades de fundición y mecanizado son 700 y 1.000 horas-hombre, respectivamente.

Formular el problema de determinar qué cantidades de cada una de las cinco piezas se han de fabricar con vistas a maximizar el beneficio obtenible.

- 5.5. Un fabricante de textiles tiene dos fábricas, dos proveedores de materias primas y tres centros de venta. El coste del transporte en ptas./tonelada de las materias primas a las dos fábricas y de éstas a los centros de venta se indican a continuación.

	Fábrica			Centro de Venta		
	A	B		1	2	3
Prov. 1	100	150	Fábrica A	400	200	100
Prov. 2	200	150	Fábrica B	300	400	200

El primer proveedor puede suministrar 10 toneladas de materia prima y el segundo 15. Los centros de venta necesitan 8, 14 y 3 toneladas de productos, respectivamente. La capacidad de procesamiento de la materia prima de las fábricas se puede considerar ilimitada.

- Formular el problema de encontrar qué cantidades se deben transportar de los proveedores a las fábricas y de estas a los centros de venta como un problema general de programación lineal.
 - Reducir el problema a uno simple de transporte con dos orígenes y tres destinos, tratando de encontrar los caminos de coste mínimo de los proveedores a los centros de venta.
 - Supóngase que la fábrica A posee una capacidad de procesamiento de materia prima de 8 toneladas y la fábrica B de 7 toneladas. Descomponer el problema en dos problemas de transporte.
- 5.6. Una empresa que posee un molino agrícola fabrica comida para ganado, ovejas y pollos. Las diversas comidas se fabrican mezclando cuatro ingredientes básicos: maíz, pescado, soja y trigo. Todos estos ingredientes poseen los siguientes elementos nutritivos: vitaminas, proteínas, calcio y grasa. Los contenidos unitarios en elementos nutritivos de cada uno de los ingredientes se indican

en la siguiente tabla.

INGREDIENTE	ELEMENTO NUTRITIVO			
	VITAMINAS	PROTEÍNAS	CALCIO	GRASA
Maíz	8	10	6	8
Trigo	6	5	10	6
Soja	10	12	6	6
Pescado	4	8	6	9

El objetivo del molino es conseguir producir 10, 6 y 8 toneladas de comida para ganado, ovejas y pollos, respectivamente. Debido a unas restricciones sólo es posible conseguir 6 toneladas de maíz, 10 de trigo, 4 de soja y 5 de pescado. El precio por kilo de estos ingredientes es 20, 12, 24 y 12 pesetas, respectivamente. El mínimo y máximo de unidades de elementos nutritivos que se permiten por kilo para la comida de ganado, de ovejas y pollos, se indica a continuación.

PRODUCTO	ELEMENTO NUTRITIVO							
	VITAMINAS		PROTEÍNAS		CALCIO		GRASA	
	MIN.	MAX.	MIN.	MAX.	MIN.	MAX.	MIN.	MAX.
Ganado	6	∞	6	∞	7	∞	4	8
Ovejas	6	∞	6	∞	6	∞	4	6
Pollos	4	6	6	∞	6	∞	4	6

Formular qué cantidades se deben mezclar de los distintos ingredientes para satisfacer la demanda a coste mínimo.

- 5.7. El encargado de la cocina de una ciudad sanitaria tiene que confeccionar un plan de menús. Debe empezar con el almuerzo. Su menú debe constar de tres platos o categorías: vegetales, carne y postre. El coste en pesetas de las diversas posibilidades que maneja se indica en la siguiente tabla.

	HIDRATOS DE CARBONO	VITAMINAS	PROTEÍNAS	GRASAS	COSTE
<i>Vegetales</i>					
Guisantes	1	3	1	0	10
Judías verdes	1	5	2	0	12
Maíz	2	6	1	2	9
Macarrones	4	2	1	1	10
Arroz	5	1	1	1	7
<i>Carne</i>					
Pollo	2	1	3	1	70
Pescado	3	6	6	1	63
Vaca	3	8	5	2	120
<i>Postre</i>					
Naranja	1	3	1	0	10
Manzana	1	2	0	0	12
Helado	1	0	0	0	12
Pudding	1	0	0	0	15

Supóngase que los requerimientos mínimos de la dieta para el almuerzo son 5 unidades de hidratos de carbono, 10 de vitaminas, 10 de proteínas y 2 de grasa.

- Formular el problema de planificar el almuerzo de mínimo coste como un programa de programación lineal.
- Pensar cómo podrían planificarse todas las comidas de la semana.

5.8. Considérese el siguiente problema de programación lineal:

$$\begin{array}{ll} \text{maximizar} & -x_1 - x_2 + 2x_3 + x_4 \\ \text{s. a} & x_1 + x_2 + x_3 + x_4 \geq 6 \\ & x_1 - x_2 - 2x_3 + x_4 \leq 4 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{array}$$

- Introducir las variables de holgura necesarias para transformarlo en forma estándar.
- Dibujar la región factible.
- Interpretar qué es la factibilidad en esa región factible.
- Si se sabe que la solución óptima de este problema debe tener dos variables iguales a cero y las otras dos serán positivas, ¿cuál debe ser el punto óptimo?

5.9. Considérese el siguiente problema de programación lineal:

$$\begin{array}{ll} \text{maximizar} & 2x_1 + x_2 \\ \text{s. a} & x_1 + 2x_2 \leq 16 \\ & 2x_1 + x_2 \leq 12 \\ & x_1, x_2 \geq 0. \end{array}$$

- Dibujar la región factible.
- Introducir las variables de holgura necesarias, x_3 y x_4 , para transformarlo en forma estándar.
- Identificar las regiones en el plano x_1, x_2 donde las variables de holgura son cero.

5.10. La calidad del aire que se respira en una zona industrial determinada depende de las emisiones contaminantes de n centrales eléctricas. Cada central usa m combustibles diferentes. Supóngase que la energía que se necesita de cada central j es b_j kilocalorías al día y que la emisión de contaminantes de la central j , debido al combustible i , cada día, es c_{ij} . Supóngase además que el combustible tipo i cuesta d_i pesetas por tonelada y que cada tonelada de ese combustible genera α_{ij} kilocalorías en la central j . El nivel de contaminación de la zona no puede exceder de b microgramos por metro cúbico. Finalmente, considérese que γ_j es un parámetro meteorológico que relaciona las emisiones de la central j con la calidad del aire en la zona.

- Formular el problema de determinar la mezcla de combustibles a utilizar en cada central como un problema de programación lineal.
- ¿Cómo se incorporarían las restricciones tecnológicas que prohíben el uso de ciertas mezclas de combustibles en determinadas centrales?
- ¿Cómo se puede asegurar la equidad entre todas las centrales?

5.11. Una región está dividida en m zonas residenciales y parques empresariales. Cada zona se representa mediante un nudo estando conectados entre sí mediante unos arcos que simbolizan las carreteras o calles que los unen. La gente que vive en una zona se desplaza a trabajar o a comprar a otras o a las mismas donde viven, de tal forma que cada nudo atrae o genera una serie de viajes. En concreto, a_{ij} representa el número de viajes que se generan en el nudo i cuyo destino son el nudo j y b_{ij} el tiempo que se invierte en ir del nudo i al j . Se desea determinar las rutas que pueden realizar las personas que viven en esa región.

- Ilustrar cómo se podría abordar este problema denominado *problema de asignación de tráfico*.

- b) Estudiar qué criterio se podría utilizar como función objetivo de este problema y la forma de resolverlo.

5.12. Una gran empresa dispone de 3.000 millones de pesetas para adjudicar a tres filiales el próximo ejercicio. Debido a compromisos adquiridos referentes a estabilidad en el empleo y a otras razones, la empresa ha garantizado a esas filiales unos niveles mínimos de dotación monetaria; son: 300 millones para la primera filial, 500 millones para la segunda y 800 millones para la tercera. Debido a la naturaleza de su negocio, la filial número 2 no puede utilizar más de 1.700 millones de pesetas sin tener que ampliar su capital, cosa que no quiere hacer el próximo ejercicio. Cada filial puede llevar a cabo varios proyectos con el dinero que reciba. Para cada uno de esos proyectos se ha fijado un beneficio mínimo obtenible. Alguno de esos proyectos además requiere sólo una cantidad determinada de dinero. Las características de los proyectos en estos términos se indican en la tabla que sigue.

FILIAL	PROYECTO	BENEFICIO	MÁXIMO INVERTIBLE
1	1	8%	600 millones
	2	6%	500 millones
	3	7%	900 millones
2	4	5%	700 millones
	5	8%	1.000 millones
	6	9%	400 millones
3	7	10%	600 millones
	8	6%	300 millones

Formular el problema de asignar los recursos disponibles a las tres filiales como un problema de programación lineal de tal forma que se maximice el beneficio.

5.13. El *estimador de norma* l_1 del modelo lineal

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{e}$$

se define como el vector $\mathbf{b} = [b_1, \dots, b_k]^T$ que minimiza

$$\sum_{i=1}^n |y_i - \mathbf{a}^i \mathbf{b}|, \quad (5.1)$$

donde $\mathbf{a}^i = [a_{i1}, \dots, a_{ik}]$.

Formular (5.1) como un problema de programación lineal.

5.14. Estudiar cómo se podría resolver el problema

$$\text{minimizar } l_\infty(a) = \max\{|a_0 + a_1 t + a_2 t^2 + a_3 t^3 - y(t)| : t \in S\}$$

aplicando programación lineal.

5.15. Se consideran 11 posibles cultivos en 8 regiones agrícolas distintas (en todas las regiones se pueden cultivar, en un principio, todas las especies). En cada región se cultivan un número de hectáreas determinado de uno o más de los cultivos seleccionados. Se trata de planificar el cultivo de todas las regiones de tal forma que se utilice más eficazmente la tierra y los otros medios de producción.

Los 11 cultivos se dividen en categorías de acuerdo con la tabla siguiente.

Cultivos de invierno	Cultivos de verano
1 Trigo	5 Algodón variedad 1
2 Cebada	6 Algodón variedad 2
3 Habas	7 Arroz
4 Lentejas	8 Maíz
	9 Mijo
	10 Sésamo (ajonjolí)
	11 Caña de azúcar

Las variables que se consideran son:

- i = número de la región (8).
- j = número del cultivo (11).
- r_{ij} = factor de ingresos netos por hectárea de cultivo j en la región i .
- x_{ij} = número de hectáreas a asignar al cultivo j en la región i .
- y_{ij} = número actual de hectáreas del cultivo j en la región i .
- w_i = área total de cultivos de invierno en la región i .
- v_i = área total de cultivos de verano en la región i .
- y_i = área total de cultivos en la región i ($w_i < y_i$ y $v_i < y_i$).
- y_{i11} = hectáreas cultivadas actuales con caña de azúcar en la región i .

El objetivo que se persigue es maximizar los ingresos en función de la producción de cada cultivo, a_{ij} , del precio de venta de lo cultivado, p_j , y de los costes de operación, c_{ij} ($r_{ij} = a_{ij}p_j - c_{ij}$), supuestos a_{ij} , p_j y c_{ij} conocidos.

Otras condiciones a tener en cuenta son:

- a) Que el número total de hectáreas que se pueden dedicar a cultivos de algodón en cada región i no debe exceder de un tercio del total de hectáreas cultivadas en esa región.
- b) Que cada región debe cultivar un número de hectáreas de caña de azúcar exactamente igual al que cultiva en la actualidad.
- c) Que la cantidad de hectáreas que se cultiven de trigo, habas y maíz deben ser al menos 0,3, 0,3 y 0,85 veces las que se cultivan en la actualidad.

5.16. Considérese el siguiente sistema de desigualdades lineales:

$$\begin{aligned}
 x_1 - 2x_2 + x_3 - x_4 - x_5 + x_6 &\leq 13 \\
 -2x_1 + x_2 - 12x_3 + 2x_4 + 3x_5 + 4x_6 &\leq 1 \\
 3x_1 + 13x_2 + 18x_3 + 17x_4 + 25x_5 + 12x_6 &\leq 3 \\
 5x_1 + 3x_2 - 8x_3 + 13x_4 + 8x_5 - 7x_6 &\leq 5 \\
 x_j &\geq 0 \quad \text{para } j = 1, \dots, 6.
 \end{aligned}$$

Formular como programa lineal el problema de determinar una solución factible, $\mathbf{x}$, del mismo que haga que se satisfagan estas desigualdades tan cerca como sea posible del valor que las haría igualdades.

5.17. Transformar el problema de programación matemática,

$$\begin{aligned}
 &\text{minimizar } 6x_1 + 5x_2 - 3x_3 \\
 &\text{s. a} \quad \begin{aligned} &x_1 + 2x_2 - 7x_3 \leq 1 \\ &|3x_1 - 5x_2 - 20| \leq 4 \end{aligned} \\
 &x_j \geq 0 \quad \text{para } j = 1, \dots, 3,
 \end{aligned}$$

en un problema de programación lineal. ¿Qué pasa si se añade la condición $|x_1 - 2x_2 + 3x_3 - 30| \geq 5$?
¿Continúa siendo posible transformar el problema en otro de programación lineal?

Capítulo 6

TEORÍA BÁSICA DE LA PROGRAMACIÓN LINEAL

CON EL OBJETIVO de profundizar en el conocimiento de lo que representa un programa lineal, antes de abordar en capítulos posteriores algunos resultados fundamentales para comprender el porqué de las diversas formas de resolver problemas de programación lineal y su mecánica, en este capítulo presentamos los aspectos geométricos más destacados y la teoría básica que fundamenta la programación lineal.

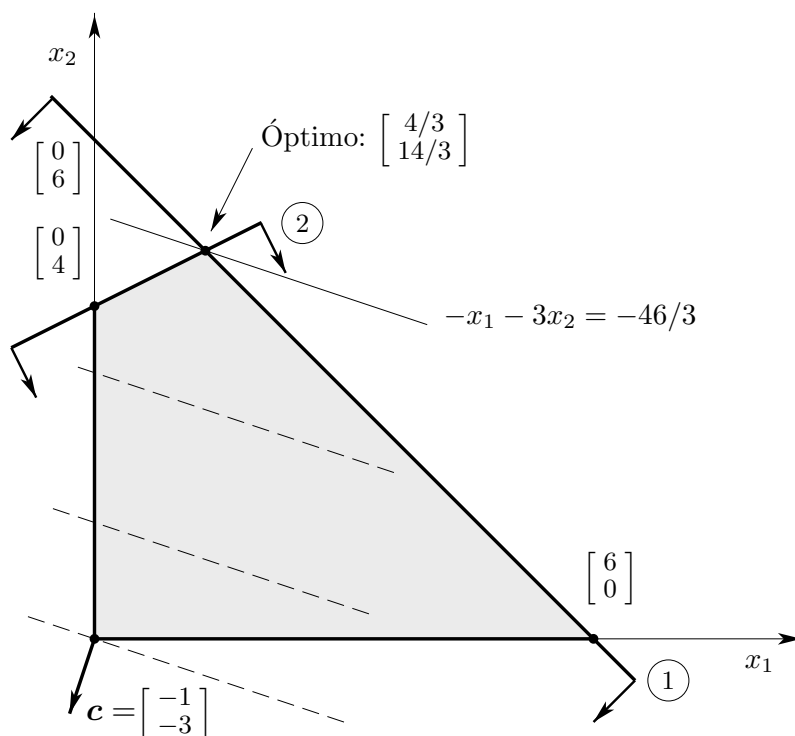
6.1 Consideraciones geométricas sobre la programación lineal

Ejemplo 6.1 Consideremos el siguiente problema:

$$\begin{aligned} \min. \quad & -x_1 - 3x_2 \\ \text{s. a} \quad & x_1 + x_2 \leq 6 \\ & -x_1 + 2x_2 \leq 8 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Lo que se plantea es determinar aquel punto de la región factible que se representa en la figura 6.1 que minimiza la variedad lineal (en este caso una recta) $z = -x_1 - 3x_2$. Como el objetivo es minimizar z , esta variedad lineal se encontrará desplazada respecto de su subespacio de referencia, $-x_1 - 3x_2 = 0$, en la dirección que minimice más dicho objetivo: esto es, en la dirección $-\mathbf{c} = [1, 3]^T$, opuesta al vector característico¹ de z . Cuando se alcanza el punto óptimo del problema, $\mathbf{x}^* = [4/3, 14/3]^T$, es imposible, manteniendo la factibilidad de la solución, mover más $z = -x_1 - 3x_2$ en la dirección de $-\mathbf{c}$.

¹Sobre este concepto volveremos inmediatamente: ver definición 6.4.

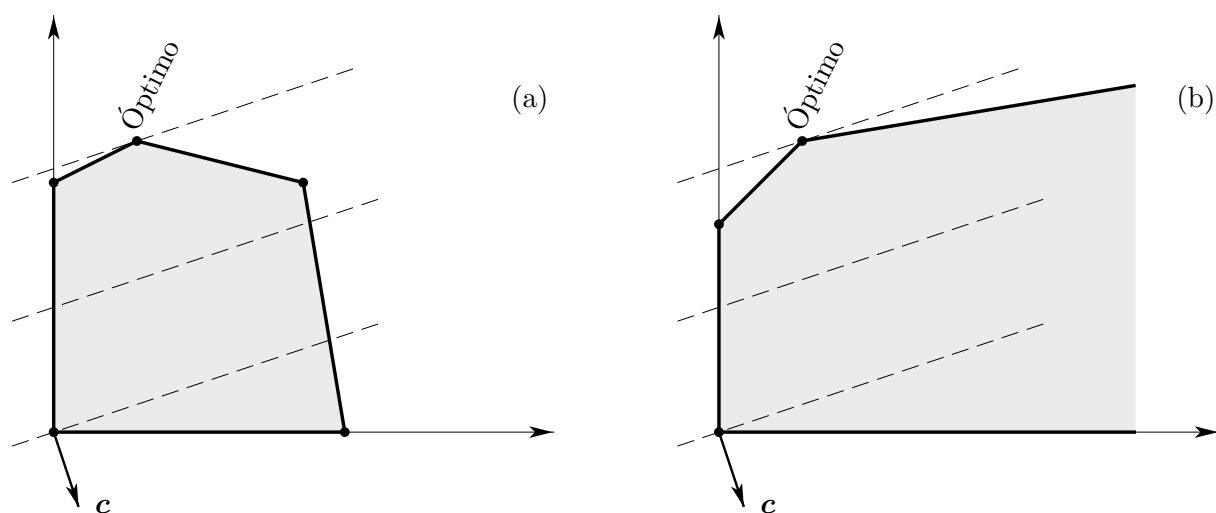
**Figura 6.1**

Resolución geométrica del problema de programación lineal del ejemplo 6.1

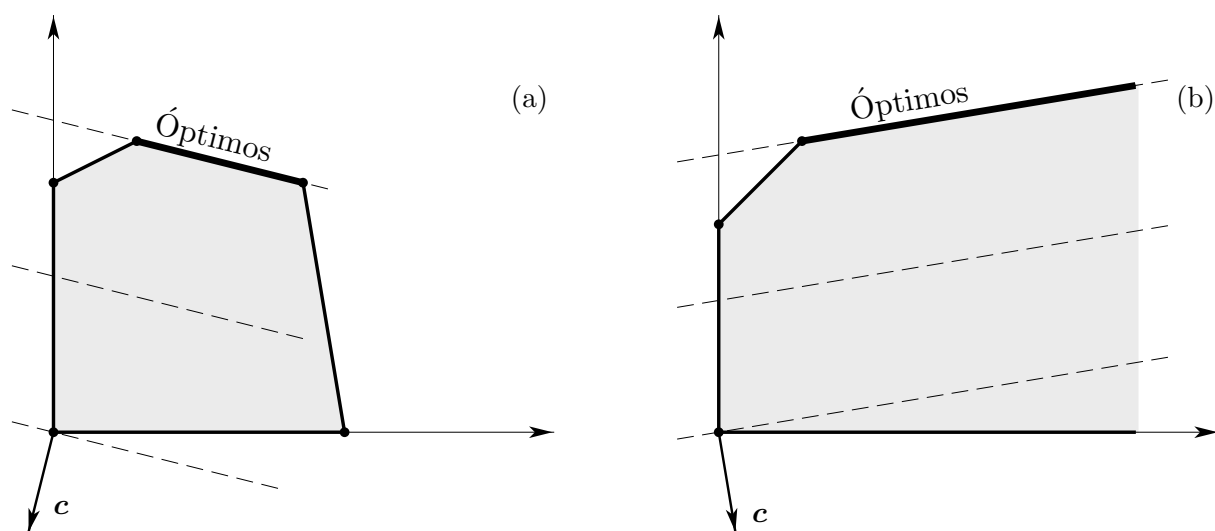
El punto óptimo, como se puede observar, se ha alcanzado en un *punto extremo* de la región factible del problema. ●

En el ejemplo anterior sólo existe un punto óptimo. La solución, sin embargo, se puede presentar bajo formas diversas:

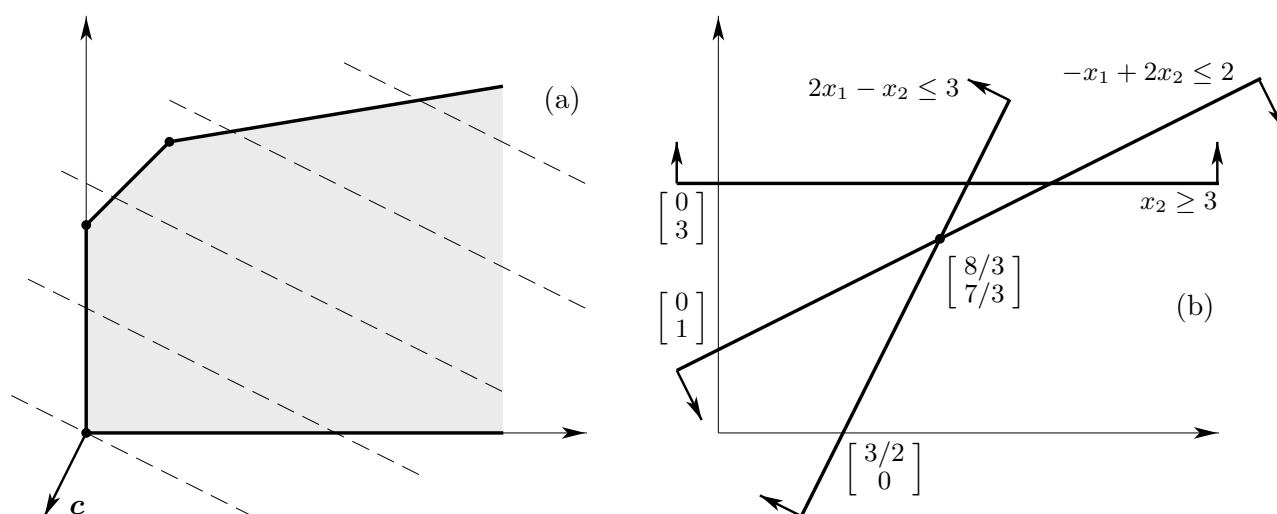
1. *Solución óptima única.* Ocurre siempre en un punto extremo de la región factible. En la figura 6.2 se describen las dos alternativas que se puede presentar este caso: región factible acotada y no acotada. El que la región factible no esté acotada no afecta a la solución; puede que sí a la forma de llegar a ella.
2. *Soluciones óptimas alternativas.* Este caso se representa en la figura 6.3. El óptimo es cualquiera de los puntos de una denominada *cara* de la región factible. En (a) la región factible está acotada mientras que en (b) no.
3. *Solución óptima no acotada.* Este caso se presenta cuando la configuración región factible-función objetivo tiene la forma de la figura 6.4 (a): es posible desplazarse tanto como se desee dentro de la región factible en la dirección $-c$ sin encontrar un punto extremo o cara de la región factible que bloquee dicho desplazamiento.

**Figura 6.2**

Solución óptima única finita: (a) región factible acotada; (b) región factible no acotada

**Figura 6.3**

Soluciones óptimas alternativas: (a) región factible acotada; (b) región factible no acotada

**Figura 6.4**

(a) Solución óptima no acotada. (b) Región factible vacía

4. *Región factible vacía.* El siguiente ejemplo ilustra este caso:

$$\begin{aligned}
 &\min. \quad -2x_1 + 3x_2 \\
 &\text{s. a} \quad \begin{aligned}
 &-x_1 + 2x_2 \leq 2 \\
 &2x_1 - x_2 \leq 3 \\
 &x_1 \geq 0 \\
 &x_2 \geq 3.
 \end{aligned}
 \end{aligned}$$

Su región factible es el $\emptyset$ pues no hay ningún $\mathbf{x} = [x_1, x_2]^T$ que satisfaga todas las condiciones. El problema se dice *no factible* o *inconsistente*. En la figura 6.4 (b) se representa este problema.

6.1.1 Representación geométrica del programa lineal en el subespacio de bienes

La representación geométrica de los programas lineales que se han presentado en el apartado anterior se refiere al denominado en la literatura especializada en programación lineal *subespacio de actividad* de $\mathbb{R}^n$: el subespacio donde se representan los puntos $\mathbf{x}$. Es el subespacio donde la expresión de la región factible, o conjunto de puntos interiores a la *zona* que delimitan los hiperplanos que definen cada una de las condiciones, es más intuitiva. En apartados posteriores veremos cómo caracterizar este subespacio.

En este apartado vamos a considerar el problema en el subespacio denominado *de bienes*: el subespacio que definen los vectores columna de la matriz A o subespacio imagen de A : $\text{Im}(A)$. Además de subespacio de bienes, en programación lineal a este subespacio también se le denomina *subespacio requisito* o *subespacio de productos*.

Definición 6.1 Un conjunto $C \subseteq \mathbb{R}^n$ se dice *convexo* si y sólo si para todo par de puntos $\mathbf{x}_1, \mathbf{x}_2 \in C$ todas las combinaciones de la forma $\mathbf{x} = \lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2$, con $0 \leq \lambda \leq 1$, están en C .

Es decir, cuando para cada par de puntos del conjunto convexo, todos los puntos de la recta que los une están en el conjunto: ver figura 6.5.

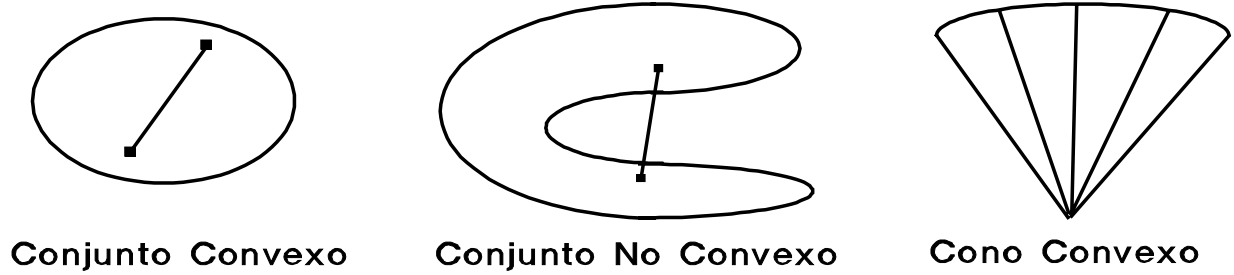


Figura 6.5

Conjuntos convexo y no convexo; cono convexo

La expresión $\mathbf{x} = \lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2$, $0 \leq \lambda \leq 1$, define la *combinación convexa* de $\mathbf{x}_1$ y $\mathbf{x}_2$. Si $0 < \lambda < 1$, es decir $\lambda \in (0, 1)$, la combinación se denomina *estrictamente convexa*. El concepto de combinación convexa se puede generalizar a cualquier número finito de puntos de la siguiente manera:

$$\mathbf{x} = \sum_{i=1}^p \lambda_i \mathbf{x}_i,$$

donde

$$\sum_{i=1}^p \lambda_i = 1, \quad \lambda_i \geq 0, \quad i = 1, \dots, p.$$

Teorema 6.1 El conjunto $K = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, de soluciones de un programa lineal, es un conjunto convexo.

DEMOSTRACIÓN. Sean $\mathbf{x}_1, \mathbf{x}_2 \in K$ y $\lambda \in [0, 1]$. Para demostrar el enunciado es suficiente probar que $\bar{\mathbf{x}} = \lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2 \in K$.

Como $\mathbf{x}_1$ y $\mathbf{x}_2$ pertenecen a K , de la definición de éste se deduce que $A\mathbf{x}_1 = \mathbf{b}$ y $\mathbf{x}_1 \geq \mathbf{0}$, y que $A\mathbf{x}_2 = \mathbf{b}$ y $\mathbf{x}_2 \geq \mathbf{0}$, respectivamente.

El hecho de que $\lambda \in [0, 1]$ implica que $\lambda \geq 0$ y que $(1 - \lambda) \geq 0$.

Combinando estas propiedades se tiene que

$$\lambda A\mathbf{x}_1 = \lambda \mathbf{b} \tag{6.1}$$

$$\lambda \mathbf{x}_1 \geq \mathbf{0} \tag{6.2}$$

$$(1 - \lambda) A\mathbf{x}_2 = (1 - \lambda) \mathbf{b} \tag{6.3}$$

$$(1 - \lambda) \mathbf{x}_2 \geq \mathbf{0}. \tag{6.4}$$

Sumando las expresiones de (6.1) y (6.3) se obtiene que

$$\lambda A\mathbf{x}_1 + (1 - \lambda)A\mathbf{x}_2 = \lambda\mathbf{b} + (1 - \lambda)\mathbf{b}.$$

Reagrupando términos se llega a que

$$A[\lambda\mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2] = [\lambda + (1 - \lambda)]\mathbf{b} = \mathbf{b}$$

Sumando las expresiones de (6.2) y (6.4) se obtiene que

$$\lambda\mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \geq \mathbf{0}.$$

A partir de las dos últimas expresiones es claro que $A\bar{\mathbf{x}} = \mathbf{b}$ y que $\bar{\mathbf{x}} \geq \mathbf{0}$, por lo que queda probado que $\bar{\mathbf{x}} \in K$. ■

Definición 6.2 Un conjunto $C \subseteq \Re^n$ se dice un *cono* si para todo $\mathbf{x} \in C$, $\lambda\mathbf{x} \in C$ para todo escalar $\lambda \in \Re$, $\lambda \geq 0$. Un cono que también es convexo se denomina *cono convexo*.

El conjunto $\{\mathbf{x} \in \Re^m : \mathbf{x} = A\boldsymbol{\alpha}, A \in \Re^{m \times n}, \boldsymbol{\alpha} \in \Re^n, \boldsymbol{\alpha} \geq \mathbf{0}\}$ es un cono convexo generado por los vectores columna de la matriz A .

Definición 6.3 Un punto $\mathbf{x}$ en un conjunto convexo C es un *punto extremo* de C si y sólo si no es interior a un segmento de recta contenido en C . Dicho de otra forma, si y sólo si,

$$\mathbf{x} = (1 - \beta)\mathbf{y} + \beta\mathbf{z} \quad \text{con} \quad 0 < \beta < 1 \quad \text{y} \quad \mathbf{y}, \mathbf{z} \in C \quad \Rightarrow \quad \mathbf{x} = \mathbf{y} = \mathbf{z}.$$

6.1.1.1 Factibilidad y condiciones de igualdad

Consideremos ahora el problema de programación lineal en forma estándar,

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

donde $\mathbf{x} \in \Re^n$ y $A \in \Re^{m \times n}$. Si el vector columna j de la matriz A se designa por $\mathbf{a}_j$, el problema se puede reescribir de la siguiente manera:

$$\begin{aligned} \min. \quad & \sum_{j=1}^n c_j x_j \\ \text{s. a} \quad & \sum_{j=1}^n \mathbf{a}_j x_j = \mathbf{b} \\ & x_1, \dots, x_n \geq 0. \end{aligned}$$

Es decir, un problema de programación lineal busca un vector $\mathbf{x} = [x_1, \dots, x_n]^T$ tal que, dados $\mathbf{a}_1, \dots, \mathbf{a}_n$, se cumpla que $\sum_{j=1}^n \mathbf{a}_j x_j = \mathbf{b}$, siendo además $\sum_{j=1}^n c_j x_j$ mínima.

Teniendo en cuenta las definiciones anteriores, el problema se puede enunciar como el de la búsqueda de un vector de escalares no negativos, $\mathbf{x}$, que combinen de forma óptima los vectores columna de la matriz A para que $\mathbf{b}$ pertenezca al cono convexo por ellos generado y se minimice el valor alcanzable por la función objetivo.

En la figura 6.6 se representan en dos dimensiones los casos de un problema con región factible no vacía y otro con región factible vacía. En el segundo caso, (b), los vectores $\mathbf{a}_1$, $\mathbf{a}_2$, $\mathbf{a}_3$ y $\mathbf{a}_4$ no se pueden combinar convexamente de ninguna manera que contenga a $\mathbf{b}$.

Ejemplo 6.2 Consideremos las regiones factibles que determinan los dos conjuntos de condiciones siguientes:

$$\begin{aligned} 2x_1 + x_2 + x_3 &= 2 \\ -x_1 + 3x_2 + x_4 &= 3 \\ x_1, x_2, x_3, x_4 &\geq 0 \end{aligned}$$

y

$$\begin{aligned} 2x_1 + x_2 + x_3 &= -1 \\ -x_1 + 3x_2 + x_4 &= 2 \\ x_1, x_2, x_3, x_4 &\geq 0. \end{aligned}$$

En la figura 6.7 se representan los conos convexos que generan los vectores $\mathbf{a}_1$, $\mathbf{a}_2$, $\mathbf{a}_3$, $\mathbf{a}_4$ y $\mathbf{b}$ de estos dos conjuntos de condiciones. En (a), $\mathbf{b}$ está contenido en dicho cono convexo; en (b) no: el primer problema es factible mientras que el segundo es inconsistente. ●

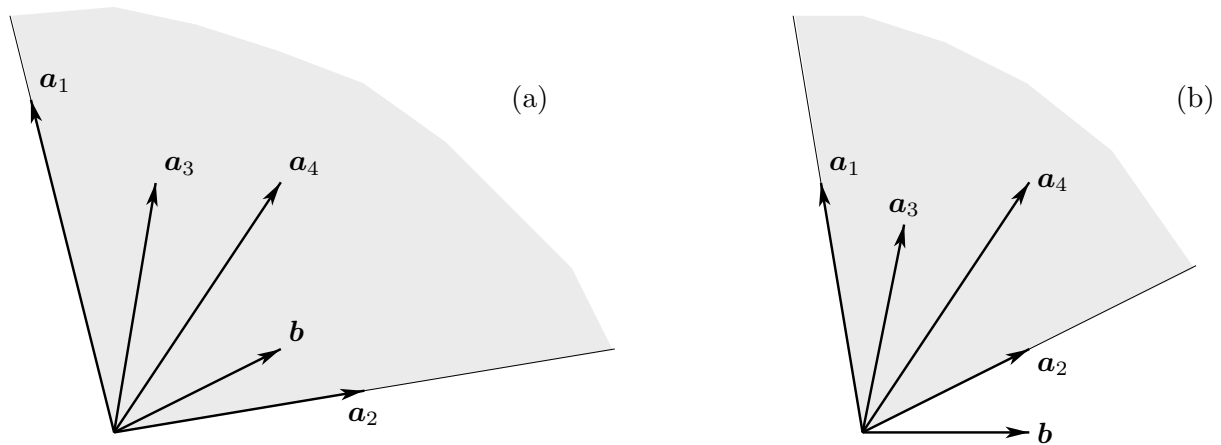
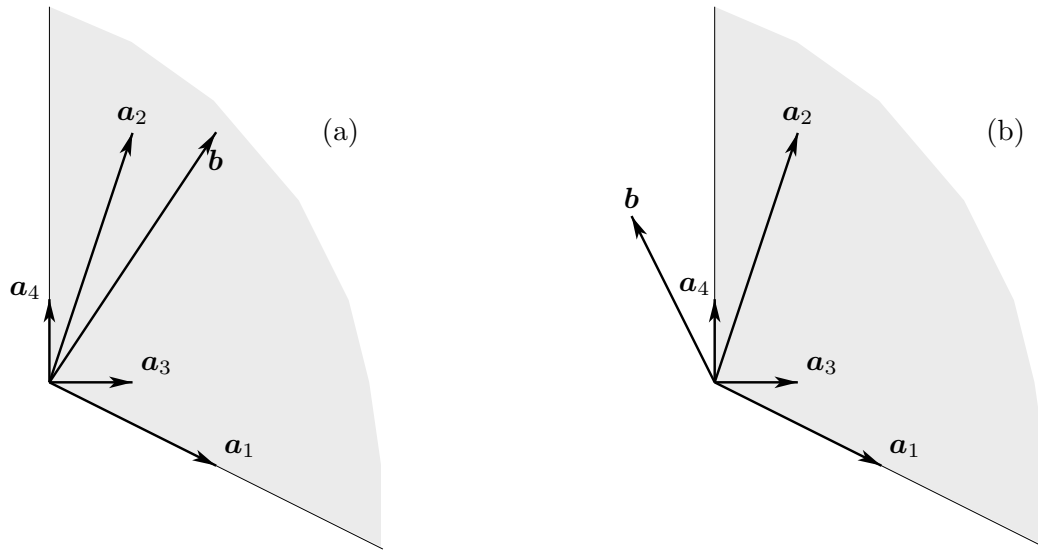


Figura 6.6

Interpretación geométrica de la factibilidad de un programa lineal: (a) región factible no vacía; (b) región factible vacía

**Figura 6.7**

Regiones factibles del ejemplo 6.2: (a) no vacía; (b) vacía

6.1.1.2 Factibilidad y condiciones de desigualdad

Consideremos los problemas de programación lineal de la siguiente forma:

$$\begin{aligned} \min. \quad & \sum_{j=1}^n c_j x_j \\ \text{s. a} \quad & \sum_{j=1}^n \mathbf{a}_j x_j \leq \mathbf{b} \\ & x_1, \dots, x_n \geq 0. \end{aligned}$$

Si existe solución factible, la intersección del cono convexo que generan los vectores $\mathbf{a}_1, \dots, \mathbf{a}_n$ y el conjunto de vectores menores o iguales que $\mathbf{b}$ debe ser no vacía.

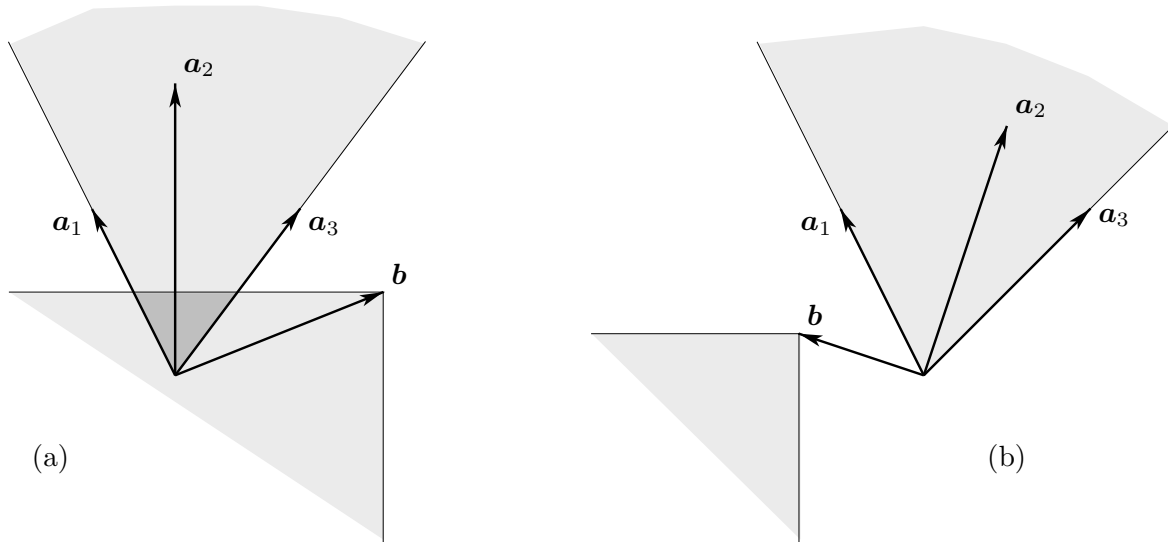
En la figura 6.8 se describen el vector $\mathbf{b}$ y los vectores columna de dos programas lineales de acuerdo con esta última forma de ver geoméricamente el problema. En (a) la región factible es no vacía y en (b) es vacía.

6.1.1.3 Óptimo

Refiriéndonos una vez más al problema de programación lineal en forma estándar, recordemos que se trata de encontrar una combinación lineal de los vectores columna de la matriz A tal que, estando $\mathbf{b}$ en el cono convexo por ellos generado, minimice la función objetivo. Es decir, encontrar unos escalares no negativos $x_1, x_2, \dots, x_n$ tales que

$$\begin{bmatrix} c_1 \\ \mathbf{a}_1 \end{bmatrix} x_1 + \begin{bmatrix} c_2 \\ \mathbf{a}_2 \end{bmatrix} x_2 + \dots + \begin{bmatrix} c_n \\ \mathbf{a}_n \end{bmatrix} x_n = \begin{bmatrix} z \\ \mathbf{b} \end{bmatrix},$$

y se minimice z . Se busca pues expresar el vector $[z, \mathbf{b}^T]^T$ en el cono convexo que generan los vectores $[c_1, \mathbf{a}_1^T]^T, \dots, [c_n, \mathbf{a}_n^T]^T$, para el valor más pequeño posible del escalar z .

**Figura 6.8**

Programa lineal con condiciones de desigualdad: (a) región factible no vacía; (b) región factible vacía

Ejemplo 6.3 Consideremos el siguiente problema de programación lineal:

$$\begin{aligned} \min. \quad & -2x_1 - 3x_2 \\ \text{s. a} \quad & x_1 + 2x_2 \leq 2 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Añadiendo la variable de holgura x_3 , el problema se puede expresar como el de encontrar unos escalares $x_1, x_2, x_3 \geq 0$ tales que

$$\begin{bmatrix} -2 \\ 1 \end{bmatrix} x_1 + \begin{bmatrix} -3 \\ 2 \end{bmatrix} x_2 + \begin{bmatrix} 0 \\ 1 \end{bmatrix} x_3 = \begin{bmatrix} z \\ 2 \end{bmatrix}$$

y además se minimice z . En la figura 6.9 se representa el cono que generan los vectores $[-2, 1]^T$, $[-3, 2]^T$, y $[0, 1]^T$. Se trata de encontrar el vector $[z, 2]^T$ de este cono convexo que haga mínimo z . La solución es $z^* = -4$ con $x_1^* = 2$ y $x_2^* = x_3^* = 0$. ●

Ejemplo 6.4 Sea ahora el programa lineal

$$\begin{aligned} \min. \quad & -2x_1 - 3x_2 \\ \text{s. a} \quad & x_1 + 2x_2 \geq 2 \\ & x_1, x_2 \geq 0. \end{aligned}$$

La solución óptima es no acotada. Si se sustrae de la primera condición la variable de holgura x_3 , el problema se reduce a encontrar unos escalares $x_1, x_2, x_3 \geq 0$ tales que se cumpla que

$$\begin{bmatrix} -2 \\ 1 \end{bmatrix} x_1 + \begin{bmatrix} -3 \\ 2 \end{bmatrix} x_2 + \begin{bmatrix} 0 \\ -1 \end{bmatrix} x_3 = \begin{bmatrix} z \\ 2 \end{bmatrix},$$

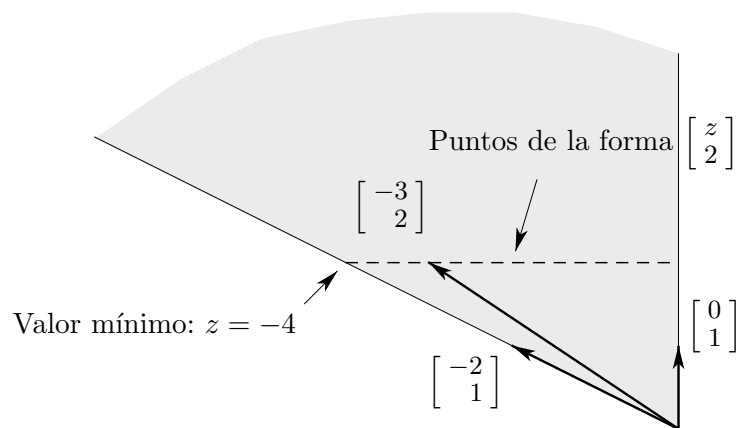


Figura 6.9
Descripción geométrica del ejemplo 6.3

y se minimice z . En la figura 6.10 se describe el cono que generan los vectores $[-2, 1]^T$, $[-3, 2]^T$ y $[0, -1]^T$. En ese cono se pueden encontrar puntos de la forma $[z, 2]^T$, con z tan pequeña como queramos: el valor óptimo de la función objetivo es, por consiguiente, $-\infty$. ●

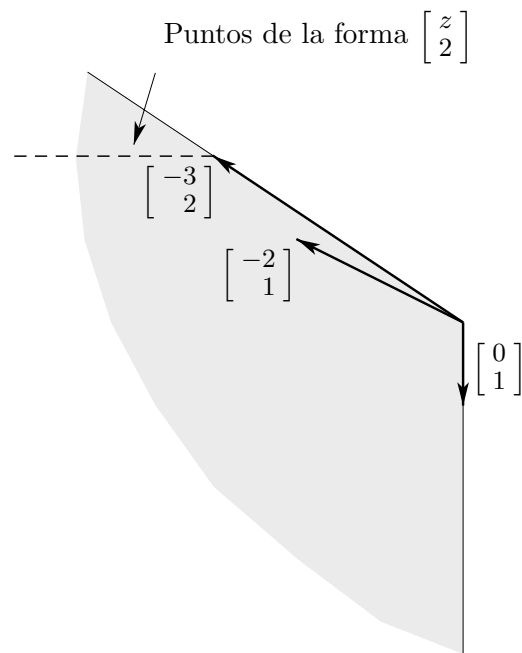


Figura 6.10
Geometría del ejemplo 6.4

6.2 Polítopos

A continuación vamos a caracterizar lo que en apartados anteriores denominábamos subespacio de actividad, y formalizar todas las impresiones que hasta este momento habrá ido adquiriendo el lector sobre la programación lineal con los ejemplos, figuras y resultados presentados. Razonaremos en $\mathbb{R}^n$.

Definición 6.4 Llamaremos *hiperplano* H de *vector característico* $\mathbf{a} \in \mathbb{R}^n$, $\mathbf{a} \neq \mathbf{0}$, al conjunto $H = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T \mathbf{x} = c\}$, con $c \in \mathbb{R}$.

De acuerdo con esto,

$$\mathbf{a}^T \mathbf{x} = c, \quad \text{ó} \quad a_1 x_1 + \cdots + a_n x_n = c,$$

es una ecuación del hiperplano. Es claro que $\hat{\mathbf{a}}^T \mathbf{x} = \hat{c}$ es ecuación de H si y sólo si existe un $\lambda \neq 0$ tal que $\hat{\mathbf{a}} = \lambda \mathbf{a}$ y $\hat{c} = \lambda c$.

Un hiperplano es el conjunto de soluciones de una ecuación lineal.

Definición 6.5 Un hiperplano en $\mathbb{R}^n$ es una variedad lineal $(n - 1)$ -dimensional.

Definición 6.6 Dado un hiperplano H por su ecuación $\mathbf{a}^T \mathbf{x} = c$, llamaremos *semiespacios cerrados de borde* H a los conjuntos

$$H_+ = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T \mathbf{x} \geq c\}$$

y

$$H_- = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T \mathbf{x} \leq c\},$$

y *semiespacios abiertos de borde* H a

$$\overset{\circ}{H}_+ = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T \mathbf{x} > c\}$$

y

$$\overset{\circ}{H}_- = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T \mathbf{x} < c\}.$$

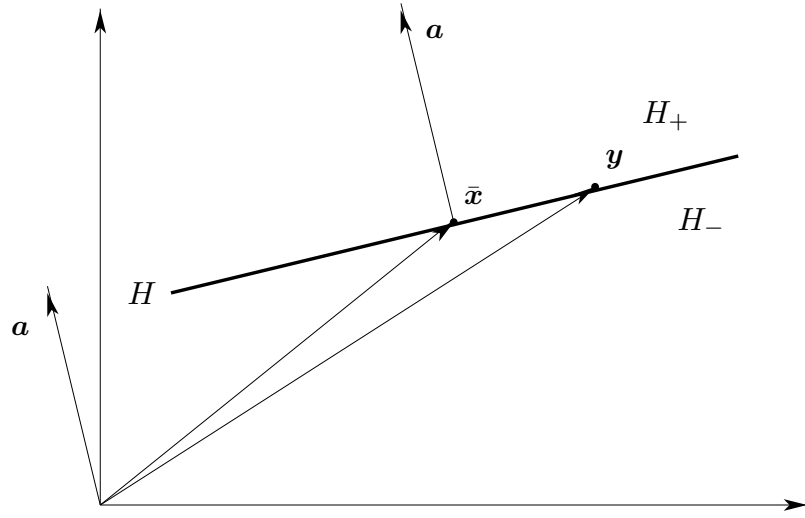
Si dos puntos $\bar{\mathbf{x}}$ e $\mathbf{y}$ pertenecen al hiperplano,

$$\mathbf{a}^T \bar{\mathbf{x}} - \mathbf{a}^T \mathbf{y} = c - c = 0.$$

Es decir, $\mathbf{a}^T(\bar{\mathbf{x}} - \mathbf{y}) = 0$. En la figura 6.11 se representa el hiperplano $-x_1 + 4x_2 = 11$, su vector característico $\mathbf{a} = [-1, 4]^T$ y los semiespacios H_+ y H_- .

Un hiperplano H y sus correspondientes semiespacios se pueden también referenciar con respecto a un punto fijo $\bar{\mathbf{x}} \in H$. De acuerdo con esto,

$$H_+ = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T(\mathbf{x} - \bar{\mathbf{x}}) \geq 0\},$$

**Figura 6.11**

Representación del hiperplano $-x_1 + 4x_2 = 11$, y los semiespacios que define

$$H_- = \{ \mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T (\mathbf{x} - \bar{\mathbf{x}}) \leq 0 \},$$

$$\overset{\circ}{H}_+ = \{ \mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T (\mathbf{x} - \bar{\mathbf{x}}) > 0 \}$$

y

$$\overset{\circ}{H}_- = \{ \mathbf{x} \in \mathbb{R}^n : \mathbf{a}^T (\mathbf{x} - \bar{\mathbf{x}}) < 0 \}.$$

El vector $\mathbf{a}$ está dirigido hacia el exterior de H_- o hacia el interior de H_+ . En efecto, si $\mathbf{y} \in H$ y $\mathbf{w} \in H_-$, se tiene que

$$\mathbf{a}^T (\mathbf{w} - \mathbf{y}) = \mathbf{a}^T \mathbf{w} - \mathbf{a}^T \mathbf{y} \leq c - c = 0.$$

Es decir, $\mathbf{a}$ forma un ángulo obtuso con cualquier vector dirigido hacia el interior de H_- : está por consiguiente dirigido hacia el exterior de H_- .

Los semiespacios de borde H son convexos; la unión de H_+ y H_- es el espacio $\mathbb{R}^n$.

Definición 6.7 Un *politopo* es un conjunto formado por la intersección de un número finito de semiespacios cerrados.

Definición 6.8 Un *politopo cónico* es un conjunto formado por la intersección de un número finito de semiespacios cerrados que pasan por un punto determinado.

Definición 6.9 Un *poliedro* es un politopo acotado y no vacío.

Es fácil comprobar que la intersección de conjuntos convexos es convexa y que por lo tanto los politopos y los poliedros son conjuntos convexos.

El conjunto $P = \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, de soluciones de un programa lineal (región factible) es un politopo convexo. En efecto, la ecuación

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n = b_1$$

es equivalente al sistema de desigualdades

$$\begin{aligned} a_1x_1 + a_2x_2 + \cdots + a_nx_n &\leq b_1 \\ a_1x_1 + a_2x_2 + \cdots + a_nx_n &\geq b_1, \end{aligned}$$

es decir, resulta de la intersección de estos dos semiespacios cerrados, por lo que P es un politopo. Que es convexo lo demuestra el teorema 6.1 de la página 383.

Definición 6.10 El conjunto intersección de todos los conjuntos convexos que contienen a un subconjunto $S \subset \Re^n$ se llama *envoltura convexa* de S y se designa por $Co(S)$.

Definición 6.11 Se denomina *hiperplano soporte* de un conjunto convexo C a un hiperplano H tal que $H \cap C \neq \emptyset$ y $C \subseteq H_+$ o $C \subseteq H_-$. Es decir, a un hiperplano que contiene al conjunto C en uno de sus semiespacios cerrados de borde H y algún punto frontera de C .

Definición 6.12 Si P es un politopo convexo y H cualquier hiperplano separador de P , la intersección $F = P \cap H$ define una *cara* de P .

Existen tres tipos especiales de caras.

Definición 6.13 Un *vértice*, una *arista* y una *faceta* son caras de un politopo convexo n -dimensional de dimensiones cero, uno y $n - 1$, respectivamente.

En un politopo convexo, obviamente, los vértices son los puntos extremos. Las aristas son segmentos de recta que unen dos puntos extremos adyacentes, o rectas semiinfinitas que parten de un punto extremo. Si $P = \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, cualquier faceta de P corresponde a su intersección con cada uno de los semiespacios que definen las desigualdades

$$\mathbf{a}_1^T \mathbf{x} \leq b_1, \dots, \mathbf{a}_m^T \mathbf{x} \leq b_m$$

y

$$x_1 \geq 0, \dots, x_n \geq 0.$$

6.3 Puntos extremos y soluciones básicas factibles

Como hemos visto en los ejemplos de apartados anteriores, si un problema de programación lineal con dos variables tiene una solución óptima finita, ésta ocurre en un punto extremo de la región factible que delimitan las soluciones factibles. Como probaremos a continuación, esto también se cumple en $\Re^n$.

Consideraremos en los sucesivos las condiciones del programa lineal en forma estándar, es decir

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0}, \end{aligned} \quad (6.5)$$

donde $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$ y $A \in \mathbb{R}^{m \times n}$ ($n > m$).

Si suponemos que el sistema $A\mathbf{x} = \mathbf{b}$ es compatible, podemos suponer sin ninguna limitación que $\text{rango}(A) = m$. Por lo tanto, de las n columnas de la matriz A podemos elegir m columnas linealmente independientes que formen una base del espacio vectorial que generan los vectores columna de A —subespacio $\text{Im}(A)$ —. Por simplicidad de notación supongamos que esas m columnas son las m primeras y designemos por B la submatriz $m \times m$ de A que forman. Como B es regular, la ecuación

$$B\mathbf{x}_B = \mathbf{b},$$

se puede resolver de forma única. El vector $\mathbf{x}^T = [\mathbf{x}_B^T, \mathbf{0}^T]$, que resulta de considerar los componentes de $\mathbf{x}_B$ como los m primeros componentes de $\mathbf{x}$, proporciona una de las soluciones de la ecuación $A\mathbf{x} = \mathbf{b}$.

Definición 6.14 Sea B cualquier submatriz no singular $m \times m$ resultante de agrupar m columnas linealmente independientes de A . Si todos los $n - m$ componentes del vector $\mathbf{x}$ no asociados a las columnas de B , a los que se denominarán *variables no básicas*, se hacen cero y se resuelve la ecuación $A\mathbf{x} = \mathbf{b}$ en los m restantes componentes, denominados *variables básicas*, la solución resultante se denomina *solución básica* asociada a la *matriz básica*, o *base*, B . Las $n - m$ columnas de A que no forman parte de B se las agrupa en una matriz $m \times (n - m)$ denominada *matriz no básica* N (asociada a las variables no básicas); en correspondencia, las variables no básicas forman $\mathbf{x}_N$.

Es bien sabido, y fácil de demostrar, que si las m primeras columnas de la matriz A son linealmente independientes, el sistema $A\mathbf{x} = \mathbf{b}$ puede, con una sucesión de multiplicaciones y restas, convertirse a la *forma canónica*:

$$\begin{array}{rcll} x_1 & + & a'_{1m+1}x_{m+1} + a'_{1m+2}x_{m+2} + \cdots + a'_{1n}x_n & = b'_1 \\ x_2 & + & a'_{2m+1}x_{m+1} + a'_{2m+2}x_{m+2} + \cdots + a'_{2n}x_n & = b'_2 \\ & & \vdots & \vdots \\ x_m & + & a'_{mm+1}x_{m+1} + a'_{mm+2}x_{m+2} + \cdots + a'_{mn}x_n & = b'_m. \end{array}$$

Ejemplo 6.5 Consideremos el poliedro de la figura 6.12, definido por

$$\begin{aligned} x_1 + x_2 &\leq 6 \\ x_2 &\leq 3 \\ x_1, x_2 &\geq 0. \end{aligned}$$

Si añadimos las variables de holgura x_3 y x_4 a la primera y segunda desigualdad, respectivamente, resulta:

$$\begin{aligned} x_1 + x_2 + x_3 &= 6 \\ x_2 + x_4 &= 3 \\ x_1, x_2, x_3, x_4 &\geq 0. \end{aligned}$$

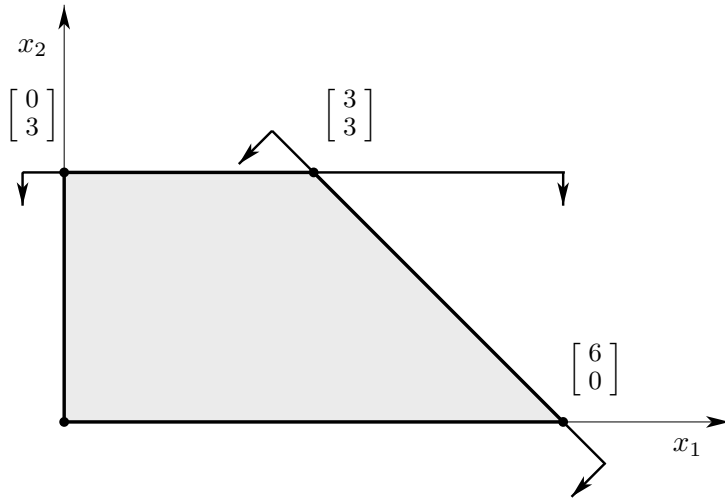


Figura 6.12
Soluciones básicas/soluciones básicas factibles

La matriz de los coeficientes de las condiciones, A , es

$$A = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}.$$

Las posibles matrices B que se pueden extraer de A y sus correspondientes soluciones básicas son las de la tabla 6.1.

Las soluciones básicas factibles son pues

$$\mathbf{x}_1 = \begin{bmatrix} 3 \\ 3 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 6 \\ 0 \\ 0 \\ 3 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} 0 \\ 3 \\ 3 \\ 0 \end{bmatrix} \quad \text{y} \quad \mathbf{x}_4 = \begin{bmatrix} 0 \\ 0 \\ 6 \\ 3 \end{bmatrix}.$$

Obsérvese que estos puntos determinan en sus dos primeros componentes los puntos extremos de la figura 6.12. ●

Deduzcamos a continuación unos resultados que nos permitirán en lo sucesivo centrarnos sólo en los puntos extremos del politopo convexo, o región factible, que definen las condiciones del programa lineal.

Teorema 6.2 (*Equivalencia entre puntos extremos y soluciones básicas*) Sean $A \in \mathbb{R}^{m \times n}$ una matriz de rango m y $\mathbf{b} \in \mathbb{R}^m$. Sea el politopo convexo

$$P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}.$$

Un vector $\mathbf{x} \in P$ es un punto extremo de P si y sólo si los vectores columna de la matriz A asociados a los componentes positivos de $\mathbf{x}$ son linealmente independientes.

Tabla 6.1
Bases y soluciones básicas del poliedro del ejemplo 6.5

$B = [\mathbf{a}_1, \mathbf{a}_2] = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$	$\mathbf{x}_B = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$ $\mathbf{x}_N = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
$B = [\mathbf{a}_1, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\mathbf{x}_B = \begin{bmatrix} x_1 \\ x_4 \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \end{bmatrix}$ $\mathbf{x}_N = \begin{bmatrix} x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
$B = [\mathbf{a}_2, \mathbf{a}_3] = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$	$\mathbf{x}_B = \begin{bmatrix} x_2 \\ x_3 \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$ $\mathbf{x}_N = \begin{bmatrix} x_1 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
$B = [\mathbf{a}_2, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$	$\mathbf{x}_B = \begin{bmatrix} x_2 \\ x_4 \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 6 \\ -3 \end{bmatrix}$ $\mathbf{x}_N = \begin{bmatrix} x_1 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
$B = [\mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$\mathbf{x}_B = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \end{bmatrix}$ $\mathbf{x}_N = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

DEMOSTRACIÓN. Supongamos sin pérdida de generalidad que los p primeros componentes del vector $\mathbf{x}$ son positivos y los $n - p$ últimos cero. Si $\mathbf{x} = [\bar{\mathbf{x}}^T, \mathbf{0}^T]^T$, $\bar{\mathbf{x}} > \mathbf{0}$, y designamos por $\bar{A}$ las p primeras columnas de la matriz A , se tiene que $A\mathbf{x} = \bar{A}\bar{\mathbf{x}} = \mathbf{b}$.

Probemos primero la necesidad de la condición enunciada. Supongamos que las columnas de $\bar{A}$ no son linealmente independientes. En este caso existirá un vector $\bar{\mathbf{w}} \neq \mathbf{0}$ tal que $\bar{A}\bar{\mathbf{w}} = \mathbf{0}$. De aquí que $\bar{A}(\bar{\mathbf{x}} \pm \varepsilon\bar{\mathbf{w}}) = \bar{A}\bar{\mathbf{x}} = \mathbf{b}$ y, para un ε suficientemente pequeño, que $(\bar{\mathbf{x}} \pm \varepsilon\bar{\mathbf{w}}) \geq \mathbf{0}$. Los puntos

$$\mathbf{y}' = \begin{bmatrix} \bar{\mathbf{x}} + \varepsilon\bar{\mathbf{w}} \\ \mathbf{0} \end{bmatrix} \quad \text{y} \quad \mathbf{y}'' = \begin{bmatrix} \bar{\mathbf{x}} - \varepsilon\bar{\mathbf{w}} \\ \mathbf{0} \end{bmatrix}$$

están, por consiguiente, en P . Además, dado que $\mathbf{x} = \frac{1}{2}(\mathbf{y}' + \mathbf{y}'')$, $\mathbf{x}$ no puede ser un punto extremo de P . Como consecuencia de esto, si $\mathbf{x}$ es un punto extremo, las columnas de la matriz $\bar{A}$ son linealmente dependientes.

Probemos ahora la suficiencia. Supongamos que $\mathbf{x}$ no es un punto extremo de P . Esto quiere decir que $\mathbf{x} = \lambda\mathbf{y}' + (1 - \lambda)\mathbf{y}''$, donde $\mathbf{y}', \mathbf{y}'' \in P$, $\mathbf{y}' \neq \mathbf{y}''$ y $0 < \lambda < 1$. Como $\mathbf{x}$ e $\mathbf{y}'$ están en P , $A(\mathbf{x} - \mathbf{y}') = A\mathbf{x} - A\mathbf{y}' = \mathbf{b} - \mathbf{b} = \mathbf{0}$. Además, dado que λ y $1 - \lambda$ son estrictamente positivos, los últimos $n - p$ componentes de $\mathbf{y}'$, y por consiguiente de $\mathbf{x} - \mathbf{y}'$, han de ser cero pues lo son los de $\mathbf{x}$. Las columnas de la matriz $\bar{A}$, en consecuencia, son linealmente dependientes. De aquí que, si las columnas de $\bar{A}$ son linealmente independientes, $\mathbf{x}$ es un punto extremo. ■

Corolario 6.1 Un punto $\mathbf{x} \in P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ es un punto extremo de P si y sólo si $\mathbf{x}$ es una solución básica factible de

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0} \end{aligned}$$

asociada a una base B .

Corolario 6.2 Un vector $\mathbf{x}$ es un punto extremo de $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ si y sólo si $\mathbf{x}$ resulta de la intersección de n hiperplanos linealmente independientes.

Corolario 6.3 Un politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ tiene un número finito de puntos extremos.

DEMOSTRACIÓN. Resulta inmediatamente del teorema anterior y del hecho de que haya sólo un número finito de posibilidades de escoger m columnas linealmente independientes entre las n de la matriz A . El número máximo de éstas y, por tanto, de puntos extremos de P es

$$C(n, m) = \binom{n}{m} = \frac{n!}{m!(n-m)!}.$$

■

Cuando A no tiene rango completo puede ocurrir que $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ sea el conjunto vacío o que alguna de las condiciones sea redundante. En lo sucesivo supondremos que $A \in \mathbb{R}^{m \times n}$ tiene m vectores fila/columna linealmente independientes.

La correspondencia entre soluciones básicas factibles y puntos extremos, en general, no es biunívoca. A cada solución básica factible le corresponde un único punto extremo en el politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, pero puede que a cada punto extremo de P le corresponda más de una solución básica factible.

Definición 6.15 Si una o más de las variables básicas de una solución básica de

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0}, \end{aligned} \tag{6.6}$$

es cero, la solución se denomina *básica degenerada*.

Definición 6.16 Una solución básica de (6.6) en la que todos sus componentes son no negativos se denomina *solución básica factible*; si algún componente es cero, la solución básica factible se dice *básica factible degenerada*.

Ejemplo 6.6 Consideremos el poliedro representado en la figura 6.13 definido por:

$$\begin{aligned} x_1 + x_2 &\leq 6 \\ x_2 &\leq 3 \\ x_1 + 2x_2 &\leq 9 \\ x_1, x_2 &\geq 0. \end{aligned}$$

Si añadimos las variables de holgura x_3 , x_4 y x_5 a la primera, segunda y tercera desigualdad, respectivamente, resulta

$$\begin{aligned} x_1 + x_2 + x_3 &= 6 \\ x_2 + x_4 &= 3 \\ x_1 + 2x_2 + x_5 &= 9 \\ x_1, x_2, x_3, x_4, x_5 &\geq 0. \end{aligned}$$

Obsérvese, como se describe en la figura 6.13, que la desigualdad $x_1 + 2x_2 \leq 9$ es redundante.

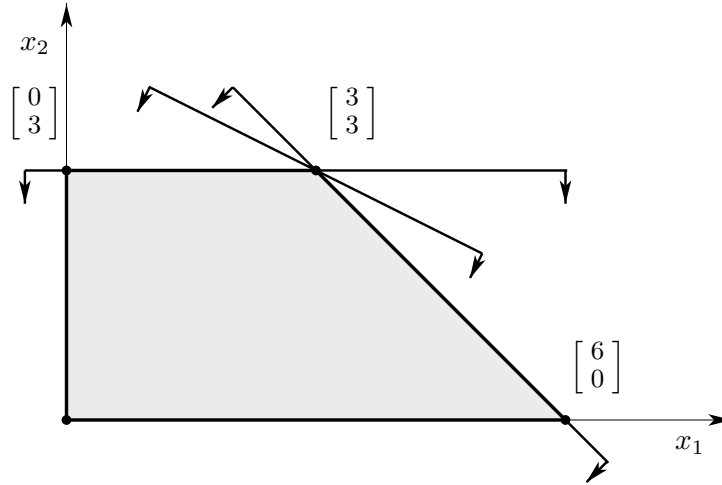


Figura 6.13

Soluciones básicas factibles degeneradas

La matriz de los coeficientes de las condiciones, A , es

$$A = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \mathbf{a}_4, \mathbf{a}_5] = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 2 & 0 & 0 & 1 \end{bmatrix}.$$

Estudiemos la solución básica que se obtiene a partir de $B = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$:

$$\begin{aligned} \mathbf{x}_B = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} &= B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 2 & 0 \end{bmatrix}^{-1} \begin{bmatrix} 6 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 0 & -2 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 0 \end{bmatrix}, \\ \mathbf{x}_N = \begin{bmatrix} x_4 \\ x_5 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \end{aligned}$$

La solución básica factible así obtenida es degenerada, pues su tercer componente es cero.

Analicemos ahora la solución básica que se obtiene a partir de considerar $B = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_4]$:

$$\begin{aligned} \mathbf{x}_B = \begin{bmatrix} x_1 \\ x_2 \\ x_4 \end{bmatrix} &= B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 0 \end{bmatrix}^{-1} \begin{bmatrix} 6 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 2 & 0 & -1 \\ -1 & 0 & 1 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 6 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 0 \end{bmatrix}, \\ \mathbf{x}_N = \begin{bmatrix} x_3 \\ x_5 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \end{aligned}$$

Como se puede ver es la misma solución que obteníamos antes. Si consideráramos $B = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_5]$ llegaríamos también a la misma solución básica degenerada: $[x_1 \ x_2 \ x_3 \ x_4 \ x_5]^T = [3 \ 3 \ 0 \ 0 \ 0]^T$. Se puede comprobar que cualquier otra solución básica es no degenerada. ●

Un problema de programación lineal se denomina *no degenerado* si todas sus soluciones básicas factibles son no degeneradas. En este caso, la correspondencia que mencionábamos anteriormente entre puntos extremos y soluciones básicas factibles sí es biunívoca.

Dos soluciones básicas factibles del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ se dicen *adyacentes* si $m - 1$ de sus componentes que forman la base son comunes. Dos soluciones adyacentes o puntos extremos están unidos por una *arista*. Suponiendo que un programa lineal es no degenerado, como $n - m$ variables no básicas pueden reemplazar una de las básicas en una solución básica factible, cualquiera de éstas (y su correspondiente punto extremo) tiene exactamente $n - m$ adyacentes. Como veremos posteriormente al analizar el algoritmo *simplex*, cualquiera de estas soluciones básicas factibles adyacentes se puede alcanzar incrementando el valor de una variable no básica desde su valor cero y decrementando el de una básica desde el valor que tenga hasta cero. Este proceso es el que se conoce en la literatura especializada en programación lineal como *pivotación*.

6.3.1 Teorema de la representación

Como probaremos a continuación, si el politopo P es un poliedro, cualquier punto de P se puede expresar como combinación convexa de los puntos extremos de P —ver corolario más adelante—. Si P no está acotado la expresión de cualquier punto de P es más complicada y requiere la siguiente definición.

Definición 6.17 Una *dirección* del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ es un vector no nulo, $\mathbf{d} \in \mathbb{R}^n$, tal que para todo $\mathbf{x}_0 \in P$ el rayo $\{\mathbf{x} \in \mathbb{R}^n : \mathbf{x} = \mathbf{x}_0 + \lambda \mathbf{d}, \lambda \geq 0\}$ pertenece a P .

De forma similar a como se introdujo el concepto de punto extremo de un conjunto convexo, ahora podemos introducir el de dirección extrema.

Definición 6.18 Una *dirección* $\mathbf{d}$ de un politopo P se dice *extrema* si no puede ponerse como combinación lineal no negativa de dos direcciones diferentes de P . Es decir, no existen dos direcciones $\mathbf{d}_1$ y $\mathbf{d}_2$ en P , $\mathbf{d}_1 \neq \mathbf{d}_2$, y unos $\alpha_1, \alpha_2 > 0$, tales que $\mathbf{d} = \alpha_1 \mathbf{d}_1 + \alpha_2 \mathbf{d}_2$.

Cualquier dirección de un politopo se puede expresar como combinación lineal no negativa de las direcciones extremas del politopo. Si P es un poliedro, obviamente, no tiene direcciones.

Teorema 6.3 Sea $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$. Un vector no nulo $\mathbf{d}$ es una dirección de P si y sólo si $\mathbf{d} \in D = \{\mathbf{d} : A\mathbf{d} = \mathbf{0}, \mathbf{d} \geq \mathbf{0}\}$.

DEMOSTRACIÓN. Comprobemos primero la necesidad de la condición. Sea $\mathbf{d}$ una dirección de P . Por definición, $\mathbf{d} \neq \mathbf{0}$ y $\mathbf{x} + \lambda \mathbf{d} \in P$ para todo $\mathbf{x} \in P$ y $\lambda \geq 0$. Entonces, para todo $\lambda \geq 0$,

$$A(\mathbf{x} + \lambda \mathbf{d}) = A\mathbf{x} + \lambda A\mathbf{d} = \mathbf{b} + \lambda A\mathbf{d} = \mathbf{b}$$

siendo $\mathbf{x} + \lambda \mathbf{d} \geq \mathbf{0}$ por pertenecer este vector a P . Si en la última expresión, sin pérdida de generalidad, se hace $\lambda = 1$, es evidente que $A\mathbf{d} = \mathbf{0}$. Como $\mathbf{x} \geq \mathbf{0}$ y $\mathbf{x} + \lambda \mathbf{d} \geq \mathbf{0}$, debe cumplirse que $\mathbf{d} \geq \mathbf{0}$ pues si no haciendo λ arbitrariamente grande se podría conseguir que no se cumpliera que $\mathbf{x} + \lambda \mathbf{d} \geq \mathbf{0}$.

Para demostrar la suficiencia sean los vectores $\mathbf{d} \in D = \{\mathbf{d} : A\mathbf{d} = \mathbf{0}, \mathbf{d} \geq \mathbf{0}, \mathbf{d} \neq \mathbf{0}\}$ y $\mathbf{x} \in P$. De la definición de D se cumple que $\mathbf{d} \neq \mathbf{0}$. Sólo queda por probar que $\mathbf{x} + \lambda \mathbf{d} \in P$ para todo $\lambda \geq 0$. Se tiene que

$$A(\mathbf{x} + \lambda \mathbf{d}) = A\mathbf{x} + \lambda A\mathbf{d} = \mathbf{b} + \mathbf{0} = \mathbf{b}$$

y como $\mathbf{x} \geq \mathbf{0}$, $\mathbf{d} \geq \mathbf{0}$ y $\lambda \geq 0$, se deduce inmediatamente que $\mathbf{x} + \lambda \mathbf{d} \geq \mathbf{0}$. Por consiguiente $\mathbf{x} + \lambda \mathbf{d} \in P$ para todo $\lambda \geq 0$. ■

De igual forma se puede probar que $\mathbf{d}$ es una dirección del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \geq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ si y sólo si $A\mathbf{d} \geq \mathbf{0}$, $\mathbf{d} \neq \mathbf{0}$ y $\mathbf{d} \geq \mathbf{0}$. Y del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ si y sólo si $A\mathbf{d} \leq \mathbf{0}$, $\mathbf{d} \neq \mathbf{0}$ y $\mathbf{d} \geq \mathbf{0}$. En general, el conjunto de direcciones de un politopo es el conjunto de soluciones del correspondiente sistema homogéneo de ecuaciones.

Ejemplo 6.7 Consideremos el politopo $P = \{[x_1, x_2]^T : x_1 - 2x_2 \geq -6, x_1 - x_2 \geq -2, x_1 \geq 0, x_2 \geq 1\}$ de la figura 6.14. Un vector no nulo $\mathbf{d} = [d_1, d_2]^T$ es una dirección de P si y sólo si se cumple que

$$\begin{aligned} d_1 - 2d_2 &\geq 0 \\ d_1 - d_2 &\geq 0 \\ d_1 &\geq 0 \\ d_2 &\geq 0. \end{aligned}$$

Dado que d_1 y d_2 son no negativos, las dos primeras desigualdades equivalen a $d_1 \geq 2d_2$ y $d_1 \geq d_2$.

En resumen, $\mathbf{d}$ es una dirección de P si y sólo si $[d_1, d_2] \neq [0, 0]$, $d_1 \geq 0$, $d_2 \geq 0$ y $d_1 \geq 2d_2$. Los vectores que cumplen estas condiciones se representan en la figura 6.14. ●

Los puntos y direcciones extremos juegan un papel fundamental en programación lineal para determinar las condiciones en que se obtiene o llega al óptimo de un problema. La interpretación gráfica de esto la pone de manifiesto la figura 6.15.

El politopo no acotado P tiene tres puntos extremos, $\mathbf{x}_1$, $\mathbf{x}_2$ y $\mathbf{x}_3$, y dos direcciones extremas, $\mathbf{d}_1$ y $\mathbf{d}_2$. El punto $\bar{\mathbf{x}}$ se puede expresar de la siguiente manera:

$$\bar{\mathbf{x}} = \mathbf{y} + \lambda \mathbf{d}_1$$

para algún $\lambda > 0$. Es decir, $\bar{\mathbf{x}}$ está en la trayectoria del rayo que parte de $\mathbf{y}$ en la dirección $\mathbf{d}_1$. Ahora bien, al estar $\mathbf{y}$ en la recta que une $\mathbf{x}_1$ y $\mathbf{x}_2$ se puede expresar como combinación convexa de $\mathbf{x}_1$ y $\mathbf{x}_2$. Es decir,

$$\mathbf{y} = \alpha \mathbf{x}_1 + (1 - \alpha) \mathbf{x}_2$$

para algún $\alpha \in (0, 1)$. Sustituyendo esta última expresión de $\mathbf{y}$ en la de $\bar{\mathbf{x}}$ se tiene que

$$\bar{\mathbf{x}} = \alpha \mathbf{x}_1 + (1 - \alpha) \mathbf{x}_2 + \lambda \mathbf{d}_1, \quad \alpha \in (0, 1), \quad \lambda > 0.$$

De forma más completa,

$$\bar{\mathbf{x}} = \alpha \mathbf{x}_1 + (1 - \alpha) \mathbf{x}_2 + 0 \mathbf{x}_3 + \lambda \mathbf{d}_1 + 0 \mathbf{d}_2, \quad \alpha \in (0, 1), \quad \lambda > 0.$$

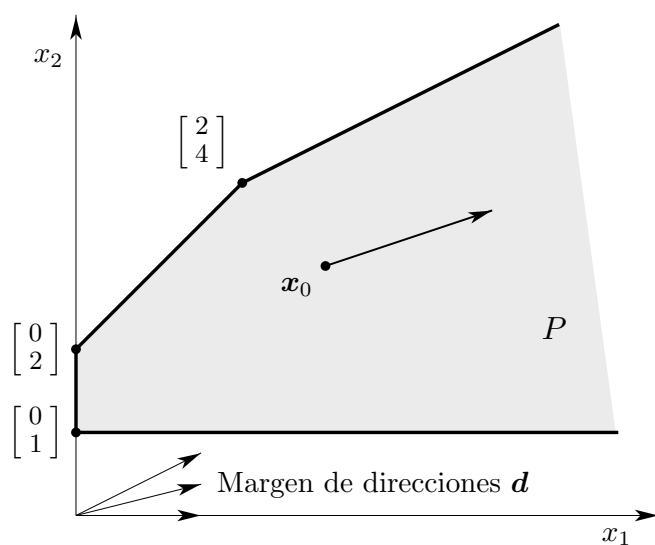


Figura 6.14
Direcciones en el politopo del ejemplo 6.7

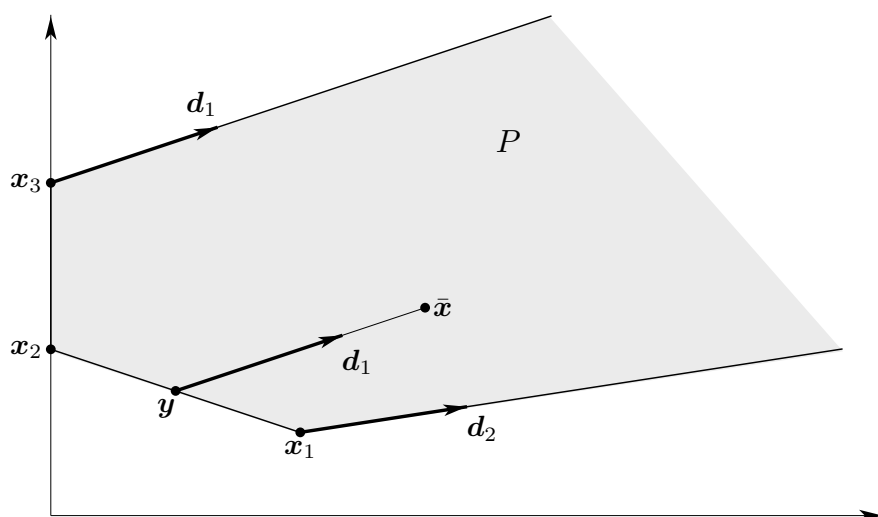


Figura 6.15
Puntos y direcciones extremos de un politopo P

En resumen, $\bar{\mathbf{x}}$ se puede expresar como suma de una combinación convexa de los puntos extremos $\mathbf{x}_1$, $\mathbf{x}_2$ y $\mathbf{x}_3$ y una no negativa de la direcciones extremas $\mathbf{d}_1$ y $\mathbf{d}_2$. Esta representación, evidentemente, no es única, pues bastaría, por ejemplo, encontrar otro punto de la recta que une $\mathbf{x}_1$ y $\mathbf{x}_2$ desde el que, en la dirección $\mathbf{d}_2$, se pudiese trazar un rayo que pasase por $\bar{\mathbf{x}}$.

El siguiente resultado, conocido como teorema de la representación, teorema de la resolución o teorema de Caratheodory, permite generalizar las últimas ideas expresadas mediante representación gráfica. Explicita que cualquier punto del politopo de soluciones factibles de un programa lineal se puede expresar como una combinación lineal convexa de los puntos extremos del politopo más una combinación no negativa de sus direcciones extremas.

Teorema 6.4 (Teorema de la representación) *Todo punto del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ se puede expresar de la forma*

$$\mathbf{x} = \sum_{i \in I} \lambda_i \mathbf{v}_i + \mathbf{d},$$

donde $\{\mathbf{v}_i : i \in I\}$ es el conjunto de puntos extremos de P , $\sum_{i \in I} \lambda_i = 1$, $\lambda_i \geq 0$, y $\mathbf{d}$, o es una dirección de P , o $\mathbf{d} = \mathbf{0}$.

DEMOSTRACIÓN. La haremos por inducción en p , número de componentes positivos de $\mathbf{x}$. Si $p = 0$, el teorema es obvio, pues $\mathbf{x} = \mathbf{0}$ es un punto extremo. Supongamos que se cumple lo enunciado para puntos con menos de p componentes positivos y que $\mathbf{x}$ tiene p componentes positivos.

Si $\mathbf{x}$ es un punto extremo, como $\mathbf{x} = \mathbf{v}_i$ para algún $i \in I$, el teorema es obvio. Supongamos por tanto que $\mathbf{x}$ no es un punto extremo. En este caso existe un vector $\mathbf{w} \neq \mathbf{0}$, con $w_i = 0$ si $x_i = 0$, tal que $A\mathbf{w} = \mathbf{0}$. Se pueden dar los tres casos siguientes:

- (a) *Que $\mathbf{w}$ tenga componentes positivos y negativos.* Consideremos los puntos $\mathbf{x}(\theta) = \mathbf{x} + \theta\mathbf{w}$ en la recta que pasa por $\mathbf{x}$ que determina $\mathbf{w}$, y sean θ' y θ'' el menor valor positivo y mayor valor negativo, respectivamente, de θ para los que $\mathbf{x}(\theta)$ tiene al menos un componente cero más que los que tiene $\mathbf{x}$. Los puntos $\mathbf{x}' = \mathbf{x}(\theta')$ y $\mathbf{x}'' = \mathbf{x}(\theta'')$ pertenecen claramente a P por lo que, por la hipótesis de inducción, al tener un componente nulo más, se pueden expresar según lo enunciado en el teorema. En consecuencia, como $\mathbf{x}$ está en la recta que une $\mathbf{x}'$ y $\mathbf{x}''$, se puede expresar de la siguiente manera

$$\begin{aligned} \mathbf{x} &= \mu\mathbf{x}' + (1 - \mu)\mathbf{x}'' \\ &= \mu \left(\sum_{i \in I} \lambda'_i \mathbf{v}_i + \mathbf{d}' \right) + (1 - \mu) \left(\sum_{i \in I} \lambda''_i \mathbf{v}_i + \mathbf{d}'' \right) \\ &= \sum_{i \in I} \left(\mu\lambda'_i + (1 - \mu)\lambda''_i \right) \mathbf{v}_i + \mu\mathbf{d}' + (1 - \mu)\mathbf{d}'', \end{aligned}$$

donde $\mu = -\theta''/(\theta' - \theta'')$.

Como $0 < \mu < 1$,

$$\begin{aligned} \lambda'_i \geq 0 \quad \text{y} \quad \lambda''_i \geq 0, \quad \text{para todo } i \in I, \quad \sum_{i \in I} \lambda'_i = \sum_{i \in I} \lambda''_i = 1, \\ A\mathbf{d}' = A\mathbf{d}'' = \mathbf{0}, \quad \mathbf{d}' \geq \mathbf{0} \quad \text{y} \quad \mathbf{d}'' \geq \mathbf{0}, \end{aligned}$$

se deduce entonces que

$$\begin{aligned}\lambda_i &= \mu\lambda'_i + (1 - \mu)\lambda''_i \geq 0 \quad \text{para todo } i \in I, \quad \sum_{i \in I} \lambda_i = 1, \\ \mathbf{d} &= \mu\mathbf{d}' + (1 - \mu)\mathbf{d}'' \geq 0 \quad \text{y} \quad A\mathbf{d} = \mathbf{0},\end{aligned}$$

quedando probado que $\mathbf{x}$ se puede expresar de la forma enunciada.

- (b) *Que $\mathbf{w} \leq \mathbf{0}$.* Definamos $\mathbf{x}'$ como en el caso (a). El punto $\mathbf{x}$ se puede expresar como $\mathbf{x} = \mathbf{x}' + \theta'(-\mathbf{w})$, con $\theta' > 0$. Como $\mathbf{x}'$ se puede expresar por inducción en la forma deseada y $(-\mathbf{w})$ es una dirección en P , $\mathbf{x}$ también se puede expresar de la forma enunciada.
- (c) *Que $\mathbf{w} \geq \mathbf{0}$.* Este caso se prueba igual que el caso (b) sin más que sustituir $\mathbf{x}'$, θ' y $-\mathbf{w}$ por $\mathbf{x}''$, $-\theta''$ y $\mathbf{w}$, respectivamente. ■

Corolario 6.4 Si el politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ es no vacío, tiene al menos un punto extremo.

Corolario 6.5 Si el politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ es cerrado y acotado (es un poliedro), todo punto $\mathbf{x} \in P$ se puede expresar como combinación convexa de sus puntos extremos.

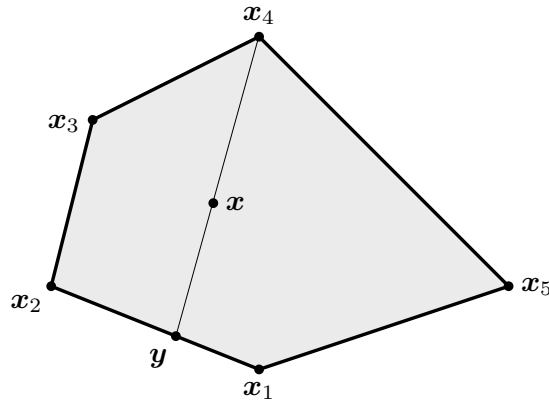


Figura 6.16

Representación de un punto de un politopo (poliedro) como combinación convexa de puntos extremos

Consideremos el poliedro de la figura 6.16, resultante de la intersección de 5 semiespacios cerrados. Cualquier punto del poliedro, por ejemplo $\mathbf{x}$, se puede representar como combinación convexa de algunos (o todos) de los 5 puntos extremos del mismo. En efecto,

$$\mathbf{x} = \lambda\mathbf{y} + (1 - \lambda)\mathbf{x}_4,$$

donde $0 < \lambda < 1$. El punto $\mathbf{y}$ también se puede representar como combinación convexa de $\mathbf{x}_1$ y $\mathbf{x}_2$. Es decir,

$$\mathbf{y} = \mu\mathbf{x}_1 + (1 - \mu)\mathbf{x}_2,$$

donde $0 < \mu < 1$. Sustituyendo,

$$\mathbf{x} = \lambda\mu\mathbf{x}_1 + \lambda(1-\mu)\mathbf{x}_2 + (1-\lambda)\mathbf{x}_4.$$

Como $\lambda \in (0, 1)$ y μ también, $\lambda\mu$, $\lambda(1-\mu)$ y $(1-\lambda)$ pertenecen a $(0, 1)$, y $\lambda\mu + \lambda(1-\mu) + (1-\lambda) = 1$. Luego $\mathbf{x}$ se ha representado mediante una combinación convexa de los puntos extremos $\mathbf{x}_1$, $\mathbf{x}_2$ y $\mathbf{x}_4$.

Ejemplo 6.8 Consideremos el politopo siguiente:

$$\begin{aligned} -3x_1 + x_2 &\leq -2 \\ -x_1 + x_2 &\leq 2 \\ -x_1 + 2x_2 &\leq 8 \\ -x_2 &\leq -2. \end{aligned}$$

Sus puntos extremos y direcciones extremas son:

$$\mathbf{x}_1 = \begin{bmatrix} 4/3 \\ 2 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \quad \text{y} \quad \mathbf{x}_3 = \begin{bmatrix} 4 \\ 6 \end{bmatrix};$$

y

$$\mathbf{d}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \text{y} \quad \mathbf{d}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Obsérvese que en cada punto extremo sólo dos de las cuatro desigualdades se hacen igualdad: ver figura 6.17.

Sea $\mathbf{x} = [4, 3]^T$ un punto del politopo. Se puede expresar de la siguiente manera:

$$\begin{bmatrix} 4 \\ 3 \end{bmatrix} = \lambda_1 \begin{bmatrix} 4/3 \\ 2 \end{bmatrix} + \lambda_2 \begin{bmatrix} 2 \\ 4 \end{bmatrix} + \lambda_3 \begin{bmatrix} 4 \\ 6 \end{bmatrix} + \mu_1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \mu_2 \begin{bmatrix} 2 \\ 1 \end{bmatrix},$$

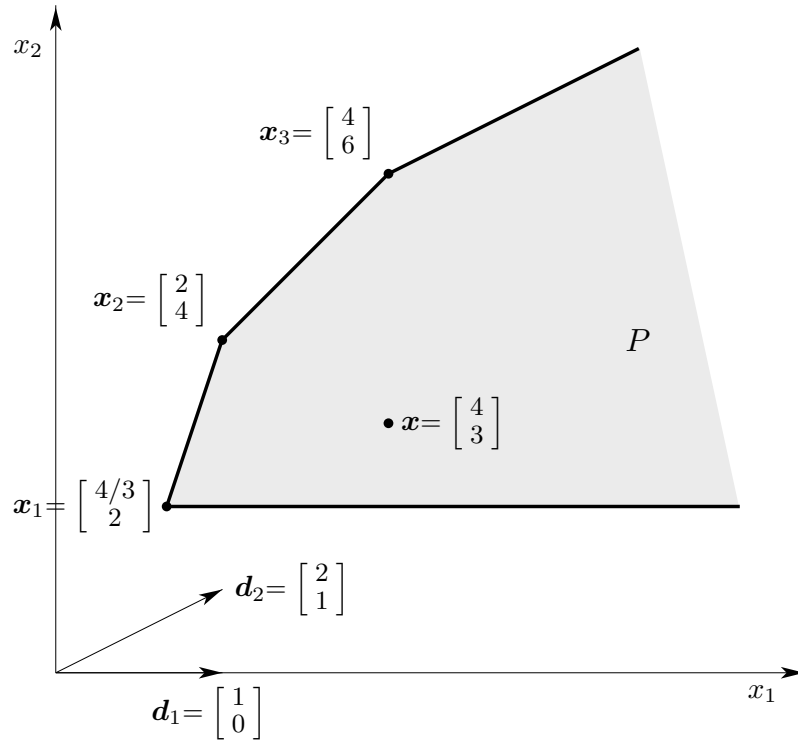
donde $\lambda_1 = \lambda_2 = \frac{1}{2}$, $\lambda_3 = 0$, $\mu_1 = \frac{7}{3}$ y $\mu_2 = 0$. Esta expresión no es única ya que haciendo $\lambda_1 = \frac{3}{4}$, $\lambda_2 = 0$, $\lambda_3 = \frac{1}{4}$, $\mu_1 = 2$ y $\mu_2 = 0$ se obtiene otra expresión de $\mathbf{x}$ como combinación de $\mathbf{x}_1$, $\mathbf{x}_2$, $\mathbf{x}_3$, $\mathbf{d}_1$ y $\mathbf{d}_2$. ●

6.3.2 Teorema fundamental de la programación lineal

En este apartado exponemos un teorema esencial para la estrategia de búsqueda de la solución de un tipo muy importante de algoritmos de programación lineal. Identifica la importancia de los puntos extremos del politopo (o poliedro) que definen las condiciones del problema en la identificación de las soluciones básicas factibles y del óptimo.

Teorema 6.5 Dado un politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ no vacío, el valor mínimo de $\mathbf{c}^T \mathbf{x}$, para $\mathbf{x} \in P$, se alcanza en un punto extremo de P (solución básica factible óptima), o $\mathbf{c}^T \mathbf{x}$ no está acotada inferiormente en P .

DEMOSTRACIÓN. Sea $V = \{\mathbf{v}_i : i \in I\}$ el conjunto de puntos extremos de P . Como P es no vacío, al menos tiene un punto extremo $\mathbf{v}_i \in V$. De acuerdo con el teorema de la representación, o el politopo P posee una dirección $\mathbf{d}$ tal que $\mathbf{c}^T \mathbf{d} < 0$, o tal dirección no existe. Consideremos estos dos casos.

**Figura 6.17**

Representación del politopo del ejemplo 6.8

- (a) El politopo P tiene una dirección $\mathbf{d}$ tal que $\mathbf{c}^T \mathbf{d} < 0$. En este caso P no está acotado y el valor de la función objetivo tiende a $-\infty$ en la dirección $\mathbf{d}$.
- (b) El politopo P no tiene una dirección $\mathbf{d}$ tal que $\mathbf{c}^T \mathbf{d} < 0$. En este caso cualquier $\mathbf{x} \in P$ se puede expresar de una de las dos maneras siguientes:

$$\mathbf{x} = \sum_{i \in I} \lambda_i \mathbf{v}_i \quad \text{donde} \quad \sum_{i \in I} \lambda_i = 1, \lambda_i \geq 0 \quad \text{o}$$

$$\mathbf{x} = \sum_{i \in I} \lambda_i \mathbf{v}_i + \bar{\mathbf{d}} \quad \text{donde} \quad \sum_{i \in I} \lambda_i = 1, \lambda_i \geq 0 \quad \text{y} \quad \mathbf{c}^T \bar{\mathbf{d}} \geq 0.$$

En ambos casos, suponiendo que $\mathbf{c}^T \mathbf{v}_{\min}$ es el menor de los elementos del conjunto $\{\mathbf{c}^T \mathbf{v}_i : i \in I\}$, se tiene que

$$\mathbf{c}^T \mathbf{x} \geq \sum_{i \in I} \lambda_i (\mathbf{c}^T \mathbf{v}_i) \geq \mathbf{c}^T \mathbf{v}_{\min} \left(\sum_{i \in I} \lambda_i \right) = \mathbf{c}^T \mathbf{v}_{\min}.$$

Es decir, el mínimo de $\mathbf{c}^T \mathbf{x}$ se alcanza en un punto extremo de P : $\mathbf{v}_{\min}$. ■

Es importante destacar que este teorema no excluye de ninguna manera la posibilidad de que la solución óptima de un programa lineal no se de en un punto extremo. Simplemente pone

de manifiesto que, de entre todas las soluciones óptimas de un programa lineal, al menos una es un punto extremo del politopo de soluciones factibles.

Ejemplo 6.9 Consideremos el politopo

$$\begin{aligned} -x_1 + x_2 &\leq 2 \\ -x_1 + 2x_2 &\leq 6 \\ x_1, x_2 &\geq 0. \end{aligned}$$

Sus puntos extremos y direcciones extremas son:

$$\mathbf{x}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \quad \text{y} \quad \mathbf{x}_3 = \begin{bmatrix} 2 \\ 4 \end{bmatrix};$$

y

$$\mathbf{d}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{y} \quad \mathbf{d}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Supongamos que sobre este politopo se quiere minimizar la función objetivo $x_1 - 3x_2$. En la figura 6.18 (a) se describe cómo el punto óptimo no está acotado. Se tiene que:

$$\begin{aligned} \mathbf{c}^T \mathbf{x}_1 &= [1, -3] \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0; \\ \mathbf{c}^T \mathbf{x}_2 &= [1, -3] \begin{bmatrix} 0 \\ 2 \end{bmatrix} = -6; \\ \mathbf{c}^T \mathbf{x}_3 &= [1, -3] \begin{bmatrix} 2 \\ 4 \end{bmatrix} = -10; \\ \mathbf{c}^T \mathbf{d}_1 &= [1, -3] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 \end{aligned}$$

y

$$\mathbf{c}^T \mathbf{d}_2 = [1, -3] \begin{bmatrix} 2 \\ 1 \end{bmatrix} = -1.$$

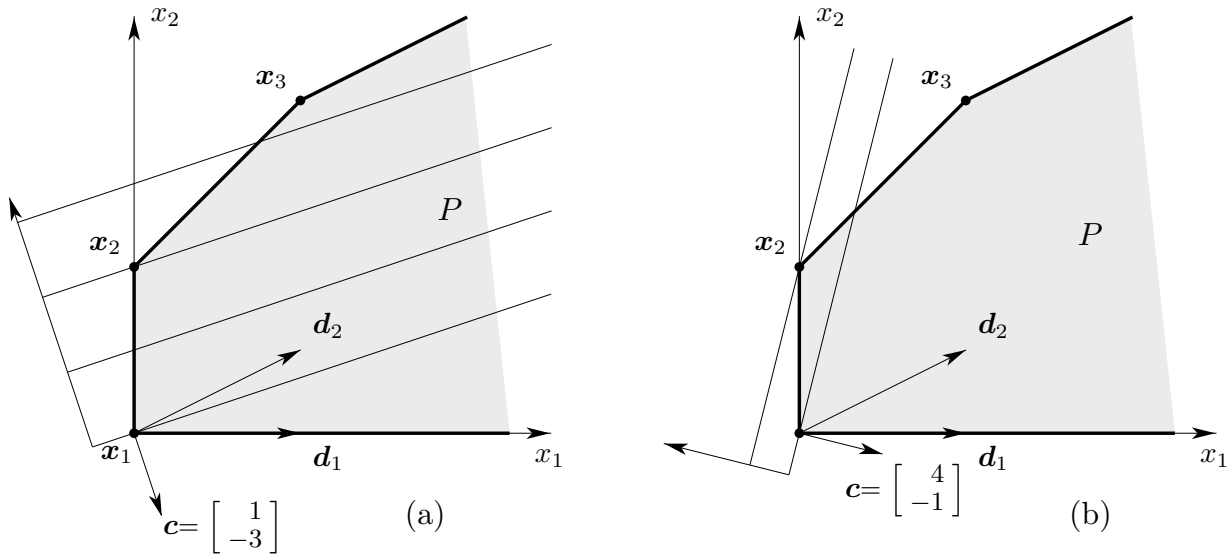
El problema es equivalente, por tanto, a

$$\begin{aligned} \text{min. } & 0\lambda_1 - 6\lambda_2 - 10\lambda_3 + \mu_1 - \mu_2 \\ \text{s. a } & \lambda_1 + \lambda_2 + \lambda_3 = 1 \\ & \lambda_1, \lambda_2, \lambda_3, \mu_1, \mu_2 \geq 0. \end{aligned}$$

Como $\mathbf{c}^T \mathbf{d}_2 = -1 < 0$ y μ_2 se puede hacer todo lo grande que queramos sin violar ninguna condición, el óptimo evidentemente no está acotado. Esto, junto con la figura 6.18 (a), ilustra la condición necesaria y suficiente de existencia de solución no acotada: esto es, que $\mathbf{c}^T \mathbf{d} < 0$.

Consideremos ahora $4x_1 - x_2$ como nueva función objetivo sobre el mismo politopo. En la figura 6.18 (b) se representa el óptimo de este problema: $\mathbf{x}_2 = [0, 2]^T$. En este caso:

$$\mathbf{c}^T \mathbf{x}_1 = [4, -1] \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0;$$

**Figura 6.18**

Direcciones extremas y óptimo: (a) solución óptima no acotada; (b) óptimo acotado

$$c^T x_2 = [4, -1] \begin{bmatrix} 0 \\ 2 \end{bmatrix} = -2;$$

$$c^T x_3 = [4, -1] \begin{bmatrix} 2 \\ 4 \end{bmatrix} = 4;$$

$$c^T d_1 = [4, -1] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 4$$

y

$$c^T d_2 = [4, -1] \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 7.$$

El problema es equivalente a

$$\begin{aligned} \min. \quad & 0\lambda_1 - 2\lambda_2 + 4\lambda_3 + 4\mu_1 + 7\mu_2 \\ \text{s. a} \quad & \lambda_1 + \lambda_2 + \lambda_3 = 1 \\ & \lambda_1, \lambda_2, \lambda_3, \mu_1, \mu_2 \geq 0. \end{aligned}$$

Como los coeficientes de μ_1 y de μ_2 en la función objetivo son positivos, se puede hacer $\mu_1 = \mu_2 = 0$. Para minimizar $-2\lambda_2 + 4\lambda_3$ sujeta a $\lambda_1 + \lambda_2 + \lambda_3 = 1$, con $\lambda_1, \lambda_2, \lambda_3 \geq 0$, se hace $\lambda_2 = 1$ y $\lambda_1 = \lambda_3 = 0$, lo que corrobora que el óptimo se alcanza en el punto extremo $x_2 = [0, 2]^T$. ●

Basándose en las consideraciones y resultados teóricos de este capítulo, ya se puede abordar el método práctico por excelencia para resolver problemas de programación lineal: el método *simplex*. Lo haremos en el siguiente capítulo.

Referencias

Para estudiar la teoría básica de la programación lineal a la que nos hemos referido en este capítulo se pueden consultar preferentemente Bazaraa, Jarvis y Sherali [1990], Ignizio y Cavalier [1994], Fang y Puthenpura [1993] y Goldfarb y Todd [1989].

La interpretación geométrica en los dos subespacios esenciales es bastante estándar; está muy bien explicada en Best y Ritter [1985], Luenberger [1984] y Goldfarb y Todd [1989]. Las explicaciones ilustradas siguen fundamentalmente a Bazaraa, Jarvis y Sherali [1990], Ignizio y Cavalier [1994], Fang y Puthenpura [1993] y Best y Ritter [1985]. La exposición de la equivalencia entre puntos extremos y soluciones básicas factibles es una modificación de Goldfarb y Todd [1989]. El teorema fundamental de la programación lineal sigue a Luenberger [1984] y Goldfarb y Todd [1989], estando apoyada en consideraciones geométricas afines de Bazaraa, Jarvis y Sherali [1990].

Para completar el estudio teórico de la programación lineal recomendamos: Schrijver [1986], desde un punto de vista más teórico y Gill, Murray y Wright [1991], mucho más práctico, con un perfecto engarce con la teoría afín de álgebra lineal.

Buenas referencias sobre programación lineal general son Chvátal [1983], Dantzig [1963], Darst [1991], Dorfman, Samuelson y Solow [1958], Fourer, Gay y Kernigham [1993], Hillier y Lieberman [1995], Karloff [1991], Minoux [1986], Murty [1983], Nash y Sofer [1996], Padberg [1995], Panik [1996], Saigal [1995], Sierksma [1996], Simonnard [1972] y [1973] y Van de Panne [1976].

Ejercicios

- 6.1. Determinar, partiendo del punto $\mathbf{x} = [1, 1, 1, 1]^T$, algún punto extremo del politopo $P = \{\mathbf{x} \in \mathbb{R}^4 : A\mathbf{x} \geq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, donde

$$A = \begin{bmatrix} 1 & 0 & 3 & 0 \\ 0 & 1 & 0 & 2 \\ 2 & 0 & 1 & 2 \\ -2 & 5 & -2 & 0 \\ 3 & 2 & 3 & 1 \\ -3 & 0 & 9 & -2 \\ 2 & 2 & 0 & 2 \end{bmatrix} \quad \text{y} \quad \mathbf{b} = \begin{bmatrix} 0,0 \\ 0,5 \\ 1,5 \\ -0,5 \\ 2,5 \\ 2,0 \\ 2,0 \end{bmatrix}.$$

¿Es ese punto extremo degenerado? ¿Por qué?

- 6.2. Determinar una solución básica factible de

$$\begin{aligned} x_1 + 2x_2 - x_3 + x_4 &= 3 \\ 2x_1 + 4x_2 + x_3 + 2x_4 &= 12 \\ x_1 + 4x_2 + 2x_3 + x_4 &= 9 \\ x_j &\geq 0 \quad \text{para } j = 1, \dots, 4. \end{aligned}$$

- 6.3. Considérese el sistema lineal de desigualdades $A\mathbf{x} \geq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, con $\mathbf{b} \geq \mathbf{0}$. Para transformarlo a forma estándar se puede introducir el vector de variables de holgura $\mathbf{y}$ de tal forma que $A\mathbf{x} - \mathbf{y} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, $\mathbf{y} \geq \mathbf{0}$. Hacer $b_k = \max_i b_i$ y considérese el nuevo sistema en forma estándar que se obtiene añadiendo la fila k a cada una de las demás filas con signo opuesto. Probar que el sistema así obtenido sólo requiere la adición de una variable de holgura para obtener una solución básica factible.

6.4. Determinar la solución general del siguiente sistema de ecuaciones:

$$\begin{aligned}x_1 + 2x_2 + x_3 &= 3 \\ -x_1 + 5x_2 + x_3 &= 6.\end{aligned}$$

6.5. Determinar todas las soluciones básicas del siguiente sistema de ecuaciones:

$$\begin{aligned}-x_1 + x_2 + x_3 + x_4 - 2x_5 &= 4 \\ x_1 - 2x_2 + x_4 - x_5 &= 3.\end{aligned}$$

6.6. Determinar si el siguiente sistema de ecuaciones lineales,

$$\begin{aligned}x_1 + 3x_2 + x_3 - x_4 &= 1 \\ 5x_2 - 6x_3 + x_4 &= 0 \\ x_1 - 2x_2 + 4x_3 &= 1:\end{aligned}$$

- a) Tiene solución.
- b) No tiene solución.
- c) Tiene muchas soluciones. ¿Cuántas en este caso?

6.7. Sea $\mathbf{v} \in \mathbb{R}^n$ un vector y $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, la traslación

$$\mathbf{f}(\mathbf{x}) = \mathbf{x} + \mathbf{v}.$$

Probar que si C es un conjunto convexo de $\mathbb{R}^n$, $\mathbf{f}(C)$ es también convexo.

6.8. ¿Cuál de los siguientes conjuntos es convexo y cuál no?

- a) $\{[x_1, x_2]^T \in \mathbb{R}^2 : x_1^2 + x_2^2 \leq 1\}$.
- b) $\{[x_1, x_2]^T \in \mathbb{R}^2 : x_1 + x_2 \leq 1, x_1 - x_2 \leq 2\}$.
- c) $\{[x_1, x_2]^T \in \mathbb{R}^2 : x_2 - x_1^2 = 0\}$.
- d) $\{[x_1, x_2, x_3]^T \in \mathbb{R}^3 : x_2 \geq x_1^2, x_1 + x_2 + x_3 \leq 6\}$.
- e) $\{[x_1, x_2]^T \in \mathbb{R}^2 : x_1 = 1, |x_2| \leq 4\}$.
- f) $\{[x_1, x_2, x_3]^T \in \mathbb{R}^3 : x_3 = |x_2|, x_1 \leq 4\}$.

6.9. Dibujar la región factible definida por las desigualdades

$$\begin{aligned}4x_1 - 3x_2 &\geq -15 \\ x_1 &\geq -3 \\ x_1 + x_2 &\geq -4 \\ -x_1 - 3x_2 &\geq 9 \\ x_1 - 3x_2 &\geq 6.\end{aligned}$$

Determinar gráficamente el mínimo de las siguientes funciones objetivo en esa región:

- a) $x_1 + 3x_2$.
- b) $x_1 + x_2$.
- c) $4x_1 - 3x_2$.
- d) $-x_1 + x_2$.

6.10. Sea A una matriz $m \times n$ distinta de cero y P el cono

$$P = \{\mathbf{w} \in \mathbb{R}^n : \mathbf{w} = A^T \mathbf{x}, \mathbf{x} \geq \mathbf{0}\}.$$

- a) Probar que P es un conjunto convexo.
b) ¿Es P un subespacio de $\mathbb{R}^n$? Explicar por qué.
- 6.11. Dibujar la envoltura convexa de los siguientes puntos:
a) $[1, 2]^T, [1, -1]^T, [1, 3]^T, [-11, 1]^T$.
b) $[-1, 2]^T, [2, 3]^T, [-1, -1]^T, [11, 0]^T$.
- 6.12. Determinar gráficamente la solución de los programas de programación lineal listados a continuación. Dibujar la región factible e indicar el comportamiento de la función objetivo en esa región. Probar gráficamente que se cumplen las condiciones de óptimo en los puntos considerados y no en los demás.
- a) minimizar $2x_1 - 5x_2$
s. a
$$\begin{aligned}x_1 + 3x_2 &\leq 10 \\2x_1 - 3x_2 &\leq 0 \\-x_1 + x_2 &\leq 3 \\-x_1 + 2x_2 &\leq 1.\end{aligned}$$
- b) minimizar $4x_1 - 5x_2$
s. a
$$\begin{aligned}x_1 + 2x_2 &\leq 3 \\x_1 - x_2 &\leq 2 \\2x_1 + x_2 &\leq 3 \\3x_1 + 4x_2 &\leq 8 \\-x_1 &\leq 0.\end{aligned}$$
- c) minimizar $23x_1 - 7x_2$
s. a
$$\begin{aligned}-4x_1 + x_2 &\leq -2 \\x_1 + x_2 &\leq 5 \\-x_1 - x_2 &\leq -1 \\-3x_1 + 2x_2 &\leq 1 \\3x_1 &\leq 0 \\-x_2 &\leq 0.\end{aligned}$$
- d) minimizar $-9x_1 - x_2$
s. a
$$\begin{aligned}6x_1 + 5x_2 &\leq 10 \\3x_1 - 2x_2 &\leq 8 \\2x_1 + x_2 &\leq 3 \\2x_1 + x_2 &\leq 3 \\-x_1 + 4x_2 &\leq -4.\end{aligned}$$
- e) minimizar $-x_1 + x_2$
s. a
$$\begin{aligned}4x_1 - 2x_2 &\leq 6 \\x_1 - 2x_2 &\leq 4 \\-x_1 + x_2 &\leq 1 \\x_1 &\leq 0 \\x_2 &\leq 0.\end{aligned}$$
- f) minimizar $x_1 - x_2$
s. a
$$\begin{aligned}3x_1 + x_2 &\geq 3 \\x_1 + 2x_2 &\geq 4 \\x_1 - x_2 &\leq 1 \\x_1 &\leq 5 \\x_2 &\leq 5.\end{aligned}$$

6.13. Determinar todas las soluciones básicas factibles del siguiente sistema de desigualdades:

$$\begin{aligned}x_1 + x_2 + x_3 &\leq 5 \\ -x_1 + x_2 + 2x_3 &\leq 6 \\ x_j &\geq 0 \quad \text{para } j = 1, 2, 3.\end{aligned}$$

6.14. ¿Tiene el siguiente politopo alguna dirección? ¿Por qué?

$$\begin{aligned}-x_1 + x_2 &= 5 \\ x_1 + x_2 + x_3 &\leq 6 \\ x_3 &\geq 1 \\ x_j &\geq 0 \quad \text{para } j = 1, 2, 3.\end{aligned}$$

6.15. Considérese el siguiente problema de programación lineal:

$$\begin{aligned}\text{maximizar} \quad & x_1 + 3x_2 \\ \text{s. a} \quad & -x_1 + x_2 \leq 4 \\ & -x_1 + 2x_2 \leq 12 \\ & x_1 + x_2 \leq 10 \\ & x_1, x_2 \geq 0.\end{aligned}$$

- Dibujar la región factible e identificar qué punto es el óptimo.
- Identificar todos los puntos extremos y reformular el problema como el de hallar la combinación convexa óptima de esos puntos extremos. Resolverlo.
- Supóngase que se elimina la tercera condición. Identificar en este caso todos los puntos extremos de la región factible resultante y reformular el problema otra vez como el de hallar la combinación convexa óptima de esos puntos extremos. Resolver este último problema, identificar la solución óptima del problema original e interpretarla.
- ¿Resulta útil esta forma de actuar? ¿Por qué?

6.16. Considérese las siguientes condiciones de un problema de programación lineal:

$$\begin{aligned}x_1 + 2x_2 &\leq 6 \\ x_1 - x_2 &\leq 4 \\ 2x_2 &\leq 2 \\ x_1, x_2 &\geq 0.\end{aligned}$$

- Dibujar la región factible.
- Identificar los puntos extremos y, en éstos, las variables básicas y las no básicas.
- Supóngase que se parte en la región factible del punto $[4, 0]^T$ moviéndose a $[14/3, 2/3]^T$. Especificar qué variable entra en la base y cuál sale.

Capítulo 7

EL MÉTODO SIMPLEX

COMO SE HA EXPUESTO en el capítulo 6, para resolver un problema de programación lineal,

$$\begin{array}{ll}\min. & \mathbf{c}^T \mathbf{x} \\ \text{s. a} & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0},\end{array}$$

se pueden estudiar los puntos extremos del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, donde $A \in \mathbb{R}^{m \times n}$ y $\mathbf{b} \in \mathbb{R}^m$, y buscar aquel en el que la función objetivo $\mathbf{c}^T \mathbf{x}$ se hace mínima. Analizar todos los puntos extremos, no obstante, dado el número posible de éstos para m y n grandes, puede resultar prohibitivo.

Para dar respuesta a este problema, George B. Dantzig, en 1947, desarrolló el denominado método *simplex*. La idea en la que se fundamenta es sencilla: primero, encontrar un punto extremo del politopo P o solución básica factible; a continuación, desplazarse desde ese punto extremo a otro, a lo largo de alguna arista de P , de tal forma que se mejore (haga menor) la función objetivo. Esta última operación se repite cuantas veces sea necesario hasta que se alcance la solución óptima o la arista escogida lleve a $-\infty$.

En este capítulo desarrollaremos la forma algebraica y el algoritmo numérico del método *simplex*. Para poder seguir la exposición es aconsejable tener en cuenta las consideraciones geométricas del capítulo anterior.

Comenzaremos la descripción del método por la que se denomina *fase II* (*phase II*): se parte de una solución básica factible y se trata de mejorar ésta. Como veremos posteriormente, la forma de operar en esta fase será también la que, en la denominada *fase I*, permitirá determinar la primera solución básica factible con la que comenzará todo el procedimiento.

7.1 Mejora de una solución básica factible

Supondremos que la matriz $A \in \mathbb{R}^{m \times n}$ ($m < n$) es de rango completo y que la región factible no es el conjunto vacío.

Si, sin pérdida de generalidad, suponemos que en una solución básica factible los m primeros componentes de $\mathbf{x}$ son no negativos —básicos—, se tendrá que

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_B \\ \mathbf{x}_N \end{bmatrix} = \begin{bmatrix} B^{-1}\mathbf{b} \\ \mathbf{0} \end{bmatrix},$$

donde A se ha ordenado de la forma $A = [B, N]$ y, de la misma manera, $\mathbf{c}^T = [\mathbf{c}_B^T, \mathbf{c}_N^T]$. El valor de la función objetivo correspondiente a esta solución básica factible es

$$z = [\mathbf{c}_B^T, \mathbf{c}_N^T] \begin{bmatrix} B^{-1}\mathbf{b} \\ \mathbf{0} \end{bmatrix} = \mathbf{c}_B^T B^{-1}\mathbf{b}. \quad (7.1)$$

Recordemos que si una solución básica factible no es degenerada, el punto que define está en la intersección en $\mathbb{R}^n$ de los m hiperplanos correspondientes a las condiciones $A\mathbf{x} = \mathbf{b}$ y los $n - m$ correspondientes a $\mathbf{x}_N = \mathbf{0}$. Si consideramos la matriz

$$M = \begin{bmatrix} B & N \\ \mathbf{0} & I \end{bmatrix}, \quad (7.2)$$

cuyos vectores fila son los vectores característicos de los n hiperplanos que determinan la solución básica, como la matriz B es regular, M también es regular.

En el punto extremo del politopo P que define una solución básica factible no degenerada confluyen $n - m$ aristas. Las direcciones de estas aristas son las que determinan las $n - m$ últimas columnas de la inversa de la matriz M . Esta inversa es

$$M^{-1} = \begin{bmatrix} B^{-1} & -B^{-1}N \\ \mathbf{0} & I \end{bmatrix}. \quad (7.3)$$

Moverse a lo largo de cada una de esas $n - m$ aristas, equivale a incrementar el valor de una variable no básica manteniendo las demás fijas a cero. Para comprobar estos dos últimos asertos basta observar que la columna q , $q > m$, de M^{-1} , es ortogonal a todas las filas de M que no son la q y, por consiguiente, es ortogonal a todos los vectores característicos de los hiperplanos que se cruzan en $\mathbf{x}$ excepto el correspondiente a $x_q = 0$. Esto quiere decir que el vector $\boldsymbol{\eta}_q = M^{-1}\mathbf{e}_q^1$ es paralelo a la intersección de los $n - 1$ hiperplanos linealmente independientes correspondientes a $A\mathbf{x} = \mathbf{b}$ y $x_k = 0$, $k > m$, $k \neq q$. El moverse a lo largo de $\boldsymbol{\eta}_q$ permitirá determinar puntos factibles pues, para un $\theta > 0$ suficientemente pequeño, los puntos de la forma

$$\mathbf{x}(\theta) = \mathbf{x} + \theta\boldsymbol{\eta}_q \quad (7.4)$$

son factibles. De hecho, $x_k(\theta) = 0$ para $k > m$, $k \neq q$, $x_q(\theta) = \theta > 0$ y

$$\mathbf{x}_B(\theta) = \mathbf{x}_B - \theta B^{-1}\mathbf{a}_q \geq \mathbf{0}, \quad (7.5)$$

¹Recordemos que $\mathbf{e}_q$ es el vector columna q -ésimo de la matriz unidad.

para un θ suficientemente pequeño, donde $\mathbf{a}_q$ es el vector columna q de A . Esta última expresión se obtiene de la estructuración de las condiciones en la forma $B\mathbf{x}_B + N\mathbf{x}_N = \mathbf{b}$, de donde resulta, despejando $\mathbf{x}_B$, que

$$\mathbf{x}_B = \underbrace{B^{-1}\mathbf{b}}_{\mathbf{x}_B} - B^{-1}N\mathbf{x}_N.$$

La función objetivo será pues $z = \mathbf{c}_B^T B^{-1}\mathbf{b} + (\mathbf{c}_N^T - \mathbf{c}_B^T B^{-1}N)\mathbf{x}_N$. Cuando $\mathbf{x}_N = \mathbf{0}$ se obtiene la expresión (7.1) antes expuesta.

Para mejorar una función objetivo desde un punto extremo, lo que interesa es encontrar de esas posibles $n - m$ aristas por las que moverse, una que consiga ese fin: es decir, una dirección de descenso respecto a la función objetivo. Para determinar direcciones de descenso se calculan los denominados *costes reducidos*:

$$\bar{c}_j = \mathbf{c}^T \boldsymbol{\eta}_j = \mathbf{c}^T M^{-1} \mathbf{e}_j = c_j - \mathbf{c}_B^T B^{-1} \mathbf{a}_j, \quad j > m.$$

Si $\bar{c}_j < 0$, el vector $\mathbf{c}$ y el $\boldsymbol{\eta}_j$ hacen ángulo obtuso ($> 90^\circ$) por lo que la función objetivo, al incrementar θ , decrece al moverse a lo largo de $\boldsymbol{\eta}_j$.

El coste reducido $\bar{c}_j$ es la derivada direccional de $z = \mathbf{c}^T \mathbf{x}$ en la dirección $\boldsymbol{\eta}_j$.

El concepto coste reducido surge del hecho de que $\bar{c}_j$ expresa el cambio que supone en la función objetivo un incremento unitario en la variable no básica x_j manteniendo todas las demás no básicas fijas. Es decir,

$$z(\theta) = \mathbf{c}^T \mathbf{x}(\theta) = \mathbf{c}^T \mathbf{x} + \theta \mathbf{c}^T \boldsymbol{\eta}_j = \mathbf{c}_B^T B^{-1} \mathbf{b} + \theta (c_j - \mathbf{c}_B^T B^{-1} \mathbf{a}_j).$$

Para mejorar la función objetivo se escoge como variable no básica a incrementar aquella cuyo coste reducido sea más negativo, es decir, la que potencialmente decremente más la función objetivo.

La dirección que se elige con este criterio no es la de máxima pendiente —recordemos el apartado 2.5.1.3—. Ésta sería aquella $\boldsymbol{\eta}_q$ tal que

$$\frac{\mathbf{c}^T \boldsymbol{\eta}_q}{\|\boldsymbol{\eta}_q\|_2} = \min_{j > m} \left\{ \frac{\mathbf{c}^T \boldsymbol{\eta}_j}{\|\boldsymbol{\eta}_j\|_2} \right\},$$

es decir, aquella de las $\boldsymbol{\eta}_j$ que formase el ángulo más obtuso, ϕ_q , con respecto al vector $\mathbf{c}$. Ese ángulo sería

$$\phi_q = \arccos \left(\frac{\mathbf{c}^T \boldsymbol{\eta}_q}{\|\mathbf{c}\|_2 \cdot \|\boldsymbol{\eta}_q\|_2} \right).$$

Para una solución básica degenerada, como el valor de alguna variable básica es cero, puede ocurrir que en alguna dirección cualquier desplazamiento θ haga que (7.4) viole la factibilidad del problema. Esto ocurre pues, como se ha estudiado, en una solución básica degenerada confluyen más de n hiperplanos: hay una redundancia de condiciones. La figura 7.1 ilustra esta situación. Si $\mathbf{x}$ es la solución que se está analizando, $\mathbf{d}_1$ es una dirección de movimiento factible; $\mathbf{d}_2$ no.

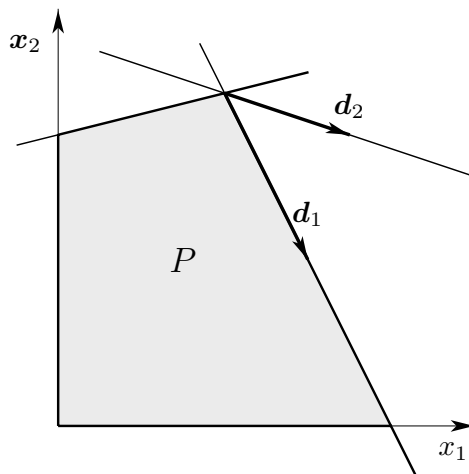


Figura 7.1

Solución básica degenerada y dirección no factible

Ejemplo 7.1 Para aclarar las ideas introducidas, consideremos el siguiente programa lineal:

$$\begin{aligned} \min. \quad & x_1 + x_2 \\ \text{s. a} \quad & x_1 + 2x_2 \leq 4 \\ & x_2 \leq 1 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Añadamos las variables de holgura x_3 y x_4 para transformar el problema en forma estándar:

$$\begin{aligned} \min. \quad & x_1 + x_2 \\ \text{s. a} \quad & x_1 + 2x_2 + x_3 = 4 \\ & x_2 + x_4 = 1 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

La matriz A es

$$A = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}.$$

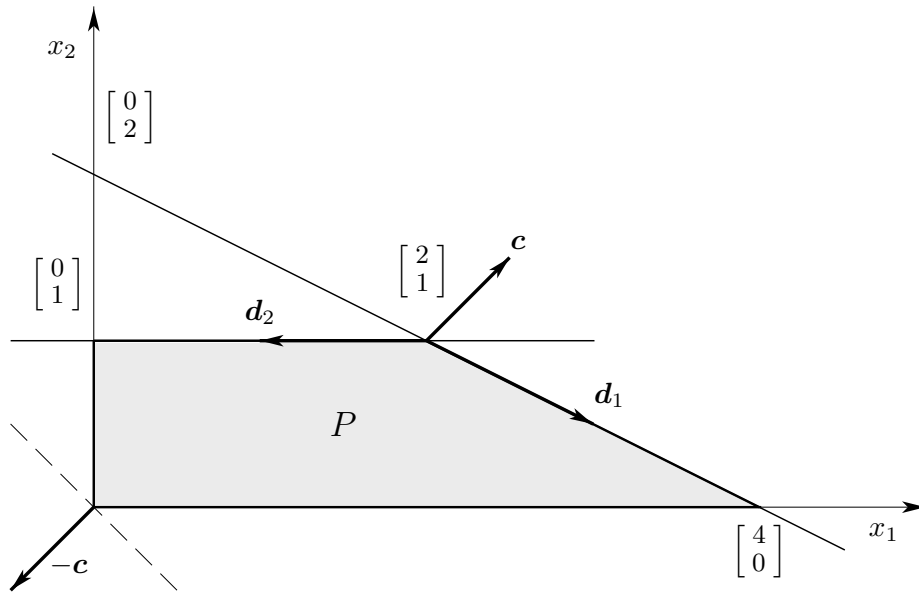
Consideremos $B = [\mathbf{a}_1, \mathbf{a}_2]$:

$$\begin{aligned} \mathbf{x}_B = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} &= B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}; \\ \mathbf{x}_N = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \end{aligned}$$

La solución básica factible que estamos considerando se puede ver en la figura 7.2. Los costes reducidos de las variables no básicas son

$$\bar{c}_3 = c_3 - \mathbf{c}_B^T B^{-1} \mathbf{a}_3 = 0 - [1, 1] \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = -1$$

y

**Figura 7.2**

Proceso de mejora de una solución básica factible del problema del ejemplo 7.1

$$\bar{c}_4 = c_3 - \mathbf{c}_B^T B^{-1} \mathbf{a}_4 = 0 - [1, 1] \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1.$$

La función objetivo decrece siguiendo la dirección $\boldsymbol{\eta}_3$ ($\mathbf{d}_2$ en la figura): incrementando x_3 . La dirección $\boldsymbol{\eta}_3$ es

$$M^{-1} \mathbf{e}_3 = \begin{bmatrix} 1 & -2 & -1 & 2 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \\ 0 \end{bmatrix}.$$

Las variables básicas se deben mover de la forma

$$\mathbf{x}_B = B^{-1} \mathbf{b} - \theta B^{-1} \mathbf{a}_3;$$

es decir, en la dirección $\mathbf{d}_2 = -B^{-1} \mathbf{a}_3 = -\begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$. •

Obsérvese que los costes reducidos de las variables básicas son cero y que los de las variables no básicas se pueden calcular obteniendo en primer lugar el vector denominado de *multiplicadores simplex*, $\boldsymbol{\pi}^T = \mathbf{c}_B^T B^{-1}$, y luego *asignar precio* (*price out*) a las columnas no básicas; es decir, calcular

$$\bar{c}_j = c_j - \boldsymbol{\pi}^T \mathbf{a}_j, \quad j > m.$$

Toda la terminología usada se deriva de la interpretación de los componentes del vector $\boldsymbol{\pi}$ como *multiplicadores de Lagrange* y como precios o valores de equilibrio en el óptimo.

Una vez elegida la variable no básica x_q que determina la dirección $\boldsymbol{\eta}_q$ a lo largo de la cual se mejora la función objetivo, la siguiente cuestión a resolver en el método *simplex* es: ¿cuánto

moverse en esa dirección? La respuesta vendrá condicionada por aquella variable básica que antes llegue a su límite, es decir, a cero, y por lo tanto bloquee el desplazamiento en la dirección escogida.

Si se hace

$$\mathbf{y} = B^{-1}\mathbf{a}_q$$

y en la dirección $\boldsymbol{\eta}_q$ se avanza una cantidad θ (en alguna referencia bibliográfica a esta cantidad se la denomina *amplitud de paso*), de las expresiones (7.4) y (7.5) se deduce que se mantendrá la factibilidad del problema, $\mathbf{x}(\theta) \geq \mathbf{0}$, si y sólo si $\mathbf{x}_B - \theta\mathbf{y} \geq \mathbf{0}$ y $\theta \geq 0$. A tenor de esto, se puede establecer el siguiente resultado.

Teorema 7.1 Si $\bar{c}_q$ es negativo y el vector $\mathbf{y} = B^{-1}\mathbf{a}_q$ es no positivo, el programa lineal

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

no está acotado: $\mathbf{x}(\theta)$ es factible para todo $\theta \geq 0$ y $z(\mathbf{x}(\theta)) \rightarrow -\infty$ cuando $\theta \rightarrow \infty$. En este caso $\mathbf{d} = \boldsymbol{\eta}_q$ es una dirección con $\mathbf{c}^T \mathbf{d} = \bar{c}_q < 0$.

Si $\mathbf{y}$ tiene positivo alguno de sus componentes, el paso θ más amplio que se puede dar en la dirección $\boldsymbol{\eta}_q$, manteniendo la factibilidad del problema, será el que determine el valor máximo en que se puedan modificar las variables básicas hasta que una de ellas llegue a cero; es decir, si las variables básicas se modifican según

$$\mathbf{x}_B = B^{-1}\mathbf{b} - \theta\mathbf{y},$$

o, desarrollando,

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \\ \vdots \\ x_{B_m} \end{bmatrix} = \begin{bmatrix} \hat{b}_1 \\ \hat{b}_2 \\ \vdots \\ \hat{b}_m \end{bmatrix} - \theta \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix},$$

donde $\hat{\mathbf{b}} = B^{-1}\mathbf{b}$, ese valor máximo de movimiento, observando esta última expresión, vendrá dado por

$$\theta = \bar{x}_q = \min \left\{ \frac{\hat{b}_i}{y_i} : y_i > 0, 1 \leq i \leq m \right\} = \frac{x_{B_p}}{y_p}.$$

El valor de la variable no básica x_q se verá incrementado después de este paso desde cero a $\bar{x}_q$, pasando a ser básica en detrimento de la variable x_{B_p} que pasará a ser no básica.

Para completar una iteración del método *simplex* lo que resta por hacer es reconfigurar la base y, por lo que se refiere a la matriz B , reemplazar el vector $\mathbf{a}_p$ por el $\mathbf{a}_q$, es decir, hacer

$$\bar{B} = B + (\mathbf{a}_q - \mathbf{a}_p)\mathbf{e}_p^T.$$

quedando

$$\bar{B} = [\mathbf{a}_{B_1}, \mathbf{a}_{B_2}, \dots, \mathbf{a}_{B_{p-1}}, \mathbf{a}_q, \mathbf{a}_{B_{p+1}}, \dots, \mathbf{a}_{B_m}].$$

Las variables básicas se modificarán de acuerdo con la siguiente expresión

$$\bar{x}_{B_i} = x_{B_i} - \theta y_i, \quad i = 1, \dots, m.$$

7.2 Finalización. Solución óptima, solución no acotada y soluciones óptimas alternativas

Hemos visto en el apartado anterior cómo mejorar una solución básica factible moviéndose a lo largo de la dirección, η_j , que define una de las aristas que confluyen en el punto extremo que define esa solución.

A continuación probaremos que todo punto $\mathbf{y} \in P$ se encuentra dentro del politopo cónico generado por una solución básica factible $\mathbf{x}$ y las direcciones η_j que parten de ese $\mathbf{x}$. En el caso de soluciones básicas no degeneradas, las direcciones η_j son factibles; en el caso de soluciones degeneradas, algunas de las η_j pueden ser no factibles.

Lema 7.1 *Dada una solución básica factible cualquiera, $\mathbf{x}'$, del programa lineal*

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

todo punto $\mathbf{y} \in P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ se puede expresar de la forma

$$\mathbf{y} = \mathbf{x}' + \sum_{j=m+1}^n y_j \eta_j, \quad y_j \geq 0,$$

donde η_j es la columna j -ésima de la matriz M^{-1} .

DEMOSTRACIÓN. Como $\mathbf{y} \in P$, $A\mathbf{y} = \mathbf{b}$ y $\mathbf{y}^T = [\mathbf{y}_B^T, \mathbf{y}_N^T] \geq \mathbf{0}$. Como además $A\mathbf{x}' = \mathbf{b}$ y $\mathbf{x}'_N = \mathbf{0}$, se tiene que

$$M(\mathbf{y} - \mathbf{x}') = \begin{bmatrix} B & N \\ 0 & I \end{bmatrix} (\mathbf{y} - \mathbf{x}') = \begin{bmatrix} \mathbf{0} \\ \mathbf{y}_N \end{bmatrix}.$$

Despejando

$$\mathbf{y} - \mathbf{x}' = M^{-1} \begin{bmatrix} \mathbf{0} \\ \mathbf{y}_N \end{bmatrix} = \begin{bmatrix} B^{-1}N \\ I \end{bmatrix} \mathbf{y}_N,$$

donde $\mathbf{y}_N \geq \mathbf{0}$. ■

De este lema se tiene que

$$z(\mathbf{y}) - z(\mathbf{x}) = \mathbf{c}^T(\mathbf{y} - \mathbf{x}) = \sum_{j=m+1}^n (\mathbf{c}^T \eta_j) y_j = \sum_{j=m+1}^n \bar{c}_j y_j,$$

para todo $\mathbf{y} \in P$. Como el vector $\mathbf{y}$ es no negativo, si los costes reducidos $\bar{c}_j$ son no negativos, la función objetivo $\mathbf{c}^T \mathbf{y} \geq \mathbf{c}^T \mathbf{x}$ para todo $\mathbf{y} \in P$. Hemos deducido de esta forma el siguiente resultado.

Teorema 7.2 *Una solución básica factible es una solución óptima del programa lineal*

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

si todos los costes reducidos (relativos a la base dada) son no negativos.

En el caso no degenerado, el recíproco de este teorema también es cierto. En el caso degenerado, sin embargo, una solución básica factible puede ser óptima aun en el caso de que algunos de los costes reducidos sean negativos, pues las direcciones de movimiento asociadas a esas variables no básicas pueden no ser factibles: en un punto $\mathbf{x}$ donde $x_j = 0$, si el componente j de esa dirección es negativo.

Corolario 7.1 *Una solución básica factible $\mathbf{x}$ es el único óptimo del programa lineal*

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

si los costes reducidos de las variables no básicas son estrictamente positivos.

Corolario 7.2 *(Soluciones óptimas alternativas) Si $\mathbf{x}$ es una solución básica factible óptima y los costes reducidos de las variables no básicas $\bar{c}_1 = \bar{c}_2 = \dots = \bar{c}_k = 0$, cualquier punto $\mathbf{y} \in P$ de la forma*

$$\mathbf{y} = \mathbf{x} + \sum_{i=1}^k y_i \boldsymbol{\eta}_i,$$

es también óptimo.

Si una solución básica factible es óptima y degenerada, y los costes reducidos correspondientes a algunas de las variables no básicas son cero, del último corolario no se deduce que la solución óptima sea múltiple. Esto es así pues en el caso degenerado $\mathbf{x}$ puede ser el único punto en P expresable en la forma enunciada en el corolario, por ser todas las direcciones $\boldsymbol{\eta}_i$ no factibles.

7.3 El algoritmo simplex

El algoritmo *simplex* para resolver

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

se describe en la tabla 7.1. Comienza su actuación desde una solución básica factible $\mathbf{x}_B$ correspondiente a $B = [\mathbf{a}_{j_1}, \mathbf{a}_{j_2}, \dots, \mathbf{a}_{j_m}]$. El conjunto $\mathbf{B} = \{j_1, \dots, j_m\}$ es el de índices de las variables básicas; x_{j_i} designa la variable básica i -ésima y $\mathbf{N}$ el conjunto de índices de las variables no básicas.

La forma en que se presenta el algoritmo *simplex* en la tabla es la que se conoce habitualmente en la literatura como *simplex revisado*.

Ejemplo 7.2 Resolver el problema de programación lineal:

$$\begin{aligned} \min. \quad & -x_1 - 3x_2 \\ \text{s. a} \quad & 2x_1 + 3x_2 \leq 6 \\ & -x_1 + x_2 \leq 1 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Para empezar, introduzcamos las variables de holgura x_3 y x_4 . Resulta:

$$\begin{aligned} \min. \quad & -x_1 - 3x_2 \\ \text{s. a} \quad & 2x_1 + 3x_2 + x_3 = 6 \\ & -x_1 + x_2 + x_4 = 1 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

Escojamos como base de partida

$$B = [\mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_1, \mathbf{a}_2] = \begin{bmatrix} 2 & 3 \\ -1 & 1 \end{bmatrix}.$$

La solución inicial, $\mathbf{x}_B = B^{-1}\mathbf{b}$, $\mathbf{x}_N = \mathbf{0}$, es pues

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 6 \\ 1 \end{bmatrix}.$$

La función objetivo para esta solución es $z = \mathbf{c}_B^T B^{-1}\mathbf{b} = [0, 0] \begin{bmatrix} 6 \\ 1 \end{bmatrix} = 0$.

Iteración 1. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -1 - [0, 0] \begin{bmatrix} 2 \\ -1 \end{bmatrix} = -1$$

y

Tabla 7.1El algoritmo *simplex* revisado (comienza a partir de una solución factible)**Paso 1** – *Asignación de precios. Comprobación de óptimo.*

- (a) Calcular los multiplicadores simplex resolviendo el sistema $B^T \boldsymbol{\pi} = \mathbf{c}_B$.
- (b) Calcular los costes reducidos de las variables no básicas, $\bar{c}_j = c_j - \boldsymbol{\pi}^T \mathbf{a}_j$, para todo $j \in \mathbf{N}$.
- (c) Comprobar que se ha llegado al óptimo: si $\bar{c}_j \geq 0$ para todo $j \in \mathbf{N}$, PARAR: la solución es óptima.

Paso 2 – *Determinar columna de pivotación.*

- (a) Determinar la variable no básica x_q que debe entrar en la base (encontrar una dirección de descenso): escoger $q \in \mathbf{N}$ tal que $\bar{c}_q = \min_{j \in \mathbf{N}} \{\bar{c}_j < 0\}$.
- (b) Resolver

$$B\mathbf{y} = \mathbf{a}_q.$$

Si $\mathbf{y} \leq \mathbf{0}$, PARAR; hay un rayo factible de soluciones a lo largo del cual $\mathbf{c}^T \mathbf{x} \rightarrow -\infty$.
Si no, seguir.

Paso 3 – *Determinar fila de pivotación. Análisis de Ratios.*Determinar la variable básica x_{j_p} que sale de la base: calcular

$$\theta = \frac{x_{j_p}}{y_p} = \min_{1 \leq i \leq m} \left\{ \frac{x_{j_i}}{y_i} : y_i > 0 \right\}.$$

Paso 4 – *Pivotación.*Adaptar la solución, la matriz B y las estructuras de datos. Hacer:

$$\begin{aligned} x_q &\leftarrow \theta = x_{j_p}/y_p \\ x_{j_i} &\leftarrow x_{j_i} - \theta y_i, \quad 1 \leq i \leq m \\ B &\leftarrow B + (\mathbf{a}_q - \mathbf{a}_{j_p}) \mathbf{e}_p^T \\ \mathbf{B} &\leftarrow \mathbf{B} \cup \{q\} \setminus \{j_p\} \\ \mathbf{N} &\leftarrow \mathbf{N} \cup \{j_p\} \setminus \{q\}. \end{aligned}$$

Ir al paso 1.

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = -3 - [0, 0] \begin{bmatrix} 3 \\ 1 \end{bmatrix} = -3.$$

Todavía no se ha llegado al óptimo pues los costes reducidos de las variables no básicas son negativos.

Iteración 1. Paso 2

Elijamos la variable no básica x_2 como aquella que ha de entrar en la base pues tiene el coste reducido más negativo.

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}_2$:

$$\mathbf{y} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Iteración 1. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_2}}{y_2} \right\} = \min \left\{ \frac{6}{3}, \frac{1}{1} \right\} = 1.$$

Es decir, saldrá $x_{B_2} = x_4$. Esto se puede deducir también analizando la expresión

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \end{bmatrix} = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = B^{-1}\mathbf{b} - \theta\mathbf{y} = \begin{bmatrix} 6 \\ 1 \end{bmatrix} - \theta \begin{bmatrix} 3 \\ 1 \end{bmatrix},$$

comprobándose que x_4 es la primera variable que llega a cero al incrementar x_2 .

Iteración 1. Paso 4

Readaptemos la solución y las estructuras correspondientes:

$$\begin{aligned} x_2 &\leftarrow \theta = 1 \\ x_3 &\leftarrow x_3 - \theta y_1 = 6 - 1 \cdot 3 = 3 \\ x_4 &\leftarrow x_4 - \theta y_2 = 1 - 1 \cdot 1 = 0. \end{aligned}$$

La solución queda, por tanto,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 0 \end{bmatrix}$$

con

$$B = [\mathbf{a}_3, \mathbf{a}_2] = \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_1, \mathbf{a}_4] = \begin{bmatrix} 2 & 0 \\ -1 & 1 \end{bmatrix}.$$

La nueva función objetivo es

$$z = \mathbf{c}_B^T B^{-1} \mathbf{b} = [0, -3] \begin{bmatrix} 1 & -3 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 1 \end{bmatrix} = -3.$$

Iteración 2. Paso 1

Calculemos los nuevos multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ -3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -3 \end{bmatrix} = \begin{bmatrix} 0 \\ -3 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -1 - [0, -3] \begin{bmatrix} 2 \\ -1 \end{bmatrix} = -4$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [0, -3] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 3.$$

Como todavía existe un coste reducido negativo, no se ha llegado al óptimo.

Iteración 2. Paso 2

Elegimos la única variable no básica capaz de mejorar la función objetivo, x_1 , como aquella que ha de entrar en la base.

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}_1$:

$$\mathbf{y} = \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 & -3 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \end{bmatrix}.$$

Iteración 2. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1} \right\} = \min \left\{ \frac{3}{5} \right\} = \frac{3}{5}.$$

Es decir, saldrá $x_{B_1} = x_3$. Analizando el porqué de esto mediante la expresión

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \end{bmatrix} = \begin{bmatrix} x_3 \\ x_2 \end{bmatrix} = B^{-1} \mathbf{b} - \theta \mathbf{y} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} - \theta \begin{bmatrix} 5 \\ -1 \end{bmatrix},$$

se comprueba que x_3 es la primera variable que llega a cero al incrementar x_1 .

Iteración 2. Paso 4

Readaptemos la solución y las estructuras correspondientes:

$$\begin{aligned} x_1 &\leftarrow \theta = \frac{3}{5} \\ x_2 &\leftarrow x_2 - \theta y_2 = 1 - \frac{3}{5} \cdot (-1) = \frac{8}{5} \\ x_3 &\leftarrow x_3 - \theta y_1 = 3 - \frac{3}{5} \cdot 5 = 0. \end{aligned}$$

La solución queda:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 8/5 \\ 0 \\ 0 \end{bmatrix}$$

con

$$B = [\mathbf{a}_1, \mathbf{a}_2] = \begin{bmatrix} 2 & 3 \\ -1 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

La nueva función objetivo es

$$z = \mathbf{c}_B^T B^{-1} \mathbf{b} = [-1, -3] \begin{bmatrix} 2 & 3 \\ -1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 6 \\ 1 \end{bmatrix} = [-1, -3] \begin{bmatrix} \frac{1}{5} & -\frac{3}{5} \\ \frac{1}{5} & \frac{2}{5} \end{bmatrix} \begin{bmatrix} 6 \\ 1 \end{bmatrix} = -\frac{27}{5}.$$

Iteración 3. Paso 1

Calculemos los nuevos multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 2 & -1 \\ 3 & 1 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ 3 \end{bmatrix} = \begin{bmatrix} \frac{1}{5} & \frac{1}{5} \\ -\frac{3}{5} & \frac{2}{5} \end{bmatrix} \begin{bmatrix} -1 \\ 3 \end{bmatrix} = \begin{bmatrix} -4/5 \\ -3/5 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [-4/5, -3/5] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{4}{5}$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [-4/5, -3/5] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{3}{5}.$$

Todos los costes reducidos son positivos por lo que se ha llegado al único óptimo del problema. La solución óptima es

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 3/5 \\ 8/5 \\ 0 \\ 0 \end{bmatrix}.$$

La función objetivo

$$z = \boldsymbol{\pi}^T \mathbf{b} = [-4/5, -3/5] \begin{bmatrix} 6 \\ 1 \end{bmatrix} = -\frac{27}{5}.$$

Las soluciones básicas factibles que se han ido obteniendo en el proceso descrito se representan en la figura 7.3. ●

Ejemplo 7.3 Solución no acotada. Resolver el siguiente problema lineal:

$$\begin{aligned} \min. \quad & -x_1 - 3x_2 \\ \text{s. a} \quad & x_1 - 2x_2 \leq 4 \\ & -x_1 + x_2 \leq 3 \\ & x_1, x_2 \geq 0. \end{aligned}$$

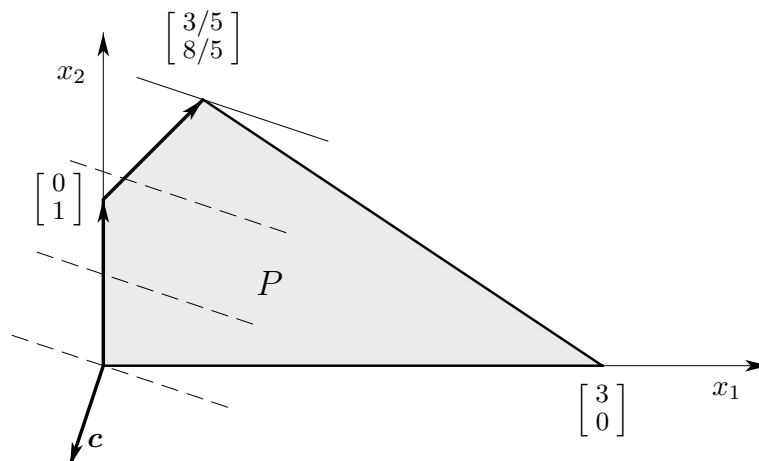


Figura 7.3

Representación del proceso seguido hasta la solución en el problema del ejemplo 7.2

Introduzcamos las variables de holgura x_3 y x_4 . El problema queda:

$$\begin{aligned} \min. \quad & -x_1 - 3x_2 \\ \text{s. a} \quad & x_1 - 2x_2 + x_3 = 4 \\ & -x_1 + x_2 + x_4 = 3 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

Escojamos como base de partida

$$B = [a_3, a_4] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [a_1, a_2] = \begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix}.$$

La solución inicial, $x_B = B^{-1}b$, $x_N = 0$, es pues

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 4 \\ 3 \end{bmatrix}.$$

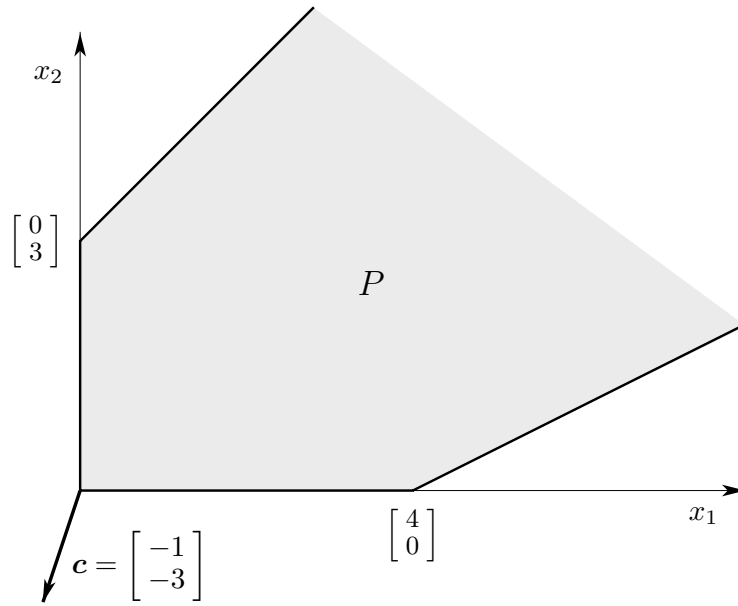
La función objetivo para esta solución es $z = [0, 0] \begin{bmatrix} 4 \\ 3 \end{bmatrix} = 0$.

La región factible de este problema es la que se ilustra en la figura 7.4.

Iteración 1. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \pi = c_B$:

$$\pi = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

**Figura 7.4**

Problema con solución no acotada del ejemplo 7.3

Los costes reducidos de las variables no básicas son

$$\bar{c}_1 = c_1 - \pi^T \mathbf{a}_1 = -1 - [0, 0] \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -1$$

y

$$\bar{c}_2 = c_2 - \pi^T \mathbf{a}_2 = -3 - [0, 0] \begin{bmatrix} -2 \\ 1 \end{bmatrix} = -3.$$

Al ser negativos los costes reducidos, esta solución no es óptima.

Iteración 1. Paso 2

Elegimos la variable no básica x_2 como aquella que ha de entrar en la base pues su coste reducido es el más negativo.

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}_2$:

$$\mathbf{y} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

Iteración 1. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_2}}{y_2} \right\} = \min \left\{ \frac{3}{1} \right\} = 3.$$

Es decir saldrá $x_{B_2} = x_4$. Como venimos haciendo, esto también se puede comprobar a partir de la expresión

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \end{bmatrix} = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = B^{-1}\mathbf{b} - \theta\mathbf{y} = \begin{bmatrix} 4 \\ 3 \end{bmatrix} - \theta \begin{bmatrix} -2 \\ 1 \end{bmatrix},$$

constatándose que, efectivamente, x_4 es la primera variable que llega a cero al incrementar x_2 .

Iteración 1. Paso 4

Readaptemos la solución y las estructuras correspondientes:

$$\begin{aligned}x_2 &\leftarrow \theta = 3 \\x_3 &\leftarrow x_3 - \theta y_1 = 4 - 3 \cdot (-2) = 10 \\x_4 &\leftarrow x_4 - \theta y_2 = 3 - 3 \cdot 1 = 0.\end{aligned}$$

La solución queda:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 10 \\ 0 \end{bmatrix}$$

con

$$B = [\mathbf{a}_3, \mathbf{a}_2] = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_1, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}.$$

Iteración 2. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 1 & 0 \\ -2 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ -3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -3 \end{bmatrix} = \begin{bmatrix} 0 \\ -3 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -1 - [0, -3] \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -4$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [0, -3] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 3.$$

Como existe un coste reducido negativo, todavía no se ha alcanzado el óptimo.

Iteración 2. Paso 2

Elegimos la única variable no básica posible, x_1 , como aquella que ha de entrar en la base.

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}_1$:

$$\mathbf{y} = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}.$$

Como todos los y_i , $i = 1, 2$ son negativos, hay un rayo de soluciones factibles a lo largo del cual $\mathbf{c}^T \mathbf{x} \rightarrow \infty$. La solución es pues no acotada. ●

Ejemplo 7.4 Soluciones óptimas alternativas. Resolver el problema lineal

$$\begin{aligned} \min. \quad & -2x_1 - 4x_2 \\ \text{s. a} \quad & x_1 + 2x_2 \leq 4 \\ & -x_1 + x_2 \leq 1 \\ & x_1, x_2 \geq 0, \end{aligned}$$

que se describe en la figura 7.5.

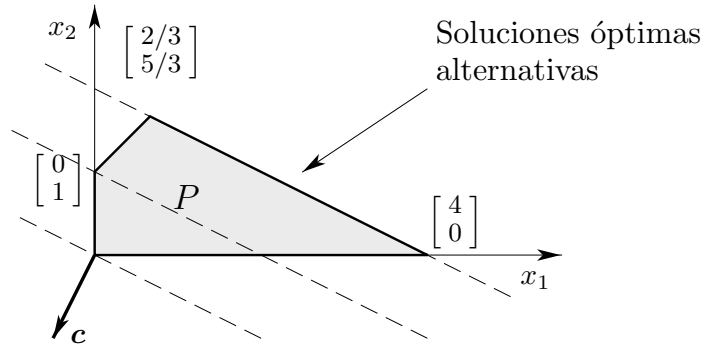


Figura 7.5

El algoritmo *simplex* resolviendo un problema con soluciones óptimas alternativas

Introduzcamos las variables de holgura x_3 y x_4 . El problema queda:

$$\begin{aligned} \min. \quad & -2x_1 - 4x_2 \\ \text{s. a} \quad & x_1 + 2x_2 + x_3 = 4 \\ & -x_1 + x_2 + x_4 = 1 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

Escojamos como base de partida

$$B = [\mathbf{a}_1, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_2, \mathbf{a}_3] = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}.$$

La solución inicial, $\mathbf{x}_B = B^{-1}\mathbf{b}$, $\mathbf{x}_N = \mathbf{0}$, es pues

$$\begin{bmatrix} x_1 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}, \quad \text{por lo que} \quad \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 0 \\ 5 \end{bmatrix}.$$

La función objetivo para esta solución es $z = [-2, 0] \begin{bmatrix} 4 \\ 1 \end{bmatrix} = -8$.

Iteración 1. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} -2 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -2 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = -4 - [-2, 0] \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 0$$

y

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [-2, 0] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2.$$

Hemos llegado a un punto óptimo. Este, sin embargo, no es único. En efecto, si incrementamos x_2 , manteniendo $x_3 = 0$, las variables x_1 y x_4 se modifican de la siguiente manera:

$$\begin{bmatrix} x_1 \\ x_4 \end{bmatrix} = B^{-1} \mathbf{b} - B^{-1} \mathbf{a}_2 x_2 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 5 \end{bmatrix} - \begin{bmatrix} 2 \\ 3 \end{bmatrix} x_2.$$

Para cualquier $x_2 < \frac{5}{3}$, la solución

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 4 - 2x_2 \\ x_2 \\ 0 \\ 5 - 3x_2 \end{bmatrix}$$

es óptima con función objetivo $z = -8$. ●

7.3.1 Degeneración y ciclado

Aunque se ha asumido que la solución de un programa lineal, $\mathbf{x}$, no es degenerada, de hecho, en la práctica, el que esto ocurra no causa grandes dificultades. Lo más que puede ocurrir es que en el paso 3 x_{j_p} sea cero, lo que dará lugar a una iteración con $\theta = 0$: $\mathbf{x}$ no cambiará. Esto ocurrirá porque a lo largo de η_q se llegará inmediatamente a la condición $x_{j_p} \geq 0$. En este caso, aunque $\mathbf{x}$ no cambie, si lo hará la base. Como $\mathbf{x}$, y por tanto $\mathbf{c}^T \mathbf{x}$, no cambian, es teóricamente posible que el algoritmo *cicle* indefinidamente a través de una sucesión de bases, y sus correspondientes soluciones.

El problema

$$\begin{array}{ll} \min. & -\frac{3}{4}x_4 + 20x_5 - \frac{1}{2}x_6 + 6x_7 \\ \text{s. a} & x_1 + \frac{1}{4}x_4 - 8x_5 - x_6 + 9x_7 = 0 \\ & x_2 + \frac{1}{2}x_4 - 12x_5 - \frac{1}{2}x_6 + 3x_7 = 0 \\ & x_3 + x_6 = 1 \\ & x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0. \end{array}$$

expuesto por E.M.L. Beale en 1955, muestra cómo el método *simplex* puede quedar atrapado en un ciclo infinito si se aplican los criterios que hemos utilizado escogiendo como variable a entrar en la base aquella que posee el coste reducido más negativo y para salir de ella una de las que proporcionan el mínimo θ .

El óptimo de este problema es $\mathbf{x} = [3/4, 0, 0, 1, 0, 1, 0]^T$ y su función objetivo óptima $-5/4$. Si se comienza su resolución partiendo de la base $B = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$, siguiendo el método *simplex*

se obtienen las bases $[\mathbf{a}_4, \mathbf{a}_2, \mathbf{a}_3]$, $[\mathbf{a}_4, \mathbf{a}_5, \mathbf{a}_3]$, $[\mathbf{a}_6, \mathbf{a}_5, \mathbf{a}_3]$, $[\mathbf{a}_6, \mathbf{a}_7, \mathbf{a}_3]$, $[\mathbf{a}_1, \mathbf{a}_7, \mathbf{a}_3]$ y otra vez $[\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$. Si se vuelve a utilizar la misma secuencia de variables a entrar y salir de la base, el método entra en un ciclo infinito.

En la práctica, evitar el ciclado no es complicado pues existen reglas sencillas para ello. Las más utilizadas son la *lexicográfica*, propuesta por G.B. Dantzig, A. Orden y P. Wolfe en 1955, y la de Bland, debida a R.G. Bland en [1977]. Se basan en el hecho de que, en ausencia de degeneración, los valores de la función objetivo que se suceden en el método *simplex* forman una sucesión estrictamente monótona decreciente, lo que garantiza que las bases no se repitan.

7.3.1.1 La regla lexicográfica

Esta regla asegura que el método *simplex* no cicla, pues a pesar de que el valor de la función objetivo, $\mathbf{c}_B^T B^{-1} \mathbf{b}$, puede permanecer constante en presencia de degeneración, el vector $[\mathbf{c}_B^T B^{-1} \mathbf{b}, \mathbf{c}_B^T B^{-1}]^T$ se puede hacer lexicográficamente monótonamente decreciente.

Esta regla primero usa la relación ya expuesta,

$$\theta = \frac{x_{j_p}}{y_p} = \min_{1 \leq i \leq m} \left\{ \frac{x_{j_i}}{y_i} : y_i > 0 \right\},$$

para determinar la variable básica x_{j_p} que ha de salir de la base. Si de aquí se deduce que sólo hay una candidata, se elige ésta. Si hay varias, se vuelven a evaluar los ratios de aquellas variables que quedan como candidatas, pero esta vez, en lugar de utilizar los x_{j_i} resultantes de $B^{-1} \mathbf{b}$, se utilizan los correspondientes elementos del vector $B^{-1} \mathbf{a}_{p_1}$, donde $\mathbf{a}_{p_1}$ es el vector columna de la matriz A que corresponde a la variable básica x_{p_1} de índice más bajo. Si haciendo esto los nuevos ratios siguen siendo iguales, se vuelven a calcular otros ratios utilizando $B^{-1} \mathbf{a}_{p_2}$, donde $\mathbf{a}_{p_2}$ es el vector columna que corresponde a la variable básica x_{p_2} con segundo índice más bajo, etc. Operando de esta manera se asegura una secuencia lexicográficamente monótonamente decreciente del vector $[\mathbf{c}_B^T B^{-1} \mathbf{b}, \mathbf{c}_B^T B^{-1}]^T$.

7.3.1.2 La regla de Bland

Mediante esta regla se elige como variable que ha de entrar en la base, de entre las de coste reducido negativo, aquella con menor índice. Para salir de la base, si el ratio es igual en varias, aquella también con menor índice.

Esta regla crea la siguiente propiedad monótona: si una variable x_q entra en la base, no la abandonará hasta que alguna otra con índice mayor, que era no básica cuando x_q entró en la base, también entre en la base. Al actuar siguiendo la regla de Bland se evita el ciclado puesto que en un ciclo cualquier variable que entre en la base debe salir de ella, lo que implica que existe alguna variable de índice mayor que entra y sale de la base. Esto, evidentemente, contradice la propiedad monótona apuntada.

7.4 Solución básica factible inicial

Hasta ahora hemos supuesto que existía una solución inicial básica factible desde la que iniciar el método *simplex*. Ahora bien, ¿cómo se llega a ella?

Si las condiciones del problema son de la forma $A\mathbf{x} \leq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, donde $A \in \mathbb{R}^{m \times n}$, y $\mathbf{b} \in \mathbb{R}^m$ tiene todos sus componentes no negativos, el problema de encontrar la solución inicial básica

factible es trivial; en efecto, añadiendo las variables de holgura a las condiciones, según hemos venido haciendo hasta ahora para convertirlas en la forma estándar, se tendrá que

$$\begin{aligned} A\mathbf{x} + \mathbf{x}_h &= \mathbf{b} \\ \mathbf{x}, \mathbf{x}_h &\geq \mathbf{0}. \end{aligned}$$

La matriz $[A, I]$ es de rango m . Una solución básica factible será $\mathbf{x}_h = \mathbf{b}$, $\mathbf{x} = \mathbf{0}$; la matriz básica correspondiente, $B = I$.

Normalmente la búsqueda de una solución inicial básica factible no es tan sencilla. Así, por ejemplo, si se diesen las mismas condiciones de antes pero el vector $\mathbf{b}$ tuviese algún componente negativo, la solución obtenida no sería válida pues violaría la condición de no negatividad que han de cumplir las variables del problema.

Otra situación en la que la forma anterior de abordar el problema no es válida tiene lugar cuando las condiciones son de la forma $A\mathbf{x} \geq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$. Añadiendo a éstas el vector de variables de holgura, $\mathbf{x}_h$, se tendría que $A\mathbf{x} - \mathbf{x}_h = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, $\mathbf{x}_h \geq \mathbf{0}$. Si el vector $\mathbf{b}$ no tuviese todos sus componentes no positivos, a priori, difícilmente se podría determinar una base B a partir de la matriz $[A, -I]$, de tal forma que $\mathbf{x}_B = B^{-1}\mathbf{b}$ cumpliera la condición de no negatividad.

Veamos algunos ejemplos concretos de lo expuesto.

Ejemplo 7.5 Consideremos las condiciones siguientes de un programa lineal cualquiera:

$$\begin{aligned} x_1 + 2x_2 &\leq 4 \\ -x_1 + x_2 &\leq 1 \\ x_1, x_2 &\geq 0. \end{aligned}$$

Añadiendo las variables de holgura x_3 y x_4 , estas condiciones se transforman en

$$\begin{aligned} x_1 + 2x_2 + x_3 &= 4 \\ -x_1 + x_2 + x_4 &= 1 \\ x_1, x_2, x_3, x_4 &\geq 0. \end{aligned}$$

Una solución inicial básica factible sería

$$\begin{aligned} B = [\mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} : \quad \mathbf{x}_B = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix} \quad y \\ \mathbf{x}_N = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \end{aligned}$$

•

Ejemplo 7.6 Sean ahora las siguientes condiciones:

$$\begin{aligned} x_1 + x_2 + x_3 &\leq 6 \\ -2x_1 + 3x_2 + 3x_3 &\geq 3 \\ x_2, x_3 &\geq 0. \end{aligned}$$

Recordemos que cuando una variable x_i no está restringida a tomar valores no negativos, como es el caso aquí de la variable x_1 , una forma de tratarla consiste en sustituirla por $x'_i - x''_i$, $x'_i \geq 0$,

$x_i'' \geq 0$. Si introducimos también las variables de holgura x_4 y x_5 , las condiciones quedan:

$$\begin{aligned} x_1' - x_1'' + x_2 + x_3 + x_4 &= 6 \\ -2x_1' + 2x_1'' + 3x_2 + 3x_3 - x_5 &= 3 \\ x_1', x_1'', x_2, x_3, x_4, x_5 &\geq 0. \end{aligned}$$

De estas condiciones no es inmediato determinar una base B que nos permita obtener una solución inicial básica factible. ●

Ejemplo 7.7 Consideremos las siguientes condiciones:

$$\begin{aligned} x_1 + x_2 - 2x_3 &\leq -3 \\ -2x_1 + x_2 + 3x_3 &\leq 7 \\ x_1, x_2, x_3 &\geq 0. \end{aligned}$$

Si en aras de hacer todos los componentes del vector $\mathbf{b}$ no negativos multiplicamos la primera condición por -1 , e introducimos las variables de holgura x_4 y x_5 , las condiciones resultarán:

$$\begin{aligned} -x_1 - x_2 + 2x_3 - x_4 &= 3 \\ -2x_1 + x_2 + 3x_3 + x_5 &= 7 \\ x_1, x_2, x_3, x_4, x_5 &\geq 0. \end{aligned}$$

Tampoco de aquí se puede determinar de una manera sencilla una base inicial del problema que configuran estas condiciones. ●

7.4.1 Variables artificiales

La forma más utilizada para evitar los problemas apuntados en el apartado anterior, y determinar de inmediato una solución básica factible de partida, consiste en introducir, temporalmente, además de las variables de holgura, un conjunto similar de variables denominadas *variables artificiales*.

La idea es muy sencilla: a las condiciones del problema en forma estándar

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{7.6}$$

donde, para simplificar y sin pérdida de generalidad, se supone que $\mathbf{b} \geq \mathbf{0}$, se le añade un vector de variables artificiales, $\mathbf{x}_a$, resultando

$$A\mathbf{x} + \mathbf{x}_a = \mathbf{b},$$

con $\mathbf{x} \geq \mathbf{0}$ y $\mathbf{x}_a \geq \mathbf{0}$.

Si se quiere conseguir una solución factible de (7.6), lógicamente, habrá que forzar a que los componentes del vector $\mathbf{x}_a$ sean cero. Es decir, resolver el problema

$$\begin{aligned} \min. \quad & \sum_{i=1}^m x_{a_i} \\ \text{s. a} \quad & A\mathbf{x} + \mathbf{x}_a = \mathbf{b} \\ & \mathbf{x}, \mathbf{x}_a \geq \mathbf{0}. \end{aligned} \tag{7.7}$$

Si el problema original (7.6) admite una solución factible, en la solución de (7.7) se tendrá que $\mathbf{x}_a = \mathbf{0}$. Si tal solución factible no existe, (7.7) acabará con algún componente de $\mathbf{x}_a$ positivo.

Para resolver este nuevo problema (7.7) se utiliza el mismo método *simplex* en lo que se ha dado en llamar en la literatura especializada la *fase I* de ese procedimiento: la búsqueda de una solución inicial básica factible.

El método *simplex* completo, por consiguiente, consta de las dos fases que se indican en la tabla 7.2.

Tabla 7.2

El método *simplex* en sus dos fases

Fase I – Partiendo de la solución inicial $\mathbf{x} = \mathbf{0}$, $\mathbf{x}_a = \mathbf{b}$, resolver

$$\begin{aligned} \min. \quad & \sum_{i=1}^m x_{a_i} \\ \text{s. a} \quad & A\mathbf{x} + \mathbf{x}_a = \mathbf{b} \\ & \mathbf{x}, \mathbf{x}_a \geq \mathbf{0}. \end{aligned}$$

Si en la solución resultante $\mathbf{x}_a \neq \mathbf{0}$, parar: el problema no tiene solución básica factible. Si $\mathbf{x}_a = \mathbf{0}$, ir a la *fase II* tomando como base inicial, B , la de la última iteración de esta fase.

Fase II – Partiendo de $\mathbf{x}_B = B^{-1}\mathbf{b}$ y $\mathbf{x}_N = \mathbf{0}$, resolver

$$\begin{aligned} \min. \quad & \mathbf{c}_B^T \mathbf{x}_B + \mathbf{c}_N^T \mathbf{x}_N \\ \text{s. a} \quad & B\mathbf{x}_B + N\mathbf{x}_N = \mathbf{b} \\ & \mathbf{x}_B, \mathbf{x}_N \geq \mathbf{0}. \end{aligned}$$

Si la solución de la *fase I* es degenerada, cualquier variable artificial $x_{a_i} = 0$ que esté en la base se puede intercambiar por una no básica x_j , o suprimirse sin más que eliminar la condición redundante correspondiente. Concretamente, si $x_{a_k} = 0$ es la variable básica k -ésima al final de la *fase I* y se cumple que $\mathbf{e}_k^T B^{-1} \mathbf{a}_j \neq 0$, la variable x_{a_k} se puede reemplazar por la x_j . Si $\mathbf{e}_k^T B^{-1} \mathbf{a}_j = 0$ para todo $\mathbf{a}_j \in N$, el sistema original $A\mathbf{x} = \mathbf{b}$ es redundante y, por tanto, se puede eliminar la fila k -ésima de la matriz A así como la fila y columna k -ésimas de B .

Ejemplo 7.8 Resolver el siguiente problema de programación lineal:

$$\begin{aligned} \min. \quad & x_1 - 2x_2 \\ \text{s. a} \quad & x_1 + x_2 \geq 2 \\ & -x_1 + x_2 \geq 1 \\ & x_2 \leq 3 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Para empezar, introduzcamos como siempre las variables de holgura. Resulta:

$$\begin{array}{llllll} \min. & x_1 & - & 2x_2 & & \\ \text{s. a} & x_1 & + & x_2 & - & x_3 & = & 2 \\ & -x_1 & + & x_2 & & - & x_4 & = & 1 \\ & & & x_2 & & & + & x_5 & = & 3 \\ & & & & & & & & & x_1, x_2, x_3, x_4, x_5 \geq 0. \end{array}$$

Como la base de partida no es de fácil obtención, introducimos las variables artificiales y planteamos el siguiente problema dentro de la *fase I*:

$$\begin{array}{llllllll} \min. & & & & & & x_6 & + & x_7 \\ \text{s. a} & x_1 & + & x_2 & - & x_3 & & + & x_6 & = & 2 \\ & -x_1 & + & x_2 & & - & x_4 & & + & x_7 & = & 1 \\ & & & x_2 & & & & + & x_5 & = & 3 \\ & & & & & & & & & & x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0. \end{array}$$

Obsérvese que en la tercera condición no es necesario introducir variable artificial alguna.

Del problema así planteado se puede obtener inmediatamente la base de partida; ésta es

$$B = [\mathbf{a}_5, \mathbf{a}_6, \mathbf{a}_7] = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 1 & -1 & 0 \\ -1 & 1 & 0 & -1 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

La solución inicial, $\mathbf{x}_B = B^{-1}\mathbf{b}$, $\mathbf{x}_N = \mathbf{0}$, es:

$$\begin{bmatrix} x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}.$$

Por tanto,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 3 \\ 2 \\ 1 \end{bmatrix}.$$

Fase I. Iteración 1. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = 0 - [1, 1, 0] \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = 0,$$

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = 0 - [1, 1, 0] \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = -2,$$

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [1, 1, 0] \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = 1$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [1, 1, 0] \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = 1.$$

No se ha llegado todavía al óptimo pues existe un coste reducido negativo.

Fase I. Iteración 1. Paso 2

Elegimos la única variable no básica posible, x_2 , como aquella que ha de entrar en la base.

Resolvemos el sistema $B\mathbf{y} = \mathbf{a}_2$:

$$\mathbf{y} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

Fase I. Iteración 1. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_2}}{y_2}, \frac{x_{B_3}}{y_3} \right\} = \min \left\{ \frac{3}{1}, \frac{2}{1}, \frac{1}{1} \right\} = 1;$$

es decir, saldrá $x_{B_3} = x_7$. O, como solemos utilizar, de evaluar la expresión $\mathbf{x}_B = B^{-1}\mathbf{b} - \theta\mathbf{y}$. Es decir,

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \\ x_{B_3} \end{bmatrix} = \begin{bmatrix} x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} - \theta \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix},$$

comprobándose que x_7 es la primera variable que llega a cero al incrementar x_2 .

Fase I. Iteración 1. Paso 4

Readaptemos la solución y las estructuras de datos correspondientes:

$$\begin{aligned}x_2 &\leftarrow \theta = 1 \\x_5 &\leftarrow x_5 - \theta y_1 = 3 - 1 \cdot 1 = 2 \\x_6 &\leftarrow x_6 - \theta y_2 = 2 - 1 \cdot 1 = 1 \\x_7 &\leftarrow x_7 - \theta y_3 = 1 - 1 \cdot 1 = 0.\end{aligned}$$

La solución queda por tanto,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 2 \\ 1 \\ 0 \end{bmatrix}$$

siendo

$$B = [\mathbf{a}_5, \mathbf{a}_6, \mathbf{a}_2] = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_1, \mathbf{a}_7, \mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 & -1 & 0 \\ -1 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Fase I. Iteración 2. Paso 1

Calculemos de nuevo los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & -1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = 0 - [1, -1, 0] \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = -2,$$

$$\bar{c}_7 = c_7 - \boldsymbol{\pi}^T \mathbf{a}_7 = 1 - [1, -1, 0] \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = 2,$$

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [1, -1, 0] \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = 1$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [1, -1, 0] \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = -1.$$

No se ha llegado todavía al óptimo.

Fase I. Iteración 2. Paso 2

Elegimos la variable no básica x_1 como aquella que ha de entrar en la base por ser la de coste reducido más negativo.

Resolvemos el sistema $B\mathbf{y} = \mathbf{a}_1$:

$$\mathbf{y} = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 1 \\ 1 & -1 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

Fase I. Iteración 2. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_2}}{y_2} \right\} = \min \left\{ \frac{2}{1}, \frac{1}{2} \right\} = \frac{1}{2};$$

es decir, saldrá $x_{B_2} = x_6$. Esto también se puede deducir de la expresión

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \\ x_{B_3} \end{bmatrix} = \begin{bmatrix} x_5 \\ x_6 \\ x_2 \end{bmatrix} = B^{-1}\mathbf{b} - \theta\mathbf{y} = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix} - \theta \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix},$$

constatándose que x_6 es la primera variable que llega a cero al incrementar x_1 .

Fase I. Iteración 2. Paso 4

Readaptemos la solución y las estructuras de datos correspondientes:

$$\begin{aligned} x_1 &= \theta = \frac{1}{2} \\ x_5 &= x_5 - \theta y_1 = 2 - \frac{1}{2} \cdot 1 = \frac{3}{2} \\ x_6 &= x_6 - \theta y_2 = 1 - \frac{1}{2} \cdot 2 = 0 \\ x_2 &= x_2 - \theta y_3 = 1 - \frac{1}{2} \cdot (-1) = \frac{3}{2}. \end{aligned}$$

La solución queda por tanto

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 3/2 \\ 0 \\ 0 \\ 3/2 \\ 0 \\ 0 \end{bmatrix}$$

siendo

$$B = [\mathbf{a}_5, \mathbf{a}_1, \mathbf{a}_2] = \begin{bmatrix} 0 & 1 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_6, \mathbf{a}_7, \mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Fase I. Iteración 3. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & -1 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\begin{aligned} \bar{c}_6 &= c_6 - \boldsymbol{\pi}^T \mathbf{a}_6 = 1 - [0, 0, 0] \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = 1, \\ \bar{c}_7 &= c_7 - \boldsymbol{\pi}^T \mathbf{a}_7 = 1 - [0, 0, 0] \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = 1, \\ \bar{c}_3 &= c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [0, 0, 0] \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = 0, \end{aligned}$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [0, 0, 0] \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = 0.$$

Todos los costes reducidos son no negativos por lo que se ha llegado al óptimo de la *fase I*.

La base de partida de la *fase II* es

$$B = [\mathbf{a}_5, \mathbf{a}_1, \mathbf{a}_2] = \begin{bmatrix} 0 & 1 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 0 & 0 \end{bmatrix}.$$

Fase II. Iteración 1. Paso 1

Calculemos como siempre los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & -1 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 1 \\ -2 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ -2 \end{bmatrix} = \begin{bmatrix} -1/2 \\ -3/2 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [-1/2, -3/2, 0] \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = -\frac{1}{2},$$

y

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [-1/2, -3/2, 0] \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = -\frac{3}{2}.$$

Como $\bar{c}_3$ y $\bar{c}_4$ son negativos, no se ha llegado aún al óptimo.

Fase II. Iteración 1. Paso 2

Elegimos la variable no básica x_4 como aquella que ha de entrar en la base.

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}_4$:

$$\mathbf{y} = \begin{bmatrix} 0 & 1 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} & 1 \\ \frac{1}{2} & -\frac{1}{2} & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 1/2 \\ -1/2 \end{bmatrix}.$$

Fase II. Iteración 1. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_2}}{y_2} \right\} = \min \left\{ \frac{\frac{3}{2}}{\frac{1}{2}}, \frac{\frac{1}{2}}{\frac{1}{2}} \right\} = 1.$$

Es decir, saldrá $x_{B_2} = x_1$. Si se analiza la expresión

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \\ x_{B_3} \end{bmatrix} = B^{-1}\mathbf{b} - \theta\mathbf{y} = \begin{bmatrix} x_5 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3/2 \\ 1/2 \\ 3/2 \end{bmatrix} - \theta \begin{bmatrix} 1/2 \\ 1/2 \\ -1/2 \end{bmatrix},$$

también se comprueba que, efectivamente, la primera variable que llega a cero al incrementar x_4 es x_1 .

Fase II. Iteración 1. Paso 4

Readaptemos la solución y las estructuras de datos correspondientes:

$$\begin{aligned} x_4 &\leftarrow \theta = 1 \\ x_5 &\leftarrow x_5 - \theta y_1 = \frac{3}{2} - 1 \cdot \frac{1}{2} = 1 \\ x_1 &\leftarrow x_1 - \theta y_2 = \frac{1}{2} - 1 \cdot \frac{1}{2} = 0 \\ x_2 &\leftarrow x_2 - \theta y_3 = \frac{3}{2} + 1 \cdot \frac{1}{2} = 2. \end{aligned}$$

La nueva solución es, por tanto,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

con

$$B = [\mathbf{a}_5, \mathbf{a}_4, \mathbf{a}_2] = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_3, \mathbf{a}_1] = \begin{bmatrix} -1 & 1 \\ 0 & -1 \\ 0 & 0 \end{bmatrix}.$$

Fase II. Iteración 2. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} -1 & 1 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} -2 \\ 0 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [-2, 0, 0] \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = -2,$$

y

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = 1 - [-2, 0, 0] \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = 3.$$

Todavía no se ha llegado al óptimo.

Fase II. Iteración 2. Paso 2

Elegimos la variable no básica x_3 como aquella que ha de entrar en la base.

Resolvemos el sistema $B\mathbf{y} = \mathbf{a}_3$:

$$\mathbf{y} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 1 \\ 1 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 1 \\ 1 & -1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix}.$$

Fase II. Iteración 2. Paso 3

La variable básica a salir de la base se obtendrá de calcular

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1} \right\} = \min \left\{ \frac{1}{1} \right\} = 1;$$

es decir, saldrá $x_{B_1} = x_5$. Viéndolo a partir de la expresión

$$\begin{bmatrix} x_{B_1} \\ x_{B_2} \\ x_{B_3} \end{bmatrix} = \begin{bmatrix} x_5 \\ x_4 \\ x_2 \end{bmatrix} = b^{-1} \mathbf{b} - \theta \mathbf{y} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} - \theta \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix},$$

se comprueba también que la primera variable que llega a cero al incrementar x_3 es x_5 .

Fase II. Iteración 2. Paso 4

Readaptamos la solución y las estructuras de datos correspondientes:

$$\begin{aligned} x_3 &\leftarrow \theta = 1 \\ x_5 &\leftarrow x_5 - \theta y_1 = 1 - 1 \cdot 1 = 0 \\ x_4 &\leftarrow x_4 - \theta y_2 = 1 + 1 \cdot 1 = 2 \\ x_2 &\leftarrow x_2 - \theta y_3 = 2 + 1 \cdot 1 = 3. \end{aligned}$$

La nueva solución es

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 1 \\ 2 \\ 0 \end{bmatrix}$$

con

$$B = [\mathbf{a}_3, \mathbf{a}_4, \mathbf{a}_2] = \begin{bmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{y} \quad N = [\mathbf{a}_5, \mathbf{a}_1] = \begin{bmatrix} 0 & 1 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Fase II. Iteración 3. Paso 1

Calculemos los multiplicadores simplex resolviendo $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -2 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son

$$\bar{c}_5 = c_5 - \boldsymbol{\pi}^T \mathbf{a}_5 = 0 - [0, 0, -2] \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = 2,$$

y

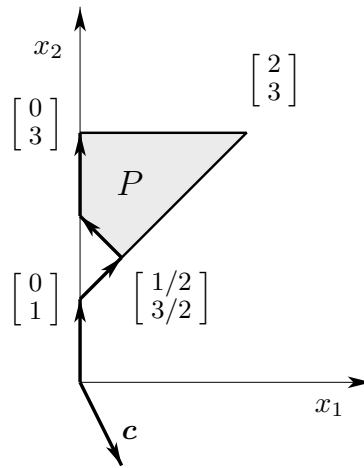
$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = 1 - [0, 0, -2] \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = 1.$$

Todos los costes reducidos de las variables no básicas son positivos por lo que se ha alcanzado el único óptimo del problema.

La solución final es

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 1 \\ 2 \\ 0 \end{bmatrix}.$$

El valor óptimo de la función objetivo es -6 . El proceso iterativo que se ha seguido en el ejemplo se representa en la figura 7.6. ●

**Figura 7.6**

Trayectoria seguida en la resolución del ejemplo 7.8 empleando las *fases I y II* del método *simplex*

7.4.2 Método de penalización o de la gran M

Esta forma de obtener una solución inicial básica factible del problema a resolver consiste en combinar en una las *fases I y II* del método *simplex* y resolver

$$\begin{aligned} \min. \quad & \sum_{i=1}^n c_i x_i + M \sum_{j=1}^m x_{a_j} \\ \text{s. a} \quad & Ax + x_a = b \\ & x, x_a \geq 0. \end{aligned}$$

La constante M se elige suficientemente grande de tal forma que esté muy penalizado que en la solución $x_a \neq 0$. Eventualmente, si existe solución factible, las variables artificiales se verán obligadas a tomar valor cero en el óptimo de este problema.

Esta alternativa tiene dos importantes inconvenientes:

- La necesidad de escoger un determinado valor fijo de M que asegure que las variables artificiales no van a estar en la base de la solución óptima.
- Que un valor muy grande de M , que domine por completo el de los elementos del vector c y los coeficientes de la matriz A , puede hacer que los errores de redondeo de los cálculos y la inestabilidad numérica del proceso lleguen a ser importantes.

7.5 Implementaciones prácticas del método simplex

Cuando un problema es de considerables dimensiones (m y n muy grandes), no resulta práctico en cada iteración del método *simplex* resolver, partiendo de cero, los sistemas de ecuaciones lineales $B^T \pi = c_B$ y $B y = a_q$. Las formas prácticas que se utilizan para resolver en ordenadores problemas de programación lineal palian en parte este inconveniente.

7.5.1 El método simplex en forma de tableau

Una de las primeras formas —sin duda la más difundida en los libros de texto de programación lineal tradicionales— en que se implementó prácticamente el método *simplex* es la que se conoce como la de los *tableau*. Aun cuando en opinión de este autor esta forma de operar introduce no pocas confusiones y formas viciadas de entender las manipulaciones algebraicas inherentes al procedimiento *simplex*, debido a su amplia difusión, pasamos a continuación a considerarla brevemente.

Se basa en un conjunto de procedimientos para manipular un *tableau* como el que sigue.

$$\begin{array}{cc|cc}
 & -z & \mathbf{x}_B & \mathbf{x}_N & TD \\
 -z & 1 & 0 & \mathbf{c}_N^T - \mathbf{c}_B^T B^{-1} N & -\mathbf{c}_B^T B^{-1} \mathbf{b} \\
 \mathbf{x}_B & 0 & I & B^{-1} N & B^{-1} \mathbf{b}
 \end{array} \quad (7.8)$$

TD designa el vector término de la derecha.

Si T es una matriz cuyos coeficientes son los del *tableau* anterior, en realidad ese *tableau* representa el sistema

$$T \begin{bmatrix} -z \\ \mathbf{x}_B \\ \mathbf{x}_N \\ -1 \end{bmatrix} = 0,$$

de $m + 1$ ecuaciones lineales y $n + 1$ variables: las n de $\mathbf{x}$ y z .

El método *simplex* en forma de *tableau* se diferencia del expuesto hasta este apartado en que el cálculo de $\boldsymbol{\pi}$ y $\bar{\mathbf{c}}_N$ del Paso 1 e $\mathbf{y}$ del Paso 2 se eliminan y el Paso 4 de pivotación se lleva a cabo directamente en el *tableau*.

Si suponemos que las variables básicas que entran y salen de la base en una determinada iteración son, respectivamente, la q y la p , y que las filas y las columnas del *tableau* de esa iteración están numeradas partiendo de cero, la operación de pivotación conlleva:

- i) dividir la fila p de T por t_{pq} , el elemento que ocupa la fila p y la columna q de la matriz T , y
- ii) para $0 \leq i \leq m$, $i \neq p$, restar de la fila i la p multiplicada por t_{iq} a fin de hacer cero el elemento q de esa fila.

La pivotación mantiene la forma de (7.8) en la nueva base. Los costes reducidos, $\bar{\mathbf{c}}_N$, los componentes básicos de la dirección de descenso $\boldsymbol{\eta}_q$, $-B^{-1}\mathbf{a}_q$, así como el vector de variables básicas, $B^{-1}\mathbf{b}$, se obtienen directamente del *tableau*.

La versión en *tableau* del método *simplex* se confunde habitualmente con el propio nombre del método pues así fue la forma en que fue descrita su mecánica originalmente por Dantzig.

El procedimiento de los *tableau* no es la forma más adecuada de implementar el método *simplex* desde los puntos de vista numérico y comercial. Para problemas de grandes dimensiones —los más frecuentes que se plantean en la industria, investigación, etc.—, en los que la matriz de coeficientes de las condiciones suele tener una estructura muy dispersa, al utilizar los *tableau* esa estructura se destruye, siendo además necesario calcular, para rehacer los cuadros, todas las columnas de la matriz $B^{-1}N$, a pesar de que sólo se necesita calcular $B^{-1}\mathbf{a}_j$.

En aras de aclarar la mecánica de esta variante del método *simplex*, no obstante las anteriores consideraciones, a continuación resolvemos un pequeño programa lineal por el método *simplex* en *tableau*.

Ejemplo 7.9 Resolvamos:

$$\begin{aligned} \min. \quad & x_1 + x_2 - 4x_3 \\ \text{s. a} \quad & x_1 + x_2 + 2x_3 \leq 9 \\ & x_1 + x_2 - x_3 \leq 2 \\ & -x_1 + x_2 + x_3 \leq 4 \\ & x_1, x_2, x_3 \geq 0. \end{aligned}$$

Para empezar introducimos como siempre las variables de holgura a fin de transformar el problema en la forma estándar:

$$\begin{aligned} \min. \quad & x_1 + x_2 - 4x_3 \\ \text{s. a} \quad & x_1 + x_2 + 2x_3 + x_4 = 9 \\ & x_1 + x_2 - x_3 + x_5 = 2 \\ & -x_1 + x_2 + x_3 + x_6 = 4 \\ & x_1, x_2, x_3, x_4, x_5, x_6 \geq 0. \end{aligned}$$

Iteración 1

	$-z$	x_1	x_2	x_3	x_4	x_5	x_6	TD
$-z$	1	1	1	-4	0	0	0	0
x_4	0	1	1	2	1	0	0	9
x_5	0	1	1	-1	0	1	0	2
x_6	0	-1	1	①	0	0	1	4

La variable que entra en la base es la no básica cuyo coste reducido es más negativo: x_3 . El ① indica que de entre las relaciones $9/2$ y $4/1$ se elige esta última, por lo que saldrá de la base aquella variable que pivota en la tercera fila: x_6 .

Iteración 2

	$-z$	x_1	x_2	x_3	x_4	x_5	x_6	TD
$-z$	1	-3	5	0	0	0	4	16
x_4	0	③	-1	0	1	0	-2	1
x_5	0	0	2	0	0	1	1	6
x_3	0	-1	1	1	0	0	1	4

Iteración 3

	$-z$	x_1	x_2	x_3	x_4	x_5	x_6	TD
$-z$	1	0	4	0	1	0	2	17
x_1	0	1	$-\frac{1}{3}$	0	$\frac{1}{3}$	0	$-\frac{2}{3}$	$\frac{1}{3}$
x_5	0	0	2	0	0	1	1	6
x_3	0	0	$\frac{2}{3}$	1	$\frac{1}{3}$	0	$\frac{1}{3}$	$\frac{13}{3}$

Se ha llegado a un punto en el que todos los costes reducidos de las variables no básicas son positivos por lo que se ha conseguido el óptimo del problema. La solución final es por tanto

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 0 \\ 13/3 \\ 0 \\ 6 \\ 0 \end{bmatrix}.$$

El valor de la función objetivo es $z = -17$. La base óptima la constituyen los vectores columna $\mathbf{a}_1$, $\mathbf{a}_5$ y $\mathbf{a}_3$. Es decir,

$$B = [\mathbf{a}_1, \mathbf{a}_5, \mathbf{a}_3] = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & -1 \\ -1 & 0 & 1 \end{bmatrix}.$$

La inversa de B es la submatriz que en el *tableau* final ocupa el lugar que en el inicial ocupaba la submatriz identidad; es decir,

$$B^{-1} = \begin{bmatrix} 1/3 & 0 & -2/3 \\ 0 & 1 & 1 \\ 1/3 & 0 & 1/3 \end{bmatrix}.$$

7.5.2 Forma producto de la inversa de la base

Esta implementación del método *simplex* revisado descrito en la tabla 7.1 de la página 420, trata de resolver eficazmente los sistemas de ecuaciones lineales $B^T \boldsymbol{\pi} = \mathbf{c}_B$ y $B\mathbf{y} = \mathbf{a}_q$. Se planteó ante la necesidad de resolver estos sistemas cuando su dimensión es muy grande.

Con este objetivo, en cada iteración sólo se genera aquella información estrictamente necesaria manteniéndose una representación explícita de la matriz B^{-1} , la cual se recalcula después de cada cambio de base. En esa representación y en su adaptación es donde los códigos modernos de programación lineal centran sus esfuerzos, diferenciándose unos de otros en la eficacia con la que las llevan a cabo. En general, si B^{-1} es la inversa de la matriz básica B en una

determinada iteración, la adaptación de la misma para la siguiente iteración se puede expresar como

$$\bar{B}^{-1} = EB^{-1},$$

donde

$$E = I - \frac{(\mathbf{y} - \mathbf{e}_p)\mathbf{e}_p^T}{y_p} = \begin{bmatrix} 1 & & -y_1/y_p & & \\ & \ddots & \vdots & & \\ & & 1 & -y_{p-1}/y_p & \\ & & & 1/y_p & \\ & & & -y_{p+1}/y_p & 1 \\ & & & \vdots & \\ & & & -y_m/y_p & & 1 \end{bmatrix} \quad (7.9)$$

$\uparrow$
columna p

y $\mathbf{y} = B^{-1}\mathbf{a}_q$. A la matriz E se la suele denominar matriz de coeficientes eta o *matriz eta*. Estas expresiones surgen del hecho de que

$$\bar{B} = BE^{-1},$$

donde

$$E^{-1} = \begin{bmatrix} 1 & & y_1 & & \\ & \ddots & \vdots & & \\ & & 1 & y_{p-1} & \\ & & & y_p & \\ & & & y_{p+1} & 1 \\ & & & \vdots & \\ & & & y_m & & 1 \end{bmatrix}.$$

Obsérvese que al multiplicar a la derecha la matriz B por la matriz E^{-1} las columnas de B quedan inalteradas, excepto la p -ésima que se transforma, según lo requerido, en $B\mathbf{y} = \mathbf{a}_q$.

Si como ocurre habitualmente el procedimiento *simplex* se inicia con una matriz B igual a la identidad, después de k iteraciones la matriz inversa de la base, B^{-1} , se puede expresar en una *forma producto*, de la siguiente manera

$$B_k^{-1} = E_k E_{k-1} \cdots E_1 B^{-1}.$$

En esta expresión cada matriz elemental E_i tiene la forma de (7.9). Para su implementación en ordenador este esquema operativo únicamente requiere almacenar los valores de los elementos de la columna de la matriz E_i que la hace diferente de la matriz identidad, y el propio valor del índice i .

A tenor de lo indicado, la iteración $k + 1$ del método *simplex* revisado que se expuso en la tabla 7.1 se puede reescribir, introduciendo la forma producto de la matriz inversa de la base, como se indica en la tabla 7.3. Recordemos que $\mathbf{B} = \{j_1, \dots, j_m\}$ es el conjunto de índices de las variables básicas.

La gran ventaja de esta forma de implementar el método *simplex* radica en su eficacia para abordar estructuras dispersas de matrices A de grandes dimensiones. Aunque B_k^{-1} puede no

Tabla 7.3

Algoritmo *simplex* revisado en la forma producto de la inversa de la base

Paso 1 – Calcular los multiplicadores simplex a partir de

$$\boldsymbol{\pi}^T = \mathbf{c}_B^T \mathbf{B}^{-1} = ((\mathbf{c}_B^T \mathbf{E}_k) \mathbf{E}_{k-1}) \cdots \mathbf{E}_1).$$

Esta operación se conoce como *transformación inversa* o BTRAN (de *backward transformation*).

Determinar los costes reducidos de las variables no básicas a partir de

$$\bar{c}_j = c_j - \boldsymbol{\pi}^T \mathbf{a}_j, \quad \text{para todo } j \in \mathbf{N}.$$

Si $\bar{c}_j \geq 0$ para todo $j \notin \mathbf{B}$, PARAR; la solución es óptima.

Paso 2 – Escoger $q \in \mathbf{N}$ tal que $\bar{c}_q = \min_{j \in \mathbf{N}} \{\bar{c}_j < 0\}$. Calcular

$$\mathbf{y} = \mathbf{B}^{-1} \mathbf{a}_q = (\mathbf{E}_k (\mathbf{E}_{k-1} \cdots (\mathbf{E}_1 \mathbf{a}_q))).$$

Esta operación se conoce como *transformación directa* o FTRAN (de *forward transformation*).

Si $\mathbf{y} \leq \mathbf{0}$, PARAR; el problema es no acotado.

Paso 3 – Si $\mathbf{x}_j = \mathbf{B}^{-1} \mathbf{b}$, establecer la variable básica x_{j_p} que sale de la base determinando la fila p sobre la que pivota la columna q a partir de la relación

$$\frac{x_{j_p}}{y_p} = \min_{1 \leq i \leq m} \left\{ \frac{x_{j_i}}{y_i} : y_i > 0 \right\}.$$

A esta operación se la denomina CHUZR.

Paso 4 – Adaptar la matriz inversa de la base y la solución a partir de

$$\mathbf{E}_{k+1} = \begin{bmatrix} 1 & & -y_1/y_p & & \\ & \ddots & \vdots & & \\ & & 1 & -y_{p-1}/y_p & \\ & & & 1/y_p & \\ & & -y_{p+1}/y_p & 1 & \\ & & \vdots & & \ddots \\ & -y_m/y_p & & & 1 \end{bmatrix}.$$

La nueva inversa de la base será $\mathbf{B}^{-1} \leftarrow \mathbf{E}_{k+1} \mathbf{E}_k \cdots \mathbf{E}_2 \mathbf{E}_1$ y la nueva solución $\mathbf{x}_j \leftarrow \mathbf{E}_{k+1} \mathbf{x}_j$.

A esta operación se la conoce como WRETA.

ser suficientemente dispersa, como se almacena como producto de matrices elementales *eta* muy dispersas, este problema se soslaya en gran medida.

Conforme avanza el proceso para resolver un determinado programa lineal, es inevitable que, particularmente en problemas de grandes dimensiones, se produzcan errores de redondeo. Para evitarlos es aconsejable refactorizar periódicamente la matriz B^{-1} ; en sucesivas iteraciones, después de esa refactorización, las transformaciones elementales *eta* que de ellas se derivasen se deberán aplicar a esa nueva B^{-1} . Esta refactorización, conocida habitualmente en la literatura especializada y en los programas comerciales disponibles por INVERT, ahorra tiempo de cálculo en las siguientes etapas, memoria de ordenador y reduce los efectos del redondeo de las operaciones algebraicas que se efectúan.

Las rutinas que hoy en día implementan la operación INVERT son extremadamente sofisticadas. Para llevarla a cabo se permutan las filas y las columnas de la matriz B a fin de que el resultado final tenga el menor número de elementos distintos de cero posible. Los esquemas publicados y empleados para optimizar esa permutación de filas y columnas son diversos. Se basan en manipulaciones y esquemas como los referidos en el capítulo 3. Los más populares son los de Hellerman y Rarich [1971] y [1972], conocidos como p^3 y p^4 , y el de Markowitz [1957], introducido en la página 261.

7.5.3 Factorización LU de la base

La forma producto de la matriz inversa de la base ha dado paso en los códigos más modernos para el tratamiento de problemas de muy grandes dimensiones (decenas o cientos de miles de variables y condiciones), a factorizaciones LU de la base numéricamente estables. Este enfoque almacena la matriz L^{-1} mediante una sucesión de matrices elementales, que difieren de la identidad en un solo elemento debajo de la diagonal principal, y matrices de permutación. La idea original de proceder así es de Bartels y Golub [1969], siendo muchas las implementaciones recientes basadas en ella (ver Reid [1982], Gill, Murray, Saunders y Wright [1986] y Gill, Murray y Wright [1991]).

En concreto, si se parte de la base B_0 (normalmente $B_0 = I$), se puede obtener —por eliminación de Gauss, por ejemplo—:

$$L_m P_m \cdots L_1 P_1 B_0 = U, \quad (7.10)$$

donde, siguiendo un razonamiento similar al realizado en el capítulo 1 al explicar la descomposición LU , se puede comprobar que $U = U_m U_{m-1} \cdots U_1$, siendo U_i la matriz que se obtiene de la identidad, I , al reemplazar su columna i por la correspondiente de U . Es decir, si, por ejemplo,

$$U = \begin{bmatrix} 3 & 2 & 1 \\ & 3 & 5 \\ & & 4 \end{bmatrix},$$

las matrices U_1 , U_2 y U_3 serán

$$U_1 = \begin{bmatrix} 3 & & \\ & 1 & \\ & & 1 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 1 & 2 & \\ & 3 & \\ & & 1 \end{bmatrix} \quad \text{y} \quad U_3 = \begin{bmatrix} 1 & & 1 \\ & 1 & 5 \\ & & 4 \end{bmatrix}.$$

Es fácil comprobar que $U = U_3 U_2 U_1$.

Después de k iteraciones se tendrá que

$$B_k = B_0 E_1^{-1} E_2^{-1} \cdots E_k^{-1};$$

es decir, que

$$L_m P_m \cdots L_1 P_1 B_k = U_m U_{m-1} \cdots U_1 E_1^{-1} E_2^{-1} \cdots E_k^{-1}.$$

Mediante esta forma de factorizar la base, la resolución del sistema de ecuaciones $B_k^T \boldsymbol{\pi} = \mathbf{c}_B$, en el paso 1 del algoritmo de la tabla 7.3, se llevaría a cabo mediante los pasos siguientes:

1. Hacer $i = k$ y $\boldsymbol{\pi} = \mathbf{c}_B$.
2. Si $i \geq 1$, hacer $\boldsymbol{\pi} \leftarrow E_i^T \boldsymbol{\pi}$, $i \leftarrow i - 1$ e ir otra vez a 2.
3. Hacer $j = 1$.
4. Si $j \leq m$, hacer $\boldsymbol{\pi} \leftarrow U_j^{-T} \boldsymbol{\pi}$, $j \leftarrow j + 1$ e ir a 4.
5. Hacer $j = m$.
6. Si $j \geq 1$, hacer $\boldsymbol{\pi} \leftarrow P_j^T L_j^T \boldsymbol{\pi}$, $j \leftarrow j - 1$ e ir a 6.

Los pasos 1 a 4 corresponden a la resolución de

$$\left(\left(\left(\boldsymbol{\pi}^T U_m \right) U_{m-1} \right) \cdots \right) E_k^{-1} = \mathbf{c}_B^T,$$

el 5 y el 6 a la operación

$$\boldsymbol{\pi}^T \leftarrow \left(\left(\boldsymbol{\pi}^T L_m P_m \right) \cdots \right) L_1 P_1.$$

Esto se hace así porque, en los pasos 1 a 4, en vez de resolver el sistema $\boldsymbol{\pi}^T B_k = \mathbf{c}_B^T$, se resuelve $(\boldsymbol{\pi}')^T X B_k = \mathbf{c}_B^T$. Como $\boldsymbol{\pi}^T = (\boldsymbol{\pi}')^T X$, los pasos 5 y 6 restablecen el verdadero valor de $\boldsymbol{\pi}^T$ multiplicando la solución obtenida en los pasos anteriores por X . Evidentemente, $X = L_m P_m \cdots L_1 P_1$.

De igual manera se resolvería el sistema $B_k \mathbf{y} = \mathbf{a}_q$:

1. Hacer $j = 1$ y $\mathbf{y} = \mathbf{a}_q$.
2. Si $j \leq m$, hacer $\mathbf{y} \leftarrow L_j P_j \mathbf{y}$, $j \leftarrow j + 1$ e ir otra vez a 2.
3. Hacer $j = m$.
4. Si $j \geq 1$, hacer $\mathbf{y} \leftarrow U_j^{-1} \mathbf{y}$, $j \leftarrow j - 1$ e ir a 4.
5. Hacer $i = 1$.
6. Si $i \leq k$, hacer $\mathbf{y} \leftarrow E_i \mathbf{y}$, $i \leftarrow i + 1$ e ir a 6.

En los pasos 1 y 2 se lleva a cabo la operación

$$\mathbf{y}' \leftarrow (L_m P_m (\cdots (L_1 P_1 \mathbf{a}_q))).$$

Del 3 al 6 se resuelve

$$U_m \left(U_{m-1} \left(\cdots \left(E_k^{-1} \mathbf{y} \right) \right) \right) = \mathbf{y}'.$$

Se procede así porque en vez de disponer de B_k se dispone de XB_k , y el sistema $B_k \mathbf{y} = \mathbf{a}_q$ es equivalente a $XB_k \mathbf{y} = X\mathbf{a}_q$, por lo que premultiplicando $\mathbf{a}_q$ por $X = L_m P_m \cdots L_1 P_1$, se llega a tal sistema y de él a la solución que nos interesa: $\mathbf{y}$.

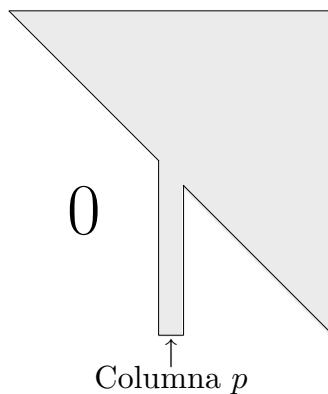
Para almacenar las matrices E_i , L_j y U_j en la memoria del ordenador sólo es necesario guardar el valor de los subíndices y los elementos distintos de cero de las columnas que designan esos subíndices. De las matrices de permutación P_i sólo es necesario guardar un puntero que indique con qué fila se intercambia la i .

Un hipotético fichero que contuviese $P_1, L_1, P_2, L_2, \dots, P_m, L_m, U_m^{-1}, \dots, U_1^{-1}, E_1, \dots, E_k$, caso de no poderse guardar en memoria todos los valores de estas matrices, se denomina *fichero eta*. Para resolver $B_k^T \boldsymbol{\pi} = \mathbf{c}_B$, ese fichero se leería de atrás hacia adelante (*backward* en inglés), en una operación que se ha dado en denominar BTRAN; para resolver $B_k \mathbf{y} = \mathbf{a}_q$, de adelante hacia atrás (*forward*), en lo que se denomina operación FTRAN.

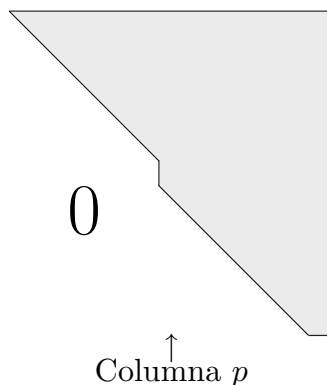
Cuando el *fichero eta* con las transformaciones BTRAN y FTRAN se hace muy grande, o se vislumbra una pérdida de estabilidad numérica, se refactoriza la base obteniéndose una nueva B_0 . Las operaciones de refactorización periódica de esta factorización puede hacerse siguiendo alguno de los esquemas que se recogen en Reid[1982], Bartels y Golub[1969], Forrest y Tomlin [1972], Saunders [1976] y Gill, Murray, Saunders y Wright [1986]. La idea general de cualquiera de ellos se basa en que si en (7.10) se agrupa $L_m P_m \cdots L_1 P_1$ bajo la expresión L^{-1} , al modificar en una iteración dada, k , la base según la expresión ya indicada, $B_{k+1} = B_k E_k^{-1}$, resultando que la columna p de la base se reemplaza por una $\mathbf{a}_q$ de A , se tiene que

$$L_m P_m \cdots L_1 P_1 B_{k+1} = L^{-1} B_{k+1} = U_m \cdots U_1 E_k^{-1} = U E_k^{-1}.$$

La matriz $L^{-1} B_{k+1}$, en general, no será estrictamente triangular superior sino que tendrá la forma que sigue.



Esta matriz se puede multiplicar a la derecha por una de permutación Q de tal forma que se obtenga otra H (matriz de Hessenberg) de la forma



Es decir, $L^{-1}B_{k+1}Q = UE_k^{-1}Q = H$. A continuación se procede a reducir H a la forma triangular superior aplicándole una sucesión de transformaciones elementales de la forma

$$M_i = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & m_i & 1 & \\ & & & \ddots & \\ & & & & 1 \end{bmatrix},$$

$i = p, p + 1, \dots, m - 1$. Al final de este último proceso se tendrá que

$$M_{m-1} \cdots M_p L^{-1} B_{k+1} Q = U_{k+1}.$$

La matriz inversa de la nueva matriz básica se obtendrá fácilmente mediante la expresión

$$B_{k+1}^{-1} = QU_{k+1}^{-1}ML^{-1},$$

donde $M = M_{m-1} \cdots M_p$. Evaluar esta matriz resulta extremadamente sencillo pues las diversas matrices de la expresión anterior son elementales. De ellas, como veíamos anteriormente, sólo hay que guardar en la memoria del ordenador una mínima cantidad de información.

De los diversos métodos de adaptación de la matriz B descompuesta en la forma LU que recoge la literatura especializada, y que usan los códigos de programación lineal más extendidos, merecen destacarse los de Forrest y Tomlin [1972], Reid [1982], Saunders [1976] y Gill, Murray, Saunders y Wright [1986].

En el apéndice E se incluye el listado en FORTRAN 77 del programa BBMI desarrollado por el autor que implementa exactamente el algoritmo de la tabla 7.3. La operación INVERT se efectúa a partir de una factorización LU de la base reordenando las filas y columnas de ésta de acuerdo con el criterio de Markowitz.

7.6 El método simplex para variables acotadas

Hasta ahora hemos supuesto que las variables que definen el programa lineal no están acotadas superiormente y, si lo están, la forma de transformar el problema en la forma estándar. En lo

sucesivo nos referiremos al problema

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u} . \end{aligned} \quad (7.11)$$

Las variables restringidas según (7.11) se dicen *acotadas*. Si la variable x_j tiene $l_j = -\infty$ y $u_j = \infty$, se dice *libre*. Si $\mathbf{l} = \mathbf{0}$ las restricciones son las usuales de no negatividad. Cualquier vector cota inferior, $\mathbf{l}$, puede transformarse en el vector cero sin más que efectuar el cambio de variables $\mathbf{x}' = \mathbf{x} - \mathbf{l}$.

La forma más rápida de abordar el problema (7.11), de acuerdo con lo hasta ahora estudiado, consiste en introducir unos vectores de holgura, $\mathbf{x}_1$ y $\mathbf{x}_2$, que transformen el problema en otro de la forma

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} + \mathbf{x}_1 = \mathbf{u} \\ & \mathbf{x} - \mathbf{x}_2 = \mathbf{l} \\ & \mathbf{x}, \mathbf{x}_1, \mathbf{x}_2 \geq \mathbf{0} \end{aligned}$$

y posteriormente resolverlo por el método habitual. El inconveniente inmediato de esta forma de actuar surge del hecho de que el número de variables se triplica y el número de condiciones pasa a ser $m+2n$. Está claro que el esfuerzo a realizar para resolver este problema se incrementa notablemente y puede llegar a ser prohibitivo.

A continuación introducimos el método más adecuado para resolver problemas como el de (7.11): el *método simplex para variables acotadas*. Se basa en la introducción de unas sencillas modificaciones en el clásico que ya hemos explicado. Considérese para ello el sistema de inecuaciones

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ \mathbf{l} &\leq \mathbf{x} \leq \mathbf{u} , \end{aligned} \quad (7.12)$$

donde la matriz $A^{m \times n}$ se supone, sin pérdida de generalidad, de rango m .

Definición 7.1 Una *solución básica factible* del sistema (7.12) es aquella solución en la que $n - m$ variables (variables no básicas) toman el valor de uno de sus límites, $\mathbf{l}$ ó $\mathbf{u}$, y las restantes m (variables básicas) corresponden a las columnas independientes de A .

De igual forma a como hacíamos en apartados anteriores, descompongamos la matriz A en $[B \ N_l \ N_u]$, con $\text{rango}(B) = m$, y de acuerdo con esto, el vector $\mathbf{x}$ en $[\mathbf{x}_B^T, \mathbf{x}_{N_l}^T, \mathbf{x}_{N_u}^T]$.

Un vector $\mathbf{x}$ se dice solución básica factible del sistema de inecuaciones (7.12) anterior si $\mathbf{x}_B$ es solución del sistema

$$B\mathbf{x}_B = \mathbf{b},$$

$\mathbf{x}_{N_l} = \mathbf{l}_{N_l}$ y $\mathbf{x}_{N_u} = \mathbf{u}_{N_u}$. Si además $\mathbf{l}_B < \mathbf{x}_B < \mathbf{u}_B$, entonces $\mathbf{x}$ es una *solución básica factible no degenerada*; por el contrario, si algún componente de $\mathbf{x}_{B_j}$ es igual a l_j o u_j , la solución se dice *básica factible degenerada*.

La idea que anima el método *simplex* para variables acotadas es muy sencilla. Supongamos que partimos de una solución básica factible. En ese punto se examinan las variables no básicas

(variables en uno de sus límites) a través del valor de sus costes reducidos. Si la solución puede ser mejorada —en un problema de minimización—, ello quiere decir que alguna variable no básica que esté en su límite inferior (para que el problema pueda seguir siendo factible sólo se podrá incrementar) tendrá un coste reducido negativo, o que otra que esté en su límite superior tendrá un coste reducido positivo.

Razonando en términos algebraicos, expresemos las condiciones de la forma

$$B\mathbf{x}_B + N_l\mathbf{x}_{N_l} + N_u\mathbf{x}_{N_u} = \mathbf{b}.$$

El vector $\mathbf{x}_B$ es:

$$\mathbf{x}_B = B^{-1}\mathbf{b} - B^{-1}N_l\mathbf{x}_{N_l} - B^{-1}N_u\mathbf{x}_{N_u}.$$

La función objetivo es

$$\begin{aligned} z &= \mathbf{c}^T\mathbf{x} = \mathbf{c}_B^T\mathbf{x}_B + \mathbf{c}_{N_l}^T\mathbf{x}_{N_l} + \mathbf{c}_{N_u}^T\mathbf{x}_{N_u} \\ &= \mathbf{c}_B^T(B^{-1}\mathbf{b} - B^{-1}N_l\mathbf{x}_{N_l} - B^{-1}N_u\mathbf{x}_{N_u}) + \mathbf{c}_{N_l}^T\mathbf{x}_{N_l} + \mathbf{c}_{N_u}^T\mathbf{x}_{N_u} \\ &= \mathbf{c}_B^TB^{-1}\mathbf{b} + (\mathbf{c}_{N_l}^T - \mathbf{c}_B^TB^{-1}N_l)\mathbf{x}_{N_l} + (\mathbf{c}_{N_u}^T - \mathbf{c}_B^TB^{-1}N_u)\mathbf{x}_{N_u}. \end{aligned}$$

El vector de costes reducidos de las variables no básicas en su límite inferior es $\mathbf{c}_{N_l}^T - \mathbf{c}_B^TB^{-1}N_l$; el de las no básicas en su límite superior, $\mathbf{c}_{N_u}^T - \mathbf{c}_B^TB^{-1}N_u$. La función objetivo se puede mejorar (decrementar) si

$$(\mathbf{c}_j - \mathbf{c}_B^TB^{-1}\mathbf{a}_j) < 0 \quad \text{para alguna } x_j = l_j$$

o

$$(\mathbf{c}_j - \mathbf{c}_B^TB^{-1}\mathbf{a}_j) > 0 \quad \text{para alguna } x_j = u_j.$$

Si se elige una determinada variable no básica, x_q , como aquella que mejora más la función objetivo de acuerdo con un criterio que se considere adecuado, y su valor lo modificamos continuamente desde el límite o cota en la que esté en la dirección hacia la otra cota, la función objetivo mejorará (decrecerá) mientras se puedan modificar las variables básicas de tal manera que persista la factibilidad del problema (se sigan satisfaciendo las condiciones). Este proceso se detendrá cuando:

- i) una variable básica llegue a uno de sus límites, o
- ii) la variable no básica que se modifica llegue al límite opuesto del que parte antes de que ocurra lo anterior.

Si ocurre el caso i), la variable básica correspondiente se hará no básica, pasando a ocupar su sitio en la base la no básica que se modifica. Si ocurre ii), la base no se modifica. Cuando ocurren simultáneamente i) y ii) resulta una solución básica factible degenerada.

En términos algebraicos otra vez, si N_l es el conjunto de los índices de las variables no básicas en su límite inferior y N_u el de las no básicas en su límite superior, y se modifica x_q , se tendrá que

$$\begin{aligned} \mathbf{x}_B &= B^{-1}\mathbf{b} - B^{-1}N_l\mathbf{x}_{N_l} - B^{-1}N_u\mathbf{x}_{N_u} - \theta B^{-1}\mathbf{a}_q \\ &= \hat{\mathbf{b}} - \theta\mathbf{y}, \end{aligned}$$

donde $\hat{\mathbf{b}} = B^{-1}\mathbf{b} - B^{-1}N_l\mathbf{x}_{N_l} - B^{-1}N_u\mathbf{x}_{N_u}$ y $\mathbf{y} = B^{-1}\mathbf{a}_q$.

En este punto caben considerar dos opciones: que $q \in N_u$, o que $q \in N_l$. Si $q \in N_l$, el incremento posible de x_q desde su límite inferior l_q , θ , ha de ser positivo para seguir manteniendo la factibilidad del problema; el máximo valor de este θ será

$$\theta = \min \begin{cases} \theta_1 = \begin{cases} \min_{1 \leq i \leq m} \left\{ \frac{\hat{b}_i - l_{B_i}}{y_i} : y_i > 0 \right\} \\ \infty \end{cases} \\ \theta_2 = \begin{cases} \min_{1 \leq i \leq m} \left\{ \frac{\hat{b}_i - u_{B_i}}{y_i} : y_i < 0 \right\} \\ \infty \end{cases} \\ u_q - l_q. \end{cases}$$

Si, por el contrario, $q \in N_u$, el incremento desde su límite superior u_q ha de ser negativo. El máximo de éste será

$$\theta = \max \begin{cases} \theta_1 = \begin{cases} \max_{1 \leq i \leq m} \left\{ \frac{\hat{b}_i - l_{B_i}}{y_i} : y_i < 0 \right\} \\ \infty \end{cases} \\ \theta_2 = \begin{cases} \max_{1 \leq i \leq m} \left\{ \frac{\hat{b}_i - u_{B_i}}{y_i} : y_i > 0 \right\} \\ \infty \end{cases} \\ l_q - u_q. \end{cases}$$

Hechos estos cálculos se deberá proceder a modificar las estructuras de datos correspondientes, a fin de reflejar los cambios habidos en la base y, partiendo de la nueva solución obtenida, reemprender el proceso.

Como se puede ver, unos simples cambios en la mecánica del proceso del método *simplex*, que refiriéndonos al algoritmo de la tabla 7.1 afectan sólo a los pasos 2 y 3, permiten abordar eficazmente el problema de las variables acotadas. Es importante hacer notar que todos los códigos que implementan el método *simplex* de una u otra forma lo hacen para variables acotadas. De hecho, en un ordenador, una variable en forma estándar, $x_j \geq 0$, en realidad se trata como si $0 \leq x_j \leq 10^{20}$, por ejemplo, ya que a todos los efectos, en una máquina que trabaje con precisión finita, la cantidad 10^{20} es suficientemente grande como para considerarla casi infinita en ese entorno.

En la tabla 7.4 se expone el algoritmo *simplex revisado para variables acotadas*. El conjunto N_l designa los índices de las variables no básicas en su límite inferior y N_u los de aquellas en su límite superior. Obsérvese que, de acuerdo con lo expuesto en el párrafo anterior sobre los límites reales al trabajar con ordenadores, la existencia de una solución no acotada se determina mediante una simple comprobación en el paso 3 o en el 4.

Tabla 7.4

Algoritmo *simplex* revisado para variables acotadas**Paso 1** – *Asignación de precios. Comprobación de óptimo.*

- (a) Calcular los multiplicadores simplex resolviendo el sistema $B^T \pi = c_B$.
- (b) Calcular los costes reducidos de las variables no básicas, $\bar{c}_j = c_j - \pi^T a_j$, para todo $j \notin B$.
- (c) Comprobar que se ha llegado al óptimo: si $\bar{c}_j \geq 0$ para todo $j \in N_l$ y $\bar{c}_j \leq 0$ para todo $j \in N_u$, PARAR; la solución es óptima.

Paso 2 – *Determinación de columna de pivotación.*

- (a) Determinar la variable no básica x_q que debe entrar en la base (encontrar una dirección de descenso): escoger $q \notin B$ tal que

$$\bar{c}_q = \max \left\{ \left| \min_{j \in N_l} \{\bar{c}_j < 0\} \right|, \max_{j \in N_u} \{\bar{c}_j > 0\} \right\}.$$

- (b) Resolver $B\mathbf{y} = \mathbf{a}_q$. Si $q \in N_l$, ir al paso 3; si $q \in N_u$, al 4.

Paso 3 – *Determinación fila de pivotación.*Determinar la variable básica x_{j_p} que sale de la base: calcular θ :

$$\theta = \min \left\{ \min_{1 \leq i \leq m} \left\{ \frac{x_{j_i} - l_{j_i}}{y_i} : y_i > 0 \right\}, \min_{1 \leq i \leq m} \left\{ \frac{x_{j_i} - u_{j_i}}{y_i} : y_i < 0 \right\}, u_q - l_q \right\}.$$

Si $\theta \geq Tol.$: PARAR; el problema es no acotado. Si no, ir al paso 5.**Paso 4** – *Determinación fila de pivotación.*Determinar la variable básica x_{j_p} que sale de la base: calcular θ :

$$\theta = \max \left\{ \max_{1 \leq i \leq m} \left\{ \frac{x_{j_i} - u_{j_i}}{y_i} : y_i > 0 \right\}, \max_{1 \leq i \leq m} \left\{ \frac{x_{j_i} - l_{j_i}}{y_i} : y_i < 0 \right\}, l_q - u_q \right\}.$$

Si $\theta \leq -Tol.$: PARAR; el problema es no acotado.**Paso 5** – *Pivotación.*Adaptar la solución y la matriz B : hacer

$$\begin{aligned} x_q &\leftarrow l_q + \theta \quad \text{ó} \quad u_q + \theta, \\ x_{j_i} &\leftarrow x_{j_i} - \theta y_i, \quad 1 \leq i \leq m, \\ B &\leftarrow B + (\mathbf{a}_q - \mathbf{a}_{j_p}) \mathbf{e}_p^T, & (\text{sólo si } \theta \neq u_q - l_q) \\ \mathbf{B} &\leftarrow \mathbf{B} \cup \{q\} \setminus \{j_p\}, & (\text{sólo si } \theta \neq u_q - l_q) \\ N_l &\leftarrow N_l \cup \{j_p\} \setminus \{q\}, & (\text{sólo si } \theta \neq u_q - l_q \text{ y } q \in N_l) \\ N_u &\leftarrow N_u \cup \{j_p\} \setminus \{q\}, & (\text{sólo si } \theta \neq u_q - l_q \text{ y } q \in N_u) \\ j_p &\leftarrow q, & (\text{sólo si } \theta \neq u_q - l_q). \end{aligned}$$

Ir al paso 1.

Ejemplo 7.10 Resolver:

$$\begin{array}{ll} \min. & -2x_1 - 4x_2 - x_3 \\ \text{s. a} & 2x_1 + x_2 + x_3 \leq 10 \\ & x_1 + x_2 - x_3 \leq 4 \\ & 0 \leq x_1 \leq 4 \\ & 0 \leq x_2 \leq 6 \\ & 1 \leq x_3 \leq 4. \end{array}$$

Se introducen como siempre las variables de holgura en las condiciones 1 y 2. El problema resulta:

$$\begin{array}{ll} \min. & -2x_1 - 4x_2 - x_3 \\ \text{s. a} & 2x_1 + x_2 + x_3 + x_4 = 10 \\ & x_1 + x_2 - x_3 + x_5 = 4 \\ & 0 \leq x_1 \leq 4 \\ & 0 \leq x_2 \leq 6 \\ & 1 \leq x_3 \leq 4 \\ & x_4, x_5 \geq 0. \end{array}$$

Como es inmediato obtener una solución factible y una base de partida, entramos directamente en la *fase II* del método *simplex*. Partiremos de:

$$B = [\mathbf{a}_4, \mathbf{a}_5] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad N_l = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3], \quad N_u = \emptyset$$

y de la solución inicial $\mathbf{x} = [0, 0, 1, 9, 5]^T$. La función objetivo en este punto es -1 .

Iteración 1. Paso 1

Calculemos los multiplicadores simplex:

$$\boldsymbol{\pi} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son:

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -2 - [0, 0] \begin{bmatrix} 2 \\ 1 \end{bmatrix} = -2,$$

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = -4 - [0, 0] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = -4$$

y

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = -1 - [0, 0] \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -1.$$

Todavía no se ha llegado al óptimo del problema pues las variables x_1 , x_2 y x_3 están en sus límites inferiores y sus costes reducidos son negativos.

Iteración 1. Paso 2

Se elige la variable x_2 para entrar en la base por ser la no básica de coste reducido más negativo. Se resuelve $B\mathbf{y} = \mathbf{a}_2$:

$$\mathbf{y} = B^{-1}\mathbf{a}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Iteración 1. Paso 3

La variable a salir de la base se determinará de calcular el incremento θ :

$$\theta = \min \left\{ \begin{array}{l} \theta_1 = \min \left\{ \frac{x_{B_1} - l_{B_1}}{y_1}, \frac{x_{B_2} - l_{B_2}}{y_2} \right\} = \left\{ \frac{9-0}{1}, \frac{5-0}{1} \right\} \\ u_2 - l_2 = 6 - 0 \end{array} \right\} = 5.$$

Saldrá, por tanto, la variable x_5 pues es la que antes llega a su límite inferior.

Iteración 1. Paso 4

Readaptemos la solución y las estructuras de datos:

$$\begin{aligned} x_2 &\leftarrow l_2 + \theta = 0 + 5 = 5 \\ x_4 &\leftarrow x_4 - \theta y_1 = 9 - 5 \cdot 1 = 4 \\ x_5 &\leftarrow x_5 - \theta y_2 = 5 - 5 \cdot 1 = 0. \end{aligned}$$

La nueva solución es $\mathbf{x} = [0, 5, 1, 4, 0]^T$, con

$$B = [\mathbf{a}_4, \mathbf{a}_2] = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad N_l = [\mathbf{a}_1, \mathbf{a}_3, \mathbf{a}_5] \quad \text{y} \quad N_u = \emptyset.$$

La nueva función objetivo es -21 .

Iteración 2. Paso 1

Calculemos los multiplicadores simplex:

$$\boldsymbol{\pi} = B^{-T}\mathbf{c}_B = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ -4 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -4 \end{bmatrix} = \begin{bmatrix} 0 \\ -4 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son:

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -2 - [0, -4] \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 2,$$

$$\bar{c}_3 = c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = -1 - [0, -4] \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -5$$

y

$$\bar{c}_5 = c_5 - \boldsymbol{\pi}^T \mathbf{a}_5 = 0 - [0, -4] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 4.$$

Como la variable x_3 está en su límite inferior y su coste reducido es negativo, todavía no se ha llegado al óptimo del problema.

Iteración 2. Paso 2

Se elige la variable x_3 para entrar en la base. Se resuelve $B\mathbf{y} = \mathbf{a}_3$:

$$\mathbf{y} = B^{-1}\mathbf{a}_3 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}.$$

Iteración 2. Paso 3

La variable a salir de la base se determinará de calcular el incremento θ :

$$\theta = \min \left\{ \begin{array}{l} \theta_1 = \min \left\{ \frac{x_{B_1} - l_{B_1}}{y_1} \right\} = \left\{ \frac{4 - 0}{2} \right\} \\ \theta_2 = \min \left\{ \frac{x_{B_2} - u_{B_2}}{y_2} \right\} = \left\{ \frac{5 - 6}{-1} \right\} \\ u_3 - l_3 = 4 - 1 \end{array} \right\} = 1.$$

Saldrá, por tanto, la variable x_2 pues es la que antes llega a uno de sus límites: el superior.

Iteración 2. Paso 4

Readaptemos la solución y las estructuras de datos:

$$\begin{aligned} x_3 &\leftarrow l_3 + \theta = 1 + 1 = 2 \\ x_4 &\leftarrow x_4 - \theta y_1 = 4 - 1 \cdot 2 = 2 \\ x_2 &\leftarrow x_2 - \theta y_2 = 5 + 1 \cdot 1 = 6. \end{aligned}$$

La nueva solución es $\mathbf{x} = [0, 6, 2, 2, 0]^T$, con

$$B = [\mathbf{a}_4, \mathbf{a}_3] = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad N_l = [\mathbf{a}_1, \mathbf{a}_5] \quad \text{y} \quad N_u = [\mathbf{a}_2].$$

La nueva función objetivo es -26 .

Iteración 3. Paso 1

Calculemos los multiplicadores simplex:

$$\boldsymbol{\pi} = B^{-T}\mathbf{c}_B = \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son:

$$\bar{c}_1 = c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -2 - [0, 1] \begin{bmatrix} 2 \\ 1 \end{bmatrix} = -3,$$

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = -4 - [0, 1] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = -5$$

y

$$\bar{c}_5 = c_5 - \boldsymbol{\pi}^T \mathbf{a}_5 = 0 - [0, 1] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = -1.$$

Como las variables x_1 y x_5 , que están en sus límites inferiores, tienen costes reducidos negativos, todavía no se ha llegado al óptimo del problema.

Iteración 3. Paso 2

Se elige la variable x_1 para entrar en la base. Se resuelve $B\mathbf{y} = \mathbf{a}_1$:

$$\mathbf{y} = B^{-1}\mathbf{a}_1 = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}.$$

Iteración 3. Paso 3

La variable a salir de la base se determinará de calcular el incremento θ :

$$\theta = \min \left\{ \begin{array}{l} \theta_1 = \min \left\{ \frac{x_{B_1} - l_{B_1}}{y_1} \right\} = \left\{ \frac{2 - 0}{3} \right\} \\ \theta_2 = \min \left\{ \frac{x_{B_2} - u_{B_2}}{y_2} \right\} = \left\{ \frac{2 - 4}{-1} \right\} \\ u_1 - l_1 = 4 - 0 \end{array} \right\} = \frac{2}{3}.$$

Saldrá la variable x_4 pues es la que antes llega a uno de sus límites: el inferior.

Iteración 3. Paso 4

Readaptemos la solución y las estructuras de datos:

$$\begin{aligned} x_1 &\leftarrow l_1 + \theta = 0 + 2/3 = 2/3 \\ x_4 &\leftarrow x_4 - \theta y_1 = 2 - (2/3) \cdot 3 = 0 \\ x_3 &\leftarrow x_3 - \theta y_2 = 2 + (2/3) \cdot 1 = 8/3. \end{aligned}$$

La nueva solución es:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 6 \\ 8/3 \\ 0 \\ 0 \end{bmatrix},$$

con

$$B = [\mathbf{a}_1, \mathbf{a}_3] = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \quad N_l = [\mathbf{a}_4, \mathbf{a}_5] \quad \text{y} \quad N_u = [\mathbf{a}_2].$$

La nueva función objetivo es -28 .

Iteración 4. Paso 1

Calculemos los multiplicadores simplex:

$$\boldsymbol{\pi} = B^{-T}\mathbf{c}_B = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}^{-1} \begin{bmatrix} -2 \\ -1 \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} \begin{bmatrix} -2 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son:

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = -4 - [-1, 0] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = -3,$$

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [-1, 0] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$$

y

$$\bar{c}_5 = c_5 - \boldsymbol{\pi}^T \mathbf{a}_5 = 0 - [-1, 0] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.$$

Los costes reducidos de las variables no básicas en sus límites inferiores son cero y -3 , y el de la variable en su límite superior 1 . Se ha llegado, por tanto, a un óptimo del problema, aunque no es el único. En efecto, si se incrementa x_5 , manteniendo $x_2 = 0$ y $x_4 = 6$ fijos, las variables x_1 y x_3 se modificarán de la forma:

$$\begin{bmatrix} x_1 \\ x_3 \end{bmatrix} = B^{-1} \mathbf{b} - B^{-1} \mathbf{a}_5 x_5 = \begin{bmatrix} 2/3 \\ 8/3 \end{bmatrix} - \begin{bmatrix} 1/3 \\ -2/3 \end{bmatrix} x_5.$$

Para cualquier $x_5 < 2$, la solución

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 2/3 - 1/3 x_5 \\ 6 \\ 8/3 + 2/3 x_5 \\ 0 \\ 0 \end{bmatrix}$$

es óptima. El valor de la función objetivo óptimo es -28 . ●

7.7 Complejidad computacional del método simplex

La complejidad computacional del método *simplex* depende del número de iteraciones necesarias para resolver un problema y del número de operaciones que requiere cada iteración. No es difícil deducir que, tal como la hemos estudiado en este capítulo, cada iteración de su versión revisada requiere alrededor de $m(n-m) + (m+1)^2$ multiplicaciones y $m(n+1)$ sumas o restas. Es decir del $O(mn)$ en los dos casos.

Por lo que se refiere al número de iteraciones necesarias, como el método va de un punto extremo a otro del politopo que definen las condiciones, dado que el número de puntos extremos de un programa lineal en forma estándar con n variables y m condiciones es $C(n, m)$, se tiene que

$$C(n, m) = \frac{n!}{m!(n-m)!} \geq \left(\frac{n}{m}\right)^m \geq 2^m,$$

siempre y cuando $n \geq 2m$. De acuerdo con esto, en el peor de los casos, parece lógico temerse un número de iteraciones exponencial. Este temor lo confirman ejemplos especialmente diseñados para analizar el comportamiento del método en las peores circunstancias posibles.

Un problema clásico del mal comportamiento del método *simplex* en su resolución es el estudiado por Klee y Minty [1972]:

$$\begin{array}{ll} \text{maximizar} & x_n \\ \text{s. a} & 0 \leq x_1 \leq 1 \\ & \delta x_{i-1} \leq x_i \leq 1 - \delta x_{i-1}, \quad i = 2, \dots, n \\ & x_i \geq 0, \quad i = 1, 2, \dots, n, \end{array}$$

para $0 < \delta < 1/2$. Este problema tiene 2^n puntos extremos.

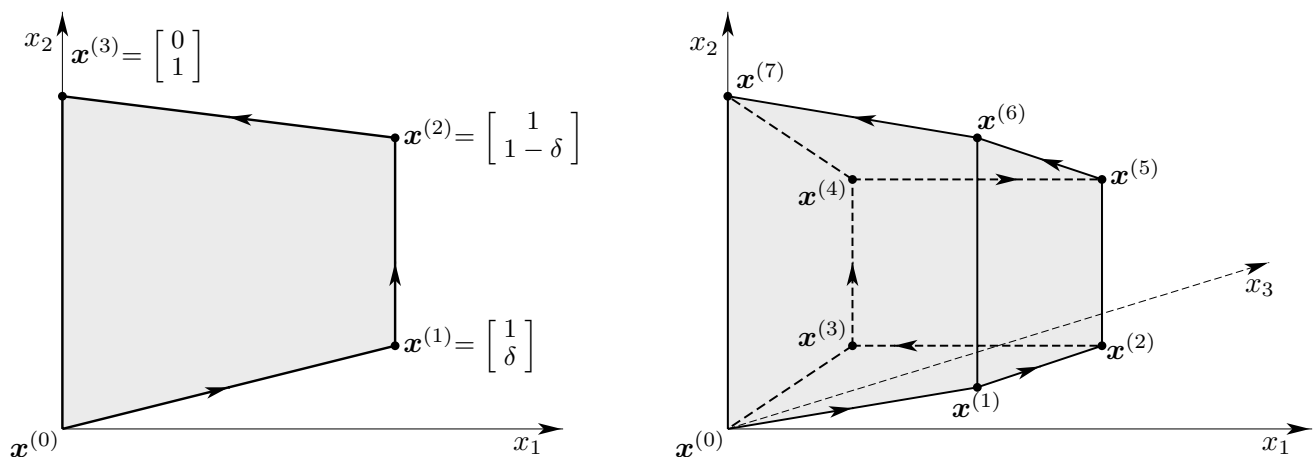


Figura 7.7

Búsqueda de la solución del problema de Klee y Minty para $n = 2$ y $n = 3$

Comenzando desde el punto $x = 0$, el cual es obviamente factible, trabajando con precisión absoluta, el método *simplex* recorrería todos y cada uno de los 2^n puntos extremos. La figura 7.7 describe este problema para $n = 2$ y $n = 3$.

Aunque se pueden definir diversas variantes del método *simplex* para evitar que en el problema de Klee y Minty se recorran todos los puntos extremos, lo cierto es que se pueden construir otros ejemplos en los que esas variantes también fallan. Lo que es contrastable, en cualquier caso, es la *complejidad exponencial teórica* del método *simplex*.

La práctica de todos los días de los últimos cincuenta años en que se ha utilizado masivamente el método *simplex* también ha permitido constatar que los casos extremos, como el ejemplo de Klee y Minty, rara vez ocurren en problemas del mundo real. Esa misma práctica también ha permitido concluir que, *grosso modo*, en problemas de tamaño moderado, el método *simplex* requiere un número de iteraciones real comprendido entre $4m$ y $6m$. En problemas grandes o muy grandes, el número de iteraciones que cabe esperar llevar a cabo para resolverlos es $\alpha \cdot m$, donde $e^\alpha < \log_2(2 + n/m)$. Experimentos basados en técnicas de simulación Monte Carlo de distribuciones de probabilidad artificiales han confirmado conclusiones similares. En resumen, la *complejidad computacional práctica* esperada del método *simplex* es del $O(m^2n)$.

Teniendo también en cuenta el efecto que el tratamiento de la dispersidad de los problemas puede acarrear en la complejidad del método, diversos estudios han llegado a establecer que la complejidad del método la define la ecuación $Km^\alpha nd^{0.33}$, donde K es una constante, $1.25 < \alpha < 2.5$ y d es el número de elementos distintos de cero en la matriz de coeficientes de las condiciones, A , dividido por nm .

Todas estas consideraciones avalan la gran eficacia del método *simplex* aunque su complejidad computacional teórica sea exponencial.

Referencias

Existen diversas formas de exponer el método *simplex*; en este capítulo hemos escogido aquella que creemos mejor ilustra, desde un punto de vista algebraico, qué es lo que se hace en el

método, su porqué y cómo llevarlo a la práctica en ordenador. Para ello se ha seguido a Bazaraa y Jarvis [1977], Bazaraa, Jarvis y Sherali [1990], Goldfarb y Todd [1989], Luenberger [1984] y Schrijver [1986]. También recomendamos Chvátal [1983], Gill, Murray y Wright [1991] y Murty [1983], Fang [1993] y Padberg [1995].

Bastantes de las consideraciones sobre la implementación práctica en ordenador del método *simplex* están basadas en la experiencia del autor al respecto. El programa BBMI listado en el apéndice E es parte del fruto de alguna de ellas.

Ejercicios

7.1. Considérese el problema de programación lineal

$$\text{minimizar } \{ \mathbf{c}^T \mathbf{x} : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0} \},$$

donde

$$A = \begin{bmatrix} 6 & 1 & 0 & 0 & 1 & 5 \\ 2 & 3 & 1 & 4 & 0 & 2 \\ -1 & 1 & 0 & 2 & 2 & 1 \end{bmatrix} \quad \text{y} \quad \mathbf{b} = \begin{bmatrix} 9 \\ 17 \\ 13 \end{bmatrix}.$$

a) Determinar la solución básica factible asociada a la base

$$B = \begin{bmatrix} 1 & 0 & 1 \\ 3 & 1 & 0 \\ 1 & 0 & 2 \end{bmatrix}.$$

b) Estudiar para cuál de los vectores $\mathbf{c}$ que se indican a continuación esta base es óptima:

$$1. \mathbf{c}_1 = [-5, -2, -1, 2, -3, -10]^T.$$

$$2. \mathbf{c}_2 = [30, 1, 0, -5, -2, 20]^T.$$

$$3. \mathbf{c}_3 = [-10, -1, 1, 6, -3, -15]^T.$$

¿Existen óptimos alternativos?

7.2. Resolver los siguientes problemas de programación lineal usando el método simplex de la tabla 7.1, partiendo de la base indicada.

$$\begin{aligned} \text{a) minimizar } & -x_1 - x_2 \\ \text{s. a } & x_1 + 3x_2 + x_3 = 9 \\ & 2x_1 + x_2 + x_4 = 8 \\ & x_1, \dots, x_4 \geq 0. \end{aligned}$$

Base inicial:

$$B = [\mathbf{a}_2, \mathbf{a}_4] = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 1/3 & 0 \\ -1/3 & 1 \end{bmatrix}.$$

$$\begin{aligned} \text{b) minimizar } & -x_1 - x_2 \\ \text{s. a } & x_1 + x_2 - x_3 = 1 \\ & x_1, \dots, x_3 \geq 0. \end{aligned}$$

Base inicial:

$$B = [\mathbf{a}_1] = [1], \quad B^{-1} = [1].$$

c) minimizar $-x_1$

$$\begin{array}{rclcl} \text{s. a} & 2x_1 + 3x_2 + x_3 & & & = 21 \\ & -x_1 + x_2 & + x_4 & & = 2 \\ & x_1 - x_2 & & + x_5 & = 3 \\ & & & & x_1, \dots, x_5 \geq 0. \end{array}$$

Base inicial:

$$B = [\mathbf{a}_3, \mathbf{a}_4, \mathbf{a}_5] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

d) minimizar $x_1 + 15x_2 - 3x_3 + 20x_4 - x_5 + 3x_6 + 25x_7$

$$\begin{array}{rclcl} \text{s. a} & 2x_1 + x_2 + x_3 & + x_5 - x_6 & & = 0 \\ & -x_1 & + x_3 & - x_5 + 2x_6 + x_7 & = 6 \\ & -2x_1 & + 2x_3 + x_4 + x_5 + 3x_6 & & = 9 \\ & & & & x_1, \dots, x_7 \geq 0. \end{array}$$

Base inicial:

$$B = [\mathbf{a}_2, \mathbf{a}_7, \mathbf{a}_4] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

e) minimizar $-14x_1 - 18x_2 - 16x_3 - 80x_4$

$$\begin{array}{rclcl} \text{s. a} & 4,5x_1 + 8,5x_2 + 6x_3 + 20x_4 + x_5 & & & = 6000 \\ & x_1 + x_2 + 4x_3 + 40x_4 & + x_6 & & = 4000 \\ & & & & x_1, \dots, x_6 \geq 0. \end{array}$$

Base inicial:

$$B = [\mathbf{a}_5, \mathbf{a}_6] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

7.3. Resolver los siguientes problemas de programación lineal usando el método simplex completo (fases I y II).

a) minimizar $10x_1 + x_2 - 7x_3 - 5x_4 - x_5$

$$\begin{array}{rclcl} \text{s. a} & x_1 - x_2 & + x_4 + x_5 & = 1 \\ & x_2 + x_3 + 2x_4 + 2x_5 & = 7 \\ & -x_2 & + 3x_4 - x_5 & = 4 \\ & & & & x_1, \dots, x_5 \geq 0. \end{array}$$

b) minimizar $x_1 - 4x_2 + x_3 + 13x_4 + 23x_5$

$$\begin{array}{rclcl} \text{s. a} & 2x_1 + x_2 + x_3 + 4x_4 - x_5 & = 6 \\ & x_1 + 2x_2 + x_3 + x_4 - 4x_5 & = 3 \\ & & & & x_1, \dots, x_5 \geq 0. \end{array}$$

c) minimizar $9x_1 + x_2 - 10x_3 + 10x_4$

$$\begin{array}{rclcl} \text{s. a} & 5x_1 + 2x_2 + 3x_3 + 60x_4 + x_5 & & & = 10 \\ & 3x_1 + x_2 - 2x_3 + x_4 & & & = 2 \\ & x_1 - 4x_2 + 7x_3 + 4x_4 & + x_6 & = 1 \\ & 2x_2 - 5x_3 - 3x_4 & & & = 5 \\ & & & & x_1, \dots, x_6 \geq 0. \end{array}$$

7.4. Considérese el problema de programación lineal

$$\text{minimizar } \{c^T \mathbf{x} : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\},$$

donde

$$A = \begin{bmatrix} 6 & 1 & 0 & 0 & 1 & -5 \\ 2 & 3 & 1 & 4 & 0 & -10 \\ -1 & 1 & 0 & 2 & 2 & -7 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 9 \\ 17 \\ 13 \end{bmatrix} \quad \text{y} \quad \mathbf{c} = \begin{bmatrix} -5 \\ -1 \\ 1 \\ 10 \\ -3 \\ -25 \end{bmatrix}.$$

Probar que no tiene solución óptima. ¿Qué ocurre si el último componente de $\mathbf{c}$ se cambia por un 20?

7.5. Considérese el problema de programación lineal

$$\begin{array}{ll} \text{minimizar} & -x_1 - x_2 \\ \text{s. a} & -x_1 + x_2 \leq 3 \\ & -x_1 + 3x_2 \leq 13 \\ & 2x_1 + x_2 \leq 16 \\ & 2x_1 - 3x_2 \leq 8 \\ & -2x_1 - 3x_2 \leq -6 \\ & x_1, x_2 \geq 0. \end{array}$$

- Escribir el problema en forma estándar.
- Resolverlo utilizando el método *simplex* completo (fases I y II).

7.6. Resolver los problemas del ejercicio 2 usando la forma de *tableau*.

7.7. Supóngase que, aplicando el método *simplex* revisado estudiado, la variable i -ésima del vector $\mathbf{x}$ sale de la base en una determinada iteración j . Probar que no puede entrar en la base en la iteración $j + 1$.

7.8. Sean $\mathbf{x}_B$ y $\mathbf{x}_{\hat{B}}$ los vectores básicos de $\mathbf{x}$ correspondientes a dos iteraciones sucesivas del método *simplex* revisado. Supóngase que todos los componentes de $\mathbf{x}_B$ son estrictamente positivos. Determinar una condición suficiente para que también lo sean los de $\mathbf{x}_{\hat{B}}$.

7.9. Considérese el modelo siguiente de problema de programación lineal:

$$\text{minimizar} \{ \mathbf{c}_1^T \mathbf{x}_1 + \mathbf{c}_2^T \mathbf{x}_2 : A_1 \mathbf{x}_1 + A_2 \mathbf{x}_2 = \mathbf{b}, \mathbf{x}_2 \geq \mathbf{0} \}.$$

Obsérvese que este modelo difiere del presentado en el capítulo en que $\mathbf{x}_1$ no está restringido a tomar valores no negativos.

- Modificar el método *simplex* revisado de la tabla 7.1 para tratar problemas de este tipo.
- ¿Cuáles serían las condiciones de óptimo a comprobar en el paso 1?

7.10. Supóngase que $y_l = (B^{-1} \mathbf{a}_q)_l = 0$ (tabla 7.1, paso 2). Probar que la condición l -ésima es redundante.

7.11. Si en lugar de escoger como variable no básica a entrar en la base aquella cuyo coste reducido es el más negativo, se eligiese aquella que produjese un mayor decremento en la función objetivo, ¿cuál habría de ser el criterio para determinar x_q en el paso 2 de la tabla 7.1?

7.12. Probar que si en la fase I del método *simplex* una variable artificial pasa a ser no básica, no será necesario volverla a hacer de nuevo básica.

7.13. Utilizar el método de la gran M para resolver el siguiente programa lineal:

$$\begin{array}{ll} \text{minimizar} & 3x_1 + x_3 \\ \text{s. a} & x_1 + 2x_2 + x_3 + x_4 = 10 \\ & x_1 - 2x_2 + 2x_3 = 6 \\ & 0 \leq x_1 \leq 4 \\ & 0 \leq x_2 \leq 4 \\ & 0 \leq x_3 \leq 4 \\ & 0 \leq x_4 \leq 12. \end{array}$$

7.14. Resolver el problema anterior mediante el método *simplex* en forma de tableau.

7.15. Indicar si el siguiente aserto es verdadero o falso: si en el método de la gran M este parámetro se escoge extremadamente grande, el método *simplex* en dos fases y el de la gran M producirán la misma sucesión de bases.

7.16. Usando el método *simplex* revisado, encontrar una solución básica factible de las siguientes condiciones

$$\begin{array}{rcl} x_1 + 2x_2 - x_3 + x_4 & = & 3 \\ 2x_1 + 4x_2 + x_3 + 2x_4 & = & 12 \\ x_1 + 4x_2 + 2x_3 + x_4 & = & 9 \\ x_1, \dots, x_4 & \geq & 0. \end{array}$$

7.17. En muchos problemas prácticos donde se aplica programación lineal es suficiente llegar a un punto tal que el valor de la función objetivo en él esté dentro de una tolerancia ε del óptimo z^* ; esto ahorra una considerable cantidad de cálculos y, por consiguiente, tiempo.

- a) Considérese un problema de programación lineal del cual se sabe que la suma de las variables está acotada superiormente por s . Si en una iteración determinada del método simplex aplicado a ese problema la función objetivo toma un valor z_0 , los costes reducidos de las variables no básicas son $\bar{c}_j$ y

$$M = \min_j \{\bar{c}_j\},$$

probar que si $M \leq \varepsilon/s$, se cumple que $z_0 - z^* \leq \varepsilon$.

- b) Aplicar este criterio a cualquiera de los problemas del ejercicio 3 con $\varepsilon = 0,8$. ¿Qué ocurre?

7.18. Considérese el siguiente programa lineal

$$\begin{array}{ll} \text{maximizar} & 3x_1 + 4x_2 \\ \text{s. a} & 2x_1 + x_2 \leq 8 \\ & -x_1 + 2x_2 \leq 6 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0. \end{array}$$

- a) Resolver el problema geoméricamente comprobando que el punto óptimo es degenerado.
b) Resolver el problema mediante el método *simplex*.
c) Identificar la condición que causa la degeneración y resolver el problema otra vez sin esa condición.
d) ¿Se puede probar que las soluciones básicas degeneradas se pueden evitar prescindiendo de determinadas condiciones sin afectar a la región factible?

Capítulo 8

DUALIDAD Y ANÁLISIS DE SENSIBILIDAD

EN ESTE CAPÍTULO se introduce un concepto de extraordinaria utilidad en programación matemática pues contribuye en gran medida a su comprensión: la dualidad. La idea que subyace en él es la de proporcionar formulaciones alternativas a los problemas de optimización a fin de facilitar su resolución numérica y aumentar la comprensión teórica de los métodos empleados para su tratamiento. Veremos que todo programa lineal tiene íntimamente asociado a él otro programa lineal denominado *dual*. Estudiaremos la interpretación en términos económicos de un programa lineal a través de las variables del dual, así como la forma de resolver el programa lineal original a partir de su formulación dual mediante el método dual del *simplex*, y cómo éste puede usarse para resolver problemas que resultan de uno lineal cambiando los límites de las variables, añadiendo condiciones, etc.

Hasta ahora nos hemos preocupado de cómo formular modelos de programación lineal, o mejor dicho, de cómo surgen los programas lineales a partir de la modelización matemática de fenómenos económicos, físicos, etc, y cómo resolver esos programas lineales. En este capítulo también nos ocuparemos de analizar la sensibilidad de la solución de los programas lineales a variaciones en la formulación de los mismos. Los parámetros que definen un programa lineal —los c_j , a_{ij} , b_i , etc.— son a menudo el reflejo de una serie de estimaciones, muchas veces subjetivas, realizadas con instrumentos sujetos a fallos. El entorno, por otro lado, que conforma el programa lineal, suele estar influido por circunstancias muy dinámicas: las demandas, los precios de materias primas, la disponibilidad de recursos productivos, etc. La solución óptima de un programa lineal sólo se refiere a una situación y conjunto de datos concretos; es importante por tanto investigar los posibles efectos que en esa solución introducen diversas modificaciones de los parámetros que definen el problema.

Lo que se pretende con los análisis de sensibilidad es responder a preguntas concretas como: ¿de qué forma habrá que modificar nuestra cadena de producción los próximos seis meses si el beneficio de nuestro producto estrella cae un 20%? o, dada nuestra disponibilidad de

recursos, ¿qué beneficio o cuántas unidades deberíamos vender de un nuevo producto para hacerlo rentable? Las respuestas a estas y muchas otras preguntas se pueden obtener mediante el análisis de sensibilidad que proporciona el estudio del programa dual de uno lineal.

Ejemplo 8.1 Una determinada empresa produce $b_1, b_2, \dots, b_m$ cantidades de m productos. Para producirlos, la empresa puede realizar cualquiera de n actividades a distintos niveles. Cada actividad j tiene un coste unitario c_j . Un determinado comprador contrata toda la producción deseando además tener control sobre las operaciones de la empresa de tal manera que pueda especificar las combinaciones y niveles de sus actividades a fin de minimizar el coste total de esa producción. Si a_{ij} representa la cantidad del producto i producido por una unidad de la actividad j , $\sum_{j=1}^n a_{ij}x_j$ representa las unidades que se producen de i . Estas unidades deben ser mayores o iguales que la cantidad requerida b_i . El problema que desea resolver el comprador es el siguiente:

$$\begin{aligned} \min. \quad & \sum_{j=1}^n c_j x_j \\ \text{s. a} \quad & \sum_{j=1}^n a_{ij} x_j \geq b_i, \quad i = 1, 2, \dots, m, \\ & x_j \geq 0 \quad j = 1, 2, \dots, n. \end{aligned} \tag{8.1}$$

En vez de tratar de controlar las operaciones de la empresa a la que compra los productos, supóngase que el comprador está de acuerdo en pagar a la empresa precios por unidad $y_1, y_2, \dots, y_m$ de cada uno de los m productos. Si a_{ij} es el número de unidades del producto i producidas por una unidad de la actividad j e y_i el precio por unidad del producto i , $\sum_{i=1}^m a_{ij}y_i$ expresa el precio por unidad de la actividad j de acuerdo con los precios $y_1, y_2, \dots, y_m$. De lo que se trata pues es de que el precio implícito de la actividad j , o sea $\sum_{i=1}^m a_{ij}y_i$, no exceda del precio real c_j : esto quiere decir que la empresa debe cumplir las restricciones $\sum_{i=1}^m a_{ij}y_i \leq c_j$, para $j = 1, 2, \dots, n$. Dentro de estas restricciones, la empresa seleccionará aquellos precios que maximicen su ganancia $\sum_{i=1}^m y_i b_i$. La empresa, en resumen, se plantea el siguiente problema:

$$\begin{aligned} \max. \quad & \sum_{i=1}^m y_i b_i \\ \text{s. a} \quad & \sum_{i=1}^m a_{ij} y_i \leq c_j, \quad j = 1, 2, \dots, n, \\ & y_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

Este último problema se conoce como el *programa dual* del planteado en (8.1). ●

8.1 Dualidad y condiciones de óptimo

Recordemos que según el teorema 7.2, de la página 418, y su primer corolario (corolario 7.1), si una solución básica $\mathbf{x}$ de un programa lineal es no degenerada, para que sea óptima es necesario y suficiente que todos los costes reducidos, $\bar{c}_j$, sean no negativos. Esta conclusión, en el caso de solución básica factible no degenerada, surge del hecho de que si y sólo si algún

coste reducido $\bar{c}_j$ es negativo se puede usar el método *simplex* para obtener otra solución básica factible que mejore el valor de la función objetivo. Cuando la solución $\mathbf{x}$ es básica factible no degenerada, como se desprende del siguiente resultado, también se pueden determinar de otra manera condiciones necesarias y suficientes para que sea óptima.

Teorema 8.1 *La solución básica factible no degenerada*

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_B \\ \mathbf{x}_N \end{bmatrix} = \begin{bmatrix} B^{-1}\mathbf{b} \\ \mathbf{0} \end{bmatrix} \quad (8.2)$$

del programa lineal

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned} \quad (8.3)$$

es óptima si y sólo si

$$\mathbf{c}^T = [\mathbf{y}^T, \bar{\mathbf{w}}^T] \begin{bmatrix} B & N \\ 0 & I \end{bmatrix}, \quad (8.4)$$

donde $\bar{\mathbf{w}} \geq \mathbf{0}$.

DEMOSTRACIÓN. Recordemos que los vectores fila de la matriz

$$M = \begin{bmatrix} B & N \\ 0 & I \end{bmatrix}$$

son linealmente independientes. Al serlo, forman una base de $\Re^n$ por lo que existe un único vector $[\mathbf{y}^T, \bar{\mathbf{w}}^T]$ que satisface (8.4).

Para completar la demostración obsérvese que $\bar{\mathbf{w}}$ es el vector de costes reducidos de las variables no básicas, $\bar{\mathbf{c}}_N$. En efecto,

$$\begin{aligned} [\mathbf{y}^T, \bar{\mathbf{w}}^T] &= \mathbf{c}^T M^{-1} = [\mathbf{c}_B^T, \mathbf{c}_N^T] \begin{bmatrix} B^{-1} & -B^{-1}N \\ 0 & I \end{bmatrix} \\ &= [\mathbf{c}_B^T B^{-1}, \mathbf{c}_N^T - \mathbf{c}_B^T B^{-1}N]. \end{aligned}$$

Por tanto, si la solución es óptima, $\bar{\mathbf{w}}$ es $\geq \mathbf{0}$. Recuérdese que $\mathbf{y}$ es el vector de *multiplicadores simplex*, $\boldsymbol{\pi}$, que se calculan en el método *simplex*.

El aserto del teorema “si” es siempre cierto aun cuando la solución (8.2) sea degenerada. ■

En términos geométricos, este último teorema viene a poner de manifiesto que en un punto extremo óptimo no degenerado, $\mathbf{x}$, del politopo que definen las condiciones de un programa lineal, el vector gradiente de la función objetivo, $\mathbf{c}$, se puede expresar como una combinación lineal de los gradientes de las restricciones de igualdad, más una no negativa de los gradientes hacia dentro de las restricciones de no negatividad que se satisfacen estrictamente. Dicho de otra forma, que el gradiente de la función objetivo está en el cono generado por los gradientes de las condiciones de igualdad.

Ejemplo 8.2 Sea el problema:

$$\begin{aligned} \min. \quad & -x_1 - 3x_2 \\ \text{s. a} \quad & -x_1 - x_2 = -4 \\ & x_1, x_2 \geq 0. \end{aligned}$$

Los vectores gradiente de la función objetivo y de las condiciones son

$$\mathbf{c} = \begin{bmatrix} -1 \\ -3 \end{bmatrix}, \quad \mathbf{a}^1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \quad \mathbf{a}^2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{y} \quad \mathbf{a}^3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Consideremos los dos puntos extremos de la figura 8.1:

- En el punto extremo $\mathbf{x} = [4, 0]^T$ se cumple la restricción de no negatividad $x_2 \geq 0$. El vector $\mathbf{c}$ no pertenece al cono que generan los gradientes de la condición de igualdad, $-x_1 - x_2 = -4$ y de $x_2 = 0$: este punto no es óptimo.
- En $\mathbf{x} = [0, 4]^T$, por el contrario (se cumple $x_1 = 0$), el vector $\mathbf{c}$ sí pertenece al cono que generan $\mathbf{a}^1$ y $\mathbf{a}^2$: este punto es el óptimo.

•

Consideremos ahora el problema de programación lineal,

$$\begin{aligned} \max. \quad & \mathbf{b}^T \mathbf{y} \\ \text{s. a} \quad & \mathbf{A}^T \mathbf{y} \leq \mathbf{c}, \end{aligned} \tag{8.5}$$

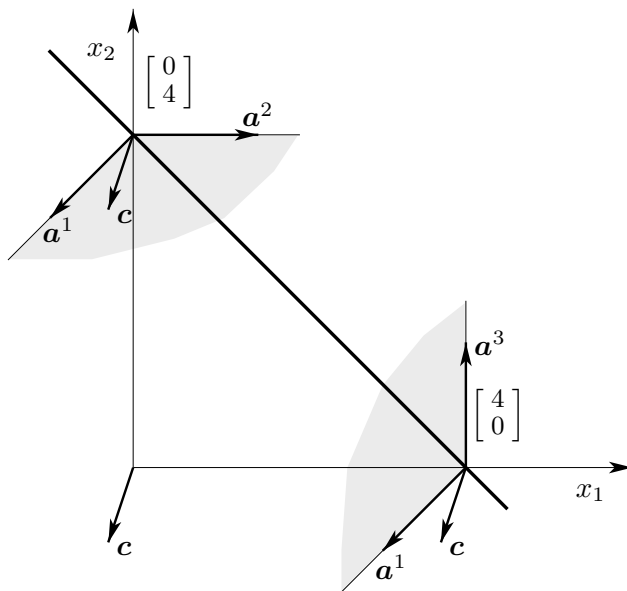


Figura 8.1

Geometría de las condiciones de óptimo del ejemplo 8.2

relacionado con el programa lineal general en forma estándar,

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{8.6}$$

A (8.5) se le denomina *programa dual* de (8.6), al que a partir de ahora denominaremos *primal*. Obsérvese que en el programa o problema dual intervienen los mismos parámetros A , $\mathbf{b}$ y $\mathbf{c}$ del primal y que, en alguna medida, el dual es la versión traspuesta del primal en la que *minimizar* se sustituye por *maximizar*. Formulando (8.5) en forma estándar se puede probar fácilmente el siguiente lema.

Lema 8.1 *El dual del problema dual*

$$\begin{aligned} \max. \quad & \mathbf{b}^T \mathbf{y} \\ \text{s. a} \quad & A^T \mathbf{y} \leq \mathbf{c}, \\ \text{es el problema primal} \quad & \\ \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{8.7}$$

El problema dual surge inmediatamente de la aplicación de las condiciones de óptimo del teorema 8.1 al problema (8.7). En efecto, esas condiciones se pueden escribir de la forma $\mathbf{c}^T = \mathbf{y}^T A + \mathbf{w}^T$, donde $\mathbf{w} = [\mathbf{0}^T, \bar{\mathbf{w}}^T]^T \geq \mathbf{0}$. Obviando la necesidad de que los primeros m componentes de $\mathbf{w}$ han de ser cero, se tiene que

$$A^T \mathbf{y} + \mathbf{w} = \mathbf{c}, \quad \mathbf{w} \geq \mathbf{0},$$

que no son sino las condiciones del problema dual (8.5) puestas en forma de igualdad mediante la introducción de las variables de holgura $\mathbf{w} \geq \mathbf{0}$.

Lema 8.2 (*Dualidad débil*) *Si $\mathbf{x}$ es una solución factible del problema primal e $\mathbf{y}$ una también factible del dual, $\mathbf{b}^T \mathbf{y} \leq \mathbf{c}^T \mathbf{x}$.*

DEMOSTRACIÓN. Como $A\mathbf{x} = \mathbf{b}$, $\mathbf{y}^T A\mathbf{x} = \mathbf{y}^T \mathbf{b}$ para todo $\mathbf{y} \in \mathbb{R}^m$. Como además $A^T \mathbf{y} \leq \mathbf{c}$ y $\mathbf{x} \geq \mathbf{0}$, $\mathbf{y}^T A\mathbf{x} \leq \mathbf{c}^T \mathbf{x}$. La combinación de estos resultados concluye la demostración. ■

Este último lema expresa que el valor de la función objetivo correspondiente a una solución factible del programa primal (dual) proporciona un límite superior (inferior) del valor de la función objetivo de cualquier solución factible —incluida la óptima— del otro problema. Consecuencia inmediata de este lema es el siguiente corolario.

Corolario 8.1 *Si $\mathbf{x}$ es una solución factible del problema primal, $\mathbf{y}$ una también factible del dual y $\mathbf{c}^T \mathbf{x} = \mathbf{b}^T \mathbf{y}$, entonces $\mathbf{x}$ e $\mathbf{y}$ son las soluciones óptimas del primal y dual, respectivamente.*

¿Existen soluciones factibles $\mathbf{x}$ e $\mathbf{y}$ que satisfagan las hipótesis de este último corolario? La respuesta a esta pregunta se obtiene del siguiente teorema.

Teorema 8.2 (Teorema de la dualidad de la programación lineal) (a) Si el problema primal o el dual tiene una solución óptima finita la tiene el otro y $\min. \mathbf{c}^T \mathbf{x} = \max. \mathbf{b}^T \mathbf{y}$.
 (b) Si el problema primal o el dual tiene una función objetivo no acotada el otro no tiene solución factible.

DEMOSTRACIÓN. De acuerdo con el resultado del lema 8.1 y el corolario 8.1, para demostrar la parte (a) del teorema sólo se necesita encontrar una solución primal óptima (finita) $\mathbf{x}$ y una dual factible $\mathbf{y}$ que satisfagan $\mathbf{c}^T \mathbf{x} = \mathbf{b}^T \mathbf{y}$. Sea $\mathbf{x}$ una solución básica factible óptima, por ejemplo,

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_B \\ \mathbf{x}_N \end{bmatrix} = \begin{bmatrix} B^{-1} \mathbf{b} \\ \mathbf{0} \end{bmatrix},$$

obtenida por el método *simplex* e $\mathbf{y}$ el vector $\boldsymbol{\pi}$ correspondiente de multiplicadores simplex; es decir, $\mathbf{y} = B^{-T} \mathbf{c}_B$. Este $\mathbf{y}$ es una solución factible del problema dual ya que

$$\mathbf{c} - A^T \mathbf{y} = \begin{bmatrix} \mathbf{c}_B \\ \mathbf{c}_N \end{bmatrix} - \begin{bmatrix} B^T \\ N^T \end{bmatrix} \boldsymbol{\pi} = \begin{bmatrix} 0 \\ \bar{\mathbf{c}}_N \end{bmatrix} \geq \mathbf{0}.$$

Además

$$\mathbf{c}^T \mathbf{x} = \mathbf{c}_B^T B^{-1} \mathbf{b} = \mathbf{y}^T \mathbf{b},$$

por lo que queda demostrada la parte (a).

La parte (b) del teorema es una conclusión inmediata del lema 8.2. ■

La demostración anterior indica que el vector de multiplicadores simplex correspondiente a la solución óptima $\mathbf{x}$ del programa primal es una solución óptima $\mathbf{y}$ del programa dual. En efecto, en cualquier iteración del método *simplex*, los multiplicadores simplex definen un vector $\mathbf{y}$ tal que $\mathbf{c}^T \mathbf{x} = \mathbf{b}^T \mathbf{y}$; este vector no es una solución factible del programa dual a no ser que todos los costes reducidos sean no negativos. Es decir, el algoritmo *simplex* mantiene de iteración a iteración la factibilidad del problema primal, y hace que $\mathbf{c}^T \mathbf{x} = \mathbf{b}^T \mathbf{y}$ mientras trata de conseguir la factibilidad del dual.

Obsérvese que la recíproca de (b) del teorema no es necesariamente cierta: si el problema primal o el dual no es factible, el otro problema no tiene por qué ser no acotado: los dos pueden ser no factibles. En la tabla 8.1 se pueden ver las implicaciones posibles.

El siguiente lema, debido a Farkas [1902], constituye un importante resultado para el estudio de sistemas lineales de igualdades y desigualdades, pudiendo ser fácilmente deducido de la parte (b) del teorema de la dualidad de la programación lineal.

Tabla 8.1
Combinaciones posibles primal-dual

		Dual		
		óptimo	no factible	no acotado
Primal	óptimo	posible	imposible	imposible
	no factible	imposible	posible	posible
	no acotado	imposible	posible	imposible

Lema 8.3 (Farkas) *El sistema de ecuaciones*

$$(I) \quad Ax = b, \quad x \geq 0,$$

no tiene solución si y sólo si la tiene el sistema

$$(II) \quad y^T A \leq 0^T, \quad b^T y > 0,$$

donde $A \in \mathbb{R}^{m \times n}$.

1^a DEMOSTRACIÓN. Consideremos el par primal-dual de problemas de programación lineal:

$$(C) \quad \begin{array}{ll} \min. & 0^T x \\ \text{s. a} & Ax = b, \\ & x \geq 0, \end{array} \quad (D) \quad \begin{array}{ll} \max. & b^T y \\ \text{s. a} & y^T A \leq 0^T. \end{array}$$

(C) no es factible si y sólo si (I) no tiene solución. (D) es no acotado si y sólo si el sistema (II) tiene solución. Ahora bien, como $y = 0$ es factible en (D), según el teorema 8.2, (C) no es factible si y sólo si (D) es no acotado. Para ser más precisos, para cualquier solución de (II), por ejemplo d , tal que $d^T A \leq 0^T$ y $b^T d > 0$, αd es una solución factible de (D) que conduce a una función objetivo no acotada al tender α a infinito. ■

2^a DEMOSTRACIÓN. El teorema se puede reformular de la siguiente manera. Si existe un $x \geq 0$ tal que $Ax = b$, no existe ningún y tal que $y^T A \leq 0^T$ y $b^T y > 0$. Recíprocamente, si no existe ningún $x \geq 0$ tal que $Ax = b$, existe un y tal que $y^T A \leq 0^T$ y $b^T y > 0$.

Supongamos que el sistema (I) tiene una solución x tal que $Ax = b$ y $x \geq 0$. Sea y un punto tal que $y^T A \leq 0^T$. En este caso $b^T y = x^T A^T y \leq 0$ pues $x \geq 0$ y $y^T A \leq 0^T$. Esto demuestra que $b^T y$ no puede ser positivo y, por lo tanto, el sistema (II) no tiene solución.

Supongamos ahora que el sistema (I) no tiene solución. Esto quiere decir que $b \notin S = \{v = Ax : x \geq 0\}$; es decir que b no pertenece al politopo cónico S . Observando la figura 8.2, está claro que si $b \notin S$, existe un hiperplano, denominado *hiperplano separador*¹ definido por un y , que separa S y b , y para el cual $y^T a_i \leq 0$, $i = 1, \dots, n$ y $y^T b > 0$, es decir, y forma un ángulo

¹La existencia de este hiperplano se demuestra formalmente en el apéndice A, página 696.

de más de 90 grados con cada uno de los vectores columna de A y de menos de 90 grados con² $\mathbf{b}$. Esto verifica que el sistema (II) tiene solución. ■

Al contrario de lo que hemos hecho aquí, el lema de Farkas se utiliza habitualmente para demostrar el teorema de la dualidad en programación lineal pues, a partir de él, la demostración de éste es trivial.

La interpretación geométrica del lema de Farkas es la siguiente:

1. Si $\mathbf{a}_i, i = 1, \dots, n$, son los n vectores columna de la matriz A , que se cumpla que $\mathbf{b} = A\mathbf{x}$, $\mathbf{x} \geq \mathbf{0}$, quiere decir que el vector $\mathbf{b} = \sum_{i=1}^n \mathbf{a}_i x_i, x_i \geq 0$; en otras palabras, que $\mathbf{b}$ pertenece al politopo cónico generado por los vectores columna de A . En la figura 8.3 se muestra un ejemplo donde el sistema (I) no tiene solución: el vector $\mathbf{b}$ no pertenece al cono generado por $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ y $\mathbf{a}_n$. La intersección del cono $\{\mathbf{y} : \mathbf{y}^T A \leq \mathbf{0}^T\}$ (conjunto formado por los vectores $\mathbf{y}$ que forman un ángulo mayor o igual de 90° con los vectores columna de la matriz A) y el semiespacio abierto $\{\mathbf{y} : \mathbf{b}^T \mathbf{y} > 0\}$, no es el conjunto vacío: el sistema (II) tiene solución, pues $\mathbf{b}$ y cualquier $\mathbf{y}$ en el cono que define la zona sombreada forma un ángulo menor de 90° y, por lo tanto, $\mathbf{b}^T \mathbf{y} > 0$.
2. El sistema (II) no tiene solución si la intersección del cono $\{\mathbf{y} : \mathbf{y}^T A \leq \mathbf{0}^T\}$ y el semiespacio abierto $\{\mathbf{y} : \mathbf{b}^T \mathbf{y} > 0\}$ es el conjunto vacío. En la figura 8.4 se muestra un ejemplo donde el sistema (II) no tiene solución. Todo vector $\mathbf{y}$ en la zona que define el cono indicado forma un ángulo mayor de 90° con $\mathbf{b}$. La tiene sin embargo (I) pues $\mathbf{b}$ pertenece al cono generado por $\mathbf{a}_1, \mathbf{a}_2$ y $\mathbf{a}_n$.

Antes de seguir adelante con las consecuencias prácticas del teorema de la dualidad, volvamos sobre la idea ya indicada de que asociado a cada programa lineal hay uno dual, y apliquemos al par primal-dual el lema de la dualidad débil, su corolario y el teorema de la dualidad. Por ejemplo, los programas lineales

$$\begin{array}{ll}
 \text{(P)} & \min. \mathbf{c}^T \mathbf{x} \\
 & \text{s. a } A\mathbf{x} \geq \mathbf{b} \\
 & \mathbf{x} \geq \mathbf{0} \\
 \text{(D)} & \max. \mathbf{b}^T \mathbf{y} \\
 & \text{s. a } A^T \mathbf{y} \leq \mathbf{c} \\
 & \mathbf{y} \geq \mathbf{0},
 \end{array} \tag{8.8}$$

son un par primal-dual. El programa dual (D) se puede obtener a partir del (P) transformando primero éste a forma estándar, convirtiendo el resultado en dual y simplificando. En efecto, (P) en forma estándar es

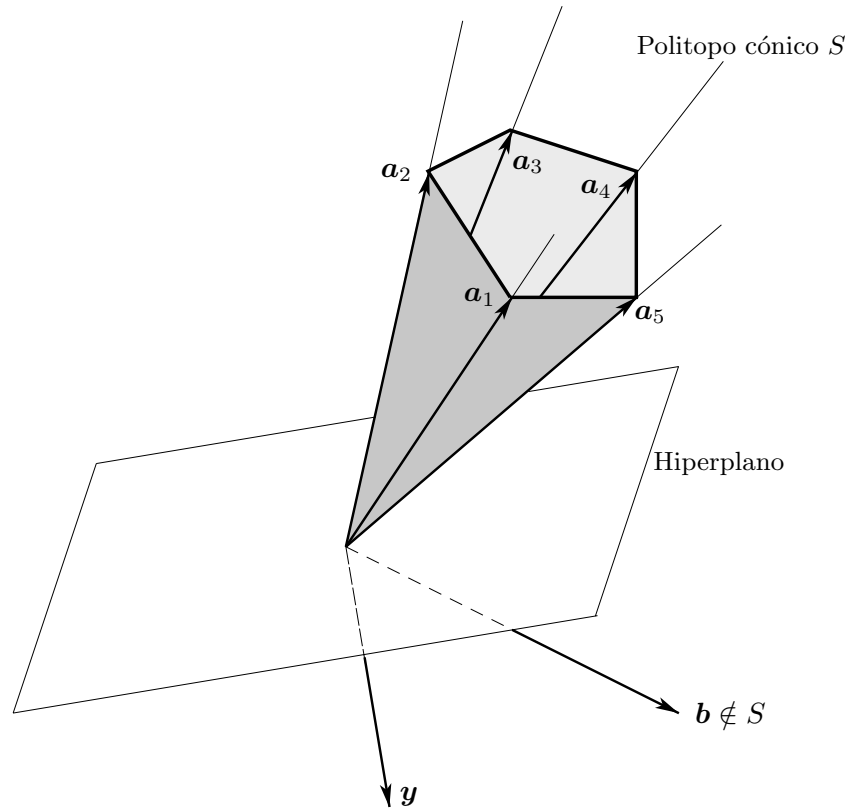
$$\begin{array}{ll}
 \text{(P')} & \min. \mathbf{c}^T \mathbf{x} \\
 & \text{s. a } A\mathbf{x} - \mathbf{x}' = \mathbf{b} \\
 & \mathbf{x}, \mathbf{x}' \geq \mathbf{0}.
 \end{array}$$

El dual de (P') es

$$\begin{array}{ll}
 \text{(D')} & \max. \mathbf{b}^T \mathbf{y} \\
 & \text{s. a } \begin{bmatrix} A^T \\ -I \end{bmatrix} \mathbf{y} \leq \begin{bmatrix} \mathbf{c} \\ \mathbf{0} \end{bmatrix}.
 \end{array}$$

De este último problema resulta inmediatamente (D) sin más que simplificar la notación.

²El hiperplano separador del politopo cónico S de la figura debería “casi” tocar a éste a lo largo de $\mathbf{a}_5$. El hiperplano soporte correspondiente, sí tocaría a $\mathbf{a}_5$.

**Figura 8.2**

Descripción geométrica de la existencia de un hiperplano separador

El par (P)-(D) se denomina habitualmente, en la literatura especializada, *forma simétrica de la dualidad*.

A continuación exponemos dos teoremas que caracterizan las soluciones óptimas del par de problemas primal-dual.

Teorema 8.3 (*Complementariedad de Holguras*) Sean $\mathbf{x}$ e $\mathbf{y}$ soluciones factibles del par de programas primal-dual en forma simétrica (P)-(D) de (8.8). Las condiciones necesarias y suficientes para que sean óptimos de sus respectivos problemas son:

$$(\mathbf{c}^T - \mathbf{y}^T A)\mathbf{x} = 0 \quad (8.9)$$

$\mathbf{y}$

$$\mathbf{y}^T (A\mathbf{x} - \mathbf{b}) = 0. \quad (8.10)$$

DEMOSTRACIÓN. Como $\mathbf{x}$ e $\mathbf{y}$ son soluciones factibles de (P) y (D), respectivamente, se tiene que

$$\mathbf{s} = A\mathbf{x} - \mathbf{b} \geq \mathbf{0}, \quad \mathbf{x} \geq \mathbf{0} \quad (8.11)$$

y

$$\mathbf{w}^T = \mathbf{c}^T - \mathbf{y}^T A \geq \mathbf{0}^T, \quad \mathbf{y} \geq \mathbf{0}. \quad (8.12)$$

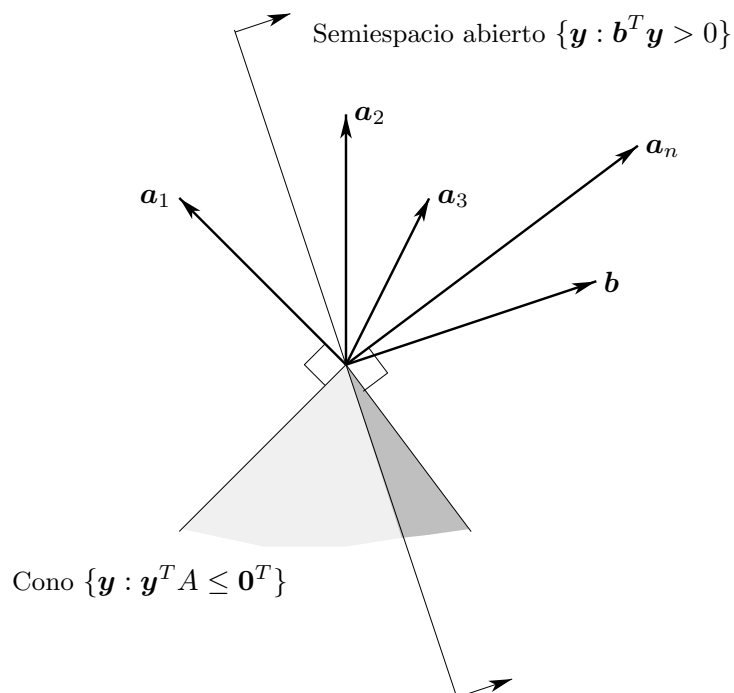


Figura 8.3

El sistema (I) del lema de Farkas no tiene solución. La tiene (II)

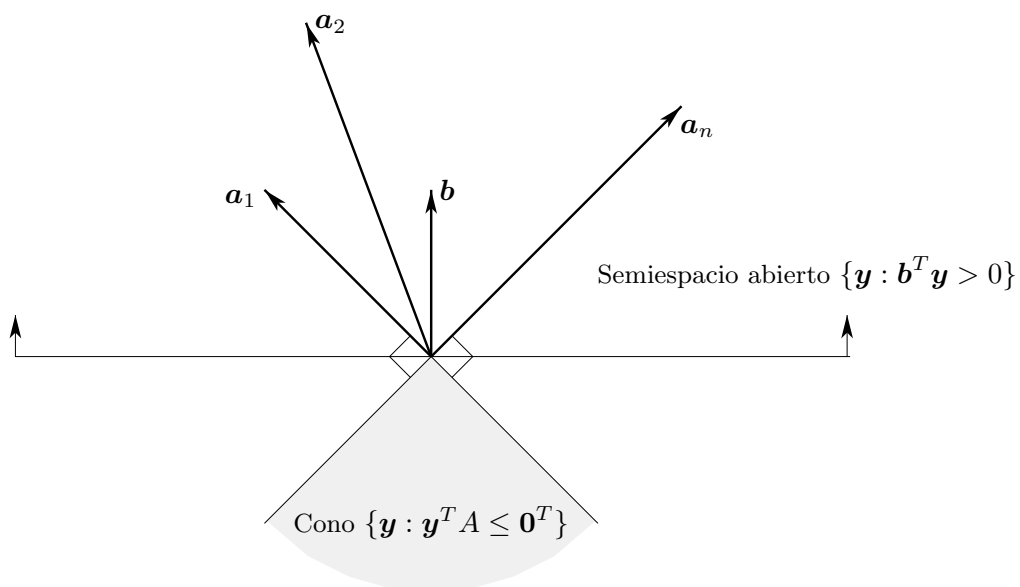


Figura 8.4

El sistema (II) del lema de Farkas no tiene solución. La tiene (I)

En consecuencia,

$$\mathbf{c}^T \mathbf{x} \geq \mathbf{y}^T A \mathbf{x} \geq \mathbf{y}^T \mathbf{b}.$$

Si las condiciones (8.9) y (8.10) se verifican estrictamente, esto es, $A\mathbf{x} = \mathbf{b}$ y $\mathbf{c}^T = \mathbf{y}^T A$, entonces $\mathbf{c}^T \mathbf{x} = \mathbf{y}^T \mathbf{b}$ y la condición de óptimo de $\mathbf{x}$ y de $\mathbf{y}$ se deduce inmediatamente del corolario 8.1. Recíprocamente, si $\mathbf{x}$ e $\mathbf{y}$ son óptimos, entonces $\mathbf{c}^T \mathbf{x} = \mathbf{y}^T \mathbf{b}$ y $\mathbf{c}^T \mathbf{x} = \mathbf{y}^T A \mathbf{x} = \mathbf{y}^T \mathbf{b}$ por lo que se cumplen (8.9) y (8.10). ■

En el caso de que el par primal-dual sea

$$\begin{array}{ll} \text{(P')} & \min. \mathbf{c}^T \mathbf{x} \\ & \text{s. a } A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{array} \quad \begin{array}{ll} \text{(D')} & \max. \mathbf{b}^T \mathbf{y} \\ & \text{s. a } A^T \mathbf{y} \leq \mathbf{c} \end{array}$$

sólo es significativa la condición (8.9), pues (8.10) es cierta para cualquier solución primal factible $\mathbf{x}$.

Debido a la condición de no negatividad de las soluciones $\mathbf{x}$ e $\mathbf{y}$ del par (P)-(D) y de los vectores $\mathbf{s}$ y $\mathbf{w}$, las condiciones (8.9) y (8.10) se pueden expresar en la forma mucho más útil que sigue.

Condiciones de Complementariedad de Holguras

$$\begin{array}{ll} w_j = (\mathbf{c} - A^T \mathbf{y})_j = 0 \text{ ó } x_j = 0, & \text{para todo } j = 1, \dots, n, \\ s_i = (A\mathbf{x} - \mathbf{b})_i = 0 \text{ ó } y_i = 0, & \text{para todo } i = 1, \dots, m. \end{array} \quad (8.13)$$

Usando estas condiciones el teorema anterior viene a decir que, en la forma simétrica (P)-(D), las soluciones factibles de este par son óptimas si y sólo si:

- i) Una variable es cero en uno de los problemas siempre que la variable de holgura correspondiente sea estrictamente positiva (la condición de desigualdad correspondiente se satisface estrictamente) en el otro problema.
- ii) Una variable de holgura es cero (la condición de desigualdad correspondiente se satisface como igualdad) en uno de los problemas siempre que la variable correspondiente sea positiva en el otro problema.

8.1.1 Condiciones de punto óptimo de Karush-Kuhn-Tucker

Estas condiciones —Kuhn y Tucker [1951]— fueron desarrolladas por estos autores para extender la teoría de Lagrange y poder caracterizar los puntos óptimos de problemas de programación lineal y no lineal sometidos a restricciones de igualdad y desigualdad. Como además del de Kuhn y Tucker existe un trabajo previo similar debido a Karush [1939], recientemente también se las denomina *condiciones de Karush-Kuhn-Tucker*.

Teorema 8.4 (*Condiciones de Kuhn-Tucker*) $\mathbf{x}$ es una solución óptima del programa lineal

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

si y sólo si existen vectores $\mathbf{y}$ y $\mathbf{w}$ tales que

$$\begin{aligned} (i) \quad & A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}, \\ (ii) \quad & A^T \mathbf{y} + \mathbf{w} = \mathbf{c}, \quad \mathbf{w} \geq \mathbf{0}, \quad y \\ (iii) \quad & \mathbf{w}^T \mathbf{x} = 0. \end{aligned}$$

DEMOSTRACIÓN. Es inmediata a partir del teorema 8.3 pues la condición (i) es la de factibilidad del primal, (ii) es la de factibilidad del dual y (iii) la de complementariedad de holguras. ■

En este teorema las variables duales $\mathbf{y}$ son los clásicos *multiplicadores de Lagrange*, si bien no del todo, pues al corresponder a condiciones de desigualdad, han de ser no negativos. La condición (iii) exige que sean cero aquellos multiplicadores que corresponden a condiciones no *activas* —condiciones que no se satisfacen estrictamente—. Esto es lógico que sea así pues esas condiciones no deben desempeñar ningún papel a la hora de decidir si un determinado punto es óptimo o no.

8.2 Interpretación económica de la dualidad

En el último apartado veíamos cómo surge de forma natural el dual de un programa lineal a partir de las condiciones de óptimo del programa primal. En éste veremos cómo, si un programa lineal posee una interpretación económica, también la tiene el dual, pudiéndose interpretar las variables de este último como precios.

Supongamos que

$$\mathbf{x}^* = \begin{bmatrix} \mathbf{x}_B^* \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} B^{-1}\mathbf{b} \\ \mathbf{0} \end{bmatrix}$$

es una solución óptima no degenerada del programa lineal en forma estándar

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{8.14}$$

Como suponemos que $\mathbf{x}_B^* > \mathbf{0}$, un pequeño cambio $\Delta\mathbf{b}$ en $\mathbf{b}$ no debe modificar la base óptima B ; en consecuencia, si se reemplaza $\mathbf{b}$ por $\mathbf{b} + \Delta\mathbf{b}$, la nueva solución óptima de (8.14) será

$$\hat{\mathbf{x}}^* = \begin{bmatrix} \hat{\mathbf{x}}_B^* \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} B^{-1}(\mathbf{b} + \Delta\mathbf{b}) \\ \mathbf{0} \end{bmatrix}.$$

Ese cambio modificará el valor de la función objetivo en la cantidad

$$\Delta z = \mathbf{c}_B^T B^{-1} \Delta \mathbf{b} = \boldsymbol{\pi}^{*T} \Delta \mathbf{b},$$

donde $\boldsymbol{\pi}^* = B^{-T} \mathbf{c}_B$ es el vector de multiplicadores simplex del problema (8.14) en el óptimo. Como ya se expuso en la demostración del teorema 8.2, $\boldsymbol{\pi}^*$ es la solución óptima del problema

$$\begin{aligned} \max. \quad & \mathbf{b}^T \mathbf{y} \\ \text{s. a} \quad & A^T \mathbf{y} \leq \mathbf{c}. \end{aligned} \tag{8.15}$$

Es evidente que π_i^* , a tenor de su expresión, indica cómo cambia la función objetivo al incrementar una unidad la cantidad disponible del recurso b_i ; es decir, ese valor π_i^* se puede considerar como un *precio o valor marginal* del recurso i -ésimo en el óptimo del problema.

Esta interpretación económica es de gran utilidad pues indica la cantidad máxima que se puede estar dispuesto a pagar por incrementar la disponibilidad del recurso i -ésimo. Obsérvese también que las condiciones de complementariedad de holguras (8.13), de la página 475, implican que el precio marginal de un recurso es cero si ese recurso no es totalmente utilizado en el óptimo del problema. Estos precios o valores marginales también se conocen habitualmente en la literatura especializada como *precios sombra* y *precios de equilibrio*.

Volvamos sobre algunos de los ejemplos clásicos de programación lineal y estudiemos sus duales y la interpretación económica de los mismos.

Ejemplo 8.3 El problema del transporte. Recordémoslo:

$$\begin{aligned} \min. \quad & \sum_{ij} c_{ij} x_{ij} \\ \text{s. a} \quad & \sum_{j=1}^n x_{ij} = a_i \text{ para } i = 1, \dots, m \\ & \sum_{i=1}^m x_{ij} = b_j \text{ para } j = 1, \dots, n \\ & x_{ij} \geq 0 \text{ para } i = 1, \dots, m \\ & \quad \quad \quad j = 1, \dots, n. \end{aligned}$$

El dual de este problema es

$$\begin{aligned} \max. \quad & \sum_{i=1}^m a_i u_i + \sum_{j=1}^n b_j v_j \\ \text{s. a} \quad & u_i + v_j \leq c_{ij}, \quad i = 1, \dots, m; \quad j = 1, \dots, n. \end{aligned} \tag{8.16}$$

Las variables duales u_i y v_j representan, respectivamente, los precios marginales de incrementar la oferta en la fábrica i y la demanda en el almacén j . Esta interpretación es coherente con la estrategia de la empresa poseedora de las fábricas y de los almacenes cuyo objetivo consiste en determinar un programa de envíos óptimo entre unas y otros. Veamos ahora una posible interpretación del problema dual en su conjunto, no sólo de las variables duales.

Supongamos que una compañía de transportes propone al fabricante anterior retirarle una unidad de producto de la fábrica i al precio u_i y entregarle una unidad de producto en el almacén j al precio v_j . Las restricciones del problema (8.16) asegurarían a la empresa de transportes que sus precios son competitivos frente a los de la empresa fabricante por lo que ésta preferiría darle la concesión del transporte de sus productos a aquella. Si se supone que la empresa de transportes conoce las cantidades disponibles en origen, a_i , y las demandas en los destinos, b_j , el problema que se debería plantear esta empresa es el indicado en (8.16), tratando de determinar los precios $u_1, \dots, u_m$ y $v_1, \dots, v_n$ de tal forma que se satisficieran las restricciones de precios antes dichas y se maximizase el dinero obtenible de la operación.

De acuerdo con el teorema de la dualidad, el fabricante del producto no se ahorraría dinero usando los servicios de la empresa de transportes en lugar de enviarlos él directamente; los medios de transporte propios de la empresa, no obstante, los podría emplear en otros usos y ganar con ello dinero. ●

Ejemplo 8.4 El problema de la dieta alimenticia. Recordemos que se formulaba de la forma siguiente:

$$\begin{aligned} \min. \quad & \sum_{j=1}^n c_j x_j \\ \text{s. a} \quad & \sum_{j=1}^n a_{ji} x_j \geq b_i, \quad i = 1, \dots, n; \\ & x_1, x_2, \dots, x_n \geq 0. \end{aligned}$$

Su dual es en consecuencia —recordemos el par (P)-(D)—:

$$\begin{aligned} \max. \quad & \sum_{i=1}^m b_i y_i \\ \text{s. a} \quad & \sum_{i=1}^m a_{ij} y_i \leq c_j, \quad j = 1, \dots, n; \\ & y_1, y_2, \dots, y_m \geq 0. \end{aligned} \tag{8.17}$$

Interpretaremos el problema dual como el que se plantea una empresa competidora de la que resuelve el primal.

Supongamos que ese competidor es una empresa que se dedica a la venta de píldoras de ingredientes nutritivos en estado puro: hierro, proteínas, etc. Si este vendedor quiere colocar en el mercado su producto, el precio de sus píldoras ha de ser competitivo con relación al de los alimentos de la empresa de dietética. Esto requiere que los precios que fije para los m ingredientes nutritivos, $y_1, \dots, y_m$, han de satisfacer las restricciones de (8.17) —recordemos que a_{ij} es la cantidad de ingrediente nutritivo i que proporciona o contiene el alimento j y c_j el coste unitario de ese alimento j —. Como los requisitos mínimos diarios de ingredientes nutritivos son b_j , $j = 1, \dots, n$, el vendedor de píldoras tratará de, a partir de esos precios y_i , maximizar el beneficio obtenible vendiendo las píldoras suficientes para completar una dieta. De ahí la función objetivo en (8.17). ●

Los precios marginales que introducíamos antes también resultan útiles para determinar la conveniencia o no de acometer nuevas actividades. Por ejemplo, en el problema de la dieta

alimenticia, supongamos que una vez obtenida la solución óptima del problema originalmente planteado se cuestione la posibilidad de comprar un nuevo alimento. La pregunta que surge de inmediato es: ¿es interesante considerar este nuevo alimento en la dieta? Para responder a esta pregunta, si a_{ik} es la cantidad de ingrediente nutritivo i que proporciona el nuevo alimento k y c_k su coste unitario, como el valor óptimo de la variable dual i -ésima, y_i , es el precio marginal de una unidad del ingrediente nutritivo i , los ingredientes nutritivos que proporcionan el alimento k tienen un valor $\sum_{i=1}^m y_i a_{ik}$. En consecuencia, si c_k es menor que ese valor, valdrá la pena comprar el nuevo alimento y considerarlo en la nueva dieta ($\mathbf{y}$ no es factible en la nueva restricción); de lo contrario, la dieta óptima actual es más ventajosa ($\mathbf{y}$ sigue siendo factible). En el caso de que se aconsejase la entrada del nuevo alimento en la dieta, se pueden utilizar los datos de la última iteración del *simplex* usado para resolver el problema original y pasar a considerar la nueva actividad —nuevo alimento— como la variable a entrar en la base para continuar el proceso de reoptimización.

8.3 El algoritmo dual del simplex

Este algoritmo fue diseñado por Lemke [1954] y Beale [1954] para hacer frente a las situaciones que surgen cuando se tiene una solución básica no factible de un programa lineal y, sin embargo, los costes reducidos de las variables no básicas son todos de óptimo (≥ 0): es decir, los multiplicadores simplex son factibles en el programa dual. Esta situación es muy frecuente en problemas de reoptimización donde se pretende obtener el nuevo óptimo de un problema al que se le añade una nueva restricción. Si la condición que se añade es, por ejemplo, de desigualdad, se puede, si no se cumple —si se cumple evidentemente no es necesario hacer nada, pues la solución primitiva sigue siendo la óptima—, añadir una nueva variable de holgura al problema y convertirla en condición de igualdad. La base óptima del problema original y la nueva variable de holgura constituirán la nueva base de partida del problema. Esta nueva base es óptima pero no factible pues el valor de la nueva variable de holgura es igual al negativo de lo que no se satisface la nueva restricción introducida. Esta forma de actuar resulta, como se expondrá, de extraordinaria utilidad en programación entera.

El método dual del *simplex*, como el primal, progresa, de iteración en iteración, de una base a otra del problema, aunque en lugar de ir manteniendo la factibilidad del programa primal, lo hace en el dual. Cuando se puede llegar a una solución del programa dual que sea factible y que también lo sea del primal, el procedimiento termina: se ha llegado al óptimo del problema.

Supongamos que se trata de resolver el problema lineal en forma estándar,

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{8.18}$$

y que la base de este problema la forman las m primeras variables del mismo. Además, que $\mathbf{x}_B = B^{-1}\mathbf{b}$, $\boldsymbol{\pi}^T = \mathbf{c}_B^T B$ y $\bar{\mathbf{c}}_N^T = \mathbf{c}_N^T - \boldsymbol{\pi}^T N \geq \mathbf{0}$. Si $\mathbf{x}_B \not\geq \mathbf{0}$, la solución $\mathbf{x}^T = [\mathbf{x}_B^T, \mathbf{0}^T]$ corresponde a un punto extremo óptimo pero no factible del politopo que definen las condiciones de (8.18).

Supongamos que $x_p < 0$. Para mejorar la situación sería conveniente trasladarse del punto extremo correspondiente no factible a otro próximo —factible o no— en el que x_p fuese cero e introducir en su lugar la variable x_q en la base. Esta nueva variable x_q se escoge de tal

forma que se mantenga la factibilidad del programa dual. A continuación analizaremos cómo seleccionar uno de los $n - m$ puntos extremos próximos posibles que sea dual factible (óptimo del primal pero no factible en él).

Recordemos de la demostración del teorema 8.1, de la página 467, que los multiplicadores simplex y los costes reducidos de las variables no básicas se pueden calcular a partir de la expresión

$$\begin{bmatrix} \boldsymbol{\pi}^T, & \bar{\mathbf{c}}_N^T \end{bmatrix} = \mathbf{c}^T M^{-1},$$

donde la matriz M es la que se definió en la fórmula (7.2) de la página 412, es decir,

$$M = \begin{bmatrix} B & N \\ 0 & I \end{bmatrix}.$$

Los vectores fila de esta matriz son los vectores característicos de los n hiperplanos que determinan la solución básica. La inversa de M es

$$M^{-1} = \begin{bmatrix} B^{-1} & -B^{-1}N \\ 0 & I \end{bmatrix}.$$

Si en una iteración del método dual del *simplex* se reemplaza la variable básica x_p por la x_q , en términos algebraicos, esto equivale a reemplazar la fila q de la matriz M (antes de ello $\mathbf{e}_q^T$) por el vector $\mathbf{e}_p^T$; es decir, M se transforma en

$$\bar{M} = M + \mathbf{e}_q(\mathbf{e}_p - \mathbf{e}_q)^T.$$

Usando la fórmula de Sherman-Morrison-Woodbury introducida en el lema 4.6 de la página 330 y el hecho de que $\mathbf{e}_q^T M^{-1} = \mathbf{e}_q^T$, se tiene que

$$\bar{M}^{-1} = M^{-1} - \frac{M^{-1} \mathbf{e}_q (\mathbf{e}_p^T M^{-1} - \mathbf{e}_q^T)}{\mathbf{e}_p^T M^{-1} \mathbf{e}_q}.$$

Multiplicando los dos miembros de esta última expresión por $\mathbf{c}^T$ se obtienen las siguientes fórmulas para obtener los nuevos multiplicadores simplex $\bar{\boldsymbol{\pi}}$ y costes reducidos $\bar{\mathbf{c}}_N$:

$$\begin{aligned} \bar{\boldsymbol{\pi}} &= \boldsymbol{\pi} + \gamma \mathbf{u}, \\ \bar{c}_j &= \bar{c}_j - \gamma \alpha_j, \quad j > m, j \neq q, \end{aligned}$$

y

$$\bar{c}_p = -\gamma,$$

donde

$$\mathbf{u}^T = \mathbf{e}_p^T B^{-1}, \quad \alpha_j = \mathbf{u}^T \mathbf{a}_j \quad \text{y} \quad \gamma = \bar{c}_q / \alpha_q.$$

Obsérvese que $\mathbf{u}^T$ es la fila p de la matriz B^{-1} y α_q es el denominado elemento *pivote* y_p del paso 3 del algoritmo *simplex* revisado de la tabla 7.1, página 420.

Para que el nuevo vector de costes reducidos $\bar{\mathbf{c}}$ sea no negativo es necesario que la variable que se escoge para entrar en la base, x_q , cumpla que

$$0 \leq -\gamma = -\bar{c}_q / \alpha_q \leq -\bar{c}_j / \alpha_j, \quad \text{para todo } \alpha_j < 0, j > m.$$

Si $\alpha_j \geq 0$ para todas las variables no básicas x_j , $\mathbf{u}^T A$ es un vector no negativo y, por tanto, $\mathbf{u}^T A\mathbf{x} = \mathbf{u}^T \mathbf{b}$ no puede tener una solución no negativa pues $\mathbf{u}^T \mathbf{b} = x_p < 0$. Esto implica que el programa (8.18) no es factible.

En la tabla 8.2 se expone una iteración del algoritmo dual del *simplex*. Como ya se utilizó en las tablas 7.1 y 7.3 de las páginas 420 y 446, $\mathbf{B} = \{j_1, \dots, j_m\}$ es el conjunto de índices de las variables básicas y $\mathbf{N}$ el de las no básicas. El algoritmo parte de una solución básica $\mathbf{x}_B$ dual factible ($A^T \boldsymbol{\pi} \leq \mathbf{c}$).

Tabla 8.2
Algoritmo dual del *simplex*

Paso 1 – Calcular el vector de multiplicadores simplex resolviendo el sistema $B^T \boldsymbol{\pi} = \mathbf{c}_B$. Determinar los costes reducidos de las variables no básicas: $\bar{c}_j = c_j - \boldsymbol{\pi}^T \mathbf{a}_j$, para todo $j \in \mathbf{N}$.

Paso 1' – Comprobar la factibilidad del programa primal: si $\mathbf{x}_B \geq \mathbf{0}$, PARAR; la solución es óptima. Si no, continuar.

Paso 2 – Determinar la variable básica x_{j_p} que debe salir de la base. Para ello, escoger $j_p \in V = \{j_i \in \mathbf{B} : x_{j_i} < 0\}$.

Paso 3 – Comprobar la no factibilidad del problema: calcular $\mathbf{u}$ resolviendo el sistema $B^T \mathbf{u} = \mathbf{e}_p$ y hacer $\alpha_j = \mathbf{u}^T \mathbf{a}_j$, para todo $j \in \mathbf{N}$. Si $\alpha_j \geq 0$ para todo $j \in \mathbf{N}$, PARAR; el problema no tiene solución.

Paso 4 – Determinar la variable no básica x_q que ha de entrar en la base: calcular

$$-\frac{\bar{c}_q}{\alpha_q} = \min \left\{ -\frac{\bar{c}_j}{\alpha_j} : \alpha_j < 0, j \in \mathbf{N} \right\} = -\gamma.$$

Paso 5 – Recalcular los costes reducidos: hacer

$$\bar{c}_j \leftarrow \bar{c}_j - \gamma \alpha_j, \quad j \in \mathbf{N}, j \neq q,$$

y

$$\bar{c}_p \leftarrow -\gamma.$$

Paso 6 – Adaptar la solución y la matriz B : calcular $\mathbf{w}$ resolviendo $B\mathbf{w} = \mathbf{a}_q$ y hacer

$$\begin{aligned} x_q &\leftarrow \theta = x_{j_p} / \alpha_q \\ x_{j_i} &\leftarrow x_{j_i} - \theta w_i, \quad 1 \leq i \leq m, i \neq p \\ B &\leftarrow B + (\mathbf{a}_q - \mathbf{a}_{j_p}) \mathbf{e}_p^T \\ \mathbf{B} &\leftarrow \mathbf{B} \cup \{q\} \setminus \{j_p\} \\ \mathbf{N} &\leftarrow \mathbf{N} \cup \{j_p\} \setminus \{q\} \\ j_p &\leftarrow q. \end{aligned}$$

Ir al paso 1'.

Utilizando las fórmulas de adaptación de los costes reducidos del paso 5, el algoritmo dual del *simplex* requiere, por iteración, esencialmente el mismo número de operaciones que el *simplex*

revisado de la tabla 7.1. En ambos casos el principal esfuerzo de cálculo se lleva a cabo en la resolución de los sistemas lineales $B^T \mathbf{u} = \mathbf{e}_p$ y $B\mathbf{w} = \mathbf{a}_q$, así como en los cálculos de los α_j y la adaptación de B . Si se adaptase en cada iteración el vector de multiplicadores, $\boldsymbol{\pi}$, en vez de $\bar{\mathbf{c}}_N$, también se requerirían efectuar más productos interiores para calcular los $\bar{c}_j$, para todo $j \in \mathbf{N}$ tal que $\alpha_j < 0$. También se podría calcular directamente $\boldsymbol{\pi}$ en cada iteración, aunque esto conllevaría resolver un sistema lineal más con B^T . Una desventaja que presenta el método dual en relación con el simplex revisado es que en aquel hay que calcular los $n - m$ productos interiores $\alpha_j = \mathbf{u}^T \mathbf{a}_j$, $j \in \mathbf{N}$; en el primal sólo hay que calcular $\boldsymbol{\pi}^T \mathbf{a}_j$ hasta que un determinado número de columnas no básicas tengan coste reducido negativo o se llegue al óptimo. Esta estrategia es la que se conoce en la literatura especializada como *evaluación parcial* de costes reducidos —*partial pricing*—.

Resolver un programa lineal en forma estándar

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

con el método dual del *simplex* es matemáticamente equivalente a resolver su dual con el método *simplex*. Este aserto no debe sorprender, pues ambos procedimientos obtienen sucesivas soluciones básicas factibles del programa dual manteniendo la condición de complementariedad de holguras. Aplicar el método *simplex* directamente al programa dual conlleva trabajar con la matriz $n \times n$, $\hat{B} = M^T$; el dual utiliza la $B \in \Re^{m \times m}$.

8.3.1 El algoritmo dual del simplex para variables acotadas

Las implementaciones prácticas en ordenador del método dual del *simplex* tienen en cuenta todas las consideraciones que hicimos en los apartados 7.5 y 7.6. En concreto, la especialización del método dual del *simplex* para resolver el problema con variables acotadas

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u} , \end{aligned}$$

cuyo dual es

$$\begin{aligned} \max. \quad & \mathbf{b}^T \boldsymbol{\pi} - \mathbf{u}^T \mathbf{y} + \mathbf{l}^T \mathbf{z} \\ \text{s. a} \quad & A^T \boldsymbol{\pi} - \mathbf{y} + \mathbf{z} = \mathbf{c} \\ & \mathbf{y}, \mathbf{z} \geq \mathbf{0} , \end{aligned}$$

es la que se describe en la tabla 8.3.

Ejemplo 8.5 Consideremos el ejemplo 7.10 de la página 453. Añadamos la restricción adicio-

Tabla 8.3
Algoritmo dual del *simplex* para variables acotadas

- Paso 1** – Calcular el vector de multiplicadores simplex resolviendo el sistema $B^T \pi = c_B$. Calcular los costes reducidos de las variables no básicas: $\bar{c}_j = c_j - \pi^T a_j$, para todo $j \notin B$.
- Paso 1'** – Comprobar la factibilidad del programa primal: si $l_B \leq x_B \leq u_B$, PARAR; la solución es óptima. Si no, continuar.
- Paso 2** – Determinar la variable básica x_{j_p} que debe salir de la base: escoger $j_p \in V \equiv \{j_i \in B : x_{j_i} < l_{j_i} \text{ ó } x_{j_i} > u_{j_i}\}$.
- Paso 3** – Comprobar la no factibilidad del problema: calcular u resolviendo el sistema $B^T u = e_p$ y hacer $\alpha_j = u^T a_j$, para todo $j \notin B$. Si $x_{j_p} < l_{j_p}$, sea $J = \{j \in N_l : \alpha_j < 0 \text{ y } j \in N_u : \alpha_j > 0\}$; si $x_{j_p} > u_{j_p}$, $J = \{j \in N_l : \alpha_j > 0 \text{ y } j \in N_u : \alpha_j < 0\}$. Si $J = \emptyset$, PARAR; el problema no es factible. Si no, continuar.
- Paso 4** – Determinar la variable no básica x_q que ha de entrar en la base: si $x_{j_p} < l_{j_p}$, calcular
- $$\max \left\{ \max \left\{ \frac{\bar{c}_j}{\alpha_j} : \alpha_j < 0, j \in N_l \right\}, \max \left\{ \frac{\bar{c}_j}{\alpha_j} : \alpha_j > 0, j \in N_u \right\} \right\} = \gamma;$$
- si $x_{j_p} > u_{j_p}$, calcular
- $$\min \left\{ \min \left\{ \frac{\bar{c}_j}{\alpha_j} : \alpha_j > 0, j \in N_l \right\}, \min \left\{ \frac{\bar{c}_j}{\alpha_j} : \alpha_j < 0, j \in N_u \right\} \right\} = \gamma.$$
- Paso 5** – Reobtener los costes reducidos: hacer $\bar{c}_j \leftarrow \bar{c}_j - \gamma \alpha_j$, $j \notin B$, $j \neq q$, y $\bar{c}_p \leftarrow -\gamma$.
- Paso 6** – Adaptar la solución y la matriz B : calcular w resolviendo $Bw = a_q$, hacer

$$\begin{aligned} x_{j_p} &\leftarrow l_{j_p} && \text{si } x_{j_p} < l_{j_p} \\ x_{j_p} &\leftarrow u_{j_p} && \text{si } x_{j_p} > u_{j_p} \\ x_q &\leftarrow x_q + \frac{x_{j_p} - l_{j_p}}{\alpha_q} && \text{si } x_{j_p} < l_{j_p} \\ x_q &\leftarrow x_q + \frac{x_{j_p} - u_{j_p}}{\alpha_q} && \text{si } x_{j_p} > u_{j_p} \\ x_{j_i} &\leftarrow x_{j_i} - \frac{x_{j_p} - l_{j_p}}{\alpha_q} w_i, && 1 \leq i \leq m, i \neq p, \quad \text{si } x_{j_p} < l_{j_p} \\ x_{j_i} &\leftarrow x_{j_i} - \frac{x_{j_p} - u_{j_p}}{\alpha_q} w_i, && 1 \leq i \leq m, i \neq p, \quad \text{si } x_{j_p} > u_{j_p} \\ B &\leftarrow B + (a_q - a_{j_p}) e_p^T \\ B &\leftarrow B \cup \{q\} \setminus \{j_p\} \\ j_p &\leftarrow q. \end{aligned}$$

Ir al paso 1'.

nal $x_3 \leq 2$. El problema resulta:

$$\begin{array}{ll} \min. & -2x_1 - 4x_2 - x_3 \\ \text{s. a} & 2x_1 + x_2 + x_3 \leq 10 \\ & x_1 + x_2 - x_3 \leq 4 \\ & 0 \leq x_1 \leq 4 \\ & 0 \leq x_2 \leq 6 \\ & 1 \leq x_3 \leq 2. \end{array}$$

Introduciendo las variables de holgura en las condiciones 1 y 2, queda:

$$\begin{array}{ll} \min. & -2x_1 - 4x_2 - x_3 \\ \text{s. a} & 2x_1 + x_2 + x_3 + x_4 = 10 \\ & x_1 + x_2 - x_3 + x_5 = 4 \\ & 0 \leq x_1 \leq 4 \\ & 0 \leq x_2 \leq 6 \\ & 1 \leq x_3 \leq 2 \\ & x_4, x_5 \geq 0. \end{array}$$

La base óptima resultante en aquel ejemplo era

$$B = [\mathbf{a}_1, \mathbf{a}_3] = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}.$$

La solución,

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 6 \\ 8/3 \\ 0 \\ 0 \end{bmatrix}.$$

Utilicemos el algoritmo de la tabla 8.3 para resolver este problema.

Iteración 1. Paso 1 y 1'

Calculemos el vector inicial de multiplicadores simplex resolviendo el sistema $B^T \boldsymbol{\pi} = \mathbf{c}_B$:

$$\boldsymbol{\pi} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}^{-1} \begin{bmatrix} -2 \\ -1 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & -\frac{2}{3} \end{bmatrix} \begin{bmatrix} -2 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Los costes reducidos de las variables no básicas son:

$$\bar{c}_2 = c_2 - \boldsymbol{\pi}^T \mathbf{a}_2 = -4 - [-1, 0] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = -3;$$

$$\bar{c}_4 = c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [-1, 0] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$$

y

$$\bar{c}_5 = c_5 - \boldsymbol{\pi}^T \mathbf{a}_5 = 0 - [-1, 0] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.$$

Todos los costes reducidos de las variables no básicas son óptimos: x_4 y x_5 en su límite inferior, costes reducidos ≥ 0 ; x_2 en su límite superior, coste reducido < 0 .

El problema primal no es factible pues $x_3 = 8/3 > 2$.

Iteración 1. Paso 2

La variable básica a salir de la base es x_3 : $x_3 > u_3$. Es la segunda de la base, luego $p = 2$.

Iteración 1. Paso 3

Resolvamos $B^T \mathbf{u} = \mathbf{e}_2$:

$$\mathbf{u} = B^{-T} \mathbf{e}_2 = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/3 \\ -2/3 \end{bmatrix}.$$

Hagamos

$$\alpha_2 = \mathbf{u}^T \mathbf{a}_2 = [1/3, -2/3] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = -1/3,$$

$$\alpha_4 = \mathbf{u}^T \mathbf{a}_4 = [1/3, -2/3] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1/3$$

y

$$\alpha_5 = \mathbf{u}^T \mathbf{a}_5 = [1/3, -2/3] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = -2/3.$$

El conjunto $J = \{j \in \mathbf{N}_l : \alpha_j > 0 \text{ y } j \in \mathbf{N}_u : \alpha_j < 0\} = \{2, 4\}$.

Iteración 1. Paso 4

Determinemos la variable no básica que ha de entrar en la base. Calculamos

$$\gamma = \min \left\{ \frac{\bar{c}_2}{\alpha_2}, \frac{\bar{c}_4}{\alpha_4} \right\} = \left\{ \frac{-3}{-1/3}, \frac{1}{1/3} \right\} = 3.$$

Entrará x_4 .

Iteración 1. Paso 5

Recalculemos los costes reducidos:

$$\bar{c}_2 \leftarrow \bar{c}_2 - \gamma \alpha_2 = -3 - 3 \left(-\frac{1}{3} \right) = -2;$$

$$\bar{c}_5 \leftarrow \bar{c}_5 - \gamma \alpha_5 = 0 - 3 \left(-\frac{2}{3} \right) = 2$$

y

$$\bar{c}_3 \leftarrow -\gamma = -3.$$

Iteración 1. Paso 6

Adaptamos la solución, la matriz B y las estructuras de datos. Para ello, resolvamos primero el sistema $B\mathbf{w} = \mathbf{a}_4$:

$$\mathbf{w} = B^{-1}\mathbf{a}_4 = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 \\ 1/3 & -2/3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \end{bmatrix}.$$

Después,

$$\begin{aligned} x_3 &\leftarrow u_3 = 2 \\ x_4 &\leftarrow x_4 + \frac{x_3 - u_3}{\alpha_4} = 0 + \frac{8/3 - 2}{1/3} = 2 \end{aligned}$$

y

$$x_1 \leftarrow x_1 - \frac{x_3 - u_3}{\alpha_4} w_1 = \frac{2}{3} - \frac{8/3 - 2}{1/3} \cdot \frac{1}{3} = 0$$

La nueva base será

$$B = [\mathbf{a}_1, \mathbf{a}_4] = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix},$$

y $B = \{1, 4\}$.

Iteración 2. Paso 1

Comprobamos que la solución $\mathbf{x} = [0, 6, 2, 2, 0]^T$ es factible (degenerada) en el programa primal: hemos llegado al óptimo del problema. El valor de la función objetivo es -26 .

Obsérvese lo fácil —una iteración— que ha sido reoptimizar el problema original utilizando el método dual del *simplex*. ●

8.4 El método primal–dual

Como hemos podido estudiar, el algoritmo dual del *simplex* parte de una solución básica factible del programa dual y define la correspondiente básica del primal de tal forma que se cumpla la condición de complementariedad de holguras. De iteración en iteración se mantiene la factibilidad del dual, y la condición de complementariedad de holguras, hasta que se consiga una solución básica factible del primal en cuyo caso se ha alcanzado el punto óptimo del problema.

El método primal–dual es similar al dual del *simplex*. Trabaja simultáneamente sobre los programas primal y dual partiendo, como el método dual, de una solución factible del programa dual. Fue originalmente desarrollado por Dantzig, Ford y Fulkerson en 1956 con el objetivo de resolver eficazmente programas lineales de flujos en redes, como los que estudiaremos en el capítulo siguiente, y problemas de transporte similares al introducido en el capítulo 5.

Para exponer la mecánica del método primal–dual, consideremos el programa primal en forma estándar. Sea $\boldsymbol{\pi}$ una solución dual factible (no necesariamente básica). Esta solución cumplirá que $c_j \geq \boldsymbol{\pi}^T \mathbf{a}_j$ para $j = 1, \dots, n$, donde $\mathbf{a}_j$ es el vector columna j de la matriz de

coeficientes de las condiciones, A . Sea $\mathbf{T}$ el conjunto de los índices de las condiciones $c_j \geq \boldsymbol{\pi}^T \mathbf{a}_j$ que se cumplen estrictamente:

$$\mathbf{T} = \{j : \boldsymbol{\pi}^T \mathbf{a}_j = c_j\}.$$

De acuerdo con el teorema de complementariedad de holguras, $\mathbf{T}$ es el conjunto de índices de las variables del programa primal que tienen valores positivos.

Consideremos el siguiente problema de programación lineal, denominado *primal restringido*:

$$\begin{aligned} \min. \quad & \mathbf{e}^T \mathbf{x}_a \\ \text{s. a} \quad & \sum_{j \in \mathbf{T}} \mathbf{a}_j x_j + \mathbf{x}_a = \mathbf{b} \\ & x_j \geq 0, \quad j \in \mathbf{T} \\ & \mathbf{x}_a \geq \mathbf{0}, \end{aligned} \tag{8.19}$$

donde $\mathbf{x}_a$ es un vector, de dimensión m , de variables artificiales. El programa dual de (8.19) es

$$\begin{aligned} \max. \quad & \mathbf{y}^T \mathbf{b} \\ \text{s. a} \quad & \mathbf{y}^T \mathbf{a}_j \leq 0, \quad j \in \mathbf{T} \\ & \mathbf{y} \leq \mathbf{e}. \end{aligned} \tag{8.20}$$

Lema 8.4 Si el programa lineal restringido tiene solución óptima con valor de la función objetivo cero, esa solución es el óptimo del programa original.

DEMOSTRACIÓN. Si con $\mathbf{x}_T$ designamos el conjunto de x_j , $j \in \mathbf{T}$, supongamos que

$$\begin{bmatrix} \mathbf{x}_T^* \\ \mathbf{x}_a^* \end{bmatrix}$$

es la solución óptima de programa lineal restringido y que el valor de la función objetivo correspondiente es cero. Como la función objetivo es cero, $\mathbf{x}_a^* = \mathbf{0}$ en ese óptimo. A partir del vector $\mathbf{x}_T^*$ se puede construir una solución factible del programa primal tal que $x_j = x_j^* \geq 0$, $j \in \mathbf{T}$, y $x_j = 0$, $j \notin \mathbf{T}$. Obsérvese que el programa lineal restringido se definió a partir de una solución dual factible, $\boldsymbol{\pi}$, tal que $c_j = \boldsymbol{\pi}^T \mathbf{a}_j$, $j \in \mathbf{T}$, y $c_j > \boldsymbol{\pi}^T \mathbf{a}_j$, $j \notin \mathbf{T}$. La condición de complementariedad de holguras también es claro que se satisface en este caso pues $(c_j - \boldsymbol{\pi}^T \mathbf{a}_j)x_j = 0$ para todo j . De acuerdo con esto se satisfacen las condiciones de Karush–Kuhn–Tucker y se completa la demostración. ■

Si el valor de la función objetivo en el óptimo del programa lineal restringido no es cero, $\mathbf{x}_T^*$ no es suficientemente bueno para definir una solución factible del primal del programa original. Es decir, se necesita otra solución dual factible con la que reconstruir el programa lineal restringido y obtener un valor de su función objetivo menor que el ya obtenido (a ser posible cero). Para lograr esto, también habrá que formar el programa restringido con variables adicionales que no pertenezcan a $\mathbf{T}$. Con tal fin, si $\mathbf{y}^*$ es el óptimo del dual (8.20) del programa lineal restringido, la condición de complementariedad de holguras implica que $\mathbf{y}^{*T} \mathbf{a}_j \leq 0$, para $j \in \mathbf{T}$. Sólo aquellas variables primales x_j cuyas correspondientes duales tengan índices $j \notin \mathbf{T}$,

tales que $\mathbf{y}^*{}^T \mathbf{a}_j > 0$, serán susceptibles de incorporarse al programa primal restringido y mejorar el valor de su función objetivo. En este sentido, $\mathbf{y}^*$ se puede considerar como una dirección de movimiento a lo largo de la cual se traslada la solución dual factible actual, $\boldsymbol{\pi}$, a una nueva $\boldsymbol{\pi}'$. Es decir,

$$\boldsymbol{\pi}' = \boldsymbol{\pi} + \alpha \mathbf{y}^*, \quad \alpha > 0.$$

De acuerdo con esto

$$c_j - \boldsymbol{\pi}'^T \mathbf{a}_j = c_j - (\boldsymbol{\pi} + \alpha \mathbf{y}^*)^T \mathbf{a}_j = (c_j - \boldsymbol{\pi}^T \mathbf{a}_j) - \alpha (\mathbf{y}^*{}^T \mathbf{a}_j).$$

Para cada $j \in \mathbf{T}$, como $c_j - \boldsymbol{\pi}^T \mathbf{a}_j = 0$ y $\mathbf{y}^*{}^T \mathbf{a}_j \leq 0$, entonces $c_j - \boldsymbol{\pi}'^T \mathbf{a}_j \geq 0$. Para mantener $\boldsymbol{\pi}'$ factible del dual habrá que considerar aquellos índices $j \notin \mathbf{T}$ tales que $\mathbf{y}^*{}^T \mathbf{a}_j > 0$. Dado que $c_j - \boldsymbol{\pi}^T \mathbf{a}_j \geq 0$, para todo $j \notin \mathbf{T}$, se puede elegir una $\alpha > 0$ adecuadamente en tanto y cuanto

$$\alpha = \frac{(c_k - \boldsymbol{\pi}^T \mathbf{a}_k)}{\mathbf{y}^*{}^T \mathbf{a}_k} = \min_j \left\{ \frac{(c_j - \boldsymbol{\pi}^T \mathbf{a}_j)}{\mathbf{y}^*{}^T \mathbf{a}_j} : j \notin \mathbf{T}, \mathbf{y}^*{}^T \mathbf{a}_j > 0 \right\}$$

siendo $c_j - \boldsymbol{\pi}'^T \mathbf{a}_j \geq 0$, $j \notin \mathbf{T}$. En particular, $c_k - \boldsymbol{\pi}'^T \mathbf{a}_k = 0$ y $c_j - \boldsymbol{\pi}'^T \mathbf{a}_j \geq 0$, $j \notin \mathbf{T}$ y $j \neq k$. La variable x_k será candidata a conformar la base del nuevo programa lineal junto con las que definían la del programa lineal restringido todavía vigente.

Siguiendo esta estrategia de añadir variables al programa lineal restringido puede ocurrir que: el valor de la función objetivo del óptimo del programa lineal restringido acabe siendo cero, en cuyo caso el lema 8.4 garantiza que se ha alcanzado el óptimo del programa original que se quería resolver; o que, el valor de la función objetivo del programa lineal restringido sea mayor que cero y además $\mathbf{y}^*{}^T \mathbf{a}_j \leq 0$, $j \notin \mathbf{T}$, en cuyo caso se puede demostrar que el programa original no será factible y su dual no acotado. En la tabla 8.4 se describe completo el algoritmo primal-dual. Parte de una solución factible del programa dual del problema planteado.

Ejemplo 8.6 Resolver el problema

$$\begin{aligned} \min. \quad & -2x_1 - x_2 \\ \text{s. a} \quad & x_1 + x_2 + x_3 = 2 \\ & x_1 + x_4 = 1 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

Iteración 1. Paso 1

El dual del problema que se desea resolver es:

$$\begin{aligned} \max. \quad & -2\pi_1 + \pi_2 \\ \text{s. a} \quad & \pi_1 + \pi_2 \leq -2 \\ & \pi_1 \leq -1 \\ & \pi_1 \leq 0 \\ & \pi_2 \leq 0. \end{aligned}$$

Las variables de este problema no están restringidas en ningún sentido.

Escojamos como solución factible de este problema dual la siguiente:

$$\boldsymbol{\pi} = \begin{bmatrix} \pi_1 \\ \pi_2 \end{bmatrix} = \begin{bmatrix} -1 \\ -3 \end{bmatrix}.$$

Tabla 8.4
Algoritmo primal-dual

Paso 1 – Determinar un vector de multiplicadores simplex, π , tal que

$$c_j - \pi^T \mathbf{a}_j \geq 0, \quad j = 1, \dots, n.$$

Definir $T = \{j : c_j - \pi^T \mathbf{a}_j = 0\}$.

Paso 2 – Comprobar que se ha llegado al óptimo del problema. Resolver el programa lineal restringido

$$\begin{aligned} \min. \quad & \mathbf{e}^T \mathbf{x}_a \\ \text{s. a} \quad & \sum_{j \in T} \mathbf{a}_j x_j + \mathbf{x}_a = \mathbf{b} \\ & x_j \geq 0, \quad j \in T \\ & \mathbf{x}_a \geq 0. \end{aligned}$$

Si el valor de la función objetivo en el óptimo de este problema es cero, PARAR; la solución actual es la óptima del problema. Si no, continuar.

Paso 3 – Resolver el programa dual del programa lineal restringido:

$$\begin{aligned} \max. \quad & \mathbf{y}^T \mathbf{b} \\ \text{s. a} \quad & \mathbf{y}^T \mathbf{a}_j \leq 0, \quad j \in T \\ & \mathbf{y} \leq \mathbf{e}. \end{aligned}$$

Sea $\mathbf{y}^*$ su solución óptima.

Paso 4 – Comprobar la no factibilidad o la existencia de solución no acotada. Si $\mathbf{y}^{*T} \mathbf{a}_j \leq 0$, $j \notin T$, PARAR; el problema original no es factible y su dual no acotado. Si no, continuar.

Paso 5 – Añadir una variable al programa lineal restringido. Escoger un índice k tal que

$$\frac{(c_k - \pi^T \mathbf{a}_k)}{\mathbf{y}^{*T} \mathbf{a}_k} = \min_j \left\{ \frac{(c_j - \pi^T \mathbf{a}_j)}{\mathbf{y}^{*T} \mathbf{a}_j} : j \notin T, \mathbf{y}^{*T} \mathbf{a}_j > 0 \right\}.$$

Definir un paso α :

$$\alpha = \frac{(c_k - \pi^T \mathbf{a}_k)}{\mathbf{y}^{*T} \mathbf{a}_k}.$$

Añadir la variable x_k a la base formando así un nuevo programa lineal restringido.

Paso 6 – Adaptar el vector π . Hacer

$$\pi = \pi + \alpha \mathbf{y}^*.$$

Ir al paso 2.

De acuerdo con esta elección, sólo la segunda de las restricciones se cumple estrictamente. Es decir, $\mathbf{T} = \{2\}$.

Iteración 1. Paso 2

Construyamos el programa primal restringido:

$$\begin{array}{ll} \min. & x_{a_1} + x_{a_2} \\ \text{s. a} & x_2 + x_{a_1} = 2 \\ & x_{a_2} = 1 \\ & x_2, x_{a_1}, x_{a_2} \geq 0. \end{array}$$

La solución óptima de este problema es

$$\begin{bmatrix} x_2 \\ x_{a_1} \\ x_{a_2} \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}.$$

Como el valor de la función objetivo es 1, todavía no se ha llegado al punto óptimo.

Iteración 1. Paso 3

Construyamos el programa dual del primal restringido y resolvámoslo:

$$\begin{array}{ll} \max. & 2y_1 + y_2 \\ \text{s. a} & y_1 \leq 0 \\ & y_1 \leq 1 \\ & y_2 \leq 1. \end{array}$$

Como las variables x_2 y x_{a_2} son básicas en el programa primal restringido, de acuerdo con la condición de complementariedad de holguras, la primera y la tercera restricciones de este programa dual se cumplirán estrictamente. En consecuencia,

$$\mathbf{y}^* = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

es la solución óptima del dual del primal restringido.

Iteración 1. Paso 4

Comprobar la no factibilidad del problema o la existencia de solución no acotada. Calculemos $\mathbf{y}^* \mathbf{a}_j$, $j \in \{1, 3, 4\}$. Los valores que se obtienen son 1, 0 y 1, respectivamente. Al ser todos no negativos, continuamos con el procedimiento.

Iteración 1. Paso 5

Calculemos $c_j - \boldsymbol{\pi}^T \mathbf{a}_j$, $j \in \{1, 3, 4\}$. Los valores que se obtienen son:

$$c_1 - \boldsymbol{\pi}^T \mathbf{a}_1 = -2 - [-1, -3] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 2;$$

$$c_3 - \boldsymbol{\pi}^T \mathbf{a}_3 = 0 - [-1, -3] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$$

y

$$c_4 - \boldsymbol{\pi}^T \mathbf{a}_4 = 0 - [-1, -3] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 3.$$

De acuerdo con esto,

$$\alpha = \min \left\{ \frac{2}{1}, \frac{3}{1} \right\} = 2,$$

con $k = 1$. Es decir, la variable x_1 deberá entrar en la base del programa lineal restringido.

Iteración 1. Paso 6

Adaptemos el vector $\boldsymbol{\pi}$. El nuevo será

$$\boldsymbol{\pi} = \boldsymbol{\pi} + \alpha \mathbf{y}^* = \begin{bmatrix} -1 \\ -3 \end{bmatrix} + 2 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}.$$

Iteración 2. Paso 2

Construyamos el nuevo programa primal restringido:

$$\begin{array}{ll} \min. & x_{a_1} + x_{a_2} \\ \text{s. a} & x_1 + x_2 + x_{a_1} = 2 \\ & x_1 + x_{a_2} = 1 \\ & x_1, x_2, x_{a_1}, x_{a_2} \geq 0. \end{array}$$

La solución óptima de este problema es

$$\begin{bmatrix} x_1 \\ x_2 \\ x_{a_1} \\ x_{a_2} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}.$$

Como el valor de la función objetivo es 0, se ha llegado ya al punto óptimo del problema que se quería resolver. Éste es:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}.$$

El valor óptimo de la función objetivo es -3 . La solución del dual del problema original como ya se ha calculado es $[-1, -1]$. ●

8.5 Análisis de sensibilidad

En la introducción de este capítulo hacíamos hincapié en la importancia que puede tener para ciertos programas lineales efectuar un análisis de su sensibilidad a variaciones de los parámetros que los definen. En el apartado dedicado al método dual del *simplex* hemos visto cómo reobtener, sin tener que hacer todos los cálculos desde el principio, la solución óptima de un programa lineal al que se añaden nuevas variables y/o nuevas condiciones, una vez obtenida la óptima del programa original. También veíamos que los multiplicadores simplex del óptimo de un programa lineal expresan lo que cambia la función objetivo de ese programa ante pequeños cambios en los elementos del vector término de la derecha de sus condiciones (en el caso de soluciones básicas factibles óptimas no degeneradas).

En este apartado profundizamos sobre cómo afectan a la función objetivo modificaciones más amplias en el vector $\mathbf{c}$ de coeficientes de esa función objetivo o en el vector $\mathbf{b}$. Este estudio se conoce habitualmente como *análisis de sensibilidad* de un programa lineal o *análisis post-optimización*. Su extensión a cuando esos cambios, más que constantes, son funciones lineales de ciertos parámetros, se conoce como *programación lineal paramétrica*. Ejemplos de esta última técnica surgen cuando las fluctuaciones de diversos costes de producción de un artículo dependen del precio de una determinada materia prima, o cuando en determinados problemas no está muy claro cuál ha de ser la función objetivo o es función de otros parámetros, etc.

Vamos a considerar en primer lugar cómo afectan a la función objetivo diversos cambios en el vector $\mathbf{c}$. En particular, consideremos la familia de programas, función del parámetro θ , definida por

$$\begin{aligned} \min. \quad & z(\theta) = [\mathbf{c} + \theta \mathbf{d}]^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{8.21}$$

Supongamos que para $\theta = \theta_0$ disponemos de una solución básica factible óptima y queremos determinar el margen de valores $\underline{\theta} \leq \theta \leq \bar{\theta}$ en el que esa solución sigue siendo óptima. Sea B la matriz básica óptima; dividamos de acuerdo con esto los vectores $\mathbf{c}$ y $\mathbf{d}$ en sus componentes básicos y no básicos: $\mathbf{c}_B$, $\mathbf{d}_B$ y $\mathbf{c}_N$, $\mathbf{d}_N$, respectivamente. La base B seguirá siendo óptima en tanto que los costes reducidos de las variables no básicas sigan siendo no negativos; es decir, mientras que $[\mathbf{c}_N + \theta \mathbf{d}_N]^T - [\mathbf{c}_B + \theta \mathbf{d}_B]^T B^{-1} N \geq \mathbf{0}^T$. Si se definen los subcostes reducidos

$$\bar{\mathbf{c}}_N^T = \mathbf{c}_N^T - \mathbf{c}_B^T B^{-1} N$$

y

$$\bar{\mathbf{d}}_N^T = \mathbf{d}_N^T - \mathbf{d}_B^T B^{-1} N,$$

la condición de no negatividad de costes reducidos se reduce en este caso a

$$\theta \bar{\mathbf{d}}_N^T \geq -\bar{\mathbf{c}}_N^T.$$

El margen de valores de θ en el que la solución sigue siendo óptima es por tanto

$$\begin{aligned} \underline{\theta} &= \max \{ \max \{ -\bar{c}_j / \bar{d}_j : \bar{d}_j > 0, j \notin \mathbf{B} \}, -\infty \} \leq \theta \\ &\leq \min \{ \min \{ -\bar{c}_j / \bar{d}_j : \bar{d}_j < 0, j \notin \mathbf{B} \}, \infty \} = \bar{\theta}. \end{aligned} \tag{8.22}$$

Para el margen de valores de θ , $\underline{\theta} \leq \theta \leq \bar{\theta}$, el valor que toma la función objetivo es una función lineal de θ . En efecto,

$$z^*(\theta) = [\mathbf{c}_B^T + \theta \mathbf{d}_B^T] B^{-1} \mathbf{b} = z^*(\theta_0) + (\theta - \theta_0) \mathbf{d}_B^T \mathbf{x}_B.$$

Si $\theta_0 = 0$, haciendo $\mathbf{d} = \mathbf{e}_j$, el margen de valores del coeficiente c_j en el que la solución óptima permanece constante, con todos los demás parámetros del problema fijos, está dado por $[c_j + \underline{\theta}, c_j + \bar{\theta}]$.

La solución óptima del problema de programación lineal paramétrica (8.21) también se puede determinar para todos los valores posibles del parámetro θ . En efecto, dado el intervalo $[\underline{\theta}, \bar{\theta}]$ de valores de θ en el que una solución básica factible es óptima, puede ocurrir que exista otra solución básica factible próxima que sea también óptima para valores de θ comprendidos en el intervalo $[\underline{\theta}, \underline{\theta}]$, donde $-\infty \leq \underline{\theta}$, o que la función objetivo $z^*(\theta)$ no esté acotada inferiormente para todo θ en el intervalo $(-\infty, \underline{\theta})$. Esta nueva solución y base, de existir, se puede obtener pivotando la variable no básica x_j en la base, lo que determina un $\underline{\theta} = -\bar{c}_j/\bar{d}_j$, de acuerdo con la expresión (8.22), pudiéndose determinar $\underline{\theta}$ a partir de la nueva base así obtenida. Si al llevar a cabo esa pivotación se detecta la existencia de un rayo no acotado, $z^*(\theta)$ no está acotada inferiormente para todo $\theta < \underline{\theta}$. De igual forma se obtendría una solución básica factible óptima próxima, de existir, para $\theta \geq \bar{\theta}$, en el margen $[\bar{\theta}, \bar{\theta}]$.

Mediante estos sencillos cálculos se pueden determinar todos los márgenes de valores del parámetro θ . A pesar de que su número puede llegar a ser 2^n , en el peor de los casos, los análisis probabilísticos que se han efectuado con el método *simplex*, a partir de la resolución del problema (8.21), han dado como resultado que ese completo análisis requeriría un límite máximo de iteraciones muy razonable: concretamente, un orden de magnitud cuadrado del tamaño del problema: $O(\min\{m^2, n^2\})$.

Consideremos a continuación el siguiente problema para estudiar el margen de valores del vector término de la derecha:

$$\begin{aligned} \min. \quad & z(\theta) = \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} + \theta \mathbf{d} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

Si B es la base óptima para algún valor $\theta = \theta_0$, el intervalo de valores $[\underline{\theta}, \bar{\theta}]$ en el cual esa base es óptima y, por lo tanto, también la solución $\mathbf{x}^T = [\mathbf{x}_B^T, \mathbf{x}_N^T] = [\bar{\mathbf{b}}^T + \theta \bar{\mathbf{d}}^T, \mathbf{0}^T]$, donde $\bar{\mathbf{b}} = B^{-1} \mathbf{b}$ y $\bar{\mathbf{d}} = B^{-1} \mathbf{d}$, está dado por

$$\begin{aligned} \underline{\theta} &= \max \left\{ \max_{1 \leq i \leq m} \{ -\bar{b}_i/\bar{d}_i : \bar{d}_i > 0 \}, -\infty \right\} \leq \theta \\ &\leq \min \left\{ \min_{1 \leq i \leq m} \{ -\bar{b}_i/\bar{d}_i : \bar{d}_i < 0 \}, \infty \right\} = \bar{\theta}. \end{aligned}$$

En este intervalo, a pesar de que la solución óptima varía linealmente con respecto a θ , la base y la solución óptima del problema dual permanecen fijas. Las bases próximas y los márgenes de valores correspondientes se pueden obtener mediante el método dual del *simplex* siempre y cuando no se detecten infactibilidades.

Referencias

Además de las referencias comentadas en el capítulo anterior, la parte teórica de la dualidad está muy bien recogida en Fletcher [1987], Simmonard [1972] y Shapiro [1979]. Para un problema general de optimización sometido a restricciones, en Fletcher [1987]. Lo expuesto en casi todo el capítulo sigue a Nemhauser, Rinnooy Kan y Todd [1989]. Lo referente al lema de Farkas se puede también seguir en Bazaraa, Jarvis y Sherali [1990].

En Dantzig [1987] —relatados por su autor— se puede encontrar una interesante y detallada relación de los avatares históricos del método *simplex* y cómo desembocaron en su publicación en 1947. Para una excelente y pormenorizada relación de la historia de la programación lineal y su contexto técnico e histórico recomendamos consultar Schrijver [1986].

Ejercicios

8.1. Considérese el problema de programación lineal:

$$\begin{array}{ll} \text{minimizar} & 3x_1 + 10x_2 + 8x_3 + 16x_4 + 20x_5 \\ \text{s. a} & \begin{array}{l} x_1 + 2x_2 + 4x_3 + 4x_4 - x_5 \geq 4,5 \\ -x_1 + x_2 - x_3 + x_4 + 4x_5 \geq 1 \\ x_1, \dots, x_5 \geq 0. \end{array} \end{array}$$

- a) Determinar todas las soluciones óptimas del problema y el valor de la función objetivo.
- b) Probar que las soluciones obtenidas son realmente óptimas.

8.2. Resolver el siguiente problema de programación lineal:

$$\begin{array}{ll} \text{minimizar} & -x_1 - 2x_2 \\ \text{s. a} & \begin{array}{l} -3x_1 + 2x_2 \leq -1 \\ x_1 - x_2 \leq 1 \\ -2x_1 + 7x_2 \leq 6 \\ 9x_1 - 4x_2 \leq 6 \\ -5x_1 + 2x_2 \leq -3 \\ 7x_1 - 3x_2 \leq 6 \\ x_1, x_2 \geq 0. \end{array} \end{array}$$

8.3. Demostrar si los puntos que se indican son la solución óptima de los siguientes programas lineales:

$$\begin{array}{ll} \text{a) minimizar} & 7x_1 + 6x_2 + 5x_3 - 2x_4 + 3x_5 \\ \text{s. a} & \begin{array}{l} x_1 + 3x_2 + 5x_3 - 2x_4 + 2x_5 \leq 4 \\ 2x_1 + 4x_2 + 4x_3 - 2x_4 + 5x_5 \leq 5 \\ 3x_1 + x_2 + 2x_3 - x_4 - 2x_5 \leq 1 \\ x_1, \dots, x_5 \geq 0. \end{array} \end{array}$$

Solución propuesta:

$$\mathbf{x}^* = \begin{bmatrix} 0 \\ 4/3 \\ 2/3 \\ 5/3 \\ 0 \end{bmatrix}.$$

$$\begin{array}{ll}
\text{b) minimizar} & 4x_1 + 5x_2 + x_3 + 3x_4 - 5x_5 + 8x_6 \\
\text{s. a} & \begin{array}{l}
x_1 - 4x_3 + 3x_4 + x_5 + x_6 \leq 1 \\
5x_1 + 3x_2 + x_3 - 5x_5 + 3x_6 \leq 4 \\
4x_1 + 5x_2 - 3x_3 + 3x_4 - 4x_5 + x_6 \leq 4 \\
- x_2 + 2x_4 + 4x_5 - 5x_6 \leq 5 \\
-2x_1 + x_2 + x_3 + x_4 + 2x_5 + 2x_6 \leq 7 \\
2x_1 - 3x_2 + 2x_3 - x_4 + 4x_5 + 5x_6 \leq 7 \\
x_1, \dots, x_6 \geq 0.
\end{array}
\end{array}$$

Solución propuesta:

$$\mathbf{x}^* = \begin{bmatrix} 0 \\ 0 \\ 5/2 \\ 7/2 \\ 0 \\ 1/2 \end{bmatrix}.$$

8.4. ¿Cómo se podría interpretar la condición de complementariedad de holguras en términos económicos?

8.5. Considérese el problema de programación lineal:

$$\begin{array}{ll}
\text{minimizar} & -4x_1 - 5x_2 - 4x_3 \\
\text{s. a} & \begin{array}{l}
x_1 + x_3 \leq 4 \\
2x_1 + x_2 - x_3 \leq 3 \\
x_2 + x_3 \leq 5 \\
- x_2 + x_3 \leq 2 \\
-x_1 + x_2 - x_3 \leq -2 \\
x_1 + x_2 + x_3 \leq 7 \\
x_1, \dots, x_3 \geq 0.
\end{array}
\end{array}$$

- ¿Es $\mathbf{x}_0 = [1, 2, 3]^T$ un punto extremo?
- Probar que $\mathbf{x}_0$ es un punto extremo.
- Construir el problema dual. ¿Cuál es su óptimo?
- Determinar todos los óptimos alternativos del programa primal.

8.6. Resolver los siguientes problemas de programación lineal usando el método dual del *simplex* de la tabla 8.2 y partiendo de la base indicada:

$$\begin{array}{ll}
\text{a) minimizar} & 10x_1 + 3x_3 + x_4 \\
\text{s. a} & \begin{array}{l}
x_1 + 2x_2 - x_3 - x_4 = -2 \\
3x_1 - 3x_2 + x_3 + 2x_4 = 5 \\
x_1, \dots, x_4 \geq 0.
\end{array}
\end{array}$$

Base inicial:

$$B = [\mathbf{a}_2, \mathbf{a}_3] = \begin{bmatrix} 2 & -1 \\ -3 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} -1 & -1 \\ -3 & -2 \end{bmatrix}.$$

$$\begin{array}{ll}
\text{b) minimizar} & 3x_1 + 2x_2 + 3x_3 + 8x_4 \\
\text{s. a} & \begin{array}{l}
x_1 + x_2 + 2x_3 + 3x_4 - x_5 = 4 \\
2x_1 - x_2 + x_3 + 4x_4 = 5 \\
x_1, \dots, x_5 \geq 0.
\end{array}
\end{array}$$

Base inicial:

$$B = [\mathbf{a}_2, \mathbf{a}_5] = \begin{bmatrix} 1 & -1 \\ -1 & 0 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 0 & -1 \\ -1 & -1 \end{bmatrix}.$$

$$\begin{array}{ll} \text{c) minimizar} & -2x_1 + 5x_2 - x_3 + x_4 \\ \text{s. a} & \begin{array}{rcl} -4x_1 + x_2 - x_3 - 3x_4 - x_5 & = & -23 \\ x_1 + x_2 - x_3 + 2x_4 - x_6 & = & 7 \\ x_1, \dots, x_6 & \geq & 0. \end{array} \end{array}$$

Base inicial:

$$B = [\mathbf{a}_2, \mathbf{a}_6] = \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 1 & 0 \\ 1 & -1 \end{bmatrix}.$$

$$\begin{array}{ll} \text{d) minimizar} & 10x_1 + 8x_2 + 3x_3 - 4x_4 \\ \text{s. a} & \begin{array}{rcl} 6x_1 + 3x_2 + 2x_3 - x_4 & = & 9 \\ 5x_1 - 2x_2 + x_3 + 4x_4 & = & 1 \\ x_1, \dots, x_4 & \geq & 0. \end{array} \end{array}$$

Base inicial:

$$B = [\mathbf{a}_3, \mathbf{a}_4] = \begin{bmatrix} 2 & -1 \\ 1 & 4 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 4/9 & 1/9 \\ -1/9 & 2/9 \end{bmatrix}.$$

$$\begin{array}{ll} \text{e) minimizar} & 6x_1 + 4x_2 + x_3 \\ \text{s. a} & \begin{array}{rcl} -x_1 + x_2 - x_3 + x_4 & = & 1 \\ -2x_1 - 2x_2 + x_3 + x_5 & = & -1 \\ x_1, \dots, x_5 & \geq & 0. \end{array} \end{array}$$

Base inicial:

$$B = [\mathbf{a}_1, \mathbf{a}_3] = \begin{bmatrix} -1 & -1 \\ -2 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} -1/3 & -1/3 \\ -2/3 & 1/3 \end{bmatrix}.$$

$$\begin{array}{ll} \text{f) minimizar} & 3x_1 + 4x_2 + x_3 + 5x_4 + 5x_5 \\ \text{s. a} & \begin{array}{rcl} 3x_1 + x_2 + x_3 + x_4 & = & -1 \\ x_1 + 2x_2 - x_3 + x_5 & = & 1 \\ x_1, \dots, x_5 & \geq & 0. \end{array} \end{array}$$

Base inicial:

$$B = [\mathbf{a}_1, \mathbf{a}_5] = \begin{bmatrix} 3 & 0 \\ 1 & 1 \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} 1/3 & 0 \\ -1/3 & 1 \end{bmatrix}.$$

8.7. Determinar el dual de

$$\begin{array}{ll} \text{minimizar} & \mathbf{c}^T \mathbf{x} \\ \text{s. a} & \begin{array}{rcl} A\mathbf{x} & = & \mathbf{b} \\ \mathbf{x} & \geq & \mathbf{a}, \end{array} \end{array}$$

donde $\mathbf{a} \geq \mathbf{0}$.

8.8. Construir un ejemplo en el que el programa primal no tenga solución factible y el dual tampoco.

8.9. Considérese el programa lineal

$$\begin{array}{ll} \text{minimizar} & \mathbf{c}^T \mathbf{x} \\ \text{s. a} & \begin{array}{rcl} A\mathbf{x} & = & \mathbf{b} \\ \mathbf{x} & \geq & \mathbf{0}. \end{array} \end{array}$$

Supóngase que este es el programa primal y que tanto él como su dual son factibles. Sea λ la solución óptima del dual:

- Si la ecuación k -ésima del programa primal se multiplica por $\mu \neq 0$, determinar una solución óptima $\mathbf{w}$ del dual.
- Supóngase que en el programa primal original se añade a la ecuación r -ésima la k -ésima multiplicada por ν . ¿Cuál es la solución óptima del correspondiente programa dual?
- Supóngase que en el programa primal original se añade a $\mathbf{c}$ la fila k -ésima de A multiplicada por μ . ¿Cuál es la solución del correspondiente programa dual?

- 8.10. Una firma textil es capaz de producir tres productos distintos x_1 , x_2 y x_3 . Su plan de producción para el próximo mes debe cumplir las siguientes condiciones:

$$\begin{aligned} x_1 + 2x_2 + 2x_3 &\leq 12 \\ 2x_1 + 4x_2 + x_3 &\leq f \\ x_1, x_2, x_3 &\geq 0. \end{aligned}$$

La primera condición la define la disponibilidad del equipo instalado, y es fija. La segunda la determina la disponibilidad de algodón.

El beneficio obtenible de los tres productos son 2, 3 y 3 unidades, respectivamente; depende del coste del algodón y de los costes fijos.

- Determinar el precio sombra, λ_2 , del algodón en función de f (usar el algoritmo dual del *simplex*). Dibujar $\lambda_2(f)$ y el beneficio neto $z(f)$ en función del coste del algodón.
- La firma puede comprar algodón en el mercado a un precio igual a 1/6. También lo puede adquirir, sin embargo, a un suministrador habitual a un precio igual a 1/12. Determinar el beneficio neto de la firma $\pi(s)$ como función de s .

- 8.11. Usando sólo las propiedades del óptimo de un programa lineal, determinar la solución $\mathbf{x}^*$ de

$$\begin{aligned} &\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} && \mathbf{c}^T \mathbf{x} \\ &\text{s. a} && \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}, \end{aligned}$$

donde $\mathbf{l}$ y $\mathbf{u}$ son vectores cuyos componentes, todos finitos, satisfacen $l_i \leq u_i$, $i = 1, \dots, n$.

- 8.12. Formular el programa dual de los siguientes programas lineales:

- $\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \mathbf{c}^T \mathbf{x}$
s. a $A\mathbf{x} \geq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$.
- $\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \mathbf{c}^T \mathbf{x}$
s. a $A\mathbf{x} \geq \mathbf{b}$, $\mathbf{a}^T \mathbf{x} \geq \beta$.
- $\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimizar}} \mathbf{c}^T \mathbf{x}$
s. a $A\mathbf{x} = \mathbf{b}$, $B\mathbf{x} \leq \mathbf{d}$, $\mathbf{x} \geq \mathbf{0}$.

- 8.13. Determinar el dual del programa lineal

$$\begin{aligned} &\underset{\mathbf{x}, \mathbf{y}}{\text{minimizar}} && \mathbf{g}^T \mathbf{z} - \mathbf{f}^T \mathbf{y} \\ &\text{s. a} && \mathbf{y} - \mathbf{z} = \mathbf{c}, \quad \mathbf{y}, \mathbf{z} \geq \mathbf{0}, \end{aligned}$$

donde $\mathbf{f}$, $\mathbf{g}$ y $\mathbf{c}$ son vectores cuyos componentes son todos finitos.

- Usar el programa dual para determinar en qué condiciones el programa primal tiene solución acotada.
- Resolver el dual y de él obtener el valor de las variables del primal.

- 8.14. Sea

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ -1 & 4 \end{bmatrix} \quad \text{y} \quad \mathbf{c} = \begin{bmatrix} 1 \\ 4 \end{bmatrix}.$$

¿Cuál de los dos sistemas siguientes tiene solución?

$$\text{Sistema 1: } A\mathbf{x} \leq \mathbf{0} \quad \mathbf{c}^T \mathbf{x} > 0.$$

$$\text{Sistema 2: } \mathbf{w}^T A = \mathbf{c} \quad \mathbf{w} \geq \mathbf{0}.$$

Ilustrar geométricamente la respuesta.

8.15. Escribir las condiciones de óptimo de cada uno de los programas lineales siguientes:

a) maximizar $\mathbf{c}^T \mathbf{x}$
s. a $\mathbf{Ax} \leq \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}.$

b) maximizar $\mathbf{c}^T \mathbf{x}$
s. a $\mathbf{Ax} \geq \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}.$

c) maximizar $\mathbf{c}^T \mathbf{x}$
s. a $A_1 \mathbf{x} = \mathbf{b}_1$
 $A_2 \mathbf{x} \geq \mathbf{b}_2, \quad \mathbf{x} \geq \mathbf{0}.$

d) maximizar $\mathbf{c}^T \mathbf{x}$
s. a $\mathbf{Ax} = \mathbf{b}$
 $\mathbf{l} \leq \mathbf{x} \leq \mathbf{u}.$

8.16. Probar que una función objetivo alcanza un mínimo en un punto extremo de un poliedro si en todos los adyacentes a ese punto el valor de la función objetivo es mayor o igual. ¿Se puede generalizar este aserto al caso de un politopo no acotado?

8.17. Considérese el problema

$$\begin{array}{ll} \text{minimizar} & \mathbf{c}^T \mathbf{x} \\ & \mathbf{x} \in \mathbb{R}^n \\ \text{s. a} & \mathbf{Ax} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}. \end{array}$$

Si $\mathbf{x}^*$ es un punto extremo óptimo, probar que el siguiente punto extremo mejor (supóngase es único) ha de ser adyacente a $\mathbf{x}$. ¿Qué pasa si la suposición de unicidad de ese punto no se tiene en cuenta?

8.18. Considérese el problema

$$\begin{array}{ll} \text{minimizar} & \mathbf{c}^T \mathbf{x} \\ & \mathbf{x} \in \mathbb{R}^n \\ \text{s. a} & \mathbf{Ax} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}, \end{array}$$

donde $m = n$, $\mathbf{c} = \mathbf{b}$ y $A = A^T$. Probar mediante dualidad que si existe un $\mathbf{x}_0$ tal que $A\mathbf{x}_0 = \mathbf{b}$, $\mathbf{x}_0$ es un punto óptimo.

8.19. Usando el lema de Farkas, probar que si el programa primal

$$\begin{array}{ll} \text{minimizar} & \mathbf{c}^T \mathbf{x} \\ & \mathbf{x} \in \mathbb{R}^n \\ \text{s. a} & \mathbf{Ax} \geq \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}, \end{array}$$

no tiene solución factible y el dual sí la tiene, el programa dual no está acotado.

Capítulo 9

PROGRAMAS LINEALES DE ESTRUCTURA ESPECIAL

EL ESFUERZO DE CÁLCULO que requiere una iteración del método *simplex* se dedica fundamentalmente a la resolución de los sistemas de ecuaciones lineales $B^T \pi = c_B$ y $By = a_q$, así como a la adaptación periódica de la representación (en los códigos modernos en la forma *LU*) que se tenga de la inversa de la matriz básica, B^{-1} . Si el problema es grande, para que el tiempo de obtención de su solución sea lo menor posible, además de utilizar técnicas de matrices dispersas, es muy conveniente aprovechar cualquier estructura especial que presente la matriz de coeficientes de las condiciones. En los próximos apartados estudiaremos algunos problemas con esa estructura especial y la forma de utilizarla para mejorar la eficacia numérica del método *simplex*.

9.1 Problemas de flujos en redes

Los problemas de flujos en redes son aquellos que, de entre todos los programas lineales, registran las cotas más altas de eficacia en su resolución. Campos de la ciencia y de la técnica como los del control de tráfico en carreteras y aéreo, transporte de energía eléctrica, control ferroviario, asignación de circuitos telefónicos en centrales, aprovechamientos hidráulicos para generación de energía eléctrica y regadíos, etc, por sólo mencionar unos pocos, se benefician a diario de los importantísimos avances que las técnicas de flujos en redes han experimentado desde los años 50 en que Kantorovich y Koopmans comenzaran su elaboración y estudio.

La modelización matemática de todo tipo de red física, natural o artificial, es sencilla. Con una breve preparación, personas no especializadas pueden acceder y comprender rápidamente el funcionamiento de los modelos matemáticos basados en ellas. Esta combinación *gran implantación-facilidad de asimilación* ha potenciado en los últimos años el estudio de la rama de las matemáticas y de la investigación operativa que se dedica a la *optimización en redes*. La matemática, la ciencia de los ordenadores, la ingeniería eléctrica, y muchas otras discipli-

nas, influyen en su desarrollo, habiéndose constituido su estudio en una auténtica ciencia a la que dedican muchos cursos las universidades y gran número de publicaciones la literatura especializada.

Para la modelización matemática de los sistemas que estudia, la optimización en redes utiliza como base la teoría de grafos. Aun cuando en el capítulo 3 ya se han introducido diversos conceptos relativos a esta teoría, a continuación se exponen las nociones más elementales adaptadas a su aplicación a la resolución de programas lineales en redes.

9.1.1 Conceptos básicos de teoría de grafos

Un *grafo*, $G = (V, E)$, es un par formado por un conjunto finito, V , de elementos denominados *vértices* o *nudos* del grafo, y por otro también finito, E , de *arcos* o *aristas*. Un arco es un par de nudos. Si los arcos de un grafo son ordenados, el grafo se denomina *digrafo* o *grafo dirigido*; si no, grafo a secas o *grafo no dirigido*. Un grafo no dirigido se puede ver como un digrafo en el que si el arco $e = (u, v) \in E$, también $e' = (v, u) \in E$. Si $e = (i, j) \in E$, este arco une un nudo de *origen* o *cola* $i = t(e)$ con otro de *destino* o *cabeza* $j = h(e)$. En la figura 9.1 se representa un grafo dirigido.

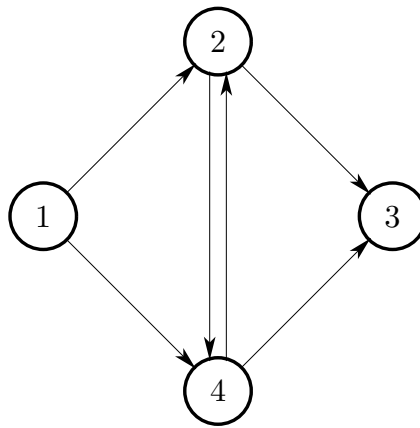


Figura 9.1

Grafo dirigido, o digrafo, de 4 nudos y 6 arcos

Se denomina *grado de un nudo* al número de nudos unidos a él. Un *camino*, P , de un grafo dirigido es una sucesión alternada $(i_0, e_1, i_1, \dots, e_l, i_l)$ de nudos y arcos en la que $e_k = (i_{k-1}, i_k)$ es un arco hacia adelante o $e_k = (i_k, i_{k-1})$ un arco hacia atrás, para $1 \leq k \leq l$. En otras palabras, una sucesión de nudos y arcos en la que el final, destino o cabeza de cada arco coincide con el origen o cola del siguiente. Ese camino va de i_0 a i_l siendo su longitud l . Una *cadena* es una estructura similar al camino excepto que no es necesario que el nudo final de cada arco coincida con el comienzo del siguiente. Un camino en el que $i_0 = i_l$ se denomina *circuito*. Una cadena en la que $i_0 = i_l$ se denomina *ciclo*. Un camino (cadena) de un digrafo se dice *hamiltoniano* si pasa una vez y nada más que una vez por cada uno de los nudos de ese grafo. Un camino (cadena) de un grafo se dice *euleriano* si comprende todas las aristas de ese grafo. Un grafo dirigido se dice *conexo* si cada par de nudos distintos se pueden unir por un camino. Un digrafo se denomina *acíclico* si no contiene ningún ciclo o circuito. Un *árbol* es

un grafo dirigido acíclico y conexo. Un *árbol maximal* es un árbol que abarca todos los nudos del digrafo. Un digrafo $H = (W, F)$ es un *subgrafo* de G si $W \subseteq V$ y $F \subseteq E$; si $W = V$ ese subgrafo se dice *subgrafo maximal*. En la figura 9.2 se describen un camino, una cadena, un circuito, un ciclo y un árbol.

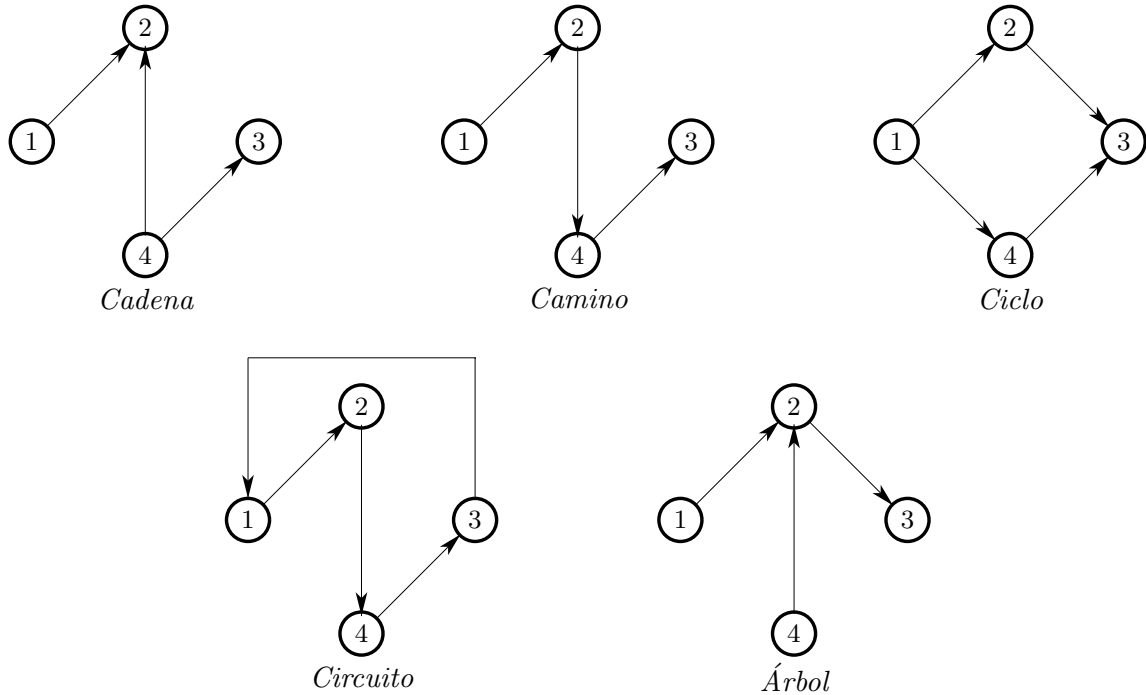


Figura 9.2

Algunas estructuras básicas de un grafo dirigido

Además de la representación esquemática, un grafo dirigido se puede caracterizar mediante la denominada *matriz de incidencia nudo-arco* del grafo. Para un digrafo $G = (V, E)$, los coeficientes de la matriz de incidencia nudo-arco, A , se definen de la siguiente manera:

$$a_{ij} = \begin{cases} +1 & \text{si } t(j) = i, \\ -1 & \text{si } h(j) = i, \\ 0 & \text{en cualquier otro caso.} \end{cases}$$

La matriz de incidencia nudo-arco del grafo de la figura 9.1 es

$$A = \begin{matrix} & \begin{matrix} (1,2) & (1,4) & (2,4) & (4,2) & (2,3) & (4,3) \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & -1 \\ 0 & -1 & -1 & 1 & 0 & 1 \end{bmatrix} \end{matrix}.$$

Mediante un grafo dirigido se tiene la posibilidad de representar la estructura de comunicaciones entre nudos. Cuando existe la posibilidad de que a lo largo de sus arcos haya *flujo* de algo —mercancías, agua, gas, etc.—, al grafo dirigido se le denomina *red*. Una red puede

representar cosas diversas: desde un problema matemático estrictamente, hasta un sistema de transporte, red de comunicaciones terrestre, marítima o aérea, etc. Un flujo en un arco dirigido (i, j) es un número $x_{ij} \geq 0$. Los flujos en los arcos de una red deben satisfacer el criterio de conservación en cada nudo. Los flujos no pueden crearse o perderse en un nudo: el flujo total que entra en un nudo debe ser igual al total que sale, a menos que ese nudo sea una *fuentes* o *nudo oferta* —suministrador de flujo—, o un *sumidero* o *nudo de demanda* —que absorbe flujo—.

9.1.2 Problemas típicos de flujos en redes

Supóngase que $G = (V, E)$ es un grafo dirigido conexo de m nudos, que en cada nudo $i \in V$ se oferta una cantidad de producto b_i ¹ de tal forma que $\sum_{i=1}^m b_i = 0$, que el flujo por un arco (i, j) se designa por x_{ij} y que el coste de transportar una unidad de producto de i a j es c_{ij} . El problema

$$\begin{aligned} \min. \quad & \sum_{i=1}^m \sum_{j=1}^m c_{ij} x_{ij} \\ \text{s. a} \quad & \sum_{j=1}^m x_{ij} - \sum_{k=1}^m x_{ki} = b_i, \quad i = 1, \dots, m \\ & l_{ij} \leq x_{ij} \leq u_{ij}, \quad i, j = 1, \dots, m, \end{aligned}$$

es el denominado *problema del coste mínimo*: el problema por excelencia de flujos en redes. Las condiciones de igualdad de este problema —ecuaciones de Kirchhoff— indican que el flujo total que sale del nudo i menos el que llega a ese nudo ha de ser igual al balance oferta/demanda en dicho nudo. Estas condiciones se suelen denominar de conservación o balance. Si las condiciones $l_{ij} \leq x_{ij} \leq u_{ij}$ se reducen a $x_{ij} \geq 0$, el problema se conoce como el *problema del transbordo*.

Si A designa la matriz de incidencia nudo arco del grafo G , el problema de mínimo coste se puede escribir de la siguiente manera:

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}. \end{aligned} \tag{9.1}$$

El de transbordo, como

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{9.2}$$

Tanto (9.1) como (9.2) son problemas de programación lineal con una formulación idéntica a la de capítulos anteriores. En principio, por tanto, se les puede aplicar sin ningún tipo de condicionamiento el método *simplex*, tal y como lo hemos descrito para resolver problemas con variables acotadas o en forma estándar. Ahora bien, una matriz como A , en la que en cada columna sólo hay un 1 y un -1 , y en la que la suma de todas las filas es cero, hace pensar en

¹Si $b_i > 0$ el nudo i es de oferta, si $b_i < 0$ de demanda y si $b_i = 0$ el nudo se denomina de *transbordo*.

la posibilidad de utilizar un procedimiento que intercale, entre los distintos pasos del método *simplex*, otro u otros pasos que saquen partido de ese hecho y que aumenten su eficacia.

Veamos ahora algunos problemas muy conocidos relativos a flujos en redes y cómo plantearlos en forma de problemas de mínimo coste o de transbordo.

El problema del camino más corto

Consiste en determinar el camino más corto (menos costoso) que debe recorrer un vehículo, fluido, etc, entre dos puntos de una determinada red. El coste de utilizar un camino —en términos monetarios o de distancia— es la suma de los costes de cada uno de los arcos o vías que debe atravesar ese vehículo.

Si se desea plantear este problema de la misma manera que el de mínimo coste, asignemos al nudo de partida del camino, b_1 , el número 1 y al de llegada, b_m , el -1 ; a todos los demás b_i , $2 \leq i \leq m-1$, el cero. A partir de aquí, su formulación es:

$$\begin{aligned} \min. \quad & \sum_{i=1}^m \sum_{j=1}^m c_{ij} x_{ij} \\ \text{s. a} \quad & \sum_{j=1}^m x_{ij} - \sum_{k=1}^m x_{ki} = \begin{cases} 1 & \text{si } i = 1 \\ 0 & \text{si } 2 \leq i \leq m-1 \\ -1 & \text{si } i = m \end{cases} \\ & x_{ij} = 0 \text{ ó } 1, \quad i, j = 1, 2, \dots, m. \end{aligned}$$

Las condiciones $x_{ij} = 0$ ó 1 indican que cada x_{ij} podrá estar o no en el camino más corto. La propiedad que tiene la matriz de condiciones de este problema de ser *totalmente unimodular* (todas sus submatrices cuadradas² tienen determinante igual a 0, $+1$ ó -1) asegura que si existe una solución óptima ésta es entera con todos los valores de las variables 0 ó 1. El problema que se ha de resolver es, entonces,

$$\begin{aligned} \min. \quad & \sum_{i=1}^m \sum_{j=1}^m c_{ij} x_{ij} \\ \text{s. a} \quad & \sum_{j=1}^m x_{ij} - \sum_{k=1}^m x_{ki} = \begin{cases} 1 & \text{si } i = 1 \\ 0 & \text{si } 2 \leq i \leq m-1 \\ -1 & \text{si } i = m \end{cases} \\ & x_{ij} \geq 0, \quad i, j = 1, 2, \dots, m. \end{aligned}$$

El problema del flujo máximo

Consiste en determinar el flujo máximo —automóviles en una red viaria, fluido de cualquier tipo en una red de tuberías, etc.— que puede atravesar una red entre dos puntos dados: entre un nudo origen u oferta y entre otro demanda o destino. Los arcos, como es lógico, tienen una capacidad máxima de absorción de flujo.

Para plantear este problema como uno de coste mínimo se procede de la siguiente manera: se añade un arco ficticio de capacidad infinita a la red original que una el nudo origen con el

² B^{-1} tiene todos sus coeficientes 0, $+1$ ó -1 y, por consiguiente, todos los elementos de $\mathbf{x}_B = B^{-1}\mathbf{b}$ también son 0 ó 1

nudo destino, asignándosele un coste igual a -1 al flujo que ha de circular por ese arco; a los demás nudos se les asigna una oferta/demanda igual a cero y, por último, se atribuye un coste igual a cero al flujo por el resto de los arcos de la red. El mínimo coste de esta red será aquel que maximice el flujo por el arco ficticio. La formulación del problema en el grafo $G = (V, E)$ es, por consiguiente,

$$\begin{aligned} \min. \quad & x_f \\ \text{s. a} \quad & \sum_{j=1}^m x_{ij} - \sum_{k=1}^m x_{ki} = \begin{cases} x_f & \text{si } i = 1 \\ 0 & \text{si } 2 \leq i \leq m-1 \\ -x_f & \text{si } i = m \end{cases} \\ & l_{ij} \leq x_{ij} \leq u_{ij}, \quad i, j = 1, 2, \dots, m \\ & x_f \geq 0, \end{aligned}$$

donde f designa el arco ficticio.

Ejemplo 9.1 Supongamos que se quiere determinar el flujo máximo entre los nudos 1 y 3 de la red de la figura 9.3(a), en la que, entre paréntesis, se indican los límites inferior y superior del flujo que puede absorber cada arco. Este flujo máximo se puede calcular obteniendo el flujo de coste mínimo en la red de la figura 9.3(b) —en ésta las capacidades mínima y máxima de flujo en los arcos y su coste se indican de la forma (l, u, c) —. ●

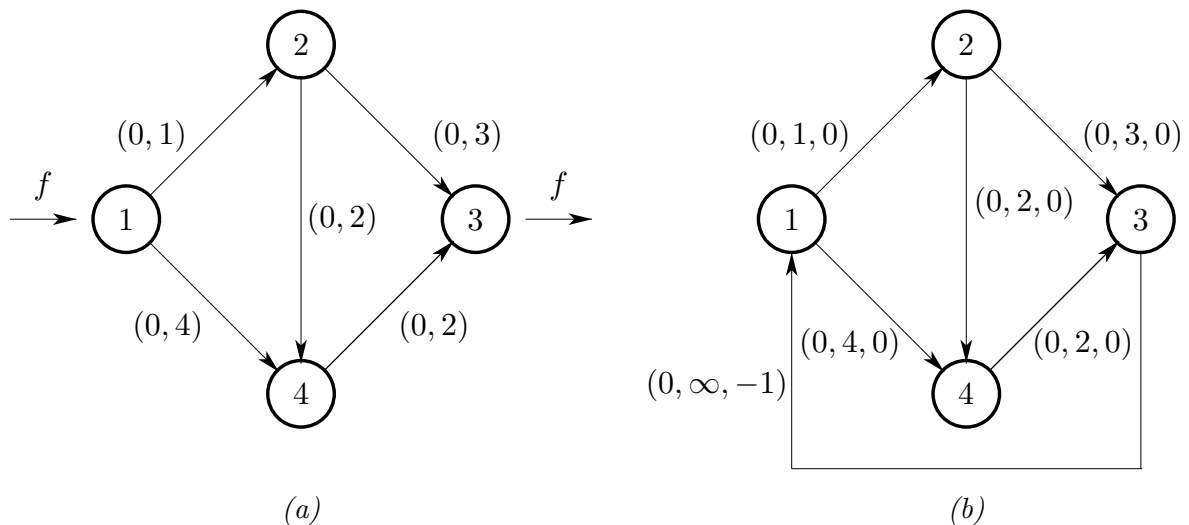
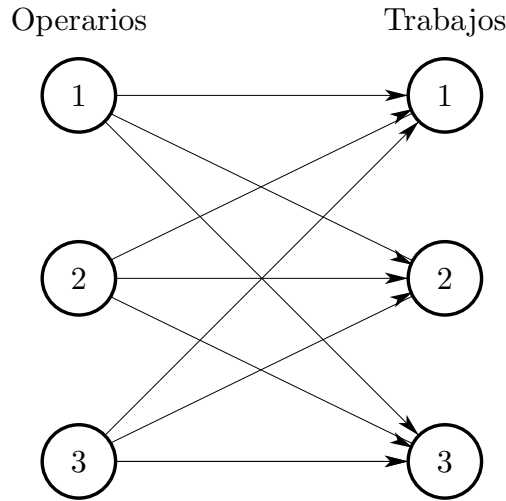


Figura 9.3

Flujo máximo en una red y su formulación como problema de coste mínimo

El problema de la asignación

Este problema es otro de los clásicos de la teoría de grafos. Consiste en asignar a m trabajos, proyectos o actividades, m operarios, cantidades de dinero, etc., de tal manera que el coste global que ello comporta sea mínimo. En la figura 9.4 se esquematiza un ejemplo en forma de

**Figura 9.4**

El problema de la asignación en forma de grafo

grafo de las posibles asignaciones de tres operarios a tres trabajos en una determinada fábrica. El problema de la asignación se puede plantear en los mismos términos del problema de coste mínimo, para un grafo $G = (V, E)$ de m nudos, de la siguiente manera:

$$\begin{aligned}
 \min. \quad & \sum_{i=1}^m \sum_{j=1}^m c_{ij} x_{ij} \\
 \text{s. a} \quad & \sum_{j=1}^m x_{ij} = 1 \quad i = 1, \dots, m \\
 & \sum_{i=1}^m x_{ij} = 1 \quad j = 1, \dots, m \\
 & x_{ij} \geq 0, \quad i, j = 1, \dots, m.
 \end{aligned}$$

Las variables que designan el flujo por los arcos, x_{ij} , sólo pueden tomar valores 0 ó 1. El hecho de que la matriz de coeficientes de las condiciones del problema sea totalmente unimodular, una vez más, asegura que los valores de la solución serán enteros 0 ó 1.

9.1.3 El método simplex para problemas de flujos en redes

Los presentados son sólo una mínima parte de los problemas de flujos en redes que se plantean habitualmente en la técnica, física, ingeniería, economía, etc. Aunque muchos de ellos poseen algoritmos específicos para resolverlos, no son sino una especialización concreta que mejora aspectos parciales del método *simplex* aplicado al problema del coste mínimo.

El estudio de la implementación específica del método *simplex* para problemas de flujos en

redes lo centraremos en resolver el problema del coste mínimo:

$$\begin{aligned} \min. \quad & \sum_{i=1}^m \sum_{j=1}^m c_{ij} x_{ij} \\ \text{s. a} \quad & \sum_{j=1}^m x_{ij} - \sum_{k=1}^m x_{ki} = b_i, \quad i = 1, \dots, m, \\ & l_{ij} \leq x_{ij} \leq u_{ij}, \quad i, j = 1, \dots, m. \end{aligned}$$

Si en el grafo dirigido que representa el problema de mínimo coste, $G = (V, E)$, r es un nudo arbitrario, que se designará como *raíz*, y A y $\mathbf{b}$ son la matriz y vector que resultan de suprimir la fila correspondiente a r en A y en $\mathbf{b}$ del problema (9.1) de la página 502, el problema de coste mínimo es equivalente a:

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}. \end{aligned}$$

Demostremos a continuación que la matriz A tiene rango $n - 1$, siendo n el número de nudos del grafo G .

Lema 9.1 Sea $H = (V, F)$ un subgrafo de un grafo dirigido conexo de n nudos $G = (V, E)$. Los siguientes asertos son equivalentes:

- (i) el número de arcos de H es $n - 1$ y H es conexo.
- (ii) el número de arcos de H es $n - 1$ y H es acíclico.
- (iii) H es conexo y acíclico.
- (iv) H es mínimamente conexo: al quitar cualquier arco H se convierte en inconexo.
- (v) H es máximamente acíclico: al añadir cualquier arco H pasa a ser cíclico.

Si se cumple cualquiera de esos asertos, el grafo H es un árbol maximal de G (recordemos la figura 9.2).

En la tabla 9.1 se expone un sencillo algoritmo para la obtención de un árbol maximal de un grafo $G = (V, E)$.

Ejemplo 9.2 Queremos obtener un árbol maximal del grafo de la figura 9.5(a). Los arcos son e_1, e_2, e_3, e_4 y e_5 .

Iteración 1. Paso 1

$H = (\{1, 2, 3, 4\}, \{e_1\})$ es acíclico; $E^1 = \{e_1\}$.

Iteración 1. Paso 2

Número de elementos en $E^1 = 1 \neq n - 1$.

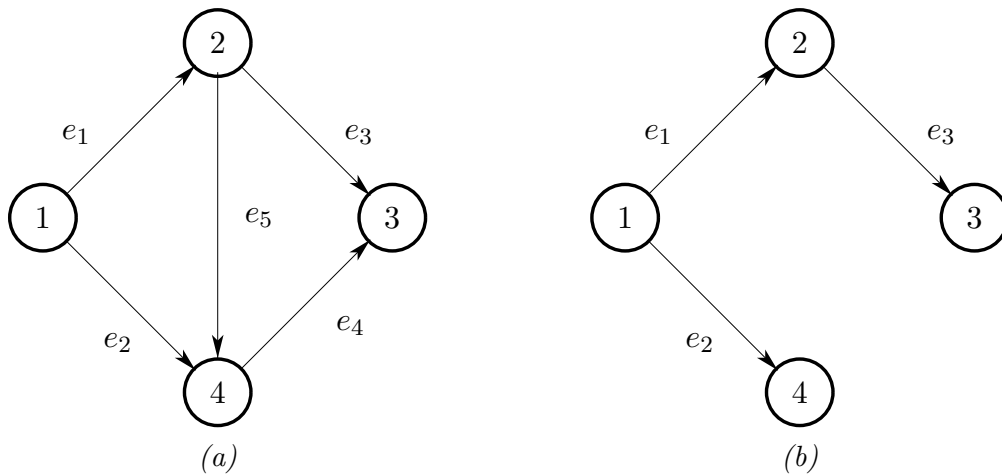
Tabla 9.1

Algoritmo para la obtención de un árbol maximal de un grafo dirigido

- Paso 0** – Establecer una lista de arcos del grafo $G = (V, E)$: $e_1, \dots, e_m$. Hacer $E^0 = \emptyset$, $i \leftarrow 1$.
Paso 1 – Si $H = (V, E^{i-1} \cup \{e_i\})$ es acíclico, hacer $E^i = E^{i-1} \cup \{e_i\}$; si no, $E^i = E^{i-1}$.
Paso 2 – Si el número de elementos de $E^i = n - 1$, parar; H es un árbol maximal. Si no, hacer $i \leftarrow i + 1$ e ir al paso 1.

Iteración 2. Paso 1

$H = (\{1, 2, 3, 4\}, \{e_1, e_2\})$ es acíclico; $E^2 = \{e_1, e_2\}$.

**Figura 9.5**

Determinación del árbol maximal de una red

Iteración 2. Paso 2

Número de elementos en $E^2 = 2 \neq n - 1$.

Iteración 3. Paso 1

$H = (\{1, 2, 3, 4\}, \{e_1, e_2, e_3\})$ es acíclico; $E^3 = \{e_1, e_2, e_3\}$.

Iteración 3. Paso 2

Número de elementos en $E^3 = 3 = n - 1$. Fin del proceso. En la figura 9.5(b) se puede ver el árbol maximal obtenido. ●

Teorema 9.1 Sea $G = (V, E)$ un grafo conexo dirigido de n nudos, $\hat{A}$ su matriz de incidencia nudo-arco, $r \in V$ un nudo arbitrario y A la matriz resultante de $\hat{A}$ al suprimir la fila r . La matriz A es de rango completo, $n - 1$. Si B es una submatriz de A de orden $n - 1$, B es regular si y sólo si sus columnas son las que definen en A los arcos de un árbol maximal de G .

DEMOSTRACIÓN. Obsérvese en primer lugar que, según el lema 9.1, todo grafo conexo tiene un árbol maximal. Para llegar a él, aparte del algoritmo de la tabla 9.1, sólo hay que ir quitando arcos del grafo hasta que el subgrafo resultante sea mínimamente conexo.

Probemos que las columnas de A correspondientes a los arcos de un ciclo del grafo G son linealmente dependientes. En efecto, si P y Q son los conjuntos de arcos hacia adelante y hacia atrás de ese ciclo, se cumplirá que

$$\sum_{e \in P} \mathbf{a}_e - \sum_{e \in Q} \mathbf{a}_e = 0,$$

donde $\mathbf{a}_e$ es el vector columna de la matriz A correspondiente al arco e . Según esto, es suficiente demostrar que cualquier submatriz B de A cuyas columnas definen los arcos de un árbol maximal es regular. Esto último es un resultado inmediato del lema que sigue. ■

Lema 9.2 Sea $H = (V, F)$ un árbol maximal del grafo G y B la correspondiente submatriz de la matriz de incidencia nudo-arco de G , A . Existe una ordenación de las filas y columnas de B que la hace triangular superior y en la que todos los elementos de la diagonal principal son distintos de cero.

DEMOSTRACIÓN. La haremos por inducción en n . Para $n = 1$ la matriz B es de orden 0. Supongamos que $n = 2$. La matriz B será 1×1 de único elemento 1 ó -1 . Supongamos que el enunciado es cierto para $n < k$. Consideremos el caso en que $n = k$. La suma de los grados³ de todos los nudos del grafo H es $2n - 2$. Como cada nudo, al ser H conexo, está unido al menos a otro —su grado es al menos 1—, habrá al menos dos nudos de grado igual a 1 —a los que llamaremos ramas—. Cojamos una de esas ramas, $i \in V$, que no sea el nudo raíz r y sea $e \in F$ el arco que une i al resto del grafo. Consideremos el grafo $H' = (V \setminus \{i\}, F \setminus \{e\})$. De acuerdo con el lema 9.1 (ii), H' es un árbol maximal de $G' = (V \setminus \{i\}, E \setminus \{e\})$, por lo que por la hipótesis de inducción podemos ordenar los nudos y los arcos de H' de tal manera que la matriz correspondiente, B' , sea triangular superior con todos los elementos de la diagonal principal distintos de cero. Añadamos ahora la fila correspondiente al nudo i al final de B' y la columna correspondiente al arco e como última columna. Se obtendrá que

$$B = \begin{bmatrix} B' & \mathbf{u} \\ 0 & \pm 1 \end{bmatrix},$$

para un $\mathbf{u}$ dado. B , por tanto, se ha podido ordenar de acuerdo con la forma pretendida en el enunciado del lema. ■

³Recordemos que el grado de un nudo es el número de nudos a los que está unido.

Un razonamiento idéntico permite concluir que también existe una ordenación de filas y columnas de B que la hace triangular inferior.

Para el caso del ejemplo 9.2 de la página 507, si eligiésemos como nudo raíz $r = 1$, la matriz B que se obtendría a partir de este árbol maximal, suprimiendo de su matriz de incidencia nudo-arco A la fila correspondiente al nudo 1, sería:

$$A = \begin{matrix} & e_1 & e_2 & e_3 \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 1 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & -1 \\ 0 & -1 & 0 \end{bmatrix} \end{matrix} \longrightarrow B = \begin{matrix} & e_1 & e_2 & e_3 \\ \begin{matrix} 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & -1 \\ 0 & -1 & 0 \end{bmatrix} \end{matrix}.$$

En la práctica, en lugar de suprimir la fila correspondiente al nudo raíz, se añade un nudo ficticio al árbol maximal —nudo 0—, unido al raíz por un único arco que sale de él y va al cero, suprimiéndose, esta vez sí, de la matriz A de incidencia nudo-arco del nuevo árbol maximal, la fila correspondiente a ese nudo ficticio. En el caso del ejemplo 9.2, el nudo 0 se une al raíz 1, resultando que:

$$B = \begin{matrix} & e_1 & e_2 & e_3 \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \end{bmatrix} \end{matrix}.$$

Esta matriz B , sin embargo, no es triangular como es de desear.

El grafo correspondiente a este nuevo árbol maximal se representa como en la figura 9.6.

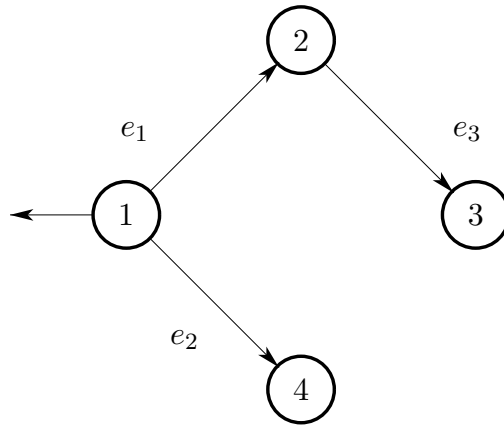


Figura 9.6

Árbol maximal del ejemplo 9.2 con nudo ficticio

Para triangularizar una base de un grafo, una vez obtenido un árbol maximal del mismo y elegido un nudo raíz, se puede utilizar el algoritmo de la tabla 9.2.

Ejemplo 9.3 Triangularizar la matriz del árbol maximal de la figura 9.6 correspondiente al ejemplo 9.2.

Tabla 9.2

Algoritmo para la triangularización de una base

- Paso 0** – Dado un árbol maximal $H = (V, F)$ de n nudos, elegir un nudo raíz r ; hacer $i \leftarrow 1$.
- Paso 1** – Encontrar una rama del árbol. Sea l esa rama y e_s el arco que lleva a ella.
- Paso 2** – Añadir a B la fila correspondiente a l .
- Paso 3** – Si $i = n - 1$, ir al paso 4; si no, hacer $H \leftarrow (V \setminus \{l\}, F \setminus \{e_s\})$, $i \leftarrow i + 1$ e ir al paso 1.
- Paso 4** – Añadir a B la fila correspondiente a r . Hacer la columna n igual a e_n .

Iteración 1. Paso 0 $H = (\{1, 2, 3, 4\}, \{e_1, e_2, e_3\})$, $i \leftarrow 1$.**Iteración 1. Paso 1**Elegimos el nudo 3: arco e_3 .**Iteración 1. Paso 2**La matriz B queda:

$$B = \begin{array}{c} \begin{array}{c} 3 \\ - \\ - \\ 1 \end{array} \end{array} \begin{array}{c} e_3 \\ \left[\begin{array}{cccc} -1 & 0 & 0 & - \\ & & & \end{array} \right] \end{array}.$$

Iteración 1. Paso 3 $i \neq n - 1$. $H = (\{1, 2, 4\}, \{e_1, e_2\})$, $i \leftarrow 2$.**Iteración 2. Paso 1**Elegimos el nudo 2: arco e_1 .**Iteración 2. Paso 2**La matriz B queda:

$$B = \begin{array}{c} \begin{array}{c} 3 \\ 2 \\ - \\ 1 \end{array} \end{array} \begin{array}{c} e_3 \quad e_1 \\ \left[\begin{array}{cccc} -1 & 0 & 0 & - \\ & 1 & -1 & 0 & - \end{array} \right] \end{array}.$$

Iteración 2. Paso 3

$i \neq n - 1$. $H = (\{1, 4\}, \{e_2\})$, $i \leftarrow 3$.

Iteración 3. Paso 1

Elegimos el nudo 4: arco e_2 .

Iteración 3. Paso 2

La matriz B queda:

$$B = \begin{array}{c} \begin{array}{ccc} & e_3 & e_1 & e_2 \end{array} \\ \begin{array}{c} 3 \\ 2 \\ 4 \\ 1 \end{array} \begin{bmatrix} -1 & 0 & 0 & - \\ 1 & -1 & 0 & - \\ 0 & 0 & -1 & - \\ & & & - \end{bmatrix} \end{array}.$$

Iteración 3. Paso 3

$i = n - 1$.

Iteración 3. Paso 4

La matriz B queda:

$$B = \begin{array}{c} \begin{array}{ccc} & e_3 & e_1 & e_2 \end{array} \\ \begin{array}{c} 3 \\ 2 \\ 4 \\ 1 \end{array} \begin{bmatrix} -1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \end{array}.$$

Esta matriz B sí es triangular. ●

El lema 9.2 también demuestra que cualquier submatriz de una de incidencia nudo-arco, A , de orden $n - 1$, tiene un determinante igual a 0, 1 ó -1 (dado que el determinante de una matriz triangular es igual al producto de los elementos de la diagonal principal). De hecho, la demostración del lema 9.2 se puede extender a demostrar que toda submatriz de A tiene un determinante igual a 0, 1 ó -1. En otras palabras, A es totalmente unimodular. Estas matrices tienen una importancia extraordinaria en programación combinatoria pues la inversa de cualquier submatriz regular de una matriz totalmente unimodular tiene todos sus elementos enteros.

Generalizando estas últimas ideas, se puede demostrar que para cualquier vector $\mathbf{b}$ con todos sus elementos enteros, las soluciones básicas del sistema $A\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, en el que A es totalmente unimodular, tienen todos sus elementos enteros.

Corolario 9.1 *El problema del transbordo*

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

tiene la propiedad de que si los elementos del vector $\mathbf{b}$ son todos enteros, cualquier solución básica también tiene todos sus elementos enteros. Más aún, cualquier solución básica $\mathbf{x}$ tiene elementos x_{ij} distintos de cero sólo si éstos son arcos de un árbol maximal: $x_{ij} \in F$, para algún árbol maximal $H = (V, F)$, $F \subseteq E$, del grafo $G = (V, E)$.

9.1.3.1 Implementación práctica del método simplex

El esquema general que se va a seguir en la exposición siguiente es estrictamente el del algoritmo de la tabla 7.4 de la página 454 (algoritmo simplex revisado para variables acotadas). Las diferencias se refieren a la especialización lógica de ciertas etapas al aplicarse a flujos en redes.

9.1.3.1.1 Paso 1. Asignación de precios. Comprobación de condiciones de óptimo

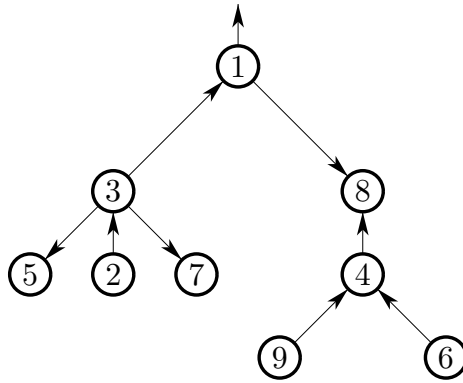
Empezaremos suponiendo que se dispone de una solución básica factible desde la que comenzar el proceso iterativo. La idea clave que caracteriza esta implementación con respecto a las que hemos visto hasta ahora radica en la representación de la matriz B en la forma triangular superior/inferior. Esto permitirá resolver muy rápidamente los sistemas de ecuaciones inherentes al método *simplex*: $B^T \boldsymbol{\pi} = \mathbf{c}_B$ y $B\mathbf{y} = \mathbf{a}_q$.

Para obtener la solución de estos sistemas de ecuaciones, en cualquier caso, es fundamental guardar eficazmente la información relativa a los nudos y arcos de la red, y la de los arcos que forman la base. Un esquema muy utilizado, correspondiente al grafo de la figura 9.7, es el de la tabla 9.3. Con $p(i)$ se designa el nudo *predecesor* del $i \in V$ en el árbol maximal⁴ correspondiente, es decir, el unido a ese i que ocupa un nivel más cercano al raíz. El signo positivo o negativo indica que el arco que lo une con su predecesor va de i a $p(i)$ o de $p(i)$ a i . Por $d(i)$ se designa la *profundidad* (*depth*) de ese nudo i en niveles o escalones con respecto al raíz (2 indica que hay que recorrer un camino de dos arcos como mínimo para llegar al raíz). Por último, $s(i)$ designa el *sucesor* —también denominado *hebra*⁵ en bastantes referencias bibliográficas— del nudo i en una lista de nudos preordenada —*preorden*— de acuerdo con un determinado criterio que se considera oportuno para ese árbol. Sobre esta última estructura, $s(\cdot)$, volveremos más adelante al hablar de su actualización de una iteración a otra.

Utilizando el algoritmo de la tabla 9.2, la matriz B ya ordenada correspondiente al árbol

⁴El árbol maximal, dentro del método *simplex*, también se suele designar en la literatura especializada como *árbol básico*.

⁵*Thread* en inglés.

**Figura 9.7**

Digrafo o grafo correspondiente a los datos de la tabla 9.3

maximal de la figura 9.7 es:

$$B = \begin{matrix} & \begin{matrix} (2,3) & (3,5) & (3,7) & (3,1) & (6,4) & (9,4) & (4,8) & (1,8) \end{matrix} \\ \begin{matrix} 2 \\ 5 \\ 7 \\ 3 \\ 6 \\ 9 \\ 4 \\ 8 \\ 1 \end{matrix} & \left[\begin{array}{cccccccccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 1 \end{array} \right] \end{matrix} . \quad (9.3)$$

Para resolver el sistema $B^T \boldsymbol{\pi} = \mathbf{c}_B$ simplemente habría que llevar a cabo una sustitución

Tabla 9.3

Estructura de datos del grafo de la figura 9.7

Nudo i	1	2	3	4	5	6	7	8	9
$p(i)$	—	+3	+1	+8	−3	+4	−3	−1	+4
$d(i)$	0	2	1	2	2	3	2	1	3
$s(i)$	3	7	5	9	2	—	8	4	6

inversa. Por ejemplo, tomando como matriz B la de (9.3), el sistema a resolver sería:

$$\begin{array}{rcl}
 \pi_2 & - \pi_3 & = c_{23} \\
 - \pi_5 & + \pi_3 & = c_{35} \\
 & - \pi_7 + \pi_3 & = c_{37} \\
 & & \pi_3 - \pi_1 = c_{31} \\
 & \pi_6 & - \pi_4 = c_{64} \\
 & \pi_9 & - \pi_4 = c_{94} \\
 & & \pi_4 - \pi_8 = c_{48} \\
 & & - \pi_8 + \pi_1 = c_{18} \\
 & & \pi_1 = 0.
 \end{array}$$

Esta sencilla operación de sustitución inversa se puede llevar a efecto muy eficazmente, con las estructuras de datos antes definidas, utilizando el algoritmo de la tabla 9.4.

Tabla 9.4

Algoritmo para la obtención de los multiplicadores simplex en el algoritmo *simplex* para flujos en redes

Paso 0 – Inicializar: $\pi_r = 0$; $M \leftarrow \{r\}$; $N \leftarrow V \setminus \{r\}$.

Paso 1 – Encontrar en la lista de nudos uno $i \in N$ tal que $p(i) \in M$. Si $N = \emptyset$, PARAR; el procedimiento ha terminado.

Paso 2 – Si $p(i) > 0$, hacer $\pi_i = \pi_{p(i)} + c_{ip(i)}$; si $p(i) < 0$, hacer $\pi_i = \pi_{p(i)} - c_{p(i)i}$.
Hacer $M \leftarrow M \cup \{i\}$, $N \leftarrow N \setminus \{i\}$ e ir al paso 1

En el ejemplo que venimos estudiando, con los datos de la tabla 9.3, se calcularían π_3 , π_2 , π_5 , π_7 , π_8 , π_4 , π_9 y π_6 .

A los multiplicadores simplex se les suele denominar, en programación en redes, *potenciales de los nudos*. Haciendo un símil hidráulico, estos multiplicadores indican la energía potencial de que dispondría un hipotético fluido que circulase por la red en un determinado nudo y, en función de la de los demás, la capacidad para poderse trasladar de ese nudo a otro.

Una vez determinados los multiplicadores simplex, la siguiente operación a realizar en el método *simplex* consiste en calcular los costes reducidos de las variables/arcos no básicos: $\bar{c}_e = c_e - \boldsymbol{\pi}^T \mathbf{a}_e$ para todo $e \in E \setminus F$. Es decir, si $e = (i, j) \in E \setminus F$, obtener

$$\Psi_1 = \{e : x_e = l_e \text{ y } c_e - \pi_i + \pi_j < 0\} \quad (9.4a)$$

y

$$\Psi_2 = \{e : x_e = u_e \text{ y } c_e - \pi_i + \pi_j > 0\}. \quad (9.4b)$$

Para realizar esta operación sólo es necesario consultar la información relativa a cuál es, para un arco $e = (i, j)$, su origen, i , y su destino, j .

Las condiciones de óptimo en los problemas de flujos en redes tienen una interpretación económica interesante. Como π_r (valor dual, multiplicador simplex o potencial del nudo raíz) es igual a 0, el coste reducido, $\bar{c}_e = c_e - \pi_i + \pi_j$, de un arco no básico, $e = (i, j) \in E \setminus F$, en su límite inferior, expresa el cambio en el valor de la función objetivo que se obtiene enviando

una unidad de flujo a través del árbol maximal desde el nudo raíz r al nudo i , y de éste al raíz, pasando por el nudo j . Un razonamiento inverso se aplicaría al caso de un arco en su límite superior. Las condiciones de óptimo indicarán que no es beneficioso hacer circular esos flujos.

9.1.3.1.2 Paso 2. Determinación de la columna de pivotación

Si $\Psi_1 \cup \Psi_2 = \emptyset$, el problema está resuelto; si no, se escoge un arco de esos dos conjuntos para entrar en la base: en general, el de coste reducido más interesante.

Definida qué variable/arco, x_q o e_q , ha de entrar en la base, la siguiente operación consiste en comprobar la existencia de una solución no acotada. Para ello hay que resolver el sistema $B\mathbf{y} = \mathbf{a}_q$.

El teorema 9.1 de la página 508 nos asegura que para un grafo $G = (V, E)$ y un árbol maximal $H = (V, F)$, entre los nudos del arco $q = (i, j) \in E \setminus F$, en el árbol H , existe una única cadena y sólo una. Es decir, sólo se puede llegar de i a j en ese árbol maximal H de una única manera. Esta cadena, junto con el arco q , constituirá un ciclo. Para resolver $B\mathbf{y} = \mathbf{a}_q$ (buscar una combinación lineal de los arcos de H con la que expresar q) habrá que determinar ese camino. Para hacerlo, bastará con examinar las estructuras de datos de la base o árbol maximal.

Si, por ejemplo, se trata de encontrar el camino entre los nudos $i = 7$ y $j = 9$ en el ejemplo de la tabla 9.3 —página 513—, procederíamos de la siguiente manera: como $d(9) > d(7)$, buscamos el $p(9) = +4$ (el arco va del nudo 9 al nudo 4); $(9, 4)$ tiene el sentido inverso al del camino que buscamos. A continuación vemos que $d(7) = d(4)$ pero $7 \neq 4$; buscamos el $p(7)$ que es -3 , en sentido por tanto también contrario al del camino que buscamos, y $p(4) = 8$, igualmente contrario. Como $3 \neq 8$, buscamos $p(3) = +1$ y $p(8) = -1$. Los arcos $(3, 1)$ y $(1, 8)$ están orientados en el sentido del camino que buscamos y llegan a un nudo común, el raíz. El camino buscado es por tanto:

$$\{7, -e_{37}, 3, +e_{31}, 1, +e_{18}, 8, -e_{48}, 4, -e_{49}, 9\}.$$

9.1.3.1.3 Paso 3. Determinación de la fila de pivotación. Análisis de ratios

Para determinar qué variable/arco ha de salir de la base al incrementar/decrementar el flujo en el arco q , primero habrá que comprobar si en el camino encontrado existen o no arcos en el sentido $i \rightarrow j$; si no, la solución sería no acotada. De existir esos arcos habrá que comprobar cómo absorbe un incremento/decremento de flujo el camino encontrado; es decir, si se incrementa el flujo en (i, j) , qué arco del camino en el sentido $i \rightarrow j$ está más cerca de su límite inferior o, en sentido contrario, más cerca de su límite superior (qué arco se *bloquea* o *satura* antes). Si se decrementa, lo contrario. En concreto, si habiendo elegido un e_q perteneciente a Ψ_1 o Ψ_2 de (9.4a) y (9.4b), haciendo

$$\delta \leftarrow \begin{cases} +1 & \text{si } e_q \in \Psi_1 \\ -1 & \text{si } e_q \in \Psi_2 \end{cases}$$

y designando como C el camino obtenido, $C = \{i, e_1, \dots, e_n, j\}$, y por $O_{e_q}(C)$ la orientación del arco e_q en ese camino, es decir,

$$O_{e_q}(C) = \begin{cases} +1, & \text{si } e_q = (i_k, i_{k+1}) \\ -1, & \text{si } e_q = (i_{k+1}, i_k), \end{cases}$$

se escogería aquel arco que determinase un θ tal que

$$\theta = \min \left\{ \min_{O_{e_k}(C)=\delta} \{x_{e_k} - l_{e_k}, \infty\}, \min_{-O_{e_k}(C)=\delta} \{u_{e_k} - x_{e_k}, \infty\}, u_q - l_q \right\}.$$

En el ejemplo de la tabla 9.3 que venimos siguiendo, si al comienzo de la iteración los valores de las variables (flujos por los arcos) son $x_{35} = x_{94} = 1$, $x_{37} = x_{18} = 2$, $x_{23} = x_{48} = 3$ y $x_{31} = x_{64} = 4$, siendo los límites inferiores de todos los flujos por los arcos del problema 0 y los superiores ∞ , como el flujo por el arco $(7, 9)$ sólo se puede incrementar, el del arco en el camino $7 \rightarrow 9$ que antes se bloquea al decrementar el flujo total por el camino (arco más cerca de su límite inferior) es el del $(1, 8)$.

9.1.3.1.4 Paso 4. Pivotación. Actualización de las estructuras de datos

Este paso consiste en adaptar la solución, la matriz B y las estructuras de datos correspondientes, de acuerdo con los cambios efectuados. Es decir, por lo que respecta a la solución, hacer:

$$x_q \leftarrow x_q + \theta\delta,$$

y

$$x_j \leftarrow x_j - \theta\delta O_{e_j}, \quad \text{para } e_j \in C.$$

En el ejemplo de la tabla 9.3, $x_{79} \leftarrow 0 + 2$, $x_{37} \leftarrow 2 + 2$, $x_{31} \leftarrow 4 - 2$, $x_{18} \leftarrow 2 - 2$, $x_{48} \leftarrow 3 + 2$ y $x_{94} \leftarrow 1 + 2$.

Para adaptar la base y B , una vez adaptados los flujos en los arcos, se definen

$$\Psi_3 = \{e_j : x_j = l_j, O_{e_j}(C) = \delta\}$$

y

$$\Psi_4 = \{e_j : x_j = u_j, -O_{e_j}(C) = \delta\}$$

y se selecciona cualquier $e_m \in \Psi_3 \cup \Psi_4$ reemplazándose en la base el arco e_m por el e_q .

De acuerdo con las consideraciones hechas sobre los multiplicadores simplex, o valores duales, se puede deducir que, de iteración en iteración, en lugar de tener que resolver el sistema $B^T \pi = c_B$ cada vez para reobtenerlos, sólo es necesario recalcular aquellos que se ven modificados por los cambios de base efectuados. En efecto, al añadir un arco cualquiera (i, j) a la base y sacar de ella otro (p, q) , por ejemplo, se obtiene un nuevo árbol maximal. El hecho de retirar de la base el arco (p, q) divide el árbol maximal original en dos subárboles: uno, H_1 , que incluirá el nudo raíz, y otro, H_2 , sin el nudo raíz. El subárbol H_2 , al entrar el arco (i, j) en la base/árbol maximal, pasará a quedar *colgado* del nudo i o del j . El arco (i, j) tendrá en el nuevo árbol maximal un nudo en H_1 y el otro en H_2 . Como $\pi_r = 0$ y $c_a - \pi_k + \pi_l = 0$, para todos los arcos $a = (k, l)$ del nuevo árbol maximal, los valores de los multiplicadores π en los nudos de H_1 permanecerán iguales mientras que los de aquellos en H_2 cambiarán en una cantidad constante: concretamente en $\bar{c}_{ij}$, si $i \in H_1$ y $j \in H_2$, o en $-\bar{c}_{ij}$, si $i \in H_2$ y $j \in H_1$.

Para llevar estas consideraciones a la práctica con las estructuras de datos que venimos estudiando, si el arco que sale de la base es $f = (p, q)$, el que entra $e = (i, j)$, y suponemos que $d(q) = d(p) + 1$, habrá que hacer

$$\pi_q \leftarrow \pi_q \pm \bar{c}_e, \quad k \leftarrow s(q)$$

y, posteriormente, mientras se cumpla que $d(k) > d(q)$,

$$\begin{aligned}\pi_k &\leftarrow \pi_k \pm \bar{c}_e \quad y \\ k &\leftarrow s(k).\end{aligned}$$

Para finalizar este paso de la pivotación, hace falta actualizar las estructuras de datos: $p(\cdot)$, $d(\cdot)$ y $s(\cdot)$. A continuación estudiamos cómo.

9.1.3.1.4.1 Actualización de $s(\cdot)$

Mientras que en el nuevo árbol maximal, $H \cup \{e\} \setminus \{f\}$, si el arco que entra es e y el que sale f , los vectores $p(\cdot)$ y $d(\cdot)$ estarán definidos de forma única, el nuevo $s(\cdot)$ dependerá del preorden en que se consideren los nudos del nuevo árbol maximal. El preorden que se debe utilizar a lo largo de toda la implementación del método *simplex* es aquel que requiera la menor cantidad de operaciones para adaptar el vector $s(\cdot)$ de iteración en iteración.

Para ilustrar ese preorden utilizaremos la figura 9.8. En aras de clarificar lo más posible la exposición, además de no incluir el sentido de los arcos del árbol maximal, los nudos de esta figura se han numerado de tal forma que $1, 2, \dots, 32$ es un preorden. El arco que entra en la base, e , es el que une los nudos 3 y 20; el que sale de la base, f , el que une el 8 y el 9.

En lo que sigue de exposición a los nudos del arco e los designaremos por e_1 y e_2 , a los del f , por f_1 y f_2 , de tal manera que $e_1 \in H_1$ (el subárbol que contiene el nudo raíz), $e_2 \in H_2$, $f_1 \in H_1$ y $f_2 \in H_2$. El camino de e_2 a f_2 en el árbol H se denomina *tronco de pivotación*. Los nudos que lo forman los designaremos por $v_1, v_2, \dots, v_h$ de tal forma que $v_1 = e_2$ y $v_h = f_2$. En el ejemplo de la figura 9.8, $v_1 = 20$, $v_2 = 17$, $v_3 = 16$, $v_4 = 12$ y $v_5 = 9$.

Consideremos un nudo arbitrario $k \in H_2$ y encontremos el subíndice más pequeño, t , tal que v_t pertenezca al camino de k al nudo raíz: para ese subíndice t escribiremos que $k \in G_t$. De esta forma H_2 se divide en subconjuntos disjuntos $G_1, G_2, \dots, G_h$. Consideraremos que cada G_t se ordena de acuerdo con el preorden establecido para H . En el ejemplo que estamos estudiando,

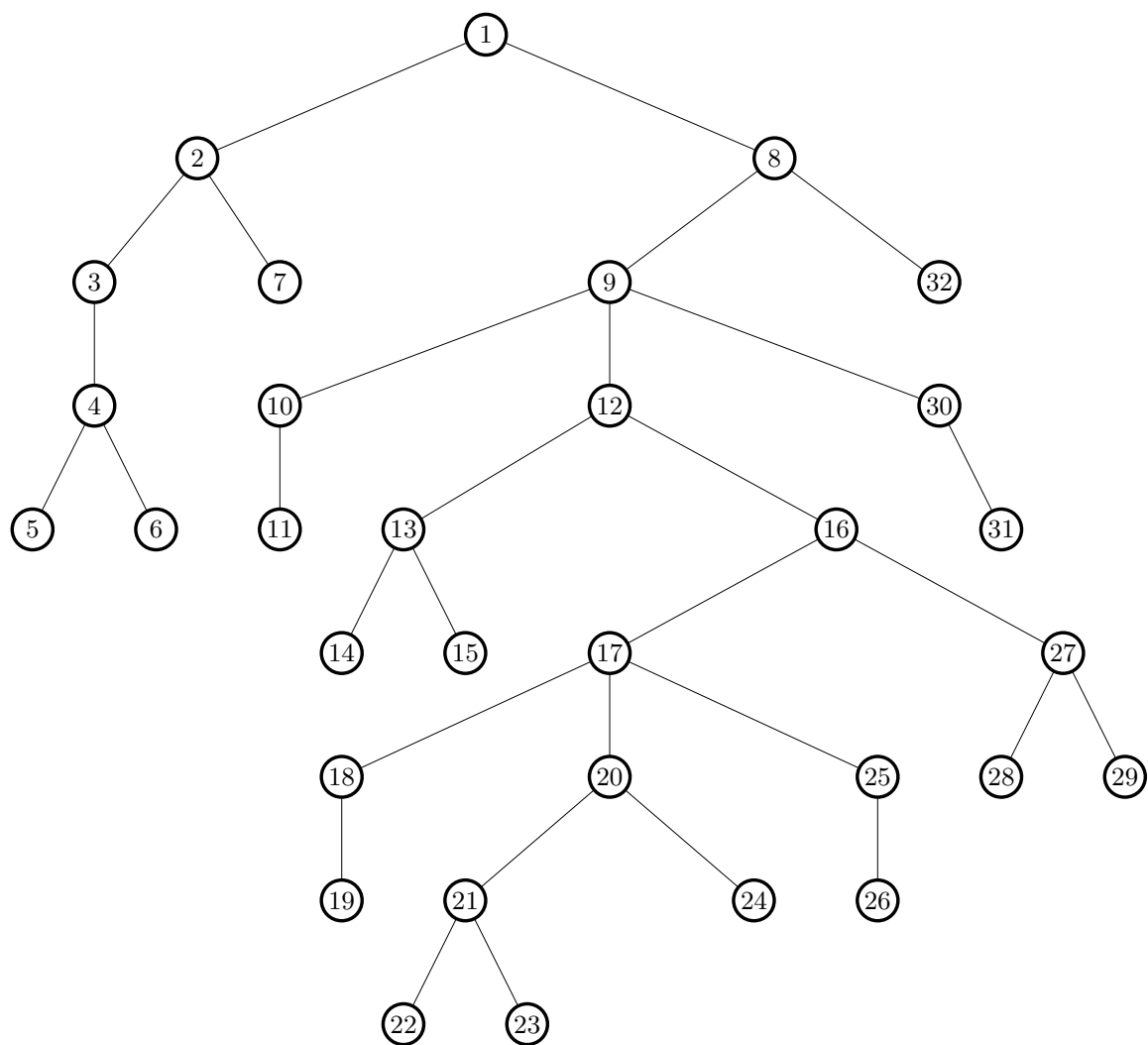
$$\begin{aligned}G_1 &= \{20, 21, 22, 23, 24\} \\ G_2 &= \{17, 18, 19, 25, 26\} \\ G_3 &= \{16, 27, 28, 29\} \\ G_4 &= \{12, 13, 14, 15\} \quad y \\ G_5 &= \{9, 10, 11, 30, 31\}.\end{aligned}$$

La concatenación de $G_1, G_2, \dots, G_h$ la designaremos por G^* . En el ejemplo, $G^* = \{20, 21, 22, 23, 24, 17, 18, 19, 25, 26, 16, 27, 28, 29, 12, 13, 14, 15, 9, 10, 11, 30, 31\}$.

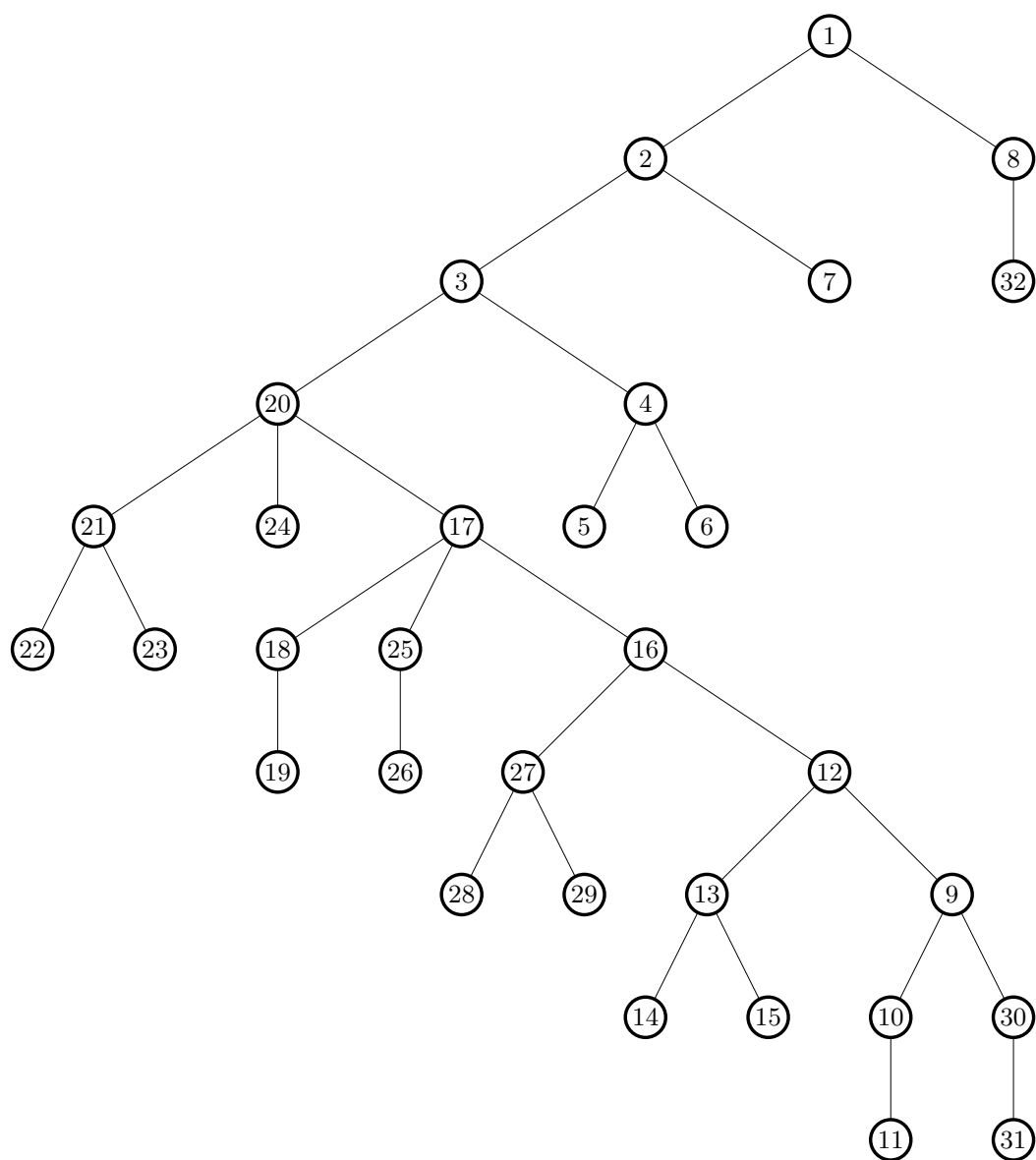
El preorden de $H \cup \{e\} \setminus \{f\}$ se obtiene del de H quitando H_2 y añadiendo G^* directamente al nudo e_1 . En nuestro ejemplo, el nuevo árbol maximal $H \cup \{e\} \setminus \{f\}$ es el de la figura 9.9.

La operación que hemos descrito requiere ciertos cambios en el vector $s(\cdot)$. En efecto, cuando se quita H_2 , hay que cerrar el hueco que se crea: si el primer nudo de H_2 , f_2 , estaba precedido por otro a , el nuevo sucesor de a , $s(a)$, será el antiguo sucesor, z , del último nudo en H_2 . Del mismo modo, para añadir G^* entre e_1 y su antiguo sucesor, b , el primer nudo de G^* , e_2 , se hace nuevo sucesor de e_1 y a b se le hace el sucesor del último nudo de G^* . En el caso de que $e_1 = a$, e_2 se hace nuevo sucesor de e_1 y z nuevo sucesor del último nudo de G^* .

Transformar H_2 en G^* requiere más cambios en $s(\cdot)$. En primer lugar, el nuevo sucesor del último nudo de cada G_t , $t \leq h - 1$, será el primer nudo v_{t+1} de G_{t+1} (como el último nudo de

**Figura 9.8**

Árbol maximal sobre el que se ilustra el proceso de adaptación del vector $s(\cdot)$ una vez efectuada una iteración del método simplex

**Figura 9.9**

Árbol maximal resultante del de la figura 9.8 una vez introducido el arco (3,20) en la base.
Sale el (8,9)

G_h es el último de G^* , su nuevo sucesor será b). Después, un nudo adicional en cada G_t , con $t \geq 2$, puede cambiar de sucesor. Para preparar este cambio, obsérvese que cada G_t , con $t \geq 2$, se separa en una *parte izquierda*, la cual contiene los nudos que aparecen antes de v_{t-1} en el antiguo preorden, y en una *parte derecha*, que contiene los nudos posteriores al v_{t-1} . La parte izquierda siempre incluye el nudo v_t ; la derecha puede estar vacía: por ejemplo, si $t = 4$ en el ejemplo que venimos estudiando. Si la parte derecha no está vacía, el último nudo de la parte izquierda cambia su sucesor pasando de ser v_{t-1} al primero de los de la parte derecha.

En la tabla 9.5 se describe un procedimiento para actualizar el vector $s(\cdot)$ en cada iteración del método *simplex* especializado para optimización de flujos en redes. La variable k escudriña $G_1, G_2, \dots, G_h$, en este orden. Cuando está en G_t , la variable i se refiere a v_t y, si $t \geq 2$, j a v_{t-1} . La variable r designa el primer nudo de la parte derecha del conjunto, de entre todos los $G_t, G_{t+1}, \dots, G_h$ con parte derecha no vacía, con el subíndice más pequeño; si tal conjunto no existe, r es el antiguo sucesor del último nudo de H_2 .

Tabla 9.5

Algoritmo para la actualización del vector $s(\cdot)$ en el método *simplex* especializado para optimización de flujos en redes

- Paso 0** – Inicialización. Hacer $a = f_1$ y, mientras $s(a) \neq f_2$, reemplazar a por $s(a)$. Hacer $b = s(e_1)$ y $i = e_2$.
- Paso 1** – Encontrar el último nudo k de G_1 e inicializar r . Hacer $k = i$ y, mientras $d(s(k)) > d(i)$, reemplazar k por $s(k)$. Hacer luego $r = s(k)$.
- Paso 2** – Si se ha llegado al final de G^* , quitar H_2 y añadir G^* . Si $i = f_2$, hacer $s(a) = r$, $s(e_1) = e_2$ y $s(k) = b$ si $e_1 \neq a$; si $e_1 = a$, $s(e_1) = e_2$ y $s(k) = r$. PARAR.
- Paso 3** – Ascender por el tronco de pivotación y adaptar $s(k)$. Hacer $j = i$, reemplazar i por $p(i)$ y luego $s(k)$ por i .
- Paso 4** – Encontrar el último nudo k en la parte izquierda de G_t . Hacer $k = i$ y, mientras $s(k) \neq j$, reemplazar k por $s(k)$.
- Paso 5** – Si la parte derecha de G_t no está vacía, adaptar $s(k)$, encontrar el último nudo k de G_t y adaptar r . Si $d(r) > d(i)$, reemplazar $s(k)$ por r , mientras $d(s(k)) > d(i)$ reemplazar k por $s(k)$ y, finalmente, reemplazar r por $s(k)$. Ir al paso 2.

9.1.3.1.4.2 Actualización de $p(\cdot)$ y $d(\cdot)$

La transformación de H en $H \cup \{e\} \setminus \{f\}$, gráficamente, se puede considerar que consta de dos pasos: primero, acoplar el arco e ; segundo, cortar el arco f . Como resultado de esto, el tronco de pivotación bascula hacia abajo: cada uno de sus nudos, v_t , colgando antes de un nudo v_{t+1} , pasa a estar colgado de v_{t-1} . A pesar de esto, sin embargo, cada G_t continúa colgado del mismo v_t que lo hacía antes del cambio. Esta observación permite hacer las dos consideraciones siguientes:

- a) El valor de $p(k)$ cambia sólo si k está en el tronco de pivotación. Los valores originales $p(v_1) = v_2, p(v_2) = v_3, \dots, p(v_h) = f_1$ cambian a $p(v_1) = e_1, p(v_2) = v_1, \dots, p(v_h) = v_{h-1}$.

- b) Para cada $t = 1, 2, \dots, h$, hay una constante c_t tal que la nueva $d^*(k)$ de cada $k \in G_t$ es igual a $d(k) + c_t$. Como $d^*(e_2) = d(e_1) + 1$, entonces $c_1 = d(e_1) - d(e_2) + 1$. Como $d^*(v_t) = d^*(v_{t-1}) + 1$ y $d(v_{t-1}) = d(v_t) + 1$, mientras $t \geq 2$, entonces, también mientras $t \geq 2$, $c_t = 2 + c_{t-1}$.

De acuerdo con esto, es muy sencillo incorporar la actualización de $p(\cdot)$ y $d(\cdot)$ al procedimiento de actualización de $s(\cdot)$ de la tabla 9.5. La actualización del vector de multiplicadores simplex también se puede incorporar a ese procedimiento.

En el apéndice F se incluye una breve descripción y el listado de CCNET, un programa en C que implementa el método *simplex* especializado para resolver el problema más general de optimización de flujos en redes: el problema del coste mínimo. Este programa utiliza los procedimientos que hemos venido exponiendo en este capítulo y las estructuras de datos $p(\cdot)$, $d(\cdot)$ y $s(\cdot)$; su actualización se lleva a cabo según acabamos de ver. CCNET está basado en el programa XNET descrito en Grigoriadis [1986].

Refiriéndonos nuevamente al ejemplo de la tabla 9.3, una vez introducido el arco (7,9) en la base y retirado el (1,8), el grafo correspondiente al nuevo árbol maximal se representa en la figura 9.10. La nueva estructura $p(i)$, $d(i)$ y $s(i)$ de la base es la de la tabla 9.6.

Para clarificar la mecánica del método *simplex* aplicado a redes, vamos a resolver un sencillo ejemplo. Insistimos una vez más que aunque el procedimiento resulta fácil, estamos basándonos en una visión directa de la red sobre la que se define el problema. En ordenador, para que esta mecánica sea eficaz, se tiene que suplir de forma acertada esta visión implementando adecuadamente las estructuras de datos que informan a las distintas fases del algoritmo del estado de la red en ese momento y cómo acceder a los distintos nudos y arcos.

Ejemplo 9.4 Resolvamos mediante el método *simplex* para flujos en redes el siguiente pro-

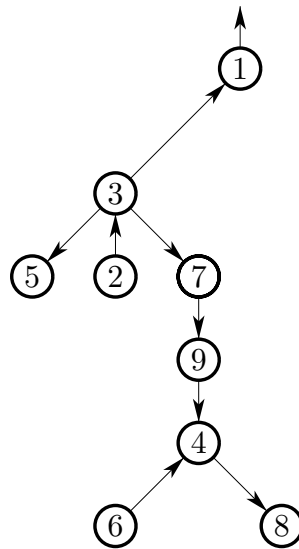


Figura 9.10

Árbol maximal del ejemplo de la tabla 9.3 una vez introducido el arco (7,9) en la base y retirado el (1,8)

Tabla 9.6
Estructura de datos del árbol de la figura 9.10

Nudo i	1	2	3	4	5	6	7	8	9
$p(i)$	–	+3	+1	–9	–3	+4	–3	–4	–7
$d(i)$	0	2	1	4	2	5	2	5	3
$s(i)$	3	7	5	6	2	8	9	–	4

blema:

$$\begin{aligned}
 \min. \quad & x_1 + x_2 + 3x_3 + 10x_4 \\
 \text{s. a} \quad & x_1 + x_3 + x_4 = 5 \\
 & -x_1 + x_2 = 0 \\
 & -x_2 - x_3 - x_4 = -5 \\
 & 0 \leq x_1 \leq 4 \\
 & 0 \leq x_2 \leq 2 \\
 & 0 \leq x_3 \leq 4 \\
 & 0 \leq x_4 \leq 10.
 \end{aligned}$$

La matriz de incidencia nudo-arco de la red que define este problema es

$$\begin{array}{cccc}
 & e_1 & e_2 & e_3 & e_4 \\
 & (1, 2) & (2, 3) & (1, 3) & (1, 3) \\
 A = \begin{array}{c} 1 \\ 2 \\ 3 \end{array} & \begin{bmatrix} 1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & -1 \end{bmatrix}
 \end{array}$$

El grafo correspondiente a este problema, habiendo ya incorporado el nudo ficticio y definido el nudo 2 como el raíz, es el de la figura 9.11.

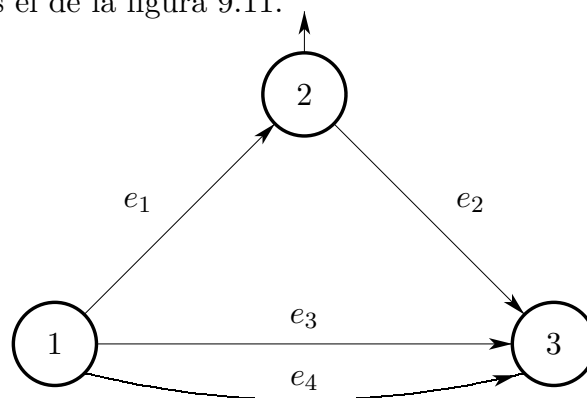


Figura 9.11
Grafo correspondiente al problema del ejemplo 9.4

Iteración 1. Paso 0 (Inicialización)

Utilizando los algoritmos de las tablas 9.1 y 9.2 se puede determinar fácilmente un árbol maximal de esta red. Será el definido por

$$B = \begin{matrix} & e_4 & e_1 \\ \begin{matrix} 3 \\ 1 \\ 2 \end{matrix} & \begin{bmatrix} -1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} \end{matrix}$$

y por el grafo de la figura 9.12. La base la forman las variables x_1 y x_4 . En aras de una adecuada exposición del ejemplo, en vez de partir como es lo habitual de $\mathbf{x}_N = [0, 0]$, comenzaremos con $\mathbf{x}_N = [0, 4]$. Es decir,

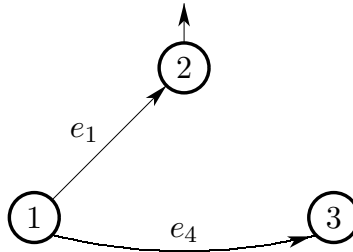
$$B = [\mathbf{a}_4, \mathbf{a}_1, \mathbf{a}_f] \quad \text{y} \quad N = [\mathbf{a}_2, \mathbf{a}_3].$$

El arco ficticio, desde el punto de vista algebraico, estará en la base siempre. En lo que afecta a la mecánica del método, este arco no se menciona y se actúa como si no existiese.

De acuerdo con las consideraciones anteriores, teniendo en cuenta las permutaciones de filas efectuadas,

$$\mathbf{x}_B = B^{-1}\mathbf{b} - B^{-1}N\mathbf{x}_N = \begin{bmatrix} -1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} -5 \\ 5 \\ 0 \end{bmatrix} - \begin{bmatrix} -1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} -1 & -1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

La solución de partida, $[x_1, x_2, x_3, x_4]$, es por tanto $[0, 0, 4, 1]$: solución degenerada. Más adelante en el capítulo indicaremos cómo llegar a una solución factible con la que comenzar el proceso.

**Figura 9.12**

Árbol maximal de la iteración 1 del ejemplo 9.4

Iteración 1. Paso 1 (Asignación de precios. Comprobación de óptimo)

De la matriz B , del hecho que $\pi_2 = 0$ (nudo raíz) y de que

$$\begin{aligned} \pi_1 - \pi_2 &= 1 \\ \pi_1 - \pi_3 &= 10, \end{aligned}$$

se deduce fácilmente que $\pi_1 = 1$ y $\pi_3 = -9$. Los costes reducidos de los arcos no básicos e_2 y e_3 son

$$\begin{aligned} \bar{c}_{e_2} &= c_{e_2} - \pi_2 + \pi_3 = 1 - 0 + (-9) = -8 \quad \text{y} \\ \bar{c}_{e_3} &= c_{e_3} - \pi_1 + \pi_3 = 3 - 1 + (-9) = -7. \end{aligned}$$

No se ha llegado al óptimo pues el coste reducido del arco no básico en su límite inferior, e_2 , es negativo. También deducimos que $\Psi_1 = \{e_2\}$ y $\Psi_2 = \emptyset$.

Iteración 1. Paso 2 (Determinación de la columna de pivotación)

La variable que se escoge para entrar en la base es el flujo en el arco e_2 , pues es la única posible. Además, como $e_2 \in \Psi_1$, $\delta = 1$.

Determinamos a continuación el camino en el árbol maximal actual entre los nudos del arco e_2 , es decir, entre el 2 y el 3. El camino, como se puede ver en la figura 9.13, es el definido por $C = \{2, e_1, 1, e_4, 3\}$. En este camino $O_{e_1} = -1$ y $O_{e_4} = 1$.

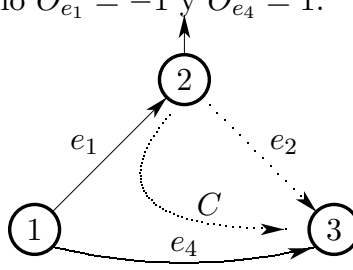


Figura 9.13

Iteración 1 Paso 2: determinación del camino para encontrar la fila de pivotación

Iteración 1. Paso 3 (Determinación de la fila de pivotación)

Definamos:

$$\begin{aligned}\Delta_1 &= \min_{O_{e_i}(C)=\delta} \{x_{e_i} - l_{e_i}, \infty\} = \min\{x_{e_4} - l_{e_4}, \infty\} = \min\{1, \infty\} = 1; \\ \Delta_2 &= \min_{-O_{e_i}(C)=\delta} \{u_{e_i} - x_{e_i}, \infty\} = \min\{u_{e_1} - x_{e_1}, \infty\} = \min\{4 - 0, \infty\} = 4 \quad \text{y} \\ \theta &= \min\{\Delta_1, \Delta_2, u_{e_2} - l_{e_2}\} = \min\{1, 4, 2\} = 1.\end{aligned}$$

El arco que sale de la base es e_4 , pues al incrementar el flujo en e_2 , el primer arco que se bloquea en C —llega a su límite inferior— es e_4 .

Iteración 1. Paso 4 (Pivotación)

Adaptamos la solución:

$$\begin{aligned}x_2 &\leftarrow x_2 + \delta\theta = 0 + 1 \cdot 1 = 1 \\ x_1 &\leftarrow x_1 - \theta\delta O_{e_1} = 0 - 1 \cdot 1 \cdot (-1) = 1 \\ x_4 &\leftarrow x_4 - \theta\delta O_{e_4} = 1 - 1 \cdot 1 \cdot 1 = 0.\end{aligned}$$

La nueva solución es

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 4 \\ 0 \end{bmatrix}.$$

Adaptamos el valor de los multiplicadores simplex:

$$\pi_3 \leftarrow \pi_3 - \bar{c}_{e_2} = -9 - (-8) = -1.$$

El nuevo árbol maximal es el de la figura 9.14.

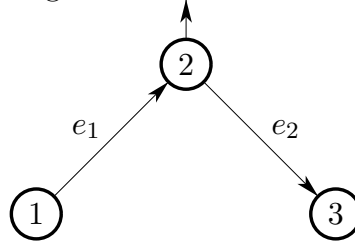


Figura 9.14
Árbol maximal de la iteración 2

Iteración 2. Paso 1

Los costes reducidos de los arcos no básicos e_3 y e_4 son

$$\begin{aligned}\bar{c}_{e_3} &= c_{e_3} - \pi_1 + \pi_3 = 3 - 1 + (-1) = 1 \quad \text{y} \\ \bar{c}_{e_4} &= c_{e_4} - \pi_1 + \pi_3 = 10 - 1 + (-1) = 8.\end{aligned}$$

Aunque el coste reducido de e_4 si es óptimo, el de e_3 , en su límite superior, no lo es: es positivo. No se ha llegado por tanto al óptimo. También deducimos que $\Psi_1 = \emptyset$ y $\Psi_2 = \{e_3\}$.

Iteración 2. Paso 2

Es evidente que la única variable posible a entrar en la base es el flujo en el arco e_3 . Además, como $e_3 \in \Psi_2$, $\delta = -1$.

A continuación determinamos el camino en el árbol maximal actual entre los nudos del arco e_3 , es decir entre 1 y 3. El camino, como se puede ver en la figura 9.15, es el definido por $C = \{1, e_1, 2, e_2, 3\}$. En este camino, $O_{e_1} = 1$ y $O_{e_2} = 1$.

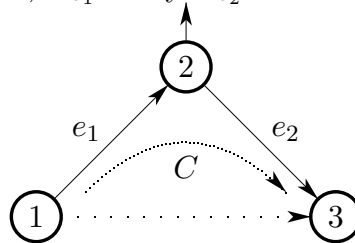


Figura 9.15
Iteración 2 Paso 2: determinación del camino para encontrar la fila de pivotación

Iteración 2. Paso 3

Definamos:

$$\Delta_1 = \min_{O_{e_i}(C)=\delta} \{x_{e_i} - l_{e_i}, \infty\} = \min\{\infty\} = \infty;$$

$$\Delta_2 = \min_{-O_{e_i}(C)=\delta} \{u_{e_i} - x_{e_i}, \infty\} = \min\{u_{e_1} - x_{e_1}, u_{e_2} - x_{e_2}, \infty\} = \min\{4 - 1, 2 - 1, \infty\} = 1$$

y

$$\theta = \min\{\Delta_1, \Delta_2, u_{e_3} - l_{e_3}\} = \min\{\infty, 1, 4\} = 1.$$

El arco que sale de la base es e_2 , pues al decrementar el flujo en e_3 , el primer arco que se bloquea en C —llega a su límite superior— es e_2 .

Iteración 2. Paso 4

Adaptamos la solución:

$$\begin{aligned} x_3 &\leftarrow x_3 + \delta\theta = 4 + 1 \cdot (-1) = 3 \\ x_1 &\leftarrow x_1 - \theta\delta O_{e_1} = 1 - 1 \cdot (-1) \cdot 1 = 2 \\ x_2 &\leftarrow x_2 - \theta\delta O_{e_2} = 1 - 1 \cdot (-1) \cdot 1 = 2. \end{aligned}$$

La nueva solución es

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 0 \end{bmatrix}.$$

Adaptamos el valor de los multiplicadores simplex:

$$\pi_3 \leftarrow \pi_3 - \bar{c}_{e_3} = -1 - 1 = -2.$$

El nuevo árbol maximal es el de la figura 9.16.

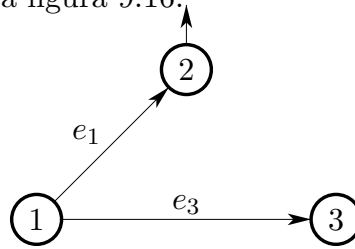


Figura 9.16

Árbol maximal de la iteración 3 del ejemplo 9.4

Iteración 3. Paso 1

Los costes reducidos de los arcos no básicos e_2 y e_4 son

$$\begin{aligned} \bar{c}_{e_2} &= c_{e_2} - \pi_2 + \pi_3 = 1 - 0 + (-2) = -1 \text{ y} \\ \bar{c}_{e_4} &= c_{e_4} - \pi_1 + \pi_3 = 10 - 1 + (-2) = 7. \end{aligned}$$

Se ha llegado al óptimo pues el coste reducido de e_2 , en su límite superior, es negativo y el de e_4 , en su límite inferior, positivo. La solución óptima es pues

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 3 \\ 0 \end{bmatrix}.$$

El valor de la función objetivo es

$$\mathbf{x}^T \mathbf{c} = 13.$$

●

9.1.3.2 Solución básica factible inicial

El procedimiento que se utiliza en la mayoría de los códigos comerciales para obtener una solución factible inicial consiste en aumentar el grafo original mediante un nudo artificial —además del ficticio que ya hemos venido considerando—, uniéndolo mediante arcos artificiales a todos los demás. Si un nudo i al que se une es de oferta ($b_i > 0$), el arco va del nudo i al artificial; si por el contrario, es de demanda ($b_i < 0$), del artificial al i . El nudo artificial a su vez se une al ficticio, considerándose como raíz el artificial. Si algún nudo es de transbordo no se une al artificial.

La idea básica de este procedimiento es exactamente la misma que la utilizada en la *fase I* del método *simplex* generalizado. Para problemas de redes se puede asignar coste unitario al flujo en los arcos artificiales y cero a los del grafo original, en cuyo caso se obtiene el equivalente a la *fase I* del método *simplex*. También se puede asignar un coste muy elevado al flujo en los arcos artificiales, en cuyo caso se obtiene el equivalente al denominado método de penalización o gran M.

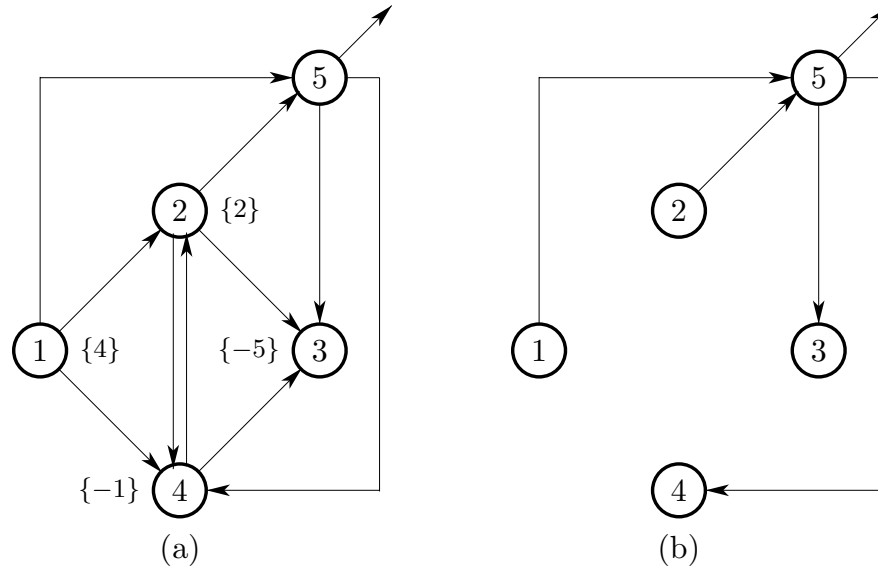
Si como ejemplo suponemos la red de la figura 9.1 de la página 500, el grafo que resulta de aplicar este procedimiento para la obtención de una solución básica factible inicial es el de la figura 9.17 (a) —las cantidades entre llaves son las demandas/ofertas (-/+); en (b) se puede ver el árbol maximal con el que se iniciaría la resolución de este problema.

9.2 El principio de descomposición de Dantzig-Wolfe

En este apartado seguimos profundizando en formas de aplicar el método *simplex* a problemas de grandes dimensiones en los que la estructura de sus condiciones hace aconsejable especializar las diversas partes o pasos del mismo con el fin de reducir considerablemente los tiempos de resolución. En concreto, nos centraremos en programas lineales del tipo

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A_0 \mathbf{x} = \mathbf{b}_0 \\ & A_1 \mathbf{x} = \mathbf{b}_1 \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{9.5}$$

A las condiciones $A_0 \mathbf{x} = \mathbf{b}_0$ se las denomina *condiciones generales* y a $A_1 \mathbf{x} = \mathbf{b}_1$, $\mathbf{x} \geq \mathbf{0}$, *condiciones especiales*. Las condiciones especiales, por ejemplo, pueden estar definidas por una

**Figura 9.17**

- (a) Grafo de la figura 9.1 aumentado en el nudo artificial 5 para obtener una solución factible inicial. (b) Árbol maximal inicial

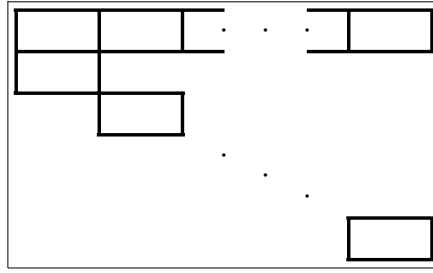
red de algún tipo o estar formadas por subconjuntos de condiciones cada uno con variables independientes de los demás.

El paradigma de este tipo de problemas lo constituye la asignación presupuestaria en una empresa multidivisional, en un ministerio con varios departamentos, en una empresa multinacional con diversas delegaciones, etc. Se trata de satisfacer las necesidades presupuestarias de N divisiones de una empresa u organismo, cada una de ellas con sus variables de decisión independientes, de tal forma que se satisfagan los requerimientos o condiciones de la empresa en su conjunto: por ejemplo, no poder endeudarse en más de una determinada cantidad de dinero, no poder disponer de más recursos de transporte que los disponibles en la empresa, etc. A estas últimas condiciones se las denomina *condiciones de enlace*. La forma del problema es la siguiente:

$$\begin{aligned}
 \min. \quad & \mathbf{c}_1^T \mathbf{x}_1 + \cdots + \mathbf{c}_N^T \mathbf{x}_N \\
 \text{s. a} \quad & A_{01} \mathbf{x}_1 + \cdots + A_{0N} \mathbf{x}_N = \mathbf{b}_0 \\
 & A_{11} \mathbf{x}_1 = \mathbf{b}_1 \\
 & \ddots = \vdots \\
 & A_{NN} \mathbf{x}_N = \mathbf{b}_N \\
 & \mathbf{x}_1, \dots, \mathbf{x}_N \geq \mathbf{0}.
 \end{aligned} \tag{9.6}$$

La matriz de coeficientes de las condiciones tiene una estructura como la de la figura 9.18.

Lo que estudiamos a continuación es la forma de resolver (9.5) eficazmente, para lo que se tendrá en cuenta, en general, que la resolución del problema sólo con las condiciones especiales es mucho más fácil. El procedimiento que exponemos es el conocido como *descomposición de Dantzig-Wolfe* [1960].


Figura 9.18

Estructura diagonal por bloques de la matriz del problema 9.6

Reescribamos el problema (9.5) de la siguiente manera:

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A_0 \mathbf{x} = \mathbf{b}_0 \\ & \mathbf{x} \in X, \end{aligned} \tag{9.7a}$$

donde X es el politopo definido así:

$$X = \{\mathbf{x} \in \Re^n : A_1 \mathbf{x} = \mathbf{b}_1, \mathbf{x} \geq \mathbf{0}\}. \tag{9.7b}$$

Supondremos que A_0 es una matriz $m_0 \times n$, A_1 otra $m_1 \times n$ y los vectores $\mathbf{c}$, $\mathbf{x}$, $\mathbf{b}_0$ y $\mathbf{b}_1$ tienen dimensiones acordes. El problema (9.6) se puede expresar de esta nueva manera sin más que hacer $A_0 = [A_{01}, A_{02}, \dots, A_{0N}]$ y

$$X = \left\{ \mathbf{x} = [\mathbf{x}_1^T, \dots, \mathbf{x}_N^T]^T : A_{jj} \mathbf{x}_j = \mathbf{b}_j, \mathbf{x}_j \geq \mathbf{0}, j = 1, \dots, N \right\}$$

o, alternativamente, considerando cada división independientemente. Es decir,

$$\begin{aligned} \min. \quad & \mathbf{c}_1^T \mathbf{x}_1 + \dots + \mathbf{c}_N^T \mathbf{x}_N \\ \text{s. a} \quad & A_{01} \mathbf{x}_1 + \dots + A_{0N} \mathbf{x}_N = \mathbf{b}_0 \\ & \mathbf{x}_1 \in X_1, \dots, \mathbf{x}_N \in X_N, \end{aligned}$$

con $X_j = \{\mathbf{x}_j : A_{jj} \mathbf{x}_j = \mathbf{b}_j, \mathbf{x}_j \geq \mathbf{0}\}$.

Las ideas que motivan la descomposición del problema así planteado son diversas: van desde la pura aplicación de la estrategia *divide y vencerás* a interpretaciones económicas muy interesantes. Una de este tipo considera que la empresa donde se plantea el problema de asignar recursos trata de descentralizar su proceso de toma de decisiones mediante la asignación de unos precios a los recursos de que dispone. Si no se imponen condiciones sobre el uso de estos recursos a una división j , ésta los comprará a los precios que determine un vector $-\boldsymbol{\pi}_0$. La división j tratará de resolver el siguiente subproblema:

$$\begin{aligned} SP_j(\boldsymbol{\pi}_0) \quad \min. \quad & (\mathbf{c}_j^T - \boldsymbol{\pi}_0^T A_{0j}) \mathbf{x}_j \\ \text{s. a} \quad & \mathbf{x}_j \in X_j. \end{aligned} \tag{9.8}$$

Evidentemente, la empresa ha de definir sus precios $-\pi_0$ de tal manera que, cuando todas las divisiones hayan resuelto sus respectivos subproblemas $SP_j(\pi_0)$, se obtenga una solución óptima, $\bar{x}_j$, tal que $\sum_{j=1}^N A_{0j}\bar{x}_j = \mathbf{b}_0$. Visto así, el problema es semejante al de la situación económica clásica: qué precios decidimos deben tener nuestros productos de tal forma que la demanda de un conjunto de mayoristas que tratan de maximizar sus ganancias sea igual a la oferta que podemos proporcionar. A una solución $[\bar{\pi}_0^T, \bar{x}_1^T, \dots, \bar{x}_N^T]^T$, tal que cada $\bar{x}_j$ es el óptimo del correspondiente subproblema $SP_j(\bar{\pi}_0)$, para cada j y $\sum_{j=1}^N A_{0j}\bar{x}_j = \mathbf{b}_0$, se la denomina *punto de equilibrio*.

El siguiente resultado define las condiciones en las cuales se presenta un punto de equilibrio.

Teorema 9.2 Si $[\bar{x}_1^T, \dots, \bar{x}_N^T]^T$ y $[\bar{\pi}_0^T, \bar{\pi}_1^T, \dots, \bar{\pi}_N^T]^T$ son las soluciones óptimas del programa primal y dual respectivamente de

$$\begin{aligned} \min. \quad & \mathbf{c}_1^T \mathbf{x}_1 + \dots + \mathbf{c}_N^T \mathbf{x}_N \\ \text{s. a} \quad & A_{01}\mathbf{x}_1 + \dots + A_{0N}\mathbf{x}_N = \mathbf{b}_0 \\ & A_{11}\mathbf{x}_1 = \mathbf{b}_1 \\ & \ddots = \vdots \\ & A_{NN}\mathbf{x}_N = \mathbf{b}_N \\ & \mathbf{x}_1, \dots, \mathbf{x}_N \geq \mathbf{0}, \end{aligned} \tag{9.9}$$

entonces $[\bar{\pi}_0^T, \bar{x}_1^T, \dots, \bar{x}_N^T]^T$ es un punto de equilibrio. Recíprocamente, si $[\bar{\pi}_0^T, \bar{x}_1^T, \dots, \bar{x}_N^T]^T$ es un punto de equilibrio, $[\bar{x}_1^T, \dots, \bar{x}_N^T]^T$ es un punto óptimo del programa primal de (9.9) y $\bar{\pi}_0$ forma parte del óptimo de su dual.

DEMOSTRACIÓN. Consideremos la primera parte. Es claro que $\sum_{j=1}^N A_{0j}\bar{x}_j = \mathbf{b}_0$ por lo que sólo resta demostrar que $\bar{x}_j$ es la solución óptima del subproblema $SP_j(\bar{\pi}_0)$. La factibilidad de ese punto es evidente. La factibilidad del dual y la condición de complementariedad de holguras de (9.9) se expresan de la siguiente manera:

$$A_{0j}^T \bar{\pi}_0 + A_{jj}^T \bar{\pi}_j \leq \mathbf{c}_j, \quad \left(A_{0j}^T \bar{\pi}_0 + A_{jj}^T \bar{\pi}_j - \mathbf{c}_j \right)^T \bar{x}_j = 0. \tag{9.10a}$$

De acuerdo con esto,

$$A_{jj}^T \bar{\pi}_j \leq \mathbf{c}_j - A_{0j}^T \bar{\pi}_0, \quad \left(A_{jj}^T \bar{\pi}_j - (\mathbf{c}_j - A_{0j}^T \bar{\pi}_0) \right)^T \bar{x}_j = 0, \tag{9.10b}$$

lo que quiere decir, por dualidad, que $\bar{x}_j$ es un punto óptimo del subproblema $SP_j(\bar{\pi}_0)$ y $\bar{\pi}_j$ óptimo de su dual. Para ver la parte recíproca del teorema, sea $\bar{\pi}_j$ una solución óptima del dual de $SP_j(\bar{\pi}_0)$. Por dualidad de este subproblema se tiene la expresión (9.10b) y, por consiguiente, (9.10a), lo cual implica que $[\bar{x}_1^T, \dots, \bar{x}_N^T]^T$ y $[\bar{\pi}_0^T, \bar{\pi}_1^T, \dots, \bar{\pi}_N^T]^T$ son puntos óptimos del primal y dual, respectivamente, de (9.9). ■

Además de establecer la existencia de un punto de equilibrio —si (9.9) tiene solución—, el teorema 9.2 proporciona en sí mismo un procedimiento para la resolución del problema. Si se supiesen cuáles son los precios $-\bar{\pi}_0$, el problema (9.9) se podría resolver resolviendo a su vez N pequeños programas lineales o subproblemas —los $SP_j(\bar{\pi}_0)$ — en vez de uno mucho más grande.

Desgraciadamente surgen dos dificultades: en primer lugar, que no está nada claro, en principio, cómo determinar adecuadamente el vector $\bar{\pi}_0$; en segundo lugar, que aun cuando se conociese $\bar{\pi}_0$, la obtención de los vectores $\bar{x}_1, \dots, \bar{x}_N$ tampoco es fácil. En efecto, suponiendo no degeneración de las soluciones básicas factibles, una solución óptima de (9.9) tendrá $m_0 + \sum_{j \geq 1} m_j$ componentes positivos (suponiendo que el vector b_j , para cada j , tiene m_j componentes). Una solución básica óptima de $SP_j(\bar{\pi}_0)$, por otro lado, tendrá sólo m_j componentes positivos por lo que todos los subproblemas juntos tendrán sólo $\sum_{j \geq 1} m_j$ componentes positivos. La conclusión es que cualquier punto de equilibrio, $\bar{\pi}_0$, obligará a que al menos uno de los subproblemas tenga soluciones óptimas alternativas y que éstas habrán de elegirse adecuadamente a fin de agotar el mercado de los recursos propios de la empresa.

Estas y otras dificultades pueden ser eliminadas formulando un nuevo problema de programación lineal de tal forma que, aplicándole el método *simplex* revisado, se genere una sucesión de vectores π_0 y, partiendo de ella, considerando explícitamente combinaciones convexas de los puntos extremos de los politopos X_j de los subproblemas.

Para facilitar la notación volvamos al problema general definido en (9.5) con un solo politopo X (posteriormente consideraremos el caso en que existan varios). La idea básica del nuevo enfoque consiste en representar el politopo X de (9.7b) en función de sus puntos extremos y alguna de sus direcciones de acuerdo con el teorema 6.4 de la página 400.

La siguiente definición formal ya fue introducida en la sección 6.3.

Definición 9.1 Una dirección d de un politopo X se dice *extrema* si no puede ponerse como combinación lineal no negativa de dos direcciones diferentes de X . Es decir, no existen dos direcciones d_1 y d_2 de X , $d_1 \neq d_2$, y unos $\alpha_1, \alpha_2 > 0$, tales que $d = \alpha_1 d_1 + \alpha_2 d_2$.

El siguiente resultado es una pequeña extensión de aquel teorema 6.4 que vamos a necesitar.

Teorema 9.3 *Todo punto x del politopo*

$$X = \{x \in \mathbb{R}^n : A_1 x = b_1, x \geq 0\}$$

se puede expresar como

$$x = \sum_{i \in I} \lambda_i v_i + \sum_{j \in J} \mu_j d_j,$$

donde $\{v_i : i \in I\}$ es el conjunto de puntos extremos, $\{d_j : j \in J\}$ el conjunto de direcciones extremas del politopo X , $\sum_{i \in I} \lambda_i = 1$, $\lambda_i \geq 0$, para todo $i \in I$, y $\mu_j \geq 0$ para todo $j \in J$. Recíprocamente, todos los puntos x expresables de esa forma pertenecen a X . Además X tiene un número finito de direcciones extremas.

La demostración de este teorema es muy similar a la del teorema 6.4 por lo que remitimos al lector a ella.

Corolario 9.2 Si los vectores columna de las matrices V y D son, respectivamente, los puntos extremos y direcciones extremas del politopo X , entonces

$$X = \{V\lambda + D\mu : e^T\lambda = 1, \lambda \geq 0, \mu \geq 0\}.$$

(Recordemos que e es un vector en el que todos sus componentes son 1.)

Si se sustituye x por su expresión como combinación de puntos y direcciones extremas, el problema (9.7) de la página 529 es equivalente al siguiente:

$$\begin{aligned} \min. \quad & c^T V\lambda + c^T D\mu \\ \text{s. a} \quad & A_0 V\lambda + A_0 D\mu = b_0 \\ & e^T \lambda = 1 \\ & \lambda \geq 0, \mu \geq 0. \end{aligned} \tag{9.11}$$

A éste se le denomina *problema maestro*. La idea del principio de descomposición de Dantzig y Wolfe es aplicar el método *simplex* revisado a este problema maestro. Obsérvese que, en contraste con (9.5), el problema (9.11) tiene sólo $m_0 + 1$ condiciones y, en cambio, un número muy elevado de columnas (una por cada punto extremo del politopo X y otra por cada dirección extrema), sólo conocidas implícitamente. Para la explicitación de estas columnas se usa lo que se denomina una *técnica de generación de columnas*.

Supongamos que disponemos de una solución básica factible $[\bar{\lambda}^T, \bar{\mu}^T]^T$ del problema maestro (9.11), con unos multiplicadores simplex asociados a las primeras m_0 condiciones de igualdad, $\bar{\pi}_0$, y $\bar{\sigma}$ asociado a la última. Si algún $\bar{\lambda}_i > 0$, entonces se conoce el correspondiente punto extremo v_i de X ; si se conoce $\bar{\mu}_j$, la dirección extrema correspondiente, d_j . Ahora bien, el conjunto de todos los puntos extremos y el de las direcciones extremas son desconocidos por lo que habrá que generarlos —dando lugar a las columnas correspondientes de (9.11)— según se vayan necesitando.

En una iteración del método *simplex* revisado aplicado al problema maestro, en primer lugar hay que encontrar un punto extremo, v_i , con coste reducido

$$c^T v_i - \bar{\pi}_0^T A_0 v_i - \bar{\sigma} < 0, \tag{9.12a}$$

o una dirección extrema, d_j , con coste reducido

$$c^T d_j - \bar{\pi}_0^T A_0 d_j < 0. \tag{9.12b}$$

Si no existe ni el uno ni la otra, la solución que se tenga en ese momento, $[\bar{\lambda}^T, \bar{\mu}^T]^T$, será el óptimo del problema maestro y, por consiguiente, $\bar{x} = V\bar{\lambda} + D\bar{\mu}$ el óptimo del problema (9.7).

Consideremos en primer lugar la expresión (9.12a). Como de lo que se trata es de encontrar un punto extremo v_i del politopo X tal que la función lineal $(c^T - \bar{\pi}_0^T A_0)v_i$ sea menor que $\bar{\sigma}$, y el mínimo de una función lineal en un politopo se alcanza como sabemos en un punto extremo (a no ser que el politopo no esté acotado inferiormente) de él parece entonces lógico considerar en este sentido un subproblema $SP(\bar{\pi}_0)$ de la forma

$$\min. \left\{ (c^T - \bar{\pi}_0^T A_0) x : x \in X \right\}.$$

Éste no es otro que el problema que introducíamos en (9.8). Veamos los posibles resultados que podemos obtener al resolver $SP(\bar{\pi}_0)$:

1. Si no es factible, el politopo X es el vacío y, en consecuencia, el problema (9.7) tampoco es factible. En este caso no se podría disponer de una solución básica factible del problema maestro.
2. $SP(\bar{\pi}_0)$ puede ser no acotado. En este caso la aplicación del método *simplex* revisado generará una dirección de descenso, η_q , desde algún punto extremo $\mathbf{v}$ de X , en la que todos los puntos de la forma $\mathbf{v} + \theta \eta_q$ estarán en X , para todo $\theta \geq 0$ y $(\mathbf{c}^T - \bar{\pi}_0^T A_0) \eta_q < 0$. De hecho, η_q tendrá la forma

$$\eta_q = \begin{bmatrix} -\mathbf{y} \\ \mathbf{e}_{q-m_1} \end{bmatrix}, \quad \text{donde } \mathbf{y} = B_1^{-1} \mathbf{a}_{1q},$$

si la matriz B_1 en ese momento está formada por las primeras m_1 columnas de A_1 y $\mathbf{a}_{1q}$ es la columna q -ésima de A_1 que entra en la base. No es muy difícil ver que η_q es una dirección extrema del politopo X , por lo que haciendo $\mathbf{d}_j = \eta_q$, se llega a la expresión (9.12b). Por supuesto que la dirección η_q no tiene por qué ser una dirección del politopo de soluciones factibles del problema (9.5) de la página 527, pues se han ignorado las condiciones $A_0 \mathbf{x} = \mathbf{b}_0$.

3. Por último, $SP(\bar{\pi}_0)$ puede tener una solución óptima finita $\bar{\mathbf{x}}$. En este caso, de acuerdo con el teorema fundamental de la programación lineal, $(\mathbf{c}^T - \bar{\pi}_0^T A_0) \mathbf{d} \geq 0$ en todas las direcciones $\mathbf{d}$ pudiendo ser entonces $\bar{\mathbf{x}}$ uno de los puntos extremos de X (el método *simplex* revisado asegura el encontrarlo). De ser esto así, (9.12b) no se cumpliría para todo j ; ninguna columna que surgiese de una dirección extrema podría ser candidata a entrar en la base. Si $(\mathbf{c}^T - \bar{\pi}_0^T A_0) \bar{\mathbf{x}} \geq \sigma$, como se minimiza sobre todo el politopo X , y por consiguiente sobre todos sus puntos extremos, la expresión (9.12a) no se cumpliría para todo i , por lo que ninguna columna que surgiese de un punto extremo sería candidata a entrar en la base, concluyéndose que $[\bar{\lambda}^T, \bar{\mu}^T]^T$ es el óptimo del problema maestro (9.11) y $\mathbf{x}^* = V\bar{\lambda} + D\bar{\mu}$ el óptimo del problema (9.5). Si por otro lado $(\mathbf{c}^T - \bar{\pi}_0^T A_0) \bar{\mathbf{x}} < \sigma$, haciendo $\mathbf{v}_i = \bar{\mathbf{x}}$ se llega a la expresión (9.12a).

De acuerdo con esto, en cualquier caso, o se comprueba que se ha alcanzado el óptimo o se genera una columna del problema maestro (9.11) que introducir en la base, en cuyo caso se continúa con otra iteración del método *simplex*.

Teorema 9.4 (a) Si el subproblema $SP(\bar{\pi}_0)$ no está acotado, al aplicarle el método simplex revisado se obtiene una dirección extrema $\mathbf{d}_j$ que satisface

$$\mathbf{c}^T \mathbf{d}_j - \bar{\pi}_0^T A_0 \mathbf{d}_j < 0,$$

de tal forma que la columna

$$\begin{bmatrix} A_0 \mathbf{d}_j \\ 0 \end{bmatrix},$$

con coste reducido $\mathbf{c}^T \mathbf{d}_j$, es buena para entrar en la base del problema maestro

$$\begin{aligned} \min. \quad & \mathbf{c}^T V \boldsymbol{\lambda} + \mathbf{c}^T D \boldsymbol{\mu} \\ \text{s. a} \quad & A_0 V \boldsymbol{\lambda} + A_0 D \boldsymbol{\mu} = \mathbf{b}_0 \\ & \mathbf{e}^T \boldsymbol{\lambda} = 1 \\ & \boldsymbol{\lambda} \geq \mathbf{0}, \boldsymbol{\mu} \geq \mathbf{0}. \end{aligned} \tag{9.13}$$

(b) Si el subproblema $SP(\bar{\pi}_0)$ tiene una solución óptima en el punto extremo $\mathbf{v}_i$ con el valor de su función objetivo menor que $\bar{\sigma}$, la columna

$$\begin{bmatrix} A_0 \mathbf{v}_i \\ 1 \end{bmatrix},$$

con coste reducido $\mathbf{c}^T \mathbf{v}_i$, es adecuada para entrar en la base del problema maestro (9.13).

(c) Finalmente, si el subproblema $SP(\bar{\pi}_0)$ alcanza un valor óptimo al menos igual a $\bar{\sigma}$, con solución óptima de su programa dual igual a $\bar{\pi}_1$, la solución básica factible del problema maestro en ese momento, $[\bar{\boldsymbol{\lambda}}^T, \bar{\boldsymbol{\mu}}^T]^T$, es óptima siendo el óptimo de su programa dual igual a $[\bar{\pi}_0^T, \bar{\sigma}]^T$; además $\mathbf{x}^* = V \bar{\boldsymbol{\lambda}} + D \bar{\boldsymbol{\mu}}$ es la solución óptima del problema

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A_0 \mathbf{x} = \mathbf{b}_0 \\ & A_1 \mathbf{x} = \mathbf{b}_1 \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{9.14}$$

siendo el óptimo de su dual $[\bar{\pi}_0^T, \bar{\pi}_1^T]^T$.

DEMOSTRACIÓN. Sólo hay que probar la última parte. Es claro que como las condiciones (9.12a) y (9.12b) en este caso no se cumplen para todo $i \in I$ y $j \in J$, el vector $[\bar{\pi}_0^T, \bar{\sigma}]^T$ es factible en el dual del problema (9.13), por lo que $[\bar{\boldsymbol{\lambda}}^T, \bar{\boldsymbol{\mu}}^T]^T$ y $[\bar{\pi}_0^T, \bar{\sigma}]^T$ son soluciones óptimas de su primal y dual, respectivamente, y por tanto $\mathbf{c}^T V \bar{\boldsymbol{\lambda}} + \mathbf{c}^T D \bar{\boldsymbol{\mu}} = \bar{\pi}_0^T \mathbf{b}_0 + \bar{\sigma}$. Por otro lado, $\mathbf{x}^*$ es un punto factible del problema (9.14) pues satisface $A_0 \mathbf{x}^* = \mathbf{b}_0$ y, por el teorema 9.3, está en el politopo X ; el valor de la función objetivo de (9.14) en este punto es $\mathbf{c}^T \mathbf{x}^* = \mathbf{c}^T V \bar{\boldsymbol{\lambda}} + \mathbf{c}^T D \bar{\boldsymbol{\mu}}$, valor óptimo del problema maestro (9.13). Como $\bar{\pi}_1$ es óptimo del dual de $SP(\bar{\pi}_0)$, es factible de ese dual y, por tanto, cumple que

$$\bar{\pi}_1^T A_1 \leq \mathbf{c}^T - \bar{\pi}_0^T A_0, \tag{9.15}$$

siendo el valor de la función objetivo de ese dual

$$\begin{aligned}\bar{\pi}_1^T \mathbf{b}_1 &\geq \bar{\sigma} = (\bar{\pi}_0^T \mathbf{b}_0 + \bar{\sigma}) - \bar{\pi}_0^T \mathbf{b}_0 \\ &= (\mathbf{c}^T V \bar{\lambda} + \mathbf{c}^T D \bar{\mu}) - \bar{\pi}_0^T \mathbf{b}_0 \\ &= \mathbf{c}^T \mathbf{x}^* - \bar{\pi}_0^T \mathbf{b}_0.\end{aligned}$$

De aquí que, según (9.15), $[\bar{\pi}_0^T, \bar{\pi}_1^T]^T$ es un punto factible del problema dual del (9.14), teniendo como valor correspondiente de la función objetivo al menos el que corresponde a $\mathbf{x}^*$ en el primal. El lema de la dualidad débil implica en este caso que $\mathbf{x}^*$ es el óptimo de (9.14) y $[\bar{\pi}_0^T, \bar{\pi}_1^T]^T$ de su dual, según se requería en el enunciado. ■

Este teorema revela que se puede resolver el problema (9.14) resolviendo el problema maestro (9.13). La convergencia del procedimiento la asegura el hecho de emplear el método *simplex* revisado para resolver un problema finito, a pesar de que los coeficientes de sus condiciones no se conozcan anticipadamente. El procedimiento expuesto terminaría bien en una solución óptima del problema maestro, y por consiguiente del original, o bien indicando la existencia de no acotación en dicho problema maestro. En este último caso es fácil deducir que el problema original tampoco estaría acotado. La demostración del teorema también prueba que, cuando el procedimiento termina en una solución óptima, el óptimo de $SP(\bar{\pi}_0)$ es precisamente $\bar{\sigma}$ y, por complementariedad de holguras, todos los puntos extremos $\mathbf{v}_i$, con λ_i positivos, son soluciones óptimas alternativas de $SP(\bar{\pi}_0)$. De esta forma se superan las dos dificultades apuntadas con posterioridad a la exposición del teorema 9.2.

Al aplicar el método *simplex* revisado al problema maestro se genera automáticamente una sucesión de vectores $-\bar{\pi}_0$ que converge a un *vector de precios de equilibrio*, $-\pi_0^*$. El problema maestro considera explícitamente la forma de combinar las soluciones óptimas de $SP(\pi_0^*)$ a fin de obtener una solución óptima del problema (9.14) la cual, probablemente, no será un punto extremo del politopo X .

9.2.1 Implementación práctica

Antes de pasar a enunciar formalmente el algoritmo de descomposición de Dantzig-Wolfe, hagamos algunas consideraciones prácticas sobre el procedimiento expuesto hasta este punto.

Supongamos primero que algunas variables aparecen en las condiciones que denominábamos generales, $A_0 \mathbf{x} = \mathbf{b}_0$, pero no así en las especiales, $A_1 \mathbf{x} = \mathbf{b}_1$ —ésta puede ser la situación, por ejemplo, de una empresa en la que ciertas variables trascienden de las divisiones—. Lo más natural en este caso es no considerar esas variables en X e introducirlas directamente en el problema maestro. Para aplicar el método *simplex* revisado a este problema maestro un poco especial, habría que comprobar en cada iteración si alguna de esas variables es buena candidata para entrar en la base y, de ser así, realizar la pivotación correspondiente. Sólo si ninguna de ellas lo es se generaría el subproblema $SP(\bar{\pi}_0)$ y se procedería como decíamos antes.

Otro asunto importante a tener en cuenta lo constituye la obtención, como siempre, de una solución básica factible inicial con la que iniciar la resolución del problema maestro. Para ello, en primer lugar se ha de encontrar por cualquier método un punto extremo $\mathbf{v}_1$ del politopo X —si se descubre que X es el vacío, el problema original a resolver no es factible y habría que parar el procedimiento—. Una vez encontrado ese punto extremo, se introducen m_0 variables artificiales en el problema maestro de tal manera que $\lambda = \mathbf{1}$, junto con esas variables artificiales,

constituyan una solución básica factible de dicho problema maestro. Luego se aplica la *fase I* del método *simplex* revisado a fin de minimizar la suma de las variables artificiales, usando una vez más la idea o el principio de descomposición del problema. Si al final de esa *fase I* todas las variables artificiales son cero, se llega como sabemos a una solución básica factible del problema maestro, pudiéndose empezar inmediatamente la *fase II*. Esta singular *fase I* la podemos equiparar a aplicar el procedimiento de descomposición a una versión de la *fase I* que surgiese del problema original y donde las variables artificiales —de las condiciones $A_0\mathbf{x} = \mathbf{b}_0$ solamente— se trasladasen al problema maestro como en el párrafo anterior.

Consideremos ahora el problema más general

$$\begin{aligned}
 \min. \quad & \mathbf{c}_0^T \mathbf{x}_0 + \mathbf{c}_1^T \mathbf{x}_1 + \cdots + \mathbf{c}_N^T \mathbf{x}_N \\
 \text{s. a} \quad & A_{00}\mathbf{x}_0 + A_{01}\mathbf{x}_1 + \cdots + A_{0N}\mathbf{x}_N = \mathbf{b}_0 \\
 & \quad \quad \quad A_{11}\mathbf{x}_1 = \mathbf{b}_1 \\
 & \quad \quad \quad \vdots \\
 & \quad \quad \quad A_{NN}\mathbf{x}_N = \mathbf{b}_N \\
 & \mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N \geq \mathbf{0}.
 \end{aligned} \tag{9.16}$$

De las consideraciones hechas antes sabemos cómo tratar las variables del vector $\mathbf{x}_0$; ahora bien, ¿podemos aplicar las ideas de la descomposición y separar los vectores $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ sin tener que considerarlos conjuntamente? La respuesta es sí: aplicando la misma idea que hasta ahora a cada politopo X_j . El problema maestro en este caso es:

$$\begin{aligned}
 \min. \quad & \mathbf{c}_0^T \mathbf{x}_0 + \mathbf{c}_1^T V_1 \boldsymbol{\lambda}_1 + \mathbf{c}_1^T D_1 \boldsymbol{\mu}_1 + \cdots + \mathbf{c}_N^T V_N \boldsymbol{\lambda}_N + \mathbf{c}_N^T D_N \boldsymbol{\mu}_N \\
 \text{s. a} \quad & A_{00}\mathbf{x}_0 + A_{01}V_1 \boldsymbol{\lambda}_1 + A_{01}D_1 \boldsymbol{\mu}_1 + \cdots + A_{0N}V_N \boldsymbol{\lambda}_N + A_{0N}D_N \boldsymbol{\mu}_N = \mathbf{b} \\
 & \quad \quad \quad \mathbf{e}^T \boldsymbol{\lambda}_1 = 1 \\
 & \quad \quad \quad \vdots \\
 & \quad \quad \quad \mathbf{e}^T \boldsymbol{\lambda}_N = 1 \\
 & \mathbf{x}_0, \boldsymbol{\lambda}_1, \boldsymbol{\mu}_1, \dots, \boldsymbol{\lambda}_N, \boldsymbol{\mu}_N \geq \mathbf{0},
 \end{aligned} \tag{9.17}$$

donde las columnas de las matrices V_j y D_j son los puntos extremos y direcciones extremas, respectivamente, del politopo X_j y los componentes de los vectores $\boldsymbol{\lambda}_j$ y $\boldsymbol{\mu}_j$ son los coeficientes de ponderación de cada politopo. Obsérvese que en el problema (9.17) hay $m_0 + N$ filas, en vez de $m_0 + 1$ como era el caso que habíamos considerado hasta ahora de un solo X : en cualquier caso, muchas menos que en (9.16). En una iteración cualquiera del procedimiento de descomposición se tendrán los multiplicadores simplex $\bar{\boldsymbol{\pi}}_0, \bar{\sigma}_1, \dots, \bar{\sigma}_N$. Si, como dijimos antes, cualquiera de las variables del vector $\mathbf{x}_0$ es adecuada para entrar en la base, se haría la pivotación correspondiente. Si no es así, se resuelven cada uno de los subproblemas $SP_j(\bar{\boldsymbol{\pi}}_0)$, $j = 1, \dots, N$. Si alguno de estos subproblemas es no acotado, la dirección extrema correspondiente que genera proporciona una columna

$$\begin{bmatrix} A_{0j} \mathbf{d}_{ji} \\ \mathbf{0} \end{bmatrix}$$

para el problema (9.17). Si no es así y la solución óptima de algún $SP_j(\bar{\boldsymbol{\pi}}_0)$ es menor que $\bar{\sigma}_j$,

el punto extremo óptimo correspondiente, $\mathbf{v}_{ji}$, proporciona otra posible columna

$$\begin{bmatrix} A_{0j}\mathbf{v}_{ji} \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \leftarrow 1 \\ \\ \\ \leftarrow j \\ \\ \leftarrow N \end{matrix}$$

para el problema maestro. Finalmente, si todos los óptimos de los subproblemas son iguales a los correspondientes $\bar{\sigma}_j$, se ha llegado al óptimo global del problema maestro y el punto $\mathbf{x}_0^* = \bar{\mathbf{x}}_0$, $\mathbf{x}_j^* = V_j \bar{\boldsymbol{\lambda}}_j + D_j \bar{\boldsymbol{\mu}}_j$, $j = 1, 2, \dots, N$, es la solución óptima del problema (9.16).

El algoritmo de la tabla 9.7 plasma todo el procedimiento a seguir para aplicar el principio de descomposición al programa

$$\begin{aligned} \min. \quad & \mathbf{c}_1^T \mathbf{x}_1 + \dots + \mathbf{c}_N^T \mathbf{x}_N \\ \text{s. a} \quad & A_{01}\mathbf{x}_1 + \dots + A_{0N}\mathbf{x}_N = \mathbf{b}_0 \\ & A_{11}\mathbf{x}_1 = \mathbf{b}_1 \\ & \vdots \\ & A_{NN}\mathbf{x}_N = \mathbf{b}_N \\ & \mathbf{x}_1, \dots, \mathbf{x}_N \geq \mathbf{0}. \end{aligned}$$

El problema maestro del que parte tiene la forma siguiente

$$\begin{aligned} \min. \quad & \mathbf{c}_1^T V_1 \boldsymbol{\lambda}_1 + \mathbf{c}_1^T D_1 \boldsymbol{\mu}_1 + \dots + \mathbf{c}_N^T V_N \boldsymbol{\lambda}_N + \mathbf{c}_N^T D_N \boldsymbol{\mu}_N \\ \text{s. a} \quad & A_{01} V_1 \boldsymbol{\lambda}_1 + A_{01} D_1 \boldsymbol{\mu}_1 + \dots + A_{0N} V_N \boldsymbol{\lambda}_N + A_{0N} D_N \boldsymbol{\mu}_N = \mathbf{b} \\ & \mathbf{e}^T \boldsymbol{\lambda}_1 = 1 \\ & \ddots \quad \quad \quad \vdots \\ & \mathbf{e}^T \boldsymbol{\lambda}_N = 1 \\ & \boldsymbol{\lambda}_1, \boldsymbol{\mu}_1, \dots, \boldsymbol{\lambda}_N, \boldsymbol{\mu}_N \geq \mathbf{0}. \end{aligned}$$

Analicemos a continuación la mecánica del procedimiento con un ejemplo.

Ejemplo 9.5 Resolver el problema:

$$\begin{aligned} \min. \quad & -2x_1 - x_2 - 3x_3 - x_4 \\ \text{s. a} \quad & x_1 + x_2 + x_3 + x_4 \leq 6 \\ & x_2 + 2x_3 + x_4 \leq 4 \\ & x_1 + x_2 \leq 6 \\ & x_2 \leq 2 \\ & -x_3 + x_4 \leq 3 \\ & x_3 + x_4 \leq 5 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

Tabla 9.7

Algoritmo de descomposición de Dantzig-Wolfe

Paso 0 – *Inicialización.* Encontrar una solución básica factible del problema maestro del programa a resolver.

Paso 1 – Calcular la solución básica. Es decir,

$$B^{-1} \begin{bmatrix} \mathbf{b} \\ \mathbf{1} \end{bmatrix}.$$

Formar el vector $[\bar{\pi}_0^T, \bar{\sigma}_1, \dots, \bar{\sigma}_N]^T = \mathbf{c}_B^T B^{-1}$, donde $\mathbf{c}_B^T$ es el coste de las variables básicas; es decir, $\mathbf{c}_j^T \mathbf{v}_{ji}$ para la variable básica λ_{ji} y $\mathbf{c}_j^T \mathbf{d}_{ji}$ para la variable básica μ_{ji} .

Paso 2 – Resolver los subproblemas

$$\begin{aligned} \min. \quad & (\mathbf{c}_i^T - \bar{\pi}_0^T A_{0i}) \mathbf{x}_i \\ \text{s. a} \quad & A_{ii} \mathbf{x}_i = \mathbf{b}_i \\ & \mathbf{x}_i \geq \mathbf{0}, \end{aligned}$$

para $i = 1, \dots, N$. Calcular para cada solución $\mathbf{x}_i^*$

$$r_i^* = (\mathbf{c}_i^T - \bar{\pi}_0^T A_{0i}) \mathbf{x}_i^*.$$

Definir

$$\begin{aligned} \Psi_1 &= \{j : SP_j(\bar{\pi}_0) \text{ acotado y } r_i^* - \bar{\sigma}_i < 0\} \text{ y} \\ \Psi_2 &= \{j : SP_j(\bar{\pi}_0) \text{ no acotado y } r_i^* < 0\}. \end{aligned}$$

Si $\Psi_1 \cup \Psi_2 = \emptyset$: PARAR; el problema está resuelto. Si no, seguir.

Paso 3 – Determinar la columna q a entrar en la base seleccionándola de Ψ_1 o Ψ_2 de acuerdo con algún criterio.

Paso 4 – *Pivotación.* Resolver $B\mathbf{y} = \mathbf{a}_q$ y calcular la variable básica a salir de la base calculando $\min_{1 \leq i \leq m_o + N} \{x_{B_i}/y_i : y_i > 0\}$. Actualizar la base e ir al paso 1.

La estructura de la matriz de coeficientes de las condiciones de este problema es diagonal en bloques. Las dos primeras condiciones son las de enlace. Las submatrices diagonales son

$$A_{11} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \quad y \quad A_{22} = \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Los politopos X_1 y X_2 del problema son los de la figura 9.19.

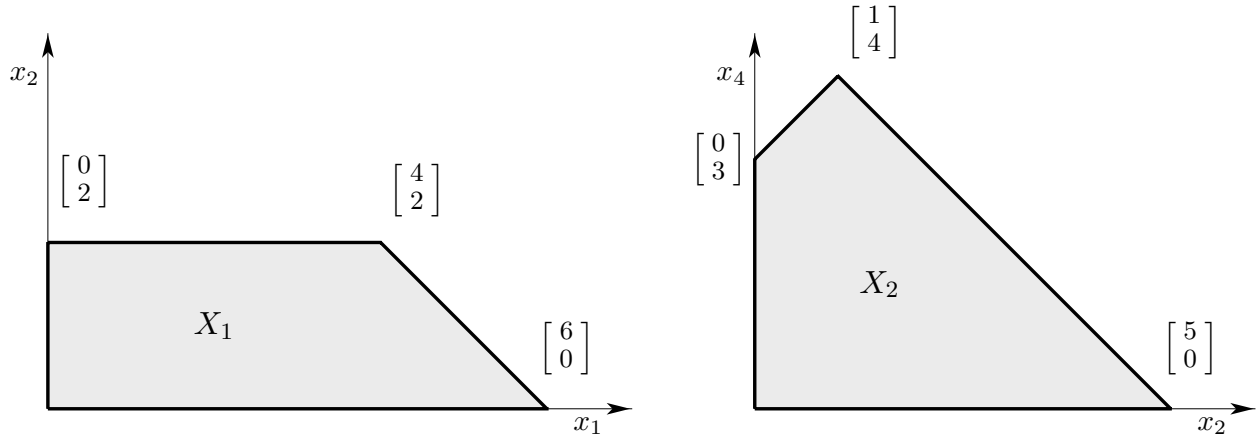


Figura 9.19
Politopos X_1 y X_2 que define el problema 9.5

El problema maestro, al no tener ninguno de estos dos politopos direcciones extremas, es

$$\begin{aligned} \min. \quad & c_1^T V_1 \lambda_1 + c_2^T V_2 \lambda_2 \\ \text{s. a} \quad & A_{01} V_1 \lambda_1 + A_{02} V_2 \lambda_2 \leq b \\ & e^T \lambda_1 = 1 \\ & e^T \lambda_2 = 1 \\ & \lambda_1, \lambda_2 \geq 0, \end{aligned}$$

donde $c_1^T = [-2, -1]$ y $c_2^T = [-3, -1]$.

Inicialmente los puntos extremos $v_{11}^T = [x_1, x_2] = [0, 0]$ y $v_{21}^T = [x_3, x_4] = [0, 0]$ pertenecen a los politopos X_1 y X_2 y satisfacen las condiciones del problema maestro. Si, además, introducimos las variables de holgura h_1 y h_2 en la primera y segunda restricción del problema maestro, obtenemos inmediatamente la primera solución básica factible:

$$\begin{bmatrix} h_1 \\ h_2 \\ \lambda_{11} \\ \lambda_{21} \end{bmatrix} = \begin{bmatrix} 6 \\ 4 \\ 1 \\ 1 \end{bmatrix}.$$

La matriz básica inicial es

$$B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \text{su inversa} \quad B^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Iteración 1. Paso 1

Los componentes del vector $\mathbf{c}_B$ son

$$c_{B_1} = 0, \quad c_{B_2} = 0, \quad c_{B_3} = \mathbf{c}_1^T \mathbf{v}_{11} = [-2, -1] \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0 \quad \text{y} \quad c_{B_4} = \mathbf{c}_2^T \mathbf{v}_{21} = [-3, -1] \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0.$$

El vector de multiplicadores simplex del problema maestro es

$$\begin{bmatrix} \bar{\pi}_{01} \\ \bar{\pi}_{02} \\ \bar{\sigma}_1 \\ \bar{\sigma}_2 \end{bmatrix} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Iteración 1. Paso 2

Resolvamos los dos subproblemas:

SUBPROBLEMA 1

$$\begin{aligned} \min. \quad & -2x_1 - x_2 \\ \text{s. a} \quad & x_1 + x_2 \leq 6 \\ & x_2 \leq 2. \end{aligned}$$

SUBPROBLEMA 2

$$\begin{aligned} \min. \quad & -3x_3 - x_4 \\ \text{s. a} \quad & -x_3 + x_4 \leq 3 \\ & x_3 + x_4 \leq 5. \end{aligned}$$

Las soluciones son $\mathbf{x}_1^* = [x_1, x_2]^T = [6, 0]^T$, con un valor de la función objetivo igual a -12 , y $\mathbf{x}_2^* = [x_3, x_4]^T = [5, 0]^T$, con un valor de la función objetivo igual a -15 .

Los costes reducidos de los puntos extremos obtenidos son

$$\begin{aligned} r_1^* &= (\mathbf{c}_1^T - \bar{\boldsymbol{\pi}}_0^T A_{01}) \mathbf{x}_1^* - \bar{\sigma}_1 = [-2, -1]^T \begin{bmatrix} 6 \\ 0 \end{bmatrix} - 0 = -12 - 0 = -12 \quad \text{y} \\ r_2^* &= (\mathbf{c}_2^T - \bar{\boldsymbol{\pi}}_0^T A_{02}) \mathbf{x}_2^* - \bar{\sigma}_2 = [-3, -1]^T \begin{bmatrix} 5 \\ 0 \end{bmatrix} - 0 = -15 - 0 = -15. \end{aligned}$$

No hemos llegado al óptimo pues estos costes reducidos son negativos. Un límite inferior de la función objetivo del problema es $\mathbf{c}_B^T B^{-1} \mathbf{b} + \sum_{i=1}^2 (\mathbf{c}_i^T - \bar{\boldsymbol{\pi}}_0^T A_{0i}) \mathbf{x}_i^* = 0 - 12 - 15 = -27$.

Iteración 1. Paso 3

Se pueden incorporar a la base tanto la variable λ_{12} como la λ_{22} , correspondientes a los nuevos puntos extremos $\mathbf{v}_{12}$ y $\mathbf{v}_{22}$. Elegimos aquella con r_i^* más negativo: λ_{22} . La nueva columna a

incorporar a la base es

$$\mathbf{a}_q = \begin{bmatrix} A_{02}\mathbf{v}_{22} \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 5 \\ 0 \end{bmatrix} \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 10 \\ 0 \\ 1 \end{bmatrix}.$$

Iteración 1. Paso 4

Resolvamos $B\mathbf{y} = \mathbf{a}_q$:

$$\mathbf{y} = B^{-1}\mathbf{a}_q = \begin{bmatrix} 5 \\ 10 \\ 0 \\ 1 \end{bmatrix}.$$

Determinemos la variable a salir de la base: calculemos

$$\min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_2}}{y_2}, \frac{x_{B_4}}{y_4} \right\} = \min \left\{ \frac{6}{5}, \frac{4}{10}, \frac{1}{1} \right\} = \frac{4}{10}.$$

Es decir, saldrá de la base $x_{B_2} = h_2$.

La nueva base es

$$B = \begin{bmatrix} 1 & 5 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \quad \text{su inversa} \quad B^{-1} = \begin{bmatrix} 1 & -1/2 & 0 & 0 \\ 0 & 1/10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1/10 & 0 & 1 \end{bmatrix}.$$

Iteración 2. Paso 1

La nueva solución es

$$\begin{bmatrix} h_1 \\ \lambda_{22} \\ \lambda_{11} \\ \lambda_{21} \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 4 \\ 2/5 \\ 1 \\ 3/5 \end{bmatrix}.$$

La mejor solución factible de momento del problema original es

$$\mathbf{x} = \lambda_{11}\mathbf{v}_{11} + \lambda_{21}\mathbf{v}_{21} + \lambda_{22}\mathbf{v}_{22} = \begin{bmatrix} 0 \\ 0 \\ 2 \\ 0 \end{bmatrix},$$

con un valor de la función objetivo igual a -6 .

Los nuevos componentes del vector $\mathbf{c}_B$ son:

$$c_{B_1} = 0, \quad c_{B_2} = \mathbf{c}_2^T \mathbf{v}_{22} = [-3, -1] \begin{bmatrix} 5 \\ 0 \end{bmatrix} = -15, \quad c_{B_3} = \mathbf{c}_1^T \mathbf{v}_{11} = 0 \quad \text{y} \quad c_{B_4} = \mathbf{c}_2^T \mathbf{v}_{21} = 0.$$

El nuevo vector de multiplicadores simplex del problema maestro,

$$\begin{bmatrix} \bar{\pi}_{01} \\ \bar{\pi}_{02} \\ \bar{\sigma}_1 \\ \bar{\sigma}_2 \end{bmatrix} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/2 & 1/10 & 0 & -1/10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -15 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -3/2 \\ 0 \\ 0 \end{bmatrix}.$$

Iteración 2. Paso 2

Planteemos los nuevos subproblemas:

$$\begin{aligned} (\mathbf{c}_1^T - \bar{\pi}_0^T A_{01}) &= [-2, -1] - [0, -3/2] \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = [-2, 1/2]; \\ (\mathbf{c}_2^T - \bar{\pi}_0^T A_{02}) &= [-3, -1] - [0, -3/2] \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = [0, 1/2]. \end{aligned}$$

Resolvámoslos:

SUBPROBLEMA 1

$$\begin{aligned} \min. \quad & -2x_1 + \frac{1}{2}x_2 \\ \text{s. a} \quad & x_1 + x_2 \leq 6 \\ & x_2 \leq 2. \end{aligned}$$

SUBPROBLEMA 2

$$\begin{aligned} \min. \quad & \frac{1}{2}x_4 \\ \text{s. a} \quad & -x_3 + x_4 \leq 3 \\ & x_3 + x_4 \leq 5. \end{aligned}$$

Las soluciones son $\mathbf{x}_1^* = [x_1, x_2]^T = [6, 0]^T$, con un valor de la función objetivo igual a -12 , y $\mathbf{x}_2^* = [x_3, x_4]^T = [5, 0]^T$, con un valor de la función objetivo igual a 0 .

Los costes reducidos de estos nuevos puntos extremos son

$$\begin{aligned} r_1^* &= (\mathbf{c}_1^T - \bar{\pi}_0^T A_{01}) \mathbf{x}_1^* - \bar{\sigma}_1 = [-2, 1/2] \begin{bmatrix} 6 \\ 0 \end{bmatrix} - 0 = -12 - 0 = -12 \text{ y} \\ r_2^* &= (\mathbf{c}_2^T - \bar{\pi}_0^T A_{02}) \mathbf{x}_2^* - \bar{\sigma}_2 = [0, 1/2] \begin{bmatrix} 5 \\ 0 \end{bmatrix} - 0 = -0 - 0 = 0. \end{aligned}$$

No hemos llegado al óptimo pues uno de estos costes reducidos es negativo. Un límite inferior de la función objetivo del problema es $\mathbf{c}_B^T B^{-1} \mathbf{b} + \sum_{i=1}^2 (\mathbf{c}_i^T - \bar{\pi}_0^T A_{0i}) \mathbf{x}_i^* = -6 - 12 - 0 = -18$.

Iteración 2. Paso 3

La única posible variable a incorporar a la base es λ_{13} , correspondiente a $\mathbf{v}_{13}$. La nueva columna a incorporar a la base es

$$\mathbf{a}_q = \begin{bmatrix} A_{01} \mathbf{v}_{13} \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 0 \end{bmatrix} \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \\ 1 \\ 0 \end{bmatrix}.$$

Iteración 2. Paso 4

Resolvamos $B\mathbf{y} = \mathbf{a}_q$:

$$\mathbf{y} = B^{-1}\mathbf{a}_q = \begin{bmatrix} 1 & -1/2 & 0 & 0 \\ 0 & 1/10 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1/10 & 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 0 \\ 1 \\ 0 \end{bmatrix}.$$

Determinemos la variable a salir de la base: calculemos

$$\min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_3}}{y_3} \right\} = \min \left\{ \frac{4}{6}, \frac{1}{1} \right\} = \frac{2}{3}.$$

Saldrá de la base $x_{B_1} = h_1$.

La nueva base es

$$B = \begin{bmatrix} 6 & 5 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \quad \text{y su inversa} \quad B^{-1} = \begin{bmatrix} 1/6 & -1/12 & 0 & 0 \\ 0 & 1/10 & 0 & 0 \\ -1/6 & 1/12 & 1 & 0 \\ 0 & -1/10 & 0 & 1 \end{bmatrix}.$$

Iteración 3. Paso 1

La nueva solución es

$$\begin{bmatrix} \lambda_{13} \\ \lambda_{22} \\ \lambda_{11} \\ \lambda_{21} \end{bmatrix} = B^{-1}\mathbf{b} = \begin{bmatrix} 1/6 & -1/12 & 0 & 0 \\ 0 & 1/10 & 0 & 0 \\ -1/6 & 1/12 & 1 & 0 \\ 0 & -1/10 & 0 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 4 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 2/5 \\ 1/3 \\ 3/5 \end{bmatrix}.$$

La mejor solución factible de momento del problema original es

$$\mathbf{x} = \lambda_{11}\mathbf{v}_{11} + \lambda_{13}\mathbf{v}_{13} + \lambda_{21}\mathbf{v}_{21} + \lambda_{22}\mathbf{v}_{22} = \begin{bmatrix} 4 \\ 0 \\ 2 \\ 0 \end{bmatrix},$$

con un valor de la función objetivo igual a -14 .

Los nuevos componentes del vector $\mathbf{c}_B$ son:

$$c_{B_1} = \mathbf{c}_1^T \mathbf{v}_{13} = -12, \quad c_{B_2} = \mathbf{c}_2^T \mathbf{v}_{22} = -15, \quad c_{B_3} = \mathbf{c}_1^T \mathbf{v}_{11} = 0 \quad \text{y} \quad c_{B_4} = \mathbf{c}_2^T \mathbf{v}_{21} = 0.$$

El nuevo vector de multiplicadores simplex del problema maestro,

$$\begin{bmatrix} \bar{\pi}_{01} \\ \bar{\pi}_{02} \\ \bar{\sigma}_1 \\ \bar{\sigma}_2 \end{bmatrix} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 1/6 & 0 & -1/6 & 0 \\ -1/12 & 1/10 & 1/12 & -1/10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -12 \\ -15 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ -1/2 \\ 0 \\ 0 \end{bmatrix}.$$

Iteración 3. Paso 2

Planteemos los nuevos subproblemas:

$$\begin{aligned} (\mathbf{c}_1^T - \bar{\boldsymbol{\pi}}_0^T A_{01}) &= [-2, -1] - [-2, -1/2] \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = [0, 3/2]; \\ (\mathbf{c}_2^T - \bar{\boldsymbol{\pi}}_0^T A_{02}) &= [-3, -1] - [-2, -1/2] \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = [0, 3/2]. \end{aligned}$$

Resolvámoslos:

SUBPROBLEMA 1

$$\begin{aligned} \min. \quad & \frac{3}{2}x_2 \\ \text{s. a} \quad & x_1 + x_2 \leq 6 \\ & x_2 \leq 2. \end{aligned}$$

SUBPROBLEMA 2

$$\begin{aligned} \min. \quad & \frac{3}{2}x_4 \\ \text{s. a} \quad & -x_3 + x_4 \leq 3 \\ & x_3 + x_4 \leq 5. \end{aligned}$$

Las soluciones son $\mathbf{x}_1^* = [x_1, x_2]^T = [0, 0]^T$, con un valor de la función objetivo igual a 0 y $\mathbf{x}_2^* = [x_3, x_4]^T = [0, 0]^T$, con un valor de la función objetivo igual también a 0.

Los costes reducidos de estos nuevos puntos extremos son

$$\begin{aligned} r_1^* &= (\mathbf{c}_1^T - \bar{\boldsymbol{\pi}}_0^T A_{01}) \mathbf{x}_1^* - \bar{\sigma}_1 = 0 - 0 = 0 \quad \text{y} \\ r_2^* &= (\mathbf{c}_2^T - \bar{\boldsymbol{\pi}}_0^T A_{02}) \mathbf{x}_2^* - \bar{\sigma}_2 = 0 - 0 = 0. \end{aligned}$$

Hemos llegado al óptimo del problema.

La solución óptima del problema original es

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \lambda_{11} \mathbf{v}_{11} + \lambda_{13} \mathbf{v}_{13} = \frac{1}{3} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \frac{2}{3} \begin{bmatrix} 6 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix}$$

y

$$\begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = \lambda_{21} \mathbf{v}_{21} + \lambda_{22} \mathbf{v}_{22} = \frac{3}{5} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \frac{2}{5} \begin{bmatrix} 5 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

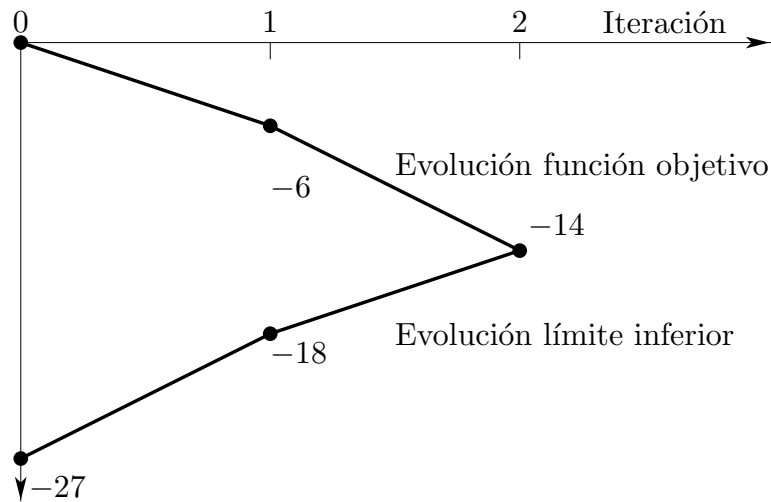
Por consiguiente,

$$\mathbf{x}^* = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 2 \\ 0 \end{bmatrix}.$$

El valor óptimo de la función objetivo es -14 .

La evolución del valor de la función objetivo del problema y del límite inferior calculado es la de la figura 9.20. ●

El procedimiento de la descomposición fue muy usado durante los años 60 y 70 debido fundamentalmente al tamaño de los problemas a resolver y a las limitaciones de los ordenadores de aquellos años en cuanto a memoria disponible y velocidad de cálculo. En la actualidad se

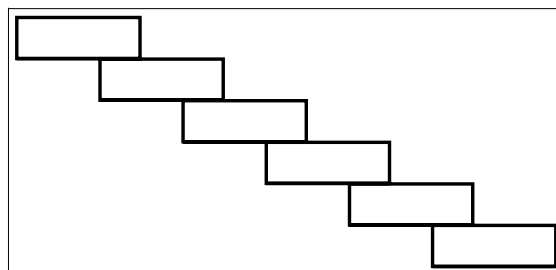
**Figura 9.20**

Evolución del valor de la función objetivo del problema del ejemplo 9.5 y del de su límite inferior calculado

utiliza para resolver problemas de estructura similar a las expuestas, debido a la gran ventaja que representa poder disponer, en cada momento del proceso, de un límite inferior para el valor de la función objetivo global. Si en la resolución de un determinado problema no es necesario obtener una solución con precisión absoluta, una vez alcanzado un error relativo aceptable, se puede interrumpir el proceso disminuyendo de esta manera el tiempo de cálculo a emplear. Esta última estrategia permite resolver en la actualidad problemas dinámicos de gran tamaño.

9.2.2 Problemas con estructura en escalera

Con este nombre se conocen aquellos problemas cuya matriz de coeficientes de las condiciones presenta una estructura como la de la figura 9.21. Surgen habitualmente en problemas de programación de actividades multiperíodo. Aunque se pueden resolver por el método *simplex* normal, su estructura, y el hecho de que en general requieren muchas más iteraciones que para resolver problemas generales de dimensiones parecidas, aconsejan el uso de procedimientos

**Figura 9.21**

Estructura en escalera de una matriz de condiciones

iterativos basados en el principio de descomposición que acabamos de exponer.

9.3 El problema del corte de materiales

Muchos de los productos primarios que emplean diversas industrias para fabricar sus productos —papel, textiles, plásticos, aluminio, etc— se obtienen del suministrador en la forma de rollos, bobinas o grandes planchas. Las dimensiones de estas formas varían de un suministrador a otro.

Si un fabricante de bobinas, por ejemplo, suministra éstas con una anchura determinada y el comprador necesita cortarlas para adaptarlas a sus necesidades (imaginemos por ejemplo el impresor de revistas o periódicos de distintos formatos), el problema que se le plantea es cómo hacer esos cortes de tal forma que se desperdicie la menor cantidad posible de material.

Supongamos que las bobinas tienen una anchura X y que las necesidades del fabricante estriban en b_i bobinas, de anchuras respectivas x_i , $i = 1, \dots, m$. Las más estrechas habrán de obtenerse a partir de las de anchura X cortándolas según un determinado *patrón*: por ejemplo, una bobina de 2 metros se puede cortar en 3 de 60 centímetros de ancho y una de 20 cm, etc. Conceptualmente, todos los patrones posibles se pueden considerar en una matriz A en la que cada coeficiente a_{ij} representa cuántas bobinas de anchura x_i produce el patrón j : $i = 1, \dots, m$; $j = 1, \dots, n$. El patrón del caso antes considerado generaría el vector columna $[3, 1, 0, \dots, 0]^T$. Si hacemos que el coste de producción de cada uno de los patrones sea uno: $c_j = 1$, $j = 1, \dots, n$, el problema que se plantea para determinar cuántas bobinas grandes son necesarias para conseguir la demanda b_i es como sigue:

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}. \end{aligned} \tag{9.18}$$

Es decir, un programa de programación lineal exactamente igual a los que venimos planteando hasta ahora. Aunque las variables del vector $\mathbf{x}$ deberían estar restringidas a tomar valores sólo enteros, habitualmente se considera buena la aproximación que se obtiene de resolver (9.18); al menos cuando las demandas b_i son suficientemente grandes.

Resolver el problema (9.18) es de por sí una tarea muy considerable. Aún en el caso de que el número de anchuras distintas, m , fuese pequeño, el número de patrones, n , que se pueden obtener es gigantesco; sólo el hecho de generar la matriz A puede ser algo prohibitivo. Por ejemplo, si las bobinas base tienen 200 pulgadas de anchura y se demandan 40 anchuras diferentes, desde 20 a 80 pulgadas, los patrones que se pueden dar exceden los 100 millones.

En 1963 Gilmore y Gomory idearon una forma de abordar éste problema eficazmente, utilizando el método *simplex* revisado, mediante la generación de las columnas de la matriz A según se hacían necesarias en lugar de todas a la vez por anticipado (algo parecido a lo que se hace en el procedimiento de descomposición antes visto).

La obtención de una solución inicial básica factible no es difícil. En efecto, si el patrón i lo forman $\lfloor X/x_i \rfloor$ bobinas de anchura x_i y ninguna de otra anchura,⁶ repitiendo este patrón para cada una de las x_i anchuras, las m primeras columnas de la matriz A pueden determinar la base —diagonal— de partida del problema. Supongamos, por consiguiente, que en cualquier

⁶El símbolo $\lfloor x \rfloor$, recordemos, designa el mayor número entero menor o igual que x

iteración del proceso de resolución del problema por el método *simplex* revisado se dispone de una solución básica factible.

El cálculo de los multiplicadores *simplex*, π , tampoco plantea mayor problema: habría que resolver el sistema $B^T \pi = e$, donde e es el vector de dimensión adecuada en el que todos sus componentes son igual a la unidad (recordemos que todos los c_j son 1).

El paso en el que se han de calcular los costes reducidos, $\bar{c}_j = 1 - \pi^T a_j$, para todo j , al objeto de determinar si se ha llegado al óptimo del problema o, de no ser así, determinar qué variable ha de entrar en la base, parece más complicado dado que no se conocen todas las columnas de la matriz A . Esta aparente dificultad, sin embargo, debido a la particular estructura del problema, se puede resolver implícitamente.

Un vector $a = [\alpha_1, \alpha_2, \dots, \alpha_m]^T \in \mathbf{Z}_+^m$ (todos sus componentes son enteros no negativos) será un vector columna de la matriz A del problema si se corresponde con un patrón factible: si $x^T a \leq X$, donde $x = [x_1, x_2, \dots, x_m]^T$. Lo que se pretende en un paso del procedimiento es saber si el coste reducido de cada vector columna no básico a , es decir $1 - \pi^T a$, es no negativo y, de no ser así, encontrar el a factible de coste reducido más negativo. En definitiva, resolver el siguiente subproblema:

$$\begin{aligned} \max. \quad & \pi^T a \\ \text{s. a} \quad & x^T a \leq X \\ & a \in \mathbf{Z}_+^m. \end{aligned} \tag{9.19}$$

Este último problema se conoce como el *problema de la mochila* (si π_i es el valor de un artículo y a_i su peso, se trata de transportar las cosas de más valor con un peso total máximo no superior a X). Si el óptimo del problema (9.19) es menor o igual que uno, quiere decir que todos los costes reducidos son no negativos y la solución básica factible que en ese momento se tenga es la óptima; si no, el valor óptimo proporcionará un nuevo vector columna, $a_q = a$ a introducir en la base, procediéndose a continuación como de costumbre. De esta forma vemos cómo un problema de las características del de corte de materiales puede ser resuelto por el método *simplex* revisado sin necesidad de conocer de antemano todas las columnas de la matriz A , pues se generan cuando haga falta.

Para resolver el problema (9.19) lo más rápido y eficaz es usar procedimientos de *ramificación y acotamiento* de programación entera. No obstante, cuando —como es lo más habitual— X y las variables x_i son números enteros, también se puede utilizar un procedimiento de *programación dinámica* como el que se detalla a continuación.

Sea $f(v)$ el valor óptimo del problema (9.19) cuando se sustituye X por v . Para $0 \leq v < x_{\min}$, $f(v) = 0$, donde $x_{\min} = \min x_i$. La expresión de $f(X)$ se obtiene usando la siguiente fórmula recursiva:

$$f(v) = \max_{\substack{1 \leq i \leq m \\ x_i \leq v}} \{f(v - x_i) + \pi_i\},$$

para $v = x_{\min}, \dots, X$. La solución óptima que lleve a $f(X)$ se puede obtener procediendo hacia atrás y llevando constancia en cada paso del índice i que maximiza la expresión entre llaves.

Ideas semejantes a las aquí expuestas para el caso unidimensional de las bobinas se pueden utilizar para determinar el mejor patrón de corte de planchas rectangulares o cuadradas. El problema, aunque parecido, sería de un tamaño considerablemente superior.

Ejemplo 9.6 A partir de bobinas de 91 pulgadas, se trata de resolver el problema de conseguir:

78 bobinas de 25,5 pulgadas,
 40 bobinas de 22,5 "
 30 bobinas de 20 " y
 30 bobinas de 15 pulgadas.

Utilizando la estrategia descrita anteriormente para obtener una solución básica factible inicial se llega a que

$$B = \begin{bmatrix} 3 & & & \\ & 4 & & \\ & & 4 & \\ & & & 6 \end{bmatrix} \quad \text{y} \quad \mathbf{x}_B = B^{-1}\mathbf{b} = \begin{bmatrix} 1/3 & & & \\ & 1/4 & & \\ & & 1/4 & \\ & & & 1/6 \end{bmatrix} \begin{bmatrix} 78 \\ 40 \\ 30 \\ 30 \end{bmatrix} = \begin{bmatrix} 26 \\ 10 \\ 15/2 \\ 5 \end{bmatrix}.$$

Iteración 1. Paso 1

Resolviendo el sistema $B^T\boldsymbol{\pi} = \mathbf{c}_B$, se obtiene que

$$\boldsymbol{\pi} = B^{-T}\mathbf{c}_B = \begin{bmatrix} 1/3 & & & \\ & 1/4 & & \\ & & 1/4 & \\ & & & 1/6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/4 \\ 1/4 \\ 1/6 \end{bmatrix}.$$

Iteración 1. Paso 2

Determinemos si se ha llegado al óptimo, o la columna que ha de entrar en la base. Resolvamos para ello:

$$\begin{aligned} \max. \quad & \frac{1}{3}a_1 + \frac{1}{4}a_2 + \frac{1}{4}a_3 + \frac{1}{6}a_4 \\ \text{s. a} \quad & 25,5a_1 + 22,5a_2 + 20a_3 + 15a_4 \leq 91 \\ & a_1, a_2, a_3, a_4 \geq 0 \\ & a_1, a_2, a_3, \text{ y } a_4 \text{ enteras.} \end{aligned}$$

La solución de este problema, obtenida por un procedimiento de ramificación y acotamiento de programación entera, es $\mathbf{a} = [2, 0, 2, 0]^T$. Como en este punto la función objetivo es mayor que 1, no se ha alcanzado el óptimo del problema; la columna $\mathbf{a}$ será la próxima a entrar en la base.

Iteración 1. Paso 3

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}$. La solución es

$$\mathbf{y} = B^{-1}\mathbf{a} = \begin{bmatrix} 1/3 & & & \\ & 1/4 & & \\ & & 1/4 & \\ & & & 1/6 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 0 \\ 1/2 \\ 0 \end{bmatrix}.$$

Iteración 1. Paso 4

Determinemos la columna que ha de salir de la base. Como siempre:

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_3}}{y_3} \right\} = \min \left\{ \frac{26}{2/3}, \frac{7,5}{1/2} \right\} = 15.$$

La columna que sale de la base es la tercera.

Iteración 1. Paso 5

La nueva base y solución son

$$B = \begin{bmatrix} 3 & 2 \\ & 4 \\ & 2 \\ & & 6 \end{bmatrix} \quad y \quad \mathbf{x}_B = \begin{bmatrix} 26 - \theta \cdot \frac{2}{3} \\ 10 \\ \theta \\ 5 \end{bmatrix} = \begin{bmatrix} 16 \\ 10 \\ 15 \\ 5 \end{bmatrix}.$$

Iteración 2. Paso 1

Resolviendo el sistema $B^T \boldsymbol{\pi} = \mathbf{c}_B$ se llega a que

$$\boldsymbol{\pi} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 1/3 & & & \\ & 1/4 & & \\ -1/3 & & 1/2 & \\ & & & 1/6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/4 \\ 1/6 \\ 1/6 \end{bmatrix}.$$

Iteración 2. Paso 2

Determinemos una vez más si se ha llegado al óptimo o la columna que ha de entrar en la base. Resolvamos:

$$\begin{aligned} \max. \quad & \frac{1}{3}a_1 + \frac{1}{4}a_2 + \frac{1}{6}a_3 + \frac{1}{6}a_4 \\ \text{s. a} \quad & 25,5a_1 + 22,5a_2 + 20a_3 + 15a_4 \leq 91 \\ & a_1, a_2, a_3, a_4 \geq 0 \\ & a_1, a_2, a_3 \text{ y } a_4, \text{ enteras.} \end{aligned}$$

La solución de este problema es $\mathbf{a} = [2, 1, 0, 1]^T$. Como la función objetivo es mayor que 1, todavía no se ha conseguido el óptimo del problema. La columna $\mathbf{a}$ será la próxima a entrar en la base.

Iteración 2. Paso 3

Resolvamos el sistema $B\mathbf{y} = \mathbf{a}$. La solución es

$$\mathbf{y} = B^{-1} \mathbf{a} = \begin{bmatrix} 1/3 & & -1/3 \\ & 1/4 & \\ & & 1/2 \\ & & & 1/6 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 1/4 \\ 0 \\ 1/6 \end{bmatrix}.$$

Iteración 2. Paso 4

Determinemos la columna que ha de salir de la base:

$$\theta = \min \left\{ \frac{x_{B_1}}{y_1}, \frac{x_{B_2}}{y_2}, \frac{x_{B_4}}{y_4} \right\} = \min \left\{ \frac{16}{2/3}, \frac{10}{1/4}, \frac{5}{1/6} \right\} = 24.$$

La columna que sale de la base es la primera.

Iteración 2. Paso 5

La nueva base y solución son

$$B = \begin{bmatrix} 2 & 2 \\ 1 & 4 \\ & 2 \\ 1 & 6 \end{bmatrix} \quad \text{y} \quad \mathbf{x}_B = \begin{bmatrix} \theta \\ 10 - \theta \cdot \frac{1}{4} \\ 15 \\ 5 - \theta \cdot \frac{1}{6} \end{bmatrix} = \begin{bmatrix} 24 \\ 4 \\ 15 \\ 1 \end{bmatrix}.$$

Iteración 3. Paso 1

Resolviendo el sistema $B^T \boldsymbol{\pi} = \mathbf{c}_B$ se obtiene que

$$\boldsymbol{\pi} = B^{-T} \mathbf{c}_B = \begin{bmatrix} 1/2 & -1/8 & & -1/12 \\ & 1/4 & & \\ -1/2 & 1/8 & 1/2 & 1/12 \\ & & & 1/6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 7/24 \\ 1/4 \\ 5/24 \\ 1/6 \end{bmatrix}.$$

Iteración 3. Paso 2

Determinemos una vez más si se ha llegado al óptimo o la columna que ha de entrar en la base; resolvamos:

$$\begin{aligned} \max. \quad & \frac{7}{24}a_1 + \frac{1}{4}a_2 + \frac{5}{24}a_3 + \frac{1}{6}a_4 \\ \text{s. a} \quad & 25,5a_1 + 22,5a_2 + 20a_3 + 15a_4 \leq 91 \\ & a_1, a_2, a_3, a_4 \geq 0 \\ & a_1, a_2, a_3 \text{ y } a_4, \text{ enteras.} \end{aligned}$$

La solución de este problema es como mucho la unidad: no se puede encontrar una solución en variables enteras cuya función objetivo sea mayor. Hemos llegado al óptimo del problema. La solución final se describe en la tabla 9.8. ●

Referencias

Existen diversas y muy buenas referencias sobre los asuntos tratados en este capítulo. Para la elaboración de lo relativo a problemas de flujos en redes y sus aspectos prácticos de implementación en ordenador, se puede seguir esencialmente a Kennington y Helgason [1980], Goldfarb y Todd [1989], Ahuja, Magnati y Orlin [1989], Chvátal [1983] y Bazaraa y Jarvis [1977]. Para lo referente a las estructuras de datos a utilizar así como cómo actualizarlas, hemos seguido a

Tabla 9.8
Resultado del problema del ejemplo 9.6

Pulgadas	Patrón de Corte				Total Bobinas
	↓	↓	↓	↓	
25,5	2		2		78
22,5	1	4			40
20			2		30
15	1			6	30
Total pulgadas	88,5	90	91	90	
Desecho, pulgadas	2,5	1	-	1	
Bobinas de 91 pulg.	24	4	15	1	44

Chvátal [1983], Bradley, Brown y Graves [1977] y Grigoriadis [1986]. Otras referencias relativas a programación lineal para flujos en redes son: Gondran y Minoux [1979], Minoux y Bartnik [1986], Lawler [1976]. Como artículos paradigmáticos en este sentido están: Bradley, Brown y Graves [1977] y Grigoriadis [1980].

Por lo que respecta al principio de descomposición de Dantzig-Wolfe, lo expuesto está basado en Goldfarb y Todd [1989], Bazaraa y Jarvis [1977] y Luenberger [1984].

El problema del corte de materiales sigue a Salkin [1977].

Ejercicios

9.1. Transformar las bases que siguen en triangulares inferiores y dibujar sus árboles maximales.

$$a) \quad B = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & -1 \\ 1 & -1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \quad \begin{matrix} \text{Nudo 1} \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix}$$

$$b) \quad B = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & -1 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \quad \begin{matrix} \text{Nudo 1} \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix}$$

$$c) \quad B = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \begin{matrix} \text{Nudo 1} \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{matrix}$$

- 9.2. Un seguidor de un equipo de fútbol americano que vive en San Francisco desea ver la *super bowl* que se celebra este año en Dallas, Texas. Nuestro amigo quiere viajar el mismo día del partido llegando no más tarde de las 7:00 de la tarde a Dallas. Desgraciadamente, como ocurre en estos casos, no dispone de todo el dinero que quisiera para viajar, por lo que tiene que volar con una compañía, la Gamma Airlines, cuya disponibilidad de vuelos San Francisco-Dallas no es muy amplia que digamos. Los vuelos disponibles son los de la siguiente tabla.

Vuelo N°	Origen	Destino	Hora Salida	Hora Llegada	Coste \$
1	San Francisco	Chicago	8:00 AM	1:00 PM	100
	Chicago	Atlanta	2:00 PM	3:00 PM	100
	Atlanta	Dallas	3:40 PM	6:00 PM	250
2	San Francisco	Atlanta	11:00 AM	4:00 PM	250
	Atlanta	Chicago	4:00 PM	5:00 PM	150
	Chicago	Dallas	5:00 PM	7:00 PM	100
3	Atlanta	Miami	4:00 PM	5:00 PM	100
	Miami	Dallas	5:00 PM	7:00 PM	100
4	San Francisco	New York	8:00 AM	2:00 PM	240
	New York	Atlanta	2:00 PM	4:00 PM	50
	Atlanta	Dallas	4:00 PM	6:00 PM	210

Para transbordar de un vuelo a otro la compañía Gamma Airlines requiere como mínimo un tiempo de una hora.

Formular el problema que se plantea nuestro amigo, tratando de minimizar el coste de su transporte a Dallas, y resolverlo usando el método *simplex* especializado a problemas de flujos en redes que se ha estudiado en este capítulo.

- 9.3. Determinar las variables duales de cada una de las bases siguientes:

a)

Arco	De Nudo	A Nudo	Coste c_j
1	6	5	10
2	3	6	8
3	7	3	6
4	4	3	9
5	6	2	7
6	1	2	3
7	6	0	0

b)

Arco	De Nudo	A Nudo	Coste c_j
1	1	0	0
2	1	3	20
3	1	4	5
4	1	2	15
5	2	5	10
6	6	2	0

c)

Arco	De Nudo	A Nudo	Coste c_j
1	1	3	70
2	2	1	30
3	1	4	90
4	1	5	50
5	5	7	100
6	8	4	40
7	8	9	80
8	4	6	60
9	4	0	0

9.4. Resolver el siguiente problema de flujos en red:

$$\begin{array}{ll}
 \text{minimizar} & 2x_1 + 4x_2 + x_3 + x_4 + 9x_5 - x_6 \\
 \text{s. a} & \begin{array}{rcl}
 x_1 - x_2 & \leq & 6 \\
 -x_1 - x_3 + x_4 + x_5 & \leq & 6 \\
 -x_2 + x_3 - x_4 & \leq & -3 \\
 -x_5 - x_6 & \leq & -5 \\
 1 \leq x_1 \leq 3 \\
 2 \leq x_2 \leq 3 \\
 2 \leq x_3 \leq 5 \\
 0 \leq x_4 \leq 3 \\
 1 \leq x_5 \leq 3 \\
 0 \leq x_6 \leq 3.
 \end{array}
 \end{array}$$

9.5. Considérese el grafo de la figura 9.22, donde las cifras entre corchetes indican las cantidades

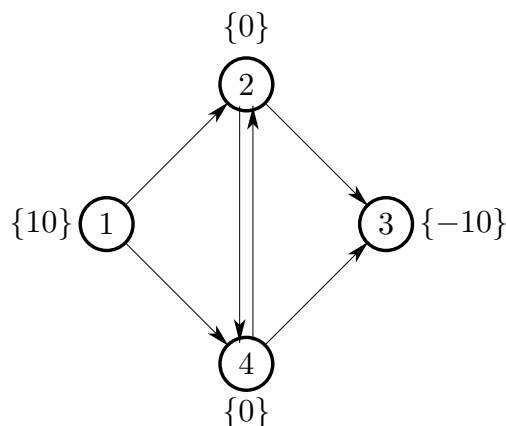


Figura 9.22
Digrafo del ejercicio 5

demandadas/ofertadas (+/-). Supóngase que se requiere que el flujo a través del nudo cuatro sea como mínimo de 5 unidades y como máximo de 5. Formular el problema de satisfacer la demanda a coste mínimo añadiendo los arcos que se consideren necesarios y cumpliendo la condición impuesta.

9.6. Probar que un problema de flujos en redes, en el que los límites inferiores de los flujos por los arcos no son cero, se puede convertir en uno en que sí son cero.

- 9.7. Probar que un problema de flujos en redes, en el que los límites superiores de los flujos por los arcos son finitos, se puede convertir en uno en que esos límites son infinito.
- 9.8. Esbozar una variante del método *simplex* especializado para flujos en redes en la que se tuviese en cuenta la posibilidad de que los arcos tuviesen *ganancia*; esto es, que cada vector columna, $\mathbf{a}_{ij}$, de la matriz de coeficientes de las condiciones, A , tuviese la forma $\mathbf{a}_{ij} = \mathbf{e}_i - p_{ij}\mathbf{e}_j$, $p_{ij} > 0$.
- 9.9. Esbozar un algoritmo para resolver un programa lineal del tipo,

$$\begin{aligned} &\text{minimizar } \mathbf{c}^T \mathbf{x} + c_{n+1}x_{n+1} \\ &\text{s. a} \quad \mathbf{Ax} + \mathbf{a}_{n+1}x_{n+1} = \mathbf{b} \\ &\quad \mathbf{x} \geq \mathbf{0}, x_{n+1} \geq 0, \end{aligned}$$

donde A es una matriz de incidencia nudo-arco. Aplicar el método considerado para resolver

$$\begin{aligned} &\text{minimizar } 2x_1 + 3x_2 + x_3 + 3x_4 + 5x_5 + 4x_6 \\ &\text{s. a} \quad \begin{array}{rcccccccl} x_1 & + & x_2 & & & & + & x_6 & = & 2 \\ -x_1 & & & + & x_3 & + & x_4 & & - & 2x_6 & = & 3 \\ & & - & x_2 & - & x_3 & & & + & x_5 & + & 3x_6 & = & -1 \\ & & & & & & - & x_4 & - & x_5 & - & x_6 & = & -4 \\ & & & & & & & & & & & x_1, \dots, x_6 & \geq & 0. \end{array} \end{aligned}$$

- 9.10. ¿Se pueden generalizar los resultados del ejercicio anterior al caso en que la matriz de coeficientes de las condiciones del problema tiene la forma $[A, D]$, donde A es una matriz de incidencia nudo-arco y D una matriz arbitraria?
- 9.11. Probar que la matriz de coeficientes de las condiciones del programa lineal de flujo en redes con variables acotadas superior e inferiormente, esto es,

$\mathbf{x}$	$\mathbf{h}_1$	$\mathbf{h}_2$	TD
A	0	0	$\mathbf{b}$
I	$-I$	0	$\mathbf{l}$
I	0	I	$\mathbf{u}$

es totalmente unimodular.

- 9.12. Resolver el siguiente problema de programación lineal mediante el método de Dantzig-Wolfe:

$$\begin{aligned} &\text{minimizar } -x_1 - x_2 - 2x_3 - x_4 \\ &\text{s. a} \quad \begin{array}{rcl} x_1 + 2x_2 + 2x_3 + x_4 & \leq & 40 \\ -x_1 + x_2 + x_3 + x_4 & \leq & 10 \\ x_1 + 3x_2 & \leq & 30 \\ 2x_1 + x_2 & \leq & 20 \\ & x_3 & \leq & 20 \\ & & x_4 & \leq & 10 \\ & & x_3 + x_4 & \leq & 15 \\ & & & x_1, \dots, x_4 & \geq & 0. \end{array} \end{aligned}$$

- 9.13. Una compañía tiene dos fábricas: una en Atlanta y la otra en Los Angeles (EE.UU.). Estas fábricas producen neveras y lavadoras/secadoras. La capacidad de producción anual de la fábrica de Atlanta es de 5.000 neveras y 7.000 lavadoras/secadoras; la de Los Angeles, 8.000 neveras y 4.000 lavadoras/secadoras. La compañía tiene tres compradores habituales de sus productos en

New York, Seattle y Miami. Las demandas anuales de estos compradores son las que se indican en la tabla siguiente.

DEMANDA\COMPRADOR	NEW YORK	SEATTLE	MIAMI
Neveras	4.000	5.000	4.000
Lavadoras/Secadoras	3.000	3.000	4.000

Los productos se transportan por ferrocarril. Los costes unitarios del transporte en dólares son los de la tabla siguiente.

Fábrica	Comprador		
	New York	Seattle	Miami
Atlanta	6 (6.000)	14 (3.000)	7 (8.000)
Los Angeles	10 (3.000)	8 (9.000)	15 (3.000)

Los valores indicados entre paréntesis corresponden al máximo número de unidades transportables. Se desea encontrar el patrón de unidades a transportar de tal forma que se satisfaga la demanda al mínimo coste.

a) Formular el problema.

b) Resolverlo aplicando el método de descomposición de Dantzig-Wolfe.

9.14. Supóngase que un programa lineal requiere $3m/2$ iteraciones para resolverlo y que se emplean las técnicas habituales de modificación de la base. ¿Se puede encontrar una descomposición óptima de las condiciones? Es decir, determinar unos m_1 y m_2 , tales que $m_1 + m_2 = m$, siendo m_1 las primeras condiciones del problema maestro, de tal forma que se minimice el esfuerzo para resolver el problema.

9.15. Resolver mediante descomposición el problema de programación lineal que sigue:

$$\begin{aligned}
 &\text{minimizar } -2x_1 + 5x_2 - 4x_3 \\
 &\text{s. a} \quad \begin{aligned}
 &x_1 + 2x_2 + a_1x_3 \leq 6 \\
 &3x_1 - 6x_2 + a_2x_3 \leq 5 \\
 &2a_1 + 3a_2 = 4 \\
 &x_1, x_2, x_3, a_1, a_2 \geq 0.
 \end{aligned}
 \end{aligned}$$

Hacer $X = \{[a_1, a_2]^T : 2a_1 + 3a_2 = 4, a_1, a_2 \geq 0\}$.

Capítulo 10

MÉTODOS DE PUNTOS INTERIORES

APESAR DE QUE desde su publicación por George B. Dantzig en 1947 el método *simplex* ha demostrado sobradamente ser altamente eficaz para resolver todo tipo de problemas de programación lineal, el hecho de que en determinadas circunstancias su complejidad sea exponencial ha motivado en los últimos años un elevado número de intentos, tanto teóricos como prácticos, de obtener otros procedimientos con mejor complejidad computacional.

El primero de gran trascendencia teórica es el debido a L. G. Khachiyan en 1979. Este autor, recopilando una serie de trabajos e ideas de los autores rusos Shor, Yudin y Nemirovskii sobre un método basado en la generación de una sucesión de elipsoides para resolver problemas de programación no lineal, los aplicó e hizo extensivos a programación lineal dando lugar al conocido como *método de los elipsoides*. Su principal ventaja teórica sobre el método *simplex* radica en que resuelve problemas de programación lineal en tiempo polinómico; es decir, posee una complejidad computacional teórica polinómica. Las implementaciones prácticas, sin embargo, han demostrado su desventaja respecto al método *simplex* ya que, por un lado, el número de iteraciones que requiere para llegar al óptimo es muy grande y, por otro, el trabajo que implica cada una de ellas es mucho más costoso que el requerido por el método *simplex*, pues no se pueden aplicar las técnicas de matrices dispersas tan características de las implementaciones en ordenador de este último. En el otoño de 1984, N. K. Karmarkar, de los laboratorios Bell de la compañía AT&T, propuso un nuevo algoritmo de complejidad polinómica para resolver problemas de programación lineal. Al contrario que el método de los elipsoides, las implementaciones prácticas a que ha dado lugar en los últimos años lo presentan como un gran rival del método *simplex* en cualquier tipo de problemas y especialmente en los de gran tamaño que surgen de aplicaciones del mundo real.

El método propuesto por Karmarkar es bastante diferente del *simplex*. Considera el pro-

grama lineal dentro de una estructura geométrica de tipo simplicial¹ y una estrategia de movimientos para obtener los distintos puntos del proceso iterativo en el interior del politopo o poliedro que definen las condiciones. Este politopo se transforma de tal manera que el punto del proceso iterativo en el que esté en cada momento el procedimiento sea el centro del politopo o poliedro en el espacio transformado. Este nuevo concepto o enfoque de tratar el problema, es decir, llegar al óptimo a través de puntos en el interior de la región factible, ha motivado numerosas nuevas ideas dentro del apartado genérico de *algoritmos o métodos de puntos interiores*, tanto para programación lineal como no lineal. En este capítulo vamos a estudiar los más extendidos, incluido el de Karmarkar, para resolver problemas prácticos de programación lineal.

10.1 Ideas básicas de los métodos de puntos interiores para programación lineal

Los procedimientos numéricos de puntos interiores para resolver problemas de programación lineal basan su estrategia en conseguir solucionar las tres cuestiones siguientes:

1. Encontrar un punto de partida factible en el interior de la región factible del problema.
2. Definir una dirección de movimiento tal que, conservando la factibilidad del problema, moviéndose a lo largo de ella, se avance hasta un punto en el que se reduzca el valor de la función objetivo (si se está minimizando evidentemente).
3. Cuándo pararse. Es decir, cuántas veces se debe realizar la operación descrita en el punto 2 y cómo identificar que se ha alcanzado el óptimo del problema.

Aun cuando genéricamente estas tres son las grandes cuestiones que debe solventar cualquier procedimiento de optimización, el cómo llevarlas a efecto en el caso de los algoritmos de puntos interiores difiere sustancialmente de lo que hace el método *simplex*.

El método *simplex*, en su forma que podemos denominar ordinaria (fase II), parte desde un punto extremo del politopo o poliedro que conforma la región factible, va a otro vértice, de éste a otro y así sucesivamente hasta que se llega al que define el óptimo del problema.

Si se desea, por el contrario, definir algún criterio o estrategia para, desde un punto interior de la región factible, moviéndose por el interior de esta región, acercarse lo más posible o incluso alcanzar el punto óptimo, ¿qué cabría plantearse?

Un primer punto a considerar sería qué dirección seguir. Si se estuviese minimizando, es evidente que por ejemplo podría utilizarse, como hemos hecho tantas veces en este libro, la dirección de máxima pendiente: en este caso $-\mathbf{c}$ por ser lineal la función objetivo.

Si, de momento, obviamos la cuestión de mantener la factibilidad del problema a lo largo de cualquier dirección de movimiento, otro asunto importante a tener en cuenta es el de lo que se puede mejorar la función objetivo a lo largo de esa dirección, en particular a lo largo de $-\mathbf{c}$, yendo de un punto a otro. Por ejemplo, si consideramos el poliedro de la figura 10.1, en la que se describe cómo poder llegar a su vértice óptimo (de acuerdo con una determinada función objetivo) por el exterior, de vértice a vértice como el método *simplex*, o por el interior, es evidente que no es lo mismo partir desde a que desde b . Desde b se puede progresar mucho

¹En la página 562 se explica lo que es esta estructura.

menos que desde a en un solo paso pues se llega antes a una condición que bloquea el paso (se llegaría antes a algo que denominaremos *pared*). Si en una estrategia de puntos interiores se parte pues desde un punto centrado en la región factible, o desde el que se “vea” centrada la región factible, el progreso hacia la solución en un solo paso o salto puede ser mucho mayor que partiendo de otro cualquiera.

Si consideramos como caso especial de programa lineal

$$\begin{array}{ll} \text{minimizar} & \mathbf{c}^T \mathbf{x} \\ \text{s. a} & \mathbf{x} \geq \mathbf{0}, \end{array}$$

y que se dispone de un punto de partida interior factible desde el que iniciar un procedimiento de búsqueda del óptimo del problema, evidentemente cualquier vector que parta desde ese punto definirá una dirección de movimiento factible. El problema que surge entonces es el de cuánto moverse a lo largo de cualquiera de esas direcciones de tal forma que se reduzca suficientemente la función objetivo y se llegue a un nuevo punto también en el interior de la región factible sin alcanzarse una pared.

Una forma ideal de resolver esto sería inscribir una esfera en $\mathbf{x} \geq \mathbf{0}$, centrándola en el punto considerado, que tuviese el radio lo más grande posible sin violar las condiciones de no negatividad. Si esta esfera fuese la región factible y el centro el punto de partida, optimizar la función objetivo en ella se podría llevar a cabo en un solo paso tomando como dirección de descenso $-\mathbf{c}$ y calculando dónde esa dirección corta a la esfera. Es decir, donde $\mathbf{c}^T \mathbf{x} = \text{cte.}$ sea tangente a esa esfera y su valor mínimo. Este proceso se representa en la figura 10.2. El punto a es el de partida y el nuevo punto del proceso iterativo b .

De la observación de la figura se comprueba que a pesar de que se ha avanzado mucho en dirección al óptimo del problema ($\mathbf{x} = \mathbf{0}$), todavía queda bastante distancia para alcanzarlo. También se puede ver que se ha avanzado más en relación con la posición del óptimo a lo largo

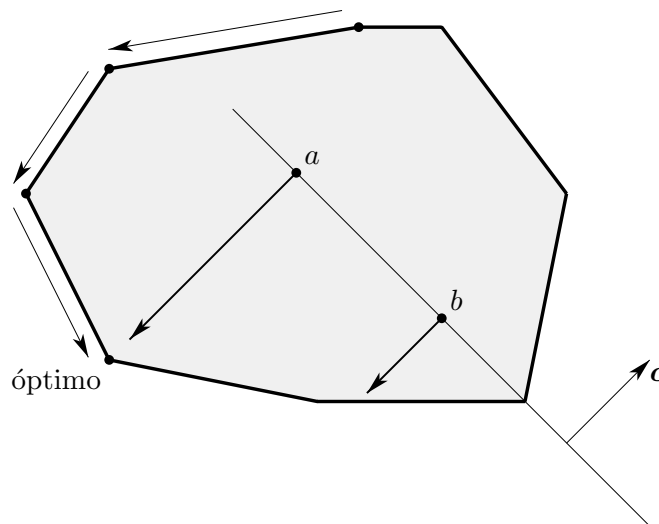
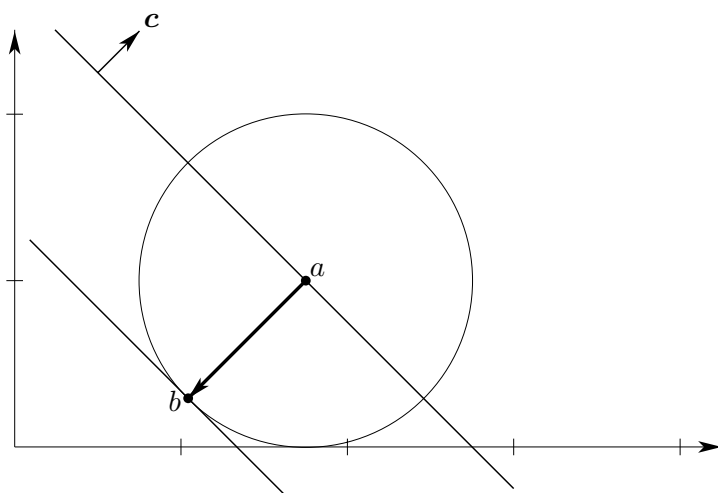


Figura 10.1

Itinerarios hacia el óptimo por el interior y exterior del poliedro que definen las condiciones de un problema

**Figura 10.2**

Máxima esfera (circunferencia en este caso) que se puede centrar en a dentro de la región factible $\mathbf{x} \geq \mathbf{0}$

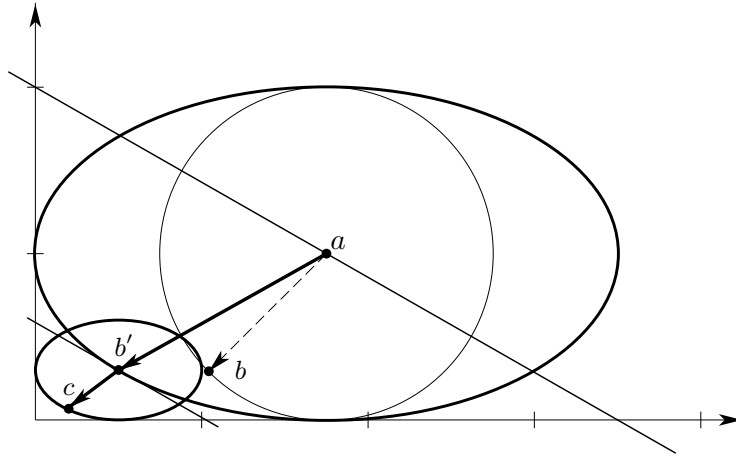
del eje vertical que del horizontal, por lo que si el punto de partida hubiese estado a igual distancia de los dos ejes se habría progresado más. Esto sugiere la posibilidad de escalar el sistema de coordenadas elegido de tal forma que el centro de la esfera que se define en $\mathbf{x}' \geq \mathbf{0}$ esté a igual distancia de los dos ejes con lo que se conseguiría que dicha esfera fuese tangente a las dos paredes que definen la región factible.

Si se escala el problema original de acuerdo con la idea apuntada, como el problema está formulado en un espacio diferente, la solución que se obtuviese en el nuevo espacio al finalizar la iteración correspondiente habría que reescalarla, o transformarla al espacio original. Esta operación se representa en la figura 10.3. En ella se describe el programa lineal en su espacio original, las direcciones de movimiento que se obtienen en el espacio original y en el escalado y los pasos que se dan a lo largo de esas direcciones. Como se puede observar, en el espacio escalado la esfera se ha transformado en una elipse y el nuevo punto del proceso iterativo pasaría de ser b a b' . El resultado es que mediante un simple escalado se ha avanzado sustancialmente más hacia el óptimo del problema. El punto c sería el nuevo punto del proceso una vez realizada una iteración más en la que se inscribiría una nueva elipse en la región factible centrándola en b' .

Esta breve introducción permite destacar dos aspectos o ideas fundamentales que todos los algoritmos de puntos interiores de la región factible deben considerar:

- I. Se debe transformar el problema original en cada punto del proceso iterativo, sin cambiar esencialmente el problema, de tal forma que el punto que se esté considerando en ese momento se sitúe en el centro de la región factible tal como resulte de la transformación.
- II. Si el punto desde el que moverse está centrado en la región factible, ese movimiento es conveniente hacerlo en la dirección que define la de máxima pendiente de la función objetivo.

A continuación pasamos a estudiar el primero de los métodos de punto interior: el publicado por Karmarkar en 1984, y que ha servido de base a otros que veremos posteriormente.

**Figura 10.3**

Máximas elipses que se pueden inscribir en a y en b' en la región factible $\mathbf{x} \geq \mathbf{0}$

10.2 El método del escalado proyectivo de Karmarkar

La forma básica de programa lineal sobre la que Karmarkar aplica su método es la siguiente:

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{0} \\ & \mathbf{e}^T \mathbf{x} = 1, \quad \mathbf{x} \geq \mathbf{0}, \end{aligned} \quad (10.1)$$

donde $\mathbf{e} = [1, 1, \dots, 1]^T$. También se supone que $A\mathbf{e} = \mathbf{0}$, por lo que el punto $\mathbf{e}/n$ es factible, y que el óptimo de (10.1) es cero. Veremos posteriormente cómo se puede transformar un problema en forma estándar en ésta tan especial.

Las condiciones de óptimo del problema (10.1) implican que

$$\mathbf{c} = A^T \mathbf{y} + \mathbf{e}z + \boldsymbol{\eta}, \quad (10.2)$$

donde $\boldsymbol{\eta}$ es el vector de multiplicadores de Lagrange de las condiciones de no negatividad de (10.1). Las condiciones de complementariedad de holguras en el óptimo, $\mathbf{x}^*$, implican también que

$$\mathbf{x}^{*T} \boldsymbol{\eta} = 0.$$

Como $\mathbf{c}^T \mathbf{x}^* = 0$, de (10.2) se tiene que

$$\mathbf{y}^T A\mathbf{x}^* + z\mathbf{e}^T \mathbf{x}^* + \boldsymbol{\eta}^T \mathbf{x}^* = 0$$

lo que implica, siendo $\mathbf{y}^T A\mathbf{x}^* = \boldsymbol{\eta}^T \mathbf{x}^* = 0$, que $z\mathbf{e}^T \mathbf{x}^* = 0$. Como las condiciones del problema hacen que $\mathbf{e}^T \mathbf{x}^* = 1$, quiere decir que $z = 0$.

El programa dual de (10.1) es

$$\begin{aligned} \max. \quad & z \\ \text{s. a} \quad & A^T \mathbf{y} + \mathbf{e}z \leq \mathbf{c}. \end{aligned} \quad (10.3)$$

Este programa dual es factible. En efecto, para cualquier $\bar{\mathbf{y}} \in \Re^m$ se puede elegir fácilmente un z (el mejor es $\min_j (\mathbf{c} - A^T \bar{\mathbf{y}})_j$) tal que $[\bar{\mathbf{y}}^T, z]^T$ es factible. Como según lo visto en el párrafo anterior $z = 0$, de lo que se trata realmente en el programa dual es encontrar un $\bar{\mathbf{y}}$ tal que $A^T \bar{\mathbf{y}} \leq \mathbf{c}$; es decir, puntos factibles.

De la expresión (10.1), el conjunto

$$\Delta = \left\{ \mathbf{x} \in \Re^n : \sum_{i=1}^n x_i = 1, \mathbf{x} \geq \mathbf{0} \right\}$$

define un polígono regular en el espacio euclídeo n -dimensional denominado *simplex*: un poliedro regular de $n + 1$ lados. En $\Re$, $\Delta = \{1\}$; en $\Re^2$, Δ es el segmento de recta que une los puntos $[0, 1]^T$ y $[1, 0]^T$; en $\Re^3$, el área triangular entre los puntos $[0, 0, 1]^T$, $[0, 1, 0]^T$ y $[1, 0, 0]^T$; etc. Es fácil ver que, en $\Re^n$, Δ posee exactamente n vértices, $C(n, 2)$ aristas y $C(n, n - 1)$ caras. Su centro está en $\mathbf{e}/n$. Analizando brevemente la figura 10.4, se puede comprobar que el radio de la hiperesfera más pequeña que circunscribe a Δ está dado por

$$R = \sqrt{\frac{n-1}{n}} \quad (10.4)$$

y el de la más grande inscrita en él

$$r = \frac{1}{\sqrt{n(n-1)}}. \quad (10.5)$$

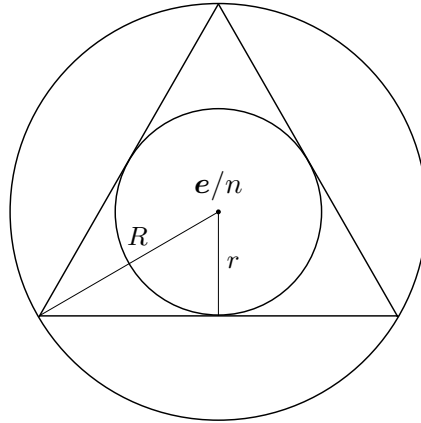


Figura 10.4

Esferas de radio mínimo y máximo que pueden circunscribir un simplex e inscribirse en él

10.2.1 Transformación proyectiva en el simplex

La idea básica del trabajo que se realiza en una iteración del algoritmo de Karmarkar es la siguiente. Supongamos que se dispone de una solución estrictamente factible $\bar{\mathbf{x}} \in \Delta$. Se lleva a cabo entonces una *transformación proyectiva* —más adelante indicaremos cómo construirla—

que convierta al simplex Δ en sí mismo y el punto $\bar{\mathbf{x}}$ en el centro del simplex, $\mathbf{e}/n$. Al permanecer fijos los puntos extremos de Δ se forzará a que una transformación lineal o afín² sea la identidad; el grado de libertad adicional de la transformación proyectiva permitirá deformar el interior de Δ de tal forma que $\bar{\mathbf{x}}$ se transforme en $\mathbf{e}/n$. La ventaja de una transformación de este tipo es que el punto que define la solución en la iteración, al transformarse, en el espacio transformado, estará lejos de las condiciones de desigualdad, $\mathbf{x} \geq \mathbf{0}$: será un punto en el interior de la región factible.³

Hecha la transformación se ignoran las condiciones de no negatividad de las variables y se inicia un movimiento en la dirección contraria a la que define el gradiente de la función objetivo transformada, proyectada sobre las ecuaciones de igualdad transformadas. En esa dirección se avanza un determinado factor de avance, o amplitud de paso, llegándose a un nuevo punto estrictamente factible en el espacio transformado. Se aplica la transformación proyectiva inversa y se consigue así un nuevo punto del proceso iterativo, $\bar{\mathbf{x}}'$.

Supongamos que $\bar{\mathbf{x}}$ está en el interior de Δ , es decir, $\bar{x}_i > 0$, $i = 1, \dots, n$ y $\sum_{i=1}^n \bar{x}_i = 1$. Definamos la matriz diagonal $\bar{X}$ como aquella que tiene como elementos de su diagonal los componentes del vector $\bar{\mathbf{x}}$, de tal forma que se cumpla que $\bar{X}\mathbf{e} = \bar{\mathbf{x}}$, es decir,

$$\bar{X} = \text{diag}(\bar{\mathbf{x}}) = \begin{bmatrix} \bar{x}_1 & 0 & \cdots & 0 \\ 0 & \bar{x}_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \bar{x}_n \end{bmatrix}.$$

Obviamente la matriz $\bar{X}$ es regular siendo su inversa, $\bar{X}^{-1}$, otra matriz diagonal con coeficientes $1/\bar{x}_i$, $i = 1, \dots, n$. La transformación proyectiva indicada se define de la siguiente manera:

$$T_{\bar{\mathbf{x}}}(\mathbf{x}) = \hat{\mathbf{x}} = \frac{\bar{X}^{-1}\mathbf{x}}{\mathbf{e}^T \bar{X}^{-1}\mathbf{x}}; \quad (10.6a)$$

y su inversa

$$T_{\bar{\mathbf{x}}}^{-1}(\mathbf{x}) = \mathbf{x} = \frac{\bar{X}\hat{\mathbf{x}}}{\mathbf{e}^T \bar{X}\hat{\mathbf{x}}}. \quad (10.6b)$$

Obsérvese que $\bar{X}^{-1}\mathbf{x}$ es un vector n -dimensional y $\mathbf{e}^T \bar{X}^{-1}\mathbf{x}$ es un escalar igual a la suma de los elementos de $\bar{X}^{-1}\mathbf{x}$. En consecuencia, los elementos de $T_{\bar{\mathbf{x}}}(\mathbf{x})$ están normalizados con suma igual a 1. Dicho de otra forma, $T_{\bar{\mathbf{x}}}(\mathbf{x}) \in \Delta$ y $T_{\bar{\mathbf{x}}}$ transforma Δ en sí mismo. También es fácilmente observable que, en efecto, $\bar{\mathbf{x}}$ se transforma en $\hat{\mathbf{x}} = \mathbf{e}/n$.

Ejemplo 10.1 Considérese el simplex de la figura 10.5. En él, sean $\mathbf{x} = [1, 0, 0]^T$, $\mathbf{y} = [0, 1, 0]^T$, $\mathbf{z} = [0, 0, 1]^T$, $\mathbf{a} = [3/10, 1/10, 3/5]^T$, $\mathbf{b} = [1/3, 0, 2/3]^T$, $\mathbf{c} = [0, 1/7, 6/7]^T$ y $\mathbf{d} = [3/4, 1/4, 0]^T$.

Como el punto $\mathbf{a}$ está en el interior de la región factible, se puede definir una transformación proyectiva con

$$\bar{X} = \begin{bmatrix} 3/10 & 0 & 0 \\ 0 & 1/10 & 0 \\ 0 & 0 & 3/5 \end{bmatrix}.$$

²Una transformación afín sería la definida por $\mathbf{x} \rightarrow A(\mathbf{x} - \mathbf{d})$, $A \in \mathbb{R}^{n \times n}$, $\mathbf{d} \in \mathbb{R}^n$

³Mientras que el método *simplex* fuerza a que en una determinada iteración parte de esas condiciones de no negatividad estén activas, el de Karmarkar evita las complejidades combinatorias que se derivan de pasar, entre un número exponencial de posibilidades, de un conjunto de esas condiciones activas a otro.

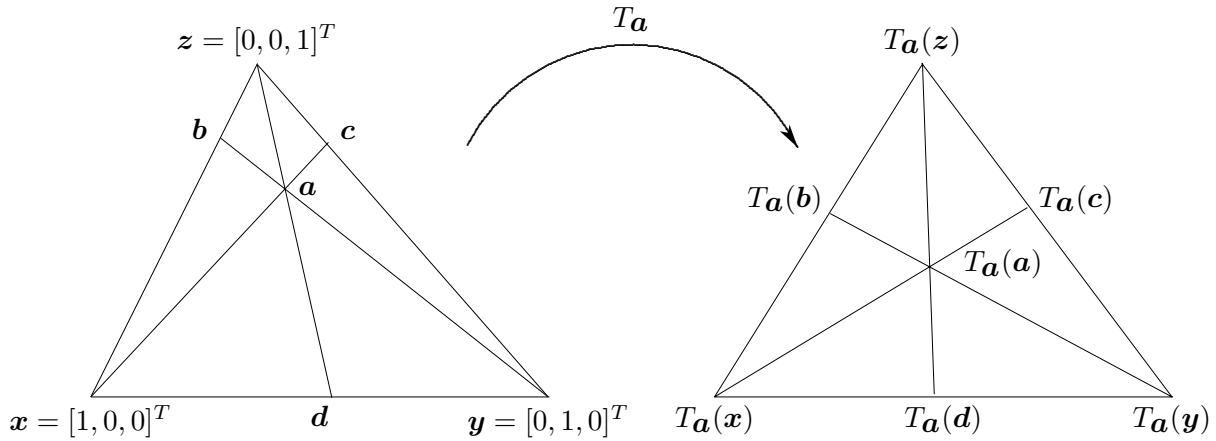


Figura 10.5
Transformación proyectiva del ejemplo 10.1

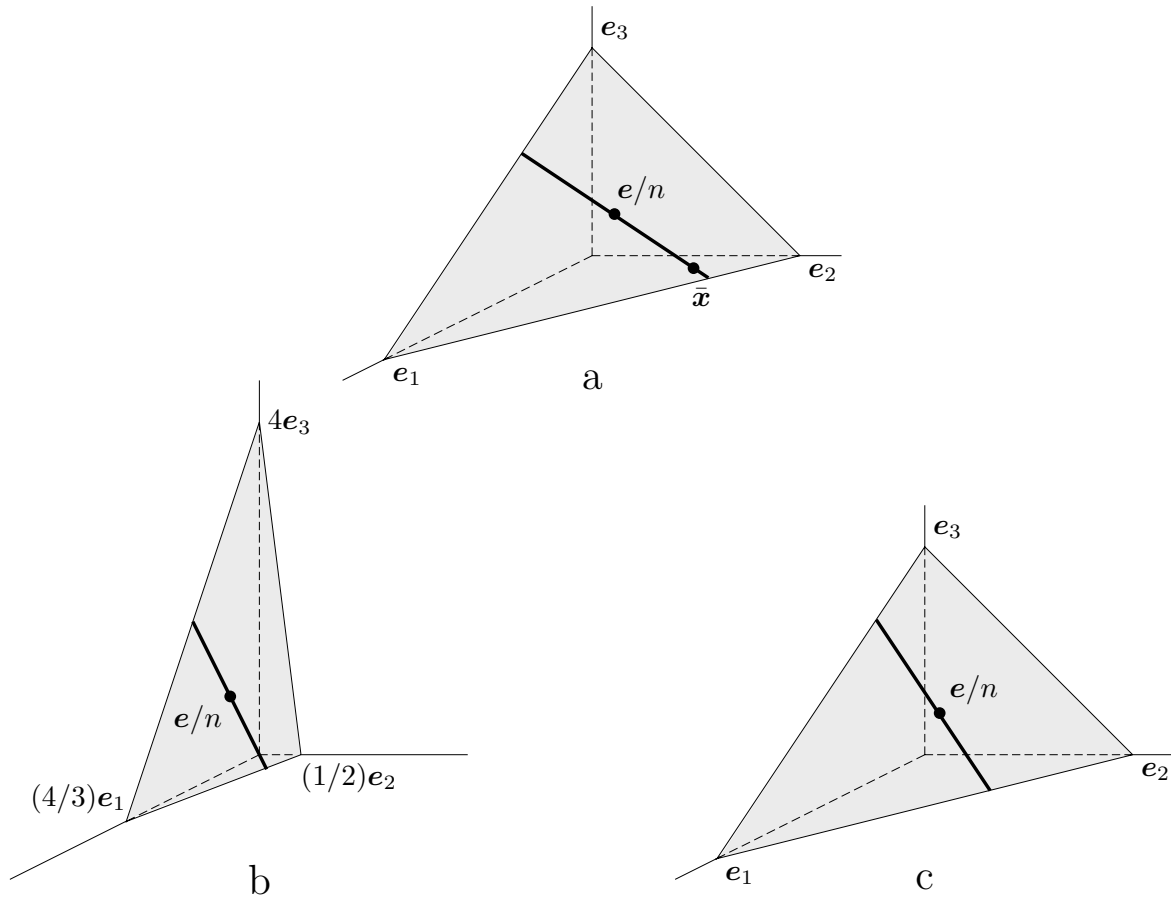
La inversa de esta matriz es, evidentemente,

$$\bar{X}^{-1} = \begin{bmatrix} 10/3 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 5/3 \end{bmatrix}.$$

Aplicando la transformación proyectiva correspondiente, T_a , se tendrá que $T_a(x) = [1, 0, 0]^T$, $T_a(y) = [0, 1, 0]^T$, $T_a(z) = [0, 0, 1]^T$, $T_a(a) = [1/3, 1/3, 1/3]^T$, $T_a(b) = [1/2, 0, 1/2]^T$, $T_a(c) = [0, 1/2, 1/2]^T$ y $T_a(d) = [1/2, 1/2, 0]^T$. El resultado de esta transformación es una distorsión de la escala y los ángulos en el espacio resultante tales que el punto a se convierte en el centro del simplex. ●

Ejemplo 10.2 Para ilustrar un poco más lo que implica una transformación proyectiva, supongamos que $n = 3$, $m = 1$ y $A = [-7, 2, 5]$. La región factible, que se representa en la figura 10.6, es el segmento que une los puntos $[2/9, 7/9, 0]^T$ y $[5/12, 0, 7/12]^T$. El punto $e/n = [1/3, 1/3, 1/3]^T$ es factible. Supongamos que estamos en el punto $\bar{x} = [1/4, 2/3, 1/12]^T$.

Para una mejor comprensión del proceso que lleva a cabo la transformación lo dividiremos en dos partes. La primera es sólo un escalado diagonal que convierte x en $\tilde{x} = (\bar{X}^{-1}x)/n$; es decir, $\tilde{x}$ se transforma sólo en e/n pero el simplex Δ queda distorsionado. En el ejemplo, $\tilde{x}$ se transforma en $[1/3, 1/3, 1/3]^T$ y Δ se convierte en el triángulo de vértices $(4/3)e_1$, $(1/2)e_2$ y $4e_3$, donde e_1 , e_2 y e_3 son los vectores unitarios de $\mathbb{R}^3$. La región factible se transforma en la intersección de este nuevo triángulo con el subespacio núcleo de la matriz $\hat{A} = A\bar{X} = [-7/4, 4/3, 5/12]$: el segmento de recta que une $[8/27, 7/18, 0]^T$ y $[5/9, 0, 7/3]^T$. Por último se transforma el simplex distorsionado en Δ mediante una proyección radial que haga que $\tilde{x}$ se transforme en $\hat{x} = \tilde{x}/e^T \tilde{x}$. El punto e/n y el subespacio núcleo de la matriz $\hat{A}$ ($\{\tilde{x} : \hat{A}\tilde{x} = 0\}$) permanecen invariantes ante esta última transformación aunque e/n se convierte en el centro del nuevo simplex no distorsionado. En nuestro ejemplo, la región factible se convierte en el segmento de recta que une los puntos $[16/37, 21/37, 0]^T$ y $[5/26, 0, 21/26]^T$. ●

**Figura 10.6**

a. Región factible original; b. Después de la primera transformación proyectiva $\bar{x}$ se convierte en e/n ; c. Después de la segunda transformación

Volvamos al caso general. Usando la expresión de x en (10.6b), sustituyéndola en (10.1), se obtiene la siguiente expresión del problema a resolver en el espacio transformado:

$$\begin{aligned} \min. & \frac{c^T \bar{X} \hat{x}}{e^T \bar{X} \hat{x}} \\ \text{s. a} & A \bar{X} \hat{x} = 0 \\ & e^T \hat{x} = 1, \quad \hat{x} \geq 0. \end{aligned}$$

Como uno de los supuestos del problema era que el óptimo es cero, basta en esta última expresión que el numerador de la función objetivo sea cero; es decir, definiendo $\hat{A} = A \bar{X}$ y $\hat{c} = \bar{X} c$, se tiene que el problema es

$$\begin{aligned} \min. & \hat{c}^T \hat{x} \\ \text{s. a} & \hat{A} \hat{x} = 0 \\ & e^T \hat{x} = 1, \quad \hat{x} \geq 0. \end{aligned}$$

Esta última expresión es idéntica a la de (10.1). Además, en el nuevo espacio, la solución actual del problema, $\hat{\mathbf{x}} = \mathbf{e}/n$, está en el centro del simplex $\Delta = \{\hat{\mathbf{x}} \in \mathbb{R}^n : \mathbf{e}^T \hat{\mathbf{x}} = 1, \hat{\mathbf{x}} \geq \mathbf{0}\}$.

Para mejorar esta solución, sea

$$S'(\mathbf{e}/n, \rho) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{e}^T \mathbf{x} = 1, \|\mathbf{x} - \mathbf{e}/n\| \leq \rho\}$$

la bola de centro $\mathbf{e}/n$ y radio ρ en el espacio afín $\{\mathbf{x} \in \mathbb{R}^n : \mathbf{e}^T \mathbf{x} = 1\}$. Es fácil ver que

$$S'(\mathbf{e}/n, r) \subseteq \Delta \subseteq S'(\mathbf{e}/n, R),$$

donde $\Delta = \{\hat{\mathbf{x}} \in \mathbb{R}^n : \mathbf{e}^T \hat{\mathbf{x}} = 1, \hat{\mathbf{x}} \geq \mathbf{0}\}$, $R = ((n-1)/n)^{1/2}$ y $r = (n(n-1))^{-1/2}$, tal como vimos en (10.4) y (10.5). Si se utiliza S' en lugar del simplex $\{\mathbf{e}^T \hat{\mathbf{x}} = 1, \hat{\mathbf{x}} \geq \mathbf{0}\}$, unas condiciones se reemplazan por otras más restrictivas pero de mejor continuidad. El subproblema a resolver entonces en las variables transformadas será:

$$\begin{aligned} \min. \quad & \hat{\mathbf{c}}^T \hat{\mathbf{x}} \\ \text{s. a} \quad & \hat{A} \hat{\mathbf{x}} = \mathbf{0} \\ & \hat{\mathbf{x}} \in S'(\mathbf{e}/n, \rho), \end{aligned} \tag{10.7}$$

para todo $\rho > 0$. Si se designa la matriz de condiciones

$$B = \begin{bmatrix} A\bar{X} \\ \mathbf{e}^T \end{bmatrix},$$

cualquier dirección de movimiento $\hat{\mathbf{d}} \in \mathbb{R}^n$ en el subespacio núcleo de B , es decir, tal que $B\hat{\mathbf{d}} = \mathbf{0}$, será factible.

La solución de (10.7) es

$$\hat{\mathbf{x}}(\rho) = \mathbf{e}/n + \rho \hat{\mathbf{d}} / \|\hat{\mathbf{d}}\|.$$

De las posibles direcciones $\hat{\mathbf{d}}$ que se pueden coger, partiendo de $\mathbf{e}/n$, lo más lógico, según venimos indicando, es tomar la de máxima pendiente de la función objetivo: en este caso $-\mathbf{c}^T \bar{X}$ o $-\bar{X}\mathbf{c}$. Para mantener la factibilidad del problema y que el nuevo punto al que se llegue esté en el interior de la región factible, esta dirección se debe proyectar en el subespacio núcleo de B . Recordando la matriz de proyección en ese subespacio, la dirección será, en suma,

$$\hat{\mathbf{d}} = - \left(I - B^T (BB^T)^{-1} B \right) \bar{X}\mathbf{c}.$$

En la tabla 10.1 se describe el proceso iterativo del algoritmo de Karmarkar para resolver el problema (10.1). El punto del proceso iterativo en la iteración k se designa, para facilitar la notación, por $\mathbf{x}^k$. El resultado que se obtiene es un $\mathbf{x}$ factible tal que $\mathbf{c}^T \mathbf{x} \leq 2^{-L} \mathbf{c}^T \mathbf{e}/n$. El algoritmo hace $\rho = \alpha/n < \alpha r$, donde $0 < \alpha < 1$ se escoge de tal manera que $\hat{\mathbf{x}}(\rho)$ sea factible y se reduzca apreciablemente la función objetivo en cada iteración. Hacer $\alpha = 1/3$ ó $(n-1)/3n$ suele bastar.

El trabajo fundamental de cada iteración del algoritmo es el derivado del cálculo de la dirección $\hat{\mathbf{d}}$.

Tabla 10.1
Algoritmo de Karmarkar

Paso 0 – *Inicialización.* Hacer $\mathbf{x}^0 = \mathbf{e}/n$; $k \leftarrow 0$.

Paso 1 – *Iteración.* Si $\mathbf{c}^T \mathbf{x}^k \leq 2^{-L} \mathbf{c}^T \mathbf{e}/n$, PARAR: se ha llegado al punto deseado $\mathbf{x} = \mathbf{x}^k$. Si no, sea $X_k = \text{diag}(\mathbf{x}^k)$; defínase un vector $\hat{\mathbf{c}} = X_k \mathbf{c}$ y $\hat{A} = AX_k$. Calcular

$$B = \begin{bmatrix} \hat{A} \\ \mathbf{e}^T \end{bmatrix}.$$

y

$$\hat{\mathbf{d}} = -\left(I - B^T (BB^T)^{-1} B\right) \hat{\mathbf{c}}.$$

Determinar $\|\hat{\mathbf{d}}\|$ y $\hat{\mathbf{x}} = \frac{\mathbf{e}}{n} + \frac{\alpha_k \hat{\mathbf{d}}}{n \|\hat{\mathbf{d}}\|}$, con $\alpha_k = 1/3$.

Hacer $\mathbf{x}^{k+1} = \frac{X_k \hat{\mathbf{x}}}{\mathbf{e}^T X_k \hat{\mathbf{x}}}$.

Hacer $k \leftarrow k + 1$ e ir al paso 1.

Ejemplo 10.3 Resolver:

$$\begin{aligned} \min. \quad & x_2 \\ \text{s. a} \quad & x_1 + x_2 - 2x_3 = 0 \\ & x_1 + x_2 + x_3 = 1 \\ & x_1, x_2, x_3 \geq 0. \end{aligned}$$

En este caso, $n = 3$ y $m = 1$. La región factible es la que se indica en la figura 10.7. El punto $\mathbf{x}^0 = [1/3, 1/3, 1/3]^T$ es factible. La solución óptima se puede ver en la figura que es $[2/3, 0, 1/3]$. El valor de la función objetivo en este punto es cero. Todos los requisitos del planteamiento de Karmarkar, pues, se cumplen.

Iteración 1

Partimos del punto $\mathbf{x}^0$ ya indicado. La matriz $X_1 = \text{diag}(1/3, 1/3, 1/3)$. Como $\mathbf{e}^T X_1^{-1} \mathbf{x} = \sum 3x_i = 3$, la transformación proyectiva es $\hat{\mathbf{x}} = \mathbf{x}$. El problema en el espacio $\hat{\mathbf{x}}$ coincide con el de la figura 10.7. El punto $\mathbf{x}^0$ se transformará en él mismo.

Calculemos $\hat{\mathbf{c}} = X_1 \mathbf{c}$; es

$$\hat{\mathbf{c}} = \begin{bmatrix} 0 \\ 1/3 \\ 0 \end{bmatrix}.$$

Calculemos $\hat{A} = AX_1$; es

$$\hat{A} = [1/3, 1/3, -2/3].$$

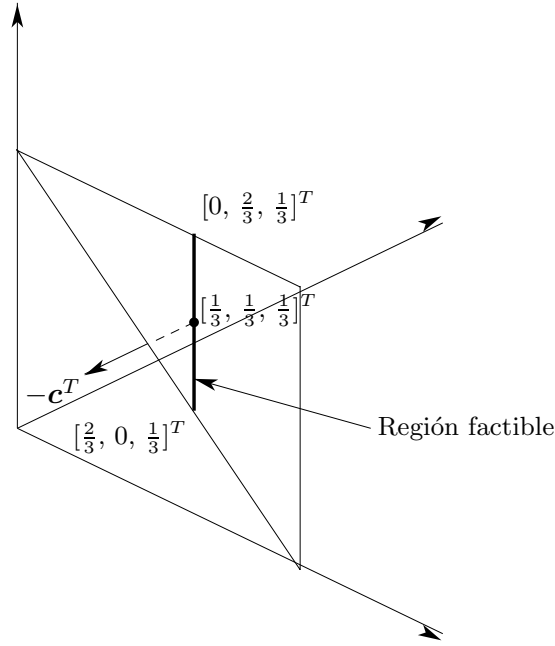


Figura 10.7
Región factible del problema del ejemplo 10.3

Calculemos B :

$$B = \begin{bmatrix} \hat{A} \\ \mathbf{e}^T \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 & -2/3 \\ 1 & 1 & 1 \end{bmatrix}.$$

Obtengamos $\hat{\mathbf{d}}$:

$$\begin{aligned} \hat{\mathbf{d}} &= -\left(I - B^T (BB^T)^{-1} B\right) \hat{\mathbf{c}} \\ &= \begin{bmatrix} 1/6 \\ -1/6 \\ 0 \end{bmatrix}. \end{aligned}$$

Por tanto, $\|\hat{\mathbf{d}}\| = \sqrt{\frac{1}{6^2} + \frac{1}{6^2} + 0} = \frac{\sqrt{2}}{6}$.

Obsérvese que en este caso la dirección $\hat{\mathbf{d}}$ nos lleva desde $[1/3, 1/3, 1/3]$ hacia la solución óptima, $[2/3, 0, 1/3]$.

El nuevo punto en el espacio, $\hat{\mathbf{x}}$, es

$$\begin{aligned} \hat{\mathbf{x}} &= \mathbf{e}/n + \alpha \hat{\mathbf{d}} / (n \|\hat{\mathbf{d}}\|) \\ &= \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} + \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{6}{\sqrt{2}} \begin{bmatrix} 1/6 \\ -1/6 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0,4119 \\ 0,2548 \\ 0,3333 \end{bmatrix}. \end{aligned}$$

Deshaciendo la transformación, el nuevo punto del proceso iterativo es

$$\mathbf{x}^1 = \frac{X_1 \hat{\mathbf{x}}}{\mathbf{e}^T X_1 \hat{\mathbf{x}}} = \begin{bmatrix} 0,4119 \\ 0,2548 \\ 0,3333 \end{bmatrix}.$$

La función objetivo en este nuevo punto es $\mathbf{c}^T \mathbf{x}^1 = 0,2548$.

Iteración 2

La matriz $X_2 = \text{diag}(0,4119, 0,2548, 0,3333)$.

Calculemos $\hat{\mathbf{c}} = X_2 \mathbf{c}$; es

$$\hat{\mathbf{c}} = \begin{bmatrix} 0 \\ 0,2548 \\ 0 \end{bmatrix}.$$

Calculemos $\hat{A} = AX_2$; es

$$\hat{A} = [0,4119, 0,2548, -0,6666].$$

Calculemos B :

$$B = \begin{bmatrix} \hat{A} \\ \mathbf{e}^T \end{bmatrix} = \begin{bmatrix} 0,4119 & 0,2548 & -0,6666 \\ 1 & 1 & 1 \end{bmatrix}.$$

Obtengamos $\hat{\mathbf{d}}$:

$$\begin{aligned} \hat{\mathbf{d}} &= -\left(I - B^T (BB^T)^{-1} B\right) \hat{\mathbf{c}} \\ &= \begin{bmatrix} 0,1243 \\ -0,1455 \\ 0,0212 \end{bmatrix}. \end{aligned}$$

Por tanto, $\|\hat{\mathbf{d}}\| = 0,1925$.

El nuevo punto en el espacio, $\hat{\mathbf{x}}$, es

$$\begin{aligned} \hat{\mathbf{x}} &= \mathbf{e}/n + \alpha \hat{\mathbf{d}} / (n \|\hat{\mathbf{d}}\|) \\ &= \begin{bmatrix} 0,4051 \\ 0,2494 \\ 0,3456 \end{bmatrix}. \end{aligned}$$

Deshaciendo la transformación, el nuevo punto del proceso iterativo es

$$\mathbf{x}^2 = \frac{X_2 \hat{\mathbf{x}}}{\mathbf{e}^T X_2 \hat{\mathbf{x}}} = \begin{bmatrix} 0,4828 \\ 0,1838 \\ 0,3333 \end{bmatrix}.$$

La función objetivo en este nuevo punto es $\mathbf{c}^T \mathbf{x}^2 = 0,1838$.

Iteración 3 y sucesivas

Los nuevos puntos que se obtienen son

$$\mathbf{x}^3 = \begin{bmatrix} 0,5397 \\ 0,1270 \\ 0,3333 \end{bmatrix}, \quad \mathbf{x}^4 = \begin{bmatrix} 0,5815 \\ 0,0852 \\ 0,3333 \end{bmatrix}, \dots, \mathbf{x}^{10} = \begin{bmatrix} 0,6604 \\ 0,0063 \\ 0,3333 \end{bmatrix}, \dots, \mathbf{x}^{20} = \begin{bmatrix} 0,6666 \\ 0,0001 \\ 0,3333 \end{bmatrix}.$$

•

10.2.2 Complejidad computacional

Para resolver

$$\begin{aligned} &\min. \quad \mathbf{c}^T \mathbf{x} \\ &\text{s. a} \quad A\mathbf{x} = \mathbf{0} \\ &\quad \mathbf{e}^T \mathbf{x} = 1, \quad \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

el método de Karmarkar necesita $O(nL)$ iteraciones. Para demostrar la complejidad computacional polinómica del método, la clave estriba en encontrar una amplitud de paso α en cada dirección de movimiento de tal forma que el valor de la función objetivo decrezca geométricamente en cada iteración. En particular, Karmarkar prueba que, para $\alpha = 1/3$,

$$\mathbf{c}^T \mathbf{x}^k \leq e^{-k/5n} (\mathbf{c}^T \mathbf{x}^0), \quad k = 1, 2, \dots \quad (10.8)$$

De acuerdo con esto, para un L suficientemente grande tal que $2^{-L}(\mathbf{c}^T \mathbf{x}^0) \approx 0$, sólo es necesario que k cumpla que

$$e^{-k/5n} \leq 2^{-L} < \frac{\varepsilon}{\mathbf{c}^T \mathbf{x}^0}, \quad (10.9)$$

pudiendo entonces terminar el algoritmo cuando se consiga la precisión que se desee. Si en (10.9) se toman logaritmos,

$$\frac{-k}{5n} \leq \ln 2^{-L} < \log_2 2^{-L} = -L.$$

Es decir, si $k > 5nL$, el algoritmo puede terminar cumpliéndose que $\mathbf{c}^T \mathbf{x}^k < \varepsilon$. El algoritmo de Karmarkar necesita, en suma, de un número de iteraciones polinómico: $O(nL)$.

Obsérvese que (10.8) es equivalente a

$$\ln (\mathbf{c}^T \mathbf{x}^k) \leq \frac{-k}{5n} + \ln (\mathbf{c}^T \mathbf{x}^0),$$

o a

$$n \ln (\mathbf{c}^T \mathbf{x}^k) \leq n \ln (\mathbf{c}^T \mathbf{x}^0) - \frac{k}{5},$$

lo que demuestra que (10.8) se cumple si el valor de la función $n \ln(\mathbf{c}^T \mathbf{x})$ se puede reducir en cada iteración en al menos la cantidad constante $1/5$.

La dirección de movimiento del método de Karmarkar es la proyección del negativo del gradiente de la función objetivo. Esta dirección se toma para reducir el valor de $\mathbf{c}^T X_k \hat{\mathbf{x}}$, el

cual es significativamente distinto del de la función $n \ln(\mathbf{c}^T \mathbf{x})$. Para ligar estos dos objetivos, Karmarkar define una *función potencial*, para cada punto $\mathbf{x}$ del interior de Δ y vector de coste $\mathbf{c}$, de la siguiente manera:

$$f(\mathbf{x}, \mathbf{c}) = n \ln(\mathbf{c}^T \mathbf{x}) - \sum_{j=1}^n \ln x_j = \sum_{j=1}^n \ln \left(\frac{\mathbf{c}^T \mathbf{x}}{x_j} \right). \quad (10.10)$$

10.3 Variantes y extensiones del método de Karmarkar

Desde su aparición en 1984, el método de Karmarkar ha suscitado una gran atención por parte de estudiosos e investigadores. De entre todas las variantes o alternativas que se han publicado, las conocidas como de *escalado afín* son las de uso más extendido, y desde luego las más utilizadas en implementaciones comerciales. Esta nueva forma de resolver un programa lineal utiliza unas transformaciones mucho más sencillas que la proyectivas de Karmarkar, las transformaciones afines. Su ventaja fundamental estriba en que es posible tratar el problema de programación lineal en su forma estándar y no en la forma de Karmarkar, mucho más artificiosa.

El algoritmo basado en transformaciones afines fue introducido por primera vez en 1967 por el matemático soviético I. I. Dikin. Con posterioridad, en 1985, E. Barnes, R. Vanderbei, M. Meketon y B. Freedman redescubrieron su utilidad. Estos autores propusieron resolver un problema de programación lineal en forma estándar utilizando el algoritmo de escalado afín primal y probaron su convergencia. En 1989, I. Adler, N. Karmarkar, M. G. C. Resende y G. Veiga diseñaron otro algoritmo similar, el denominado de escalado afín dual, para resolver problemas de programación lineal con restricciones de desigualdad (duales).

Los algoritmos de puntos interiores basados en transformaciones afines son hoy en día objeto de múltiples experimentaciones. Muchos códigos comerciales que resuelven de forma eficaz problemas de programación lineal disponen de variantes que permiten comparar los tiempos de resolución que necesita el clásico *simplex* y las diferentes implementaciones de los métodos de escalado afín. Y ello a pesar de que N. Megido y M. Shub han demostrado que la trayectoria que siguen estos métodos hasta el óptimo depende en gran medida del punto de partida y que, por consiguiente, se puede perder la complejidad polinómica. No obstante, una variante de los métodos de escalado afín, la primal-dual, consigue una complejidad polinómica utilizando funciones de barrera logarítmicas. La idea es tratar de evitar mediante estas funciones que a lo largo de las direcciones de movimiento los puntos que se obtengan se acerquen a las paredes de la región factible.

10.4 El método primal de escalado afín

Uno de los grandes inconvenientes del algoritmo de Karmarkar estriba en la obtención de las transformaciones proyectivas. Una forma de evitarlo es usar transformaciones afines en lugar de proyectivas dando lugar así a los denominados algoritmos de *escalado afín*. Como ya hemos mencionado, el inconveniente que presentan desde el punto de vista teórico es que no se ha podido probar que, utilizándolas, el tiempo de resolución que necesitan para resolver problemas de programación lineal sea polinómico.

En el método de Karmarkar, el simplex

$$\Delta = \{\mathbf{x} \in \mathbb{R}^n : x_1 + x_2 + \cdots + x_n = 1, x_i \geq 0, i = 1, \dots, n\}$$

y su centro, $\mathbf{e}/n = [1/n, \dots, 1/n]^T$, permitían que, partiendo de ese centro, gracias a una transformación proyectiva, utilizando como dirección de movimiento la de máxima pendiente de la función objetivo, se pudiese progresar adecuadamente hacia el mínimo del problema. Si se trabaja con el problema en forma estándar,

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned} \tag{10.11}$$

no se dispone de la estructura que define el *simplex* pudiendo incluso darse que la región factible no sea acotada. La única estructura disponible en este caso es la que define la intersección del espacio afín $\{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}\}$, formado por las condiciones del problema, y el “octante” positivo $\{\mathbf{x} \in \mathbb{R}^n : \mathbf{x} \geq \mathbf{0}\}$.

El octante positivo, evidentemente, no dispone de punto central. El punto $\mathbf{e} = [1, \dots, 1]^T$, sin embargo, está a igual distancia de cada una de sus paredes. En tanto y cuanto, evidentemente, a lo largo de cualquier dirección, desde $\mathbf{e}$, se avance un paso menor de la unidad, el punto resultante estará dentro del octante no negativo. Si se puede definir una transformación que convierta un punto interior de la región factible en $\mathbf{e}$, de forma similar a como hace Karmarkar en su método, se puede definir una estrategia de avance hacia el óptimo del problema de la siguiente manera:

1. Partiendo de un punto interior de la región factible, definir y aplicar una transformación de tal forma que ese punto se transforme en $\mathbf{e}$.
2. Obtener la dirección de máxima pendiente de la función objetivo proyectada en el subespacio núcleo de los vectores columna que define la matriz de condiciones transformada, y moverse a lo largo de ella un paso tal que no se violen las condiciones de no negatividad y que se llegue a otro punto interior de la región factible.
3. Transformar el nuevo punto al espacio original y repetir este proceso hasta acercarse lo suficientemente al punto óptimo del problema.

10.4.1 Transformación afín del octante positivo

La transformación a la que se refería la estrategia anterior es la que se denomina *transformación afín* o *transformación afín primal*.

Sea $\mathbf{x}^k \in \mathbb{R}^n$ una solución estrictamente factible ($\mathbf{x}^k > \mathbf{0}$) del problema de programación lineal en forma estándar (10.11), y considérese la matriz diagonal

$$X_k = \text{diag}(\mathbf{x}^k) = \begin{bmatrix} x_1^k & 0 & \cdots & 0 \\ 0 & x_2^k & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & x_n^k \end{bmatrix}.$$

La matriz X_k es obviamente regular, siendo su inversa, X_k^{-1} , otra matriz diagonal con coeficientes $1/x_i^k$. Una transformación afín, del octante no negativo en sí mismo, se define de la siguiente manera:

$$T_k(\mathbf{x}) = \mathbf{y} = X_k^{-1}\mathbf{x}.$$

Obsérvese que esta transformación convierte $\mathbf{x}$ en el punto interior $\mathbf{e}$ equidistante de todas las condiciones de no negatividad del problema. La figura 10.8 describe geoméricamente esta transformación afín en dos dimensiones. Tal como se ha definido, la transformación convierte

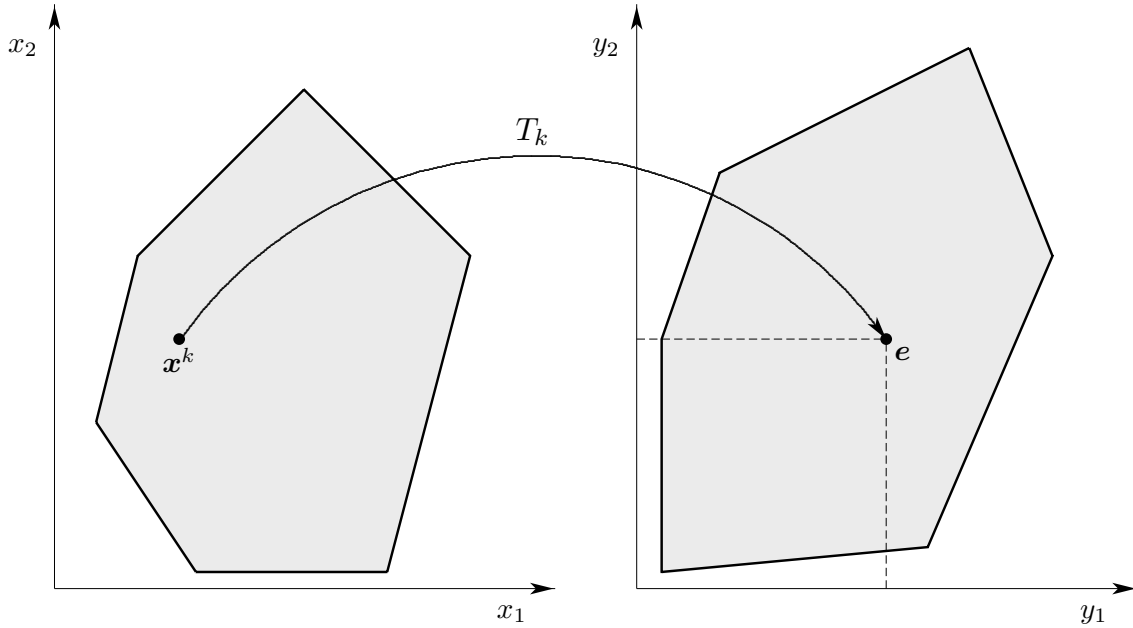


Figura 10.8

Descripción geométrica de la transformación afín en dos dimensiones

un punto extremo de la región factible original en otro punto extremo de la región factible en el espacio transformado, y un punto en una arista de la región factible original en otro también en una arista en el espacio transformado.

El problema de programación lineal en el nuevo espacio es

$$\begin{aligned} \min. \quad & (\mathbf{c}^k)^T \mathbf{y} \\ \text{s. a} \quad & A_k \mathbf{y} = \mathbf{b} \\ & \mathbf{y} \geq \mathbf{0}, \end{aligned} \tag{10.12}$$

donde $\mathbf{c}^k = X_k \mathbf{c}$ y $A_k = A X_k$. Si partiendo de $\mathbf{y}^k$ nos movemos en una dirección $\mathbf{d}_y^k$ en el subespacio núcleo de la matriz A_k , y a lo largo de ella se avanza un paso α_k , el nuevo punto obtenido, $\mathbf{y}^{k+1} = \mathbf{e} + \alpha_k \mathbf{d}_y^k$ permanecerá en el interior de la región factible de (10.12). En el espacio original, $\mathbf{x}^{k+1} = T_k^{-1}(\mathbf{y}^{k+1}) = X_k \mathbf{y}^{k+1}$ será un nuevo punto interior de la región factible del problema en forma estándar (10.11).

Como de lo que se trata es de minimizar el valor de la función objetivo, siguiendo con la estrategia apuntada, la idea es escoger como dirección que haga decrecer ese valor la proyección de la dirección de máxima pendiente de la función objetivo en el subespacio núcleo de A_k . Si se define como matriz de proyección en el subespacio $\ker(A_k)$

$$P_k = I - A_k^T (A_k A_k^T)^{-1} A_k = I - X_k A^T (A X_k^2 A^T)^{-1} A X_k,$$

la dirección de movimiento $\mathbf{d}_y^k$ será:

$$\mathbf{d}_y^k = P_k (-\mathbf{c}^k) = - \left(I - X_k A^T (A X_k^2 A^T)^{-1} A X_k \right) X_k \mathbf{c}.$$

Obsérvese que la matriz de proyección P_k estará definida en tanto y cuanto A sea de rango completo y $\mathbf{x}^k > \mathbf{0}$. Es también fácilmente comprobable que $A X_k \mathbf{d}_y^k = \mathbf{0}$. En la figura 10.9 se representa la obtención de $\mathbf{d}_y^k$.

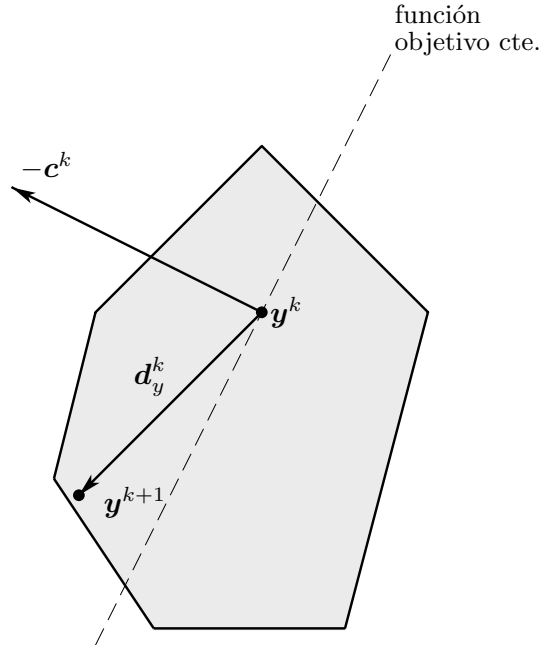


Figura 10.9

Obtención de la dirección en el subespacio núcleo de A_k

Determinada la dirección de movimiento a lo largo de la cual desplazarse en el espacio transformado, la siguiente cuestión a resolver es cuánto moverse en esa dirección. Es decir, escoger un $\alpha_k > 0$ tal que

$$\mathbf{y}^{k+1} = \mathbf{y}^k + \alpha_k \mathbf{d}_y^k > \mathbf{0}.$$

Obsérvese que si $\mathbf{d}_y^k \geq \mathbf{0}$, α_k puede ser cualquier valor real positivo sin que el nuevo punto se salga de la región factible. Si, por el contrario, $d_{y_i}^k < 0$, para algún i , el valor de α_k habrá de ser menor que

$$\frac{y_i^k}{-d_{y_i}^k} = \frac{1}{-d_{y_i}^k}.$$

De acuerdo con esto, en cada paso k del proceso, tomando $0 < \alpha < 1$, se escogerá una amplitud de paso en la dirección $\mathbf{d}_y^k$ dada por

$$\alpha_k = \min_i \left\{ \frac{\alpha}{-d_{y_i}^k} : d_{y_i}^k < 0 \right\}.$$

Cuanto más próxima a 1 sea α_k , más cerca estará el nuevo punto $\mathbf{y}^{k+1}$ de una pared definida por una condición de no negatividad.

Una vez obtenido el nuevo punto $\mathbf{y}^{k+1}$ en el espacio transformado, la siguiente operación que se debe realizar es convertir ese punto en su correspondiente, $\mathbf{x}^{k+1}$, en el espacio original. Esto se realiza mediante la transformación inversa, T_k^{-1} . Es decir,

$$\begin{aligned} \mathbf{x}^{k+1} &= T_k^{-1}(\mathbf{y}^{k+1}) = X_k \mathbf{y}^{k+1} \\ &= \mathbf{x}^k + \alpha_k X_k \mathbf{d}_y^k \\ &= \mathbf{x}^k - \alpha_k X_k P_k X_k \mathbf{c} \\ &= \mathbf{x}^k - \alpha_k X_k \left(I - X_k A^T (A X_k^2 A^T)^{-1} A X_k \right) X_k \mathbf{c} \\ &= \mathbf{x}^k - \alpha_k X_k^2 \left(\mathbf{c} - A^T (A X_k^2 A^T)^{-1} A X_k^2 \mathbf{c} \right) \\ &= \mathbf{x}^k - \alpha_k X_k^2 \left(\mathbf{c} - A^T \mathbf{w}^k \right), \end{aligned}$$

donde

$$\mathbf{w}^k = (A X_k^2 A^T)^{-1} A X_k^2 \mathbf{c}. \quad (10.13)$$

Lo que quiere decir que la dirección de movimiento en el espacio original es $\mathbf{d}_x^k = -X_k^2(\mathbf{c} - A^T \mathbf{w}^k)$ y la amplitud del paso a dar α_k . En el espacio transformado la dirección de movimiento es $\mathbf{d}_y^k = -X_k(\mathbf{c} - A^T \mathbf{w}^k)$.

Como, de acuerdo con esto, $\mathbf{d}_y^k = -P_k \mathbf{c}^k$, $\mathbf{d}_x^k = X_k \mathbf{d}_y^k$ y P_k es la matriz de proyección sobre el subespacio al que pertenece $\mathbf{d}_y^k$,

$$\begin{aligned} \mathbf{c}^T \mathbf{x}^{k+1} &= \mathbf{c}^T (\mathbf{x}^k + \alpha_k \mathbf{d}_x^k) \\ &= \mathbf{c}^T \mathbf{x}^k + \alpha_k \mathbf{c}^T X_k \mathbf{d}_y^k \\ &= \mathbf{c}^T \mathbf{x}^k + \alpha_k (\mathbf{c}^k)^T \mathbf{d}_y^k \\ &= \mathbf{c}^T \mathbf{x}^k - \alpha_k (\mathbf{d}_y^k)^T P_k \mathbf{d}_y^k \\ &= \mathbf{c}^T \mathbf{x}^k - \alpha_k \|\mathbf{d}_y^k\|^2. \end{aligned} \quad (10.14)$$

Lo que implica que el punto $\mathbf{x}^{k+1}$ mejora el valor de la función objetivo a lo largo de la dirección calculada si $\mathbf{d}_y^k \neq \mathbf{0}$.

Lema 10.1 Si existe un $\mathbf{x}^k \in \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} > \mathbf{0}\}$ y $\mathbf{d}_y^k > \mathbf{0}$, el problema de programación lineal

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

no está acotado.

DEMOSTRACIÓN. Como $\mathbf{d}_y^k$ pertenece al subespacio núcleo de $A_k = AX_k$ y $\mathbf{d}_y^k > \mathbf{0}$, el punto $\mathbf{y}^{k+1} = \mathbf{y} + \alpha_k \mathbf{d}_y^k$ será factible en el problema

$$\begin{aligned} \min. \quad & (\mathbf{c}^k)^T \mathbf{y} \\ \text{s. a} \quad & A_k \mathbf{y} = \mathbf{b} \\ & \mathbf{y} \geq \mathbf{0}, \end{aligned}$$

para cualquier $\alpha_k > 0$. En consecuencia, α_k se puede hacer tan grande como se quiera resultando de (10.14) que $\mathbf{c}^T \mathbf{x}^{k+1}$ tenderá a $-\infty$ para $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k X_k \mathbf{d}_y^k \in \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$. ■

Lema 10.2 Si existe un $\mathbf{x}^k \in \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} > \mathbf{0}\}$ y $\mathbf{d}_y^k = \mathbf{0}$, cualquier solución factible del problema

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

es óptima.

DEMOSTRACIÓN. Como $\mathbf{d}_y^k = -P_k X_k \mathbf{c} = \mathbf{0}$, $X_k \mathbf{c}$ pertenecerá al subespacio complemento ortogonal del subespacio núcleo de AX_k . Como el complemento ortogonal de $\ker(AX_k)$ es el subespacio que determinan los vectores fila de AX_k , $\text{Im}((AX_k)^T)$, existirá un vector $\mathbf{u}^k$ tal que

$$(AX_k)^T \mathbf{u}^k = X_k \mathbf{c} \quad \text{o} \quad (\mathbf{u}^k)^T AX_k = \mathbf{c}^T X_k.$$

Como X_k^{-1} existe, $(\mathbf{u}^k)^T A = \mathbf{c}^T$. Para cualquier punto factible $\mathbf{x}$,

$$\mathbf{c}^T \mathbf{x} = (\mathbf{u}^k)^T A \mathbf{x} = (\mathbf{u}^k)^T \mathbf{b}.$$

Como $(\mathbf{u}^k)^T \mathbf{b}$ no depende de $\mathbf{x}$, el valor de $\mathbf{c}^T \mathbf{x}$ es constante en $\{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$. ■

Lema 10.3 Si el problema de programación lineal

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \end{aligned}$$

está acotado inferiormente y el valor de su función objetivo no es constante, la sucesión $\mathbf{c}^T \mathbf{x}^k$, $k = 1, 2, \dots$ está definida y es estrictamente decreciente.

DEMOSTRACIÓN. Es consecuencia directa de los dos lemas anteriores y de la ecuación (10.14). ■

Si $\mathbf{x}^k$ es un punto extremo de la región factible, el vector $\mathbf{w}^k$ definido en (10.13) es igual a $(B^T)^{-1}\mathbf{c}_B$, es decir, al vector de multiplicadores simplex que definíamos en el método *simplex*. Aquí le denominaremos *estimador dual*. Si, de nuevo, $\mathbf{x}^k$ es un punto extremo,

$$\mathbf{r}^k = \mathbf{c} - A^T \mathbf{w}^k$$

es igual a $\mathbf{c} - A^T(B^T)^{-1}\mathbf{c}_B$, el vector de costes reducidos del método *simplex*, al que denominaremos igual aquí.

Obsérvese que si $\mathbf{r}^k \geq \mathbf{0}$, el estimador dual $\mathbf{w}^k$ es una solución factible del problema dual y $(\mathbf{x}^k)^T \mathbf{r}^k = \mathbf{e}^T X_k \mathbf{r}^k$ se convierte en la diferencia entre el valor de la función objetivo del programa primal y la del dual⁴, es decir,

$$\mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k = \mathbf{e}^T X_k \mathbf{r}^k.$$

En el caso de que $\mathbf{e}^T X_k \mathbf{r}^k = 0$, con $\mathbf{r}^k \geq \mathbf{0}$, se habrá alcanzado un punto factible en el programa primal, $\mathbf{x}^k$, la factibilidad del dual con $\mathbf{w}^k$ y se cumplirán las condiciones de complementariedad de holguras. Es decir, $\mathbf{x}^k$ es el óptimo del programa primal y $\mathbf{w}^k$ del dual.

En la tabla 10.2 se describe, como resumen de todo lo expuesto en este apartado, el algoritmo primal de escalado afín.

Ejemplo 10.4 Resolver:

$$\begin{aligned} \text{minimizar} \quad & -2x_1 + x_2 \\ \text{sujeta a} \quad & x_1 - x_2 + x_3 = 15 \\ & x_2 + x_4 = 15 \\ & x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

En este problema,

$$A = \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 15 \\ 15 \end{bmatrix} \quad \text{y} \quad \mathbf{c} = [-2, 1, 0, 0]^T.$$

Comencemos el proceso iterativo partiendo del punto $\mathbf{x}^0 = [10, 2, 7, 13]^T$, el cual como se comprueba fácilmente, está en el interior de la región factible del problema.

⁴Duality gap.

Tabla 10.2

Algoritmo primal de escalado afín

Paso 0 – *Inicialización.* Hacer $k = 0$ y definir un $\mathbf{x}^0 > \mathbf{0}$ tal que $A\mathbf{x}^0 = \mathbf{b}$.**Paso 1** – *Cálculo de los estimadores duales.* Calcular el vector de estimadores duales

$$\mathbf{w}^k = (AX_k^2 A^T)^{-1} AX_k^2 \mathbf{c},$$

donde X_k es la matriz diagonal cuyos coeficientes son los elementos del vector $\mathbf{x}^k$.**Paso 2** – *Cálculo de los costes reducidos.* Calcular el vector de costes reducidos

$$\mathbf{r}^k = \mathbf{c} - A^T \mathbf{w}^k.$$

Paso 3 – *Comprobar si el punto es óptimo.* Si $\mathbf{r}^k \geq \mathbf{0}$ y $\mathbf{e}^T X_k \mathbf{r}^k \leq \varepsilon$ (una tolerancia dada), PARAR. El problema está resuelto: $\mathbf{x}^k$ es la solución del programa primal y $\mathbf{w}^k$ del dual. Si no, ir al siguiente paso.**Paso 4** – *Calcular la dirección de movimiento.* Calcular

$$\mathbf{d}_y^k = -X_k \mathbf{r}^k.$$

Paso 5 – *Comprobar si existe solución no acotada.* Si $\mathbf{d}_y^k > \mathbf{0}$, PARAR. El problema es no acotado. Si $\mathbf{d}_y^k = \mathbf{0}$, PARAR. El punto $\mathbf{x}^k$ es el óptimo del problema. Si no, ir al paso 6.**Paso 6** – *Cálculo de la amplitud de paso.* Calcular la amplitud del paso a dar en la dirección $\mathbf{d}_y^k$:

$$\alpha_k = \min_i \left\{ \frac{\alpha}{-d_{y_i}^k} : d_{y_i}^k < 0 \right\} \quad \text{con } 0 < \alpha < 1.$$

Paso 7 – *Obtener nuevo punto.* Realizar la traslación al nuevo punto:

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k X_k \mathbf{d}_y^k.$$

Hacer $k \leftarrow k + 1$ e ir al paso 1.

Iteración 1

La matriz que define la primera transformación es

$$X_0 = \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 7 & 0 \\ 0 & 0 & 0 & 13 \end{bmatrix}.$$

Calculemos el vector de estimadores duales:

$$\begin{aligned} \mathbf{w}^0 &= (AX_0^2A^T)^{-1} AX_0^2\mathbf{c} = \left(\begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 7 & 0 \\ 0 & 0 & 0 & 13 \end{bmatrix}^2 \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \right)^{-1} \\ &\quad \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 7 & 0 \\ 0 & 0 & 0 & 13 \end{bmatrix}^2 \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -1,33353 \\ -0,00771 \end{bmatrix}. \end{aligned}$$

Los costes reducidos son:

$$\mathbf{r}^0 = \mathbf{c} - A^T\mathbf{w}^0 = \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -1,33353 \\ -0,00771 \end{bmatrix} = \begin{bmatrix} -0,66647 \\ -0,32582 \\ 1,33353 \\ 0,00771 \end{bmatrix}.$$

Como alguno de los costes reducidos es negativo y $\mathbf{e}^T X_0 \mathbf{r}^0 = 2,1186$, todavía no se ha llegado al óptimo del problema.

Calculemos la dirección de movimiento:

$$\mathbf{d}_y^0 = -X_0 \mathbf{r}^0 = - \begin{bmatrix} 10 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 7 & 0 \\ 0 & 0 & 0 & 13 \end{bmatrix} \begin{bmatrix} -0,66647 \\ -0,32582 \\ 1,33353 \\ 0,00771 \end{bmatrix} = \begin{bmatrix} 6,6647 \\ 0,6516 \\ -9,3347 \\ -0,1002 \end{bmatrix}.$$

Si se escoge un $\alpha = 0,99$, la amplitud de paso será

$$\alpha_0 = \frac{0,99}{9,3347} = 0,1061.$$

La nueva solución será

$$\mathbf{x}^1 = \mathbf{x}^0 + \alpha_0 X_0 \mathbf{d}_y^0 = \begin{bmatrix} 17,06822 \\ 2,13822 \\ 0,07000 \\ 12,86178 \end{bmatrix}.$$

El valor de la función objetivo en este nuevo punto es

$$\mathbf{c}^T \mathbf{x} = [-2, 1, 0, 0] \begin{bmatrix} 17,06822 \\ 2,13822 \\ 0,07000 \\ 12,86178 \end{bmatrix} = -31,99822.$$

Obsérvese que en sólo una iteración la función objetivo ha pasado de ser -18 a $-31,99822$.

Iteración 2 y sucesivas

Continuando con el proceso de la misma forma que hemos hecho en la iteración precedente, se conseguirían los siguientes puntos:

$$\mathbf{x}^2 = \begin{bmatrix} 29,82964 \\ 14,87138 \\ 0,04174 \\ 0,12862 \end{bmatrix}, \mathbf{x}^3 = \begin{bmatrix} 29,98380 \\ 14,99871 \\ 0,01491 \\ 0,00129 \end{bmatrix}, \mathbf{x}^4 = \begin{bmatrix} 29,99862 \\ 14,99877 \\ 0,00015 \\ 0,00123 \end{bmatrix}, \mathbf{x}^5 = \begin{bmatrix} 29,99987 \\ 14,99999 \\ 0,00011 \\ 0,00001 \end{bmatrix}, \mathbf{x}^6 = \begin{bmatrix} 30,00000 \\ 15,00000 \\ 0,00000 \\ 0,00000 \end{bmatrix}.$$

El valor de la función objetivo en el óptimo es -45 . ●

10.4.2 Solución de partida del método

Hasta ahora hemos supuesto que partíamos de una solución inicial en el interior de la región factible del problema. A continuación veremos dos formas, similares a las que se expusieron para resolver el mismo problema en el caso del método *simplex*, de llegar a ella.

10.4.2.1 El método de la gran M

Consiste en añadir una variable artificial x^a , a la que se asocia un coste en la función objetivo muy grande, M , a fin de que $[1, 1, \dots, 1]^T \in \mathbb{R}^{n+1}$ sea una solución en el interior de la región factible del problema

$$\begin{aligned} &\text{minimizar } \mathbf{c}^T \mathbf{x} + Mx^a \\ &\text{sujeta a } \quad [A|\mathbf{b} - A\mathbf{e}] \begin{bmatrix} \mathbf{x} \\ x^a \end{bmatrix} = \mathbf{b} \\ &\quad \mathbf{x} \geq \mathbf{0}, \quad x^a \geq 0, \end{aligned}$$

donde $\mathbf{e} = [1, 1, \dots, 1]^T \in \mathbb{R}^n$. Comparada con la estrategia utilizada en el caso del método *simplex*, ésta sólo utiliza $n + 1$ variables en vez de las $n + m$ del *simplex*. Utilizando esta forma de actuar, con un M suficientemente grande, el procedimiento llegará a una solución óptima del problema original o se comprobará que el mismo es no acotado. Si al final del proceso la variable x^a es positiva, el problema no es factible.

10.4.2.2 El método en dos fases

Si se escoge un punto arbitrario $\mathbf{x}^0 > \mathbf{0}$, se calcula $\mathbf{v} = \mathbf{b} - A\mathbf{x}^0$ y resulta que $\mathbf{v} = \mathbf{0}$, $\mathbf{x}^0$ es un punto interior de la región factible pudiéndose por tanto utilizar para comenzar el método primal de escalado afín. Si no es así, se define como se hacía en el método *simplex* una primera fase para obtener ese punto interior desde el que comenzar todo el procedimiento. Esta *fase I* tratará de resolver el problema

$$\begin{aligned} &\text{minimizar } u \\ &\text{sujeta a } \quad [A|\mathbf{v}] \begin{bmatrix} \mathbf{x} \\ u \end{bmatrix} = \mathbf{b} \\ &\quad \mathbf{x} \geq \mathbf{0}, \quad u \geq 0. \end{aligned} \tag{10.15}$$

Es fácilmente comprobable que

$$\hat{\mathbf{x}}^0 = \begin{bmatrix} \mathbf{x}^0 \\ u^0 \end{bmatrix} = \begin{bmatrix} \mathbf{x}^0 \\ 1 \end{bmatrix} > \mathbf{0},$$

y que por lo tanto está en el interior de la región factible de (10.15). A este problema se le puede aplicar el algoritmo primal de escalado afín, el cual, como la solución está acotada inferiormente por 0, siempre llegará a una solución óptima, $[\mathbf{x}^*, u^*]^T$. Si en esa solución $u^* > 0$, el problema original no es factible. En cualquier otro caso el punto $\mathbf{x}^*$, siempre y cuando $\mathbf{x}^* > \mathbf{0}$ (cosa que ocurre casi siempre), se podrá utilizar para comenzar el procedimiento habitual.

10.4.3 Reglas de parada del método

Se utilizan las más lógicas: las que definen las condiciones de un punto óptimo de un programa lineal de Karush-Kuhn-Tucker. Es decir: 1. factibilidad del programa primal; 2. factibilidad del dual; y 3. complementariedad de holguras.

10.4.3.1 Factibilidad del programa primal

Se ha de cumplir que

$$A\mathbf{x}^k = \mathbf{b}, \quad \mathbf{x}^k \geq \mathbf{0}.$$

En la práctica, teniendo en cuenta el entorno finito de la aritmética que utiliza el ordenador donde se debe implementar el método, se comprueba que

$$\rho_p = \frac{\|A\mathbf{x}^k - \mathbf{b}\|}{\|\mathbf{b}\| + 1}, \quad \mathbf{x}^k \geq \mathbf{0}.$$

Si ρ_p es suficientemente pequeño y $\mathbf{x}^k \geq \mathbf{0}$, se puede considerar que se cumplen las condiciones de factibilidad del programa primal.

10.4.3.2 Factibilidad del programa dual

La factibilidad del programa dual requiere que los costes reducidos

$$\mathbf{r}^k = \mathbf{c} - A^T \mathbf{w}^k \geq \mathbf{0},$$

donde $\mathbf{w}^k$ es el vector de estimadores duales. Desde el punto de vista práctico, una vez más, si

$$\rho_d = \frac{\|\mathbf{r}^k\|}{\|\mathbf{c}\| + 1}, \quad \mathbf{x}^k \geq \mathbf{0},$$

donde $\|\mathbf{r}^k\|$ y $\|\mathbf{c}\|$ se calculan sólo con aquellos componentes $r_i^k < 0$, la factibilidad del dual se considerará conseguida si ρ_d es suficientemente pequeño.

10.4.3.3 Complementariedad de holguras

La condición de complementariedad de holguras requiere que

$$(\mathbf{x}^k)^T \mathbf{r}^k = \mathbf{e}^T X_k \mathbf{r}^k = 0.$$

Como

$$\mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k = \mathbf{e}^T X_k \mathbf{r}^k,$$

donde $\mathbf{x}^k$ es un punto factible del primal y $\mathbf{w}^k$ del dual,

$$\rho_c = \mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k$$

calibrará la condición de complementariedad de holguras. Eligiendo ρ_c suficientemente pequeño se podrá verificar que se cumple esta condición.

10.4.4 Complejidad computacional del método primal de escalado afín

Como se ha podido comprobar, comparado con el método de Karmarkar, el método primal de escalado afín resulta mucho más natural y menos complicado. Su implementación en ordenador también es más sencilla. No requiere asumir que el valor de la función objetivo óptimo debe ser cero, ni tampoco la estructura simplicial. El número de iteraciones que requiere para llegar al óptimo del problema con una tolerancia $\varepsilon = 10^{-3}$, según R. Vanderbei, M. Mekeon y B. Freedman, es $7,3885m^{-0,0187}n^{0,1694}$.

Como ya hemos indicado anteriormente, a pesar de que en la práctica el método funciona muy bien, no se ha podido probar que su complejidad computacional sea polinómica. De hecho, N. Megido y M. Shub han comprobado que al resolver el problema de Klee-Minty, si el punto de partida está muy cerca de los límites de la región factible, el método, como en el caso del *simplex*, puede llegar a recorrer las proximidades de todos los puntos extremos de esa región factible.

Para evitar caer en el efecto que produce el acercarse mucho a los límites de la región factible se han diseñado diversas estrategias. A continuación estudiamos dos de las más utilizadas.

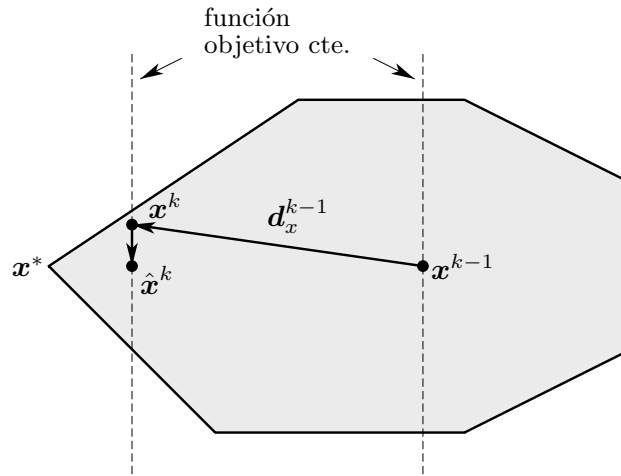
10.4.4.1 Método del empuje potencial

La idea consiste en “empujar” al punto que se esté considerando en un momento determinado del proceso de resolución del problema, $\mathbf{x}^k$, hacia el interior de la región factible sin empeorar el valor de la función objetivo. En la figura 10.10 se describe geométricamente lo que se pretende con esta forma de actuar.

Se parte de un $\mathbf{x}^{k-1}$, a lo largo de la dirección $\mathbf{d}_x^{k-1}$ se llega a $\mathbf{x}^k$, el cual se recentra en el interior de la región factible del problema, obteniéndose $\hat{\mathbf{x}}^k$. Ese recentrado se lleva a cabo mediante un *empuje potencial* tal que el valor de la función objetivo permanece constante pero el punto se aleja de la zona conflictiva que representa estar cerca de un límite exterior (una pared) de la región factible.

Para conseguir esto, se define la *función potencial* $p(\mathbf{x})$, para cada $\mathbf{x} > \mathbf{0}$, de la siguiente manera:

$$p(\mathbf{x}) = - \sum_{j=1}^n \ln x_j.$$

**Figura 10.10**

Representación de la idea en la que se basa el método de empuje potencial

El valor de la función potencial será tanto más grande cuanto más cerca esté el punto $\mathbf{x}$ de la condición $x_j = 0$. Esta función introduce una especie de fuerza que empuja a $\mathbf{x}$ hacia el interior de la región factible. Con ella se resuelve el siguiente problema:

$$\begin{aligned} &\text{minimizar } p(\mathbf{x}) \\ &\text{sujeta a } \quad A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} > \mathbf{0} \\ &\quad \quad \mathbf{c}^T \mathbf{x} = \mathbf{c}^T \mathbf{x}^k, \end{aligned} \tag{10.16}$$

problema denominado de empuje potencial. En este problema, la primera condición fuerza a que su solución esté en el interior de la región factible del problema de programación lineal original, la segunda que se mantenga como valor de la función objetivo el que existe en $\mathbf{x}^k$ y la función objetivo que el punto que se obtenga esté alejado de las paredes de la región factible.

Para resolver (10.16) mediante el método primal de escalado afín, por un lado, se tiene en cuenta la condición $\mathbf{c}^T \mathbf{x} = \mathbf{c}^T \mathbf{x}^k$ separadamente del resto del problema y, por otro, se mejora $p(\mathbf{x})$ lo que se considere necesario sin necesidad de llegar al óptimo, puesto que basta con separar el punto de la pared más próxima una cierta cantidad.

La forma de llevar esto a la práctica consiste en comenzar con $\mathbf{x}^k$ como punto de partida de (10.16), luego proyectar el negativo del gradiente de $p(\mathbf{x})$ en el subespacio núcleo de A y utilizar esta proyección como posible dirección, $\mathbf{p}^k$, a lo largo de la que moverse. Para mantener el valor de la función objetivo constante, primero se proyecta el negativo del gradiente de la función objetivo $\mathbf{c}^T \mathbf{x}$ en el subespacio núcleo de A , designándose por $\mathbf{g}$. Con posterioridad, para recentrar $\mathbf{x}^k$, se toma como dirección para hacerlo la componente de $\mathbf{p}^k$ que es ortogonal a $\mathbf{g}$, designándola $\hat{\mathbf{d}}_x^k$. Finalmente, a lo largo de esa dirección se calcula una amplitud de paso adecuada.

Si $P = I - A^T(AA^T)^{-1}A$ es la matriz de proyección sobre el subespacio núcleo de A y

$\nabla p(\mathbf{x})$ el gradiente de la función potencial,

$$\mathbf{p}^k = -P \left(\nabla p(\mathbf{x}^k) \right) = \left[I - A^T (AA^T)^{-1} A \right] \left(\frac{1}{\mathbf{x}^k} \right),$$

donde

$$\frac{1}{\mathbf{x}^k} = \left(\frac{1}{x_1^k}, \dots, \frac{1}{x_n^k} \right)^T.$$

Igualmente,

$$\mathbf{g} = -P\mathbf{c} = - \left[I - A^T (AA^T)^{-1} A \right] \mathbf{c}.$$

Descompongamos ahora el vector $\mathbf{p}^k$ en dos componentes: uno a lo largo de $\mathbf{g}$ y el otro ortogonal a éste. El primer componente se puede expresar de la forma $\mu\mathbf{g}$, para algún $\mu > 0$. Su ortogonal, de la siguiente manera:

$$\hat{\mathbf{d}}_x^k = \mathbf{p}^k - \mu\mathbf{g}.$$

La condición de ortogonalidad implica que

$$\left(\hat{\mathbf{d}}_x^k \right)^T \mathbf{g} = 0,$$

lo que permite obtener el valor de μ :

$$\mu = \frac{\left(\mathbf{p}^k \right)^T \mathbf{g}}{\mathbf{g}^T \mathbf{g}}.$$

De aquí que

$$\hat{\mathbf{d}}_x^k = \mathbf{p}^k - \left[\frac{\left(\mathbf{p}^k \right)^T \mathbf{g}}{\mathbf{g}^T \mathbf{g}} \right] \mathbf{g}.$$

El siguiente aspecto que hay que considerar es el cálculo de la amplitud del paso, κ , a dar a lo largo de $\hat{\mathbf{d}}_x^k$ partiendo de $\mathbf{x}^k$. El valor máximo de κ estará dado por

$$\max \kappa = \frac{1}{\max_j \left\{ -\hat{d}_{x_j}^k / x_j^k \right\}}.$$

La amplitud de paso se puede calcular por cualquier método que se considere oportuno buscando en el intervalo $(0, \max \kappa)$.

De esta forma de tratar de evitar una excesiva aproximación a las paredes de la región factible son dignos de tenerse en cuenta los siguientes aspectos:

1. Al aplicar el método del empuje potencial después de cada iteración del método primal de escalado afín, como la matriz P es la misma que la calculada en esa iteración y la búsqueda de κ mediante un procedimiento de búsqueda lineal es relativamente sencilla, la mayor cantidad de trabajo que conlleva el empuje potencial es la evaluación de la función potencial en esa búsqueda lineal.

2. Si la previsible mejora que se obtiene de aplicar el empuje potencial, separándose de las paredes de la región factible, no se constata que es tal, no será necesario continuar actuando así. Esto se puede comprobar después de 4 ó 5 iteraciones.
3. La función potencial del método de Karmarkar era

$$f(\mathbf{x}, \mathbf{c}) = n \ln(\mathbf{c}^T \mathbf{x}) - \sum_{j=1}^n \ln x_j = \sum_{j=1}^n \ln \left(\frac{\mathbf{c}^T \mathbf{x}}{x_j} \right).$$

Es decir, $f(\mathbf{x}, \mathbf{c}) = n \ln(\mathbf{c}^T \mathbf{x}) - p(\mathbf{x})$, supuesto $\mathbf{c}^T \mathbf{x} > 0$. Cuando se aplica el método de escalado afín seguido del empuje potencial es como si se conservara la estrategia del procedimiento de Karmarkar pues el primer término lo reduce el propio método de escalado afín mientras que del segundo se encarga el empuje potencial.

4. Parece lógico pensar que si se conserva la estrategia del procedimiento de Karmarkar, el método primal de escalado afín junto con el empuje potencial pudieran dar como resultado un algoritmo de complejidad computacional polinómica. Todavía, no obstante, no se ha podido probar esto rigurosamente.

10.4.4.2 Método de función barrera logarítmica

Este otro procedimiento es una alternativa más para tratar de evitar que cada iteración del método primal de escalado afín parta de puntos próximos a las paredes de la región factible. - La idea es incorporar una función barrera en la función objetivo original que adopte valores muy altos cuando un punto está próximo a los límites de la región factible. Si, lógicamente, se minimiza esta nueva función objetivo, los puntos que se vayan obteniendo en el proceso iterativo, automáticamente, se alejarán de las paredes de la región factible. En la figura 10.11 se describe un pequeño ejemplo en el que se sustituye la función $3 - x/2$ por una de barrera logarítmica que impide que el valor de la solución se acerque a la pared que determina la condición $x \leq 6$.

El problema que se resuelve es

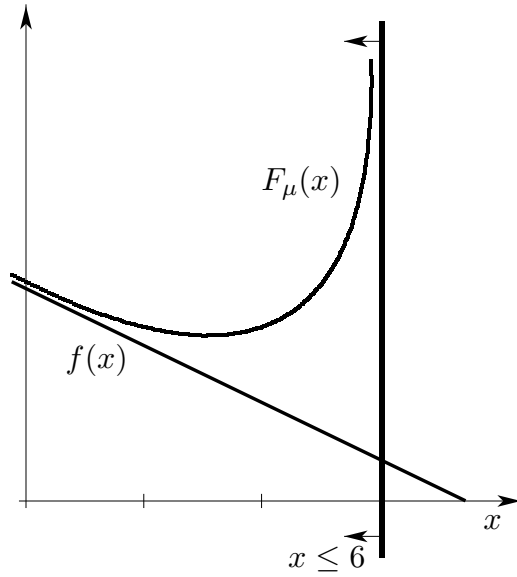
$$\begin{aligned} \min. \quad & F_\mu(\mathbf{x}) = \mathbf{c}^T \mathbf{x} - \mu \sum_{j=1}^n \ln x_j \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} > \mathbf{0}, \end{aligned} \tag{10.17}$$

donde $\mu > 0$ es un escalar. Si $\mathbf{x}^*(\mu)$ es el óptimo de este problema y tiende a $\mathbf{x}^*$ al tender μ a cero, $\mathbf{x}^*$ será el óptimo del problema original. Obsérvese que la condición de que $\mathbf{x} > \mathbf{0}$ está implícitamente recogida en la definición de la función barrera logarítmica.

Para resolver (10.17) se utiliza un método de descenso basado en el cálculo de una dirección de descenso, $\mathbf{d}$, a lo largo de la cual luego se calcula una amplitud de paso. Si el punto en el que se está es $\mathbf{x}^k$ y la dirección de descenso es $\mathbf{d}$, el nuevo punto vendrá dado por la expresión

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha \mathbf{d}.$$

El cálculo de $\mathbf{d}$ y α han de asegurar que $A\mathbf{x}^{k+1} = \mathbf{b}$ y que $F_\mu(\mathbf{x}^{k+1}) < F_\mu(\mathbf{x}^k)$.

**Figura 10.11**

Función barrera logarítmica del problema: minimizar $f(x) = 3 - x/2$ sujeta a $x \leq 6$

La dirección de descenso más habitual es la de Newton. Se determina de la aproximación cuadrática derivada del desarrollo en serie de Taylor de la función objetivo truncado a partir de los términos de segundo orden, sujeta a mantener la factibilidad del problema. Es decir,

$$\begin{aligned} &\text{minimizar } \frac{1}{2} \mathbf{d}^T \nabla^2 F_\mu(\mathbf{x}) \mathbf{d} + (\nabla F_\mu(\mathbf{x}))^T \mathbf{d} \\ &\text{sujeta a } \quad A\mathbf{d} = \mathbf{0}, \end{aligned} \quad (10.18)$$

donde $\nabla F_\mu(\mathbf{x}) = \mathbf{c} - \mu X^{-1} \mathbf{e}$ y $\nabla^2 F_\mu(\mathbf{x}) = \mu X^{-2}$. La dirección de descenso deberá estar por tanto en el subespacio nulo de la matriz A , $\ker(A)$.

Si a (10.18) se le aplican las condiciones de punto óptimo de Karush-Kuhn-Tucker se obtiene que

$$\mathbf{c} - \mu X^{-1} \mathbf{e} + \mu X^{-2} \mathbf{d}_\mu = A^T \lambda_\mu,$$

donde $\mathbf{d}_\mu$ es la dirección de Newton y λ_μ el vector de multiplicadores de Lagrange de las condiciones. Combinando estas condiciones y las de factibilidad del problema se obtiene el siguiente sistema de ecuaciones lineales:

$$\begin{bmatrix} \mu X^{-2} & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} -\mathbf{d}_\mu \\ \lambda_\mu \end{bmatrix} = \begin{bmatrix} \mathbf{c} - \mu X^{-1} \mathbf{e} \\ \mathbf{0} \end{bmatrix}.$$

Si la matriz X^{-2} no es singular, la solución de este último sistema de ecuaciones se puede obtener resolviendo

$$\begin{aligned} \frac{1}{\mu} A X^2 A^T \lambda_\mu &= \frac{1}{\mu} A X^2 (\mathbf{c} - \mu X^{-1} \mathbf{e}) \\ \mu X^{-2} \mathbf{d}_\mu &= A^T \lambda_\mu - (\mathbf{c} - \mu X^{-1} \mathbf{e}). \end{aligned}$$

De la primera de estas dos ecuaciones se deduce que

$$\lambda_\mu = \left(AX^2A^T\right)^{-1} AX^2 \left(\mathbf{c} - \mu X^{-1}\mathbf{e}\right).$$

Sustituyendo λ_μ en la segunda se obtiene que

$$\mathbf{d}_\mu = -\frac{1}{\mu}X \left[I - XA^T \left(AX^2A^T\right)^{-1} AX \right] (X\mathbf{c} - \mu\mathbf{e}).$$

Si suponemos que estamos en $\mathbf{x}^k$ y se calcula la dirección $\mathbf{d}_\mu$, comparándola con la que se obtiene en el método primal de escalado afín, $\mathbf{d}_x^k$, se tiene que

$$\mathbf{d}_\mu = \frac{1}{\mu}\mathbf{d}_x^k + X_k \left[I - X_k A^T \left(AX_k^2A^T\right)^{-1} AX_k \right] \mathbf{e}.$$

El componente adicional, $X_k[I - X_k A^T (AX_k^2 A^T) AX_k] \mathbf{e} = X_k P_k \mathbf{e}$, se puede ver como una fuerza que empuja a que la solución esté lejos de los límites de la región factible. De aquí que en algunas referencias bibliográficas a este componente se le denomine *fuerza de centrado* y al método que incluye la función barrera logarítmica, *método primal de escalado afín con fuerza de centrado*.

10.5 El método dual de escalado afín

Recordemos una vez más que el dual del problema de programación lineal en forma estándar

$$\begin{aligned} \min. \quad & \mathbf{c}^T \mathbf{x} \\ \text{s. a} \quad & A\mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

es

$$\begin{aligned} \text{maximizar} \quad & \mathbf{b}^T \mathbf{w} \\ \text{sujeta a} \quad & A^T \mathbf{w} + \mathbf{s} = \mathbf{c} \\ & \mathbf{s} \geq \mathbf{0}, \end{aligned} \tag{10.19}$$

donde $\mathbf{w} \in \Re^m$ no está restringido en ningún sentido.

De una forma similar a como lo hacía el método dual del *simplex*, el método dual de escalado afín comienza con una solución factible del programa dual y avanza mejorando la función objetivo hasta conseguir la factibilidad del programa primal, en cuyo caso se habrá alcanzado el punto óptimo de los programas primal y dual.

En (10.19), $[\mathbf{w}, \mathbf{s}]^T \in \Re^{m+n}$ está en el interior de la región factible si $A^T \mathbf{w} + \mathbf{s} = \mathbf{c}$ y $\mathbf{s} > \mathbf{0}$. En este caso no tiene sentido hablar de si $\mathbf{w}$ está centrado en la región factible dado que no está restringido en ningún sentido; en el caso de $\mathbf{s}$, por el contrario, sí, por lo que la idea es, como en casos anteriores, llevarlo al centro de la región factible siempre que ello sea posible.

10.5.1 Ideas básicas del método dual de escalado afín

Como el primal, el método dual de escalado afín consta de tres fases principales: 1. búsqueda de una solución de la que partir; 2. calcular una dirección a lo largo de la cual moverse a una solución mejor; y 3. pararse cuando la solución esté lo suficientemente próxima al punto óptimo. Como hicimos al describir el método primal, estudiemos primero qué es lo que se hace en el núcleo fundamental del procedimiento y luego veremos cómo comenzar y dónde parar.

Si en una determinada iteración k se dispone de una solución, $[\mathbf{w}^k, \mathbf{s}^k]^T$, en el interior de la región factible, tal que $A^T \mathbf{w}^k + \mathbf{s}^k = \mathbf{c}$, siendo $\mathbf{s}^k > \mathbf{0}$, su objetivo es encontrar una buena dirección de movimiento $[\mathbf{d}_w^k, \mathbf{d}_s^k]^T$ tal que moviéndose a lo largo de ella un paso $\beta_k > 0$ se llegue a una nueva solución $[\mathbf{w}^{k+1}, \mathbf{s}^{k+1}]^T$ tal que

$$\mathbf{w}^{k+1} = \mathbf{w}^k + \beta_k \mathbf{d}_w^k \quad (10.20)$$

$$\mathbf{s}^{k+1} = \mathbf{s}^k + \beta_k \mathbf{d}_s^k, \quad (10.21)$$

que satisfaga

$$A^T \mathbf{w}^{k+1} + \mathbf{s}^{k+1} = \mathbf{c} \quad (10.22)$$

$$\mathbf{s}^{k+1} > \mathbf{0}, \quad (10.23)$$

y en la que se cumpla que

$$\mathbf{b}^T \mathbf{w}^{k+1} \geq \mathbf{b}^T \mathbf{w}^k. \quad (10.24)$$

Si en (10.22) se sustituyen $\mathbf{w}^{k+1}$ y $\mathbf{s}^{k+1}$ por sus expresiones de (10.20) y (10.21), teniendo en cuenta además que $A^T \mathbf{w}^k + \mathbf{s}^k = \mathbf{c}$, se tiene que

$$A^T \mathbf{d}_w^k + \mathbf{d}_s^k = \mathbf{0}. \quad (10.25)$$

Para mejorar la función objetivo se deberá cumplir además que

$$\mathbf{b}^T \mathbf{w}^{k+1} = \mathbf{b}^T (\mathbf{w}^k + \beta_k \mathbf{d}_w^k) \geq \mathbf{b}^T \mathbf{w}^k,$$

por lo que la dirección $\mathbf{d}_w^k$ debe cumplir que $\mathbf{b}^T \mathbf{d}_w^k \geq 0$. Para conseguir que $\mathbf{s}^{k+1} > \mathbf{0}$ se aplica el método de escalado afín. La idea básica en este sentido consiste en recentrar el punto $\mathbf{s}^k$ en $\mathbf{e} = [1, \dots, 1]^T \in \Re^n$ en el espacio transformado, como hacíamos en el método primal de escalado afín, consiguiéndose así que cualquier desplazamiento desde ese nuevo punto pueda hacerse en al menos una unidad sin alcanzar ninguna de las paredes que determinan las condiciones de no negatividad.

Para llevar a cabo la transformación afín se define la matriz de escalado, $S_k = \text{diag}(\mathbf{s}^k)$. Mediante ella, $S_k^{-1} \mathbf{s}^k = \mathbf{e}$, transformándose el vector $\mathbf{s}$ en uno $\mathbf{u} \geq \mathbf{0}$ tal que

$$\mathbf{u} = S_k^{-1} \mathbf{s}$$

y

$$\mathbf{s} = S_k \mathbf{u}.$$

Además, si en el espacio resultante de la transformación $\mathbf{d}_u^k$ es la dirección de movimiento a lo largo de la cual se puede mejorar el valor de la función objetivo, ésta dirección en el espacio original será

$$\mathbf{d}_s^k = S_k \mathbf{d}_u^k.$$

Para que una dirección en el espacio transformado sea adecuada y se mejore la función objetivo del problema original, es decir, se cumpla (10.25),

$$\begin{aligned} A^T \mathbf{d}_w^k + \mathbf{d}_s^k = \mathbf{0} &\Rightarrow A^T \mathbf{d}_w^k + S_k \mathbf{d}_u^k = \mathbf{0} \\ &\Rightarrow S_k^{-1} A^T \mathbf{d}_w^k + \mathbf{d}_u^k = \mathbf{0} \Rightarrow S_k^{-1} A^T \mathbf{d}_w^k = -\mathbf{d}_u^k. \end{aligned}$$

Multiplicando ambos miembros por AS_k^{-1} :

$$AS_k^{-2} A^T \mathbf{d}_w^k = -AS_k^{-1} \mathbf{d}_u^k.$$

Si se supone que la matriz A es de rango completo,

$$\mathbf{d}_w^k = -\left(AS_k^{-2} A^T\right)^{-1} AS_k^{-1} \mathbf{d}_u^k.$$

Haciendo $Q_k = (AS_k^{-2} A^T)^{-1} AS_k^{-1}$, esta última expresión se puede escribir

$$\mathbf{d}_w^k = -Q_k \mathbf{d}_u^k.$$

Esta ecuación pone de manifiesto que la dirección $\mathbf{d}_w^k$ está definida a partir de $\mathbf{d}_u^k$ en el espacio transformado. Si se puede encontrar una dirección $\mathbf{d}_u^k$ tal que haga que se cumpla $\mathbf{b}^T \mathbf{d}_w^k \geq 0$, se habrá conseguido nuestro objetivo. Para ello, si

$$\mathbf{d}_u^k = -Q_k^T \mathbf{b}$$

entonces

$$\mathbf{b}^T \mathbf{d}_w^k = \mathbf{b}^T Q_k \mathbf{d}_u^k = \mathbf{b}^T Q_k Q_k^T \mathbf{b} = \|\mathbf{b}^T Q_k\|^2 \geq 0.$$

Combinando esta última expresión con $\mathbf{d}_w^k = -Q_k \mathbf{d}_u^k$, se obtiene que

$$\mathbf{d}_w^k = \left(AS_k^{-2} A^T\right)^{-1} \mathbf{b}.$$

De acuerdo con esto, teniendo en cuenta (10.25), se obtiene la dirección de movimiento en el espacio original:

$$\mathbf{d}_s^k = -A^T \mathbf{d}_w^k = -A^T \left(AS_k^{-2} A^T\right)^{-1} \mathbf{b}.$$

Una vez calculada la dirección de movimiento, $[\mathbf{d}_w^k, \mathbf{d}_s^k]^T$, la amplitud del paso a dar a lo largo de ella vendrá dada por el requisito de que $\mathbf{s}^{k+1}$ sea estrictamente positivo. En este sentido,

- Si $\mathbf{d}_s^k = \mathbf{0}$ el problema dual presenta una función objetivo constante en la región factible y $[\mathbf{w}^k, \mathbf{s}^k]^T$ es la solución óptima del programa dual.
- Si $\mathbf{d}_s^k \geq \mathbf{0}$ y $\mathbf{d}_s^k \neq \mathbf{0}$, el programa dual es no acotado.
- En cualquier otro caso

$$\beta_k = \min_i \left\{ \frac{\alpha s_i^k}{-d_{s_i}^k} : d_{s_i}^k < 0 \right\}, \quad \text{donde } 0 < \alpha < 1.$$

Si, de la misma forma que definíamos los estimadores duales en el método primal de escalado afín, se define

$$\mathbf{x}^k = -S_k^{-2}\mathbf{d}_s^k,$$

entonces $A\mathbf{x}^k = AS_k^{-2}A^T\mathbf{d}_w^k = \mathbf{b}$. Es decir, $\mathbf{x}^k$ se puede ver como el vector de *estimadores primales* en el método dual de escalado afín. Cuando el vector de estimadores primales cumpla que $\mathbf{x}^k \geq \mathbf{0}$, se habrá conseguido una solución factible del programa primal con una diferencia⁵ entre valor objetivo del primal y del dual de $\mathbf{c}^T\mathbf{x}^k - \mathbf{b}^T\mathbf{w}^k$. Si $\mathbf{c}^T\mathbf{x}^k - \mathbf{b}^T\mathbf{w}^k = 0$, $[\mathbf{w}^k, \mathbf{s}^k]^T$ es la solución óptima del dual y $\mathbf{x}^k$ el óptimo del primal.

En la tabla 10.3 se describe, como síntesis de todo lo expuesto en este apartado, el algoritmo dual de escalado afín.

Ejemplo 10.5 Resuélvase el dual de:

$$\begin{aligned} &\text{minimizar } -2x_1 + x_2 \\ &\text{sujeta a } \begin{aligned} x_1 - x_2 + x_3 &= 15 \\ x_2 + x_4 &= 15 \\ x_1, x_2, x_3, x_4 &\geq 0. \end{aligned} \end{aligned}$$

Este dual es:

$$\begin{aligned} &\text{maximizar } 15w_1 + 15w_2 \\ &\text{sujeta a } \begin{aligned} w_1 + s_1 &= -2 \\ -w_1 + w_2 + s_2 &= 1 \\ w_1 + s_3 &= 0 \\ w_2 + s_4 &= 0 \\ s_1, s_2, s_3, s_4 &\geq 0. \end{aligned} \end{aligned}$$

Es fácilmente comprobable que $\mathbf{w}^0 = [-3, -3]^T$ y $\mathbf{s}^0 = [1, 1, 3, 3]^T$ constituyen una solución factible de este problema en el interior de su región factible.

Iteración 1

La matriz de la primera transformación es:

$$S_0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}.$$

Las direcciones de movimiento son

$$\mathbf{d}_w^0 = (AS_0^{-2}A^T)^{-1}\mathbf{b} = \left(\begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}^{-2} \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \right)^{-1} \begin{bmatrix} 15 \\ 15 \end{bmatrix} = \begin{bmatrix} 23,53211 \\ 34,67890 \end{bmatrix}$$

⁵Duality gap.

Tabla 10.3
Algoritmo dual de escalado afín

Paso 0 – *Inicialización.* Hacer $k = 0$ y definir un $[\mathbf{w}^0, \mathbf{s}^0]^T$ tal que $A^T \mathbf{w}^0 + \mathbf{s}^0 = \mathbf{c}$ y $\mathbf{s}^0 > \mathbf{0}$

Paso 1 – *Cálculo de las direcciones de movimiento.* Sea $S_k = \text{diag}(\mathbf{s}^k)$, calcular

$$\mathbf{d}_w^k = (AS_k^{-2}A^T)^{-1} \mathbf{b} \quad \text{y} \quad \mathbf{d}_s^k = -A^T \mathbf{d}_w^k.$$

Paso 2 – *Comprobación de problema no acotado.* Si $\mathbf{d}_s^k = \mathbf{0}$, PARAR. La solución $[\mathbf{w}^k, \mathbf{s}^k]^T$ es el óptimo del programa dual. Si $\mathbf{d}_s^k \geq \mathbf{0}$, PARAR. El programa dual no está acotado.

Paso 3 – *Cálculo de la solución del primal.* Calcular el punto del primal mediante la expresión

$$\mathbf{x}^k = -S_k^{-2} \mathbf{d}_s^k.$$

Paso 4 – *Comprobar si se ha llegado al óptimo.* Si $\mathbf{x}^k \geq \mathbf{0}$ y $\mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k \leq \varepsilon$, donde ε es un valor pequeño positivo, PARAR. Las soluciones $[\mathbf{w}^k, \mathbf{s}^k]^T$ y $\mathbf{x}^k$ son las óptimas del programa dual y del primal, respectivamente. Si no, ir al paso siguiente.

Paso 5 – *Cálculo de la amplitud de paso.* Calcular la amplitud del paso a dar en la dirección de movimiento:

$$\beta_k = \min_i \left\{ \frac{\alpha s_i^k}{-d_{s_i}^k} : d_{s_i}^k < 0 \right\} \quad \text{donde } 0 < \alpha < 1.$$

Paso 6 – *Obtener nuevo punto.* Realizar la traslación al nuevo punto:

$$\mathbf{w}^{k+1} = \mathbf{w}^k + \beta_k \mathbf{d}_w^k$$

$$\mathbf{s}^{k+1} = \mathbf{s}^k + \beta_k \mathbf{d}_s^k.$$

Hacer $k \leftarrow k + 1$ e ir al paso 1.

y

$$\mathbf{d}_s^0 = -A^T \mathbf{d}_w^0 = - \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 23,53211 \\ 34,67890 \end{bmatrix} = \begin{bmatrix} -23,53211 \\ -11,14679 \\ -23,53211 \\ -34,67890 \end{bmatrix}.$$

La solución del primal es

$$\mathbf{x}^0 = -S_0^{-2} \mathbf{d}_s^0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 3 \end{bmatrix}^{-2} \begin{bmatrix} -23,53211 \\ -11,14679 \\ -23,53211 \\ -34,67890 \end{bmatrix} = \begin{bmatrix} 23,53211 \\ 11,14679 \\ 2,61467 \\ 3,85321 \end{bmatrix}.$$

El *duality gap* es $\mathbf{c}^T \mathbf{x}^0 - \mathbf{b}^T \mathbf{w}^0 = 54,08257$; muy lejos todavía de cero.

La amplitud de paso β_0 se calcula de la expresión

$$\beta_0 = \min_i \left\{ \frac{0,99 s_i^0}{-d_{s_i}^0} : d_{s_i}^0 < 0 \right\} = \frac{0,99 \cdot 1}{23,53211} = 0,04207.$$

Las nuevas variables son

$$\mathbf{w}^1 = \mathbf{w}^0 + \beta_0 \mathbf{d}_w^0 = \begin{bmatrix} -3 \\ -3 \end{bmatrix} + 0,04207 \begin{bmatrix} 23,53211 \\ 34,67890 \end{bmatrix} = \begin{bmatrix} -2,01000 \\ -1,54105 \end{bmatrix}.$$

y

$$\mathbf{s}^1 = \mathbf{s}^0 + \beta_0 \mathbf{d}_s^0 = \begin{bmatrix} 1 \\ 1 \\ 3 \\ 3 \end{bmatrix} + 0,04207 \begin{bmatrix} 23,53211 \\ 11,14679 \\ 2,61467 \\ 3,85321 \end{bmatrix} = \begin{bmatrix} 0,01000 \\ 0,53105 \\ 2,01000 \\ 1,54105 \end{bmatrix}.$$

Iteración 2 y sucesivas

Continuando con el proceso de la misma forma que hemos hecho en la iteración precedente se conseguirían los puntos de la tabla 10.4. El valor de la solución es $\mathbf{x} = [30, 15, 0, 0]^T$, $\mathbf{w} = [-2, -1]^T$ y $\mathbf{s} = [0, 0, 2, 1]^T$. El óptimo de la función objetivo es -45 . ●

Tabla 10.4

Proceso de convergencia del algoritmo dual de escalado afín aplicado al ejemplo 10.5

k	$\mathbf{w}^k$		$\mathbf{s}^k$				Duality gap
2	-2,00961	-1,01491	0,00901	0,00531	2,00961	1,01492	9,8605
3	-2,00010	-1,00395	0,00010	0,00386	2,00009	1,00395	0,3723
4	-2,00009	-1,00013	0,00009	0,00004	2,00009	1,00013	0,0610
5	-2,00000	-1,00000	0,00000	0,00000	2,00000	1,00000	0,0033

10.5.2 Solución de partida del método

A continuación estudiaremos dos formas de encontrar una solución inicial del programa dual en el interior de la región factible. Su objetivo, determinar una solución $[\mathbf{w}^0, \mathbf{s}^0]^T$ tal que $A^T \mathbf{w}^0 + \mathbf{s}^0 = \mathbf{c}$ y $\mathbf{s}^0 > \mathbf{0}$. En el caso especial de que $\mathbf{c} > \mathbf{0}$, se puede escoger como esa solución que se busca $\mathbf{w}^0 = \mathbf{0}$ y $\mathbf{s}^0 = \mathbf{c}$.

10.5.2.1 El método de la gran M

Este método consiste en añadir una o más variables artificiales, w^a , y considerar el siguiente problema:

$$\begin{aligned} & \text{maximizar } \mathbf{b}^T \mathbf{w} + Mw^a \\ & \text{sujeta a } \quad A^T \mathbf{w} + \mathbf{p}w^a + \mathbf{s} = \mathbf{c} \\ & \quad \mathbf{s} \geq \mathbf{0}, \end{aligned} \tag{10.26}$$

donde $\mathbf{w}$ y w^a no están restringidas en ningún sentido y $\mathbf{p} \in \Re^n$ es un vector definido por:

$$p_i = \begin{cases} 1 & \text{si } c_i \leq 0 \\ 0 & \text{si } c_i > 0. \end{cases}$$

A continuación se define $\bar{c} = \max_i |c_i|$, se escoge un $\theta > 0$ y se hace $\mathbf{w} = \mathbf{0}$, $w^a = -\theta\bar{c}$ y $\mathbf{s} = \mathbf{c} + \theta\bar{c}\mathbf{p}$. De acuerdo con esto, la solución $[\mathbf{0}, -\theta\bar{c}, \mathbf{c} + \theta\bar{c}\mathbf{p}]^T$ es factible en (10.26), con $\mathbf{c} + \theta\bar{c}\mathbf{p} > \mathbf{0}$, y se puede empezar a resolver el problema.

Obsérvese que la variable w^a comienza con el valor $-\theta\bar{c} < 0$, forzándose luego a que, a lo largo del proceso, su valor se incremente dado que M es un número positivo grande y su presencia está por tanto muy penalizada. En algún punto del procedimiento su valor se hará no negativo a no ser que el problema original (10.19) sea no factible. Cuando el valor de w^a se aproxime a cero o incluso se haga cero, pongamos en la iteración k , se hace $\hat{\mathbf{w}} = \mathbf{w}^k$ y $\hat{\mathbf{s}} = \mathbf{s}^k + \mathbf{p}w^a$ empezándose desde ese punto el procedimiento dual de escalado afín. Si w^a no tiende a hacerse cero, se puede demostrar fácilmente que el problema (10.19) no es factible.

10.5.2.2 Método de la condición artificial o del límite superior

En este método se asume que, para un número suficientemente grande M , una de las soluciones óptimas del problema de programación lineal original pertenecerá a la bola de centro el origen y radio M , $S(\mathbf{0}, M)$, considerándose para su estudio el siguiente problema de programación lineal:

$$\begin{aligned} & \text{minimizar } \mathbf{c}^T \mathbf{x} \\ & \text{sujeta a } \quad A\mathbf{x} = \mathbf{b} \\ & \quad \mathbf{0} \leq \mathbf{x} \leq \mathbf{u}, \end{aligned}$$

donde $\mathbf{u} = [M, M, \dots, M]^T \in \Re^n$. La condición adicional que supone acotar superiormente al vector $\mathbf{x}$ se crea artificialmente con el fin de que el dual de este problema tenga una solución en el interior de su región factible trivial. Este dual es

$$\begin{aligned} & \text{maximizar } \mathbf{b}^T \mathbf{w} - \mathbf{u}^T \mathbf{v} \\ & \text{sujeta a } \quad A^T \mathbf{w} + \mathbf{s} - \mathbf{v} = \mathbf{c} \\ & \quad \mathbf{s} \geq \mathbf{0}, \mathbf{v} \geq \mathbf{0}, \end{aligned}$$

no estando restringido el vector $\mathbf{w}$ en ningún sentido. Si $\bar{c}$ y θ se definen como en el apartado anterior, $\mathbf{w}^0 = \mathbf{0}$, $\mathbf{v}^0 = \theta \bar{c} \mathbf{e} > \mathbf{0}$ y $\mathbf{s}^0 = \mathbf{c} + \theta \bar{c} \mathbf{e} > \mathbf{0}$ constituyen una solución interior de la región factible desde la que se puede comenzar el procedimiento iterativo del método dual de escalado afín.

La clave de esta forma de actuar estriba en el valor que se asigne a M . Debe ser lo suficientemente grande como para incluir al menos una solución óptima de (10.19). Si el problema original no está acotado, la elección de M puede ser muy problemática.

10.5.3 Reglas de parada del método

Como en el caso del método primal, se utilizan las que definen las condiciones de punto óptimo de un programa lineal de Karush-Kuhn-Tucker. Es decir: 1. factibilidad del programa primal; 2. factibilidad del dual; y 3. complementariedad de holguras. La factibilidad del dual se mantiene a lo largo de todo el algoritmo por lo que sólo se necesita comprobar la factibilidad del primal y la complementariedad de holguras.

Si se combinan

$$\mathbf{d}_s^k = -A^T \mathbf{d}_w^k = -A^T \left(AS_k^{-2} A^T \right)^{-1} \mathbf{b}$$

y

$$\mathbf{x}^k = -S_k^{-2} \mathbf{d}_s^k,$$

el valor de la solución del primal es

$$\mathbf{x}^k = -S_k^{-2} \mathbf{d}_s^k = S_k^{-2} A^T \left(AS_k^{-2} A^T \right)^{-1} \mathbf{b}. \quad (10.27)$$

Es fácilmente comprobable que las condiciones $A\mathbf{x} = \mathbf{b}$ se satisfacen automáticamente para cualquier $\mathbf{x}^k$ definido según esta última expresión. Si $\mathbf{x}^k \geq \mathbf{0}$ se satisfará la factibilidad del primal. Si el problema dual (10.19) de un programa lineal en forma estándar se convierte a su vez a la forma estándar y se le aplica el método primal de escalado afín, el valor de los estimadores duales en la iteración k se obtendrá de una expresión idéntica a (10.27).

Cuando se alcance un punto factible del programa dual, $\mathbf{w}^k$, y otro factible del primal, $\mathbf{x}^k$, se calcula la complementariedad de holguras, $\sigma_c = \mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k$. Cuando σ_c sea menor que una cantidad predeterminada se podrá finalizar el método.

10.5.4 Mejora de la complejidad computacional del método dual de escalado afín

Como en el caso del método primal, no existe una demostración teórica general que demuestre la complejidad computacional polinómica del método dual de escalado afín. Para mejorarla, en cualquier caso, se pueden utilizar estrategias similares a las empleadas en el método primal: empuje potencial y la función barrera logarítmica. En lo que sigue analizaremos brevemente la segunda.

10.5.4.1 Método de función barrera logarítmica

La idea, como en el algoritmo primal, es incorporar una función de barrera en la función objetivo que adopte valores muy altos cuando un punto esté próximo a los límites de la región

factible. Con este objetivo se considera el siguiente problema no lineal:

$$\begin{aligned} \min. \quad & F_\mu(\mathbf{w}, \mu) = \mathbf{b}^T \mathbf{w} + \mu \sum_{j=1}^n \ln(c_j - \mathbf{a}_j^T \mathbf{w}) \\ \text{s. a} \quad & A^T \mathbf{w} < \mathbf{c}, \end{aligned} \quad (10.28)$$

donde $\mu > 0$ es un escalar y $\mathbf{a}_j^T$ es el traspuesto del vector columna j de la matriz A . Obsérvese que si $\mathbf{w}^*(\mu)$ es la solución óptima de este problema y tiende a $\mathbf{w}^*$ al tender μ a cero, $\mathbf{w}^*$ será el óptimo del dual del programa lineal original que se busca.

La función lagrangiana del problema (10.28) es

$$L(\mathbf{w}, \lambda) = \mathbf{b}^T \mathbf{w} + \mu \sum_{j=1}^n \ln(c_j - \mathbf{a}_j^T \mathbf{w}) + \lambda^T (\mathbf{c} - A^T \mathbf{w}),$$

donde λ es el vector de multiplicadores simplex. Como $c_j - \mathbf{a}_j^T \mathbf{w} > 0$, la condición de complementariedad de holguras requiere que $\lambda = \mathbf{0}$. Las condiciones de Karush-Kuhn-Tucker son

$$\mathbf{b} - \mu A S^{-1} \mathbf{e} = \mathbf{0}$$

y $\mathbf{s} > \mathbf{0}$. Si se supone que $\mathbf{w}^k$ y $\mathbf{s}^k = \mathbf{c} - A^T \mathbf{w}^k > \mathbf{0}$ son una solución dual factible, de las condiciones de Karush-Kuhn-Tucker se puede determinar la dirección de Newton:

$$\Delta \mathbf{w} = \frac{1}{\mu} \left(A S_k^{-2} A^T \right)^{-1} \mathbf{b} - \left(A S_k^{-2} A^T \right)^{-1} A S_k^{-1} \mathbf{e}.$$

Comparada con la dirección $\mathbf{d}_w^k$ del método dual de escalado afín,

$$\mathbf{d}_w^k = \left(A S_k^{-2} A^T \right)^{-1} \mathbf{b},$$

$\Delta \mathbf{w}$ posee un término adicional: $-(A S_k^{-2} A^T)^{-1} A S_k^{-1} \mathbf{e}$. Este término es el que procura que la dirección lleve a un punto que no esté en los límites de la región factible.

Diversos autores han demostrado que, escogiendo adecuadamente el parámetro μ y la amplitud del paso a dar en cada iteración, la complejidad computacional de este método es polinómica. El número de iteraciones necesario para alcanzar el óptimo es como mucho $O(\sqrt{n})$.

10.6 El método primal-dual

Sea un programa de programación lineal en forma estándar

$$\begin{aligned} & \text{minimizar } \mathbf{c}^T \mathbf{x} \\ & \text{sujeta a } \quad A \mathbf{x} = \mathbf{b} \\ & \quad \mathbf{x} \geq \mathbf{0} \end{aligned} \quad (10.29)$$

y su dual

$$\begin{aligned} & \text{maximizar } \mathbf{b}^T \mathbf{w} \\ & \text{sujeta a } \quad A^T \mathbf{w} + \mathbf{s} = \mathbf{c} \\ & \quad \mathbf{s} \geq \mathbf{0}, \end{aligned} \quad (10.30)$$

en el que $\mathbf{w}$ no está restringido. Impongamos las siguientes condiciones:

- A1. El conjunto $S = \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} > \mathbf{0}\}$ no está vacío.
- A2. El conjunto $T = \{(\mathbf{w} \in \Re^m, \mathbf{s} \in \Re^n) : A^T\mathbf{w} + \mathbf{s} = \mathbf{c}, \mathbf{s} > \mathbf{0}\}$ no está vacío.
- A3. La matriz A es de rango completo.

En estas condiciones, de acuerdo con el teorema de la dualidad, los problemas (10.29) y (10.30) tienen solución óptima, coincidiendo los valores de sus funciones objetivo. Además el conjunto de soluciones óptimas de ambos están acotados.

Al problema primal (10.29) se le puede aplicar la técnica de la función barrera logarítmica para reemplazar las condiciones de no negatividad y así llegar a considerar el siguiente problema:

$$\begin{aligned} (P_\mu) : \quad & \text{minimizar } \mathbf{c}^T \mathbf{x} - \mu \sum_{j=1}^n \ln x_j \\ & \text{sujeta a } \quad A\mathbf{x} = \mathbf{b} \\ & \quad \mu > 0, \end{aligned}$$

donde $\mu > 0$ es el parámetro de penalización o barrera. De la misma manera, mediante una función barrera logarítmica, el problema dual se formula de la siguiente manera:

$$\begin{aligned} (D_\mu) : \quad & \text{maximizar } \mathbf{b}^T \mathbf{w} + \mu \sum_{j=1}^n \ln s_j \\ & \text{sujeta a } \quad A^T \mathbf{w} + \mathbf{s} = \mathbf{c} \\ & \quad \mu > 0. \end{aligned}$$

La función lagrangiana de (P_μ) es:

$$L_p(\mathbf{x}, \mathbf{w}, \mu) = \mathbf{c}^T \mathbf{x} - \mu \sum_{j=1}^n \ln x_j - \mathbf{w}^T (\mathbf{b} - A\mathbf{x}).$$

La de (D_μ) :

$$L_d(\mathbf{x}, \mathbf{w}, \mu) = \mathbf{b}^T \mathbf{w} + \mu \sum_{j=1}^n \ln s_j - \mathbf{x}^T (A^T \mathbf{w} + \mathbf{s} - \mathbf{c}).$$

Como las funciones objetivo de (P_μ) y (D_μ) son estrictamente convexas, estos problemas tendrán un solo mínimo y máximo (un mínimo global y un máximo global).

Las condiciones necesarias de primer orden que debe cumplir el mínimo de (P_μ) son

$$\begin{aligned} \frac{\partial L_p}{\partial \mathbf{x}} &= 0 = \mathbf{c} - \mu X^{-1} \mathbf{e} - A^T \mathbf{w} \quad \text{y} \\ \frac{\partial L_p}{\partial \mathbf{w}} &= 0 = \mathbf{b} - A\mathbf{x}, \end{aligned}$$

donde $X = \text{diag}(x_1, x_2, \dots, x_n)$. Las del máximo de (D_μ)

$$\begin{aligned}\frac{\partial L_d}{\partial \mathbf{x}} &= 0 = A^T \mathbf{w} + \mathbf{s} - \mathbf{c}, \\ \frac{\partial L_d}{\partial \mathbf{w}} &= 0 = \mathbf{b} - A\mathbf{x} \quad \text{y} \\ \frac{\partial L_d}{\partial \mathbf{s}} &= 0 = \mu S^{-1} \mathbf{e} - \mathbf{x},\end{aligned}$$

donde $S = \text{diag}(s_1, s_2, \dots, s_n)$. Sustituyendo $\mathbf{c} = \mathbf{s} + A^T \mathbf{w}$ se tiene que

$$\frac{\partial L_p}{\partial \mathbf{x}} = 0 = \mathbf{s} + A^T \mathbf{w} - \mu X^{-1} \mathbf{e} - A^T \mathbf{w} = \mathbf{s} - \mu X^{-1} \mathbf{e},$$

por lo que $X S \mathbf{e} = \mu \mathbf{e}$. Obsérvese que esta condición, componente a componente, se puede escribir

$$x_j s_j = \mu, \quad j = 1, \dots, n. \quad (10.31)$$

Las condiciones necesarias de primer orden, para un mismo μ , de los programas primal y dual son pues

$$\begin{aligned}A\mathbf{x} &= \mathbf{b} && \text{(factibilidad del programa primal)} \\ A^T \mathbf{w} + \mathbf{s} &= \mathbf{c} && \text{(factibilidad del programa dual)} \\ X S \mathbf{e} - \mu \mathbf{e} &= \mathbf{0} && \text{(complementariedad de holguras)}.\end{aligned} \quad (10.32)$$

En el supuesto de que se cumple la condición A3 antes indicada, el vector $\mathbf{x}$ determina de forma única $\mathbf{w}$ a partir de la expresión (10.31) y de la relación $\mathbf{c} = A^T \mathbf{w} + \mathbf{s}$. Si la solución del sistema de ecuaciones lineales (10.32) se designa mediante $[\mathbf{x}(\mu), \mathbf{w}(\mu), \mathbf{s}(\mu)]^T$, para cada $\mu > 0$, obviamente $\mathbf{x}(\mu) \in S$ y $[\mathbf{w}(\mu), \mathbf{s}(\mu)]^T \in T$. El *duality gap* es

$$\begin{aligned}g(\mu) &= \mathbf{c}^T \mathbf{x}(\mu) - \mathbf{b}^T \mathbf{w}(\mu) \\ &= (\mathbf{c}^T - \mathbf{w}(\mu)^T A) \mathbf{x}(\mu) \\ &= \mathbf{s}(\mu)^T \mathbf{x}(\mu) = n\mu.\end{aligned}$$

Al tender $\mu \rightarrow 0$, $g(\mu)$ converge a cero, lo que implica que $\mathbf{x}(\mu)$ y $\mathbf{w}(\mu)$ convergen a la solución de los problemas (10.29) y (10.30), respectivamente.

Si para $\mu > 0$ designamos mediante Γ la curva o camino de las soluciones del sistema de ecuaciones lineales (10.32), es decir,

$$\Gamma = \left\{ [\mathbf{x}(\mu), \mathbf{w}(\mu), \mathbf{s}(\mu)]^T : [\mathbf{x}(\mu), \mathbf{w}(\mu), \mathbf{s}(\mu)]^T \text{ es la solución de (10.32) para algún } \mu > 0 \right\},$$

al tender $\mu \rightarrow 0$ el camino Γ tiende a proporcionar una solución $[\mathbf{w}^*, \mathbf{s}^*]^T$ óptima de los programas primal y dual: (10.29) y (10.30).

El procedimiento primal-dual genera, dada una solución de partida $[\mathbf{x}^0 \in S, [\mathbf{w}^0, \mathbf{s}^0]^T \in T]$, escogiendo adecuadamente unas direcciones de movimiento $[\mathbf{d}_x^k, \mathbf{d}_w^k, \mathbf{d}_s^k]^T$, y a lo largo de ellas unas amplitudes de paso β^k , una sucesión de soluciones $[\mathbf{x}^k \in S, [\mathbf{w}^k, \mathbf{s}^k]^T \in T]$ que converge a la solución de (10.29) y (10.30).

10.6.1 Dirección y amplitud de movimiento

El método primal-dual se basa en tomar como direcciones de movimiento las de Newton resultantes de resolver el sistema (10.32). Es decir, recordando los conceptos del capítulo 4 referidos a la resolución de sistemas de ecuaciones no lineales, $\mathbf{f} : \Re^m \rightarrow \Re^n$, utilizando la aproximación de $\mathbf{f}$ mediante el modelo resultante del desarrollo en serie de Taylor truncándolo a partir de los términos de segundo orden, esa dirección de Newton es el resultado de resolver el sistema de ecuaciones

$$J(\mathbf{x}^k)(\mathbf{x} - \mathbf{x}^k) = -\mathbf{f}(\mathbf{x}^k),$$

donde $J(\mathbf{x}^k)$ es la matriz Jacobiana del sistema en $\mathbf{x}^k$, es decir,

$$J(\mathbf{x}_k) = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_n(\mathbf{x})}{\partial x_n} \end{bmatrix}_{\mathbf{x}=\mathbf{x}_k}.$$

En el caso que nos ocupa, refiriéndonos al sistema que definen las condiciones de Karush-Kuhn-Tucker, (10.32), la dirección de Newton $[\mathbf{d}_x^k, \mathbf{d}_w^k, \mathbf{d}_s^k]^T$ se determina a partir de la resolución del sistema

$$\begin{bmatrix} A & 0 & 0 \\ 0 & A^T & I \\ S_k & 0 & X_k \end{bmatrix} \begin{bmatrix} \mathbf{d}_x^k \\ \mathbf{d}_w^k \\ \mathbf{d}_s^k \end{bmatrix} = - \begin{bmatrix} A\mathbf{x}^k - \mathbf{b} \\ A^T \mathbf{w}^k + \mathbf{s}^k - \mathbf{c} \\ X_k S_k \mathbf{e} - \mu^k \mathbf{e} \end{bmatrix},$$

donde $X_k = \text{diag}(x_1^k, x_2^k, \dots, x_n^k)$ y $S_k = \text{diag}(s_1^k, s_2^k, \dots, s_n^k)$. Efectuando el producto de matrices resulta el siguiente sistema:

$$\begin{aligned} A\mathbf{d}_x^k &= \mathbf{t}^k \\ A^T \mathbf{d}_w^k + \mathbf{d}_s^k &= \mathbf{u}^k \\ S_k \mathbf{d}_x^k + X_k \mathbf{d}_s^k &= \mathbf{v}^k, \end{aligned} \tag{10.33}$$

donde $\mathbf{t}^k = \mathbf{b} - A\mathbf{x}^k$, $\mathbf{u}^k = \mathbf{c} - A^T \mathbf{w}^k - \mathbf{s}^k$ y $\mathbf{v}^k = \mu^k \mathbf{e} - X_k S_k \mathbf{e}$. Obsérvese que si $\mathbf{x}^k \in S$ y $[\mathbf{w}^k, \mathbf{s}^k]^T \in T$, entonces $\mathbf{t}^k = \mathbf{0}$ y $\mathbf{u}^k = \mathbf{0}$.

Para resolver (10.33) se multiplican ambos miembros de la primera ecuación por $AX_k S_k^{-1}$ resultando

$$AX_k S_k^{-1} A^T \mathbf{d}_w^k = AX_k S_k^{-1} \mathbf{u}^k - AX_k S_k^{-1} \mathbf{d}_s^k. \tag{10.34}$$

Despejando $\mathbf{d}_s^k$ de la tercera ecuación se tiene que

$$\mathbf{d}_s^k = X_k^{-1} \mathbf{v}^k - X_k^{-1} S_k \mathbf{d}_x^k.$$

Haciendo $X_k^{-1} \mathbf{v}^k = \mu^k X_k^{-1} \mathbf{e} - S_k \mathbf{e} = \mathbf{p}^k$,

$$AX_k S_k^{-1} \mathbf{d}_s^k = AX_k S_k^{-1} \mathbf{p}^k - AX_k S_k^{-1} X_k^{-1} S_k \mathbf{d}_x^k = AX_k S_k^{-1} \mathbf{p}^k - \mathbf{t}^k.$$

Sustituyendo esta expresión en (10.34) se obtiene que

$$\mathbf{d}_w^k = [AX_k S_k^{-1} A^T]^{-1} (AX_k S_k^{-1} (\mathbf{u}^k - \mathbf{p}^k) + \mathbf{t}^k), \tag{10.35}$$

donde $X_k S_k^{-1}$ es una matriz diagonal definida positiva. Calculada $\mathbf{d}_w^k$, $\mathbf{d}_s^k$ y $\mathbf{d}_x^k$ se pueden obtener fácilmente a partir de

$$\mathbf{d}_s^k = \mathbf{u}^k - A^T \mathbf{d}_w^k \quad (10.36)$$

y

$$\mathbf{d}_x^k = X_k S_k^{-1} [\mathbf{p}^k - \mathbf{d}_s^k]. \quad (10.37)$$

Si $\mathbf{x}^k \in S$ y $[\mathbf{w}^k, \mathbf{s}^k]^T \in T$, las expresiones de $\mathbf{d}_w^k$, $\mathbf{d}_x^k$ y $\mathbf{d}_s^k$ se pueden simplificar quedando:

$$\begin{aligned} \mathbf{d}_w^k &= -[A \hat{D}_k^2 A^T]^{-1} A S_k^{-1} \mathbf{v}^k \\ \mathbf{d}_s^k &= -A^T \mathbf{d}_w^k \\ \mathbf{d}_x^k &= S_k^{-1} [\mathbf{v}^k - X_k \mathbf{d}_s^k], \end{aligned}$$

donde $\hat{D}_k^2 = X_k S_k^{-1}$ y $\hat{D}_k = \text{diag}(\sqrt{\mathbf{x}^k / \mathbf{s}^k})$.

Los componentes de la dirección de movimiento calculados están relacionados entre sí. En efecto, si se designa

$$\mathbf{r}^k(\mu) = X_k^{-1} \hat{D}_k \mathbf{v}^k(\mu)$$

y

$$Q = \hat{D}_k A^T (A \hat{D}_k^2 A^T)^{-1} A \hat{D}_k$$

los componentes de la dirección de movimiento $[\mathbf{d}_x^k, \mathbf{d}_w^k, \mathbf{d}_s^k]^T$ se puede reescribir de la siguiente manera:

$$\begin{aligned} \mathbf{d}_x^k &= \hat{D}_k (I - Q) \mathbf{r}^k(\mu) \\ \mathbf{d}_w^k &= - (A \hat{D}_k^2 A^T)^{-1} A \hat{D}_k \mathbf{r}^k(\mu) \\ \mathbf{d}_s^k &= \hat{D}_k^{-1} Q \mathbf{r}^k(\mu). \end{aligned}$$

Como la matriz Q es la de proyección ortogonal en el subespacio $\text{Im}(\hat{D}_k A^T)$,

$$\begin{aligned} \hat{D}_k^{-1} \mathbf{d}_x^k + \hat{D}_k \mathbf{d}_s^k &= \mathbf{r}^k(\mu) \\ (\mathbf{d}_x^k)^T \mathbf{d}_s^k &= (\hat{D}_k^{-1} \mathbf{d}_x^k)^T \hat{D}_k \mathbf{d}_s^k = 0. \end{aligned}$$

Si se combinan las ecuaciones (10.35), (10.36) y (10.37) se tiene que

$$\mathbf{d}_x^k = \mu^k \hat{D}_k P_k \hat{D}_k X_k^{-1} \mathbf{e} - \hat{D}_k P_k \hat{D}_k \mathbf{c} + \hat{D}_k^2 A^T (A \hat{D}_k^2 A^T)^{-1} \mathbf{t}^k, \quad (10.38)$$

donde $\hat{D}_k^2 = X_k S_k^{-1}$ y $P_k = I - \hat{D}_k A^T (A \hat{D}_k^2 A^T)^{-1} A \hat{D}_k$ es la matriz de proyección en $\ker(A \hat{D}_k)$.

Si se definen

$$\begin{aligned} \mathbf{d}_{x_{cen}}^k &= \mu^k \hat{D}_k P_k \hat{D}_k X_k^{-1} \mathbf{e} \\ \mathbf{d}_{x_{obj}}^k &= -\hat{D}_k P_k \hat{D}_k \mathbf{c} \quad y \\ \mathbf{d}_{x_{fac}}^k &= \hat{D}_k^2 A^T (A \hat{D}_k^2 A^T)^{-1} \mathbf{t}^k, \end{aligned}$$

la ecuación (10.38) se convierte en

$$\mathbf{d}_x^k = \mathbf{d}_{x_{cen}}^k + \mathbf{d}_{x_{obj}}^k + \mathbf{d}_{x_{fac}}^k.$$

El término $\mathbf{d}_{x_{cen}}^k$ se le conoce como *dirección de centrado*, pues no es sino la proyección del vector de empuje $1/\mathbf{x}^k$ que ayuda a que los puntos que se van obteniendo con el algoritmo se mantengan alejados de las paredes de la región factible. El término $\mathbf{d}_{x_{obj}}^k$ es la *dirección de reducción de la función objetivo*, pues es la proyección del negativo del vector gradiente de la función objetivo del programa primal que lleva a la reducción del valor de esta función objetivo. Por último, el término $\mathbf{d}_{x_{fac}}^k$ es la denominada *dirección de factibilidad* pues $\mathbf{t}^k$ es una medida de la factibilidad del programa primal. Obsérvese además que $A\mathbf{d}_{x_{cen}}^k = \mathbf{0}$ y que $A\mathbf{d}_{x_{obj}}^k = \mathbf{0}$, por lo que estas dos direcciones están en el subespacio núcleo de la matriz A . A la factibilidad del primal sólo le afecta $\mathbf{d}_{x_{fac}}^k$.

En la práctica, el método primal-dual comienza en una solución arbitraria $[\mathbf{x}^0, \mathbf{w}^0, \mathbf{s}^0]^T$, con $\mathbf{x}^0, \mathbf{s}^0 > \mathbf{0}$. El valor de $\mathbf{t}^0$ puede ser muy grande pues $\mathbf{x}^0$ puede estar muy alejado de ser factible. Desde ese punto el principal esfuerzo se dedica a encontrar una solución factible lo más centrada posible en la región factible. Conseguido esto, el algoritmo tratará de conseguir que $\mathbf{t}^k = \mathbf{0}$. De esta forma el término $\mathbf{d}_{x_{fac}}^k$ se anulará.

10.6.1.1 Amplitud de movimiento

Después de calculada la dirección de Newton en una iteración k , el algoritmo primal-dual desplazará la solución a:

$$\begin{aligned}\mathbf{x}^{k+1} &= \mathbf{x}^k + \beta_p^k \mathbf{d}_x^k \\ \mathbf{w}^{k+1} &= \mathbf{w}^k + \beta_d^k \mathbf{d}_w^k \\ \mathbf{s}^{k+1} &= \mathbf{s}^k + \beta_d^k \mathbf{d}_s^k.\end{aligned}$$

Esto implicaría la elección de unas amplitudes de paso adecuadas, β_p^k y β_d^k , en la dirección de Newton, tales que $\mathbf{x}^{k+1} \in S$ y $[\mathbf{w}^{k+1}, \mathbf{s}^{k+1}]^T \in T$. Una manera simple de elegir β_p^k y β_d^k es hacer

$$\beta_p^k = \frac{1}{\max \left\{ 1, -d_{x_i}^k / \alpha x_i^k \right\}}$$

y

$$\beta_d^k = \frac{1}{\max \left\{ 1, -d_{s_i}^k / \alpha s_i^k \right\}},$$

donde $\alpha < 1$, $d_{x_i}^k$ es el componente i -ésimo de $\mathbf{d}_x^k$, x_i^k el componente i -ésimo de $\mathbf{x}^k$, $d_{s_i}^k$ el i -ésimo de $\mathbf{d}_s^k$ y s_i^k el componente i -ésimo de $\mathbf{s}^k$.

10.6.2 Ajuste del parámetro de penalización y reglas de parada del método

El parámetro μ^k debe reducirse de iteración en iteración a fin de que al finalizar el procedimiento se satisfagan las condiciones de complementariedad de holguras

$$XSe - \mu e = \mathbf{0}.$$

De esta ecuación se puede deducir que $\mu_{med} = \mathbf{s}^T \mathbf{x} / n$. Si esta expresión se introduce en las de $\mathbf{x}^k$ y $\mathbf{s}^k$ se obtendrá una buena medida del valor del parámetro en el punto en el que se esté. En muchos casos, un valor de μ^k un poco más pequeño, por ejemplo,

$$\sigma \left[\left(\mathbf{s}^k \right)^T \mathbf{x}^k \right] / n,$$

con $\sigma < 1$, suele acelerar la convergencia del método. La elección de este último parámetro, en muchos casos, es crítica pues contribuye en no poca medida a la rápida convergencia del método. En las referencias que se dictan al final de este capítulo se puede estudiar cómo calcularlo.

10.6.2.1 Reglas de parada del método

Se utilizan las que venimos aplicando: la factibilidad del primal, la del dual y la complementariedad de holguras.

La factibilidad del primal la mide $\mathbf{t}^k$, la del dual $\mathbf{u}^k$ y la complementariedad de holguras $\mathbf{v}^k$.

Como síntesis de todas las ideas expuestas relativas al método primal-dual, en la tabla 10.5 se describe el algoritmo primal-dual de puntos en el interior de la región factible de un programa lineal.

Ejemplo 10.6 Resolver el mismo problema de los ejemplos 10.4 y 10.5:

$$\begin{aligned} &\text{minimizar } -2x_1 + x_2 \\ &\text{sujeta a } \quad \quad x_1 - x_2 + x_3 \quad \quad = 15 \\ &\quad \quad \quad \quad x_2 \quad \quad + x_4 = 15 \\ &\quad \quad \quad \quad x_1, x_2, x_3, x_4 \geq 0. \end{aligned}$$

El dual de este problema es, como es sabido:

$$\begin{aligned} &\text{maximizar } 15w_1 + 15w_2 \\ &\text{sujeta a } \quad \quad w_1 \quad \quad + s_1 = -2 \\ &\quad \quad -w_1 + \quad w_2 + s_2 = 1 \\ &\quad \quad \quad \quad w_1 \quad \quad + s_3 = 0 \\ &\quad \quad \quad \quad w_2 + s_4 = 0 \\ &\quad \quad \quad \quad s_1, s_2, s_3, s_4 \geq 0. \end{aligned}$$

Comencemos tomando como solución de partida $\mathbf{x}^0 = [1, 1, 1, 1]^T$, $\mathbf{w}^0 = [0, 0]^T$ y $\mathbf{s}^0 = [1, 1, 1, 1]^T$. Las matrices X_0 , S_0 y $\hat{D}_0^2$ son la identidad. El parámetro $\mu^0 = 1 \cdot \sigma$. Elegimos $\sigma = 0,09$

Tabla 10.5
Algoritmo primal-dual de puntos interiores

Paso 0 – *Inicialización.* Hacer $k = 0$ y definir una solución $[\mathbf{x}^0, \mathbf{w}^0, \mathbf{s}^0]^T$ tal que $\mathbf{x}^0 > \mathbf{0}$ y $\mathbf{s}^0 > \mathbf{0}$. Escoger unos números $\varepsilon_1, \varepsilon_2$ y ε_3 suficientemente pequeños y un $0 < \sigma < 1$.

Paso 1 – *Cálculos intermedios.* Calcular $\mu^k = \sigma (\mathbf{x}^k)^T \mathbf{s}^k / n$, $\mathbf{t}^k = \mathbf{b} - A\mathbf{x}^k$, $\mathbf{u}^k = \mathbf{c} - A^T \mathbf{w}^k - \mathbf{s}^k$, $\mathbf{v}^k = \mu^k \mathbf{e} - X_k S_k \mathbf{e}$, $\mathbf{p}^k = X_k^{-1} \mathbf{v}^k$ y $\hat{D}_k^2 = X_k S_k^{-1}$, donde X_k y S_k son matrices diagonales cuyos coeficientes son los x_i^k y s_i^k , respectivamente.

Paso 2 – *Comprobación de solución óptima.* Si

$$\mu^k < \varepsilon_1, \quad \frac{\|\mathbf{t}\|}{\|\mathbf{b}\| + 1} < \varepsilon_2 \quad \text{y} \quad \frac{\|\mathbf{u}\|}{\|\mathbf{c}\| + 1} < \varepsilon_3,$$

PARAR. La solución conseguida es la óptima. Si no, ir al siguiente paso.

Paso 3 – *Cálculo de las direcciones de movimiento.* Calcular

$$\begin{aligned} \mathbf{d}_w^k &= \left(A \hat{D}_k^2 A^T \right)^{-1} \left(A \hat{D}_k^2 (\mathbf{u}^k - \mathbf{p}^k) + \mathbf{t}^k \right) \\ \mathbf{d}_s^k &= \mathbf{u}^k - A^T \mathbf{d}_w^k \\ \mathbf{d}_x^k &= \hat{D}_k^2 (\mathbf{p}^k - \mathbf{d}_s^k). \end{aligned}$$

Paso 4 – *Comprobación de problema no acotado.* Si $\mathbf{t}^k = \mathbf{0}$, $\mathbf{d}_x^k > \mathbf{0}$ y $\mathbf{c}^T \mathbf{d}_x^k < 0$ el problema primal no está acotado. Si $\mathbf{u}^k = \mathbf{0}$, $\mathbf{d}_s^k > \mathbf{0}$ y $\mathbf{b}^T \mathbf{d}_w^k > 0$, es el dual el que no está acotado. En cualquiera de estos dos casos, PARAR. Si no seguir con el procedimiento.

Paso 5 – *Cálculo de la amplitud del paso.* Calcular los pasos β_p y β_d :

$$\beta_p^k = \frac{1}{\max \{1, -d_{x_i}^k / \alpha x_i^k\}}$$

y

$$\beta_d^k = \frac{1}{\max \{1, -d_{s_i}^k / \alpha s_i^k\}},$$

donde $\alpha < 1$, por ejemplo 0,99.

Paso 6 – *Obtener el nuevo punto.* Realizar la traslación al nuevo punto:

$$\begin{aligned} \mathbf{x}^{k+1} &\leftarrow \mathbf{x}^k + \beta_p \mathbf{d}_x^k \\ \mathbf{w}^{k+1} &\leftarrow \mathbf{w}^k + \beta_d \mathbf{d}_w^k \\ \mathbf{s}^{k+1} &\leftarrow \mathbf{s}^k + \beta_d \mathbf{d}_s^k. \end{aligned}$$

Hacer $k \leftarrow k + 1$ e ir al paso 1.

Calculemos $\mathbf{t}^0$, $\mathbf{u}^0$, $\mathbf{v}^0$ y $\mathbf{p}^0$:

$$\begin{aligned}\mathbf{t}^0 &= \mathbf{b} - A\mathbf{x}^0 = \begin{bmatrix} 15 \\ 15 \end{bmatrix} - \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 14 \\ 13 \end{bmatrix} \\ \mathbf{u}^0 &= \mathbf{c} - A^T \mathbf{w}^0 - \mathbf{s}^0 = \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ 0 \\ -1 \\ -1 \end{bmatrix} \\ \mathbf{v}^0 &= \mu^0 \mathbf{e} - X_0 S_0 \mathbf{e} = 0,09 \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0,91 \\ -0,91 \\ -0,91 \\ -0,91 \end{bmatrix} \\ \mathbf{p}^0 &= X_0^{-1} \mathbf{v}^0 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -0,91 \\ -0,91 \\ -0,91 \\ -0,91 \end{bmatrix} = \begin{bmatrix} -0,91 \\ -0,91 \\ -0,91 \\ -0,91 \end{bmatrix}.\end{aligned}$$

Iteración 1

Determinemos la dirección de movimiento:

$$\begin{aligned}\mathbf{d}_w^0 &= (A\hat{D}_0^2 A^T)^{-1} (A\hat{D}_k^2 (\mathbf{u}^0 - \mathbf{p}^0) + \mathbf{t}^0) = \\ &\left(\begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \right)^{-1} \left(\begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \right. \\ &\left. \left(\begin{bmatrix} -3 \\ 0 \\ -1 \\ -1 \end{bmatrix} - \begin{bmatrix} -0,91 \\ -0,91 \\ -0,91 \\ -0,91 \end{bmatrix} \right) + \begin{bmatrix} 14 \\ 13 \end{bmatrix} \right) = \begin{bmatrix} 7,1280 \\ 10,4740 \end{bmatrix}; \\ \mathbf{d}_s^0 &= \mathbf{u}^0 - A^T \mathbf{d}_w^0 = \begin{bmatrix} -3 \\ 0 \\ -1 \\ -1 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 7,1280 \\ 10,4740 \end{bmatrix} = \begin{bmatrix} -10,1280 \\ -3,3460 \\ -8,1280 \\ -11,4740 \end{bmatrix} \quad \text{y} \\ \mathbf{d}_x^0 &= \hat{D}_0^2 (\mathbf{p}^0 - \mathbf{d}_s^0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \left(\begin{bmatrix} -0,91 \\ -0,91 \\ -0,91 \\ -0,91 \end{bmatrix} - \begin{bmatrix} -10,1280 \\ -3,3460 \\ -8,1280 \\ -11,4740 \end{bmatrix} \right) = \begin{bmatrix} 9,2180 \\ 2,4360 \\ 7,2180 \\ 10,5640 \end{bmatrix}.\end{aligned}$$

Como $\mathbf{t}^0 > \mathbf{0}$ el programa primal no es todavía factible por lo que se continúa el procedimiento.

Si se elige un $\alpha = 0,99$ y se calculan las amplitudes de paso se obtiene que:

$$\beta_p^k = \frac{1}{\max \left\{ 1, -d_{x_i}^k / \alpha x_i^k \right\}} = 1,0$$

y

$$\beta_d^k = \frac{1}{\max \left\{ 1, -d_{s_i}^k / \alpha s_i^k \right\}} = \frac{1}{11,4740/0,99} = 0,086282.$$

Las nuevas soluciones en $k + 1 = 1$ son:

$$\begin{aligned} \mathbf{x}^1 &\leftarrow \mathbf{x}^0 + \beta_p \mathbf{d}_x^0 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + 1,0 \begin{bmatrix} 9,2180 \\ 2,4360 \\ 7,2180 \\ 10,5640 \end{bmatrix} = \begin{bmatrix} 10,2045 \\ 3,4090 \\ 8,2045 \\ 11,5910 \end{bmatrix}; \\ \mathbf{w}^1 &\leftarrow \mathbf{w}^0 + \beta_d \mathbf{d}_w^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0,086282 \begin{bmatrix} 7,1280 \\ 10,4740 \end{bmatrix} = \begin{bmatrix} 0,61462 \\ 0,90442 \end{bmatrix} \quad y \\ \mathbf{s}^1 &\leftarrow \mathbf{s}^0 + \beta_d \mathbf{d}_s^0 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + 0,086282 \begin{bmatrix} -10,1280 \\ -3,3460 \\ -8,1280 \\ -11,4740 \end{bmatrix} = \begin{bmatrix} 0,12865 \\ 0,71019 \\ 0,29981 \\ 0,01000 \end{bmatrix}. \end{aligned}$$

La nueva solución es factible en el primal.

Iteración 2 y sucesivas

Si se sigue iterando, con un valor de $\sigma = 0,09$, las soluciones que se obtienen son las de la tabla 10.6. El valor de la solución es $\mathbf{x} = [30, 15, 0, 0]^T$ y el óptimo de la función objetivo -45 . ●

Tabla 10.6

Proceso de convergencia del algoritmo primal-dual de puntos interiores aplicado al ejemplo 10.6

k	$\mathbf{x}^k$				$\mathbf{w}^k$		$\mathbf{s}^k$			
2	18,78860	3,87065	0,08204	11,12935	0,21206	0,72874	0,00129	0,48331	0,52572	0,00904
3	21,10035	6,10117	0,00082	8,89883	-0,50512	0,49004	0,00088	0,00483	1,00371	0,00854
4	29,91021	14,91101	0,00081	0,08899	-0,52090	0,47905	0,00062	0,00005	0,00081	0,08899
5	29,99832	14,99911	0,00079	0,00089	-1,69004	-0,69004	0,00003	0,00000	1,79337	0,79337
6	29,99999	14,99999	0,00000	0,00000	-2,00000	-1,00000	0,00000	0,00000	2,00000	1,00000

10.6.3 Solución de partida del método

Para poder aplicar el método se parte de un punto cualquiera $[\mathbf{x}^0, \mathbf{w}^0, \mathbf{s}^0] \in \mathcal{R}^{n+m+n}$ tal que $\mathbf{x}^0 > \mathbf{0}$ y $\mathbf{s}^0 > \mathbf{0}$.

Si se cumple que $A\mathbf{x}^0 = \mathbf{b}$ y $A^T\mathbf{w}^0 + \mathbf{s}^0 = \mathbf{c}$, entonces $\mathbf{x}^0 \in S$ y $[\mathbf{w}^0, \mathbf{s}^0]^T \in T$ por lo que ese punto se puede utilizar como solución de partida del método primal-dual. Si no es así, se considera el par de programas primal y dual siguiente:

$$\begin{aligned}
 (AP) \quad & \text{minimizar } \mathbf{c}^T \mathbf{x} + \pi x_{n+1} \\
 & \text{sujeta a} \quad A\mathbf{x} + (\mathbf{b} - A\mathbf{x}^0) x_{n+1} = \mathbf{b} \\
 & \quad \quad \quad \left(A^T\mathbf{w}^0 + \mathbf{s}^0 - \mathbf{c}\right)^T \mathbf{x} + x_{n+2} = \lambda \\
 & \quad \quad \quad \mathbf{x} \geq \mathbf{0}, x_{n+1}, x_{n+2} \geq 0,
 \end{aligned}$$

donde x_{n+1} y x_{n+2} son dos variables artificiales y π y λ dos números positivos suficientemente grandes; y

$$\begin{aligned}
 (AD) \quad & \text{maximizar } \mathbf{b}^T \mathbf{w} + \lambda w_{m+1} \\
 & \text{sujeta a} \quad A^T \mathbf{w} + \left(A^T \mathbf{w}^0 - \mathbf{s}^0 - \mathbf{c}\right) w_{m+1} + \mathbf{s} = \mathbf{c} \\
 & \quad \quad \quad (\mathbf{b} - A\mathbf{x}^0)^T \mathbf{w} + s_{n+1} = \pi \\
 & \quad \quad \quad w_{m+1} + s_{n+2} = 0 \\
 & \quad \quad \quad \mathbf{s} \geq \mathbf{0}, s_{n+1}, s_{n+2} \geq 0,
 \end{aligned}$$

donde w_{m+1} , s_{n+1} y s_{n+2} son variables artificiales.

Si se escogen unos π y λ tales que

$$\begin{aligned}
 \pi &> (\mathbf{b} - A\mathbf{x}^0)^T \mathbf{w}^0 \\
 \lambda &> \left(A^T \mathbf{w}^0 + \mathbf{s}^0 - \mathbf{c}\right)^T \mathbf{x}^0,
 \end{aligned}$$

entonces $[\mathbf{x}^0, x_{n+1}^0, x_{n+2}^0]^T$ y $[\mathbf{w}^0, w_{m+1}^0, \mathbf{s}^0, s_{n+1}^0, s_{n+2}^0]^T$ son soluciones factibles de los problemas (AP) y (AD) , respectivamente, donde

$$\begin{aligned}
 x_{n+1}^0 &= 1 \\
 x_{n+2}^0 &= \lambda - \left(A^T \mathbf{w}^0 + \mathbf{s}^0 - \mathbf{c}\right)^T \mathbf{x}^0 \\
 w_{m+1}^0 &= -1 \\
 s_{n+1}^0 &= \pi - (\mathbf{b} - A\mathbf{x}^0)^T \mathbf{w}^0 \\
 s_{n+2}^0 &= 1.
 \end{aligned}$$

En este caso el algoritmo primal-dual que acabamos de analizar se puede comenzar a aplicar al par (AP) – (AD) con una solución de partida factible conocida.

Teorema 10.1 Sean $\mathbf{x}^*$ y $[\mathbf{w}^*, \mathbf{s}^*]^T$ las soluciones óptimas de los problemas primal y dual originales (10.29) y (10.30). Si además de cumplirse que

$$\begin{aligned}\pi &> (\mathbf{b} - A\mathbf{x}^0)^T \mathbf{w}^0 \\ \lambda &> (A^T \mathbf{w}^0 + \mathbf{s}^0 - \mathbf{c})^T \mathbf{x}^0,\end{aligned}$$

se supone que

$$\begin{aligned}\lambda &> (A^T \mathbf{w}^0 + \mathbf{s}^0 - \mathbf{c})^T \mathbf{x}^* \\ \pi &> (\mathbf{b} - A\mathbf{x}^0)^T \mathbf{w}^*,\end{aligned}$$

entonces se verifica que:

- (i) Una solución factible $[\bar{\mathbf{x}}, \bar{x}_{n+1}, \bar{x}_{n+2}]^T$ de (AP) es óptima si y sólo si $\bar{\mathbf{x}}$ es el óptimo del programa primal original (10.29) y $\bar{x}_{n+1} = 0$.
- (ii) Una solución factible $[\bar{\mathbf{w}}, \bar{w}_{m+1}, \bar{\mathbf{s}}, \bar{s}_{n+1}, \bar{s}_{n+2}]^T$ de (AD) es óptima si y sólo si $[\bar{\mathbf{w}}, \bar{\mathbf{s}}]^T$ es el óptimo del programa dual original (10.30) y $\bar{w}_{m+1} = 0$.

DEMOSTRACIÓN. Como la solución $\mathbf{x}^*$, además de óptima, también es factible del programa primal original (10.29), si se hace que $x_{n+1}^* = 0$ y $x_{n+2}^* = \lambda - (A^T \mathbf{w}^0 + \mathbf{s}^0 - \mathbf{c})^T \mathbf{x}^*$, entonces $[\mathbf{x}^*, x_{n+1}^*, x_{n+2}^*]^T$ es factible en (AP). Supongamos que $[\mathbf{x}, x_{n+1}, x_{n+2}]^T$ es factible en (AP) con $x_{n+1} > 0$, se cumple entonces que

$$\mathbf{c}^T \mathbf{x}^* + \pi x_{n+1}^* = \mathbf{w}^{*T} \mathbf{b} = \mathbf{w}^{*T} (A\mathbf{x} + (\mathbf{b} - A\mathbf{x}^0) x_{n+1}).$$

Obsérvese que $A^T \mathbf{w}^* + \mathbf{s}^* = \mathbf{c}$, $x_{n+1} > 0$ y $\pi > (\mathbf{b} - A\mathbf{x}^0)^T \mathbf{w}^*$. Entonces

$$\mathbf{c}^T \mathbf{x}^* + \pi x_{n+1}^* < (\mathbf{c} - \mathbf{s}^*)^T \mathbf{x} + \pi x_{n+1} \leq \mathbf{c}^T \mathbf{x} + \pi x_{n+1},$$

pues $\mathbf{s}^{*T} \mathbf{x} \geq 0$. Esto quiere decir que $[\mathbf{x}, x_{n+1}, x_{n+2}]^T$ no puede ser la solución óptima de (AP) a menos que $x_{n+1} = 0$. Ahora bien, debido a la continuidad, $[\mathbf{x}^*, x_{n+1}^*, x_{n+2}^*]^T$ es una solución óptima de (AP). Por consiguiente, si una solución factible $[\bar{\mathbf{x}}, \bar{x}_{n+1}, \bar{x}_{n+2}]^T$ de (AP) es óptima, entonces $\bar{x}_{n+1} = 0$ y $\mathbf{c}^T \bar{\mathbf{x}} = \mathbf{c}^T \mathbf{x}^*$. Como $\bar{\mathbf{x}}$ satisface todas las restricciones del problema primal original, (10.29), debe ser su solución óptima.

Si, por el contrario, $[\bar{\mathbf{x}}, 0, \bar{x}_{n+2}]^T$ es una solución factible de (AP) y $\bar{\mathbf{x}}$ una solución óptima de (10.29), el valor de la función objetivo $\mathbf{c}^T \bar{\mathbf{x}} + \pi \bar{x}_{n+1}$ coincide con el valor mínimo de $\mathbf{c}^T \mathbf{x}^* + \pi x_{n+1}^*$, siendo por tanto $[\bar{\mathbf{x}}, 0, \bar{x}_{n+2}]^T$ una solución óptima de (AP). Esto concluye la demostración del punto (i). La demostración del punto (ii) se lleva a cabo de manera similar. ■

10.6.4 Complejidad computacional del método

El método primal-dual es un algoritmo de complejidad computacional polinómica. Si se escoge adecuadamente el parámetro β^k de cada iteración, se verifica el siguiente resultado.

Teorema 10.2 Si la amplitud de paso de cada iteración $\beta^k < 1$, entonces

$$\beta^k \geq \frac{4(\sigma - \tau)}{n(1 - 2\sigma + \theta^k \sigma^2)} \geq \frac{4(\sigma - \tau)}{n(1 + \sigma^2)\theta^k} \quad (10.39a)$$

$$\mathbf{c}^T \mathbf{x}^{k+1} - \mathbf{b}^T \mathbf{w}^{k+1} = \left(1 - (1 - \sigma)\beta^k\right) \left(\mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k\right) \quad (10.39b)$$

$$\theta^{k+1} - \sigma/\tau \leq (1 - \nu) \left(\theta^k - \sigma/\tau\right) \quad \text{si } \sigma/\tau < \theta^k \quad (10.39c)$$

$$\theta^{k+1} \leq \sigma\tau \quad \text{si } \theta^k \leq \sigma/\tau, \quad (10.39d)$$

donde

$$\nu = \frac{4(\sigma - \tau)\tau}{n(1 + \sigma^2) + 4(\sigma - \tau)\tau}.$$

Además, si $\beta^k = 1$,

$$\mathbf{c}^T \mathbf{x}^{k+1} - \mathbf{b}^T \mathbf{w}^{k+1} = \sigma \left(\mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k\right)$$

y

$$\theta^{k+1} \leq \sigma/\tau.$$

De acuerdo con (10.39b), el *duality gap*, $\mathbf{c}^T \mathbf{x}^k - \mathbf{b}^T \mathbf{w}^k$, alcanza una precisión ε en

$$O\left(n \ln \left(\frac{\mathbf{c}^T \mathbf{x}^0 - \mathbf{b}^T \mathbf{w}^0}{\varepsilon}\right)\right)$$

iteraciones. Es decir, el algoritmo termina en, como mucho,

$$O\left(n \ln \theta^0\right) + O\left(n \ln \left(\frac{\mathbf{c}^T \mathbf{x}^0 - \mathbf{b}^T \mathbf{w}^0}{\varepsilon}\right)\right)$$

iteraciones.

Para que se cumplan estas predicciones hay varias formas de definir los parámetros utilizados. En concreto, se pueden hacer, $0 \leq \tau < \sigma < 1$. Por ejemplo, $\sigma = 1/2$, $\tau = 1/4$ y $\beta^k \geq 4/n\theta^k$.

Referencias

Todo el material que se expone en este capítulo es muy reciente. Sigue esencialmente a Fletcher [1987], Gill, Murray, Saunders, Tomlin y Wright [1986], Goldfarb y Todd [1989], Bazaraa, Jarvis y Sherali [1990], Arbel [1993], Padberg [1995], Fang y Puthenpura [1993], Vanderbei [1996] y Wright [1997]. También es útil consultar Mehrotra [1992] y Lustig, Marsten y Shanno [1992].

Dos de las direcciones más interesantes de INTERNET donde se listan códigos de programación lineal que utilizan técnicas de puntos interiores son las que siguen.

<http://www.siam.org/books/swright/>

<http://www.princeton.edu/~rvdb/LPbook/>

También puede resultar útil consultar de vez en cuando información sobre el estado del arte en la tecnología de puntos interiores para optimización. A este respecto, el *Argonne National Laboratory*, de los Estados Unidos, mantiene continuamente actualizada información en la siguiente dirección.

<http://www.mcs.anl.gov/home/otc/InteriorPoint/>

Ejercicios

10.1. En el espacio euclídeo n -dimensional:

- Dado un punto del simplex Δ , ¿cómo se puede saber si es un punto extremo de Δ ? ¿Y si está en una arista del mismo? ¿Y si está en el interior de él? ¿Y si está en el centro?
- Probar que el simplex Δ posee n puntos extremos y $C(n, 2)$ aristas.
- Probar que la distancia desde el centro del simplex a cualquiera de los puntos extremos de Δ es

$$R = \sqrt{\frac{n-1}{n}}$$

y que la distancia del centro a cualquiera de los lados del simplex es

$$r = \frac{1}{\sqrt{n(n-1)}}.$$

10.2. Probar que la función

$$\sum_{j=1}^n \ln x_j$$

alcanza su valor máximo en

$$\mathbf{x}^* = \frac{\mathbf{e}}{n} \quad \text{para } \mathbf{x} \in \Delta.$$

10.3. Si se dispone de dos algoritmos, A y B, el primero para resolver sistemas de ecuaciones lineales y el segundo para optimizar programas lineales:

- ¿Cómo se puede usar el algoritmo A para resolver un problema de programación lineal?
- ¿Cómo se puede usar el algoritmo B para resolver un sistema de ecuaciones lineales?
- Combinando a) y b), ¿cuál es la conclusión? ¿Por qué?

10.4. Considérese el siguiente problema de programación lineal:

$$\begin{aligned} &\text{minimizar } -x_1 + 1 \\ &\text{sujeta a} \quad \begin{aligned} x_3 - x_4 &= 0 \\ x_1 + x_2 + x_3 + x_4 &= 1 \\ x_1, x_2, x_3, x_4 &\geq 0. \end{aligned} \end{aligned}$$

- Dibujar la región factible de este problema. Teniendo en cuenta que el punto $[0, 0, 1/2, 1/2]^T$ es un punto extremo, mediante el método *simplex*, determinar desde ese punto extremo cuál sería la dirección que habría que tomar para mejorar la función objetivo y representarla en el gráfico.

- b) Si $[0,01, 0,01, 0,49, 0,49]^T$ es un punto factible en el interior de la región factible y próximo al punto extremo de a), utilizar el algoritmo de Karmarkar para determinar la dirección de movimiento desde este punto y representarla en el gráfico.
- c) Utilizar el algoritmo primal de escalado afín para determinar la dirección de movimiento desde $[0,01, 0,01, 0,49, 0,49]^T$ y representarla en el gráfico.
- d) Utilizar el algoritmo primal de escalado afín con función barrera logarítmica para determinar la dirección de movimiento desde el mismo punto y representarla en el gráfico.
- e) Comparar las direcciones obtenidas en los puntos a) a d). ¿Qué comentarios se pueden hacer al respecto? ¿Por qué?

10.5. Volviendo al problema del ejercicio anterior:

- a) Obtener el dual del problema planteado y dibujar su región factible.
- b) Probar que $[1, -2]^T$ es un punto en el interior de esa región factible.
- c) Aplicar el método dual de escalado afín para determinar la dirección de movimiento desde ese punto y representarla en el gráfico dibujado.
- d) ¿Se dirige esa dirección hacia el óptimo del programa dual?
- e) Aplicar el algoritmo dual de escalado afín con función barrera logarítmica para determinar la dirección de movimiento desde el punto considerado y representarla sobre el gráfico de la región factible.
- f) ¿Es mejor la dirección que se calcula en e) que la de c)? ¿Por qué?

10.6. Considérese otra vez el ejercicio 4:

- a) Partiendo de la solución $\mathbf{x} = [0,01, 0,01, 0,49, 0,49]^T$, factible en el programa primal, y de $\mathbf{w} = [1, -2]^T$, factible en el dual, aplicar el método primal-dual y calcular la dirección de movimiento.
- b) Representar $\mathbf{d}_x$ y $\mathbf{d}_w$ sobre los correspondientes gráficos.
- c) ¿Qué se puede observar en esa representación?

10.7. Considérese el siguiente problema de programación lineal:

$$\begin{aligned} &\text{minimizar } \mathbf{c}^T \mathbf{x} \\ &\text{sujeta a } \quad \mathbf{A} \mathbf{x} = \mathbf{b} \\ &\quad \quad \mathbf{x} \geq \mathbf{q}, \end{aligned}$$

donde A es una matriz $m \times n$ de rango completo y $\mathbf{q} \in \mathbb{R}^n$.

- a) Convertir este problema a forma estándar considerando sólo n variables.
- b) Determinar el dual del programa del punto anterior. Probar que cuando $\mathbf{q} = \mathbf{0}$ se obtiene un programa dual que podemos considerar normal.
- c) Teniendo como objetivo desarrollar un procedimiento de puntos interiores para resolver este problema, y siguiendo la estrategia de transformar un punto en el interior de su región factible $\mathbf{x}^k$, tal que $\mathbf{A} \mathbf{x}^k = \mathbf{b}$ y $\mathbf{x}^k > \mathbf{q}$, en otro centrado en el primer octante de $\mathbb{R}^n$, es decir, en $\mathbf{e} = [1, \dots, 1]^T$:
 - i) Determinar la transformación que nos permite conseguir lo indicado y probar que es biyectiva entre los conjuntos $\{\mathbf{x} \in \mathbb{R}^n : \mathbf{x} \geq \mathbf{q}\}$ y $\{\mathbf{y} \in \mathbb{R}^n : \mathbf{y} \geq \mathbf{0}\}$.
 - ii) Escribir el programa lineal resultante en el espacio transformado.

- iii) Proyectar en el espacio transformado el negativo del gradiente de la función objetivo en el subespacio núcleo de la matriz de condiciones. ¿Cuál es la dirección de movimiento?
- iv) Determinar la dirección de movimiento en el espacio original.
- v) Aplicar el método primal de escalado afín al problema lineal en forma estándar de a).
- vi) Siguiendo con lo empezado en iii), ¿cómo se escoge una amplitud de paso apropiada para seguir conservando la factibilidad del problema?
- vii) Deducir una fórmula para calcular el nuevo punto del proceso iterativo que estamos desarrollando.
- viii) ¿Qué reglas se pueden utilizar para parar el método?
- ix) ¿Cómo se puede actuar para determinar un buen punto de partida del método?
- x) Escribir detalladamente el algoritmo completo a que daría lugar la forma de actuar que se ha seguido en este ejercicio para resolver un problema de programación lineal con límite inferior en sus variables.

10.8. Considérese el algoritmo primal de escalado afín con función barrera logarítmica. Defínase como P_{AX_k} la proyección en el espacio núcleo de la matriz AX_k , $\ker(AX_k)$, y pruébese que la dirección de movimiento desde un punto $\mathbf{x}^k$ está dada por la siguiente expresión:

$$\mathbf{d}_{\mu}^k = -X_k P_{AX_k} \left(\frac{X_k \mathbf{c}}{\mu^k} - \mathbf{e} \right).$$

Tercera parte

Programación entera

Capítulo 11

PROGRAMACIÓN LINEAL EN VARIABLES ENTERAS

EN ESTE CAPÍTULO comenzamos el estudio de una rama de la programación matemática cuyas áreas de aplicación abarcan muchas disciplinas de gran interés: la programación entera. La *programación entera* trata los problemas de maximizar o minimizar una función de diversas variables sujeta a condiciones de igualdad y/o desigualdad, restringiéndose todas o alguna de esas variables a tomar valores enteros.

Desde que Ralph Gomory, al final de los años 50 y comienzos de los 60, iniciase sus trabajos en esta rama de la optimización, muchas han sido las mejoras introducidas en sus algoritmos característicos y amplísima la diversificación experimentada en su campo de aplicación. Entre las áreas de aplicación actuales cabe citar: distribución de mercancías, programación de la producción en factorías, secuenciación de maquinaria en procesos industriales y productivos, asignación de grupos generadores de energía eléctrica, asignaciones presupuestarias, localización de elementos productivos, diseño de circuitos electrónicos, procesos de manufactura flexible, etc. En general, en todas aquellas donde se trata de resolver el problema de asignar recursos de cualquier tipo sólo disponibles en cantidades discretas.

Las aplicaciones en el campo de la matemática aplicada también son muy variadas: teoría de grafos, combinatoria, lógica, etc. De igual manera, otras áreas del conocimiento como la estadística (análisis de datos), biología molecular, física (determinación de estados de mínima energía, rayos X), criptografía (diseño de códigos), cristalografía, etc, incluso la política (diseño de circunscripciones electorales), se benefician de los avances de esta rama de la programación matemática.

La formulación general del problema que estudia la programación entera es la siguiente:

$$\begin{aligned} & \underset{\mathbf{x} \in \mathbf{Z}^n, \mathbf{y} \in \mathbb{R}^p}{\text{maximizar}} && \mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y} \\ & \text{sujeta a} && A\mathbf{x} + G\mathbf{y} \leq \mathbf{b} \\ & && \mathbf{x}, \mathbf{y} \geq \mathbf{0}. \end{aligned} \tag{11.1}$$

Este problema también se conoce como *programa entero mixto* pues algunas de las variables involucradas son enteras y otras continuas.

Análogamente a como hacíamos en programación lineal, al conjunto $S = \{\mathbf{x} \in \mathbf{Z}^n, \mathbf{y} \in \mathbb{R}^p : A\mathbf{x} + G\mathbf{y} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}, \mathbf{y} \geq \mathbf{0}\}$ se le denomina *región factible*. Un punto $[\mathbf{x}^T, \mathbf{y}^T]^T \in S$ se denomina factible. Como es lógico, al punto $[\mathbf{x}^{*T}, \mathbf{y}^{*T}]^T \in S$, tal que

$$\mathbf{c}^T \mathbf{x}^* + \mathbf{h}^T \mathbf{y}^* \geq \mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y}, \quad \text{para todo } [\mathbf{x}^T, \mathbf{y}^T]^T \in S,$$

se le denomina solución óptima del problema (11.1).

Un programa en variables enteras también se puede definir de la siguiente manera:

$$\begin{aligned} & \underset{\mathbf{x} \in \mathbf{Z}^n}{\text{max.}} && \mathbf{c}^T \mathbf{x} \\ & \text{s. a} && A\mathbf{x} \leq \mathbf{b} \\ & && \mathbf{x} \geq \mathbf{0}. \end{aligned}$$

Tal programa se conoce también como *programa entero puro*, programa lineal en variables enteras o, simplemente, *programa entero*.

Un caso particular de programa entero es aquel en el que las variables sólo pueden tomar valores 0 ó 1. Otro, el *programa combinatorio*. Éste, en términos genéricos, se puede formular como sigue:

Dado un conjunto finito $N = \{1, \dots, n\}$ y un vector n -dimensional $\mathbf{c}^T = [c_1, \dots, c_n]$, si para un conjunto $F \subseteq N$ se define $d(F) = \sum_{j \in F} c_j$ y una colección de subconjuntos $\mathcal{F}$ de N , se trata de

$$\underset{F \in \mathcal{F}}{\text{maximizar}} d(F).$$

En lo que sigue de capítulo nos centraremos en los aspectos más prácticos de la programación entera: en aquellos orientados a resolver de la forma más eficaz posible los programas enteros que se pueden plantear. En programación entera, desgraciadamente, no existe la contrapartida del método *simplex* de programación lineal: no existe un método universal que, partiendo y sirviéndose de las propiedades de convexidad del problema a resolver, llegue a una solución de éste. Esto es así, sencillamente, porque en programación entera la convexidad desaparece, no pudiéndose utilizar por tanto la noción de gradiente para caracterizar y buscar el óptimo de un problema, haciéndose necesario emplear métodos de resolución específicos del aspecto combinatorio de las variables enteras.

Los métodos que analizaremos en éste capítulo y en el siguiente son los que, en principio, se pueden aplicar a cualquier problema de programación entera:

- El de los truncamientos o planos cortantes de Gomory. Basado en profundos aspectos teóricos pero que muy a menudo consiguen prestaciones numéricas decepcionantes (los analizamos, no obstante, por contribuir en gran medida a profundizar en el conocimiento del problema).
- Los de enumeración implícita. También denominados de ramificación y acotamiento (*Branch and Bound*), basados en principios muy simples, que se distinguen por ser los más eficaces para problemas de todo tipo.

Existen muchos métodos orientados a la resolución de problemas enteros especiales que aumentan de forma considerable las prestaciones de los generales; todos ellos, no obstante, están limitados por los aspectos combinatorios del problema a resolver, por lo que todavía tratar problemas con varios miles de variables conlleva mucho tiempo de ordenador.

11.1 Formulación y ejemplos de programas lineales en variables enteras

Uno de los problemas que más a menudo surge como programa entero es el de la planificación de nuevo equipamiento industrial, material o de recursos humanos. Pensemos, por ejemplo, en el problema con que se enfrenta el planificador de la flota de autobuses urbanos (o coches de metro) de una gran ciudad a la hora de prever las nuevas necesidades de autobuses para unos años futuros. Si los fabricantes de vehículos los facturan a unos precios determinados, con un mantenimiento necesario por año y unidad, y con una vida útil dada, el problema de tomar la decisión adecuada de mínimo coste sobre qué modelo de autobús elegir y cuántas unidades del mismo adquirir, es un problema típico de programación entera.

Gestión de un servicio hospitalario

En un determinado servicio de asistencia hospitalaria i enfermos están a la espera de una operación. Cada enfermo, i , necesita una operación de duración D_i . Habida cuenta de la disponibilidad de cirujanos, la suma de las duraciones de las operaciones que pueden tener lugar cada día del período de estudio, j , es igual a T_j . Se trata de minimizar una función económica suma de las penalizaciones por esperas de los diferentes enfermos (esta penalización es una función lineal creciente de la espera). El problema se formula de la siguiente manera:

$$\begin{aligned}
 &\max. \sum_i \sum_j c_{ij} x_{ij} \\
 &\text{s. a} \quad \sum_i D_i x_{ij} \leq T_j \quad \text{para todo } j \\
 &\quad \sum_j x_{ij} = 1 \quad \text{para todo } i \\
 &\quad x_{ij} = 0 \text{ ó } 1 \quad \text{para todo } i \text{ y } j.
 \end{aligned}$$

La variable de decisión x_{ij} es 1 si el enfermo i se le opera el día j y 0 en el caso contrario.

El problema de la mochila

Este problema, típico en programación entera, ya lo introdujimos al hablar del problema del corte de materiales. Se trata de, dado un transportista que dispone de un vehículo con capacidad de transporte b , en el que se pueden alojar diferentes objetos j de volúmenes a_j y valores c_j , maximizar el valor de lo transportado en cada viaje. Su formulación es:

$$\begin{aligned} \max. \quad & \sum_j c_j x_j \\ \text{s. a} \quad & \sum_j a_j x_j \leq b \\ & x_j = 0 \text{ ó } 1. \end{aligned}$$

Planificación de la generación de energía eléctrica de una empresa o país

Se trata de planificar cómo se va a hacer frente al crecimiento de la demanda de energía eléctrica durante T años, a partir de uno futuro que se considere oportuno. Si se designa como variables de decisión x_{lst} , la cantidad de potencia instalada que se demanda de un tipo de generación s —nuclear, carbón, fuel, etc.—, del nivel de demanda l el año t y y_{ist} , la cantidad de nueva potencia que el grupo i del tipo generador s puede aportar el año t —100 MW, 250 MW, 500 MW ó 1000 MW—, y se decide que como función objetivo se ha de considerar la de minimizar el coste total a valor presente de satisfacer la demanda total de energía eléctrica los T años del estudio, la formulación del problema a que da lugar este planteamiento es la que sigue:

$$\begin{aligned} \min. \quad & \sum_i \sum_s \sum_t \left(\sum_{t_1=t}^T \frac{1}{(1+\alpha)^{t_1}} \right) \left(\frac{1}{(1+\alpha)^{t-1}} C_{cap_{is}} + \sum_{t_1=t}^T \frac{1}{(1+\alpha)^{t_1}} C_{f_{is}} \right) y_{ist} \\ & + \sum_i \sum_l \sum_s \sum_t \frac{1}{(1+\alpha)^T} \left(\sum_{l_1=l}^L F_{l_1} \right) C_{v_{is}} z_{lst} \\ \text{s. a} \quad & \sum_{l=1}^L z_{lst} - \sum_i \sum_{t_1=1}^T y_{ist_1} \leq Q_{0s} \quad \text{para todo } s \text{ y } t, \\ & \sum_{l_1=1}^l \sum_s z_{l_1st} = D_{lt} \quad \text{para todo } l \text{ y } t, \\ & y_{ist} \text{ entera} \quad \text{para todo } i, s \text{ y } t, \end{aligned}$$

donde: $C_{cap_{is}}$ es el coste de capital de un nuevo grupo generador i del tipo s ;
 $C_{f_{is}}$ el coste fijo de operación y mantenimiento del nuevo grupo i del tipo s ;
 $C_{v_{is}}$ el coste variable del mismo grupo anterior s ;
 D_{lt} la demanda del nivel l el año t ;
 F_l la fracción del año en la cual el nivel de demanda se sitúa en l ;
 Q_{0s} la capacidad instalada al comienzo del estudio del tipo s ;
 α la tasa de descuento a aplicar en el estudio;

y donde, suponiendo que $x_{1st} \leq x_{2st} \leq \dots \leq x_{Lst}$, se ha hecho $z_{lst} = x_{lst} - x_{l-1st}$.

La complejidad del modelo definitivo a adoptar depende del número de tramos en que se divida la demanda, L , grupos de generación considerados, etc.

Dicotomías

Una dicotomía ocurre habitualmente en programación matemática en problemas donde hay condiciones del tipo *una o la otra*. La programación entera es un instrumento adecuado para resolverlas. Como ejemplo, supongamos que se plantea el problema de colorear un mapa —en televisión y telecomunicaciones se plantean a menudo variantes de este problema—, que se dispone sólo de cuatro niveles cromáticos, y que se trata de hacerlo de forma que dos regiones contiguas no tengan el mismo color. Sean $r = 1, 2, \dots, R$ las regiones del mapa a colorear y $t_r = 0, 1, 2$ y 3 la tonalidad que se les puede adjudicar. Se trata pues de que dos regiones, r y s , cumplan que

$$t_r - t_s \neq 0$$

y que

$$t_r - t_s \geq 1 \quad \text{o} \quad t_s - t_r \geq 1.$$

Si se introduce la variable entera δ_{rs} (0 ó 1), las relaciones

$$t_r - t_s \geq 1 - 4\delta_{rs}$$

y

$$t_s - t_r \geq -3 + 4\delta_{rs},$$

reemplazan la dicotomía.

En términos más generales, supongamos que en un programa matemático determinado se han de cumplir simultáneamente k de las p relaciones siguientes:

$$\begin{aligned} G_1(\mathbf{x}) &\geq 0 \\ G_2(\mathbf{x}) &\geq 0 \\ &\vdots \\ G_p(\mathbf{x}) &\geq 0, \end{aligned}$$

donde $\mathbf{x}$ está definido en una región de factibilidad S . Las alternativas las podemos reemplazar por:

$$\begin{aligned} G_1(\mathbf{x}) - \delta_1 L_1 &\geq 0 \\ G_2(\mathbf{x}) - \delta_2 L_2 &\geq 0 \\ &\vdots \\ G_p(\mathbf{x}) - \delta_p L_p &\geq 0, \end{aligned}$$

donde L_i es un límite inferior del valor de la función $G_i(\mathbf{x})$ en S y δ_i , (0 ó 1), cumple que

$$\delta_1 + \delta_2 + \dots + \delta_p = p - k.$$

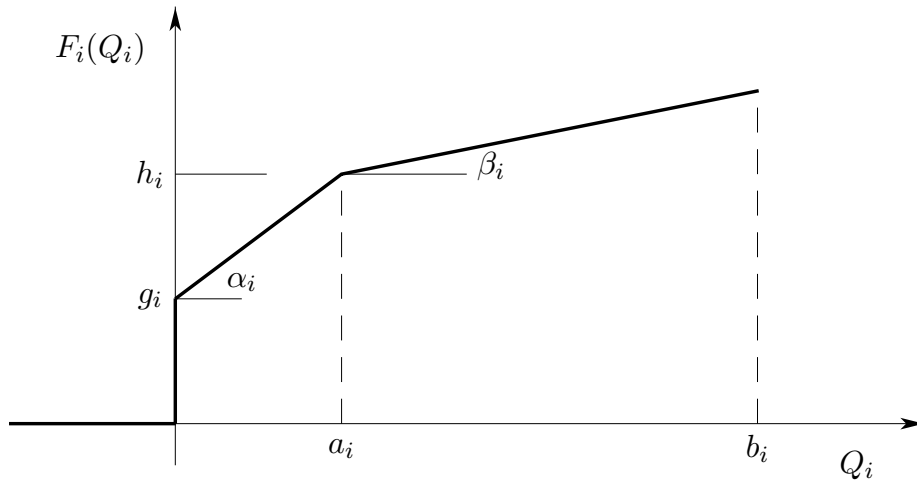
En el caso del problema de los cuatro colores anterior, $k = 1$, $p = 2$. Además, $L_r = -4$ y $L_s = -4$, lo que da como resultado:

$$\begin{aligned} t_r - t_s - 1 + 4\delta_{rs} &\geq 0 \\ t_s - t_r - 1 + 4\delta_{sr} &\geq 0, \end{aligned}$$

con $\delta_{rs} + \delta_{sr} = 1$.

Función objetivo no convexa. El problema de la localización de almacenes

Un producto se fabrica en unos determinados talleres y se distribuye a j clientes. Para el período de estudio se conoce la demanda de cada cliente D_j . La distribución directa por el fabricante desde un taller al cliente es demasiado costosa, por lo que se propone disponer de unos ciertos almacenes, i , más cerca de los lugares donde se encuentran los clientes. Si x_{ij} es la cantidad de producto depositado en el almacén i que se entrega al cliente j , su coste de distribución es d_{ij} . Por el contrario, si Q_i es la cantidad de producto transportada de un taller al almacén i , siendo allí depositada hasta que se distribuya, los gastos correspondientes al transporte, construcción del almacén y *stock* de esa cantidad de producto, son una función cóncava de Q_i cuya forma es la de la figura 11.1.

**Figura 11.1**

Función objetivo cóncava del problema de la localización de almacenes

El problema de satisfacer la demanda de los clientes a coste mínimo se formula de la siguiente manera:

$$\begin{aligned}
 \min. \quad & \sum_i (g_i Y_i + \alpha_i y_i + (h_i - \beta_i a_i) Z_i + \beta_i z_i) + \sum_i \sum_j d_{ij} x_{ij} \\
 \text{s. a} \quad & \sum_i x_{ij} = D_j \quad \text{para todo } j \\
 & \sum_j x_{ij} = y_i + z_i \quad \text{para todo } i \\
 & y_i \leq a_i Y_i \quad " \\
 & z_i \leq b_i Z_i \quad " \\
 & Y_i + Z_i = 1 \quad " \\
 & Y_i = 0 \text{ ó } 1 \quad " \\
 & Z_i = 0 \text{ ó } 1 \quad \text{para todo } i.
 \end{aligned}$$

La variable $Y_i = 0$ y la $Z_i = 0$ indican que no se construye el almacén i ; si $Y_i = 1$ y $Z_i = 0$ se construye el almacén i pero de un tamaño a_i y si $Y_i = 0$ y $Z_i = 1$ se construye el almacén de un tamaño comprendido entre a_i y b_i . La condición $Y_i + Z_i = 1$ expresa el hecho evidente de

que no se puede construir un mismo almacén de dos tamaños distintos. La cantidad depositada en cada almacén, Q_i , es $y_i + z_i$. La condición $y_i \leq a_i Y_i$ expresa que si $Y_i = 0$, $y_i = 0$, y que si $Y_i = 1$, $y_i \leq a_i$. La condición $z_i \leq b_i Z_i$ expresa lo mismo para el otro tamaño de almacén.

Problema con condiciones no lineales. Optimización de la operación de un parque de centrales térmicas.

La optimización de la operación de un parque de centrales térmicas para generar energía en un sistema eléctrico de generación y transporte, simplificada, se puede plantear como un problema de optimización entera con función objetivo lineal o no lineal y con condiciones lineales y no lineales.

Si la potencia que suministran al sistema los diferentes grupos de las centrales térmicas se designa por p_i , y por x_i la variable que indica si una central está en funcionamiento o no (0 ó 1), la formulación del problema apuntado da lugar a:

$$\begin{aligned} \min. \quad & \sum_i (a_i x_i + \alpha_i p_i) \\ \text{s. a} \quad & \mathbf{g}(\mathbf{p}) \geq \mathbf{0} \\ & m_i x_i \leq p_i \leq M_i x_i \\ & x_i = 0 \text{ ó } 1, \end{aligned}$$

donde la función vectorial $\mathbf{g}(\mathbf{p})$ es una expresión (no lineal) de las leyes de Kirchhoff. La función de costes de cada unidad térmica en relación con la potencia que puede suministrar tiene la forma que se indica en la figura 11.2. El coeficiente a_i corresponde a una producción nula (sólo pérdidas caloríficas); m_i es la potencia de mínimo técnico de generación por debajo de la cual la unidad no se acopla a la red de transporte y M_i la máxima potencia que puede suministrar.

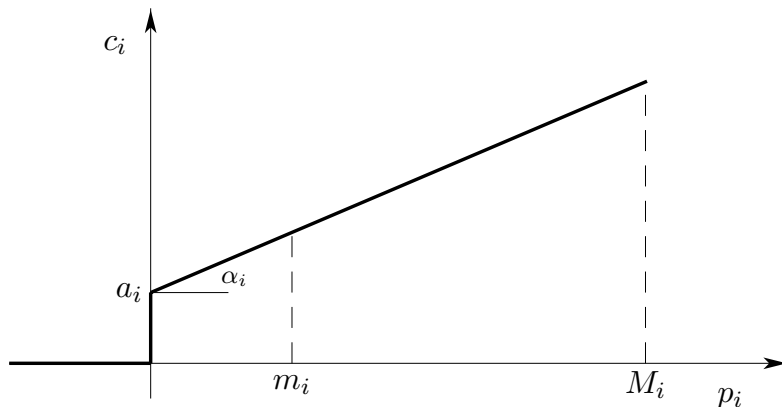


Figura 11.2

Función de costes de un grupo de una central térmica

11.1.1 Problemas de estructura especial

El problema del representante de comercio

Un representante de comercio parte de una ciudad 0 debiendo realizar un periplo de visitas que comprenda las ciudades $1, 2, \dots, n$, las cuales debe visitar una vez y nada más que una, regresando a su punto de origen 0. La distancia entre dos ciudades i y j es $d_{ij} > 0$. El problema consiste en determinar el itinerario de visitas de las ciudades de tal forma que se minimice la distancia total recorrida.

Si además de atribuir el número 0 a la ciudad de origen se le atribuye el $n + 1$, el problema consiste en visitar, partiendo de la ciudad 0, las n restantes ciudades y terminar en $n + 1$.

Para formular correctamente el problema se introducen las *variables binarias* x_{ij} , $i = 0, 1, \dots, n$, $j = 1, \dots, n + 1$, de tal forma que $x_{ij} = 1$ si el representante va de la ciudad i a la j , y $x_{ij} = 0$ en caso contrario. Garantizar que cada ciudad sólo se visita una vez (excepto la 0) se puede conseguir introduciendo la siguiente condición en el problema:

$$\sum_{i=0}^n x_{ij} = 1 \quad j = 1, \dots, n + 1, i \neq j.$$

De igual forma, asegurar que cada ciudad (excepto la $n + 1$) se abandona una sola vez se consigue introduciendo la condición

$$\sum_{j=1}^{n+1} x_{ij} = 1 \quad j = 0, 1, \dots, n, i \neq j.$$

Estas condiciones no evitan, sin embargo, que puedan darse los bucles como los que se indican en la figura 11.3, donde la solución es $x_{01} = x_{12} = x_{26} = 1$, $x_{34} = x_{45} = x_{53} = 1$ y $x_{ij} = 0$ para las demás variables. Para eliminar estos bucles se añaden a la formulación del problema las siguientes condiciones:

$$\alpha_i - \alpha_j + (n + 1)x_{ij} \leq n, \quad i = 0, 1, \dots, n, j = 1, \dots, n + 1, i \neq j,$$

donde α_i es un número real asociado a la ciudad i .

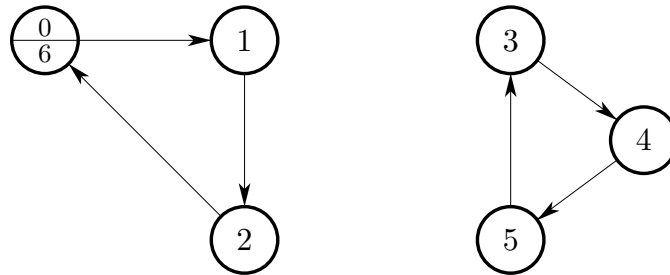


Figura 11.3

Bucles en el problema del representante de comercio

Para comprobar que una solución que contenga bucles no satisface estas últimas condiciones, consideremos un bucle cualquiera que no contenga la ciudad origen/destino. Si sumamos las

desigualdades correspondientes a las variables $x_{ij} = 1$ alrededor del bucle, los $\alpha_i - \alpha_j$ se hacen cero quedando la desigualdad $(n+1)N \leq nN$ (donde N es el número de arcos en el bucle), lo que constituye una contradicción. En la figura 11.3, para el bucle $3 \rightarrow 4 \rightarrow 5$, las condiciones son $\alpha_3 - \alpha_4 + 6 \leq 5$, $\alpha_4 - \alpha_5 + 6 \leq 5$ y $\alpha_5 - \alpha_3 + 6 \leq 5$. Si se suman todas se llega a que $18 \leq 15$, lo cual es imposible evidentemente.

Por el contrario, para comprobar que las desigualdades referidas se satisfacen cuando no hay bucles, definamos $\alpha_0 = 0$, $\alpha_{n+1} = n+1$ y $\alpha_i = K$ si la ciudad i es la que hace el número k de las visitadas. Cuando $x_{ij} = 1$, se tiene que $\alpha_i - \alpha_j + (n+1) = K - (K+1) + (n+1) = n$. Además, como $1 \leq \alpha_i \leq n+1$ ($i = 1, \dots, n+1$), la diferencia $\alpha_i - \alpha_j$ es siempre $\leq n$ (para todo i y j), satisfaciéndose por consiguiente las condiciones cuando $x_{ij} = 0$.

En resumen, la formulación completa del problema del representante de comercio es la siguiente:

$$\begin{aligned} \min. \quad & \sum_{i=0}^n \sum_{\substack{j=1 \\ j \neq i}}^{n+1} d_{ij} x_{ij} \\ \text{s. a} \quad & \sum_{i=0}^n x_{ij} = 1 \quad j = 1, \dots, n+1, i \neq j \\ & \sum_{j=1}^{n+1} x_{ij} = 1 \quad i = 0, 1, \dots, n, i \neq j \\ & \alpha_i - \alpha_j + (n+1)x_{ij} \leq n \quad i = 0, 1, \dots, n, j = 1, \dots, n+1, i \neq j \\ & x_{ij} = 0 \text{ ó } 1 \quad i = 0, 1, \dots, n, j = 1, \dots, n+1, i \neq j, \end{aligned}$$

donde $x_{0n+1} = 0$ (pues $x_{ij} = 0$ para $i = j$).

Como se puede intuir, muchos programas de programación entera (en particular aquellos en los que las variables son binarias) se pueden formular como problemas de flujos en redes.

En concreto, el del representante de comercio se puede referir a un conjunto de nudos $V = \{0, 1, \dots, n\}$ y a otro de arcos E . Los nudos representan las ciudades que hay que visitar y los arcos pares ordenados de ciudades entre las que es posible viajar. Para un arco $(i, j) \in E$, d_{ij} es la distancia entre las ciudades i y j . Las variables del problema, x_{ij} , serán las mismas de antes aunque en este caso indicarán si hay o no flujo en el arco (i, j) . Las condiciones de que a cada ciudad se llegue una sola vez y también se salga una sola vez las plasman las expresiones:

$$\sum_{\{i:(i,j) \in E\}} x_{ij} = 1 \quad \text{para } j \in V \quad \text{y} \quad \sum_{\{j:(i,j) \in E\}} x_{ij} = 1 \quad \text{para } i \in V.$$

Los bucles se eliminan observando que para cualquier¹ $U \subset V$, con $2 \leq |U| \leq |V| - 2$, de existir alguno, se viola al menos una de las condiciones

$$\sum_{\{(i,j) \in E: i \in U, j \in V \setminus U\}} x_{ij} \geq 1.$$

¹ $|X|$ indica el cardinal —número de elementos— del conjunto X .

El problema se formula de la siguiente manera:

$$\begin{aligned}
 \min. \quad & \sum_{(i,j) \in E} d_{ij} x_{ij} \\
 \text{s. a} \quad & \sum_{\{i: (i,j) \in E\}} x_{ij} = 1 \text{ para } j \in V \\
 & \sum_{\{j: (i,j) \in E\}} x_{ij} = 1 \text{ para } i \in V \\
 & \sum_{\{(i,j) \in E: i \in U, j \in V \setminus U\}} x_{ij} \geq 1 \\
 & \mathbf{x} \in \mathbf{B}^{|E|}.
 \end{aligned}$$

Problemas teóricos de grafos. “Set covering”

Se plantea sobre un grafo no dirigido tratándose de encontrar el subgrafo que contiene el mínimo número de arcos tal que de cada nudo del grafo parta o llegue al menos un arco perteneciente al subgrafo. Su formulación como programa entero es la siguiente:

$$\begin{aligned}
 \min. \quad & \mathbf{c}^T \mathbf{x} \\
 \text{s. a} \quad & E\mathbf{x} \geq \mathbf{e} \\
 & \mathbf{x} \in \mathbf{B}^n.
 \end{aligned}$$

La matriz E tiene todos sus elementos igual a 0 ó 1.

Si las condiciones de desigualdad son de estricta igualdad, al problema se le conoce como el del *set partitioning*.

Problemas teóricos de grafos. “Simple matching”

Se trata de, sobre un grafo no dirigido, encontrar el subgrafo que contiene el máximo número de arcos tal que 2 arcos del subgrafo no incidan a la vez en un mismo nudo del grafo. Su formulación como programa entero es la siguiente:

$$\begin{aligned}
 \min. \quad & \sum_j x_j \\
 \text{s. a} \quad & E\mathbf{x} \leq \mathbf{e} \\
 & x_j = 0 \text{ ó } 1, \quad j = 1, 2, \dots
 \end{aligned}$$

11.1.2 Modelización con variables binarias

Hasta ahora hemos presentado algunos problemas que dan lugar a la formulación de programas enteros tanto con variables binarias como con enteras en general. Dado que existe cierta especialización en programación entera referida a variables binarias, algunas veces se recurre a convertir un programa entero con variables enteras generales a otro con variables binarias exclusivamente. Para poder hacerlo basta tener en cuenta el siguiente resultado.

Teorema 11.1 *Supónganse programas enteros puros o mixtos en los que las variables enteras $x_j \leq u_j$. Esos programas son equivalentes a otros puros o mixtos con variables binarias.*

DEMOSTRACIÓN. Reemplazar cada x_j por:

- (i) $\sum_{k=1}^{u_j} t_{kj}$, donde las t_{kj} son variables 0 ó 1, y omitir los límites $x_j \leq u_j$ (dado que los $\sum_{k=1}^{u_j} t_{kj}$ pueden, a lo sumo, ser u_j);

o por,

- (ii) $\sum_{k=0}^{l_j} 2^k t_{kj}$, donde las t_{kj} son variables 0 ó 1 y l_j es el entero más pequeño tal que² $\sum_{k=0}^{l_j} 2^k = 2^{l_j+1} - 1 \geq u_j$, y conservar las condiciones $x_j = \sum_{k=0}^{l_j} 2^k t_{kj} \leq u_j$.

El resultado del teorema se obtiene inmediatamente pues cualquiera de las sustituciones permite a las variables x_j tomar cualquier valor entero entre 0 y u_j y todo entero siempre se puede escribir en base de numeración 2. ■

Ejemplo 11.1 Si la variable entera x_j tiene un límite superior $u_j = 107$, entonces en (ii) $l_j = 6$, pues

$$\sum_{k=0}^6 2^k = 2^7 - 1 \geq 107 \quad \text{y} \quad \sum_{k=0}^5 2^k = 2^6 - 1 < 107.$$

Por consiguiente, x_j se debe reemplazar por

$$\sum_{k=0}^6 2^k t_{kj} = t_{0j} + 2t_{1j} + 4t_{2j} + 8t_{3j} + 16t_{4j} + 32t_{5j} + 64t_{6j},$$

debiéndose mantener la condición $\sum_{k=0}^6 2^k t_{kj} \leq 107$. ●

Obsérvese cómo crece el tamaño del problema al hacerlo u_j . Evidentemente, aun cuando la ventaja de resolver un programa entero en variables binarias sea mucha, en muchos casos puede no resultar rentable modelizarlo así.

11.2 Resolución gráfica de programas enteros

Recordemos de la programación lineal que las condiciones del programa entero

$$\begin{aligned} & \underset{\mathbf{x} \in \mathbb{Z}^n, \mathbf{y} \in \mathbb{R}^p}{\text{maximizar}} && \mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y} \\ & \text{sujeta a} && A\mathbf{x} + G\mathbf{y} \leq \mathbf{b} \\ & && \mathbf{x}, \mathbf{y} \geq \mathbf{0}, \end{aligned}$$

en el caso en que $\mathbf{x} \in \mathbb{R}^n$ y $\mathbf{y} \in \mathbb{R}^p$, definen un politopo (poliedro si es acotado) en uno de cuyos puntos extremos o vértices la función objetivo $\mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y}$ alcanza el máximo. En

²La suma de la serie geométrica finita $a_0 + a_1 + \dots + a_N$ es $(a_0 - a_0 r^{N+1})/(1 - r)$, donde $r = \frac{a_{i+1}}{a_i}$, $i = 0, 1, \dots, N-1$.

general, desafortunadamente, las variables x_j en ese punto extremo no tienen por qué ser enteras. Como el politopo contiene todas las soluciones en las que $\mathbf{x}$ es entero, gráficamente se puede determinar esa solución hallando primero el punto extremo óptimo del problema considerando todas las variables como si fuesen continuas y luego, desplazando el hiperplano $\mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y}$ hacia dentro del politopo, encontrando el primer punto $[\mathbf{x}^T, \mathbf{y}^T]^T$ en el que $\mathbf{x}$ sea entero.

Ejemplo 11.2 Resuélvase el problema de programación entera

$$\begin{aligned} \max. \quad & \frac{3}{2}x_1 + x_2 \\ \text{s. a} \quad & -x_1 + 4x_2 \leq 16 \\ & x_1 + x_2 \leq 10 \\ & 5x_1 + x_2 \leq 36 \\ & x_1, x_2 \geq 0 \\ & x_1 \text{ y } x_2 \text{ enteras.} \end{aligned}$$

La región factible de este problema, sin tener en cuenta la condición de que las variables han de ser enteras, es el poliedro convexo que generan los puntos extremos 0, A, B, C y D de la figura 11.4. El valor máximo de la función objetivo en este poliedro se alcanza en C. Dado, sin embargo, que la auténtica región factible la constituyen los puntos interiores enteros de ese poliedro, el máximo del problema se alcanza en el punto $\mathbf{x}^* = [6, 4]^T$. ●

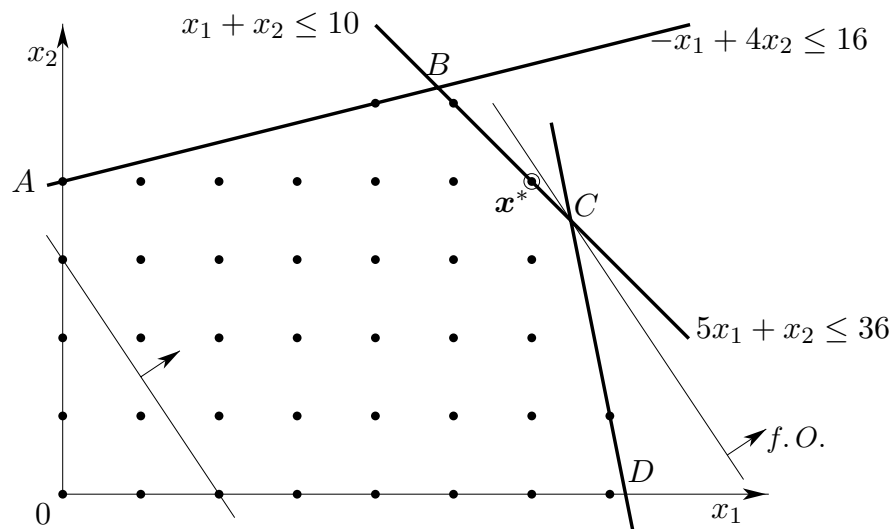


Figura 11.4

Región factible del problema del ejemplo 11.2

11.3 Propiedades de la región factible de los programas enteros

El conjunto o región factible de un programa entero, S , lo definen los puntos enteros del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$; es decir, $S = P \cap \mathbb{Z}^n$. Como se recordará del teorema 9.3

de la página 531, un politopo se puede expresar como una combinación lineal convexa de sus puntos extremos y una combinación lineal no negativa de sus direcciones extremas.

La *envoltura convexa* de S , expresada recordemos por $Co(S)$, es el conjunto de puntos que pueden expresarse como combinación convexa finita de puntos de S . Es decir:

$$Co(S) = \left\{ \mathbf{x} \in \mathbb{R}^n : \mathbf{x} = \sum_{i=1} \lambda_i \mathbf{x}_i, \sum_{i=1} \lambda_i = 1, \lambda_i \in \mathbb{R}, \lambda_i \geq 0 \right\},$$

donde $\mathbf{x}_1, \mathbf{x}_2, \dots$ es un conjunto finito de puntos de S . Si S es finito, por tanto, su envoltura convexa, $Co(S)$, es un poliedro cuyos puntos extremos constituyen un subconjunto de S . Este resultado se cumple en tanto y cuanto $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ sea un *politopo racional*: un politopo en el que los coeficientes de A y los elementos de $\mathbf{b}$ son racionales.

Teorema 11.2 *Si P es un politopo racional y $S = P \cap \mathbb{Z}^n \neq \emptyset$, la envoltura convexa de S , $Co(S)$, es un politopo racional cuyos puntos extremos constituyen un subconjunto de S y cuyas direcciones extremas son las direcciones extremas de P .*

Este teorema es inmediatamente extensible a la región factible de un programa entero mixto con parámetros racionales.

El teorema 11.2 tiene unas consecuencias muy interesantes. En primer lugar nos dice que el programa entero max. $\{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in S\}$, donde $S = P \cap \mathbb{Z}^n$, se puede formular como un programa lineal de la forma

$$\max. \{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in Co(S)\}. \quad (11.2)$$

Aun cuando interesante en sí misma, esta apreciación sin embargo no permite vislumbrar ningún procedimiento para resolver más cómodamente el programa entero original, pues para resolver el lineal que se deriva de ella habría que generar las desigualdades correspondientes a la definición de $Co(S)$, siendo el número de éstas, probablemente, extremadamente grande. En lo que sigue nos centraremos brevemente en la forma de construir ese $Co(S)$, o mejor dicho, un politopo Q , $Co(S) \subseteq Q \subseteq P$, tal que la solución del problema max. $\{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in Q\}$ sea la solución óptima del programa lineal (11.2).

El solo hecho de saber que un programa entero se puede reformular como uno lineal permite concluir que, si el programa entero no es ni no acotado ni no factible, entonces tiene una solución óptima; en otras palabras, el máximo de $\mathbf{c}^T \mathbf{x}$ se alcanzará en un punto extremo de S , y más concretamente en un punto extremo de $Co(S)$.

11.4 Algunas relajaciones de la formulación de programas enteros

La idea de las relajaciones consiste en resolver un programa entero a través de la determinación de cotas superiores, que designaremos por z_{PE} , del valor óptimo de la función objetivo de dicho programa entero. El cálculo de estas cotas superiores es esencial en programación entera pues la forma más elemental de determinar el carácter óptimo o no de una solución factible $\mathbf{x}$ es encontrar un $w \geq z_{PE}$ tal que $w = \mathbf{c}^T \mathbf{x}$. En términos prácticos, cuando $w = \mathbf{c}^T \mathbf{x} + \varepsilon$, $\varepsilon > 0$, se habrá establecido que la solución que determine esa w , $\mathbf{x}$, estará ε próxima a la real.

La idea básica que preside las relajaciones es sustituir el programa entero original por otro más fácil de resolver —subproblema— en el que se *relajan* —de ahí el concepto— alguna o algunas de las condiciones, obteniéndose de su resolución una cota de la función objetivo del programa original. Mediante un proceso iterativo que integre estas relajaciones y la resolución de los subproblemas, se va acotando cada vez más la función objetivo deseada hasta que se llega a la solución final.

Una *relajación* del programa entero

$$(PE) \quad \max. \{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in S \}$$

la constituye cualquier subproblema de la forma

$$(PR) \quad \max. \{ z_{PR}(\mathbf{x}) : \mathbf{x} \in S_{PR} \}$$

con las propiedades siguientes:

$$(R1) \quad S_{PR} \supseteq S$$

y

$$(R2) \quad \mathbf{c}^T \mathbf{x} \leq z_{PR}(\mathbf{x}) \quad \text{para } \mathbf{x} \in S.$$

Las propiedades (R1) y (R2) se escogen con el siguiente criterio.

Proposición 11.1 *Si el problema (PR) no es factible, tampoco lo es (PE). Si (PR) es factible, los valores de las funciones objetivo cumplen que $z_{PE} \leq z_{PR}$.*

11.4.1 Relajación lineal

La relajación más evidente que se puede utilizar en un programa entero resulta de cancelar la condición de que las variables han de ser enteras. Si el conjunto o región factible del programa entero es $S = P \cap \mathbf{Z}^n$, la relajación lineal es

$$(PL) \quad \max. \{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in P \},$$

donde $P = \{ \mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0} \}$. Según el teorema 11.2 de la página 625, si $S \neq \emptyset$, las direcciones extremas de P y $Co(S)$ coinciden.

Proposición 11.2 *Si (PL) es no acotado, el programa entero (PE) es no factible o no acotado.*

A cada programa lineal se le pueden asociar infinitas relajaciones lineales pues para cada polítopo $Q \supseteq Co(S)$, una relajación lineal la constituye el programa

$$(PL(Q)) \quad \max. \{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in Q \}.$$

Mejorar la función objetivo de un programa entero que define una solución, e incluso obtener una solución inicial, en la que las variables que han de ser enteras lo sean, requiere hacer algo más. Veamos a continuación cómo se pueden definir desigualdades que poder añadir a la relajación lineal con el fin de ir acotando más y más esa función objetivo y obtener la solución deseada.

11.4.1.1 Generación de desigualdades

Definición 11.1 Dado un politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$, una *desigualdad* $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ (o $[\boldsymbol{\pi}^T, \pi_0]^T$) se dice *válida* en P , si $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ para todo $\mathbf{x} \in P$.

Obsérvese que $[\boldsymbol{\pi}^T, \pi_0]^T$ es una desigualdad válida si y solo si P está en el semiespacio cerrado $\{\mathbf{x} \in \mathbb{R}^n : \boldsymbol{\pi}^T \mathbf{x} \leq \pi_0\}$ o, lo que es equivalente, si y sólo si la solución de $\max. \{\boldsymbol{\pi}^T \mathbf{x} : \mathbf{x} \in P\} \leq \pi_0$.

Definición 11.2 Si $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ es una desigualdad válida en un politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$, al conjunto $F = \{\mathbf{x} \in P : \boldsymbol{\pi}^T \mathbf{x} = \pi_0\}$ se le denomina *cara* de P . La desigualdad se dice *representa* a F . Una cara se dice *apropiada* si no es el conjunto vacío y se cumple que $F \neq P$.

La cara F representada por $[\boldsymbol{\pi}^T, \pi_0]^T$ no es el vacío si y sólo si la solución de $\max. \{\boldsymbol{\pi}^T \mathbf{x} : \mathbf{x} \in P\} = \pi_0$. Cuando F no es el vacío se dice que $[\boldsymbol{\pi}^T, \pi_0]^T$ *soporta* a P .

Definición 11.3 Una cara F del politopo $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$ es una *faceta* si $\dim(F) = \dim(P) - 1$.

En el conjunto $S = P \cap \mathbb{Z}^n$ una desigualdad válida en S lo es en $Co(S)$. Dadas dos desigualdades válidas, $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ y $\boldsymbol{\gamma}^T \mathbf{x} \leq \gamma_0$, no siendo una múltiplo escalar de la otra, se dice que $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ es más fuerte que, o *domina*, a $\boldsymbol{\gamma}^T \mathbf{x} \leq \gamma_0$, si $\boldsymbol{\pi} \geq \boldsymbol{\gamma}$, $\pi_0 \leq \gamma_0$ y al menos una de las desigualdades se cumple estrictamente. Una desigualdad válida *maximal* es aquella que no está dominada por ninguna otra en S . El conjunto de desigualdades maximales contiene todas aquellas desigualdades que definen facetas de $Co(S)$ (pero no todas las desigualdades maximales definen facetas).

A continuación expondremos tres métodos para generar desigualdades válidas en S . Lo que se persigue con esto es formular el programa entero que se trata de resolver como un programa lineal de acuerdo con $\max. \{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in Co(S)\}$. Obsérvese que cualquier combinación lineal no negativa de desigualdades válidas en S es también válida en S .

Redondeo entero

Recordemos que $\lfloor a \rfloor$, $a \in \mathbb{R}$, designa el máximo número entero menor o igual que a .

Proposición 11.3 Si $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ es una desigualdad válida en $S \subseteq \mathbb{Z}^n$, siendo $\boldsymbol{\pi}$ un vector entero, $\boldsymbol{\pi}^T \mathbf{x} \leq \lfloor \pi_0 \rfloor$ es una desigualdad válida en S .

DEMOSTRACIÓN. Para todo $\mathbf{x} \in S$, $\boldsymbol{\pi}^T \mathbf{x}$ es un número entero y por consiguiente menor o igual que el máximo número entero menor o igual que π_0 . ■

La generación de desigualdades que propicia el redondeo entero se representa en la figura 11.5. Supongamos que $\boldsymbol{\pi}^T \mathbf{x} \leq \pi_0$ es una combinación lineal de las condiciones $A\mathbf{x} \leq \mathbf{b}$,

donde π es un vector entero y π_0 un número racional, y el máximo común divisor de los elementos de π es 1. Redondear π_0 equivale a desplazar el hiperplano $\pi^T x = \pi_0$ hacia dentro de la región $Ax \leq b$ hasta que se encuentre un punto entero y . Este punto y , sin embargo, no tiene por qué estar en la región factible por lo que el hiperplano $\pi^T x = \lfloor \pi_0 \rfloor$ no tiene por qué contener ningún punto de S .

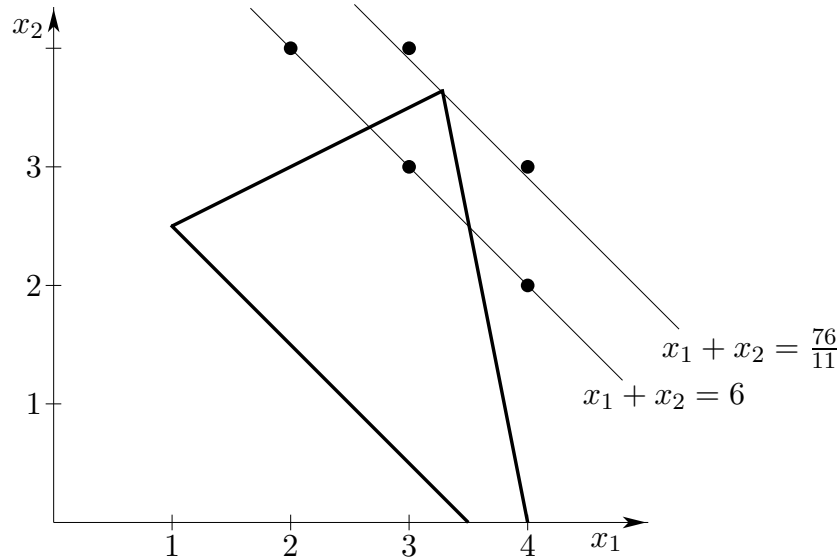


Figura 11.5

Generación de desigualdades por redondeo entero

Ejemplo 11.3 Sea $S = \{x \in \mathbf{Z}^n : Ax \leq b, x \geq 0\}$ con

$$A = \begin{bmatrix} -1 & 2 \\ 5 & 1 \\ -2 & -2 \end{bmatrix} \quad \text{y} \quad b = \begin{bmatrix} 4 \\ 20 \\ -7 \end{bmatrix}. \quad (11.3)$$

En la figura 11.6 se puede ver el politopo que definen las condiciones $Ax \leq b$ y $x \geq 0$ (el exterior), los puntos enteros factibles y $Co(S)$ (el politopo interior). Se tiene que

$$S = \left\{ \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \begin{bmatrix} 4 \\ 0 \end{bmatrix} \right\} = \{x_1, x_2, \dots, x_6\}.$$

La envoltura $Co(S) = \{x : x = \sum_{i=1}^6 \lambda_i x_i, \lambda_i \geq 0 \text{ para } i = 1, \dots, 6 \text{ y } \sum_{i=1}^6 \lambda_i = 1\}$. Como

$$\begin{bmatrix} 3 \\ 2 \end{bmatrix} = \frac{1}{2} \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} + \frac{1}{2} \cdot \begin{bmatrix} 3 \\ 3 \end{bmatrix} \quad \text{y} \quad \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \frac{1}{2} \cdot \begin{bmatrix} 2 \\ 2 \end{bmatrix} + \frac{1}{2} \cdot \begin{bmatrix} 4 \\ 0 \end{bmatrix},$$

$Co(S)$ es el poliedro definido por los cuatro puntos extremos

$$\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 3 \\ 3 \end{bmatrix} \quad \text{y} \quad \begin{bmatrix} 4 \\ 0 \end{bmatrix}.$$

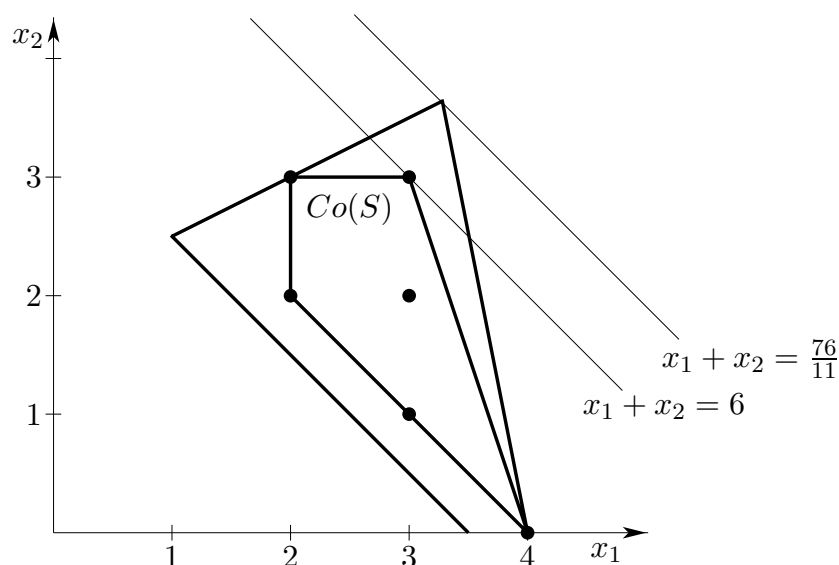


Figura 11.6
Región factible del problema del ejemplo 11.3

En este caso es muy sencillo conseguir una representación de $Co(S)$ mediante desigualdades; éstas son:

$$\begin{aligned} -x_1 &\leq -2 \\ x_2 &\leq 3 \\ -x_1 - x_2 &\leq -4 \\ 3x_1 + x_2 &\leq 12. \end{aligned}$$

Si se combinan las desigualdades $A\mathbf{x} \leq \mathbf{b}$ de (11.3), con pesos respectivos $\frac{4}{11}$, $\frac{3}{11}$ y 0, se obtiene la desigualdad válida en S

$$x_1 + x_2 \leq \frac{76}{11}.$$

Como no existen puntos de $\mathbf{Z}^2$ tales que $x_1 + x_2 = \frac{76}{11}$, ese hiperplano —una recta en este caso— se puede desplazar hacia dentro hasta que encuentre un punto de $\mathbf{Z}^2$ que cumpla $x_1 + x_2 \leq 6$, la cual es una desigualdad válida maximal en S . En este ejemplo, por casualidad, la recta $x_1 + x_2 = 6$ contiene un punto de S . ●

Desigualdades disyuntivas

Se trata de combinar dos desigualdades, cada una de las cuales es válida en una parte de S , para conseguir otra válida en S , aunque menos fuerte.

Proposición 11.4 Supóngase que $S = S^1 \cup S^2$ y que $\boldsymbol{\pi}^{iT} \mathbf{x} \leq \pi_0^i$ es una desigualdad válida en $S^i \subseteq \mathbb{R}^n$, $i = 1, 2$. Entonces

$$\sum_{j=1}^n \min \{ \pi_j^1, \pi_j^2 \} x_j \leq \max \{ \pi_0^1, \pi_0^2 \}$$

es válida en S .

En particular, considérense las dos siguientes desigualdades válidas en S , con $x_k \in \mathbf{Z}$:

$$(a) \quad \sum_{j=1}^n \pi_j x_j - \alpha(x_k - \delta) \leq \pi_0$$

y

$$(b) \quad \sum_{j=1}^n \pi_j x_j + \beta(x_k - \delta - 1) \leq \pi_0,$$

donde $\alpha \geq 0$, $\beta \geq 0$ y δ es un número entero. Obsérvese que (a) indica que

$$(c) \quad \sum_{j=1}^n \pi_j x_j \leq \pi_0$$

es válida para $\mathbf{x} \in S$, con $x_k \leq \delta$, y (b) que (c) es válida para $\mathbf{x} \in S$ con $x_k \geq \delta + 1$. De aquí que (c) sea válida en S pues como $x_k \in \mathbf{Z}$, $x_k \leq \delta$ ó $x_k \geq \delta + 1$.

Ejemplo 11.4 Un ejemplo de desigualdades disyuntivas se describe en la figura 11.7. En este caso $S = P \cap \mathbf{Z}^2$ y

$$P = \left\{ \mathbf{x} \in \mathbb{R}^2 : -x_1 + x_2 \leq \frac{1}{2}, \frac{1}{2}x_1 + x_2 \leq \frac{5}{4}, x_1 \leq 2, x_1, x_2 \geq 0 \right\}.$$

Las dos primeras condiciones se pueden reescribir de la siguiente manera:

$$\begin{aligned} -\frac{1}{4}x_1 + x_2 - \frac{3}{4}(x_1 - 0) &\leq \frac{1}{2}, \\ -\frac{1}{4}x_1 + x_2 + \frac{3}{4}(x_1 - 1) &\leq \frac{1}{2}. \end{aligned}$$

Usando la disyunción $x_1 \leq 0$ ó $x_1 \geq 1$ se llega a la desigualdad $-\frac{1}{4}x_1 + x_2 \leq \frac{1}{2}$, válida en $\mathbf{x} \in S = P \cap \mathbf{Z}^2$. ●

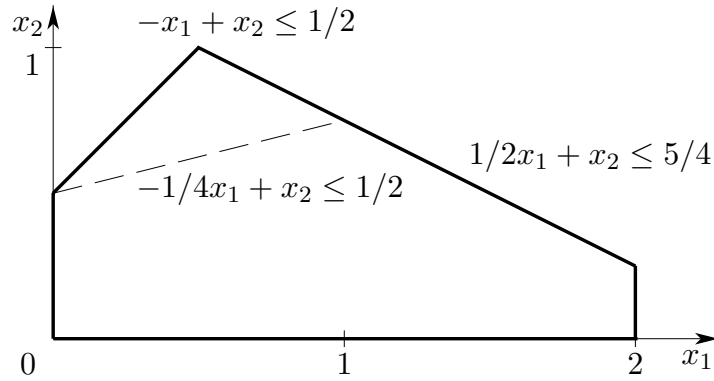


Figura 11.7

Ilustración del ejemplo 11.4 sobre desigualdades disyuntivas

Desigualdades superaditivas

Definición 11.4 Una función real F , con dominio de definición $D \subseteq \mathbb{R}^m$, $0 \in D$ y $F(0) = 0$, se denomina *superaditiva* en D , si $F(d_1) + F(d_2) \leq F(d_1 + d_2)$ para todo $d_1, d_2, d_1 + d_2 \in D$.

Definición 11.5 Una función real F , con dominio de definición $D \subseteq \mathbb{R}^m$, $0 \in D$ y $F(0) = 0$, se denomina *no decreciente* en D , si dados dos $d_1, d_2 \in D$, tales que $d_1 < d_2$, se cumple que $F(d_1) \leq F(d_2)$.

Sean $\mathbf{a}_j$, $j = 1, \dots, n$, los vectores columna de la matriz A . Si la función F es superaditiva no decreciente y $\mathbf{x} \in S = \{\mathbf{x} \in \mathbb{Z}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$, entonces

$$\sum_{j=1}^n F(\mathbf{a}_j)x_j \leq \sum_{j=1}^n F(\mathbf{a}_jx_j) \leq F\left(\sum_{j=1}^n \mathbf{a}_jx_j\right) \leq F(\mathbf{b}).$$

Proposición 11.5 Si la función F es superaditiva no decreciente en $\mathbb{R}^m$, entonces

$$\sum_{j=1}^n F(\mathbf{a}_j)x_j \leq F(\mathbf{b})$$

es una desigualdad válida en $S = \{\mathbf{x} \in \mathbb{Z}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$.

Ejemplo 11.5 Las funciones

- (i) $F(d) = \lfloor ud \rfloor$ para $\mathbf{u} \in \mathbb{R}^m$, $\mathbf{u} \geq \mathbf{0}$, descrita³ en la figura 11.8-(1) para $m = 1$ y $\mathbf{u} = 1$, y
- (ii) $F_\alpha(d) = \lfloor d \rfloor + (f_d - \alpha)^+ / (1 - \alpha)$, donde $0 < \alpha < 1$, $f_d = d - \lfloor d \rfloor$, y $x^+ = \max\{x, 0\}$ (representada en la figura 11.8-(2) para $\alpha = 1/2$),

³Obsérvese que F es la función que caracteriza el redondeo entero.

se pueden usar para generar desigualdades válidas en problemas de programación entera mixta. ●

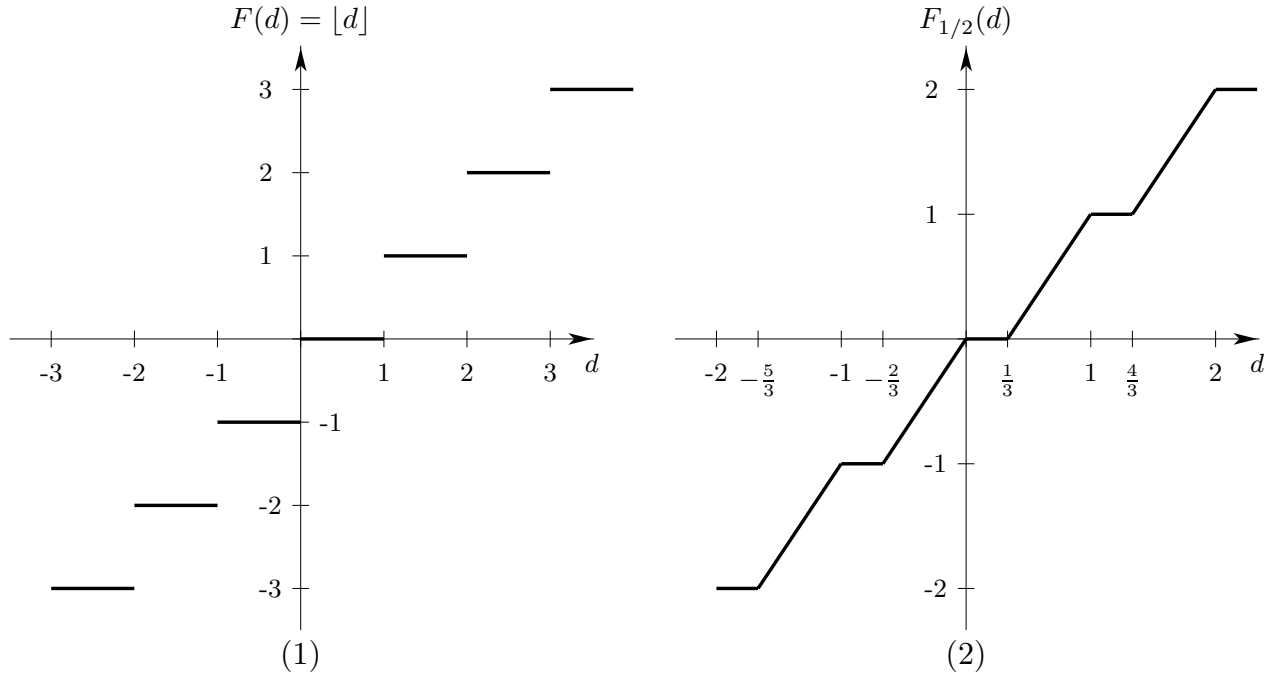


Figura 11.8

Funciones del ejemplo 11.5 para generar desigualdades válidas

Aunque la relajación más habitual en programación entera es la lineal, a continuación estudiaremos brevemente otras también frecuentemente usadas.

11.4.2 Relajación lagrangiana

Muy a menudo las condiciones $A\mathbf{x} \leq \mathbf{b}$ de un programa entero se pueden dividir en dos conjuntos, $A^1\mathbf{x} \leq \mathbf{b}^1$ y $A^2\mathbf{x} \leq \mathbf{b}^2$, donde la matriz A^1 tiene $m_1 < m$ filas, de tal forma que el programa relajado

$$\max. \{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in Q \}, \quad \text{donde } Q = \{ \mathbf{x} \in \mathbf{Z}^n : A^2 \mathbf{x} \leq \mathbf{b}^2, \mathbf{x} \geq \mathbf{0} \},$$

resultante de dejar de considerar las condiciones $A^1\mathbf{x} \leq \mathbf{b}^1$, es mucho más fácil de resolver. Es decir, el problema suele constar de un conjunto de condiciones *difíciles* y de otro formado por otras menos difíciles o que plantean menos dificultades para tenerlas en cuenta. Caso típico de esta situación la constituye aquel en el que las condiciones $A^2\mathbf{x} \leq \mathbf{b}^2$ son de red.

El proceder de esta forma separando las condiciones en dos conjuntos de ellas conduce a la siguiente familia de relajaciones:

$$RL(\boldsymbol{\lambda}) \quad \max. \{ z(\boldsymbol{\lambda}, \mathbf{x}) : \mathbf{x} \in Q \},$$

donde $z(\boldsymbol{\lambda}, \mathbf{x}) = \mathbf{c}^T \mathbf{x} + \boldsymbol{\lambda}^T (\mathbf{b}^1 - A^1 \mathbf{x})$ y $\boldsymbol{\lambda} \in \Re^{m_1}$, $\boldsymbol{\lambda} \geq \mathbf{0}$. Al problema $RL(\boldsymbol{\lambda})$ se le denomina *relajación lagrangiana* del programa entero original con respecto a las condiciones $A^1\mathbf{x} \leq \mathbf{b}^1$.

Obsérvese que $\lambda^T(b^1 - A^1x) \geq 0$ para todo $x \in S$, pues $\lambda \geq 0$, por lo que $z(\lambda, x) \geq c^T x$ para todo $x \in S$. El problema RL(0), que se obtiene al no considerar las condiciones $A^1x \leq b^1$, proporciona un primer límite superior de la función objetivo del programa original. Escogiendo un $\lambda \neq 0$, sin embargo, se puede determinar un límite mejor de la función objetivo deseada. El problema de determinar aquella λ que dé el menor límite superior de la función objetivo del programa entero en su conjunto origina el planteamiento de un programa dual que es el que se resuelve realmente.

11.4.3 Descomposición y separación de costes

Esta forma de abordar el problema consiste en descomponerlo en dos a partir de la observación de que el programa entero original es equivalente, en un espacio $2n$ -dimensional, al siguiente:

$$\max. \{c^1{}^T x^1 + c^2{}^T x^2 : x^1 \in Q^1, x^2 \in Q^2, x^1 - x^2 = 0\},$$

donde $Q^i = \{x^i \in \mathbf{Z}^n : A^i x^i \leq b^i, x^i \geq 0\}$ para $i = 1, 2$ y $c^1 + c^2 = c$. Tomándose la relajación lagrangiana de este problema con respecto a las condiciones $x^1 - x^2 = 0$, se obtienen los subproblemas

$$z^1(\lambda) = \max. \{(c - \lambda)^T x^1 : x^1 \in Q^1\} \quad \text{y} \quad z^2(\lambda) = \max. \{\lambda^T x^2 : x^2 \in Q^2\},$$

donde $\lambda \in \Re^n$. El vector de multiplicadores, λ , no está restringido dado que $x^1 - x^2 = 0$ es un conjunto de condiciones de igualdad. Resolviendo estos subproblemas se obtiene un valor que acota la función objetivo del programa entero. Ese valor es:

$$z_{PE} \leq z^1(\lambda) + z^2(\lambda),$$

para todo $\lambda \in \Re^n$.

11.4.4 Descomposición de Benders

Si el método de la relajación lagrangiana se puede ver como el que trata condiciones complicadas, éste es en alguna medida el dual para variables complicadas. En particular, si se considera el programa entero mixto

$$\begin{aligned} (PEM) \quad & \max. \quad c^T x + h^T y \\ & \text{s. a} \quad Ax + Gy \leq b \\ & \quad \quad x, y \geq 0 \\ & \quad \quad x \in X \subseteq \mathbf{Z}^n, \quad y \in \Re^p, \end{aligned}$$

las variables enteras x son las que complican el problema pues de lo contrario éste sería un simple programa lineal; visto de otra manera, las variables y complican un problema entero puro al convertirlo en uno mixto.

El procedimiento de Benders (1962) reformula el problema (PEM) en uno en $\mathbf{Z}^n \times \Re$, esto es, en uno en el que solamente se considera una variable continua. Su formulación normalmente

contiene una cantidad importante de condiciones lineales aunque al no estar muchas de ellas activas en la solución óptima se pueden considerar sólo temporalmente.

Para llegar a esa formulación, supóngase que las variables enteras $\mathbf{x}$ se fijan en un determinado valor. El programa lineal resultante es

$$\begin{aligned} PL(\mathbf{x}) \quad & \max. \quad \mathbf{h}^T \mathbf{y} \\ \text{s. a} \quad & G\mathbf{y} \leq \mathbf{b} - A\mathbf{x} \\ & \mathbf{y} \geq \mathbf{0} \\ & \mathbf{y} \in \mathbb{R}^p, \end{aligned}$$

y su dual

$$\min. \left\{ \mathbf{u}^T (\mathbf{b} - A\mathbf{x}) : \mathbf{u} \in Q \right\},$$

donde $Q = \{\mathbf{u} \in \mathbb{R}^m : \mathbf{u}^T G \geq \mathbf{h}, \mathbf{u} \geq \mathbf{0}\}$. La ventaja de trabajar con el dual es que su región factible, Q , es independiente de $\mathbf{x}$. Si $Q = \emptyset$, el programa lineal, $PL(\mathbf{x})$, o no es factible o tiene una solución no acotada. Si $Q \neq \emptyset$, el programa lineal $PL(\mathbf{x})$ no es factible o tiene óptimo. En particular $PL(\mathbf{x})$ no es factible si y sólo si Q tiene una dirección extrema $\mathbf{v}$ tal que $\mathbf{v}^T (\mathbf{b} - A\mathbf{x}) < 0$; de no ser así, el valor óptimo de $PL(\mathbf{x})$ es $\mathbf{u}^T (\mathbf{b} - A\mathbf{x})$, donde $\mathbf{u}$ es un punto extremo de Q .

Si $\mathbf{v}^j$, para $j \in J$, son las direcciones extremas de Q y $\mathbf{u}^k$ para $k \in K$, los puntos extremos de Q , cuando $Q \neq \emptyset$, el programa entero mixto, PEM , se puede reformular de la siguiente manera:

$$\begin{aligned} \max. \quad & \left(\mathbf{c}^T \mathbf{x} + \min_{k \in K} \left\{ \left(\mathbf{u}^k \right)^T (\mathbf{b} - A\mathbf{x}) \right\} \right), \\ \text{s. a} \quad & (\mathbf{v}^j)^T (\mathbf{b} - A\mathbf{x}) \geq 0 \quad \text{para } j \in J, \\ & \mathbf{x} \in X, \end{aligned}$$

formulación de Benders del problema entero mixto PEM .

Teorema 11.3 *El programa entero mixto*

$$\begin{aligned} (PEM) \quad & \max. \quad \mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y} \\ \text{s. a} \quad & A\mathbf{x} + G\mathbf{y} \leq \mathbf{b} \\ & \mathbf{x}, \mathbf{y} \geq \mathbf{0} \\ & \mathbf{x} \in X \subseteq \mathbf{Z}^n, \quad \mathbf{y} \in \mathbb{R}^p, \end{aligned}$$

se puede reformular de la siguiente manera:

$$\begin{aligned} \max. \quad & \eta \\ \text{s. a} \quad & \eta \leq \mathbf{c}^T \mathbf{x} + \left(\mathbf{u}^k \right)^T (\mathbf{b} - A\mathbf{x}) \quad \text{para } k \in K, \\ & \left(\mathbf{v}^k \right)^T (\mathbf{b} - A\mathbf{x}) \geq 0 \quad \text{para } j \in J, \\ & \mathbf{x} \in X, \quad \eta \in \mathbb{R}. \end{aligned}$$

Referencias

Existen varios libros recientes muy buenos sobre programación entera. Cabe destacar Nemhauser y Wolsey [1988] y [1989], Schrijver [1986] y Salkin y Mathur [1989]. Para este capítulo hemos seguido a Nemhauser y Wolsey [1988], Schrijver [1986] y Salkin [1975].

Entre las referencias clásicas de programación entera, además de las mencionadas, están Salkin [1975], Garfinkel y Nemhauser [1972], Hu [1970] y Greenberg [1971].

Ejercicios

- 11.1. Una empresa produce un determinado producto en m fábricas distintas y satisface con ello la demanda de n clientes (la demanda de éstos es d_j). Si una de esas fábricas funciona (o debe construirse), su coste de operación/construcción es $f_i \geq 0$, produciendo $M_i > 0$ unidades de producto. El coste de enviar una unidad de producto de la fábrica i al cliente j es g_{ij} .

Formular el problema de determinar cuántas fábricas debe tener disponibles esa empresa de tal forma que se satisfaga la demanda de los clientes a mínimo coste.

- 11.2. Probar que en el problema planteado en el ejercicio anterior las desigualdades que se definen implican que las cantidades a transportar de las fábricas a los clientes deben ser enteras.
- 11.3. Resolver el problema del ejercicio 1 para el caso de dos fábricas y tres clientes cuyos datos se indican a continuación.

g_{ij}	1	2	3	d_1	d_2	d_3	f_1	f_2
1	5	7	6	150	170	260	90	100
2	5	6	7					

- 11.4. Reformular el problema del ejercicio 1 para tener en cuenta las siguientes condiciones:

- Sólo se pueden abrir un conjunto de fábricas dado.
- El coste de abrir una fábrica consta de una parte fija, K_i , y otra dependiente de su producción.
- Las fábricas pueden estar separadas por regiones de interés estratégico, función del número de clientes, debiendo cada región k tener al menos l_k fábricas abiertas.
- Se dispone de un total de F pesetas para construir fábricas.

- 11.5. Considérese la variante siguiente del problema de la mochila:

$$\begin{aligned} \max. \quad & \sum_j c_j x_j \\ \text{s. a} \quad & \sum_j a_j x_j \leq b \\ & x_j \geq 0 \text{ y entera,} \end{aligned}$$

donde c_j es el valor del objeto j , a_j su peso, b la capacidad de transporte de la mochila y x_j el número de objetos a transportar en ella.

- a) Si no se requiere que x_j sea entera, resolver el problema cuyos datos son

c_j	8	10	9	8	6	2	7	5
a_j	4	2	7	3	5	1	3	5

y $b = 23$. Sugerencia: hacer $z_j = a_j x_j$, $j = 1, \dots, n$, y transformar el problema en otro en las variables z_j . La solución de éste es obvia.

b) Determinar una solución básica factible entera del problema.

c) A partir de la respuesta anterior, determinar el rango de valores de una solución entera óptima.

11.6. Considérese el problema del representante de comercio con $d_{ij} = d_{ji}$ para todo i y j . ¿Se puede simplificar la formulación del problema en este caso?

11.7. Dado un mapa dividido en regiones, ¿se pueden colorear las regiones usando sólo cuatro colores de tal forma que dos adyacentes no estén coloreadas igual?

a) Formular el problema como un programa lineal entero.

b) ¿Por qué no es posible definir una función objetivo para este problema?

11.8. Considérese el programa entero

$$\begin{array}{ll} \text{maximizar} & \mathbf{c}^T \mathbf{x} \\ \text{s. a} & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{0} \leq \mathbf{x} \leq \mathbf{e} \\ & x_j = 0 \text{ ó } 1 \end{array}$$

y el programa cuadrático

$$\begin{array}{ll} \text{maximizar} & \mathbf{c}^T \mathbf{x} - M\mathbf{x}^T(\mathbf{e} - \mathbf{x}) \\ \text{s. a} & A\mathbf{x} \leq \mathbf{b} \\ & \mathbf{0} \leq \mathbf{x} \leq \mathbf{e}, \end{array}$$

donde $\mathbf{e}$ es un vector en el que todos sus componentes son 1 y M un valor positivo muy grande.

a) Probar que si $\mathbf{x}^*$ es la solución óptima del programa cuadrático y es entera, es la solución óptima del programa entero.

b) Probar que si $\mathbf{x}^*$ es una solución óptima del programa cuadrático y no tiene todos sus componentes enteros, el programa entero no tiene solución factible.

c) Probar que si el programa cuadrático no tiene solución factible, el programa entero no tiene solución entera factible.

¿Se puede resolver el programa entero resolviendo el cuadrático?

11.9. Considérese el problema de la mochila cuadrático siguiente:

$$\begin{array}{ll} \text{max.} & \sum_j c_j x_j + \sum_{i=1}^{j-1} \sum_{j=2}^n c_{ij} x_i x_j \\ \text{s. a} & \sum_j a_j x_j \leq b \\ & x_j = 0 \text{ ó } 1. \end{array}$$

Reformularlo como programa entero haciendo $y_{ij} = x_i x_j$.

11.10. Formular como programa entero el que se plantea un inversor en bolsa que quiere maximizar sus beneficios cumpliendo una serie de condiciones que afectan al plazo de amortización, presupuesto total, número mínimo de títulos públicos y privados y rentabilidad.

11.11. Expresar la dicotomía

$$g(x) \geq 0 \quad \text{ó} \quad h(x) \geq 0, \quad \text{pero no ambas,}$$

con variables binarias.

11.12. Resolver gráficamente el programa entero

$$\begin{array}{ll}\text{maximizar} & 2x - y \\ \text{s. a} & 5x + 7y \leq 45 \\ & -2x + y \leq 1 \\ & 2x - 5y \leq 5 \\ & x, y \geq 0 \\ & x \text{ e } y \text{ enteras.}\end{array}$$

11.13. Aplicar la relajación lagrangiana a

$$\begin{array}{ll}\text{maximizar} & 2x + 5y \\ \text{s. a} & 4x + y \leq 28 \\ & 2x + 4y \leq 27 \\ & x - y \leq 1 \\ & x, y \geq 0 \\ & x \text{ e } y \text{ enteras.}\end{array}$$

11.14. Reformular mediante Benders el problema estudiado en este capítulo de la planificación de la generación de energía eléctrica en un país.

11.15. Reformular mediante Benders el programa entero mixto

$$\begin{array}{ll}\text{maximizar} & 2x_1 + x_2 + 3x_3 + 7y_1 + 5y_2 \\ \text{s. a} & 9x_1 + 4x_2 + 14x_3 + 35y_1 + 24y_2 \leq 80 \\ & -x_1 - 2x_2 + 3x_3 - 2y_1 + 4y_2 \leq 10 \\ & x_1, x_2, x_3, y_1, y_2 \geq 0 \\ & x_1, x_2 \text{ y } x_3 \text{ enteras.}\end{array}$$

Capítulo 12

ALGORITMOS GENERALES DE RELAJACIÓN

UNA VEZ PRESENTADAS las cuestiones teóricas esenciales de los programas enteros y sus diversas formulaciones, en este capítulo analizamos los algoritmos más usados para resolver programas enteros generales del tipo

$$(PE) \quad \max. \left\{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in S \right\}.$$

donde $S = \{\mathbf{x} \in \Re^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\} \cap \mathbf{Z}^n$. Supondremos que la región o conjunto factible es no vacía y que el valor de la función objetivo del problema, z_{PE} , está acotado.

Los dos grupos de algoritmos que se analizarán siguen un esquema general basado en relajaciones sucesivas del problema original como el que se indica en la tabla 12.1.

Obsérvese que la sucesión de cotas superiores que este algoritmo produce verifican $z_R^{(i+1)} \leq z_R^{(i)}$. En muchas relajaciones se usa como función objetivo $z_R^{(i)}(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$, para todo i , por lo que tan pronto como se llegue a una solución $\mathbf{x}^{(i)} \in S$, se habrá alcanzado el óptimo de (PE) . En este caso, el paso de mejora de la solución satisface que $S_R^{(i+1)} \subset S_R^{(i)}$ para todo i . Es aconsejable, por tanto, elegir $S_R^{(i+1)} \subseteq S_R^{(i)} \setminus \{\mathbf{x}^{(i)}\}$, pues si no $z_R^{(i+1)} = z_R^{(i)}$.

12.1 El algoritmo de los planos cortantes de Gomory

Este algoritmo es uno de los primeros que se emplearon para resolver el problema general de programación entera. Para la exposición de su mecánica supondremos que el problema sólo tiene variables enteras; es decir, que $S = \{\mathbf{x} \in \mathbf{Z}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$.

Introducido por el que muchos consideran el padre de la programación entera, Gomory [1960] y [1963], este algoritmo obtiene la solución en un número finito de pasos. Su uso, después de los primeros años de su difusión, no ha sido muy amplio debido a los pobres resultados

Tabla 12.1

Algoritmo general para programas enteros basado en relajaciones sucesivas

Paso 0 – *Inicialización.* Hacer $i = 1$, $w^* = \infty$ y $z^* = -\infty$. Escoger un $S_R^{(1)} \supseteq S$ tal que $z_R^{(1)} \geq \mathbf{c}^T \mathbf{x}$ para $\mathbf{x} \in S$.

Paso 1 – *Relajación.* Resolver el problema PE relajado:

$$(R^{(i)}) \quad \max. \left\{ z_R^{(i)}(\mathbf{x}) : \mathbf{x} \in S_R^{(i)} \right\}.$$

Paso 2 – *Comprobación de óptimo.* Sea $\mathbf{x}^{(i)}$ la solución del problema anterior. Si $\mathbf{x}^{(i)} \in S$: PARAR; ésta es la solución óptima de (PE) con función objetivo $w^* = \mathbf{c}^T \mathbf{x}^{(i)} = z^*$; si no, seguir.

Paso 3 – *Mejora de la solución.* Hacer $w^* = z_R^{(i)}$, $z^* = \mathbf{c}^T \mathbf{x}^{(i)}$ si $\mathbf{x}^{(i)} \in S$ y $i \leftarrow i + 1$. Escoger $S_R^{(i+1)}$ tal que $S \subseteq S_R^{(i+1)} \subseteq S_R^{(i)}$ y $z_R^{(i+1)}(\mathbf{x})$ tal que $\mathbf{c}^T \mathbf{x} \leq z_R^{(i+1)}(\mathbf{x}) \leq z_R^{(i)}(\mathbf{x})$ para $\mathbf{x} \in S$ con $S_R^{(i+1)} \neq S_R^{(i)}$ o $z_R^{(i+1)}(\mathbf{x}) \neq z_R^{(i)}(\mathbf{x})$. Ir al paso 1

numéricos de sus implementaciones prácticas. Recientemente, gracias a los últimos avances de la programación poliédrica y de otras disciplinas afines, se ha vuelto a investigar en alguna de sus variantes con el fin de resolver problemas de estructura especial. A pesar de no aportar mucho desde el punto de vista práctico, no obstante, a continuación lo expondremos brevemente pues, entre otras cosas, facilita la comprensión de un procedimiento muy sencillo para eliminar y mejorar posibles soluciones en el otro método general que veremos posteriormente, el de ramificación y acotamiento o *branch and bound*. Para facilitar la exposición, de acuerdo con las ideas de Gomory, consideraremos que el problema que resuelve el correspondiente algoritmo, esto es

$$\max. \left\{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in S^{(e)} \right\}, \quad \text{donde } S^{(e)} = \{ \mathbf{x} \in \mathbf{Z}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0} \},$$

está escrito de la siguiente forma:

$$(PE) \quad \max. \left\{ x_0 : [x_0, \mathbf{x}^T]^T \in S^{(0)} \right\},$$

donde

$$S^{(0)} = \left\{ x_0 \in \mathbf{Z}, \mathbf{x} \in \mathbf{Z}^n : x_0 - \mathbf{c}^T \mathbf{x} = 0, A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0} \right\}.$$

Supondremos que se dispone de una solución y base óptimas de la relajación lineal del programa entero. El problema se puede escribir entonces:

$$\begin{aligned} & \text{maximizar } x_0 \\ & \text{s. a} \quad x_{B_i} + \sum_{j \in H} \bar{a}_{ij} x_j = \bar{a}_{i0} \quad \text{para } i = 0, 1, \dots, m \\ & \quad x_{B_0} \in \mathbf{Z}, x_{B_i} \in \mathbf{Z}_+, \quad \text{para } i = 1, \dots, m \\ & \quad x_j \in \mathbf{Z}_+ \text{ para } j \in H, \end{aligned} \tag{12.1}$$

donde¹ $x_0 = x_{B_0}$, x_{B_i} , $i = 1, \dots, m$, son las variables básicas y x_j , $j \in H \subset N = \{1, \dots, n\}$, las no básicas.² Como esta base es factible en el programa primal y dual se cumple que $\bar{a}_{i0} \geq 0$ para $i = 1, \dots, m$, y que $\bar{a}_{0j} \geq 0$, para $j \in H$.

Suponiendo que en el problema (12.1) existe un i tal que $\bar{a}_{i0} \notin \mathbf{Z}$, se tiene el siguiente resultado.

Proposición 12.1 (*Corte Fraccionario de Gomory*) Si $\bar{a}_{i0} \notin \mathbf{Z}$,

$$\sum_{j \in H} f_{ij} x_j = f_{i0} + x_{n+1}, \quad x_{n+1} \in \mathbf{Z}_+,$$

es una desigualdad válida en $S^{(0)}$, donde $f_{ij} = \bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor$ para $j \in H$ y $f_{i0} = \bar{a}_{i0} - \lfloor \bar{a}_{i0} \rfloor$.

Ejemplo 12.1 Considérese el programa entero

$$\begin{array}{ll} (PE) & \text{maximizar} \quad 7x_1 + 2x_2 \\ & \text{s. a} \quad \begin{array}{l} -x_1 + 2x_2 \leq 4 \\ 5x_1 + x_2 \leq 20 \\ -2x_1 - 2x_2 \leq -7 \\ \mathbf{x} \in \mathbf{Z}_+^2. \end{array} \end{array}$$

Introduzcamos las variables de holgura x_3 , x_4 y x_5 . El programa³ queda:

$$\begin{array}{ll} \text{maximizar} & 7x_1 + 2x_2 \\ \text{s. a} & \begin{array}{ll} -x_1 + 2x_2 + x_3 & = 4 \\ 5x_1 + x_2 + x_4 & = 20 \\ -2x_1 - 2x_2 + x_5 & = -7 \\ x_1, x_2 \in \mathbf{Z}_+; & x_3, x_4, x_5 \geq 0. \end{array} \end{array}$$

Este problema, escrito en la forma de la expresión (12.1), esta vez restringiendo las variables x_3 , x_4 y x_5 a tomar valores enteros, es:

$$\begin{array}{ll} \text{maximizar} & x_0 \\ \text{s. a} & \begin{array}{ll} x_0 - 7x_1 - 2x_2 & = 0 \\ -x_1 + 2x_2 + x_3 & = 4 \\ 5x_1 + x_2 + x_4 & = 20 \\ -2x_1 - 2x_2 + x_5 & = -7 \\ x_0 \in \mathbf{Z}, x_j \in \mathbf{Z}_+ & \text{para } j = 1, \dots, 5. \end{array} \end{array}$$

¹En lo sucesivo, para simplificar la notación, designaremos mediante $\mathbf{Z}_+$ el conjunto de los enteros no negativos (≥ 0).

²Recuérdese de la programación lineal que $\mathbf{x}_B = B^{-1}\mathbf{b} - B^{-1}N\mathbf{x}_N$ y $z = \mathbf{c}_B B^{-1}\mathbf{b} + (\mathbf{c}_N - \mathbf{c}_B B^{-1}N)\mathbf{x}_N$. Expresión muy parecida a la aquí expuesta.

³Aunque las variables de holgura serán enteras, no hay ninguna necesidad de restringirlas a tomar valores enteros.

La solución de la relajación lineal de este programa entero es

$$\begin{array}{rclcl} x_0 & + & \frac{3}{11}x_3 & + & \frac{16}{11}x_4 & = & \frac{332}{11} \\ x_1 & - & \frac{1}{11}x_3 & + & \frac{2}{11}x_4 & = & \frac{36}{11} \\ x_2 & + & \frac{5}{11}x_3 & + & \frac{1}{11}x_4 & = & \frac{40}{11} \\ & & \frac{8}{11}x_3 & + & \frac{6}{11}x_4 & + & x_5 = \frac{75}{11}, \end{array}$$

donde $x_3 = x_4 = 0$.

El corte de Gomory que se puede obtener de la primera ecuación es

$$\frac{3}{11}x_3 + \frac{5}{11}x_4 = \frac{2}{11} + x_6, \quad x_6 \in \mathbf{Z}_+.$$

En función de las variables originales, x_1 y x_2 (despejando de las ecuaciones anteriores), el corte es $2x_1 + x_2 \leq 10$. ●

El algoritmo completo para resolver programas enteros puros (no hay variables reales) basándose en los cortes de Gomory se describe en la tabla 12.2. Este algoritmo converge a una solución en un número finito de iteraciones.

Cuando se añade un corte, la nueva base, que incluye x_{n+i} como variable básica, es factible del dual. La factibilidad del primal se incumple sólo en que x_{n+i} es negativa. De acuerdo con esto y con lo expuesto en el capítulo 8, para reoptimizar el problema lo más adecuado es usar el método dual del *simplex*.

Si $PR^{(i)}$ es no acotado, el programa entero, PE , es no acotado o no factible. Si PE es factible, existe una solución factible en la que $\sum_{j \in N} x_j \leq d$, donde d es un entero adecuadamente grande. En consecuencia, se puede añadir una condición $\sum_{j \in N} x_j \leq d$ al programa relajado $PR^{(i)}$ de tal forma que PE será no acotado si y sólo si el problema modificado tiene una solución factible.

Ejemplo 12.1 (Continuación) La última solución de $PR^{(1)}$ a la que hacíamos referencia era

$$\begin{array}{rclcl} x_0 & + & \frac{3}{11}x_3 & + & \frac{16}{11}x_4 & = & \frac{332}{11} \\ x_1 & - & \frac{1}{11}x_3 & + & \frac{2}{11}x_4 & = & \frac{36}{11} \\ x_2 & + & \frac{5}{11}x_3 & + & \frac{1}{11}x_4 & = & \frac{40}{11} \\ & & \frac{8}{11}x_3 & + & \frac{6}{11}x_4 & + & x_5 = \frac{75}{11}, \end{array}$$

donde $x_3 = x_4 = 0$. Es decir $\left[x_0^{(1)}, \left(\mathbf{x}^{(1)} \right)^T \right] = [x_0^{(1)}, x_1^{(1)}, \dots, x_5^{(1)}] = \left[\frac{332}{11}, \frac{36}{11}, \frac{40}{11}, 0, 0, \frac{75}{11} \right]$. El corte de Gomory respecto de la tercera ecuación es

$$\frac{5}{11}x_3 + \frac{1}{11}x_4 = \frac{7}{11} + x_6, \quad x_6 \in \mathbf{Z}_+.$$

Tabla 12.2

El algoritmo de los planos cortantes de Gomory

Paso 0 – *Inicialización.* Hacer

$$S_R^{(1)} = \{x_0 \in \mathbb{R}, \mathbf{x} \in \mathbb{R}^n : x_0 - \mathbf{c}^T \mathbf{x} = 0, A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}\},$$

$$i = 1 \text{ y } z_R^{(1)} = x_0.$$

Paso 1 – *Resolución de la relajación lineal.* Resolver el programa entero relajado:

$$(PR^{(i)}) \quad \max. \left\{ x_0 : [x_0, \mathbf{x}^T]^T \in S_R^{(i)} \right\}.$$

Si $PR^{(i)}$ es factible y tiene solución óptima, $[x_0^{(i)}, (\mathbf{x}^{(i)})^T]^T$, continuar.**Paso 2** – *Comprobación de óptimo.* Si $\mathbf{x}^{(i)} \in \mathbf{Z}_+^n$, ésta es la solución óptima del programa entero.**Paso 3** – *Comprobación de no factibilidad.* Si $PR^{(i)}$ no es factible, el programa entero original no es factible.**Paso 4** – *Adición de un corte de Gomory.* Escoger una fila $x_{B_i} + \sum_{j \in H^{(i)}} \bar{a}_{ij}^{(i)} x_j = \bar{a}_{i0}^{(i)}$ con $\bar{a}_{i0}^{(i)} \notin \mathbf{Z}$. Hacer

$$\sum_{j \in H^{(i)}} f_{kj} x_j - x_{n+i} = f_{k0}, \quad k = 1, \dots, m, \quad x_{n+i} \in \mathbf{Z}_+,$$

el corte de Gomory de esa fila. Hacer

$$S_R^{(i+1)} = S_R^{(i)} \cap \left\{ x_0 \in \mathbb{R}, \mathbf{x} \in \mathbb{R}^{n+i} : \sum_{j \in H^{(i)}} f_{kj} x_j - x_{n+i} = f_{k0}, \mathbf{x} \geq \mathbf{0} \right\}.$$

Paso 5 – Hacer $i \leftarrow i + 1$ e ir al paso 1.Añadiendo este corte a $PR^{(1)}$ se obtiene $PR^{(2)}$. Su solución es

$$\begin{array}{rcccccl} x_0 & & + \frac{7}{5}x_4 & + \frac{3}{5}x_6 & = & \frac{149}{5} \\ x_1 & & + \frac{1}{5}x_4 & - \frac{1}{5}x_6 & = & \frac{17}{5} \\ x_2 & & & + x_6 & = & 3 \\ & & \frac{2}{5}x_4 + x_5 & + \frac{8}{5}x_6 & = & \frac{29}{5} \\ x_3 & + \frac{1}{5}x_4 & & - \frac{11}{5}x_6 & = & \frac{7}{5}. \end{array}$$

El corte de Gomory correspondiente a la primera de estas ecuaciones es:

$$\frac{2}{5}x_4 + \frac{3}{5}x_6 = \frac{4}{5} + x_7, \quad x_7 \in \mathbf{Z}_+.$$

Añadiendo este nuevo corte a $PR^{(2)}$ se obtiene $PR^{(3)}$. Éste tiene como solución:

$$\begin{array}{rclcl}
 x_0 & + & x_4 & + & x_7 = 29 \\
 x_1 & + & \frac{1}{3}x_4 & - & \frac{1}{3}x_7 = \frac{11}{3} \\
 x_2 & - & \frac{2}{3}x_4 & + & \frac{5}{3}x_7 = \frac{5}{3} \\
 & & \frac{5}{3}x_4 + x_5 & - & \frac{11}{3}x_7 = \frac{13}{3} \\
 x_3 & - & \frac{2}{3}x_4 & + & \frac{8}{3}x_7 = \frac{11}{3} \\
 & & \frac{2}{3}x_4 & + & x_6 - \frac{5}{3}x_7 = \frac{4}{3}.
 \end{array}$$

El siguiente corte se elige de la fila $x_2 - \frac{2}{3}x_4 + \frac{5}{3}x_7 = \frac{5}{3}$. Es⁴

$$\frac{1}{3}x_4 + \frac{2}{3}x_7 = \frac{2}{3} + x_8, \quad x_8 \in \mathbf{Z}_+.$$

La solución óptima de $PR^{(4)}$ —que también lo es del problema original—, resultante de añadir este último corte de Gomory, es

$$[x_0^{(4)}, x_1^{(4)}, \dots, x_6^{(4)}, x_7^{(4)}, x_8^{(4)}] = [28, 4, 0, 8, 0, 1, 3, 1, 0].$$

En términos de las variables originales, los tres cortes añadidos al problema son $x_2 \leq 3$, $2x_1 + x_2 \leq 9$ y $3x_1 + x_2 \leq 12$. El proceso que se ha seguido se ilustra en la figura 12.1. ●

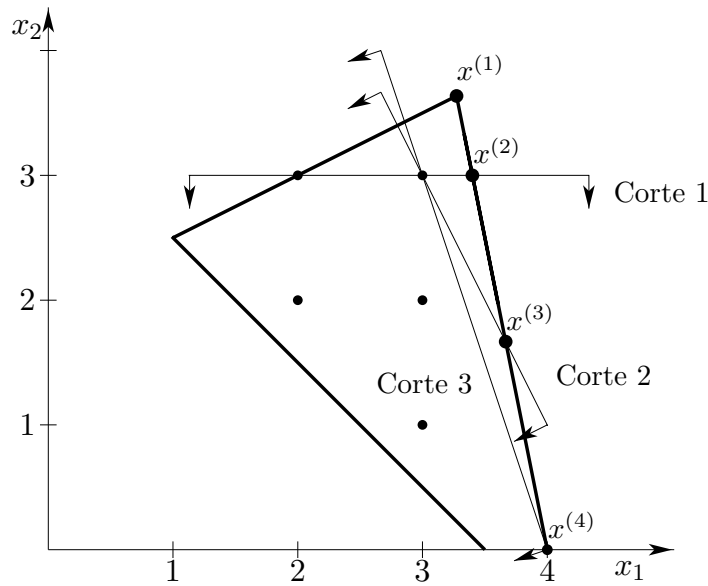


Figura 12.1

Resolución del problema del ejemplo 12.1 mediante el algoritmo de los planos cortantes de Gomory

⁴Recordemos $\lfloor -\frac{2}{3} \rfloor = -\frac{2}{3} - (-\frac{3}{3}) = \frac{1}{3}$.

12.1.1 Extensión a programas enteros mixtos

La extensión del procedimiento basado en los planos cortantes de Gomory para tratar programas enteros donde hay presencia de variables que no están restringidas a tomar valores enteros es inmediata. Supongamos para ello que en la relajación lineal de un programa entero mixto, PEM , $x_i \in \mathbf{Z}_+$ es una variable básica dada por

$$x_i + \sum_{h \in H_I} \bar{a}_{ij} x_j + \sum_{j \in H \setminus H_I} \bar{a}_{ij} y_j = \bar{a}_{i0},$$

donde H es el conjunto de índices de las variables no básicas, H_I el subconjunto de H de índices de las variables no básicas enteras y $\bar{a}_{i0} \notin \mathbf{Z}$. El corte de Gomory a usar en este caso es

$$\begin{aligned} & \sum_{(j \in H_I: f_{ij} < f_{i0})} f_{ij} x_j + \frac{f_{i0}}{1 - f_{i0}} \sum_{(j \in H_I: f_{ij} > f_{i0})} (1 - f_{ij}) x_j \\ & + \sum_{(j \in H \setminus H_I: \bar{a}_{ij} > 0)} \bar{a}_{ij} y_j - \frac{f_{i0}}{1 - f_{i0}} \sum_{(j \in H \setminus H_I: \bar{a}_{ij} < 0)} \bar{a}_{ij} y_j \geq f_{i0}. \end{aligned}$$

Todos los demás pasos del algoritmo permanecen exactamente igual.

En programación entera mixta no es razonable suponer que el valor de la función objetivo, x_0 , tendrá valor entero. No se puede usar, por consiguiente, la fila correspondiente a la función objetivo para definir los correspondientes cortes.

12.2 Algoritmos de ramificación y acotamiento o branch and bound

Estos algoritmos se inscriben dentro de la familia de los denominados *enumerativos*. El conjunto $\{S^{(i)} : i = 1, \dots, k\}$ se denomina *división de la región factible de un programa entero*, S , si $\cup_{i=1}^k S^{(i)} = S$. Una división se denomina *partición* si $S^{(i)} \cap S^{(j)} = \emptyset$ para $i, j = 1, \dots, k$, $i \neq j$.

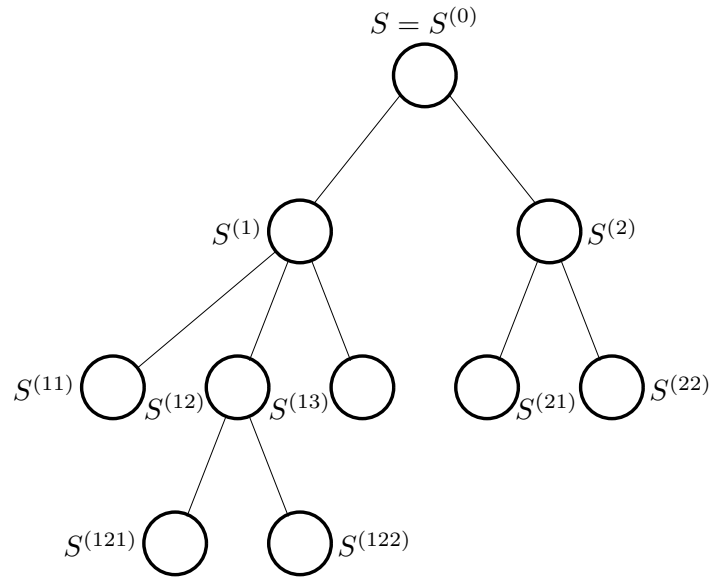
Proposición 12.2 *Sea el programa entero*

$$(PE^{(i)}) \quad \max. \left\{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in S^{(i)} \right\},$$

donde $\{S^{(i)}\}_{i=1}^k$ es una división de S . La función objetivo $z_{PE} = \max_{i=1, \dots, k} \{z_{PE}^{(i)}\}$.

Esta proposición expresa formalmente el concepto que coloquialmente se conoce como *divide y vencerás*. Es decir, si es difícil optimizar un programa entero en toda la región factible, S , quizás sea más fácil hacerlo por separado en pequeñas partes de ese S y luego unificar los resultados.

La *división* apuntada se suele hacer de forma *recursiva* según se ilustra en la figura 12.2. En ella, los *nudos hijos* de uno dado ($S^{(11)}$, $S^{(12)}$ y $S^{(13)}$ son los nudos hijos de $S^{(1)}$) representan una división de la región factible que atañe al *nudo padre*.

**Figura 12.2**

División recursiva de una región factible

Cuando $S \subseteq \mathbf{B}^n$ (variables 0 ó 1), la forma más simple de hacer la división recursiva apuntada es la indicada en la figura 12.3. En ella $S^{(\delta_1 \dots \delta_k)} = S \cap \{\mathbf{x} \in \mathbf{B}^n : x_j = \delta_j \in \{0, 1\}, \text{ para } j = 1, \dots, k\}$; esta división es una partición de S .

Llevada a un extremo, la división del conjunto factible contempla la enumeración de todos los elementos de S . En problemas prácticos, cuando el número de variables es muy grande, la enumeración y análisis de todos los elementos del conjunto factible es inviable. Para llevar a la práctica estos métodos hay que emplear algún procedimiento o regla para eliminar ramas en el árbol⁵ de búsqueda que se forma.

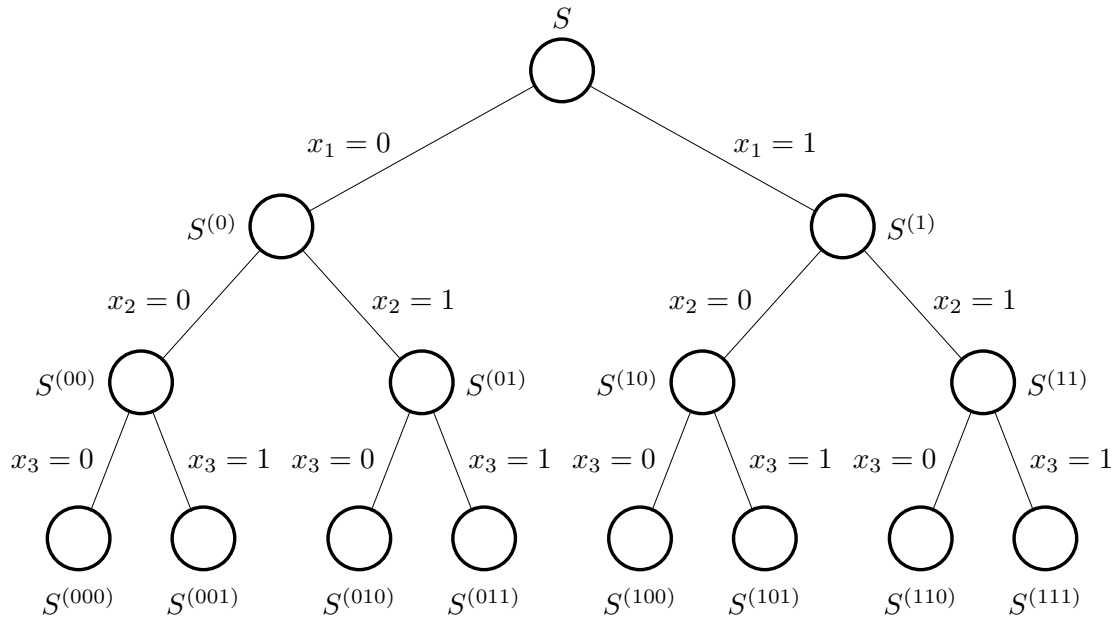
Si la región factible S se divide en subconjuntos $\{S^{(1)}, \dots, S^{(k)}\}$, y se puede establecer de alguna manera que no es necesario, a partir de un determinado momento, seguir dividiendo un $S^{(i)}$, se dice entonces que el árbol enumerativo puede *podarse* a partir del nudo correspondiente a ese $S^{(i)}$.

Proposición 12.3 *El árbol enumerativo se puede podar a partir del nudo correspondiente a $S^{(i)}$ si se cumplen cualquiera de las siguientes condiciones:*

1. *No factibilidad:* $S^{(i)} = \emptyset$.
2. *Optimalidad:* se ha llegado al óptimo de $PE^{(i)}$.
3. *Existencia de un valor dominante* $z_{PE}^{(i)} \leq z_{PE}$.

Para no tener que resolver el programa entero $PE^{(i)}$ se pueden usar diversos procedimientos.

⁵ A este árbol lo denominaremos en lo sucesivo *árbol enumerativo* o *árbol de enumeración*.


Figura 12.3

División recursiva de una región factible de un problema en variables 0 ó 1

Nosotros utilizaremos el más habitual: la relajación de programación lineal de ese $PE^{(i)}$; esto es $PL^{(i)}$ con $S^{(i)} \subseteq S_{PL}^{(i)}$ y $z_{PL}^{(i)}(\mathbf{x}) \geq \mathbf{c}^T \mathbf{x}$ para $\mathbf{x} \in S^{(i)}$.

Proposición 12.4 *El árbol enumerativo puede podarse a partir del nudo correspondiente a un conjunto $S^{(i)}$ si se cumple cualquiera de las condiciones siguientes:*

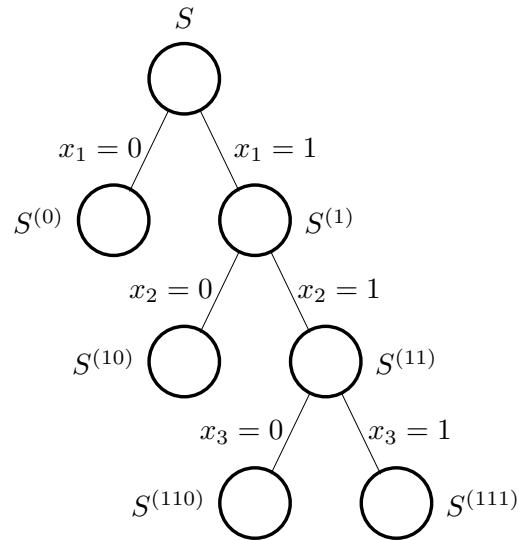
1. *El programa $PL^{(i)}$ no es factible.*
2. *La solución óptima $\mathbf{x}_{PL}^{(i)}$ de $PL^{(i)}$ cumple que $\mathbf{x}_{PL}^{(i)} \in S^{(i)}$ y que $z_{PL}^{(i)} = \mathbf{c}^T \mathbf{x}_{PL}^{(i)}$.*
3. *$z_{PL}^{(i)} \leq \underline{z}_{PE}$, donde $\underline{z}_{PE}$ es el valor de alguna solución factible de PE .*

DEMOSTRACIÓN. La condición 1 implica que $S^{(i)} = \emptyset$. La condición 2 implica que $\mathbf{x}_{PL}^{(i)}$ es la solución óptima de $PE^{(i)}$. La condición 3 implica que $z_{PE}^{(i)} \leq z_{PE}$. ■

Ejemplo 12.2 Considérese el programa entero

$$\begin{aligned}
 (Z_{PE}) \quad & \text{maximizar} \quad -100x_1 + 72x_2 + 36x_3 \\
 \text{s. a} \quad & -2x_1 + x_2 \leq 0 \\
 & -4x_1 + x_3 \leq 0 \\
 & x_1 + x_2 + x_3 \geq 0 \\
 & \mathbf{x} \in \mathbf{B}^3.
 \end{aligned}$$

La división de este ejemplo y su árbol enumerativo correspondiente son los de la figura 12.4.

**Figura 12.4**

División recursiva de la región factible del problema en variables 0 ó 1 del ejemplo 12.2

La solución de la relajación lineal de este programa entero es $[1/2, 1, 1]^T$, con un valor de la función objetivo igual a 58. Usando los criterios de la proposición 12.4 para podar el árbol enumerativo, se puede ver que

$$S^{(0)} = \{\mathbf{x} \geq \mathbf{0} : x_1 = 0, x_2 \leq 0, x_3 \leq 0, x_2 + x_3 \geq 1\} = \emptyset,$$

por lo que el nudo $S^{(0)}$ se desecha. La condición de óptimo de los nudos $S^{(110)}$ y $S^{(111)}$ se cumple pues estos conjuntos contienen las soluciones $[1, 1, 0]^T$ y $[1, 1, 1]^T$, respectivamente, por lo que, como, $z_{PE}^{(110)} < z_{PE}^{(111)} = 8$, se tendrá que $\underline{z}_{PE} = z_{PE}^{(111)} = 8$. Aplicando la tercera condición de la mencionada proposición al nudo $S^{(10)}$ se ve que $z_{PL}^{(10)} = -64 < \underline{z}_{PE}$, por lo que se desecha. Como resultado final se obtiene que $\mathbf{x} = [1, 1, 1]^T$ es la solución óptima del programa entero. El valor de su función objetivo es $Z_{PE} = 8$. ●

Usando relajación de algún tipo para desechar ramas, los métodos enumerativos siguen el esquema del algoritmo general presentado en la página 640. Los algoritmos enumerativos de relajación se conocen habitualmente en la literatura especializada como de *enumeración implícita* o *branch and bound*.⁶ El esquema general de tales algoritmos es el de la tabla 12.3. En ella, $\mathcal{L}$ designa una colección de programas enteros, $PE^{(i)}$, cada uno de los cuales tiene la forma $z_{PE}^{(i)} = \max. \{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in S^{(i)}\}$, donde $S^{(i)} \subseteq S$. Asociado a cada problema de $\mathcal{L}$ hay una cota superior del valor de la función objetivo del programa entero a resolver, $\bar{z}^{(i)} \geq z_{PE}^{(i)}$.

12.2.1 Algoritmos de ramificación y acotamiento con relajación lineal

Casi todos los códigos comercialmente disponibles para resolver problemas de programación entera generales que usan estrategias de tipo *branch and bound* utilizan relajación lineal y

⁶En lo sucesivo y en todo el libro designaremos estos algoritmos indistintamente como algoritmos de ramificación y acotamiento o *branch and bound*.

Tabla 12.3El algoritmo de ramificación y acotamiento o *branch and bound*

- Paso 0** – *Inicialización*: Hacer $\mathcal{L} = \{PE\}$, $S^{(0)} = S$, $\bar{z}^{(0)} = \infty$ y $\underline{z}_{PE} = -\infty$.
- Paso 1** – *Comprobación de final*. Si $\mathcal{L} = \emptyset$, la solución $\mathbf{x}$ que daba la función objetivo $\underline{z}_{PE} = \mathbf{c}^T \mathbf{x}$ es la óptima.
- Paso 2** – *Selección de problema, relajación lineal y resolución*. Seleccionar y borrar de $\mathcal{L}$ un problema $PE^{(i)}$. Resolver su relajación lineal $PL^{(i)}$. Sea $z_{PL}^{(i)}$ el valor óptimo de la función objetivo de este problema y $\mathbf{x}_{PL}^{(i)}$ su solución óptima (si existe).
- Paso 3** – *Poda*. Si $z_{PL}^{(i)} \leq \underline{z}_{PE}$ ir al paso 1 (nótese que si la relajación se resuelve por un algoritmo dual, este paso es aplicable tan pronto como el valor de la función objetivo del programa dual se hace inferior a $\underline{z}_{PE}$).
 Si $\mathbf{x}_{PL}^{(i)} \notin S^{(i)}$ ir al paso 4.
 Si $\mathbf{x}_{PL}^{(i)} \in S^{(i)}$ y $\mathbf{c}^T \mathbf{x}_{PL}^{(i)} > \underline{z}_{PE}$, hacer $\underline{z}_{PE} = \mathbf{c}^T \mathbf{x}_{PL}^{(i)}$. Borrar de $\mathcal{L}$ todos los problemas tales que $\bar{z}^{(i)} \leq \underline{z}_{PE}$. Si $\mathbf{c}^T \mathbf{x}_{PL}^{(i)} = z_{PL}^{(i)}$ ir al paso 1; si no, al paso 4.
- Paso 4** – *División*. Sea $\{S^{(ij)}\}_{j=1}^k$ una división de S . Añadir los problemas $\{PE^{(ij)}\}_{j=1}^k$ a $\mathcal{L}$, donde $\bar{z}^{(ij)} = z_{PL}^{(i)}$ para $j = 1, \dots, k$.
 Ir al paso 1.

divisiones. En este apartado consideraremos los programas enteros generales

$$(PE) \quad \max. \left\{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in S \right\}, \quad \text{donde } S = \{ \mathbf{x} \in \mathbf{Z}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0} \},$$

y la forma de resolverlos utilizando el procedimiento de ramificación y acotamiento o *branch and bound* que utiliza relajaciones lineales. Aun cuando por simplicidad en la exposición nos referiremos a programas enteros del tipo que definimos en su momento como puro, la realidad es que la extensión de todo lo que expliquemos a programas entero-mixtos

$$(PEM) \quad \max. \left\{ \mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y} : [\mathbf{x}^T, \mathbf{y}^T]^T \in T \right\},$$

donde

$$T = \{ \mathbf{x} \in \mathbf{Z}^n, \mathbf{y} \in \mathbb{R}^m : \mathbf{x} \geq \mathbf{0}, \mathbf{y} \geq \mathbf{0}, A\mathbf{x} + G\mathbf{y} \leq \mathbf{b} \},$$

es inmediata.

En la relajación inicial, el conjunto factible, S , se reemplaza por $S_{PL}^{(0)} = \{ \mathbf{x} \in \mathbb{R}^n : \mathbf{x} \geq \mathbf{0}, A\mathbf{x} \leq \mathbf{b} \}$, haciéndose en cada relajación $z_{PL}(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$.

12.2.1.1 Criterios de poda o rechazo de ramas del árbol enumerativo

Al resolver las relajaciones lineales, los criterios de rechazo indicados en la proposición 12.3, esto es, no factibilidad, óptimo y valor dominante, son aplicables directamente. Supóngase que la relajación lineal en el nudo i del árbol enumerativo es

$$(PL^{(i)}) \quad \max. \left\{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in S_{PL}^{(i)} \right\}, \quad \text{donde } S_{PL}^{(i)} = \left\{ \mathbf{x} \in \mathbb{R}^n : \mathbf{x} \geq \mathbf{0}, A^{(i)}\mathbf{x} \leq \mathbf{b}^{(i)} \right\}.$$

Si el programa lineal $PL^{(i)}$ tiene solución óptima, designémosla para facilitar la notación por $\mathbf{x}^{(i)}$. Las condiciones para eliminar, rechazar o “podar”⁷ el nudo i , y por tanto sus nudos hijos, son:

1. $S_{PL}^{(i)} = \emptyset$ (no factibilidad);
2. $\mathbf{x}^{(i)} \in \mathbf{Z}_+^n$ (óptimo) y
3. $z_{PL}^{(i)} \leq \underline{z}_{PE}$, donde $\underline{z}_{PE}$ es el mejor valor (valor dominante) de las soluciones factibles de PE encontradas hasta ese momento. Obsérvese que si se resuelve $PL^{(i)}$ por el algoritmo dual del *simplex*, el rechazo se puede llevar a cabo antes de encontrar el óptimo de $PL^{(i)}$. También se puede usar un criterio del tipo $z_{PL}^{(i)} \leq \underline{z}_{PE} + \varepsilon$, para una tolerancia $\varepsilon > 0$ dada.

12.2.1.2 División

Como se usa relajación lineal en cada nudo del árbol enumerativo, la división se debe hacer añadiendo condiciones lineales. Una forma evidente de hacer esto es tomar $S = S^{(1)} \cup S^{(2)}$ con $S^{(1)} = S \cap \{\mathbf{x} \in \Re^n : \mathbf{x} \geq \mathbf{0}, \mathbf{d}^T \mathbf{x} \leq d_0\}$ y $S^{(2)} = S \cap \{\mathbf{x} \in \Re^n : \mathbf{x} \geq \mathbf{0}, \mathbf{d}^T \mathbf{x} \geq d_0 + 1\}$, donde $[\mathbf{d}^T, d_0]^T \in \mathbf{Z}^{n+1}$. Si $\mathbf{x}^{(0)}$ es la solución del programa lineal (relajación inicial) inicial:

$$(PL^{(0)}) \quad \max. \left\{ \mathbf{c}^T \mathbf{x} : \mathbf{x} \in \Re^n, \mathbf{x} \geq \mathbf{0}, A\mathbf{x} \leq \mathbf{b} \right\},$$

el vector $[\mathbf{d}^T, d_0]^T$ se puede escoger de tal manera que $d_0 < \mathbf{d}^T \mathbf{x}^{(0)} < d_0 + 1$. Proceder así es lo más recomendable pues se obtiene un $\mathbf{x}^{(0)} \notin S_{PL}^{(1)} \cup S_{PL}^{(2)}$, lo que permite la posibilidad de que, para $i = 1, 2$, $z_{PL}^{(i)} = \max. \{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in S_{PL}^{(i)}\} < z_{PL}^{(0)}$.

En la práctica sólo se usan vectores $[\mathbf{d}^T, d_0]^T$ muy concretos:

- *Dicotomías de variables.* En este caso $\mathbf{d} = \mathbf{e}_j$ para algún $j \in N$. El punto $\mathbf{x}^{(0)}$ será no factible en las relajaciones resultantes si $x_j^{(0)} \notin \mathbf{Z}$ y $d_0 = \lfloor x_j \rfloor$ (ver figura 12.5). Nótese que si $x_j \in \mathbf{B}$ la rama izquierda hace $x_j = 0$ y la derecha $x_j = 1$.

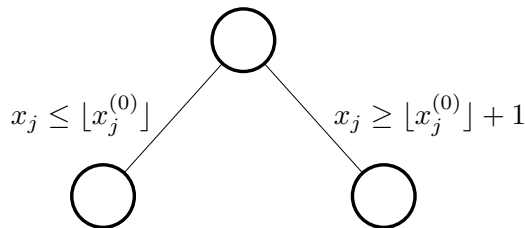


Figura 12.5

División, por dicotomía de la variable x_j , en un árbol enumerativo

Una importante ventaja práctica de esta división es que las condiciones que se añaden a la relajación lineal correspondiente son simples cotas inferiores o superiores. Sólo será

⁷Usaremos indistintamente estos términos.

necesario en la optimización del nuevo programa lineal tener en cuenta esas nuevas cotas y resolverlo mediante el método dual del *simplex*; el tamaño de la base seguirá siendo el mismo.

- *Dicotomías de cotas superiores generalizadas.* Si el problema tuviese *cotas superiores generalizadas*, esto es $\sum_{j \in Q} x_j = 1$, para algún $Q \subseteq N$, la división a llevar cabo sería la de la figura 12.6. Obsérvese que $\mathbf{x}^{(0)}$ será no factible en la relajación resultante si $0 < \sum_{j \in Q_1} x_j^{(0)} < 1$, donde Q_1 es un subconjunto no vacío de Q .

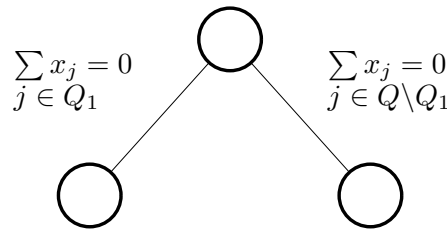


Figura 12.6

Dicotomía debida a la existencia de cotas superiores generalizadas

- Si una variable x_j está acotada ($l_j \leq x_j \leq u_j$), otra forma de proceder sería considerar cada valor entero posible de x_j por separado (ver figura 12.7). Como se puede comprender, si el número de variables es un poco elevado, pudiendo tomar cada una varios valores enteros, el problema puede adquirir un tamaño gigantesco. Ningún código comercial usa esta posibilidad.

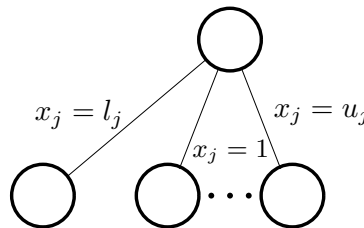


Figura 12.7

División del árbol enumerativo en tantas ramas como valores enteros puede tomar la variable x_j

Nótese que cada una de las divisiones expuestas es una partición. En lo que sigue supondremos que la división se realiza exclusivamente por dicotomías de variables.

Proposición 12.5 Si $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}\}$ es acotado, un árbol enumerativo desarrollado a partir de dicotomías de las variables es finito, supuesto que en cada nudo i donde se requiera una división se escoge una dicotomía de la forma $x_j \leq \lfloor x_j^{(i)} \rfloor$, $x_j \geq \lfloor x_j^{(i)} \rfloor + 1$, donde $x_j^{(i)}$ no es entera. En particular, si $\omega_j = \lceil \max\{x_j : \mathbf{x} \in P\} \rceil$, ningún camino del árbol enumerativo puede contener más de $\sum_{j \in N} \omega_j$ ramas.

DEMOSTRACIÓN. Una vez añadida la condición $x_j \leq d$, para algún $d \in \{0, \dots, \omega_j - 1\}$, las únicas condiciones que pueden aparecer más adelante en un camino desde el nudo raíz a una rama del árbol enumerativo son $x_j \leq d'$, para $d' \in \{0, \dots, d - 1\}$, y $x_j \geq \bar{d}$, para $\bar{d} \in \{1, \dots, d\}$. De aquí se deduce que el número máximo de condiciones que involucren a x_j surgirán de añadir $x_j \leq d$ para todo $d \in \{0, \dots, \omega_j - 1\}$, $x_j \geq d$ para todo $d \in \{1, \dots, \omega_j\}$, $x_j \geq d$ para todo $d \in \{1, \dots, \alpha\}$ y $x_j \leq d$ para todo $d \in \{\alpha, \dots, \omega_j - 1\}$. Cada uno de estos casos requiere ω_j condiciones de x_j habiendo, por consiguiente, $\sum_{j \in N} \omega_j$ en total en cualquier camino. ■

El tamaño del árbol enumerativo depende en gran medida de la bondad de las cotas obtenidas de las relajaciones lineales que se llevan a cabo.

Proposición 12.6 Si el nudo t del árbol enumerativo con conjunto de condiciones $S^{(t)}$ es tal que $\max. \{\mathbf{c}^T \mathbf{x} : \mathbf{x} \in S_{PL}^{(t)}\} > z_{PE}$, ese nudo no se puede podar o rechazar.

Esto indica que, independientemente de cómo se desarrolle el árbol enumerativo, las cotas obtenidas de las relajaciones lineales son el factor primordial que caracteriza la eficiencia del algoritmo *branch and bound*. En el apartado siguiente analizamos las estrategias a tener en cuenta a la hora de seleccionar qué nudo del árbol enumerativo considerar a partir de un punto dado del proceso de optimización y qué variable se debe seleccionar para llevar a cabo una división de ese árbol.

12.2.1.3 Selección del nudo a estudiar

Una de las operaciones de más trascendencia del algoritmo *branch and bound* queda reflejada en el paso 2 de éste: dada una lista $\mathcal{L}$ de subproblemas activos o, dicho de otra forma, de subárboles del árbol enumerativo con nudos no rechazados, qué nudo elegir como el próximo a examinar. Las opciones son dos: (1) analizar uno siguiendo unas reglas preestablecidas o (2) escoger uno de acuerdo con la información disponible en ese momento sobre cotas de las variables, número de nudos activos, etc. La primera podríamos encuadrarla dentro de un apartado denominado de reglas apriorísticas; la segunda en otro de reglas adaptativas.

Existen multitud de reglas apriorísticas que se pueden utilizar. La más extendida es la denominada LIFO (del inglés *Last In First Out*). Viene a decir que primero se deben analizar todos los nudos hijos de un determinado nudo y luego volver hacia atrás y escoger otro del mismo nivel del nudo padre que no haya sido todavía analizado. Esta regla, junto con otra que determine qué variable va a ser la de ramificación siguiente y con cuál de las dos ramas a que da lugar hay que empezar, permiten analizar el problema en su totalidad. Un ejemplo se ilustra en la figura 12.8. En ella se han numerado los nudos según el orden en que se consideran. Los nudos subrayados son los podados o rechazados. La regla LIFO tiene dos ventajas fundamentales:

1. La relajación lineal de un nudo hijo se obtiene de la lineal del nudo padre sin más que añadir una condición simple de cota en una variable. A partir de la solución óptima del programa lineal que define el nudo padre, utilizando el método dual del *simplex*, sin reinvertir la base ni modificar las estructuras de datos sustancialmente, se obtiene la que define el nudo hijo.
2. La experiencia dice que las soluciones factibles se encuentran más bien en puntos *profundos* del árbol enumerativo.

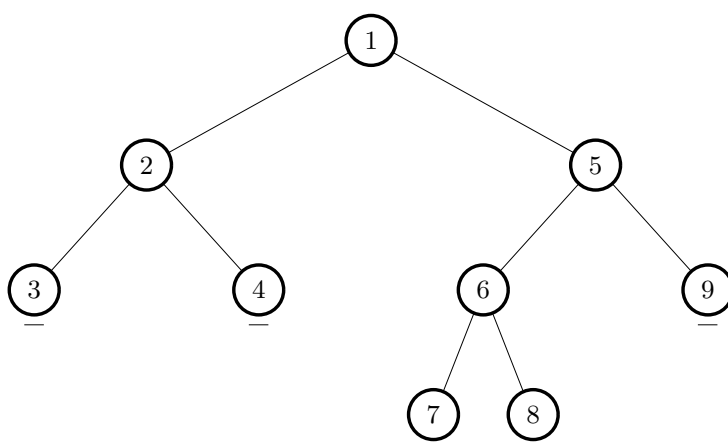


Figura 12.8

Selección de los nudos de un árbol enumerativo de acuerdo con la regla LIFO

Otra regla apriorística que se puede utilizar es la denominada *búsqueda en amplitud o anchura*. Consiste en analizar todos los nudos de un determinado nivel antes de pasar a otro más profundo. Aunque esta variante no obtiene resultados prácticos relevantes comparándolos con los de la anterior, sí es muy usada en códigos especializados en la resolución de programas especiales en variables binarias que utilizan técnicas heurísticas.

Cualquiera que sea la regla, los criterios más razonables que deben considerarse para seleccionar un nudo para su análisis son:

- Escoger aquel que se ha de estudiar en cualquier caso. Si de acuerdo con la proposición 12.6 sólo existe un nudo con la cota más alta, estudiarlo. Es decir, cuando un nudo se rechaza, escoger aquel de los todavía no analizados que tenga la cota superior más alta independientemente de su posición en el árbol. Dicho de otra forma, si $\mathcal{L}$ es el conjunto de nudos a analizar, escoger aquel $i \in \mathcal{L}$ que maximice $\bar{z}^{(i)}$.
- Escoger aquel nudo que con mayor probabilidad contenga una solución óptima. Aunque esto puede parecer evidente, la razón de proceder así estriba en que una vez obtenida una solución óptima, aunque no se pueda inmediatamente probar que es el caso, sí se habrá conseguido el máximo valor posible de z_{PE} . Esto es muy importante para sucesivas podas. Para hacerlo, si $\hat{z}^{(i)} \leq \bar{z}^{(i)}$, escoger un nudo $i \in \mathcal{L}$ que maximice $\hat{z}^{(i)}$.

- A pesar de que intentar encontrar una solución óptima es muy deseable, a veces puede resultar más práctico tratar de encontrar rápidamente una solución factible $\hat{\mathbf{x}}$ tal que $\mathbf{c}^T \hat{\mathbf{x}} > \underline{z}_{PE}$. El criterio

$$\max_{i \in \mathcal{L}} \frac{\bar{z}^{(i)} - \underline{z}_{PE}}{\bar{z}^{(i)} - \hat{z}^{(i)}},$$

que se denomina de *mejora rápida*, trata de conseguir este objetivo. Obsérvese que un nudo i en el que $\hat{z}^{(i)} > \underline{z}_{PE}$ será más adecuado que otro j en el que $\hat{z}^{(j)} \leq \underline{z}_{PE}$. Más aún, los nudos para los cuales $\bar{z}^{(i)} - \hat{z}^{(i)}$ es pequeña deberán tener preferencia. Es de suponer que tales nudos proporcionarán rápidamente una solución factible del problema. El criterio de la mejora rápida se usa en bastantes códigos comerciales como la opción por defecto para utilizar una vez que se conoce la primera solución factible.

12.2.1.4 Selección de la variable de ramificación

Supongamos que ya se ha elegido el nudo i que se va a analizar. Asociado a ese nudo habrá un programa lineal de solución $\mathbf{x}^{(i)}$. El siguiente paso a llevar a cabo —paso 4 según la tabla 12.3 de la página 649— consiste en escoger la variable que va a definir la división que parta de ese nudo i . En lo que sigue restringiremos esas posibilidades de elección al conjunto índice $N^{(i)} = \{j \in N : x_j^{(i)} \notin \mathbf{Z}\}$. La evidencia empírica de muchos años de utilización de este procedimiento demuestra la importancia fundamental que tiene el escoger bien un $j \in N^{(i)}$. Los criterios para ello suelen variar de unos programas a otros. Los que exponemos a continuación son los más aceptados y se basan en el cálculo previo a la toma de esa decisión de unas *degradaciones* o *penalizaciones* en que se puede incurrir en el valor de $\bar{z}^{(i)}$ al requerir que una variable x_j que no es entera deba serlo.

12.2.1.4.1 Selección basada en penalizaciones

Para la exposición que sigue mantenemos esencialmente la notación introducida en la expresión 12.1 de la página 640; esto es, el vector solución del programa lineal que está siendo considerado en el nudo i es

$$x_{B_i} = \bar{a}_{i0} + \sum_{j=m+1}^n \bar{a}_{ij}(-x_j) \quad \text{para } i = 1, \dots, m,$$

y el valor de la función objetivo

$$x_0 = \bar{a}_{00} + \sum_{j=m+1}^n \bar{a}_{0j}(-x_j).$$

Obsérvese que en esta última expresión los $\bar{a}_{0j}$, $j = m+1, \dots, n$, son los costes reducidos de las variables no básicas. Consideraremos la elección de la variable de ramificación dentro de dos apartados generales: que esa variable se elige del conjunto de las básicas, y del de las no básicas.

Variables básicas

Supongamos que en el nudo i alguna variable básica x_p no es entera debiendo serlo, es decir

$$x_p = \bar{a}_{p0} + \sum_{j=m+1}^n \bar{a}_{pj}(-x_j) = n_{p0} + f_{p0},$$

donde n_{p0} designa la parte entera de x_p , $\lfloor x_p \rfloor$, y f_{p0} la parte fraccionaria.⁸

Como ya mencionamos en el apartado correspondiente, la división que definirá la variable x_p es

$$x_p \geq n_{p0} + 1 \quad \text{y} \quad x_p \leq n_{p0}.$$

De la teoría de dualidad, y más concretamente del algoritmo dual del *simplex* de la tabla 8.3, página 483, podemos deducir que la imposición de la nueva cota $x_p \geq n_{p0} + 1$ a la variable x_p implicará un empeoramiento (reducción) del valor de la función objetivo; como la variable básica x_p , realizando una iteración del método dual del *simplex*, pasaría como mínimo a ser no básica, otra variable no básica, x_j , que estuviese en uno de sus límites, se incrementaría o decrementaría según estuviese en el inferior o superior. Es decir, se reduciría el valor de la función objetivo en una cantidad o *penalización* que vendría dada por la expresión

$$P_U = \min_{j, \bar{a}_{pj} < 0} (1 - f_{p0}) \frac{\bar{a}_{0j}}{-\bar{a}_{pj}}, \quad \text{si } x_j = l_j,$$

o por

$$P_U = \min_{j, \bar{a}_{pj} > 0} (1 - f_{p0}) \frac{\bar{a}_{0j}}{-\bar{a}_{pj}}, \quad \text{si } x_j = u_j.$$

Razonando de manera similar, la imposición de la nueva cota $x_p \leq n_{p0}$ a la variable x_p implicará un empeoramiento (reducción) del valor de la función objetivo que vendrá dado por la expresión

$$P_D = \min_{j, \bar{a}_{pj} > 0} f_{p0} \frac{\bar{a}_{0j}}{\bar{a}_{pj}}, \quad \text{si } x_j = l_j,$$

o por

$$P_D = \min_{j, \bar{a}_{pj} < 0} f_{p0} \frac{\bar{a}_{0j}}{\bar{a}_{pj}}, \quad \text{si } x_j = u_j.$$

Cualquier solución entera que se pudiese obtener partiendo del nudo i estaría por consiguiente acotada superiormente por

$$\max\{x_0 - P_U, x_0 - P_D\}. \quad (12.2)$$

Estas cotas no son necesariamente las menores, de las superiores, que impone el hecho de que la variable haya de ser entera (volveremos sobre este asunto más adelante). Se podrá conseguir una mejor solución desde el nudo i si y sólo si

$$\min\{P_U, P_D\} < x_0 - \bar{z}_{PE} \quad (12.3)$$

para cada variable básica que no sea entera debiendo serlo. Si esta condición no se cumple se abandona el nudo i pues no tiene interés seguir analizándolo.

⁸Recordemos $\lfloor 3, 22 \rfloor = 3$; $\lfloor -4, 12 \rfloor = -5$.

Si todas las variables x_p cumplen la condición (12.3), la cuestión siguiente a solventar es qué variable de ramificación se elige. Existen dos principios básicos que cabe considerar para tomar esta decisión. El primero es elegir aquella con la penalización asociada más pequeña. El segundo, escoger aquella variable con una penalización asociada más grande y comenzar imponiendo el límite opuesto al que determina esa penalización. Si, por ejemplo, la penalización más grande asociada a una variable x_p es P_U , comenzar analizando el problema que resulte de imponer a esa variable la nueva cota n_{p0} incluyendo en la lista de nudos a analizar más adelante el que resulte de imponer la cota $n_{p0} + 1$. La justificación de este proceder es evidente: si se almacenan las peores soluciones, una vez que se encuentre una buena, se rechazarán rápidamente buena parte de los nudos que queden en la lista.

Antes de continuar conviene aclarar una cuestión. Cuando varias de las variables (no todas) que debiendo ser enteras y no lo son violan la condición (12.3), conviene simultáneamente acotar todas esas variables con los límites opuestos a los que violan la condición, almacenando dichos límites en la lista: es probable que sean rechazados muy poco después.

Variables no básicas

En el apartado anterior se han analizado las penalizaciones que sobre la función objetivo acarrear la imposición de unos nuevos límites al problema como consecuencia del hecho de que una determinada variable básica, debiendo ser entera, no lo es.

Si una variable x_q es no básica en una determinada solución, ha de ser entera y está en uno de sus límites superior o inferior entero, cualquier cambio en su estado, para seguir siendo entera, deberá serlo en una cantidad como mínimo igual a 1. La penalización por efectuar este cambio está inmediatamente determinada por el coste reducido de esta variable, es decir $\bar{a}_{0q}$. Si la mejor solución entera factible obtenida hasta ese momento es $\mathbf{x}_{0c}$, la que determina —continua o entera— aquella en la que la variable que nos ocupa toma el valor x_q es $\mathbf{x}_0$ y la penalización $\bar{a}_{0q}$ cumple que $\bar{a}_{0q} \geq \mathbf{x}_0 - \mathbf{x}_{0c}$, está claro que no interesa moverla del estado en que se encuentra pues nunca mejoraría $\mathbf{x}_{0c}$: se impondría un nuevo límite inferior o superior a x_q igual al límite en el que estuviese para que en lo sucesivo permaneciese fija. Todo esto, claro está, para cualquier rama de búsqueda de inferior nivel que partiese de algún nudo en el que se hacen estas consideraciones.

La condición de que las variables no básicas también han de ser enteras se puede usar para determinar unas penalizaciones P_U y P_D más estrictas. En efecto, recordemos que al imponer una nueva cota $x_p \geq n_{p0} + 1$ a una variable, suponiendo que en una iteración del método dual del *simplex* esa x_p pasase a ser como mínimo no básica con esa nueva cota, una variable no básica x_q habría de moverse del límite en el que estuviese. Si estuviese en su límite inferior o superior, ese incremento vendría dado por la expresión

$$\frac{1 - f_{p0}}{-\bar{a}_{pq}} \quad \text{si } \bar{a}_{pq} < 0 \text{ y } x_q = l_q \quad (12.4a)$$

o por

$$\frac{1 - f_{p0}}{-\bar{a}_{pq}} \quad \text{si } \bar{a}_{pq} > 0 \text{ y } x_q = u_q. \quad (12.4b)$$

De igual manera, al imponer la nueva cota $x_p \leq n_{p0}$ a la variable, alguna variable no básica x_q

variaría su valor en una cantidad dada por la expresión

$$\frac{f_{p0}}{\bar{a}_{pq}} \quad \text{si } \bar{a}_{pq} > 0 \text{ y } x_q = l_q \quad (12.5a)$$

o por

$$\frac{f_{p0}}{\bar{a}_{pq}} \quad \text{si } \bar{a}_{pq} < 0 \text{ y } x_q = u_q. \quad (12.5b)$$

Ahora bien, al considerar variables x_q no básicas enteras, las cantidades (12.4) y (12.5) serán por lo general no enteras. Si son estrictamente mayor que cero y menor que uno, se puede volver a usar el hecho de que x_q debe ser entera, en el sentido de pensar que si cambia su estado, para seguir siendo entera, ese cambio deberá ser una unidad, como mínimo, siendo la penalización de la función objetivo por ese cambio su coste reducido $\bar{a}_{0q}$. En consecuencia, la penalización por incrementar la variable básica x_p a, al menos, $n_{p0} + 1$, incrementando o decrementando una variable no básica x_q que debe ser entera, será

$$\max \left\{ \bar{a}_{0q}, \bar{a}_{0q} \frac{1 - f_{p0}}{-\bar{a}_{pq}} \right\} \quad \text{si } \bar{a}_{pq} < 0 \text{ y } x_q = l_q$$

o

$$\max \left\{ \bar{a}_{0q}, \bar{a}_{0q} \frac{1 - f_{p0}}{-\bar{a}_{pq}} \right\} \quad \text{si } \bar{a}_{pq} > 0 \text{ y } x_q = u_q.$$

De igual manera, la penalización por decrementar x_p a, al menos, n_{p0} , incrementando o decrementando una variable no básica x_q que debe ser entera, será

$$\max \left\{ \bar{a}_{0q}, \bar{a}_{0q} \frac{f_{p0}}{\bar{a}_{pq}} \right\} \quad \text{si } \bar{a}_{pq} > 0 \text{ y } x_q = l_q$$

o

$$\max \left\{ \bar{a}_{0q}, \bar{a}_{0q} \frac{f_{p0}}{\bar{a}_{pq}} \right\} \quad \text{si } \bar{a}_{pq} < 0 \text{ y } x_q = u_q.$$

Las nuevas penalizaciones P_U y P_D que utilizar serán pues

$$P_U^* = \min_{j, \bar{a}_{pj} < 0} \left\{ \begin{array}{l} \bar{a}_{0j}(1 - f_{p0})/(-\bar{a}_{pj}), \quad j \in M \\ \max\{\bar{a}_{0j}, \bar{a}_{0j}(1 - f_{p0})/(-\bar{a}_{pj})\}, \quad j \in J \end{array} \right\} \quad \text{si } x_j = l_j \quad (12.6a)$$

o

$$P_U^* = \min_{j, \bar{a}_{pj} > 0} \left\{ \begin{array}{l} \bar{a}_{0j}(1 - f_{p0})/(-\bar{a}_{pj}), \quad j \in M \\ \max\{\bar{a}_{0j}, \bar{a}_{0j}(1 - f_{p0})/(-\bar{a}_{pj})\}, \quad j \in J \end{array} \right\} \quad \text{si } x_j = u_j \quad (12.6b)$$

y

$$P_D^* = \min_{j, \bar{a}_{pj} > 0} \left\{ \begin{array}{l} \bar{a}_{0j}f_{p0}/\bar{a}_{pj}, \quad j \in M \\ \max\{\bar{a}_{0j}, \bar{a}_{0j}f_{p0}/\bar{a}_{pj}\}, \quad j \in J \end{array} \right\} \quad \text{si } x_j = l_j \quad (12.7a)$$

o

$$P_D^* = \min_{j, \bar{a}_{pj} < 0} \begin{cases} \bar{a}_{0j} f_{p0} / \bar{a}_{pj}, & j \in M \\ \max\{\bar{a}_{0j}, \bar{a}_{0j} f_{p0} / \bar{a}_{pj}\}, & j \in J \end{cases} \quad \text{si } x_j = u_j, \quad (12.7b)$$

donde J es el conjunto índice de las variables no básicas que han de ser enteras y M el de las no básicas que no han de ser enteras.

Estas penalizaciones más estrictas benefician el poder encontrar una buena solución entera sin necesidad de analizar todos los nudos. Igual que antes, cualquier solución entera que se pudiese obtener a partir del nudo i estaría acotada superiormente por

$$\max\{x_0 - P_U^*, x_0 - P_D^*\}.$$

El nudo i se podrá desechar si, para cualquier x_p ,

$$\min\{P_U^*, P_D^*\} \geq x_0 - \underline{z}_{PE}. \quad (12.8)$$

Como ya adelantábamos, este criterio no es el único que se puede adoptar. En efecto, recordemos que el corte de Gomory asociado a x_p ,

$$-f_{p0} + \sum_{j=m+1}^n f_{pj} x_j \geq 0,$$

se habría de satisfacer en cualquier solución que se obtuviese partiendo del problema actual.

Asociado a ese corte de Gomory se puede derivar la penalización P_G en que se incurrirá por cumplirla. Su expresión es:

$$P_G^* = \min_j \begin{cases} f_{p0} \bar{a}_{0j} / \bar{a}_{pj} & \text{si } \bar{a}_{pj} \geq 0 \quad j \in M \\ (1 - f_{p0}) \bar{a}_{0j} / (-\bar{a}_{pj}) & \text{si } \bar{a}_{pj} < 0 \quad j \in M \\ f_{p0} \bar{a}_{0j} / f_{pj} & \text{si } f_{pj} \leq f_{p0} \quad j \in J \\ (1 - f_{p0}) \bar{a}_{0j} / (1 - f_{pj}) & \text{si } f_{pj} > f_{p0} \quad j \in J \end{cases} \quad \text{si } x_j = l_j; \quad (12.9a)$$

$$P_G^* = \min_j \begin{cases} f_{p0} \bar{a}_{0j} / \bar{a}_{pj} & \text{si } \bar{a}_{pj} \leq 0 \quad j \in M \\ (1 - f_{p0}) \bar{a}_{0j} / (-\bar{a}_{pj}) & \text{si } \bar{a}_{pj} > 0 \quad j \in M \\ f_{p0} (-\bar{a}_{0j}) / f_{pj} & \text{si } f_{pj} \leq f_{p0} \quad j \in J \\ (1 - f_{p0}) (-\bar{a}_{0j}) / (1 - f_{pj}) & \text{si } f_{pj} > f_{p0} \quad j \in J \end{cases} \quad \text{si } x_j = u_j. \quad (12.9b)$$

Comparando (12.9), (12.6) y (12.7) se deduce inmediatamente que

$$P_G^* \geq \min(P_U^*, P_D^*),$$

lo que trae como consecuencia que la condición (12.8) para desechar el nudo i se puede reemplazar por

$$P_G^* \geq x_0 - \underline{z}_{PE}.$$

El resumen de todas estas consideraciones es que, incorporando los sencillos cálculos descritos al analizar un nudo, es posible determinar unas penalizaciones que permiten desechar muchas alternativas de ramificación: esto permite reducir sustancialmente el tiempo de cálculo de obtención de la solución óptima de un programa entero (puro o mixto) por el procedimiento *branch and bound*.

Ejemplo 12.3 Volvamos al ejemplo 12.1 de la página 641:

$$\begin{aligned}
 (PE) \quad & \text{maximizar} \quad 7x_1 + 2x_2 \\
 & \text{s. a} \quad \begin{aligned} -x_1 + 2x_2 &\leq 4 \\ 5x_1 + x_2 &\leq 20 \\ -2x_1 - 2x_2 &\leq -7 \end{aligned} \\
 & \quad \mathbf{x} \in \mathbf{Z}_+^2.
 \end{aligned}$$

Una vez introducidas las variables de holgura x_1 , x_2 y x_3 , la solución de la relajación lineal de este programa entero, como hemos visto, es

$$\begin{aligned}
 x_0 &+ \frac{3}{11}x_3 + \frac{16}{11}x_4 = \frac{332}{11} \\
 x_1 &- \frac{1}{11}x_3 + \frac{2}{11}x_4 = \frac{36}{11} \\
 x_2 &+ \frac{5}{11}x_3 + \frac{1}{11}x_4 = \frac{40}{11} \\
 &\frac{8}{11}x_3 + \frac{6}{11}x_4 + x_5 = \frac{75}{11},
 \end{aligned}$$

donde $x_3 = x_4 = 0$. De aquí que $z_{PL}^{(0)} = x_0 = 332/11$ y $\mathbf{x}^{(0)} = [36/11, 40/11, 0, 0, 75/11]^T$.

Ilustremos la utilización de las penalizaciones. Las variables básicas son x_1 , x_2 y x_5 . Esta última no se requiere que sea entera. Las variables no básicas de la solución obtenida están todas en su límite inferior.

Para empezar, como todavía no tenemos ninguna solución factible de z_{PE} , determinemos las penalizaciones por bifurcar a:

$$x_1 \leq 3:$$

$$P_{1D}^{0-} = \min_{j, \bar{a}_{pj} > 0} \{ \bar{a}_{0j} f_{p0} / \bar{a}_{pj}, j \in M \} = \min \left\{ \frac{16}{11} \cdot \frac{3}{11} / \frac{2}{11} \right\} = \frac{24}{11}.$$

$$x_1 \geq 4:$$

$$P_{1U}^{0+} = \min_{j, \bar{a}_{pj} < 0} \{ \bar{a}_{0j} (1 - f_{p0}) / (-\bar{a}_{pj}), j \in M \} = \min \left\{ \frac{3}{11} \cdot (1 - \frac{3}{11}) / \frac{1}{11} \right\} = \frac{24}{11}.$$

$$x_2 \leq 3:$$

$$P_{2D}^{0-} = \min_{j, \bar{a}_{pj} > 0} \{ \bar{a}_{0j} f_{p0} / \bar{a}_{pj}, j \in M \} = \min \left\{ \frac{16}{11} \cdot \frac{7}{11} / \frac{1}{11}, \frac{3}{11} \cdot \frac{7}{11} / \frac{5}{11} \right\} = \frac{21}{55}.$$

La penalización P_{2D}^{0+} la podemos considerar ∞ pues de la solución del programa lineal anterior se observa que al incrementar la variable x_2 , x_3 y x_4 decrecen.

De las penalizaciones calculadas, la mejor (menos mala) se obtiene al hacer $x_2 \leq 3$. Añadamos esa condición, es decir, $x_2 = \frac{40}{11} - \frac{5}{11}x_3 - \frac{1}{11}x_4 \leq 3$. La relajación lineal resultante anterior más esta condición queda:

$$\begin{aligned}
 x_0 &+ \frac{3}{11}x_3 + \frac{16}{11}x_4 = \frac{332}{11} \\
 x_1 &- \frac{1}{11}x_3 + \frac{2}{11}x_4 = \frac{36}{11} \\
 x_2 &+ \frac{5}{11}x_3 + \frac{1}{11}x_4 = \frac{40}{11} \\
 &\frac{8}{11}x_3 + \frac{6}{11}x_4 + x_5 = \frac{75}{11} \\
 &- \frac{5}{11}x_3 - \frac{1}{11}x_4 + x_6 = -\frac{7}{11}.
 \end{aligned}$$

Después de una iteración del método dual del *simplex* se llega a la solución óptima siguiente:

$$\begin{array}{rclcl}
 x_0 & + & \frac{7}{5}x_4 & + & \frac{3}{5}x_6 & = & \frac{149}{5} \\
 x_1 & + & \frac{1}{5}x_4 & - & \frac{1}{5}x_6 & = & \frac{17}{5} \\
 x_2 & & & + & x_6 & = & 3 \\
 & & \frac{2}{5}x_4 + x_5 + \frac{8}{5}x_6 & = & \frac{29}{5} \\
 x_3 + \frac{1}{5}x_4 & - & \frac{11}{5}x_6 & = & \frac{7}{5} .
 \end{array}$$

De aquí que $z_{PL}^{(1)} = x_0 = 149/5$ y $\mathbf{x}^{(1)} = [17/5, 3, 7/5, 0, 29/5]^T$, con $x_6 = 0$. La única variable que debe ser entera en esta solución y no lo es es x_1 .

Calculemos ahora las penalizaciones por bifurcar a:

$x_1 \leq 3$:

$$P_{1D}^{1-} = \min_{j, \bar{a}_{pj} > 0} \{ \bar{a}_{0j} f_{p0} / \bar{a}_{pj}, j \in M \} = \min \left\{ \frac{7}{5} \cdot \frac{2}{5} / \frac{1}{5} \right\} = \frac{14}{5}.$$

$x_1 \geq 4$:

$$P_{1U}^{1+} = \min_{j, \bar{a}_{pj} < 0} \{ \bar{a}_{0j} (1 - f_{p0}) / (-\bar{a}_{pj}), j \in M \} = \min \left\{ \frac{3}{5} \cdot (1 - \frac{2}{5}) / \frac{1}{5} \right\} = \frac{9}{5}.$$

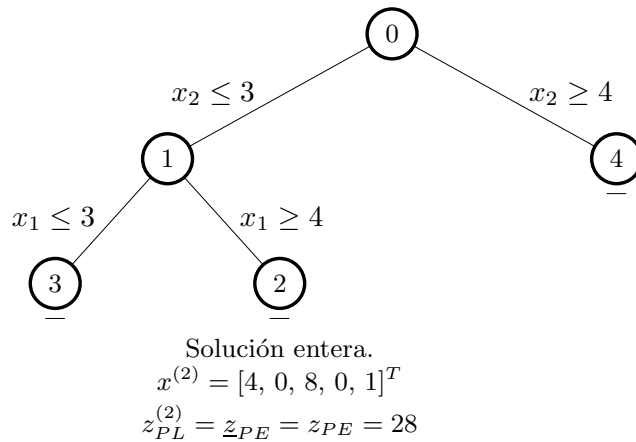
De estas penalizaciones la mejor se obtiene al hacer $x_1 \geq 4$. Añadamos esa condición, es decir, $x_1 = \frac{17}{5} - \frac{1}{5}x_4 + \frac{1}{5}s \geq 4$. La relajación lineal resultante anterior más esta condición queda:

$$\begin{array}{rclcl}
 x_0 & + & \frac{7}{5}x_4 & + & \frac{3}{5}x_6 & = & \frac{149}{5} \\
 x_1 & + & \frac{1}{5}x_4 & - & \frac{1}{5}x_6 & = & \frac{17}{5} \\
 x_2 & & & + & x_6 & = & 3 \\
 & & \frac{2}{5}x_4 + x_5 + \frac{8}{5}x_6 & = & \frac{29}{5} \\
 x_3 + \frac{1}{5}x_4 & - & \frac{11}{5}x_6 & = & \frac{7}{5} \\
 & - & \frac{1}{5}x_4 & + & \frac{1}{5}x_6 - x_7 & = & \frac{3}{5} .
 \end{array}$$

Después de una iteración del método dual del *simplex* se llega a la primera solución factible entera $\mathbf{x}^{(2)} = [4, 0, 8, 0, 1]^T$, con $z_{PE} = 28$.

Después se analizaría el nudo 3 (ver figura 12.9), rechazándose inmediatamente pues del cálculo anterior de penalizaciones se vería que la función objetivo obtenible sería $149/5 - P_{1D}^{1-} = 135/5 = 27 < 28$. Posteriormente también se rechazaría el nudo 4. ●

En el apéndice E, página 779 y siguientes, se puede ver la salida de ordenador resultante de resolver este último ejemplo con el programa BBMI, que implementa un procedimiento *branch and bound* para resolver grandes programas enteros, enteros mixtos y, por supuesto, lineales, siguiendo todas las directrices aquí expuestas, incluidas las penalizaciones, así como las técnicas de matrices dispersas más modernas expuestas en los capítulos 3 y 7.

**Figura 12.9**

Árbol enumerativo del problema del ejemplo 12.3

Ejemplo 12.4 Para estudiar las prestaciones del programa BBMI al resolver, según todo lo que acabamos de exponer, un programa entero puro, resolvamos con él el siguiente problema:

$$\begin{aligned}
 &\text{minimizar} && x_1 + 10x_2 \\
 &\text{s. a} && 66x_1 + 14x_2 \geq 1430 \\
 &&& -82x_1 + 28x_2 \geq 1306 \\
 &&& x_1, x_2 \text{ enteras.}
 \end{aligned}$$

El fichero de datos que requeriría este problema es el que sigue (la clave PROCESO ITERATIVO se incluye en los datos para que el programa facilite toda la información posible sobre los pasos que se efectúan).

```

PROCESO ITERATIVO
NAME          Ej12.2.3
ROWS
  N  OBJ
  G  FILA1
  G  FILA2
COLUMNS
INT
  X1      OBJ      1          FILA1      66
  X1      FILA2     -82
  X2      OBJ      10         FILA1      14
  X2      FILA2     28
RHS
  RHS     FILA1     1430
  RHS     FILA2     1306
ENDATA

```

El resultado que se obtiene con BBMI se lista a continuación.

Problema Ej12.2.3

*** Estadísticas del Problema

3 Fila(s)
 2 Restricción(es)
 2 Variable(s) de decisión
 2 Variable(s) de holgura/artificiales
 6 Elementos no cero
 2 Variable(s) entera(s)
 Densidad de la matriz de coeficientes A: 100.000%

*** Estadísticas de INVERT

3 elementos no cero en la base
 0 columnas estructurales en la base
 0 vectores columna antes del "bump"
 3 vectores columna después del "bump"
 L: 0 elementos no cero; 0 vectores ETA
 U: 2 elementos no cero; 2 vectores ETA
 Total: 0 elementos no en la diagonal; 2 vectores ETA
 Máximo de transformaciones ETA: 2; máximo número de elementos ETA: 9
 Error relativo en x: .000000D+00

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra de-a	Cost.Red	Sale de-a	Paso	Pivote	El.Eta
1	2	1	.2736000D+04	2	L->B	-.42D+02	3 B->L	.10D+03	-.14D+02	2
2	0	1	.6863538D+03	1	L->B	-.46D+02	4 B->L	.73D+01	.21D+03	5

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: Ej12.2.3
 No. de iteraciones: 3
 Valor de la función objetivo: 686.353805073431

*** FILAS

No.	..Fila..	enValor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJ	BS 686.35381	-686.35381	Ninguno	Ninguno	1.000
2	FILA1	LI 1430.0000	.00000000	1430.0000	Ninguno	.2830
3	FILA2	LI 1306.0000	.00000000	1306.0000	Ninguno	.2156

*** COLUMNAS

No.	.Columna enValor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	X1 BS 7.2616822	1.0000000	.00000000	Ninguno	.000
2	X2 BS 67.909212	10.000000	.00000000	Ninguno	.000

Variable Separación	Nivel	Dirección	Nudos en Lista	Variables Ent. No Ent.	Iteración	Valor Func. Obj.
2	1	X> 68	1	2	4 D	6.8724242D+02
1	2	X> 8	2	1	6 D	7.0871429D+02
2	3	X> 71	3	1	7 D	7.1800000D+02

* Nueva solución entera; z(PE)= 718.00000; Tiempo desde última: -.0017 seg.

--- SOLUCION ENTERA ---

Tiempo desde última: .0017 seg.
 Nombre del problema: Ej12.2.3
 No. de iteraciones: 7
 Valor de la función objetivo: 718.000000000000

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Li.Inferior.	.Li.Superior.	Val.Dual.
1	OBJ	BS	718.00000	-718.00000	Ninguno	Ninguno	1.000
2	FILA1	BS	1522.0000	92.000000	1430.0000	Ninguno	.1269E-15
3	FILA2	BS	1332.0000	26.000000	1306.0000	Ninguno	.1349E-16

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Li.Inferior.	.Li.Superior.	Cos.Red.
1	X1	LIB	8.0000000	1.0000000	8.0000000	Ninguno	1.00
2	X2	LIB	71.000000	10.000000	71.000000	Ninguno	10.0

Variable	Nivel	Dirección	Nodos	Variables	Iteración	Valor	
Separación			en Lista	Ent. No Ent.		Func. Obj.	
2	3	X< 70	2	-Nudo desechado en BKTRAK-		1.0000000D+20	
1	2	X< 7	1	0	9 P	6.9842857D+02	
2FIJ	1	3	X> 70	3	1	10 D	7.0700000D+02

* Nueva solución entera; z(PE)= 707.00000; Tiempo desde última: -.0017 seg.

--- SOLUCION ENTERA ---

Tiempo desde última: -.0017 seg.
Nombre del problema: Ej12.2.3
No. de iteraciones: 10
Valor de la función objetivo: 707.000000000000

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Li.Inferior.	.Li.Superior.	Val.Dual.
1	OBJ	BS	707.00000	-707.00000	Ninguno	Ninguno	1.000
2	FILA1	BS	1442.0000	12.000000	1430.0000	Ninguno	.1269E-15
3	FILA2	BS	1386.0000	80.000000	1306.0000	Ninguno	.1349E-16

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Li.Inferior.	.Li.Superior.	Cos.Red.
1	X1	EQB	7.0000000	1.0000000	7.0000000	Ninguno	1.00
2	X2	LIB	70.000000	10.000000	70.000000	Ninguno	10.0

Variable	Nivel	Dirección	Nodos	Variables	Iteración	Valor
Separación			en Lista	Ent. No Ent.		Func. Obj.
2	3	X< 69	2	-Nudo desechado en BKTRAK-		1.0000000D+20
1	3	X< 6	1	-Nudo desechado en BKTRAK-		7.4457143D+02
2	1	X< 67	0	-Nudo desechado en BKTRAK-		1.0000000D+20

--- SOLUCION ENTERA OPTIMA ---

Nombre del problema: Ej12.2.3
No. de iteraciones: 10
Valor de la función objetivo: 707.000000000000

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Li.Inferior.	.Li.Superior.	Val.Dual.
-----	----------	----	---------------	---------------	---------------	---------------	-----------

1 OBJ	BS 707.00000	-707.00000	Ninguno	Ninguno	1.000
2 FILA1	BS 1442.0000	12.000000	1430.0000	Ninguno	.1269E-15
3 FILA2	BS 1386.0000	80.000000	1306.0000	Ninguno	.1349E-16

*** COLUMNAS

No. .Columna enValor.... Coste en F.O. .Lí.Inferior. .Lí.Superior. Cos.Red.

1 X1	EQB 7.0000000	1.0000000	7.0000000	Ninguno	.000
2 X2	LIB 70.000000	10.000000	70.000000	Ninguno	10.0

Tiempo total de CPU en cálculos: .0603 segundos

La evolución de los cálculos que realiza BBMI hacia la solución se representa sobre la región factible en la figura 12.10. En ella también se incluye el árbol enumerativo que genera.

Los nudos del árbol totalmente en negro son los que se desechan sin realizar ninguna iteración más pues la previsible función objetivo que obtendrían sería peor que la calculada hasta ese momento. La solución en los otros nudos se indica con un número que corresponde en la otra parte de la figura a su posición en la región factible. ●

Referencias

Esencialmente, las mismas del capítulo anterior: Nemhauser y Wolsey [1988] y [1989]; Schrijver [1986]; Salkin y Mathur [1989] y las más antiguas: Salkin [1975], Garfinkel y Nemhauser [1972], Hu [1970] y Greenberg [1971]. Además de éstas, para la elaboración del apartado relativo a los algoritmos de ramificación y acotamiento, o *branch and bound*, se ha seguido a Tomlin [1970]. Mucho de lo expuesto está también basado en la experiencia del autor en la elaboración del programa BBMI. Éste sigue las pautas marcadas en Dakin [1965] por lo que respecta a la forma básica de proceder para resolver programas enteros mixtos.

Ejercicios

12.1. Resolver los siguientes programas enteros. Utilizar el programa BBMI si se considera conveniente.

- a) minimizar $x_1 + 4x_2$
 s. a $2x_1 + x_2 \leq 8$
 $x_1 + 2x_2 \geq 6$
 $x_1, x_2 \geq 0$ y enteras.
- b) maximizar $3x_1 + 2x_2$
 s. a $x_1 + x_2 \leq 3,5$
 $0 \leq x_1 \leq 2$
 $0 \leq x_2 \leq 2$
 x_1, x_2 enteras.
- c) maximizar $4x_1 + 5x_2 + x_3$
 s. a $3x_1 + 2x_2 \leq 10$
 $x_1 + 4x_2 \leq 11$
 $3x_1 + 3x_2 + x_3 \leq 13$
 $x_1, x_2 \geq 0$ y enteras.

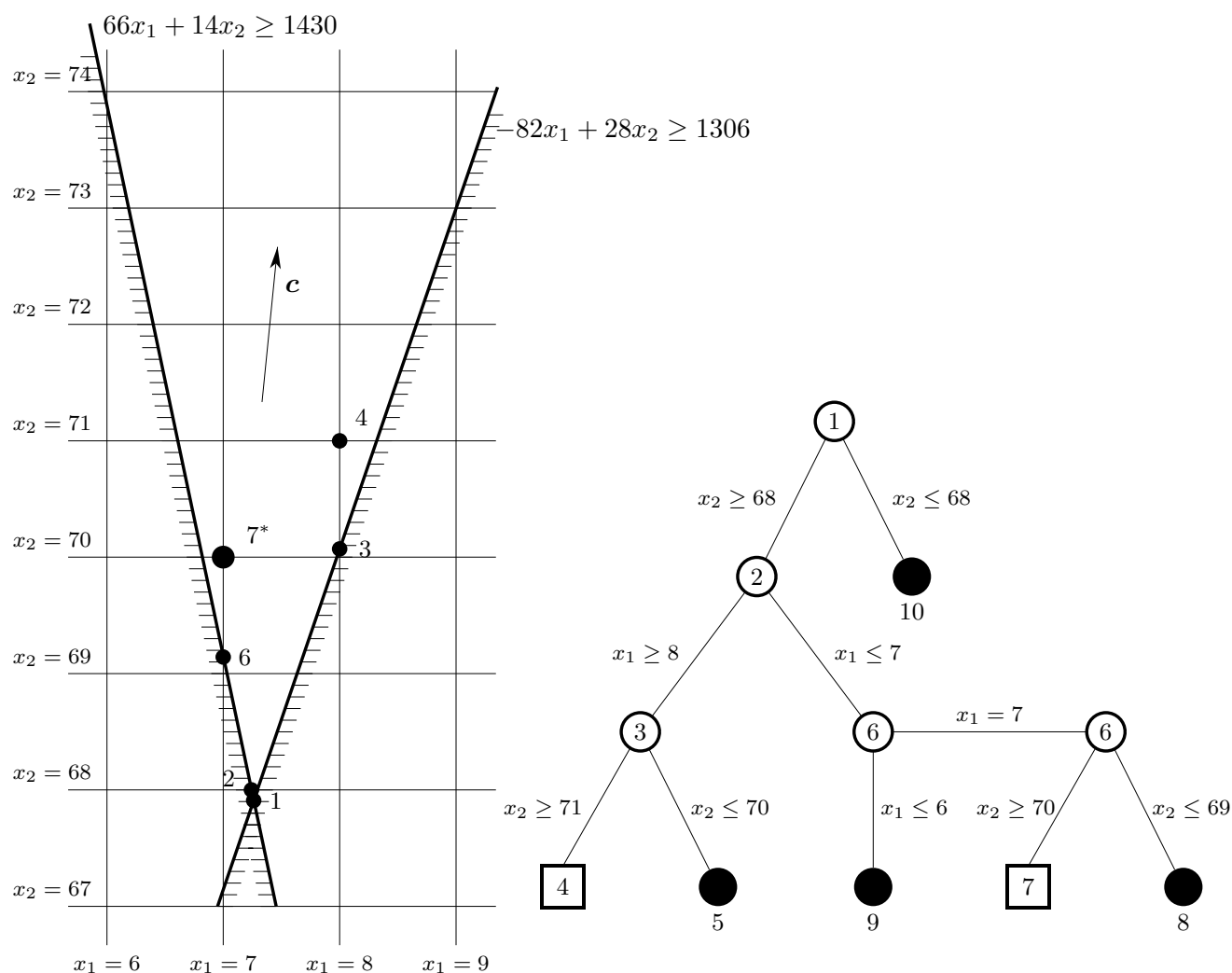


Figura 12.10

Región factible y árbol enumerativo del problema del ejemplo 12.4

d) maximizar $x_1 + x_2 + x_3$

$$\begin{aligned} \text{s. a} \quad & -4x_1 + 5x_2 + 2x_3 \leq 4 \\ & -2x_1 + 5x_2 \leq 5 \\ & 3x_1 - 2x_2 + 2x_3 \leq 6 \\ & 2x_1 - 5x_2 \leq 1 \end{aligned}$$

 $10 \geq x_1, x_2, x_3 \geq 0$ y enteras.e) minimizar $3x_1 + 5x_2 + 6x_3 + 9x_4 + 10x_5 + 10x_6$

$$\begin{aligned} \text{s. a} \quad & -2x_1 + 6x_2 - 3x_3 + 4x_4 + x_5 - 2x_6 \geq 2 \\ & -5x_1 - 3x_2 + x_3 + 3x_4 - 2x_5 + x_6 \geq -2 \\ & 5x_1 - x_2 + 4x_3 - 2x_4 + 2x_5 - x_6 \geq 3 \end{aligned}$$

 $x_1, x_2, x_3, x_4, x_5, x_6 \geq 0$ y enteras.

f) maximizar $2x_1 - x_2 + 5x_3 - 3x_4 + 4x_5$
s. a $3x_1 - 2x_2 + 7x_3 - 5x_4 + 4x_5 \leq 6$
 $x_1 - x_2 + 2x_3 - 4x_4 + 2x_5 \leq 0$
 $1 \geq x_1, x_2, x_3, x_4, x_5 \geq 0$ y enteras.

g) maximizar $x_1 - x_2 + x_3 - x_4$
s. a $x_1 + x_2 + x_3 + x_4 = 8$
 $0 \leq x_1 \leq 8$
 $-4 \leq x_2 \leq 4$
 $-2 \leq x_3 \leq 4$
 $0 \leq x_4 \leq 10$
 x_1, x_2, x_3, x_4 enteras.

h) maximizar $4x_1 - 2x_2 + 7x_3$
s. a $x_1 + 5x_3 \leq 10$
 $x_1 + x_2 - x_3 \leq 1$
 $6x_1 - 5x_2 \leq 0$
 $x_1, x_2, x_3 \geq 0$ y enteras.

i) minimizar $3x_1 + 2x_2$
s. a $3x_1 + x_2 \geq 6$
 $x_1 + x_2 \geq 3$
 $x_1, x_2 \geq 0$ y enteras.

j) maximizar $x_1 + 5x_2$
s. a $x_1 + 10x_2 \leq 20$
 $0 \leq x_1 \leq 2$
 x_1, x_2 enteras.

k) maximizar $3x_1 + 3x_2 + 13x_3$
s. a $-3x_1 + 6x_2 + 7x_3 \leq 8$
 $6x_1 - 3x_2 + 7x_3 \leq 8$
 $0 \leq x_i \leq 5, i = 1, 2, 3$
 $x_1, \dots, x_3$ enteras.

12.2. Resolver el siguiente programa entero

$$\begin{array}{ll}\text{maximizar} & 2x_1 + 5x_2 \\ \text{s. a} & 4x_1 + x_2 \leq 28 \\ & x_1 + 4x_2 \leq 27 \\ & x_1 - x_2 \leq 1 \\ & x_1, x_2 \in \mathbf{Z}_+.\end{array}$$

- a) Gráficamente.
b) Mediante el método *branch and bound*.

12.3. La solución de la relajación lineal del programa entero del ejercicio anterior es

$$\begin{array}{rclcl} x_0 & + & \frac{1}{5}x_3 + \frac{6}{5}x_4 & = & 38 \\ x_1 & + & \frac{4}{15}x_3 - \frac{1}{15}x_4 & = & \frac{17}{3} \\ x_2 & - & \frac{1}{15}x_3 + \frac{4}{15}x_4 & = & \frac{16}{3} \\ & - & \frac{1}{3}x_3 + \frac{1}{3}x_4 + x_5 & = & \frac{2}{3}. \end{array}$$

Resolver el problema mediante el algoritmo de los planos cortantes de Gomory.

12.4. Considérese el programa entero

$$\begin{array}{ll} \text{maximizar} & -x_{n+1} \\ & 2x_1 - 2x_2 + \cdots + 2x_n + x_{n+1} = n \\ & x_1, \dots, x_{n+1}, 0 \text{ ó } 1. \end{array}$$

Probar que el algoritmo *branch and bound* aplicado a este problema, cuando n es impar, requiere enumerar un número exponencial de nudos.

12.5. Considérese un programa entero mixto con una sola variable entera. Probar que el algoritmo *branch and bound* aplicado a ese problema no tendrá más de tres nudos. ¿Por qué?

12.6. Resolver el programa entero

$$\begin{array}{ll} \text{maximizar} & y \\ \text{s. a} & x_1 + x_2 + y \leq 2 \\ & -x_1 + y \leq 0 \\ & x_1, x_2, y \in \mathbf{Z}_+ \end{array}$$

mediante el algoritmo de los planos cortantes de Gomory. Reemplazar posteriormente la condición $y \in \mathbf{Z}_+$ por $y \in \mathbb{R}$ y resolverlo nuevamente mediante el mismo algoritmo.

Cuarta parte

Apéndices

Apéndice A

REPASO DE MATEMÁTICAS: DEFINICIONES, NOTACIONES Y RELACIONES BÁSICAS

EN ESTE APÉNDICE se recopilan algunos conceptos y resultados que pueden resultar útiles para leer y comprender este libro. Sólo se introducen brevemente aquellos relacionados con las materias objeto del libro o en los que se basan; en ningún caso pretenden ser un exhaustivo recordatorio de matemáticas elementales.

A.1 Conjuntos

Un *conjunto*, es una colección de objetos: los números naturales, las soluciones a un problema determinado, los municipios de una provincia, etc. Se identifica por una letra mayúscula: el conjunto S , el conjunto de los números naturales $\mathbf{N}$, el de los enteros $\mathbf{Z}$, el de los reales $\mathbf{R}$, complejos $\mathbf{C}$, racionales $\mathbf{Q}$, etc.

Los componentes de un conjunto se denominan *elementos*. Si un elemento a pertenece a un conjunto se indica $a \in S$. Los conjuntos se definen mediante la enumeración entre llaves de sus elementos, $S = \{a, b, \dots\}$, o especificando, también entre llaves, la propiedad que los caracteriza, $S = \{x : x \in \mathbf{R}, x \leq 2\}$: números reales menores o iguales que dos.

El conjunto sin elementos se denomina *vacío*, designándose $\emptyset$. Ejemplo: el conjunto S de los números reales x que son mayores que 1 y menores que 0: esto es, $S = \{x \in \mathbf{R} : x > 1, x < 0\}$.

Si S y S' son dos conjuntos y todos los elementos del conjunto S' lo son de S , se dice que S' es un *subconjunto* del conjunto S , o que está contenido en S' , expresándose $S' \subset S$ o $S \supset S'$.

La *unión* de dos conjuntos S y T , expresada $S \cup T$, es el conjunto formado por los elementos que pertenecen a S o a T .

La *intersección* de S y T , expresada $S \cap T$, es el conjunto formado por los elementos que pertenecen a S y a T .

Si S' es un subconjunto de S , el *complemento* de S' en S es el conjunto formado por los elementos de S que no pertenecen a S' .

Si a y b son números reales, $a \leq b$, el conjunto de números de la recta real x , $a \leq x \leq b$, se indica $[a, b]$. El formado por los x , $a < x \leq b$, por $(a, b]$. El de los x , $a < x < b$, por (a, b) .

Si S es un conjunto de números reales acotados superiormente y no vacío, existe un número real y tal que $x \leq y$ y tal que $y \leq z$ si z es cota superior de S , para todo $x \in S$. Al número y se le denomina *cota superior mínima* o *supremo* de S ; se expresa

$$\sup_{x \in S} (x) \quad \text{o} \quad \sup \{x : x \in S\}.$$

De forma similar se define la *cota inferior máxima* o *ínfimo* de S :

$$\inf_{x \in S} (x) \quad \text{o} \quad \inf \{x : x \in S\}.$$

A.2 Aplicaciones

Dados dos conjuntos S y T , una *aplicación* f de S en T , expresada como $f : S \rightarrow T$, es una asociación o criterio que a cada elemento de S hace corresponder uno de T .

La *imagen* de un elemento $x \in S$ con la aplicación $f : S \rightarrow T$ es el elemento $f(x) \in T$. El *conjunto imagen* $f(S) = \{f(x) \in T, \text{ para todo } x \in S\}$. La imagen de un subconjunto $S' \subset S$ con la aplicación f sería, por consiguiente, el subconjunto imagen $f(S')$. El conjunto S se conoce como *origen* o *dominio de definición* y el T como *dominio de valores*. Una aplicación $f : S \rightarrow T$ se dice *inyectiva* si para cualquier par de elementos $x, y \in S$, $x \neq y$, se cumple que $f(x) \neq f(y)$. Ejemplo, la aplicación $f : \mathbb{R} \rightarrow \mathbb{R}$, definida por $f(x) = x^2$, no es inyectiva, pues $f(1) = f(-1) = 1$.

Una aplicación $f : S \rightarrow T$ se dice *suprayectiva* si el conjunto imagen $f(S)$ es igual a todo el conjunto T ; es decir, para todo $y \in T$ existe un $x \in S$ tal que $f(x) = y$.

Una aplicación se dice *biyectiva* si es inyectiva y suprayectiva. Ejemplo, si J_n es el conjunto de los números enteros de 1 a n , $J_n = \{1, \dots, n\}$, y se define una aplicación $\sigma : J_n \rightarrow J_n$ que modifica el orden de disposición de los elementos de J_n —estas aplicaciones se denominan *permutaciones*—, tal aplicación es biyectiva.

Un conjunto S se dice *numerable* si existe una biyección entre $\mathbf{N}$ y S : a cada elemento k , $1 \leq k \leq n$, se le asocia un elemento $a_k \in S$, esto es: $k \mapsto a_k$.

Una *sucesión* de elementos de un conjunto T es una aplicación de $\mathbf{N}$ en T : a cada elemento $n \geq 1$ se le hace corresponder un $x^{(n)} \in T$: $n \mapsto x^{(n)}$. Tal sucesión se expresa como $\{x^{(1)}, x^{(2)}, \dots\}$ o $\{x^{(n)}\}_{n \geq 1}$.

Una *función* es un caso particular de aplicación en donde los conjuntos origen e imagen son conjuntos de números.

Los conjuntos dotados de ciertas leyes de composición o asociación interna —adición, multiplicación, división o cualquier otra—, se dice que poseen una *estructura*. Las estructuras fundamentales son: *grupo*, *anillo* ($\mathbf{Z}$ por ejemplo), *cuerpo* ($\mathbb{R}$ y $\mathbf{C}$ por ejemplo) y *espacio vectorial*.

A.3 Espacios vectoriales

Sea K un cuerpo, un *espacio vectorial* E sobre K es un conjunto dotado de una ley de composición interna, *adición*, con la siguientes propiedades —grupo conmutativo—:

$$\begin{aligned}x + y &= y + x \\(x + y) + z &= x + (y + z) \\x + \emptyset &= x \\x + (-x) &= \emptyset,\end{aligned}$$

y una ley de composición externa, *producto por un escalar*, de la que el dominio de operadores es K , con las siguientes propiedades:

$$\begin{aligned}1 \cdot x &= x \\ \alpha(\beta x) &= (\alpha\beta)x \\ (\alpha + \beta)x &= \alpha x + \beta x \\ \alpha(x + y) &= \alpha x + \alpha y,\end{aligned}$$

válidas cualesquiera que sean x, y, z en E y α, β en K ; a $\emptyset$ se le denomina elemento neutro y a $-x$ el opuesto de x . Es usual denominar *vectores* a los elementos de E y *escalares* a los de K . En las aplicaciones que se estudian en el libro los casos más importantes ocurren cuando $K = \mathbb{R}$ o $K = \mathbb{C}$. Con la notación $\mathbf{K}$ designaremos a cualquiera de los cuerpos $\mathbb{R}$ o $\mathbb{C}$ y por $\mathbf{x}$ un vector cualquiera de un espacio vectorial.

Un ejemplo característico de espacio vectorial lo constituye el formado por sucesiones ordenadas de n elementos cualesquiera de K , o n -uplas $\mathbf{x} = (x_1, \dots, x_n)$, definiendo la suma de vectores mediante

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n)$$

y el producto por un escalar mediante

$$\alpha(x_1, \dots, x_n) = (\alpha x_1, \dots, \alpha x_n).$$

Si X es un conjunto arbitrario, el conjunto de aplicaciones $\varphi : X \rightarrow K$ se estructura también como un espacio vectorial definiendo las operaciones

$$\begin{aligned}(\varphi + \psi) : \mathbf{x} &\longmapsto \varphi(\mathbf{x}) + \psi(\mathbf{x}) \\ (\lambda\varphi) : \mathbf{x} &\longmapsto \lambda\varphi(\mathbf{x}).\end{aligned}$$

El ejemplo anterior es un caso particular de este espacio vectorial tomando $X = \{1, 2, \dots, n\}$.

Un *subespacio* M , de un espacio vectorial E sobre un cuerpo K , es un subconjunto no vacío cerrado respecto de las operaciones de adición y producto por un escalar; es decir, se cumple que:

$$\begin{aligned}\forall \mathbf{x}, \mathbf{y} \in M &\implies \mathbf{x} + \mathbf{y} \in M, \\ \forall \mathbf{x} \in M \text{ y } \forall \lambda \in K &\implies \lambda \mathbf{x} \in M.\end{aligned}$$

La intersección de una familia cualquiera de subespacios de E es también un subespacio.

Si X es un subconjunto cualquiera de E , el subespacio $\mathcal{L}(X)$ *generado* por X es la intersección de todos los subespacios que contienen a X . Cuando $\mathcal{L}(X) = E$, se dice que X es una *parte generadora* de E .

Dados vectores $\mathbf{x}_1, \dots, \mathbf{x}_n$ y escalares $\lambda_1, \dots, \lambda_n$, el vector formado según la expresión

$$\mathbf{x} = \lambda_1 \mathbf{x}_1 + \dots + \lambda_n \mathbf{x}_n$$

se dice que es *combinación lineal* de los vectores $\mathbf{x}_1, \dots, \mathbf{x}_n$ de coeficientes $\lambda_1, \dots, \lambda_n$. Un subconjunto X de E es un subespacio si y sólo si contiene a cualquier combinación lineal de cualquier subconjunto finito de vectores de X . También se demuestra que el subespacio $\mathcal{L}(X)$ es el conjunto de todas las combinaciones lineales de vectores de X .

Un conjunto de vectores $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_k$ se dicen *linealmente dependientes* si existen escalares λ_i , no todos cero, tales que $\sum_{i=1}^k \lambda_i \mathbf{x}_i = \mathbf{0}$; *linealmente independientes*, si

$$\sum_{i=1}^k \lambda_i \mathbf{x}_i = \mathbf{0} \implies \lambda_i = 0, \quad 0 \leq i \leq k.$$

Una parte X de un espacio vectorial E se dice que es una *familia libre* si los vectores de cualquier subconjunto finito de X son linealmente independientes.

La *dimensión* de un subespacio es el máximo número de vectores linealmente independientes en el subespacio.

Una *base* de un espacio vectorial E es cualquier subconjunto B de E que sea, simultáneamente, una parte libre y generadora de E ; dicho de otra forma, una base de un espacio vectorial es un conjunto —normalmente se supone ordenado (numerado)— de vectores linealmente independientes que generan dicho espacio. Se demuestra que cualquier espacio vectorial tiene una base y que todas las bases de un mismo espacio tienen la misma cardinalidad —se pueden poner en biyección—. Cuando el cardinal de las bases es un número natural, $n \in \mathbf{N}$, se dice que el espacio es de dimensión finita n . En un espacio vectorial K^n ,

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \dots, \quad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix},$$

forman una base en dicho espacio; éste, por tanto, tiene dimensión n . Esta base se denomina *base canónica* de K^n . En esta base, cualquier vector $\mathbf{x}^T = [x_1, x_2, \dots, x_n]$ se puede expresar de la siguiente forma:

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} + \dots + x_n \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}.$$

Si A y B son subconjuntos de un espacio vectorial E , el conjunto $A + B$ se define como:

$$A + B = \{\mathbf{a} + \mathbf{b} : \mathbf{a} \in A, \mathbf{b} \in B\}.$$

Cuando A y B son subespacios, también lo es la suma $A + B$. Si además $A \cap B = \emptyset$, la suma se denomina *directa*, escribiéndose $A \oplus B$. Si $A \oplus B = E$, cualquier vector $\mathbf{c} \in E$ se descompone de manera única como $\mathbf{c} = \mathbf{a} + \mathbf{b}$, con $\mathbf{a} \in A$ y $\mathbf{b} \in B$; también se dice que A y B son *subespacios suplementarios*.

A.3.1 Espacios normados

Si en un espacio vectorial E sobre $\mathbf{K}$ ($\mathbb{R}$ o $\mathbb{C}$) se define una *norma vectorial* como una aplicación $\|\cdot\| : E \rightarrow \mathbb{R}$ que verifica

$$\begin{aligned}\|\mathbf{v}\| = 0 &\implies \mathbf{v} = \mathbf{0} \text{ y } \mathbf{x} \neq \mathbf{0} \implies \|\mathbf{x}\| > 0, \\ \|\alpha\mathbf{v}\| &= |\alpha|\|\mathbf{v}\| \text{ para } \alpha \in K \text{ y } \mathbf{v} \in E, \\ \|\mathbf{u} + \mathbf{v}\| &\leq \|\mathbf{u}\| + \|\mathbf{v}\| \quad \forall \mathbf{u}, \mathbf{v} \in E,\end{aligned}$$

se dice que E es un *espacio vectorial normado*. La condición $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ se conoce como *regla del triángulo*. Es una generalización del hecho de que un lado de un triángulo no puede ser mayor que la suma de los otros dos: ver figura A.1. Una variante también útil de esta regla es la siguiente:

$$\|\mathbf{u} - \mathbf{v}\| \geq \|\mathbf{u}\| - \|\mathbf{v}\|.$$

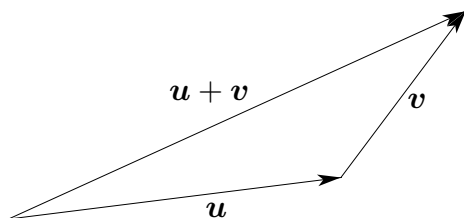


Figura A.1

Representación gráfica de la regla del triángulo

En el espacio vectorial $\mathbf{K}^n$, para $1 \leq p < \infty$, se tiene la familia de normas

$$\|\mathbf{x}\|_p = \left[|x_1|^p + \cdots + |x_n|^p \right]^{1/p},$$

denominadas *normas p de Hölder*. Casos particulares lo constituyen las correspondientes a $p = 1$ y $p = 2$:

$$\begin{aligned}\|\mathbf{x}\|_1 &= \sum_{i=1}^n |x_i| \\ \|\mathbf{x}\|_2 &= \sqrt{|x_1|^2 + \cdots + |x_n|^2}.\end{aligned}$$

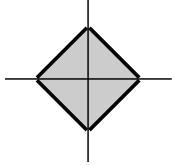
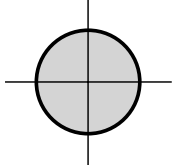
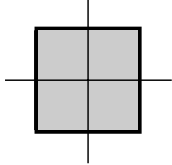
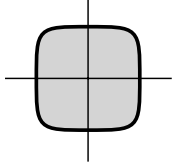
Esta última se denomina en $\mathbb{R}^n$ *norma euclídea*. También en $\mathbf{K}^n$ es una norma la dada por

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|.$$

Estas normas cumplen, cualquiera que sea $\mathbf{x} \in \mathbf{K}^n$, que

$$\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq n\|\mathbf{x}\|_\infty.$$

Tabla A.1Forma de la bola unidad para diferentes normas en $\mathbb{R}^2$

$\ \mathbf{x}\ _1 = \sum_{i=1}^2 x_i $	
$\ \mathbf{x}\ _2 = \sqrt{ x_1 ^2 + x_2 ^2} = \sqrt{\mathbf{x}^T \mathbf{x}}$	
$\ \mathbf{x}\ _\infty = \max_{1 \leq i \leq 2} x_i $	
$\ \mathbf{x}\ _p = \left[x_1 ^p + x_2 ^p \right]^{1/p}, \quad (1 \leq p < \infty)$	

Si la bola cerrada unidad en $\mathbb{R}^2$ es el conjunto $\{\mathbf{x} \in \mathbb{R}^2 : \|\mathbf{x}\| \leq 1\}$, sus formas para las normas vectoriales 1, 2, ∞ , y p son las que representa la tabla A.1.

En el espacio $C[0, 1]$, de funciones continuas del intervalo $[0, 1]$ en $\mathbf{C}$, son normas las dadas por

$$\|f\|_p = \left[\int_0^1 |f(t)|^p dt \right]^{1/p}$$

y por

$$\|f\|_\infty = \max_{t \in [0, 1]} |f(t)|.$$

En un espacio vectorial normado se define la *distancia* entre dos elementos $\mathbf{u}$ y $\mathbf{v}$ mediante

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|.$$

Esta definición convierte a cualquier espacio vectorial normado en un *espacio métrico*.

Sea E un espacio vectorial normado; se dice que una sucesión¹ $\{\mathbf{x}^{(n)}\}$ en E *converge* a un

¹A lo largo del libro, cuando así lo aconseja la dificultad de la notación, una sucesión también se designa por $\{\mathbf{x}_n\}$; sus integrantes, $\mathbf{x}^{(k)}$.

límite $\mathbf{v} \in E$, si para todo $\varepsilon > 0$, existe un $N \in \mathbf{N}$ tal que a partir de él, $n \geq N$, se cumple que $\|\mathbf{x}^{(n)} - \mathbf{v}\| < \varepsilon$.

Cuando una sucesión $\{\mathbf{x}^{(n)}\}$ admite un vector límite $\mathbf{v}$, sólo tiene ese vector como límite —si existe límite es único—: se escribe $\lim_{n \rightarrow \infty} \mathbf{x}^{(n)} = \mathbf{v}$. Es equivalente decir que $\lim_{n \rightarrow \infty} \mathbf{x}^{(n)} = \mathbf{v}$ y que $\lim_{n \rightarrow \infty} \|\mathbf{x}^{(n)} - \mathbf{v}\| = 0$. En particular, $\mathbf{x}^{(n)} \rightarrow 0$ si y sólo si $\|\mathbf{x}^{(n)}\| \rightarrow 0$.

Una sucesión $\{\mathbf{x}^{(n)}\}$ en un espacio vectorial normado por $\|\cdot\|$ se denomina *sucesión de Cauchy*, si para cada $\varepsilon > 0$, existe un $N \in \mathbf{N}$ tal que cualesquiera que sean $p, q \geq N$, se cumple que $\|\mathbf{x}^{(p)} - \mathbf{x}^{(q)}\| < \varepsilon$. Toda sucesión convergente es una sucesión de Cauchy pero pueden existir espacios normados con sucesiones de Cauchy que no son convergentes. Un espacio vectorial normado se dice *completo* si toda sucesión de Cauchy en él tiene límite.

Un *espacio de Banach* es un espacio vectorial completo respecto de la norma a él asociada. Todo espacio vectorial normado de dimensión finita es un espacio de Banach. En un espacio de dimensión infinita esto no es cierto; por ejemplo, es fácil ver que en $C[0, 1]$ la sucesión de funciones cuyas gráficas son las de la figura A.2 es una sucesión de Cauchy para cualquier norma $\|\cdot\|_p$, pero no tiene límite en $C[0, 1]$.

A.3.2 Espacios con producto interior

Sea E un espacio vectorial sobre un cuerpo $\mathbf{K}$ ($\mathbf{R}$ o $\mathbf{C}$); una *forma sesquilineal* —vez y media lineal— sobre E es una aplicación, $\langle \cdot | \cdot \rangle : E \times E \rightarrow \mathbf{K}$, que verifica²:

- 1) $\langle \alpha \mathbf{u} + \beta \mathbf{v} | \mathbf{w} \rangle = \alpha \langle \mathbf{u} | \mathbf{w} \rangle + \beta \langle \mathbf{v} | \mathbf{w} \rangle$ y
- 2) $\langle \mathbf{u} | \alpha \mathbf{v} + \beta \mathbf{w} \rangle = \bar{\alpha} \langle \mathbf{u} | \mathbf{v} \rangle + \bar{\beta} \langle \mathbf{u} | \mathbf{w} \rangle$,

²La barra designa complejo conjugado.

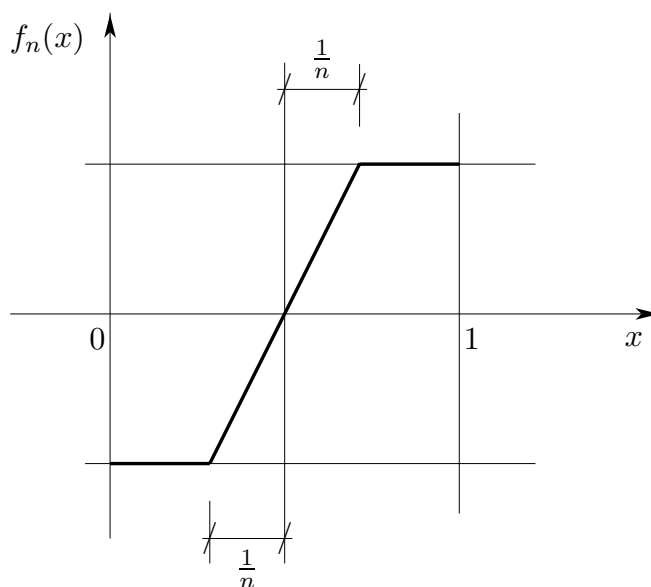


Figura A.2

Gráfica de una de las funciones de una sucesión de Cauchy

cualesquiera que sean $\mathbf{u}, \mathbf{v}, \mathbf{w}$ en E y α, β en $\mathbf{K}$. Si además se cumple que

$$\langle \mathbf{u} | \mathbf{v} \rangle = \overline{\langle \mathbf{v} | \mathbf{u} \rangle},$$

la forma se denomina *hermítica*. Es claro que $\langle \mathbf{u} | \mathbf{u} \rangle$ es siempre un número real. Cuando se cumple que

$$\mathbf{u} \neq \mathbf{0} \implies \langle \mathbf{u} | \mathbf{u} \rangle > 0,$$

se dice que la forma es definida positiva, denominándosela también *producto escalar*. Una forma sesquilineal sobre $\mathfrak{R}$ es siempre *bilineal*.

Un *espacio prehilbertiano* es un espacio vectorial sobre $\mathbf{K}$ dotado de una forma hermítica definida positiva. Todo espacio prehilbertiano es un espacio normado mediante

$$\|\mathbf{v}\| = \sqrt{\langle \mathbf{v} | \mathbf{v} \rangle}.$$

En la demostración de que esta definición corresponde a la de una norma en E juega un papel importante la *desigualdad de Cauchy-Schwarz*: a saber,

$$|\langle \mathbf{u} | \mathbf{v} \rangle| \leq \|\mathbf{u}\| \cdot \|\mathbf{v}\|.$$

Un *espacio de Hilbert* es un espacio prehilbertiano completo respecto de la norma que deriva del producto escalar $\|\cdot\| = \sqrt{\langle \cdot, \cdot \rangle}$. Dicho de otra forma, un espacio prehilbertiano que con esta norma da un espacio de Banach.

El *espacio euclídeo* n -dimensional, expresado $\mathfrak{R}^n$ o $\mathbf{E}^n$, es un espacio de Hilbert de dimensión finita.

Dos vectores cuyo producto escalar es cero se denominan *ortogonales*; si su $\|\cdot\|_2$ es igual a la unidad, se denominan *ortonormales*. Para dos vectores ortogonales se tiene la identidad

$$\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2,$$

que es una generalización del teorema de Pitágoras. En un espacio prehilbertiano, el único vector ortogonal a todos los vectores del espacio es el vector nulo; si este espacio es de dimensión finita es posible construir una base ortonormalizada.

Una familia cualquiera de vectores distintos del nulo y ortogonales dos a dos es una familia libre. Si M es un subespacio de un espacio prehilbertiano E de dimensión finita, el *subespacio ortogonal* de M , $M^\perp$, es el subespacio formado por todos los vectores ortogonales a los de M , siendo un subespacio suplementario de M ; es decir $M \oplus M^\perp = E$. Cualquier $\mathbf{x} \in E$, por consiguiente, se puede expresar como $\mathbf{x} = \mathbf{a} + \mathbf{b}$, con $\mathbf{a} \in M$ y $\mathbf{b} \in M^\perp$.

A.3.3 Aplicaciones lineales

Dados dos espacios vectoriales E y F sobre el mismo cuerpo K , se define una *aplicación lineal*, u *homomorfismo* f de E en F , como una aplicación $f : E \rightarrow F$ que verifica

$$f(\lambda \mathbf{x} + \mu \mathbf{y}) = \lambda f(\mathbf{x}) + \mu f(\mathbf{y}),$$

cualesquiera que sean los vectores $\mathbf{x}, \mathbf{y}$ en E y los escalares λ y μ . Existen dos casos particulares interesantes: el primero cuando $E = F$, en este caso se dice que f es un *operador lineal* de E o

endomorfismo de E ; el segundo cuando $F = K$ —el cuerpo base—, en cuyo caso la aplicación se denomina *forma lineal* sobre E .

El conjunto $\mathcal{L}(E, F)$ de todas las aplicaciones lineales del espacio E en el espacio F se estructura como un espacio vectorial si se definen las siguientes operaciones:

- a) adición $(f + g) :$ $(f + g)(\mathbf{x}) = f(\mathbf{x}) + g(\mathbf{x}) \quad \forall \mathbf{x} \in E;$
 b) producto por un escalar $\lambda f :$ $(\lambda f)(\mathbf{x}) = \lambda f(\mathbf{x}) \quad \forall \mathbf{x} \in E \text{ y } \forall \lambda \in K.$

En particular, el conjunto $\mathcal{L}(E, K)$ de formas lineales es un espacio vectorial denominado *dual* de E , representándose con E^* .

Para una aplicación lineal $f : E \rightarrow F$, el conjunto de vectores de F que son la imagen de los de un subespacio de E forma un subespacio de F . En particular, la imagen de todo E es un subespacio de F que se denomina *subespacio imagen* de f , representándose mediante $\text{Im}(f)$. Análogamente, el conjunto anti-imagen de un subespacio de F forma un subespacio de E . En particular, la anti-imagen del subespacio nulo de F forma lo que se denomina el *núcleo* de la aplicación, representándose por $\ker(f)$. Así pues

$$\ker(f) = \{\mathbf{x} \in E : f(\mathbf{x}) = \mathbf{0}\}.$$

Si $\mathbf{b} \in F$, la ecuación lineal $f(\mathbf{x}) = \mathbf{b}$ tiene solución si y sólo si $\mathbf{b} \in \text{Im}(f)$. En ese caso el conjunto de todas las soluciones es la *variedad lineal* —traslación de un subespacio— dada por $\mathbf{x}_0 + \ker(f)$, donde $\mathbf{x}_0$ es una solución particular de la ecuación. En particular, la aplicación es inyectiva si y sólo si $\ker(f) = \emptyset$.

Sean E y F dos espacios prehilbertianos sobre el cuerpo $\mathbf{K}$; si $f : E \rightarrow F$ es una aplicación lineal, la *aplicación traspuesta* de f es la aplicación $f^* : F \rightarrow E$ que cumple

$$\langle \mathbf{x} | f^*(\mathbf{y}) \rangle = \langle f(\mathbf{x}) | \mathbf{y} \rangle,$$

cualesquiera que sean los vectores $\mathbf{x} \in E$ e $\mathbf{y} \in F$. Particularmente importante es el caso en que $E = F$: f^* se dice entonces que es el *operador adjunto* de f . Cuando un operador f de E cumple que $f^* = f$ se denomina *operador autoadjunto*. En el caso de que E sea un espacio vectorial real, también se dice que f es *un operador simétrico* y cuando es un espacio vectorial complejo, que f es *un operador hermítico*. Un operador simétrico cumple que

$$\langle \mathbf{x} | f(\mathbf{y}) \rangle = \langle f(\mathbf{x}) | \mathbf{y} \rangle,$$

mientras que uno hermítico, que

$$\langle \mathbf{x} | f(\mathbf{y}) \rangle = \overline{\langle f(\mathbf{x}) | \mathbf{y} \rangle}.$$

Un operador f de E es *unitario* cuando es invertible y su inverso coincide con su adjunto. Es decir, si $f^* = f^{-1}$. Para un operador unitario se tiene que

$$\langle f(\mathbf{x}) | f(\mathbf{y}) \rangle = \langle f^*(f(\mathbf{x})) | \mathbf{y} \rangle = \langle \mathbf{x} | \mathbf{y} \rangle,$$

de manera que $\|f(\mathbf{x})\| = \|\mathbf{x}\|$. Por este motivo a los operadores unitarios también se les denomina *operadores isométricos*.

A.4 Matrices

Sean dos espacios vectoriales E y F de dimensiones finitas n y m sobre el mismo cuerpo K . Una aplicación lineal $g : E \rightarrow F$, $g \in \mathcal{L}(E, F)$, está caracterizada o representada en dos bases $\{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n\}$ de E y $\{\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_m\}$ de F por una tabla de coeficientes, *matriz* asociada, de m filas y n columnas:

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \in K^{m \times n}.$$

Los elementos a_{ij} están definidos por

$$g(\mathbf{e}_j) = \sum_{i=1}^m a_{ij} \mathbf{f}_i, \quad 1 \leq j \leq n.$$

El vector columna j -ésimo

$$\begin{bmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{mj} \end{bmatrix}$$

representa el vector $g(\mathbf{e}_j)$ en la base $(\mathbf{f}_i)$. A partir de la matriz A se pueden calcular los componentes $y_1, y_2, \dots, y_m$ del vector $\mathbf{y} = g(\mathbf{x})$ en la base $(\mathbf{f}_i)$, conociendo los componentes $x_1, x_2, \dots, x_n$ en la base $(\mathbf{e}_j)$. En efecto:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = x_1 \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix} + x_2 \begin{bmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{m2} \end{bmatrix} + \cdots + x_n \begin{bmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{mn} \end{bmatrix}.$$

Expresión que también se puede escribir de la siguiente forma:

$$\mathbf{y} = \sum_{i=1}^n x_i \mathbf{a}_i,$$

donde $\mathbf{a}_i$ es el vector columna i -ésimo de la matriz A . Así pues, si se fijan dos bases en E y F , cada aplicación lineal, $g : E \rightarrow F$, queda unívocamente representada por una matriz. Recíprocamente, toda matriz en $K^{m \times n}$ define unívocamente una aplicación lineal entre dos espacios E y F de dimensiones n y m en los que se han fijado dos bases. En particular, se pueden identificar las matrices $m \times n$ con las aplicaciones lineales de K^n en K^m .

Las matrices de m filas y n columnas con elementos en el cuerpo K forman un espacio vectorial, $K^{m \times n}$, sobre dicho cuerpo K .

Si E y F son dos espacios de dimensión finita dotados de un producto escalar y la aplicación $\alpha \in \mathcal{L}(E, F)$ se representa en dos bases ortonormalizadas mediante una matriz A , la aplicación $\alpha^T \in \mathcal{L}(F, E)$, traspuesta de α , viene representada por la matriz A^T , traspuesta de A .

El *núcleo* y la *imagen* de una matriz $A \in K^{m \times n}$, $\ker(A)$ y $\text{Im}(A)$, respectivamente, se definen como los subespacios de K^n y K^m que son el núcleo y la imagen de la aplicación lineal asociada:

$$\left. \begin{aligned} \ker(A) &= \{\mathbf{x} \in K^n : A\mathbf{x} = \mathbf{0}\} \\ \text{Im}(A) &= \{\mathbf{y} \in K^m : \mathbf{y} = A\mathbf{x}, \mathbf{x} \in K^n\} \end{aligned} \right\}_{A \in K^{m \times n}}.$$

Dicho de otra forma, la imagen de una matriz es el subespacio generado por los vectores columna de la matriz; los vectores fila también generan un subespacio que no es otro que la imagen de A^T .

Para una matriz $A \in \mathfrak{R}^{m \times n}$ se cumple que:

$$\begin{aligned} \ker(A^T) &= (\text{Im}(A))^\perp; \\ \text{Im}(A^T) &= (\ker(A))^\perp; \\ \ker(A) &= (\text{Im}(A^T))^\perp; \\ \text{Im}(A) &= (\ker(A^T))^\perp. \end{aligned}$$

El *rango* de una matriz es la dimensión³ de su subespacio imagen:

$$\text{rango}(A) = \dim(\text{Im}(A))$$

Una matriz $A \in K^{m \times n}$ se dice de *rango completo* si $\text{rango}(A) = \min(m, n)$. Una matriz cuadrada $A \in K^{n \times n}$ se denomina *singular* si $\text{rango}(A) < n$; *regular* si $\text{rango}(A) = n$.

La aplicación asociada a una matriz $A \in \mathfrak{R}^{m \times n}$ es suprayectiva si $\text{rango}(A) = m$. Para una matriz $A \in K^{m \times n}$ se cumple que

$$\dim(\ker(A)) + \text{rango}(A) = n,$$

o, alternativamente, $\dim(\ker(A)) = n - \text{rango}(A)$. La aplicación lineal asociada a A es, por tanto, inyectiva, si y sólo si $\text{rango}(A) = n$.

A.4.1 Normas de matrices

Aun cuando en lo que sigue nos limitaremos a *matrices cuadradas*, la mayor parte de las definiciones y resultados son extensibles a matrices rectangulares; también supondremos que las matrices son reales.

Las matrices cuadradas de orden n forman un espacio vectorial con un producto, esto es, un *álgebra*. Una *norma matricial* es una norma vectorial compatible con el producto. Se define formalmente sobre $\mathfrak{R}^{m \times n}$ como una aplicación $\|\cdot\| : \mathfrak{R}^{m \times n} \rightarrow \mathfrak{R}$ que cumple:

- 1) $\|A\| = 0 \implies A = 0$.
- 2) $\|\lambda A\| = |\lambda| \cdot \|A\|$.
- 3) $\|A + B\| \leq \|A\| + \|B\|$.
- 4) $\|AB\| \leq \|A\| \cdot \|B\|$.

³Recordemos: máximo número de vectores linealmente independientes.

Existen normas sobre el espacio $\Re^{m \times n}$ que no son normas matriciales pues no cumplen la propiedad 4). Así, si se define

$$\|A\| = \max_{1 \leq i, j \leq n} |a_{ij}|,$$

se satisfacen 1), 2) y 3); sin embargo, tomando $A = B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, es fácil ver que $\|AB\| = 2 > \|A\| \cdot \|B\| = 1$, por lo que no se cumple 4).

Un ejemplo importante de norma matricial es la *norma de Frobenius*, definida como:

$$\|A\|_F^2 = \sum_{1 \leq i, j \leq n} a_{ij}^2 = \text{traza}(A^T A),$$

donde la *traza* de una matriz A de orden n es $\sum_{i=1}^n a_{ii}$. Es fácil ver que esta norma deriva del producto escalar $\langle A|B \rangle = \text{traza}(A^T B)$, que configura al espacio de las matrices cuadradas como un espacio prehilbertiano. La norma de Frobenius cumple que

$$\|AB\|_F \leq \|A\|_F \cdot \|B\|_F,$$

como se puede comprobar fácilmente.

Una norma matricial $\|\cdot\|$ sobre $\Re^{m \times n}$ se dice *consistente* con una norma vectorial $\|\cdot\|'$ sobre $\Re^n$ cuando para cada matriz A y cada vector x se cumple que

$$\|Ax\|' \leq \|A\| \cdot \|x\|'.$$

Por ejemplo, la norma de Frobenius y la norma euclídea de $\Re^n$ son consistentes pues

$$\|Ax\|_2 \leq \|A\|_F \cdot \|x\|_2.$$

Se demuestra que para toda norma matricial es posible construir una norma vectorial consistente. Recíprocamente, a toda norma vectorial sobre $\Re^n$ se le puede asociar una norma matricial consistente. Una norma matricial consistente con una cierta norma vectorial $\|\cdot\|$ se construye mediante la definición

$$\|A\| = \sup_{\mathbf{0} \neq \mathbf{x} \in \Re^n} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}.$$

Esta norma matricial se dice *inducida* por la norma vectorial. Ejemplo: la norma matricial inducida por la norma euclídea de $\Re^n$ es la *norma espectral*:

$$\|A\|_2 = \sup_{\mathbf{0} \neq \mathbf{x} \in \Re^n} \left[\frac{\mathbf{x}^T A^T A \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \right]^{1/2} = \sqrt{\lambda_{\max}(A^T A)} = \sigma_{\max}(A),$$

donde λ designa un autovalor de A y σ un valor singular. Si $\|\cdot\|$ es la norma inducida por una cierta norma vectorial y $\|\cdot\|'$ es una norma matricial cualquiera consistente con esa norma vectorial, se cumple, para toda matriz A , que $\|A\| \leq \|A\|'$. En particular, para la norma espectral y la norma de Frobenius, se cumple que

$$\|A\|_2 \leq \|A\|_F.$$

Las normas matriciales inducidas más usadas son

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}| \quad \text{y}$$

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|.$$

Ejemplo A.1 El efecto que produce aplicar la transformación lineal basada en la matriz

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}$$

sobre la bola unidad definida a partir de las normas $\|\cdot\|_1$, $\|\cdot\|_2$ y $\|\cdot\|_\infty$ en $\mathbb{R}^2$, se representa en la figura A.3. La aplicación transforma el vector $e_1 = [1, 0]^T$ en sí mismo y $e_2 = [0, 1]^T$ en $[2, 2]^T$. Tomando la norma 1, el vector unitario que más se amplifica al aplicarle la transformación es $[0, 1]^T$ (o $[0, -1]^T$), que pasa a ser $[2, 2]^T$. Su factor de amplificación, en términos de la norma 1, es 4.

Tomando la norma 2, el vector unitario que más se amplifica es el que se representa en la figura con una recta discontinua. El factor de amplificación es 2,9208.

Para la norma ∞ , igualmente, el vector unitario que más se amplifica es el que se representa también con la recta discontinua: $[1, 1]^T$, que pasa a transformarse en $[3, 2]^T$. El factor de amplificación correspondiente es en este caso 3 ya que

$$\begin{aligned} \|[1, 1]^T\|_\infty &= 1 \\ \|[3, 2]^T\|_\infty &= 3. \end{aligned}$$

•

A.4.2 Matrices ortogonales, matrices de permutación y matrices de proyección

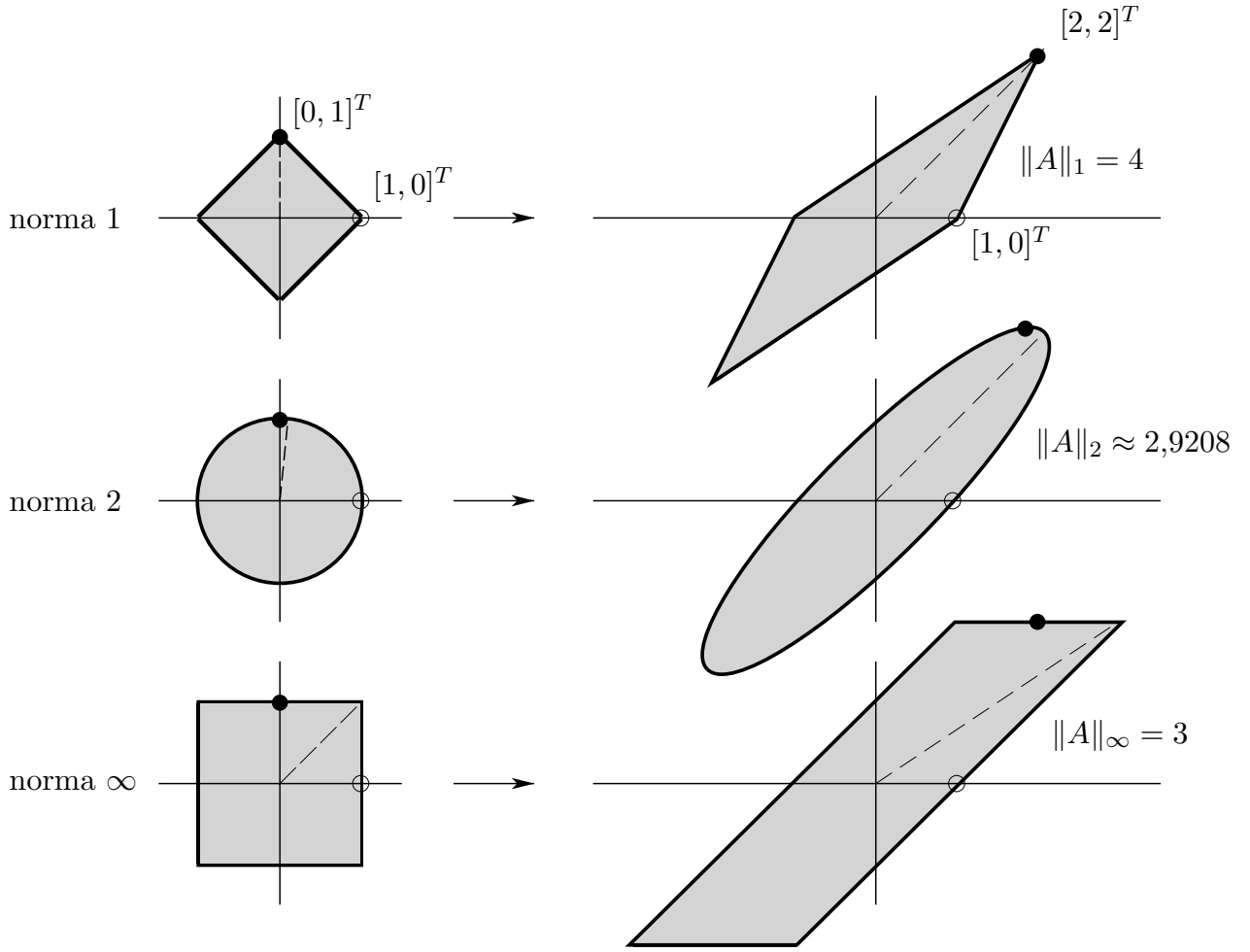
Una matriz $Q \in \mathbb{R}^{m \times n}$ se dice *ortogonal* si verifica que $Q^T Q = I$; es decir, cuando sus vectores columna son ortogonales dos a dos y de norma euclídea unitaria (ortonormales). Si $Q \in \mathbb{R}^{n \times n}$ es ortogonal, se cumple que $Q Q^T = Q^T Q = I$.

Las matrices ortogonales $Q \in \mathbb{R}^{m \times n}$ verifican:

$$\left. \begin{aligned} \|Q\|_2 &= 1 \\ \|Q\|_F &= n^{1/2} \\ \|QA\|_2 &= \|A\|_2 \\ \|QA\|_F &= \|A\|_F \end{aligned} \right\} \quad \text{si } m \geq n$$

y

$$\left. \begin{aligned} \|Q\|_2 &= 1 \\ \|Q\|_F &= m^{1/2} \\ \|AQ\|_2 &= \|A\|_2 \\ \|AQ\|_F &= \|A\|_F \end{aligned} \right\} \quad \text{si } m \leq n$$

**Figura A.3**

Efecto de una aplicación lineal sobre la bola unidad para diferentes normas

Una *matriz de permutación* es una matriz cuadrada cuyas columnas están formadas por las de la matriz unidad permutadas. Una matriz de permutación es una matriz ortogonal.

Una matriz se dice *simétrica* si se verifica que $A^T = A$. Para una matriz cualquiera $A \in \mathbb{R}^{m \times n}$, la matriz $A^T A$ es simétrica.

Se denomina *proyector* o *matriz de proyección* a una matriz $P \in \mathbb{R}^{n \times n}$ que verifica que

$$P^2 = P$$

Si P además es simétrica, se denomina *proyector ortogonal* o *matriz de proyección ortogonal*. Si, en este último caso, F es el subespacio imagen de la matriz P (el mismo que el de la matriz P^T), $P\mathbf{x}$ define la *proyección ortogonal* del vector $\mathbf{x}$ sobre F .

Se denomina *proyector suplementario* de P al proyector $S = I - P$. Si $F = \text{Im}(P)$ y $G = \text{ker}(P)$, entonces $F = \text{ker}(S)$ y $G = \text{Im}(S)$.

En el caso de un proyector ortogonal P en el que $F = \text{Im}(P)$, se tiene que $\mathbb{R}^n = F \oplus F^\perp$,

verificándose que $\|P\mathbf{x}\|_2 \leq \|\mathbf{x}\|_2$ y que

$$\|\mathbf{x} - P\mathbf{x}\|_2 = \min_{\mathbf{y} \in \text{Im}(P)=F} \|\mathbf{x} - \mathbf{y}\|_2.$$

A.5 Autovalores, valores singulares y formas cuadráticas

A.5.1 Autovalores

Sea A una matriz cuadrada de orden n y elementos en $\mathbf{K}$ ($\mathbb{R}$ o $\mathbf{C}$). Un vector no nulo $\mathbf{u} \in \mathbf{K}^n$ se denomina *vector propio* de A si para algún $\lambda \in \mathbf{K}$ se cumple que

$$A\mathbf{u} = \lambda\mathbf{u}.$$

A este λ se le denomina *valor propio* o *autovalor* de la matriz A . El conjunto de los valores propios de una matriz A se denomina *espectro* de A , designándose por $\Lambda(A)$. El *radio espectral*, $\rho(A)$, se define de la siguiente manera:

$$\rho(A) = \max_{1 \leq i \leq n} |\lambda_i|.$$

Para que un número λ sea valor propio de A , el sistema lineal y homogéneo de ecuaciones dado por

$$(\lambda I - A)\mathbf{x} = \mathbf{0}$$

debe tener soluciones distintas de la trivial $\mathbf{x} = \mathbf{0}$. Esto equivale a que

$$\det(A - \lambda I) = 0.$$

Esta es una ecuación polinómica de grado n en λ que se denomina *ecuación característica* de la matriz A . La ecuación característica admite la raíz $\lambda = 0$ si y sólo si $\det(A) = 0$. Una matriz es invertible, por tanto, si y sólo si no admite al cero como vector propio.

Una matriz real de orden n no tiene necesariamente valores propios reales pero, como consecuencia del teorema fundamental del álgebra, cualquier matriz compleja tiene al menos un valor propio complejo. El número máximo de valores propios es n .

Siendo λ un valor propio de la matriz A , el conjunto de soluciones del sistema de ecuaciones

$$(\lambda I - A)\mathbf{x} = \mathbf{0}$$

es un subespacio de $\mathbf{K}^n$ que se denomina *subespacio propio* asociado al valor propio λ , designándose con E_λ . Si n_λ es la multiplicidad de λ como raíz de la ecuación característica de A , se cumple que

$$\dim(E_\lambda) \leq n_\lambda.$$

La intersección de subespacios propios correspondientes a valores propios distintos se reduce al subespacio nulo; esto es,

$$\lambda \neq \mu \implies E_\lambda \cap E_\mu = \emptyset.$$

De este modo, la suma de subespacios propios es directa. Se cumple que

$$\bigoplus_{\lambda \in \Lambda(A)} E_\lambda = \mathbf{K}^n$$

si y sólo si para cada $\lambda \in \Lambda(A)$, $\dim(E_\lambda) = n_\lambda$; en ese caso existe una base de $\mathbf{K}^n$ formada toda ella por vectores propios de A . Siendo V una matriz cuadrada invertible de orden n cuyas columnas son los vectores de esa base, se tiene que

$$AV = VD,$$

donde $D = \text{diag}(\lambda_1, \dots, \lambda_n)$. Alternativamente, se puede escribir que

$$V^{-1}AV = D,$$

por lo que la matriz A es *semejante* a una matriz diagonal; se dice entonces que la matriz A es diagonalizable por semejanza.

Toda matriz real y simétrica tiene todos sus valores propios reales y es diagonalizable por semejanza. Se demuestra además que los subespacios propios correspondientes a valores propios distintos son ortogonales. De aquí se sigue que es siempre posible formar una base ortonormalizada de vectores propios para una matriz real y simétrica A . Existe entonces una matriz ortogonal Q tal que se verifica que

$$Q^T A Q = D, \quad \text{con } Q^T = Q^{-1},$$

y, de aquí que, toda matriz real y simétrica es *congruente ortogonal* con su reducida diagonal. Este resultado fundamental de la teoría de matrices es la versión elemental del denominado teorema espectral.

A.5.2 Valores singulares

La noción de autovalor no tiene significado para matrices rectangulares. En éstas, por el contrario, se introduce el concepto de *valor singular*. Si A es una matriz rectangular $m \times n$ con elementos en $\mathbb{R}$, se definen sus *valores singulares* $\sigma_i, i = 1, \dots, \min\{m, n\}$, como las raíces cuadradas positivas de los valores propios de la matriz cuadrada $A^T A \in \mathbb{R}^{n \times n}$.

Se demuestra que si $A \in \mathbb{R}^{m \times n}$, existen dos matrices ortogonales,

$$U = [\mathbf{u}_1, \dots, \mathbf{u}_m] \in \mathbb{R}^{m \times m}$$

y

$$V = [\mathbf{v}_1, \dots, \mathbf{v}_n] \in \mathbb{R}^{n \times n},$$

tales que

$$U^T A V = \text{diag}(\sigma_1, \dots, \sigma_p), \quad p = \min\{m, n\},$$

y donde

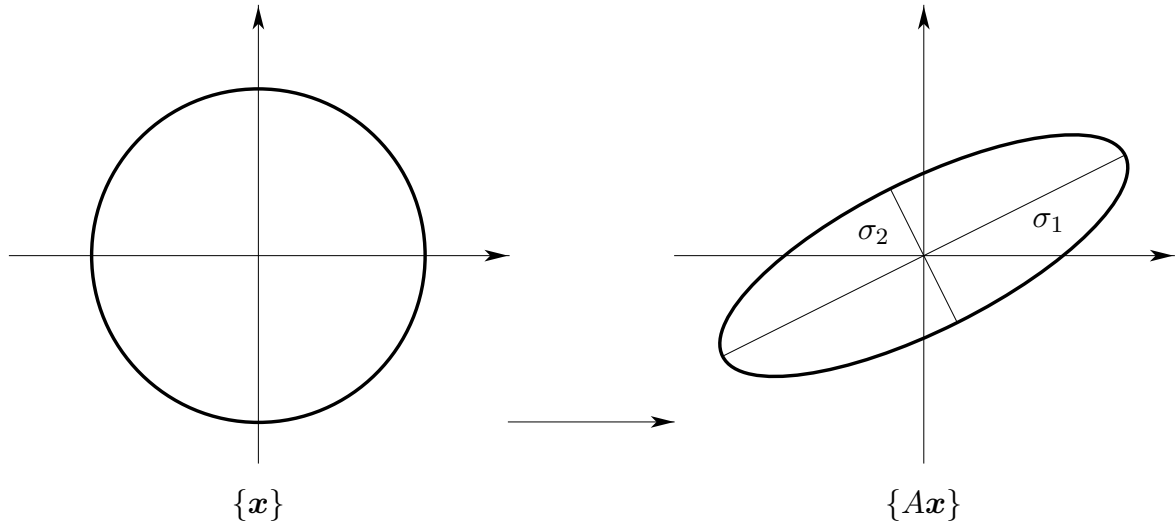
$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0.$$

Los vectores $\mathbf{u}_i$ se denominan vectores singulares izquierdos; los $\mathbf{v}_i$, vectores singulares derechos.

Los valores singulares de A son las longitudes de los semiejes del hiperelipsoide E definido por

$$E = \{\mathbf{y} : \mathbf{y} = A\mathbf{x}, \|\mathbf{x}\|_2 = 1\}.$$

Es decir, las longitudes de los semiejes del hiperelipsoide imagen de la esfera unidad resultante de la aplicación que caracteriza la matriz A . En la figura A.4 se describe gráficamente el caso en que $m = n = 2$.

**Figura A.4**

Representación en dos dimensiones de una transformación lineal de la esfera unidad

Para una matriz $A \in \mathfrak{R}^{m \times n}$ cuya descomposición en valores singulares es $A = U\Sigma V^T$, se define su *matriz pseudoinversa*, $A^\dagger$, como

$$A^\dagger = V\Sigma^\dagger U^T,$$

donde

$$\Sigma^\dagger = \text{diag}(\sigma_1^{-1}, \dots, \sigma_r^{-1}, 0, \dots, 0) \in \mathfrak{R}^{n \times m}.$$

Si $A \in \mathfrak{R}^{m \times n}$ es de rango completo y $m > n$,

$$A^\dagger = (A^T A)^{-1} A^T;$$

si $m < n$

$$A^\dagger = A^T (A A^T)^{-1}.$$

Para cualquier matriz $A \in \mathfrak{R}^{m \times n}$, la matriz $A^\dagger A$ es la matriz $n \times n$ de proyección ortogonal sobre el subespacio de los vectores fila de A , $A A^\dagger$ la $m \times m$ de proyección ortogonal sobre la imagen de la matriz A (subespacio de sus vectores columna) y $(I - A^\dagger A)$ la de proyección ortogonal sobre el núcleo de A .

A.5.3 Formas cuadráticas

Una *forma cuadrática* en n variables es un polinomio de segundo grado en esas variables. La expresión más general de una forma cuadrática es

$$q(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x},$$

donde $Q = Q^T$ es una matriz simétrica de orden n . Nos limitaremos al análisis de formas cuadráticas con coeficientes reales.

Mediante una transformación lineal de variables, $\mathbf{x} = T\mathbf{y}$, una forma cuadrática se puede reducir a la forma canónica de *suma de cuadrados* siguiente:

$$q(\mathbf{x}) = \sum_{i=1}^p y_i^2 - \sum_{i=p+1}^{p+q} y_i^2.$$

El *rango* de la forma es $p + q$ y la *signatura* $p - q$ (p números positivos y q negativos).

Una forma cuadrática real es *definida positiva* si para todo vector $\mathbf{x} \neq \mathbf{0}$, $q(\mathbf{x}) > 0$. El rango y signatura de una forma cuadrática definida positiva valen n . Si Q la forman los coeficientes q_{ij} y se introducen los números *menores* como

$$\Delta_i = \det \begin{bmatrix} q_{11} & q_{12} & \cdots & q_{1i} \\ q_{21} & q_{22} & \cdots & q_{2i} \\ \vdots & \vdots & \ddots & \vdots \\ q_{i1} & q_{i2} & \cdots & q_{ii} \end{bmatrix},$$

la forma cuadrática asociada a Q es definida positiva si y sólo si todos los menores Δ_i son positivos.

Sean $\lambda_1, \dots, \lambda_n$ los valores propios —que sabemos son reales— de la matriz Q ; por el teorema espectral, existe una matriz ortogonal P tal que

$$P^T Q P = \text{diag}(\lambda_1, \dots, \lambda_n).$$

Haciendo en la forma cuadrática $q(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$ el cambio de variables $\mathbf{x} = P\mathbf{y}$, se tiene que

$$q(\mathbf{x}) = \mathbf{y}^T P^T Q P \mathbf{y} = \lambda_1 y_1^2 + \cdots + \lambda_n y_n^2,$$

lo que hace ver que el rango de la forma cuadrática es el número total —teniendo en cuenta las multiplicidades— de valores propios no nulos de Q , mientras que la signatura coincide con la diferencia entre los números de valores propios positivos y negativos. En particular, la forma cuadrática asociada a Q es definida positiva si y sólo si todos los valores propios de Q son positivos.

En ciertos casos es importante acotar el cociente de una forma cuadrática al cuadrado de la norma euclídea, es decir, el cociente

$$r(\mathbf{x}) = \frac{\mathbf{x}^T Q \mathbf{x}}{\mathbf{x}^T \mathbf{x}}, \quad \mathbf{x} \neq \mathbf{0}.$$

Mediante una transformación ortogonal $\mathbf{x} = P\mathbf{y}$, este cociente se escribe como

$$r(\mathbf{x}) = \frac{\lambda_1 y_1^2 + \cdots + \lambda_n y_n^2}{y_1^2 + \cdots + y_n^2},$$

de manera que se deducen las acotaciones

$$\lambda_{\min}(Q) \leq \frac{\mathbf{x}^T Q \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \leq \lambda_{\max}(Q).$$

Estas acotaciones no se pueden mejorar ya que si $Q\mathbf{v} = \lambda\mathbf{v}$,

$$\frac{\mathbf{v}^T Q \mathbf{v}}{\mathbf{v}^T \mathbf{v}} = \lambda.$$

Una matriz A se dice *definida positiva* si la forma cuadrática $\mathbf{x}^T A \mathbf{x}$ es positiva para todo vector $\mathbf{x} \neq \mathbf{0}$. De forma similar se definen matrices *semidefinida positiva*, *definida negativa* y *semidefinida negativa*, si $\mathbf{x}^T A \mathbf{x} \geq 0$, < 0 y ≤ 0 , respectivamente, para todo vector $\mathbf{x} \neq \mathbf{0}$. La matriz A se dice *indefinida* si la forma $\mathbf{x}^T A \mathbf{x}$ es positiva para algún $\mathbf{x}$ y negativa para otros.

Criterio A.1 Para que una matriz simétrica sea definida positiva, es necesario que todos los coeficientes de la diagonal principal sean positivos.

Criterio A.2 Para que una matriz simétrica A sea definida positiva es necesario que el coeficiente de mayor valor absoluto esté en la diagonal principal. Más concretamente,

$$\max_{i \neq j} |a_{ij}| < \max_k a_{kk}.$$

La matriz

$$\begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix},$$

de acuerdo con el criterio A.1, no puede ser definida positiva, aunque cumple el criterio A.2. Por el contrario, la matriz

$$\begin{bmatrix} 5 & 2 & 7 \\ 2 & 5 & 2 \\ 7 & 2 & 5 \end{bmatrix},$$

satisface el criterio A.1 pero no el A.2. La matriz

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 4 & 5 \end{bmatrix},$$

satisface los dos criterios enunciados pero no es definida positiva ya que, por ejemplo,

$$q([2, -3, 1]^T) = -5.$$

Criterio A.3 Si en cada fila de una matriz simétrica A el elemento de la diagonal principal es mayor que la suma de los valores absolutos de todos los demás elementos de la fila, es decir, si

$$a_{kk} > \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| \quad k = 1, \dots, n,$$

A es definida positiva.

DEMOSTRACIÓN. Para $\mathbf{x} \neq \mathbf{0}$ se tendrá que

$$\begin{aligned}
 q(\mathbf{x}) &= \sum_i \sum_j a_{ij} x_i x_j \geq \sum_i a_{ii} x_i^2 - \sum_i \sum_{j \neq i} |a_{ij}| |x_i| |x_j| \\
 &> \sum_i \left(\sum_{j \neq i} |a_{ij}| \right) |x_i|^2 - \sum_i \sum_{j \neq i} |a_{ij}| |x_i| |x_j| \\
 &= \sum_i \sum_{j \neq i} |a_{ij}| |x_i| (|x_i| - |x_j|) = \sum_i \sum_{j \neq i} |a_{ij}| |x_j| (|x_j| - |x_i|) \\
 &= \frac{1}{2} \sum_i \sum_{j \neq i} |a_{ij}| (|x_i| - |x_j|)^2 \geq 0.
 \end{aligned}$$

■

Es importante destacar que este último criterio define una condición suficiente, no necesaria. En efecto, la matriz

$$\begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & 2 \\ 2 & 2 & 3 \end{bmatrix},$$

es definida positiva, pues

$$q(\mathbf{x}) = x_1^2 + x_2^2 + x_3^2 + 2(x_1 + x_2 + x_3)^2,$$

lo que atestigua que, cualquiera que sea $\mathbf{x} \neq \mathbf{0}$, $q(\mathbf{x}) > 0$. Esta matriz, sin embargo, no satisface el criterio A.3.

Como ya se ha visto, una matriz simétrica definida positiva tiene todos sus autovalores reales y positivos; si es semidefinida, algún autovalor es cero. Si la matriz es negativa definida, todos sus autovalores son negativos.

Un teorema muy interesante para averiguar el orden de magnitud de los autovalores de una matriz es el *teorema de Gerschgorin*, que dice que si $A \in \mathfrak{R}^{n \times n}$ es una matriz simétrica con autovalores $\lambda_1, \lambda_2, \dots, \lambda_n$, entonces

$$\begin{aligned}
 \min_{1 \leq i \leq n} \lambda_i &\geq \min_{1 \leq i \leq n} \left\{ a_{ii} - \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \right\}, \\
 \max_{1 \leq k \leq n} \lambda_k &\leq \max_{1 \leq k \leq n} \left\{ a_{kk} + \sum_{\substack{j=1 \\ j \neq k}}^n |a_{kj}| \right\}.
 \end{aligned}$$

Como corolario de este teorema se puede probar también que si A es de diagonal estrictamente dominante y simétrica, es definida positiva.

Se dice que una matriz compleja A , de coeficientes a_{ij} , cuadrada y de orden n , es de *diagonal estrictamente dominante* por filas, o simplemente de diagonal dominante por filas, cuando cumple que

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad i = 1, \dots, n.$$

Puede darse una definición análoga de matriz de diagonal dominante por columnas.

A.6 Topología

En un espacio vectorial normado se define una *bola abierta*, $S(\mathbf{x}_0, r)$, de centro $\mathbf{x}_0$ y radio r , como el conjunto de puntos $\mathbf{x}$ que verifican $\|\mathbf{x} - \mathbf{x}_0\| < r$. Es decir:

$$S(\mathbf{x}_0, r) = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x} - \mathbf{x}_0\| < r\}.$$

Una *bola cerrada*, $\bar{S}(\mathbf{x}_0, r)$, se define, por el contrario, como el conjunto de puntos $\mathbf{x}$ que verifican $\|\mathbf{x} - \mathbf{x}_0\| \leq r$. Es decir:

$$\bar{S}(\mathbf{x}_0, r) = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x} - \mathbf{x}_0\| \leq r\}.$$

Consideraremos en lo que sigue de este apartado un subconjunto S del espacio vectorial métrico hasta ahora estudiado (puede ser, por ejemplo, $\mathbb{R}^n$).

Un punto $\mathbf{y} \in S$ es un *punto interior* del conjunto S si existe un ε tal que

$$\|\mathbf{x} - \mathbf{y}\| < \varepsilon \Rightarrow \mathbf{x} \in S.$$

En otras palabras, existe una bola abierta $S(\mathbf{y}, \varepsilon)$ de centro $\mathbf{y}$ y radio ε contenida íntegramente en S .

El conjunto de todos los puntos interiores del conjunto S se denomina *interior* de S . Este conjunto puede, evidentemente, ser vacío. Ejemplo: un plano del espacio $\mathbb{R}^3$.

Un subconjunto de S se dice *abierto* si coincide con su interior; es decir, si alrededor de todo punto de S existe una bola abierta contenida íntegramente en S . Ejemplo, la bola abierta unidad, $S(\mathbf{x}, 1) = \{\mathbf{x} : \|\mathbf{x}\| < 1\}$; el espacio $\mathbb{R}^n$ en su totalidad. En general los subconjuntos o conjuntos abiertos se caracterizan por no tener límites definidos o ser disjuntos de su frontera (ver más adelante la definición del concepto frontera).

Un *entorno* de un punto $\mathbf{x}$, $\mathcal{E}(\mathbf{x})$, es un conjunto abierto que contiene a $\mathbf{x}$. En otras palabras, $\mathcal{E}(\mathbf{x})$ es un entorno de $\mathbf{x}$ si contiene una bola abierta de centro $\mathbf{x}$.

Se dice que un punto $\mathbf{x}$ es un *punto de acumulación* del subconjunto S si en todo entorno de $\mathbf{x}$ existen un número infinito de puntos de S .

Un punto $\mathbf{x}$ se denomina *punto de adherencia* del subconjunto S cuando todo entorno de dicho punto $\mathbf{x}$ contiene al menos un punto de S ; es decir, para todo ε existe un $\mathbf{y} \in S$ tal que $\|\mathbf{x} - \mathbf{y}\| < \varepsilon$. El conjunto de todos los puntos de adherencia se denomina *adherencia*. La adherencia de la bola abierta $S(\mathbf{x}, 1) = \{\mathbf{x} : \|\mathbf{x}\| < 1\}$ es la cerrada $\bar{S}(\mathbf{x}, 1) = \{\mathbf{x} : \|\mathbf{x}\| \leq 1\}$.

Se denomina *frontera* de un conjunto a la parte de la adherencia que no está en el interior.

Un conjunto, o subconjunto, se dice *cerrado* si coincide con su adherencia. La adherencia de cualquier conjunto S es el conjunto cerrado más pequeño que contiene a S . Se puede demostrar que un conjunto es cerrado si y sólo si toda sucesión convergente de elementos de S tiene un límite en ese conjunto.

Un conjunto, o subconjunto, se dice *compacto* si es cerrado y acotado (contenido en una bola de radio $r < \infty$). Un importante resultado, debido a Weierstrass, dice que si S es un conjunto compacto, de cada sucesión o sucesión infinita $\{\mathbf{x}^{(n)}\}_{n \in \mathbf{N}}$ de elementos de dicho conjunto es posible extraer una subsucesión

$$\{\mathbf{x}^{(\ell)}\}_{\ell \in L} \quad L \subset \mathbf{N}$$

que converge a un elemento del propio conjunto S .

Si $\{r^{(k)}\}$ es una sucesión de números reales y $s^{(k)} = \sup \{r^{(i)} : i \geq k\}$, entonces $\{s^{(k)}\}$ converge a un número real s_0 ; a este número se le denomina *límite superior* de $\{r^{(k)}\}$ y se expresa como

$$\limsup \left(r^{(k)} \right) \quad \text{o} \quad \overline{\lim}_{k \rightarrow \infty} \left(r^{(k)} \right).$$

El límite superior de una sucesión de números reales es el mayor punto de acumulación de la sucesión. De forma similar se define el límite inferior.

A.7 Teorema de la proyección

Gran parte de las teorías de sistemas de ecuaciones y de optimización que se estudian en el libro están basadas en unos pocos resultados simples e intuitivos. Entre estos, quizás el más sencillo y usado sea el teorema de la proyección. Su aplicación en la teoría de mínimos cuadrados lineales es fundamental. En un espacio Euclídeo ordinario de tres dimensiones determina que la distancia más corta de un punto exterior a un plano a ese plano la proporciona la perpendicular al plano desde dicho punto. La expresión formal de este teorema en espacios de Hilbert es la que sigue.

Teorema A.1 Sea H un espacio de Hilbert y M un subespacio cerrado de H . Para todo vector $\mathbf{x} \in H$ existe un único vector $\mathbf{m}_0 \in M$ tal que $\|\mathbf{x} - \mathbf{m}_0\|_2 \leq \|\mathbf{x} - \mathbf{m}\|_2$, para todo $\mathbf{m} \in M$. La condición necesaria y suficiente además para que $\mathbf{m}_0 \in M$ sea el vector mínimo único es que $\mathbf{x} - \mathbf{m}_0$ sea ortogonal a M .

DEMOSTRACIÓN. Primero probaremos que si $\mathbf{m}_0$ es un vector que minimiza $\|\mathbf{x} - \mathbf{m}\|$, $\mathbf{x} - \mathbf{m}_0$ es ortogonal a M . Supongamos para ello, por el contrario, que existe un $\mathbf{m}$ que no es ortogonal a $\mathbf{x} - \mathbf{m}_0$; sin pérdida de generalidad podemos suponer que $\|\mathbf{m}\| = 1$ y que $\langle \mathbf{x} - \mathbf{m}_0 | \mathbf{m} \rangle = \delta \neq 0$. Definamos el vector $\mathbf{m}_1 \in M$ como $\mathbf{m}_1 = \mathbf{m}_0 + \delta \mathbf{m}$. Tendremos que

$$\begin{aligned} \|\mathbf{x} - \mathbf{m}_1\|_2^2 &= \|\mathbf{x} - \mathbf{m}_0 - \delta \mathbf{m}\|_2^2 = \|\mathbf{x} - \mathbf{m}_0\|_2^2 - \langle \mathbf{x} - \mathbf{m}_0 | \delta \mathbf{m} \rangle - \langle \delta \mathbf{m} | \mathbf{x} - \mathbf{m}_0 \rangle + |\delta|^2 \\ &= \|\mathbf{x} - \mathbf{m}_0\|_2^2 - |\delta|^2 < \|\mathbf{x} - \mathbf{m}_0\|_2^2. \end{aligned}$$

De esta manera, si $\mathbf{x} - \mathbf{m}_0$ no es ortogonal a M , $\mathbf{m}_0$ no es el mínimo que decíamos.

Veamos ahora cómo, si $\mathbf{x} - \mathbf{m}_0$ es ortogonal al subespacio M , $\mathbf{m}_0$ es el único vector de M que minimiza $\|\mathbf{x} - \mathbf{m}\|_2$. En efecto, para todo $\mathbf{m} \in M$, el teorema de Pitágoras dice que

$$\|\mathbf{x} - \mathbf{m}\|_2^2 = \|\mathbf{x} - \mathbf{m}_0 + \mathbf{m}_0 - \mathbf{m}\|_2^2 = \|\mathbf{x} - \mathbf{m}_0\|_2^2 + \|\mathbf{m}_0 - \mathbf{m}\|_2^2.$$

Por lo tanto $\|\mathbf{x} - \mathbf{m}\|_2 > \|\mathbf{x} - \mathbf{m}_0\|_2$ para $\mathbf{m} \neq \mathbf{m}_0$.

Demostraremos ahora la existencia de un $\mathbf{m}_0$ que minimiza $\|\mathbf{x} - \mathbf{m}\|_2$. Si $\mathbf{x} \in M$, entonces $\mathbf{m}_0 = \mathbf{x}$ y todo estaría probado como es obvio. Si $\mathbf{x} \notin M$, definamos un $\delta = \inf_{\mathbf{m} \in M} \|\mathbf{x} - \mathbf{m}\|_2$; lo que queremos es obtener un $\mathbf{m}_0 \in M$ tal que $\|\mathbf{x} - \mathbf{m}_0\|_2 = \delta$.

A tal fin, sea $\{\mathbf{m}^{(i)}\}$ una sucesión de vectores en M tal que $\|\mathbf{x} - \mathbf{m}^{(i)}\|_2 \rightarrow \delta$. Por la ley del paralelogramo⁴ se tiene que

$$\|(\mathbf{m}^{(j)} - \mathbf{x}) + (\mathbf{x} - \mathbf{m}^{(i)})\|_2^2 + \|(\mathbf{m}^{(j)} - \mathbf{x}) - (\mathbf{x} - \mathbf{m}^{(i)})\|_2^2 = 2\|\mathbf{m}^{(j)} - \mathbf{x}\|_2^2 + 2\|\mathbf{x} - \mathbf{m}^{(i)}\|_2^2.$$

Reordenando, se obtiene

$$\|\mathbf{m}^{(j)} - \mathbf{m}^{(i)}\|_2^2 = 2\|\mathbf{m}^{(j)} - \mathbf{x}\|_2^2 + 2\|\mathbf{x} - \mathbf{m}^{(i)}\|_2^2 - 4\left\|\mathbf{x} - \frac{\mathbf{m}^{(i)} + \mathbf{m}^{(j)}}{2}\right\|_2^2.$$

Para todo i, j , el vector $(\mathbf{m}^{(i)} + \mathbf{m}^{(j)})/2$ está en M pues éste es un espacio vectorial (lineal). De la definición de δ se deduce que $\|\mathbf{x} - (\mathbf{m}^{(i)} + \mathbf{m}^{(j)})/2\|_2 \geq \delta$, por lo que

$$\|\mathbf{m}^{(j)} - \mathbf{m}^{(i)}\|_2^2 \leq 2\|\mathbf{m}^{(j)} - \mathbf{x}\|_2^2 + 2\|\mathbf{x} - \mathbf{m}^{(i)}\|_2^2 - 4\delta^2.$$

Como $\|\mathbf{m}^{(i)} - \mathbf{x}\|_2^2 \rightarrow \delta^2$ cuando $i \rightarrow \infty$, $\|\mathbf{m}^{(j)} - \mathbf{m}^{(i)}\|_2^2 \rightarrow 0$ cuando $i, j \rightarrow \infty$. Es decir, $\{\mathbf{m}^{(i)}\}$ es una sucesión de Cauchy; como M es un subespacio cerrado, la sucesión $\{\mathbf{m}^{(i)}\}$ tiene un límite $\mathbf{m}_0$ en M y, debido a la continuidad de la norma, $\|\mathbf{x} - \mathbf{m}_0\|_2 \rightarrow \delta$. ■

A.8 Funciones

Recordemos que una función es un caso particular de aplicación donde los conjuntos origen e imagen son conjuntos de números.

Una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ se dice *continua* en $\mathbf{x}$ si para toda sucesión $\{\mathbf{x}^{(k)}\}$ que converge a $\mathbf{x}$ (expresado $\mathbf{x}^{(k)} \rightarrow \mathbf{x}$), se cumple que $f(\mathbf{x}^{(k)}) \rightarrow f(\mathbf{x})$. De forma equivalente, f se dice continua en $\mathbf{x}$ si dado un $\varepsilon > 0$, existe un $\delta > 0$ tal que

$$\|\mathbf{y} - \mathbf{x}\| < \delta \implies \|f(\mathbf{y}) - f(\mathbf{x})\| < \varepsilon.$$

Una función $f : \mathbb{R} \rightarrow \mathbb{R}$ se dice satisface la *condición de Lipschitz* con constante γ en un conjunto X , si para todo x e y pertenecientes a X se cumple que

$$|f(x) - f(y)| \leq \gamma|x - y|.$$

Una función que satisface la condición de Lipschitz en un conjunto X se dice *continua γ -Lipschitz* en ese X , designándose $f \in \text{Lip}_\gamma(X)$.

Dada una norma vectorial $\|\cdot\|$ en $\mathbb{R}^n$ y otra matricial $\|\cdot\|$ en $\mathbb{R}^{m \times n}$, $m, n > 0$, una función $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$ se dice satisface la *condición de Lipschitz* con constante γ en un abierto $D \subset \mathbb{R}^n$, si para todo $\mathbf{x}$ e $\mathbf{y}$ pertenecientes a D se cumple que

$$\|\mathbf{g}(\mathbf{x}) - \mathbf{g}(\mathbf{y})\| \leq \gamma\|\mathbf{x} - \mathbf{y}\|.$$

⁴Para $\mathbf{u}, \mathbf{w} \in M$, $|\mathbf{u} + \mathbf{w}|^2 + |\mathbf{u} - \mathbf{w}|^2 = 2|\mathbf{u}|^2 + 2|\mathbf{w}|^2$.

Una función $\mathbf{g}$ que satisface la condición de Lipschitz en D se dice *continua γ -Lipschitz* en ese D , designándose $\mathbf{g} \in \text{Lip}_\gamma(D)$.

Un resultado muy interesante referido a funciones continuas es el *teorema de Weierstrass*, que dice que una función continua definida en un conjunto compacto S tiene un punto donde alcanza un mínimo en S . Es decir, existe un $\mathbf{x}^* \in S$ tal que para todo $\mathbf{x} \in S$, $f(\mathbf{x}) \geq f(\mathbf{x}^*)$.

Un conjunto de funciones $f_1, f_2, \dots, f_m$ de $\mathbb{R}^n$ en $\mathbb{R}$ se puede considerar como una función vectorial

$$\mathbf{f} = [f_1, f_2, \dots, f_m]^T.$$

Esta función asigna a todo vector $\mathbf{x} \in \mathbb{R}^n$ otro vector $\mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})]^T$ de $\mathbb{R}^m$. Tal función vectorial se dice continua si lo es cada uno de sus componentes $f_1, f_2, \dots, f_m$.

Si cada uno de los componentes de $\mathbf{f} = [f_1, f_2, \dots, f_m]^T$ es continua en algún conjunto abierto de $\mathbb{R}^n$, se dice $\mathbf{f} \in C$. Si además cada función componente tiene derivadas parciales de primer orden continuas en ese abierto, se dice que $\mathbf{f} \in C^1$. En general, si las funciones componentes tienen derivadas parciales de orden p continuas, se indica $\mathbf{f} \in C^p$.

Si $f : \mathbb{R}^n \rightarrow \mathbb{R}$ y $f \in C^1$, se define el *vector gradiente* de f como el vector

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right]^T.$$

También se suele expresar como $\mathbf{f}_x(\mathbf{x})$.

Si $f \in C^2$, se define la *Hessiana*, o *matriz Hessiana*, de f en $\mathbf{x}$ como la matriz $n \times n$

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial^2 x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial^2 x_2} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_1} & \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f(\mathbf{x})}{\partial^2 x_n} \end{bmatrix}.$$

A esta matriz también se la designa habitualmente como $\mathbf{F}(\mathbf{x})$.

Para una función vectorial $\mathbf{f} = [f_1, f_2, \dots, f_m]^T$, si $\mathbf{f} \in C^1$, se define la *matriz Jacobiana* o, simplemente, la *Jacobiana*, como la matriz $m \times n$

$$\nabla \mathbf{f}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \frac{\partial f_1(\mathbf{x})}{\partial x_2} & \dots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \frac{\partial f_2(\mathbf{x})}{\partial x_1} & \frac{\partial f_2(\mathbf{x})}{\partial x_2} & \dots & \frac{\partial f_2(\mathbf{x})}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m(\mathbf{x})}{\partial x_1} & \frac{\partial f_m(\mathbf{x})}{\partial x_2} & \dots & \frac{\partial f_m(\mathbf{x})}{\partial x_n} \end{bmatrix}.$$

Si $\mathbf{f} \in C^2$, es posible definir m Hessianas $\mathbf{F}_1(\mathbf{x}), \mathbf{F}_2(\mathbf{x}), \dots, \mathbf{F}_m(\mathbf{x})$ correspondientes a cada una de las m funciones componentes.

Un resultado de extraordinaria importancia es el conocido *teorema de Taylor*. Dice que si $f : \mathbb{R}^n \rightarrow \mathbb{R}$ y $f \in C^1$ en una región que contiene el segmento $[\mathbf{x}_1, \mathbf{x}_2]$, es decir puntos $\alpha\mathbf{x}_1 + (1 - \alpha)\mathbf{x}_2$, $0 \leq \alpha \leq 1$, existe un θ , $0 \leq \theta \leq 1$, tal que

$$f(\mathbf{x}_2) = f(\mathbf{x}_1) + \nabla^T f(\theta\mathbf{x}_1 + (1 - \theta)\mathbf{x}_2)(\mathbf{x}_2 - \mathbf{x}_1).$$

Además, si $f \in C^2$, existe un θ , $0 \leq \theta \leq 1$, tal que

$$f(\mathbf{x}_2) = f(\mathbf{x}_1) + \nabla^T f(\mathbf{x}_1)(\mathbf{x}_2 - \mathbf{x}_1) + \frac{1}{2}(\mathbf{x}_2 - \mathbf{x}_1)^T \mathbf{F}(\theta\mathbf{x}_1 + (1 - \theta)\mathbf{x}_2)(\mathbf{x}_2 - \mathbf{x}_1),$$

donde $\mathbf{F}$ denota la Hessiana de f .

A.8.1 Condiciones necesarias y suficientes de primer y segundo orden que ha de cumplir un punto mínimo

Se trata de definir condiciones necesarias y suficientes para determinar si un punto $\mathbf{x}^*$ cumple

$$\underset{\mathbf{x}}{\text{minimizar}} f(\mathbf{x}),$$

donde $f : \Omega \rightarrow \mathbb{R}$ y $\Omega \in \mathbb{R}^n$.

Un punto $\mathbf{x}^* \in \Omega$ se dice que es un *mínimo relativo* de la función $f : \Omega \rightarrow \mathbb{R}$ si existe un $\epsilon > 0$ tal que $f(\mathbf{x}) \geq f(\mathbf{x}^*)$ para todo $\mathbf{x} \in \Omega$ a una distancia menor que ϵ de $\mathbf{x}^*$. Es decir, para todo $\mathbf{x} \in \Omega$ tal que $|\mathbf{x} - \mathbf{x}^*| < \epsilon$. Si $f(\mathbf{x}) > f(\mathbf{x}^*)$ para todo $\mathbf{x} \in \Omega$, $\mathbf{x} \neq \mathbf{x}^*$, a una distancia menor que ϵ de $\mathbf{x}^*$, se dice que $\mathbf{x}^*$ es un *mínimo relativo estricto* de f en Ω .

Proposición A.1 (Condiciones necesarias de primer orden) Sea Ω un subconjunto de $\mathbb{R}^n$ y una función $f : \Omega \rightarrow \mathbb{R}$, $f \in C^1$. Si $\mathbf{x}^*$ es un mínimo relativo de f en Ω , para toda dirección $\mathbf{d} \in \mathbb{R}^n$, factible desde $\mathbf{x}^*$, se cumple que $\nabla f(\mathbf{x}^*)\mathbf{d} \geq 0$.

Corolario A.1 Sea Ω un subconjunto de $\mathbb{R}^n$ y una función $f : \Omega \rightarrow \mathbb{R}$, $f \in C^1$. Si $\mathbf{x}^*$ es un mínimo relativo de f en Ω y $\mathbf{x}^*$ es un punto interior de Ω , se cumple que $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

Proposición A.2 (Condiciones necesarias de segundo orden) Sea Ω un subconjunto de $\mathbb{R}^n$ y una función $f : \Omega \rightarrow \mathbb{R}$, $f \in C^2$. Si $\mathbf{x}^*$ es un mínimo relativo de f en Ω , para toda dirección $\mathbf{d} \in \mathbb{R}^n$, factible desde $\mathbf{x}^*$, se cumple que:

$$\begin{aligned} \nabla f(\mathbf{x}^*)\mathbf{d} &\geq 0. \\ \text{Si } \nabla f(\mathbf{x}^*)\mathbf{d} &= 0, \text{ entonces } \mathbf{d}^T \nabla^2 f(\mathbf{x}^*)\mathbf{d} \geq 0. \end{aligned}$$

Proposición A.3 (Condiciones necesarias de segundo orden) Sea $\mathbf{x}^*$ un punto interior de Ω y supóngase que también es un mínimo relativo de $f : \Omega \rightarrow \mathbb{R}$, $f \in C^2$. Entonces:

$$\begin{aligned} \nabla f(\mathbf{x}^*) &= \mathbf{0}. \\ \text{Para todo } \mathbf{d}, \mathbf{d}^T \nabla^2 f(\mathbf{x}^*)\mathbf{d} &\geq 0. \end{aligned}$$

Proposición A.4 (Condiciones suficientes de segundo orden) Sea $f \in C^2$ una función definida en una región en la cual $\mathbf{x}^*$ es un punto interior. Supóngase además que:

$$\nabla f(\mathbf{x}^*) = 0.$$

La matriz Hessiana $\nabla^2 f(\mathbf{x}^*)$ es definida positiva.

$\mathbf{x}^*$ es entonces un mínimo relativo estricto de f .

Las demostraciones de estas proposiciones y corolario se pueden seguir en Luenberger [1984].

A.9 Conjuntos convexos. Existencia de los hiperplanos separador y soporte

Los dos resultados que incluimos a continuación son de gran importancia en la teoría de conjuntos convexos. El primero demuestra que dado un punto fuera de un conjunto convexo, existe un hiperplano que contiene al punto sin tocar al conjunto convexo. El segundo, que dado un punto frontera de un conjunto convexo, existe un hiperplano que contiene ese punto frontera y el conjunto convexo en uno de sus semiespacios cerrados.

Teorema A.2 Sea C un conjunto convexo e $\mathbf{y}$ un punto exterior a la adherencia de C . Existe un vector $\mathbf{a}$ tal que $\mathbf{a}^T \mathbf{y} < \inf_{\mathbf{x} \in C} \mathbf{a}^T \mathbf{x}$.

DEMOSTRACIÓN. Sea

$$\delta = \inf_{\mathbf{x} \in C} \|\mathbf{x} - \mathbf{y}\|_2 > 0.$$

Existe un $\mathbf{x}_0$ en la frontera de C tal que $\|\mathbf{x}_0 - \mathbf{y}\|_2 = \delta$. Esto es así pues la función continua $f(\mathbf{x}) = \|\mathbf{x} - \mathbf{y}\|_2$ alcanza su mínimo en cualquier conjunto cerrado y acotado por lo que sólo es necesario considerar $\mathbf{x}$ en la intersección de la adherencia de C y la bola abierta de centro $\mathbf{y}$ y radio 2δ .

A continuación probaremos que $\mathbf{a} = \mathbf{x}_0 - \mathbf{y}$ satisface las condiciones del enunciado del teorema. En efecto, para cualquier α , $0 \leq \alpha \leq 1$, al ser C un conjunto convexo, el punto $\mathbf{x}_0 + \alpha(\mathbf{x} - \mathbf{x}_0) \in C$, por lo que

$$\|\mathbf{x}_0 + \alpha(\mathbf{x} - \mathbf{x}_0) - \mathbf{y}\|_2^2 \geq \|\mathbf{x}_0 - \mathbf{y}\|_2^2.$$

Desarrollando,

$$2\alpha(\mathbf{x}_0 - \mathbf{y})^T(\mathbf{x} - \mathbf{x}_0) + \alpha^2\|\mathbf{x} - \mathbf{x}_0\|_2^2 \geq 0.$$

Considerando esta expresión cuando $\alpha \rightarrow 0+$, se tiene que

$$(\mathbf{x}_0 - \mathbf{y})^T(\mathbf{x} - \mathbf{x}_0) \geq 0$$

o que

$$\begin{aligned} (\mathbf{x}_0 - \mathbf{y})^T \mathbf{x} &\geq (\mathbf{x}_0 - \mathbf{y})^T \mathbf{x}_0 = (\mathbf{x}_0 - \mathbf{y})^T \mathbf{y} + (\mathbf{x}_0 - \mathbf{y})^T (\mathbf{x}_0 - \mathbf{y}) \\ &= (\mathbf{x}_0 - \mathbf{y})^T \mathbf{y} + \delta^2. \end{aligned}$$

Haciendo $\mathbf{a} = \mathbf{x}_0 - \mathbf{y}$ queda probado el teorema. ■

La interpretación geométrica de este teorema es que, dado un conjunto convexo C y un punto $\mathbf{y}$ exterior a la adherencia de C , existe un hiperplano que contiene a $\mathbf{y}$ estando C en uno de sus semiespacios abiertos. Este hiperplano (de vector característico $\mathbf{a}$ en el teorema) se denomina *hiperplano separador* de C e $\mathbf{y}$.

Teorema A.3 *Sea C un conjunto convexo e $\mathbf{y}$ un punto frontera de C . Existe un hiperplano que contiene a $\mathbf{y}$ y a C en uno de sus semiespacios cerrados.*

DEMOSTRACIÓN. Sea $\{\mathbf{y}^{(k)}\}$ una sucesión de puntos exteriores a la adherencia de C . Sea $\{\mathbf{a}^{(k)}\}$ la sucesión de puntos normalizados, $\|\mathbf{a}^{(k)}\|_2 = 1$, obtenida de aplicar el teorema anterior a la sucesión anterior, tales que,

$$\left(\mathbf{a}^{(k)}\right)^T \mathbf{y}^{(k)} < \inf_{\mathbf{x} \in C} \left(\mathbf{a}^{(k)}\right)^T \mathbf{x}.$$

Como $\{\mathbf{a}^{(k)}\}$ es una sucesión acotada, una subsucesión $\{\mathbf{a}^{(k)}\}$, $k \in \mathcal{H}$, convergerá a un límite $\mathbf{a}$. Para este $\mathbf{a}$ se tiene que, para cualquier $\mathbf{x} \in C$,

$$\mathbf{a}^T \mathbf{y} = \lim_{k \in \mathcal{H}} \left(\mathbf{a}^{(k)}\right)^T \mathbf{y}^{(k)} \leq \lim_{k \in \mathcal{H}} \left(\mathbf{a}^{(k)}\right)^T \mathbf{x} = \mathbf{a}^T \mathbf{x}.$$

■

Un hiperplano que contiene un conjunto convexo C en uno de sus semiespacios cerrados y que contiene algún punto frontera de C se denomina *hiperplano soporte* de C .

De acuerdo con esta definición, el teorema anterior dice que, dado un conjunto convexo C y un punto frontera $\mathbf{y}$ de C , existe un hiperplano soporte de C en $\mathbf{y}$.

Apéndice B

ERRORES DE REDONDEO Y ARITMÉTICA DE PRECISIÓN FINITA

LAS PRESTACIONES de los algoritmos que resuelven problemas numéricos —muy en particular la solución de sistemas de ecuaciones lineales y problemas de optimización lineales y enteros—, dependen en gran medida de la exactitud con la que se pueden representar los números reales en la máquina donde se han de utilizar. Para diseñarlos y codificarlos en un lenguaje que *entienda* la máquina, es necesario conocer cómo opera la aritmética de esa máquina.

Los ordenadores y calculadoras donde se implementan los algoritmos no guardan o almacenan la información relativa a un número con precisión infinita, sino mediante una aproximación empaquetada en grupos de *bits*, denominados *bytes* (grupos de 8 bits). Casi todas las máquinas permiten al usuario escoger entre diversas formas de *representar* un determinado número. Estas representaciones varían casi siempre en función del número de bits utilizados —longitud de palabra— y de si se hace en *formato entero* —sin decimales (*integer*)— o en *formato de coma o punto flotante* (*real*).

B.1 Sistema de numeración en un ordenador de cálculo

La representación de un número entero en un ordenador en formato entero es exacta, siempre y cuando su valor esté comprendido dentro de los límites que admite la máquina.

Por el contrario, en formato real sólo es posible representar un conjunto finito de números, F , de la recta real: $F \subset \mathbb{R}$. Este conjunto F , denominado *sistema de numeración*, está formado por elementos de la forma

$$f = \pm m \times \beta^{e-t}$$

y está caracterizado por cuatro parámetros:

β , la *base de numeración* de la máquina;

t , la *precisión* de la máquina; y

e , el *margen de exponentes*, $e_{\min} \leq e \leq e_{\max}$.

A $e_{\min}$ se le denomina *límite de underflow*, L , y a $e_{\max}$ *límite de overflow*, U . El conjunto F está compuesto por todos los números reales f de la forma

$$f = \pm \left(\frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_t}{\beta^t} \right) \times \beta^e = \pm 0, d_1 d_2 \cdots d_t \times \beta^e,$$

donde los números enteros (dígitos) $d_1, \dots, d_t$ satisfacen

$$0 \leq d_i \leq \beta - 1, \quad i = 1, \dots, t$$

y $L \leq e \leq U$. El número entero m , que se denomina *mantisa*, cumple que $0 \leq m \leq \beta^t - 1$. Al número entero e se le denomina *exponente*, a $0, d_1 \cdots d_t$ o a $d_1/\beta + \cdots + d_t/\beta^t$ *fracción*. El máximo valor de la fracción es $1 - \beta^{-t}$, el cual corresponde a $d_i = \beta - 1$, $i = 1, \dots, t$; el menor, β^{-1} , que corresponde a $d_1 = 1$, $d_2 = \cdots = d_t = 0$. Si en una máquina, para todo $f \neq 0$ de F , $d_1 \neq 0$, el sistema de numeración de coma flotante de esa máquina se dice *normalizado*, siendo en este caso $m \geq \beta^{t-1}$. Obsérvese que si $0 \neq f \in F$, entonces $m \leq |f| \leq M$, donde

$$m = \beta^{L-1}$$

y

$$M = \beta^U (1 - \beta^{-t}).$$

En la tabla B.1 se representan algunos valores típicos de los parámetros que definen el sistema de numeración de diversas máquinas. Los ordenadores personales IBM-PC siguen el estándar IEEE de la tabla.

El conjunto F no es evidentemente infinito: tiene exactamente $2(\beta - 1)\beta^{t-1}(U - L + 1) + 1$ elementos. Estos elementos no están igualmente espaciados sobre la recta real sino en potencias sucesivas de β . A modo de ejemplo, si se trabaja en una máquina en la que $\beta = 2$, $t = 3$, $L = -1$ y $U = 2$, los 33 elementos del conjunto F de esa máquina son los representados por pequeñas muescas en el eje de la figura B.1.

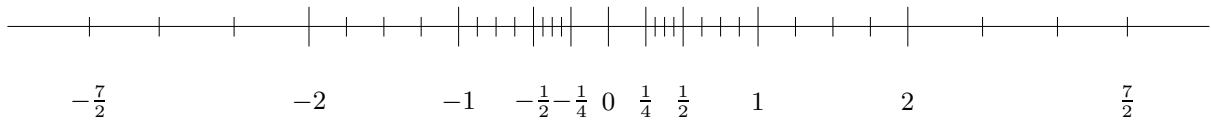


Figura B.1

Conjunto F de números reales representables en un ordenador con $\beta = 2$, $t = 3$, $L = -1$ y $U = 2$

Como el conjunto F es finito, no hay posibilidad de representar toda la recta de números reales en detalle. Por supuesto que aquellos números mayores que el máximo elemento de F y

Tabla B.1

Parámetros de la aritmética de precisión finita de diversas máquinas

Máquina	Precisión	β	t	L	U
Cray X-MP	Simple	2	48	−8192	8191
	Doble	2	96	−8192	8191
DEC-Alpha	Simple	2	24	−125	128
	Doble	2	53	−1021	1024
SIEMENS 7090	Simple	16	6	−64	63
	Doble	16	14	−64	63
IBM 3090	Simple	16	6	−64	63
	Doble	16	14	−64	63
	Extendida	16	28	−64	63
IEEE estándar 754, 1985	Simple	2	24	−125	128
	Doble	2	53	−1021	1024
HP Apollo 9000	Simple	2	24	−125	128
	Doble	2	53	−1021	1024
	Extendida	2	113	−16381	16384

menores que el mínimo no pueden representarse de ninguna manera. Tampoco lo podrán ser aquellos más pequeños que el más pequeño de F . Cada elemento o número de F representa todos los números reales de un intervalo dado alrededor de él.

Para calcular los parámetros β y t de una máquina, si no se conocen e interesan por algún motivo, se puede utilizar el código en FORTRAN 90 que se lista a continuación.

```

PROGRAM Beta_y_t
  real (kind=2) :: a=1,b=2,t=1,beta,f
  !
  do; if (f(a+1)-a/=1) exit
    a = 2*a
  end do
  !
  do; if (a+b/=a) exit
    b = 2*b
  end do
  !
  beta = (a+b)-a
  a = beta
  do; if (f(a+1)-a/=1) exit
    t = t+1
    a = a*beta
  end do
  !
  print *, "beta=", beta, ", t=", t
  !
END PROGRAM Beta_y_t

real (kind=2) FUNCTION f(a)
  real(kind=2), intent(in) :: a
  f = a
END FUNCTION f

```

Los resultados que se obtendrían al hacerlo funcionar en un ordenador personal serían los siguientes.

```
beta= 2.0000000000000000 , t= 53.0000000000000000
```

El algoritmo en el que se basa este código utiliza el hecho de que los números enteros que se pueden representar exactamente en coma flotante en una máquina son $1, 2, \dots, \beta^t$ y

$$\beta^t + \beta, \beta^t + 2\beta, \beta^t + 3\beta, \dots, \beta^{t+1}, \beta^{t+1} + \beta^2, \dots$$

En el intervalo $[\beta^t, \beta^{t+1}]$, los números de la recta real que se pueden representar están separados por una distancia igual a β . Este intervalo debe contener un número a que es una potencia de 2, $a = 2^k$. El primer bucle del código explora cuál es ese número a (o mejor dicho, la representación en coma flotante de a), probando sucesivas potencias 2^i para averiguar si 2^i y 2^{i+1} se pueden representar. El bucle que sigue añade sucesivas potencias de 2 hasta que se obtiene el siguiente número de coma flotante que se puede representar. Si a éste se le resta a se obtiene la base de numeración, β . Por último, t se determina como la potencia más pequeña de β para la cual su distancia al número en coma flotante más próximo es mayor que 1.

Debido a la optimización que llevan a cabo casi todos los compiladores que traducen el código que se le da a la máquina al lenguaje que ella entiende (denominado *ensamblador* o código de máquina), puede ocurrir que determinadas operaciones se simplifiquen produciendo resultados, en este caso, no deseados¹ a efectos teóricos. Por ejemplo, al hacer `if ((a+1)-a/=1)`, lo habitual es que el compilador lo traduzca a `if (1/=1)`. Para evitar esto es por lo que hemos introducido la `function f(a)`, que no hace otra cosa que devolver el propio argumento. También se podría haber guardado el resultado `a+1` en otra variable, aunque lo normal es que un buen compilador tampoco se dejase engañar por esta estratagema y, adelantándose un par de pasos, simplificase de igual manera. Si esto no ocurriese, el código en FORTRAN 90 que se podría emplear, por ejemplo, sería el siguiente.

```
PROGRAM Beta_y_t
  real (kind=2) :: a=1,b=2,t=1,beta,am1=2
  !
  do; if (am1-a/=1) exit
    a = 2*a
    am1 = a+1
  end do
  !
  do; if (a+b/=a) exit
    b = 2*b
  end do
  !
  beta = (a+b)-a
  a = beta
  am1 = a+1
  do; if (am1-a/=1) exit
    t = t+1
    a = a*beta
    am1 = a+1
  end do
  !
```

¹Esto no quiere decir, en ningún caso, que los compiladores falseen los resultados obtenibles, simplemente que para ahorrar pasos intermedios, si se quiere que se lleven a cabo en cualquier caso, aun a costa de la eficacia en tiempo de cálculo, hay que tener cuidado.

```

      print *, "beta=", beta, ", t=", t
!
END PROGRAM Beta_y_t

```

Utilizando el mismo compilador que el empleado para compilar el código anterior, FTN90 de NAG, se conseguiría el mismo resultado que antes.

Los parámetros del sistema de numeración en los que un determinado código se tendrá que desenvolver se pueden conocer en FORTRAN 90 muy fácilmente. Basta invocar las funciones `tiny`, `huge`, `epsilon`, `digits`, `maxexponent`, `minexponent`, `radix` y `range`.

B.2 Precisión de un ordenador. Errores de redondeo

Para disponer de un modelo de cómo se representa un número en el sistema de numeración en coma flotante de un ordenador, se define el conjunto G ,

$$G = \{x \in \mathbb{R} : m \leq |x| \leq M\} \cup \{0\}, \quad (\text{B.1})$$

y la aplicación $fl(\cdot) : G \rightarrow F$ de tal forma que

$$fl(x) = \begin{cases} \text{el elemento } c \in F \text{ más cerca de } x, \text{ si se redondea, y} \\ \text{el elemento } c \in F \text{ más cercano a } x \text{ que satisfaga } |c| \leq |x|, \text{ si se trunca.} \end{cases} \quad (\text{B.2})$$

Si un número x está situado a igual distancia de dos elementos de F y se redondea, se puede representar por cualquiera de los dos que no sea cero. Se dice que $fl(x)$ produce *overflow* si $|fl(x)| > \max\{|y| : y \in F\}$ y *underflow* si $0 < |fl(x)| < \min\{|y| : 0 \neq y \in F\}$.

El *redondeo* y *truncamiento* se pueden ilustrar en una hipotética máquina que trabajase con $\beta = 10$ y con $t = k$, de la siguiente manera:

- Truncando, el número

$$x = +0,d_1d_2d_3 \dots d_kd_{k+1}d_{k+2} \dots \times 10^n$$

perdería los dígitos $d_{k+1}, \dots$ quedando $+0,d_1d_2d_3 \dots d_k \times 10^n$.

- Redondeando, se añadiría $5 \times 10^{n-(k+1)}$ al número en cuestión y luego se truncaría.

Si en esa hipotética máquina $t = 3$, el número 0,3246 se representaría truncando como 0,324 y redondeando como 0,325. Como es obvio, si se utiliza poca precisión, el efecto del redondeo o truncamiento en la diferencia de los resultados obtenidos y esperados en procesos donde se realizan muchos cálculos puede ser importante.

El truncamiento o redondeo dan lugar a lo que se conoce comúnmente como *error de redondeo*. Afecta a todas las operaciones que se realizan en un ordenador o máquina calculadora.

Definición B.1 Si x^* es la representación del número x en un ordenador, el *error absoluto* de esa representación es $|x - x^*|$.

El error absoluto de la representación de un número x con el sistema de coma flotante de un ordenador es $|x - fl(x)|$.

Definición B.2 Si el número x^* es la representación de x , el *error relativo* de esa representación es

$$\frac{|x - x^*|}{|x|}.$$

Se puede comprobar fácilmente que si $x \in G$ y la máquina donde se representa usa redondeo, se cumple que

$$\frac{|x - fl(x)|}{|x|} \leq \frac{1}{2}\beta^{1-t}.$$

Si usa truncamiento, que

$$\frac{|x - fl(x)|}{|x|} \leq \beta^{1-t}.$$

A β^{1-t} se la denomina *precisión de la máquina* o *epsilon*, ϵ ó ϵ_M , de la máquina. El *epsilon* de una máquina es la distancia en la recta real de 1,0 al número en coma flotante más próximo. Es claro que $\epsilon_M = \beta^{1-t}$ es la distancia entre 1,0 y β en la recta real. La distancia entre 1,0 y $1/\beta$ es $\beta^{-t} = \epsilon_M/\beta$. El valor de *epsilon* define la cantidad más pequeña que añadida a uno tiene representación en la máquina: $1 + \epsilon_M > 1$. Volveremos sobre esta idea inmediatamente pues nos servirá para calcular ϵ_M si no se conoce.

A $u = \frac{1}{2}\beta^{1-t}$ se le denomina *unidad de redondeo*. Si $x \in \mathfrak{R}$ está comprendido entre el margen de valores de F , se cumple que

$$fl(x) = x(1 + \delta), \quad |\delta| < u.$$

donde la aplicación $fl(\cdot)$ es la definida en (B.2) con redondeo. En efecto, suponiendo que $x > 0$, este número se puede expresar de la siguiente manera:

$$x = \mu \times \beta^{e-t}, \quad \beta^{t-1} \leq \mu \leq \beta^t - 1.$$

Es decir, x está entre los dos números de coma flotante adyacentes

$$y_1 = \lfloor \mu \rfloor \beta^{e-t} \quad \text{y} \quad y_2 = \lceil \mu \rceil \beta^{e-t}.$$

En consecuencia, $fl(x) = y_1$ ó y_2 cumpliéndose que

$$|fl(x) - x| \leq \frac{|y_2 - y_1|}{2} \leq \frac{\beta^{e-t}}{2}.$$

De aquí que

$$\left| \frac{fl(x) - x}{x} \right| \leq \frac{\frac{1}{2}\beta^{e-t}}{\mu \times \beta^{e-t}} \leq \frac{1}{2}\beta^{1-t} = u.$$

Esta última desigualdad se cumple estrictamente a no ser que $\mu = \beta^{t-1}$ en cuyo caso $x = fl(x)$, cumpliéndose entonces que $|\delta| = 0$.

De las consideraciones anteriores se deduce que cuando se va a trabajar con una máquina o codificar para ella un programa, es muy conveniente conocer su precisión o *epsilon*. Aunque hoy en día muchos ordenadores y compiladores la facilitan sin más que invocarla mediante

la correspondiente función implícita, es interesante poder calcularla rápidamente. El siguiente código en FORTRAN 90 permite hacerlo cuando se trabaje en precisión simple o en doble. La función implícita `epsilon()` proporciona el valor que se busca sin necesidad de calcularlo. El código también se utiliza para conocer los valores en precisión doble de `radix()`, `digits()`, `huge()`, `maxexponent()`, `minexponent()`, `precision()`, `range()` y `tiny()` en la máquina en que se esté trabajando.

```
PROGRAM Epsmaq
  real      :: eps=1, xp1
  real (kind=2) :: epsd=1, xp1d
  !
  xp1 = eps+1
  do; if (xp1==1) exit
    eps = eps/2; xp1 = eps+1
  enddo
  !
  xp1d = epsd+1
  do; if (xp1d==1) exit
    epsd = epsd/2; xp1d = epsd+1
  enddo
  !
  print '(2(a,1pe16.9)/2(a,1pe16.9))', "eps", 2*eps, ", epsd", 2*epsd,      &
        "eps de máquina", epsilon(eps), ", epsd de máquina", epsilon(epsd)
  !
  print '(/2(a,i4), a, 1pe16.9/4(a,i6)/a, 1pe16.9)',      &
        "radix", radix(epsd), ", digits", digits(epsd), ", huge", huge(epsd), &
        "maxexponent", maxexponent(epsd), ", minexponent", minexponent(epsd), &
        ", precision", precision(epsd), ", range", range(epsd),      &
        "tiny", tiny(epsd)
  !
END PROGRAM Epsmaq
```

Los resultados que este pequeño código proporciona son los que siguen.

```
eps 1.192092896E-07, epsd 2.220446049E-16
eps de máquina 1.192092896E-07, epsd de máquina 2.220446049E-16

radix 2, digits 53, huge 1.797693135+308
maxexponent 1024, minexponent -1021, precision 15, range 307
tiny 2.225073859-308
```

Definición B.3 Se dice que en un ordenador o máquina calculadora x^* aproxima o *representa* un número x con t dígitos significativos si t es el entero más grande no negativo para el cual

$$\frac{|x - x^*|}{|x|} < 0,5 \times 10^{1-t}.$$

El número 0,0123 aproxima a 0,012345 con 3 dígitos significativos pues

$$\frac{|0,012345 - 0,0123|}{|0,012345|} = 0,0036452 < 0,5 \times 10^{-2} = 0,005.$$

De acuerdo con esto, todos los dígitos de la representación de un número en un ordenador comenzando por el primero distinto de cero por la izquierda se denominan dígitos significativos.

Si el sistema de numeración que usa la máquina está normalizado, todos los dígitos de la mantisa serán significativos.

B.3 Aritmética en un ordenador

El sistema de numeración, F , de un ordenador tiene definidas entre sus elementos operaciones aritméticas. Éstas, dada la finitud del conjunto F , son operaciones aritméticas de precisión finita. Si $\square$ designa cualquiera de las cuatro operaciones aritméticas elementales $+$, $-$, $\times$ y $\div$, y $|a\square b| \in G$, donde este conjunto G es el definido en (B.1), el resultado de cualquier cómputo o cálculo en ordenador con ellas dará como resultado $fl(a\square b)$. Cualquier operación aritmética en ordenador cumple que $fl(a\square b) = (a\square b)(1 + \delta)$, donde $|\delta| < u$. Si las operaciones aritméticas en una máquina se representan por $\oplus$, $\ominus$, $\otimes$ y $\oslash$, el resultado de aplicarlas es:

$$\begin{aligned} a \oplus b &= fl(fl(a) + fl(b)); \\ a \ominus b &= fl(fl(a) - fl(b)); \\ a \otimes b &= fl(fl(a) \times fl(b)) \quad \text{y} \\ a \oslash b &= fl(fl(a) \div fl(b)). \end{aligned}$$

El error relativo de cualquiera de estas operaciones es

$$\frac{|a\square b - fl(a\square b)|}{|a\square b|} < u.$$

Ejemplo B.1 En una máquina $\beta = 10$, $t = 3$ y la representación se hace mediante truncamiento. La operación aritmética $(10^{-3} + 1) - 1$ dará como resultado

$$fl[fl(10^{-3} + 1) - 1] = 0.$$

El error relativo es 1. Por el contrario,

$$fl[10^{-3} + fl(1 - 1)] = 10^{-3},$$

resultado exacto. La aritmética de coma flotante, por consiguiente, no siempre es asociativa. ●

Ejemplo B.2 Supongamos que se quiere operar con los números $x = \frac{1}{3}$ e $y = \frac{5}{7}$ en una máquina con $\beta = 10$ y $t = 5$ que usa truncamiento. En la siguiente tabla se observan los resultados obtenibles y los errores de los mismos.

Operación	Resultado	Valor Real	Error Absoluto	Error Relativo
$x \oplus y$	$0,10476 \times 10^1$	$22/21$	$0,190 \times 10^{-4}$	$0,182 \times 10^{-4}$
$y \ominus x$	$0,38095 \times 10^0$	$8/21$	$0,238 \times 10^{-5}$	$0,625 \times 10^{-5}$
$x \otimes y$	$0,23809 \times 10^0$	$5/21$	$0,524 \times 10^{-5}$	$0,220 \times 10^{-4}$
$y \oslash x$	$0,21428 \times 10^1$	$15/7$	$0,571 \times 10^{-4}$	$0,267 \times 10^{-4}$

Como el error máximo relativo es $0,267 \times 10^{-4}$, la aritmética de ese ordenador para estos cálculos produce errores satisfactorios. Supóngase, sin embargo, que se tiene $u = 0,714251$, $v = 98765,9$ y $w = 0,111111 \times 10^{-4}$. La representación en la máquina de estos números será

$fl(u) = 0,71425 \times 10^0$, $fl(v) = 0,98765 \times 10^5$ y $fl(w) = 0,11111 \times 10^{-4}$. En la tabla siguiente se puede comprobar como $y \ominus u$ produce un error absoluto pequeño pero un error relativo bastante grande.

Operación	Resultado	Valor Real	Error Absoluto	Error Relativo
$y \ominus u$	$0,30000 \times 10^{-4}$	$0,34714 \times 10^{-4}$	$0,471 \times 10^{-5}$	0,136
$(y \ominus u) \oslash w$	$0,27000 \times 10^1$	$0,31243 \times 10^1$	0,424	0,136
$(y \ominus u) \otimes v$	$0,29629 \times 10^1$	$0,34285 \times 10^1$	0,465	0,136
$u \oplus v$	$0,98765 \times 10^5$	$0,98766 \times 10^5$	$0,161 \times 10^1$	$0,163 \times 10^{-4}$

La división posterior por w y multiplicación por v amplifica el error absoluto sin modificar el error relativo. La adición del número grande u y el pequeño v produce un error absoluto grande pero no un gran error relativo. ●

Este ejemplo sirve de introducción para considerar otro aspecto de la aritmética de precisión finita de extraordinaria importancia: el fenómeno conocido como *cancelación catastrófica* o simplemente *error numérico de cancelación*. Se refiere a la pérdida extrema de dígitos significativos que conllevan ciertas operaciones entre números similares entre sí: concretamente al restarlos. En efecto, para llevar a cabo sumas y multiplicaciones, los números en un ordenador son primeramente convertidos a un formato de mantisa común, desplazando el primer dígito significativo de una de ellas a la posición equivalente del otro. De esta forma, si ese desplazamiento supera los dígitos significativos que la máquina permite, se puede perder toda la información del número desplazado. Por ejemplo, considérese la función $f(x) = (1 - \cos x)/x^2$. Si $x = 1,2 \times 10^{-5}$ el valor de $\cos x$, redondeado a diez² dígitos significativos, es

$$c = 0,9999999999$$

por lo que

$$1 - c = 0,0000000001.$$

El valor de la función es entonces

$$\frac{1 - c}{x^2} = \frac{10^{-10}}{1,44 \times 10^{-10}} = 0,6944 \dots$$

el cual es erróneo pues $0 \leq f(x) < 1/2$ para todo $x \neq 0$. El problema de este ejemplo estriba en que $1 - c$ sólo posee un dígito significativo. La resta $1 - c$ es exacta pero la operación que conlleva produce un resultado de la misma magnitud que el error en c . Es decir, la resta amplifica mucho la importancia del error que se obtiene al calcular c . Para evitar este problema, como $\cos x = 1 - 2 \sin^2(x/2)$, es fácil ver que

$$f(x) = \frac{1}{2} \left(\frac{\sin(x/2)}{x/2} \right)^2.$$

Calculando $f(x)$ mediante esta fórmula y aproximando $\sin(x/2)$ con 10 dígitos significativos, el resultado es $f(x) = 0,5$, el cual se aproxima al valor exacto en 10 dígitos.

²El valor con quince dígitos es 0,99999999992800.

El ejemplo más típico de error de cancelación lo constituye el cálculo de e^{-x} . Si recordamos, la expresión del desarrollo en serie de e^x es:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Si se desea calcular $e^{-5,5}$ mediante desarrollo en serie en una máquina con $\beta = 10$ y $t = 5$, los resultados que se obtienen son los siguientes:

$$\begin{aligned} e^{-5,5} &= +1,0000 - 5,5000 + 15,125 - 27,730 + 38,129 \\ &\quad - 41,942 + 38,446 - 30,208 + 20,768 - 12,692 + 6,9803 - 3,4902 + 1,5997 - \dots \\ &= +0,0026363. \end{aligned}$$

La suma se termina después de 25 términos pues los siguientes no aportan dígitos significativos al resultado. El valor real de $e^{-5,5}$ es 0,00408677. Como se puede comprobar, la diferencia es muy importante: tanto que los dos resultados apenas se parecen.

El problema de esta forma de proceder surge de sumar y restar valores relativamente distintos entre sí —uno o varios órdenes de magnitud— con la consiguiente pérdida de dígitos significativos. La solución para este problema en este caso consistiría en obtener el desarrollo en serie de $e^{5,5}$ y luego calcular el recíproco. La respuesta que se obtendría sería:

$$e^{-5,5} = \frac{1}{e^{5,5}} = \frac{1}{1 + 5,5 + 15,125 + \dots} = 0,0040865.$$

Muchos algoritmos tienen en cuenta que se puedan presentar los problemas expuestos y tratar de evitarlos siempre que sea posible.

Los errores de cancelación no tienen por qué ser necesariamente malos en sí mismos. En efecto, pueden ser un buen síntoma para detectar que el problema que los refleja está intrínsecamente mal condicionado y alertar de ello, u ocurrir que de su resultado no dependa significativamente el cálculo general. Un ejemplo de esta última circunstancia se presenta cuando $x \gg y \approx z > 0$. Evidentemente la cancelación que se pueda producir al calcular $x + (y - z)$ no tiene relevancia.

Para profundizar un poco más en el fenómeno del error de cancelación, consideremos la resta $\hat{x} = \hat{a} - \hat{b}$, donde $\hat{a} = a(1 + \Delta a)$ y $\hat{b} = b(1 + \Delta b)$. Las cantidades Δa y Δb designan los errores relativos ya presentes en los datos de esta operación debidos, por ejemplo, a cálculos previos. Si $x = a - b$, el error relativo de x es

$$\frac{|x - \hat{x}|}{|x|} = \frac{|-a\Delta a - b\Delta b|}{|a - b|} \leq \max(|\Delta a|, |\Delta b|) \frac{|a| + |b|}{|a - b|}.$$

El error relativo de x es grande cuando $|a - b| \ll |a| + |b|$, es decir, cuando se produce una cancelación importante en la resta. Esto atestigua el que una operación de resta en la que la cancelación es importante no hace sino magnificar los errores o imprecisiones ya existentes en los datos; en este caso en $\hat{a}$ y $\hat{b}$.

B.3.1 Solución de una ecuación cuadrática

Desde el punto de vista matemático, resolver la ecuación cuadrática $ax^2 + bx + c = 0$ es un problema trivial. Desde el punto de vista numérico con precisión finita, no tanto, presentándose algunas peculiaridades dignas de tenerse en cuenta.

Como es sabido, si a , b y c son números reales y $a \neq 0$, las dos soluciones, x_1 y x_2 , están dadas por las expresiones:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad (\text{B.3})$$

y

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}. \quad (\text{B.4})$$

Veamos lo que ocurre al aplicar directamente estas fórmulas en una máquina en la que $\beta = 10$, $t = 8$, $L = -50$ y $U = 50$ (bastante más precisión de lo que es habitual).

Si $a = 1$, $b = -10^5$ y $c = 1$, las raíces auténticas de la ecuación son

$$x_1 = 99999,999990$$

y

$$x_2 = 0,0000100000000001.$$

Si se usan las expresiones (B.3) y (B.4) en la máquina apuntada, el resultado es

$$x_1 = 100000,00$$

y

$$x_2 = 0,0.$$

Al calcular x_1 y x_2 ha habido un error de cancelación catastrófico, pues

$$fl((-10^5)^2 - 4) = 10^{10}.$$

La forma de evitar este error es bien conocida. Consiste en usar el signo de b para determinar cuál de las expresiones (B.3) o (B.4) produciría menor error de cancelación, y usarla para calcular la raíz de mayor valor absoluto a partir de la fórmula

$$x_1 = \frac{-\left(b + \text{signo}(b)\sqrt{b^2 - 4ac}\right)}{2a}.$$

Como $ax^2 + bx + c = a(x - x_1)(x - x_2)$, lo que implica que $ax_1x_2 = c$, la otra raíz se podría calcular usando la expresión

$$x_2 = \frac{c}{ax_1}.$$

Aplicando este criterio en el ejemplo apuntado se obtendrían las raíces $x_1 = 100000,00$ y $x_2 = 0,00001$: ambas buenas soluciones.

B.3.2 Más errores. Una suma de infinitos sumandos

Como es sabido

$$\sum_{k=1}^{\infty} k^{-2} = \frac{\pi^2}{6} = 1,644934066848 \dots$$

Supongamos que no se conoce el resultado y que se desea realizar la operación con un ordenador programándola adecuadamente y aproximándola lo más posible a su resultado exacto.

Lo más lógico parece codificar un bucle, en el que k haga de índice, mediante el cual se sumen los términos correspondientes hasta que la suma no varíe, pues no se pueden aportar más dígitos significativos de acuerdo con el número de éstos que proporcione ese ordenador. En FORTRAN, en un ordenador personal, utilizando precisión simple, el valor que se obtiene a partir de $k = 4096$ es 1,64472532. Este resultado sólo se aproxima al exacto en cuatro dígitos significativos cuando lo esperable, de acuerdo con el sistema de numeración de la máquina, serían 6 dígitos.

La explicación del porqué ocurre esto radica en la forma en que se efectúa la suma: primero los números mayores y después los más pequeños. Éstos últimos difícilmente contribuyen al resultado significativamente. En efecto, para $k = 4096$, se lleva a cabo la siguiente operación: $s + 4096^{-2}$. El término $4096^{-2} = 2^{-24}$ y $s \approx 1,6$. Como ya hemos visto, la precisión simple utiliza una mantisa de 24 bits por lo que 4096^{-2} , frente a $s \approx 1,6$, se sale de la ventana de valores que admite la palabra de ordenador que ha de representar el resultado. Igual ocurriría con los términos sucesivos.

La forma de remediar esto es sumar primero los términos más pequeños y luego los grandes. Proceder así, sin embargo, requiere saber cuántos términos se habrán de utilizar para aproximar adecuadamente el resultado antes de comenzar los cálculos. Si se utilizan 10^9 , el valor que se obtiene con el código en FORTRAN 90 que sigue es 1,6449340658482266, es decir, el exacto con 9 dígitos significativos.

```
PROGRAM Suma_de_serie
!
!   real (kind=2) :: suma=0,uno=1
!
!   do i=1000000000,1,-1
!       suma=suma+uno/(dble(i)*i)
!   end do
!
!   print *,suma
!
END PROGRAM Suma_de_serie
```

Referencias

Como resumen de este apéndice en una sola idea, cabe insistir en la importancia que tiene, a la hora de diseñar o codificar algoritmos en ordenador, tener en cuenta que se estará trabajando en un entorno numérico finito. Habrá que tener mucho cuidado con las singularidades que puedan presentar los problemas a resolver con un determinado algoritmo, los criterios con los que se decida cuándo se ha de parar un proceso iterativo que alcance la solución por aproximaciones sucesivas, la precisión máxima obtenible, cuándo se ha de considerar que un número es cero, etc.

Existen excelentes referencias de cálculo numérico y teoría de errores en entornos numéricos finitos donde se pueden encontrar mucho más ampliados los estudios sobre representación y operaciones con números en ordenadores, y los errores que esto puede acarrear. Para la elaboración de este anexo se ha seguido Golub y Van Loan [1989], Forsythe, Malcolm y Moler [1977], Stoer y Bulirsch [1980] y, fundamentalmente, Higham [1996].

Apéndice C

REDES ELÉCTRICAS: FLUJOS POR SUS ELEMENTOS Y POTENCIAS INYECTADAS EN SUS NUDOS

PARTIENDO DE UN ESQUEMA GENERALIZADO¹ de un elemento (una línea o un transformador) de una red eléctrica de transporte de energía o transmisión de potencia, en este apéndice se desarrollan, en función de las tensiones en los nudos, los argumentos de estas tensiones y posiciones de los reguladores² en los transformadores, las ecuaciones que relacionan la potencia inyectada en cada nudo de la red y los flujos de potencia en los elementos de la misma. Estas ecuaciones³ son utilizadas repetidas veces en el texto.

C.1 Línea

Analizaremos primero el caso más sencillo: aquel en que no están presentes transformadores con regulador variable. El elemento de transporte, línea, que une dos nudos i y j se representa

¹Denominado esquema en II.

²En inglés denominados *taps*.

³Para profundizar en el estudio de estas cuestiones aconsejamos al lector consultar las referencias indicadas al final del apéndice.

según el esquema II de la figura C.1.

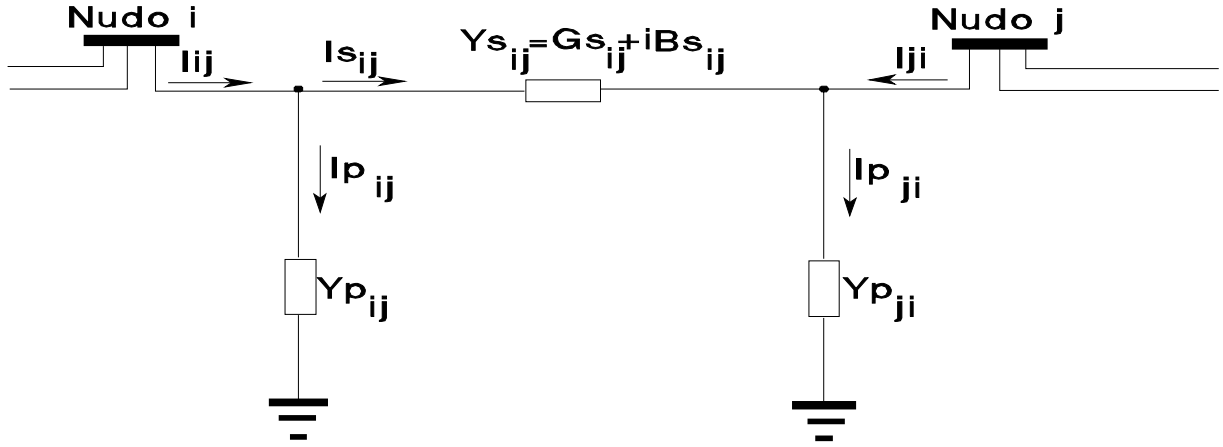


Figura C.1

Esquema en II de una línea entre dos nudos i y j

C.1.1 Potencias inyectadas en los nudos

Si designamos por P_i y Q_i las potencias activa y reactiva inyectadas —balance de potencia generada menos potencia absorbida o demandada— en el nudo i , se tiene que

$$P_i + iQ_i = V_i I_i^*,$$

donde I_i^* es la conjugada de la intensidad —compleja— que sale del nudo i y V_i la tensión —también compleja— en el nudo i .

La intensidad que sale de un nudo i es la suma de todas las que circulan por los elementos que parten de dicho nudo; esto es:

$$I_i^* = \sum_{j=1}^n I_{ij}^*,$$

donde n es el número de nudos de la red. El valor de estas I_{ij}^* será cero, obviamente, entre aquellos nudos donde no haya conexión directa. La expresión que liga las intensidades que circulan entre los nudos i y j (ver el esquema que describe la figura C.1) con las tensiones en ambos nudos es:

$$\begin{aligned} I_{ij}^* &= Y_{ij}^* (V_i^* - V_j^*) = (V_i^* - V_j^*) Y_{sij}^* + V_i^* Y_{p_{ij}}^* \\ &= V_i^* (Y_{p_{ij}}^* + Y_{sij}^*) + V_j^* (-Y_{sij}^*). \end{aligned}$$

A la expresión $-Y_{sij}^* = Y_{ij}^*$ se le denomina admitancia mutua de los nudos i y j ; a $Y_{p_{ij}}^* + Y_{sij}^* = Y_{ii}^*$, admitancia propia del nudo i (con respecto claro está a la línea entre i y j).

Con estas expresiones, las potencias inyectadas en los nudos saldrán de estas otras:

$$P_i = \text{Real} \left\{ V_i \sum_{j=1}^n Y_{ij}^* V_j^* \right\}$$

$$Q_i = \text{Ima.} \left\{ V_i \sum_{j=1}^n Y_{ij}^* V_j^* \right\}.$$

Las tensiones y las admitancias —magnitudes complejas— se expresan de la siguiente forma:

$$V_i = |V_i| (\cos \theta_i + i \sen \theta_i)$$

$$V_j = |V_j| (\cos \theta_j + i \sen \theta_j)$$

$$Y_{ij} = |Y_{ij}| (\cos \delta_{ij} + i \sen \delta_{ij}).$$

Las potencias inyectadas serán, por consiguiente:

$$P_i = \text{Real} \left\{ |V_i| (\cos \theta_i + i \sen \theta_i) \sum_{j=1}^n |V_j| |Y_{ij}| (\cos \theta_j - i \sen \theta_j) (\cos \delta_{ij} - i \sen \delta_{ij}) \right\}$$

$$Q_i = \text{Ima.} \left\{ |V_i| (\cos \theta_i + i \sen \theta_i) \sum_{j=1}^n |V_j| |Y_{ij}| (\cos \theta_j - i \sen \theta_j) (\cos \delta_{ij} - i \sen \delta_{ij}) \right\}.$$

Desarrollando la expresión entre llaves se llega a:

$$|V_i| (\cos \theta_i + i \sen \theta_i) \sum_{j=1}^n |V_j| |Y_{ij}| (\cos \theta_j \cos \delta_{ij} - \sen \theta_j \sen \delta_{ij} - i (\sen \theta_j \cos \delta_{ij} + \sen \delta_{ij} \cos \theta_j)).$$

Recordando que

$$\begin{aligned} \cos(\alpha + \beta) &= \cos \alpha \cos \beta - \sen \alpha \sen \beta \\ \cos(\alpha - \beta) &= \cos \alpha \cos \beta + \sen \alpha \sen \beta \\ \sen(\alpha + \beta) &= \sen \alpha \cos \beta + \sen \beta \cos \alpha \\ \sen(\alpha - \beta) &= \sen \alpha \cos \beta - \sen \beta \cos \alpha, \end{aligned}$$

se puede simplificar la expresión anterior, obteniendo

$$|V_i| (\cos \theta_i + i \sen \theta_i) \sum_{j=1}^n |V_j| |Y_{ij}| (\cos(\theta_j + \delta_{ij}) - i \sen(\theta_j + \delta_{ij})).$$

Si se introduce el término $(\cos \theta_i + i \sen \theta_i)$ dentro del sumatorio, se obtiene

$$\begin{aligned} |V_i| \sum_{j=1}^n |V_j| |Y_{ij}| (\cos(\theta_j + \delta_{ij}) \cos \theta_i + \sen(\theta_j + \delta_{ij}) \sen \theta_i \\ + i (\cos(\theta_j + \delta_{ij}) \sen \theta_i - \sen(\theta_j + \delta_{ij}) \cos \theta_i)). \end{aligned}$$

Reagrupando términos en esta última expresión se llega a

$$|V_i| \sum_{j=1}^n |V_j| |Y_{ij}| (\cos(\theta_i - \theta_j - \delta_{ij}) + i \sin(\theta_i - \theta_j - \delta_{ij})).$$

En resumen, las potencias activas y reactivas inyectadas en el nudo i son:

$$\begin{aligned} P_i &= |V_i| \sum_{j=1}^n |V_j| |Y_{ij}| \cos(\theta_i - \theta_j - \delta_{ij}) \\ Q_i &= |V_i| \sum_{j=1}^n |V_j| |Y_{ij}| \sin(\theta_i - \theta_j - \delta_{ij}). \end{aligned}$$

Como en el caso que nos ocupa $Y_{ii} = Y_{p_{ij}} + Y_{s_{ij}} = G_{p_{ij}} + iB_{p_{ij}} + G_{s_{ij}} + iB_{s_{ij}}$ e $Y_{ij} = -Y_{s_{ij}} = -G_{s_{ij}} - iB_{s_{ij}}$, también se pueden expresar como sigue.

$$\begin{aligned} P_i &= |V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n (G_{p_{ij}} + G_{s_{ij}}) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| [G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j)] \\ Q_i &= -|V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n (B_{p_{ij}} + B_{s_{ij}}) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| [G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j)]. \end{aligned}$$

Si el nudo tiene algún condensador o reactancia a él conectado, $B_{p_{ij}}$ debería englobar la del condensador/reactancia y las de tierra de las líneas conectadas a ese nudo.

C.1.2 Flujos de potencia entre los nudos

La potencia aparente que circula de un nudo i a otro j es

$$S_{ij} = P_{ij} + iQ_{ij} = V_i I_{ij}^*. \quad (\text{C.1})$$

La intensidad I_{ij}^* , medida en el nudo i y definida como positiva en la dirección $i \rightarrow j$, es

$$I_{ij}^* = \underbrace{(V_i^* - V_j^*) Y_{s_{ij}}^*}_{I_{s_{ij}}^*} + \underbrace{V_i^* Y_{p_{ij}}^*}_{I_{p_{ij}}^*}$$

por lo que la ecuación (C.1) queda

$$S_{ij} = V_i (V_i^* - V_j^*) Y_{s_{ij}}^* + |V_i|^2 Y_{p_{ij}}^*. \quad (\text{C.2})$$

Los complejos $V_i, V_j, Y_{s_{ij}}$ e $Y_{p_{ij}}$ son:

$$\begin{aligned} V_i &= |V_i| (\cos \theta_i + i \sen \theta_i); \\ V_j &= |V_j| (\cos \theta_j + i \sen \theta_j); \\ Y_{s_{ij}} &= |Y_{s_{ij}}| (\cos \delta_{s_{ij}} + i \sen \delta_{s_{ij}}); \\ Y_{p_{ij}} &= |Y_{p_{ij}}| (\cos \delta_{p_{ij}} + i \sen \delta_{p_{ij}}). \end{aligned}$$

Sustituyendo estas expresiones en la ecuación (C.2):

$$\begin{aligned} S_{ij} &= |V_i| (\cos \theta_i + i \sen \theta_i) \left(|V_i| (\cos \theta_i - i \sen \theta_i) - |V_j| (\cos \theta_j - i \sen \theta_j) \right) \\ &\quad \cdot |Y_{s_{ij}}| (\cos \delta_{s_{ij}} - i \sen \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| (\cos \delta_{p_{ij}} - i \sen \delta_{p_{ij}}) \\ &= |V_i| (\cos \theta_i + i \sen \theta_i) \left(|V_i| \cos \theta_i - i |V_i| \sen \theta_i - |V_j| \cos \theta_j + i |V_j| \sen \theta_j \right) \\ &\quad \cdot |Y_{s_{ij}}| (\cos \delta_{s_{ij}} - i \sen \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| \cos \delta_{p_{ij}} - i |V_i|^2 |Y_{p_{ij}}| \sen \delta_{p_{ij}} \\ &= \left(|V_i|^2 \cos \theta_i \cos \theta_i + i |V_i|^2 \sen \theta_i \cos \theta_i - i |V_i|^2 \cos \theta_i \sen \theta_i + |V_i|^2 \sen \theta_i \sen \theta_i \right. \\ &\quad \left. - |V_i| |V_j| \cos \theta_i \cos \theta_j - i |V_i| |V_j| \sen \theta_i \cos \theta_j + i |V_i| |V_j| \cos \theta_i \sen \theta_j \right. \\ &\quad \left. - |V_i| |V_j| \sen \theta_i \sen \theta_j \right) |Y_{s_{ij}}| (\cos \delta_{s_{ij}} - i \sen \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| \cos \delta_{p_{ij}} \\ &\quad - i |V_i|^2 |Y_{p_{ij}}| \sen \delta_{p_{ij}} \\ &= \left(|V_i|^2 - |V_i| |V_j| \cos(\theta_i - \theta_j) - i |V_i| |V_j| \sen(\theta_i - \theta_j) \right) |Y_{s_{ij}}| (\cos \delta_{s_{ij}} - i \sen \delta_{s_{ij}}) \\ &\quad + |V_i|^2 |Y_{p_{ij}}| \cos \delta_{p_{ij}} - i |V_i|^2 |Y_{p_{ij}}| \sen \delta_{p_{ij}} \\ &= |V_i|^2 |Y_{s_{ij}}| (\cos \delta_{s_{ij}} - i \sen \delta_{s_{ij}}) - |V_i| |V_j| |Y_{s_{ij}}| \cos(\theta_i - \theta_j - \delta_{s_{ij}}) \\ &\quad - i |V_i| |V_j| |Y_{s_{ij}}| \sen(\theta_i - \theta_j - \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| \cos \delta_{p_{ij}} - i |V_i|^2 |Y_{p_{ij}}| \sen \delta_{p_{ij}} \\ &= |V_i|^2 |Y_{s_{ij}}| \cos \delta_{s_{ij}} - |V_i| |V_j| |Y_{s_{ij}}| \cos(\theta_i - \theta_j - \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| \cos \delta_{p_{ij}} \\ &\quad - i \left(|V_i|^2 |Y_{s_{ij}}| \sen \delta_{s_{ij}} + |V_i| |V_j| |Y_{s_{ij}}| \sen(\theta_i - \theta_j - \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| \sen \delta_{p_{ij}} \right). \end{aligned}$$

La parte real de esta última expresión es la potencia activa, la imaginaria la reactiva. En resumen, se tienen las expresiones que siguen.

$$\begin{aligned} P_{ij} &= |V_i|^2 |Y_{s_{ij}}| \cos \delta_{s_{ij}} - |V_i| |V_j| |Y_{s_{ij}}| \cos(\theta_i - \theta_j - \delta_{s_{ij}}) + |V_i|^2 |Y_{p_{ij}}| \cos \delta_{p_{ij}} \\ Q_{ij} &= -|V_i|^2 |Y_{s_{ij}}| \sen \delta_{s_{ij}} - |V_i| |V_j| |Y_{s_{ij}}| \sen(\theta_i - \theta_j - \delta_{s_{ij}}) - |V_i|^2 |Y_{p_{ij}}| \sen \delta_{p_{ij}}. \end{aligned}$$

Estas últimas ecuaciones, teniendo en cuenta que $Y_{s_{ij}} = G_{s_{ij}} + iB_{s_{ij}}$ e $Y_{p_{ij}} = G_{p_{ij}} + iB_{p_{ij}}$, se

pueden expresar de la forma que sigue.

$$\begin{aligned} P_{ij} &= |V_i|^2 G_{s_{ij}} - |V_i||V_j| G_{s_{ij}} \cos(\theta_i - \theta_j) - |V_i||V_j| B_{s_{ij}} \sin(\theta_i - \theta_j) + |V_i|^2 G_{p_{ij}} \\ Q_{ij} &= -|V_i|^2 B_{s_{ij}} - |V_i||V_j| G_{s_{ij}} \sin(\theta_i - \theta_j) + |V_i||V_j| B_{s_{ij}} \cos(\theta_i - \theta_j) - |V_i|^2 B_{p_{ij}}. \end{aligned}$$

Ecuaciones similares se pueden obtener para los flujos P_{ji} y Q_{ji} sin más que sustituir en estas últimas las i por j y las j por i .

C.2 Transformador

Consideremos ahora el caso en que exista un transformador entre el nudo i y el nudo j según se representa en la figura C.2.

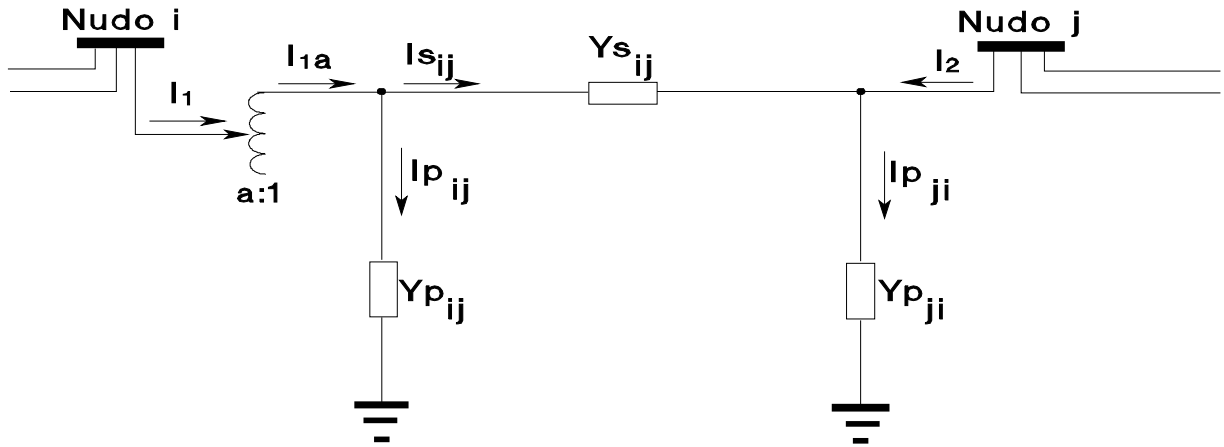


Figura C.2

Transformador entre los nudos i y j

El modelo de transformador que se elige es aquel que la literatura especializada denomina “ $a:1$ ”. Es decir, se considera que la tensión del nudo j es la que tiene como referencia la unidad. El nudo i , por consiguiente, tendrá una tensión aV_j .

Veamos a continuación cómo se puede tener en cuenta la presencia del regulador en un nuevo esquema equivalente en Π para poder seguir utilizando las ideas hasta ahora expuestas.

C.2.1 Esquema equivalente con el regulador del transformador en el primario

El esquema que se considera en este caso es el de la figura C.2. La intensidad que sale del nudo i , una vez *atravesado* el regulador, $I_{ij}a$, es:

$$I_{ij}a = \frac{S_{ij}^*}{\left(\frac{V_i}{a}\right)^*} = \left(\frac{V_i}{a} - V_j\right) Y_{s_{ij}} + \frac{V_i}{a} Y_{p_{ij}}.$$

La que sale del nudo j :

$$I_{ji} = \frac{S_{ji}^*}{V_j^*} = \left(V_j - \frac{V_i}{a} \right) Y_{sij} + V_j Y_{pij}.$$

Reescribiendo estas dos ecuaciones agrupando términos se obtiene que

$$I_{ij} = \left(\frac{Y_{sij}}{a^2} + \frac{Y_{pij}}{a^2} \right) V_i + \left(-\frac{Y_{sij}}{a} \right) V_j \quad (C.3)$$

$$I_{ji} = \left(-\frac{Y_{sij}}{a} \right) V_i + \left(Y_{sij} + Y_{pij} \right) V_j. \quad (C.4)$$

En el caso de una línea hubiésemos llegado a las siguientes expresiones:

$$\begin{aligned} I_{ij} &= (Y_{sij} + Y_{pij}) V_i + (-Y_{sij}) V_j \\ I_{ji} &= (-Y_{sij}) V_i + (Y_{sij} + Y_{pij}) V_j. \end{aligned}$$

Escritas en forma matricial:

$$I = |Y_{nudos}| V.$$

La matriz $|Y_{nudos}|$ es la denominada matriz de admitancias de nudos: define las admitancias propias y mutuas de los nudos de la red.

Volviendo al caso de transformador, éste, para su estudio como un elemento más de transporte, se puede aproximar por un esquema en Π (según se describe en la figura C.3) convencional sin regulador haciendo la admitancia en serie Y_1 igual a Y_{sij}/a , la admitancia-shunt Y_2 igual a

$$\frac{Y_{sij}(1-a)}{a^2} + \frac{Y_{pij}}{a^2}$$

y la Y_3 a

$$\frac{Y_{sij}(a-1)}{a} + Y_{pij}.$$

De esta forma conseguimos que las admitancias resultantes propias de los nudos i y j y la mutua ij verifiquen las ecuaciones (C.3) y (C.4).

C.2.2 Esquema equivalente con el regulador del transformador en el secundario

El esquema que se considera en este caso es el de la figura C.4. Siguiendo un razonamiento similar al del caso en que el regulador estaba en el primario se deduce que la intensidad que sale del nudo i es:

$$I_{ij} = \frac{S_{ij}^*}{V_i^*} = V_i Y_{pij} + (V_i - aV_j) Y_{sij};$$

y la que sale del nudo j :

$$\frac{I_{ji}}{a} = \frac{S_{ji}^*}{(aV_j)^*} = aV_j Y_{pij} + (aV_j - V_i) Y_{sij}.$$

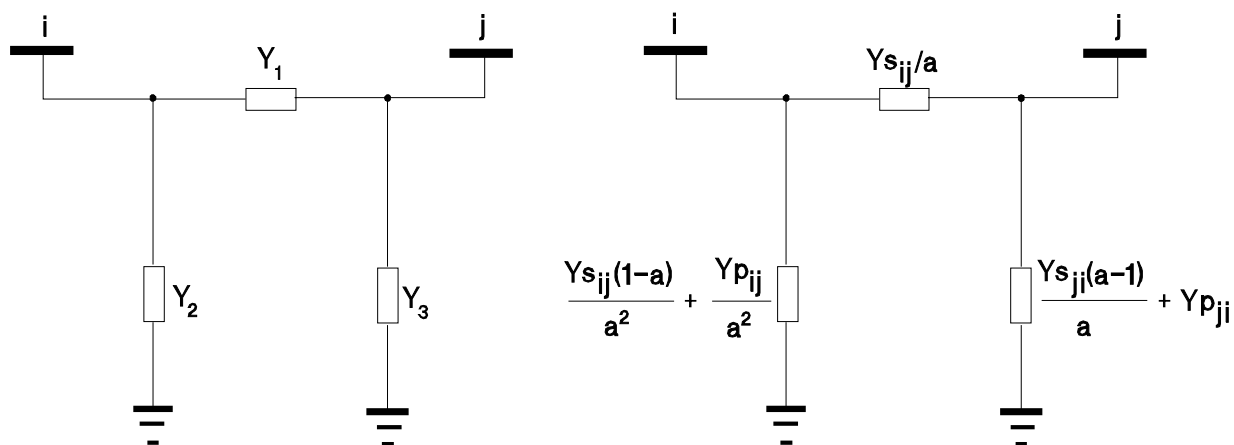


Figura C.3

Esquema en Π del transformador entre i y j con el regulador conectado a i

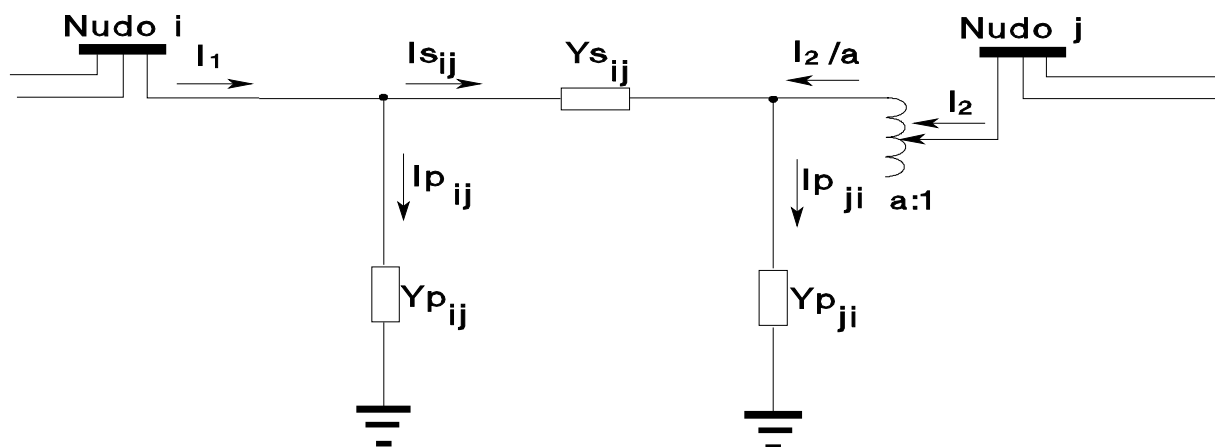


Figura C.4

Transformador entre i y j

Reagrupando términos

$$I_{ij} = (Y_{s_{ij}} + Y_{p_{ij}}) V_i + (-aY_{s_{ij}}) V_j \quad (C.5)$$

$$I_{ji} = (-aY_{s_{ij}}) V_i + a^2 (Y_{s_{ij}} + Y_{p_{ij}}) V_j. \quad (C.6)$$

Igual que en el caso en que el regulador estaba en el primario, el nuevo esquema en Π (ver figura C.5) daría $Y_1 = aY_{s_{ij}}$; la admitancia-shunt Y_2 sería

$$(1 - a)Y_{s_{ij}} + Y_{p_{ij}}$$

y la Y_3

$$a(a - 1)Y_{s_{ij}} + a^2 Y_{p_{ij}}.$$

Estas nuevas admitancias cumplen las ecuaciones (C.5) y (C.6).

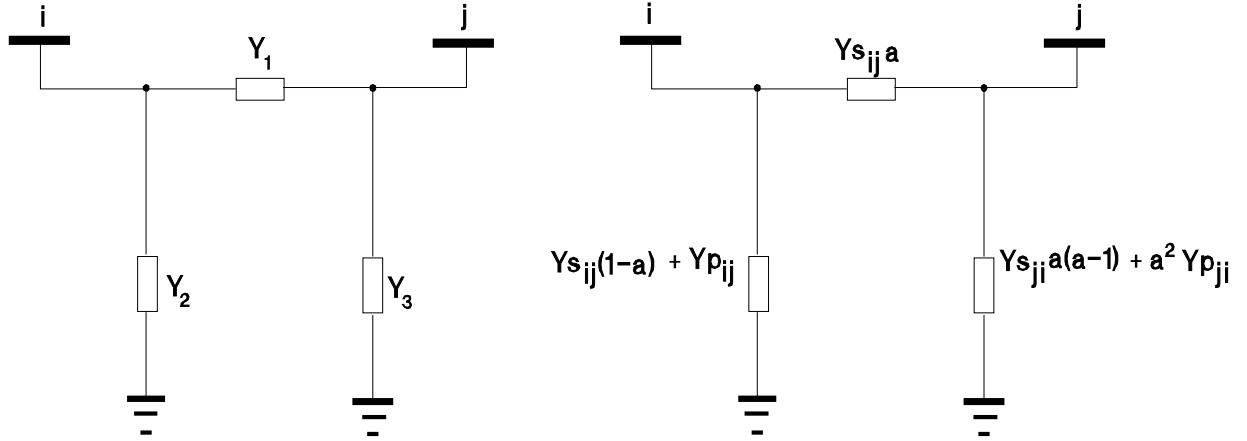


Figura C.5

Esquema en Π del transformador entre i y j con el regulador conectado a j

C.2.3 Potencias inyectadas en los nudos

De igual forma que en el apartado C.1.1 para el caso de una línea, teniendo en cuenta los nuevos esquemas en Π de la figuras C.3 y C.5, las potencias inyectadas son las que se indican a continuación.

• Regulador en el primario

$$P_i = |V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n \left(\frac{G_{p_{ij}} + G_{s_{ij}}}{a^2} \right) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n \frac{|V_j|}{a} \left[G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j) \right]$$

$$Q_i = -|V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n \left(\frac{B_{p_{ij}} + B_{s_{ij}}}{a^2} \right) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n \frac{|V_j|}{a} \left[G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j) \right]$$

$$\begin{aligned}
P_j &= |V_j|^2 \sum_{\substack{i=1 \\ i \neq j}}^n (G_{p_{ij}} + G_{s_{ij}}) - |V_j| \sum_{\substack{i=1 \\ i \neq j}}^n |V_i| a \left[G_{s_{ij}} \cos(\theta_j - \theta_i) + B_{s_{ij}} \sin(\theta_j - \theta_i) \right] \\
Q_j &= -|V_j|^2 \sum_{\substack{i=1 \\ i \neq j}}^n (B_{p_{ij}} + B_{s_{ij}}) - |V_j| \sum_{\substack{i=1 \\ i \neq j}}^n |V_i| a \left[G_{s_{ij}} \sin(\theta_j - \theta_i) - B_{s_{ij}} \cos(\theta_j - \theta_i) \right].
\end{aligned}$$

• **Regulador en el secundario**

$$\begin{aligned}
P_i &= |V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n (G_{p_{ij}} + G_{s_{ij}}) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| a \left[G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j) \right] \\
Q_i &= -|V_i|^2 \sum_{\substack{j=1 \\ j \neq i}}^n (B_{p_{ij}} + B_{s_{ij}}) - |V_i| \sum_{\substack{j=1 \\ j \neq i}}^n |V_j| a \left[G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j) \right]
\end{aligned}$$

$$\begin{aligned}
P_j &= |V_j|^2 \sum_{\substack{i=1 \\ i \neq j}}^n \left(\frac{G_{p_{ij}} + G_{s_{ij}}}{a^2} \right) - |V_j| \sum_{\substack{i=1 \\ i \neq j}}^n \frac{|V_i|}{a} \left[G_{s_{ij}} \cos(\theta_j - \theta_i) + B_{s_{ij}} \sin(\theta_j - \theta_i) \right] \\
Q_j &= -|V_j|^2 \sum_{\substack{i=1 \\ i \neq j}}^n \left(\frac{B_{p_{ij}} + B_{s_{ij}}}{a^2} \right) - |V_j| \sum_{\substack{i=1 \\ i \neq j}}^n \frac{|V_i|}{a} \left[G_{s_{ij}} \sin(\theta_j - \theta_i) - B_{s_{ij}} \cos(\theta_j - \theta_i) \right].
\end{aligned}$$

C.2.4 Flujos de potencia entre los nudos

Las fórmulas de los flujos por el elemento transformador entre los nudos i y j son las que siguen.

• **Regulador en el primario**

$$\begin{aligned}
P_{ij} &= |V_i|^2 \left(\frac{G_{s_{ij}} + G_{p_{ij}}}{a^2} \right) - \frac{|V_i||V_j|}{a} \left(G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j) \right) \\
Q_{ij} &= -|V_i|^2 \left(\frac{B_{s_{ij}} + B_{p_{ij}}}{a^2} \right) - \frac{|V_i||V_j|}{a} \left(G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j) \right)
\end{aligned}$$

$$\begin{aligned}
P_{ji} &= |V_j|^2 \left(G_{s_{ij}} + G_{p_{ij}} \right) - \frac{|V_i||V_j|}{a} \left(G_{s_{ij}} \cos(\theta_j - \theta_i) + B_{s_{ij}} \sin(\theta_j - \theta_i) \right) \\
Q_{ji} &= -|V_j|^2 \left(B_{s_{ij}} + B_{p_{ij}} \right) - \frac{|V_i||V_j|}{a} \left(G_{s_{ij}} \sin(\theta_j - \theta_i) - B_{s_{ij}} \cos(\theta_j - \theta_i) \right).
\end{aligned}$$

• Regulador en el secundario

$$\begin{aligned} P_{ij} &= |V_i|^2 (G_{s_{ij}} + G_{p_{ij}}) - |V_i||V_j|a (G_{s_{ij}} \cos(\theta_i - \theta_j) + B_{s_{ij}} \sin(\theta_i - \theta_j)) \\ Q_{ij} &= -|V_i|^2 (B_{s_{ij}} + B_{p_{ij}}) - |V_i||V_j|a (G_{s_{ij}} \sin(\theta_i - \theta_j) - B_{s_{ij}} \cos(\theta_i - \theta_j)) \end{aligned}$$

$$\begin{aligned} P_{ji} &= |V_j|^2 a^2 (G_{s_{ij}} + G_{p_{ij}}) - |V_i||V_j|a (G_{s_{ij}} \cos(\theta_j - \theta_i) + B_{s_{ij}} \sin(\theta_j - \theta_i)) \\ Q_{ji} &= -|V_j|^2 a^2 (B_{s_{ij}} + B_{p_{ij}}) - |V_i||V_j|a (G_{s_{ij}} \sin(\theta_j - \theta_i) - B_{s_{ij}} \cos(\theta_j - \theta_i)) . \end{aligned}$$

Referencias

Lo expuesto en este apéndice ha sido desarrollado por el autor basándose en la teoría estándar sobre análisis de sistemas de generación y transporte de energía eléctrica. Buenas referencias en este sentido son las clásicas: Brown [1975], Elgerd [1982], Stagg y El-Abiad [1968] y Stevenson [1975].

Apéndice D

CASUÍSTICA DE PROGRAMACIÓN LINEAL

ESTE APÉNDICE ESTÁ DEDICADO a estudiar en profundidad dos casos de la realidad económico/industrial empresarial cuyo planteamiento da lugar a la formulación de programas lineales de una cierta complejidad. Se refieren uno a la gestión financiera del dinero disponible en caja en una empresa u organismo, y a la gestión de la explotación de una refinería de petróleo el otro.

D.1 Gestión financiera a corto plazo

Uno de los cometidos más importantes de los responsables financieros de las grandes, medianas e incluso pequeñas empresas consiste en gestionar el dinero (liquidez) disponible a corto plazo, así como otros productos financieros, de tal forma que se produzcan los mayores beneficios posibles de ella evitando que, de no hacerse así, vayan a parar a los bancos y entidades depositarios de los mismos.

En este apéndice nos planteamos el trabajo a llevar a cabo por un agente financiero de una empresa y las decisiones que debe tomar para que, a partir de los resultados que se pueden derivar de utilizar programación lineal, su gestión e inversiones sean óptimas por lo que respecta a la obtención del máximo de los beneficios monetarios obtenibles.

Supongamos que nuestro hombre debe decidir sobre qué hacer con el dinero disponible y obtenible a corto plazo, en cuatro períodos consecutivos de 10, 20, 30 y 60 días, respectivamente. Es decir, maximizar los beneficios derivados de su gestión en los próximos cuatro meses, divididos en períodos según lo indicado.

La cartera de valores (bonos, acciones, etc.) negociables que posee la Empresa al comienzo del primer período se compone de 5 tipos diferentes cuyos nominales son 100 MM de pesetas, 75 MM, 750 MM, 600 MM y 900 MM, respectivamente. Los primeros 4 tipos vencen en los

períodos 1 a 4 de estudio, mientras que los del último tipo lo hacen más allá del horizonte de este análisis. Todos los valores, cualesquiera que sea su tipo, se pueden vender antes de que venzan descontando (perdiendo) la cantidad correspondiente. En el período número 2, además, vencen otros valores cuyo nominal es de 100 MM de pesetas, aunque no se pueden negociar pues esa cantidad está reservada para otros menesteres.

La única fuente de financiación ajena a los propios recursos de la Empresa con la que cuenta nuestro agente es una línea de crédito abierta con un banco por un montante total de 850 MM de ptas. Los préstamos se pueden solicitar al comienzo de cada uno de los cuatro períodos de estudio debiendo reintegrarse al cabo de un año junto con un interés del 0,7% mensual. No se permiten la amortizaciones de esta deuda antes de su vencimiento. Los costes unitarios, F_j , que para los períodos de estudio supone financiarse de esta manera son los que se indican en la tabla que sigue.

Coste de los créditos

F_1	0,0280
F_2	0,0257
F_3	0,0210
F_4	0,0140

Obsérvese que, por ejemplo, $0,0257 = [(60 + 30 + 20)/30] \times 0,7/100$, es decir, si se pide un crédito en el segundo período, el coste unitario que ello representa en los días restantes de estudio es de 0,0257 pesetas por cada una concedida.

Si en un período se compra pasivo que no sea crédito, los términos del mercado en el que opera la Empresa dictan que se debe devolver su nominal a 30 días o, si se desea hacerlo a los 10 días, se puede obtener un beneficio o prima del 2% (para designarlo rápidamente $2 \cdot 10 / (N - 30)$). De acuerdo con esto, los beneficios que obtendría la Empresa por la compra de este pasivo son los de la tabla que sigue.

**Coste del pasivo
no crediticio**

C_{12}	0,0204
C_{13}	0,0000
C_{22}	0,0204
C_{23}	0,0000
C_{33}	0,0204
C_{34}	0,0000
C_{44}	0,0204

Los valores de esta tabla se deducen considerando que si a los 10 días de adquirir un compromiso interesa reembolsarlo con un beneficio del 2%, la ganancia unitaria de esta operación es $1/(1 - 0,02) = 0,0204082 \approx 0,0204$.

Todas las obligaciones contraídas por la Empresa antes del comienzo del período de estudio han sido satisfechas, por lo que el pasivo que se adquiera se habrá de devolver dentro de este período no pudiéndose, tampoco, comprometerse con nada que suponga pagos posteriores.

En los cuatro períodos en que se divide el estudio se prevé adquirir pasivo no crediticio por un montante de 400 MM, 650 MM, 1.400 MM y 2.300 MM de pesetas, respectivamente. Se supone que estas adquisiciones se efectúan el primer día de cada uno de los cuatro períodos y que los pagos también hay que efectuarlos al comienzo del período correspondiente.

Los costes unitarios asociados con la compra y venta de productos financieros o valores para cada uno de los períodos de estudio se describen en la tabla D.1. Si, por ejemplo, se invierte 1 millón de pesetas el primer período en valores del tipo que sea que vencen en el cuarto período, lo que se gana mediante esta operación es

$$D_{41} \cdot 1 = 0,008 \text{ MM de ptas.}$$

Si se obtiene 1 millón de pesetas en el período 2 por la venta de valores que vencen más allá del período de estudio, período 5, el coste asociado a esta otra operación es

$$E_{52} \cdot 1 = 0,019 \text{ MM de ptas.}$$

Es decir, se pierden 19.000 pesetas por costes derivados de la operación.

Se supone que al comienzo del período de estudio hay en caja 100 MM de pesetas y que los flujos de dinero a la caja son -1.000 MM de ptas (salida de ella), -1.500 MM, 2.000 MM y 4.500 MM, en los períodos 1, 2, 3 y 4, respectivamente. El dinero mínimo que debe haber en caja al final de cada período (o comienzo del siguiente) puede ser 0 ptas. También se requiere que el balance medio diario en caja sea como mínimo de 100 MM de pesetas.

D.1.1 Modelo del problema a optimizar

Si de lo que se trata globalmente con todas estas operaciones, en principio, es maximizar las ganancias que nuestro financiero puede conseguir, veamos cómo plantear este proyecto ayudándonos de su modelización como un problema de programación lineal.

Tabla D.1

Costes unitarios de la compra o venta de valores o productos financieros

Costes de compraventa de valores

Compra		Venta	
D_{21}	0,0010	E_{21}	0,0020
D_{31}	0,0040	E_{31}	0,0050
D_{41}	0,0080	E_{41}	0,0100
D_{51}	0,0160	E_{51}	0,0200
D_{32}	0,0025	E_{32}	0,0037
D_{42}	0,0070	E_{42}	0,0087
D_{52}	0,0150	E_{52}	0,0190
D_{43}	0,0040	E_{43}	0,0050
D_{53}	0,0120	E_{53}	0,0150
D_{54}	0,0080	E_{54}	0,0100

Definamos las variables del problema:

- x_{gj} : cantidad pagada en el período j por compromisos adquiridos en el período g .
- y_{ij} : cantidad invertida en el período j en valores que vencen en el período i .
- z_{ij} : cantidad obtenida por la venta de valores en el período j que vencen en el período i .
- w_g : crédito obtenido en el período g .
- b_j : balance de caja en el período j .

Utilicemos además la siguiente notación para designar otras magnitudes del problema:

- A : balance de caja medio que se necesita en cada período.
- C_{gj} : remuneración unitaria de la cantidad x_{gj} .
- D_{ij} : remuneración unitaria de las inversiones en valores y_{ij} .
- E_{ij} : pérdida unitaria por la venta de z_{ij} .
- F_g : coste del crédito pedido en el período g .
- L_g : cantidad total de pasivo no crediticio comprometido en el período g .
- N_g : flujo de caja neto del período j (ingresos menos gastos).
- R : cantidad total de crédito disponible a corto plazo.
- S_i : valor nominal total de los valores de la cartera inicial que vencen en el período j .
- a_{gj} : $1 + C_{gj}$.
- d_{ij} : $1 + D_{ij}$.
- e_{ij} : $1 + E_{ij}$.

Tal como se han planteado, las condiciones que se le han impuesto al agente financiero de la Empresa son las que siguen a continuación.

Pagos. La representación general de las condiciones que se refieren a los pagos a efectuar por la Empresa es:

$$\sum_{j=g}^4 a_{gj} x_{gj} = L_g, \quad g = 1, \dots, 4.$$

Por ejemplo, para $g = 1$, como los compromisos adquiridos en el período 1 se pueden pagar en el 2 con una prima del 2%, o en el 3 a valor nominal, la condición correspondiente es $1,0204x_{12} + x_{13} = 400$.

Créditos. La cantidad total disponible para créditos no debe exceder de 850 MM de pesetas. Es decir,

$$w_1 + w_2 + w_3 + w_4 \leq 850.$$

Venta de valores. La expresión general de las condiciones impuestas a estas ventas es:

$$\sum_{j=1}^{i-1} e_{ij} z_{ij} \leq S_i \quad i = 2, \dots, 5.$$

Es decir, para $i = 2$, de acuerdo con la tabla D.1, los valores que vencen en el período 2 se pueden vender en el 1 con un descuento unitario del E_{21} sobre su valor nominal, obteniéndose $1/e_{21}$ de su valor. La correspondiente condición es pues $1,002z_{21} \leq 75$.

Balance de caja. El balance medio diario que se impone a nuestro agente se refleja en la siguiente condición:

$$10b_1 + 20b_2 + 30b_3 + 60b_4 \geq 12.000.$$

Flujos de caja. Las condiciones que se refieren a los flujos de caja tienen la siguiente forma:

$$b_g - b_{g-1} = \sum_{i=g+1}^5 z_{ig} + w_g - \sum_{j=1}^g x_{gj} - \sum_{i=g+1}^5 y_{ig} + S_g - \sum_{j=1}^{g-1} e_{gj} z_{gj} - \sum_{j=1}^{g-1} d_{gj} y_{gj} + N_g.$$

Para $g = 1$ se tiene la siguiente condición:

$$b_1 - 100 = z_{21} + z_{31} + z_{41} + z_{51} + w_1 - y_{21} - y_{31} - y_{41} - y_{51} + 100 - 1.000.$$

Función objetivo. El objetivo que se pretende conseguir es la maximización del beneficio obtenible en todo el período de estudio. Es decir,

$$\text{maximizar } \sum_{j=1}^4 C_{gj} x_{gj} + \sum_{i=j+1}^5 \sum_{j=1}^4 (D_{ij} y_{ij} - E_{ij} z_{ij}) - \sum_{g=1}^4 F_g w_g.$$

Todas las variables están restringidas a tomar valores no negativos.

Modelo general

Teniendo en cuenta todas las consideraciones hechas hasta este punto, el modelo del problema que tiene entre manos el agente financiero de esta Empresa es:

$$\begin{aligned} \text{max. } & 0,0204x_{12} + 0,0204x_{22} + 0,0204x_{33} + 0,0204x_{44} \\ & + 0,001y_{21} + 0,004y_{31} + 0,0025y_{32} + 0,008y_{41} + 0,007y_{42} \\ & + 0,004y_{43} + 0,016y_{51} + 0,015y_{52} + 0,012y_{53} + 0,008y_{54} \\ & - 0,002z_{21} - 0,005z_{31} - 0,0037z_{32} - 0,01z_{41} - 0,0087z_{42} \\ & - 0,005z_{43} - 0,02z_{51} - 0,019z_{52} - 0,015z_{53} - 0,01z_{54} \\ & - 0,028w_1 - 0,0257w_2 - 0,021w_3 - 0,014w_4 \end{aligned}$$

$$\text{s. a } 1,0204x_{12} + x_{13} = 400 \quad (\text{D.1})$$

$$1,0204x_{22} + x_{23} = 650 \quad (\text{D.2})$$

$$1,0204x_{33} + x_{34} = 1.400 \quad (\text{D.3})$$

$$1,0204x_{44} = 2300 \quad (\text{D.4})$$

$$w_1 + w_2 + w_3 + w_4 \leq 850 \quad (\text{D.5})$$

$$1,002z_{21} \leq 75 \quad (\text{D.6})$$

$$1,005z_{31} + 1,0037z_{32} \leq 750 \quad (\text{D.7})$$

$$1,01z_{41} + 1,0087z_{42} + 1,005z_{43} \leq 600 \quad (\text{D.8})$$

$$1,02z_{51} + 1,019z_{52} + 1,015z_{53} + 1,01z_{54} \leq 900 \quad (\text{D.9})$$

$$10b_1 + 20b_2 + 30b_3 + 60b_4 \geq 12.000 \quad (\text{D.10})$$

$$b_1 - z_{21} - z_{31} - z_{41} - z_{51} + y_{21} + y_{31} + y_{41} + y_{51} - w_1 = -800 \quad (\text{D.11})$$

$$b_2 - b_1 + x_{12} + x_{22} - z_{32} - z_{42} - z_{52} + y_{32} + y_{42} + y_{52} - w_2 - 1,001y_{21} + 1,002z_{21} = -1.425 \quad (\text{D.12})$$

$$\begin{aligned} b_3 - b_2 + x_{13} + x_{23} + x_{33} - z_{43} - z_{53} + y_{43} + y_{53} - w_3 + 1,005z_{31} + 1,0037z_{32} \\ - 1,004y_{31} - 1,0025y_{32} = 2.750 \end{aligned} \quad (\text{D.13})$$

$$\begin{aligned} b_4 - b_3 + x_{34} + x_{44} + y_{54} - z_{54} - w_4 + 1,01z_{41} + 1,0087z_{42} + 1,005z_{43} \\ - 1,008y_{41} - 1,007y_{42} - 1,004y_{43} = 5.100. \end{aligned} \quad (\text{D.14})$$

Las condiciones (D.1) a (D.4) son las de los pagos a efectuar en cada uno de los 4 períodos. La (D.5) es la que impone el límite de créditos. (D.6) a (D.9) tienen que ver con las ventas de valores. El balance de caja diario es la condición (D.10). Las condiciones (D.11) a (D.14) son las del flujo de caja en cada período.

Resolución del problema

Para resolver el problema que se ha planteado se puede utilizar cualquier paquete de *software* disponible en el mercado para optimizar programas lineales. Si se usa el que se presenta y estudia en este libro, BBMI, el fichero de datos que contiene toda la información necesaria para resolverlo es el que sigue.

```
MAXIMIZAR
NAME          Padberg1
ROWS
N  OBJETIVO
E  Pagos1
E  Pagos2
E  Pagos3
E  Pagos4
L  Credito
L  Valores2
L  Valores3
L  Valores4
L  Valores5
G  Caja
E  F.Caja1
E  F.Caja2
E  F.Caja3
E  F.Caja4
COLUMNS
X12      OBJETIVO  0.0204      Pagos1      1.0204
X12      F.Caja2    1                F.Caja3     1
X22      OBJETIVO  0.0204      Pagos2      1.0204
X22      F.Caja2    1                F.Caja4     1
X33      OBJETIVO  0.0204      Pagos3      1.0204
X33      F.Caja3    1                F.Caja1     1
X44      OBJETIVO  0.0204      Pagos4      1.0204
X44      F.Caja4    1                F.Caja3     1
X13      Pagos1     1                F.Caja3     1
X23      Pagos2     1                F.Caja3     1
X34      Pagos3     1                F.Caja4     1
Y21      OBJETIVO  0.001        F.Caja1     1
Y21      F.Caja2   -1.001
Y31      OBJETIVO  0.004        F.Caja1     1
Y31      F.Caja3   -1.004
Y32      OBJETIVO  0.0025       F.Caja2     1
Y32      F.Caja3   -1.0025
Y41      OBJETIVO  0.008        F.Caja1     1
```

Y41	F.Caja4	-1.008		
Y42	OBJETIVO	0.007	F.Caja2	1
Y42	F.Caja4	-1.007		
Y43	OBJETIVO	0.004	F.Caja3	1
Y43	F.Caja4	-1.004		
Y51	OBJETIVO	0.016	F.Caja1	1
Y52	OBJETIVO	0.015	F.Caja2	1
Y53	OBJETIVO	0.012	F.Caja3	1
Y54	OBJETIVO	0.008	F.Caja4	1
Z21	OBJETIVO	-0.002	Valores2	1.002
Z21	F.Caja1	-1	F.Caja2	1.002
Z31	OBJETIVO	-0.005	Valores3	1.005
Z31	F.Caja1	-1	F.Caja3	1.005
Z32	OBJETIVO	-0.0037	F.Caja2	-1
Z32	Valores3	1.0037	F.Caja3	1.0037
Z41	OBJETIVO	-0.01	F.Caja1	-1
Z41	Valores4	1.01	F.Caja4	1.01
Z42	OBJETIVO	-0.0087	F.Caja2	-1
Z42	Valores4	1.0087	F.Caja4	1.0087
Z43	OBJETIVO	-0.005	F.Caja3	-1
Z43	Valores4	1.005	F.Caja4	1.005
Z51	OBJETIVO	-0.02	F.Caja1	-1
Z51	Valores5	1.02		
Z52	OBJETIVO	-0.019	Valores5	1.019
Z52	F.Caja2	-1		
Z53	OBJETIVO	-0.015	Valores5	1.015
Z53	F.Caja3	-1		
Z54	OBJETIVO	-0.01	Valores5	1.01
Z54	F.Caja4	-1		
W1	OBJETIVO	-0.028	Credito	1
W1	F.Caja1	-1		
W2	OBJETIVO	-0.0257	Credito	1
W2	F.Caja2	-1		
W3	OBJETIVO	-0.021	Credito	1
W3	F.Caja3	-1		
W4	OBJETIVO	-0.014	Credito	1
W4	F.Caja4	-1		
B1	Caja	10	F.Caja1	1
B1	F.Caja2	-1		
B2	Caja	20	F.Caja2	1
B2	F.Caja3	-1		
B3	Caja	30	F.Caja3	1
B3	F.Caja4	-1		
B4	Caja	60	F.Caja4	1
RHS				
RHS1	Pagos1	400		
RHS1	Pagos2	650		
RHS1	Pagos3	1400		
RHS1	Pagos4	2300		
RHS1	Credito	850		
RHS1	Valores2	75		
RHS1	Valores3	750		
RHS1	Valores4	600		
RHS1	Valores5	900		
RHS1	Caja	12000		
RHS1	F.Caja1	-800		
RHS1	F.Caja2	-1425		
RHS1	F.Caja3	2750		
RHS1	F.Caja4	5100		

ENDATA

Los resultados que se obtienen con su concurso en un ordenador personal se listan a continuación.

Problema Padberg1

*** Estadísticas del Problema

15 Fila(s)
14 Restricción(es)
35 Variable(s) de decisión
14 Variable(s) de holgura/artificiales
103 Elementos no cero
0 Variable(s) entera(s)
Densidad de la matriz de coeficientes A: 19.619%

*** Estadísticas de INVERT

15 elementos no cero en la base
0 columnas estructurales en la base
0 vectores columna antes del "bump"
15 vectores columna después del "bump"
L: 0 elementos no cero; 0 vectores ETA
U: 1 elementos no cero; 1 vectores ETA
Total: 0 elementos no en la diagonal; 1 vectores ETA
Máximo de transformaciones ETA: 1; máximo número de elementos ETA: 118
Error relativo en x: .000000D+00

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra	de-a	Cost.Red	Sale	de-a	Paso	Pivote	El.Eta
1	9	23	.2682500D+05	35	L->B	.61D+02	49	B->L	.51D+04	.10D+01	1	
2	7	19	.9725000D+04	3	L->B	.20D+01	38	B->L	.14D+04	.10D+01	3	
3	6	17	.6952989D+04	19	L->B	.20D+01	42	B->L	.75D+03	.10D+01	6	
4	6	15	.5456720D+04	5	L->B	.20D+01	36	B->L	.40D+03	.10D+01	10	
5	5	13	.4656720D+04	6	L->B	.20D+01	48	B->L	.23D+03	.10D+01	12	
6	4	18	.4200742D+04	4	L->B	.10D+01	39	B->L	.23D+04	.10D+01	14	
7	3	17	.1900742D+04	21	L->B	.10D+01	37	B->L	.42D+03	.10D+01	18	
8	2	11	.1480810D+04	22	L->B	.10D+01	43	B->L	.59D+03	.10D+01	25	
9	2	9	.8859851D+03	24	L->B	.10D+01	46	B->U	.56D+02	-.10D+01	30	
10	1	5	.8301750D+03	8	L->B	.10D+01	44	B->L	.83D+03	.10D+01	33	
11	1	5	.2805606D+01	28	L->B	.10D+01	47	B->U	.28D+01	-.10D+01	37	
12	0	4	.5908148D+02	17	L->B	.80D-02	45	B->L	.16D+04	.60D+02	41	
13	0	5	.6173035D+02	1	L->B	.68D-02	5	B->L	.39D+03	.10D+01	44	
14	0	4	.6187611D+02	2	L->B	.68D-02	21	B->L	.22D+02	.10D+01	52	
15	0	6	.6436638D+02	13	L->B	.56D-02	40	B->L	.45D+03	.98D+00	60	
16	0	3	.6544360D+02	29	L->B	.13D-02	28	B->L	.85D+03	.10D+01	67	
17	0	2	.6567111D+02	20	L->B	.30D-03	19	B->L	.75D+03	.10D+01	74	
18	0	1	.6569587D+02	32	L->B	.30D-03	8	B->L	.82D+02	.10D+01	81	

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: Padberg1

No. de iteraciones: 19

Valor de la función objetivo: 65.6958680270542

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJETIVO	BS	65.695868	-65.695868	Ninguno	Ninguno	1.000
2	Pagos1	EQ	400.00000	.00000000	400.00000	400.00000	-.1203E-01
3	Pagos2	EQ	650.00000	.00000000	650.00000	650.00000	-.1203E-01
4	Pagos3	EQ	1400.0000	.00000000	1400.0000	1400.0000	.8201E-02
5	Pagos4	EQ	2300.0000	.00000000	2300.0000	2300.0000	.1215E-01
6	Credito	LS	850.00000	.00000000	Ninguno	850.00000	.6977E-02
7	Valores2	BS	.00000000	75.000000	Ninguno	75.000000	.0000
8	Valores3	LS	750.00000	.00000000	Ninguno	750.00000	.1684E-01
9	Valores4	LS	600.00000	.00000000	Ninguno	600.00000	.1577E-01
10	Valores5	LS	900.00000	.00000000	Ninguno	900.00000	.1374E-01
11	Caja	LI	12000.000	.00000000	12000.000	Ninguno	-.1333E-03
12	F.Caja1	EQ	-800.00000	.00000000	-800.00000	-800.00000	.3401E-01
13	F.Caja2	EQ	-1425.0000	.00000000	-1425.0000	-1425.0000	.3268E-01
14	F.Caja3	EQ	2750.0000	.00000000	2750.0000	2750.0000	.1203E-01
15	F.Caja4	EQ	5100.0000	.00000000	5100.0000	5100.0000	.8000E-02

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	X12	BS	392.00314	.20400000E-01	.00000000	Ninguno	.000
2	X22	BS	457.41006	.20400000E-01	.00000000	Ninguno	.000
3	X33	BS	1372.0110	.20400000E-01	.00000000	Ninguno	.000
4	X44	BS	2254.0180	.20400000E-01	.00000000	Ninguno	.000
5	X13	LI	.00000000	.00000000	.00000000	Ninguno	.000
6	X23	BS	183.25878	.00000000	.00000000	Ninguno	.000
7	X34	LI	.00000000	.00000000	.00000000	Ninguno	-.162E-01
8	Y21	LI	.00000000	.10000000E-02	.00000000	Ninguno	-.301E-03
9	Y31	LI	.00000000	.40000000E-02	.00000000	Ninguno	-.179E-01
10	Y32	LI	.00000000	.25000000E-02	.00000000	Ninguno	-.181E-01
11	Y41	LI	.00000000	.80000000E-02	.00000000	Ninguno	-.179E-01
12	Y42	LI	.00000000	.70000000E-02	.00000000	Ninguno	-.176E-01
13	Y43	BS	444.73025	.40000000E-02	.00000000	Ninguno	.000
14	Y51	LI	.00000000	.16000000E-01	.00000000	Ninguno	-.180E-01
15	Y52	LI	.00000000	.15000000E-01	.00000000	Ninguno	-.177E-01
16	Y53	LI	.00000000	.12000000E-01	.00000000	Ninguno	-.320E-04
17	Y54	BS	2506.2166	.80000000E-02	.00000000	Ninguno	.000
18	Z21	LI	.00000000	-.20000000E-02	.00000000	Ninguno	-.732E-03
19	Z31	LI	.00000000	-.50000000E-02	.00000000	Ninguno	-.420E-05
20	Z32	BS	747.23523	-.37000000E-02	.00000000	Ninguno	.000
21	Z41	LI	.00000000	-.10000000E-01	.00000000	Ninguno	-.108E-02
22	Z42	BS	594.82502	-.87000000E-02	.00000000	Ninguno	.000
23	Z43	LI	.00000000	-.50000000E-02	.00000000	Ninguno	-.169E-01
24	Z51	BS	882.35294	-.20000000E-01	.00000000	Ninguno	.000
25	Z52	LI	.00000000	-.19000000E-01	.00000000	Ninguno	-.320E-03
26	Z53	LI	.00000000	-.15000000E-01	.00000000	Ninguno	-.169E-01
27	Z54	LI	.00000000	-.10000000E-01	.00000000	Ninguno	-.159E-01
28	W1	LI	.00000000	-.28000000E-01	.00000000	Ninguno	-.967E-03
29	W2	BS	850.00000	-.25700000E-01	.00000000	Ninguno	.000
30	W3	LI	.00000000	-.21000000E-01	.00000000	Ninguno	-.159E-01
31	W4	LI	.00000000	-.14000000E-01	.00000000	Ninguno	-.130E-01
32	B1	BS	82.352941	.00000000	.00000000	Ninguno	.000
33	B2	LI	.00000000	.00000000	.00000000	Ninguno	-.180E-01
34	B3	LI	.00000000	.00000000	.00000000	Ninguno	-.320E-04
35	B4	BS	186.27451	.00000000	.00000000	Ninguno	.000

Tiempo total de CPU en cálculos: .0617 segundos

Los valores esenciales de la solución del problema se representan en la tabla que sigue. Las variables que no se listan son cero.

Función objetivo: 65,697			
x_{12}	392,003	z_{32}	747,235
x_{22}	456,644	z_{41}	594,059
x_{13}	0,000	z_{42}	0,000
x_{23}	184,040	z_{51}	882,353
x_{33}	1.372,011	w_2	850,000
x_{44}	2.254,018	b_1	676,412
y_{21}	0,000	b_2	0,000
y_{43}	443,949	b_3	0,000
y_{54}	2.604,442	b_4	87,265

La solución óptima genera unos ingresos íntegros de 65,697 MM de pesetas. Estos ingresos provienen de un total de 113,896 MM de pesetas de ingresos (beneficios obtenidos por pronto pago más los resultantes de las inversiones) menos 48,199 MM de gastos (venta de valores antes de su vencimiento e intereses del crédito). Del total de beneficios, 91,284 MM provienen del pronto pago y 22,612 de las inversiones en valores. Del total de gastos, 21,845 MM son debidos al uso del crédito y el resto, 26,354 MM, a la venta de valores antes de su vencimiento.

D.1.2 Análisis de sensibilidad

Estudiaremos diversas modificaciones del problema y su incidencia en los resultados. Por ejemplo, qué pasa si se suprime la condición de que el balance medio diario en caja ha de ser 100 MM de pesetas. Es decir, resolver el problema suprimiendo la condición (D.10).

Si se lleva a cabo este primer análisis, modificando adecuadamente el fichero de datos, se obtiene la siguiente solución.

Problema Padberg2

```
*** Estadísticas del Problema
    14 Fila(s)
    13 Restricción(es)
    35 Variable(s) de decisión
    13 Variable(s) de holgura/artificiales
    99 Elementos no cero
    0 Variable(s) entera(s)
Densidad de la matriz de coeficientes A: 20.204%

*** Estadísticas de INVERT
    14 elementos no cero en la base
    0 columnas estructurales en la base
    0 vectores columna antes del "bump"
    14 vectores columna después del "bump"
L:    0 elementos no cero;    0 vectores ETA
```

U: 0 elementos no cero; 0 vectores ETA
 Total: 0 elementos no en la diagonal; 0 vectores ETA
 Máximo de transformaciones ETA: 0; máximo número de elementos ETA: 113
 Error relativo en x: .000000D+00

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra	de-a	Cost.Red	Sale	de-a	Paso	Pivote	El.Eta
1	8	20	.1482500D+05	3	L->B	.20D+01	38	B->L	.14D+04	.10D+01	0	
2	7	19	.1205299D+05	4	L->B	.20D+01	39	B->L	.23D+04	.10D+01	3	
3	6	18	.7498971D+04	21	L->B	.20D+01	43	B->L	.59D+03	.10D+01	6	
4	6	16	.6304912D+04	19	L->B	.20D+01	45	B->U	.21D+03	-.10D+01	10	
5	5	17	.5892001D+04	8	L->B	.20D+01	42	B->L	.54D+03	.10D+01	14	
6	5	14	.4808103D+04	5	L->B	.20D+01	36	B->L	.40D+03	.10D+01	19	
7	4	12	.4008103D+04	6	L->B	.20D+01	47	B->L	.23D+03	.10D+01	21	
8	3	17	.3552125D+04	7	L->B	.20D+01	37	B->L	.43D+03	.98D+00	23	
9	2	11	.2699494D+04	16	L->B	.10D+01	3	B->L	.95D+03	.10D+01	28	
10	2	8	.1730114D+04	24	L->B	.10D+01	44	B->L	.88D+03	.10D+01	32	
11	2	6	.8468783D+03	28	L->B	.10D+01	46	B->U	.90D+00	-.10D+01	36	
12	1	2	.8459820D+03	17	L->B	.10D+01	48	B->L	.85D+03	.10D+01	40	
13	0	7	.5396437D+02	3	L->B	.17D-01	16	B->L	.95D+03	.10D+01	42	
14	0	12	.6281300D+02	1	L->B	.23D-01	5	B->L	.39D+03	.10D+01	46	
15	0	11	.6329992D+02	2	L->B	.23D-01	7	B->L	.22D+02	.10D+01	54	
16	0	6	.6578922D+02	13	L->B	.56D-02	40	B->L	.45D+03	.98D+00	62	
17	0	3	.6686644D+02	29	L->B	.13D-02	28	B->L	.85D+03	.10D+01	69	
18	0	2	.6709395D+02	20	L->B	.30D-03	19	B->L	.75D+03	.10D+01	76	
19	0	1	.6727111D+02	22	L->B	.30D-03	21	B->L	.59D+03	.10D+01	83	

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: Padberg2
 No. de iteraciones: 20
 Valor de la función objetivo: 67.2711081310103

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJETIVO	BS	67.271108	-67.271108	Ninguno	Ninguno	1.000
2	Pagos1	EQ	400.00000	.00000000	400.00000	400.00000	-.1203E-01
3	Pagos2	EQ	650.00000	.00000000	650.00000	650.00000	-.1203E-01
4	Pagos3	EQ	1400.0000	.00000000	1400.0000	1400.0000	.8201E-02
5	Pagos4	EQ	2300.0000	.00000000	2300.0000	2300.0000	.1215E-01
6	Credito	LS	850.00000	.00000000	Ninguno	850.00000	.6977E-02
7	Valores2	BS	.00000000	75.000000	Ninguno	75.000000	.0000
8	Valores3	LS	750.00000	.00000000	Ninguno	750.00000	.1684E-01
9	Valores4	LS	600.00000	.00000000	Ninguno	600.00000	.1577E-01
10	Valores5	LS	900.00000	.00000000	Ninguno	900.00000	.1344E-01
11	F.Caja1	EQ	-800.00000	.00000000	-800.00000	-800.00000	.3371E-01
12	F.Caja2	EQ	-1425.0000	.00000000	-1425.0000	-1425.0000	.3268E-01
13	F.Caja3	EQ	2750.0000	.00000000	2750.0000	2750.0000	.1203E-01
14	F.Caja4	EQ	5100.0000	.00000000	5100.0000	5100.0000	.8000E-02

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
-----	----------	----	---------------	---------------	---------------	---------------	----------

1	X12	BS	392.00314	.20400000E-01	.00000000	Ninguno	.000
2	X22	BS	457.49241	.20400000E-01	.00000000	Ninguno	.000
3	X33	BS	1372.0110	.20400000E-01	.00000000	Ninguno	.000
4	X44	BS	2254.0180	.20400000E-01	.00000000	Ninguno	.000
5	X13	LI	.00000000	.00000000	.00000000	Ninguno	-.173E-17
6	X23	BS	183.17474	.00000000	.00000000	Ninguno	.000
7	X34	LI	.00000000	.00000000	.00000000	Ninguno	-.162E-01
8	Y21	BS	82.352941	.10000000E-02	.00000000	Ninguno	.000
9	Y31	LI	.00000000	.40000000E-02	.00000000	Ninguno	-.176E-01
10	Y32	LI	.00000000	.25000000E-02	.00000000	Ninguno	-.181E-01
11	Y41	LI	.00000000	.80000000E-02	.00000000	Ninguno	-.176E-01
12	Y42	LI	.00000000	.70000000E-02	.00000000	Ninguno	-.176E-01
13	Y43	BS	444.81428	.40000000E-02	.00000000	Ninguno	.000
14	Y51	LI	.00000000	.16000000E-01	.00000000	Ninguno	-.177E-01
15	Y52	LI	.00000000	.15000000E-01	.00000000	Ninguno	-.177E-01
16	Y53	LI	.00000000	.12000000E-01	.00000000	Ninguno	-.320E-04
17	Y54	BS	2692.5755	.80000000E-02	.00000000	Ninguno	.000
18	Z21	LI	.00000000	-.20000000E-02	.00000000	Ninguno	-.103E-02
19	Z31	LI	.00000000	-.50000000E-02	.00000000	Ninguno	-.305E-03
20	Z32	BS	747.23523	-.37000000E-02	.00000000	Ninguno	.000
21	Z41	LI	.00000000	-.10000000E-01	.00000000	Ninguno	-.298E-03
22	Z42	BS	594.82502	-.87000000E-02	.00000000	Ninguno	.000
23	Z43	LI	.00000000	-.50000000E-02	.00000000	Ninguno	-.169E-01
24	Z51	BS	882.35294	-.20000000E-01	.00000000	Ninguno	.000
25	Z52	LI	.00000000	-.19000000E-01	.00000000	Ninguno	-.192E-04
26	Z53	LI	.00000000	-.15000000E-01	.00000000	Ninguno	-.166E-01
27	Z54	LI	.00000000	-.10000000E-01	.00000000	Ninguno	-.156E-01
28	W1	LI	.00000000	-.28000000E-01	.00000000	Ninguno	-.127E-02
29	W2	BS	850.00000	-.25700000E-01	.00000000	Ninguno	.000
30	W3	LI	.00000000	-.21000000E-01	.00000000	Ninguno	-.159E-01
31	W4	LI	.00000000	-.14000000E-01	.00000000	Ninguno	-.130E-01
32	B1	LI	.00000000	.00000000	.00000000	Ninguno	-.103E-02
33	B2	LI	.00000000	.00000000	.00000000	Ninguno	-.206E-01
34	B3	LI	.00000000	.00000000	.00000000	Ninguno	-.403E-02
35	B4	LI	.00000000	.00000000	.00000000	Ninguno	-.800E-02

Tiempo total de CPU en cálculos: .0503 segundos

El resumen de estos resultados se describe en la tabla que sigue

Función objetivo: 67,271			
x_{12}	392,003	z_{32}	747,235
x_{22}	457,492	z_{41}	0,000
x_{13}	0,000	z_{42}	594,825
x_{23}	183,175	z_{51}	882,353
x_{33}	1.372,011	w_2	850,000
x_{44}	2.254,018	b_1	0,000
y_{21}	82,353	b_2	0,000
y_{43}	444,814	b_3	0,000
y_{54}	2.692,575	b_4	0,000

Es decir, la supresión de esa condición le proporciona a la Empresa unos ingresos adicionales de 1,575 MM de pesetas.

Si se exigiese que, en vez de 0, el balance en caja al final de cada uno de los 4 períodos de

estudio fuese 100 MM de pesetas, habría que añadir las siguientes condiciones al problema:

$$b_j \geq 100, \quad j = 1, \dots, 4.$$

El fichero de datos que BBMI requiere para estudiar este caso es el que se lista a continuación.

```

MAXIMIZAR
NAME          Padberg3
ROWS
  N  OBJETIVO
  E  Pagos1
  E  Pagos2
  E  Pagos3
  E  Pagos4
  L  Credito
  L  Valores2
  L  Valores3
  L  Valores4
  L  Valores5
  E  F.Caja1
  E  F.Caja2
  E  F.Caja3
  E  F.Caja4
COLUMNS
  X12  OBJETIVO  0.0204      Pagos1  1.0204
  X12  F.Caja2   1          Pagos2  1.0204
  X22  OBJETIVO  0.0204      Pagos3  1.0204
  X22  F.Caja2   1          Pagos4  1.0204
  X33  OBJETIVO  0.0204      Pagos1  1
  X33  F.Caja3   1          Pagos2  1
  X44  OBJETIVO  0.0204      Pagos3  1
  X44  F.Caja4   1          Pagos4  1
  X13  Pagos1    1          F.Caja3  1
  X23  Pagos2    1          F.Caja3  1
  X34  Pagos3    1          F.Caja4  1
  Y21  OBJETIVO  0.001      F.Caja1  1
  Y21  F.Caja2   -1.001
  Y31  OBJETIVO  0.004      F.Caja1  1
  Y31  F.Caja3   -1.004
  Y32  OBJETIVO  0.0025     F.Caja2  1
  Y32  F.Caja3   -1.0025
  Y41  OBJETIVO  0.008      F.Caja1  1
  Y41  F.Caja4   -1.008
  Y42  OBJETIVO  0.007      F.Caja2  1
  Y42  F.Caja4   -1.007
  Y43  OBJETIVO  0.004      F.Caja3  1
  Y43  F.Caja4   -1.004
  Y51  OBJETIVO  0.016      F.Caja1  1
  Y52  OBJETIVO  0.015      F.Caja2  1
  Y53  OBJETIVO  0.012      F.Caja3  1
  Y54  OBJETIVO  0.008      F.Caja4  1
  Z21  OBJETIVO  -0.002     Valores2 1.002
  Z21  F.Caja1   -1        F.Caja2 1.002
  Z31  OBJETIVO  -0.005     Valores3 1.005
  Z31  F.Caja1   -1        F.Caja3 1.005
  Z32  OBJETIVO  -0.0037    F.Caja2 -1
  Z32  Valores3  1.0037    F.Caja3 1.0037
  Z41  OBJETIVO  -0.01      F.Caja1 -1
  Z41  Valores4  1.01      F.Caja4 1.01

```

Z42	OBJETIVO	-0.0087	F.Caja2	-1
Z42	Valores4	1.0087	F.Caja4	1.0087
Z43	OBJETIVO	-0.005	F.Caja3	-1
Z43	Valores4	1.005	F.Caja4	1.005
Z51	OBJETIVO	-0.02	F.Caja1	-1
Z51	Valores5	1.02		
Z52	OBJETIVO	-0.019	Valores5	1.019
Z52	F.Caja2	-1		
Z53	OBJETIVO	-0.015	Valores5	1.015
Z53	F.Caja3	-1		
Z54	OBJETIVO	-0.01	Valores5	1.01
Z54	F.Caja4	-1		
W1	OBJETIVO	-0.028	Credito	1
W1	F.Caja1	-1		
W2	OBJETIVO	-0.0257	Credito	1
W2	F.Caja2	-1		
W3	OBJETIVO	-0.021	Credito	1
W3	F.Caja3	-1		
W4	OBJETIVO	-0.014	Credito	1
W4	F.Caja4	-1		
B1	F.Caja1	1	F.Caja2	-1
B2	F.Caja2	1	F.Caja3	-1
B3	F.Caja3	1	F.Caja4	-1
B4	F.Caja4	1		
RHS				
RHS1	Pagos1	400		
RHS1	Pagos2	650		
RHS1	Pagos3	1400		
RHS1	Pagos4	2300		
RHS1	Credito	850		
RHS1	Valores2	75		
RHS1	Valores3	750		
RHS1	Valores4	600		
RHS1	Valores5	900		
RHS1	F.Caja1	-800		
RHS1	F.Caja2	-1425		
RHS1	F.Caja3	2750		
RHS1	F.Caja4	5100		
BOUNDS				
LO BOUND1	B1	100		
LO BOUND1	B2	100		
LO BOUND1	B3	100		
LO BOUND1	B4	100		
ENDATA				

Obsérvese que la única diferencia con el descrito anteriormente es que, en la sección BOUNDS, se han añadido unos límites inferiores a las variables b_i , $i = 1, \dots, 4$. La solución que se

obtendría haciendo funcionar a BBMI con estos datos es la de la tabla que sigue.

Función objetivo: 63,895			
x_{12}	392,003	z_{32}	747,235
x_{22}	357,387	z_{41}	17,647
x_{13}	0,000	z_{42}	577,155
x_{23}	285,322	z_{51}	882,353
x_{33}	1.372,011	w_2	850,000
x_{44}	2.254,018	b_1	100,000
y_{21}	0,000	b_2	100,000
y_{43}	342,667	b_3	100,000
y_{54}	2.590,020	b_4	100,000

El nuevo requisito que se le impone al agente supone, por tanto, que la Empresa deja de ingresar 1,801 MM de pesetas.

D.1.2.1 Cambio en las condiciones de la adquisición del pasivo no crediticio

Si en vez de 2-10/N-30, es decir, una prima del 2% a 10 días o el valor nominal a 30, las condiciones se cambian a 1-10/N-60, el 1% a 10 días y el nominal a 60, habría que replantearse un poco el modelo.

En primer lugar, como ahora se pueden adquirir compromisos hasta para 60 días, se hace necesario introducir las variables x_{14} y x_{24} . Por otro lado, los coeficientes en la función objetivo de las variables x_{12} , x_{22} , x_{33} y x_{44} , es decir los C_{gj} , han de ser $1/(1 - 0,01) - 1 = 0,0101$. Las correspondientes a_{gj} , lógicamente, deberán hacerse $1 + C_{gj} = 1,0101$.

El nuevo modelo que se tiene entonces es:

$$\begin{aligned} \text{max. } & 0,0101x_{12} + 0,0101x_{22} + 0,0101x_{33} + 0,0101x_{44} \\ & + 0,001y_{21} + 0,004y_{31} + 0,0025y_{32} + 0,008y_{41} + 0,007y_{42} \\ & + 0,004y_{43} + 0,016y_{51} + 0,015y_{52} + 0,012y_{53} + 0,008y_{54} \\ & - 0,002z_{21} - 0,005z_{31} - 0,0037z_{32} - 0,01z_{41} - 0,0087z_{42} \\ & - 0,005z_{43} - 0,02z_{51} - 0,019z_{52} - 0,015z_{53} - 0,01z_{54} \\ & - 0,028w_1 - 0,0257w_2 - 0,021w_3 - 0,014w_4 \end{aligned}$$

$$\text{s. a } 1,0101x_{12} + x_{13} + x_{14} = 400$$

$$1,0101x_{22} + x_{23} + x_{24} = 650$$

$$1,0101x_{33} + x_{34} = 1.400$$

$$1,0101x_{44} = 2.300$$

$$w_1 + w_2 + w_3 + w_4 \leq 850$$

$$1,002z_{21} \leq 75$$

$$1,005z_{31} + 1,0037z_{32} \leq 750$$

$$1,01z_{41} + 1,0087z_{42} + 1,005z_{43} \leq 600$$

$$1,02z_{51} + 1,019z_{52} + 1,015z_{53} + 1,01z_{54} \leq 900$$

$$10b_1 + 20b_2 + 30b_3 + 60b_4 \geq 12.000$$

$$b_1 - z_{21} - z_{31} - z_{41} - z_{51} + y_{21} + y_{31} + y_{41} + y_{51} - w_1 = -800$$

$$b_2 - b_1 + x_{12} + x_{22} - z_{32} - z_{42} - z_{52} + y_{32} + y_{42} + y_{52} - w_2 - 1,001y_{21} + 1,002z_{21} = -1.425$$

$$\begin{aligned}
& b_3 - b_2 + x_{13} + x_{23} + x_{33} - z_{43} - z_{53} + y_{43} + y_{53} - w_3 + 1,005z_{31} + 1,0037z_{32} \\
& - 1,004y_{31} - 1,0025y_{32} = 2.750 \\
& b_4 - b_3 + x_{14} + x_{24} + x_{34} + x_{44} + y_{54} - z_{54} - w_4 + 1,01z_{41} + 1,0087z_{42} + 1,005z_{43} \\
& - 1,008y_{41} - 1,007y_{42} - 1,004y_{43} = 5.100.
\end{aligned}$$

Usando BBMI para resolver este nuevo problema, se obtiene la solución que sigue.

Problema Padberg4

*** Estadísticas del Problema

15 Fila(s)
 14 Restricción(es)
 37 Variable(s) de decisión
 14 Variable(s) de holgura/artificiales
 107 Elementos no cero
 0 Variable(s) entera(s)
 Densidad de la matriz de coeficientes A: 19.279%

*** Estadísticas de INVERT

15 elementos no cero en la base
 0 columnas estructurales en la base
 0 vectores columna antes del "bump"
 15 vectores columna después del "bump"
 L: 0 elementos no cero; 0 vectores ETA
 U: 1 elementos no cero; 1 vectores ETA
 Total: 0 elementos no en la diagonal; 1 vectores ETA
 Máximo de transformaciones ETA: 1; máximo número de elementos ETA: 122
 Error relativo en x: .000000D+00

Ite.	Inf.	Nopt.	Sum.Inf/F.Obj	Entra	de-a	Cost.Red	Sale	de-a	Paso	Pivote	El.Eta
1	9	25	.2682500D+05	37	L->B	.61D+02	51	B->L	.51D+04	.10D+01	1
2	7	21	.9725000D+04	3	L->B	.20D+01	40	B->L	.14D+04	.10D+01	3
3	6	19	.6938999D+04	21	L->B	.20D+01	44	B->L	.75D+03	.10D+01	6
4	6	18	.5442730D+04	5	L->B	.20D+01	38	B->L	.40D+03	.10D+01	10
5	5	15	.4642730D+04	6	L->B	.20D+01	50	B->L	.21D+03	.10D+01	12
6	4	20	.4214733D+04	4	L->B	.10D+01	41	B->L	.23D+04	.10D+01	14
7	3	19	.1914733D+04	7	L->B	.10D+01	5	B->L	.40D+03	.10D+01	18
8	3	17	.1514733D+04	8	L->B	.10D+01	39	B->L	.36D+02	.10D+01	23
9	2	8	.1478731D+04	23	L->B	.10D+01	48	B->U	.54D+02	-.10D+01	26
10	1	6	.1425000D+04	10	L->B	.10D+01	45	B->L	.54D+03	.10D+01	31
11	1	6	.8841316D+03	26	L->B	.10D+01	46	B->L	.88D+03	.10D+01	37
12	1	4	.8963152D+00	30	L->B	.10D+01	49	B->U	.90D+00	-.10D+01	41
13	0	6	.1844383D+02	18	L->B	.12D-01	6	B->L	.61D+03	.10D+01	45
14	0	5	.2622781D+02	19	L->B	.80D-02	47	B->L	.97D+03	.60D+02	50
15	0	5	.2622895D+02	31	L->B	.13D-02	30	B->L	.90D+00	.10D+01	53
16	0	4	.2659811D+02	34	L->B	.31D-03	37	B->L	.12D+04	.17D+00	57
17	0	3	.2666554D+02	22	L->B	.30D-03	10	B->L	.22D+03	.10D+01	63
18	0	1	.2668519D+02	15	L->B	.32D-04	18	B->L	.61D+03	.10D+01	68

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: Padberg4

No. de iteraciones: 19
 Valor de la función objetivo: 26.6851862404083

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJETIVO	BS	26.685186	-26.685186	Ninguno	Ninguno	1.000
2	Pagos1	EQ	400.00000	.00000000	400.00000	400.00000	-.8000E-02
3	Pagos2	EQ	650.00000	.00000000	650.00000	650.00000	-.8000E-02
4	Pagos3	EQ	1400.0000	.00000000	1400.0000	1400.0000	-.1913E-02
5	Pagos4	EQ	2300.0000	.00000000	2300.0000	2300.0000	.2079E-02
6	Credito	BS	2.0305780	847.96942	Ninguno	850.00000	.0000
7	Valores2	BS	.00000000	75.000000	Ninguno	75.000000	.0000
8	Valores3	LS	750.00000	.00000000	Ninguno	750.00000	.9887E-02
9	Valores4	LS	600.00000	.00000000	Ninguno	600.00000	.8860E-02
10	Valores5	LS	900.00000	.00000000	Ninguno	900.00000	.6891E-02
11	Caja	LI	12000.000	.00000000	12000.000	Ninguno	-.1328E-03
12	F.Caja1	EQ	-800.00000	.00000000	-800.00000	-800.00000	.2703E-01
13	F.Caja2	EQ	-1425.0000	.00000000	-1425.0000	-1425.0000	.2570E-01
14	F.Caja3	EQ	2750.0000	.00000000	2750.0000	2750.0000	.1203E-01
15	F.Caja4	EQ	5100.0000	.00000000	5100.0000	5100.0000	.8000E-02

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	X12	LI	.00000000	.10100000E-01	.00000000	Ninguno	-.752E-02
2	X22	LI	.00000000	.10100000E-01	.00000000	Ninguno	-.752E-02
3	X33	BS	1386.0014	.10100000E-01	.00000000	Ninguno	.000
4	X44	BS	2277.0023	.10100000E-01	.00000000	Ninguno	.000
5	X13	LI	.00000000	.00000000	.00000000	Ninguno	-.403E-02
6	X23	LI	.00000000	.00000000	.00000000	Ninguno	-.403E-02
7	X14	BS	400.00000	.00000000	.00000000	Ninguno	.000
8	X24	BS	650.00000	.00000000	.00000000	Ninguno	.000
9	X34	LI	.00000000	.00000000	.00000000	Ninguno	-.609E-02
10	Y21	LI	.00000000	.10000000E-02	.00000000	Ninguno	-.303E-03
11	Y31	LI	.00000000	.40000000E-02	.00000000	Ninguno	-.109E-01
12	Y32	LI	.00000000	.25000000E-02	.00000000	Ninguno	-.111E-01
13	Y41	LI	.00000000	.80000000E-02	.00000000	Ninguno	-.110E-01
14	Y42	LI	.00000000	.70000000E-02	.00000000	Ninguno	-.106E-01
15	Y43	BS	613.99861	.40000000E-02	.00000000	Ninguno	.000
16	Y51	LI	.00000000	.16000000E-01	.00000000	Ninguno	-.110E-01
17	Y52	LI	.00000000	.15000000E-01	.00000000	Ninguno	-.107E-01
18	Y53	LI	.00000000	.12000000E-01	.00000000	Ninguno	-.320E-04
19	Y54	BS	1789.4523	.80000000E-02	.00000000	Ninguno	.000
20	Z21	LI	.00000000	-.20000000E-02	.00000000	Ninguno	-.723E-03
21	Z31	BS	523.58765	-.50000000E-02	.00000000	Ninguno	.000
22	Z32	BS	222.96942	-.37000000E-02	.00000000	Ninguno	.000
23	Z41	BS	594.05941	-.10000000E-01	.00000000	Ninguno	.000
24	Z42	LI	.00000000	-.87000000E-02	.00000000	Ninguno	-.658E-05
25	Z43	LI	.00000000	-.50000000E-02	.00000000	Ninguno	-.991E-02
26	Z51	BS	882.35294	-.20000000E-01	.00000000	Ninguno	.000
27	Z52	LI	.00000000	-.19000000E-01	.00000000	Ninguno	-.322E-03
28	Z53	LI	.00000000	-.15000000E-01	.00000000	Ninguno	-.996E-02
29	Z54	LI	.00000000	-.10000000E-01	.00000000	Ninguno	-.896E-02
30	W1	LI	.00000000	-.28000000E-01	.00000000	Ninguno	-.972E-03

31	W2	BS	2.0305780	-.25700000E-01	.00000000	Ninguno	.000
32	W3	LI	.00000000	-.21000000E-01	.00000000	Ninguno	-.897E-02
33	W4	LI	.00000000	-.14000000E-01	.00000000	Ninguno	-.600E-02
34	B1	BS	1200.0000	.00000000	.00000000	Ninguno	.000
35	B2	LI	.00000000	.00000000	.00000000	Ninguno	-.110E-01
36	B3	LI	.00000000	.00000000	.00000000	Ninguno	-.465E-04
37	B4	LI	.00000000	.00000000	.00000000	Ninguno	-.290E-04

Tiempo total de CPU en cálculos: .0002 segundos

El valor que se obtiene en la función objetivo es 26,685. Resulta una pérdida de ganancias de $65,697 - 26,685 = 39,012$ MM de pesetas.

Si la mitad del pasivo no crediticio se obtuviese a 2-10/N-30 y la otra mitad a 1-10/N-60, habría que reconfigurar otra vez el modelo. Se introduciría un nuevo índice, h , para las variables x resultando x_{hgj} : cantidad pagada con el tipo remunerativo h (1 ó 2) en el período j por compromisos adquiridos en el período g .

Las condiciones (D.1) a (D.4) habría que desdoblarlas en dos grupos: uno para cada tipo de remuneración. Además habría que reemplazar cada x_{gj} por $x_{1gj} + x_{2gj}$ dondequiera que fuese preciso.

El nuevo modelo que resultaría llevando a cabo estas modificaciones es el siguiente:

$$\begin{aligned}
 \text{max. } & 0,0204x_{112} + 0,0204x_{122} + 0,0204x_{133} + 0,0204x_{144} \\
 & 0,0101x_{212} + 0,0101x_{222} + 0,0101x_{233} + 0,0101x_{244} \\
 & +0,001y_{21} + 0,004y_{31} + 0,0025y_{32} + 0,008y_{41} + 0,007y_{42} \\
 & +0,004y_{43} + 0,016y_{51} + 0,015y_{52} + 0,012y_{53} + 0,008y_{54} \\
 & -0,002z_{21} - 0,005z_{31} - 0,0037z_{32} - 0,01z_{41} - 0,0087z_{42} \\
 & -0,005z_{43} - 0,02z_{51} - 0,019z_{52} - 0,015z_{53} - 0,01z_{54} \\
 & -0,028w_1 - 0,0257w_2 - 0,021w_3 - 0,014w_4
 \end{aligned}$$

$$\text{s. a } 1,0204x_{112} + x_{113} = 200$$

$$1,0204x_{122} + x_{123} = 325$$

$$1,0204x_{133} + x_{134} = 700$$

$$1,0204x_{144} = 1.150$$

$$1,0101x_{212} + x_{213} + x_{214} = 200$$

$$1,0101x_{222} + x_{223} + x_{224} = 325$$

$$1,0101x_{233} + x_{234} = 700$$

$$1,0101x_{244} = 1.150$$

$$w_1 + w_2 + w_3 + w_4 \leq 850$$

$$1,002z_{21} \leq 75$$

$$1,005z_{31} + 1,0037z_{32} \leq 750$$

$$1,01z_{41} + 1,0087z_{42} + 1,005z_{43} \leq 600$$

$$1,02z_{51} + 1,019z_{52} + 1,015z_{53} + 1,01z_{54} \leq 900$$

$$10b_1 + 20b_2 + 30b_3 + 60b_4 \geq 12.000$$

$$b_1 - z_{21} - z_{31} - z_{41} - z_{51} + y_{21} + y_{31} + y_{41} + y_{51} - w_1 = -800$$

$$b_2 - b_1 + x_{112} + x_{212} + x_{122} + x_{222} - z_{32} - z_{42} - z_{52} + y_{32} + y_{42} + y_{52} - w_2 - 1,001y_{21} + 1,002z_{21} = -1.425$$

$$b_3 - b_2 + x_{113} + x_{213} + x_{123} + x_{223} + x_{133} + x_{233} - z_{43} - z_{53} + y_{43} + y_{53} - w_3 + 1,005z_{31} + 1,0037z_{32}$$

$$- 1,004y_{31} - 1,0025y_{32} = 2.750$$

$$b_4 - b_3 + x_{214} + x_{224} + x_{134} + x_{234} + x_{144} + x_{244} + y_{54} - z_{54} - w_4 + 1,01z_{41} + 1,0087z_{42} + 1,005z_{43} - 1,008y_{41} - 1,007y_{42} - 1,004y_{43} = 5.100.$$

Los valores de las variables que son distintos de cero en la solución óptima de este nuevo problema, utilizando una vez más BBMI para resolverlo, son los de la tabla que sigue.

Función objetivo: 46,822			
x_{112}	196,002	y_{43}	620,994
x_{122}	318,503	y_{54}	2.332,968
x_{133}	686,005	z_{31}	523,588
x_{144}	1.127,009	z_{32}	222,969
x_{233}	693,001	z_{41}	594,059
x_{244}	1.138,501	z_{51}	882,353
x_{214}	200,000	w_2	516,535
x_{224}	325,000	b_1	1.200,000

El coste (pérdida de beneficios) de esta nueva modalidad de remunerar el pasivo no crediticio es $65,697 - 46,822 = 18,875$ MM de pesetas.

D.1.3 Solución factible inicial

Cualquier solución factible del programa lineal deberá equilibrar el balance. Lo que hemos hecho hasta ahora es obtener, de entre todas las factibles, aquella que optimiza las ganancias de la Empresa jugando con sus distintas opciones.

Si se trata única y exclusivamente de cumplir todas las condiciones que se han impuesto, para obtener la deseada solución factible se pueden realizar ciertas simplificaciones. Por ejemplo:

- Hay que aprovechar todos los beneficios posibles del pronto pago.
- Hay que utilizar la línea de crédito al máximo cuanto antes.
- Hay que retrasar la venta de valores lo más posible pero se venderá si ello supone obtener mayores beneficios por pronto pago.

Con estas premisas, tanteando un poco, se puede obtener una solución factible como la de la tabla D.2 que sigue.

Los valores de las variables que hemos definido con anterioridad que son significativamente distintos de cero para la solución que define esta tabla, así como la función objetivo correspondiente, se describen a continuación.

Función objetivo: 64,621			
x_{12}	392,003	z_{32}	747,226
x_{22}	457,899	z_{42}	594,776
x_{33}	1.372,010	z_{52}	882,900
x_{44}	2.254,018	w_1	800,000
x_{23}	182,760	w_2	50,000
y_{53}	445,240	b_4	200,000
y_{54}	2.045,980		

Tabla D.2

Balance equilibrado a partir del cual se obtiene una solución factible inicial del problema de la gestión financiera a corto plazo

	Período 1	Período 2	Período 3	Período 4
Balance en caja (CP) ^a	100,00	0,00	0,00	0,00
Ventas	100,00	2.299,90 ^b	2.000,00	4.500,00
Efectivo disponible	200,00	2.299,90	2.000,00	4.500,00
Gasto	1.000,00	2.349,90 ^c	1.554,76 ^d	2.254,02 ^e
Balance en caja (FP) ^f	-800,00	-50,00	445,24	2.245,98
Balance mínimo	0,00	0,00	0,00	0,00
Total	-800,00	-50,00	445,24	2.246,00
Para invertir	0,00	0,00	445,24	2.045,98 ^g
Crédito	800,00	50,00	0,00	0,00

^aComienzo del período.

^bDe vender todos los valores: $75,00 + 750,00 \cdot (1 - 0,0037) + 600 \cdot (1 - 0,0087) + 900 \cdot (1 - 0,019)$.

^cFlujo neto a caja, -1.500, más lo necesario, -849,90, para aprovechar el crédito al máximo.

^dLos 1.400 con prima de pronto pago más algo del pasivo de períodos anteriores.

^eLos 2.300 con prima de pronto pago.

^fFin del período.

^gPara mantener el balance medio diario en caja de 100 MM de pesetas ($60 \cdot 200 = 12.000$).

Además de ésta existen otras soluciones que verifican el balance de la tabla anterior. En efecto, de acuerdo con la nota *c* de esa tabla, $x_{12} + x_{22} = 849,90$. Además, como exige el modelo $1,0204x_{12} + x_{13} = 400$ y $1,0204x_{22} + x_{23} = 650$. Como tanto x_{12} como x_{22} tienen el mismo coeficiente en la función objetivo, basta con hacer $x_{13} = 0$ para obtener una solución factible. También $y_{43} + y_{53} = 445,24$. Como y_{53} tiene un mayor coeficiente en la función objetivo y se trata de maximizarla, se puede hacer $y_{43} = 0$.

La función objetivo 64,621 resulta de la diferencia entre unos ingresos totales de $113,020 = 91,309$ (por pronto pago)+ $21,711$ (por inversiones) y de unos gastos de $48,399 = 23,685$ (coste del crédito)+ $24,714$ (venta de valores).

Aunque tal vez absurdo desde el punto de vista práctico, es interesante teóricamente preguntarse cuál sería el mínimo de todos los posibles ingresos totales que podría obtener el agente de la Empresa. Para conseguir esto, habría que hacer uso de todo el crédito, los 850 MM de pesetas, en el primer período, pues es cuando resulta más caro, vender todos los valores de la cartera inicial también en el primer período y, por último, devolver el pasivo no crediticio sin beneficiarse del pronto pago. Estas, probablemente irracionales, decisiones —en aras de minimizar los ingresos de su gestión—, corresponden a los siguientes valores de las variables tal como las hemos definido anteriormente: $w_1 = 850$, $x_{13} = 400$, $x_{23} = 650$, $x_{34} = 1.400$, $x_{44} = 2.254,02$, $z_{21} = 74,85$, $z_{31} = 746,27$, $b_1 = 2.347,53$, $b_2 = 847,53$, $b_3 = 1.797,53$ y $b_4 = 2.643,51$. Las variables y , por supuesto, serían cero. El valor de la función objetivo resultante, ganancias con esta forma de actuar, sería -5,29 MM de pesetas. Es decir, se perdería dinero. A esta misma solución se llegaría, como se puede comprobar fácilmente, con el concurso del programa BB-MI, si en el modelo inicial que se ha planteado se minimizase la función objetivo en vez de

maximizarla.

D.1.3.1 Análisis de los valores duales de las condiciones

Los valores duales de las condiciones, o vector de *multiplicadores simplex*, en la solución del modelo inicial se pueden usar para tratar de gestionar mejor las finanzas de la Empresa. Este vector, cuyo valor es

$$\lambda = \begin{bmatrix} -0,0120 \\ -0,0120 \\ 0,0082 \\ 0,0122 \\ 0,0070 \\ 0,0000 \\ 0,0168 \\ 0,0158 \\ 0,0137 \\ -0,0001 \\ 0,0340 \\ 0,0327 \\ 0,0120 \\ 0,0080 \end{bmatrix},$$

indica lo siguiente:

- De los valores duales de las cuatro primeras condiciones se deduce que si se aumenta la cantidad de dinero dedicada a comprar activo en los últimos dos períodos se aumentarán los beneficios de la Empresa; si se dedican más recursos, por el contrario, para los dos primeros períodos, los beneficios disminuirán.
- Que si aumenta la disponibilidad de crédito (condición (5)), se aumentarán los beneficios de la Empresa. El interés que se está dispuesto a pagar por un aumento extra de esta línea de crédito es el 0,7%.
- Si se aumentan para negociar con ellas las cantidades disponibles en valores de los tipos 3, 4 y 5 (condiciones (7) a (9)), el beneficio de la Empresa aumentaría. Si parte de los valores que cuantifican 100 MM de pesetas que se reservan para otros menesteres en el período 2, por el contrario, se pusiesen a disposición del agente para utilizarlos, no se mejoraría en nada el beneficio obtenible: el valor dual de la condición (6) es cero.
- Que elevando el balance medio diario que hay que mantener en caja se reducirían los beneficios obtenibles por la Empresa. Esto lo indica el valor $-0,0001$ correspondiente a la condición (10). Esto es así pues se reducirían las oportunidades de inversión o incrementarían los costes financieros.
- Que una elevación de los flujos de caja en todos los períodos de estudio mejorarían las expectativas de ganancia de la Empresa: los *multiplicadores simplex* correspondientes a las condiciones (11) a (14) son todos positivos.

D.2 Gestión operativa de una refinería de crudo de petróleo

Una refinería de crudo de petróleo produce aceites lubricantes, disolventes, combustibles líquidos de varios tipos y calidades, así como algunos residuos como el asfalto, los alquitranes y el coque de petróleo. En la figura D.1 se representa muy simplificada el proceso productivo general de una industria de este tipo.

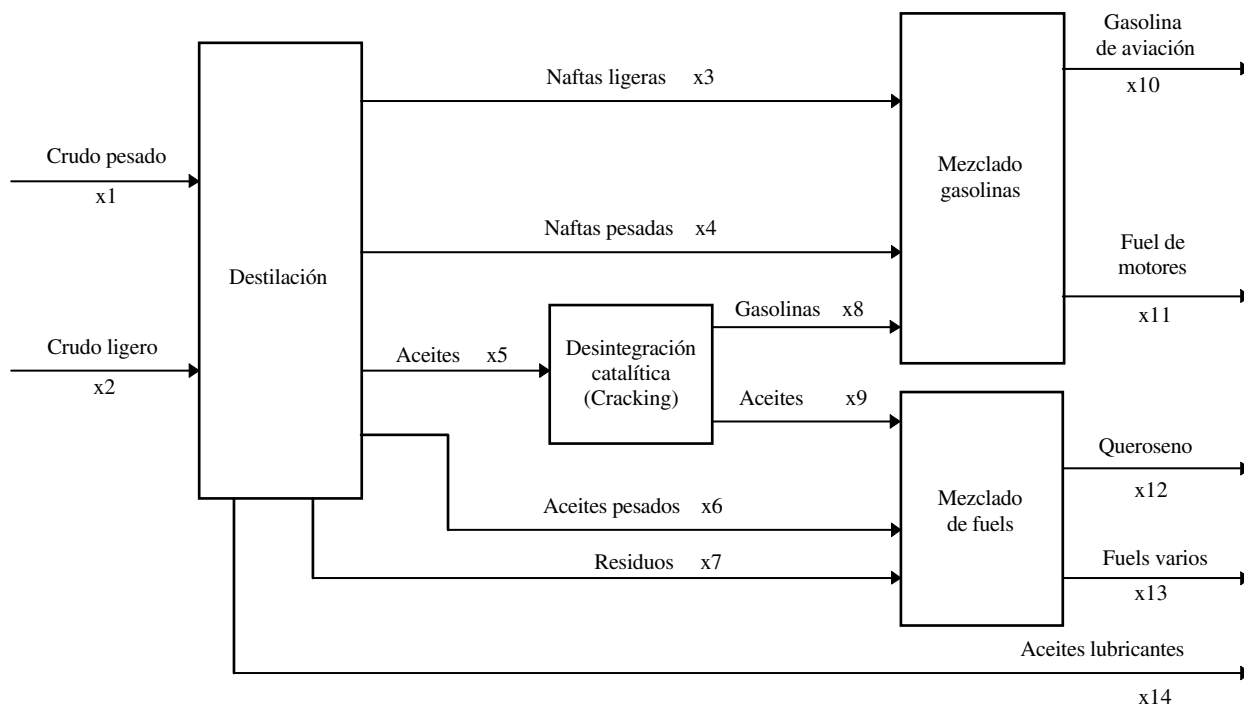


Figura D.1

Proceso productivo simplificado de una refinería de crudo de petróleo

Para producir estos artículos a partir de diversas variedades de crudo de petróleo se necesita el concurso de gran cantidad de energía calorífica en forma de vapor de agua, para poner en marcha determinadas reacciones químicas y multitud de procesos mecánicos, así como energía eléctrica.

Dadas las importantes cantidades de vapor de agua y electricidad que consume una refinería, en vez de comprárselas a suministradores externos resulta más rentable producirlas in situ a partir de los propios combustibles líquidos que elabora y vende, o de los derivados gaseosos que se desprenden de los procesos productivos y que en muchos casos de no utilizarse para estos menesteres se desecharían.

Quemar combustibles fósiles para producir calor, y a partir de éste vapor de agua y electricidad para consumirlos en los procesos químicos que apuntábamos, lógicamente, reduce la capacidad neta de la refinería de producir artículos vendibles al mercado. El vapor de agua que se produce no se obtiene a presiones homogéneas. Si se produce a alta presión, se debe despresurizar para poderlo utilizar en otros procesos donde se necesite menos presión. Esta despresurización se puede llevar a efecto haciendo pasar el vapor a través de turbinas capaces de mover un alternador y por tanto generar energía eléctrica. Si la refinería es capaz de producir más energía eléctrica que la que necesita en un momento dado, se la puede vender a la

compañía eléctrica que opera en la zona, produciéndose de esta forma unos ingresos adicionales a los que se originan por la venta de los artículos que elabora.

Un complejo industrial de refinado de crudo de petróleo es en suma una instalación —con calderas, turbinas de vapor y gas, motores eléctricos, alternadores, torres de destilación, reactores químicos, *reformadores*¹, etc.— con capacidad de autoabastecer sus necesidades energéticas, y con la facultad de decidir en un momento dado si comprar o vender energía eléctrica según sus requerimientos y los precios del mercado. Nada desdeñable tampoco es su inherente capacidad para aprovechar mejor que en otros procesos productivos industriales el poder energético de los combustibles que produce/utiliza, y por tanto contribuir así al ahorro energético de la compañía mercantil a la que pertenece y a la economía productiva del país en el que opera.

En lo que sigue se va a describir simplificada el proceso físico de producción de vapor de agua en una típica refinería de crudo y la optimización que con él se puede llevar a cabo para producir energía eléctrica, de tal manera que se ahorre lo más posible en el recibo que la compañía eléctrica que la abastece le presenta periódicamente. El problema que se formulará partirá de un modelo lineal de esa descripción física y de su operación diaria, y buscará optimizar ésta última hora a hora.

En lo sucesivo, al hablar de *unidades de producción* nos estaremos refiriendo a los diversos dispositivos o mecanismos del proceso productivo de la refinería —unidad de desintegración catalítica (*cracking* o *craqueo*), columnas de fraccionamiento, cámaras de mezcla, etc.—, así como a los diversos dispositivos productores de vapor de agua y energía eléctrica. Su función específica dentro del conjunto productivo de la refinería no es relevante para el problema que aquí presentamos.

D.2.1 Producción de vapor de agua y electricidad en una refinería de petróleo

En la figura D.2 se representan esquemáticamente los procesos que se llevan a cabo para producir vapor de agua en la refinería tipo que estudiaremos en este apéndice.

El vapor recalentado a alta presión se produce en las calderas principales (cuadro 5) y en los turbogeneradores de gas, TG (cuadro 1). Para calentar el agua de las calderas y convertirla en vapor se utiliza el fuel almacenado en los tanques correspondientes. Este fuel, antes de quemarse en las calderas, se precalienta en la unidad del cuadro 7, denominada *precalentador de fuel*, PC, y, mediante el concurso de vapor de agua, se pulveriza en la unidad del cuadro 6, denominada *pulverizador de fuel*, PF. El agua que se utiliza en las calderas es agua desionizada, que ha sido tratada en el *desaireador* del cuadro 9, y precalentada. Este precalentamiento se lleva a cabo en el *precalentador* del cuadro 4, PCC. El agua desionizada se almacena en el tanque correspondiente y se supone que está disponible a coste cero y en cantidad sin límite. Las calderas, además de vapor de agua, producen la denominada *agua de descarga* la cual se envía a un *tanque de destilación flash* (representado en la figura por el cuadro 8), TDF. En este tanque se separa el agua y el condensado de vapor de agua. El agua se vuelve a enviar al *desaireador* mientras que el condensado se tira por el drenaje. Los productos que se desechan por el drenaje se representan en la figura D.2 mediante un pequeño recuadro negro.

En las calderas de los turbogeneradores de gas, TG, se quema el gas que resulta como residuo de diversos procesos de la refinería y con el calor desprendido se calienta el agua. Como consecuencia de esto se produce vapor de agua a alta y baja presión. Haciendo pasar el agua de descarga de las calderas de estos turbogeneradores a través de un tanque de destilación

¹Instalaciones donde se mejora el octanaje de las gasolinas o se realizan otros procesos químicos.

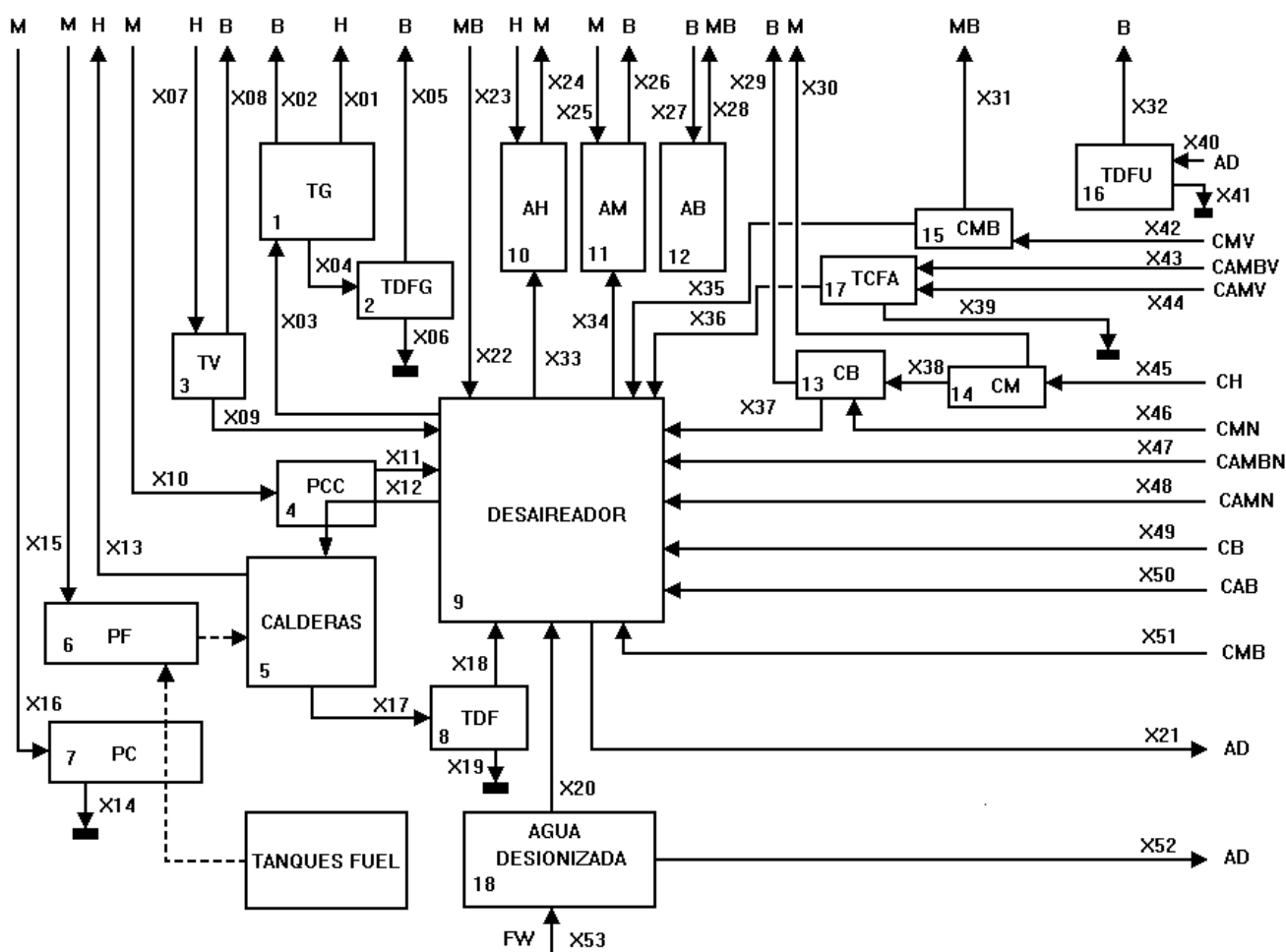


Figura D.2

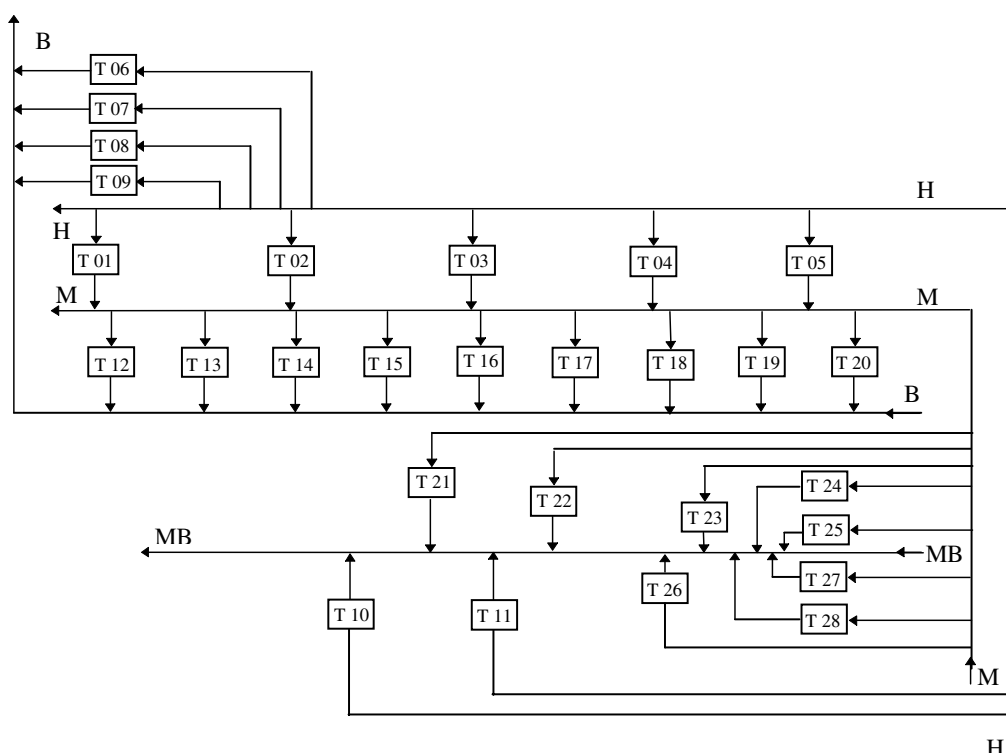
Esquema productivo de vapor de agua de una refinería de crudo de petróleo

flash, recuadro 2, TDFG, también se produce una cierta cantidad de vapor a baja presión. En el proceso de combustión del gas, además de producir vapor de agua, se puede producir energía eléctrica mediante la utilización de unas turbinas de gas. Haciendo pasar el vapor de agua a alta presión que se ha obtenido en este proceso a través de turbinas de vapor, también se puede producir energía eléctrica además de vapor de agua a más baja presión de acuerdo con la etapa o etapas de extracción que se utilicen. El vapor que se recoge de la última etapa de extracción de las turbinas de vapor se condensa en el correspondiente condensador y luego se envía al desaireador.

El sistema productor de electricidad a base de vapor de agua consta de 28 turbinas. El esquema de cómo operan, qué vapor utiliza y qué vapor producen se representa en la figura D.3.

El vapor de agua que no se obtiene mediante los procesos descritos hasta ahora se produce por condensación *flash*, por reducción de entalpía², en las unidades de producción antes men-

²La entalpía es una magnitud termodinámica igual a la suma de la energía interna de un fluido y el producto

**Figura D.3**

Esquema productivo de las turbinas de vapor de la refinería

cionadas y en diversas turbinas. En los condensadores *flash* de media, CM, baja, CB, y muy baja, CMB, presión, los condensados procedentes de las unidades de producción a alta y media presión se someten a un proceso de reducción de presión para producir vapor de agua a media, baja y muy baja presión.

Los atemperadores de alta, AH, media, AM, y baja, AB, reducen la entalpía del vapor de agua que a ellos entra a media, baja y muy baja presión.

En las tablas D.3 y D.4 se especifican las producciones o consumos de cada una de las unidades de producción a las que nos estamos refiriendo, así como los consumos de vapor de agua y producción de energía eléctrica de las 28 turbinas de vapor de que se compone el sistema que estamos estudiando. Como se puede observar, el vapor de agua, a distintas presiones, se produce o consume en todas las unidades de producción. Algunas de éstas poseen sus propias calderas para producir vapor de agua.

De las calderas que hay en las unidades de producción se deriva un agua, AD, a partir de la cual también se puede producir vapor de agua a baja presión en un tanque de destilación *flash*: cuadro 16 de la figura D.2, TDFU.

Los condensados aceitosos de media y muy baja presión que resultan de algunas de las

de su presión por su volumen. Tiene la dimensión de una energía: kJ/kg, kcal/kg.

La energía interna la integran la energía cinética de traslación de las moléculas, la energía cinética de rotación de las moléculas, la energía cinética vibratoria de los átomos de las moléculas y la energía potencial de las moléculas debida a la fuerza de atracción entre ellas. La energía interna aumenta al aumentar la separación de las moléculas. Depende del volumen y de la temperatura del fluido.

Tabla D.3

Producción/consumo horario de agua, vapor de agua y condensados de las diversas unidades de producción de la refinería

Unidad	H	M	B	MB	AC	AD	AA	ADI	CH	CM	CB	CMB	AM	AB	AMB
U01		3,50			4,6		-7,60								
U03		5,80		1,5			-5,60								
U04		16,80								-9,3					
U06		1,80					-1,60								
U07			6,10									-6,1			
U08		5,10		0,2									-12,6		-0,2
U09				1,8			-1,80								
U10	4,4			1,2	7,5							-10,2			
U11	3,1	-12,40	-5,50		39,0	-1,40	-20,60			-0,5	-0,50				
U12	11,5	2,60							-11,5						
U13	4,4	-26,90	2,52		26,0	-1,00				-1,3	-2,40				
U14		0,30	0,10								-0,10				
U15		2,80	52,00		4,3			0,5		-3,1	-55,70			-0,30	
U16					3,8										
U17	1,2	-18,60	-2,90		24,0				-1,4		-1,40				
U18			20,10								-20,10				
U19	0,2	-19,80	0,20		24,5	-1,00	-3,47	0,4		-0,3	-0,20				
U20	-1,3	-19,70	1,20		28,5	-1,28	-5,90				-1,10				
U21	-30,0	10,10	2,82		33,5	-1,30	-29,10				-1,82				
U22							-3,80								
U23			5,40		0,5									-5,90	
U24	4,0		2,80		0,7			1,1	-4,7					-2,80	
U25		5,00											-4,8		
U26	-8,0	-0,13				-0,65		15,6							
U27	7,1	10,90	16,33	2,5					-3,3	-2,1	-7,70	-3,4	-0,6	-8,63	-3,1

Leyendas:

H	Vapor a alta presión	CH	Condensado a alta presión
M	Vapor a media presión	CM	Condensado a media presión
B	Vapor a baja presión	CB	Condensado a baja presión
MB	Vapor a muy baja presión	CMB	Condensado a muy baja presión
AC	Agua de caldera principal del desaireador	AM	Cond. aceitoso a media presión
AD	Agua de descarga de las calderas	AB	Cond. aceitoso a baja presión
AA	Agua ácida	AMB	Cond. aceitoso a muy baja presión
ADI	Agua desionizada		

Tabla D.4

Requisitos horarios de energía eléctrica y combustibles en las distintas unidades de producción, y consumos de vapor y potencias de las turbinas

Unidad	G	FO1	FO2	SG	SFO1	EL
U01		3,60				0,800
U02						0,115
U03		6,40				1,245
U04				1,08		0,820
U05				3,77		
U06		1,36		0,96		0,395
U07	-5,000					0,050
U08	-3,925			0,01		0,220
U10				0,25		0,105
U11			5,65			1,620
U12		1,73				1,270
U13			9,22	0,35		4,350
U14					0,89	1,130
U15	-17,103					0,925
U16						0,093
U17				0,73		0,685
U18						0,205
U19			2,59			1,140
U20			2,03			2,975
U21			2,34			6,330
U22						0,002
U23						0,140
U24	-0,856					0,720
U25		1,59				0,565
U26	-1,578			1,14		0,545
U27				1,02		9,555

Leyendas:

G Gas (combustible)
 FO1 Gas o fuel del tipo 1
 FO2 Gas o fuel del tipo 2
 SG Sólo gas
 SFO1 Sólo fuel del tipo 1
 EL Energía eléctrica

Turbina	Unidad	Vapor Tn/h	Potencia MW
H a M			
T01	U19	9,50	0,372
T02	U11	3,30	0,116
T03	U21	9,00	0,280
T04	U26	4,20	0,111
T05	U16	37,00	0,045
H a B			
T06	U19	4,60	0,278
T07	U11	6,60	0,306
T08	U11	4,60	0,204
T09	U20	0,50	0,013
H a MB			
T10	U22	8,00	0,460
T11	U22	6,10	0,145
M a B			
T12	U15	7,30	0,164
T13	U13	7,30	0,144
T14	U13	4,80	0,092
T15	U13	3,30	0,052
T16	U20	2,70	0,036
T17	U21	3,30	0,044
T18	U20	2,40	0,030
T19	U21	1,30	0,015
T20	U13	2,10	0,024
M a MB			
T21	U12	2,00	0,184
T22	U16	4,70	0,250
T23	U16	2,20	0,100
T24	U16	3,80	0,150
T25	U16	0,90	0,015
T26	U12	0,90	0,013
T27	U22	1,00	0,011
T28	U16	2,00	0,015

unidades de producción se procesan en un tanque de condensado *flash*: cuadro 17, TCFA. El exceso de vapor a muy baja presión que no se necesita se libera a la atmósfera.

La implantación física de la refinería consta de cuatro circuitos de tuberías de vapor de agua: a alta, H; media, M; baja, B; y muy baja presión, MB. Las pérdidas de carga y de poder calorífico del fluido a través de las correspondientes tuberías se consideran despreciables teóricamente. Las diferentes unidades de producción de la refinería y las turbinas de vapor se conectan a estos circuitos en paralelo con el fin de que si alguna se avería ello no afecte al resto de la producción.

El agua ácida, AA, que se produce en las unidades de producción se tira por los drenajes.

Debido a diferencias tecnológicas, a las unidades de producción 1 a 9 se las denomina *viejas* y a las 10 a 27 *nuevas*. Las viejas y las nuevas producen condensados de distintas características.

Las líneas discontinuas de la figura D.2 se refieren a los circuitos de transporte de fuel.

Como ejemplo del balance de flujos entrantes y salientes, en la figura D.4 se representa el de la unidad de producción 11: la columna de destilación de vacío. En este balance no se indica ni la electricidad que se consume en esta unidad, ni el fuel que se quema. Tampoco se pone de manifiesto cómo opera funcionalmente la misma pues ello no afecta a la producción/consumo de vapor de agua tal como los estamos considerando aquí.

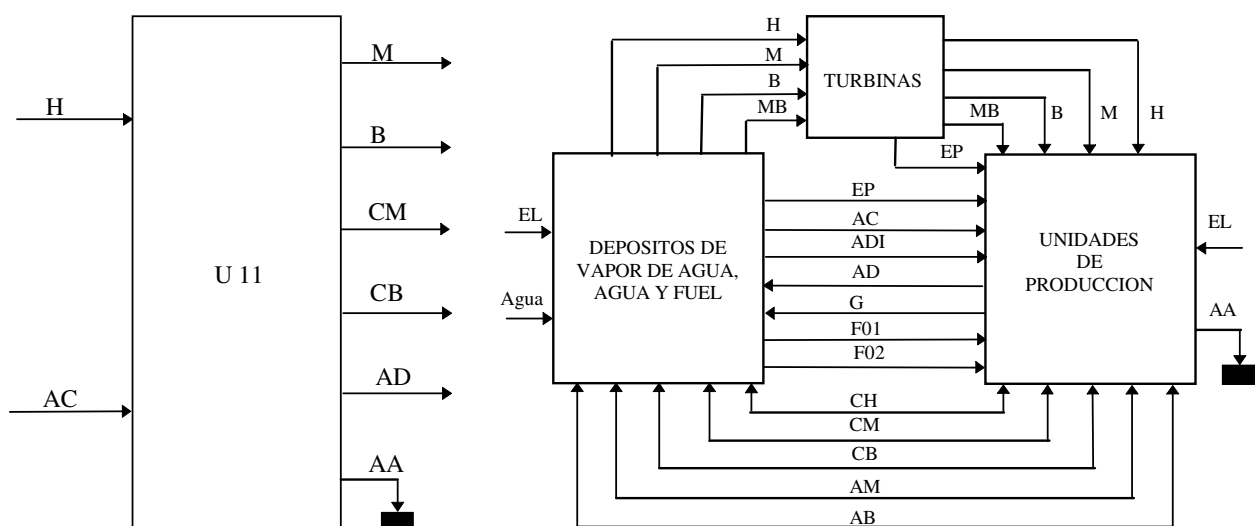


Figura D.4

Fluidos que se consumen y producen en la unidad de producción número 11 y esquema de flujos energéticos en la refinería

En las tablas D.3 y D.4 a que hacíamos referencia con anterioridad, las cantidades positivas indican consumo horario en toneladas; las negativas producción. Los consumos y producciones de todas las unidades de producción se han prorrateado linealmente con el fin de que si una unidad operase a menos capacidad de la nominal, su producción y consumo se puedan interpolar linealmente. Si no funcionan, evidentemente, las producciones y consumos serían cero. No se tienen en cuenta, por tanto, los mínimos técnicos operativos reales de esas unidades de producción.

En la producción de vapor de agua se utilizan fundamentalmente tres combustibles: gas, fuel de tipo 1 y fuel de tipo 2. El gas se produce en la refinería y se quema en los hogares

de las diversas calderas de las unidades de producción y en los turbogeneradores. Como su composición no es homogénea, o se consume en la refinería o se tiene que quemar en unas antorchas al aire libre pues su valor en el mercado no hace rentable su comercialización. Los hogares de las calderas de las unidades de producción son de cuatro tipos: los que sólo pueden quemar gas, SG; los que pueden quemar gas y fuel del tipo 1, FO1; los que pueden quemar gas y fuel del tipo 2, FO2; y los que sólo pueden quemar fuel del tipo 1, SFO1. El fuel que se quema en las calderas principales es del tipo 1 y necesita ser precalentado. El de este mismo tipo que se quema en las unidades de producción no hay que precalentarlo. Los tanques de fuel se consideran suficientemente grandes como para abastecer cualquier situación que requiera la refinería; se rellenan automáticamente.

Como ya hemos indicado, la refinería es capaz de producir energía eléctrica en los turbogeneradores de gas, en los turbogeneradores de vapor y en las turbinas de vapor. La electricidad adicional que necesita la refinería se adquiere a una compañía eléctrica que opera en la zona. En la tabla D.4 se describen las cantidades de vapor que necesitan las turbinas de vapor de agua para producir energía eléctrica, qué cantidad de energía son capaces de producir y en qué unidad de producción se encuentran ubicadas. Si una unidad de producción opera a menos del 100% de su capacidad, la correspondiente turbina de vapor no funciona. Como ya se indicó anteriormente, en la tabla D.4 también se indican los combustibles que se producen y consumen en todas las unidades de producción, así como sus necesidades de energía eléctrica. Las cantidades positivas indican consumos y las negativas producciones.

En la figura D.4 se describen esquemáticamente los flujos de energía en la refinería; EL se refiere a la energía eléctrica consumida y EP a la producida.

D.2.2 Modelo del problema a optimizar

El objetivo económico que se persigue es satisfacer todas las necesidades de vapor de agua de la refinería al menor coste posible.

Dado que todos los costes son lineales y las condiciones que se imponen se han planteado de tal manera que conducen a relaciones lineales, parece aconsejable utilizar programación lineal para optimizar los aspectos económicos y productivos de la fabricación horaria de vapor de agua en la refinería.

La **función objetivo** de este problema consistirá en minimizar los costes operativos de la refinería para producir vapor que determinan las compras de energía eléctrica a la empresa suministradora y los derivados del consumo de combustibles que produce la refinería que podrían venderse.

Las **variables** del problema son de tres tipos: las cantidades de vapor de agua, agua y condensados; el volumen de electricidad y fuel consumido; y las 55 variables binarias, 0 ó 1, que indican qué unidades de producción y qué turbinas funcionan y cuáles no.

Aunque las variables correspondientes a las unidades de producción que están funcionando pueden adoptar valores continuos, lo que refleja que dichas unidades pueden funcionar a un rendimiento por debajo del nominal, en principio supondremos que sólo pueden ser 0 ó 1. Esto permitirá una mayor flexibilidad al modelo, o mejor dicho, a la interpretación y uso que se pudiese hacer de los resultados que se obtengan, pues no olvidemos que lo que se está modelando es la operación horaria de la producción de vapor de la refinería. El que las variables que definen el funcionamiento de las unidades de producción sólo tomen valores 0 ó 1 tiene la ventaja de que a partir de una solución se puede programar la posición de las válvulas de los circuitos de vapor de la refinería a lo largo de todo un día (u otro período) de producción.

Las **restricciones** o condiciones del problema son también de tres tipos: las de balance de masa, las de balance de energía y las que denominaremos tecnológicas. Las de balance de masa establecen la conservación del flujo en cada uno de los recuadros de la figura D.2; es decir, la masa que entra en cada uno de ellos es igual a la que sale. Las de balance de energía hacen uso de las magnitudes entálpicas de la tabla D.5 para convertir los diversos flujos y hacerlos comparables y poderlos relacionar. Esas magnitudes se basan en condiciones generalizadas de conservación de flujo y utilizan números conocidos de diversos manuales de termodinámica.

Desde el punto de vista de la termodinámica teórica supondremos que la calidad del vapor de agua y de los condensados es la misma en cualquier punto de la red de tuberías que los transportan. Esta asunción nos permitirá trabajar con datos de entalpías constantes, aunque la realidad sea un poco distinta.³ Una mayor profundidad en la definición de esta cuestión llevaría a que la entalpía fuese otra variable a considerar, lo que daría lugar a condiciones no lineales.

Las condiciones tecnológicas son las que imponen las especificaciones de los manuales operativos de la maquinaria utilizada en las diversas unidades de producción y turbinas.

El poder calorífico del fuel que se utiliza en la caldera principal es 8.540 kcal/kg. En su precalentamiento, este fuel absorbe 22,5 kcal/kg. El agua que se precalienta absorbe en este proceso 28 kcal/kg. Como ya hemos indicado anteriormente, esto se lleva a cabo haciéndola pasar a través del PCC. Lógicamente, se supone que la cantidad de agua que llega a este dispositivo para ser precalentada es igual a la que sale.

En la tabla D.5 se describen las diversas entalpías en kcal/kg de los fluidos que circulan por la red de tuberías de vapor de agua de la refinería. Los tanques *flash* CM, CB, CMB, TDFG y TDFU producen vapor saturado seco o saturado líquido a presiones diversas según describe la

³En muchos casos prácticos de más envergadura que el que analizamos aquí esta hipótesis es perfectamente asumible a efectos de diseño.

Tabla D.5

Entalpías en kcal/kg de los diversos fluidos de vapor de agua de la refinería

Presión	Vapor recalentado	Vapor saturado seco	Vapor saturado líquido
Vapor de agua			
Alta	772	-	263
Media	732	665	194
Baja	677	656	150
Muy baja	686	651	134
Condensados			
Alta			255
Media			181
Baja			144
Muy baja			126
Agua desionizada		10	
Vapor que se tira del TCFA		640	
Agua del TCFA al desaireador		100	
Agua del TDF al desaireador		640	
Agua de los turbogeneradores de vapor al desaireador		40	
Agua del PCC a la caldera principal		130	

figura D.2. El vapor que va a la red es saturado seco, el resto es saturado líquido. Las turbinas descargan vapor saturado líquido. El agua de descarga de las calderas de las unidades de producción, AD, tiene una entalpía de 190 kcal/kg. El fluido que va de las calderas principales al TDF, y de los TG al TDFG, es vapor saturado líquido a alta presión. El precalentador de agua de las calderas principales, PCC, produce un fluido condensado de media presión que se vierte en el desaireador. El fluido que se desecha de TDFG y TDFU tiene una entalpía de 150 kcal/kg. El que también se tira de TDF y PC, 100 kcal/kg y 181 kcal/kg respectivamente. El desaireador produce agua con una entalpía de 102 kcal/kg.

La refinería compra la energía eléctrica a la compañía local a 4.440 pesetas el MWh y vende la producida que le sobra a 3.550.

El precio del fuel del tipo 1 es 12.000 pesetas la tonelada; el del tipo 2, 10.000 pesetas, también la tonelada.

Condiciones tecnológicas

Como ya hemos apuntado, estas condiciones son las que imponen las características físicas del funcionamiento de los diversos dispositivos que componen el sistema productivo de vapor de agua de la refinería.

Para los **turbogeneradores de vapor** estas condiciones indican que estas máquinas no pueden aceptar más de 100 Tn/hora de vapor de agua a alta presión y que la cantidad de agua de condensación que resulta del proceso termodinámico de las correspondientes turbinas es superior a 3 Tn/hora e inferior a 60. Para incorporar la electricidad que se produce, en vez de recurrir a un balance de energía, se introduce la siguiente relación:

$$0,122(\text{vapor a baja presión}) + 0,249(\text{agua de condensación}) - (\text{energía eléctrica}) = 1,143.$$

De igual manera, los **turbogeneradores de gas** deben satisfacer las siguientes condiciones tecnológicas:

$$\begin{aligned} (\text{gas quemado}) - 0,206(\text{energía eléctrica}) &= 4,199 \\ (\text{vapor a baja presión}) - 0,355(\text{energía eléctrica}) &= 12,7 \\ (\text{vapor a alta presión}) - 0,617(\text{energía eléctrica}) &= 35,2. \end{aligned}$$

La cantidad máxima de gas que se puede quemar en los turbogeneradores de gas es 10,7 Tn/hora. La cantidad de agua que va al tanque de destilación *flash* es el 2,5% de la de vapor de agua a alta presión que produce.

Las **calderas principales** producen un mínimo de 20 Tn/hora de vapor de agua y un máximo de 60. La cantidad de fluido que se envía al tanque de destilación *flash* es el 2,5% del vapor que en ellas se produce.

En el **pulverizador de fuel** se utiliza una cantidad de vapor de agua igual al 30% de la de fuel que se pulveriza.

Los **atemperadores** de alta presión no pueden tratar más de 60 Tn/hora de vapor a alta presión; los de media y baja presión tienen ese límite establecido en 50 Tn/hora.

Por lo que se refiere al **desaireador**, la cantidad máxima de agua que puede producir es 423,3 Tn/hora.

Para satisfacer los requisitos de los turbogeneradores de gas, la refinería se ve obligada a comprar cada hora a la compañía eléctrica que la abastece 1,48 MWh.

Para aclarar con un caso concreto los tres tipos de condiciones que hemos indicado anteriormente, consideremos las calderas del recuadro 5 de la figura D.2. La condición de conservación

de masa dice que la cantidad de vapor de agua a alta presión que se produce más la cantidad de agua que se lleva al tanque de destilación *flash* debe ser igual a la cantidad de agua que llega del desaireador. Cada uno de estos fluidos poseerá una entalpía que indicará su energía interna más el producto de su presión por su volumen. Según la tabla D.5, la entalpía del vapor de agua recalentado a alta presión es 772 kcal/kg, la del agua que va al tanque 263 kcal/kg y la que llega del desaireador ya precalentada 130 kcal/kg. Sabiendo que el poder calorífico del fuel que se quema en la caldera es 8.540 kcal/kg, el balance de energía en la caldera es: 772 (entalpía del vapor producido) + 263 (entalpía del agua que va al tanque de destilación) – 130 (la del agua que entra) – 8.540 (la del fuel que se quema) = 0. Según acabamos de ver, la condición tecnológica en este caso sería (agua al tanque de destilación) – 0,025 (cantidad de vapor a alta presión producido) = 0: la cantidad de agua que va al tanque de destilación *flash* es el 2,5% de la del vapor de agua a alta presión.

D.2.3 Formulación del problema

Definamos primero las variables que se van a utilizar. Las que definen las masas de vapor de agua que se producen en el sistema y que circulan por él, x_i , $1 \leq i \leq 54$, se representan en la figura D.2. Los arcos que en esta figura atraviesan un recuadro indican que la correspondiente variable interviene sólo en el balance energético y no en el balance de masa. Mediante la variable x_{54} se designan el exceso de vapor de baja presión que se libera a la atmósfera.

Las variables 0 ó 1 t_j , $1 \leq j \leq 28$ y u_j , $1 \leq j \leq 27$, designan qué turbinas y qué unidades de producción estarán o no funcionando una determinada hora.

Otras variables son:

- w_{01} : energía eléctrica superior a 1,48 MWh que se compra a la compañía eléctrica.
- w_{02} : energía eléctrica que producen los turbogeneradores de gas.
- w_{03} : energía eléctrica que producen los turbogeneradores de vapor.
- w_{04} : energía eléctrica que producen las turbinas.
- w_{05} : energía eléctrica que consumen las unidades de producción.
- w_{06} : energía eléctrica que se vende al exterior.
- f_{01} : cantidad de fuel del tipo 1 que se consume en las calderas principales.
- f_{02} : fuel del tipo 1 que consumen los hogares FO1 y SFO1.
- f_{03} : fuel del tipo 2 que consumen los hogares FO2.
- f_{04} : gas que consumen los turbogeneradores de gas.
- f_{05} : gas que consumen los hogares SG.
- f_{06} : gas que consumen los hogares de FO1.
- f_{07} : gas que consumen los hogares de FO2.

Función objetivo. El objetivo que se pretende conseguir es la minimización de los costes operativos horarios de la refinería para producir vapor de agua que determinan las compras de energía eléctrica y los consumos de combustibles que podrían venderse. Teniendo en cuenta los datos que se han dado, esta función es

$$\text{minimizar } 6.571,2x_0 + 4.440w_{01} - 3.550w_{06} + 12.000f_{01} + 12.000f_{02} + 10.000f_{03}.$$

La constante 6.571,2 tiene en cuenta el hecho de que los turbogeneradores de gas requieren comprar a la compañía eléctrica un mínimo de 1,48 MWh cada hora a 4.440 pesetas el MWh. La variable x_0 es constante e igual a 1.

Esta función objetivo sólo pretende optimizar los costes que hemos mencionado, por lo que habrá que imponer como condición que la producción regular de artículos que se venden de la refinería sea la máxima posible. Es decir, que la refinería funcione para maximizar su objetivo productivo primario: los productos derivados del petróleo. Para conseguir esto se fijará el nivel productivo de las unidades de producción al 100%. Si no se hiciese así, con la función objetivo tal como está planteada, dejando libre el valor de las variables u_j entre 0 y 1, se primaría el que las unidades de producción no trabajasen para ganar dinero y sí para vender energía eléctrica a la compañía local.

Condiciones de balance de masa, de energía y tecnológicas Empezaremos escribiendo estas condiciones para cada uno de los dispositivos de la figura D.2.

(1) Turbogeneradores de gas, TG:

$$\begin{aligned} -x_{01} - x_{02} + x_{03} - x_{04} &= 0 \\ x_{01} - 0,617w_{02} &= 35,2 \\ x_{02} - 0,355w_{02} &= 12,7 \\ f_{04} - 0,206w_{02} &= 4,199 \end{aligned}$$

(2) Tanque de destilación flash de los turbogeneradores de gas, TDFG:

$$\begin{aligned} -0,025x_{01} + x_{04} &= 0 \\ x_{04} - x_{05} - x_{06} &= 0 \\ 263x_{04} - 656x_{05} - 150x_{06} &= 0. \end{aligned}$$

(3) Turbogeneradores de vapor, TV:

$$\begin{aligned} x_{07} - x_{09} - x_{08} &= 0 \\ 0,122x_{08} + 0,249x_{09} - w_{03} &= 1,143 \end{aligned}$$

(4) Precalentador del agua de las calderas principales, PCC:

$$\begin{aligned} x_{10} - x_{11} &= 0 \\ 732x_{10} - 181x_{11} - 28x_{12} &= 0. \end{aligned}$$

(5) Calderas principales:

$$\begin{aligned} x_{12} - x_{13} - x_{17} &= 0 \\ -130x_{12} + 772x_{13} + 263x_{17} - 8.540f_{01} &= 0. \end{aligned}$$

(6) Pulverizador de fuel, PF:

$$x_{15} - 0,3f_{01} = 0.$$

(7) Precalentador de fuel, PC:

$$\begin{aligned} -x_{14} + x_{16} &= 0 \\ 22,5f_{01} + 181x_{14} - 732x_{16} &= 0. \end{aligned}$$

(8) Tanque de destilación flash, TDF:

$$\begin{aligned} -0,025x_{13} + x_{17} &= 0 \\ x_{17} - x_{18} - x_{19} &= 0 \\ 263x_{17} - 640x_{18} - 100x_{19} &= 0. \end{aligned}$$

(9) Desaireador:

$$\begin{aligned} -x_{03} + x_{09} + x_{11} - x_{12} + x_{18} + x_{20} - x_{21} + x_{22} - x_{33} - x_{34} + x_{35} + \\ x_{36} + x_{37} + x_{47} + x_{48} + x_{49} + x_{50} + x_{51} &= 0 \\ -102x_{03} + 40x_{09} + 181x_{11} - 102x_{12} + 640x_{18} + 10x_{20} - 102x_{21} + 686x_{22} - \\ 102x_{33} - 102x_{34} + 134x_{35} + 100x_{36} + 150x_{37} + 126x_{47} + \\ 181x_{48} + 144x_{49} + 144x_{50} + 126x_{51} &= 0. \end{aligned}$$

(10) Atemperador de alta presión, AH:

$$\begin{aligned} x_{23} - x_{24} + x_{33} &= 0 \\ 772x_{23} - 732x_{24} + 102x_{33} &= 0. \end{aligned}$$

(11) Atemperador de media presión, AM:

$$\begin{aligned} x_{25} - x_{26} + x_{34} &= 0 \\ 732x_{25} - 677x_{26} + 102x_{34} &= 0. \end{aligned}$$

(12) Atemperador de baja presión, AB:

$$x_{27} - x_{28} = 0.$$

(13) Tanque flash de baja presión, CB:

$$\begin{aligned} -x_{29} - x_{37} + x_{38} + x_{46} &= 0 \\ -656x_{29} - 150x_{37} + 194x_{38} + 181x_{46} &= 0. \end{aligned}$$

(14) Tanque flash de media presión, CM:

$$\begin{aligned} -x_{30} - x_{38} + x_{45} &= 0 \\ -665x_{30} - 194x_{38} + 255x_{45} &= 0. \end{aligned}$$

(15) Tanque flash de muy baja presión, CMB:

$$\begin{aligned} -x_{31} - x_{35} + x_{42} &= 0 \\ -651x_{31} - 134x_{35} + 181x_{42} &= 0. \end{aligned}$$

(16) Tanque de destilación flash, TDFU:

$$\begin{aligned} -x_{32} + x_{40} - x_{41} &= 0 \\ -656x_{32} + 190x_{40} - 150x_{41} &= 0. \end{aligned}$$

(17) Tanque de condensación flash de aceitosos, TCFA:

$$\begin{aligned} -x_{36} - x_{39} + x_{43} + x_{44} &= 0 \\ -100x_{36} - 640x_{39} + 126x_{43} + 181x_{44} &= 0. \end{aligned}$$

(18) Tanques de agua desionizada:

$$-x_{20} - x_{52} + x_{53} = 0.$$

Ahora formulamos las condiciones de balance de masa para cada una de las cuatro redes de tuberías.

Red a alta presión

$$\begin{aligned} x_{01} - x_{07} + x_{13} - x_{23} - 9,5t_{01} - 3,3t_{02} - 9t_{03} - 4,2t_{04} - 37t_{05} - 4,6t_{06} - 6,6t_{07} - 4,6t_{08} - 0,5t_{09} - 8t_{10} - \\ 6,1t_{11} - 4,4u_{10} - 3,1u_{11} - 11,5u_{12} - 4,4u_{13} - 1,2u_{17} - 0,2u_{19} + 1,3u_{20} + 30u_{21} - 4u_{24} + 8u_{26} - 7,1u_{27} &= 0. \end{aligned}$$

Red a media presión

$$\begin{aligned}
& -x_{10} - x_{15} - x_{16} + x_{24} - x_{25} + x_{30} + 9,5t_{01} + 3,3t_{02} + 9t_{03} + 4,2t_{04} + 37t_{05} - 7,3t_{12} - 7,3t_{13} - 4,8t_{14} - \\
& 3,3t_{15} - 2,7t_{16} - 3,3t_{17} - 2,4t_{18} - 1,3t_{19} - 2,1t_{20} - 2t_{21} - 4,7t_{22} - 2,2t_{23} - 3,8t_{24} - 0,9t_{25} - 0,9t_{26} - \\
& t_{27} - 2t_{28} - 3,5u_{01} - 5,8u_{03} - 16,8u_{04} - 1,8u_{06} - 5,1u_{08} + 12,4u_{11} - 2,6u_{12} + 26,9u_{13} - 0,3u_{14} - \\
& 2,8u_{15} + 18,6u_{17} + 19,8u_{19} + 19,7u_{20} - 10,1u_{21} - 5u_{25} - 0,13u_{26} - 10,9u_{27} = 0.
\end{aligned}$$

Red a baja presión

$$\begin{aligned}
& x_{02} + x_{05} + x_{08} + x_{26} - x_{27} + x_{29} + x_{32} + 4,6t_{06} + 6,6t_{07} + 4,6t_{08} + 0,5t_{09} + 7,3t_{12} + 7,3t_{13} + 4,8t_{14} + \\
& 3,3t_{15} + 2,7t_{16} + 3,3t_{17} + 2,4t_{18} + 1,3t_{19} + 2,1t_{20} - 6,1u_{07} + 5,5u_{11} - 2,52u_{13} - 0,1u_{14} - 52u_{15} + \\
& 2,9u_{17} - 20,1u_{18} - 0,2u_{19} - 1,2u_{20} - 2,82u_{21} - 5,4u_{23} - 2,8u_{24} - 16,33u_{27} = 0.
\end{aligned}$$

Red a muy baja presión

$$\begin{aligned}
& -x_{22} + x_{28} + x_{31} - x_{54} + 8t_{10} + 6,1t_{11} + 2t_{21} + 4,7t_{22} + 2,2t_{23} + 3,8t_{24} + 0,9t_{25} + 0,9t_{26} + t_{27} + 2t_{28} - \\
& 1,5u_{03} - 0,2u_{08} - 1,8u_{09} - 1,2u_{10} - 2,5u_{27} = 0.
\end{aligned}$$

El agua de la caldera principal, AC, el agua desionizada, ADI, el agua de descarga de las calderas de las unidades de producción, AD, y los diversos condensados de esas unidades de producción dan lugar a las siguientes 12 condiciones de conservación de masa.

$$\begin{aligned}
& x_{21} - 4,6u_{01} - 7,5u_{10} - 39u_{11} - 26u_{13} - 4,3u_{15} - 3,8u_{16} - 24u_{17} - 24,5u_{19} - \\
& 28,5u_{20} - 33,5u_{21} - 0,5u_{23} - 0,7u_{24} = 0 \\
& x_{40} - 1,4u_{11} - u_{13} - u_{19} - 1,28u_{20} - 1,3u_{21} - 0,65u_{26} = 0 \\
& x_{42} - 9,3u_{04} = 0 \\
& x_{43} - 0,2u_{08} = 0 \\
& x_{44} - 12,6u_{08} = 0 \\
& x_{45} - 11,5u_{12} - 1,4u_{17} - 4,7u_{24} - 3,3u_{27} = 0 \\
& x_{46} - 0,5u_{11} - 1,3u_{13} - 3,1u_{15} - 0,3u_{19} - 2,1u_{27} = 0 \\
& x_{47} - 3,1u_{27} = 0 \\
& x_{48} - 4,8u_{25} - 0,6u_{27} = 0 \\
& x_{49} - 0,5u_{11} - 2,4u_{13} - 0,1u_{14} - 55,7u_{15} - 1,4u_{17} - 20,1u_{18} - 0,2u_{19} - 1,1u_{20} - 1,82u_{21} - 7,7u_{27} = 0 \\
& x_{50} - 0,3u_{15} - 5,9u_{23} - 2,8u_{24} - 8,63u_{27} = 0 \\
& x_{51} - 6,1u_{07} - 10,2u_{10} - 3,4u_{27} = 0 \\
& x_{52} - 0,5u_{15} - 0,4u_{19} - 1,1u_{24} - 15,6u_{26} = 0.
\end{aligned}$$

La gestión de los combustibles que no son el que se quema en la caldera principal la reflejan las siguientes relaciones:

$$\begin{aligned}
& f_{05} - 1,08u_{04} - 3,77u_{05} - 0,96u_{06} - 0,01u_{08} - 0,25u_{10} - 0,35u_{13} - 0,73u_{17} - 1,14u_{26} - 1,02u_{27} = 0 \\
& f_{04} + f_{05} + f_{06} + f_{07} - 5u_{07} - 3,925u_{08} - 17,103u_{15} - 0,856u_{24} - 1,578u_{26} = 0 \\
& f_{02} + f_{06} - 3,6u_{01} - 6,4u_{03} - 1,36u_{06} - 1,73u_{12} - 0,89u_{14} - 1,59u_{25} = 0 \\
& f_{03} + f_{07} - 5,65u_{11} - 9,22u_{13} - 2,59u_{19} - 2,03u_{20} - 2,34u_{21} = 0 \\
& -f_{02} + 0,89u_{14} \leq 0.
\end{aligned}$$

La producción, consumo y gestión de la energía eléctrica dan lugar a las siguientes ecuaciones:

$$\begin{aligned}
& w_{04} - 0,372t_{01} - 0,116t_{02} - 0,28t_{03} - 0,111t_{04} - 0,045t_{05} - 0,278t_{06} - 0,306t_{07} - 0,204t_{08} - 0,013t_{09} - \\
& 0,46t_{10} - 0,145t_{11} - 0,164t_{12} - 0,144t_{13} - 0,092t_{14} - 0,052t_{15} - 0,036t_{16} - 0,044t_{17} - 0,03t_{18} - \\
& 0,015t_{19} - 0,024t_{20} - 0,184t_{21} - 0,25t_{22} - 0,1t_{23} - 0,15t_{24} - 0,015t_{25} - 0,013t_{26} - 0,011t_{27} - 0,015t_{28} = 0 \\
& w_{05} - 0,8u_{01} - 0,115u_{02} - 1,245u_{03} - 0,82u_{04} - 0,395u_{06} - 0,05u_{07} - 0,22u_{08} - 0,105u_{10} - \\
& 1,62u_{11} - 1,27u_{12} - 4,35u_{13} - 1,13u_{14} - 0,925u_{15} - 0,093u_{16} - 0,685u_{17} - 0,205u_{18} - \\
& 1,14u_{19} - 2,975u_{20} - 6,33u_{21} - 0,002u_{22} - 0,140u_{23} - 0,72u_{24} - 0,565u_{25} - 0,545u_{26} - 9,555u_{27} = 0 \\
& w_{01} + w_{02} + w_{03} + w_{04} - w_{05} - w_{06} = 0.
\end{aligned}$$

El último grupo de condiciones se refiere a las que definen las capacidades operativas de los distintos dispositivos del sistema de producción de vapor de agua y a las que plasman el hecho de

que las turbinas de vapor de las diversas unidades de producción sólo pueden funcionar si dichas unidades lo hacen a plena capacidad. Estas últimas condiciones son fácilmente formulables si se tiene en cuenta que las t_i , $1 \leq i \leq 28$, y u_j , $1 \leq j \leq 27$, sólo pueden adoptar valores 0 ó 1. Si una turbina i pertenece a una unidad de producción j , y se define la condición $t_i - u_j \leq 0$, evidentemente, si $u_j = 0$, t_i debe ser también 0. Si $u_j = 1$, t_i puede ser 0 ó 1. En cualquier caso, recordemos que para satisfacer las necesidades de producción de la refinería supondremos que las unidades de producción funcionan al tope de su capacidad, es decir, $u_j = 1$, $1 \leq j \leq 27$. Más adelante analizaremos qué ocurre si alguna de estas unidades está averiada o parcialmente en mantenimiento. Las condiciones son pues

$$\begin{aligned} x_{07} &\leq 100, 3 \leq x_{09} \leq 60, 20 \leq x_{13} \leq 60, x_{23} \leq 60, x_{25} \leq 50, x_{27} \leq 50 \\ x_{03} + x_{12} + x_{21} + x_{33} + x_{34} &\leq 423,3, f_{04} \leq 10,7 \end{aligned}$$

y

$$\begin{aligned} t_{01} - u_{19} &\leq 0, t_{02} - u_{11} \leq 0, t_{03} - u_{21} \leq 0, t_{04} - u_{26} \leq 0, t_{05} - u_{16} \leq 0, t_{06} - u_{19} \leq 0, t_{07} - u_{11} \leq 0, \\ t_{08} - u_{11} &\leq 0, t_{09} - u_{20} \leq 0, t_{10} - u_{22} \leq 0, t_{11} - u_{22} \leq 0, t_{12} - u_{15} \leq 0, t_{13} - u_{13} \leq 0, t_{14} - u_{13} \leq 0, \\ t_{15} - u_{13} &\leq 0, t_{16} - u_{20} \leq 0, t_{18} - u_{2001} \leq 0, t_{17} - u_{21} \leq 0, t_{19} - u_{21} \leq 0, t_{20} - u_{13} \leq 0, t_{21} - u_{12} \leq 0, \\ t_{22} - u_{16} &\leq 0, t_{23} - u_{16} \leq 0, t_{24} - u_{16} \leq 0, t_{25} - u_{16} \leq 0, t_{26} - u_{12} \leq 0, t_{27} - u_{22} \leq 0, t_{28} - u_{16} \leq 0. \end{aligned}$$

El programa lineal que se plantea como resultado de todas estas restricciones y de la función objetivo formulada antes, tiene 95 variables. De éstas, 55 son binarias 0 ó 1 aunque 28, las correspondientes a las unidades de producción, están fijadas a 1.

Resolución del problema

El modelo que define la función objetivo que se ha descrito y el conjunto de condiciones formulado constituye lo que denominaremos el Modelo 1.

Si el Modelo 1 se traduce a formato MPS, con el objetivo de resolverlo posteriormente con BBMI, el resultado que se obtiene es el que se lista a continuación.

NAME	MODEL-R1	E	c36
ROWS		E	c37
N	obj	E <th>c38</th>	c38
E	c2	L <th>c47</th>	c47
E	c3	E <th>c49</th>	c49
E	c4	E <th>c50</th>	c50
E	c5	E <th>c51</th>	c51
E	c6	E <th>c52</th>	c52
E	c7	E <th>c53</th>	c53
E	c8	E <th>c54</th>	c54
E	c9	E <th>c55</th>	c55
E	c10	E <th>c56</th>	c56
E	c11	E <th>c57</th>	c57
E	c12	E <th>c58</th>	c58
E	c13	E <th>c59</th>	c59
E	c14	E <th>c60</th>	c60
E	c15	E <th>c61</th>	c61
E	c16	E <th>c62</th>	c62
E	c17	E <th>c63</th>	c63
E	c18	E <th>c64</th>	c64
E	c19	E <th>c65</th>	c65
E	c20	E <th>c66</th>	c66
E	c21	E <th>c67</th>	c67
E	c22	E <th>c68</th>	c68
E	c23	E <th>c69</th>	c69
E	c24	L <th>c70</th>	c70
E	c25	E <th>c71</th>	c71
E	c26	E <th>c72</th>	c72
E	c27	E <th>c73</th>	c73
E	c28	L <th>c74</th>	c74
E	c29	L <th>c75</th>	c75
E	c30	L <th>c76</th>	c76
E	c31	L <th>c77</th>	c77
E	c32	L <th>c78</th>	c78
E	c33	L <th>c79</th>	c79
E	c34	L <th>c80</th>	c80
E	c35	L <th>c81</th>	c81

L	c82				x38	c28	1	c29	194
L	c83				x38	c30	-1	c31	-194
L	c84				x39	c36	-1	c37	-640
L	c85				x40	c34	1	c35	190
L	c86				x40	c54	1		
L	c87				x41	c34	-1	c35	-150
L	c88				x42	c32	1	c33	181
L	c89				x42	c55	1		
L	c90				x43	c36	1	c37	126
L	c91				x43	c56	1		
L	c92				x44	c36	1	c37	181
L	c93				x44	c57	1		
L	c94				x45	c30	1	c31	255
L	c95				x45	c58	1		
L	c96				x46	c28	1	c29	181
L	c97				x46	c59	1		
L	c98				x47	c21	1	c22	126
L	c99				x47	c60	1		
L	c100				x48	c21	1	c22	181
L	c101				x48	c61	1		
COLUMNS					x49	c21	1	c22	144
x0	obj	6571.2			x49	c62	1		
x01	c2	-1	c3	1	x50	c21	1	c22	144
x01	c6	-0.025	c49	1	x50	c63	1		
x02	c2	-1	c4	1	x51	c21	1	c22	126
x02	c51	1			x51	c64	1		
x03	c2	1	c21	-1	x52	c38	-1	c65	1
x03	c22	-102	c47	1	x53	c38	1		
x04	c2	-1	c6	1	x54	c52	-1		
x04	c7	1	c8	263	w01	obj	4440	c73	1
x05	c7	-1	c8	-656	w02	c3	-0.617	c4	-0.355
x05	c51	1			w02	c5	-0.206	c73	1
x06	c7	-1	c8	-150	w03	c10	-1	c73	1
x07	c9	1	c49	-1	w04	c71	1	c73	1
x08	c9	-1	c10	0.122	w05	c72	1	c73	-1
x08	c51	1			w06	obj	-3550	c73	-1
x09	c9	-1	c10	0.249	f01	obj	12000	c14	-8540
x09	c21	1	c22	40	f01	c16	-0.3	c18	22.5
x10	c11	1	c12	732	f02	obj	12000	c68	1
x10	c50	-1			f02	c70	-1		
x11	c11	-1	c12	-181	f03	obj	10000	c69	1
x11	c21	1	c22	181	f04	c5	1	c67	1
x12	c12	-28	c13	1	f05	c66	1	c67	1
x12	c14	-130	c21	-1	f06	c67	1	c68	1
x12	c22	-102	c47	1	f07	c67	1	c69	1
x13	c13	-1	c14	772	INT				
x13	c15	-0.025	c49	1	u01	c50	-3.5	c53	-4.6
x14	c17	-1	c18	181	u01	c68	-3.6	c72	-0.8
x15	c16	1	c50	-1	u02	c72	-0.115		
x16	c17	1	c18	-732	u03	c50	-5.8	c52	-1.5
x16	c50	-1			u03	c68	-6.4	c72	-1.245
x17	c13	-1	c14	263	u04	c50	-16.8	c55	-9.3
x17	c15	1	c19	1	u04	c66	-1.08	c72	-0.82
x17	c20	263			u05	c66	-3.77		
x18	c19	-1	c20	-640	u06	c50	-1.8	c66	-0.96
x18	c21	1	c22	640	u06	c68	-1.36	c72	-0.395
x19	c19	-1	c20	-100	u07	c51	-6.1	c64	-6.1
x20	c21	1	c22	10	u07	c67	-5	c72	-0.05
x20	c38	-1			u08	c50	-5.1	c52	-0.2
x21	c21	-1	c22	-102	u08	c56	-0.2	c57	-12.6
x21	c47	1	c53	1	u08	c66	-0.01	c67	-3.925
x22	c21	1	c22	686	u08	c72	-0.22		
x22	c52	-1			u09	c52	-1.8		
x23	c23	1	c24	772	u10	c49	-4.4	c52	-1.2
x23	c49	-1			u10	c53	-7.5	c64	-10.2
x24	c23	-1	c24	-732	u10	c66	-0.25	c72	-0.105
x24	c50	1			u11	c49	-3.1	c50	12.4
x25	c25	-1	c26	732	u11	c51	5.5	c53	-39
x25	c50	-1			u11	c54	-1.4	c59	-0.5
x26	c25	-1	c26	-677	u11	c62	-0.5	c69	-5.65
x26	c51	1			u11	c72	-1.62	c75	-1
x27	c27	1	c51	-1	u11	c80	-1	c81	-1
x28	c27	-1	c52	1	u12	c49	-11.5	c50	-2.6
x29	c28	-1	c29	-656	u12	c58	-11.5	c68	-1.73
x29	c51	1			u12	c72	-1.27	c94	-1
x30	c30	-1	c31	-665	u12	c99	-1		
x30	c50	1			u13	c49	-4.4	c50	26.9
x31	c32	-1	c33	-651	u13	c51	-2.52	c53	-26
x31	c52	1			u13	c54	-1	c59	-1.3
x32	c34	-1	c35	-656	u13	c62	-2.4	c66	-0.35
x32	c51	1			u13	c69	-9.22	c72	-4.35
x33	c21	-1	c22	-102	u13	c86	-1	c87	-1
x33	c23	1	c24	102	u13	c88	-1	c93	-1
x33	c47	1			u14	c50	-0.3	c51	-0.1
x34	c21	-1	c22	-102	u14	c62	-0.1	c68	-0.89
x34	c25	1	c26	102	u14	c70	0.89	c72	-1.13
x34	c47	1			u15	c50	-2.8	c51	-52
x35	c21	1	c22	134	u15	c53	-4.3	c59	-3.1
x35	c32	-1	c33	-134	u15	c62	-55.7	c63	-0.3
x36	c21	1	c22	100	u15	c65	-0.5	c67	-17.103
x36	c36	-1	c37	-100	u15	c72	-0.925	c85	-1
x37	c21	1	c22	150	u16	c53	-3.8	c72	-0.093
x37	c28	-1	c29	-150	u16	c78	-1	c95	-1

u16	c96	-1	c97	-1	t21	c50	-2	c52	2
u16	c98	-1	c101	-1	t21	c71	-0.184	c94	1
u17	c49	-1.2	c50	18.6	t22	c50	-4.7	c52	4.7
u17	c51	2.9	c53	-24	t22	c71	-0.25	c95	1
u17	c58	-1.4	c62	-1.4	t23	c50	-2.2	c52	2.2
u17	c66	-0.73	c72	-0.685	t23	c71	-0.1	c96	1
u18	c51	-20.1	c62	-20.1	t24	c50	-3.8	c52	3.8
u18	c72	-0.205			t24	c71	-0.15	c97	1
u19	c49	-0.2	c50	19.8	t25	c50	-0.9	c52	0.9
u19	c51	-0.2	c53	-24.5	t25	c71	-0.015	c98	1
u19	c54	-1	c59	-0.3	t26	c50	-0.9	c52	0.9
u19	c62	-0.2	c65	-0.4	t26	c71	-0.013	c99	1
u19	c69	-2.59	c72	-1.14	t27	c50	-1	c52	1
u19	c74	-1	c79	-1	t27	c71	-0.011	c100	1
u20	c49	1.3	c50	19.7	t28	c50	-2	c52	2
u20	c51	-1.2	c53	-28.5	t28	c71	-0.015	c101	1
u20	c54	-1.28	c62	-1.1	RHS				
u20	c69	-2.03	c72	-2.975	rhs	c3	35.2		
u20	c82	-1	c89	-1	rhs	c4	12.7		4.199
u20	c90	-1			rhs	c10	1.143	c5	423.3
u21	c49	30	c50	-10.1	BOUNDS				
u21	c51	-2.82	c53	-33.5	FX bnd	x0	1		
u21	c54	-1.3	c62	-1.82	UP bnd	x07	100		
u21	c69	-2.34	c72	-6.33	LO bnd	x09	3		
u21	c76	-1	c91	-1	UP bnd	x09	60		
u21	c92	-1			LO bnd	x13	20		
u22	c72	-0.002	c83	-1	UP bnd	x13	60		
u22	c84	-1	c100	-1	UP bnd	x23	60		
u23	c51	-5.4	c53	-0.5	UP bnd	x25	50		
u23	c63	-5.9	c72	-0.14	UP bnd	x27	50		
u24	c49	-4	c51	-2.8	UP bnd	f04	10.7		
u24	c53	-0.7	c58	-4.7	FX bnd	u01	1		
u24	c63	-2.8	c65	-1.1	FX bnd	u02	1		
u24	c67	-0.856	c72	-0.72	FX bnd	u03	1		
u25	c50	-5	c61	-4.8	FX bnd	u04	1		
u25	c68	-1.59	c72	-0.565	FX bnd	u05	1		
u26	c49	8	c50	0.13	FX bnd	u06	1		
u26	c54	-0.65	c65	-15.6	FX bnd	u07	1		
u26	c66	-1.14	c67	-1.578	FX bnd	u08	1		
u26	c72	-0.545	c77	-1	FX bnd	u09	1		
u27	c49	-7.1	c50	-10.9	FX bnd	u10	1		
u27	c51	-16.33	c52	-2.5	FX bnd	u11	1		
u27	c58	-3.3	c59	-2.1	FX bnd	u12	1		
u27	c60	-3.1	c61	-0.6	FX bnd	u13	1		
u27	c62	-7.7	c63	-8.63	FX bnd	u14	1		
u27	c64	-3.4	c66	-1.02	FX bnd	u15	1		
u27	c72	-9.555			FX bnd	u16	1		
t01	c49	-9.5	c50	9.5	FX bnd	u17	1		
t01	c71	-0.372	c74	1	FX bnd	u18	1		
t02	c49	-3.3	c50	3.3	FX bnd	u19	1		
t02	c71	-0.116	c75	1	FX bnd	u20	1		
t03	c49	-9	c50	9	FX bnd	u21	1		
t03	c71	-0.28	c76	1	FX bnd	u22	1		
t04	c49	-4.2	c50	4.2	FX bnd	u23	1		
t04	c71	-0.111	c77	1	FX bnd	u24	1		
t05	c49	-37	c50	37	FX bnd	u25	1		
t05	c71	-0.045	c78	1	FX bnd	u26	1		
t06	c49	-4.6	c51	4.6	FX bnd	u27	1		
t06	c71	-0.278	c79	1	UP bnd	t01	1		
t07	c49	-6.6	c51	6.6	UP bnd	t02	1		
t07	c71	-0.306	c80	1	UP bnd	t03	1		
t08	c49	-4.6	c51	4.6	UP bnd	t04	1		
t08	c71	-0.204	c81	1	UP bnd	t05	1		
t09	c49	-0.5	c51	0.5	UP bnd	t06	1		
t09	c71	-0.013	c82	1	UP bnd	t07	1		
t10	c49	-8	c52	8	UP bnd	t08	1		
t10	c71	-0.46	c83	1	UP bnd	t09	1		
t11	c49	-6.1	c52	6.1	UP bnd	t10	1		
t11	c71	-0.145	c84	1	UP bnd	t11	1		
t12	c50	-7.3	c51	7.3	UP bnd	t12	1		
t12	c71	-0.164	c85	1	UP bnd	t13	1		
t13	c50	-7.3	c51	7.3	UP bnd	t14	1		
t13	c71	-0.144	c86	1	UP bnd	t15	1		
t14	c50	-4.8	c51	4.8	UP bnd	t16	1		
t14	c71	-0.092	c87	1	UP bnd	t17	1		
t15	c50	-3.3	c51	3.3	UP bnd	t18	1		
t15	c71	-0.052	c88	1	UP bnd	t19	1		
t16	c50	-2.7	c51	2.7	UP bnd	t20	1		
t16	c71	-0.036	c89	1	UP bnd	t21	1		
t17	c50	-3.3	c51	3.3	UP bnd	t22	1		
t17	c71	-0.044	c91	1	UP bnd	t23	1		
t18	c50	-2.4	c51	2.4	UP bnd	t24	1		
t18	c71	-0.03	c90	1	UP bnd	t25	1		
t19	c50	-1.3	c51	1.3	UP bnd	t26	1		
t19	c71	-0.015	c92	1	UP bnd	t27	1		
t20	c50	-2.1	c51	2.1	UP bnd	t28	1		
t20	c71	-0.024	c93	1	ENDATA				

En la tabla D.6, columna 2, se describe la solución completa del Modelo 1. La producción de energía eléctrica de la refinería que se vende a la compañía eléctrica cada hora es 8,427

MWh. En la columna 6, Modelo 1', se representa la solución del mismo Modelo 1 sin tener en cuenta que las variables t_i son enteras 0 ó 1.

D.2.4 Análisis de sensibilidad

Si se deseara optimizar la misma función objetivo añadiendo la condición de que no se venda energía eléctrica, bastaría hacer $w_{06} = 0$. Este problema lo denominamos Modelo 2. Su solución se describe en la tabla D.6, columna 3. Como se puede observar, la nueva restricción supone unas pérdidas a la refinería de $310.929,1 - 298.506,9 = 12.422$, pesetas por hora. La energía eléctrica que se produce en los turbogeneradores de gas y vapor se reduce sustancialmente, mientras que la de las turbinas de vapor es prácticamente la misma. La solución de la columna 7, Modelo 2', es la del Modelo 2 sin restringir que las variables t_i sean 0 ó 1.

También puede ser interesante saber la máxima cantidad de energía eléctrica que se puede vender cada hora en condiciones normales. Para ello, basta con cambiar la función objetivo haciendo la nueva igual a

$$\text{maximizar } w_{06}$$

y forzar a que $w_{01} = 0$. Si se designa este nuevo problema como Modelo 3, la solución del mismo se describe en la columna 4 de la tabla D.6. La máxima cantidad de energía que se puede vender es 16,260 MW. La producción de los turbogeneradores de gas es la misma que en el Modelo 1 mientras que la de los turbogeneradores de vapor y las turbinas de las unidades de producción prácticamente se duplica. En la columna 8 de la tabla D.6, bajo Modelo 3', se describe la solución del Modelo 3 sin restringir que las variables t_i adopten valores 0 ó 1.

Una situación habitual que también se puede estudiar fácilmente es la que se presenta cuando alguna de las unidades de producción no se puede utilizar al 100% de su capacidad. Si por ejemplo la unidad número 15 estuviese al 50% de su capacidad, haciendo $u_{15} = 0,5$ se obtendría el Modelo 4. Su solución es la de la columna 5 de la tabla D.6. El coste de operación sería 389.970,5 pesetas por hora. Esto supondría un incremento respecto del óptimo de 91.463,6 pesetas por hora. La explicación de esto reside en el hecho de que la unidad de producción número 15 es la que más gas produce. Si se reduce a la mitad, $17.103/2 = 8.551,5$ Tn/h, este combustible hay que remplazarlo con fuel con lo que esto representa en coste pues estos últimos combustibles se pueden vender mientras que el gas, como ya se indicó, no. De la solución se deduce que los hogares que pueden quemar gas o fuel de tipo 1 pasan a quemar sólo fuel, incrementándose el consumo de éste de 7.118 Tn/h a 15.570 Tn/h.

Tabla D.6

Soluciones óptimas de los diversos modelos del problema de la refinería

Vari.	Modelo 1	Modelo 2	Modelo 3	Modelo 4	Modelo 1'	Modelo 2'	Modelo 3'
Coste	298.506,9	310.929,1	16,260	389.970,5	298.450,2	310.877,4	16,296
x_0	1,000	1,000		1,000	1,000	1,000	
x_{01}	54,671	50,305	54,671	54,373	54,671	50,292	54,671
x_{02}	23,903	21,390	23,903	23,731	23,903	21,383	23,903
x_{03}	79,941	72,953	79,941	79,464	79,941	72,932	79,941
x_{04}	1,366	1,257	1,366	1,359	1,366	1,257	1,366
x_{05}	0,305	0,280	0,305	0,303	0,305	0,280	0,305
x_{06}	1,061	0,976	1,061	1,055	1,061	0,976	1,061
x_{07}	78,071	73,705	100,000	77,773	78,071	73,692	100,000
x_{09}	27,173	52,811	40,000	28,303	27,106	52,832	40,000
x_{08}	50,897	20,893	60,000	49,469	50,965	20,859	60,000
x_{10}	1,041	1,041	3,122	1,041	1,041	1,041	3,125
x_{11}	1,041	1,041	3,122	1,041	1,041	1,041	3,125
x_{12}	20,500	20,500	61,449	20,500	20,500	20,500	61,500
x_{13}	20,000	20,000	59,950	20,000	20,000	20,000	60,000
x_{14}	0,061	0,061	0,184	0,061	0,061	0,061	0,185
x_{15}	0,453	0,453	1,359	0,453	0,453	0,453	1,360
x_{16}	0,061	0,500	1,498	0,500	0,061	0,500	1,500
x_{17}	0,500	0,061	0,184	0,061	0,500	0,061	0,185
x_{18}	0,150	0,150	0,452	0,150	0,150	0,150	0,452
x_{19}	0,349	0,349	1,046	0,349	0,349	0,349	1,047
x_{20}	85,191	85,106	87,565	85,185	85,191	85,106	87,600
x_{21}	196,900	196,900	196,900	194,750	196,900	196,900	196,900
x_{22}	4,189	3,509	7,489	8,689	4,182	3,505	7,494
x_{23}	0,000	0,000	0,007	0,000	0,000	0,000	0,000
x_{24}	0,000	0,017	0,000	0,036	0,000	0,000	0,000
x_{25}	0,779	8,454	8,454	8,454	0,000	8,454	8,454
x_{26}	0,854	10,900	10,900	10,900	0,000	10,900	10,900
x_{27}	0,044	23,463	23,463	22,008	0,000	23,463	23,463
x_{28}	0,044	18,193	18,193	18,193	0,000	18,193	18,193
x_{29}	2,029	1,899	1,899	1,899	2,029	1,899	1,899
x_{30}	2,706	9,300	9,300	9,300	2,706	9,300	9,300
x_{31}	0,845	12,600	12,600	12,600	0,845	12,600	12,600
x_{32}	0,524	7,300	7,300	5,750	0,524	7,300	7,300
x_{33}	0,000	0,000	0,122	0,000	0,000	0,000	0,000
x_{34}	0,074	0,000	0,129	0,000	0,000	0,000	0,000
x_{35}	8,454	0,179	0,000	0,379	8,454	0,000	0,000
x_{36}	10,900	0,197	0,000	0,416	10,900	0,000	0,000
x_{37}	23,463	0,064	1,791	1,443	23,463	0,000	0,000
x_{38}	18,193	6,630	6,630	6,630	18,193	6,630	6,630
x_{39}	1,899	91,020	91,020	63,170	1,899	91,020	91,020
x_{40}	6,630	3,100	3,100	3,100	6,630	3,100	3,100
x_{41}	6,105	5,400	5,400	5,400	6,105	5,400	5,400
x_{42}	9,300	20,900	20,900	20,900	9,300	20,900	20,900
x_{43}	0,200	17,630	17,630	17,480	0,200	17,630	17,630
x_{44}	12,600	19,700	19,700	19,700	12,600	19,700	19,700
x_{45}	20,900	0,200	0,200	0,200	20,900	0,200	0,200
x_{46}	7,300	6,105	6,105	6,105	7,300	6,105	6,105

continúa en la siguiente página

<i>continuación</i>							
Vari.	Modelo 1	Modelo 2	Modelo 3	Modelo 4	Modelo 1'	Modelo 2'	Modelo 3'
t_{03}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{04}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{05}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{06}	0,000	0,000	1,000	0,000	0,000	0,000	1,000
t_{07}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{08}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{09}	0,000	0,000	1,000	0,000	0,000	0,000	0,000
t_{10}	0,000	0,000	0,000	0,000	0,000	0,000	0,228
t_{11}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{12}	1,000	1,000	1,000	0,000	1,000	1,000	1,000
t_{13}	1,000	1,000	1,000	1,000	1,000	1,000	1,000
t_{14}	1,000	1,000	1,000	1,000	1,000	1,000	1,000
t_{15}	1,000	1,000	1,000	1,000	1,000	1,000	1,000
t_{16}	0,000	0,000	1,000	1,000	0,000	0,525	1,000
t_{17}	0,000	0,000	1,000	1,000	0,225	0,000	1,000
t_{18}	0,000	0,000	1,000	0,000	0,000	0,000	0,461
t_{19}	0,000	1,000	0,000	0,000	0,000	0,000	0,000
t_{20}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{21}	1,000	1,000	1,000	1,000	1,000	1,000	1,000
t_{22}	1,000	1,000	1,000	1,000	1,000	1,000	1,000
t_{23}	0,000	1,000	1,000	1,000	1,000	1,000	1,000
t_{24}	1,000	0,000	1,000	1,000	0,430	0,252	1,000
t_{25}	0,000	1,000	0,000	1,000	0,000	0,000	0,000
t_{26}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{27}	0,000	0,000	0,000	0,000	0,000	0,000	0,000
t_{28}	0,000	0,000	0,000	0,000	0,000	0,000	0,000

Referencias

En cualquier buen libro de programación lineal se pueden encontrar ejemplos más o menos complejos que pueden ayudar al lector a familiarizarse con la formulación de problemas de programación lineal a partir de diversos casos reales. Los dos que se han presentado en este apéndice se han obtenido de Padberg [1995].

Apéndice E

El PROGRAMA BBMI

E L PROGRAMA BBMI está diseñado para la resolución de grandes problemas de programación lineal o programación entera del siguiente tipo:

$$\begin{aligned} &\text{minimizar o maximizar } \mathbf{c}^T \mathbf{x} + \mathbf{h}^T \mathbf{y} \\ &\text{sujeta a } \quad \mathbf{Ax} + \mathbf{Gy} \leq \mathbf{b} \\ &\quad \quad \quad \mathbf{l}_x \leq \mathbf{x} \leq \mathbf{u}_x \\ &\quad \quad \quad \mathbf{l}_y \leq \mathbf{y} \leq \mathbf{u}_y \\ &\quad \quad \quad \mathbf{x} \in \Re^n, \mathbf{y} \in \mathbf{Z}^p. \end{aligned} \tag{E.1}$$

La versión que se lista más adelante está adaptada para su uso en ordenadores personales, donde la limitación de memoria en algunos casos puede ser un condicionante importante. Con ella se han resuelto problemas lineales de más de 2.000 variables y 1.000 condiciones.

La sencillez de entrada de datos al programa, y por ende el manejo del mismo, es una de las características primordiales que perseguía su elaboración. Esta entrada de datos está dividida en dos bloques: aquellos que se refieren a los datos del problema a estudiar, para lo cual se utiliza en toda su extensión el formato estándar MPS (Mathematical Programming Systems), ampliado con la palabra clave INT para definir qué variables han de ser enteras; y los que especifican los parámetros que se desea que utilice el programa para resolver el problema que haya que estudiar.

E.1 Datos del problema. Formato MPS

Los datos del problema que se quiera que BBMI resuelva deberán estar en un fichero cuyo nombre el propio programa demandará por pantalla al comienzo de la ejecución mediante el requerimiento:

Fichero de datos del problema?

En este fichero, como decíamos, los datos del problema han de suministrarse siguiendo el formato Mathematical Programming Systems. Los **resultados** y el proceso de optimización aparecerán en un fichero que el programa automáticamente designa como **nombre.pbb**, donde *nombre* es el introducido por pantalla. En el sistema operativo MS/DOS ese nombre estará limitado a 8 caracteres.

Mediante el formato MPS se le suministran al programa las características y parámetros del programa lineal que tenga que resolver, a partir de ciertas palabras cabecera clave que dividen al fichero en varias secciones según se esquematiza a continuación.

```
NAME
ROWS
:
COLUMNS
:
INT          No estándar
:
RHS
:
RANGES      Opcional
:
BOUNDS      Opcional
:
ENDATA
```

La disposición y orden de estas secciones de datos en el fichero correspondiente han de ser las indicadas. Cada una de las palabras clave cabecera debe comenzar en la columna 1. Los datos que conforman cada sección deben seguir el siguiente formato en cada línea:

Columnas	2-3	5-12	15-22	25-36	40-47	50-61	(E.2)
Contenido	Clave	Nombre	Nombre	Valor	Nombre	Valor	

También se permiten líneas de comentarios siempre que comiencen por el carácter * en la columna 1.

El formato MPS es sin duda el más extendido entre los códigos comerciales que resuelven problemas de optimización. Para el tratamiento de programas enteros hemos incorporado la palabra clave INT ya mencionada (aunque se usa en algunos programas comerciales para la resolución de programas enteros, no es estándar), con el fin de informar a BBMI sobre qué variables debe considerar como enteras. Todo lo demás sigue estrictamente las directrices de ese formato.

E.1.1 Clave NAME

Mediante esta palabra clave, con el formato¹ del siguiente ejemplo,

¹El símbolo □ designa un espacio en blanco.

NAME_____PROBLE1

se le indica a BBMI el nombre del problema que se va a resolver (en el caso del ejemplo, PROBLE1). Este nombre aparecerá al comienzo de cada bloque de información significativa en la salida o fichero de resultados.

Obsérvese que NAME debe comenzar **en la primera columna** y el **nombre** propiamente dicho **en la 15**, pudiéndose continuar hasta la 22. Todo lo posterior a la columna 22, aun cuando puede servir de orientación al usuario, no será tenido en cuenta por el programa.

NAME debe ser la primera palabra clave del formato MPS que se debe incluir en el fichero de datos. Puede estar precedida de comentarios y, como veremos más adelante, por información relativa a los parámetros de cómo se quiere que el programa se resuelva el problema.

E.1.2 Sección ROWS

En esta sección, un ejemplo de la cual puede ser

```
ROWS
  N___FILA1
  G___FILA2
  L___FILA3
  E___FILA4
```

se deben definir las condiciones o restricciones del problema y el tipo de éstas. Se debe incluir una línea para cada condición. En la **columna dos o en la tres** de cada una de esas líneas deberá aparecer una **clave** por medio de la cual se indique el tipo de condición que se está definiendo. A partir de la **columna cinco**, y ocupando como máximo **hasta la doce**, se definirá el **nombre** que se quiere asignar a dicha condición. Los distintos tipos de condición que se pueden definir y sus palabras clave asociadas son las que siguen.

Clave	Condición Tipo
E	=
G	≥
L	≤
N	Función Objetivo
N	Libre

Las condiciones de los tipos E, G y L se usan para definir restricciones de la forma

$$\mathbf{a}^T \mathbf{x} = b_1 \quad \mathbf{a}^T \mathbf{x} \geq b_1 \quad \text{y} \quad \mathbf{a}^T \mathbf{x} \leq b_1,$$

respectivamente. Los coeficientes distintos de cero de estas condiciones aparecerán en la sección correspondiente a COLUMNS.

El tipo de condición N se puede referir bien a una condición que no tiene límites, y que por lo tanto no es una condición como tal ($-\infty \leq \mathbf{c}^T \mathbf{x} \leq \infty$ no es realmente una condición), o a la propia función objetivo.

E.1.3 Sección COLUMNS

En esta sección se definen los coeficientes distintos de cero de las condiciones previamente definidas en la sección ROWS. Un ejemplo de esta sección de datos es el que sigue.

```

1  5____12  15____22  25____36  40____47  50____61  (Posición)
COLUMNS
X01      FILA3      1.2              FILA16      -33.12
X01      FILA19     14.2              FILA12      1123.4533128
X02      FILA1       1              FILA2       3.5
X02      FILA1     88.E02
X06      FILA6       1              FILA7      99.99
X06      FILA89     1.33E12

```

Para cada variable de un problema, x_j , en COLUMNS se define el nombre que se le quiere dar y el valor de los coeficientes distintos de cero, a_{ij} , correspondientes a la columna que define esa variable en la matriz A o G de las condiciones del problema (E.1). **Todos los coeficientes distintos de cero de una columna/variable se deben especificar antes de pasar a los correspondientes a otra.** Si una columna posee varios elementos distintos de cero, no importa el orden en que se definan.

La información de las columnas 1 a 4, una vez leída la palabra clave COLUMNS, no se utiliza, salvo que se desee incorporar en ellas algún comentario. Las **columnas 5 a 12** definirán el **nombre de la columna/variable** de la que se va a informar al programa sobre cuáles son sus **elementos distintos de cero**. Las **columnas 15 a 22** indican el nombre de una **fila previamente definida** en la que existe un coeficiente distinto de cero que afecta a la variable antes dicha. En las **columnas 25 a 36** se habrá de dar el **valor numérico** de ese coeficiente. En las **columnas 40 a 47** y **50 a 61** se puede **repetir** el mismo patrón. Si las **columnas 15 a 22** y **40 a 47** se dejan **en blanco**, los **valores** de las columnas 25 a 36 y 50 a 61 se **ignorarán**.

Es conveniente saber que los valores numéricos BBMI los lee con formato F12.0. Aunque de esta forma la disposición de los espacios en blanco en el campo reservado para el número no debe plantear ningún problema, conviene comprobar a qué atenerse al respecto en la máquina que se utilice. En principio, es lo mismo escribir _____1123.45, ____1.12345E03, _1123.45_____ ó 1.12345E3____. De igual manera, es lo mismo 120_____, ____120_____ y _____120.

En el ejemplo incluido líneas más arriba, la variable X01 tiene cuatro coeficientes distintos de cero en las filas FILA3, FILA12, FILA16 y FILA19.

E.1.3.1 Clave INT

Esta palabra clave debe interrumpir la entrada normal de variables/columnas en la sección COLUMNS **a partir del momento en el que todas las demás variables que se van a**

definir sean enteras. Si en un ejemplo como el anterior se encuentra

1	5	12	15	22	25	36	40	47	50	61	(Posición)
COLUMNS											
	X01		FILA3		1.2			FILA16		-33.12	
	X01		FILA19		14.2			FILA12		1123.4533128	
	X02		FILA1		1			FILA2		3.5	
	X02		FILA1		88.E02						
INT											
	X06		FILA6		1			FILA7		99.99	
	X06		FILA89		1.33E12						

la variable X06 y las que se definan posteriormente serán las que estén restringidas a tomar valores enteros.

Es **importante** recalcar que la formulación del problema se habrá de llevar a cabo de tal manera que las **variables enteras** del mismo estén dispuestas **en las últimas columnas de la matriz de condiciones**.

E.1.4 Sección RHS

En esta sección se especifican los valores del vector $\mathbf{b}$ de un problema como (E.1). Un ejemplo sería:

1	5	12	15	22	25	36	40	47	50	61	(Posición)
RHS											
	RH1		FILA1		1			FILA09		-3	
	RH1		FILA19		14.2			FILA12		1123.4533128	
	RH1		FILA4		1						
	RH1		FILA5		10.E12						

Los principios sobre orden de los datos, valores, etc, son exactamente los mismos que los indicados en la sección COLUMNS.

Solamente es necesario especificar al programa los elementos del vector $\mathbf{b}$ que son distintos de cero.

E.1.5 Sección RANGES

Los márgenes se usan para caracterizar condiciones de la forma

$$l \leq \mathbf{a}^T \mathbf{x} \leq u, \quad (\text{E.3})$$

sin que sea necesario definir las dos condiciones $l \leq \mathbf{a}^T \mathbf{x}$ y $\mathbf{a}^T \mathbf{x} \leq u$ por separado. El margen de una condición del tipo (E.3) es $r = u - l$. Si se desea informar al programa que una determinada condición tiene margen de valores, habrá que definir en la sección RHS uno de sus límites, l o u , y en ésta el valor de r . Los l y u que resultarán de definir un margen dependerán del tipo de condición que tenga ese margen y del signo de r . La tabla que sigue ilustra los posibles

resultados.

<i>Condición</i>	<i>Tipo</i>	<i>Signo de r</i>	Límite Inferior l	Límite Superior u
E		+	b	$b + r $
E		-	$b - r $	b
G		+ o -	b	$b + r $
L		+ o -	$b - r $	b

Un ejemplo completo puede ser

1 5 12 15 22 25 36 40 47 50 61 (Posición)

ROWS

E FUN01

E FUN02

G CAPITAL1

L CAPITAL2

⋮

COLUMNS

X01 FUN01 1.2 FUN02 -33.12

X01 CAPITAL1 14.2 CAPITAL2 1123.4533128

X02 FUN01 1 FUN02 3.5

X02 CAPITAL1 88.E02

⋮

INT

X06 FUN01 1 FUN02 99.99

X06 CAPITAL1 1.33E12

⋮

RHS

RH1 FUN01 4.2

RH1 FUN02 -3.12

RH1 CAPITAL1 4

RH1 CAPITAL2 5

⋮

RANGES

RANGE01 FUN01 1 FUN02 -3

RANGE01 CAPITAL1 2 CAPITAL2 1

Las restricciones de este ejemplo tienen los siguientes límites:

$$\begin{array}{rclclcl}
 4,20 & \leq & \text{FUN01} & \leq & 5,20 \\
 -6,12 & \leq & \text{FUN02} & \leq & -3,12 \\
 4,00 & \leq & \text{CAPITAL1} & \leq & 6,00 \\
 4,00 & \leq & \text{CAPITAL2} & \leq & 5,00.
 \end{array}$$

El orden de los datos, valores, etc, siguen las mismas directrices que las indicadas en la sección COLUMNS.

E.1.6 Sección BOUNDS

En esta sección se definen los límites o cotas entre los que estarán definidas las variables del problema.

Los valores **por defecto** que el programa asigna a todas las variables x_j son **0 y ∞** ; es decir $0 \leq x_j \leq \infty$. La realidad es que, trabajando en precisión finita, no tiene mucho sentido hablar de ∞ , sino de una cantidad muy grande con respecto a las demás: 10^{20} . Si no desea que los límites de las variables sean los que asigna el programa, se pueden definir en esta sección.

En la sección BOUNDS, la *clave* de las columnas 2 a 3 del formato general (E.2), se refiere al tipo de límite que se va a definir para la variable cuyo nombre aparecerá entre las columnas 15 a 22. Un ejemplo de esta sección es el que sigue.

```

1      5_____12   15_____22   25_____36   40_____47   50_____61   (Posición)
BOUNDS
UP BOUND1      X01      4
LO BOUND1      X01     -8
UP BOUND1      X02      9
:
UP BOUND1      X05     60
LO BOUND1      X05     -2
:
FR BOUND1      X07
UP BOUND1      X07      1
FX BOUND1      X10      5

```

Las **posibles claves** que pueden aparecer en las **columnas 2 y 3** son las de la tabla siguiente.

Clave	Límite Tipo	Límites Resultantes
LO	Inferior	$b \leq x \leq u$
UP	Superior	$l \leq x \leq b$
FX	Fijo	$b \leq x \leq b$
FR	Libre	$-\infty \leq x \leq +\infty$
MI	Menos ∞	$-\infty \leq x \leq u$
PL	Más ∞	$l \leq x \leq +\infty$

El valor b indica la cantidad que se introduce en las columnas 25 a 36 y u y l otros límites ya/o por suministrar al programa, o los que él asigna por defecto.

El efecto en los límites internos que el programa asigna a las variables del ejemplo anterior es:

$$\begin{array}{rclclcl}
 -8 & \leq & X01 & \leq & 4 \\
 -\infty & \leq & X02 & \leq & 9 \\
 -2 & \leq & X05 & \leq & 60 \\
 -\infty & \leq & X07 & \leq & 1 \\
 5 & \leq & X10 & \leq & 5.
 \end{array}$$

Insistimos en que al referirnos a ∞ el ordenador lo interpreta como una cantidad muy grande ($\approx 10^{20}$).

Antes de finalizar este apartado es importante aclarar que, **caso de necesitarse definir las dos secciones de datos RANGES y BOUNDS**, es aconsejable que RANGES preceda a BOUNDS.

E.2 Parámetros y especificaciones de la resolución

En esta sección se consideran los parámetros variables que se pueden especificar al programa a fin de modificar ligeramente la forma en que se lleva a cabo el proceso de optimización del problema que se va a estudiar.

La definición de estos parámetros de lleva a cabo mediante unas palabras clave que a continuación exponemos. Caso de ser necesarias, se han de incluir en el **mismo fichero de datos** del problema a resolver y **antes** de estos: es decir, inmediatamente **antes** de la palabra clave NAME.

Los parámetros son los que se listan seguidamente:

MAXIMIZAR Se refiere a que el problema cuyos datos siguen tiene una función objetivo que se ha de maximizar. Si no se especifica, **el programa asume por defecto que la función objetivo del problema se ha de minimizar**.

FRECUENCIA DE REINVERSION nnn Se refiere a la frecuencia nnn con la que se quiere que se reinvierta la base. **Por defecto, el programa lo hará cada 50 iteraciones**.

LIMITE DE ITERACIONES nnn Se refiere al número máximo de iteraciones simplex nnn que se permite realizar al programa. **Por defecto, el programa realiza un número de iteraciones máximo igual a 100.000**.

TOLERANCIA DE CERO xxx.x Se refiere al número que identifica cuándo otro cualquiera se puede considerar cero. Es decir si un número x es $-Tol \leq x \leq Tol$, a los efectos de ciertos cálculos, el programa lo considera como si fuese cero. El programa asume **por defecto** el valor $1,0 \times 10^{-6}$.

TOLERANCIA DE COSTES xxx.x Se refiere al número que identifica cuándo un coste reducido se considera positivo o negativo. Es decir, si un coste reducido, $\bar{c}_j$, es $\bar{c}_j \geq Tol$, el programa lo considerará positivo; si $\bar{c}_j \leq -Tol$, negativo. Si $-Tol < \bar{c}_j < Tol$, cero. El programa asume **por defecto** un valor $1,0 \times 10^{-6}$.

TOLERANCIA DE PIVOTE xxx.x Se refiere al número que identifica cuándo un elemento pivote se considera cero. Este elemento pivote se refiere al y_p del paso 3 del algoritmo de la tabla 7.1 de la página 420, en la que se expone el método simplex revisado. **Por defecto** el programa asume una tolerancia igual a $3,6 \times 10^{-11}$.

PROCESO ITERATIVO nnn Esta especificación se deberá hacer constar cuando se desee que el programa explice completamente en la unidad nnn el proceso de resolución de un **programa entero**; es decir que informe en todo momento sobre qué variables de separación utiliza, en qué nudo del árbol enumerativo está, si las iteraciones son del método primal del simplex o dual, etc. No es aconsejable su inclusión si se desea que la salida de ordenador no sea muy extensa. Si no se indica ninguna unidad nnn, pero sí las palabras clave **PROCESO ITERATIVO**, el programa incluirá esa información en el mismo fichero de salida, Nombre.pbb, intercalándola allí donde sea necesario.

COTA INICIAL F.O. nnn Esta especificación se deberá hacer constar cuando se desee orientar al programa sobre cuál puede ser una **cota inferior de la función objetivo**, caso de estar **maximizando**, o **cota superior** caso de estar **minimizando**, cuando se resuelva un **programa entero**. Su inclusión puede disminuir el tiempo total de resolución de forma apreciable; sobre todo si es próxima al valor final. Si no se incluye, el programa no asume ninguna.

E.3 Resultados

E.3.1 Programas lineales

Como ejemplo de las posibilidades del programa BBMI, primero resolveremos el programa lineal del ejemplo 7.10 de la página 453:

$$\begin{aligned}
 &\text{min. } -2x_1 - 4x_2 - x_3 \\
 &\text{s. a } \begin{aligned}
 &2x_1 + x_2 + x_3 \leq 10 \\
 &x_1 + x_2 - x_3 \leq 4 \\
 &0 \leq x_1 \leq 4 \\
 &0 \leq x_2 \leq 6 \\
 &1 \leq x_3 \leq 4.
 \end{aligned}
 \end{aligned}$$

El fichero de datos de este ejemplo es el que se detalla a continuación.

```

NAME          Ejelibro
ROWS
  N  OBJ
  L  FILA1
  L  FILA2
COLUMNS
  X1      OBJ      -2          FILA1      2
  X1      FILA2     1
  X2      OBJ      -4          FILA1      1
  X2      FILA2     1
  X3      OBJ      -1          FILA1      1
  X3      FILA2    -1
RHS
  RHS1    FILA1     10
  RHS1    FILA2     4
BOUNDS
  UP       X1       4
  UP       X2       6
  LO       X3       1
  UP       X3       4
ENDATA

```

El resultado obtenido con BBMI es el que sigue.

Problema Ejelibro

```

*** Estadísticas del Problema
    3 Fila(s)
    2 Restricción(es)
    3 Variable(s) de decisión
    2 Variable(s) de holgura/artificiales
    9 Elementos no cero
    0 Variable(s) entera(s)
Densidad de la matriz de coeficientes A: 100.000%

*** Estadísticas de INVERT
    3 elementos no cero en la base
    0 columnas estructurales en la base
    0 vectores columna antes del "bump"
    3 vectores columna después del "bump"
L:   0 elementos no cero;    0 vectores ETA
U:   0 elementos no cero;    0 vectores ETA
Total:  0 elementos no en la diagonal;    0 vectores ETA
Máximo de transformaciones ETA:    0; máximo número de elementos ETA:   12
Error relativo en x:   .000000D+00

Ite. Inf. Nopt. Sum.Inf/F.Obj Entra de-a Cost.Red Sale de-a Paso Pivote El.Eta
  1   0   3   -.2100000D+02  2 L->B -.40D+01  5 B->L .50D+01 .10D+01  0
  2   0   1   -.2600000D+02  3 L->B -.50D+01  2 B->U .10D+01 -.10D+01  3
  3   0   2   -.2800000D+02  1 L->B -.30D+01  4 B->L .67D+00 .30D+01  6

      --- SOLUCION INICIAL PROGRAMA LINEAL ---
      -----

Nombre del problema: Ejelibro
No. de iteraciones:    4
Valor de la función objetivo:    -28.00000000000000

*** FILAS

No. ..Fila.. en ....Valor.... ..Holgura... .Lí.Inferior. .Lí.Superior. Val.Dual.

  1 OBJ      BS -28.000000    28.000000      Ninguno      Ninguno      1.000
  2 FILA1     LS 10.000000    .00000000      Ninguno      10.000000    -1.000
  3 FILA2     LS  4.000000    .00000000      Ninguno      4.000000     .0000

*** COLUMNAS

No. .Columna en ....Valor.... Coste en F.O. .Lí.Inferior. .Lí.Superior. Cos.Red.

  1 X1      BS .66666667    -2.0000000    .00000000    4.0000000    .000
  2 X2      LS 6.0000000    -4.0000000    .00000000    6.0000000    -3.00
  3 X3      BS 2.6666667    -1.0000000    1.0000000    4.0000000    .000

Tiempo total de CPU en cálculos:    .0002 segundos

```

El problema de la dieta alimenticia

El problema quizás más clásico de programación lineal es el denominado de la dieta alimenticia. Se trata —ver capítulo 5— de elaborar una dieta diaria para un colectivo de personas de tal forma que se suministre a cada individuo una cantidad mínima de varios ingredientes nutritivos. En la tabla E.1 se especifica un problema de este tipo. Para su resolución con BBMI, el fichero de datos es el que sigue.

NAME	Dieta
------	-------

Tabla E.1

Especificaciones numéricas de un problema de dieta alimenticia como el introducido en el capítulo 5

	CARNE	SOJA EN HARINA	SOJA EN GRANO	MINERALES	SAL	ALFALFA	VITAMINAS	FACTOR DE CRECIMIENTO	PESCADO	
Función objetivo	5,80	2,63	3,08	1,13	1,00	2,26	35,72	6,00	7,00	
Cantidad total	1	1	1	1	1	1	1	1	1	= 100
Alfalfa						1				≥ 1
Vitaminas							1			≥ 1,1
Factor de crecimiento								1		≥ 5
Pescado									1	≥ 5
Proteínas	0,55	0,450	0,500			0,17	0,25	0,25	0,63	≥ 43
Riboflavina	0,26	0,130	0,120			0,70	41,6	2	0,20	≥ 70
Niacina	0,23	0,090	0,045			0,14	20,4	0,4	0,25	≥ 45
Acido pantoténico	0,20	0,055	0,060			0,14	9	0,4	0,04	≥ 16
Fósforo	0,40	0,065	0,060	0,26		0,02	0,1	0,05	0,30	≥ 14
Calcio	0,80	0,025	0,020	3		0,15	0,05	0,05	0,50	≥ 35
Sal 1	0,10			0,9	10					≥ 19
Sal 2	0,10			0,9	10					≥ 24

ROWS

N OBJETIVO

E CAN.TOT.

G ALFALFA

G VITAMINA

G FAC.CRE.

G PESCADO

G PROTEINA

G RIBOFLA.

G NIACINA

G ACI.PAN.

G FOSFORO

G CALCIO

G SAL 1

L SAL 2

COLUMNS

CARNE OBJETIVO 5.8

CARNE	CAN.TOT.	1.0
CARNE	PROTEINA	0.55
CARNE	RIBOFLA.	0.26
CARNE	NIACINA	0.23
CARNE	ACI.PAN.	0.02
CARNE	FOSFORO	0.4
CARNE	CALCIO	0.8
CARNE	SAL 1	0.1
CARNE	SAL 2	0.1
SOJA-HAR	OBJETIVO	2.63
SOJA-HAR	CAN.TOT.	1.0
SOJA-HAR	PROTEINA	0.45
SOJA-HAR	RIBOFLA.	0.13
SOJA-HAR	NIACINA	0.09
SOJA-HAR	ACI.PAN.	0.055
SOJA-HAR	FOSFORO	0.065
SOJA-HAR	CALCIO	0.025
SOJA-GRA	OBJETIVO	3.08
SOJA-GRA	CAN.TOT.	1.0
SOJA-GRA	PROTEINA	0.5
SOJA-GRA	RIBOFLA.	0.12
SOJA-GRA	NIACINA	0.045
SOJA-GRA	ACI.PAN.	0.06
SOJA-GRA	FOSFORO	0.06
SOJA-GRA	CALCIO	0.02
MINERAL.	OBJETIVO	1.13
MINERAL.	CAN.TOT.	1.0
MINERAL.	FOSFORO	0.26
MINERAL.	CALCIO	3.0
MINERAL.	SAL 1	0.9
MINERAL.	SAL 2	0.9
SAL	OBJETIVO	1.0
SAL	CAN.TOT.	1.0
SAL	SAL 1	10.0
SAL	SAL 2	10.0
ALFALFA	OBJETIVO	2.26
ALFALFA	CAN.TOT.	1.0
ALFALFA	ALFALFA	1.0
ALFALFA	PROTEINA	0.17
ALFALFA	RIBOFLA.	0.7
ALFALFA	NIACINA	0.14
ALFALFA	ACI.PAN.	0.14
ALFALFA	FOSFORO	0.02
ALFALFA	CALCIO	0.15
VITAMINA	OBJETIVO	35.72
VITAMINA	CAN.TOT.	1.0
VITAMINA	VITAMINA	1.0

VITAMINA	PROTEINA	0.25
VITAMINA	RIBOFLA.	41.6
VITAMINA	NIACINA	20.4
VITAMINA	ACI.PAN.	9.0
VITAMINA	FOSFORO	0.1
VITAMINA	CALCIO	0.05
FAC.CRE.	OBJETIVO	6.0
FAC.CRE.	CAN.TOT.	1.0
FAC.CRE.	FAC.CRE.	1.0
FAC.CRE.	PROTEINA	0.25
FAC.CRE.	RIBOFLA.	2.0
FAC.CRE.	NIACINA	0.4
FAC.CRE.	ACI.PAN.	0.4
FAC.CRE.	FOSFORO	0.05
FAC.CRE.	CALCIO	0.05
PESCADO	OBJETIVO	7.0
PESCADO	CAN.TOT.	1.0
PESCADO	PESCADO	1.0
PESCADO	PROTEINA	0.63
PESCADO	RIBOFLA.	0.2
PESCADO	NIACINA	0.25
PESCADO	ACI.PAN.	0.04
PESCADO	FOSFORO	0.3
PESCADO	CALCIO	0.5
RHS		
RHS2	CAN.TOT.	100
RHS2	ALFALFA	1
RHS2	VITAMINA	1.1
RHS2	FAC.CRE.	5
RHS2	PESCADO	5
RHS2	PROTEINA	43
RHS2	RIBOFLA.	70
RHS2	NIACINA	45
RHS2	ACI.PAN.	16
RHS2	FOSFORO	14
RHS2	CALCIO	35
RHS2	SAL 1	19
RHS2	SAL 2	24
ENDATA		

La resolución de este problema con BBMI daría el resultado que sigue.

Problema Dieta

```

*** Estadísticas del Problema
  14 Fila(s)
  13 Restricción(es)
   9 Variable(s) de decisión
  13 Variable(s) de holgura/artificiales
  72 Elementos no cero

```


0 Variable(s) entera(s)
 Densidad de la matriz de coeficientes A: 57.143%

*** Estadísticas de INVERT
 14 elementos no cero en la base
 0 columnas estructurales en la base
 0 vectores columna antes del "bump"
 14 vectores columna después del "bump"
 L: 0 elementos no cero; 0 vectores ETA
 U: 11 elementos no cero; 11 vectores ETA
 Total: 0 elementos no en la diagonal; 11 vectores ETA
 Máximo de transformaciones ETA: 11; máximo número de elementos ETA: 86
 Error relativo en x: .000000D+00

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra	de-a	Cost.Red	Sale	de-a	Paso	Pivote	El.Eta
1	12	9	.3541000D+03	7	L->B	-.73D+02	10	B->L	.10D+03	.10D+01	11	
2	7	8	.8200000D+02	5	L->B	-.96D+01	21	B->L	.19D+01	-.10D+02	20	
3	6	7	.6376000D+02	4	L->B	-.29D+01	5	B->L	.21D+02	.90D-01	31	
4	5	7	.3490000D+02	9	L->B	-.16D+01	15	B->L	.61D+02	-.38D+00	41	
5	2	2	.6000000D+01	6	L->B	-.10D+01	11	B->L	.10D+01	-.10D+01	51	
6	1	1	.5000000D+01	8	L->B	-.10D+01	13	B->L	.50D+01	-.10D+01	62	
7	0	7	.6014543D+03	15	L->B	-.76D+02	17	B->L	.39D+01	.53D+02	70	
8	0	6	.5708227D+03	21	L->B	-.61D+01	22	B->L	.50D+01	.10D+01	80	
9	0	5	.5673594D+03	11	L->B	-.46D+01	15	B->L	.76D+00	.46D+00	92	
10	0	5	.5483786D+03	5	L->B	-.80D+01	20	B->L	.24D+01	.33D+02	103	
11	0	4	.4423352D+03	2	L->B	-.22D+01	19	B->L	.47D+02	.87D-01	115	
12	0	3	.4363499D+03	15	L->B	-.37D+01	11	B->L	.16D+01	.51D+01	127	
13	0	2	.4032097D+03	1	L->B	-.27D+01	15	B->L	.12D+02	.13D+00	139	
14	0	3	.4004950D+03	19	L->B	-.40D+01	14	B->L	.68D+00	.79D+01	150	
15	0	2	.3977449D+03	22	L->B	-.55D+00	21	B->L	.50D+01	.10D+01	161	
16	0	1	.3963783D+03	3	L->B	-.23D+00	19	B->L	.59D+01	.70D-01	172	

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: Dieta
 No. de iteraciones: 17
 Valor de la función objetivo: 396.378264562255

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJETIVO	BS	396.37826	-396.37826	Ninguno	Ninguno	1.000
2	CAN.TOT.	EQ	100.00000	.00000000	100.00000	100.00000	-2.703
3	ALFALFA	LI	1.0000000	.00000000	1.0000000	Ninguno	2.657
4	VITAMINA	BS	1.5888503	.48885029	1.1000000	Ninguno	.0000
5	FAC.CRE.	LI	5.0000000	.00000000	5.0000000	Ninguno	5.057
6	PESCADO	LI	5.0000000	.00000000	5.0000000	Ninguno	.9198
7	PROTEINA	LI	43.000000	.00000000	43.000000	Ninguno	10.98
8	RIBOFLA.	BS	90.253400	20.253400	70.000000	Ninguno	.0000
9	NIACINA	LI	45.000000	.00000000	45.000000	Ninguno	1.731
10	ACI.PAN.	BS	20.527492	4.5274915	16.000000	Ninguno	.0000
11	FOSFORO	LI	14.000000	.00000000	14.000000	Ninguno	3.319
12	CALCIO	LI	35.000000	.00000000	35.000000	Ninguno	.8790
13	SAL 1	LI	19.000000	.00000000	19.000000	Ninguno	.3703
14	SAL 2	BS	19.000000	5.0000000	Ninguno	24.000000	.1110E-15

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	CARNE	BS	15.967949	5.8000000	.00000000	Ninguno	.000
2	SOJA-HAR	BS	58.430072	2.6300000	.00000000	Ninguno	.000
3	SOJA-GRA	BS	5.9137668	3.0800000	.00000000	Ninguno	.000
4	MINERAL.	BS	5.8890572	1.1300000	.00000000	Ninguno	.000

5	SAL	BS	1.2103054	1.0000000	.00000000	Ninguno	.000
6	ALFALFA	BS	1.0000000	2.2600000	.00000000	Ninguno	.000
7	VITAMINA	BS	1.5888503	35.720000	.00000000	Ninguno	.000
8	FAC.CRE.	BS	5.0000000	6.0000000	.00000000	Ninguno	.000
9	PESCADO	BS	5.0000000	7.0000000	.00000000	Ninguno	.000

Tiempo total de CPU en cálculos: .0016 segundos

E.3.2 Programas enteros puros

Como ejemplo de programa entero puro, resolvamos el del ejemplo 12.1 de la página 641:

$$\begin{aligned}
 &\max. && 7x_1 + 2x_2 \\
 &\text{s. a} && -x_1 + 2x_2 \leq 4 \\
 &&& 5x_1 + x_2 \leq 20 \\
 &&& -2x_1 - 2x_2 \leq -7 \\
 &&& \mathbf{x} \in \mathbf{Z}_+^2.
 \end{aligned}$$

El fichero de datos de este ejemplo es el que se detalla a continuación.

```

MAXIMIZAR
NAME                Demos-bb
ROWS
  N  OBJ
  L  ROW1
  L  ROW2
  L  ROW3
COLUMNS
INT
  COL1    OBJ      7                ROW1      -1
  COL1    ROW2     5                ROW3      -2
  COL2    OBJ      2                ROW1       2
  COL2    ROW2     1                ROW3      -2
RHS
  RHS1    ROW1      4
  RHS1    ROW2     20
  RHS1    ROW3     -7
ENDATA

```

El resultado obtenido con BBMI es el que sigue.

Problema Demos-bb

```

*** Estadísticas del Problema
  4 Fila(s)
  3 Restricción(es)
  2 Variable(s) de decisión
  3 Variable(s) de holgura/artificiales
  8 Elementos no cero
  2 Variable(s) entera(s)
Densidad de la matriz de coeficientes A: 100.000%

```

*** Estadísticas de INVERT

```

4 elementos no cero en la base
0 columnas estructurales en la base
0 vectores columna antes del "bump"
4 vectores columna después del "bump"
L: 0 elementos no cero; 0 vectores ETA
U: 0 elementos no cero; 0 vectores ETA
Total: 0 elementos no en la diagonal; 0 vectores ETA
Máximo de transformaciones ETA: 0; máximo número de elementos ETA: 12
Error relativo en x: .000000D+00

```

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra	de-a	Cost.Red	Sale	de-a	Paso	Pivote	El.Eta
1	1	2	.7000000D+01		1	L->B	.20D+01	5	B->L	.35D+01	-.20D+01	0
2	0	1	.2800000D+02		5	L->B	.35D+01	4	B->L	.10D+01	.25D+01	4
3	0	1	.3018182D+02		2	L->B	.60D+00	3	B->L	.36D+01	.22D+01	8

--- SOLUCION INICIAL PROGRAMA LINEAL ---

```

Nombre del problema: Demos-bb
No. de iteraciones: 4
Valor de la función objetivo: 30.1818181818182

```

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJ	BS	30.181818	-30.181818	Ninguno	Ninguno	1.000
2	ROW1	LS	4.0000000	.00000000	Ninguno	4.0000000	.2727
3	ROW2	LS	20.000000	.00000000	Ninguno	20.000000	1.455
4	ROW3	BS	-13.818182	6.8181818	Ninguno	-7.0000000	.2220E-15

*** COLUMNAS

No.	.Columna en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	COL1	BS	3.2727273	7.0000000	.00000000	Ninguno
2	COL2	BS	3.6363636	2.0000000	.00000000	Ninguno

* Nueva solución entera; z(PE)= 28.00000; Tiempo desde última: .0595 seg.

--- SOLUCION ENTERA OPTIMA ---

```

Nombre del problema: Demos-bb
No. de iteraciones: 6
Valor de la función objetivo: 28.0000000000000

```

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJ	BS	28.000000	-28.000000	Ninguno	Ninguno	1.000
2	ROW1	BS	-4.0000000	8.0000000	Ninguno	4.0000000	-.2017E-16
3	ROW2	LS	20.000000	.00000000	Ninguno	20.000000	2.000
4	ROW3	BS	-8.0000000	1.0000000	Ninguno	-7.0000000	.8882E-15

*** COLUMNAS

No.	.Columna en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	COL1	LIB	4.0000000	7.0000000	4.0000000	Ninguno
2	COL2	BS	.00000000	2.0000000	.00000000	Ninguno

Tiempo total de CPU en cálculos: .0591 segundos

Obsérvese que sólo se imprime la solución inicial de la relajación inicial y la óptima entera. Si quisiéramos que el programa facilitase todo el proceso de optimización habría que confeccionar el fichero de datos como sigue.

```

MAXIMIZAR
PROCESO ITERATIVO
NAME          Demos-b1
ROWS
  N  OBJ
  L  ROW1
  L  ROW2
  L  ROW3
COLUMNS
INT
  COL1      OBJ      7          ROW1      -1
  COL1      ROW2     5          ROW3      -2
  COL2      OBJ      2          ROW1       2
  COL2      ROW2     1          ROW3      -2
RHS
  RHS1      ROW1      4
  RHS1      ROW2     20
  RHS1      ROW3     -7
ENDATA

```

El resultado obtenido con BBMI en este caso es el que sigue.

Problema Demos-b1

*** Estadísticas del Problema

```

  4 Fila(s)
  3 Restricción(es)
  2 Variable(s) de decisión
  3 Variable(s) de holgura/artificiales
  8 Elementos no cero
  2 Variable(s) entera(s)
Densidad de la matriz de coeficientes A: 100.000%

```

*** Estadísticas de INVERT

```

  4 elementos no cero en la base
  0 columnas estructurales en la base
  0 vectores columna antes del "bump"
  4 vectores columna después del "bump"
L:   0 elementos no cero;    0 vectores ETA
U:   0 elementos no cero;    0 vectores ETA
Total:  0 elementos no en la diagonal;    0 vectores ETA
Máximo de transformaciones ETA:    0; máximo número de elementos ETA:   12
Error relativo en x:   .000000D+00

```

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra de-a	Cost.Red	Sale de-a	Paso	Pivote	El.Eta
1	1	2	.7000000D+01	1 L->B	.20D+01	5 B->L	.35D+01	-.20D+01		0
2	0	1	.2800000D+02	5 L->B	.35D+01	4 B->L	.10D+01	.25D+01		4
3	0	1	.3018182D+02	2 L->B	.60D+00	3 B->L	.36D+01	.22D+01		8

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: Demos-b1

No. de iteraciones: 4
 Valor de la función objetivo: 30.18181818182

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJ	BS	30.181818	-30.181818	Ninguno	Ninguno	1.000
2	ROW1	LS	4.0000000	.00000000	Ninguno	4.0000000	.2727
3	ROW2	LS	20.000000	.00000000	Ninguno	20.000000	1.455
4	ROW3	BS	-13.818182	6.8181818	Ninguno	-7.0000000	.2220E-15

*** COLUMNAS

No.	.Columna en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	COL1	BS 3.2727273	7.0000000	.00000000	Ninguno	.000
2	COL2	BS 3.6363636	2.0000000	.00000000	Ninguno	.000

Variable Separación	Nivel	Dirección	Nudos en Lista	Variables Ent. No Ent.	Iteración	Valor Func. Obj.
2	1	X<	3	1	2	5 D
1	2	X>	4	2	1	6 D

* Nueva solución entera; z(PE)= 28.00000; Tiempo desde última: .0019 seg.

--- SOLUCION ENTERA ---

Tiempo desde última: .0019 seg.
 Nombre del problema: Demos-b1
 No. de iteraciones: 6
 Valor de la función objetivo: 28.0000000000000

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Lí.Inferior.	.Lí.Superior.	Val.Dual.
1	OBJ	BS	28.000000	-28.000000	Ninguno	Ninguno	1.000
2	ROW1	BS	-4.0000000	8.0000000	Ninguno	4.0000000	-.2017E-16
3	ROW2	LS	20.000000	.00000000	Ninguno	20.000000	2.000
4	ROW3	BS	-8.0000000	1.0000000	Ninguno	-7.0000000	.8882E-15

*** COLUMNAS

No.	.Columna en	Valor....	Coste en F.O.	.Lí.Inferior.	.Lí.Superior.	Cos.Red.
1	COL1	LIB 4.0000000	7.0000000	4.0000000	Ninguno	-3.00
2	COL2	BS .88817842E-15	2.0000000	.00000000	Ninguno	.000

Variable Separación	Nivel	Dirección	Nudos en Lista	Variables Ent. No Ent.	Iteración	Valor Func. Obj.
1	2	X<	3	1	-Nudo desechado en BKTRAK-	2.7000000D+01
2	1	X>	4	0	-Nudo desechado en BKTRAK-	-1.0000000D+20

--- SOLUCION ENTERA OPTIMA ---

Nombre del problema: Demos-b1
 No. de iteraciones: 6
 Valor de la función objetivo: 28.0000000000000

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Li.Inferior.	.Li.Superior.	Val.Dual.
1	OBJ	BS	28.000000	-28.000000	Ninguno	Ninguno	1.000
2	ROW1	BS	-4.000000	8.000000	Ninguno	4.000000	-.2017E-16
3	ROW2	LS	20.000000	.000000	Ninguno	20.000000	2.000
4	ROW3	BS	-8.000000	1.000000	Ninguno	-7.000000	.8882E-15

*** COLUMNAS

No.	.Columna en	Valor....	Coste en F.O.	.Li.Inferior.	.Li.Superior.	Cos.Red.
1	COL1	LIB	4.000000	7.000000	4.000000	Ninguno
2	COL2	BS	.000000	2.000000	.000000	Ninguno

Tiempo total de CPU en cálculos: .0489 segundos

E.3.3 Programas enteros mixtos

Como ejemplo de programa entero mixto resolveremos uno de 35 variables. Las 28 primeras son continuas; de la 29 a la 35 son enteras, estando restringidas a tomar sólo valores 0 ó 1. La estructura del problema por lo que se refiere a los coeficientes distintos de cero es la de la figura E.1. El fichero de datos de este ejemplo es el que sigue.

```

MAXIMIZAR
NAME          P5b
ROWS
N  OBJ
E  E1C1
E  E1C2
G  E1A1
G  E1A2
G  E1T1
G  E1T2
E  E2C1
E  E2C2
G  E2A1
G  E2A2
G  E2T1
G  E2T2
E  E3C1
E  E3C2
G  E3A1
G  E3A2
G  E3J1
G  E3J2
L  E3D1
L  E3D2
G  E3D3
L  E3JP
G  E3T1
G  E3T2
E  E4C1
E  E4C2
G  E4A1
G  E4A2
G  E4J1
G  E4J2
L  E4D1
L  E4D2
G  E4D3
L  E4JP
G  E4T1
G  E4T2
E  E5C1
E  E5C2

```


G	E5A1				
G	E5A2				
G	E5T1				
G	E5T2				
E	BIF1				
E	BIF2				
L	BIF3				
COLUMNS					
	T0	E1C1	1	E1T2	-1
	DTC1	E1C1	1	E1C2	0.53
	DTC1	E1A1	-0.0775	E1A2	-1
	TA1	E1C2	1	E1T1	1
	TA1	BIF1	1		
	TA2	E1C2	-1	E1T2	1
	A1	OBJ	-0.75	E1A1	1
	T1	E1C1	-1	E1T1	-1
	T1	E2C1	1	E2T2	-1
	DTC2	E2C1	1	E2C2	2.024
	DTC2	E2A1	-0.0823	E2A2	-1
	TB1	E2C2	1	E2T1	1
	TB2	E2C2	-1	E2T2	1
	A2	OBJ	-0.75	E2A1	1
	T2	E2C1	-1	E2T1	-1
	T2	E3C1	1	E3T2	-1
	DTC3	E3C1	1	E3C2	0.53
	DTC3	E3A1	-0.103	E3A2	-1
	DTC3	E3J1	1	E3J2	-1
	DTC3	E3D2	-1	E3D3	-1
	TD1	E3C2	1	E3T1	1
	TD1	BIF2	1		
	TD2	E3C2	-1	E3T2	1
	A3	OBJ	-0.75	E3A1	1
	D3	E3A1	0.035	E3D1	1
	D3	E3D2	1	E3D3	1
	T3	E3C1	-1	E3T1	-1
	T3	E4C1	1	E4T2	-1
	DTC4	E4C1	1	E4C2	0.291
	DTC4	E4A1	-0.090	E4A2	-1
	DTC4	E4J1	1	E4J2	-1
	DTC4	E4D2	-1	E4D3	-1
	TE1	E4C2	1	E4T1	1
	TE2	E4C2	-1	E4T2	1
	A4	OBJ	-0.75	E4A1	1
	D4	E4A1	0.031	E4D1	1
	D4	E4D2	1	E4D3	1
	T4	E4C1	-1	E4T1	-1
	T4	E5C1	1	E5T2	-1
	DTC5	E5C1	1	E5C2	0.438
	DTC5	E5A1	-0.088	E5A2	-1
	TF1	E5C2	1	E5T1	1
	TF2	E5C2	-1	E5T2	1
	TF2	BIF1	-1	BIF2	-1
	A5	OBJ	-0.75	E5A1	1
	T5	OBJ	0.962	E5C1	-1
	T5	E5T1	-1.		
INT					
	I1	E1A1	-3.26	E1A2	125
	I1	E1T1	-272	E1T2	-272
	I1	BIF3	1		
	I2	E2A1	-3.469	E2A2	125
	I2	E2T1	-141	E2T2	-141
	I3	E3A1	-1.425	E3A2	125
	I3	E3JP	-1	E3T1	-272
	I3	E3T2	-272	BIF3	1
	J3	E3A1	-1.425	E3J1	-85
	J3	E3J2	85	E3D1	-125
	J3	E3D2	125	E3D3	-125
	J3	E3JP	1		
	I4	E4A1	-1.255	E4A2	64
	I4	E4JP	-1	E4T1	-205

I4	E4T2	-205		
J4	E4A1	-1.255	E4J1	-40
J4	E4J2	40	E4D1	-65
J4	E4D2	65	E4D3	-65
J4	E4JP	1		
I5	E5A1	-3.7	E5A2	125
I5	E5T1	-272	E5T2	-272
RHS				
RHS	E1T1	-256		
RHS	E1T2	-256		
RHS	E2T1	-125		
RHS	E2T2	-125		
RHS	E3J1	-45		
RHS	E3J2	-40		
RHS	E3D2	125		
RHS	E3D3	-125		
RHS	E3T1	-256		
RHS	E3T2	-256		
RHS	E4J2	-40		
RHS	E4D2	65		
RHS	E4D3	-65		
RHS	E4T1	-189		
RHS	E4T2	-189		
RHS	E5T1	-256		
RHS	E5T2	-256		
RHS	BIF3	1		
BOUNDS				
FX BOUND	T0	93.3		
LO BOUND	TA2	182		
FX BOUND	TB1	218		
LO BOUND	TB2	120		
LO BOUND	TD2	182		
FX BOUND	TE1	282		
LO BOUND	TE2	120		
FX BOUND	TF1	349		
UP BOUND	I1	1		
UP BOUND	I2	1		
UP BOUND	I3	1		
UP BOUND	I4	1		
UP BOUND	I5	1		
UP BOUND	J3	1		
UP BOUND	J4	1		
ENDATA				

Los resultados obtenidos (sólo solución inicial de la relajación lineal y entera óptima) son los que siguen.

Problema P5b

*** Estadísticas del Problema

46 Fila(s)
 45 Restricción(es)
 35 Variable(s) de decisión
 45 Variable(s) de holgura/artificiales
 129 Elementos no cero
 7 Variable(s) entera(s)
 Densidad de la matriz de coeficientes A: 8.012%

*** Estadísticas de INVERT

46 elementos no cero en la base
 0 columnas estructurales en la base
 0 vectores columna antes del "bump"
 46 vectores columna después del "bump"
 L: 0 elementos no cero; 0 vectores ETA
 U: 26 elementos no cero; 26 vectores ETA
 Total: 0 elementos no en la diagonal; 26 vectores ETA
 Máximo de transformaciones ETA: 26; máximo número de elementos ETA: 175
 Error relativo en x: .000000D+00

Ite.	Inf.	Nopt.	Sum.	Inf/F.Obj	Entra	de-a	Cost.Red	Sale	de-a	Paso	Pivote	El.Eta
1	6	7	.1066300D+04	3	L->B	.10D+01	78	B->L	.00D+00	.10D+01		26
2	6	6	.1066300D+04	26	L->B	.20D+01	79	B->U	.00D+00	-.10D+01		29
3	6	5	.1066300D+04	13	L->B	.30D+01	37	B->L	.18D+03	.10D+01		35
4	4	4	.5203000D+03	4	L->B	.10D+01	49	B->L	.00D+00	.10D+01		43
5	4	4	.5203000D+03	6	L->B	.10D+01	42	B->L	.00D+00	.10D+01		52
6	4	4	.5203000D+03	9	L->B	.10D+01	43	B->U	.98D+02	-.10D+01		56
7	3	3	.4223000D+03	11	L->B	.10D+01	48	B->L	.00D+00	.10D+01		58
8	3	3	.4223000D+03	14	L->B	.10D+01	73	B->U	.17D+03	-.10D+01		65
9	2	2	.2553000D+03	17	L->B	.10D+01	60	B->L	.00D+00	.10D+01		75
10	2	2	.2553000D+03	20	L->B	.10D+01	61	B->U	.16D+03	-.10D+01		85
11	1	1	.9330000D+02	23	L->B	.10D+01	72	B->L	.00D+00	.10D+01		87
12	1	1	.9330000D+02	28	L->B	.10D+01	36	B->U	.93D+02	-.10D+01		100
13	0	5	.8975460D+02	2	L->B	.96D+00	39	B->L	.00D+00	.10D+01		115
14	0	5	.8975460D+02	29	L->B	.12D+03	38	B->L	.00D+00	.13D+02		134
15	0	5	.2002940D+03	5	L->B	.85D+01	29	B->U	.13D+02	-.77D-01		154
16	0	4	.2002940D+03	7	L->B	.96D+00	45	B->L	.00D+00	.10D+01		174
17	0	4	.2002940D+03	30	L->B	.12D+03	44	B->L	.00D+00	.14D+02		190
18	0	4	.2518301D+03	10	L->B	.80D+01	46	B->L	.64D+01	.19D+02		206
19	0	3	.2518301D+03	12	L->B	.96D+00	51	B->L	.00D+00	.10D+01		222
20	0	3	.2518301D+03	31	L->B	.12D+03	50	B->L	.00D+00	.14D+02		239
21	0	4	.2518301D+03	15	L->B	.77D+01	80	B->L	.00D+00	.70D-01		258
22	0	4	.2680367D+03	29	U->B	-.51D+02	53	B->L	.32D+00	-.13D+03		277
23	0	4	.2815437D+03	32	L->B	.33D+02	56	B->L	.40D+00	.21D+03		308
24	0	3	.2815437D+03	18	L->B	.96D+00	63	B->L	.00D+00	.10D+01		339
25	0	3	.2815437D+03	33	L->B	.62D+02	62	B->L	.00D+00	.70D+01		353
26	0	4	.3148538D+03	21	L->B	.80D+01	70	B->L	.41D+01	.38D+02		368
27	0	3	.3148538D+03	24	L->B	.96D+00	75	B->L	.00D+00	.10D+01		383
28	0	3	.3148538D+03	35	L->B	.12D+03	74	B->L	.00D+00	.15D+02		398
29	0	3	.3850864D+03	27	L->B	.74D+01	76	B->L	.95D+01	.27D+02		413
30	0	2	.3982691D+03	16	L->B	.11D+00	55	B->L	.13D+03	.12D+01		428
31	0	1	.3982691D+03	22	L->B	.23D-01	66	B->L	.00D+00	.10D+01		472
32	0	1	.3985560D+03	34	L->B	.57D+00	69	B->L	.50D+00	.10D+01		477

--- SOLUCION INICIAL PROGRAMA LINEAL ---

Nombre del problema: P5b
 No. de iteraciones: 33
 Valor de la función objetivo: 398.555969357191

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Li.Inferior.	.Li.Superior.	Val.Dual.
1	OBJ	BS	398.55597	-398.55597	Ninguno	Ninguno	1.000
2	E1C1	EQ	.00000000	.00000000	.00000000	.00000000	-.3255
3	E1C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
4	E1A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
5	E1A2	LI	.00000000	.00000000	.00000000	Ninguno	-.2674
6	E1T1	BS	288.40591	544.40591	-256.00000	Ninguno	.0000
7	E1T2	BS	288.40591	544.40591	-256.00000	Ninguno	.0000
8	E2C1	EQ	.00000000	.00000000	.00000000	.00000000	-.3255
9	E2C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
10	E2A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
11	E2A2	LI	.00000000	.00000000	.00000000	Ninguno	-.2638
12	E2T1	LI	-125.00000	.00000000	-125.00000	Ninguno	-.2154
13	E2T2	BS	229.83684	354.83684	-125.00000	Ninguno	.0000
14	E3C1	EQ	.00000000	.00000000	.00000000	.00000000	-.5409
15	E3C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
16	E3A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
17	E3A2	LI	.00000000	.00000000	.00000000	Ninguno	-.2564
18	E3J1	BS	40.000000	85.000000	-45.000000	Ninguno	.0000
19	E3J2	LI	-40.000000	.00000000	-40.000000	Ninguno	-.2335
20	E3D1	BSD	-.18735014E-15	.18735014E-15	Ninguno	.00000000	.0000
21	E3D2	LS	125.00000	.00000000	Ninguno	125.00000	.8825E-01
22	E3D3	LI	-125.00000	.00000000	-125.00000	Ninguno	-.6200E-01

23	E3JP	BSD-.99990545E-16	.99990545E-16	Ninguno	.00000000	.0000
24	E3T1	BS-225.93431	30.065689	-256.00000	Ninguno	.0000
25	E3T2	BS-34.684311	221.31569	-256.00000	Ninguno	.0000
26	E4C1	EQ .00000000	.00000000	.00000000	.00000000	-.5409
27	E4C2	EQ .00000000	.00000000	.00000000	.00000000	.0000
28	E4A1	LI .00000000	.00000000	.00000000	Ninguno	-.7500
29	E4A2	LI .00000000	.00000000	.00000000	Ninguno	-.4734
30	E4J1	BS 12.076708	12.076708	.00000000	Ninguno	.0000
31	E4J2	BS-12.076708	27.923292	-40.000000	Ninguno	.0000
32	E4D1	LS .00000000	.00000000	Ninguno	.00000000	.2325E-01
33	E4D2	BS 33.210948	31.789052	Ninguno	65.000000	.0000
34	E4D3	BS-32.204556	32.795444	-65.000000	Ninguno	.0000
35	E4JP	LS .00000000	.00000000	Ninguno	.00000000	.5700
36	E4T1	LI-189.00000	.00000000	-189.00000	Ninguno	-.1460
37	E4T2	BS-147.42392	41.576082	-189.00000	Ninguno	.0000
38	E5C1	EQ .00000000	.00000000	.00000000	.00000000	-.6869
39	E5C2	EQ .00000000	.00000000	.00000000	.00000000	.0000
40	E5A1	LI .00000000	.00000000	.00000000	Ninguno	-.7500
41	E5A2	LI .00000000	.00000000	.00000000	Ninguno	-.6209
42	E5T1	LI-256.00000	.00000000	-256.00000	Ninguno	-.2751
43	E5T2	BS-148.62305	107.37695	-256.00000	Ninguno	.0000
44	BIF1	EQ .00000000	.00000000	.00000000	.00000000	.0000
45	BIF2	EQ .00000000	.00000000	.00000000	.00000000	.0000
46	BIF3	LS 1.0000000	.00000000	Ninguno	1.0000000	30.98

*** COLUMNAS

No. .Columna enValor.... Coste en F.O. .Lí.Inferior. .Lí.Superior. Cos.Red.

1	T0	FX 93.300000	.00000000	93.300000	93.300000	.326
2	DTC1	BS-.67515438E-14	.00000000	.00000000	Ninguno	.000
3	TA1	BS 381.70591	.00000000	.00000000	Ninguno	.000
4	TA2	BS 381.70591	.00000000	182.00000	Ninguno	.000
5	A1	BS-.12689502E-14	.75000000	.00000000	Ninguno	.000
6	T1	BS 93.300000	.00000000	.00000000	Ninguno	.000
7	DTC2	BS 117.34023	.00000000	.00000000	Ninguno	.000
8	TB1	FX 218.00000	.00000000	218.00000	218.00000	.215
9	TB2	BS 455.49662	.00000000	120.00000	Ninguno	.000
10	A2	BS 12.913527	-.75000000	.00000000	Ninguno	.000
11	T2	BS 210.64023	.00000000	.00000000	Ninguno	.000
12	DTC3	BS 125.00000	.00000000	.00000000	Ninguno	.000
13	TD1	BS 381.70591	.00000000	.00000000	Ninguno	.000
14	TD2	BS 447.95591	.00000000	182.00000	Ninguno	.000
15	A3	BS 11.350000	-.75000000	.00000000	Ninguno	.000
16	D3	BS 125.00000	.00000000	.00000000	Ninguno	.000
17	T3	BS 335.64023	.00000000	.00000000	Ninguno	.000
18	DTC4	BS 32.204556	.00000000	.00000000	Ninguno	.000
19	TE1	FX 282.00000	.00000000	282.00000	282.00000	.146
20	TE2	BS 291.37153	.00000000	120.00000	Ninguno	.000
21	A4	BS 3.1474922	-.75000000	.00000000	Ninguno	.000
22	D4	BS 32.707752	.00000000	.00000000	Ninguno	.000
23	T4	BS 367.84478	.00000000	.00000000	Ninguno	.000
24	DTC5	BS 74.671039	.00000000	.00000000	Ninguno	.000
25	TF1	FX 349.00000	.00000000	349.00000	349.00000	.275
26	TF2	BS 381.70591	.00000000	.00000000	Ninguno	.000
27	A5	BS 8.7813141	-.75000000	.00000000	Ninguno	.000
28	T5	BS 442.51582	.96200000	.00000000	Ninguno	.000
29	I1	BS-.22730299E-15	.00000000	.00000000	1.0000000	.000
30	I2	BS .93872180	.00000000	.00000000	1.0000000	.000
31	I3	BS 1.0000000	.00000000	.00000000	1.0000000	.000
32	J3	BS 1.0000000	.00000000	.00000000	1.0000000	.000
33	I4	BS .50319619	.00000000	.00000000	1.0000000	.000
34	J4	BS .50319619	.00000000	.00000000	1.0000000	.000
35	I5	BS .59736831	.00000000	.00000000	1.0000000	.000

* Nueva solución entera; z(PE)= 296.75024; Tiempo desde última: .0598 seg.

* Nueva solución entera; z(PE)= 301.48830; Tiempo desde última: .0512 seg.

--- SOLUCION ENTERA OPTIMA ---

Nombre del problema: P5b
 No. de iteraciones: 54
 Valor de la función objetivo: 301.488300000000

*** FILAS

No.	..Fila..	en	Valor....	...Holgura...	.Li.Inferior.	.Li.Superior.	Val.Dual.
1	OBJ	BS	301.48830	-301.48830	Ninguno	Ninguno	1.000
2	E1C1	EQ	.00000000	.00000000	.00000000	.00000000	-.6600E-01
3	E1C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
4	E1A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
5	E1A2	LI	.00000000	.00000000	.00000000	Ninguno	-.1956E-01
6	E1T1	BS	305.93860	561.93860	-256.00000	Ninguno	.0000
7	E1T2	BS	305.93860	561.93860	-256.00000	Ninguno	.0000
8	E2C1	EQ	.00000000	.00000000	.00000000	.00000000	-.6600E-01
9	E2C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
10	E2A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
11	E2A2	LI	.00000000	.00000000	.00000000	Ninguno	-.2081E-01
12	E2T1	BS	124.70000	249.70000	-125.00000	Ninguno	.0000
13	E2T2	BS	124.70000	249.70000	-125.00000	Ninguno	.0000
14	E3C1	EQ	.00000000	.00000000	.00000000	.00000000	-.6600E-01
15	E3C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
16	E3A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
17	E3A2	BSD	-.71054274E-14	-.71054274E-14	.00000000	Ninguno	.0000
18	E3J1	BS	40.000000	85.000000	-45.000000	Ninguno	.0000
19	E3J2	BSD	-40.000000	-.22030988E-14	-40.000000	Ninguno	.0000
20	E3D1	LS	.00000000	.00000000	Ninguno	.00000000	.1500E-01
21	E3D2	LS	125.00000	.00000000	Ninguno	125.00000	.1125E-01
22	E3D3	BSD	-125.00000	-.71054274E-14	-125.00000	Ninguno	.0000
23	E3JP	LS	.00000000	.00000000	Ninguno	.00000000	1.069
24	E3T1	BS	-91.061400	164.93860	-256.00000	Ninguno	.0000
25	E3T2	BS	100.18860	356.18860	-256.00000	Ninguno	.0000
26	E4C1	EQ	.00000000	.00000000	.00000000	.00000000	-.6600E-01
27	E4C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
28	E4A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
29	E4A2	BSD	.00000000	.00000000	.00000000	Ninguno	.2168E-18
30	E4J1	BSD	.00000000	.00000000	.00000000	Ninguno	.0000
31	E4J2	BS	.00000000	40.000000	-40.000000	Ninguno	.0000
32	E4D1	LS	.00000000	.00000000	Ninguno	.00000000	.2325E-01
33	E4D2	BS	.00000000	65.000000	Ninguno	65.000000	.0000
34	E4D3	BS	.00000000	65.000000	-65.000000	Ninguno	.0000
35	E4JP	LS	.00000000	.00000000	Ninguno	.00000000	.5700
36	E4T1	BS	63.700000	252.70000	-189.00000	Ninguno	.0000
37	E4T2	BS	63.700000	252.70000	-189.00000	Ninguno	.0000
38	E5C1	EQ	.00000000	.00000000	.00000000	.00000000	-.6600E-01
39	E5C2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
40	E5A1	LI	.00000000	.00000000	.00000000	Ninguno	-.7500
41	E5A2	BS	10.300000	10.300000	.00000000	Ninguno	-.2082E-16
42	E5T1	LI	-256.00000	.00000000	-256.00000	Ninguno	-.8960
43	E5T2	BS	-91.061400	164.93860	-256.00000	Ninguno	.0000
44	B1F1	EQ	.00000000	.00000000	.00000000	.00000000	.0000
45	B1F2	EQ	.00000000	.00000000	.00000000	.00000000	.0000
46	B1F3	BSD	1.0000000	.24231919E-16	Ninguno	1.0000000	.0000

*** COLUMNAS

No.	.Columna	en	Valor....	Coste en F.O.	.Li.Inferior.	.Li.Superior.	Cos.Red.
1	T0	FX	93.300000	.00000000	93.300000	93.300000	.000
2	DTC1	LI	.00000000	.00000000	.00000000	Ninguno	-.117E-01
3	TA1	BS	399.23860	.00000000	.00000000	Ninguno	.000
4	TA2	BS	399.23860	.00000000	182.00000	Ninguno	.000
5	A1	BS	.00000000	-.75000000	.00000000	Ninguno	.000

6	T1	BS	93.300000	.00000000	.00000000	Ninguno	.000
7	DTC2	LI	.00000000	.00000000	.00000000	Ninguno	-.165E-01
8	TB1	FX	218.00000	.00000000	218.00000	218.00000	.000
9	TB2	BS	218.00000	.00000000	120.00000	Ninguno	.000
10	A2	BS	.00000000	-.75000000	.00000000	Ninguno	.000
11	T2	BS	93.300000	.00000000	.00000000	Ninguno	.000
12	DTC3	BS	125.00000	.00000000	.00000000	Ninguno	.000
13	TD1	BS	399.23860	.00000000	.00000000	Ninguno	.000
14	TD2	BS	465.48860	.00000000	182.00000	Ninguno	.000
15	A3	BS	11.350000	-.75000000	.00000000	Ninguno	.000
16	D3	BS	125.00000	.00000000	.00000000	Ninguno	.000
17	T3	BS	218.30000	.00000000	.00000000	Ninguno	.000
18	DTC4	LI	.00000000	.00000000	.00000000	Ninguno	-.150E-02
19	TE1	FX	282.00000	.00000000	282.00000	282.00000	.000
20	TE2	BS	282.00000	.00000000	120.00000	Ninguno	.000
21	A4	BS	-.29869770E-16	-.75000000	.00000000	Ninguno	.000
22	D4	BS	.00000000	.00000000	.00000000	Ninguno	.000
23	T4	BS	218.30000	.00000000	.00000000	Ninguno	.000
24	DTC5	BS	114.70000	.00000000	.00000000	Ninguno	.000
25	TF1	FX	349.00000	.00000000	349.00000	349.00000	.000
26	TF2	BS	399.23860	.00000000	.00000000	Ninguno	.000
27	A5	BS	13.793600	-.75000000	.00000000	Ninguno	.000
28	T5	BS	333.00000	.96200000	.00000000	Ninguno	.000
29	I1	BS	.00000000	.00000000	.00000000	1.0000000	.000
30	I2	BS	.00000000	.00000000	.00000000	1.0000000	.000
31	I3	BS	1.0000000	.00000000	.00000000	1.0000000	.000
32	J3	EQB	1.0000000	.00000000	1.0000000	1.0000000	.000
33	I4	LI	.00000000	.00000000	.00000000	1.0000000	-.371
34	J4	BS	.00000000	.00000000	.00000000	1.0000000	.000
35	I5	EQB	1.0000000	.00000000	1.0000000	1.0000000	.000

Tiempo total de CPU en cálculos: .1095 segundos

E.4 Listado de BBMI


```

PROGRAM Bbmi
C*****
C*****
C
C      BBMI resuelve problemas de programación lineal,
C      entera y programación mixta lineal-entera.
C
C
C      ESTRUCTURA de SUBROUTINAS
C
C      -----
C      | Nivel0 | Nivel1 | Nivel2 | Nivel3 | Nivel4 | Nivel5 |
C      -----
C
C      |
C      | / PARAME
C      | INPUT
C      |
C      | /
C      | INVERT < WRETA
C      | \ FTRAN
C      |
C      | NORMAL < FORMC
C      | BTRAN
C      | PRICE
C      | UNPACK
C      | FTRAN
C      | CHUZR
C      | \ WRETA
C      |
C      | WRILPR
C      |
C      | /
C      | TESTX < BTRAN
C      | WRILPR
C      |
C      | PENLTS < UNPACK
C      | FTRAN
C      | BRANCH
C      |
C      | /
C      | BKTRAK < UNPACK
C      | FTRAN
C      |
C      | BBMI < /
C      | INVERT < WRETA
C      | \ FTRAN
C      |
C      | CYCLE < NORMAL < FORMC
C      | BTRAN
C      | PRICE
C      | UNPACK
C      | FTRAN
C      | CHUZR
C      | \ WRETA
C      |
C      | BANDB <
C      |
C      | /
C      | TESTX < BTRAN
C      | WRILPR
C      |
C      | DCHUZR < UNPACK
C      | FTRAN
C      |
C      | CYCLE
C      | INVERT
C      | WRETA
C      | \ DCHUZR
C      |
C      | \ WRILPR
C      |
C      -----
C*****
C
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      common /iolog/ iu,lo,logu
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C      + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /nodbb/ ipart(1000),ivid(1500,2),nvare,list1,listsa,noint,
C      + idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
C      + xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C      + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
C      + ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
C      + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C      + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C      + yen(1000)
C
C      + yen(1000)
C
C      call parame
C
C*****
C      Entrada datos del problema a resolver.
C*****
C      call input
C      call cputim (time1,cput)
C
C*****
C      Se resuelve relajación lineal inicial y escriben resultados.
C*****
C      call normal
C      call wrilpr (iuno)
C      if (nvare.le.ncol.and.qstat.ne.'INFEAS') then
C
C*****
C      Si el programa tiene variables enteras, se resuelve aplicando
C      estrategia Branch-and-Bound y se escriben resultados.
C*****
C      call bandb
C      call wrilpr (3)
C
C      endif
C      call cputim (time2,cput)
C      write (lo,9000) cput-time1
C
C      9000 format(/'Tiempo total de CPU en cálculos:',f9.4,' segundos')
C
C      Fin de -BBMI-
C      end
C
C      subroutine bandb
C
C      Rutina que dirige el proceso de optimización del programa
C      entero o mixto mediante un procedimiento Branch-and-Bound
C      con relajaciones lineales.
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      common /iolog/ iu,lo,logu
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C      + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /nodbb/ ipart(1000),ivid(1500,2),nvare,list1,listsa,noint,
C      + idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
C      + xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C      + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
C      + ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
C      + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C      + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C      + yen(1000)
C
C
C      if (itcnt.ge.itrfrq) return
C      list1 = 0
C      listsa = 0
C      idepth = 0
C
C
C
C*****
C      Se comprueba si la solución inicial PL es entera.
C*****
C      call testx
C      if (qstat.eq.'ENTERA') return
C      inibb = 1
C
C*****
C      Nudo analizándose no rechazado; calcular penalizaciones por
C      separar una u otra variable.
C      Las nuevas ramas del árbol se generan en BRANCH a la que
C      llama PENLTS.
C      Si en PENLTS se rechaza el nudo (IDIR=0), buscar en BKTRAK
C      el siguiente a analizar.
C      Si no se desecha, utilizar el método dual del simplex para
C      reoptimizar.
C*****
C      150 continue
C      call penlts (irowpd,idir,iptypd)
C      if (idir.eq.0) then
C      if (logite.eq.1) write (logu,550)
C      call cycle (*500,*150)
C      endif
C      icolit = jh(irowpd)
C      if (icolit.le.nrow) then
C      icolit = icolit+ncol-nrow-1
C      else
C      icolit = icolit-nrow

```



```

endif
qupdo = '      ',
if (idir.eq.1) then
  iax = ipart(irowpd)
  write (qupdo,10) 'X<',iax
else
  iax = ipart(irowpd)+1
  write (qupdo,10) 'X>',iax
endif
C
C*****
C Sale de la base la variable IROWPD; determinar la columna de
C pivotación JCOLPD y adaptar la solución.
C Si no es posible (JCOLPD=0), existe degeneración dual.
C*****
250 continue
call dchuzc (jcolpd,irowpd,npivod,iptypd,thetad,ivout)
itcnt = itcnt+1
itsinv = itsinv+1
if (jcolpd.eq.0) then
  idepth = idepth+1
  if (logite.eq.1) write (logu,600) icolit,qfix,idepth,qupdo,
+ listsa,noint,itcnt,' D','No factible'
  call cycle (*500,*150)
endif
C
C*****
C Comprobar si hay que reinvertir la base.
C*****
if (nelem.gt.nemax.or.itsinv.ge.invfqr) then
  call invert
  itsinv = 0
else
  call wreta (irowpd)
endif
C
C*****
C Comienzo ciclo iterativo dual del simplex.
C*****
if (itcnt.ge.itrfqr) then
  write (lo,')' *** Límite de iteraciones sobrepasado;',
+ ' ' el programa se para')
  stop
endif
C
C-----
C Escoger variable básica X(IROWPD) que debe salir de la base.
C-----
call dchuzr (irowpd,iptypd)
if (irowpd.ne.0) go to 250
idepth = idepth+1
if (x(1).le.xincva) then
  qplus = 'ABANDONO'
  if (logite.eq.1) write (logu,700) icolit,qfix,idepth,qupdo,
+ listsa,noint,itcnt,' D',x(1)*min,qplus
  call cycle (*500,*150)
endif
qplus = '      ',
if (logite.eq.1) write (logu,700) icolit,qfix,idepth,qupdo,lists,
+ noint,itcnt,' D',x(1)*min,qplus
call testx
if (qstat.eq.'ENTERA') call cycle (*500,*150)
go to 150
C
500 continue
return
C
10 format(a,i4)
550 format('----- Nudo desechado en PENLTS -----')
600 format(i6,a8,i5,7x,a,i7,4x,i7,8x,i4,a,6x,a)
700 format(i6,a8,i5,7x,a,i7,4x,i7,8x,i4,a,1pd18.7,1x,a)
C
C Fin de -BANDB-
end
C
subroutine bktrak
C
C Se escoge un nudo del árbol enumerativo siguiendo la
C regla LIFO (Last In First Out).
C*****
implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
common /iolog/ iu,lo,logu
C
common /defpa/ nrmax,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+ ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,lists,
+ idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+ xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),

```

```

+ kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+ ncol,nelem,neta,nleta,ninf,min,nopt,logite,qname(4000),
+ qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+ b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+ yen(1000)

```

```

C
C ntemp3 = 0
C qfix = '      ',
C
C
C*****
C Si la lista de nudos del árbol esta vacía, terminar.
C*****
50 continue
if (listl.eq.0) return
C
C*****
C Si no esta vacía, coger el siguiente.
C*****
if (xiobnd(listl).gt.xincva) then
  inco = ivid(listl,1)
  if (inco.ge.0) then
    dp = xlb(inco)
    xlb(inco) = xub(inco)+uno
    xub(inco) = vbnd(listl)
    if (kinbas(inco).le.0) then
      kinbas(inco) = 0
      ntemp3 = 1
    endif
  else
    inco = -inco
    dp = xub(inco)
    xub(inco) = xlb(inco)-uno
    xlb(inco) = vbnd(listl)
    if (kinbas(inco).le.0) then
      kinbas(inco) = -1
      ntemp3 = 1
    endif
  endif
  idvid(listl,1) = -ivid(listl,1)
  idepth = ivid(listl,2)
  vbnd(listl) = dp
  xiobnd(listl) = -1.0D50
  listsa = listsa-1
  if (ntemp3.eq.0) return

```

```

C
C-----
C                                     -1      -1
C Calcular la solución que determina nudo LISTL: x = B  b - B  Nx
C                                     N
C-----

```

```

do i = 1,nrow
  y(i) = b(i)
end do
do j = 1,ncol
  if (kinbas(j).eq.(-1)) then
    de = xub(j)
  else if (kinbas(j).eq.0) then
    de = xlb(j)
  else
    cycle
  endif
  do i = la(j),la(j+1)-1
    ir = ia(i)
    y(ir) = y(ir)-a(i)*de
  end do
end do
call ftran (1)
do i = 1,nrow
  x(i) = y(i)
end do
return

```

```

C
C*****
C Nudo no es de interés.
C Adaptar límites de las variables y escoger otro nudo.
C*****
else
  inco = ivid(listl,1)
  icolit = iabs(inco)
  if (xiobnd(listl).ne.(-1.0D50)) then
    listsa = listsa-1
    inco = ivid(listl,1)
    qupdo = '      ',
    if (inco.gt.0) then
      iax = idnint(xub(icolit))+1
      write (qupdo,10) 'X>',iax
    else
      iax = idnint(xlb(icolit))-1
      write (qupdo,10) 'X<',iax
    endif
    if (icolit.le.nrow) then
      icolit = icolit+ncol-nrow-1

```

```

        else
            icolit = icolit-nrow
        endif
        if (logite.eq.1) write (logu,550) icolit,qfix,ivid(listl,2)+
+      1,qupdo,listsa,xiobnd(listl)*min
    endif
    if (inco.ge.0) then
        if (kinbas(inco).eq.(-1)) then
            ntemp3 = 1
            dp = xub(inco)-xlb(inco)
            dy = vbnd(listl)-xub(inco)
            if (dp.lt.dy) kinbas(inco) = 0
        endif
        xub(inco) = vbnd(listl)
    else
        inco = -inco
        if (kinbas(inco).eq.0) then
            ntemp3 = 1
            dy = xlb(inco)-vbnd(listl)
            dp = xub(inco)-xlb(inco)
            if (dp.lt.dy) kinbas(inco) = -1
        endif
        xlb(inco) = vbnd(listl)
    endif
    listl = listl-1
    go to 50
endif
C
10 format(a,i4)
550 format(i6,a8,i5,7x,a,i7,' -Nudo desechado en BKTRAK-',1pd14.7)
C
C      Fin de -BKTRAK-
C      end

subroutine branch(icol,idir)
C
C      Se generan dos nuevas ramas del árbol enumerativo separando
C      la variable X(ICOL).
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      common /iolog/ iu,lo,logu
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+      ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,listsa,noint,
+      idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+      xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+      kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+      ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
+      qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+      b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+      yen(1000)
C
C
C*****
C      ICOL indica variable de separación.
C      IDIR indica dirección de bifurcación escogida (1 o -1). Añadir
C      opuesta a la escogida a la lista de las a analizar más tarde
C*****
C      listl = listl+1
C      listsa = listsa+1
C      if (listl.gt.maxnod) then
C          write (lo,20) maxnod
C          stop
C      endif
C      if (idir.eq.(-1)) vbnd(listl) = xlb(icol)
C      if (idir.eq.1) vbnd(listl) = xub(icol)
C      ivid(listl,1) = idir*icol
C      ivid(listl,2) = idepth
C      xiobnd(listl) = xival
C      return
C
20 format('El número de nudos excede',i5,'; incrementar "MAXNOD"',
+      ' y los vectores correspondientes')
C
C      Fin de -BRANCH-
C      end

subroutine btran
C
C      t -1
C      Se calculan los multiplicadores simplex pi=c B realizando
C      B t
C      transformación inversa del vector c ; es decir
C      B
C      t
C      pi=((c E)E )...E
C      B k) k-1 1
C
C
C***** Descripción de vectores afectados *****
C      Y      := Vector de costes c
C      B
C      LE, IE, E := Vectores que definen en forma dispersa las
C      matrices elementales eta.
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+      ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+      kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+      ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
+      qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+      b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+      yen(1000)
C
C
C      if (neta.le.0) return
C      do i = neta,1,-1
C          ll = le(i)
C          kk = le(i+1)-1
C          ipiv = ie(ll)
C          dsum = cero
C          if (kk.gt.ll) then
C              do j = ll+1,kk
C                  ir = ie(j)
C                  dsum = dsum+e(j)*y(ir)
C              end do
C          endif
C          y(ipiv) = (y(ipiv)-dsum)/e(ll)
C      end do
C
C      return
C
C      Fin de -BTRAN-
C      end

subroutine chuzr(hrtype,jcolp,irowp,npivot,iptype,theta,ivout)
C
C      Esta rutina, una vez efectuada la transformación directa
C      -1
C      Y = B a
C      jcolp
C      del vector columna de la matriz de condiciones A, a
C      jcolp
C      que entrará en la base, selecciona la fila IROWP sobre la
C      que pivotará dicha columna en esta iteración del método
C      simplex; selecciona de esta forma la variable básica X
C      que ha de salir de la base.
C      irowp
C
C***** Descripción de parámetros *****
C      KINBAS := Estado en el que se encuentran los componentes del
C      del vector X de variables de decisión:
C      =0, variable en su límite inferior;
C      =-1, variable en su límite superior;
C      =-2, variable libre: especificada FR;
C      =>0, variable básica, el número indica
C      sobre qué vector fila pivota.
C
C      HRTYPE := Se calcula en FORMC de tal forma que:
C      =-2 si X(j) < XLB(j)-ZTOLZE;
C      = 0 si X(j) es factible y
C      = 2 si X(j) > XUB(j)+ZTOLZE.
C
C      XLB, XUB := Límites inferior y superior de las variables.
C
C      JH      := Vector que indica la columna con la que pivota
C      cada vector fila.
C
C      X      := Valor que en esta iteración toma el vector de
C      variables de decisión.
C      -1
C      Y      := El ya mencionado B a
C      jcolp
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      integer hrtype(*)
C
C      common /iolog/ iu,lo,logu
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+      ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,listsa,noint,

```



```

move = theta.gt.stepmn
unbndd = theta.ge.stepmx.or.irowp.eq.0
if (.not.unbndd) then
  if (hitlow) unbndd = xlb(jh(irowp)).le.(-plinfy)
  if (.not.hitlow) unbndd = xub(jh(irowp)).ge.plinfy
endif
C
C   if (.not.move) theta = cero
C
C   if (unbndd) then
C     write (lo, '( ' Solución no acotada ' ) ' )
C     stop
C   endif
C
C-----
C Se adapta el vector solución al final de una pivotación del
C método simplex.
C
C       x =x  -THETA*y ; i<=i<=m.
C       j      i
C       i      i
C-----
C   if (irowp.eq.nrwml) then
C-----
C   No ha habido pivotación; una variable no básica va de uno
C de sus límites al otro. Cambiar su estatus: si = -1,
C hacerlo 0; si = 0, hacerlo -1.
C-----
C       kinbas(jcolp) = -(kinbas(jcolp)+1)
C       ivout = jcolp
C       rpivot = cero
C       npivot = 0
C   else
C-----
C   Pivotación normal.
C-----
C       npivot = 1
C       rpivot = y(irowp)*is
C       ivout = jh(irowp)
C       kinbas(jcolp) = irowp
C       if (hitlow) then
C         kinbas(ivout) = 0
C         iptype = 0
C       else
C         kinbas(ivout) = -1
C         iptype = -1
C       endif
C       jh(irowp) = jcolp
C       if (.not.move) go to 1
C   endif
C
C-----
C Adaptar el vector x.
C-----
C   do i = 1,nrwml
C     x(i) = x(i)-y(i)*theta
C   end do
C 1 continue
C   x(irowp) = x(nrwml)
C   if (rcost.lt.cero) then
C     do i = 1,nrow
C       y(i) = -y(i)
C     end do
C   endif
C   return
C
C Fin de -CHUZR-
C end
C
C subroutine cycle(*,*)
C
C Esta rutina lleva a cabo la resolución de la relajación
C lineal del nudo determinado en BKTRAK. Se utiliza
C el método primal del simplex.
C
C*****
C   implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C   common /iolog/ iu,lo,logu
C
C   common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,lists,noint,
C + idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
C + xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C   common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwml,
C + ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
C + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C + yen(1000)

```

```

C
C   1 continue
C   call bktrak
C
C*****
C Si la lista de nudos a analizar esta vacía, terminar.
C*****
C   if (listl.eq.0) return 1
C
C*****
C Si no esta vacía, optimizar nudo mediante simplex.
C*****
C   call normal
C   inco = ivid(listl,1)
C   icolit = iabs(inco)
C   qupdo = ' '
C   if (inco.gt.0) then
C     iax = idmint(xub(icolit))
C     write (qupdo,10) 'X<',iax
C   else
C     iax = idmint(xlb(icolit))
C     write (qupdo,10) 'X>',iax
C   endif
C   if (icolit.le.nrow) then
C     icolit = icolit+ncol-nrow-1
C   else
C     icolit = icolit-nrow
C   endif
C   idepth = idepth+1
C   if (qstat.eq.'FEAS') then
C     if (x(1).le.xincva) then
C       qplus = 'ABANDONO'
C       if (logite.eq.1) write (logu,20) icolit,qfix,idepth,qupdo,
C + listsa,noint,itcnt,' P',x(1)*min,qplus
C       go to 1
C     endif
C     qplus = ' '
C     if (logite.eq.1) write (logu,20) icolit,qfix,idepth,qupdo,
C + listsa,noint,itcnt,' P',x(1)*min,qplus
C   else
C     if (logite.eq.1) write (logu,30) icolit,qfix,idepth,qupdo,
C + listsa,noint,itcnt,' P'
C     go to 1
C   endif
C
C*****
C Comprobar si solución conseguida es entera factible.
C Si es, coger otro nudo de la lista y comenzar ciclo primal.
C Si no, retornar para calcular penalizaciones.
C*****
C   call testx
C   if (qstat.eq.'ENTERA') go to 1
C   return 2
C
C 10 format(a,i4)
C 20 format(i6,a8,i5,7x,a,i7,4x,i7,8x,i4,a,1pd18.7,1x,a)
C 30 format(i6,a8,i5,7x,a,i7,4x,i7,8x,i4,a,4x,'No factible')
C
C Fin de -CYCLE-
C end
C
C subroutine dchuzc(jcolp,irowp,npivot,iptype,theta,ivout)
C
C Se determina la variable no básica JCOLP que ha de entrar en
C la base de acuerdo al método dual del simplex.
C
C Si el programa lineal del nudo analizándose no es factible
C se hace JCOLP=0.
C
C*****
C   implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C   common /defpa/ nrmax,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C   common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,lists,noint,
C + idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
C + xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C   common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwml,
C + ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
C + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C + yen(1000)
C
C   jcolp = 0
C   if (iptype.eq.(-1)) then
C

```

```

C*****
C  Se ha impuesto el límite x      <=n
C                                irowp irowp
C*****
      dp = plinfy
      do j = 1,ncol
        if (dabs(xub(j)-xlb(j)).gt.ztolze.and.kinbas(j).le.0) then
          call unpack (j)
          call ftran (iuno)
          if (kinbas(j).eq.0) then
            if (y(irowp).gt.ztolpv) then
              de = y(1)/y(irowp)
              if (de.lt.dp) then
                jcolp = j
                dp = de
              endif
            endif
          else if (y(irowp).lt.(-ztolpv)) then
            de = y(1)/y(irowp)
            if (de.lt.dp) then
              jcolp = j
              dp = de
            endif
          endif
        endif
      end do
      if (jcolp.eq.0) return

C-----
C      -1
C  Se calcula B N      y el paso
C                                jcolp
C-----
      call unpack (jcolp)
      call ftran (iuno)
      theta = (x(irowp)-xub(jh(irowp)))/y(irowp)
      else
C*****
C  Se ha impuesto el límite x      >=n      +1
C                                irowp irowp
C*****
      dp = -plinfy
      do j = 1,ncol
        if (dabs(xub(j)-xlb(j)).gt.ztolze.and.kinbas(j).le.0) then
          call unpack (j)
          call ftran (iuno)
          if (kinbas(j).eq.(-1)) then
            if (y(irowp).gt.ztolpv) then
              de = y(1)/y(irowp)
              if (de.gt.dp) then
                jcolp = j
                dp = de
              endif
            endif
          else if (y(irowp).lt.(-ztolpv)) then
            de = y(1)/y(irowp)
            if (de.gt.dp) then
              jcolp = j
              dp = de
            endif
          endif
        endif
      end do
      if (jcolp.eq.0) return

C-----
C      -1
C  Se calcula B N      y el paso
C                                jcolp
C-----
      call unpack (jcolp)
      call ftran (iuno)
      theta = (x(irowp)-xlb(jh(irowp)))/y(irowp)
      npivot = 1
      do i = 1,nrow
        x(i) = x(i)-y(i)*theta
      end do
C
      if (kinbas(jcolp).eq.(-1)) then
        x(irowp) = xub(jcolp)+theta
      else
        x(irowp) = xlb(jcolp)+theta
      endif
      ivout = jh(irowp)
      kinbas(jcolp) = irowp
      kinbas(ivout) = iptype
      jh(irowp) = jcolp
      return
C
C  Fin de -DCHUZC-
end

```

```

subroutine dchuzr(irowp,iptype)
C
C  Se determina para una iteración del método simplex dual la
C  variable x      que ha de salir de la base.
C                                irowp
C
C  Si el problema es factible en el primal se acaba; se ha
C  llegado al óptimo haciéndose IROWP=0.
C*****
C  implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C  common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C  + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C  common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,lists,a,noint,
C  + idepth,qfix,xincva,xival,revbnd,dipart(1000),xinc(4000),
C  + xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C  common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C  + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
C  + ncol,nelem,neta,nleta,neta,ninf,min,nopt,logite,qname(4000),
C  + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C  + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C  + yen(1000)
C
C*****
C  Se escoge la fila con mayor no factibilidad.
C*****
      irowp = 0
      qstat = 'FEAS'
      dp = -plinfy
      do i = 2,nrow
        inco = jh(i)
        if (x(i).le.xlb(inco)-ztolze) then
C-----
C  La variable básica que pivota en la fila I esta por debajo de
C  su límite inferior
C-----
          qstat = 'INFEAS'
          de = xlb(inco)-x(i)
          if (de.gt.dp) then
            dp = de
            iptype = 0
            irowp = i
          endif
        else if (x(i).ge.xub(inco)+ztolze) then
C-----
C  La variable básica que pivota en la fila I esta por encima de
C  su límite superior
C-----
          qstat = 'INFEAS'
          de = x(i)-xub(inco)
          if (de.gt.dp) then
            dp = de
            iptype = -1
            irowp = i
          endif
        endif
      end do
C
      return
C
C  Fin de -DCHUZR-
end

double precision function dnorm (n,x)
C
C  implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C  doubleprecision x(n)
C
      sum = 0.0D+00
      if (n.gt.0) then
        do i = 1,n
          sum = sum+dabs(x(i))
        end do
      endif
      d = n
      d = sum/dsqrt(d)
      dnorm = d
C
      return
C
C  Fin de -DNORM-
end

subroutine formc(hrtype)
C

```

```

C      Se forma el vector de coste c en Y. En la Phase II es Y(1)=1
C      B
C      y todos los demás componentes cero. En Phase I Y(1)=0
C      añadiéndose un una variable artificial Y(i)=1 si x <1
C      i i
C      o Y(i)=-1 si x >u (recordemos que el problema es de
C      i i
C      maximización).
C
C***** Descripción de vectores afectados *****
C      XLB, XUB := Límites inferior y superior de las variables
C      del problema.
C
C      JH      := Vector que indica la columna con la que pivota
C      cada vector fila.
C
C      X      := Vector solución actual =B-1 b.
C
C      Y      := Vector de coste a formar.
C
C      QSTAT  := Estado de la solución = 'I' si no es factible;
C              = 'F' si es factible.
C
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      integer hrtype(*)
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C      + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C      + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
C      + ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
C      + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C      + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C      + yen(1000)
C
C      qstat = 'INFEAS'
C      suminf = cero
C      y(1) = cero
C      ninf = 0
C
C      do i = 2,nrow
C      icol = jh(i)
C      xi = x(icol)
C      res = xlb(icol)-xi
C      if (res.le.ztolze) then
C      res = xi-xub(icol)
C      if (res.le.ztolze) then
C      y(i) = cero
C      hrtype(i) = 0
C      else
C      y(i) = -uno
C      hrtype(i) = 2
C      ninf = ninf+1
C      suminf = suminf+res
C      endif
C      else
C      y(i) = uno
C      hrtype(i) = -2
C      ninf = ninf+1
C      suminf = suminf+res
C      endif
C      end do
C
C      if (ninf.eq.0) then
C      y(1) = uno
C      qstat = 'FEAS'
C      endif
C
C      return
C
C      Fin de -FORMC-
C      end
C
C      subroutine ftran(ipar)
C
C      Se lleva a cabo la transformación directa siguiente:
C
C      y <--- B-1 a
C      jcolp
C
C      La matriz B-1 esta almacenada como producto de matrices
C      elementales ETA.
C
C***** Descripción de los parametros *****
C
C      IPAR := Indica que transformaciones ETA se van a aplicar

```

```

C      al vector a
C      jcolp
C
C      Y      := Vector de entrada con a-1 y salida con B-1 a
C      jcolp jcolp
C
C      LE, IE, E := Vectores donde se guarda toda la información
C      relativa a la matriz inversa de la base en
C      forma de producto de matrices elementales.
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C      + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
C      + kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
C      + ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
C      + qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
C      + b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
C      + yen(1000)
C
C      if (ipar.eq.1) then
C      nfe = 1
C      nle = neta
C      else
C      nfe = nleta+1
C      nle = neta
C      endif
C      if (nfe.gt.nle) return
C      do ik = nfe,nle
C      ll = le(ik)
C      kk = le(ik+1)-1
C      ipiv = ie(ll)
C      if (dabs(y(ipiv)).gt.0) then
C      y(ipiv) = y(ipiv)/e(ll)
C      if (kk.gt.ll) then
C      do j = ll+1,kk
C      ir = ie(j)
C      y(ir) = y(ir)-e(j)*y(ipiv)
C      end do
C      endif
C      endif
C      end do
C
C      return
C
C      Fin de -FTRAN-
C      end
C
C      subroutine input
C
C      Entrada de los datos del problema a resolver.
C
C***** Descripción de vectores afectados *****
C
C      B      := Termino de la derecha del problema.
C
C      KINBAS := Estado en el que se encuentran los componentes
C      del vector X de variables de decisión:
C      =0, variable en su límite inferior;
C      =-1, variable en su límite superior;
C      =-2, variable libre: especificada FR;
C      =>0, variable básica, el número indica
C      el vector fila sobre el que pivota.
C
C      QNAME  := Vector CHARACTER donde se guardan los nombres
C      atribuidos en el fichero MPS a las variables
C      de decisión y a las condiciones.
C
C      XLB, XUB := Límites inferior y superior de las variables.
C
C      JH      := Vector que indica la columna con la que pivota
C      cada vector fila.
C
C      LA, IA, A := Vectores donde se guarda toda la información
C      relativa a la matriz de los coeficientes
C      de las condiciones.
C
C      QNAMPO  := Variable CHARACTER con el nombre del caso a
C      resolver.
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      character*1 idf*20,char1*80,qn(3)*8,k1,k2,k3,k4
C
C      common /iolog/ iu,lo,logu
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
C      + ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos

```

```

C      common /nodbb/ ipart(1000),ivid(1500,2),nvare,list1,lists2,noint,
+      idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+      xioibnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+      kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+      ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
+      qnampo,qstat,a(15000),suminf,rcostr,rpivot,x(1000),y(1000),
+      b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+      yen(1000)
C
C      min = -1
C      inibb = 0
C      nrow = 1
C      itcnt = 0
C      nvare = 0
C      invfrq = 50
C      itrfrq = 100000
C      logite = 0
C      qcs = ' '
C      a(1) = uno
C      b(1) = cero
C      ia(1) = 1
C      la(1) = 1
C      jh(1) = 1
C      kinbas(1) = 1
C      xincva = -1.D75
C      nsolen = 0
C      initbd = 0
C
C      write (*,'(A)') ' Archivo de datos del problema?'
C      read (*,'(A)') idf
C      open (iu,file=idf)
C      open (lo,file=idf(:index(idf,' ') - 1)//'.pbb')
C
C*****
C      Leer las especificaciones del proceso de optimización.
C*****
1 continue
C      read (iu,'(A)') char1
C
C      if (index(char1,'NAME' ) .ne. 1) then
C      if (index(char1,'MAXIMIZAR') .ne. 0) then
C      min = 1
C      go to 1
C      endif
C      ind = index(char1,'FRECUENCIA DE REINVERSION')
C      if (ind .ne. 0) then
C      read (char1(ind+22:),'(BN,I50)') invfrq
C      go to 1
C      endif
C      ind = index(char1,'LIMITE DE ITERACIONES')
C      if (ind .ne. 0) then
C      read (char1(ind+18:),'(BN,I50)') itrfrq
C      go to 1
C      endif
C      ind = index(char1,'COTA INICIAL F.O.')
C      if (ind .ne. 0) then
C      read (char1(ind+17:),'(BN,I50)') initbd
C      go to 1
C      endif
C      ind = index(char1,'PROCESO ITERATIVO')
C      if (ind .ne. 0) then
C      read (char1(ind+17:),'(BN,I50)') logu
C      if (logu .ne. 0) then
C      open(logu,file=idf//'.LOG')
C      else
C      logu = lo
C      endif
C      logite = 1
C      go to 1
C      endif
C      ind = index(char1,'TOLERANCIA DE CERO')
C      if (ind .ne. 0) then
C      read (char1(ind+15:),'(BN,D50.0)') ztolze
C      go to 1
C      endif
C      ind = index(char1,'TOLERANCIA DE COSTES')
C      if (ind .ne. 0) then
C      read (char1(ind+17:),'(BN,D50.0)') ztcost
C      go to 1
C      endif
C      ind = index(char1,'TOLERANCIA DE PIVOTE')
C      if (ind .ne. 0) then
C      read (char1(ind+17:),'(BN,D50.0)') ztolpv
C      go to 1
C      endif
C      go to 1
C      endif
C
C      if (initbd .ne. 0) xincva = dble(initbd)*min

```

```

      backspace iu
C*****
C      Leer el fichero de datos en formato MPS.
C*****
5 continue
C      read (iu,8000) k1,k2,k3,k4,qn(1),qn(2),vtemp1,qn(3),vtemp2
C
C      if (k1 .ne. 'E' .or. k2 .ne. 'N') then
C      if (k1 .eq. ' ') then
C      go to (210,320,500,550,595) 1
C      endif
C      if (k1 .eq. 'N' .and. k2 .eq. 'A') then
C      qnampo = qn(2)
C      go to 5
C      endif
C      if (k1 .eq. 'R' .and. k2 .eq. '0') then
C      l = 1
C      go to 5
C      endif
C      if (k1 .eq. 'C' .and. k2 .eq. '0') then
C      l = 2
C      go to 5
C      endif
C      if (k1 .eq. 'R' .and. k2 .eq. 'H') then
C      l = 3
C      go to 5
C      endif
C      if (k1 .eq. 'B' .and. k2 .eq. '0') then
C      l = 4
C      go to 5
C      endif
C      if (k1 .eq. 'R' .and. k2 .eq. 'A') then
C      l = 5
C      go to 5
C      endif
C      if (k1 .eq. 'I' .and. k2 .eq. 'N' .and. k3 .eq. 'T') then
C      nvare = ncol+1
C      go to 5
C      endif
C      write (lo,'('' *** Clave '' ,A,' ' no reconocida'')')
C      +      k1//k2//k3//k4
C      stop
C
C*****
C      ROWS: nueva fila.
C*****
210 continue
C      nrow = nrow+1
C      qname(nrow) = qn(1)
C
C-----
C      Determinar tipo de fila.
C-----
C      if (k2 .eq. 'L' .or. k3 .eq. 'L') then
C      xlb(nrow) = cero
C      xci(nrow) = cero
C      xub(nrow) = plinfy
C      xcs(nrow) = plinfy
C      a(nrow) = uno
C      endif
C      if (k2 .eq. 'E' .or. k3 .eq. 'E') then
C      xlb(nrow) = cero
C      xci(nrow) = cero
C      xub(nrow) = cero
C      xcs(nrow) = cero
C      a(nrow) = uno
C      endif
C      if (k2 .eq. 'G' .or. k3 .eq. 'G') then
C      xlb(nrow) = cero
C      xci(nrow) = cero
C      xub(nrow) = plinfy
C      xcs(nrow) = plinfy
C      a(nrow) = -uno
C      endif
C      if (k2 .eq. 'N' .or. k3 .eq. 'N') then
C      nrow = nrow-1
C      qname(1) = qn(1)
C      ncol = nrow
C      go to 5
C      endif
C
C      b(nrow) = 0.0
C      ia(nrow) = nrow
C      la(nrow) = nrow
C      jh(nrow) = nrow
C      kinbas(nrow) = nrow
C      nelem = nrow
C      ncol = nrow
C      go to 5
C
C*****
C      COLUMNS: coeficientes de la matriz de condiciones.

```

```

C*****
320  continue
      j = 2
      if (dabs(vtemp1).lt.ztolze) then
        if (dabs(vtemp2).lt.ztolze) go to 5
        j = 3
        vtemp1 = vtemp2
      endif
C-----
C Comprobar si el vector columna ya estaba definido.
C-----
      if (qn(1).ne.qcs) then
        do i = nrow+1,ncol
          if (qn(1).eq.qname(i)) then
            write (lo,8250) qn(1)
            stop
          endif
        end do
        if (ncol+1.gt.ncolma) then
          write (lo,8555) ncolma
          stop
        endif
        ncol = ncol+1
        qcs = qn(1)
        qname(ncol) = qcs
        la(ncol) = nelelem+1
        xlb(ncol) = cero
        xci(ncol) = cero
        xub(ncol) = plinfy
        xcs(ncol) = plinfy
        kinbas(ncol) = 0
      endif
C-----
C Comprobar si la fila existe.
C-----
330  continue
      do i = 1,nrow
        if (qn(j).eq.qname(i)) then
          nelelem = nelelem+1
          if (nelelem.ge.namax) then
            write (lo,8550) namax
            stop
          endif
          ia(nelelem) = i
          if (qn(j).eq.qname(1).and.min.eq.1) vtemp1 = -vtemp1
          a(nelelem) = singl(vtemp1)
          la(ncol+1) = nelelem+1
          if (j.eq.3.or.dabs(vtemp2).lt.ztolze) go to 5
          j = 3
          vtemp1 = vtemp2
          go to 330
        endif
      end do
      write (lo,8300) qn(j),qn(1)
      stop
C-----
C *****
C RHS: término de la derecha.
C *****
500  continue
      j = 2
      if (dabs(vtemp1).lt.ztolze) then
        if (dabs(vtemp2).lt.ztolze) go to 5
        j = 3
        vtemp1 = vtemp2
      endif
C-----
C Comprobar si la fila existe.
C-----
530  continue
      do i = 1,nrow
        if (qn(j).eq.qname(i)) then
          b(i) = vtemp1
          if (j.eq.3.or.dabs(vtemp2).lt.ztolze) go to 5
          j = 3
          vtemp1 = vtemp2
          go to 530
        endif
      end do
      write (lo,8350) qn(j),qn(1)
      stop
C-----
C *****
C BOUNDS: límites de las variables.
C *****
550  continue
      j = 2
C-----
C Comprobar si la fila existe.

```

```

C-----
580  continue
      do i = nrow+1,ncol
        if (qn(j).eq.qname(i)) then
          if (k2.eq.'U'.and.k3.eq.'P') then
            xub(i) = vtemp1
            xcs(i) = vtemp1
          endif
          if (k2.eq.'L'.and.k3.eq.'O') then
            xlb(i) = vtemp1
            xci(i) = vtemp1
          endif
          if (k2.eq.'F'.and.k3.eq.'X') then
            xlb(i) = vtemp1
            xci(i) = vtemp1
            xub(i) = vtemp1
            xcs(i) = vtemp1
          endif
          if (k2.eq.'F'.and.k3.eq.'R') then
            xlb(i) = -plinfy
            xci(i) = -plinfy
            xub(i) = plinfy
            xcs(i) = plinfy
            kinbas(i) = -2
          endif
          if (k2.eq.'M'.and.k3.eq.'I') then
            xlb(i) = -plinfy
            xci(i) = -plinfy
            kinbas(i) = -1
          endif
          if (k2.eq.'P'.and.k3.eq.'L') then
            xub(i) = plinfy
            xcs(i) = plinfy
          endif
          if (j.eq.3) go to 5
          j = 3
          vtemp1 = vtemp2
          if (qn(j).eq.' ') go to 5
          go to 580
        endif
      end do
      write (lo,8400) qn(j),qn(1)
      go to 5
C-----
C *****
C RANGES: margen de validez de los elementos término de derecha
C *****
595  continue
      j = 2
      if (dabs(vtemp1).lt.ztolze) then
        if (dabs(vtemp2).lt.ztolze) go to 5
        j = 3
        vtemp1 = vtemp2
      endif
C-----
C Comprobar si la fila existe.
C-----
597  continue
      do i = 1,nrow
        if (qn(j).eq.qname(i)) then
          if (dabs(xub(i)).lt.ztolze) then
            if (vtemp1.gt.ztolze) then
              xub(i) = vtemp1
              xcs(i) = vtemp1
              a(i) = -1.0
            else
              xub(i) = dabs(vtemp1)
              xcs(i) = dabs(vtemp1)
            endif
          else
            xub(i) = dabs(vtemp1)
            xcs(i) = dabs(vtemp1)
          endif
          if (j.eq.3) go to 5
          j = 3
          vtemp1 = vtemp2
          if (qn(j).eq.' ') go to 5
          go to 597
        endif
      end do
      write (lo,8450) qn(j),qn(1)
      stop
C-----
C *****
C Fin de INPUT; se imprimen algunas estadísticas.
C *****
      endif
      if (nvare.eq.0) nvare = ncol+1
      write (lo,8650) qnampo
      write (lo,8700) nrow,nrow-1,ncol-nrow,nrow-1,nelelem-nrow,ncol-
nvare
+ +1,dble(nelelem-nrow)*100./(nrow*(ncol-nrow))

```



```

        nrwm1 = nrow+1
        close(iu)
        return
C
6500 format(2i3,12,i3)
7000 format(4i4,i10)
8000 format(4a1,a,2x,a,2x,f12.0,3x,a,2x,f12.0)
8250 format('*** Variable ',a,' previamente definida')
8300 format('*** La condición ',a,' en la línea COLUMN ',a,
+ ' no existe.')
8350 format('*** La condición ',a,' en la línea RHS ',a,' no exis-
te.')
8400 format('<Cuidado! La variable ',a,' de la línea BOUNDS ',a,
+ ' no existe o los coeficientes de la misma son nulos')
8450 format('*** La condición ',a,' en la línea RANGES ',a,
+ ' no existe.')
8550 format('*** El número de coeficientes de la matriz A', ' exce-
de de',i7,'; el programa se para.')
8555 format('*** El número de variables del problema', ' exce-
de de',i7,'; el programa se para.')
8650 format('Problema ',a)
8700 format('*** Estadísticas del Problema'/4x,i5,' Fila(s)'/4x,i5,
+ ' Restricción(es)'/4x,i5,' Variable(s) de decisión'/4x,i5,
+ ' Variable(s) de ', 'holgura/artificiales'/4x,i5,
+ ' Elementos no cero'/4x,i5,' Variable(s) entera(s)'/4x,
+ 'Densidad de la matriz de coeficientes A: ',f8.3,'%')
C
C Fin de -INPUT-
end

subroutine invert
C
C Se refactoriza la inversa de la matriz básica B en la
C forma LU.
C
C***** Descripción de vectores afectados *****
C
C B := Término de la derecha del problema.
C
C KINBAS := Estado en el que se encuentran los componentes
C del vector X de variables de decisión:
C =0, variable en su límite inferior;
C =-1, variable en su límite superior;
C =-2, variable libre: especificada FR;
C =>0, variable básica, el número indica
C el vector fila sobre el que pivota.
C
C XLB, XUB := Límites inferior y superior de las variables.
C
C JH := Vector que indica la columna con la que pivota
C cada vector fila.
C
C LA, IA, A := Vectores donde se guarda toda la información
C relativa a la matriz de los coeficientes
C de las condiciones.
C
C -1 -1 -1
C X := vector B b - B Nx = B (b-Nx)
C
C LE, IE, E := Vectores donde se guarda toda la información
C relativa a la matriz de las
C transformaciones elementales eta.
C
C MREG, HREG, VREG := Vectores de trabajo.
C
C*****
C
C implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C integer mreg(4000),hreg(4000),vreg(4000)
C
C common /iolog/ iu,lo,logu
C
C common /defpa/ nrmax,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+ ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+ kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+ ncol,nlelem,neta,nleta,ninf,min,nopt,logite,qname(4000),
+ qnampp,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+ b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+ yen(1000)
C
C
C Definir parámetros.
C
C ztrel = 0.05
10 continue
neta = 0
nleta = 0

```

```

        nueta = 0
        nelem = 0
        nlelem = 0
        nuelem = 0
        nabove = 0
        le(1) = 1
        lr1 = 1
        kr1 = 0
        lr4 = nrwm1
        kr4 = nrow
C
C-----
C Se ponen las variables de holgura y artificiales en la parte
C 4, el resto en la parte 1
C-----
        do i = 1,nrow
            if (jh(i).le.nrow) then
                lr4 = lr4-1
                mreg(lr4) = jh(i)
                vreg(lr4) = jh(i)
            else
                kr1 = kr1+1
                vreg(kr1) = jh(i)
            endif
            hreg(i) = -1
            jh(i) = 0
        end do
C
        kr3 = lr4-1
        lr3 = lr4
C
        do i = lr4,kr4
            ir = mreg(i)
            hreg(ir) = 0
            jh(ir) = ir
            kinbas(ir) = ir
        end do
C
C-----
C Recuperar vectores de debajo del pico y contar filas.
C-----
        nbnonz = kr4-lr4+1
        if (kr1.ne.0) then
            j = lr1
210         continue
            iv = vreg(j)
            ll = la(iv)
            kk = la(iv+1)-1
            ircnt = 0
            do i = ll,kk
                nbnonz = nbnonz+1
                ir = ia(i)
                if (hreg(ir).lt.0) then
                    ircnt = ircnt+1
                    hreg(ir) = hreg(ir)-1
                    irp = ir
                endif
            end do
            if (ircnt-1.gt.0) go to 300
            if (ircnt-1.eq.0) go to 250
            write (lo,8000)
            kinbas(iv) = 0
            vreg(j) = vreg(kr1)
            kr1 = kr1-1
            if (.gt.kr1) go to 310
            go to 210
250         continue
            vreg(j) = vreg(kr1)
            kr1 = kr1-1
            lr3 = lr3-1
            vreg(lr3) = iv
            mreg(lr3) = irp
            hreg(irp) = 0
            jh(irp) = iv
            kinbas(iv) = irp
            if (.gt.kr1) go to 310
            go to 210
300         continue
            if (.ge.kr1) go to 310
            j = j+1
            go to 210
C
C-----
C Recuperar el resto de los vectores de encima y debajo del
C pico y establecer contador de mérito de las columnas.
C-----
310         continue
            nvrem = 0
            if (kr1.ne.0) then
                j = lr1
320         continue
                iv = vreg(j)
                ll = la(iv)

```

```

      kk = la(iv+1)-1
      ircnt = 0
      do i = 11, kk
        ir = ia(i)
        if (hreg(ir).eq.(-2)) then
C-----
C   Pivota por encima del pico (parte de L).
C-----
          nabove = nabove+1
          irowp = ir
          call unpack (iv)
          call wreta (irowp)
          nleta = neta
          jh(ir) = iv
          kinbas(iv) = ir
          vreg(j) = vreg(kr1)
          kr1 = kr1-1
          nvrem = nvrem+1
          hreg(ir) = iv
          go to 940
        else if (hreg(ir).lt.0) then
          ircnt = ircnt+1
          irp = ir
        endif
      end do
C
      if (ircnt-1.gt.0) go to 1000
      if (ircnt-1.eq.0) go to 900
      write (10,8000)
      kinbas(iv) = 0
      vreg(j) = vreg(kr1)
      nvrem = nvrem+1
      kr1 = kr1-1
      if (j.gt.kr1) go to 1010
      go to 320
C-----
C   Poner vector debajo del pico.
C-----
      900      continue
             vreg(j) = vreg(kr1)
             nvrem = nvrem+1
             kr1 = kr1-1
             lr3 = lr3-1
             vreg(lr3) = iv
             mreg(lr3) = irp
             hreg(irp) = 0
             jh(irp) = iv
             kinbas(iv) = irp
C-----
C   Cambiar contador de filas.
C-----
      940      continue
             do ii = 11, kk
               iir = ia(ii)
               if (hreg(iir).lt.0) hreg(iir) = hreg(iir)+1
             end do
             if (j.gt.kr1) go to 1010
             go to 320
      1000      continue
             if (j.ge.kr1) go to 1010
             j = j+1
             go to 320
      1010      continue
             if (nvrem.gt.0) go to 310
C-----
C   Obtener contador de meritos.
C-----
      if (kr1.ne.0) then
        do j = lr1, kr1
          iv = vreg(j)
          ll = la(iv)
          kk = la(iv+1)-1
          imcnt = 0
          do i = 11, kk
            ir = ia(i)
            if (hreg(ir).lt.0) imcnt = imcnt-hreg(ir)-1
          end do
          mreg(j) = imcnt
        end do
C-----
C   Ordenar columnas segun mérito usando el algo. SHELL.
C-----
      1106      isd = 1
             continue
             if (kr1.lt.2*isd) go to 1108
             isd = 2*isd
             go to 1106
      1108      continue

```

```

             isd = isd-1
             continue
             if (isd.gt.0) then
               isk = 1
             1102      continue
                     isj = isk
                     isl = isk+isd
                     isy = mreg(isl)
                     isz = vreg(isl)
             1103      continue
                     if (isy.lt.mreg(isj)) go to 1104
             1105      continue
                     isl = isj+isd
                     mreg(isl) = isy
                     vreg(isl) = isz
                     isk = isk+1
                     if (isk+isd.le.kr1) go to 1102
                     isd = (isd-1)/2
                     go to 1101
             1104      continue
                     isl = isj+isd
                     mreg(isl) = mreg(isj)
                     vreg(isl) = vreg(isj)
                     isj = isj-isd
                     if (isj.gt.0) go to 1103
                     go to 1105
             endif
C-----
C   Fin de rutina de ordenación.
C-----
C-----
C-----
C   Sacar etas debajo del pico (parte de U).
C-----
             endif
             endif
             nsick = 0
             nbelow = 0
             nelast = nemax
             ntlast = ntmax
             le(ntlast+1) = nelast+1
C-----
C
      lr = lr3
      if (lr3.ge.lr4) lr = lr4
      if (lr.le.kr4) then
        jk = kr4+1
        do jj = lr, kr4
          jk = jk-1
          iv = vreg(jk)
          i = mreg(jk)
          nbelow = nbelow+1
          if (iv.le.nrow) nsick = nsick+1
          ll = la(iv)
          kk = la(iv+1)-1
          if (kk.gt.ll.or.dabs(a(ll)-1.0).gt.ztolze) then
            nueta = nueta+1
            do j = 11, kk
              ir = ia(j)
              if (ir.eq.i) then
                ep = dble(a(j))
              else
                ie(nelast) = ir
                e(nelast) = dble(a(j))
                nelast = nelast-1
                nuelem = nuelem+1
              endif
            end do
            ie(nelast) = i
            e(nelast) = ep
            le(ntlast) = nelast
            nelast = nelast-1
            ntlast = ntlast-1
            nuelem = nuelem+1
          endif
        end do
      endif
      end do
      if (kr1.ne.0) then
C-----
C   Descomponer en forma LU en pico.
C-----
        do j = lr1, kr1
          iv = vreg(j)
          call unpack (iv)
          call ftran (idos)
          ztoly = ztolpv
          continue
          ymax = cero
          irowp = 0
          ircmin = -999999
          do i = 1, nrow

```

```

        yval = dabs(y(i))
        if (yval.ge.ztoly.and.hreg(i).lt.0) then
            ymax = dmax1(yval,ymax)
            if (hreg(i).gt.ircmin) then
                ircmin = hreg(i)
                irowp = i
            endif
        endif
    end do
    if (irowp.eq.0) then
        write (lo,8000)
        kinbas(iv) = 0
        cycle
    endif
    ztmin = ztrel*ymax
    if (dabs(y(irowp)).lt.ztmin) then
        ztoly = ztmin
        go to 2070
    endif
    incr = hreg(irowp)+3
C-----
C  Escribir etas L y U.
C-----
        if (j.lt.kr1) then
            nelelem = nelelem+1
            ie(nelelem) = irowp
            e(nelelem) = y(irowp)
        endif
    do i = 1,nrow
        if (i.ne.irowp.and.dabs(y(i)).gt.toleta) then
            if (hreg(i).lt.0) then
C-----
C  Elementos eta de L.
C-----
                nelelem = nelelem+1
                ie(nelelem) = i
                e(nelelem) = y(i)
            else
C-----
C  Elementos eta de U.
C-----
                ie(nelast) = i
                e(nelast) = y(i)
                nelast = nelast+1
                nuelelem = nuelelem+1
            endif
        endif
    end do
    jh(irowp) = iv
    kinbas(iv) = irowp
    nueta = nueta+1
    ie(nelast) = irowp
    if (j.eq.kr1) then
        e(nelast) = y(irowp)
    else
        e(nelast) = uno
        neta = neta+1
        le(neta+1) = nelelem+1
    endif
    nuelelem = nuelelem+1
    le(ntlast) = nelast
    nelast = nelast+1
    ntlast = ntlast+1
C-----
C  Adaptar contadores de filas.
C-----
    do i = 1,nrow
        if (dabs(y(i)).gt.toleta.and.hreg(i).lt.0) then
            hreg(i) = hreg(i)-incr
            if (hreg(i).ge.0) hreg(i) = -1
        endif
    end do
    hreg(irowp) = 0
end do
C-----
C  Mezclar etas L y U.
C-----
        endif
        nleta = neta
        neta = nleta+nueta
        nlelem = nelelem
        nelelem = nuelelem+1
        if (nelelem.ge.nemax) then
            write (lo,8005)
            stop
        endif
        if (nuelelem.gt.0) then
C-----

```

```

C-----
C  Desplazar vectores IE y E de los elementos de U.
C-----
        nf = nemax-nuelem+1
        incr = 0
        do i = nf,nemax
            incr = incr+1
            iaux = nlelem+incr
            ie(iaux) = ie(i)
            e(iaux) = e(i)
        end do
C-----
C  Redefinir vector LE de los elementos eta.
C-----
        idif = nemax-nlelem-nuelem
        nf = ntmax-nueta+1
        incr = 0
        do i = nf,ntmax
            incr = incr+1
            le(nleta+incr) = le(i)-idif
        end do
        le(neta+1) = nelelem+1
    endif
C-----
C  Insertar holgura en columnas borradas.
C-----
    do i = 1,nrow
        if (jh(i).eq.0) then
            jh(i) = i
            irowp = i
            call unpack (i)
            call ftran (iuno)
            call wreta (irowp)
        endif
    end do
C-----
C  Adaptar x:

$$x = B \begin{matrix} -1 & -1 & -1 \\ b & -B & Nx \\ N & & N \end{matrix} = B \begin{matrix} -1 \\ (b-Nx) \\ N \end{matrix}$$

C-----
    do i = 1,nrow
        y(i) = b(i)
    end do
    do j = 1,ncol
        if (kinbas(j).eq.(-2)) then
            de = 0
        else if (kinbas(j).eq.(-1)) then
            de = xub(j)
        else if (kinbas(j).eq.0) then
            de = xlb(j)
        else
            cycle
        endif
        do i = la(j),la(j+1)-1
            ir = ia(i)
            y(ir) = y(ir)-a(i)*de
        end do
    end do
    call ftran (iuno)
    do i = 1,nrow
        x(i) = y(i)
    end do
C-----
C  Imprimir estadísticas de inversión.
C-----
        nofd = nelelem-neta
        nstr = nrow-nsclck
        if (inibb.eq.0) write (lo,8500) nbnonz,nstr,nabove,nbelow,nlelem,
            + nleta,nuelem,nueta,nofd,neta
C-----
C  Calcular en y:  $b - B \begin{pmatrix} -1 \\ b - Nx \\ N \end{pmatrix} - Nx$  y chequear error rela.
C-----
    do i = 1,nrow
        y(i) = b(i)
    end do
    x2max = cero
    do j = 1,ncol
        if (kinbas(j).eq.(-2)) then
            de = 0
        else if (kinbas(j).eq.(-1)) then
            de = xub(j)
        else if (kinbas(j).eq.0) then
            de = xlb(j)
        else
            de = x(kinbas(j))
        endif
    end do

```

```

do i = la(j),la(j+1)-1
  ir = ia(i)
  y(ir) = y(ir)-a(i)*de
end do
end do
call ftran (iuono)
eemax = cero
do i = 1,nrow
  eemax = dmax1(dabs(y(i)),eemax)
  x(i) = x(i)+y(i)
  x2max = dmax1(dabs(x(i)),x2max)
end do
if (x2max.ne.cero) ermax = eemax/x2max
if (inibb.eq.0) write (lo,'(4X,''Error relativo en x:''',D14.6,)')
+   ermax
if (ermax.le.ztolze) go to 8800
if (ztrel.gt.0.1) go to 8800
ztrel = 0.5
go to 10
C
8800 continue
if (inibb.eq.0) write (lo,8900)
return
C
8000 format('*** Matriz singular')
8005 format('*** Espacio para vectores ETA insuficiente; ',
+   'el programa se para:/' incrementa E(.) y NEMAX')
8500 format('*** Estadísticas de INVERT'/4x,i5,
+   'elementos no cero en la base'/4x,i5,
+   'columnas estructurales en la base'/4x,i5,
+   'vectores columna antes del "bump"/4x,i5,
+   'vectores columna después del "bump"/4x,i5,
+   'elementos no cero; ',i5,' vectores ETA'/4x,'U: ',i5,
+   'elementos no cero; ',i5,' vectores ETA'/4x,'Total: ',i5,
+   'elementos no en la diagonal; ',i5,' vectores ETA')
8900 format('///Ite. Inf. Nopt. Sum. Inf/F.Obj',
+   ' Entra de-a Cost.Red Sale de-a Paso ',',', Pivo-
te El.Eta')
C
C Fin de -INVERT-
end

subroutine normal
C
C Se coordinan todas las subrutinas necesarias para llevar a
C cabo la resolución de un programa lineal por el método
C simplex revisado.
C
C***** Descripción de vectores afectados *****
C
C B := Termino de la derecha del problema.
C
C KINBAS := Estado en el que se encuentran los componentes
C del vector X de variables de decisión:
C =0, variable en su límite inferior;
C =-1, variable en su límite superior;
C =-2, variable libre: especificada FR;
C =>0, variable básica, el número indica
C el vector fila sobre el que pivota.
C
C QSTAT := Variable CHARACTER que indica el estado de la
C solución: 'I', no factible; 'F', factible.
C
C XLB, XUB := Límites inferior y superior de las variables.
C
C JH := Vector que indica la columna con la que pivota
C cada vector fila.
C
C LA, IA, A := Vectores donde se guarda toda la información
C relativa a la matriz de los coeficientes
C de las condiciones.
C
C X := vector  $B^{-1}b - B^{-1}N_{\text{X}} = B^{-1}(b - N_{\text{X}})$ 
C
C LE, IE, E := Vectores donde se guarda toda la información
C relativa a la matriz de las
C transformaciones elementales eta.
C
C YTEMP1, YTEMP2, YTEMP3 := vectores de trabajo.
C
C*****
C implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C integer hrtype(4000)
C
C character*1 ast,movel*4,move2*4
C
C common /iolog/ iu,lo,logu
C
C common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+   ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos

```

```

C
common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,lists,a,noint,
+   idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+   xioibnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+   kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+   ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
+   qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+   b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+   yen(1000)
C
C if (itsinv.ge.invfrq) then
C   call invert
C   itsinv = 0
C endif
C
C*****
C Comienzo iteraciones del método simplex revisado.
C Paso 1. Asignación de precios.
C
C*****
1500 continue
call formc (hrtype)
call btran
C
C*****
C Paso 2. Determinación columna de pivotación JCOLP.
C*****
call price (jcolp)
C
C-----
C Comprobar si se ha llegado al óptimo.
C-----
itcnt = itcnt+1
itsinv = itsinv+1
if (jcolp.eq.0) return
C
call unpack (jcolp)
call ftran (i)
C
C*****
C Paso 3. Determinación fila de pivotación IROWP Y pivotación.
C*****
call chuzr (hrtype,jcolp,irowp,npivot,iptype,theta,ivout)
C
C*****
C Paso 4. Escritura de resultados intermedios.
C*****
if (inibb.eq.0) then
  ast = ' '
  if (npivot.eq.1) then
    if (rcost.gt.0.0) then
      move1 = 'L->B'
    else
      move1 = 'U->B'
    endif
    if (iptype.eq.(-1)) then
      move2 = 'B->U'
    else
      move2 = 'B->L'
    endif
  else
    ast = '*'
    if (rcost.gt.0.0) then
      move1 = 'L->U'
      move2 = 'L->U'
    else
      move1 = 'U->L'
      move2 = 'U->L'
    endif
  endif
endif
if (jcolp.le.nrow) then
  ivines = jcolp-ncol-nrow-1
else
  ivines = jcolp-nrow
endif
if (ivout.le.nrow) then
  ivoute = ivout+ncol-nrow-1
else
  ivoute = ivout-nrow
endif
sumobj = x(1)*min
if (ninf.gt.0) sumobj = suminf
write (lo,8000) itcnt,ninf,nopt,sumobj,ivines,ast,movel,rcost*
+   min,ivoute,ast,movel,theta,rpivot,nelem
endif
C
if (npivot.eq.1) then
  if (nelem.ge.nemax) then
    call invert
    itsinv = 0

```

$$\begin{array}{c} \diagup \quad \diagdown \\ \diagdown \quad \diagup \\ \diagup \quad \diagdown \end{array} \quad \begin{array}{l} X \geq N + 1 \\ P \quad P \end{array}$$

```

C      X <= N          /P D          P \ \ IDIR=1
C      P           P   //         U   \ \
C      /             / IDIR=-1       \
C      /--\         /--\
C      |  |         |  |
C      --/         --/
C*****
C      implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C      common /defpa/ nrmx,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+      , ztolpv,ztcost,tolata,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C      common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,listsa,noint
+      idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+      xioibnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+      kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+      ncol,nelem,neta,nleta,nueta,ninf,min,nopt,logite,qname(4000),
+      qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+      b(1000),xc(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+      yen(1000)
C
C      qfix = ' ',
C      do i = 2,nrow
C        if (dfpart(i).le.ztolze) then
C          pu(i) = cero
C          pd(i) = cero
C          pg(i) = cero
C        else
C          pu(i) = plinfy
C          pd(i) = plinfy
C          pg(i) = plinfy
C        endif
C      end do
C*****
C      Se analizan todas las variables (COL=J=2,NCOL) no básicas:
C      KINBAS(J)= -1 o KINBAS(J)=0.
C*****
C      do j = 2,ncol
C        if (kinbas(j).le.0.and.dabs(xub(j)-xlb(j)).gt.ztolze) then
C-----
C      Se calcula
C
C              -1
C            B  N
C              J
C-----
C      call unpack (j)
C      call ftran (1)
C-----
C      Se comprueban incrementos positivos para variables no básicas
C      en su límite inferior y negativos para no básicas
C      en su límite superior.
C-----
C      if (kinbas(j).eq.(-1)) then
C        do i = 1,nrow
C          y(i) = -y(i)
C        end do
C      endif
C-----
C      Para las variables no básicas que no han de tomar
C      valores enteros hacemos el valor TAOK=0;
C      para aquellas que sí, TAOj=Y(1).
C          Y(1) = A      (ver referencia).
C              00
C-----
C      if (j.lt.nvare) then
C        ta0j = 0.0
C      else
C        ta0j = y(1)
C      endif
C-----
C      Comprobar bifurcación forzada con respecto a la variable xj;
C      es decir, si al mover una variable no básica que ha de tomar
C      un valor entero una unidad, se obtiene peor función
C      objetivo que la calculada hasta este momento; crear una
C      nueva rama en la que esa variable este fija.
C-----
C      if (j.ge.nvare) then
C        xival = x(1)-y(1)
C        if (xival.le.xincva) then
C          idir = 2*kinbas(i)+1

```

```

        icol = j
        call branch (icol,idir)
        if (idir.eq.(-1)) xlb(icol) = xub(j)
        if (idir.eq.1) xub(icol) = xlb(j)
        if (icol.le.nrow) then
            icol = icol+ncol-nrow-1
        else
            icol = icol-nrow
        endif
        if (qfix.eq.' ' ) then
            write (qfix,'(A,I4,A)') 'FIJ',icol,' '
        else
            write (qfix,'(A,I4,A)') 'FIJ',icol,'*'
        endif
        cycle
    endif
endif

do i = 2,nrow
    if (jh(i).ge.nvare.and.dfp(i).gt.ztolze) then

C-----
C   Calcular la penalización P que determinara el acotar la
C   U
C   variable básica x(i), que toma un valor no entero,
C   superiormente y consecuentemente una no básica x saldrá
C   de su límite. j
C-----
        if (y(i).lt.(-ztolpv)) then
            de = dmax1(y(1)*dfpart(i)-1/y(i),ta0j)
            pu(i) = dmin1(de,pu(i))
        endif

C-----
C   Calcular la penalización P que determinara el acotar la
C   D
C   variable básica x(i), que toma un valor no entero,
C   inferiormente y consecuentemente una no básica x saldrá
C   de su límite. j
C-----
        if (y(i).gt.ztolpv) then
            de = dmax1(y(1)*dfpart(i)/y(i),ta0j)
            pd(i) = dmin1(de,pd(i))
        endif

C-----
C   Calcular la penalización P asociada a la introducción de un
C   corte de Gomory. G
C-----
C   Primero, analizar las variables no básicas x que no han
C   de tomar valores enteros. j
C-----
        if (j.lt.nvare) then
            if (y(i).gt.0.0) then
                de = dfpart(i)*y(1)/y(i)
                pg(i) = dmin1(de,pg(i))
            else if (y(i).lt.0.0) then
                de = (dfpart(i)-1)*y(1)/y(i)
                pg(i) = dmin1(de,pg(i))
            endif
        else
            else
        endif

C-----
C   Segundo, analizar las variables no básicas x que sí han de
C   tomar valores enteros. j
C-----
        dp = dabs(y(i))-dble(idint(dabs(y(i))))
        if (dp.gt.ztolze.and.dp.lt.1-ztolze) then
            if (y(i).lt.0.0) dp = 1-dp
            if (dp.gt.dfp(i)) then
                de = (1-dfp(i))*y(1)/(1-dp)
            else
                de = dfpart(i)*y(1)/dp
            endif
            pg(i) = dmin1(de,pg(i))
        endif
    endif
end do
endif
end do

C-----
C*****
C   Calcular la penalización P máxima y comprobar si se pueden
C   G G
C   abandonar las ramas que partirían del nudo en que nos
C   encontramos y que resultarían de imponer nuevos límites a
C   variables que debiendo ser enteras no cumplen este requisito
C   de momento.
C*****
pen = cero
do i = 2,nrow
    if (jh(i).ge.nvare) pen = dmax1(pen,pg(i))

```

```

        end do
        xival = x(1)-pen
        if (xival.le.xincva) then
            idir = 0
            return
        endif
    C-----
C*****
C   Comprobar si las penalizaciones P o P por imponer nuevos
C   U D
C   límites a la variable x(i) hacen que las soluciones
C   continuas obtenibles son peores que la "titular"
C   entera hasta este momento.
C*****
ntemp2 = 0
do i = 2,nrow
    if (jh(i).ge.nvare.and.x(1)-dmax1(pd(i),pu(i)).le.xincva) then
        if (pu(i).le.pd(i)) then

C-----
C   Asi es: se añade a la lista de nudos a analizar
C   posteriormente el que resulta de bifurcar desde el nudo en
C   que nos encontramos en la dirección resultante de acotar
C   la variable x(i) inferiormente. Seguiremos inmediatamente
C   analizando la rama resultante de acotar x(i) superiormente.
C-----
            xival = x(1)-pd(i)
            idir = -1
            irowp = i
            icol = jh(irowp)
            ntemp2 = 1
            call branch (icol,idir)
            xlb(icol) = dble(ipart(i)+1)
        else

C-----
C   Asi es: se añade a la lista de nudos a analizar
C   posteriormente el que resulta de bifurcar desde el nudo en
C   que nos encontramos en la dirección resultante de acotar
C   la variable x(i) superiormente. Seguiremos inmediatamente
C   analizando la rama resultante de acotar x(i) inferiormente.
C-----
            xival = x(1)-pu(i)
            idir = 1
            irowp = i
            icol = jh(irowp)
            ntemp2 = 1
            call branch (icol,idir)
            xub(icol) = dble(ipart(i))
        endif
    endif
end do

C-----
C*****
C   Las penalizaciones P o P por imponer nuevos límites a la
C   U D
C   variable x(i) no hacen presagiar que las soluciones
C   continuas obtenibles son peores que la "titular" entera
C   hasta este momento; se elige con respecto a qué variable
C   bifurcar y en qué dirección.
C*****
if (ntemp2.eq.0) then
    pen = 0.0
    irowp = 0
end do

C-----
C   Se determina como variable de bifurcación aquella que tenga
C   una mayor penalización P o P esperando así agotar
C   D U
C   pronto el árbol que parta de aquí.
C-----
do i = 2,nrow
    if (jh(i).ge.nvare) then
        if (pu(i).le.pd(i)) then
            if (pd(i).gt.pen) then
                pen = pd(i)
                irowp = i
                idir = -1
                revbnd = dble(ipart(i)+1)
            endif
        else if (pu(i).gt.pen) then
            pen = pu(i)
            irowp = i
            idir = 1
            revbnd = dble(ipart(i))
        endif
    endif
end do

C-----
C   Problemas... Todas las penalizaciones P y P de las variables
C   son cero: degeneración. D U
C   Elegir cualquier variable básica no entera como variable

```

```

C de bifurcación; anadir a la lista de nudos a analizar
C la rama resultante de acotar superiormente la variable x(i)
C y analizar a continuación la derivada de acotar
C inferiormente la variable.
C-----
      if (irowp.eq.0) then
        do irowp = 2,nrow
          if (jh(irowp).ge.nvare.and.dfpert(irowp).gt.ztolze)
+         exit
          end do
          pen = pu(irowp)
          idir = 1
          revbnd = dble(ipart(irowp))
          endif
          icol = jh(irowp)
          xival = x(1)-pen
          call branch (icol,idir)
          if (idir.eq.(-1)) xlb(icol) = revbnd
          if (idir.eq.1) xub(icol) = revbnd
        endif
        if (idir.eq.(-1)) iptype = 0
        if (idir.eq.1) iptype = -1
C
C return
C
C Fin de -PENLTS-
C end

subroutine price(jcolp)
C
C Se asignan precios a las columnas no básicas y se determina
C la columna no básica JCOLP a entrar en la base.
C
C      t -1      t t -1      t
C      Z = C B b + (c -c B N)x ; en este caso c = 0.
C      B      N B      N
C
C El coste reducido de la columna no básica a es
C
C      t -1
C      -c B a .
C      B      j
C
C***** Descripción de vectores afectados *****
C
C KINBAS := Estado en el que se encuentran los componentes
C del vector X de variables de decisión:
C =0, variable en su límite inferior;
C =-1, variable en su límite superior;
C =-2, variable libre: especificada FR;
C =>0, variable básica, el número indica
C el vector fila sobre el que pivota.
C
C XLB, XUB := Límites inferior y superior de las variables.
C
C LA, IA, A := Vectores donde se guarda toda la información
C relativa a la matriz de los coeficientes
C de las condiciones.
C
C Y := El vector columna a .
C j
C*****
C implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C common /defpa/ nrmax,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+ ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,listsa,noint,
+ idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+ xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+ kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+ ncol,nelem,neta,nleta,neta,ninf,min,nopt,logite,qname(4000),
+ qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+ b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+ yen(1000)
C
C cmin = plinfy
C cmax = -plinfy
C nopt = 0
C
C*****
C Cálculo de los costes reducidos de todas las variables no
C básicas.
C
C      t -1
C      r = c - c B a .
C      j j B j
C
C en este caso c =0.

```

```

C
C j
C
C La función onjetivo se podra mejorar (este es un problema de
C maximización, si
C
C      r > 0 para x=1 ---> KINBAS(J)=0
C      j j
C
C 0
C      r < 0 FOR x=u ---> KINBAS(J)=-1
C      j j
C
C*****
C jcolp = 0
C do j = 1,ncol
C if (dabs(xub(j)-xlb(j)).gt.ztolze.and.kinbas(j).le.0) then
C dsum = cero
C do i = la(j),la(j+1)-1
C dsum = dsum-a(i)*y(ia(i))
C end do
C if (kinbas(j).eq.(-2)) then
C jcolp = j
C rcost = dsum
C if (dabs(dsum).gt.ztcost) nopt = nopt+1
C else if (kinbas(j).eq.0) then
C if (dsum.gt.ztcost) nopt = nopt+1
C if (dsum.gt.cmax) then
C cmax = dsum
C jcol1 = j
C endif
C else
C if (dsum.lt.(-ztcost)) nopt = nopt+1
C if (dsum.lt.cmin) then
C cmin = dsum
C jcol2 = j
C endif
C endif
C endif
C end do
C
C if (jcolp.ne.0) return
C if (cmax.lt.ztcost) then
C if (cmin.gt.(-ztcost)) then
C jcolp = 0
C rcost = cero
C else
C jcolp = jcol2
C rcost = cmin
C endif
C else if (cmin.le.(-ztcost)) then
C if (cmax.le.dabs(cmin)) then
C jcolp = jcol2
C rcost = cmin
C else
C jcolp = jcol1
C rcost = cmax
C endif
C else
C jcolp = jcol1
C rcost = cmax
C endif
C
C return
C
C Fin de -PRICE-
C end

subroutine testx
C
C Se comprueba la solución óptima obtenida del programa lineal
C rechazándose el nudo no generándose, por tanto, a partir
C de el ninguna rama si:
C (1) la solución PL no es factible (QSTAT = 'N');
C (2) el valor de la función objetivo de PL es peor que el
C de la mejor solución entera obtenida hasta este
C momento;
C (3) la solución PL es entera.
C*****
C implicit integer*4(i-n),real*8(a-h,o-p,r-z),character*8(q)
C
C common /iolog/ iu,lo,logu
C
C common /defpa/ nrmax,ncolma,nemax,ntmax,maxnod,namax,itsinv,ztolze
+ ,ztolpv,ztcost,toleta,epmcd,plinfy,cero,uno,dos,iuno,idos
C
C common /nodbb/ ipart(1000),ivid(1500,2),nvare,listl,listsa,noint,
+ idepth,qfix,xincva,xival,revbnd,dfpart(1000),xinc(4000),
+ xiobnd(1500),vbnd(1500),pu(1000),nsolen,pd(1000),pg(1000)
C
C common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+ kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+ ncol,nelem,neta,nleta,neta,ninf,min,nopt,logite,qname(4000),
+ qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),

```

```

+   b(1000), xci(4000), xlb(4000), xcs(4000), xub(4000), e(35000),
+   yen(1000)
C
C   xival = x(1)
C
C*****
C   Calcular parte entera y fraccionaria de cada variable básica.
C*****
do i = 2, nrow
  if (x(i).ge.(-ztolze)) then
    ipart(i) = idint(x(i)+ztolze)
  else
    ipart(i) = idint(x(i)+ztolze)-1
  endif
  dfpart(i) = dabs(x(i)-dble(ipart(i)))
end do
C
C*****
C   Comprobar si se ha obtenido una solución entera.
C*****
noint = 0
do i = 2, nrow
  if (jh(i).ge.nvare.and.dfpart(i).gt.ztolze) noint = noint+1
end do
if (noint.ne.0) return
C
C*****
C   Si; se ha obtenido una solución entera. Si es mejor que la
C   actual guardarla y sacar informacion.
C*****
qstat = 'ENTERA'
if (inibb.eq.0) then
  write (lo, '(** ** La solución inicial del programa** **',
+    ' ** lineal es la solución ENTERA OPTIMA ** **')
  return
endif
if (xival.le.xincva) return
xincva = xival
if (nvare.le.ncol) then
  do i = nvare, ncol
    kinben(i) = kinbas(i)
    if (kinbas(i).eq.(-1)) then
      xinc(i) = dnint(xub(i))
      if (dabs(xub(i)-xlb(i)).le.ztolze) kinben(i) = -3
    endif
    if (kinbas(i).eq.0) then
      xinc(i) = dnint(xlb(i))
      if (dabs(xub(i)-xlb(i)).le.ztolze) kinben(i) = -3
    endif
    if (kinbas(i).gt.0) xinc(i) = dnint(x(kinbas(i)))
  end do
endif
C
do i = 1, nvare-1
  kinben(i) = kinbas(i)
  if (kinbas(i).eq.(-1)) then
    xinc(i) = xub(i)
    if (dabs(xub(i)-xlb(i)).le.ztolze) kinben(i) = -3
  endif
  if (kinbas(i).eq.0) then
    xinc(i) = xlb(i)
    if (dabs(xub(i)-xlb(i)).le.ztolze) kinben(i) = -3
  endif
  if (kinbas(i).gt.0) xinc(i) = x(kinbas(i))
end do
do i = 2, nrow
  y(i) = cero
end do
y(1) = uno
call btran
do i = 1, nrow
  yen(i) = y(i)
end do
nsolen = nsolen+1
call wrilpr (2)
return
C
C   Fin de -TESTX-
end

subroutine unpack(iv)
C
C   Se expande un vector columna de la matriz A guardado como
C   disperso insertando ceros apropiadamente.
C
C***** Descripción de vectores afectados *****
C
C   IV      := Índice del vector columna a expandir.
C
C   LA, IA, A := Vectores donde se guarda toda la información
C               relativa a la matriz de los coeficientes
C               de las condiciones.

```

```

C
C   Y      := El vector columna a .
C
C               iv
C*****
C
C   implicit integer*4(i-n), real*8(a-h, o-p, r-z), character*8(q)
C
C   common /defpa/ nrmx, ncolma, nemax, ntmax, maxnod, namax, itsinv, ztolze
+   , ztolpv, ztcost, toleta, epmc, plinfy, cero, uno, dos, iuno, idos
C
C   common /parlp/ jh(1000), la(4001), ia(15000), le(8001), ie(35000),
+   kinbas(4000), kinben(4000), inibb, itcnt, invfrq, itrfrq, nrow, nrwm1,
+   ncol, nelem, neta, nleta, nueta, ninf, min, nopt, logite, qname(4000),
+   qnamo, qstat, a(15000), suminf, rcost, rpivot, x(1000), y(1000),
+   b(1000), xci(4000), xlb(4000), xcs(4000), xub(4000), e(35000),
+   yen(1000)
C
C
C   do i = 1, nrow
+   y(i) = cero
  end do
C
  ll = la(iv)
  kk = la(iv+1)-1
  do i = ll, kk
    ir = ia(i)
    y(ir) = dble(a(i))
  end do
C
  return
C
C   Fin de -UNPACK-
end

subroutine wreta(irowp)
C
C   Se guarda la nueva matriz elemental eta resultado de la
C   iteración y pivotación correspondiente.
C*****
C
C   implicit integer*4(i-n), real*8(a-h, o-p, r-z), character*8(q)
C
C   common /defpa/ nrmx, ncolma, nemax, ntmax, maxnod, namax, itsinv, ztolze
+   , ztolpv, ztcost, toleta, epmc, plinfy, cero, uno, dos, iuno, idos
C
C   common /parlp/ jh(1000), la(4001), ia(15000), le(8001), ie(35000),
+   kinbas(4000), kinben(4000), inibb, itcnt, invfrq, itrfrq, nrow, nrwm1,
+   ncol, nelem, neta, nleta, nueta, ninf, min, nopt, logite, qname(4000),
+   qnamo, qstat, a(15000), suminf, rcost, rpivot, x(1000), y(1000),
+   b(1000), xci(4000), xlb(4000), xcs(4000), xub(4000), e(35000),
+   yen(1000)
C
C
C   nelem = nelem+1
  ie(nelem) = irowp
  e(nelem) = y(irowp)
C
  do i = 1, nrow
    if (i.ne.irowp.and.dabs(y(i)).gt.ztolpv) then
      nelem = nelem+1
      ie(nelem) = i
      e(nelem) = y(i)
    endif
  end do
C
  neta = neta+1
  le(neta+1) = nelem+1
  return
C
C   Fin de -WRETA-
end

subroutine wrilpr(ipa)
C
C   Se escriben los resultados del problema.
C
C*****
C
C   implicit integer*4(i-n), real*8(a-h, o-p, r-z), character*8(q)
C
C   character*3 ql1, qlb, qul, qulb, qeq, qeqb, qbas, qbs
C   character*14 char1, char2
C
C   common /iolog/ iu, lo, logu
C
C   common /defpa/ nrmx, ncolma, nemax, ntmax, maxnod, namax, itsinv, ztolze
+   , ztolpv, ztcost, toleta, epmc, plinfy, cero, uno, dos, iuno, idos
C
C   common /nodbb/ ipart(1000), ivid(1500, 2), nvare, list1, listsa, noint,
+   idepth, qfix, xincva, xival, revbnd, dfpart(1000), xinc(4000),
+   xiobnd(1500), vbnd(1500), pu(1000), nsolen, pd(1000), pg(1000)

```



```

C      common /parlp/ jh(1000),la(4001),ia(15000),le(8001),ie(35000),
+      kinbas(4000),kinben(4000),inibb,itcnt,invfrq,itrfrq,nrow,nrwm1,
+      ncol,nelem,neta,nleta,nufeta,ninf,min,nopt,logite,qname(4000),
+      qnampo,qstat,a(15000),suminf,rcost,rpivot,x(1000),y(1000),
+      b(1000),xci(4000),xlb(4000),xcs(4000),xub(4000),e(35000),
+      yen(1000)
C
C      data qbas/' BS'/
C      data qll/' LI'/
C      data qul/' LS'/
C      data qeq/' EQ'/
C      data qfx/' FX'/
C      data qlb/' LIB'/
C      data qulb/' LSB'/
C      data qeqb/' EQB'/
C      data qfr/' FR'/
C      data qbsd/' BSD'/
C
C      go to (1,2,3) ipa
C
C*****
C Imprimir valor de la función objetivo y alguna estadística.
C*****
1 continue
if (qstat.eq.'INFEAS') then
write (lo,910)
stop
endif
write (lo,10)
lode = lo
go to 8
2 continue
call cputim (time,cput)
lode = logu
write (lo,13) x(1)*min,time
if (logite.eq.0) return
write (logu,12) time
go to 8
3 continue
if (inibb.eq.0) return
lode = lo
if (nsolen.eq.0) then
write (lo,16)
return
endif
write (lo,11)
do i = 1,ncol
if (kinben(i).eq.0) xlb(i) = xinc(i)
if (kinben(i).eq.(-1)) xub(i) = xinc(i)
if (kinben(i).eq.(-3)) then
xlb(i) = xinc(i)
xub(i) = xinc(i)
endif
if (kinben(i).gt.0) x(kinben(i)) = xinc(i)
kinbas(i) = kinben(i)
end do
do i = 1,nrow
y(i) = yen(i)
end do
C
8 continue
write (lode,14) qnampo,itcnt,x(1)*min
C
C*****
C Imprimir resultados de las condiciones: ROWS
C*****
char1 = ' Ninguno '
write (lode,20) 1,qname(1),qbas,x(1)*min,(-x(1)*min),char1,char1,
+ 1.0
do i = 2,nrow
ytempi = 0.0
yi = y(i)
bi = b(i)
xubi = xub(i)
if (a(i).gt.0.0) then
d1 = -xubi
d2 = bi
if (kinbas(i).gt.0) then
ytempi = x(kinbas(i))
bact = bi-ytempi
qbs = qbas
if (ytempi.lt.ztolze) qbs = qbsd
else if (kinbas(i).eq.0) then
bact = bi
qbs = qul
else
bact = bi-xubi
qbs = qll
endif
else
d1 = bi
d2 = xubi
if (kinbas(i).gt.0) then
ytempi = x(kinbas(i))
bact = bi+ytempi
qbs = qbas
if (ytempi.lt.ztolze) qbs = qbsd
else if (kinbas(i).eq.0) then
bact = bi
qbs = qll
else
bact = bi+xubi
qbs = qul
endif
endif
if (dabs(xubi-xlb(i)).lt.ztolze) then
qbs = qeq
if (kinbas(i).gt.0) qbs = qbsd
bact = bi
d1 = bi
d2 = bi
endif
write (char1,'(G14.8)') d1
if (d1.le.(-plinfy)+ztolze) char1 = ' Ninguno '
write (char2,'(G14.8)') d2
if (d2.ge.plinfy-ztolze) char2 = ' Ninguno '
C
write (lode,20) i,qname(i),qbs,bact,ytempi,char1,char2,yi*min
end do
C
C*****
C Imprimir resultados de las variables: COLUMNS.
C*****
write (lode,30)
C
do i = nrow+1,ncol
redco = cero
if (kinbas(i).eq.(-2)) then
xtemp = cero
qbs = qfr
else if (kinbas(i).eq.(-1)) then
xtemp = xub(i)
if (dabs(xub(i)-xcs(i)).le.ztolze) then
qbs = qul
else
qbs = qulb
endif
else if (kinbas(i).eq.0) then
xtemp = xlb(i)
if (dabs(xlb(i)-xci(i)).le.ztolze) then
qbs = qll
else
qbs = qllb
endif
else
xtemp = x(kinbas(i))
qbs = qbas
go to 45
endif
do j = la(i),la(i+1)-1
redco = redco-y(ia(j))*a(j)
end do
45 continue
if (dabs(xub(i)-xlb(i)).le.ztolze) then
xtemp = xlb(i)
if (dabs(xub(i)-xcs(i)).le.ztolze.and.dabs(xlb(i)-xci(i))
+ .le.ztolze) then
qbs = qfx
else
qbs = qeqb
endif
endif
C
write (char1,'(G14.8)') xlb(i)
if (xci(i).le.(-plinfy)+ztolze) char1 = ' Ninguno '
C
write (char2,'(G14.8)') xub(i)
if (xcs(i).ge.plinfy-ztolze) char2 = ' Ninguno '
C
act2 = 0.0
do j = la(i),la(i+1)-1
if (ia(j).eq.1) act2 = -a(j)*min
end do
write (lode,40) i-nrow,qname(i),qbs,xtemp,act2,char1,char2,
+ redco*min
end do
C
if (logite.eq.1.and.ipa.ne.3) write (logu,150)
C
return
C
10 format(/21x,'--- SOLUCION INICIAL PROGRAMA LINEAL ---'/25x,32('
+ /)

```

```

11 format(/26x,'--- SOLUCION ENTERA OPTIMA ---'/30x,22('-')/)
16 format(/31x,'--- NO SE HA ENCONTRADO SOLUCION ENTERA ---
'/34x,35(
+      '-')/)
12 format(/34x,'--- SOLUCION ENTERA ---'/38x,15('-')//
+      'Tiempo desde última:',f9.4,' seg.')
13 format(/'* Nueva solución entera; z(PE)='f17.5,
+      '; Tiempo desde última:',f9.4,' seg.')
14 format(4x,'Nombre del problema: ',a/4x,'No. de iteracio-
nes: ',i9/
LAS'//
+      4x,'Valor de la función objetivo: ',g30.15//')*** FI-
+      ' No. ..Fila.. en ....Valor.... ',
+      '...Holgura... .Lí.Inferior. ','.Lí.Superior. Val.Dual.'/)
20 format(i4,ix,a,a,2g14.8,2a14,g10.4)
30 format(/'*** COLUMNAS'//') No. .Columna en ',
+      '...Valor.... Coste en F.O. .Lí.',
+      'Inferior. .Lí.Superior. Cos.Red.'/)
40 format(i4,ix,a,a,2g14.8,2a14,g9.3)
150 format(/' Variable',28x,'Nudos',
+      ' Variables Valor '/
+      'Separación Nivel Dirección en Lista ',
+      'Ent. No Ent. Iteración Func. Obj.')
910 format('El Problema No es Factible')

```

```

C
C      Fin de -WRILPR-
end

subroutine cputim(cpur,cput)

C
integer*2 ih,im,is,ihs

C
doubleprecision cpur,cput

C
save aux
data aux/0.0/

C
call gettim (ih,im,is,ihs)
aux1 = ih*3600.0+im*60.0+is+ihs/100.0
cpur = dble(aux1-aux)
cput = dble(aux1)
aux = aux1

C
return

C
C      Fin de -CPUTIM-
end

```


Apéndice F

EL PROGRAMA CCNET

EL PROGRAMA CCNET es una especialización escrita en C del método *simplex* revisado, diseñada para resolver problemas de flujos en redes de mínimo coste de grandes dimensiones de la forma:

$$\begin{aligned} &\text{minimizar } \mathbf{c}^T \mathbf{x} \\ &\text{sujeta a } \quad A\mathbf{x} = \mathbf{b} \\ &\quad \quad \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}, \end{aligned} \tag{F.1}$$

donde $\mathbf{c}, \mathbf{x}, \mathbf{l}, \mathbf{u} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$ y $A \in \mathbb{R}^{m \times n}$ es una matriz de incidencia nudo-arco en la que cada vector columna, $\mathbf{a}_j$, está dado por:

$$a_{ij} = \begin{cases} -1, & \text{si el arco } j \text{ llega al nudo } i. \\ 1, & \text{si el arco } j \text{ sale del nudo } i. \\ 0, & \text{si los nudos } i \text{ y } j \text{ no están unidos por ningún arco.} \end{cases}$$

Para evitar la singularidad existente en la matriz A , tal como se indica en el capítulo 9, se añade un vector unitario $\mathbf{e}_1 = (1, 0, \dots, 0)$ para conseguir rango completo. Este vector columna, como es sabido, se puede interpretar como un arco artificial desde el nudo raíz (nudo 1) a uno ficticio.

Los elementos del vector $\mathbf{b}$ representan los flujos exógenos a los nudos de acuerdo con el criterio siguiente:

$$\begin{aligned} b_i &< 0, & \text{si } i \text{ es un nudo de demanda o sumidero;} \\ b_i &> 0, & \text{si } i \text{ es un nudo de oferta o fuente;} \\ b_i &= 0, & \text{si } i \text{ es un nudo de transbordo.} \end{aligned}$$

El algoritmo utilizado en CCNET, su mecánica, estructuras de datos y la forma de adaptarlas, siguen fielmente lo expuesto en el capítulo 9.

La solución básica factible de partida se obtiene creando una base de arcos (variables) artificiales, es decir, un árbol con raíz en el nudo 1 al que se unen todos los restantes de la

red mediante unos arcos artificiales orientados de tal forma que se acomoden todos los flujos exógenos dados. Una vez creada la base de partida, el programa sigue el método de Grigoriadis [1986], denominado de Penalización Gradual (GPM: *Gradual Penalty Method*). En el primer paso de este método se asigna una penalización a todos los arcos artificiales del árbol inicial. Posteriormente, se itera normalmente hasta obtener una solución óptima con esa penalización. Si hay arcos artificiales con flujo positivo en ese óptimo, se aumenta la penalización, se vuelven a calcular las variables duales y se continúa iterando. El método termina cuando:

- en la solución óptima no hay arcos artificiales con flujos positivos;
- la penalización es suficientemente grande:
 - el problema no tiene solución factible pues hay arcos artificiales en el árbol básico con flujo positivo, o
 - el problema es no acotado pues existe un ciclo negativo de capacidad infinita.

Si no se puede aumentar más la penalización, y hay al menos un arco artificial positivo en el árbol básico, se produce una salida del programa, indicándose tal circunstancia.

F.1 Ejecución del programa

Al lanzar el programa para resolver un problema, se puede especificar si se desea o no que en el fichero de resultados aparezca la información relativa a todo el proceso iterativo que ha sido necesario efectuar. Si se desea, además de teclear el nombre del ejecutable de CCNET, hay que incluir la especificación `-i`. Es decir,

```
CCNET -i
```

ejecutará el programa CCNET requiriéndosele que incluya entre los resultados información relativa al proceso iterativo llevado a cabo.

F.2 Datos del problema

Los datos del problema a resolver deberán estar en un fichero cuyo nombre el propio programa demandará al comienzo de la ejecución mediante el requerimiento:

Fichero entrada:

Los **resultados** y el proceso de optimización aparecerán en un fichero que el programa automáticamente designa como **nombre.cnt**, donde *nombre* es el mismo que el introducido como respuesta al requerimiento mencionado.

En el fichero de datos se introducen los valores de las variables que definen el problema a resolver. El formato con el que se pueden escribir es libre; no así el orden en que deben

escribirse. Éste es:

- LÍNEA 1: m, n (enteros).
 LÍNEA 2: $mxit$ (entero), frq (doble), $p0$ (doble), $pbar$ (doble), $ncand$ (entero), $cpmax$ (doble).
 LÍNEA 3 A $n + 3$: para cada arco de la red, nudo origen, $from$ (entero); nudo destino, to (entero); coeficiente, $coste$ (doble), en la función objetivo; cota inferior, l (doble) y cota superior, u , (doble).
 LÍNEA $n + 4$: $nonz$ (entero), número de flujos exógenos distintos de cero en la red.
 LÍNEA $n + 5$: vector, bnz (entero), con las posiciones en el vector $\mathbf{b}$ de los $nonz$ flujos exógenos distintos de cero.
 LÍNEA $n + 6$: vector, rhs (doble), con los valores de los elementos del vector $\mathbf{b}$ distintos de cero (flujos exógenos).

Una vez introducidos los datos numéricos de cada línea, se pueden añadir todos aquellos comentarios que puedan contribuir a clarificar el significado de los valores. Este es el caso de los datos del ejemplo que se incluye en el apartado dedicado a los resultados obtenibles.

El significado de algunas de las variables apuntadas es el siguiente:

- m número de nudos de la red.
 n número de arcos.
 $mxit$ número máximo de iteraciones a llevar a cabo.
 frq frecuencia con la que se desea se realice el *pricing* de los arcos; en % de n .
 Los arcos candidatos para entrar en la base se seleccionan tomando cada k -ésimo arco de la lista de arcos, donde $k = frq \times n \times 0,01 + 0,5$. En cada iteración, el arco que se toma para esta búsqueda se elige secuencialmente desde 1 a $k - 1$. La mejor elección de frq depende de la clase de problema que se quiere resolver. Generalmente, frq debe ser elegido en el rango 0,8%–8,0%. Un valor aconsejable para este parámetro es 1,5.
 $ncand$ número de arcos de entre los que se selecciona el que va a entrar en la base. En este caso, frq viene dado en % de $ncand$.
 $p0$ multiplicador constante de la penalización para los arcos artificiales. Se usa para controlar la magnitud del valor de la penalización inicial. Un valor aconsejable de este parámetro es 1.
 $pbar$ base de la exponencial del multiplicador de $p0$ (debe ser mayor que 1). Se usa para controlar el índice de crecimiento de la penalización en los pasos sucesivos del procedimiento. Un valor aconsejable para este parámetro es 1,5.
 $cpmax$ valor máximo de la penalización. Normalmente este valor debe ser grande: mayor que $p0$.

F.3 Resultados

Como ejemplo de las posibilidades de CCNET resolveremos el ejemplo 9.4 de la página 521, es decir:

$$\begin{aligned}
 \min. \quad & x_1 + x_2 + 3x_3 + 10x_4 \\
 \text{s. a} \quad & x_1 + x_3 + x_4 = 5 \\
 & -x_1 + x_2 = 0 \\
 & -x_2 - x_3 - x_4 = -5 \\
 & 0 \leq x_1 \leq 4 \\
 & 0 \leq x_2 \leq 2 \\
 & 0 \leq x_3 \leq 4 \\
 & 0 \leq x_4 \leq 10.
 \end{aligned}$$

El fichero de datos de este problema es el que se detalla a continuación.

3, 4	Tamaño del problema: nudos(M), arcos(N)
500, 8.0, 1.0, 1.5, 100	mxit, frq, p0, pbar, cpmx
1, 2, 1.0, 0.0, 4.0	Arco 1: nudo FROM, nudo TO, coste, cotas
2, 3, 1.0, 0.0, 2.0	Arco 2: " " " "
1, 3, 3.0, 0.0, 4.0	Arco 3: " " " "
1, 3, 10., 0.0, 10.	Arco 4: " " " "
2	Número de flujos exógenos no iguales a cero
1, 3	Nudos con flujos exógenos no iguales a cero
5.0, -5.0	Flujos exógenos

El resultado que produce CCNET es el que sigue.

--- ESTADISTICAS DEL PROBLEMA ---

3 Nudos
4 Arcos
2 Arcos artificiales

Iter.	Artif.	Sum.Inf/F.Obj	Entra	de->a	Cost.Red	Sale	de->a	Paso	Artpen
1	1	5.00000	1	L->B	-9.000	A		0.000	10.000
2	1	3.00000	2*	L->U	-8.000	2*	L->U	2.000	10.000
3	0	13.00000	3	L->B	-7.000	A		3.000	

--- SOLUCION PROGRAMA LINEAL CON CONDICIONES DE RED ---

Valor de la función objetivo:	13.000
No. de iteraciones:	3
No. de pasos con aumento de flujo nulo:	1
No. de pasos en los que el árbol no cambia:	1
No. de incrementos del coste de penalizac.:	1
Ultimo multiplicador del coste de penalizac.:	1.000
Ultima penalización para arcos artificiales:	10.000

*** NUDOS ***

Nudo	 Valor	 Valor dual
NUD01	5.000	0.000
NUD02	0.000	-1.000
NUD03	-5.000	-3.000

*** ARCOS ***

Arco ...	Estado ...	Valor ...	Coste F.O. ...	L.Inf. ...	L.Sup. ...	Coste Red.
X1	BS	2.000	1.000	0.000	4.000	0.000
X2	LS	2.000	1.000	0.000	2.000	-1.000
X3	BS	3.000	3.000	0.000	4.000	0.000
X4	LI	0.000	10.000	0.000	10.000	7.000

Tiempo de cálculos: 0.05 segundos

Para que el lector compruebe la diferencia práctica que puede suponer resolver un problema de optimización en redes con un procedimiento especialmente diseñado al efecto y con otro de propósito general, basado en el mismo algoritmo *simplex* pero sin especializar sus estructuras de datos, en la tabla F.1 se indican los segundos de *cpu* obtenidos, llegando a la misma solución, en un ordenador HP APOLLO 9000 730 (23 MFLOPS LINPACK precisión doble), con los programas CCNET, BBMI y MINOS 5.1, para resolver unos problemas generados con el código NETGEN, ver Klingman [1974]. El programa MINOS 5.1, ver Murtagh y Saunders [1987], es un programa general para optimización lineal y no lineal de problemas de muy grandes dimensiones. Constituye, sin duda, un estándar mundial entre los códigos de optimización.

Tabla F.1

Segundos de *c.p.u.* invertidos en una estación de trabajo HP APOLLO 9000 730 para resolver diversos problemas de optimización en redes

Nudos	Arcos	CCNET	BBMI	MINOS 5.1
150	900	0,04	0,70	0,88
500	3000	0,28	10,10	9,81
1000	5000	0,65	51,33	48,47
3000	10000	3,59	419,90	650,31
5000	30000	18,92	2.208,56	2.782,93
10000	85000	195,69	22.129,18	24.478,51

F.4 Listado de CCNET


```

/*****
/*
/* Rutinas para la resolución de un problema lineal de
/* flujo en una red utilizando el método simplex de
/* Penalización Gradual (GPM: Gradual Penalty Method)
/*
*****/

/*****
/* Declaración de variables
*****/

/* Variables externas */

int bt,cycl,icom,irt,it,it0,ith,itmi,itmj,itmjd;
int karc,lgam,lencyc,lnod,natlrt,nswp,numart,oldarc,pas;
double artpen,aug,cp,cycst,mac,magnc,maxf,mic,objv;
double tolbg,tolnf,tolpn,tolsm,tolz,totinf;
int *arc,*d,*f,*p,*r;
double *x,*u,*cred;

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int m,mxit,n,modwr;
double cpmx,frq,p0,pbar;
int *bnz,*from,*to;
double *c,*h,*lb,*rhs;
FILE *ouf;

main(argc,argv)
int argc;
char *argv[];
{
    FILE *inf;
    static char app[] = ".cnt";
    char filein[16];
    char lin[85];
    char flt[20];
    int i,nonz;
    float time,time_ib();
    void dim(),xnet(),imprime();

    if(!strcmp(argv[argc-1],"-i")) modwr = 1;
    else modwr = 0;

    printf("\n Fichero entrada: ");
    scanf("%s",filein);
    inf = fopen(filein,"r");

    fgets(lin,85,inf);
    sscanf(lin,"%ld%c%ld",&m,&n);
    fgets(lin,85,inf);
    sscanf(lin,"%ld%c%lf%c%lf%c%lf%c%lf",&mxix,&frq,&p0,
        &pbar,&cpmx);

    dim();

/* Lectura de los vectores FROM, TO, COSTES, CAPACIDAD */

    for(i=0;i<n;i++){
        fgets(lin,85,inf);
        sscanf(lin,"%ld%c%ld%c%lf%c%lf%c%lf",&from[i],&to[i],
            &c[i],&lb[i],&h[i]);
    }

/* Lectura de los RHS distintos de cero */

    fgets(lin,85,inf);
    sscanf(lin,"%ld",&nonz);

    for (i=0;i<nonz;i++) fscanf(inf,"%ld%c",&bnz[i]);
    fgets(lin,85,inf);

/* Lectura del valor de los RHS */

    for (i=0;i<nonz;i++) fscanf(inf,"%lf%c",&rhs[bnz[i]]);

    fclose(inf);

    sprintf(flt,"%s",filein,app);
    printf(" Fichero de salida: %s\n",flt);
    ouf=fopen(flt,"w");
    fprintf(ouf,"n          --- ESTADISTICAS DEL ");
    fprintf(ouf,"PROBLEMA ---\n          -----");
    fprintf(ouf,"-----\n\n");
    fprintf(ouf,"      %4d Nudos\n",m);
    fprintf(ouf,"      %4d Arcos\n",n);
    fprintf(ouf,"      %4d Arcos artificiales\n",m-1);
    if(modwr){
        fprintf(ouf," Iter. Artif. Sum.Inf/F.Obj Entra de-");
        fprintf(ouf,">a Cost.Red Sale de->a Paso");
    }

```

```

        fprintf(ouf,"      Artpen\n");
    }

    time = time_ib();
    xnet();
    time = time_ib();

    imprime(time);
    return;
}

/*****
#include <time.h>

float time_ib()
{
    float aux1,aux2;
    static float aux;

    aux1=clock()/(float)CLOCKS_PER_SEC;
    aux2=aux1-aux;
    aux=aux1;

    return(aux2);
}

*****/

void dim()
{
    extern int *arc,*d,*f,*p,*r,*bnz;
    extern double *x,*u,*cred;
    extern void *calloc();

    arc = (int *)calloc(m,sizeof(int));
    d = (int *)calloc(m,sizeof(int));
    f = (int *)calloc(m,sizeof(int));
    p = (int *)calloc(m,sizeof(int));
    r = (int *)calloc(m,sizeof(int));
    from = (int *)calloc(n,sizeof(int));
    to = (int *)calloc(n,sizeof(int));
    bnz = (int *)calloc(m,sizeof(int));

    c = (double *)calloc(n,sizeof(double));
    h = (double *)calloc(n,sizeof(double));
    lb = (double *)calloc(n,sizeof(double));
    x = (double *)calloc(m,sizeof(double));
    u = (double *)calloc(m,sizeof(double));
    rhs = (double *)calloc(m+1,sizeof(double));
    cred = (double *)calloc(n,sizeof(double));
}

/*****
/* XNET()
/*
/* Entrada directa al método simplex
*****/

void xnet()
{
    void xnmach(),xnchlim1(),xnsmpx(),xnchlim2();

    xnmach();
    xnchlim1();
    xnsmpx();
    xnchlim2();
}

/*****
/* XNCHLIM1()
/*
/* Cambio de variable para resolver el problema con
/* todas las cotas inferiores iguales a cero.
*****/

void xnchlim1()
{
    int i;

    aug= 0.0;
    for(i=0;i<n;i++)

/* Cambio de variable para los arcos con capacidad mínima
distinta de cero */

        if(lb[i] != 0.0){

/* -- Cambio de la capacidad */
            h[i] -= lb[i];

/* -- Cambio del flujo exógeno (lado derecho) */
            rhs[from[i]] -= lb[i];
            rhs[to[i]] += lb[i];

```

```

/* -- Cambio en la función de coste */
aug += c[i]*lb[i];
}
}

/*****
/*                               */
/*      XNCHLIM2()                */
/*                               */
/*      Se deshace el cambio de variable hecho en XNCHLIM1() */
/*      para los arcos con capacidad mínima distinta de cero. */
*****/

void xnchlim2()
{
    int i;

    for(i=0;i<n;i++)

/* Cambio de variable para los arcos con capacidad mínima
distinta de cero */

        if(lb[i] != 0.0){

/* -- Cambio de la capacidad */
            if (h[i] < 0.0) h[i] -= lb[i];
            else h[i] += lb[i];

/* -- Cambio del flujo exógeno (lado derecho) */
            rhs[from[i]] += lb[i];
            rhs[to[i]] -= lb[i];
        }
    }

/*****
/*                               */
/*      XNMACH()                  */
/*                               */
/*      Definición de los valores de tolerancia por defecto */
/*      que se utilizan para esta implementación.          */
*****/

void xnmach()
{
/* Número de bits por longitud de palabra entera */
    bt = 32;

/* Tolerancia de infactibilidad */
    tolnf = 1.0e-09;

/* Número suficientemente grande que representa el infinito */
    tolbq = 1.0e20;

/* Número suficientemente pequeño que representa el cero */
    tolsm = 1.0e-30;

/* Tolerancia del cero; usada para comprobar contra cero */
    tolz = 1.0e-10;

/* Tolerancia del pricing */
    tolpn = -1.0e-04;
}

/*****
/*                               */
/*      XNSMPX()                  */
/*                               */
/*      Control del flujo del método simplex para una red uti- */
/*      lizando el método de penalización.                    */
/*                               */
/*      Secuencia de subrutinas llamadas desde xnsmpx :        */
/*      xnnit()          */
/*      xnpeny()         */
/*      xndual()         */
/*      xnph0()          */
/*      xnobjv()         */
/*      prext(sub,cs)    */
*****/

void xnsmpx()
{
    void xninit(),xnpeny(),xndual(),xnobjv(),prext();
    int xnph0();
    int numrt;
    double cpn,pennew;

    xninit();

/* Penalizaciones iniciales */

    xnpeny(&cpn,&pennew,&numrt);

/* Se comprueba si la primera penalización calculada es dema-
siado grande para esta máquina */
    if (numrt != 0) prext(3,1);

/* La penalización para los arcos artificiales es acertada */
    cp = cpn;
    artpen = pennew;

/* Variables duales */
    xndual();

    numrt = xnph0();
    if (numrt != 0) prext(3,2);

/* Cálculo del valor objetivo de la solución óptima */
    xnobjv();
}

/*****
/*                               */
/*      XNINIT()                  */
/*                               */
/*      Comprobación de los datos pbar, p0, cpmax y cálculo de */
/*      it.                                                            */
/*                               */
/*      Secuencia de subrutinas llamadas desde xninit:              */
/*      xninar()             */
/*      xnintr()             */
*****/

void xninit()
{
    void xninar(),xnintr(),prext();

/* Contador del número de iteraciones */
    itmj = 0;

/* Contador del número de veces que 'ith=3' */
    itmi = 0;

/* Contador del número de veces que 'maxf=0' (caso degenerado)*/
    itmjd = 0;

    artpen = 0.0;
    cp = 0.0;
    pas = 0;

    if((pbar <= 1.0) || (p0 <= tolz) || (cpmax < p0)) prext(4,1);
    it = (int)(frq*n*0.01+0.5);

    it = (it > 1) ? it : 1;

    xninar();
    xnintr();
}

/*****
/*                               */
/*      XNINAR()                  */
/*                               */
/*      Comprobación de los datos de capacidad, coste, from, */
/*      to y cálculo de los costes máximo y mínimo.          */
*****/

void xninar()
{
    double fabs();
    int j;
    double imic,imac,flow;

    mac = -tolbg;
    mic = tolbq;

    for(j=0;j<n;j++){
        if(h[j] < 0.0) prext(5,1);
        if(fabs(c[j]) <= tolsm) c[j] = 0.0;

/* mac = máximo coste
mic = mínimo coste */

        if(c[j] > mac) mac = c[j];
        if(c[j] < mic) mic = c[j];

        if((from[j] > m) || (to[j] > m) || (from[j] <= 0) ||
            (to[j] <= 0) || (from[j] == to[j])) prext(5,2);
    }

/* Asignación de los flujos exógenos a los arcos del árbol
artificial inicial; x[i] = flujo por el arco 'i' */

    for(j=1,flow=0.0;j<=m;j++){

```

```

        x[j] = rhs[j];
        flow += rhs[j];
    }
    if(fabs(flow) > tolnf) prext(5,3);

    imic = fabs(mic);
    imac = fabs(mac);
    magnc = (imic > imac) ? imic : imac;
}

/*****
/*      XNINTR()
/*
/*      Construcción del árbol inicial con arcos artificiales
/* de profundidad 1 y asignación de los valores iniciales de
/* las x[i] (= flujo por arco 'i').
/*
*****/

void xnintr()
{
    int i;

    totinf = 0.0;
    irt = 1;

/* Se hace siempre que irt sea el nudo 1 */

    for(i=1;i<=m;i++){
        u[i] = 0.0;
        f[i] = irt;
        p[i] = i+1;
        if(i < m) r[i+1] = i;
        d[i] = 1;
        arc[i] = -1;
        if(x[i] < 0) x[i] = -x[i];
        else if(x[i] > 0) f[i] = -irt;
        totinf += x[i];
    }

    totinf -= x[irt];
    x[irt] = 0.0;
    p[m] = irt;
    r[irt] = m;
    f[irt] = 0;
    d[irt] = 0;
    numart = m-1;
}

/*****
/*      XNPENY()
/*
/*      Cálculo de la penalización para los arcos artificiales.
/*
/*      Variables de entrada:
/*
/*      - cp      : último multiplicador del coste de penali-
/* zación.
/*      - artpen  : último coste de penalización de los arcos
/* artificiales.
/*      - pas     : número de veces que se ha calculado o in-
/* crementado el coste de penalización.
/*      - cpmax   : valor máximo para 'cp'.
/*
/*      Variables de salida:
/*
/*      numrtm = 0 penalización incrementada:
/*
/*      - cpn     : multiplicador del coste de penalización
/* para el nuevo paso.
/*      - pennew  : nuevo coste de penalización para los ar-
/* cos artificiales.
/*
/*      numrtm = 1 no se puede incrementar la penalización :
/*
/*      Variables de interés:
/*
/*      - pas     : número de veces que se ha calculado o in-
/* crementado el coste de penalización.
*****/

void xnpeny(cpn,pennew,numrtm)
double *cpn,*pennew;
int *numrtm;
{
    double fabs(),pow();
    int ictr,isame;
    double big,cmagm,cpt,difc,ppw,trialp;

    ictr = 0;
    if(cp >= cpmax) ictr = 1;

    else {
        big = tolb;
        isame = 0;
        difc = mac - mic;
        *numrtm = 0;

        cmagm = magnc*(m-1)+1;
        if(fabs(difc) >= 1.0) cpt = (cmagm-mic)/difc;
        else {
            cpt = cmagm/(1.0+fabs(mic));
            isame = 1;
        }
        cpt = (cpt < cpmax) ? cpt : cpmax;

        do {
            pas++;
            ppw = 1.0;
            if (pas != 1)
                if(pas-2 > bt) {
                    ictr = 1;
                    pas--;
                }
            else {
                ppw = pow(pbar,pow(2.0,(double)(pas-2)));
                if(ppw >= big/p0) {
                    ictr = 1;
                    pas--;
                }
            }
        }

/* Cálculo de cpn y pennew */

        if (ictr == 0) {
            *cpn = p0*ppw;

/* -- No se incrementa la penalización por encima del max
necesitado*/
            if(*cpn > cpt) *cpn = cpt;
            trialp = (isame == 0) ? (*cpn)*difc+mic :
                (*cpn)*(1+fabs(mic));
            *pennew = (double) ((int) (trialp + 0.5));
        } while ((*pennew <= artpen) && (ictr == 0));

/* No se puede incrementar mas el valor de la penalización */

        if(ictr == 1) {
            *cpn = cp;
            *pennew = artpen;
            *numrtm = 1;
        }
    }

/*****
/*      XNDUAL()
/*
/*      Cálculo de las variables duales basadas en el árbol
/* actual.
*****/

void xndual()
{
    int i,idi,nfathr;
    double costa;

    u[irt] = 0.0;
    i = p[irt];
    while(i != irt) {
        idi = arc[i];
        costa = (idi == -1) ? artpen : c[idi];
        nfathr = f[i];
        if(nfathr > 0) costa = -costa;
        u[i] = u[abs(nfathr)] + costa;
        i = p[i];
    }
}

/*****
/*      XNPHO()
/*
/*      Implementación del método de penalización gradual (GPM).
/*
/*      Secuencia de subrutinas llamadas desde xnp0 :
/*
/*      xnp20()
/*      xnatl()
/*      xnpudf()
/*      xnpivt()
/*      xnpeny()
/*      xndual()
/*      xncycl()
*****/

```

```

int xnph0()
{
    void xnpr20(),xnatl(),xnupdf(),xnpivt(),xnpeny(),xndual(),
        xncycl();
    int xnchk();
    int iext,nchrt,npenrt,numrtn;
    double cpn,pennew;

    numrtn = nswp = it0 = iext = cycl = 0;

    while (iext != 1) {
        if(mxit > itmj) {
            xnpr20();
            if(karc != -1) {

/* Cálculo del arco que sale de la base */

                xnatl();
                oldarc = arc[lnd];
                switch(natlrt) {

/* -- Paso no degenerado: se actualiza el flujo para los arcos
en el BEP */
                    case 1: xnupdf();
                        if(ith != 3) {
                            xnpivt();
                            itmj++;
                        }
                        else itmi++;
                        break;

/* -- Paso degenerado (maxf= 0) */
                    case 2: itmjd++;
                        xnpivt();
                        itmj++;
                        break;

/* -- Arco artificial en un ciclo; se calcula una penalización
mayor */
                    case 3: xnpeny(&cpn,&pennew,&npenrt);
                        if(npenrt == 0) {
                            /* Se ha podido incrementar la penalización*/
                            cp = cpn;
                            artpen = pennew;
                            xndual();
                        }
                        else numrtn = iext = 1;
                        break;

/* -- Aumento infinito de flujo; cálculo del ciclo y salida */
                    case 4: xncycl();
                        iext = cycl = 1;
                        break;
                }
                if(modwr) wrstat();
            }
            else {

/* No hay ningún arco candidato para entrar en la base */

                nchrt = xnchk();
                if(nchrt == 0) iext = 1;
                else if(nchrt == 2) numrtn = iext = 1;
            }
        }
        else prext(6,1);
    }
    return(numrtn);
}

/*****
/*
/* XNPR20()
/*
/* Cálculo de los costes reducidos y del arco que entra
/* en la base 'karc'.
/*
/* Devuelve :
/*
/* - karc = -1 todos los arcos sobre los que
/* se ha calculado pricing son optimos.*/
/*
/* - karc != -1 arco 'karc' entra en la base.
*****/

void xnpr20()
{
    int i,j,iter,ity;
    double iz,minz;

    itp = 1;
    karc = -1;
    iter = 0;

```

```

/* Se calcula el 'pricing' con todos los arcos */

while(iter == 0) {
    if(it0 >= it){
        it0 = 0;
        nswp++;
    }
    it0++;
    minz = tolbg;
    for(i=it0-1;i<n;i+=it){
        iz = u[to[i]] - u[from[i]] + c[i];
        cred[i] = iz;
        if(h[i] < 0) iz = -iz;

/* Se toma el primer arco con coste reducido menor */
        if(iz < tolbn) {
            if (iz < minz) {
                minz = iz;
                karc = i;
            }
        }

/* No hay ningún arco candidato para entrar en la base */

        if((karc != -1) || (ity >= it)) iter = 1;
        else itp++;
    }
}

/*****
/*
/* XNATL()
/*
/* Cálculo del BEP, del arco que deja la base 'lnd', y
/* del aumento de flujo a lo largo del BEP.
/*
/* Devuelve :
/*
/* - natlrt = 1 aumento de flujo finito (paso no dege-
/* nerado)
/*
/* = 2 paso degenerado
/*
/* = 3 aumento de flujo infinito; arco arti-
/* ficial en ciclo.
/*
/* = 4 aumento de flujo infinito no causado
/* por artificiales.
/*
/* Variables de interés :
/*
/* - kbnd = 1 si 'karc' esta saturado y no esta en el
/* árbol.
/*
/* = 0 si 'karc' esta a nivel 0.
/*
/* - lgam = 1 si el arco que sale 'lnd' esta en el
/* lado del from(karc) del BEP.
/*
/*
/* - ith = 1 si 'lnd' deja el árbol en su cota in-
/* ferior.
/*
/* = 2 si 'lnd' deja el árbol en su cota su-
/* perior.
/*
/* = 3 si no se necesita el paso de pivotaje;
/* esto pasa cuando el aumento de flujo
/* máximo esta limitado por arco que entra
/* 'karc'.
*****/

void xnatl()
{
    void prext();
    double fabs();
    int cmb,ext,i,idarc,idir,idk,idp,ieps,ijgam,kbnd,kfr,kfrfa;
    int kto,ktofa,nartub,node,nodefa;
    double artflo,curflo,flowi,hkarc,maxfk,mxfapf;

    artflo = tolbg;

/* Si nartub=1, uno o mas arcos artificiales en el BEP alcan-
zarán sus cotas superiores (infinitas) */

    nartub = 0;
    ith = 0;
    cmb = 0;
    ext = 0;

    hkarc = h[karc];
    kfr = from[karc];
    kto = to[karc];

    maxfk = hkarc;
    kbnd = (hkarc < 0) ? 1 : -1;

/* maxfk es el aumento de flujo máximo en 'karc' */

    maxfk = fabs(maxfk);

```

```

/* mxfafp es el aumento de flujo máximo en el BEP sin
   considerar 'karc' */

mxfafp = tolbg;

/* Diferencia en profundidades de los nudos 'kfr' y 'kto' */
idp = d[kfr] - d[kto];
if(idp != 0){
  if(idp > 0){

/* -- El lado del nudo 'from' es mas profundo */
    iep = kbnd;
    ikgam = 1;
    node = kfr;
  }
  else{

/* -- El lado del nudo 'to' es mas profundo */
    iep = kbnd;
    idp = -idp;
    ikgam = 0;
    node = kto;
  }

/* Se recorre hacia arriba el lado mas profundo y se busca el
   arco que deja la base */

    lgam = ikgam;
    for(i=0; (i<idp && ext==0); i++) {
      idarc = arc[node];
      nodefa = f[node];
      idir = (nodefa > 0) ? 1 : -1;
      curflo = x[node];

/* Se comprueba la direccion del arco 'idarc' con respecto al
   BEP */

      if(idir == iep){

/* Arco 'idarc' en la misma direccion que el BEP; puede incre-
   mentar su flujo (caso ith = 2). */

        if(idarc == -1) {

/* Arco 'idarc' es artificial basico; puede incrementar su
   flujo a un flujo infinito */

          flowi = artflo;
          if(mxfafp >= flowi) nartub = cmb = 1;
        }
        else{
          flowi = fabs(h[idarc]);
          if(flowi < tolbg) flowi = flowi - curflo;
          if(mxfafp > flowi) cmb = 1;
        }
        if(cmb == 1){

/* El arco que sale 'lnod' se quedara saturado */

          ith = 2;
          lnod = node;
          mxfafp = flowi;
          cmb = 0;
        }
      }

/* Arco 'idarc' en la direccion opuesta al BEP; puede dismi-
   nuir su flujo. */

      else if(mxfafp > curflo) {

/* El arco que sale 'lnod' disminuye a su cota inferior (= 0)*/

        ith = 1;
        mxfafp = curflo;
        lnod = node;
      }

/* Se mueve al siguiente nudo hacia arriba en el árbol */

      node = abs(nodefa);
      if(mxfafp <= tols) ext = 1;
    }

/* Se comprueba en que lado del BEP esta el nudo 'node' (últi-
   mo nudo procesado anteriormente) */

    if(ikgam == 0) kto = node;
    else kfr = node;
  }

/* Profundidades iguales. Se recorre el BEP simultáneamente

```

```

por ambos lados. (Se ejecuta el siguiente if si mxfafp>0)*/

if(ext == 0){
  while(kfr != kto){

/* Avanza un nivel hacia arriba desde el lado del nudo 'to'
   ( misma logica del for anterior ) */

    idarc = arc[kto];
    ktofa = f[kto];
    idir = (ktofa > 0) ? 1 : -1;
    curflo = x[kto];
    if(idir == kbnd) {
      if(idarc == -1){
        flowi = artflo;
        if(mxfafp >= flowi) nartub = cmb = 1;
      }
      else{
        flowi = fabs(h[idarc]);
        if(flowi < tolbg) flowi = flowi - curflo;
        if(mxfafp > flowi) cmb = 1;
      }
    }
    if(cmb == 1) {
      ith = 2;
      lgam = 0;
      mxfafp = flowi;
      lnod = kto;
      cmb = 0;
    }
  }
  else if(mxfafp > curflo) {
    ith = 1;
    mxfafp = curflo;
    lnod = kto;
    lgam = 0;
  }
  kto = abs(ktofa);
  kfrfa = f[kfr];
  if(mxfafp > tols){

/* Avanza un nivel hacia arriba desde el lado del nudo 'from'
   ( misma lógica del for anterior ) */

    idarc = arc[kfr];
    idir = (kfrfa > 0) ? 1 : -1;
    curflo = x[kfr];
    if(idir == kbnd){
      if(mxfafp > curflo) {
        ith = 1;
        mxfafp = curflo;
        lnod = kfr;
        lgam = 1;
      }
    }
    else{
      if(idarc == -1){
        flowi = artflo;
        if(mxfafp >= flowi) nartub = cmb = 1;
      }
      else{
        flowi = fabs(h[idarc]);
        if(flowi < tolbg) flowi = flowi - curflo;
        if(mxfafp > flowi) cmb = 1;
      }
    }
    if(cmb == 1){
      ith = 2;
      lgam = 1;
      mxfafp = flowi;
      lnod = kfr;
      cmb = 0;
    }
  }
}

kfr = abs(kfrfa);

if(mxfafp <= tols)

/* Continúa avanzando hacia arriba en el BEP hasta alcanzar un
   nudo comun */

  while(kfr != kto){
    kfr = abs(f[kfr]);
    kto = abs(f[kto]);
  }
}

/* Se ha llegado al nudo comun del BEP */

icom = kfr;

maxf = (mxfafp < maxfk) ? mxfafp : maxfk;

```



```

/* > Paso degenerado ? */
if(maxf <= tolsm){
/* Paso degenerado (maxf= 0; notar que 'ith' no puede ser 3)*/
    maxf = 0.0;
    if(hkarc < 0.0){
/* El arco entrante 'karc' esta saturado. Será pivoteado des-
pués (en xnpivt) y se pone su flujo igual al flujo actual
evitando así una llamada a xnupdf */
        h[karc] = -hkarc;
        maxf = -hkarc;
    }
    if(ith == 2){
/* El arco que sale 'lnod' queda saturado */
        idk = arc[lnod];
        h[idk] = -h[idk];
    }
/* Se asigna natlrt= 2 para hacer un paso de pivoteo */
    natlrt = 2;
}
else{
/* Paso no degenerado; se asigna ith= 3 (ningún cambio en el
árbol) si maxf está limitado por el flujo del arco
que entra */
    if(maxf == maxfk) ith = 3;
/* > Aumento de flujo infinito ? */
    if(maxf >= tolbq) {
        natlrt = 4;
        if(nartub) natlrt = 3;
    }
/* Aumento de flujo finito (aqui maxf= 0 ) */
    else natlrt = 1;
}
}

/*****
/*      XNUPDF()
/*
/*      Actualización del flujo por los arcos en el BEP.
/*
/*      XNUPDF() es llamada solo despues de XNATL() y siempre
/*      que:
/*
/*      1) 'maxf' es finito y distinto de cero.
/*      2) ith = 1,2, ó 3.
/*      3) XNATL devuelve natlrt = 1.
/*
/*      Devuelve:
/*      - maxf : nuevo flujo del arco que entra 'karc'.*/
/*      - x(.) : flujos actualizados.
*****/

void xnupdf()
{
    void prext();
    int i,idarc,idk,ip,iter;
    double augflo,change,floj;

/* maxf = aumento de flujo en el arco 'karc',indicado en xnatl*/
    floj = maxf;
    if(h[karc] < 0) {
/* Arco 'karc' está saturado y no está en el árbol */
        h[karc] = -h[karc];
        maxf = -maxf;
        floj = maxf + h[karc];
        if(ith == 2){
/* Se pone el arco que sale fuera del árbol (no básico) y sa-
turado */
            idk = arc[lnod];
            h[idk] = -h[idk];
        }
    }
}

```

```

    else{
/* Arco 'karc' es no básico en su cota inferior */
        if(ith == 2){
/* Se pone el arco que sale fuera del árbol (no básico) y
saturado */
            idk = arc[lnod];
            h[idk] = -h[idk];
        }
/* Se pone arco 'karc' no basico y saturado */
        else if(ith == 3) h[karc] = -h[karc];
    }
/* Se recorre el BEP y se actualizan los flujos: primero por
un lado del BEP hasta llegar al nudo común (icom), y luego
por el otro lado */
    i = from[karc];
    augflo = maxf;
    iter = 0;
    while(iter <= 1){
        if(i != icom){
            idarc = arc[i];
            ip = f[i];
            change = (ip < 0) ? -augflo : augflo;
            x[i] += change;
            if(idarc == -1) totinf += change;
            i = abs(ip);
        }
        else{
            if(iter == 0){
                augflo = -augflo;
                i = to[karc];
            }
            iter++;
        }
    }
/* maxf es ahora el nuevo flujo en 'karc' */
    maxf = floj;
}

/*****
/*      XNPIVT()
/*
/*      Actualización de la base, después de que el arco
/*      'karc' ha entrado en la base en sustitución del arco
/*      'lnod'.
/*
/*      Variables de entrada:
/*
/*      - lnod : arco que sale de la base.
/*      - karc : arco que entra en la base.
/*      - maxf : nuevo flujo en el arco que entra 'karc'.*/
/*      - lgam : = 1 si 'lnod' está en el lado del
/*                'from(karc)' en el BEP.
/*                = 0 en caso contrario.
/*      - numart : no. arcos artificiales en el árbol.
/*
/*      Variables de interés:
/*
/*      - nrk : nudo raíz del subárbol que se va a ac-
/*                tualizar.
/*      - nsr : el nuevo primer hijo de 'nrk'
/*      - iq : nudo tal que p(iq) = lnod
*****/

void xnpivt()
{
    int dir,i,id1,id2,ideld,idir1,idir2,iq,isdpth,is,isav,iter;
    int j,lf,lr,nsr,nrk;
    double ckarc,delu,flo1,flo2;

    if(arc[lnod] == -1) numart--;
    ckarc = c[karc];
    if(lgam == 1){
/* El arco 'lnod' está en el lado del 'from(karc)' en el BEP */
        nsr = from[karc];
        nrk = to[karc];
    }
    else{
/* El arco 'lnod' está en el lado del 'to(karc)' en el BEP */

```

```

    nsr = to[karc];
    nrk = from[karc];
}

iq = r[l nod];

lf = f[nsr];
idir1 = (lf < 0) ? -1 : 1;
lf = abs(lf);

flo1 = x[nsr];
idi = arc[nsr];
f[nsr] = nrk;
if(lgam == 1) f[nsr] = -f[nsr];
x[nsr] = maxf;
arc[nsr] = karc;
isdpth = d[nsr];
dir = (f[nsr] >= 0) ? -1 : 1;

/* Diferencia entre los nuevos y los antiguos valores duales
   (delu = coste reducido del arco que entra) */

delu = u[nrk] - u[nsr] + dir*ckarc;

/* Diferencia entre las nuevas y las antiguas profundidades */

ideld = d[nrk] - d[nsr] + 1;

/* Inicialización de las variables del siguiente bucle; */
/*
/* Variables de interés: */
/* i = recorre los nudos del subárbol. */
/* is = indica el nudo raíz del subárbol. */
/* lr = sucesor de 'nsr' en el árbol sin actualizar */
/* (nudo raíz del subárbol recorrido anteriormente). */
/* lf = recorre el tronco principal de pivoteaje hacia */
/* arriba en el árbol sin actualizar. */

lr = p[nsr];
isav = p[nrk];
j = lr;
i = nrk;
is = nsr;
iter = 0;

/* Se recorren los nudos del subárbol por medio de los sucesores */
while(iter == 0){
    p[i] = is;
    r[is] = i;
    i = is;

/* Actualización de las variables duales y la profundidad */

    u[i] += delu;
    d[i] += ideld;

/* Se actualizan los sucesores de 'is' por el lado izquierdo
   del subárbol */

    while(p[i] != lr){
        i = p[i];
        u[i] += delu;
        d[i] += ideld;
    }
    if(d[j] > isdpth){

/* Se actualizan los sucesores de 'is' por el lado derecho del
   subárbol */

        p[i] = j;
        r[j] = i;
    }
    while(d[j] > isdpth){
        i = j;
        u[i] += delu;
        d[i] += ideld;
        j = p[j];
    }

    if(is == l nod) iter = 1;
    else{

/* Se mueve al siguiente nudo en el tronco de pivoteaje */

        lr = is;
        is = lf;
        lf = f[is];

/* Se actualiza la información para el nuevo nudo 'is' */

        idir2 = (lf < 0) ? -1 : 1;
        lf = abs(lf);

        isdpth = d[is];
        ideld = d[lr] - isdpth + 1;
        f[is] = -idir1*lr;
        idir1 = idir2;
        flo2 = x[is];
        x[is] = flo1;
        flo1 = flo2;
        id2 = arc[is];
        arc[is] = id1;
        id1 = id2;
    }
}

/* Ya se han recorrido todos los nudos en el tronco de pivoteaje;
   se conecta el subárbol reordenado al árbol principal */

if(iq == nrk){

/* El nudo raíz del BEP no cambia de sucesor;
   se actualiza solo el último nudo del subárbol */

    p[i] = j;
    r[j] = i;
}
else {

/* Actualización del último nudo del subárbol */

    p[i] = isav;
    r[isav] = i;

/* Actualización del nudo del arco que sale y permanece en el
   árbol principal */

    p[iq] = j;
    r[j] = iq;
}
}

/*****
/*                                */
/*                                */
/*                                */
/*                                */
/*****

void xnobjv()
{
    double fabs();
    int i,iarc;

    objv = 0;

    for(i=1;i<=m;i++){

/* Arcos con flujo positivo y no artificiales en el árbol */

        iarc = arc[i];
        if((fabs(x[i]) > tolz) && (iarc != -1))
            objv += x[i]*c[iarc];
    }

/* Arcos saturados fuera del árbol */

    for(i=0;i<n;i++)
        if(h[i] < 0) objv += (-h[i])*c[i];
}

/*****
/*                                */
/*                                */
/*                                */
/*                                */
/*                                */
/*****

int xnchk()
{
    void xndual(),prext();
    int nirtn,numrtn;
    double cm,dbig,dflm,npenc,ptry,xnumhi;

    nirtn = 0;
    if(totinf > tolnf){

/* Existen arcos artificiales positivos;

```

```

    si 'artpen' > (m-1)*máximo coste, entonces se garantiza que
    no existe solución factible. */
    if((artpen/(m-1)) >= magnc) prext(15,1);
    else {

/* No se puede decidir si hay solución factible; se incrementa
   el valor de la penalización para todos los arcos artificia-
   les. */

    cm = magnc;
    xnpeny(&ptry,&npenc,&numrtn);
    if(numrtn == 0) {

/* xnpeny sugiere que la penalización debe ser aumentada.
   Se comprueba si es posible implementar este incremento;
   ( > es la longitud de palabra suficiente para el peor caso
   de coste reducido máximo con la nueva penalización ? ) */

    dflm = (double) m;
    dbig = tolbg/dflm;
    xnumhi = cm*(dflm - 2)/dflm + 2*npenc/dflm;
    if(xnumhi < dbig){

/* Se puede incrementar la penalización */

    cp = ptry;
    artpen = npenc;
    xndual();
    nirtn = 1;
    }

/* No se puede incrementar mas la penalización; se acaba el
   proceso */

    if((numrtn != 0) || (xnumhi >= dbig)) nirtn = 2;
    }
    return(nirtn);
}

/*****
/* XNCYCL()
/*
/* Identifica y clasifica el ciclo negativo (cuya existen-*/
/* cia es conocida anteriormente a la llamada a esta subru-*/
/* tina).
/* Mientras se traza el BEP esta subrutina no considera la*/
/* dirección de los arcos.
/*
/* Devuelve:
/* - lencyc : número de arcos en el ciclo.
/* - cycst : suma de los costes de los arcos en*/
/* el ciclo; (sin tener en cuenta su */
/* dirección).
/* - r : vector de la lista de los arcos en*/
/* el ciclo; (nota: la lista del pre-*/
/* orden inverso se pierde despues de*/
/* un ciclo negativo).
*****/

void xncycl()
{
    int iarc,iter,next,node,i;
    double artcs,costa;

/* Asignación a 'cycst' del coste del arco que entra 'karc' */

    cycst = c[karc];

/* Coste de los arcos artificiales */

    artcs = artpen;
    lencyc = 1;
    r[1] = karc;
    next = node = from[karc];
    iter = 0;
    while(iter <= 1)
        if(next != icom){
            lencyc++;
            iarc = arc[next];
            r[lencyc] = iarc;
            costa = (iarc != -1) ? c[iarc] : artcs;
            cycst += costa;
            next = abs(f[next]);
        }
    else{
        if(node != to[karc]) next = node = to[karc];
        iter++;
    }
}

```

```

/*****
/* PREXT()
/*
/* Impresión de los códigos de error o salidas anormales */
/* en las que el problema no es factible, o no se puede */
/* alcanzar el óptimo.
*****/

void prext(sub,cs)
int sub,cs;
{
    switch(sub){
        case 3 : printf("    Stop en xnsmpx.\n    Razon: ");
            if(cs == 1){
                printf("el 'numrtn' de xnpeny es distinto de ");
                printf("0.\nLa primera penalización no puede");
                printf(" ser calculada porque es muy grande ");
                printf("para esta máquina.\n");
            }
            if(cs == 2){
                printf("el 'numrtn' de xnph0 es distinto de 0");
                printf(".\nArco artificial en ciclo; la penal");
                printf("ización para los arcos artificiales, ");
                printf("particularmente aquellos en el ciclo,");
                printf(" no se puede incrementar más.\n");
            }
            break;
        case 4 : printf("    Stop en xninit.\n    Razón: ");
            if(cs == 1) {
                printf("pbar <= 1, p0 <= tolz o ");
                printf("cpmax < p0\n");
            }
            break;
        case 5 : printf("    Stop en xninar\n    Razón: ");
            if(cs == 1) {
                printf("hay un arco de capacidad negativa o ");
                printf("con límite inferior mayor que la ca");
                printf("pacidad.\n");
            }
            if(cs == 2){
                printf("mala definición de los vectores from,");
                printf(" to, o ambos.\n");
            }
            if(cs == 3) printf("Problema infactible\n");
            break;
        case 6 : printf("    Stop en xnph0\n    Razon: ");
                printf("Se ha sobrepasado el máximo número de i");
                printf("teraciones\n");
                break;
        case 15: printf("    Stop en xnchk\n    Razon: ");
                printf("No existe solución factible, porque art");
                printf("pen > (m-1)*magnc\n");
                break;
    }
    exit(0);
}

/*****
/* IMPRIME()
/*
/* Impresión de los resultados del proceso.
*****/

#include <stdio.h>

void imprime(time)
float time;
{
    int i,j,alt=0,deg=0;

    fprintf(ouf,"\n\n    --- SOLUCION PROGRAMA LINEAL CON ");
    fprintf(ouf,"CONDICIONES DE RED ---\n");
    fprintf(ouf,"-----\n\n\n");

    fprintf(ouf," Valor de la función objetivo:          ");
    fprintf(ouf,"%14.3f\n",objv+aug);
    fprintf(ouf," No. de iteraciones:          ");
    fprintf(ouf," %8d\n",itmj+itmi);
    fprintf(ouf," No. de pasos con aumento de flujo nulo:    ");
    fprintf(ouf," %4d\n",itmjd);
    fprintf(ouf," No. de pasos en los que el árbol no cambia:");
    fprintf(ouf," %4d\n",itmi);
    fprintf(ouf," No. de incrementos del coste de penalizac.:");
    fprintf(ouf," %4d\n",pas);
    fprintf(ouf," Ultimo multiplicador del coste de penalizac");
    fprintf(ouf," : %8.3f\n",cp);
    fprintf(ouf," Ultima penalización para arcos artificiales");
    fprintf(ouf," : %8.3f\n",artpen);
    if(cycl){
        fprintf(ouf," No. de arcos en el ciclo:    %21d\n",lencyc);
    }
}

```

```

    fprintf(ouf, " Suma de costes de los arcos en el ciclo:");
    fprintf(ouf, "%13.3f\n", cycst);
    fprintf(ouf, "\n\n\n PROBLEMA NO FACTIBLE, por la existe");
    fprintf(ouf, "ncia de un ciclo negativo.\n");
    fprintf(ouf, " Arcos que forman el ciclo:\n\n");
    for(i=1; i<=lencyc; i++)
        fprintf(ouf, " X%-3d\n", r[i+1]);
}

fprintf(ouf, "\n\n\n *** NUDOS ***\n\n");
fprintf(ouf, " Nudo ..... Valor ..... Valor dual\n");
for (i=1; i<=m; i++)
    fprintf(ouf, " NUDO%-3d%-11.3f%-14.3f\n", i, rhs[i], u[i]);
fprintf(ouf, "\n");

fprintf(ouf, "\n\n\n *** ARCOS ***\n\n ");
fprintf(ouf, "Arco ... Estado ... Valor ... Coste F.O. ...");
fprintf(ouf, " L.Inf. ... L.Sup. ... Coste Red.\n");
for(i=0; i<n; i++){
    fprintf(ouf, " X%-3d ", i+1);
    if(h[i] < 0.0) {
        if(cred[i] == 0.0) alt = 1;
        fprintf(ouf, "LS%-12.3f%-12.3f%-13.3f%-11.3f%-13.3f\n", -h[i],
            c[i], lb[i], -h[i], cred[i]);
    }
    else {
        j=1;
        while((arc[j] != i) && (j <= m)) j++;
        if(i == arc[j]) {
            if((x[j] == h[i]) || (x[j] == lb[i])) deg = 1;
            fprintf(ouf, "BS%-12.3f%-12.3f%-13.3f", x[j]+lb[i], c[i],
                lb[i]);
            if(h[i] == tolbg) fprintf(ouf, "%11.0e%-13.3f\n", h[i],
                cred[i]);
            else fprintf(ouf, "%11.3f%-13.3f\n", h[i], cred[i]);
        }
        else {
            if(cred[i] == 0.0) alt = 1;
            fprintf(ouf, "LI%-12.3f%-12.3f%-13.3f", lb[i], c[i], lb[i]);
            if(h[i] == tolbg) fprintf(ouf, "%11.0e%-13.3f\n", h[i],
                cred[i]);
            else fprintf(ouf, "%11.3f%-13.3f\n", h[i], cred[i]);
        }
    }
}
fprintf(ouf, "\n");
if (alt)
    fprintf(ouf, " Existen soluciones optimas alternativas\n");
if (deg) fprintf(ouf, " Problema degenerado\n");
fprintf(ouf, "\n Tiempo de cálculos: %5.2f segundos\n", time);
fclose(ouf);
}

/*****
/*          WRSTAT()
/*
/*  Impresión de resultados intermedios del proceso.
/*
*****/

wrstat()

```

```

{
    int i;
    double aux = 0.0;
    static char lb[] = "L->B";
    static char lu[] = "L->U";
    static char bl[] = "B->L";
    static char bu[] = "B->U";
    static char ul[] = "U->L";
    static char ub[] = "U->B";
    void xnobjv();
    aux = objv;

    fprintf(ouf, "%5d %5d", itmi+itmj, numart);
    if(numart > 0) aux = totinf;
    else{
        xnobjv();
        aux = objv;
    }
    fprintf(ouf, " %11.5f %5d", aux, karc+1);
    switch(natlrnt){
        case 1: if(ith != 3){
            if(h[karc] > 0.0) fprintf(ouf, " %s", lb);
            else fprintf(ouf, " %s", ub);
            fprintf(ouf, " %9.3f", cred[karc]);
            if(olddarc != -1){
                fprintf(ouf, " %5d", olddarc+1);
                if(h[arc[lnd]] > 0.0) fprintf(ouf, " %s", bl);
                else fprintf(ouf, " %s", bu);
            }
            else fprintf(ouf, " A ");
        }
        else{
            if(h[karc] < 0) {
                fprintf(ouf, "* %s %9.3f", lu, cred[karc]);
                fprintf(ouf, " %5d* %s", karc+1, lu);
            }
            else {
                fprintf(ouf, "* %s %9.3f", ul, cred[karc]);
                fprintf(ouf, " %5d* %s", karc+1, ul);
            }
        }
        if(numart)
            fprintf(ouf, " %9.3f %9.3f", maxf, artpen);
        else
            fprintf(ouf, " %9.3f", maxf);
        break;

        case 2: if(maxf == 0.0) fprintf(ouf, " %s", lb);
            else fprintf(ouf, " %s", ub);
            fprintf(ouf, " %9.3f", cred[karc]);
            if(olddarc != -1){
                fprintf(ouf, " %5d", olddarc+1);
                if(h[arc[lnd]] > 0.0) fprintf(ouf, " %s", bl);
                else fprintf(ouf, " %s", bu);
            }
            else fprintf(ouf, " A ");
            if(numart)
                fprintf(ouf, " %9.3f %9.3f", maxf, artpen);
            else
                fprintf(ouf, " %9.3f", maxf);
            break;
    }
    fprintf(ouf, "\n");
}

```


Apéndice G

VERSIONES EN C y FORTRAN 90 DE LOS PROGRAMAS DEL TEXTO EN FORTRAN 77

EN ESTE APÉNDICE se listan las versiones en C y FORTRAN 90 de los códigos presentados en el texto. Como en el caso de sus homólogos en FORTRAN 77, los que aquí incluimos sólo intentan presentar una forma de implementar los algoritmos expuestos base de esos códigos. Para entender estas implementaciones conviene tener en cuenta que codificar un programa en cualquier lenguaje se parece mucho a redactar cualquier texto: la personalidad de su autor queda reflejada en la forma de hacerlo y en su fondo. El fondo, en este caso, lo definen los algoritmos que se pretenden implementar por lo que la fidelidad a ellos pretende ser absoluta.

Aun cuando el lector se puede hacer la pregunta de por qué *traducir* a FORTRAN 90 los códigos que se han listado en el texto en FORTRAN 77, creemos que puede ser útil por dos motivos esencialmente:

- Porque FORTRAN 90 ha introducido unos cambios muy significativos y trascendentes con respecto al estándar que definió el FORTRAN 77, que no son trivialmente asumibles y que potencian de forma sustancial las prestaciones de éste frente a sus “competidores” C y C++.
- Porque una buena forma de aprender cómo utilizar un nuevo lenguaje de programación y beneficiarse de sus nuevas prestaciones es contrastarlo, con ejemplos concretos, con códigos de probada utilidad en uno antiguo. Como los que se han estudiado en el texto en FORTRAN 77 son la mayoría de ellos muy sencillos, puede resultar de gran utilidad

práctica para el lector recurrir a éstos en FORTRAN 90 y familiarizarse y entrenarse con un lenguaje con el que tal vez tenga que convivir en el futuro.

Por lo que respecta a las versiones en C, es indudable que es en este lenguaje, y en C++, donde se diseñan muchas de las nuevas aplicaciones en los mundos científico y tecnológico. Aunque el C gana terreno día a día al FORTRAN, los programas de diseño y análisis tradicionales de la ingeniería, de la técnica y del análisis numérico siguen codificados y se codifican en FORTRAN. La aparición del nuevo estándar FORTRAN 90 quizás detenga esa pérdida de terreno del FORTRAN con respecto al C. Los nuevos por venir, HPF (High Performance Fortran) y FORTRAN 95, qué duda cabe, contribuirán a mejorar el panorama de lenguajes disponibles para diseño e implementación de nuevas técnicas, procedimientos de simulación, etc.: en definitiva, a aumentar las posibilidades que los ordenadores ofrecen para mejorar la calidad de vida de la sociedad y resolver los retos tecnológicos que ésta se plantea. Auténticos lenguajes como los que implícitamente proporcionan los paquetes de *software* Matlab, Mathematica o Gauss (hay otros, evidentemente) también deberán aportar lo suyo a este respecto.

G.1 Códigos en C

G.1.1 Códigos del capítulo 1

1. Programa Gauss de la página 15.

```
/*
-- Eliminación de Gauss
*/

#include <stdio.h>
#include <math.h>
#define SWAPf(a,b) {float temp=a; a=b; b=temp;}
#define N 3
#define NN 4

main(){
    int i, j, k, l;
    float smax, r, c, x[NN];
    static float a[NN][NN] = {2,1,0,4,2,-4,-2,3,
                              -7,-9,4,1,-2,8,2,0,-3,-12,-1,2};

    /* *** Triangularización *** */

    for (k=0, l=0; k<N; k++){
        smax = fabs(a[k][k]);
        for (i=k+1; i<=N; i++) if (fabs(a[i][k]) > smax){
            l = i;
            smax = fabs(a[i][k]);
        }
        if (l) for (i=k; i<=N+1; i++) SWAPf(a[l][i],a[k][i]);
        for (i=k+1; i<=N; i++){
            r = a[i][k]/a[k][k];
            for (j=k+1; j<=NN; j++) a[i][j] -= r*a[k][j];
        }
    }

    /* *** Sustitución inversa *** */

    x[N] = a[N][NN]/a[N][N];
    for (i=N-1; i>=0; i--){
        for (j=i+1, c=a[i][NN]; j<=N; j++) c -= a[i][j]*x[j];
        x[i] = c/a[i][i];
    }

    printf("Solución = [%f,%f,%f,%f]\n", x[0], x[1], x[2], x[3]);
    return 0;
}
```

2. Programa Gaussc de la página 19.

```
/*
-- Resolución de un sistema lineal regular cualquiera mediante
eliminación de Gauss
```

```
*/

#include <stdio.h>
#include <math.h>
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

main(){
    char fil[14], linea[81];
    int i, iaux, ip, *ipvt, *ivector(), j, k, l, n, pi;
    float **a, *b, c, **matriz(), r, r1, smax, *vector(), *x;
    FILE *f1;

    printf ("Dimension de la Matriz A?\n");
    scanf ("%d", &n);
    printf ("Fichero de datos?\n");
    scanf ("%s", &fil);
    f1 = fopen (fil, "r");
    a = matriz(n,n); /* Reservar memoria para A */
    b = vector(n); /* " " para b */
    x = vector(n); /* " " para x */
    n -= 1;
    for (j=0; j<=n; j++){
        fgets (linea,81,f1);
        for (i=0; i<=n; i++) a[j][i]=atof(linea+i*5);
    }
    fgets (linea,81,f1);
    for (i=0; i<=n; i++) b[i]=atof(linea+i*5);

    ipvt = ivector(n); /* Reservar memoria para ipvt */
    for (i=0; i<=n; i++) ipvt[i]=i;

    /* *** Triangularización *** */

    for (k=0, l=0; k<=n; k++){
        smax = fabs(a[ipvt[k]][k]);
        for (i=k+1; i<=n; i++){
            ip = ipvt[i];
            if (fabs(a[ip][k]) > smax){
                l = i;
                smax = fabs(a[ip][k]);
            }
        }
        if (l) SWAPi(ipvt[k],ipvt[l]);
        pi = ipvt[k];
        r1 = 1.0/a[pi][k];
        for (i=k+1; i<=n; i++){
            ip = ipvt[i];
            r = a[ip][k]*r1;
            for (j=k+1; j<=n; j++) a[ip][j] -= r*a[pi][j];
            a[ip][k] = -r;
        }
    }

    for (k=0; k<=n; k++){
        ip = ipvt[k];
        for (i=k+1; i<=n; i++){
            pi = ipvt[i];
            b[pi] += a[pi][k]*b[ip];
        }
    }
}
```

```

/* *** Sustitución inversa *** */

x[n] = b[ipvt[n]]/a[ipvt[n]][n];
for (i=n-1; i>=0; i--){
    pi = ipvt[i];
    for (j=i+1; j<=n; j++) c -= a[pi][j]*x[j];
    x[i] = c/a[pi][i];
}

printf("Solución: ");
for (i=0; i<=n; i++) printf("%f, ", x[i]);
printf("\n");

return 0;
}

float **matriz(nf,nc)
int nf,nc;
{
    int i;
    float **m, *m1;

    m1 = (float *) calloc(nf*nc,sizeof(float));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (float **) malloc(nf*sizeof(float *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

float *vector(n)
int n;
{
    float *m;

    m = (float *) malloc(n*sizeof(float));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

int *ivector(n)
int n;
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

```

3. Programa Crout de la página 30

```

/*
-- Descomposición LU1 mediante el método de Crout
*/

#include <stdio.h>
#include <math.h>

#define N 2

main (){

    int i, j, k, l;
    float sum;
    static float a[N+1][N+1]={10.,10.,20.,20.,25.,40.,30.,50.,61};

/* *** Descomposición *** */

    for (k=0; k<=N; k++){
        for (i=k; i<=N; i++){
            for (l=0, sum=0; l<=k-1; l++) sum += a[i][l]*a[l][k];
            a[i][k] -= sum;
        }
        for (i=k+1; i<=N; i++){

```

```

            for (l=0, sum=0; l<=k-1; l++) sum += a[k][l]*a[l][i];
            a[k][i] = (a[k][i]-sum)/a[k][k];
        }
    }

    printf("Matriz A factorizada:\n");
    for (i=0; i<=N; i++){
        for (j=0; j<=N; j++) printf ("%7.2f", a[i][j]);
        printf ("\n");
    }

    return 0;
}

```

4. Programa Croutp de la página 33.

```

/*
-- Descomposición LU1 mediante el método de Crout
con pivotación
*/

#include <stdio.h>
#include <math.h>
#define N 2
#define SWAPf(a,b) {float temp=a; a=b; b=temp;}
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

main (){

    int i, iaux, ipvt[N+1], j, k, l;
    float aux, sum, smax;
    static float a[N+1][N+1]={10.,10.,20.,20.,25.,40.,30.,50.,61};

    for (i=0; i<=N; i++) ipvt[i]=i;

/* *** Descomposición *** */

    for (k=0; k<=N; k++){
        for (i=k, l=0, smax=0; i<=N; i++){
            for (j=0, sum=0; j<=k-1; j++) sum += a[i][j]*a[j][k];
            a[i][k] -= sum;
            if (fabs(a[i][k]) > smax){
                l = i;
                smax = fabs(a[i][k]);
            }
            if (l) {
                for (j=0; j<=N; j++) SWAPf(a[l][j],a[k][j]);
                SWAPi(ipvt[l],ipvt[k]);
            }
        }
        for (i=k+1; i<=N; i++){
            for (l=0, sum=0; l<=k-1; l++) sum += a[k][l]*a[l][i];
            a[k][i] = (a[k][i]-sum)/a[k][k];
        }
    }

    for (j=0; j<=N; j++) printf ("%d", ipvt[j]+1);
    printf ("\n");
    printf("Matriz A factorizada:\n");
    for (i=0; i<=N; i++){
        for (j=0; j<=N; j++) printf ("%8.4f", a[i][j]);
        printf ("\n");
    }

    return 0;
}

```

5. Programa Croutl1u de la página 36.

```

/*
-- Descomposición L1U mediante el método de Crout
*/

#include <stdio.h>
#include <math.h>

#define N 2

main (){

    int i, j, k, l;
    float sum;
    static float a[N+1][N+1]={10.,10.,20.,20.,25.,40.,30.,50.,61};

/* *** Descomposición *** */

    for (k=0; k<=N; k++){
        for (j=k; j<=N; j++){

```



```

        for (l=0, sum=0; l<=k-1; l++) sum += a[k][l]*a[l][j];
        a[k][j] -= sum;
    }
    for (i=k+1; i<=N; i++){
        for (l=0, sum=0; l<=k-1; l++) sum += a[i][l]*a[l][k];
        a[i][k] = (a[i][k]-sum)/a[k][k];
    }
}

printf ("Matriz A factorizada:\n");
for (i=0; i<=N; i++){
    for (j=0; j<=N; j++) printf ("%8.3f", a[i][j]);
    printf ("\n");
}

return 0;
}

```

6. Programa Dool de la página 38.

```

/*
-- Descomposición LU mediante el método de Doolittle
*/

#include <stdio.h>
#include <math.h>

#define N 3
#define SWAPf(a,b) {float temp=a; a=b; b=temp;}
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

main (){

    int i, iaux, imax, ipvt[N+1], j, k, l;
    float aux, sum, smax;
    static float a[N+1][N+1]={1,-4,1,1,1,0,1,3,-2,0,-1,0,0,0,1,0};

    for (i=0; i<=N; i++) ipvt[i]=i;

/* *** Descomposición *** */

    for (j=0; j<=N; j++){
        for (i=0; i<=j-1; i++){
            for (k=0, sum=a[i][j]; k<=i-1; k++)
                sum -= a[i][k]*a[k][j];
            a[i][j] = sum;
        }
        smax = 0.0;
        for (i=j; i<=N; i++){
            for (k=0, sum=a[i][j]; k<=j-1; k++)
                sum -= a[i][k]*a[k][j];
            a[i][j] = sum;
            if (fabs(sum) >= smax){
                imax = i;
                smax = fabs(sum);
            }
        }
        if (j != imax){
            for (k=0; k<=N; k++) SWAPf(a[imax][k], a[j][k]);
            SWAPi(ipvt[imax], ipvt[j]);
        }
        if (j != N){
            if (!a[j][j]) a[j][j] = 1.e-20;
            for (i=j+1, aux=1/a[j][j]; i<=N; i++) a[i][j] *= aux;
        }
    }
    if (!a[N][N]) a[N][N] = 1.e-20;

    for (j=0; j<=N; j++) printf ("%d ", ipvt[j]+1);
    printf ("\n");
    printf ("Matriz A factorizada:\n");
    for (i=0; i<=N; i++){
        for (j=0; j<=N; j++) printf ("%8.4f", a[i][j]);
        printf ("\n");
    }

    return 0;
}

```

7. Programa Chol de la página 45.

```

/*
-- Resolución de un sistema de ecuaciones lineales Ax=b
en el que la matriz A es definida positiva y se
factoriza por Cholesky G'G
*/

#include <stdio.h>

```

```

#include <math.h>

#define N 3

main (){

    int i, j, k;
    float sum;
    static float a[N+1][N+1]={5,1,-2,0,1,2,0,0,-2,0,4,1,0,0,1,3},
        b[N+1]={1,5,14,15};

/* *** Descomposición G'G *** */

    for (k=0; k<=N; k++){
        for (j=0, sum=a[k][k]; j<=k-1; j++)
            sum -= pow(a[k][j],2.0);
        a[k][k] = sqrt(sum);
        for (i=k+1; i<=N; i++){
            for (j=0, sum=a[i][k]; j<=k-1; j++)
                sum -= a[i][j]*a[k][j];
            a[i][k] = sum/a[k][k];
        }
    }

/* *** Sustitución directa *** */

    for (i=0; i<=N; i++){
        for (j=0; j<=i-1; j++) b[i] -= a[i][j]*b[j];
        b[i] /=a[i][i];
    }

/* *** Sustitución inversa *** */

    b[N] /=a[N][N];
    for (i=N-1; i>=0; i--){
        for (j=i+1; j<=N; j++) b[i] -= a[j][i]*b[j];
        b[i] /=a[i][i];
    }

    printf ("Matriz A factorizada:\n");
    for (i=0; i<=N; i++){
        for (j=0; j<=i; j++) printf ("%9.4f", a[i][j]);
        printf ("\n");
    }
    printf ("Solución:\n");
    for (j=0; j<=N; j++) printf ("%9.4f", b[j]);

    return 0;
}

```

8. Programa Aasen de la página 57.

```

/*
-- Descomposición LTL' mediante el método de Aasen
*/

#include <stdio.h>
#include <math.h>

#define N 2
#define Nm1 3
#define SWAPf(a,b) {float temf=a; a=b; b=temf;}
#define SWAPi(a,b) {int temi=a; a=b; b=temi;}

main (){

    int i, iaux, ipvt[Nm1], iq, j, k, k1;
    float aux, sum, smax, alfa[Nm1], beta[Nm1], l[Nm1][Nm1],
        h[Nm1], v[Nm1], lo[Nm1];
    float a[Nm1][Nm1]={1,10,20,10,1,30,20,30,1};

    for (i=0; i<=N; i++) ipvt[i] = i;

/* *** Factorización *** */

    for (j=0; j<=N; j++){
        if (!j) h[0] = a[0][0];
        else if (j == 1){
            h[0] = beta[0];
            h[1] = a[1][1];
        } else{
            lo[0] = 0;
            for (k=1; k<=j-1; k++) lo[k] = l[j][k];
            lo[j] = 1;
            h[j] = a[j][j];
            h[0] = beta[0]*lo[1];
            for (k=1; k<=j-1; k++){
                h[k] = beta[k-1]*lo[k-1]+
                    alfa[k]*lo[k]+beta[k]*lo[k+1];
                h[j] -= lo[k]*h[k];
            }
        }
    }
}

```

```

    }
    if (j==0 || j==1) alfa[j] = h[j];
    else alfa[j] = h[j]-beta[j-1]*l[j][j-1];
    if (j <= N-1){
        for (k=j+1, smax=0, iq=j; k<=N; k++){
            for (k1=0, sum=0; k1<=j; k1++){
                sum -= l[k][k1]*h[k1];
                v[k] = a[k][j]+sum;
                if (fabs(v[k]) > smax){
                    smax = fabs(v[k]);
                    iq = k;
                }
            }
            SWAPf(v[j+1],v[iq]);
            SWAPi(ipvt[j],ipvt[iq]);
            for (k=1; k<=j; k++) SWAPf(l[j+1][k],l[iq][k]);
            for (k=j+1; k<=N; k++) SWAPf(a[j+1][k],a[iq][k]);
            for (k=j+1; k<=N; k++) SWAPf(a[k][j+1],a[k][iq]);
            beta[j] = v[j+1];
        }
    }
    if (j <= N-2){
        for (k=j+2; k<=N; k++) l[k][j+1] = v[k];
        if (v[j+1] for (k=j+2; k<=N; k++) l[k][j+1] /=v[j+1];
    }
}

for (j=0; j<=N; j++) printf ("%9.4f", alfa[j]);
printf ("\n");
for (j=0; j<=N; j++) printf ("%9.4f", beta[j]);
printf ("\n");
printf ("Matriz L:\n");
for (i=1; i<=N; i++){
    for (j=0; j<=i-1; j++) printf ("%9.4f", l[i][j]);
    printf ("\n");
}
for (j=0; j<=N; j++) printf ("%d", ipvt[j]+1);

return 0;
}

```

9. Programa Bunch y rutina bunchkauf de la página 66.

```

/*
-- Descomposición UBU' mediante el método de Bunch y Kaufman
*/

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define N 2
#define Nm1 3
#define ALPHA (1.0+sqrt(17.0))/8.0
#define SWAPf(a,b) {float temf=a; a=b; b=temf;}
#define SWAPi(a,b) {int temi=a; a=b; b=temi;}

void bunchkauf(float a[][Nm1], int ipvt[]);
void cambia (int k, float *a, float *b);
void apory (int k, float mult, float *a, float *b);
int colmax (int k, float *a);

main (){
    int i, ipvt[Nm1], j;
    static float a[Nm1][Nm1]={1,10,20,10,1,30,20,30,1};

    /* *** Factorización en rutina bunchkauf *** */

    bunchkauf (a, ipvt);

    for (i=0; i<=N; i++){
        for (j=0; j<=N; j++) printf ("%9.4f", a[i][j]);
        printf ("\n");
    }
    for (j=0; j<=N; j++) printf ("%d ", ipvt[j]);

    return 0;
}

void bunchkauf(float a[][Nm1], int ipvt[])
{
    int colmax(), i, imax, j, jmax, k, kstep, swap;
    float aux, absakk, ak, akml, deno, bk, bkml, colm, mulk,
        mulkml, rowm;

    k = N;
    while (k > 0){
        absakk = fabs(a[k][k]);

```

```

        imax = colmax(k-1,&a[0][k]);
        colm = fabs(a[imax][k]);
        if (absakk >= ALPHA*colm){
            kstep = 1;
            swap = 0;
        } else {
            for (j=imax+1, rowm=0; j<=k; j++){
                rowm=max(rowm,fabs(a[imax][j]));
            }
            if (imax) {
                jmax = colmax(imax-1,&a[0][imax]);
                rowm = max(rowm,fabs(a[jmax][imax]));
            }
            if (fabs(a[imax][imax]) >= ALPHA*rowm){
                kstep = 1;
                swap = 1;
            } else if (absakk >= ALPHA*colm*(colm/rowm)) {
                kstep = 1;
                swap = 0;
            } else {
                kstep = 2;
                swap = imax != k-1 ? 1 : 0;
            }
        }
        if (!max(absakk,colm)){
            ipvt[k] = k;
            k -= kstep;
            continue;
        }
        if (kstep == 1) {
            ipvt[k] = k;
            if (swap){
                cambia(imax,&a[0][imax],&a[0][k]);
                for (j=k; j>=imax; j--) SWAPf(a[j][k],a[imax][j]);
                ipvt[k] = imax;
            }
            for (j=k-1; j>=0; j--){
                mulk = -a[j][k]/a[k][k];
                apory(j,mulk,&a[0][k],&a[0][j]);
                a[j][k] = mulk;
            }
        } else {
            ipvt[k] = ipvt[k-1] = 1-k;
            if (swap){
                cambia(imax,&a[0][imax],&a[0][k-1]);
                for (j=k-1; j>=imax; j--)
                    SWAPf(a[j][k-1],a[imax][j]);
                SWAPf(a[k-1][k],a[imax][k]);
                ipvt[k] = -imax;
            }
            if (k > 1){
                ak = a[k][k]/a[k-1][k];
                akml = a[k-1][k-1]/a[k-1][k];
                deno = 1.e0-ak*akml;
                for (j=k-2; j>=0; j--){
                    bk = a[j][k]/a[k-1][k];
                    bkml = a[j][k-1]/a[k-1][k];
                    mulk = (akml*bk-bkml)/deno;
                    mulkml = (ak*bkml-bk)/deno;
                    apory(j,mulk,&a[0][k],&a[0][j]);
                    apory(j,mulkml,&a[0][k-1],&a[0][j]);
                    a[j][k] = mulk;
                    a[j][k-1] = mulkml;
                }
            }
        }
        k -= kstep;
    }
    if (!k) ipvt[0] = 0;
}

int colmax (int k, float *a)
{
    int i, imax;
    float dmax;
    for (i=0, dmax=0; i<=k; i++){
        if (fabs(*(a+Nm1*i)) > dmax){
            imax = i;
            dmax = fabs(*(a+Nm1*i));
        }
    }
    return imax;
}

void cambia (int k, float *a, float *b)
{
    int i;
    float aux;
    for (i=0; i<=k; i++){
        aux = *(a+Nm1*i);
        *(a+Nm1*i) = *(b+Nm1*i);
        *(b+Nm1*i) = aux;
    }
}

```

```
void apory (int k, float mult, float *a, float *b)
{
    int i;
    for (i=0; i<=k; i++) *(b+Nm1*i) += mult*(*(a+Nm1*i));
}
```

10. Programa Grmsch de las página 87.

```
/*
-- Resolución del problema lineal de mínimos cuadrados por
  Gram-Schmidt
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

#define M 3
#define Mm1 4
#define N 2
#define Nm1 3

double prod (int k, double *a, int n, double *b, int nn);

main (){

    int i, j, k, n=N, nm1=Nm1, m=M, uno=1;
    double dmax, u[Nm1][Nm1], x[Nm1], temp, prod();
    static double a[Mm1][Nm1]={1,1,1,0,0,0,0,0,0,0,0,0},
        b[Mm1]={1,0,0,0};

    a[1][0] = sqrt(DBL_EPSILON)*10;
    a[2][1] = sqrt(DBL_EPSILON)*10;
    a[3][2] = sqrt(DBL_EPSILON)*10;

    /* *** Ortonormalización de columnas de A *** */

    for (j=0; dmax=0; j<=n; j++){
        for (i=0; i<=j-1; i++){
            u[i][j] = prod(m,&a[0][i],nm1,&a[0][j],nm1);
            for (k=0; k<=m; k++) a[k][j] -= u[i][j]*a[k][i];
        }
        temp = sqrt(prod(m,&a[0][j],nm1,&a[0][j],nm1));
        u[j][j] = temp;
        for (k=0; k<=m; k++) a[k][j] /= temp;
        if (temp > dmax) dmax = temp;
        if (dmax+temp == dmax){
            printf ("Stop: Existe dependen. lineal columna %d\n",j);
            return 0;
        }
    }
    x[n] = prod(m,&a[0][n],nm1,&b,uno)/u[n][n];
    for (i=n-1; i>=0; i--){
        temp = prod(m,&a[0][i],nm1,&b,uno);
        for (j=i+1; j<=n; j++) temp -= u[i][j]*x[j];
        x[i] = temp/u[i][i];
    }
    for (j=0; j<=n; j++){
        temp = prod(m,&a[0][j],nm1,&b,uno);
        for (i=0; i<=m; i++) b[i] -= temp*a[i][j];
    }

    for (j=0; j<=n; j++) printf ("%f ", x[j]);
    printf ("\n");
    for (j=0; j<=n; j++) printf ("%f ", b[j]);
    printf ("\n");

    return 0;
}

double prod (int k, double *a, int n, double *b, int nn)
{
    int i;
    double pro;
    for (i=0, pro=0; i<=k; i++) pro += *(a+n*i)*(*(b+nn*i));
    return pro;
}
```

11. Programa Qrdes de las página 96.

```
/*
-- Resolución del problema lineal de mínimos cuadrados ||Ax-b||
  mediante transformaciones ortogonales de householder
  de la matriz A (rango completo)
*/

#include <stdio.h>
```

```
#include <math.h>
#include <stdlib.h>

#define M 3
#define N 2
#define SIGN(a,b) (b>=0 ? fabs(a) : -fabs(a))

main (){

    int i, j, k, l;
    float beta, betas[N+1], d[N+1], dmax, s, sigma, x[N+1],
        sum, wj;
    static float a[M+1][N+1]={1,1,1,1,2,4,1,3,9,1,4,16},
        b[M+1]={2,3,5,6};

    /* *** Reducción QA = R *** */

    for (j=0; j<=N; j++){
        for (i=j, dmax=0; i<=M; i++) dmax=max(dmax,fabs(a[i][j]));
        if (!dmax) {
            printf ("Stop: La matriz A no es de rango completo\n");
            abort();
        }
        for (i=j+1, beta=0; i<=M; i++) beta += pow(a[i][j],2);
        wj = a[j][j];
        sigma = SIGN(sqrt(beta+wj*wj),wj);
        wj += sigma;
        beta = 2/(beta+wj*wj);
        a[j][j] = wj;
        d[j] = -sigma;
        betas[j] = beta;
        for (l=j+1; l<=N; l++){
            for (k=j, s=0; k<=M; k++) s += a[k][j]*a[k][l];
            s *= beta;
            for (k=j; k<=M; k++) a[k][l] -= a[k][j]*s;
        }
        for (k=j, s=0; k<=M; k++) s += a[k][j]*b[k];
        s *= beta;
        for (k=j; k<=M; k++) b[k] -= a[k][j]*s;
    }

    /* *** Resolución Rx = b *** */

    x[N] = b[N]/d[N];
    for (i=N-1; i>=0; i--){
        for (k=i+1, s=0; k<=N; k++) s += a[i][k]*x[k];
        x[i] = (b[i]-s)/d[i];
    }

    /* *** Suma de residuos al cuadrado *** */

    for (i=N+1, sum=0; i<=M; i++) sum += pow(b[i],2);

    /* *** Vector de residuos *** */

    for (i=N; i>=0; i--){
        for (k=i+1, s=0; k<=M; k++) s += a[k][i]*b[k];
        s *= betas[i];
        b[i] = -a[i][i]*s;
        for (k=i+1; k<=M; k++) b[k] -= a[k][i]*s;
    }

    printf ("Solución:");
    for (j=0; j<=N; j++) printf ("%f ",x[j]);
    printf ("\n");
    printf ("Suma de residuos al cuadrado: %f\n",sum);
    printf ("Vector de residuos:");
    for (j=0; j<=N; j++) printf ("%f ", b[j]);
    printf ("\n");

    return 0;
}
```

12. Programa Mincud de las página 102.

```
/*
-- Resolución del problema lineal general de mínimos cuadrados
  ||Ax-b|| mediante transformaciones ortogonales de Householder
  de A teniendo en cuenta su posible rango incompleto
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define M 3
#define Mm1 4
#define N 3
#define Nm1 4
#define TAU 0.000001
```

```

#define SIGN(a,b) (b>0 ? fabs(a) : -fabs(a))
#define DIVD(a) ((1.0+a)>1.0 ? a : DBL_EPSILON)
#define SWAPf(a,b) {double temp=a; a=b; b=temp;}

void h1 (double *beta, int i, int j, int m, double *w, double *x,
        int n);
void h1 (double *beta, int i, int j, int m, double *w, double *x,
        int n);

main (){

    int i, imax, ipvt[Nm1], ira, j, k, k1, kp1, l,
        m=M, n=N, nm1=Nm1, mm1=Mm1;
    double beta, beta1[Mm1], s, sigma, dmax, x[Nm1],
        sum, w[Mm1], w1[Mm1][Nm1], a1[Nm1][Mm1];
    static double a[Mm1][Nm1]={1,1,1,1,1,2,4,4,1,3,9,9,1,4,16,16},
        b[Mm1]={2,3,5,6};

/*
*** Reducción QAP=
| R   R |
| 0   0 | *** */

    ira = min(m,n);
    for (i=0; i<=ira; i++){
        for (j=i, imax=i, dmax=0; j<=n; j++){
            for (k=i, s=0; k<=m; k++) s += pow(a[k][j],2);
            if (s > dmax) {
                dmax = s;
                imax = j;
            }
        }
        ipvt[i] = imax;
        if (imax!=i) for (j=0; j<=m; j++){
            SWAPf(a[j][i],a[j][imax]);
        }
        h1 (&beta,i,i+1,m,w,&a[0][i],nm1);
        for (j=i+1; j<=n; j++) h2 (&beta,i,i+1,m,w,&a[0][j],nm1);
        h2 (&beta,i,i+1,m,w,b,1);
    }

    k = ira;
    for (j=0; j<=ira; j++){
        if (fabs(a[j][j]) < TAU){
            k = j-1;
            break;
        }
    }
    kp1 = k+1;

    for (i=kp1, sum=0; i<=m; i++) sum += pow(b[i],2);

    for (i=0; i<=k; i++) for (j=0; j<=n; j++) a1[j][i] = a[i][j];

    if (k != n) {
        for (i=k; i>=0; i--){
            h1(&beta1[i],i,kp1,n,&w1[i][0],&a1[0][i],mm1);
            for (j=i-1; j>=0; j--){
                h2(&beta1[i],i,kp1,n,&w1[i][0],&a1[0][j],mm1);
            }
        }
    }

    x[k] = b[k]/a1[k][k];
    for (i=k-1; i>=0; i--){
        for (k1=i+1, s=0; k1<=k; k1++) s += a1[k1][i]*x[k1];
        x[i] = (b[i]-s)/a1[i][i];
    }

    if (k != n) {
        for (j=kp1; j<=n; j++) x[j] = 0;
        for (i=0; i<=k; i++) h2(&beta1[i],i,kp1,n,&w1[i][0],x,1);
    }

    for (j=ira; j>=0; j--){
        if (ipvt[j] != j){
            l = ipvt[j];
            SWAPf(x[l],x[j]);
        }
    }

    printf ("Rango de la matriz A: %d\n",k+1);
    printf ("Solución:");
    for (j=0; j<=n; j++) printf ("%7.4f ",x[j]);
    printf ("\n");
    printf ("Suma de residuos al cuadrado: %f\n",sum);

    return 0;
}

void h1 (double *beta, int i, int j, int m, double *w, double *x,
        int n)
{
    int k;
    double sigma;

```

```

    for (k=j, *beta=0; k<=m; k++){
        *(w+k) = *(x+n*k);
        *beta += pow(*(w+k),2);
    }
    *(w+i) = *(x+n*i);
    sigma = SIGN(sqrt(*beta*pow(*(w+i),2)),*(x+n*i));
    *(w+i) += sigma;
    *beta = 2.0/DIVD(*beta*pow(*(w+i),2));
    *(x+n*i) = -sigma;
}

void h2 (double *beta, int i, int j, int m, double *w, double *x,
        int n)
{
    int k;
    double s;

    s = (*(w+i))*(*(x+n*i));
    for (k=j; k<=m; k++) s += (*(w+k))*(*(x+n*k));
    s *= *beta;
    *(x+n*i) -= (*(w+i))*s;
    for (k=j; k<=m; k++) *(x+n*k) -= (*(w+k))*s;
}

```

13. Programa Givens de la página 107.

```

/*
-- Resolución del problema lineal de mínimos cuadrados ||Ax-b||
mediante transformaciones ortogonales de Givens de la
matriz A (rango completo)
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define M 3
#define N 2

main (){

    int i, j, k;
    float c, s, sum, q, t, x[N+1];
    static float a[M+1][N+1]={1,1,1,1,2,4,1,3,9,1,4,16},
        b[M+1]={2,3,5,6};

/* *** Reducción QA = R *** */

    for (i=0; i<=N; i++){
        for (k=i+1; k<=M; k++){
            if (1.0+fabs(a[k][i]) == 1.0) continue;
            if (fabs(a[k][i]) >= fabs(a[i][i])){
                t = a[i][i]/a[k][i];
                s = 1/sqrt(1+t*t);
                c = s*t;
            } else {
                t = a[k][i]/a[i][i];
                c = 1/sqrt(1+t*t);
                s = c*t;
            };
            a[i][i] = c*a[i][i]+s*a[k][i];
            for (j=i+1; j<=N; j++){
                q = c*a[i][j]+s*a[k][j];
                a[k][j] = -s*a[i][j]+c*a[k][j];
                a[i][j] = q;
            }
            q = c*b[i]+s*b[k];
            b[k] = -s*b[i]+c*b[k];
            b[i] = q;
        }
    }

/* *** Resolución Rx = b *** */

    x[N] = b[N]/a[N][N];
    for (i=N-1; i>=0; i--){
        for (k=i+1, sum=0; k<=N; k++) sum += a[i][k]*x[k];
        x[i] = (b[i]-sum)/a[i][i];
    }

/* *** Suma de residuos al cuadrado *** */

    for (i=N+1, sum=0; i<=M; i++) sum += pow(b[i],2);

    printf ("Solución:");
    for (j=0; j<=N; j++) printf ("%f ",x[j]);
    printf ("\n");
    printf ("Suma de residuos al cuadrado: %f\n",sum);

    return 0;
}

```

14. Programa Fastgivens de las página 112.

```

/*
-- Resolución del problema lineal de mínimos cuadrados ||Ax-b||
mediante transformaciones rápidas de Givens de la
matriz A (rango completo)
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define M 3
#define N 2
#define SWAPf(a,b) {float temp=a; a=b; b=temp;}

main (){

    int i, j, k;
    float c, d[M+1], r1, r2, s, sqrd, sum, t, x[N+1];
    static float a[M+1][N+1]={1,1,1,1,2,4,1,3,9,1,4,16},
        b[M+1]={2,3,5,6};

/* *** Reducción QA = R *** */

    for (i=0; i<=M; i++) d[i] = 1;
    for (j=0; j<=N; j++){
        for (i=j+1; i<=M; i++){
            if (1.0+fabs(a[i][j]) == 1.0) continue;
            c = d[j]*pow(a[j][j],2);
            s = d[i]*pow(a[i][j],2);
            if (s <= c){
                r2 = a[i][j]/a[j][j];
                r1 = d[i]*r2/d[j];
                c /= s+c;
                d[j] *= c;
                d[i] *= c;
                for (k=j; k<=N; k++){
                    t = a[j][k]+r1*a[i][k];
                    a[i][k] -= r2*a[j][k];
                    a[j][k] = t;
                }
                t = b[j]+r1*b[i];
                b[i] -= r2*b[j];
                b[j] = t;
            } else {
                r2 = a[j][j]/a[i][j];
                r1 = d[j]*r2/d[i];
                s /= s+c;
                SWAPf(d[j],d[i]);
                d[j] *= s;
                d[i] *= s;
                for (k=j; k<=N; k++){
                    t = a[i][k]+r1*a[j][k];
                    a[i][k] = a[j][k]-r2*a[i][k];
                    a[j][k] = t;
                }
                t = b[i]+r1*b[j];
                b[i] = b[j]-r2*b[i];
                b[j] = t;
            }
        }
    }
    for (i=0; i<=M; i++){
        sqrd = sqrt(d[i]);
        for (j=0; j<=N; j++) a[i][j] *= sqrd;
        b[i] *= sqrd;
    }

/* *** Resolución Rx = b *** */

    x[N] = b[N]/a[N][N];
    for (i=N-1; i>=0; i--){
        for (k=i+1, sum=0; k<=N; k++) sum += a[i][k]*x[k];
        x[i] = (b[i]-sum)/a[i][i];
    }

/* *** Suma de residuos al cuadrado *** */

    for (i=N+1, sum=0; i<=M; i++) sum += pow(b[i],2);

    printf ("Solución:");
    for (j=0; j<=N; j++) printf ("%f ",x[j]);
    printf ("\n");
    printf ("Suma de residuos al cuadrado: %f\n",sum);

    return 0;
}

```

15. Programa Svdre y rutina dcmsvd de la página 128.

```

/*
-- Resolución del problema lineal general de mínimos cuadrados
||Ax-b|| mediante descomposición en valores singulares
de la matriz A
*/

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define N 2
#define Nm1 3
#define M 4
#define Mm1 5
#define SIGN(a,b) (b>0 ? fabs(a) : -fabs(a))

void dcmsvd( double a[][Nm1], int m, int n, double sv[Nm1],
    double v[][Nm1]);

main (){

    int i, j, k, m=M, n=N, ns;
    double s, sp, sv[Nm1], v[Mm1][Nm1], x[Nm1], tmp[Mm1],
        sm;
    static double a[Mm1][Nm1]={1,6,11,2,7,12,3,8,13,4,9,14,5,10,
        15}, b[Mm1]={5,5,5,5,5};

/* *** Descomposición en rutina dcmsvd *** */

    dcmsvd (a,m,n,sv,v);

    for (i=0, sm=0; i<=n; i++) sm=max(sm,sv[i]);
    sp = sm*1.0e-6;
    for (j=0, ns=0; j<=n; j++){
        s = 0;
        if (sv[j] > sp){
            ns++;
            for (i=0; i<=m; i++) s += a[i][j]*b[i];
            s /= sv[j];
        } else sv[j] = 0;
        tmp[j] = s;
    }
    for (j=0; j<=n; j++){
        for (k=0, s=0; k<=n; k++) s += v[j][k]*tmp[k];
        x[j] = s;
    }

    printf ("Rango de la matriz A: %d\n",ns);
    printf ("Solución: ");
    for (j=0; j<=N; j++) printf ("%9.4f", x[j]);
    printf ("\n");
    printf ("Valores singulares de A: ");
    for (j=0; j<=ns; j++) printf ("%9.4f", sv[j]);
    printf ("\n");

    return 0;
}

void dcmsvd( double a[][Nm1], int m, int n, double sv[Nm1],
    double v[][Nm1] )
{
    int i, imax, ifl, its, j, jj, jmax, k, l, nm;
    double aux, anorm, c, f, g, h, rmax, s,
        rv1[20], x, y, z;

    g = anorm = 0;
    for (i=0; i<=n; i++){
        l = i+1;
        rv1[i] = g;
        g = s = 0;
        if (i <= m){
            for (k=i, rmax=0; k<=m; k++){
                rmax = max(rmax,fabs(a[k][i]));
            }
            if (rmax){
                for (k=i; k<=m; k++) s += pow(a[k][i],2);
                f = a[i][i];
                g = -SIGN(sqrt(s),f);
                h = f*g-s;
                a[i][i] = f-g;
                for (j=l; j<=n; j++){
                    for (k=i, s=0; k<=m; k++) s += a[k][i]*a[k][j];
                    f = s/h;
                    for (k=i; k<=m; k++) a[k][j] += f*a[k][i];
                }
            }
        }
        sv[i] = g;
        g = s = 0;
        if (i <= m && i != n){

```

```

    for (k=1; rmax=0; k<=n; k++){
        rmax = max(rmax, fabs(a[i][k]));
    }
    if (rmax){
        for (k=1; k<=n; k++) s += pow(a[i][k], 2);
        f = a[i][1];
        g = -SIGN(sqrt(s), f);
        h = f*g-s;
        a[i][1] = f-g;
        for (k=1; k<=n; k++) rv1[k] = a[i][k]/h;
        for (j=1; j<=m; j++){
            for (k=1; s=0; k<=n; k++) s += a[j][k]*a[i][k];
            for (k=1; k<=n; k++) a[j][k] += s*rv1[k];
        }
    }
    anorm = max(anorm, fabs(sv[i])+fabs(rv1[i]));
}

for (i=n; i>=0; i--){
    if (i < n){
        if (g){
            for (j=1; j<=n; j++) v[j][i] = (a[i][j]/a[i][1])/g;
            for (j=1; j<=n; j++){
                for (k=1, s=0; k<=n; k++) s += a[i][k]*v[k][j];
                for (k=1; k<=n; k++) v[k][j] += s*v[k][i];
            }
        }
        for (j=1; j<=n; j++) v[i][j] = v[j][i] = 0;
    }
    v[i][i] = 1;
    g = rv1[i];
    l = i;
}

for (i=n; i>=0; i--){
    l = i+1;
    g = sv[i];
    for (j=1; j<=n; j++) a[i][j] = 0;
    if (g){
        g = 1/g;
        for (j=1; j<=n; j++){
            for (k=1, s=0; k<=m; k++) s += a[k][i]*a[k][j];
            f = (s/a[i][i])*g;
            for (k=i; k<=m; k++) a[k][j] += f*a[k][i];
        }
        for (j=i; j<=m; j++) a[j][i] *= g;
    } else for (j=i; j<=m; j++) a[j][i] = 0;
    ++a[i][i];
}

for (k=n; k>=0; k--){
    for (its=0; its<=30; its++){
        ifl = 1;
        for (l=k; l>=0; l--){
            nm = l-1;
            if (fabs(rv1[l])+anorm == anorm){
                ifl = 0;
                break;
            }
            if (fabs(sv[nm])+anorm == anorm) break;
        }
        if (ifl) {
            c = 0;
            s = 1;
            for (i=l; i<=k; i++){
                f = s*rv1[i];
                if (fabs(f)+anorm != anorm){
                    g = sv[i];
                    h = sqrt(f*f+g*g);
                    sv[i] = h;
                    c = g/h;
                    s = (-f/h);
                    for (j=0; j<=m; j++){
                        y = a[j][nm];
                        z = a[j][i];
                        a[j][nm] = y*c+z*s;
                        a[j][i] = z*c-y*s;
                    }
                }
            }
        }
        z = sv[k];
        if (l == k){
            if (z < 0){
                sv[k] = -z;
                for (j=0; j<=n; j++) v[j][k] = (-v[j][k]);
            }
            break;
        }
        if (its == 30){
            printf ("No hay convergencia\n");
            abort();
        }
    }
}

```

```

x = sv[l];
nm = k-1;
y = sv[nm];
g = rv1[nm];
h = rv1[k];
f = ((y-z)*(y+z)+(g-h)*(g+h))/(2*h*y);
g = sqrt(f*f+1);
f = ((x-z)*(x+z)+h*((y/(f+SIGN(g,f)))-h))/x;

c = s = 1;
for (j=1; j<=nm; j++){
    i = j+1;
    g = rv1[i];
    y = sv[i];
    h = s*g;
    g *= c;
    z = sqrt(f*f+h*h);
    rv1[j] = z;
    c = f/z;
    s = h/z;
    f = x*c+g*s;
    g = g*c-x*s;
    h = y*s;
    y *= c;
    for (jj=0; jj<=n; jj++){
        x = v[jj][j];
        z = v[jj][i];
        v[jj][j] = x*c+z*s;
        v[jj][i] = z*c-x*s;
    }
    z = sqrt(f*f+h*h);
    sv[j] = z;
    if (z){
        c = f/z;
        s = h/z;
    }
    f = c*g+s*y;
    x = c*y-s*g;
    for (jj=0; jj<=m; jj++){
        y = a[jj][j];
        z = a[jj][i];
        a[jj][j] = y*c+z*s;
        a[jj][i] = z*c-y*s;
    }
    rv1[l] = 0;
    rv1[k] = f;
    sv[k] = x;
}
}
}
}

```

16. Programa Mci de la página 134.

```

/*
-- Resolución del problema lineal general de mínimos cuadrados
||Ax-b|| sujeto a las condiciones Cx=d
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define M1 3
#define M1m1 4
#define M2 1
#define M2m1 2
#define N 2
#define Nm1 3
#define TAU 0.000001
#define SIGN(a,b) ((b)>0 ? fabs(a) : -fabs(a))
#define SWAPf(a,b) {float temp=a; a=b; b=temp;}

void h1 ( double *beta, int i, int j, int m, double *w,
double *x, int n );
void h2 ( double *beta, int i, int j, int m, double *w,
double *x, int n );

main (){
    int i, imax, ipvt[Nm1], ira, j, k, k1, l,
    m1=M1, m2=M2, n=N, nm1=Nm1, n1;
    double beta, s, sigma, dmax, x[Nm1],
    sum, w[M1m1];
    static double a[M1m1][Nm1]={0.2113,0.4524,0.6538,0.0824,0.8075,
    0.4899,0.7599,0.4832,0.7741,0.0087,0.6135,0.9626},
    b[M1m1]={3.0775,3.1671,4.0485,4.1237},
    c[M2m1][Nm1]={0.8096,0.2749,0.9933,0.8474,0.8807,
    0.8360},
    d[M2m1]={4.3393,5.1169};
}

```

```

/*
*** Reducción QCP=| 11 12| ***
|0 0 | */

ira = min(m2,n);
for (i=0; i<=ira; i++){
  for (j=i, dmax=0, imax=i; j<=n; j++){
    for (k=i, s=0; k<=m2; k++) s += pow(c[k][j],2);
    if (s > dmax) {
      dmax = s;
      imax = j;
    }
  }
  ipvt[i] = imax;

  /* Intercambiar columnas de C y A si ha lugar */

  if (imax != i){
    for (j=0; j<=m2; j++) SWAPf(c[j][i],c[j][imax]);
    for (j=0; j<=m1; j++) SWAPf(a[j][i],a[j][imax]);
  }

  /* Aplicar transformación de Householder a columnas i a n
    de C y a vector d */

  h1(&beta,i,i+1,m2,&w,&c[0][i],nm1);
  for (j=i+1; j<=n; j++) h2(&beta,i,i+1,m2,&w,&c[0][j],nm1);
  h2(&beta,i,i+1,m2,&w,&d,1);
}

k1 = ira;
for (j=0; j<=ira; j++){
  if (fabs(c[j][j]) <= TAU){ /* Se determina rango de C */
    k1 = j-1;
    break;
  }
}

/*
Determinar A y B
2 */

for (i=0; i<=m1; i++){
  a[i][0] /= c[0][0];
  for (j=1; j<=ira; j++){
    for (k=0, s=0; k<=j-1; k++) s += a[i][k]*c[k][j];
    a[i][j] = (a[i][j]-s)/c[j][j];
  }
  for (j=ira+1; j<=n; j++){
    for (k=0, s=0; k<=ira; k++) s += a[i][k]*c[k][j];
    a[i][j] -= s;
  }
  for (k=0, s=0; k<=ira; k++) s += a[i][k]*d[k];
  b[i] -= s;
}

/* Aplicar transformación de Householder a columnas ira a n
de A para así obtener R
2 22 */

for (i=ira+1; i<=n; i++){
  k = i-ira-1;
  h1(&beta,k,k+1,m1,&w,&a[0][i],nm1);
  for (j=i+1; j<=n; j++) h2(&beta,k,k+1,m1,&w,&a[0][j],nm1);
  h2(&beta,k,k+1,m1,&w,&b,1);
}

/* Resolver el sistema

|R R || | |d|
| 11 12||x| = | 1|
|0 R || | |c|
| 22|| | |1| */

n1 = n-ira-1;
x[n] = b[n1]/a[n1][n];
for (i=n1-1; i>=1; i--){
  for (j=i+1, s=0; j<=n1; j++) s += a[i][j]*x[j];
  x[i+ira] = (b[i-1]-s)/a[i][i+ira];
}
for (i=ira; i>=0; i--){
  for (j=i+1, s=0; j<=n; j++) s += c[i][j]*x[j];
  x[i] = (d[i]-s)/c[i][i];
}

for (j=ira; j>=0; j--){
  if (ipvt[j] != j){
    l = ipvt[j];
    SWAPf(x[l],x[j]);
  }
}

/* Deshacer permutación */

```

```

printf ("Rango de la matriz C: %d\n",k1);
printf ("Solución:");
for (j=0; j<=n; j++) printf ("%7.4f ",x[j]);
printf ("\n");

return 0;

}

void h1 (double *beta, int i, int j, int m, double *w, double *x,
int n)
{
  int k;
  double sigma;
  *beta = 0;
  for (k=j; k<=m; k++){
    *(w+k) = *(x+n*k);
    *beta += pow(*(w+k),2);
  }
  *(w+i) = *(x+n*i);
  sigma = SIGN(sqrt(*beta+pow(*(w+i),2)),*(x+n*i));
  *(w+i) += sigma;
  *beta = 2/(*beta+pow(*(w+i),2));
  *(x+n*i) = -sigma;
}

void h2 ( double *beta, int i, int j, int m, double *w,
double *x, int n )
{
  int k;
  double s;
  s = (*(w+i))*(*(x+n*i));
  for (k=j; k<=m; k++) s += (*(w+k))*(*(x+n*k));
  s **= beta;
  *(x+n*i) -= (*(w+i))*s;
  for (k=j; k<=m; k++) *(x+n*k) -= (*(w+k))*s;
}

```

G.1.2 Códigos del capítulo 2

1. Programa Jacobi de la página 147.

```

/*
-- Resolución de un sistema lineal de ecuaciones Ax = b
mediante el método iterativo de Jacobi
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define N 3

main (){

  int i, j;
  float s1, sm, su, x[N+1], y[N+1];
  static float a[N+1][N+1]={10,-1,2,0,-1,11,-1,3,2,-1,10,-1,
0,3,-1,8},
b[N+1]={6,25,-11,15};

  /* *** Proceso iterativo *** */

  do {
    for (i=0, s1=0; i<=N; i++){
      for (j=0, su=b[i]; j<=i-1; j++) su -= a[i][j]*x[j];
      for (j=i+1; j<=N; j++) su -= a[i][j]*x[j];
      y[i] = su/a[i][i];
      s1 = max(s1,fabs(y[i]));
    }
    for (i=0, sm=0; i<=N; i++){
      if (fabs(x[i]-y[i])/s1 > sm) sm = fabs(x[i]-y[i])/s1;
      x[i] = y[i];
    }
    for (i=0; i<=N; i++) printf ("%9.5f ",x[i]);
    printf ("\n");
  } while (sm >= 0.001);

  return 0;
}

```

2. Programa GaussSeidel de la página 151.

```

/*
-- Resolución de un sistema lineal de ecuaciones Ax = b
mediante el método iterativo de Gauss-Seidel
*/

```

```

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define N 3

main (){

    int i, j;
    float s1, sm, su, xi;
    static float a[N+1][N+1]={10,-1,2,0,-1,11,-1,3,2,-1,10,-1,
        0,3,-1,8},
        b[N+1]={6,25,-11,15}, x[N+1]={0,0,0,0};

    /* *** Proceso iterativo *** */

    do {
        for (i=0, s1=0, sm=0; i<=N; i++){
            for (j=0, su=b[i]; j<=N; j++) su -= a[i][j]*x[j];
            xi = x[i]+su/a[i][i];
            if (fabs(x[i]-xi) > sm) sm = fabs(x[i]-xi);
            x[i] = xi;
            s1 = max(s1,fabs(x[i]));
        }
        sm /= s1;
        for (i=0; i<=N; i++) printf ("%9.5f ",x[i]);
        printf ("\n");
    } while (sm >= 0.001);

    return 0;
}

```

3. Programa Sor de la página 165.

```

/*
-- Resolución de un sistema lineal de ecuaciones Ax = b mediante
    el método iterativo de relajaciones sucesivas, S.O.R
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#define N 2
#define W 1.25

main (){

    int i, j;
    float s1, sm, su, xi;
    static float a[N+1][N+1]={4,3,0,3,4,-1,0,-1,4},
        b[N+1]={24,30,-24}, x[N+1]={1,1,1};

    /* *** Proceso iterativo *** */

    do {
        for (i=0, s1=0, sm=0; i<=N; i++){
            for (j=0, su=b[i]; j<=i-1; j++) su -= a[i][j]*x[j];
            for (j=i+1; j<=N; j++) su -= a[i][j]*x[j];
            xi = (1-W)*x[i]+W*su/a[i][i];
            if (fabs(x[i]-xi) > sm) sm = fabs(x[i]-xi);
            x[i] = xi;
            s1 = max(s1,fabs(x[i]));
        }
        sm /= s1;
        for (i=0; i<=N; i++) printf ("%9.5f ",x[i]);
        printf ("\n");
    } while (sm >= 0.001);

    return 0;
}

```

4. Programa Cg de la página 188.

```

/*
-- Resolución de un sistema lineal de ecuaciones Ax = b
    mediante el método de los gradientes conjugados
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <float.h>

#define N 5

float xnorm (float *x, int n);

```

```

main (){

    int i, j, k=0, n=N;
    float alfak, betak, p[N+1], r[N+1], ro0, ro1, su, xn, w[N+1];
    static float a[N+1][N+1]={4,-1,0,-1,0,0,-1,4,-1,0,-1,0,0,-1,
        4,0,0,-1,-1,0,0,4,-1,0,0,-1,0,-1,4,-1,0,0,-1,0,
        -1,4}, b[N+1]={0,5,0,6,-2,6}, x[N+1]={0,0,0,0,0,0};

    /* *** Proceso iterativo *** */

    for (i=0; i<=N; i++) r[i] = b[i];
    ro0 = ro1 = xnorm(&r,n);
    xn = FLT_EPSILON*100*sqrt(xnorm(&b,n));

    do {
        if (!k) for (i=0; i<=n; i++) p[i] = r[i];
        else { betak = ro1/ro0;
            for (i=0; i<=n; i++) p[i] = r[i] + betak*p[i];
        }
        for (i=0; i<=n; i++){
            for (j=0, su=0; j<=n; j++) su += a[i][j]*p[j];
            w[i] = su;
        }
        for (i=0, su=0; i<=n; i++) su += p[i]*w[i];
        alfak = ro1/su;
        for (i=0; i<=n; i++) x[i] += alfak*p[i];
        for (i=0; i<=n; i++) r[i] -= alfak*w[i];
        ro0 = ro1;
        ro1 = xnorm(&r,n);
        k = k+1;
        printf ("%d:",k);
        for (i=0; i<=N; i++) printf ("%9.5f ",x[i]);
        printf ("\n");
    } while (sqrt(ro1) >= xn);

    return 0;
}

float xnorm (float *x, int n)
{
    int i;
    float aux;
    for (i=0, aux=0; i<=n; i++) aux += pow(*(x+i),2);
    return aux;
}

```

5. Programa Cgp de la página 192.

```

/*
-- Resolución de un sistema lineal de ecuaciones Ax = b
    mediante el método de los gradientes conjugados con
    preconditionamiento
*/

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <float.h>

double prod (double *x, double *y, int n);
double **matriz(int nf, int nc);
double *vector(int n);

main (){

    int i, j, k=0, n;
    double alfak, **a, *b, betak, *M, *p, *r, ro1, *x, xn, *w, *z,
        *zm2, *rm2, **aux, *baux;

    printf ("Dimensión de la Matriz A?\n");
    scanf ("%d", &n);
    a = matriz(n,n);
    aux = matriz(n,n);
    b = vector(n);
    baux = vector(n);
    M = vector(n);
    p = vector(n);
    r = vector(n);
    rm2 = vector(n);
    w = vector(n);
    x = vector(n);
    z = vector(n);
    zm2 = vector(n);
    n -= 1;

    /* *** Generación aleatoria de los datos del problema *** */

    for (i=0; i<=n; i++){
        for (j=0; j<=n; j++) aux[i][j] = (double)(rand())/RAND_MAX;
        baux[i] = i;
    }
}

```



```

    }
    for (i=0; i<=n; i++) for (j=0; j<=n; j++)
        a[i][j] = prod(&aux[i][0], &aux[j][0], n);
    for (i=0; i<=n; i++) b[i] = prod(&a[i][0], baux, n);

/* *** Obtención del preconditionador *** */

    for (i=0; i<=n; i++) M[i] = sqrt(prod(&a[i][0], &a[i][0], n));

/* *** Proceso iterativo *** */

    for (i=0; i<=n; i++) r[i] = b[i];
    ro1 = prod(b, b, n);
    xn = DBL_EPSILON*1000*sqrt(ro1);

    do {
        for (i=0; i<=n; i++) z[i] = r[i]/M[i];
        if (!k) for (i=0; i<=n; i++) p[i] = z[i];
        else { betak = prod(z, r, n)/prod(zm2, rm2, n);
              for (i=0; i<=n; i++) p[i] = z[i] + betak*p[i];
            }
        for (i=0; i<=n; i++) w[i] = prod(&a[i][0], p, n);
        alfak = prod(z, r, n)/prod(p, w, n);
        for (i=0; i<=n; i++) x[i] += alfak*p[i];
        for (i=0; i<=n; i++) rm2[i] = r[i];
        for (i=0; i<=n; i++) r[i] -= alfak*w[i];
        for (i=0; i<=n; i++) zm2[i] = z[i];
        ro1 = prod(r, r, n);
        k = k+1;
        printf ("%d:", k);
        for (i=0; i<=n; i++) printf ("%f ", x[i]);
        printf ("\n");
        printf ("%f:", ro1);
        printf ("\n");
    } while (sqrt(ro1) >= xn);

    return 0;
}

double prod (double *x, double *y, int n)

```

```

{
    int i;
    double aux;
    for (i=0, aux=0; i<=n; i++) aux += (*(x+i))*(*(y+i));
    return aux;
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc, sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

double *vector(int n)
{
    double *v;

    v = (double *) calloc(n, sizeof(double));
    if (!v){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return v;
}

```

G.1.3 Códigos del capítulo 3

Una *traducción* literal de los trozos de programas escritos en FORTRAN de este capítulo a C no tiene mucho sentido puesto que la forma de almacenamiento de matrices dispersas y las operaciones con ellas realizadas tienen mucho que ver con el lenguaje que se elige y la máquina donde se implementan estas operaciones. Las presentadas se han realizado con vistas a su implementación en FORTRAN 77; en C, probablemente, deberían ser muy distintas. No obstante, para compactar todo lo expuesto en FORTRAN 77 en su versión en C, a continuación se listan trozos de programas en C más o menos similares a los presentados en FORTRAN 77.

1. Si se define una matriz dispersa como una estructura de la forma

```

typedef struct {
    int ifi[ELE], ico[ELE];
    float val[ELE];
} MATdis;

```

donde ELE es el número de elementos distintos de cero que hay en esa matriz, el programa de la página 203, relativo a la recuperación en `VEC(·)` del vector fila i de una matriz guardada de acuerdo con el esquema de almacenamiento por coordenadas, es el que sigue.

```

void recpr (float *v, MATdis *pa, int i)
{
    int ii=0;
    while (pa->ifi[ii]!=i) ii++;
    for(; pa->ifi[ii]==i; ii++) v[pa->ico[ii]] = pa->val[ii];
}

```

2. Si se define ahora la matriz dispersa como una estructura de la forma

```
typedef struct {
    int ia[N], ico[ELE];
    float val[ELE];
} MATdis;
```

donde ELE es el número de elementos distintos de cero que hay en esa matriz y N el número de filas, el programa de la página 204, relativo a la recuperación en `VEC(·)` del vector fila i de una matriz guardada de acuerdo con el esquema de almacenamiento por filas, es el que sigue.

```
void recpr (float *v, MATdis *pa, int i)
{
    int ii;
    for (ii=pa->ia[i]; ii<pa->ia[i+1]; ii++)
        v[pa->ico[ii]] = pa->val[ii];
}
```

3. El programa de la página 204 para recuperar la columna k de esa misma matriz es el que sigue.

```
void recpr (float *v, MATdis *pa, int k)
{
    int ii, j;
    for (j=0; j<=M; j++){
        for (ii=pa->ia[j]; ii<pa->ia[j+1]; ii++) {
            if (pa->ico[ii]>k) break;
            else if (pa->ico[ii]==k) v[j] = pa->val[ii];
        }
    }
}
```

4. Si se define ahora la matriz dispersa como una estructura de la forma

```
typedef struct {
    int ia[N], ifa[N];
    float val[ELE];
} MATdis;
```

donde ELE es el número de elementos distintos de cero que hay en esa matriz y N el número de filas, el programa de la página 206, relativo a la recuperación en `VEC(·)` del vector fila i de una matriz guardada de acuerdo con el esquema de almacenamiento por filas, es el que sigue.

```
void recpr (float *v, MATdis *pa, int i)
{
    int ii, j;

    for (ii=pa->ia[i], j=0; ii<pa->ia[i+1]; ii++, j++)
        v[pa->ifa[i]+j] = pa->val[ii];
}
```

5. Si se define ahora la matriz dispersa como una estructura de la forma

```
typedef struct {
    int ifi[N], ico[ELE], link[ELE];
    float val[ELE];
} MATdis;
```

donde ELE es el número de elementos distintos de cero que hay en esa matriz y N el número de filas, el programa de la página 207, relativo a la recuperación en `VEC(·)` del vector fila *i* de una matriz guardada de acuerdo con el esquema de almacenamiento por listas encadenadas, es el que sigue.

```
void recpr (float *v, MATdis *pa, int i)
{
    int ii;
    ii = pa->ifl[i];
    do {
        v[pa->ico[ii]] = ps->val[ii];
        ii = pa->link[ii];
    } while (ii);
}
```

6. En lo que sigue supondremos la matriz dispersa almacenada según el esquema de filas/columnas. Es decir, definido por la estructura anterior

```
typedef struct {
    int i[N], ico[ELE];
    float val[ELE];
} MATdis;
```

donde ELE es el número de elementos distintos de cero que hay en esa matriz y N su número de filas.

El programa de la página 209 para efectuar el producto interior de dos vectores —en este caso el vector IA no es necesario— es el que sigue.

```
float proin (float h, MATdis *pa, MATdis *pb, int n)
{
    int i, j;
    float g;
    for (i=0, h=0; i<=n, i++){
        for (j=0; j<=n; j++) {
            if (pb->ico[j]>pa->ico[i]) break;
            else if (pb->ico[j]==pa->ico[i])
                g+=pa->val[pa->ico[i]]*pb->val[pb->ico[j]];
        }
    }
    return g;
}
```

7. El programa de la página 209 para efectuar el producto interior de dos vectores —en este caso el vector I tampoco es necesario— es el que sigue.

```
float proin (float h, MATdis *pa, MATdis *pb, int n)
{
    int i, *ip;
    float g;
    ip = (int *) calloc(n*sizeof(int));
    for (i=0; i<=n, i++) ip[pa->ico[i]] = i;
    for (i=0, h=0; i<=n, i++) if (ip[pb->ico[i]])
        g+=pa->val[ip[pb->ico[i]]]*pb->val[i];
    return g;
}
```

8. El programa de la página 210 para efectuar el producto de una matriz por un vector es el que se lista a continuación.

```
void axb (MATdis *pa, float *b, float *c, int m)
{
    int i, j;
    float s;
    for (i=0, i<=m, i++){
        for (j=pa->i[i], s=0; j<pa->i[i+1]; j++){
            s += pa->val[j]*b[pa->ico[j]];
            c[i] = s;
        }
    }
}
```

9. El programa de la página 211 para efectuar el producto de un vector por una matriz es el que sigue.

```
void bxa (MATdis *pa, float *b, float *c, int n)
{
    int i, j;
    float bi;
    for (i=0, i<=n, i++){
        bi = b[i];
        for (j=pa->i[i], s=0; j<pa->i[i+1]; j++){
            c[pa->ico[j]] += pa->val[j]*bi;
        }
    }
}
```

10. El equivalente en C al programa de la página 214 para efectuar la suma simbólica de dos matrices dispersas es el siguiente.

```
void bmas (MATdis *pa, MATdis *pb, MATdis *pc, int n)
{
    int i, *ip, j, nu=1;
    ip = (int *) calloc(n*sizeof(int));
    for (i=0, i<=n, i++){
        pc->i[i] = nu;
        for (j=pa->i[i]; j<pa->i[i+1]; j++, nu++){
            pc->ico[nu] = pa->ico[j];
            ip[pa->ico[j]] = i;
        }
        for (j=pb->i[i]; j<pb->i[i+1]; j++){
            if (ip[pb->ico[j]] != i){
                pc->ico[nu] = pb->ico[j];
                nu++;
            }
        }
        pc->i[n+1] = nu;
    }
}
```

11. Un programa en C similar al de la página 215 para efectuar la suma numérica de dos matrices dispersas es el siguiente.

```
void bmas (MATdis *pa, MATdis *pb, MATdis *pc, int n)
{
    int i, j;
    float *x;
    x = (float *)calloc(n*sizeof(float));
```

```
    for (i=0, i<=n, i++){
        for (j=pa->i[i];j<pa->i[i+1];j++) x[pa->ico[j]] =pa->val[j];
        for (j=pb->i[i];j<pb->i[i+1];j++) x[pb->ico[j]]+=pb->val[j];
        for (j=pc->i[i];j<pc->i[i+1];j++) pc->val[j]=x[pc->ico[j]];
    }
}
```

12. El programa de la página 216 para efectuar el producto simbólico de dos matrices dispersas en C podría ser como sigue.

```
void bmas (MATdis *pa, MATdis *pb, MATdis *pc, int n)
{
    int i, *ip, j, k, kk, nu=1;
    ip = (int *) calloc(n*sizeof(int));
    for (i=0, i<=n, i++){
        pc->i[i] = nu;
        for (j=pa->i[i];j<pa->i[i+1];j++){
            for (k=pb->i[pa->ico[j]];k<pb->i[pa->ico[j]+1];k++){
                kk = pb->ico[k];
                if (ip[kk] != i){
                    pc->ico[nu] = kk;
                    ip[kk]      = i;
                    nu++;
                }
            }
        }
        pc->i[n+1] = nu;
    }
}
```

13. El programa de la página 216 para efectuar ese producto en forma numérica, esta vez en C, sería como el que sigue.

```
void bmas (MATdis *pa, MATdis *pb, MATdis *pc, int n)
{
    int i, j, k;
    float *x;
    x = (float *)calloc(n*sizeof(float));
    for (i=0, i<=n, i++){
        for (j=pa->i[i];j<pa->i[i+1];j++)
            for (k=pb->i[pa->ico[j]];k<pb->i[pa->ico[j]+1];k++)
                x[pb->ico[k]] += pa->val[j]*pb->val[k];
        for (j=pc->i[i];j<pc->i[i+1];j++) pc->val[j]=x[pc->ico[j]];
    }
}
```

14. El programa de la página 218 para trasponer una matriz dispersa es el siguiente.

```
void atra (MATdis *pa, MATdis *pat, int m, int n)
{
    int i, j, k, l;
    for (i=0; i<pa->i[m+1]; i++) if (pa->ico[i]+2<=n+1)
                                pat->i[pa->ico[i]+2]++;

    pat->i[0] = 1;
    pat->i[1] = 1;
    for (i=2; i<n+1; i++) pat->i[i] += pat->i[i-1];
    for (i=0; i<=m; i++){
        for (j=pa->i[i]; j<pa->i[i+1]; j++){
            k      = pa->ico[j]+1;
            l      = pat->ico[k];
            pat->val[l] += pa->val[j];
        }
    }
}
```

```

        l = pat->i[k];
        pat->ico[l] = i;
        pat->val[l] = pa->val[l];
        pat->i[k] = l+1;
    }
}
}

```

G.1.4 Códigos del capítulo 4

1. Programa Bisec de la página 285.

```

/*
Resolución de  $x\sin(x)-1=0$  mediante el método de la
bisección
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

float fx(float x);

main (){

    float a=1.0, b=2.0, fa, fb, c, fc;

    fa = fx(a);
    fb = fx(b);
    if (fa*fb>0) printf("El intervalo [a,b] no contiene solución\n");

/* *** Proceso iterativo *** */

    while (fabs(a-b) > FLT_EPSILON*10){
        c = (a+b)/2;
        fc = fx(c);
        if (fc==0) {
            a = c;
            b = c;
        } else if (fb*fc>0) {
            b = c;
            fb = fc;
        } else {
            a = c;
            fa = fc;
        }
        printf ("%9.7f,%9.7f\n",a,b);
    }

    return 0;
}

float fx(float x)
{
    float f;
    f = x*sin(x)-1;
    return f;
}

```

2. Programa Newt de la página 290.

```

/*
Resolución de  $X - \text{SEN}(X) = 0$  mediante el método de Newton
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

main (){

    float x=1, x0;

/* *** Proceso iterativo *** */

    do {
        x0 = x;
        x = x0-(x0*x0*x0-sin(x0))/(3*x0*x0-cos(x0));
        printf ("%10.6f\n",x);
    } while (fabs(x-x0) >= FLT_EPSILON);

    return 0;
}

```

3. Programa Newton de la página 298.

```

/*
Resolución de  $x^2-1=0$  mediante el método de Newton
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

double fx(double x);
double derfx(double x);

main (){

    double x=2, x0=0;

/* *** Proceso iterativo *** */

    while (fabs(x-x0) > FLT_EPSILON){
        x0 = x;
        x = x0-fx(x0)/derfx(x0);
        printf ("%19.16f\n",x);
    }

    return 0;
}

double fx(double x)
{
    double f;
    f = x*x-1;
    return f;
}

double derfx(double x)
{
    double f;
    f = 2*x;
    return f;
}

```

4. Programa Newtondf de la página 298.

```

/*
Resolución de  $x^2-1=0$  mediante el método de Newton
(Derivadas por diferencias finitas)
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

#define h sqrt(DBL_EPSILON)

double fx(double x);
double derfx(double x);

main (){

    double x=2, x0=0;

/* *** Proceso iterativo *** */

    while (fabs(x-x0) > FLT_EPSILON){
        x0 = x;
        x = x0-fx(x0)/derfx(x0);
        printf ("%19.16f\n",x);
    }

    return 0;
}

double fx(double x)
{
    double f;

```

```

    f = x*x-1;
    return f;
}

double derfx(double x)
{
    double f;
    f = (fx(x+h)-fx(x))/h;
    return f;
}

```

5. Programa Newtonsecante de la página 301.

```

/*          3
-- Resolución de x - SEN(X)=0 mediante el método de secantes
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

double fx(double x);
double secfx(double x, double y);

main (){

    double x2, x1=1, x0=1.1;

/* *** Proceso iterativo *** */

    x2 = x1-fx(x1)/secfx(x0,x1);
    while (fabs(x2-x1) > FLT_EPSILON){
        x0 = x1;
        x1 = x2;
        x2 = x1-fx(x1)/secfx(x0,x1);
        printf ("%19.16f\n",x1);
    }

    return 0;
}

double fx(double x)
{
    double f;
    f = x*x*x-sin(x);
    return f;
}

double secfx(double x, double y)
{
    double f;
    f = (fx(y)-fx(x))/(y-x);
    return f;
}

```

6. Programa Muller de la página 305.

```

/*          3
Resolución de x - sen(x)=0 mediante el método de Muller
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#define SIGN(a,b) (b>=0 ? fabs(a) : -fabs(a))

double fx(double x);

main (){

    double x0=1.5, x1=1.2, x2=1.0, fx0, fx1, fx2,
           c, d0, d1, det, b, a, di, z, x3, u;

    fx0 = fx(x0);
    fx1 = fx(x1);
    fx2 = fx(x2);

/* *** Proceso iterativo *** */

    while (fabs(fx2) > DBL_EPSILON){
        c = fx2;
        d0 = x0-x2;
        d1 = x1-x2;
        det = d0*d1*(x0-x1);
        b = (d0*d0*(fx1-fx2)-d1*d1*(fx0-fx2))/det;
        a = (d1*(fx0-fx2)-d0*(fx1-fx2))/det;
        di = 0.0;
        if (b*b-4*a*c >= 0) di = sqrt(b*b-4*a*c);

```

```

        z = (-2)*c/(b+SIGN(1.0,b)*di);
        x3 = x2+z;
        if (fabs(x3-x1)<fabs(x3-x0)) { /* Escoger nuevos */
            u = x1; /* x0, x1 y x2 los */
            x1 = x0; /* más próximos a */
            x0 = u; /* x3. */
            u = fx1;
            fx1 = fx0;
            fx0 = u;
        }
        if (fabs(x3-x2)<fabs(x3-x1)) {
            u = x2;
            x1 = u;
            u = fx2;
            fx1 = u;
        }
        x2 = x3;
        fx2 = fx(x2);
        printf ("%9.7f\n",x1);
    }

    return 0;
}

double fx(double x)
{
    double f;
    f = x*x*x-sin(x);
    return f;
}

```

7. Programa Newtonmod de la página 299.

```

/*          3
-- Resolución de x - SEN(X)=0 mediante el método de Newton
modificado
*/

#include <stdio.h>
#include <math.h>
#include <float.h>

double fx(double x);

main (){

    double x1=1, x2, dx;

/* *** Proceso iterativo *** */

    dx = 3*x1*x1-cos(x1);
    x2 = x1-fx(x1)/dx;
    while (fabs(x2-x1) > DBL_EPSILON){
        printf ("%19.16f\n",x1);
        x1 = x2;
        x2 = x1-fx(x1)/dx;
    }

    return 0;
}

double fx(double x)
{
    double f;
    f = x*x*x-sin(x);
    return f;
}

```

8. Programa Newtrp de la página 307.

```

/*
-- Resolución de un sistema de ecuaciones no lineales
cualquiera mediante el método de Newton
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define N 3
#define TOL sqrt(DBL_EPSILON)
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
double dmax(double *x, int n);
void fx(double *f, double *x);

```

```

void derfx(double **j, double *x);
void gauss(double **a, double *b, double *x, int n);

main (){
    int i, n=N, nm1=N-1;
    double **j, *f, *x, *x1, dnor;

    j = matriz(n,n);
    f = vector(n);
    x = vector(n);
    x1 = vector(n);

    for (i=0; i<=nm1; i++) x[i] = x1[i] = 1;

/* *** Proceso iterativo *** */
    do {
        for (i=0; i<=nm1; i++) x[i] = x1[i];
        fx(f,x);
        derfx(j,x);
        gauss(j,f,x,nm1);
        for (i=0; i<=nm1; i++) x1[i] -= x[i];
        for (i=0; i<=nm1; i++) printf("%e ",x1[i]);
        printf("\n");
        dnor = dmax(x,nm1)/dmax(x1,nm1);
        printf ("%e\n",dnor);
    }
    while (dnor > TOL);

    return 0;
}

void fx(double *f, double *x)
{
    f[0] = 3*x[0]-cos(x[1]*x[2])-0.5;
    f[1] = x[0]*x[0]-81*pow((x[1]+0.1),2)+sin(x[2])+1.06;
    f[2] = exp((-x[0])*x[1])+20*x[2]+(10*acos(-1.e0)-3)/3;
}

void derfx(double **j, double *x)
{
    j[0][0] = 3.;
    j[0][1] = sin(x[1]*x[2])*x[2];
    j[0][2] = sin(x[1]*x[2])*x[1];
    j[1][0] = 2*x[0];
    j[1][1] = (-162)*(x[1]+0.1);
    j[1][2] = cos(x[2]);
    j[2][0] = -exp((-x[0])*x[1])*x[1];
    j[2][1] = -exp((-x[0])*x[1])*x[0];
    j[2][2] = 20;
}

void gauss(double **a, double *b, double *x, int n)
{
    int i, ip, *ipvt, j, k, l, pi;
    double c, r, r1, smax;

    ipvt = ivector(n);
    for (i=0; i<=n; i++) ipvt[i]=i;

/* *** Triangularización *** */

    for (k=0; k<n; k++){
        smax = fabs(a[ipvt[k]][k]);
        for (i=k+1, l=0; i<=n; i++){
            ip = ipvt[i];
            if (fabs(a[ip][k]) > smax){
                l = i;
                smax = fabs(a[ip][k]);
            }
        }
        if (l) SWAPi(ipvt[k],ipvt[l]);
        pi = ipvt[k];
        r1 = 1.0/a[pi][k];
        for (i=k+1; i<=n; i++){
            ip = ipvt[i];
            r = a[ip][k]*r1;
            for (j=k+1; j<=n; j++) a[ip][j] -= r*a[pi][j];
            a[ip][k] = -r;
        }
    }

    for (k=0; k<n; k++){
        ip = ipvt[k];
        for (i=k+1; i<=n; i++){
            pi = ipvt[i];
            b[pi] += a[pi][k]*b[ip];
        }
    }

/* *** Sustitución inversa *** */

```

```

x[n] = b[ipvt[n]]/a[ipvt[n]][n];
for (i=n-1; i>=0; i--){
    pi = ipvt[i];
    for (j=i+1, c=b[pi]; j<=n; j++) c -= a[pi][j]*x[j];
    x[i] = c/a[pi][i];
}
free(ipvt);
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc,sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

int *ivector(int n)
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

```

9. Programa Newtrpdf de la página 314.

```

/*
-- Resolución de un sistema de ecuaciones no lineales
cualquiera mediante el método de Newton
(Derivadas por Diferencias Finitas)
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define N 3
#define TOL sqrt(DBL_EPSILON)
#define H sqrt(DBL_EPSILON)
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
double dmax(double *x, int n);
void fx(double *f, double *x, int n);
void derfx(double **j, double *x, int n, double *f, double *f1,
            double *x1);
void gauss(double **a, double *b, double *x, int n);

```



```

main (){
    int i, n=N, nm1=N-1;
    double **j, *f, *f1, *x, *x1, *x2, dnor;

    j = matriz(n,n);
    f = vector(n);
    f1 = vector(n);
    x = vector(n);
    x1 = vector(n);
    x2 = vector(n);

    for (i=0; i<=nm1; i++) x[i] = x1[i] = 1;

/* *** Proceso iterativo *** */
    do {
        for (i=0; i<=nm1; i++) x[i] = x1[i];
        fx(f,x,nm1);
        derfx(j,x,nm1,f,f1,x2);
        gauss(j,f,x,nm1);
        for (i=0; i<=nm1; i++) x1[i] -= x[i];
        for (i=0; i<=nm1; i++) printf("%e ",x1[i]);
        printf ("\n");
        dnor = dmax(x,nm1)/dmax(x1,nm1);
        printf ("%e\n",dnor);
    }
    while (dnor > TOL);

    return 0;
}

void fx(double *f, double *x, int n)
{
    f[0] = 3*x[0]-cos(x[1]*x[2])-0.5;
    f[1] = x[0]*x[0]-81*pow((x[1]+0.1),2)+sin(x[2])+1.06;
    f[2] = exp((-x[0])*x[1])+20*x[2]+(10*acos(-1.e0)-3)/3;
}

void derfx(double **j, double *x, int n, double *f, double *f1,
            double *x1)
{
    int i, k;

    for (i=0; i<=n; i++) x1[i] = x[i];
    for (i=0; i<=n; i++){
        x1[i] += H;
        fx(f,x1,n);
        for (k=0; k<=n; k++) f1[k] = f[k];
        fx(f,x,n);
        for (k=0; k<=n; k++) f1[k] = (f1[k]-f[k])/H;
        for (k=0; k<=n; k++) j[i][k] = f1[k];
        x1[i] = x[i];
    }
}

void gauss(double **a, double *b, double *x, int n)
{
    int i, ip, *ipvt, j, k, l, pi;
    double c, r, r1, smax;

    ipvt = ivector(n);
    for (i=0; i<=n; i++) ipvt[i]=i;

/* *** Triangularización *** */

    for (k=0; k<n; k++){
        smax = fabs(a[ipvt[k]][k]);
        for (i=k+1, l=0; i<=n; i++){
            ip = ipvt[i];
            if (fabs(a[ip][k]) > smax){
                l = i;
                smax = fabs(a[ip][k]);
            }
        }
        if (l) SWAPi(ipvt[k],ipvt[l]);
        pi = ipvt[k];
        r1 = 1.0/a[pi][k];
        for (i=k+1; i<=n; i++){
            ip = ipvt[i];
            r = a[ip][k]*r1;
            for (j=k+1; j<=n; j++) a[ip][j] -= r*a[pi][j];
            a[ip][k] = -r;
        }
    }

    for (k=0; k<n; k++){
        ip = ipvt[k];
        for (i=k+1; i<=n; i++){
            pi = ipvt[i];
            b[pi] += a[pi][k]*b[ip];
        }
    }
}

```

```

    }

/* *** Sustitución inversa *** */

    x[n] = b[ipvt[n]]/a[ipvt[n]][n];
    for (i=n-1; i>=0; i--){
        pi = ipvt[i];
        c = b[pi];
        for (j=i+1; j<=n; j++) c -= a[pi][j]*x[j];
        x[i] = c/a[pi][i];
    }
    free(ipvt);
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc,sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

int *ivector(int n)
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

```

10. Programa Newjac de la página 317.

```

/*
-- Resolución de un sistema de ecuaciones no lineales
cualquiera mediante el método de Newton
(Variante Jacobi)
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define N 3
#define TOL sqrt(DBL_EPSILON)

double *vector(int n);
double dmax(double *x, int n);
void fx(double *f, double *x, int n);
void derfx(double *j, double *x, int n);

main (){

```

```

int i, n=N, nm1=N-1;
double *j, *f, *x, *x1, dnor;

j = vector(n);
f = vector(n);
x = vector(n);
x1 = vector(n);

for (i=0; i<=nm1; i++) x[i] = x1[i] = 1;

/* *** Proceso iterativo *** */

do {
    for (i=0; i<=nm1; i++) x[i] = x1[i];
    fx(f,x,nm1);
    derfx(j,x,nm1);
    for (i=0; i<=nm1; i++) x[i] = f[i]/j[i];
    for (i=0; i<=nm1; i++) x1[i] = x[i];
    for (i=0; i<=nm1; i++) printf("%e ",x1[i]);
    printf ("\n");
    dnor = dmax(x,nm1)/dmax(x1,nm1);
    printf ("%e\n",dnor);
}
while (dnor > TOL);

return 0;

}

void fx(double *f, double *x, int n)
{
    f[0] = 3*x[0]-cos(x[1]*x[2])-0.5;
    f[1] = x[0]*x[0]-81*pow((x[1]+0.1),2)+sin(x[2])+1.06;
    f[2] = exp((-x[0])*x[1])+20*x[2]+(10*acos(-1.e0)-3)/3;
}

void derfx(double *j, double *x, int n)
{
    j[0] = 3;
    j[1] = (-162)*(x[1]+0.1);
    j[2] = 20;
}

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

```

11. Programa Newsor de la página 319.

```

/*
-- Resolución de un sistema de ecuaciones no lineales
   cualquier mediante el método de Newton
   (Variante Sobrerrelajación)
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define N 3
#define TOL sqrt(DBL_EPSILON)
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
double dmax(double *x, int n);
void fx(double *f, double *x, int n);
void derfx(double **j, double *x, int n, double omega);
void susdi(double **a, double *b, double *x, int n);

main (){

    int i, n=N, nm1=N-1;
    double **j, *f, *x, *x1, dnor, omega;

```

```

j = matriz(n,n);
f = vector(n);
x = vector(n);
x1 = vector(n);

printf ("Valor de OMEGA?\n");
scanf ("%lf", &omega);

for (i=0; i<=nm1; i++) x[i] = x1[i] = 1;

/* *** Proceso iterativo *** */

do {
    for (i=0; i<=nm1; i++) x[i] = x1[i];
    fx(f,x,nm1);
    derfx(j,x,nm1,omega);
    susdi(j,f,x,nm1);
    for (i=0; i<=nm1; i++) x1[i] = x[i];
    for (i=0; i<=nm1; i++) printf("%e ",x1[i]);
    printf ("\n");
    dnor = dmax(x,nm1)/dmax(x1,nm1);
    printf ("%e\n",dnor);
}
while (dnor > TOL);

return 0;

}

void fx(double *f, double *x, int n)
{
    f[0] = 3*x[0]-cos(x[1]*x[2])-0.5;
    f[1] = x[0]*x[0]-81*pow((x[1]+0.1),2)+sin(x[2])+1.06;
    f[2] = exp((-x[0])*x[1])+20*x[2]+(10*acos(-1.e0)-3)/3;
}

void derfx(double **j, double *x, int n, double omega)
{
    j[0][0] = 3*(1+omega);
    j[1][0] = 2*x[0];
    j[1][1] = (-162)*(x[1]+0.1)*(1+omega);
    j[2][0] = -exp((-x[0])*x[1])*x[1];
    j[2][1] = -exp((-x[0])*x[1])*x[0];
    j[2][2] = 20*(1+omega);
}

void susdi(double **a, double *b, double *x, int n)
{
    int i, j;
    double c;

    /* *** Sustitución directa *** */

    x[0] = b[0]/a[0][0];
    for (i=1; i<=n; i++){
        for (j=0, c=b[i]; j<=i-1; j++) c -= a[i][j]*x[j];
        x[i] = c/a[i][i];
    }

    double **matriz(int nf, int nc)
    {
        int i;
        double **m, *m1;

        m1 = (double *) calloc(nf*nc,sizeof(double));
        if (!m1){
            printf("Error de asignación de memoria en matriz\n");
            abort();
        }
        m = (double **) malloc(nf*sizeof(double *));
        if (!m){
            printf("Error de asignación de memoria en matriz\n");
            abort();
        }
        for (i=0; i<nf; i++){
            m[i] = m1;
            m1 += nc;
        }
        return m;
    }

    double *vector(int n)
    {
        double *m;

        m = (double *) malloc(n*sizeof(double));
        if (!m){
            printf("Error de asignación de memoria en vector\n");
            abort();
        }
        return m;
    }

```

```

}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

```

12. Programa Broyden de la página 323.

```

/*
-- Resolución de un sistema de ecuaciones no lineales
   cualquiera mediante el método cuasi Newton
   resultante de aplicar la fórmula de Broyden
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define N 3
#define TOL sqrt(DBL_EPSILON)
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
double dmax(double *x, int n);
void fx(double *f, double *x);
void derfx(double **j, double *x);
void broyd(double **j, double *y, double *s, int n);
void gauss(double **a, double *b, double *x, int n);

main (){
    int i, n=N, nm1=N-1;
    double **j, *f, *f1, *s, *xk, *xkm1, *y, dnor;

    j = matriz(n,n);
    f = vector(n);
    f1 = vector(n);
    s = vector(n);
    xk = vector(n);
    xkm1 = vector(n);
    y = vector(n);

    for (i=0; i<=nm1; i++) xk[i] = 1;
    derfx(j,xk);

/* *** Proceso iterativo *** */
    do {
        fx(f,xk);
        for (i=0; i<=nm1; i++) f1[i] = f[i];
        gauss(j,f,s,nm1);
        for (i=0; i<=nm1; i++) xkm1[i] = xk[i]-s[i];
        fx(f,xkm1);
        for (i=0; i<=nm1; i++) y[i] = f[i]-f1[i];
        broyd(j,y,s,nm1);
        for (i=0; i<=nm1; i++) xk[i] = xkm1[i];
        for (i=0; i<=nm1; i++) printf("%e ",xkm1[i]);
        printf("\n");
        dnor = dmax(s,nm1)/dmax(xkm1,nm1);
        printf ("%e\n",dnor);
    }
    while (dnor > TOL);

    return 0;
}

void fx(double *f, double *x)
{
    f[0] = 3*x[0]-cos(x[1]*x[2])-0.5;
    f[1] = x[0]*x[0]-81*pow((x[1]+0.1),2)+sin(x[2])+1.06;
    f[2] = exp((-x[0])*x[1])+20*x[2]+(10*acos(-1.e0)-3)/3;
}

void derfx(double **j, double *x)
{
    j[0][0] = 3.;
    j[0][1] = sin(x[1]*x[2])*x[2];
    j[0][2] = sin(x[1]*x[2])*x[1];
    j[1][0] = 2*x[0];
    j[1][1] = (-162)*(x[1]+0.1);
    j[1][2] = cos(x[2]);
    j[2][0] = -exp((-x[0])*x[1])*x[1];
    j[2][1] = -exp((-x[0])*x[1])*x[0];
    j[2][2] = 20;
}

```

```

}

void broyd(double **j, double *y, double *s, int n)
{
    int i, jj;
    double sum, prod;

    for (i=0, prod=0; i<=n; i++) prod += pow(s[i],2);
    for (i=0; i<=n; i++){
        for (jj=0, sum=0; jj<=n; jj++) sum += j[i][jj]*s[jj];
        y[i] = (y[i]+sum)/prod;
    }
    for (i=0; i<=n; i++) for(jj=0; jj<=n; jj++)
        j[i][jj] -= y[i]*s[jj];
}

void gauss(double **a, double *b, double *x, int n)
{
    int i, ip, *ipvt, j, k, l, pi;
    double c, r, r1, smax;

    ipvt = ivector(n);
    for (i=0; i<=n; i++) ipvt[i]=i;

/* *** Triangularización *** */
    for (k=0; k<n; k++){
        smax = fabs(a[ipvt[k]][k]);
        for (i=k+1, l=0; i<=n; i++){
            ip = ipvt[i];
            if (fabs(a[ip][k]) > smax){
                l = i;
                smax = fabs(a[ip][k]);
            }
            if (l) SWAPi(ipvt[k],ipvt[l]);
            pi = ipvt[k];
            r1 = 1.0/a[pi][k];
            for (i=k+1; i<=n; i++){
                ip = ipvt[i];
                r = a[ip][k]*r1;
                for (j=k+1; j<=n; j++) a[ip][j] -= r*a[pi][j];
                a[ip][k] = -r;
            }
        }

        for (k=0; k<n; k++){
            ip = ipvt[k];
            for (i=k+1; i<=n; i++){
                pi = ipvt[i];
                b[pi] += a[pi][k]*b[ip];
            }
        }
    }

/* *** Sustitución inversa *** */
    x[n] = b[ipvt[n]]/a[ipvt[n]][n];
    for (i=n-1; i>=0; i--){
        pi = ipvt[i];
        c = b[pi];
        for (j=i+1; j<=n; j++) c -= a[pi][j]*x[j];
        x[i] = c/a[pi][i];
    }
    free(ipvt);

    double **matriz(int nf, int nc)
    {
        int i;
        double **m, *m1;

        m1 = (double *) calloc(nf*nc,sizeof(double));
        if (!m1){
            printf("Error de asignación de memoria en matriz\n");
            abort();
        }
        m = (double **) malloc(nf*sizeof(double *));
        if (!m){
            printf("Error de asignación de memoria en matriz\n");
            abort();
        }
        for (i=0; i<nf; i++){
            m[i] = m1;
            m1 += nc;
        }
        return m;
    }

    double *vector(int n)
    {
        double *m;

```

```

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

int *ivector(int n)
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

```

13. Programa Newtarmijo de la página 333.

```

/*
Resolución de un sistema de ecuaciones no lineales
mediante el método de Newton con el mecanismo
de salvaguarda de Armijo.
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define N 3
#define TOL sqrt(DBL_EPSILON)
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
double dnor(double *f, int n);
void fx(double *f, double *x);
void derfx(double **j, double *x);
void gauss(double **j, double *f, double *x, int n);

main (){

    int    n=N, iml=N-1, i;
    double **j, *f, *x, *x1, *s, dnr, dnx, alfa;

    j = matriz(n,n);
    f = vector(n);
    s = vector(n);
    x = vector(n);
    x1 = vector(n);

    for (i=0; i<n; i++) x[i] = 0;
    fx(f,x);
    dnx = dnor(f,n);

    while (dnx>TOL) {
        derfx(j,x);
        gauss(j,f,s,iml);
        for (i=0; i<n; i++) x1[i] = x[i]-s[i];
        fx(f,x1);
        dnr = dnor(f,n);
        while (dnr>(1.0-alfa/2)*dnx) {
            alfa = alfa/2;
            for (i=0; i<n; i++) x1[i] = x[i]-alfa*s[i];
            fx(f,x1);
            dnr = dnor(f,n);
        }
        printf ("%e,%e,%e,%e,%e\n",x[0],x[1],x[2],alfa,dnr);
        for (i=0; i<n; i++) x[i] = x1[i];
        dnx = dnr;
    }

    return 0;
}

void fx(double *f, double *x)
{

```

```

    f[0] = 6*atan(x[0]-10)-2*exp(-x[1])-
        2*exp(-x[2])+2*x[1]+2*x[2]-9;
    f[1] = 2*atan(x[0]-10)-4*exp(-x[1])-
        exp(-x[2])+7*x[1]-2*x[2]-3;
    f[2] = 2*atan(x[0]-10)-exp(-x[1])-
        3*exp(-x[2])-x[1]+5*x[2]-3;
}

void derfx(double **j, double *x)
{
    j[0][0] = 6/(1+(x[0]-10)*(x[0]-10));
    j[0][1] = 2*exp(-x[1])+2;
    j[0][2] = 2*exp(-x[2])+2;
    j[1][0] = 2/(1+(x[0]-10)*(x[0]-10));
    j[1][1] = 4*exp(-x[1])+7;
    j[1][2] = exp(-x[2])-2;
    j[2][0] = 2/(1+(x[0]-10)*(x[0]-10));
    j[2][1] = exp(-x[1])-1;
    j[2][2] = 3*exp(-x[2])+5;
}

void gauss(double **a, double *b, double *x, int n)
{
    int    i, ip, *ipvt, j, k, l, pi;
    double c, r, rl, smax;

    ipvt = ivector(n+1);
    for (i=0; i<=n; i++) ipvt[i]=i;

    /* *** Triangularización *** */

    for (k=0; k<n; k++){
        smax = fabs(a[ipvt[k]][k]);
        for (i=k+1, l=0; i<=n; i++){
            ip = ipvt[i];
            if (fabs(a[ip][k]) > smax){
                l = i;
                smax = fabs(a[ip][k]);
            }
            if (l) SWAPi(ipvt[k],ipvt[l]);
            pi = ipvt[k];
            r1 = 1.0/a[pi][k];
            for (i=k+1; i<=n; i++){
                ip = ipvt[i];
                r = a[ip][k]*r1;
                for (j=k+1; j<=n; j++) a[ip][j] -= r*a[pi][j];
                a[ip][k] = -r;
            }
        }

        for (k=0; k<n; k++){
            ip = ipvt[k];
            for (i=k+1; i<=n; i++){
                pi = ipvt[i];
                b[pi] += a[pi][k]*b[ip];
            }
        }

    /* *** Sustitución inversa *** */

    x[n] = b[ipvt[n]]/a[ipvt[n]][n];
    for (i=n-1; i>=0; i--){
        pi = ipvt[i];
        for (j=i+1, c=b[pi]; j<=n; j++) c -= a[pi][j]*x[j];
        x[i] = c/a[pi][i];
    }
    free(ipvt);
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc,sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

```

```

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

int *ivector(int n)
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

double dnor(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<n; i++) dm = x[i]*x[i];
    return dm;
}

```

14. Programa Gausnewt de la página 347

```

/*
-- Resolución de un problema no lineal de mínimos
    cuadrados mediante el método de Gauss-Newton
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define M 4
#define N 2
#define TOL sqrt(DBL_EPSILON)
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}
#define SIGN(a,b) (b>0 ? fabs(a) : -fabs(a))

double *vector(int n);
double **matriz(int nf, int nc);
double dmax(double *x, int n);
void fx(double *f, double *x);
void derfx(double **j, double *x);
void qrdcs(double **a, double *b, double *x,
           double *ax1, int m, int n, double *s);

main (){

    int i, n=N, nm1=N-1, m=M, mm1=M-1;
    double **j, *f, *x, *p, *ax1, s, dnor;

    j = matriz(m,n);
    f = vector(m);
    x = vector(n);
    p = vector(n);
    ax1 = vector(m);

    for (i=0; i<=nm1; i++) x[i] = 1;

    /* *** Proceso iterativo *** */

    do {
        fx(f,x);
        derfx(j,x);
        qrdcs(j,f,ax1,mm1,nm1,&s);
        for (i=0; i<=nm1; i++) x[i] -= p[i];
        for (i=0; i<=nm1; i++) printf("%e ",x[i]);
        printf ("\n");
        dnor = dmax(p,nm1)/dmax(x,nm1);
        printf ("%e, %e\n",dnor,s);
    }
    while (dnor > TOL);

    return 0;
}

void fx(double *f, double *x)
{
    f[0] = exp(x[0]-2*x[1])-0.5;

```

```

    f[1] = exp(x[0]-x[1])-1;
    f[2] = exp(x[0])-2;
    f[3] = exp(x[0]+x[1])-4;
}

void derfx(double **j, double *x)
{
    j[0][0] = exp(x[0]-2*x[1]);
    j[0][1] = (-2)*exp(x[0]-2*x[1]);
    j[1][0] = exp(x[0]-x[1]);
    j[1][1] = -exp(x[0]-x[1]);
    j[2][0] = exp(x[0]);
    j[2][1] = 0;
    j[3][0] = exp(x[0]+x[1]);
    j[3][1] = exp(x[0]+x[1]);
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc,sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

void qrdcs(double **a, double *b, double *x,
           double *d, int m, int n, double *s)
{
    int i, j, k, l;
    double beta, sigma, sum, dmax, wj;

    /* *** Reducción QA = R *** */

    for (j=0; j<=n; j++){
        for (i=j, dmax=0; i<=n; i++)
            dmax = max(dmax,fabs(a[i][j]));
        if (!dmax) {
            printf ("Stop: La matriz A no es de rango completo\n");
            abort();
        }
        for (i=j+1, beta=0; i<=m; i++) beta += pow(a[i][j],2);
        wj = a[j][j];
        sigma = SIGN(sqrt(beta+wj*wj),wj);
        wj += sigma;
        beta = 2/(beta+wj*wj);
        a[j][j] = wj;
        d[j] = -sigma;
        for (l=j+1; l<=n; l++){
            for (k=j, sum=0; k<=m; k++) sum += a[k][j]*a[k][l];
            sum *= beta;
            for (k=j; k<=m; k++) a[k][l] -= a[k][j]*sum;
        }
        for (k=j, sum=0; k<=m; k++) sum += a[k][j]*b[k];
        sum *= beta;
        for (k=j; k<=m; k++) b[k] -= a[k][j]*sum;
    }
}

```

```

/* *** Resolución Rx = b *** */
x[n] = b[n]/d[n];
for (i=n-1; i>=0; i--){
    for (k=i+1, sum=0; k<=n; k++) sum += a[i][k]*x[k];
    x[i] = (b[i]-sum)/d[i];
}

/* *** Suma de residuos al cuadrado *** */
for (i=n+1, *s=0; i<=m; i++) *s += pow(b[i],2);
}

```

15. Programa Levmar de la página 353.

```

/*
-- Resolución de un problema no lineal de mínimos cuadrados
mediante el método de Levenberg-Marquardt
*/

#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>

#define M 4
#define N 2
#define SWAPi(a,b) {int temp=a; a=b; b=temp;}

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
double dmax(double *x, int n);
double prod(double *x, double *y, int n, int m, int l1);
void fx(double *f, double *x);
void derfx(double **j, double *x);
void gauss(double **a, double *b, double *x, int n);

main (){
    int i, l, n=N, nm1=N-1, m=M, mm1=M-1, uno=1;
    double **j, **jtj, **a, *f, *f1, *x, *p, *b, amu, dnor, res,
        res1, TOL=sqrt(DBL_EPSILON);

    j = matriz(m,n);
    jtj = matriz(n,n);
    a = matriz(n,n);
    f = vector(m);
    f1 = vector(m);
    x = vector(n);
    p = vector(n);
    b = vector(n);

    amu = 0.1;
    for (i=0; i<=nm1; i++) x[i] = 1;

    /* *** Proceso iterativo *** */

    do {
        fx(f,x);
        derfx(j,x);
        for (i=0; i<=nm1; i++) for (l=0; l<=nm1; l++)
            jtj[i][l] = prod(&j[0][i], &j[0][l], mm1, n, n);
        for (i=0; i<=nm1; i++) b[i] = prod(&j[0][i], f, mm1, n, uno);
        res = prod(f, f, mm1, uno, uno);
        for (i=0; i<=nm1; i++) for (l=0; l<=nm1; l++)
            a[i][l] = jtj[i][l];
        do {
            for (i=0; i<=nm1; i++) a[i][i] = jtj[i][i]+amu;
            gauss(a, b, p, nm1);
            for (i=0; i<=nm1; i++) b[i] = x[i]-p[i];
            fx(f1, b);
            res1 = prod(f1, f1, mm1, uno, uno);
            amu *= 10;
        } while (res1 > res);
        for (i=0; i<=nm1; i++) x[i] = b[i];
        for (i=0; i<=nm1; i++) f[i] = f1[i];
        dnor = dmax(p, nm1)/dmax(x, nm1);
        for (i=0; i<=nm1; i++) printf("%e ", x[i]);
        printf("\n");
        amu /= 100;
        printf (" %e, %e, %e\n", res1, dnor, amu);
    } while (dnor > TOL);

    return 0;
}

void fx(double *f, double *x)
{
    f[0] = exp(x[0]-2*x[1])-5;
    f[1] = exp(x[0]-x[1])-1;

```

```

    f[2] = exp(x[0])-2;
    f[3] = exp(x[0]+x[1])+4;
}

void derfx(double **j, double *x)
{
    j[0][0] = exp(x[0]-2*x[1]);
    j[0][1] = (-2)*exp(x[0]-2*x[1]);
    j[1][0] = exp(x[0]-x[1]);
    j[1][1] = -exp(x[0]-x[1]);
    j[2][0] = exp(x[0]);
    j[2][1] = 0;
    j[3][0] = exp(x[0]+x[1]);
    j[3][1] = exp(x[0]+x[1]);
}

void gauss(double **a, double *b, double *x, int n)
{
    int i, ip, *ipvt, j, k, l, pi;
    double c, r, r1, smax;

    ipvt = ivector(n);
    for (i=0; i<=n; i++) ipvt[i]=i;

    /* *** Triangularización *** */

    for (k=0; k<=n; k++){
        smax = fabs(a[ipvt[k]][k]);
        for (i=k+1, l=0; i<=n; i++){
            ip = ipvt[i];
            if (fabs(a[ip][k]) > smax){
                l = i;
                smax = fabs(a[ip][k]);
            }
        }
        if (l) SWAPi(ipvt[k], ipvt[l]);
        pi = ipvt[k];
        r1 = 1.0/a[pi][k];
        for (i=k+1; i<=n; i++){
            ip = ipvt[i];
            r = a[ip][k]*r1;
            for (j=k+1; j<=n; j++) a[ip][j] -= r*a[pi][j];
            a[ip][k] = -r;
        }
    }

    for (k=0; k<=n; k++){
        ip = ipvt[k];
        for (i=k+1; i<=n; i++){
            pi = ipvt[i];
            b[pi] += a[pi][k]*b[ip];
        }
    }

    /* *** Sustitución inversa *** */

    x[n] = b[ipvt[n]]/a[ipvt[n]][n];
    for (i=n-1; i>=0; i--){
        pi = ipvt[i];
        for (j=i+1, c=b[pi]; j<=n; j++) c -= a[pi][j]*x[j];
        x[i] = c/a[pi][i];
    }
    free(ipvt);
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc, sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<=nf; i++, m1+=nc) m[i] = m1;
    return m;
}

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }

```

```

    }
    return m;
}

int *ivector(int n)
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

double dmax(double *x, int n)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm = max(fabs(x[i]),dm);
    return dm;
}

double prod(double *x, double *y, int n, int m, int l1)
{
    int i;
    double dm;
    for (i=0, dm=0; i<=n; i++) dm += (*(x+m*i))*(*(y+l1*i));
    return dm;
}

```

G.1.5 Códigos del apéndice B

1. Programa Beta-y-t de la página 701

```

/*
  Cálculo de los parámetros beta y t de una máquina
  */

#include <stdio.h>
#include <math.h>
#include <float.h>

main (){

    double a=1,b=2,t=1,beta,f();

    while ((f(a+1)-a)==1) a *= 2;

    while ((a+b)==a) b *= 2;

    beta = (a+b)-a;
    a = beta;
    while ((f(a+1)-a)==1) {
        t++;
        a *= beta;
    }

    printf ("beta = %f, t = %15.9e\n",beta,t);
    return 0;
}

double f(s)
{
    double s;
    {
        return s;
    }
}

```

2. Programa EPSILON de la página 705.

```

/*
  Cálculo de la precisión de una máquina
  */

#include <stdio.h>
#include <math.h>
#include <float.h>

main (){

    float eps=1.0, xp1;
    double epsd=1.0, xpid;

    /* *** Cálculo recurrente *** */

```

```

    xp1 = eps+1;
    while (xp1!=1) {
        eps /= 2;
        xp1 = eps+1;
    }

    xpid = epsd+1;
    while (xpid!=1) {
        epsd /= 2;
        xpid = epsd+1;
    }

    printf ("eps = %15.9e, epsd = %15.9e\n",eps*2,epsd*2);
    return 0;
}

```

3. Programa Suma-de-serie de la página 710.

```

/*
  Suma de una serie de infinitos sumandos
  */

#include <stdio.h>
#include <math.h>
#include <float.h>
#define R 1e10

main (){

    double i,suma,sqrt();

    suma = 0.0;
    for (i=R;i>0.0;i--) suma += 1.0/(i*i);

    printf ("suma = %15.14f\n",suma);
    printf ("pi = %15.14f\n",sqrt(suma*6));
    return 0;
}

```

G.1.6 Códigos del apéndice H

1. Programa Condest de la página 882

```

/*
  Estimación del número de condición de una matriz.
  */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define N 10

int *ivector(int n);
double *vector(int n);
double **matriz(int nf, int nc);
void gau(int n, double **a, int *ipvt);
double rcond(int n, int *ipvt, double **a, double anorm, double *z);

main (){

    int i, j, n=N, *ipvt;
    double cond, anorm, aux, **a, *z;

    FILE *f_in; /* entrada de datos */
    FILE *f_out; /* salida de datos */

    ipvt = ivector(n);
    a = matriz(n,n);
    z = vector(n);

    if ((f_in=fopen("clin1","r"))==NULL) {
        printf("No se ha podido abrir el fichero clin1\n");
        exit(-1);
    }
    f_out=fopen("sal.txt","w");
    if ((f_out=fopen("sal.txt","w"))==NULL) {
        printf("No se ha podido abrir el fichero sal.txt\n");
        exit(-1);
    }

    for (i=0;i<n;i++) for (j=0;j<n;j++) fscanf(f_in,"%lf",&a[i][j]);

    /*
    norma 1 de la matriz

```

```

        */
        anorm=0.0;
        for (i=0;i<n;i++) {
            aux=0.0;
            for (j=0;j<n;j++) aux+=fabs(a[j][i]);
            if (anorm<aux) anorm=aux;
        }

/*
Estimación del número de condición. Factorizar primero PA=LU
*/
        gau(n,a,ipvt);
        for(j=0;j<n;j++) z[j]=0.0;
        cond=rcond(n,ipvt,a,anorm,z);

        fprintf(f_out,"La solución es cond = %15.7le\n",cond);
        fclose(f_in);
        fclose(f_out);

        return 0;
}

/* -----
Subrutina rcond
*/
double rcond(int n, int *ipvt, double **a, double anorm, double *z)
{
    int i,k,l;
    double ek,z1,z2,z3,auxil,wk,wkm,sm,sumb,sumz,s,t;
    double ynorm,cond;

    ek=1.0;
    for (k=0;k<n;k++){
        if (z[k]>0.0) if (ek>0.0) ek=-ek;
        else
            if (ek<0.0) ek=-ek;
        if (fabs(ek-z[k])>fabs(a[k][k])) {
            s= fabs(a[k][k])/fabs(ek-z[k]);
            for(l=0;l<n;l++) z[l]=s*z[l];
            ek=s*ek;
        }
        wk=ek-z[k];
        wkm=(-ek)-z[k];
        s=fabs(wk);
        sm = fabs(wkm);
        if (a[k][k]!=0.0) {
            wk=wk/a[k][k];
            wkm=wkm/a[k][k];
        }
        else {
            wk=1.0;
            wkm=1.0;
        }
        z1=0.0;
        for(l=k+1;l<n;l++) z1=z1+fabs(z[l]+wkm*a[k][l]);
        sm=sm+z1;
        for(l=k+1;l<n;l++) z[l]=z[l]+wk*a[k][l];
        auxil=0.0;
        for(l=k+1;l<n;l++) auxil=auxil+fabs(z[l]);
        s=s+auxil;
        if (s<sm) {
            t=wkm-wk;
            wk=wkm;
            for(l=k+1;l<n;l++) z[l]=z[l]+t*a[k][l];
        }
        z[k]=wk;
    }
    z2=0.0;
    for(l=0;l<n;l++) z2=z2+fabs(z[l]);
    s=1.0/z2;
    for(l=0;l<n;l++) z[l]=s*z[l];
    for(k=n-1;k>=0;k--) {
        sumz=0.0;
        for(l=k+1;l<n;l++) sumz=sumz+a[l][k]*z[l];
        z[k]=z[k]+sumz;
        if (fabs(z[k])>1.0) {
            s=1.0/fabs(z[k]);
            for (l=0;l<n;l++) z[l]=s*z[l];
        }
        i=ipvt[k];
        t=z[i];
        z[i]=z[k];
        z[k]=t;
    }
    sumb=0.0;
    for(l=0;l<n;l++) sumb=sumb+fabs(z[l]);
    s=1.0/sumb;
    for(l=0;l<n;l++) z[l]=s*z[l];
    ynorm=1.0;
    for(k=0;k<n;k++){
        i=ipvt[k];
        t=z[i];
        z[i]=z[k];
        z[k]=t;
        for (l=k+1;l<n;l++) z[l]=z[l]+t*a[l][k];
        sumb=sumb+fabs(z[l]);
        s=1.0/sumb;
        for (l=0;l<n;l++) z[l]=s*z[l];
        ynorm=ynorm*s;
        if (anorm==0.0) cond=0.0;
        else cond=anorm/ynorm;
    }
    return cond;
}

/* -----
Subrutina gauss
*/
void gau(int n, double **a, int *ipvt)
{
    int i,j,k,l;
    double smax, r, r1;

    for (k=0;k<n-1;k++) {
        smax=0.0;
        for (i=k+1;i<n;i++) {
            if (fabs(a[i][k])>smax) {
                l=i;
                smax=fabs(a[i][k]);
            }
        }
        ipvt[k]=l;
        if (l!=k) {
            r=a[l][k];
            a[l][k]=a[k][k];
            a[k][k]=r;
        }
        r1=-1.0/a[k][k];
        for (j=k+1;j<n;j++) a[j][k]=a[j][k]*r1;
        for (i=k+1;i<n;i++) {
            r=a[l][i];
            if (l!=k) {
                a[l][i]= a[k][i];
                a[k][i]=r;
            }
            for (j=k+1;j<n;j++) a[j][i]=a[j][i]+r*a[j][k];
        }
    }
    ipvt[n-1]=n-1;
}

double **matriz(int nf, int nc)
{
    int i;
    double **m, *m1;

    m1 = (double *) calloc(nf*nc,sizeof(double));
    if (!m1){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    m = (double **) malloc(nf*sizeof(double *));
    if (!m){
        printf("Error de asignación de memoria en matriz\n");
        abort();
    }
    for (i=0; i<nf; i++){
        m[i] = m1;
        m1 += nc;
    }
    return m;
}

```



```

double *vector(int n)
{
    double *m;

    m = (double *) malloc(n*sizeof(double));
    if (!m){
        printf("Error de asignación de memoria en vector\n");
        abort();
    }
    return m;
}

int *ivector(int n)
{
    int *m;

    m = (int *) malloc(n*sizeof(int));
    if (!m){
        printf("Error de asignación de memoria en ivector\n");
        abort();
    }
    return m;
}

```

G.2 Códigos en Fortran 90

G.2.1 Códigos del capítulo 1

1. Programa Gauss de la página 15.

```

PROGRAM Eliminacion_de_Gauss
!
!   integer, parameter :: n=4
!   integer             :: k,l,i
!   real                :: a(n,n+1),x(n),temp(n+1),smax
!
!   data a/2.,-4.,4.,0.,1.,-2.,1.,-3.,0.,3.,-2.,-12.,4.,-7.,8., &
!       -1.,2.,-9.,2.,2./
!
!   *** Eliminación de Gauss ***
!
!   * Triangularización *
!
!   do k=1,n-1
!       l = 0; smax = abs(a(k,k))
!       do i=k+1,n
!           if (abs(a(i,k))>smax) then
!               l = i; smax = abs(a(i,k))
!           endif
!       enddo
!       if (l/=0) then
!           temp = a(l,:); a(l,:)=a(k,:); a(k,:)=temp;
!       endif
!       a(k,:) = a(k,:)/a(k,k)
!       do i=k+1,n
!           a(i,k+1:n+1) = a(i,k+1:n+1)-a(i,k)*a(k,k+1:n+1)
!       enddo
!   enddo
!
!   * Sustitución inversa *
!
!   x(n) = a(n,n+1)/a(n,n)
!   do i=n-1,1,-1
!       x(i) = (a(i,n+1)-dot_product(a(i,i+1:n),x(i+1:n)))/a(i,i)
!   enddo
!
!   print *,x
!
!   END PROGRAM Eliminacion_de_Gauss

```

2. Programa Gaussc de la página 19.

```

PROGRAM Gaussc
!
!   integer, allocatable :: ipoint(:)
!   integer              :: pi,n,i,k,ip,l,iaux
!   real, allocatable    :: a(:,,:),b(:,),x(:)
!   real                :: smax,r,r1
!   character            :: fil*12
!
!   *** Resolución de un sistema lineal regular cualquiera Ax=b
!       mediante eliminación de Gauss ***

```

```

!
!   print *, "Dimensión de la Matriz A?"
!   read *, n
!   allocate (ipoint(n),a(n,n),b(n),x(n))
!   print *, "Fichero de datos?"
!   read '(A)',fil
!   open (10,FILE=fil)
!   read (10,*) a,b
!
!   ipoint = (/i,i=1,n)/
!
!   * Triangularización *
!
!   do k=1,n-1
!       l = 0; smax = abs(a(ipoint(k),k))
!       do i=k+1,n
!           ip = ipoint(i)
!           if (abs(a(ip,k))>smax) then
!               l = i; smax = abs(a(ip,k))
!           endif
!       enddo
!       if (l/=0) then
!           iaux = ipoint(k); ipoint(k) = ipoint(l); ipoint(l) = iaux
!       endif
!       pi = ipoint(k)
!       r1 = 1/a(pi,k)
!       do i=k+1,n
!           ip = ipoint(i)
!           r = a(ip,k)*r1
!           a(ip,k+1:n) = a(ip,k+1:n)-r*a(pi,k+1:n)
!           a(ip,k) = -r
!       enddo
!   enddo
!
!   do k=1,n-1
!       pi = ipoint(k)
!       do i=k+1,n
!           ip = ipoint(i)
!           b(ip) = b(ip)+a(ip,k)*b(pi)
!       enddo
!   enddo
!
!   * Sustitución inversa *
!
!   x(n) = b(ipoint(n))/a(ipoint(n),n)
!   do i=n-1,1,-1
!       ip = ipoint(i)
!       x(i) = (b(ip)-dot_product(a(ip,i+1:n),x(i+1:n)))/a(ip,i)
!   enddo
!
!   print '( "Solución:",9f7.3:)',x(:)
!
!   END PROGRAM Gauss

```

3. Programa Crout de la página 30.

```

PROGRAM Crout
!
!   integer, parameter :: n=3
!   integer            :: i,j,k
!   real               :: a(n,n)
!
!   data a/10.,20.,30.,10.,25.,50.,20.,40.,61/
!
!   *** Factorización LU1 por el método de Crout ***
!
!   do k=1,n
!       do i=k,n
!           a(i,k)=a(i,k)-dot_product(a(i,1:k-1),a(1:k-1,k))
!       enddo
!       do i=k+1,n
!           a(k,i)=(a(k,i)-dot_product(a(k,1:k-1),a(1:k-1,i)))/a(k,k)
!       enddo
!   enddo
!
!   print '(3f7.2)',(a(j,:),j=1,n)
!
!   END PROGRAM Crout

```

4. Programa Croutp de la página 33.

```

PROGRAM Croutp
!
!   implicit none
!
!   integer, parameter :: n = 3
!
!   integer , dimension(n) :: ipvt
!   integer :: i, k, l, j, iaux

```

```

real , dimension(n,n) :: a
real :: smax, suma, aux
!
data a/10.,20.,30.,10.,25.,50.,20.,40.,61/
!
do i = 1,n
  ipvt(i) = i
end do
!
! *** Factorización LU1 con pivotación: método de Crout ***
!
do k = 1,n
  l = 0
  smax = 0.0
  do i = k,n
    suma = 0.0
    l = 1
    if (k-1>0) then
      suma = sum(a(i,:k-1)*a(:k-1,k))
      l = k
    endif
    a(i,k) = a(i,k)-suma
    if (abs(a(i,k))>smax) then
      smax = abs(a(i,k))
      l = i
    endif
  end do
  if (l/=0) then
    do j = 1,n
      aux = a(l,j)
      a(l,j) = a(k,j)
      a(k,j) = aux
    end do
    iaux = ipvt(l)
    ipvt(l) = ipvt(k)
    ipvt(k) = iaux
  endif
  do i = k+1,n
    suma = 0.0
    suma = sum(a(k,:k-1)*a(:k-1,i))
    a(k,i) = (a(k,i)-suma)/a(k,k)
  end do
end do
!
print *,ipvt
print *,(a(i,:),i=1,n)
!
END PROGRAM Croutp

```

5. Programa Crout11u de la página 36.

```

PROGRAM Crout11u
!
implicit none
!
integer, parameter :: n = 3
!
integer :: k, j, l, i
real , dimension(n,n) :: a
real :: suma
!
data a/10.,20.,30.,10.,25.,50.,20.,40.,61/
!
! *** Factorización L1U por el método de Crout ***
!
do k = 1,n
  do j = k,n
    suma = 0.0
    suma = sum(a(k,:k-1)*a(:k-1,j))
    a(k,j) = a(k,j)-suma
  end do
  do i = k+1,n
    suma = 0.0
    suma = sum(a(i,:k-1)*a(:k-1,k))
    a(i,k) = (a(i,k)-suma)/a(k,k)
  end do
end do
!
print 20,(a(i,:),i=1,n)
!
20 format(3f7.2)
!
END PROGRAM Crout11u

```

6. Programa Dool de la página 38.

```

PROGRAM Dool
!

```

```

implicit none
!
integer, parameter :: n = 4
!
integer , dimension(n) :: ipvt
integer :: i, j, k, imax, iaux
real , dimension(n,n) :: a
real :: suma, amax, dum
!
data a/1.,1.,-2.,0.,-4.,0.,0.,1.,1.,-1.,1.,1.,3.,0.,0./
!
do i = 1,n
  ipvt(i) = i
end do
!
! *** Factorización L1U con pivotación; método de Doolittle ***
!
do j = 1,n
  do i = 1,j-1
    suma = a(i,j)
    suma = suma-sum(a(i,:i-1)*a(:i-1,j))
    a(i,j) = suma
  end do
  amax = 0.0
  do i = j,n
    suma = a(i,j)
    suma = suma-sum(a(i,:j-1)*a(:j-1,j))
    a(i,j) = suma
    if (abs(suma)>amax) then
      imax = i
      amax = abs(suma)
    endif
  end do
  if (j/=imax) then
    do k = 1,n
      dum = a(imax,k)
      a(imax,k) = a(j,k)
      a(j,k) = dum
    end do
    iaux = ipvt(imax)
    ipvt(imax) = ipvt(j)
    ipvt(j) = iaux
  endif
  if (j/=n) then
    if (a(j,j).eq.0.0) a(j,j) = 1.0e-20 ! Se divide la
    dum = 1.0/a(j,j) ! columna por
    a(j+1:n,j) = a(j+1:n,j)*dum ! A(j,j)
  endif
end do
if (a(n,n).eq.0.0) a(n,n) = 1.0e-20
!
print *,ipvt
print 1,((a(i,j),j=1,n),i=1,n)
!
1 format(4f8.3)
!
END PROGRAM Dool

```

7. Programa Chol de la página 45.

```

PROGRAM Chol
!
implicit none
!
integer, parameter :: n = 4
!
integer :: i, j
real , dimension(n,n) :: a
real , dimension(n) :: b
real :: suma
!
data a/5.,1.,-2.,0.,1.,2.,0.,0.,-2.,0.,4.,1.,0.,0.,1.,3./
data b/1.,5.,14.,15./
!
! *** Factorización de Cholesky G G ***
!
do i = 1,n
  suma = a(i,i)
  suma = suma-sum(a(:i-1,i)**2)
  a(i,i) = sqrt(suma)
  do j = i+1,n
    suma = a(i,j)
    suma = suma-sum(a(:i-1,i)*a(:i-1,j))
    a(i,j) = suma/a(i,i)
  end do
end do
!
! *** Sustitución directa
!
do i = 1,n
  b(i) = b(i)-sum(a(:i-1,i)*b(:i-1))
  b(i) = b(i)/a(i,i)

```

```

      end do
!
! *** Sustitución inversa
!
      b(n) = b(n)/a(n,n)
      do i = n-1,1,-1
        b(i) = b(i)-sum(a(i,i+1:n)*b(i+1:n))
        b(i) = b(i)/a(i,i)
      end do
!
      print 1,(a(i,:),i=1,n)
      print 1,b
!
1 format(4f9.4)
!
      END PROGRAM Chol

```

8. Programa Aasen de la página 57.

```

      PROGRAM Aasen
!
!      implicit none
!
!      integer, parameter :: n = 3
!
!      integer , dimension(n) :: ipvt
!      integer :: i, j, k, iq,iaux
!      real , dimension(n,n) :: a
!      real , dimension(n) :: alfa, beta
!      real , dimension(n,n) :: l
!      real , dimension(n) :: h, v
!      real , dimension(0:n) :: l0
!      real :: smax, suma, aux
!
!      data a/1.,10.,20.,10.,1.,30.,20.,30.,1./
!
!      do i = 1,n
!        ipvt(i) = i
!      end do
!
!      T
!      *** Factorización LTL ***
!
!      do j = 1,n
!        if (j==1) then
!          h(j) = a(1,1)
!        else if (j==2) then
!          h(1) = beta(1)
!          h(2) = a(2,2)
!        else
!          l0(0) = 0.
!          l0(1) = 0.
!          l0(2:j-1) = l(j,2:j-1)
!          l0(j) = 1
!          h(j) = a(j,j)
!          h(:j-1) = beta(0:j-2)*l0(:j-2)+alfa(:j-1)*l0(1:j-1)+ &
!            beta(:j-1)*l0(2:j)
!          h(j) = h(j)-sum(l0(1:j-1)*h(:j-1))
!        endif
!
!        if (j==1.or.j==2) then
!          alfa(j) = h(j)
!        else
!          alfa(j) = h(j)-beta(j-1)*l(j,j-1)
!        endif
!
!        if (j<=n-1) then
!          smax = 0.
!          iq = j
!          do k = j+1,n
!            suma = 0.
!            suma = -sum(l(k,:j)*h(:j))
!            v(k) = a(k,j)+suma
!            if (abs(v(k))>smax) then
!              smax = abs(v(k))
!              iq = k
!            endif
!          end do
!          aux = v(j+1)
!          v(j+1) = v(iq)
!          v(iq) = aux
!          do k = 2,j
!            aux = l(j+1,k)
!            l(j+1,k) = l(iq,k)
!            l(iq,k) = aux
!          end do
!         iaux = ipvt(j+1)
!          ipvt(j+1) = ipvt(iq)
!          ipvt(iq) =iaux
!          do k = j+1,n
!            aux = a(j+1,k)
!            a(j+1,k) = a(iq,k)

```

```

      a(iq,k) = aux
!      end do
!      do k = j+1,n
!        aux = a(k,j+1)
!        a(k,j+1) = a(k,iq)
!        a(k,iq) = aux
!      end do
!      beta(j) = v(j+1)
!      endif
!      if (j<=n-2) then
!        l(j+2:n,j+1) = v(j+2:n)
!        if (v(j+1)/=0.) then
!          l(j+2:n,j+1) = l(j+2:n,j+1)/v(j+1)
!        endif
!      endif
!      end do
!
!      print *,alfa
!      print *,beta(:n-1)
!      print *,((l(j,i),j=i+1,n),i=1,n-1)
!      print *,ipvt
!
!      END PROGRAM Aasen

```

9. Programa Bunch y rutina bunchkauf de la página 66.

```

      PROGRAM Bunch
!
!      implicit none
!
!      integer, parameter :: n = 3
!
!      integer , dimension(n) :: ipvt
!      integer :: i, j
!      real , dimension(n,n) :: a
!
!      data a/1.,10.,20.,10.,1.,30.,20.,30.,1./
!
!      call bunchkauf (a,n,ipvt)
!      print *,((a(i,j),j=1,n),i=1,n)
!      print *,ipvt
!
!      END PROGRAM Bunch
!
!      SUBROUTINE bunchkauf (a,n,ipvt)
!
!      implicit none
!
!      integer , intent(in) :: n
!      integer , intent(inout) :: ipvt(n)
!      real a(n,n)
!
!      integer :: k, km1, imax, kstep, j, jmax
!      real :: mulk, mulkm1, alpha, absakk, colmax, rowmax, t, &
!        ak, akm1, deno, bk, bkm1
!      logical :: swap
!
!      integer , external :: isamax
!
!      a(-,-) Al final de la factorización contiene la matriz
!            diagonal en bloques y los multiplicadores necesarios
!            para obtenerla. T
!            Se puede escribir de la forma a=u*b*u , donde u es
!            el producto de matrices de permutación y matrices
!            triangular superior.
!
!      ipvt() Indicador de la pivotaciones realizadas.
!
!      alpha = (1.0e0+sqrt(17.0e0))/8.0e0
!      k = n
!      do while(k>1)
!
!      *** Determinar pivotación diagonal.
!      kstep indica el tamaño del bloque;
!      swap si se realizan intercambios de filas y columnas.
!
!      km1 = k-1
!      absakk = abs(a(k,k)) ! Se determina el mayor
!      imax = isamax(k-1,a(1,k)) ! elemento no en la
!      colmax = abs(a(imax,k)) ! diagonal de columna
!      if (absakk>=alpha*colmax) then
!        kstep = 1
!        swap = .false.
!      else
!        rowmax = 0.0e0
!        Mayor elemento no en la diagonal en la fila imax.
!        rowmax = amax1(rowmax,maxval(abs(a(imax,imax+1:k))))
!        if (imax/=1) then
!          jmax = isamax(imax-1,a(1,imax))

```

```

        rowmax = amax1(rowmax,abs(a(jmax,imax)))
    endif
    if (abs(a(imax,imax))>=alpha*rowmax) then
        kstep = 1
        swap = .true.
    else if (absakk>=alpha*colmax*(colmax/rowmax)) then
        kstep = 1
        swap = .false.
    else
        kstep = 2
        swap = imax/=km1
    endif
    endif
    if (amax1(absakk,colmax)==0.0e0) then
        ipvt(k) = k          ! La columna k es cero;
        go to 190           ! seguir a otra.
    endif
    if (kstep==1) then
!
! * Bloque pivote 1 x 1 *
!
        ipvt(k) = k
        if (swap) then
            call sswap (imax,a(1,imax),a(1,k))! Intercambiar
            do j = k,imax,-1                ! filas y
                t = a(j,k)                  ! columnas.
                a(j,k) = a(imax,j)
                a(imax,j) = t
            end do
            ipvt(k) = imax
        endif
!
        do j = k-1,1,-1                    ! Eliminación.
            mulk = -a(j,k)/a(k,k)
            call saxpy (j,mulk,a(1,k),a(1,j))
            a(j,k) = mulk
        end do
        else
! KSTEP=2
!
! * Bloque pivote 2 x 2 *
!
        ipvt(k) = 1-k
        ipvt(k-1) = ipvt(k)
        if (swap) then
            call sswap (imax,a(1,imax),a(1,k-1)) ! Intercambiar
            do j = k-1,imax,-1                ! filas y
                t = a(j,k-1)                  ! columnas.
                a(j,k-1) = a(imax,j)
                a(imax,j) = t
            end do
            t = a(k-1,k)
            a(k-1,k) = a(imax,k)
            a(imax,k) = t
            ipvt(k) = -imax
        endif
!
        if (k-2/=0) then                    ! Eliminación
            ak = a(k,k)/a(k-1,k)
            akm1 = a(k-1,k-1)/a(k-1,k)
            deno = 1.0e0-ak*akm1
            do j = k-2,1,-1
                bk = a(j,k)/a(k-1,k)
                bkm1 = a(j,k-1)/a(k-1,k)
                mulk = (akm1*bk-bkm1)/deno
                mulkm1 = (ak*bkm1-bk)/deno
                call saxpy (j,mulk,a(1,k),a(1,j))
                call saxpy (j,mulkm1,a(1,k-1),a(1,j))
                a(j,k) = mulk
                a(j,k-1) = mulkm1
            end do
        endif
190    endif
        continue
        k = k-kstep
        ipvt(1) = 1
!
        return
    END SUBROUTINE bunchkauf
!
    SUBROUTINE sswap(n,a,b)
!
    implicit none
!
    integer , intent(in) :: n
    real , intent(inout) :: a(n)
    real , intent(inout) :: b(n)
!
    integer :: i
    real :: aux
!
    do i = 1,n
        aux = a(i)

```

```

        a(i) = b(i)
        b(i) = aux
    end do
    return
END SUBROUTINE sswap
!
    INTEGER FUNCTION isamax (n,a)
!
    implicit none
!
    integer , intent(in) :: n
    real , intent(in) :: a(n)
!
    integer :: i
    real :: dmax
!
    isamax = 1
    dmax = abs(a(1))
    do i = 2,n
        if (abs(a(i))>dmax) then
            isamax = i
            dmax = abs(a(i))
        endif
    end do
    return
END FUNCTION isamax
!
    SUBROUTINE saxpy(j,t,a,b)
!
    implicit none
!
    integer , intent(in) :: j
    real , intent(in) :: t
    real , intent(in) :: a(1)
    real , intent(inout) :: b(1)
!
    b(:j) = b(:j)+t*a(:j)
    return
END SUBROUTINE saxpy

```

10. Programa Grmsch de la página 87.

```

    PROGRAM Grmsch
!
    implicit none
!
    integer , parameter :: m = 4
    integer , parameter :: n = 3
!
    integer :: j, i, k
    real(kind=2), dimension(m,n) :: a
    real(kind=2), dimension(n,n) :: u
    real(kind=2), dimension(n) :: x
    real(kind=2), dimension(m) :: b, res
    real(kind=2) :: epsi, dmax, temp
!
    real(kind=2), external :: prod
!
    data a/1.d0,0.,0.,0.,1.d0,0.,0.,0.,1.d0,0.,0.,0./
    data b/1.d0,0.,0.,0./
!
    *** Ortonormalizar columnas de A ***
!
    epsi = dsqrt(epsilon(1.d0))*10
    a(2,1) = epsi
    a(3,2) = epsi
    a(4,3) = epsi
    dmax = 0.d0
    do j = 1,n
        do i = 1,j-1
            u(i,j) = prod(m,a(1,i),a(1,j))
            a(:,j) = a(:,j)-u(i,j)*a(:,i)
        end do
        temp = dsqrt(prod(m,a(1,j),a(1,j)))
        u(j,j) = temp
        a(:,j) = a(:,j)/temp
        k = m+1
    end do
!
    * Comprobar dependencia lineal de los vectores columna *
!
    dmax = dmax1(temp,dmax)
    if (dmax+temp==dmax) then
        print *, 'Stop: dependencia lineal de columna ',k
        stop
    endif
    end do
!
    T
!
    *** Resolver Ux=E b ***
!
    x(n) = prod(m,a(1,n),b)/u(n,n)
    do i = n-1,1,-1

```

```

        temp = prod(m,a(1,i),b)
        temp = temp-sum(u(i,i+1:n)*x(i+1:n))
        x(i) = temp/u(i,i)
    end do
!
! *** Residuos: sustraer del vector b sus componentes en la base
! ortonormal que define E ***
!
    res = b
    do j = 1,n
        temp = prod(m,a(1,j),res)
        res(:m) = res(:m)-temp*a(:m,j)
    end do
!
    print *,x,res
!
END PROGRAM Grmsch
!
REAL(kind=2) FUNCTION prod (n,x,y)
!
implicit none
!
integer , intent(in) :: n
real(kind=2), intent(in) :: x(1)
real(kind=2), intent(in) :: y(1)
!
real(kind=2) :: suma
!
suma = 0.d0
if (n==0) then
    prod = 0.d0
else
    suma = dot_product(x(:n),y(:n))
    prod = suma
endif
!
return
END FUNCTION prod

```

11. Programa Qrdes de la página 96.

```

PROGRAM Qrdes
!
implicit none
!
integer, parameter :: m = 4
integer, parameter :: n = 3
!
integer :: j, i, l
real, dimension(m,n) :: a
real, dimension(m) :: b
real, dimension(n) :: d, x, betas
real :: rmm, beta, wj, sigma, s, s1
!
data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16./
data b/2.,3.,5.,6./
!
*** Reducción QA=R ***
!
do j = 1,n
    rmm = 0.0
    rmm = amax1(rmm,maxval(abs(a(j:m,j))))
    if (rmm==0.0) stop 'Matriz A de rango incompleto'
    beta = 0.0
    beta = sum(a(j+1:m,j)**2)
    wj = a(j,j)
    sigma = sign(sqrt(beta+wj*wj),wj)
    wj = wj+sigma
    beta = 2.0/(beta+wj*wj)
    a(j,j) = wj
    d(j) = -sigma
    betas(j) = beta
    do l = j+1,n
        s = beta*sum(a(j:m,j)*a(j:m,l))
        a(j:m,l) = a(j:m,l)-a(j:m,j)*s
    end do
    s = beta*sum(a(j:m,j)*b(j:m))
    b(j:m) = b(j:m)-a(j:m,j)*s
end do
!
*** Resolución Rx = b
!
x(n) = b(n)/d(n)
do i = n-1,1,-1
    x(i) = (b(i)-sum(a(i,i+1:n)*x(i+1:n)))/d(i)
end do
!
*** Suma de residuos al cuadrado
!
s1 = sum(b(n+1:m)**2)
!

```

```

! *** Vector de residuos
!
do i = n,1,-1
    s = betas(i)*sum(a(i+1:m,i)*b(i+1:m))
    b(i) = -a(i,i)*s
    b(i+1:m) = b(i+1:m)-a(i+1:m,i)*s
end do
!
print 50,x
print 60,s1
print 70,b
!
50 format('x=(',f6.4,',',f6.4,',',f6.4,')''')
60 format('Suma de residuos al cuadrado=',f9.6)
70 format('Vector de residuos',4f8.4)
!
END PROGRAM Qrdes

```

12. Programa Mincud de la página 102.

```

PROGRAM Mincud
!
implicit none
!
integer, parameter :: m = 4
integer, parameter :: n = 4
!
integer , dimension(n) :: ipiv
integer :: ira, i, imax, j, k, kp1, l
real(kind=2), dimension(m,n) :: a
real(kind=2), dimension(n,m) :: a1
real(kind=2), dimension(m) :: b, w
real(kind=2), dimension(n,m) :: w1
real(kind=2), dimension(n) :: x
real(kind=2), dimension(m) :: betal
real(kind=2) :: tau, rmax, h, tmp, beta, s1, suma
!
data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16.,1.,4.,9.,16./
data b/2.,3.,5.,6./
data tau/0.000001/
!
!R   R |
*** Reducción QAP=| 11 12| ***
!  |0   0 |
!
ira = min0(m,n)
do i = 1,ira
    imax = i
    rmax = 0.0
    do j = i,n
        h = sum(a(i:m,j)**2)
        if (h>rmax) then
            rmax = h
            imax = j
        endif
    end do
    ipiv(i) = imax
    if (imax/=i) then
        do j = 1,m
            tmp = a(j,i)
            a(j,i) = a(j,imax)
            a(j,imax) = tmp
        end do
        ! Intercambio de columnas.
    endif
    if (i+1<=m) then
        call h1 (beta,i,i+1,m,w,a(1,i)) ! Aplicar trans.
        do j = i+1,n
            call h2 (beta,i,i+1,m,w,a(1,j))! de Householder
        end do
        call h2 (beta,i,i+1,m,w,b) ! a columnas i a N.
        ! Aplicar trans. a b
    endif
end do
!
k = ira
! Calc. rango de A.
do j = 1,ira
    if (dabs(a(j,j))<=tau) then
        k = j-1
        exit
    endif
end do
kp1 = k+1
s1 = sum(b(kp1:m)**2)
a1(:n,:k) = transpose(a(:k,:n))
if (k/=n) then
    ! Residuos al cuadra.
    ! Trasponer matriz A.
    Reducir R a cero y R a T.
    12 11
!
do i = k,1,-1
    call h1 (beta1(i),i,kp1,n,w1(1,i),a1(1,i))
    do j = i-1,1,-1
        call h2 (beta1(i),i,kp1,n,w1(1,i),a1(1,j))
    end do

```

```

      end do
    endif
!
    x(k) = b(k)/a1(k,k)          ! Resolución de Tx=Qb
    do i = k-1,1,-1
      suma = sum(a1(i+1:k,i)*x(i+1:k))
      x(i) = (b(i)-suma)/a1(i,i)
    end do
!
    if (k/=n) then
      x(kp1:n) = 0.0             ! Aplicar trans de
                                ! reduc. de R a
      do i = 1,k
        call h2 (beta1(i),i,kp1,n,w1(1,i),x) ! x. 12
      end do
    endif
    do j = ira,1,-1
      if (ipiv(j)/=j) then
        l = ipiv(j)              ! Deshacer permutación intro-
                                ! ducida por pivotaciones.
        tmp = x(l)
        x(l) = x(j)
        x(j) = tmp
      endif
    end do
!
    print '(,' Rango de A:',I3)',k
    print '(,' Solución:',f68.4)',x
    print '(,' Suma de residuos al cuadrado:',f9.6)',s1
!
    END PROGRAM Mincuaad

    SUBROUTINE h1 (beta,i,j,m,w,x)
!
    implicit none
!
    integer , intent(in) :: i
    integer , intent(in) :: j
    integer , intent(in) :: m
    real(kind=2) , intent(inout) :: beta
    real(kind=2) , intent(inout) :: w(m)
    real(kind=2) , intent(inout) :: x(m)
!
    real :: sigma
!
    Construir transformación
!
    w(j:m) = x(j:m)
    beta = dot_product(w(j:m),w(j:m))
    w(i) = x(i)
    sigma = sign(sqrt(beta+w(i)*w(i)),x(i))
    w(i) = w(i)+sigma
    beta = 2.0/(beta+w(i)*w(i))
    x(i) = -sigma
!
    return
    END SUBROUTINE h1

    SUBROUTINE h2(beta,i,j,m,w,x)
!
    implicit none
!
    integer , intent(in) :: i
    integer , intent(in) :: j
    integer , intent(in) :: m
    real(kind=2) , intent(in) :: beta
    real(kind=2) , intent(in) :: w(m)
    real(kind=2) , intent(inout) :: x(m)
!
    real(kind=2) :: s
!
    Aplicar transformación de Householder.
!
    s = beta*(w(i)*x(i)+dot_product(w(j:m),x(j:m)))
    x(i) = x(i)-w(i)*s
    x(j:m) = x(j:m)-w(j:m)*s
!
    return
    END SUBROUTINE h2

```

13. Programa Givens de la página 107.

```

    PROGRAM Givens
!
    implicit none
!
    integer, parameter :: m = 4
    integer, parameter :: n = 3
!
    integer :: i, k, j
    real, dimension(m,n) :: a
    real, dimension(m) :: b
    real, dimension(n) :: x

```

```

    real :: t, s, c, q, suma
!
    data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16./
    data b/2.,3.,5.,6./
!
    *** Reducción QA=R ***
!
    do i = 1,n
      do k = i+1,m
        if (1.0+abs(a(k,i))/=1.0) then
          if (abs(a(k,i))>=abs(a(i,i))) then
            t = a(i,i)/a(k,i)
            s = 1.0/sqrt(1.0+t*t)
            c = s*t
          else
            t = a(k,i)/a(i,i)
            c = 1.0/sqrt(1.0+t*t)
            s = c*t
          endif
          a(i,i) = c*a(i,i)+s*a(k,i)
          do j = i+1,n
            q = c*a(i,j)+s*a(k,j)
            a(k,j) = ((-s*a(i,j))+c*a(k,j))
            a(i,j) = q
          end do
          q = c*b(i)+s*b(k)
          b(k) = ((-s*b(i))+c*b(k))
          b(i) = q
        endif
      end do
    end do
!
    *** Resolución Rx = b ***
!
    x(n) = b(n)/a(n,n)
    do i = n-1,1,-1
      suma = sum(a(i,i+1:n)*x(i+1:n))
      x(i) = (b(i)-suma)/a(i,i)
    end do
!
    *** Suma de residuos al cuadrado
!
    s = dot_product(b(n+1:m),b(n+1:m))
!
    print 50,x
    print 60,s
!
    50 format(' X=(',f6.4,',',f6.4,',',f6.4,')')
    60 format(' Suma de residuos al cuadrado=',f9.6)
!
    END PROGRAM Givens

```

14. Programa Fastgivens de la página 112.

```

    PROGRAM Fastgivens
!
    implicit none
!
    integer, parameter :: m = 4
    integer, parameter :: n = 3
!
    integer :: i, j, k
    real, dimension(m,n) :: a
    real, dimension(m) :: b
    real, dimension(n) :: x
    real, dimension(m) :: d
    real :: c, s, r2, r1, t, sqrd, suma
!
    data a/1.,1.,1.,1.,1.,2.,3.,4.,1.,4.,9.,16./
    data b/2.,3.,5.,6./
!
    *** Reducción QA=R ***
!
    d(:m) = 1.0
    do j = 1,n
      do i = j+1,m
        if (1.0+abs(a(i,j))/=1.0) then
          c = d(j)*a(j,j)**2
          s = d(i)*a(i,j)**2
          if (s<=c) then
            r2 = a(i,j)/a(j,j)
            r1 = d(i)*r2/d(j)
            c = c/(s+c)
            d(j) = c*d(j)
            d(i) = c*d(i)
            do k = j,n
              t = a(j,k)+r1*a(i,k)
              a(i,k) = a(i,k)-r2*a(j,k)
              a(j,k) = t
            end do
            t = b(j)+r1*b(i)

```

```

        b(i) = b(i)-r2*b(j)
        b(j) = t
    else
        r2 = a(j,j)/a(i,j)
        r1 = d(j)*r2/d(i)
        s = s/(s+c)
        t = d(j)
        d(j) = d(i)
        d(i) = t
        d(j) = s*d(j)
        d(i) = s*d(i)
        do k = j,n
            t = a(i,k)+r1*a(j,k)
            a(i,k) = a(j,k)-r2*a(i,k)
            a(j,k) = t
        end do
        t = b(i)+r1*b(j)
        b(i) = b(j)-r2*b(i)
        b(j) = t
    endif
end do
end do
do i = 1,m
    sqrd = sqrt(d(i))
    a(i,i:n) = sqrd*a(i,i:n)
    b(i) = sqrd*b(i)
end do
!
! *** Resolución Rx = b ***
!
x(n) = b(n)/a(n,n)
do i = n-1,1,-1
    suma = sum(a(i,i+1:n)*x(i+1:n))
    x(i) = (b(i)-suma)/a(i,i)
end do
!
! *** Suma de residuos al cuadrado ***
!
s = dot_product(b(n+1:m),b(n+1:m))
!
print 50,x
print 60,s
!
50 format(' x=(',f6.4,',',f6.4,',',f6.4,')')
60 format(' Suma de residuos al cuadrado=',f9.6)
!
END PROGRAM Fastgivens

```

15. Programa Svdre y rutina dcmsvd de la página 128.

```

PROGRAM Svdre
!
implicit none
!
integer, parameter :: m = 5
integer, parameter :: n = 3
!
integer :: ns, j
real, dimension(m,n) :: a
real, dimension(n) :: sv
real, dimension(m,n) :: v
real, dimension(m) :: b
real, dimension(n) :: x
real, dimension(m) :: tmp
real :: sm, sp, s
!
data a/1.,2.,3.,4.,5.,6.,7.,8.,9.,10.,11.,12.,13.,14.,15./
data b/5.,5.,5.,5.,5./
!
call dcmsvd (a,m,n,sv,v)
!
ns = 0
sm = 0.0
sm = amax1(sm,maxval(sv(:n)))
sp = sm*1.0E-6
do j = 1,n
    s = 0.0
    if (sv(j)>sp) then
        ns = ns+1
        s = sum(a(:,j)*b(:m))
        s = s/sv(j)
    else
        sv(j) = 0.0
    endif
    tmp(j) = s
end do
do j = 1,n

```

```

        s = 0.0
        s = sum(v(j,:n)*tmp(:n))
        x(j) = s
    end do
!
    print '( Rango de A:',I3)',ns
    print '( Solución:',3(F11.7:',','))',x
    print '( Valores singulares de A:',3(F11.7:',','))',sv
!
end program svdre
!
SUBROUTINE dcmsvd(a,m,n,sv,v)
!
implicit none
!
integer, intent(in) :: m
integer, intent(in) :: n
real, intent(inout) :: a(m,n)
real, intent(inout) :: sv(n)
real, intent(inout) :: v(m,n)
!
integer :: i, l, k, j, its, nm, jj
real, dimension(20) :: rv1
real :: g, anorm, s, rmax, f, h, c, y, z, x
!
g = 0.0
anorm = 0.0
!
*** Reducir matriz A a matriz bidiagonal.
!
do i = 1,n
    l = i+1
    rv1(i) = g
    g = 0.0
    s = 0.0
    if (i<=m) then
        rmax = 0.0
        rmax = amax1(rmax,maxval(abs(a(i:m,i))))
        if (rmax/=0.0) then
            s = sum(a(i:m,i)**2)
            f = a(i,i)
            g = -sign(sqrt(s),f)
            h = f*g-s
            a(i,i) = f-g
            do j = l,n
                s = 0.0
                s = sum(a(i:m,i)*a(i:m,j))
                f = s/h
                a(i,m,j) = a(i,m,j)+f*a(i:m,i)
            end do
        endif
    endif
    sv(i) = g
    g = 0.0
    s = 0.0
    if (i<=m.and.i/=n) then
        rmax = 0.0
        rmax = amax1(rmax,maxval(abs(a(i,l:n))))
        if (rmax/=0.0) then
            s = sum(a(i,l:n)**2)
            f = a(i,l)
            g = -sign(sqrt(s),f)
            h = f*g-s
            a(i,l) = f-g
            rv1(l:n) = a(i,l:n)/h
            do j = l,m
                s = sum(a(j,l:n)*a(i,l:n))
                a(j,l:n) = a(j,l:n)+s*rv1(l:n)
            end do
        endif
    endif
    anorm = amax1(anorm,abs(sv(i))+abs(rv1(i)))
end do
!
*** Acumular en la matriz V las transformaciones
! por la derecha hechas a A. ***
!
do i = min(m,n),1,-1
    if (i<n) then
        if (g/=0.0) then
            v(l:n,i) = (a(i,l:n)/a(i,l))/g
            do j = l,n
                s = sum(a(i,l:n)*v(l:n,j))
                v(l:n,j) = v(l:n,j)+s*v(l:n,i)
            end do
        endif
        v(i,l:n) = 0.0
        v(l:n,i) = 0.0
    endif
    v(i,i) = 1.0
    g = rv1(i)
    l = i
end do

```

```

!
! *** Acumular en la matriz A las transformaciones
! por la izquierda hechas a A.
!
do i = n,1,-1
  l = i+1
  g = sv(i)
  a(i,l:n) = 0.0
  if (g/=0.0) then
    g = 1.0/g
    do j = l,n
      s = 0.0
      s = sum(a(l:m,i)*a(l:m,j))
      f = (s/a(i,i))*g
      a(i:m,j) = a(i:m,j)+f*a(i:m,i)
    end do
    a(i:m,i) = a(i:m,i)*g
  else
    a(i:m,i) = 0.0
  endif
  a(i,i) = a(i,i)+1.0
end do

!
! *** Diagonalizar la matriz bidiagonal almacenada en sv(.) y en
! rv1(.). Sólo se realizan 30 iteraciones como máximo.
!
do k = n,1,-1
  do its = 1,30
    do l = k,1,-1
      nm = l-1
      if (abs(rv1(l))+anorm==anorm) exit
      if (abs(sv(nm))+anorm==anorm) then
        c = 0.0
        s = 1.0
        do i = l,k
          f = s*rv1(i)
          rv1(i) = c*rv1(i)
          if (abs(f)+anorm==anorm) exit
          g = sv(i)
          h = sqrt(f*f+g*g)
          sv(i) = h
          c = g/h
          s = -f/h
          do j = 1,m
            y = a(j,nm)
            z = a(j,i)
            a(j,nm) = y*c+z*s
            a(j,i) = ((-y*s))+z*c
          end do
        end do
        exit
      endif
    end do
    z = sv(k)
    if (l==k) then
      if (z<0.0) then
        sv(k) = -z
        v(:n,k) = -v(:n,k)
      endif
    endif
    exit
  end if
  if (its==30) stop 'No hay convergencia'
  x = sv(l)
  nm = k-1
  y = sv(nm)
  g = rv1(nm)
  h = rv1(k)
  f = ((y-z)*(y+z)+(g-h)*(g+h))/(2.0*h*y)
  g = sqrt(f*f+1.0)
  f = ((x-z)*(x+z)+h*(y/(f+sign(g,f))-h))/x
  c = 1.0
  s = 1.0
  do j = l,nm
    i = j+1
    g = rv1(i)
    y = sv(i)
    h = s*g
    g = c*g
    z = sqrt(f*f+h*h)
    rv1(j) = z
    c = f/z
    s = h/z
    f = x*c+g*s
    g = ((-x*s))+g*c
    h = y*s
    y = y*c
    do jj = 1,n
      x = v(jj,j)
      z = v(jj,i)
      v(jj,j) = x*c+z*s
      v(jj,i) = ((-x*s))+z*c
    end do
    z = sqrt(f*f+h*h)
  end do
end do

```

```

sv(j) = z
if (z/=0.0) then
  c = f/z
  s = h/z
endif
f = c*g+s*y
x = ((-s*g))+c*y
do jj = 1,m
  y = a(jj,j)
  z = a(jj,i)
  a(jj,j) = y*c+z*s
  a(jj,i) = ((-y*s))+z*c
end do
end do
rv1(l) = 0.0
rv1(k) = f
sv(k) = x
end do
end do
return
END SUBROUTINE dcmsvd

```

16. Programa Mci de la página 134.

```

PROGRAM Mci
!
implicit none
!
integer, parameter :: m1 = 4
integer, parameter :: m2 = 2
integer, parameter :: n = 3
!
integer, dimension(n) :: ipiv
integer :: ira, i, imax, j, k, n1, l
real(kind=2), dimension(m1,n) :: a
real(kind=2), dimension(m1) :: b
real(kind=2), dimension(m2,n) :: c
real(kind=2), dimension(m2) :: d
real(kind=2), dimension(m1) :: w
real(kind=2), dimension(n) :: x
real(kind=2) :: tau, rmax, h, tmp, beta, s
!
data a/0.2113,0.0824,0.7599,0.0087,0.4524,0.8075,0.4832, &
0.6135,0.6538,0.4899,0.7741,0.9626/
data b/3.0775,3.1671,4.0485,4.1237/
data c/0.8096,0.8474,0.2749,0.8807,0.9933,0.8360/
data d/4.3393,5.1169/
data tau/0.000001/
!
*** Reducción QCP=| 11 12| ***
!
|R R |
|0 0 |
!
ira = min0(m2,n)
do i = 1,ira
  imax = i
  rmax = 0.0
  do j = i,n
    ! Búsqueda de columna con
    h = sum(c(i:m2,j)**2) ! mayor norma euclídea
    if (h>rmax) then ! en componentes I a N de
      rmax = h ! matriz C.
      imax = j
    endif
  end do
  ipiv(i) = imax
  if (imax/=i) then
    do j = 1,m2
      ! Intercambio de columnas:
      tmp = c(j,i) !
      c(j,i) = c(j,imax) ! en matriz C.
      c(j,imax) = tmp !
    end do
    do j = 1,m1
      ! -----
      tmp = a(j,i) !
      a(j,i) = a(j,imax) ! en matriz A.
      a(j,imax) = tmp !
    end do
  endif
  if (i+1.le.m2) then
    call h1 (beta,i,i+1,m2,w,c(1,i)) ! Aplicar transf.
    do j = i+1,n ! de Householder a
      call h2 (beta,i,i+1,m2,w,c(1,j))! columnas i a n
    end do ! de la matriz C.
    call h2 (beta,i,i+1,m2,w,d) ! Aplicar tra. a d.
  endif
end do
!
k = ira ! Calc. rango de C.
do j = 1,ira
  if (dabs(c(j,j))<=tau) then
    k = j-1
    exit
  endif
end do
end do

```



```

!
do i = 1,m1
  a(i,1) = a(i,1)/c(1,1)      ! Determinar A y B
  do j = 2,ira
    s = sum(a(i,:j-1)*c(:j-1,j))
    a(i,j) = (a(i,j)-s)/c(j,j)
  end do
  do j = ira+1,n
    s = sum(a(i,:ira)*c(:ira,j))
    a(i,j) = a(i,j)-s
  end do
  s = 0.0
  k = 1
  if (ira>0) then
    s = sum(a(i,:ira)*d(:ira))
    k = ira+1
  endif
  b(i) = b(i)-s
end do

!
do i = ira+1,n
  k = i-ira
  call h1 (beta,k,k+1,m1,w,a(1,i)) ! Aplicar trans. de Householder a
  do j = i+1,n
    call h2 (beta,k,k+1,m1,w,a(1,j)) ! columnas ira+1 a N
  end do
  call h2 (beta,k,k+1,m1,w,b)      ! de matriz A; es decir a A^2
end do
! Aplicar trans. a b.

n1 = n-ira
x(n) = b(n1)/a(n1,n)
do i = n1-1,1,-1
  s = sum(a(i,i+1+ira:n1+ira)*x(i+1+ira:n1+ira))
  x(i+ira) = (b(i)-s)/a(i,i+ira)
end do
! |R R || |D |
! | 11 12||x|=| 1 |
do i = ira,1,-1
  s = sum(c(i,i+1:n)*x(i+1:n))
  x(i) = (d(i)-s)/c(i,i)
end do
! | 0 R || |C |
! | 22|| | 1 |

do j = ira,1,-1
  if (ipiv(j) /= j) then
    l = ipiv(j)
    tmp = x(l)
    x(l) = x(j)
    x(j) = tmp
  endif
end do

!
print '( " Rango de C: ",I3)',k
print '( " Solución: ", 6(F8.4: ', '))',x

!
END PROGRAM Mci

SUBROUTINE h1(beta,i,j,m,w,x)
!
implicit none
!
integer, intent(in) :: i
integer, intent(in) :: j
integer, intent(in) :: m
real(kind=2), intent(inout) :: beta
real(kind=2), intent(inout) :: w(m)
real(kind=2), intent(inout) :: x(m)

!
real :: sigma

beta = 0.0
w(j:m) = x(j:m)
beta = dot_product(w(j:m),w(j:m))
w(i) = x(i)
sigma = sign(sqrt(beta+w(i)*w(i)),x(i))
w(i) = w(i)+sigma
beta = 2.0/(beta+w(i)*w(i))
x(i) = -sigma

!
return
END SUBROUTINE h1

SUBROUTINE h2(beta,i,j,m,w,x)
!
implicit none
!
integer, intent(in) :: i
integer, intent(in) :: j
integer, intent(in) :: m
real(kind=2), intent(in) :: beta
real(kind=2), intent(in) :: w(m)
real(kind=2), intent(inout) :: x(m)

!
real(kind=2) :: s

s = w(i)*x(i)

```

```

s = s+dot_product(w(j:m),x(j:m))
s = s*beta
x(i) = x(i)-w(i)*s
x(j:m) = x(j:m)-w(j:m)*s

!
return
END SUBROUTINE h2

```

G.2.2 Códigos del capítulo 2

1. Programa Jacobi de la página 147.

```

PROGRAM Jacobi
!
implicit none
!
integer, parameter :: n = 4
!
integer :: i
real, dimension(n,n) :: a
real, dimension(n) :: b, x, y
real :: s1, su, sm

!
data a/10.,-1.,2.,0.,-1.,11.,-1.,3.,2.,-1.,10.,-1.,0.,3., &
-1.,8./
data b/6.,25.,-11.,15./
data sm/1.0/

!
x = 0.

!
*** Proceso iterativo ***
!
do while (sm>=0.001)
  s1 = 0.0
  do i = 1,n
    su = b(i)-sum(a(i,:i-1)*x(:i-1))- &
sum(a(i,i+1:n)*x(i+1:n))
    y(i) = su/a(i,i)
    s1 = amax1(s1,abs(y(i)))
  end do
  sm = maxval(abs(x(n)-y(:n))/s1)
  x = y
  print *,x
! Salida de resultados
end do

END PROGRAM Jacobi

```

2. Programa GaussSeidel de la página 151.

```

PROGRAM GaussSeidel
!
implicit none
!
integer, parameter :: n = 4
!
integer :: i
real, dimension(n,n) :: a
real, dimension(n) :: b, x
real :: s1, su, sm, xi

!
data a/10.,-1.,2.,0.,-1.,11.,-1.,3.,2.,-1.,10.,-1.,0., &
3.,-1.,8./
data b/6.,25.,-11.,15./
data sm/1.0/

!
x = 0.

!
*** Proceso iterativo ***
!
do while (sm>=0.001)
  s1 = 0.
  sm = 0.
  do i = 1,n
    su = b(i)-sum(a(i,:i-1)*x(:i-1))
    xi = x(i)+su/a(i,i)
    sm = amax1(sm,abs(x(i)-xi))
    x(i) = xi
    s1 = amax1(s1,abs(x(i)))
  end do
  sm = sm/s1
  print *,x
! Salida de resultados
end do

END PROGRAM GaussSeidel

```

3. Programa Sor de la página 165.

```

PROGRAM Sor
!
! implicit none
!
! integer, parameter :: n = 3
!
! integer :: i
! real, dimension(n,n) :: a
! real, dimension(n) :: b, x
! real :: s1, su, sm, xi, w
!
! data a/4.,3.,0.,3.,4.,-1.,0.,-1.,4./
! data b/24.,30.,-24./
! data sm/1.0/
!
! x = 1.
! w = 1.25
!
! *** Proceso iterativo ***
!
! do while (sm>=0.001)
!   s1 = 0.
!   sm = 0.
!   do i = 1,n
!     su = b(i)-sum(a(i,:i-1)*x(:i-1))- &
!           sum(a(i,i+1:n)*x(i+1:n))
!     xi = (1-w)*x(i)+w*su/a(i,i)
!     sm = amax1(abs(x(i)-xi),sm)
!     x(i) = xi
!     s1 = amax1(s1,abs(x(i)))
!   end do
!   sm = sm/s1
!   print *,x
! end do
!
! END PROGRAM Sor

```

4. Programa Steep de la página 176.

```

PROGRAM Steep
!
! implicit none
!
! integer, parameter :: n = 50
!
! integer :: k, i
! real(kind=2), dimension(n,n) :: a
! real(kind=2), dimension(n) :: b, x, r
! real(kind=2) :: ro1, xnormb, rar
!
! open(10,file='stp.dat')
!
! read (10,*) a,b
!
! x = 0.0
! r = b
! ro1 = dot_product(r,r)
!
! *** Proceso iterativo ***
!
! xnormb = epsilon(1.0)*dsqrt(ro1)*5
! k = 1
! do while (dsqrt(ro1)>xnormb)
!   rar = 0.0
!   do i = 1,n
!     rar = rar+r(i)*dot_product(a(1:n,i),r)
!   end do
!   x = x+(ro1/rar)*r
!   do i = 1,n
!     r(i) = b(i)-dot_product(a(1:n,i),x)
!   end do
!   ro1 = dot_product(r,r)
!   k = k+1
!   print *,k,ro1 ! Resultados de iteraciones
! end do
!
! print '(10f8.5)',x ! Solución
! END PROGRAM Steep

```

5. Programa Cg de la página 188.

```

PROGRAM Cg
!
! implicit none
!
! integer, parameter :: n = 6
!
! integer :: k, i

```

```

real, dimension(n,n) :: a
real, dimension(n) :: b, x, r, p, w
real :: ro0, ro1, xnormb, betak, alfak
!
! data a/4.,-1.,0.,-1.,0.,0.,-1.,4.,-1.,0.,-1.,0.,0.,-1., &
!       4.,0.,0.,-1.,-1.,0.,0.,4.,-1.,0.,0.,-1.,0.,-1.,4., &
!       -1.,0.,0.,-1.,0.,-1.,4./
! data b/0.,5.,0.,6.,-2.,6./
!
! x = 0.
! r = b
! ro0 = dot_product(r,r)
! ro1 = ro0
!
! *** Proceso iterativo ***
!
! xnormb = epsilon(1.0)*sqrt(ro0)*5
! k = 0
! do while(sqrt(ro1)>xnormb)
!   if (k==0) then
!     p = r
!   else
!     betak = ro1/ro0
!     p = r+betak*p
!   endif
!   do i = 1,n
!     w(i) = dot_product(a(1:n,i),p)
!   end do
!   alfak = ro1/dot_product(p,w)
!   x = x+alfak*p
!   r = r-alfak*w
!   ro0 = ro1
!   ro1 = dot_product(r,r)
!   k = k+1
!   print *,k,x
! end do
!
! END PROGRAM Cg

```

6. Programa Cgp de la página 192.

```

PROGRAM Cgp
!
! implicit none
!
! integer, parameter :: n = 40
!
! integer :: i, j, k
! real :: ra
! real(kind=2), dimension(n,n) :: a
! real(kind=2), dimension(n) :: b, x, r, p, w, m, z, zm2, rm2
! real(kind=2), dimension(n) :: aux
! real(kind=2), dimension(n) :: baux
! real(kind=2) :: xnormb, ro1, betak, alfak
!
! do i = 1,n
!   do j = 1,n
!     ! Generación aleatoria
!     ! de un problema con
!     call random_number (ra)
!     ! solución = | 1 |
!     aux(i,j) = dble(ra)
!     ! | 2 |
!   end do
!   ! | 3 |
!   baux(i) = dble(i)
!   ! | . |
! end do
! ! | . |
! do i = 1,n
!   ! T
!   do j = 1,n
!     ! A=AUX *AUX
!     a(i,j) = dot_product(aux(1:n,i),aux(1:n,j))
!   end do
! end do
! do i = 1,n
!   b(i) = dot_product(a(1:n,i),baux)
! end do
!
! do i = 1,n
!   ! Obtención del preconditionador
!   m(i) = dsqrt(dot_product(a(1:n,i),a(1:n,i)))
! end do
!
! *** Proceso iterativo ***
!
! xnormb = epsilon(1.00)*1000.*sqrt(dot_product(b,b))
! x = 0.
! r = b
! k = 0
! ro1 = dot_product(b,b)
! do while (dsqrt(ro1)>xnormb)
!   z = r/m
!   if (k==0) then
!     p = z
!   else
!     betak = dot_product(z,r)/dot_product(zm2,rm2)
!     p = z+betak*p
!   endif
!   do i = 1,n

```

```

        w(i) = dot_product(a(1:n,i),p)
    end do
    alfab = dot_product(z,r)/dot_product(p,w)
    x = x+alfak*p
    rm2 = r
    r = r-alfak*w
    zm2 = z
    ro1 = dot_product(r,r)
    k = k+1
    print *,k,x,ro1          ! Salida de resultados
end do
!
END PROGRAM Cgp

```

```

!
tol = epsilon(1.0)
x0 = 0
x = 1
do while (abs(x-x0)>tol)
    x0 = x
    x = x0-(x0**3-sin(x0))/(3*x0*x0-cos(x0))
    print '(f10.7)',x          ! Salida de resultados
end do
END PROGRAM Newt

```

G.2.3 Códigos del capítulo 3

No creemos que tenga especial interés listar la versión en FORTRAN 90 de los programas del capítulo en FORTRAN 77. Las versiones adecuadas se deberán parecer bastante a las de C listadas anteriormente.

G.2.4 Códigos del capítulo 4

1. Programa Bisec de la página 285.

```

PROGRAM Bisec
implicit none
!
real :: a, b, fa, fb, tol, c, fc
!
real , external :: fx
!
*** Resolución de la ecuación x*sin(x)-1=0 ***
!
data a/1.0/
data b/2.0/
!
fa = fx(a)
fb = fx(b)
if (fa*fb>0) stop 'El intervalo [a,b] no contiene solución'
!
tol = epsilon(1.0)*10
do while (abs(a-b)>tol)
    c = (a+b)/2.
    fc = fx(c)
    if (fc==0) then
        a = c
        b = c
    else if (fb*fc>0) then
        b = c
        fb = fc
    else
        a = c
        fa = fc
    endif
    print '(2f10.7)',a,b
end do
!
END PROGRAM Bisec
!
REAL FUNCTION fx (x)
!
implicit none
!
real , intent(in) :: x
!
fx = x*sin(x)-1
return
END FUNCTION fx

```

2. Programa Newt de la página 290.

```

!
PROGRAM Newt
!
implicit none
!
real :: x, x0, tol

```

3. Programa Newton de la página 298.

```

PROGRAM Newton
!
implicit none
!
real(kind=2), parameter :: eps = epsilon(1.D0)
!
real(kind=2) :: x1 = 2., x0 = 0.
!
real(kind=2) , external :: fx, derfx
!
do while (dabs(fx(x1))>eps)
    x0 = x1
    x1 = x0-fx(x0)/derfx(x0)
    print *,x1
end do
!
END PROGRAM Newton
!
REAL(kind=2) FUNCTION fx(x)
!
implicit none
!
real(kind=2) , intent(in) :: x
!
fx = x**2-1.
return
END FUNCTION fx
!
REAL(kind=2) FUNCTION derfx(x)
!
implicit none
!
real(kind=2) , intent(in) :: x
!
derfx = 2.0*x
return
END FUNCTION derfx

```

4. Programa Newtondf de la página 298.

```

PROGRAM Newtondf
!
implicit none
!
real(kind=2), parameter :: eps = epsilon(1.d0)
!
real(kind=2) :: h, x1 = 2., x0 = 0.
!
real(kind=2) , external :: fx, derfx
!
h = dsqrt(eps)
do while (dabs(fx(x1))>eps)
    x0 = x1
    x1 = x0-fx(x0)/derfx(x0,h)
    print *,x1
end do
!
end program newtondf
!
REAL(kind=2) FUNCTION fx(x)
!
implicit none
!
real(kind=2) , intent(in) :: x
!
fx = x**2-1.
return
END FUNCTION fx
!
REAL(kind=2) FUNCTION derfx(x,h)
!
implicit none
!
real(kind=2) x
real(kind=2) , intent(in) :: h
!

```

```

      real(kind=2) , external :: fx
!
      derfx = (fx(x+h)-fx(x))/h
      return
      END FUNCTION derfx

```

5. Programa Newtonmod de la página 299.

```

      PROGRAM Newtonmod
!
      implicit none
!
      real(kind=2), parameter :: eps = epsilon(1.d0)
!
      real(kind=2) :: x1, dx, x2
!
      real(kind=2) , external :: fx
!
! *** Resolución de la ecuación x**3-sen(x)=0 ***
!
      x1 = 1.
      dx = 3.0*x1*x1-dcos(x1)
      x2 = x1-fx(x1)/dx
!
      do while (dabs(fx(x2))>eps)
!
         print *,x1                      ! Salida de resultados
         x1 = x2
         x2 = x1-fx(x1)/dx
      end do
!
      end program newtonmod
!
      REAL(kind=2) FUNCTION fx(x)
!
      implicit none
!
      real(kind=2) , intent(in) :: x
!
      fx = x**3-dsin(x)
      return
      END FUNCTION fx

```

6. Programa Newtonsecante de la página 301.

```

      PROGRAM Newtonsecante
!
      implicit none
!
      real(kind=2), parameter :: eps = epsilon(1.D0)
!
      real(kind=2) :: x0, x1, x2
!
      real(kind=2) , external :: fx, secfx
!
! *** Resolución de la ecuación x**3-sen(x)=0 ***
!
      x0 = 1.1
      x1 = 1.0
      x2 = x1-fx(x1)/secfx(x0,x1)
!
      do while (dabs(fx(x2))>eps)
!
         x0 = x1
         x1 = x2
         x2 = x1-fx(x1)/secfx(x0,x1)
         print *,x2                      ! Salida de resultados
      end do
!
      END PROGRAM Newtonsecante
!
      REAL(kind=2) FUNCTION fx (x)
!
      implicit none
!
      real(kind=2) , intent(in) :: x
!
      fx = x**3-dsin(x)
      return
      END FUNCTION fx
!
      REAL(kind=2) FUNCTION secfx (x0,x1)
!
      implicit none
!
      real(kind=2) x0
      real(kind=2) x1
!
      real(kind=2) , external :: fx

```

```

      secfx = (fx(x1)-fx(x0))/(x1-x0)
      return
      END FUNCTION secfx

```

7. Programa Muller de la página 305.

```

      PROGRAM Muller
!
      implicit none
!
      real :: x0, x1, x2, fx0, fx1, fx2, eps, c, d0, d1, det, &
         b, a, di, z, x3, u
!
      real, external :: fx
!
! *** Resolución de la ecuación x**3-sen(x)=0 ***
!
      data x0/1.5/
      data x1/1.2/
      data x2/1.0/
!
      fx0 = fx(x0)
      fx1 = fx(x1)
      fx2 = fx(x2)
!
      eps = epsilon(1.0)
      do while(abs(fx2)>eps)
!
         c = fx2
         d0 = x0-x2
         d1 = x1-x2
         det = d0*d1*(x0-x1)
         b = (d0*d0*(fx1-fx2)-d1*d1*(fx0-fx2))/det
         a = (d1*(fx0-fx2)-d0*(fx1-fx2))/det
         di = 0.
         if (b*b-4*a*c.ge.0) di = sqrt(b*b-4*a*c)
         z = (-2)*c/(b+sign(1.0,b)*di)
         x3 = x2+z
         if (abs(x3-x1)<abs(x3-x0)) then ! Escoger como nuevos
            u = x1                      ! x0, x1 y x2 los
            x1 = x0                      ! más próximos a
            x0 = u                        ! x3.
            u = fx1
            fx1 = fx0
            fx0 = u
         endif
         if (abs(x3-x2)<abs(x3-x1)) then
            u = x2
            x1 = u
            u = fx2
            fx1 = u
         endif
         x2 = x3
         fx2 = fx(x2)
         print '(F9.7)',x2
      end do
!
      END PROGRAM Muller
!
      REAL FUNCTION fx(x)
!
      implicit none
!
      real, intent(in) :: x
!
      fx = x**3-sin(x)
      return
      END FUNCTION fx

```

8. Programa Newtrp de la página 307.

```

      PROGRAM Newtrp
!
      implicit none
!
      integer, parameter :: n = 3
!
      real(kind=2), dimension(n) :: f
      real(kind=2), dimension(n,n) :: j
      real(kind=2), dimension(n) :: x, x1, s
      real(kind=2) :: tol, dnr
!
      real(kind=2) , external :: dnor
!
      tol = dsqrt(epsilon(1.0d0))
      x = 1.0
      call fx (f,x,n)
      dnr = dnor(f,n)
!

```

```

! *** Proceso iterativo ***
!
do while (dnr>=tol)
  call derfx (j,x,n)
  call gauss (j,f,s,n)
  x1 = x-s
  call fx (f,x1,n)
  dnr = dnor(f,n)
  print *,x1,dnr          ! Salida de resultados
  x = x1
end do
!
END PROGRAM Newtrp

SUBROUTINE fx(f,x,n)
!
implicit none
!
integer , intent(in) :: n
real(kind=2) , intent(out) :: f(n)
real(kind=2) , intent(in) :: x(n)
!
f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
f(2) = x(1)**2-81*(x(2)+0.1)**2+dsin(x(3))+1.06
f(3) = dexp((-x(1)*x(2)))+20*x(3)+(10*dcos(-1.0d0)-3)/3
!
return
END SUBROUTINE fx

SUBROUTINE derfx(j,x,n)
!
implicit none
!
integer , intent(in) :: n
real(kind=2) , intent(out) :: j(n,n)
real(kind=2) , intent(in) :: x(n)
!
j(1,1) = 3.0
j(1,2) = dsin(x(2)*x(3))*x(3)
j(1,3) = dsin(x(2)*x(3))*x(2)
j(2,1) = 2.0*x(1)
j(2,2) = -162.0*(x(2)+0.1)
j(2,3) = dcos(x(3))
j(3,1) = -dexp((-x(1)*x(2)))*x(2)
j(3,2) = -dexp((-x(1)*x(2)))*x(1)
j(3,3) = 20.0
!
return
END SUBROUTINE derfx

REAL(kind=2) FUNCTION dnor(x,n)
!
implicit none
!
integer , intent(in) :: n
real(kind=2) , intent(in) :: x(n)
!
dnor = sum(x**2)
!
dnor = dsqrt(dnor)
return
END FUNCTION dnor

SUBROUTINE gauss(a,b,x,n)
!
implicit none
!
integer, intent(in) :: n
real(kind=2) , intent(inout) :: a(n,n)
real(kind=2) , intent(inout) :: b(n)
real(kind=2) , intent(inout) :: x(n)
!
integer, dimension(10) :: ipvt
integer :: pi, i, k, l, ip,iaux
real(kind=2) :: smax, r, r1, c
!
*** Resolución del sistema lineal mediante eliminación de Gauss
!
do i = 1,n
  ipvt(i) = i
end do
!
*** Triangularización ***
!
do k = 1,n-1
  l = 0
  smax = dabs(a(ipvt(k),k))
  do i = k+1,n
    ip = ipvt(i)
    if (dabs(a(ip,k))>smax) then
      l = i
      smax = dabs(a(ip,k))
    end if
  end do
  if (l/=0) then
    if (l/=0) then
      iaux = ipvt(k)
      ipvt(k) = ipvt(l)
      ipvt(l) = iaux
    end if
    pi = ipvt(k)
    r1 = 1.0/a(pi,k)
    do i = k+1,n
      ip = ipvt(i)
      r = a(ip,k)*r1
      a(ip,k+1:n) = a(ip,k+1:n)-r*a(pi,k+1:n)
      a(ip,k) = -r
    end do
  end do
  do k = 1,n-1
    ip = ipvt(k)
    do i = k+1,n
      pi = ipvt(i)
      b(pi) = b(pi)+a(pi,k)*b(ip)
    end do
  end do
!
*** Sustitución inversa ***
!
x(n) = b(ipvt(n))/a(ipvt(n),n)
do i = n-1,1,-1
  pi = ipvt(i)
  c = b(pi)
  c = c-sum(a(pi,i+1:n)*x(i+1:n))
  x(i) = c/a(pi,i)
end do
!
return
END SUBROUTINE gauss

```

9. Programa Newtrpdf de la página 314.

```

PROGRAM Newtrpdf
!
implicit none
!
integer, parameter :: n = 3
!
integer, dimension(n) :: ipvt
real(kind=2), dimension(n) :: f
real(kind=2), dimension(n,n) :: j
real(kind=2), dimension(n) :: x, x1, f1, s
real(kind=2) :: tol, dnr, h
!
real(kind=2), external :: dnor
!
tol = dsqrt(epsilon(1.0d0))
h = tol
x = 1.0
call fx (f,x,n)
dnr = dnor(f,n)
!
! *** Proceso iterativo ***
!
do while (dnr>tol)
  call derfxdf (j,x,n,f,f1,x1,h)
  call gauss (j,f,s,ipvt,n)
  x1 = x-s
  call fx (f,x1,n)
  dnr = dnor(f,n)
  print *,x1,dnr          ! Salida de resultados
  x = x1
end do
!
END PROGRAM Newtrpdf

SUBROUTINE fx(f,x,n)
!
implicit none
!
integer , intent(in) :: n
real(kind=2) , intent(out) :: f(n)
real(kind=2) , intent(in) :: x(n)
!
f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+
      & (10.0*dcos(-1.0D0)-3.0)/3.0
!
return
END SUBROUTINE fx

SUBROUTINE derfxdf(j,x,n,f,f1,x1,h)

```

```

!
!      implicit none
!
!      integer n
!      real(kind=2) , intent(in) :: h
!      real(kind=2) , intent(out) :: j(n,n)
!      real(kind=2) x(n)
!      real(kind=2) f(n)
!      real(kind=2) f1(n)
!      real(kind=2) x1(n)
!
!      integer :: i
!
!      x1(:n) = x(:n)
!      call fx (f,x,n)
!      do i = 1,n
!         x1(i) = x1(i)+h
!         call fx (f1,x1,n)
!         j(:n,i) = (f1(:n)-f(:n))/h
!         x1(i) = x(i)
!      end do
!
!      return
!      END SUBROUTINE derfxdf
!
!      REAL(kind=2) FUNCTION dnor (x,n)
!
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(in) :: x(n)
!
!      dnor = sum(x**2)
!
!      dnor = dsqrt(dnor)
!      return
!      END FUNCTION dnor
!
!      SUBROUTINE gauss(a,b,x,ipvt,n)
!
!      implicit none
!
!      integer , intent(in) :: n
!      integer , intent(inout) :: ipvt(n)
!      real(kind=2) , intent(inout) :: a(n,n)
!      real(kind=2) , intent(inout) :: b(n)
!      real(kind=2) , intent(inout) :: x(n)
!
!      integer :: pi, i, k, l, ip, iaux
!      real(kind=2) :: smax, r, r1, c
!
!      *** Resolución del sistema lineal mediante eliminación
!      de Gauss ***
!
!      do i = 1,n
!         ipvt(i) = i
!      end do
!
!      *** Triangularización ***
!
!      do k = 1,n-1
!         l = 0
!         smax = dabs(a(ipvt(k),k))
!         do i = k+1,n
!            ip = ipvt(i)
!            if (dabs(a(ip,k))>smax) then
!               l = i
!               smax = dabs(a(ip,k))
!            endif
!         end do
!         if (l/=0) then
!            iaux = ipvt(k)
!            ipvt(k) = ipvt(l)
!            ipvt(l) = iaux
!         endif
!         pi = ipvt(k)
!         r1 = 1.0/a(pi,k)
!         do i = k+1,n
!            ip = ipvt(i)
!            r = a(ip,k)*r1
!            a(ip,k+1:n) = a(ip,k+1:n)-r*a(pi,k+1:n)
!            a(ip,k) = -r
!         end do
!      end do
!
!      do k = 1,n-1
!         ip = ipvt(k)
!         do i = k+1,n
!            pi = ipvt(i)
!            b(pi) = b(pi)+a(pi,k)*b(ip)
!         end do
!      end do
!
!

```

```

!      *** Sustitución inversa ***
!
!      x(n) = b(ipvt(n))/a(ipvt(n),n)
!      do i = n-1,1,-1
!         pi = ipvt(i)
!         c = b(pi)-sum(a(pi,i+1:n)*x(i+1:n))
!         x(i) = c/a(pi,i)
!      end do
!
!      return
!      END SUBROUTINE gauss

```

10. Programa Newjac de la página 317.

```

!      PROGRAM Newjac
!
!      implicit none
!
!      integer, parameter :: n = 3
!
!      real(kind=2), dimension(n) :: f, j, x, x1
!      real(kind=2) :: tol, dnr
!
!      real(kind=2) , external :: dnor
!
!      tol = dsqrt(epsilon(1.0D0))
!      x = 1.0
!      call fx (f,x,n)
!      dnr = dnor(f,n)
!
!      do while(dnr>tol)
!         call derfx (j,x,n)
!         x1 = x-f/j
!         call fx (f,x1,n)
!         dnr = dnor(f,n)
!         print *,x1,dnr
!         x = x1
!      end do
!
!      END PROGRAM Newjac
!
!      SUBROUTINE fx(f,x,n)
!
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(out) :: f(n)
!      real(kind=2) , intent(in) :: x(n)
!
!      f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
!      f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
!      f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+ &
!            (10.0*dacos(-1.0D0)-3.0)/3.0
!
!      return
!      END SUBROUTINE fx
!
!      SUBROUTINE derfx(j,x,n)
!
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(out) :: j(n)
!      real(kind=2) , intent(in) :: x(n)
!
!      j(1) = 3.0
!      j(2) = -162.0*(x(2)+0.1)
!      j(3) = 20.0
!
!      return
!      END SUBROUTINE derfx
!
!      REAL(kind=2) FUNCTION dnor (x,n)
!
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(in) :: x(n)
!
!      dnor = sum(x**2)
!
!      dnor = dsqrt(dnor)
!      return
!      END FUNCTION dnor

```

11. Programa Newsor de la página 319.

```

!      PROGRAM Newsor

```

```

!
!   implicit none
!
!   integer, parameter :: n = 3
!
!   real(kind=2), dimension(n) :: f
!   real(kind=2), dimension(n,n) :: j
!   real(kind=2), dimension(n) :: x, x1, s
!   real(kind=2) :: tol, dnr, omega, ro
!
!   real(kind=2), external :: dnor
!
!   tol = dsqrt(epsilon(1.0D0))
!   x = 1.0
!   print '(a)', 'Valor de OMEGA ---->'
!   read '(bn,f9.0)', omega
!   ro = (1-omega)/omega
!   call fx (f,x,n)
!   dnr = dnor(f,n)
!
!   do while (dnr>tol)
!       call derfx (j,x,n,ro)
!       call sustdi (j,f,s,n)
!       x1 = x-s
!       call fx (f,x1,n)
!       dnr = dnor(f,n)
!       print *,x1,dnr
!       x = x1
!   end do
!
!   END PROGRAM Newsor
!
!   SUBROUTINE fx(f,x,n)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(out) :: f(n)
!   real(kind=2) , intent(in) :: x(n)
!
!   f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
!   f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
!   f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+      &
!         (10.0*dacos(-1.0D0)-3.0)/3.0
!
!   return
!   END SUBROUTINE fx
!
!   SUBROUTINE derfx(j,x,n,ro)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(in) :: ro
!   real(kind=2) , intent(out) :: j(n,n)
!   real(kind=2) , intent(in) :: x(n)
!
!   j(1,1) = 3.0*(1.0+ro)
!   j(2,1) = 2.0*x(1)
!   j(2,2) = -162.0*(x(2)+0.1)*(1.0+ro)
!   j(3,1) = -dexp((-x(1)*x(2)))*x(2)
!   j(3,2) = -dexp((-x(1)*x(2)))*x(1)
!   j(3,3) = 20.0*(1.0+ro)
!
!   return
!   END SUBROUTINE derfx
!
!   REAL(kind=2) FUNCTION dnor (x,n)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(in) :: x(n)
!
!   dnor = sum(x**2)
!
!   dnor = dsqrt(dnor)
!   return
!   END FUNCTION dnor
!
!   SUBROUTINE sustdi(a,b,x,n)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(in) :: a(n,n)
!   real(kind=2) , intent(in) :: b(n)
!   real(kind=2) , intent(inout) :: x(n)
!
!   integer :: i
!   real(kind=2) :: c
!
!   *** Sustitución directa ***

```

```

!   x(1) = b(1)/a(1,1)
!   do i = 2,n
!       c = b(i)
!       c = c-sum(a(i,:i-1)*x(:i-1))
!       x(i) = c/a(i,i)
!   end do
!
!   return
!   END SUBROUTINE sustdi

```

12. Programa Broyden de la página 323.

```

!   PROGRAM Broyden
!
!   implicit none
!
!   integer, parameter :: n = 3
!
!   integer, dimension(n) :: ip
!   real(kind=2), dimension(n) :: f
!   real(kind=2), dimension(n,n) :: j, ja
!   real(kind=2), dimension(n) :: x, x1, f1, y, s
!   real(kind=2) :: tol, dnr
!
!   real(kind=2), external :: dnor
!
!   tol = dsqrt(epsilon(1.0D0))
!   x = 1.d0
!   j(1,1) = 3.d0
!   j(2,2) = -178.2d0
!   j(3,3) = 20.d0
!   call fx (f,x,n)
!   dnr = dnor(f,n)
!
!   *** Proceso iterativo ***
!
!   do while(dnr>tol)
!       f1 = f
!       ja = j
!       call gauss (ja,f,s,ip,n)
!       x1 = x-s
!       call fx (f,x1,n)
!       dnr = dnor(f,n)
!       print *,x1,dnr      ! Salida de resultados
!       y = f-f1
!       call broyd (j,y,s,n)
!       x = x1
!   end do
!
!   END PROGRAM Broyden
!
!   SUBROUTINE fx(f,x,n)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(out) :: f(n)
!   real(kind=2) , intent(in) :: x(n)
!
!   f(1) = 3*x(1)-dcos(x(2)*x(3))-0.5
!   f(2) = x(1)**2-81.0*(x(2)+0.1)**2+dsin(x(3))+1.06
!   f(3) = dexp((-x(1)*x(2)))+20.0*x(3)+      &
!         (10.0*dacos(-1.0D0)-3.0)/3.0
!
!   return
!   END SUBROUTINE fx
!
!   REAL(kind=2) FUNCTION dnor (x,n)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(in) :: x(n)
!
!   dnor = sum(x**2)
!
!   dnor = dsqrt(dnor)
!   return
!   END FUNCTION dnor
!
!   SUBROUTINE broyd(a,y,s,n)
!
!   implicit none
!
!   integer , intent(in) :: n
!   real(kind=2) , intent(inout) :: a(n,n)
!   real(kind=2) , intent(inout) :: y(n)
!   real(kind=2) , intent(in) :: s(n)
!
!   integer :: i

```

```

!      real(kind=2) :: prod
!      prod = sum(s**2)
!      do i = 1,n
!        y(i) = (y(i)+sum(a(i,:)*s))/prod
!      end do
!      a = a-spread(y,dim=2,ncopies=n)*spread(s,dim=1,ncopies=n)
!      return
!      END SUBROUTINE broyd
!
!      SUBROUTINE gauss(a,b,x,ipvt,n)
!      implicit none
!
!      integer , intent(in) :: n
!      integer , intent(inout) :: ipvt(n)
!      real(kind=2) , intent(inout) :: a(n,n)
!      real(kind=2) , intent(inout) :: b(n)
!      real(kind=2) , intent(inout) :: x(n)
!
!      integer :: ip, pi, l, i, k,iaux
!      real(kind=2) :: smax, r, r1, c
!
!      *** Resolución del sistema de ecuaciones lineales
!      *** mediante eliminación de Gauss
!
!      do i = 1,n
!        ipvt(i) = i
!      end do
!
!      *** Triangularización ***
!
!      do k = 1,n-1
!        l = 0
!        smax = dabs(a(ipvt(k),k))
!        do i = k+1,n
!          ip = ipvt(i)
!          if (dabs(a(ip,k))>smax) then
!            l = i
!            smax = dabs(a(ip,k))
!          endif
!        end do
!        if (l/=0) then
!         iaux = ipvt(k)
!          ipvt(k) = ipvt(l)
!          ipvt(l) =iaux
!        endif
!        pi = ipvt(k)
!        r1 = 1.0/a(pi,k)
!        do i = k+1,n
!          ip = ipvt(i)
!          r = a(ip,k)*r1
!          a(ip,k+1:n) = a(ip,k+1:n)-r*a(pi,k+1:n)
!          b(ip) = b(ip)-r*b(pi)
!        end do
!      end do
!
!      *** Sustitución inversa ***
!
!      x(n) = b(ipvt(n))/a(ipvt(n),n)
!      do i = n-1,1,-1
!        pi = ipvt(i)
!        c = b(pi)-sum(a(pi,i+1:n)*x(i+1:n))
!        x(i) = c/a(pi,i)
!      end do
!
!      return
!      END SUBROUTINE gauss

```

```

!      dnx = dnor(f,n)
!
!      do while(dnx>tol)
!        call derfx (j,x,n)
!        call gauss (j,f,s,n)
!        x1 = x-s
!        call fx (f,x1,n)
!        dnr = dnor(f,n)
!        alfa = 1.D0
!        do while(dnr>(1.D0-alfa/2.)*dnx)
!          alfa = alfa/2.
!          x1 = x-alfa*s
!          call fx (f,x1,n)
!          dnr = dnor(f,n)
!        end do
!        print *,x1,alfa,dnr      ! Salida de resultados
!        x = x1
!        dnx = dnr
!      end do
!
!      end program newtarmijo
!
!      SUBROUTINE fx(f,x,n)
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(out) :: f(n)
!      real(kind=2) , intent(in) :: x(n)
!
!      f(1) = 6*datan(x(1)-10)-2*dexp(-x(2))-      &
!        2*dexp(-x(3))+2*x(2)+2*x(3)-9
!      f(2) = 2*datan(x(1)-10)-4*dexp(-x(2))-      &
!        dexp(-x(3))+7*x(2)-2*x(3)-3
!      f(3) = 2*datan(x(1)-10)-dexp(-x(2))-      &
!        3*dexp(-x(3))-x(2)+5*x(3) -3
!
!      return
!      END SUBROUTINE fx
!
!      SUBROUTINE derfx(j,x,n)
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(out) :: j(n,n)
!      real(kind=2) , intent(in) :: x(n)
!
!      j(1,1) = 6/(1+(x(1)-10)**2)
!      j(1,2) = 2*dexp(-x(2))+2
!      j(1,3) = 2*dexp(-x(3))+2
!      j(2,1) = 2/(1+(x(1)-10)**2)
!      j(2,2) = 4*dexp(-x(2))+7
!      j(2,3) = dexp(-x(3))-2
!      j(3,1) = 2/(1+(x(1)-10)**2)
!      j(3,2) = dexp(-x(2))-1
!      j(3,3) = 3*dexp(-x(3))+5
!
!      return
!      END SUBROUTINE derfx
!
!      REAL(kind=2) FUNCTION dnor (x,n)
!
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(in) :: x(n)
!
!      dnor = dsqrt(sum(x**2))
!
!      return
!      END FUNCTION dnor
!
!      SUBROUTINE gauss(a,b,x,n)
!      implicit none
!
!      integer , intent(in) :: n
!      real(kind=2) , intent(inout) :: a(n,n)
!      real(kind=2) , intent(inout) :: b(n)
!      real(kind=2) , intent(inout) :: x(n)
!
!      integer , dimension(10) :: ipvt
!      integer :: pi, i, k, l, ip,iaux
!      real(kind=2) :: smax, r, r1, c
!
!      *** Resolución del sistema lineal de ecuaciones mediante
!      *** eliminación de Gauss
!
!      do i = 1,n
!        ipvt(i) = i
!      end do

```

13. Programa Newtarmijo de la página 333.

```

!      PROGRAM Newtarmijo
!
!      implicit none
!
!      integer, parameter :: n = 3
!
!      real(kind=2), dimension(n) :: f
!      real(kind=2), dimension(n,n) :: j
!      real(kind=2), dimension(n) :: x, x1, s
!      real(kind=2) :: tol, dnr, dnx, alfa
!
!      real(kind=2) , external :: dnor
!
!      tol = dsqrt(epsilon(1.0d0))
!      x = 0.D0
!      call fx (f,x,n)

```

```

!      do i = 1,n
!        ipvt(i) = i
!      end do

```



```

!
! *** Triangularización ***
!
do k = 1,n-1
  l = 0
  smax = dabs(a(ipvt(k),k))
  do i = k+1,n
    ip = ipvt(i)
    if (dabs(a(ip,k))>smax) then
      l = i
      smax = dabs(a(ip,k))
    endif
  end do
  if (l/=0) then
   iaux = ipvt(k)
    ipvt(k) = ipvt(l)
    ipvt(l) = iaux
  endif
  pi = ipvt(k)
  r1 = 1.0/a(pi,k)
  do i = k+1,n
    ip = ipvt(i)
    r = a(ip,k)*r1
    a(ip,k+1:n) = a(ip,k+1:n)-r*a(pi,k+1:n)
    a(ip,k) = -r
  end do
end do

do k = 1,n-1
  ip = ipvt(k)
  do i = k+1,n
    pi = ipvt(i)
    b(pi) = b(pi)+a(pi,k)*b(ip)
  end do
end do

!
! *** Sustitución inversa ***
!
x(n) = b(ipvt(n))/a(ipvt(n),n)
do i = n-1,1,-1
  pi = ipvt(i)
  c = b(pi)-sum(a(pi,i+1:n)*x(i+1:n))
  x(i) = c/a(pi,i)
end do

!
return
END SUBROUTINE gauss

f(3) = dexp(x(1))-2.0
f(4) = dexp(x(1)+x(2))-4.0

!
return
END SUBROUTINE fx

SUBROUTINE derfx(j,x,m,n)      ! Evaluación de la matriz
!
implicit none
!
integer, intent(in) :: m
integer, intent(in) :: n
real(kind=2), intent(out) :: j(m,n)
real(kind=2), intent(in) :: x(n)
!
j(1,1) = dexp(x(1))-2.0*x(2)
j(1,2) = -2.0*dexp(x(1))-2.0*x(2)
j(2,1) = dexp(x(1))-x(2)
j(2,2) = -dexp(x(1))-x(2)
j(3,1) = dexp(x(1))
j(3,2) = 0.0
j(4,1) = dexp(x(1)+x(2))
j(4,2) = dexp(x(1)+x(2))
!
return
END SUBROUTINE derfx

SUBROUTINE qrdes(a,b,x,m,n,s1)
!
implicit none
!
integer, intent(in) :: m
integer, intent(in) :: n
real(kind=2), intent(out) :: s1
real(kind=2), intent(inout) :: a(m,n)
real(kind=2), intent(inout) :: b(m)
real(kind=2), intent(inout) :: x(n)
!
integer :: j, i, l
real(kind=2), dimension(20) :: d
real(kind=2) :: wj, s, suma, rmax, beta, sigma
!
! *** Resolución del problema lineal de mínimos cuadrados con
! factorización QR por transformaciones de Householder.
!
!
! *** Reducción QA=R y vector b a b' ***
!
do j = 1,n
  rmax = 0.0d0
  rmax = dmax1(rmax,maxval(dabs(a(j:m,j))))
  if (rmax==0.0) stop 'Matriz A de rango incompleto'
  beta = sum(a(j+1:m,j)**2)
  wj = a(j,j)
  sigma = sign(dsqrt(beta+wj*wj),wj)
  wj = wj+sigma
  beta = 2.0/(beta+wj*wj)
  a(j,j) = wj
  d(j) = -sigma
  do l = j+1,n
    s = beta*sum(a(j:m,j)*a(j:m,l))
    a(j:m,l) = a(j:m,l)-a(j:m,j)*s
  end do
  s = sum(a(j:m,j)*b(j:m))
  s = beta*s
  b(j:m) = b(j:m)-a(j:m,j)*s
end do

!
! *** Resolución Rx = b' ***
!
x(n) = b(n)/d(n)
do i = n-1,1,-1
  suma = sum(a(i,i+1:n)*x(i+1:n))
  x(i) = (b(i)-suma)/d(i)
end do

!
! *** Suma de residuos al cuadrado ***
!
s1 = sum(b(n+1:m)**2)

!
return
END SUBROUTINE qrdes

PROGRAM Gausnewt
!
implicit none
!
integer, parameter :: m = 4
integer, parameter :: n = 2
!
integer :: i
real(kind=2), dimension(m) :: f
real(kind=2), dimension(m,n) :: j
real(kind=2), dimension(n) :: x, p
real(kind=2) :: tol, dnor, s
!
tol = dsqrt(epsilon(1.0d0))
x = 1.0
!
! *** Proceso iterativo ***
!
do i = 1,100
  call fx(f,x,m,n)
  call derfx(j,x,m,n)
  call qrdes(j,f,p,m,n,s)
  x = x-p
  dnor = maxval(dabs(p))/maxval(dabs(x))
  if (dnor<tol) stop
  print *,x,s,dnor      ! Salida de resultados
end do

!
end program gausnewt

SUBROUTINE fx(f,x,m,n)      ! Cálculo de residuos
!
implicit none
!
integer, intent(in) :: m
integer, intent(in) :: n
real(kind=2), intent(out) :: f(m)
real(kind=2), intent(in) :: x(n)
!
f(1) = dexp(x(1))-2.0*x(2)-0.5
f(2) = dexp(x(1)-x(2))-1.0

PROGRAM Levmar
!
implicit none
!
integer, parameter :: m = 12

```

14. Programa Gausnewt de la página 347.

15. Programa Levmar de la página 353.

```

integer, parameter :: n = 3
!
integer :: k, i, l
real(kind=2), dimension(m) :: f
real(kind=2), dimension(m,n) :: j
real(kind=2), dimension(n,n) :: jtj, a
real(kind=2), dimension(n) :: x, s
real(kind=2) :: tol, dnor, mu, res
real(kind=2), dimension(n) :: b
real(kind=2), dimension(m) :: f1
real(kind=2) :: res1
!
tol = dsqrt(epsilon(1.0d0))*10
x(1) = 200. ! Valores de partida
x(2) = 30. ! de los parámetros
x(3) = -0.4 ! a estimar
mu = 0.01d0
!
do k = 1,100
call fx(f,x)
call derfx(j,x,m,n)
do i = 1,n
do l = 1,n
jtj(i,l) = dot_product(j(1:m,i),j(1:m,l))
end do
end do
2 continue
do i = 1,n
b(i) = dot_product(j(1:m,i),f)
end do
res = dot_product(f,f)
a = jtj
do i = 1,n
a(i,i) = jtj(i,i)+mu
end do
call gauss(a,b,s,n)
b = x-s
call fx(f1,b)
res1 = dot_product(f1,f1)
if (res1<res) then
x = b
f = f1
dnor = maxval(dabs(s))/maxval(dabs(x))
if (dnor<=tol) stop
print *,x,res1,mu,dnor
mu = mu/10.D0
else
mu = mu*10.D0
go to 2
endif
end do
!
END PROGRAM Levmar
!
SUBROUTINE fx(f,x)
!
implicit none
!
real(kind=2), intent(out) :: f(*)
real(kind=2), intent(in) :: x(*)
!
f(1) = x(1)/(1+x(2)*dexp(x(3)))-5.308
f(2) = x(1)/(1+x(2)*dexp(2*x(3)))-7.24
f(3) = x(1)/(1+x(2)*dexp(3*x(3)))-9.638
f(4) = x(1)/(1+x(2)*dexp(4*x(3)))-12.866
f(5) = x(1)/(1+x(2)*dexp(5*x(3)))-17.069
f(6) = x(1)/(1+x(2)*dexp(6*x(3)))-23.192
f(7) = x(1)/(1+x(2)*dexp(7*x(3)))-31.443
f(8) = x(1)/(1+x(2)*dexp(8*x(3)))-38.558
f(9) = x(1)/(1+x(2)*dexp(9*x(3)))-50.156
f(10) = x(1)/(1+x(2)*dexp(10*x(3)))-62.948
f(11) = x(1)/(1+x(2)*dexp(11*x(3)))-75.995
f(12) = x(1)/(1+x(2)*dexp(12*x(3)))-91.972
!
return
END SUBROUTINE fx
!
SUBROUTINE derfx(j,x,m,n)
!
implicit none
!
integer, intent(in) :: m
integer, intent(in) :: n
real(kind=2), intent(out) :: j(m,n)
real(kind=2), intent(in) :: x(n)
!
j(1,1) = 1/(1+x(2)*dexp(x(3)))
j(1,2) = -x(1)*dexp(x(3))/(1+x(2)*dexp(x(3)))**2
j(1,3) = -x(1)*x(2)*dexp(x(3))/(1+x(2)*dexp(x(3)))**2
j(2,1) = 1/(1+x(2)*dexp(2*x(3)))
j(2,2) = -x(1)*dexp(2*x(3))/(1+x(2)*dexp(2*x(3)))**2
j(2,3) = -x(1)*x(2)*dexp(2*x(3))*2/(1+x(2)*dexp(2*x(3)))**2
j(3,1) = 1/(1+x(2)*dexp(3*x(3)))
j(3,2) = -x(1)*dexp(3*x(3))/(1+x(2)*dexp(3*x(3)))**2
j(3,3) = -x(1)*x(2)*dexp(3*x(3))*3/(1+x(2)*dexp(3*x(3)))**2
j(4,1) = 1/(1+x(2)*dexp(4*x(3)))
j(4,2) = -x(1)*dexp(4*x(3))/(1+x(2)*dexp(4*x(3)))**2
j(4,3) = -x(1)*x(2)*dexp(4*x(3))*4/(1+x(2)*dexp(4*x(3)))**2
j(5,1) = 1/(1+x(2)*dexp(5*x(3)))
j(5,2) = -x(1)*dexp(5*x(3))/(1+x(2)*dexp(5*x(3)))**2
j(5,3) = -x(1)*x(2)*dexp(5*x(3))*5/(1+x(2)*dexp(5*x(3)))**2
j(6,1) = 1/(1+x(2)*dexp(6*x(3)))
j(6,2) = -x(1)*dexp(6*x(3))/(1+x(2)*dexp(6*x(3)))**2
j(6,3) = -x(1)*x(2)*dexp(6*x(3))*6/(1+x(2)*dexp(6*x(3)))**2
j(7,1) = 1/(1+x(2)*dexp(7*x(3)))
j(7,2) = -x(1)*dexp(7*x(3))/(1+x(2)*dexp(7*x(3)))**2
j(7,3) = -x(1)*x(2)*dexp(7*x(3))*7/(1+x(2)*dexp(7*x(3)))**2
j(8,1) = 1/(1+x(2)*dexp(8*x(3)))
j(8,2) = -x(1)*dexp(8*x(3))/(1+x(2)*dexp(8*x(3)))**2
j(8,3) = -x(1)*x(2)*dexp(8*x(3))*8/(1+x(2)*dexp(8*x(3)))**2
j(9,1) = 1/(1+x(2)*dexp(9*x(3)))
j(9,2) = -x(1)*dexp(9*x(3))/(1+x(2)*dexp(9*x(3)))**2
j(9,3) = -x(1)*x(2)*dexp(9*x(3))*9/(1+x(2)*dexp(9*x(3)))**2
j(10,1) = 1/(1+x(2)*dexp(10*x(3)))
j(10,2) = -x(1)*dexp(10*x(3))/(1+x(2)*dexp(10*x(3)))**2
j(10,3) = -x(1)*x(2)*dexp(10*x(3))*10/(1+x(2)*dexp(10*x(3)))**2
j(11,1) = 1/(1+x(2)*dexp(11*x(3)))
j(11,2) = -x(1)*dexp(11*x(3))/(1+x(2)*dexp(11*x(3)))**2
j(11,3) = -x(1)*x(2)*dexp(11*x(3))*11/(1+x(2)*dexp(11*x(3)))**2
j(12,1) = 1/(1+x(2)*dexp(12*x(3)))
j(12,2) = -x(1)*dexp(12*x(3))/(1+x(2)*dexp(12*x(3)))**2
j(12,3) = -x(1)*x(2)*dexp(12*x(3))*12/(1+x(2)*dexp(12*x(3)))**2
!
return
END SUBROUTINE derfx
!
SUBROUTINE gauss(a,b,x,n)
!
implicit none
!
integer, intent(in) :: n
real(kind=2), intent(inout) :: a(n,n)
real(kind=2), intent(inout) :: b(n)
real(kind=2), intent(inout) :: x(n)
!
integer, dimension(10) :: ipvt
integer :: pi, i, k, l, ip, iaux
real(kind=2) :: smax, r, r1, c
!
*** Resolución del sistema lineal de ecuaciones mediante
eliminación de Gauss
!
!
do i = 1,n
ipvt(i) = i
end do
!
*** Triangularización ***
!
do k = 1,n-1
l = 0
smax = dabs(a(ipvt(k),k))
do i = k+1,n
ip = ipvt(i)
if (dabs(a(ip,k))>smax) then
l = i
smax = dabs(a(ip,k))
endif
end do
if (l/=0) then
iaux = ipvt(k)
ipvt(k) = ipvt(l)
ipvt(l) = iaux
endif
pi = ipvt(k)
r1 = 1.0/a(pi,k)
do i = k+1,n
ip = ipvt(i)
r = a(ip,k)*r1
a(ip,k+1:n) = a(ip,k+1:n)-r*a(pi,k+1:n)
a(ip,k) = -r
end do
end do
do k = 1,n-1
ip = ipvt(k)
do i = k+1,n
pi = ipvt(i)
b(pi) = b(pi)+a(pi,k)*b(ip)
end do
end do

```

```

! *** Sustitución inversa ***
!
x(n) = b(ipvt(n))/a(ipvt(n),n)
do i = n-1,1,-1
  pi = ipvt(i)
  c = b(pi)-sum(a(pi,i+1:n)*x(i+1:n))
  x(i) = c/a(pi,i)
end do
!
return
END SUBROUTINE gauss

```

16. Programa Condest de la página 882.

```

PROGRAM Condest
!
! *** Estimación del número de condición 1 de una matriz
!
implicit none
!
integer, parameter :: n = 10
!
integer, dimension(n) :: ipvt
integer :: i
real(kind=2), dimension(n,n) :: a
real(kind=2), dimension(n) :: z
real(kind=2) :: cond, anorm

open(10,file='clin1')
read (10,*) a
!
! --- Norma 1 de la matriz
!
anorm = 0
do i = 1,n
  anorm = dmax1(anorm,sum(dabs(a(:,i))))
end do
!
! - Estimación del número de condición. Factorizar primero PA=LU.
!
call gau (a,ipvt,n)
z = 0
call rcond (a,n,ipvt,anorm,cond,z)
!
print *,cond
!
end program Condest

subroutine rcond(a,n,ipvt,anorm,cond,z)
!
implicit none
!
integer n
integer , intent(in) :: ipvt(n)
real(kind=2), intent(in) :: anorm
real(kind=2), intent(out) :: cond
real(kind=2), intent(inout) :: a(n,n), z(n)
!
integer :: k, j, l, i
real(kind=2) :: ek, wk, wkm, sm, s, t, ynorm

ek = 1.0
do k = 1,n
  ek = dsign(ek,-z(k))
  if (dabs(ek-z(k))>dabs(a(k,k))) then
    s = dabs(a(k,k))/dabs(ek-z(k))
    z = z*s
    ek = s*ek
  endif
  wk = ek-z(k)
  wkm = -ek-z(k)
  s = dabs(wk)
  sm = dabs(wkm)
  if (a(k,k)/=0.0) then
    wk = wk/a(k,k)
    wkm = wkm/a(k,k)
  else
    wk = 1.0
    wkm = 1.0
  endif
  sm = sm+sum(dabs(z(k+1:n)+wkm*a(k,k+1:n)))
  z(k+1:n) = z(k+1:n)+wk*a(k,k+1:n)
  s = s+sum(dabs(z(k+1:n)))
  if (s<sm) then
    t = wkm-wk
    wk = wkm
    z(k+1:n) = z(k+1:n)+t*a(k,k+1:n)
  endif
  z(k) = wk
end do
s = 1.0/sum(dabs(z))

```

```

z = z*s
!
do k = n,1,-1
  z(k) = z(k)+sum(a(k+1:n,k)*z(k+1:n))
  if (dabs(z(k))>1.0) then
    s = 1.0/dabs(z(k))
    z = z*s
  endif
  l = ipvt(k)
  t = z(l)
  z(l) = z(k)
  z(k) = t
end do
s = 1.0/sum(dabs(z))
z = z*s
!
!
ynorm = 1.0
!
do k = 1,n
  l = ipvt(k)
  t = z(l)
  z(l) = z(k)
  z(k) = t
  z(k+1:n) = z(k+1:n)+t*a(k+1:n,k)
  if (dabs(z(k))>1.0) then
    s = 1.0/dabs(z(k))
    z = z*s
    ynorm = s*ynorm
  endif
end do
s = 1.0/sum(dabs(z))
z = z*s
ynorm = s*ynorm
!
do k = n,1,-1
  if (dabs(z(k))>dabs(a(k,k))) then
    s = dabs(a(k,k))/dabs(z(k))
    z = z*s
    ynorm = s*ynorm
  endif
  if (a(k,k)/=0.0) then
    z(k) = z(k)/a(k,k)
  else
    z(k) = 1.0
  endif
  t = -z(k)
  z(:,k-1) = z(:,k-1)+t*a(:,k-1,k)
end do
s = 1.0/sum(dabs(z))
z = z*s
ynorm = s*ynorm
if (anorm/=0.0) then
  cond = 0.0
else
  cond = anorm/ynorm
endif
return
!
end subroutine rcond

subroutine gau(a,ipvt,n)
!
! *** Eliminación de Gauss de la matriz A
!
implicit none
!
integer, intent(in) :: n
integer, intent(out) :: ipvt(n)
real(kind=2), intent(inout) :: a(n,n)
!
integer :: l, i, j, k
real(kind=2) :: smax, r, r1
!
! --- Triangularización -
!
do k = 1,n-1
  smax = 0
  do i = k,n
    if (dabs(a(i,k))>smax) then
      l = i
      smax = dabs(a(i,k))
    endif
  end do
  ipvt(k) = l
  if (l/=k) then
    r = a(l,k)
    a(l,k) = a(k,k)
    a(k,k) = r
  endif
  r1 = -1.0/a(k,k)
  a(k+1:n,k) = a(k+1:n,k)*r1
  do i = k+1,n
    r = a(l,i)

```

```
      if (l/=k) then
        a(l,i) = a(k,i)
        a(k,i) = r
      endif
      a(k+1:n,i) = a(k+1:n,i)+r*a(k+1:n,k)
    end do

    end do
    !
    return
  end subroutine gau
```


Apéndice **H**

ESTIMACIÓN DEL NÚMERO DE CONDICIÓN DE MATRICES CUADRADAS

COMO SE RECORDARÁ de la definición dada en el capítulo 1, el número de condición de una matriz cuadrada, A ,

$$\kappa(A) = \|A\| \|A^{-1}\|$$

para una norma matricial $\|\cdot\|$ consistente con una norma vectorial, informa, entre otras cosas, de la proximidad de esa matriz a la singularidad, teniendo una importancia extraordinaria para determinar la sensibilidad del vector solución de un sistema de ecuaciones lineales $A\mathbf{x} = \mathbf{b}$ a pequeños cambios tanto en el término de la derecha, $\mathbf{b}$, como en los coeficientes que definen la matriz A . Cuantificar o estimar el número de condición de una matriz puede ser necesario para calibrar qué va a ocurrir con las operaciones en las que interviene o con el algoritmo que se le aplique para factorizarla, reducirla, etc.

Como se puede deducir de la observación de su definición, determinar estrictamente el número de condición de una matriz cuadrada es una operación prohibitiva desde el punto de vista del número de cálculos a realizar. A pesar de que contar con la información que suministra puede resultar interesante, puede que no el precio que hay que pagar por ello en términos del número de operaciones a llevar a cabo y del tiempo del ordenador a invertir en ello. Para evitar esto, en los últimos años se han desarrollado un conjunto de buenos algoritmos para estimar un número aproximado al del número de condición de una matriz, haciéndolo con una cantidad significativamente inferior de operaciones de cómputo.

H.1 El estimador de Cline, Moler, Stewart y Wilkinson

Si se desea obtener el número $\kappa_1(A) = \|A\|_1 \|A^{-1}\|_1$, el cálculo de $\|A\|_1$ no representa mayor dificultad ya que

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|.$$

Ahora bien, $\|A^{-1}\|_1$ no es tan fácil. Una forma de hacerlo sería estimando el valor de $\|\hat{X}\|_1$, donde $\hat{X} = [\hat{x}_1, \dots, \hat{x}_n]$ y $\hat{x}_i$ se obtendría resolviendo $A\hat{x}_i = \mathbf{e}_i$, llegándose a que $\hat{\kappa}_1(A) = \|A\|_1 \|\hat{X}\|_1$ sería una buena aproximación de $\kappa_1(A)$. El número de operaciones necesario para calcular $\hat{\kappa}_1(A)$ sería del orden de 3 veces el que se requeriría para obtener la matriz $\hat{X}$.

En 1971, Cline, Moler, Stewart y Wilkinson propusieron un algoritmo para estimar $\kappa_1(A)$ en $O(n^2)$ operaciones, que se utiliza mucho desde entonces, siendo referencia obligada al respecto. Se basa en la utilización de la siguiente implicación:

$$A\mathbf{y} = \mathbf{d} \quad \implies \quad \|A^{-1}\|_1 \geq \|\mathbf{y}\|_1 / \|\mathbf{d}\|_1.$$

La idea básica del procedimiento de obtención de $\hat{\kappa}_1(A)$ radica en escoger $\mathbf{d}$ de tal manera que la solución $\mathbf{y}$ tenga una norma grande y hacer entonces

$$\hat{\kappa}_1(A) = \|A\|_1 \|\mathbf{y}\|_1 / \|\mathbf{d}\|_1.$$

Lo buena o mala que sea la aproximación $\hat{\kappa}_1(A)$ de $\kappa_1(A)$ dependerá de lo proxima que esté la relación $\|\mathbf{y}\|_1 / \|\mathbf{d}\|_1$ a su valor máximo, $\|A^{-1}\|_1$.

Para comprender como funciona este algoritmo, considérese primero el caso en que $A = T$ es una matriz triangular superior. La relación entre $\mathbf{d}$ e $\mathbf{y}$ estará totalmente definida por la siguiente versión de un proceso de sustitución inversa:

$$\begin{aligned} & p(1:n) = 0 \\ & \textbf{for } j = n \textbf{ to } 1 \\ & \quad \text{Escoger } d(j) \\ & \quad y(j) = (d(j) - p(j)) / T(j, j) \\ & \quad p(1:j-1) = p(1:j-1) + y(j)T(1:j-1, j) \\ & \textbf{end} \end{aligned} \tag{H.1}$$

La diferencia entre este último y un proceso de sustitución inversa ordinario, $T\mathbf{y} = \mathbf{d}$, es que en (H.1) el término de la derecha, $\mathbf{d}$, hay que elegirlo de tal manera que la norma de $\mathbf{y}$ sea grande con respecto a la de él.

Una forma de forzar a que $\mathbf{y}$ crezca con respecto a $\mathbf{d}$ consiste en obligar a que $d(j)$ sea -1 ó $+1$, maximizando así $y(j)$. Si $p(j) \geq 0$, $d(j)$ se hará -1 ; si $p(j) < 0$, $d(j)$ se hará $+1$. En otras palabras, en (H.1), $d(j)$ debe hacerse igual a $-\text{sign}(p(j))$. Como $d(1:n) = [\pm 1, \dots, \pm 1]^T$, el estimador del número de condición $\kappa_1(A)$ de $A = T$ es

$$\hat{\kappa}_1(A) = \|T\|_1 \|\mathbf{y}\|_1.$$

Se puede conseguir un estimador más fiable si $d(j) \in \{-1, +1\}$ se escoge de tal manera que que crezcan simultáneamente $y(j)$ y $p(1:j-1, j) + y(j)T(1:j-1, j)$. En concreto, en el paso j

se calcularían

$$\begin{aligned} y(j)^+ &= (1 - p(j))/T(j, j) \\ s(j)^+ &= |y(j)^+| + \|p(1:j-1) + y(j)^+T(1:j-1, j)\|_1 \\ y(j)^- &= (-1 - p(j))/T(j, j) \\ s(j)^- &= |y(j)^-| + \|p(1:j-1) + y(j)^-T(1:j-1, j)\|_1 \end{aligned}$$

y se haría

$$y(j) = \begin{cases} y(j)^+ & \text{si } s(j)^+ \geq s(j)^- \\ y(j)^- & \text{si } s(j)^+ < s(j)^-. \end{cases}$$

El algoritmo completo para estimar el número de condición $\kappa_1(T)$ de una matriz $T \in \mathbb{R}^{n \times n}$, triangular superior regular, es el que se lista en la tabla H.1.

Tabla H.1

Algoritmo para la estimación del número de condición $\kappa_1(T)$ de una matriz triangular superior

```

p(1:n) = 0
for j = n to 1
    y(j)+ = (1 - p(j))/T(j, j)
    y(j)- = (-1 - p(j))/T(j, j)
    p(j)+ = p(1:j-1) + y(j)+T(1:j-1, j)
    p(j)- = p(1:j-1) + y(j)-T(1:j-1, j)
    if |y(j)+| + ||p(j)+||1 ≥ |y(j)-| + ||p(j)-||1 then
        y(j) = y(j)+
        p(1:j-1) = p(j)+
    else
        y(j) = y(j)-
        p(1:j-1) = p(j)-
    end
end
κ = ||y||1 ||T||1
y = y/||y||1

```

Para estimar el número de condición $\kappa_1(A)$ de una matriz $A \in \mathbb{R}^{n \times n}$, de la que se conoce su factorización $PA = LU$, habría que llevar a cabo las siguientes operaciones:

- Aplicar una versión del algoritmo de la tabla H.1 a la matriz U^T para obtener una solución de $U^T \mathbf{y} = \mathbf{d}$ de norma grande.
- Resolver $L^T \mathbf{r} = \mathbf{y}$, $L\mathbf{w} = P\mathbf{r}$ y $U\mathbf{z} = \mathbf{w}$.
- Calcular $\hat{\kappa}_1(A) = \|A\|_1 \|\mathbf{z}\|_1 / \|\mathbf{r}\|_1$.

Hay que hacer notar que $\|\mathbf{z}\|_1 \leq \|A^{-1}\|_1 \|\mathbf{r}\|_1$.

Esta forma de llegar al número de condición deseado se basa en las siguientes reglas heurísticas:

- Si A está mal condicionada y $PA = LU$, la matriz U estaría también mal condicionada. L , por el contrario estaría bien condicionada.
- El vector solución de $A^T P^T \mathbf{r} = \mathbf{d}$ tiende a estar situado en la dirección del vector singular izquierdo asociado al valor singular $\sigma_{\min}(A)$.
- Términos de la derecha como $\mathbf{r}$ producen soluciones de norma grande en el sistema $A\mathbf{z} = \mathbf{r}$.

A continuación se lista un código en FORTRAN 77 que utiliza el algoritmo de la tabla H.1 para estimar el número de condición κ_1 de una *matriz de Hilbert*, $H_{10} \in \mathbb{R}^{10 \times 10}$. La matriz de Hilbert, cuyos coeficientes se definen de la siguiente manera,

$$h_{ij} = \frac{1}{i+j-1},$$

es una matriz conocida por estar muy mal condicionada. De hecho, sus números de condición son:

$$\begin{aligned}\kappa_1(H_{10}) &= 3,5353 \times 10^{13} \\ \kappa_2(H_{10}) &= 1,6025 \times 10^{13} \\ \kappa_\infty(H_{10}) &= 3,5353 \times 10^{13}.\end{aligned}$$

```

PROGRAM Condest
C
C *** Estimación del número de condición 1 de una matriz de Hilbert.
C
  parameter (n=10)
  double precision a(n,n),z(n),cond,anorm,absum
  integer ipvt(n)
C
  open (10,file='clin1')
  read (10,*) a
C
C - Norma 1 de la matriz
C
  anorm = 0
  do i=1,n
    anorm = dmax1(anorm,absum(a(1,i),n))
  end do
C
C - Estimación del número de condición. Factorizar primero PA=LU.
C
  call gau (a,ipvt,n)
  call rcond (a,n,ipvt,anorm,cond,z)
C
  print *,cond
C
  end

  subroutine rcond (a,n,ipvt,anorm,cond,z)
C
C *** Se estima es número de condición 1 de PA=LU.
C
  integer ipvt(n)
  double precision a(n,n),z(n),anorm,ek,wk,wkm,sm,s,t,ynorm,cond,
+      absum,produ

```

```

C          T
C - Resolver U y=d
C
    ek = 1.0
    do k=1,n
        if (z(k).ne.0.0) ek = dsign(ek,-z(k))
        if (dabs(ek-z(k)).gt.dabs(a(k,k))) then
            s = dabs(a(k,k))/dabs(ek-z(k))
            call scalm (z,s,n)
            ek = s*ek
        end if
        wk = ek-z(k)
        wkm = -ek-z(k)
        s = dabs(wk)
        sm = dabs(wkm)
        if (a(k,k).ne.0.0) then
            wk = wk/a(k,k)
            wkm = wkm/a(k,k)
        else
            wk = 1.0
            wkm = 1.0
        end if
        do j=k+1,n
            sm = sm+dabs(z(j)+wkm*a(k,j))
            z(j) = z(j)+wk*a(k,j)
            s = s+dabs(z(j))
        end do
        if (s.lt.sm) then
            t = wkm-wk
            wk = wkm
            do j=k+1,n
                z(j) = z(j)+t*a(k,j)
            end do
        end if
        z(k) = wk
    end do
    s = 1.0/absum(z,n)
    call scalm (z,s,n)
C          T
C - Resolver L r=y
C
    do k=n,1,-1
        if (k.lt.n) z(k) = z(k)+produ(a(k+1,k),z(k+1),n-k)
        if (dabs(z(k)).gt.1.0) then
            s = 1.0/dabs(z(k))
            call scalm (z,s,n)
        end if
        l = ipvt(k)
        t = z(l)
        z(l) = z(k)
        z(k) = t
    end do
    s = 1.0/absum(z,n)
    call scalm (z,s,n)
C
    ynorm = 1.0
C
C - Resolver Lw=Pr
C

```

```
do k=1,n
  l = ipvt(k)
  t = z(l)
  z(l) = z(k)
  z(k) = t
  do i=1,n-k
    z(k+i) = z(k+i)+t*a(k+i,k)
  end do
  if (dabs(z(k)).gt.1.0) then
    s = 1.0/dabs(z(k))
    call scaln (z,s,n)
    ynorm = s*ynorm
  end if
end do
s = 1.0/absum(z,n)
call scaln (z,s,n)
ynorm = s*ynorm
C
C - Resolver Uz=w
C
do k=n,1,-1
  if (dabs(z(k)).gt.dabs(a(k,k))) then
    s = dabs(a(k,k))/dabs(z(k))
    call scaln (z,s,n)
    ynorm = s*ynorm
  end if
  if (a(k,k).ne.0.0) then
    z(k) = z(k)/a(k,k)
  else
    z(k) = 1.0
  end if
  t = -z(k)
  do i=1,k-1
    z(i) = z(i)+t*a(i,k)
  end do
end do
s = 1.0/absum(z,n)
call scaln (z,s,n)
ynorm = s*ynorm
if (anorm.eq.0.0) then
  cond = 0.0
else
  cond = anorm/ynorm
end if
return
end

double precision function absum (a,n)
double precision a(n)
absum = 0.0
do i=1,n
  absum = absum +dabs(a(i))
end do
return
end

double precision function produ (a,b,n)
double precision a(n),b(n)
produ = 0.0
```

```

do i=1,n
  produ = produ+a(i)*b(i)
end do
return
end

subroutine scaln (a,t,n)
double precision a(n),t
do i=1,n
  a(i) = a(i)*t
end do
return
end

subroutine gau (a,ipvt,n)
C
C *** Eliminación de Gauss de la matriz A
C
  integer ipvt(n),l,i,j,k,n
  double precision a(n,n),smax,r,r1
C
C - Triangularización -
C
  do k = 1,n-1
    smax = 0
    do i = k,n
      if (dabs(a(i,k)).gt.smax) then
        l = i
        smax = dabs(a(i,k))
      endif
    end do
    ipvt(k) = l
    if (l.ne.k) then
      r = a(l,k)
      a(l,k) = a(k,k)
      a(k,k) = r
    endif
    r1 = -1.0/a(k,k)
    do i = k+1,n
      a(i,k) = a(i,k)*r1
    enddo
    do i = k+1,n
      r = a(l,i)
      if (l.ne.k) then
        a(l,i) = a(k,i)
        a(k,i) = r
      endif
      do j = k+1,n
        a(j,i) = a(j,i)+r*a(j,k)
      end do
    end do
  end do
  ipvt(n) = n
C
  return
end

```

Si se ejecuta este programa, la solución que se obtiene es

2.678770454061916E+013

Como se puede observar, la estimación del valor real de $\kappa_1(H_{10}) = 3,5353 \times 10^{13}$ que se obtiene es muy buena.

H.2 El algoritmo de Hager

Propuesto por W. Hager en 1984, este algoritmo se basa en estimar el valor de $\|A^{-1}\|$ a partir de una técnica de optimización. Si se hace $A^{-1} = B$, designándose los coeficientes de B mediante b_{ij} , y se define la función

$$f(\mathbf{x}) = \|B\mathbf{x}\|_1 = \sum_{i=1}^n \left| \sum_{j=1}^n b_{ij}x_j \right|,$$

entonces

$$\|B\|_1 = \|A^{-1}\|_1 = \max \{f(\mathbf{x}) : \|\mathbf{x}\|_1 \leq 1\}.$$

El problema se plantea en términos de encontrar el máximo de la función convexa $f(\mathbf{x})$ en el conjunto convexo

$$S = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x}\|_1 \leq 1\}.$$

Como se ha estudiado en la parte del libro dedicada a la programación lineal, el máximo de una función convexa dentro del conjunto de soluciones que define un conjunto convexo se produce en un punto extremo de ese conjunto convexo. El algoritmo completo de Hager es el de la tabla H.2. Según hace notar Hager en su artículo, Hager [1984], el algoritmo converge normalmente en muy pocas iteraciones (en la mayoría de los casos en dos iteraciones).

A continuación se lista un programa escrito para MATLAB basado en el algoritmo de Hager para estimar el número de condición $\kappa_1(A)$.

```
function [cnd,iter] = condhag(A);
%
%CONDHAG Esta función estima el número de condición 1 de la matriz A.
%
%[CND,ITER] = produce el número de condición estimado CND y el número
%                de iteraciones empleado para conseguirlo.
%
%El programa necesita de las rutinas SUSINV y PIVPAR.
%
[m,n] = size(A);
if m~=n
    disp('la matriz A no es cuadrada');
    return;
end;
rho = 0;
b = zeros(n,1);
y = zeros(n,1);
for i = 1 : n
    b(i) = 1/n;
end;
flag = 0;
iter = 0;
while flag == 0
%
% Resolver Ax = b usando PIVPAR.
%
    [stora,U,M] = pivpar(A);
```

Tabla H.2

El algoritmo de Hager para estimar el número de condición 1 de una matriz A

Paso 0 – Hacer $\rho = \|A^{-1}\|_1 = 0$ y $\mathbf{b} = [1/n, \dots, 1/n]^T$.

Paso 1 – Resolver $A\mathbf{x} = \mathbf{b}$.

Paso 2 – Comprobar si $\|\mathbf{x}\|_1 \leq \rho$: si es así, ir al paso 6; si no, hacer $\rho = \|\mathbf{x}\|_1$ e ir al paso 3.

Paso 3 – Resolver $A^T \mathbf{z} = \mathbf{y}$, donde $y_i = 1$, si $x_i \geq 0$, e $y_i = -1$, si $x_i < 0$.

Paso 4 – Hacer $j = \max_{1 \leq i \leq n} \{|z_i|\}$.

Paso 5 – Si $|z_j| > \mathbf{z}^T \mathbf{b}$, hacer

$$\mathbf{b} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \leftarrow \text{fila } j$$

y volver al paso 1. Si no se cumple lo anterior, seguir al paso 6.

Paso 6 – Hacer $\|A^{-1}\|_1 = \rho$. El número de condición $\hat{\kappa}_1(A) = \rho \|A\|_1$.

```

bprim = M*b;
x = susinv(U,bprim) ;
if norm(x,1) <= rho
    flag = 1;
    return;
end;
iter = iter + 1;
rho = norm(x,1);
for i = 1:n
    if x(i) >= 0
        y(i) = 1;
    else
        y(i) = -1;
    end;
end;
%
% Resolver A'z = y usando PIVPAR.
%
c1 = A';
[stora,U,M] = pivpar(c1);
bprim = M * y;
z = susinv(U,bprim);
[j] = absmax(z);
if abs(z(j)) > z'*b
    b = zeros(n,1);
    b(j) = 1;
else

```

```
        flag = 1;
    end;
    cnd = rho * norm(A,1);
end;
end;

function [A,U,M] = pivpar(A);
%
%PIVPAR Triangulariza, mediante eliminación de Gauss con pivotación parcial,
%    la matriz A.
%
%[A,U,M] = pivpar(A) produce una matriz triangular superior U
%                    y una matriz triangular inferior permutada M,
%                    tales que MA = U. La parte triangular inferior de A
%                    contiene los multiplicadores y la triangular superior U.
%
[m,n] = size(A);
if m~=n
    disp('La matriz T no es cuadrada')
    return;
end;
M = eye(n,n);
U = zeros(n,n);
for k = 1:n-1
    M1 = eye(n,n);
    d = k ;
    s = A(k,k);
    for i = k+1:n
        if abs(A(i,k)) > abs(s)
            s = A(i,k);
            d = i;
        end;
    end;
    if (s == 0)
        disp('se ha encontrado un elemento pivote cero')
        A = [];
        U = [];
        M = [];
        return;
    end;
    p(k) = d;
    if d ~= k
        [A(k,k:n),A(d,k:n)] = inter(A(k,k:n),A(d,k:n));
        [M(k,:),M(d,:)] = inter(M(k,:),M(d,:));
    end;
    M1(k+1:n,k) = -A(k+1:n,k)/A(k,k);
    A(k+1:n,k) = M1(k+1:n,k);
    A(k+1:n,k+1:n) = A(k+1:n,k+1:n) +M1(k+1:n,k) * A(k,k+1:n);
    M(k+1:n,1:n) = M(k+1:n,1:n) +M1(k+1:n,k) * M(k,1:n);
end;
U = triu(A);
end;

function [y] = susinv(T,b);
%
%SUSINV Sustitución inversa.
%
%y = susinv(T,b) Calcula la solución y del sistema Ty = b.
%
```

```

[m,n] = size(T);
if m~=n
    error('la matriz T no es cuadrada')
end;
y = zeros(n,1);
for i = n:-1:1
    sum = 0;
    if (i ~= n)
        sum = T(i,i+1:n)*y(i+1:n);
    end;
    if (T(i,i) == 0)
        error('la matriz T es singular')
    end;
    y(i) = (b(i)-sum)/T(i,i);
end;
end;

function [p] = absmax(y);
%
%ABSMAX Determina la posición p en el vector y del coeficiente de
%         mayor valor absoluto.
%
%p = absmax(y) Devuelve la posición p.
%
[m,n] = size(y);
s = y(1);
n1 = 1;
for k=1:m
    if abs(y(k))>abs(s)
        p = k;
        s = y(k);
    end;
end;
end;

function [y,z] = inter(y,z);
%
%INTER Intercambia dos vectores.
%
c = y;
y = z;
z = c;
end;

```

Usando este estimador, el valor que proporciona para la matriz de Hilbert que usábamos antes es

3.535374081066318e+013

habiendo utilizado 2 iteraciones para conseguirlo.

Referencias

Todo lo que se expone en este apéndice se puede encontrar detalladamente en cualquier buen libro de cálculo numérico o de álgebra lineal numérica. Lo presentado está basado en Golub y Van Loan [1996], Highman [1996] y Datta [1995]. Los programas son del autor; están basados

en los correspondientes de LINPACK y MATMOD. Este último disponible con el libro de Datta.

Apéndice I

SOFTWARE DISPONIBLE EN INTERNET

EXISTE UNA GRAN CANTIDAD de *software* disponible para resolver los problemas que hemos abordado en este libro. Como fácilmente puede imaginar el lector, el más probado, robusto y con total garantía de funcionar en las circunstancias más adversas no suele ser de dominio público,¹ es decir, no se puede obtener si no se abona una cierta cantidad de dinero. No obstante, gracias a INTERNET, cada vez es más el de buena calidad que la comunidad científico-técnica tiene a su disposición estando conectado a la red.

Para saber qué *software* emplear en cada circunstancia, como hemos recalcado en varias ocasiones a lo largo del libro, es importante conocer bien el problema que se tiene que resolver, la forma en la que se presenta, su previsible dificultad numérica, su tamaño e incluso la impresión personal al respecto de la persona que lo ha de resolver. Como la disponibilidad de programas de ordenador para resolver problemas de sistemas de ecuaciones lineales y no lineales, así como de programación lineal y entera, cada vez es mayor, una buena forma de proceder desde nuestro punto de vista consistiría en tratar de resolver lo más rápidamente posible un prototipo del problema que se estudie, utilizando si es factible *software* de dominio público, o programas específicos de alto nivel como MATLAB, MATHEMATICA u otros parecidos, y luego, si se tiene que resolver el mismo problema muchas veces más, o va a ser parte de un proceso más complejo, plantearse la idoneidad de adquirir las rutinas concretas que interesen de alguno de los paquetes de programas que indicaremos posteriormente, o programarse uno mismo en FORTRAN o C la forma de dar solución al problema tomando como base el diseño del prototipo que mencionábamos.

Una guía muy buena para conocer el *software* disponible para resolver los problemas que hemos presentado en el libro, así como cualquier problema de optimización en general, es *Optimization Software Guide* de Moré y Wright [1993]. Información al respecto, actualizada

¹Aunque no siempre sea así.

continuamente, se puede obtener de unos de los mejores sitios de INTERNET con información y documentación sobre matemática aplicada industrialmente: su dirección de WORLD WIDE WEB, o simplemente WEB, es la que sigue.

<http://www.netlib.org>

Esta información también está disponible vía FTP en

<ftp://ftp.netlib.org>

En estas direcciones se puede obtener directamente copia del mucho *software* disponible en INTERNET de dominio público, como LINPACK, LAPACK, EISPACK, MINPACK, así como enlaces (*links*), con direcciones donde se puede encontrar otro mucho tanto de dominio público como de pago.

La sociedad norteamericana INFORMS, en su revista *OR/MS Today*, publica periódicamente unas revisiones muy interesantes sobre los programas de ordenador que se encuentran disponibles, tanto de dominio público como de pago. La dirección WEB donde se pueden consultar estos estudios es

<http://lionhrtpub.com/ORMS.html>

En los estudios se revisan programas para optimización, álgebra lineal numérica, toma de decisiones, estadística y otras muchas ramas del cómputo.

En la sociedad norteamericana *Society for Industrial and Applied Mathematics*, editora de la guía antes indicada, también se pueden consultar listas actualizadas de *software* matemático. Su dirección de WEB es la que sigue.

<http://www.siam.org>

Una dirección donde se pueden obtener sin coste alguno todos los algoritmos que la Association for Computing Machinery ha ido recolectando las últimas décadas en su serie *Collected Algorithms from ACM* es la siguiente

<http://www.acm.org/calgo>

I.1 Software de pago

Los paquetes de ordenador generales que consideramos más interesante dentro de este dominio son:

NAG La librería de Numerical Algorithms Group contiene programas de ordenador para tratar cualquier tipo de problemas de cómputo que se puedan dar. Se venden versiones para ordenador personal, estación de trabajo, grandes ordenadores, ordenadores con arquitectura en paralelo, etc. También incluye compiladores y otros productos generales de *software*. La dirección WEB donde se puede recabar información sobre este paquete es la que sigue.

<http://www.nag.co.uk>

- IMSL Estas siglas se refieren a International Mathematical and Statistical Libraries. Contiene rutinas en FORTRAN y C para resolver cualquier tipo de problema relacionado con álgebra lineal numérica, optimización, ecuaciones diferenciales, estadística, integración numérica, elementos finitos, etc. La dirección de INTERNET donde se puede encontrar información sobre esta librería es la que sigue.
- <http://www.vni.com/products/ims1/index.html>**
- MATLAB Sobre este programa ya hemos incidido varias veces a lo largo de todo el libro. Información directa sobre él se puede consultar en la dirección siguiente
- <http://www.mathworks.com>**
- MATHEMATICA Esta es una alternativa muy interesante de MATLAB. La dirección de INTERNET donde se puede obtener información al respecto es la que sigue.
- <http://www.wri.com>**
- CPLEX Bajo este nombre se agrupan un amplio conjunto de programas escritos en C para resolver problemas de programación lineal utilizando varios algoritmos, secuenciales y paralelos, de los más modernos y eficientes: el *simplex*, de puntos interiores, etc. La dirección de INTERNET donde se puede consultar información sobre este paquete es la que sigue.
- <http://www.cplex.com>**
- OSL Esta librería de IBM, Optimization Software Library, contiene programas de ordenador para resolver problemas computacionales de optimización de cualquier tipo. La dirección de INTERNET donde se puede recabar información sobre este paquete es la que sigue.
- <http://www.research.ibm.com/osl>**

I.2 Software de dominio público

Como ya hemos indicado, existe gran cantidad de *software* de dominio público disponible para resolver problemas como los que hemos presentado en este libro.

Dada la gran novedad que ha supuesto, y supone, poder acceder a INTERNET cualquiera que sea la circunstancia, existe una proliferación muy importante de direcciones donde se pueden encontrar cosas valiosas. El problema de listar aquí siquiera algunas es que en unos pocos meses la información puede resultar irrelevante, bien porque las personas encargadas de mantener los datos o los programas que en ellas figuren no se dediquen más a ello, o simplemente porque desaparezcan las direcciones. Nuestra mejor recomendación es dirigirse a las direcciones de instituciones consolidadas como las que hemos listado al comienzo de este capítulo y en la sección anterior.

En cualquier caso, a continuación se citan cuatro direcciones que llevan ya varios años funcionando muy bien y en las cuales se puede consultar mucha información relativa a *software* disponible, artículos interesantes, libros y últimas novedades a estos respectos.

<http://www.simtel.net>

<http://mat.gsia.cmu.edu>

<http://www.mats.mu.oz.au/~worms>

<http://ucsu.colorado.edu/~xu/software.html>

En el libro de Brian J. Thomas, *The Internet for Scientists and Engineers*, de 1996, se listan una gran cantidad de direcciones de INTERNET donde se puede encontrar respuesta a cualquiera de las necesidades del lector en asuntos científico-tecnológicos.

Apéndice J

EL SOFTWARE DEL LIBRO

EL SOFTWARE QUE SE suministra con el libro, en el CD-ROM que se adjunta, está almacenado según el esquema que representa la figura J.1. Todos los programas que se han codificado y listado para probar los algoritmos que se han presentado en el libro se incluyen en el disco en sus versiones en FORTRAN 77, FORTRAN 90 y C. También están las versiones más recientes de BBMI y CCNET y bastantes casos y ejemplos, académicos y comerciales, con los que se pueden poner a prueba las prestaciones de estos dos códigos.

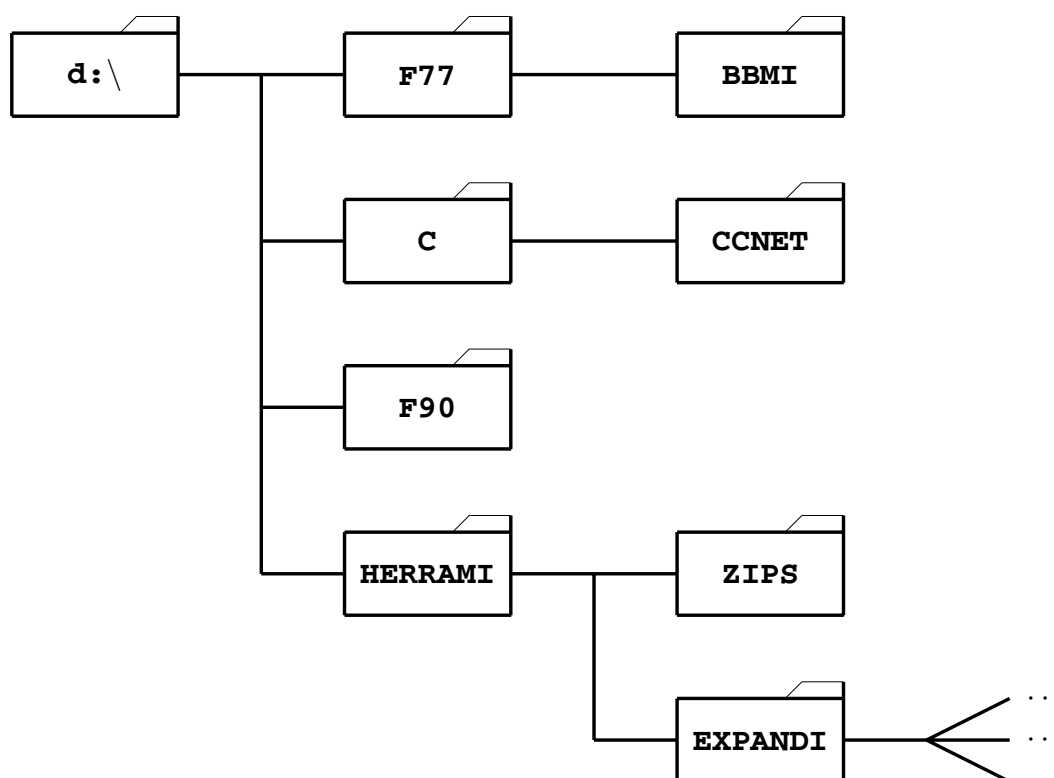
Los códigos fuente en FORTRAN 77, FORTRAN 90 y C se encuentran en los subdirectorios, o carpetas, correspondientes. Los casos de prueba para el programa BBMI, y su ejecutable `bbmi.exe`, se encuentran en el subdirectorio o carpeta con su nombre. La carpeta CCNET contiene el código fuente de CCNET, su ejecutable `ccnet.exe` y varios ficheros para probar sus prestaciones.

La carpeta HERRAMI contiene programas útiles si no se dispone de compilador de FORTRAN o de C y se quiere modificar y compilar los fuentes que se suministran. Los compiladores que se proporcionan son de dominio público por lo que su bondad no se espera que sea comparable a la de comerciales como los que se han usado para procesar los códigos que se presentan en el libro: *FORTRAN Powerstation 4.0* de Microsoft y *Borland C/C++ 5.0*. En las carpetas que cuelgan de HERRAMI están todos esos programas comprimidos y algunos descomprimidos. Entre estos últimos se pueden encontrar un compilador de C y otro de FORTRAN 77, y todos los programas auxiliares que necesitan. Para expandir los ficheros que están en la carpeta ZIPS, lo que daría lugar a algo similar a lo que cuelga de EXPANDI, hay que seguir las instrucciones que figuran en los ficheros `readme.1st`, `readme.dj` y similares.

La dirección de INTERNET de donde se ha obtenido gran parte del *software* de dominio público que se incluye en la carpeta HERRAMI del CD-ROM es la siguiente.

`http://www.simtel.net`

Esta dirección, mantenida por Keith Petersen, es un centro mundial de distribución muy interesante para encontrar programas de todo tipo de los denominados de dominio público: *shareware*

**Figura J.1**

Representación de la disposición del *software* del libro que se incluye en el CD-ROM que se adjunta al mismo

y *freeware*. De toda la información que se puede obtener en este servidor, a la que nos referimos está ubicada dentro del apartado denominado *DJ Delorie's DJGPP*. Como se actualiza periódicamente, es conveniente consultarla de vez en cuando por si ha cambiado algo de los programas o se ha incluido nuevo *software* mejor que el que hemos utilizado para elaborar el CD.

Además de los ficheros `readme.1st` y `readme.dj`, en la dirección

`ftp://x2ftp.oulu.fi`

en el directorio `/pub/msdos/programming/djgpp2`, existe un fichero, `djtut255.zip` donde se dan instrucciones detalladas (en ficheros `.txt` y `.html`) sobre cómo proceder con los programas apuntados. El número 255 puede variar dependiendo de la versión de que se trate.

Bibliografía

- [1] AASEN, J.O. 1971. *On the Reduction of a Symmetric Matrix to Tridiagonal Form*. BIT 11, pp. 233-242.
- [2] ABADIE, J. ED. 1970. *Integer and Nonlinear Programming*. North-Holland Publishing Company.
- [3] ACTON, F.S. 1990. *Numerical Methods that Work*. The Mathematical Association of America.
- [4] ADAMS, J.C., BRAINERD, W.S., MARTIN, J.T., SMITH, B.T. Y WAGENER, J.L. 1992. *FORTRAN 90 Handbook. Complete ANSI/ISO Reference*. Intertext Publications, MacGraw-Hill Book Company.
- [5] ADOBE SYSTEMS INCORPORATED 1990. *PostScript Language. Reference Manual*. Addison-Wesley Publishing Company.
- [6] ADOBE SYSTEMS INCORPORATED 1986. *PostScript Language. Tutorial and Cookbook*. Addison-Wesley Publishing Company.
- [7] AHUJA, R.K., MAGNATI, T.L. Y ORLIN, J.B. 1989. *Network Flows*. En Handbooks in Operations Research and Management Science. Volume 1: Optimization. Nemhauser, G.L., Rinnooy Kan, A.H.G. y Todd, M.J. eds. North-Holland Publishing Company.
- [8] ALJ, A. Y FAURE. R. 1988. *Investigación operativa. Elementos fundamentales. Vol. 1*. Masson, S.A.
- [9] ALVARADO, F.L. 1979. *A Note on Sorting Sparse Matrices*. Proceedings of the IEEE 67, pp. 1362-1363.
- [10] ALVARADO, F.L. 1990. *Manipulation and Visualization of Sparse Matrices*. ORSA J. Computing 2, pp. 186-207.
- [11] ANDERS, G.J. 1990. *Probability Concepts in Electric Power Systems*. John Wiley and Sons.
- [12] ANDERSON, E., BAI, Z., BISCHOF, C., DEMMEL, J., DONGARRA, J., DU CROZ, J., GREENBAUM, A., HAMMARLING S., MCKENNEY, A., OSTROUCHOV, S. Y SORENSEN, D. 1992. *LA-PACK User's Guide*. SIAM.
- [13] ANDERSON, E., BAI, Z., BISCHOF, C., DEMMEL, J., DONGARRA, J., DU CROZ, J., GREENBAUM, A., HAMMARLING S., MCKENNEY, A., OSTROUCHOV, S. Y SORENSEN, D. 1995. *LA-PACK User's Guide. Second Edition*. SIAM.
- [14] ARBEL, A. 1993. *Exploring Interior-Point Linear Programming. Algorithms and Software*. The MIT Press.
- [15] ARMIJO, L. 1966. *Minimization of Functions Having Lipschitz Continuos First Partial Derivatives*. Pacific J. Math. 16, pp. 1-3.
- [16] ARRILLAGA, J. Y ARNOLD, C.P. 1990. *Computer Analysis of Power Systems*. John Wiley and Sons.
- [17] ATKINSON, K. 1993. *Elementary Numerical Analysis*. John Wiley and Sons.

- [18] ATKINSON, L.V., HARLEY, P.J. y HUDSON, J.D. 1989. *Numerical Methods with FORTRAN 77. A Practical Introduction*. Addison-Wesley Publishing Company.
- [19] ATTEIA, M. y PRADEL, M. 1990. *Éléments d'Analyse Numérique*. Cepadues-Editions.
- [20] AVRIEL, M. 1976. *Nonlinear Programming. Analysis and Methods*. Prentice Hall.
- [21] AXELSSON, O. 1996. *Iterative Solution Methods*. Cambridge University Press.
- [22] BARTELS, R.H. y GOLUB, G.H. 1969. *The Simplex Method of Linear Programming Using LU Decomposition*. Communications of the ACM 12, pp. 266-268.
- [23] BAZARAA, M.S. y JARVIS, J.J. 1977. *Linear Programming and Network Flows*. John Wiley and Sons.
- [24] BAZARAA, M.S., JARVIS, J.J. y SHERALI, H.D. 1990. *Linear Programming and Network Flows*. John Wiley and Sons.
- [25] BAZARAA, M.S. y SHETTY, C.M. 1979. *Nonlinear Programming. Theory and Algorithms*. John Wiley and Sons.
- [26] BEALE, E.M.L. 1954. *An Alternative Method for Linear Programming*. Proceedings of the Cambridge Philosophical Society 50, pp. 513-523.
- [27] BECKMANN, M.J. 1968. *Dynamic Programming of Economic Decisions*. Springer Verlag.
- [28] BELLMAN, R. 1960. *Introduction to Matrix Analysis*. McGraw-Hill Book Company.
- [29] BELLMAN, R. 1972. *Dynamic Programming*. Princeton University Press.
- [30] BELLMAN, R. 1985. *Introducción al análisis matricial*. Editorial Reverté.
- [31] BELLMAN, R. y DREYFUS, S.E. 1962. *Applied Dynamic Programming*. Princeton University Press.
- [32] BERGE, C. 1970. *Graphes et Hypergraphes*. Dunod.
- [33] BERGEN, A.R. 1986. *Power Systems Analysis*. Prentice Hall.
- [34] BERMAN, A. y PLEMMONS, R.J. 1974. *Cones and Iterative methods for Best Least Squares Solutions of Linear Systems*. SIAM J. Numer. Anal. 11, pp. 145-154.
- [35] BERTSEKAS, D. P. 1982. *Constrained Optimization and Lagrange Multiplier Methods*. Academic Press, Inc.
- [36] BERTSEKAS, D. P. 1991. *Linear Network Optimization: Algorithms and Codes*. The MIT Press.
- [37] BERTSEKAS, D. P. 1995. *Nonlinear Programming*. Athena Scientific.
- [38] BERTSEKAS, D. P. y TSITSIKLIS, J.N. 1989. *Parallel and Distributed Computation. Numerical Methods*. Prentice Hall.
- [39] BERTSIMAS, D. y TSITSIKLIS, J.N. 1997. *Introduction to Linear Optimization*. Athena Scientific.
- [40] BEST, M.J. y RITTER, K. 1985. *Linear Programming. Active Set Analysis and Computer Programs*. Prentice Hall.
- [41] BJÖRK, *rA* 1990. *Least Squares Methods*. En Handbook of Numerical Analysis. Volume 1: Finite Difference Methods (Part 1); Solution of Equations in $\mathbb{R}^n$ (Part 1). Ciarlet, P.G. y Lions, J.L. eds. North-Holland Publishing Company.
- [42] BJÖRK, *rA* 1996. *Numerical Methods for Least Squares Problems*. SIAM.
- [43] BJÖRK, *rA* y ELFVING, T. 1979. *Accelerated Projection Methods for Computing Psudoinverse Solutions of Systems of Linear Equations*. BIT 19, pp. 145-163.
- [44] BOGGS, P.T., BYRD, R.H. y SCHNABEL, R.B. EDS. 1985. *Numerical Optimization 1984*. SIAM.

-
- [45] BLAND, R.G. 1977. *New Finite Pivoting Rules for the Simplex Method*. Mathematics of Operations Research 2, pp. 103-107.
- [46] BRADLEY, G.H., BROWN, G.G. Y GRAVES, G.W. 1977. *Design and Implementation of Large Scale Transshipment Algorithms*. Management Science 24, pp. 1-34.
- [47] BRAINERD, W.S., GOLDBERG, C.H. Y ADAMS, J.C. 1990. *Programmer's Guide to FORTRAN 90*. Intertext Publications, MacGraw-Hill Book Company.
- [48] BRAINERD, W.S., GOLDBERG, C.H. Y ADAMS, J.C. 1996. *Programmer's Guide to FORTRAN 90*. Springer Verlag.
- [49] BROWN, H.E. 1975. *Solution of Large Networks by Matrix Methods*. John Wiley and Sons.
- [50] BROYDEN, C.G. 1965. *A Class of Methods for Solving Nonlinear Simultaneous Equations*. Mathematics of Computation 19, pp. 577-593.
- [51] BUNCH, J.R. 1971. *Analysis of the Diagonal Pivoting Method*. SIAM J. Numer. Anal. 8, pp. 656-680.
- [52] BUNCH, J.R. 1974. *Partial Pivoting Strategies for Symmetric Matrices*. SIAM J. Numer. Anal. 11, pp. 521-528.
- [53] BUNCH, J.R. Y KAUFMAN, L. 1977. *Some Stable Methods for Calculating Inertia and Solving Symmetric Linear Systems*. Mathematics of Computation 31, pp. 163-179.
- [54] BUNCH, J.R., KAUFMAN, L. Y PARLETT, B.N. 1976. *Decomposition of a Symetric Matrix*. Numerische Mathematik 27, pp. 95-109.
- [55] BUNCH, J.R. Y PARLETT, B.N. 1971. *Direct Methods for Solving Symmetric Indefinite Systems of Linear Equations*. SIAM J. Numer. Anal. 8, pp. 639-655.
- [56] BUNCH, J.R. Y ROSE, D.J. EDS. 1976. *Sparse Matrix Computations*. Academic Press, Inc.
- [57] BURDEN, R.L. Y FAIRES, J.D. 1985. *Análisis numérico*. Grupo Editorial Iberoamérica.
- [58] BUZZI-FERRARIS, G. 1993. *Scientific C++*. Building Numerical Libraries the Object-Oriented Way. Addison-Wesley Publishing Company.
- [59] CIARLET, P.G. 1988. *Introduction à L'Analyse Numérique Matricielle et à L'Optimisation*. Masson, S.A.
- [60] CIARLET, P.G. 1989. *Introduction to Numerical Linear Algebra and Optimisation*. Cambridge University Press.
- [61] CIARLET, P.G. Y LIONS, J.L. EDS. 1990. *Handbook of Numerical Analysis. Volume 1: Finite Difference Methods (Part 1); Solution of Equations in $\mathbb{R}^n$ (Part 1)*. North-Holland Publishing Company.
- [62] CIRIANI, T.A. Y LEACHMAN, R.C. 1993. *Optimization in Industry. Mathematical Programming and Modeling Techniques in Practice*. John Wiley and Sons.
- [63] CLINE, A.K., MOLER, C.B., STEWART, G.W. Y WILKINSON, J.H. 1979. *An Estimate for the Condition Number of a Matrix*. SIAM J. Numer. Anal. 16, pp. 368-375.
- [64] COLEMAN, T.F., EDENBRANDT, A. Y GILBERT, J.R. 1986. *Predicing Fill for Sparse Orthogonal Factorization*. Journal ACM 33, pp. 517-532.
- [65] COLEMAN, T.F. Y LI, Y. EDS. 1990. *Large-Scale Numerical Optimization*. SIAM.
- [66] COLEMAN, T.F. Y VAN LOAN, C. 1988. *Handbook for Matrix Computations*. SIAM.
- [67] CONTE, S.D. Y DE BOOR, C. 1981. *Elementary Numerical Analysis. An Algorithmic Approach*. McGraw-Hill Book Company.
- [68] COOK, T.M. Y RUSSELL, R.A. 1977. *Introduction to Management Science*. Prentice Hall.

- [69] CORMEN, T.H., LEISERSON, C.E. Y RIVEST, R.L. 1992. *Introduction to Management to Algorithms*. The MIT Press, McGraw-Hill Book Company.
- [70] COWELL, W.R. ED. 1984. *Sources and Development of Mathematical Software*. Prentice Hall.
- [71] CUTHILL, E. Y MCKEE, J. 1969. *Reducing the bandwidth of Sparse Symmetric Matrices*. Proceedings of the 24th National Conference of the Association for Computing Machinery (ACM), pp. 157-172. Brandon Systems Press.
- [72] CHAMBERLAND, L. 1995. *FORTRAN 90. A Reference Guide*. Prentice Hall PTR.
- [73] CHAN, T.F. 1982. *An Improved Algorithm for Computing the Singular Value Decomposition*. ACM Trans. on Mathematical Software 8, pp. 72-83.
- [74] CHAN, T.F. 1982. *Algorithm 581: An Improved Algorithm for Computing the Singular Value Decomposition*. ACM Trans. on Mathematical Software 8, pp. 84-88.
- [75] CHENEY, W. Y KINCAID, D. 1985. *Numerical Mathematics and Computing*. Brooks/Cole Publishing Company.
- [76] CHIVERS, I. Y SLEIGHTHOLME, J. 1995. *Introducing FORTRAN 90*. Springer Verlag.
- [77] CHONG, E.K.P. Y ŽAK, S.H. 1996. *An Introduction to Optimization*. John Wiley and Sons.
- [78] CHU, E., GEORGE, A., LIU, J. Y NG, E. 1984. *SPARSPAK: Waterloo Sparse Matrix Package. User's Guide for SPARSPAK-A*. Research Report CS-84-36, Department of Computer Science, University of Waterloo, Waterloo, Ontario, Canadá.
- [79] CHVÁTAL, V. 1983. *Linear Programming*. W.H. Freeman and Company.
- [80] DAHLQUIST, G. Y BJÖRK, R. 1974. *Numerical Methods*. Prentice Hall.
- [81] DAKIN, R.J. 1965. *A Tree-Search Algorithm for Mixed Integer Programming Problems*. Computer Journal 8, pp. 250-255.
- [82] DANTZIG, G.B. 1963. *Linear Programming and Extensions*. Princeton University Press.
- [83] DANTZIG, G.B. 1987. *Origins of the Simplex Method*. Technical Report SOL 87-5. Systems Optimization Laboratory, Department of Operations Research, Stanford University.
- [84] DANTZIG, G.B. Y WOLFE, PH. 1960. *Decomposition Principle for Linear Programming*. Operations Research 8, pp. 101-111.
- [85] DARST, R.B. 1991. *Introduction to Linear Programming: Applications and Extensions*. Marcel Dekker, Inc.
- [86] DATTA, B.N. 1995. *Numerical Linear Algebra and Applications*. Brooks/Cole Publishing Company.
- [87] DAVENPORT, J.H. SIRET, Y. Y TOURNIER, E. 1993. *Computer Algebra. Systems and Algorithms for Algebraic Computation*. Academic Press.
- [88] DE LA FUENTE, J.L. 1986. *Programación en redes no lineales. El problema de redes eléctricas*. Actas Optimización de Flujos en Redes'86.
- [89] DE LA FUENTE, J.L. 1987. *Programación no lineal: Aplicaciones en análisis, gestión y planificación de sistemas eléctricos*. Actas I Seminario Internacional de Investigación Operativa-Programación Matemática'86. J.P. Vilaplana y L.F. Escudero eds., pp. 255-278.
- [90] DE LA FUENTE, J.L. Y LUMBRERAS, J. 1987. *A New Implementation of an Optimal Power Flow System Based on a General Purpose Nonlinear Programming Program*. Proc. IEEE PICA, pp. 422-428.
- [91] DE LA FUENTE, J.L. 1988. *Application of Nonlinear Network Optimization Techniques to Large Scale Power Scheduling Problems*. TIMS/ORSA National Meeting.

-
- [92] DE LA FUENTE, J.L. 1989. *La programación matemática y la planificación de redes eléctricas*. Actas Programación Matemática'89.
- [93] DEMIDOVICH, B.P. Y MARON, I.A. 1985. *Cálculo Numérico Fundamental*. Paraninfo.
- [94] DENNIS, J.E. Y SCHNABEL, R.B. 1983. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. Prentice Hall.
- [95] DENNIS, J.E. Y SCHNABEL, R.B. 1996. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. SIAM.
- [96] DOMMEL, H.W. Y TINNEY, W.F. 1968. *Optimal Power Flow Solutions*. IEEE Trans. on Power Apparatus and Systems PAS-87, pp. 1866-1876.
- [97] DONGARRA, J.J., BUNCH, J.R., MOLER, C.B. Y STEWART, G.W. 1979. *LINPACK Users' Guide*. SIAM.
- [98] DONGARRA, J.J., DUFF, I.S., SORENSEN, D.C. Y VAN DER VORST, H.A. 1991. *Solving Linear Systems on Vector and Shared Memory Computers*. SIAM.
- [99] DORFMAN, R., SAMUELSON, P. Y SOLOW, R. 1958. *Linear Programming and Economic Analysis*. McGraw-Hill Kogakusha Ltd.
- [100] DUFF, I.S. 1981. *MA32 - A Package for Solving Sparse Unsymmetric Systems Using the Frontal Method*. AERE Harwell Report AERE-R 10079. Computer Science & Systems Division, AERE Harwell.
- [101] DUFF, I.S. 1981. *MA32 - A Package for Solving Sparse Unsymmetric Systems Using the Frontal Method*. AERE Harwell Report AERE-R 10079. Computer Science & Systems Division, AERE Harwell.
- [102] DUFF, I.S., GRIMES, G. Y LEWIS, J.G. 1989. *Sparse Matrix Test Problems*. ACM Trans. on Mathematical Software 15, pp. 1-14.
- [103] DUFF, I.S. Y REID, J.K. 1979. *Some Design Features of a Sparse Matrix Code*. ACM Trans. on Mathematical Software 5, pp.18-35.
- [104] DUFF, I.S., ERISMAN, A.M. Y REID, J.K. 1986. *Direct Methods for Sparse Matrices*. Oxford University Press.
- [105] DUFF, I.S. Y STEWART, G.W. EDS. 1979. *Sparse Matrix Proceedings 1978*. SIAM.
- [106] EISENSTAT, S.C., SCHULTZ, M.H. Y SHERMAN, A.H. 1981. *Algorithms and Data Structures for Sparse Symmetric Gaussian Elimination*. SIAM J. Sci. and Statist. Comput. 2, pp. 225-237.
- [107] ELGERD, O.I. 1983. *Electric Energy Systems Theory: An Introduction*. McGraw-Hill Book Company.
- [108] EL-HAWARY, M.E. Y CHRISTENSEN, G.S. 1979. *Optimal Economic Operation of Electric Power Systems*. Academic Press, Inc.
- [109] ELLIS, T.M.R., PHILIPS, I.R. Y LAHEY, T.M. 1994. *FORTRAN 90 Programming*. Addison-Wesley Publishing Company.
- [110] ENGELN-MÜLLGES, G. Y UHLIG, F. 1996. *Numerical Algorithms with C*. Springer Verlag.
- [111] ENGELN-MÜLLGES, G. Y UHLIG, F. 1996. *Numerical Algorithms with FORTRAN*. Springer Verlag.
- [112] ERISMAN, A.M., NEVES, K.W. Y DWARAKANATH, M.H. EDS. 1980. *Electric Power Problems: The Mathematical Challenge*. SIAM.
- [113] EVANS, J.R. Y MINIEKA, E. 1992. *Optimization Algorithms for Network and Graphs*. Marcel Dekker, Inc.
- [114] FAIRES, J.D. Y BURDEN, R.L. 1993. *Numerical Methods*. PWS Publishing Company.

- [115] FANG, S.C., PUTHENPURA, S. 1993. *Linear Optimization and Extensions. Theory and Algorithms*. Prentice Hall.
- [116] FARKAS, J. 1902. *Theorie der Einfachen Ungleichungen*. Journal für die Reine und Angewandte Mathematik 124, pp. 1-27.
- [117] FIACCO, A.V. y MCCORMICK, G.P. 1968. *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*. John Wiley and Sons.
- [118] FIACCO, A.V. y MCCORMICK, G.P. 1990. *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*. SIAM.
- [119] FLETCHER, R. 1987. *Practical Methods of Optimization*. John Wiley and Sons.
- [120] FORD, L.R. y FULKERSON, D.R. 1962. *Flows in Networks*. Princeton University Press.
- [121] FORREST, J.J.H. y TOMLIN, J.A. 1972. *Updated Triangular Factors of the Basis to Maintain Sparsity in the Product Form Simplex Method*. Mathematical Programming 2, pp. 263-278.
- [122] FORSYTHE, G.E., MALCOLM, M.A. y MOLER, C.B. 1977. *Computer Methods for Mathematical Computations*. Prentice Hall.
- [123] FOURER, R., GAY, D.M. y KERNIGHAM, B.W. 1993. *AMPL. A Modeling Language for Mathematical Programming*. Boyd & Fraser Publishing Company.
- [124] GAL, T. 1979. *Postoptimal Analysis, Parametric Programming, and Related Topics*. McGraw-Hill Book Company.
- [125] GANDER, W. y HŘEBÍČEK, JIŘÍ 1993. *Solving Problems in Scientific Computing Using Maple and Matlab*. Springer Verlag.
- [126] GARBOW, B.S., BOYLE, J.M., DONGARRA, J.J. y MOLER, C.B. 1977. *Matrix Eigensystem Routines-EISPACK Guide Extension*. Springer Verlag.
- [127] GARCÍA, C.B. y ZANGWILL, W.I. 1981. *Pathways to Solutions, Fixed Points, and Equilibria*. Prentice Hall.
- [128] GARFINKEL, R.S. y NEMHAUSER, G.L. 1972. *Integer Programming*. John Wiley and Sons.
- [129] GEORGE, A. 1971. *Computer Implementation of the Finite Element Method*. Ph.D. Dissertation, Computer Science Department Report STAN-CS-71208, Stanford University.
- [130] GEORGE, A. 1980. *An Automatic One-way Disecction Algorithm for Irregular Finite Element Problems*. SIAM J. Numer. Anal. 17, pp. 740-751.
- [131] GEORGE, A. y HEATH, M.T. 1980. *Solution of Sparse Linear Least Squares Problems Using Givens Rotations*. Linear Algebra and its Applications 34, pp. 69-83.
- [132] GEORGE, A. y LIU, J.W. 1979. *The Design of a User Interface for a Sparse Matrix Package*. ACM Trans. on Mathematical Software 5, pp. 139-162.
- [133] GEORGE, A. y LIU, J.W. 1979. *An Implementation of a Pseudoperipheral Node Finder*. ACM Trans. on Mathematical Software 5, pp. 284-295.
- [134] GEORGE, A. y LIU, J.W. 1981. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice Hall.
- [135] GEORGE, A. y NG, E. 1985. *An Implementation of Gaussian Elimination with Partial Pivoting for Sparse Systems*. SIAM J. Sci. and Statist. Comput. 6, pp. 390-409.
- [136] GEORGE, A. y NG, E. 1984. *SPARSPAK: Waterloo Sparse Matrix Package. User's Guide for SPARSPAK-B*. Research Report CS-84-37, Department of Computer Science, University of Waterloo, Waterloo, Ontario, Canadá.
- [137] GERALD, C.F. y WHEATLEY, P.O. 1994. *Applied Numerical Analysis*. Addison-Wesley Publishing Company.

-
- [138] GIBBS, N.E., POOLE, W.G. y STOCKMEYER, P.K. 1976. *An Algorithm for Reducing the Bandwidth and profile of a Sparse Matrix*. SIAM J. Numer. Anal. 13, pp. 236-250.
 - [139] GILL, P.E. y MURRAY, W. 1974. *Numerical Methods for Constrained Optimization*. Academic Press, Inc.
 - [140] GILL, P.E., MURRAY, W. y WRIGHT, M.H. 1981. *Practical Optimization*. Academic Press, Inc.
 - [141] GILL, P.E., MURRAY, W. y WRIGHT, M.H. 1991. *Numerical Linear Algebra and Optimization. Volume 1*. Addison-Wesley Publishing Company.
 - [142] GILL, P.E., MURRAY, W., SAUNDERS, M.A. y WRIGHT, M.H. 1986. *Maintaining LU Factors of a General Sparse Matrix*. Technical Report SOL 86-8. Systems Optimization Laboratory, Department of Operations Research, Stanford University.
 - [143] GILL, P.E., MURRAY, W., SAUNDERS, M.A., TOMLIN, J.A. y WRIGHT, M.H. 1986. *On Projected Newton Barrier Methods for Linear Programming and an Equivalence to Karmarkar's Projective Method*. Technical Report SOL 85-11R, revision of May 1986. Systems Optimization Laboratory, Department of Operations Research, Stanford University.
 - [144] GILLET, B.E. 1976. *Introduction to Operations Research: A Computer Oriented Algorithmic Approach*. McGraw-Hill Book Company.
 - [145] GOLDFARB, D. y TODD, M.J. 1989. *Linear Programming*. En Handbooks in Operations Research and Management Science. Volume 1: Optimization. Nemhauser, G.L., Rinnooy Kan, A.H.G. y Todd, M.J. eds. North-Holland Publishing Company.
 - [146] GOLDSTEIN, A.A. 1965. *On Steepest Descent*. SIAM J. Control 3, pp. 147-151.
 - [147] GOLOVINA, L.I. 1974. *Algebra lineal y algunas de sus aplicaciones*. Editorial Mir.
 - [148] GOLUB, G.H. y MEURANT, G.A. 1983. *Résolution Numérique des Grands Systèmes Linéaires*. Editions Eyrolles.
 - [149] GOLUB, G.H. y O'LEARY, D.P. 1989. *Some History of the Conjugate Gradient and Lanczos Algorithms: 1948-1976*. SIAM Review 31, pp.50-102.
 - [150] GOLUB, G.H. y REINSCH, C. 1970. *Singular Value Decomposition and Least Squares Solutions*. Numerische Mathematik 14, pp. 403-20.
 - [151] GOLUB, G.H. y VAN LOAN, C.F. 1983. *Matrix Computations*. The Johns Hopkins University Press.
 - [152] GOLUB, G.H. y VAN LOAN, C.F. 1989. *Matrix Computations. Second Edition*. The Johns Hopkins University Press.
 - [153] GOLUB, G.H. y VAN LOAN, C.F. 1996. *Matrix Computations. Third Edition*. The Johns Hopkins University Press.
 - [154] GOMORY, R.E. 1960. *An Algorithm for the Mixed Integer Problem*. RAND Corporation paper RM-2597.
 - [155] GOMORY, R.E. 1963. *An Algorithm for Integer Solutions to Linear Programs*. en Recent Advances in Mathematical Programming. Graves, R. y Wolfe, Ph. eds. McGraw-Hill Book Company.
 - [156] GONDRAN, M. y MINOUX, M. 1979. *Graphes et Algorithmes*. Editions Eyrolles.
 - [157] GONIN, R. y MONEY, A.H. 1989. *Nonlinear L_p Estimation*. Marcel Dekker, Inc.
 - [158] GOOSSENS, M., MITTELBACH, F. y SAMARIN, A. 1994. *The L^AT_EX Companion*. Addison-Wesley Publishing Company.
 - [159] GOOSSENS, M., RAHTZ, S. y MITTELBACH, F. 1997. *The L^AT_EX Graphics Companion. Illustrating Documents with T_EX and PostScript*. Addison-Wesley Publishing Company.

- [160] GRAINGER, J.J. y STEVENSON JR., W.D. 1994. *Power Systems Analysis*. McGraw-Hill Book Company.
- [161] GREENBERG, H. 1971. *Integer Programming*. Academic Press, Inc.
- [162] GRIGORIADIS, M.D. 1986. *An Efficient Implementation of the Network Simplex Method*. Mathematical Programming Study 26, pp. 83-111.
- [163] GRÖTSCHEL, M., LOVÁSZ, L. y SCHRIJVER, A. 1988. *Geometric Algorithms and Combinatorial Optimization*. Springer Verlag.
- [164] HÄFELE, W. y KIRCHMAYER, L.K. EDS. 1981. *Modeling of Large-Scale Energy Systems*. Pergamon Press.
- [165] HAGER, W.W. 1984. *Condition Estimates*. SIAM J. Sci. and Statist. Comput. 5, pp. 311-316.
- [166] HAGER, W.W. 1988. *Applied Numerical Linear Algebra*. Prentice Hall.
- [167] HALL, M. 1956. *An Algorithm for Distinct Representatives*. Amer. Math. Monthly 63, pp. 716-717.
- [168] HALMOS, P.R. 1974. *Finite-Dimensional Vector Spaces*. Springer Verlag.
- [169] HAMMER, P.L., JOHNSON, E.L. y KORTE, B.H. EDS. 1979. *Discrete Optimization III*. North-Holland Publishing Company.
- [170] HÄMMERLIN, G. y HOFFMANN, K. H. 1991. *Numerical Mathematics*. Springer Verlag.
- [171] HAMMING, R.W. 1986. *Numerical Methods for Scientists and Engineers*. Dover Publications, Inc.
- [172] HARWELL LABORATORY 1987. *HARWELL Subroutine Library: A Catalogue of Subroutines (1987)*. Computer Science and Systems Division, AERE Harwell.
- [173] HELLERMAN, E. y RARICK, D. 1971. *Reinversion with the Preassigned Pivot Procedure*. Mathematical Programming 1, pp. 195-216.
- [174] HELLERMAN, E. y RARICK, D. 1972. *The Partitioned Preassigned Pivot Procedure (P^4)*. En Sparse Matrices and their Applications. Rose, D.J. y Willoughby, R.A. eds. Plenum Press.
- [175] HENRICI, P. 1982. *Essentials of Numerical Analysis with Pocket Calculator Demonstrations*. John Wiley and Sons.
- [176] HESTENES, M. 1980. *Conjugate Direction Methods in Optimization*. Springer Verlag.
- [177] HIGHAM, N.J. 1996. *Accuracy and Stability of Numerical Algorithms*. SIAM.
- [178] HILDEBRAND, F.B. 1987. *Introduction to Numerical Analysis*. Dover Publications, Inc.
- [179] HILLIER, F.S. y LIEBERMAN, G.J. 1974. *Introduction to Operations Research*. Holden-Day Inc.
- [180] HILLIER, F.S. y LIEBERMAN, G.J. 1995. *Introduction to Mathematical Programming*. McGraw-Hill, Inc.
- [181] HIMMELBLAU, D.M. ED. 1973. *Decomposition of Large-Scale Problems*. North-Holland Publishing Company.
- [182] HOCKNEY, R.W. 1996. *The Science of Computer Benchmarking*. SIAM.
- [183] HORN, R.A. y JOHNSON, C.R. 1985. *Matrix Analysis*. Cambridge University Press.
- [184] HOUSEHOLDER, A.S. 1975. *The Theory of Matrices in Numerical Analysis*. Dover Publications, Inc.
- [185] HU, T.C. 1970. *Integer Programming and Network Flows*. Addison-Wesley Publishing Company.
- [186] IGNIZIO, J.P. y CAVALIER, T.M. 1994. *Linear Programming*. Prentice Hall.
- [187] ILIĆ, M.D. y LIU, S. 1996. *Hierarchical Power Systems Control. Its Value in a Changing Industry*. Springer Verlag.

-
- [188] INFANGER, G. 1994. *Planning under Uncertainty. Solving Large-Scale Stochastic Linear Programs*. Boyd & Fraser Publishing Company.
 - [189] ISAACSON, E. Y KELLER, H.B. 1994. *Analysis of Numerical Methods*. Dover Publications, Inc.
 - [190] JACOBS, D. ED. 1977. *The State of the Art in Numerical Analysis*. Academic Press, Inc.
 - [191] JENNINGS, A. Y MCKEOWN, J.J. 1992. *Matrix Computation. Second Edition*. John Wiley and Sons.
 - [192] KARLOFF, H. 1991. *Linear Programming*. Birkhäuser.
 - [193] KARMARKAR, N. 1984. *A New Polynomial-Time Algorithm for Linear Programming*. *Combinatorics* 4, pp. 373-395.
 - [194] KARUSH, W. 1939. *Minima of Functions of Several Variables with Inequalities as Side Constraints*. M.Sc. Dissertation, Department of Mathematics, University of Chicago.
 - [195] KAUFMANN, A. Y HENRY-LABORDÈRE, A. 1974. *Méthodes et Modèles de la Recherche Opérationnelle*. Dunod.
 - [196] KENNINGTON, J.L. Y HELGASON, R.V. 1980. *Algorithms for Network Programming*. John Wiley and Sons.
 - [197] KINCAID, D.R. Y HAYES, L.J. EDS. 1990. *Iterative Methods for Large Linear Systems*. Academic Press, Inc.
 - [198] KLEE, V. Y MINTY, G.J. 1972. *How good is the Simplex Algorithm?* En *Inequalities III*. Shisha, O. ed. Academic Press, Inc.
 - [199] KLINGMAN, D., NAPIER, A. Y STUTZ, J. 1974. *NETGEN-A Program for Generating Large-scale (Un)Capacitated Assignment, Transportation and Minimum Cost Flow Network Problems*. *Management Science* 20, pp. 814-821.
 - [200] KOLMAN, B. Y BECK, R.E. 1995. *Elementary Linear Programming with Applications*. Academic Press, Inc.
 - [201] KOPKA, H. Y DALY, P.W. 1995. *A Guide to L^AT_EX 2_ε. Document Preparation for Beginners and Advanced Users*. Addison Wesley Publishing Company.
 - [202] KUESTER, J.L. Y MIZE, J.H. 1973. *Optimization Techniques with FORTRAN*. McGraw-Hill Book Company.
 - [203] KUHN, H.W. Y TUCKER, A.W. 1951. *Nonlinear Programming*. En *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*. University of California Press.
 - [204] KÜNZI, H.P. Y KRELLE, W. 1969. *La Programmation Non Linéaire*. Gauthier-Villars.
 - [205] LAMPORT, L. 1994. *L^AT_EX. A Document Preparation System. User's Guide and Reference Manual*. Addison-Wesley Publishing Company.
 - [206] LAND, A. Y POWELL, S. 1973. *FORTRAN Codes for Mathematical Programming*. John Wiley and Sons.
 - [207] LANG, S. 1968. *Analysis I*. Addison-Wesley Publishing Company.
 - [208] LANG, S. 1969. *Analysis II*. Addison-Wesley Publishing Company.
 - [209] LANG, S. 1983. *Linear Algebra*. Addison-Wesley Publishing Company.
 - [210] LARSON, R.E. 1968. *State Increment Dynamic Programming*. American Elsevier Publishing Company, Inc.
 - [211] LASCAUX, P. Y THÉODOR, R. 1986. *Analyse Numérique Matricielle Appliquée à l'Art de l'Ingénieur. Tome 1*. Masson, S.A.

- [212] LASCAUX, P. Y THÉODOR, R. 1987. *Analyse Numérique Matricielle Appliquée a l'Art de l'Ingénieur. Tome 2*. Masson, S.A.
- [213] LASDON, L.S. 1970. *Optimization Theory for Large Systems*. Macmillan Publishing Company Inc.
- [214] LAWLER, E.L. 1976. *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart and Winston.
- [215] LAWSON, C.L. Y HANSON, R.J. 1974. *Solving Least Squares Problems*. Prentice Hall.
- [216] LAWSON, C.L. Y HANSON, R.J. 1995. *Solving Least Squares Problems*. SIAM.
- [217] LEIFMAN, L.J. ED. 1990. *Functional Analysis, Optimization, and Mathematical Economics*. Oxford University Press.
- [218] LEMKE, C.E. 1954. *The Dual Method of Solving the Linear Programming Problem*. Naval Research Logistics Quarterly 1, pp. 36-47.
- [219] LERMAN, S.R. 1993. *Problem Solveing and Computation for Scientists and Engineers. An Introduction Using C*. Prentice Hall.
- [220] LEVENBERG, K. 1944. *A Method for the Solution of Certain Problems in Least Squares*. Quart. Appl. Math. 2, pp. 164-168.
- [221] LIEBMAN, J., LASDON, L., SCHRAGE, L. Y WARREN, A. 1986. *Modeling and Optimization with GINO*. The Scientific Press.
- [222] LIEBMAN, J., LASDON, L., SCHRAGE, L. Y WARREN, A. 1986. *Modeling and Optimization with GINO*. The Scientific Press.
- [223] LINDFIELD, G. Y PENNY, J. 1995. *Numerical Methods Using Matlab*. Ellis Horwood.
- [224] LUENBERGER, D.G. 1969. *Optimization by Vector Space Methods*. John Wiley and Sons.
- [225] LUENBERGER, D.G. 1984. *Linear and Nonlinear Programming*. Addison-Wesley Publishing Company.
- [226] LUENBERGER, D.G. 1989. *Programación lineal y no lineal*. Addison-Wesley Iberoamericana.
- [227] LUSTIG, I.J., MARSTEN, R.E. Y SHANNO, D.F. 1992. *On Implementing Mehrotra's Predictor-Corrector Interior-Point Method for Linear Programming*. SIAM J. Optimization, Vol. 2, No. 3, pp. 435-449.
- [228] MANDL, C. 1979. *Applied Network Optimization*. Academic Press, Inc.
- [229] MANGASARIAN, O.L. 1994. *Nonlinear Programming*. SIAM.
- [230] MANNEBACK, P. 1985. *On Some Numerical Methods for Solving Large Sparse Linear Least Squares Problems*. Ph.D. Dissertation, Facultés Iniversitaires Notre-Dame de la Paix, Namur, Bélgica.
- [231] MARKOWITZ, H.M. 1957. *The Elimination Form of the Inverse and its Application to Linear Programming*. Management Science 3, pp. 255-269.
- [232] MARQUARDT, D. 1963. *An Algorithm for Least Squares Estimation of Nonlinear Parameters*. SIAM J. Appl. Math. 11, pp. 431-441.
- [233] MATHEWS, J.H. 1992. *Numerical Methods for Mathematics, Science, and Engineering*. Prentice Hall.
- [234] MEHROTRA, S. 1992. *On the Implementation of a Primal-Dual Interior Point Method*. SIAM J. Optimization, Vol. 2, No. 4, pp. 575-601.
- [235] MESIROV, J.P. ED. 1991. *Very Large Scale Computation in the 21st Century*. SIAM.
- [236] METCALF, M. Y REID, J. 1990. *FORTAN 90 Explained*. Oxford University Press.

-
- [237] METCALF, M. Y REID, J. 1996. *FORTRAN 90/95 Explained*. Oxford University Press.
 - [238] MCCORMICK, G.P. 1983. *Nonlinear Programming*. John Wiley and Sons.
 - [239] MOLER, C.B., LITTLE, J.N. Y BANGERT, S. 1987. *PC-Matlab User's Guide*. The MathWorks, Inc.
 - [240] MORÉ, J.J. Y WRIGHT, S.J. 1993. *Optimization Software Guide*. SIAM.
 - [241] MINOUX, M. 1986. *Mathematical Programming: Theory and Algorithms*. John Wiley and Sons.
 - [242] MINOUX, M. Y BARTNIK, G. 1986. *Graphes, Algorithmes, Logiciels*. Dunod.
 - [243] MURTAGH, B.A. Y SAUNDERS, M.A. 1978. *Large Scale Linearly Constrained Optimization*. Mathematical Programming 14, pp. 41-72.
 - [244] MURTAGH, B.A. Y SAUNDERS, M.A. 1982. *A Projected Lagrangian Algorithm and its Implementation for Sparse Nonlinear Constraints*. Mathematical Programming Study 16, pp. 84-117.
 - [245] MURTAGH, B.A. Y SAUNDERS, M.A. 1987. *MINOS 5.1 User's Guide*. Systems Optimization Laboratory, Department of Operations Research, Stanford University.
 - [246] MURTY, K.G. 1983. *Linear Programming*. John Wiley and Sons.
 - [247] MURTY, K.G. 1992. *Network Programming*. Prentice Hall.
 - [248] NAG 1992. *C Library Manual*. Numerical Algorithms Group, Mark 2, Oxford, England.
 - [249] NAG 1993. *FORTRAN Library Manual*. Numerical Algorithms Group, Mark 16, Oxford, England.
 - [250] NAKAMURA, S. 1996. *Numerical Analysis and Graphic Visualization with Matlab*. Prentice Hall PTR.
 - [251] NASH, S.G. Y SOFER, A. 1996. *Linear and Nonlinear Programming*. The McGraw-Hill Companies.
 - [252] NEMHAUSER, G.L., RINNOOY KAN, A.H.G. Y TODD, M.J. EDS. 1989. *Handbooks in Operations Research and Management Science. Volume 1: Optimization*. North-Holland Publishing Company.
 - [253] NEMHAUSER, G.L. Y WOLSEY, L.A. 1988. *Integer and Combinatorial Optimization*. John Wiley and Sons.
 - [254] NEMHAUSER, G.L. Y WOLSEY, L.A. 1989. *Integer Programming*. En Handbooks in Operations Research and Management Science. Volume 1: Optimization. Nemhauser, G.L., Rinnooy Kan, A.H.G. y Todd, M.J. eds. North-Holland Publishing Company.
 - [255] NERING, E.D. Y TUCKER, A.W. 1993. *Linear Programs and Related Problems*. Academic Press, Inc.
 - [256] NESTEROV, Y. Y NEMIROVSKII, A. 1994. *Interior-Point Polynomial Algorithms in Convex Programming*. SIAM.
 - [257] NIEDERREITER, H. 1992. *Random Number Generation and Quasi-Monte Carlo Methods*. SIAM.
 - [258] ORCHARD-HAYS, W. 1968. *Advanced Linear Programming Computing Techniques*. McGraw-Hill Book Company.
 - [259] ORTEGA, J.M. 1988. *Introduction to Parallel and Vector Solution of Linear Systems*. Plenum Press.
 - [260] ORTEGA, J.M. Y RHEINBOLDT, W.C. 1970. *Iterative Solution of Nonlinear Equations in Several Variables*. Academic Press, Inc.
 - [261] PADBERG, M. 1995. *Linear Programming and Extensions*. Springer Verlag.
 - [262] PAI, M. A. 1986. *Computer Techniques in Power System Analysis*. McGraw-Hill Book Company.
 - [263] PAIGE, C.C. 1979. *Computer Solution of Perturbation Analysis of Generalized Linear Least Squares Problems*. Mathematics of Computation 33, pp. 171-184.

- [264] PAIGE, C.C. 1979. *Fast Numerically Stable Computations for Generalized Linear Least Squares Problems*. SIAM J. Numer. Anal. 16, pp. 165-171.
- [265] PANNELL, D.J. 1997. *Introduction to Practical Linear Programming*. John Wiley and Sons.
- [266] PANIK, M.J. 1996. *Linear Programming: Mathematics, Theory and Algorithms*. Kluwer Academic Publishers.
- [267] PARKER, R.G. Y RARDIN, R.L. 1988. *Discrete Optimization*. Academic Press, Inc.
- [268] PARTER, S.V. 1961. *The Use of Linear Graphs in Gaussian Elimination*. SIAM Review 3, pp. 119-130.
- [269] PATEL, R.V. LAUB, A.J. Y VAN DOOREN, P.M. EDS. 1994. *Numerical Linear Algebra Techniques for Systems and Control*. IEEE Press.
- [270] PFAFFENBERGER, R.C. Y WALKER, D.A. 1976. *Mathematical Programming for Economics and Business*. The Iowa State University Press.
- [271] PHILLIPS, C. Y CORNELIUS, B. 1986. *Computational Numerical Methods*. Ellis Horwood Limited.
- [272] PHILLIPS, D.T., RAVINDRAN, A. Y SOLBERG, J. 1976. *Operations Research: Principles and Practice*. John Wiley and Sons.
- [273] PHILLIPS, G.M. Y TAYLOR, P.J. 1996. *Theory and Applications of Numerical Analysis*. Academic Press, Inc.
- [274] PIERRE, D.A. 1986. *Optimization Theory with Applications* 1986. Dover Publications, Inc.
- [275] PIERRE, D.A. Y LOWE, M.J. 1975. *Mathematical Programming Via Augmented Lagrangians. An Introduction with Computer Programs*. Addison-Wesley Publishing Company.
- [276] PIKE, R.W. 1986. *Optimization for Engineering Systems*. Van Nostrand Reinhold Company.
- [277] PISSANETZKY, S. 1984. *Sparse Matrix Technology*. Academic Press, Inc.
- [278] PLYBON, B.F. 1992. *An Introduction to Applied Numerical Analysis*. PWS-Kent Publishing Company.
- [279] POWELL, M.J.D. ED. 1982. *Nonlinear Optimization*. Academic Press, Inc.
- [280] PRESS, W.H., FLANNERY, B.P., TEUKOLSKY, S.A. Y VETTERLING, W.T. 1986. *Numerical Recipes in FORTRAN. The Art of Scientific Computing*. Cambridge University Press.
- [281] PRESS, W.H., TEUKOLSKY, S.A., VETTERLING, W.T. Y FLANNERY, B.P. 1992. *Numerical Recipes in C. The Art of Scientific Computing*. Second Edition. Cambridge University Press.
- [282] PRESS, W.H., TEUKOLSKY, S.A., VETTERLING, W.T. Y FLANNERY, B.P. 1996. *Numerical Recipes in FORTRAN 90. The Art of Parallel Scientific Computing*. Cambridge University Press.
- [283] REID, J.K. ED. 1971. *Large Sparse Sets of Linear Equations*. Academic Press, Inc.
- [284] REDWINE, C. 1995. *Upgrading to FORTRAN 90*. Springer Verlag.
- [285] REID, J.K. 1982. *A Sparsity-Exploiting Variant of the Bartels-Golub Decomposition for Linear Programming Bases*. Mathematical Programming 24, pp. 55-69.
- [286] REKLAITIS, G.V., RAVINDRAN, A. Y RAGSDALL, K.M. 1983. *Engineering Optimization. Methods and Applications*. John Wiley and Sons.
- [287] RIAZA, R. Y ÁLVAREZ, M. 1996. *Cálculo Infinitesimal. Vol. I*. Sociedad de Amigos de la Escuela Técnica Superior de Ingenieros Industriales de Madrid.
- [288] RIAZA, R. Y ÁLVAREZ, M. 1997. *Cálculo Infinitesimal. Vol. II*. Sociedad de Amigos de la Escuela Técnica Superior de Ingenieros Industriales de Madrid.
- [289] RÍBNIKOV, K. 1987. *Historia de las matemáticas*. Editorial Mir.

-
- [290] RICE, J.R. 1966. *Experiments on Gram-Schmidt Orthogonalization*. Mathematics of Computation 20, pp. 325-328.
- [291] RICE, J.R. 1983. *Matrix Computations and Mathematical Software*. McGraw-Hill Book Company.
- [292] RICE, J.R. 1993. *Numerical Methods, Software, and Analysis*. Academic Press, Inc.
- [293] ROOS, C. TERLAKY, T. Y VIAL, J.-PH. 1997. *Theory and Algorithms for Linear Optimization. An Interior Point Approach*. John Wiley and Sons.
- [294] ROSE, D.J. Y WILLOUGHBY, R.A. EDS. 1972. *Sparse Matrices and Their Applications*. Plenum Press.
- [295] ROSEN, J.B., MANGASARIAN, O.L. Y RITTER, K. EDS. 1970. *Nonlinear Programming*. Academic Press, Inc.
- [296] RUTISHAUSER, H. 1990. *Lectures on Numerical Mathematics*. Birkhäuser.
- [297] SAAD, Y. 1994. *SPARSKIT: A Basic Tool Kit for Sparse Matrix Computations. Version 2*. Computer Science Department, University of Minnesota, Minneapolis, EE. UU.
- [298] SAIGAL, R. 1995. *Linear Programming. A Modern Integrated Analysis*. Kluwer Academic Publishers.
- [299] SALKIN, H.M. 1975. *Integer Programming*. Addison-Wesley Publishing Company.
- [300] SALKIN, H.M. Y MATHUR, K 1989. *Introduction to Integer Programming*. North-Holland Publishing Company.
- [301] SALKIN, H.M. Y SAHA, J. EDS. 1975. *Studies in Linear Programming*. North-Holland Publishing Company.
- [302] SARGENT, R.W.H. Y WESTERBERG, A.W. 1964. *Speed-up in Chemical Engineering Design*. Trans. Inst. Chem. Eng. 42, pp.190-197.
- [303] SAUNDERS, M.A. 1976. *A Fast Stable Implementation of the Simplex Method using Bartels-Golub Updating*. En *Sparse Matrix Computations*. Bunch, J.R. y Rose, D.J. eds. Academic Press, Inc.
- [304] SAVILLE, D.J. Y WOOD, G.R. 1991. *Statistical Methods: The Geometric Approach*. Springer Verlag.
- [305] SCHENDEL, U. 1989. *Sparse Matrices. Numerical Aspects with Applications to Scientists and Engineers*. Ellis Horwood Limited.
- [306] SCHERER, C.R. 1977. *Estimating Electric Power System Marginal Costs*. North-Holland Publishing Company.
- [307] SCHITTKOWSKI, K. 1980. *Nonlinear Programming Codes. Information, Tests, Performance*. Lecture Notes in Economics and Mathematical Systems, 183. Springer Verlag.
- [308] SCHITTKOWSKI, K. 1981. *Test Examples for Nonlinear Programming Codes*. Lecture Notes in Economics and Mathematical Systems, 187. Springer Verlag.
- [309] SCHITTKOWSKI, K. ED. 1985. *Computational Mathematical Programming*. Springer Verlag.
- [310] SCHRAGE, L. 1989. *User's Manual for Linear, Integer, and Quadratic Programming with LINDO*. The Scientific Press.
- [311] SCHRAGE, L. 1991. *LINDO. An Optimization Modeling System*. Boyd & Fraser Publishing Company.
- [312] SCHRIJVER, A. 1986. *Theory of Linear and Integer Programming*. John Wiley and Sons.
- [313] SEDGEWICK, R. 1992. *Algorithms in C++*. Addison-Wesley Publishing Company.
- [314] SENIOR, T.B.A. 1986. *Mathematical Methods in Electrical Engineering*. Cambridge University Press.

- [315] SHAPIRO, J.F. 1979. *Mathematical Programming. Structures and Algorithms*. John Wiley and Sons.
- [316] SIDDALL, J.N. 1982. *Optimal Engineering Design. Principles and Applications*. Marcel Dekker, Inc.
- [317] SIERKSMA G. 1996. *Linear and Integer Programming: Theory and Practice*. Marcel Dekker, Inc.
- [318] SIMONNARD, M. 1972. *Programmation Linéaire. Technique du Calcul Économique. Fondements*. Dunod.
- [319] SIMONNARD, M. 1973. *Programmation Linéaire. Technique du Calcul Économique. Extensions*. Dunod.
- [320] SMITH, B.T., BOYLE, J.M., DONGARRA, J.J., GARBOW, B.S., IKEBE, Y., KLEMA, V.C. y MOLER, C.B. 1976. *Matrix Eigensystem Routines-EISPACK Guide*. Springer Verlag.
- [321] SORDET, J. 1970. *La Programmation Linéaire Appliquée à l'Entreprise*. Dunod.
- [322] SPEDICATO, E. ED. 1991. *Computer Algorithms for Solving Linear Algebraic Equations. The State of the Art*. Springer Verlag, NATO Scientific Affairs.
- [323] STAGG, G.W. y EL-ABIAD, A.H. 1968. *Computer Methods in Power Systems Analysis*. McGraw-Hill Book Company.
- [324] STEVENSON, W. D. 1984. *Elements of Power System Analysis*. McGraw-Hill Book Company.
- [325] STEWART, G.W. 1973. *Introduction to Matrix Computations*. Academic Press, Inc.
- [326] STEWART, G.W. 1996. *Afternotes on Numerical Analysis*. SIAM.
- [327] STOER, J. y BULIRSCH, R. 1980. *Introduction to Numerical Analysis*. Springer Verlag.
- [328] STOTT, B. y ALSAÇ, O. 1974. *Fast Decoupled Load Flow*. IEEE Trans. on Power Apparatus and Systems PAS-93, pp. 859-869.
- [329] STRANG, G. 1976. *Linear Algebra and its Applications*. Academic Press, Inc.
- [330] SULTAN, A. 1993. *Linear Programming. An Introduction with Applications*. Academic Press, Inc.
- [331] TARJAN, R. 1972. *Depth-First Search and Linear Graph Algorithms*. SIAM J. Computing 1, pp. 146-160.
- [332] TEWARSON, R.P. 1973. *Sparse Matrices*. Academic Press, Inc.
- [333] THOMAS, B.J. 1996. *The Internet for Scientists and Engineers*. Oxford University Press.
- [334] TINNEY, W.F. y WALKER, J.W. 1967. *Direct Solution of Sparse Network Equations by Optimally Ordered Triangular Factorizations*. Proceedings of the IEEE 55, pp.1801-1809.
- [335] TOMLIN, J.A. 1970. *Branch-and-Bound Methods for Integer and Non-Convex Programming*. En Integer and Nonlinear Programming. Abadie, J. ed. North-Holland Publishing Company.
- [336] TOMLIN, J.A. 1972. *Pivoting for Size and Sparsity in Linear Programming*. J. Inst. Maths. Appls. 10, pp. 289-295.
- [337] TREFETHEN, L.N. y BAU, D. 1997. *Numerical Linear Algebra*. SIAM.
- [338] VAN DE PANNE, C. 1976. *Linear Programming and Related Techniques*. North-Holland Publishing Company.
- [339] VAN LOAN, C.F. 1997. *Introduction to Scientific Computing. A Matrix-Vector Approach Using Matlab*. Prentice Hall.
- [340] VANDERBEI, R.J. 1995. *LOQO: An Interior Point Code for Quadratic Programming*. Princeton University.

-
- [341] VANDERBEI, R.J. 1996. *Linear Programming. Foundations and Extensions*. Kluwer Academic Publishers.
 - [342] VARGA, R.S. 1962. *Matrix Iterative Analysis*. Prentice Hall.
 - [343] WALSH, G.R. 1975. *Methods of Optimization*. John Wiley and Sons.
 - [344] WATKINS, D.S. 1991. *Fundamentals of Matrix Computations*. John Wiley and Sons.
 - [345] WHITE, R.E. 1985. *An Introduction to Finite Element Method with Applications to Nonlinear Problems*. John Wiley and Sons.
 - [346] WHITEHOUSE, G.E. Y WECHSLER, B. 1976. *Applied Operations Research: A Survey*. John Wiley and Sons.
 - [347] WILKINSON, J.H. 1965. *The Algebraic Eigenvalue Problem*. Oxford University Press.
 - [348] WILKINSON, J.H. 1994. *Rounding Errors in Algebraic Processes*. Dover Publications Inc.
 - [349] WINSTON, W.L. 1994. *Operations Research. Applications and Algorithms*. Duxbury Press.
 - [350] WOLFE, M.A. 1978. *Numerical Methods for Unconstrained Optimization. An Introduction*. Van Nostrand Reinhold Company.
 - [351] WOLFE, P. 1961. *A Duality Theorem for Non-Linear Programming*. Quart. Appl. Math. 19, N^o 3.
 - [352] WOLFE, P. 1967. *Methods of Nonlinear Programming*. En *Nonlinear Programming*. Abadie J. ed. North-Holland Publishing Company.
 - [353] WOOD, A.J. Y WOLLENBERG, B.F. 1984. *Power Generation Operation and Control*. John Wiley and Sons.
 - [354] WRIGHT, S.J. 1997. *Primal-Dual Interior Point Methods*. SIAM.
 - [355] YOUNG, D.M. Y GREGORY, R.T. 1988. *A Survey of Numerical Mathematics. Vol. I y II*. Dover Publications, Inc.

Índice de materias

β , base de numeración de un ordenador, 700

A

Aasen, método de 53–58

número de operaciones, 54

Abierto, conjunto o subconjunto, 691

Accesibilidad, de un nudo en un grafo, en un digrafo, 230, 248

matriz de, en un digrafo, 248

Adherencia, 691

de un conjunto, 691

punto de, 691

Adyacente,

conjunto,

de un digrafo, 248

de un grafo, 229

nudo,

de un digrafo, 248

de un grafo, 229

Algebra, 681

Algoritmo, 3

Algoritmo(s), ver también Método(s)

Aasen, para la factorización LTL^T de una matriz simétrica indefinida sin pivotación, 55

con pivotación, 56

actualización del vector $s(\cdot)$ en el método *simplex* especializado para optimización de flujos en redes, 520

árbol maximal, obtención en un digrafo, 507

branch and bound, enumerativos, o de ramificación y acotamiento, 649

Broyden, cuasi Newton para solución de sistemas de ecuaciones no lineales, 323

Bunch y Kaufman, para la factorización UBU^T de una matriz simétrica indefinida con pivotación, 63

eliminación de Gauss con pivotación parcial, para solución de $Ax = b$, 16

eliminación directa, para resolución de MCI, 132

escalado afín, de puntos interiores para programación lineal,

primal, 578

dual, 591

primal-dual, 602

Cholesky, para la factorización $G^T G$ de una matriz simétrica definida positiva, por filas, 44

por columnas, 46

Cholesky, con pivotación, para la factorización $G^T G$ de una matriz simétrica semidefinida positiva, 49

Cholesky, sin pivotación, para la factorización $G^T G$ de una matriz simétrica semidefinida positiva, 48

Crout, para la factorización LU_1 de una matriz, 30

con pivotación parcial, 33

Crout, para la factorización $L_1 U$ de una matriz, 36

Cuthill-McKee, para reducción del ancho de banda de una matriz dispersa simétrica, 238

selección nudo inicial, 241

Cuthill-McKee inverso, para reducción de la envolvente de una matriz dispersa simétrica, 242

Dantzig y Wolfe, descomposición para programación lineal, 538

dual del *simplex*, 481

para variables acotadas, 483

Doolittle, para la factorización $L_1 U$ de una matriz, 37

factorización LDL^T , de una matriz simétrica, 41

Gauss-Newton, para solución de problemas no lineales de mínimos cuadrados, 346

Gauss-Seidel, para solución de $Ax = b$

- iterativamente, 150
- George y Heath, para la ortogonalización de una matriz dispersa y resolución de mínimos cuadrados, 272
- Golub-Kahan, etapa k del de Golub-Reinsch para la descomposición numérica en valores singulares de una matriz cualquiera, 121
- Golub-Reinsch, para la descomposición numérica en valores singulares de una matriz cualquiera, 122
- Gradientes conjugados, para solución de ecuaciones normales, $A^T(A\mathbf{x} - \mathbf{b})$, 196
- Gradientes conjugados, para solución de $A\mathbf{x} = \mathbf{b}$, 188
- Gradientes conjugados con preconditionamiento, para solución de $A\mathbf{x} = \mathbf{b}$, 191
- grado mínimo, para reducción del número de elementos de relleno en una matriz dispersa, 235
- Gram-Schmidt clásico, para la ortonormalización de los vectores columna de una matriz, 84
- Gram-Schmidt modificado, para la ortonormalización de los vectores columna de una matriz, 86
- Hall, para la búsqueda de un transversal completo de una matriz dispersa, 251–254
- inverso de Cuthill-McKee, para reducir la envolvente de una matriz dispersa simétrica, 242
- Jacobi, para solución de $A\mathbf{x} = \mathbf{b}$ iterativamente, 147
- Jacobi, variante de Newton-Raphson para sistemas de ecuaciones no lineales, 316
- Karmarkar, escalado proyectivo, para programación lineal, 567
- Levenberg-Marquardt, para solución de problemas no lineales de mínimos cuadrados, 353
- Máxima pendiente, para solución de $A\mathbf{x} = \mathbf{b}$, 176
- multiplicadores simplex, obtención en programación lineal para flujos en redes, 514
- Newton-Raphson, para sistemas de ecuaciones no lineales, 307
- Newton-Raphson para sistemas de ecuaciones no lineales con el criterio de salvaguarda de Armijo, 333
- Paige, para la resolución del problema generalizado de mínimos cuadrados, 130
- planos cortantes de Gomory, 643
- primal-dual del *simplex*, 489
- programas enteros, algoritmo general basado en relajaciones sucesivas lineales, 640
- puntos interiores, para solución de programas lineales,
 - primal de escalado afín, 578
 - dual de escalado afín, 591
 - primal-dual, 602
- relajación SOR, para solución de $A\mathbf{x} = \mathbf{b}$ iterativamente, 165
- Sargent y Westerberg, para triangularizar por bloques una matriz dispersa no simétrica, 256
- simplex* revisado, para programación lineal, 420
 - dual del *simplex*, 481
 - en dos fases, 432
 - forma producto de la inversa de la base, 446
 - para variables acotadas, 454
 - primal-dual del *simplex*, 489
- solución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ mediante transformaciones ortogonales de Householder, 95
- solución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ mediante transformaciones ortogonales de Givens, 108
- solución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|A\mathbf{x} - \mathbf{b}\|_2$ mediante transformaciones ortogonales rápidas de Givens, 113
- solución de $A\mathbf{x} = \mathbf{b}$ siendo A dispersa, 221
- solución de problemas de mínimos cuadrados con matrices dispersas mediante ecuaciones normales, 269
- ramificación y acotamiento o *branch and bound*, 649
- Tarjan, para triangularizar por bloques una matriz dispersa no simétrica, 260
- triangularización de una base, en programación lineal para flujos en redes mediante el método *simplex*, 510
- Almacenamiento, en ordenador de matrices dispersas, 202–208
 - por coordenadas, 202
 - por filas/columnas, 203
 - por listas encadenadas, 207
 - por perfil o envolvente, 204
- Amplitud de paso, 171
 - en el método *simplex*, 416

- Análisis post-optimización, en programación lineal, 492
- Análisis de sensibilidad, en programación lineal, 492–493
- Ancho de banda de una matriz, 205
- simétrica, 206
- Aplicación, 672
- biyectiva, 672
- dominio de definición, origen, 672
- dominio de valores, 672
- imagen, 672
- inyectiva, 672
- lineal, 678
- núcleo, 679
- permutaciones, 672
- suprayectiva, 672
- traspuesta, 679
- Árbol, de un digrafo, 500
- maximal, 500
- algoritmo para obtención en digrafo, 507
- Árbol, de un grafo, 230
- ascendiente/descendiente, relaciones, 230
- enraizado, 230
- cociente, 230
- maximal, 230
- nudo raíz, 230
- numeración monótona, 230
- partición, 230
- Arbol de enumeración o enumerativo, en programación entera, 646
- poda, 646
- criterios de, 647
- ver también algoritmos enumerativos, de ramificación y acotamiento o *branch and bound*
- Arco(s) de un grafo, de un digrafo, 227, 248, 500
- nudo origen, 227, 248, 500
- nudo destino, 227, 248, 500
- Aristas de un grafo, ver Arcos de un grafo
- Aritmética en ordenador, 699
- Armijo, regla de, 331
- Ascendiente/descendiente, relaciones en un árbol, 230
- Asignación de precio, *price out*, en programación lineal, 415, 420, 454
- Asignación de tráfico, problema, 375
- Autovalor, 682
- B**
- B*, base o matriz básica de un programa lineal, 392
- Banach, espacio vectorial de, 677
- Barrera, función, para programación lineal, 585
- Bartels y Golub, factorización *LU* de la base en *simplex*, 447
- Base, de un espacio vectorial, 674
- canónica, 674
- Base, de un programa lineal, 392
- Básica, solución de un programa lineal, 392
- factible, 395
- factible degenerada, 395
- BBMI, programa, 450, 660
- listado, 791
- Bilineal, forma, 678
- Bit, 699
- Biyectiva, aplicación, 672
- Bland, regla para el método *simplex*, 429
- Bola,
- abierta, 691
- cerrada, 691
- BOUNDS, 771
- Branch and bound*, algoritmo enumerativo, o de ramificación y acotamiento, 649
- Broyden, método para solución de sistemas de ecuaciones no lineales, 321
- convergencia, 326
- fórmula de, 321
- implementación práctica, 330
- BTRAN, operación del *simplex*, 446
- Bunch y Kaufman, método de, 60–66
- número de operaciones, 63
- Bunch y Parlett, método de, 59
- número de operaciones, 60
- Búsqueda, en amplitud o anchura, en programación entera, 653
- Byte, 699
- C**
- C**, cuerpo de los números complejos, 671, 672
- Cabeza, de un arco, ver Nudo Destino
- Cadena, en un grafo dirigido, 500
- euleriana, 500
- hamiltoniana, 500
- Camino,
- de un digrafo, 248, 500
- creciente, 251
- euleriano, 500
- hamiltoniano, 500
- de un grafo, 229
- longitud, 229
- nudo de partida, 229
- nudo de llegada, 229

- Canónica,
 - base de un espacio vectorial, 674
 - forma de las condiciones de un programa lineal, 392
- Cara, de un politopo, 627
 - apropiada, 627
- Cauchy, método de, 174
 - ver también método de máxima pendiente
 - sucesión de, 677
- Cauchy-Schwarz, desigualdad de 678
- CCNET, programa, 521
 - listado, 819
- Chan, método de, 116
 - número de operaciones, 127
- Cholesky, factorización, 41–49
 - con pivotación para matrices simétricas semidefinidas positivas, 49
 - existencia y unicidad, 42
 - número de operaciones, 47
- CHUZR, operación del *simplex*, 446
- Ciclado, en programación lineal, 429
 - ver también reglas lexicográfica y de Bland
- Ciclo,
 - en un digrafo, 248, 500
 - en un grafo, 229
 - euleriano, 500
 - hamiltoniano 500
- Circuito, de un grafo dirigido, 500
- Coefficientes de coste, de programa lineal, 366
- Cola, de un arco, ver Nudo Origen
- COLUMNS, 768
- Combinaciones convexas, 383
- Combinación lineal, de vectores, 674
- Compacto, subconjunto o conjunto, 691
- Complementariedad de holguras, teorema, en programación lineal, 473
 - condición o condiciones, 475
- Complemento, de un subconjunto, 672
- Completo, espacio vectorial, 677
- Componentes conexos, 230
 - fuertemente conexos, en un digrafo, 249
- Condición,
 - de complementariedad de holguras, en programación lineal, 475
 - de Lipschitz, 693
 - número de, de una matriz, 69
- Condicionamiento,
 - de un problema, 66
 - de un sistema de ecuaciones lineales, 66
- Condiciones,
 - de complementariedad de holguras, 475
 - de enlace, en descomposición de Dantzig-Wolfe, 528
 - de Karush-Kuhn-Tucker, 476
 - de Kuhn-Tucker, 476
 - de un programa lineal, 366
 - necesarias y suficientes de primer y segundo orden de un mínimo, 695
- Conexo,
 - digrafo, 249
 - grafo, 230
 - componentes conexos, 230
- Conjunto, 671
 - abierto, 691
 - accesible, de un grafo, 230
 - adyacente
 - de un digrafo, 248
 - de un grafo, 229
 - cerrado, 691
 - compacto, 691
 - complemento, de un subconjunto, 672
 - convexo, 383
 - punto extremo, 384
 - elementos de, 671
 - interior de, 691
 - numerable, 672
 - separador, de un grafo, 230
 - vacío, 671
- Cono, 384
 - convexo, 384
- Continuidad, 693
 - de Lipschitz, 693
- Convergencia, de una sucesión en espacio vectorial normado, 677
- Convergencia, orden, 281
 - cuadrática, 281
 - lineal, 281
 - superlineal, 281
- Convergencia, tasa o relación, 281
- Convexo,
 - conjunto, 383
 - cono, 384
- Coordenadas, forma de almacenar matrices dispersas en un ordenador, 202
- Corte de materiales, problema, 546
- Corte fraccionario de Gomory, 641
- Coste reducido, en programación lineal, 412
- Cota
 - inferior máxima, o ínfimo, 672
 - superior mínima, o supremo, 672
- COTA INICIAL F.O., 772
- Cramer, fórmulas, 22

- Criterio o regla
 de Armijo, 331
 de Markowitz, 261
 Crout, método de, 29–36
 versión LU_1 , 29
 con pivotación, 32
 número de operaciones, 34
 versión L_1U , 34
 y su relación con la eliminación de Gauss, 31
 Cuadrática, forma, 170, 687
 Cuasi Newton, métodos para resolver sistemas de ecuaciones no lineales, 320–331
 Broyden, 320–331
 Cuthill-McKee,
 algoritmo para la reducción del ancho de banda de una matriz dispersa simétrica, 238–243
 selección nudo inicial, 241
 inverso, algoritmo para la reducción de la envolvente de una matriz dispersa simétrica, 242–243
- D**
- Dantzig-Wolfe, descomposición de, 527–545
 Definida positiva,
 forma cuadrática, 687
 matriz, 689
 Degeneración, en programación lineal, 429
 Dependencia lineal, vectores de espacio vectorial, 674
Depth, profundidad, estructura de datos para programación lineal para flujos en redes, 512
 Descomposición, ver también Factorización,
 de Benders, 633
 de Dantzig-Wolfe, 527–545
 problema maestro, 532
 problemas con estructura en escalera, 545
 en valores singulares, 75
 numérica, 115
 Desigualdad,
 de Cauchy-Schwarz, 678
 dominante, 627
 válida, 627
 maximal, 627
 Desigualdades, generación de, en programación entera, 627–632
 desigualdades disyuntivas, 630
 desigualdades superaditivas, 631
 redondeo entero, 627
 Destino, nudo destino de un arco, 227, 248, 500
 Diagonal dominante, matriz de, 156, 691
 Diámetro, de un grafo, 229
 Dicotomía, problema, 617
 Dieta alimenticia, problema, 368
 Diferencias finitas, método de Newton-Raphson,
 para ecuaciones no lineales de una variable, 295
 para sistemas de ecuaciones no lineales, 313
 Dígitos significativos, número, de una máquina, 705
 Digrafo, ver también Grafo dirigido, 227, 248, 500
 acíclico, 500
 conexo, 249, 500
 de eliminación, 202
 fuertemente conexo, 249
 componente, 249
 Dimensión, de espacio vectorial, 674
 Dirección,
 de descenso, en métodos iterativos para solución de $Ax = b$, 170
 de máxima pendiente, 171
 direcciones conjugadas, 177
 relajación en una variable, 171
 relajación SOR, 173
 de un politopo, 397
 extrema, 397, 531
 Direcciones conjugadas, 178
 Dispersas, matrices, 202
 Distancia,
 en espacio vectorial normado, 677
 entre dos nudos de un digrafo, 248
 entre dos nudos de un grafo, 229
 División, de la región factible de un programa entero, 645
 Dominio
 de definición, de una aplicación, 672
 de valores, de una aplicación, 672
 Doolittle, método de, 36–39
 número de operaciones, 36
 Dual,
 combinaciones posibles primal-dual, 471
 del *simplex*, algoritmo, 481
 espacio vectorial, 679
 programa, 468
 de uno lineal en forma estándar, 468
 Dualidad, en programación lineal, 465–493
 débil, 469
 interpretación económica, 476
- E**
- Ecuación característica, de una matriz, 685
 Ecuaciones normales, 79, 267

- condicionamiento, 81
 - y dispersidad, 81
 - y mínimos cuadrados lineales, 79–81
 - con matrices dispersas, 267
 - EISPACK, 138
 - Elemento pivote en eliminación de Gauss, 9
 - ver también Pivote
 - Elementos de relleno, *fill-in*, 219
 - Eliminación,
 - digrafo de, 250
 - e interpretación grafo-teórica de la eliminación de Gauss en matrices dispersas de estructura no simétrica, 250
 - grafo de, 231
 - e interpretación grafo-teórica de la eliminación de Gauss en matrices dispersas de estructura simétrica, 231
 - Eliminación directa, método para resolución de MCI, 132
 - Eliminación de Gauss, 7–23
 - en matrices dispersas, 226–261
 - de estructura simétrica, 226–246
 - de estructura no simétrica, 246–261
 - y teoría de grafos, 231, 250
 - matriz de transformación, 10
 - multiplicadores, 9
 - número de operaciones, 20–23
 - pivotación, 14–20
 - parcial, 15
 - total, 15
 - y factorización *LU*, 24
 - Emisión de deuda, problema, 369
 - Endomorfismo, 678
 - Entorno, de un punto, 691
 - Enumerativo, *branch and bound*, algoritmo de ramificación y acotamiento, 649
 - Envoltura convexa, 391, 625
 - Envolvente de una matriz, 205
 - simétrica, 206
 - Epsilon* de una máquina, precisión, 704
 - Errores numéricos,
 - de cancelación, 267, 707
 - de la representación de un número en ordenador, 703
 - absoluto, 703
 - relativo, 703
 - de redondeo, 703
 - Escalar(es), 673
 - Escalado afín, para programación lineal, 571
 - Escalado proyectivo de Karmarkar, para programación lineal, 561
 - Espacio vectorial, 673
 - base, 674
 - dimensión, 674
 - dual, 679
 - métrico, 677
 - normado, 675
 - completo, 677
 - de Banach, 677
 - de Hilbert, 678
 - euclídeo, 678
 - prehilbertiano, 678
 - subespacio, 674
 - Espectral, norma de una matriz, 682
 - Espectro de una matriz, 152, 685
 - Esquema iterativo,
 - de descenso para solución de $Ax = b$, 170
 - dirección, 170
 - factor de avance o amplitud de paso, 171
 - de Gauss-Seidel,
 - para mínimos cuadrados, 195
 - para sistemas de ecuaciones no lineales, 318
 - para solución de $Ax = b$, 150
 - de Jacobi,
 - para mínimos cuadrados, 194
 - para sistemas de ecuaciones no lineales, 316
 - para solución de $Ax = b$, 146
 - de relajación SOR,
 - para mínimos cuadrados, 195
 - para sistemas de ecuaciones no lineales, 319
 - para solución de $Ax = b$, 164
 - de Richardson, 144
 - de SGS (Symmetric Gauss-Seidel), para solución de $Ax = b$, 169
 - de SSOR, para solución de $Ax = b$, 169
 - Estimación del estado, de sistemas eléctricos, 336
 - Estimador,
 - de mínimos cuadrados, 341
 - de máxima verosimilitud, 341
 - de norma l_1 , 376
 - Estructuras de datos de programación lineal para flujos en redes, 512
 - Estructura de niveles,
 - de un digrafo, 250
 - de un grafo general, 230
 - Estudio de cargas, 279
 - Euclídeo, espacio vectorial, 678
 - Excentricidad, de un nudo en un grafo, 229
- F**
- Faceta, de un politopo, 627
 - Factorización

- de Cholesky para matrices simétricas definidas positivas, 41–47
- de Cholesky con pivotación para matrices simétricas semidefinidas positivas, 49
- LBL^T , matrices simétricas indefinidas, 58–66
 - Bunch y Kaufman, 60
 - Bunch y Parlett, 59
- LDL^T , matrices simétricas, 40–41
- LTL^T , matrices simétricas indefinidas, 50–58
 - Aasen, 53–58
 - Parlett y Reid, 50
- LU , 24–39
 - condiciones de existencia, 26
 - de la base en *simplex*, 447
 - métodos directos para su obtención, 29–39
 - unicidad, 28
 - y eliminación de Gauss, 31
- LU_1 , 29
 - método de Crout, 29
- L_1U , 34
 - método de Crout, 34
 - método de Doolittle, 36–39
- matrices simétricas, 39–66
 - definidas positivas, Cholesky, 41–47
 - indefinidas, 50
 - LDL^T , 40
 - semidefinidas positivas, 47
- QR , 88–115
 - por transformaciones ortogonales de Householder, 90–105
 - por transformaciones ortogonales de Givens, 105–110
 - por transformaciones ortogonales rápidas de Givens, 110–115
- Familia libre, en espacio vectorial, 674
- Farkas, lema, 471
- Fase I y II, del método *simplex*, 432
- Fill-in*, 219
 - ver también Elementos de relleno
- Filas/columnas, forma de almacenar matrices dispersas en un ordenador, 203
- Flujo de cargas, ver estudios de cargas
- Flujos en redes, problemas lineales de, 501
- Forma,
 - bilineal, 678
 - cuadrática, 170, 687
 - rango, 687
 - signatura, 687
 - hermítica, 678
 - lineal, 678
 - sesquilineal, 678
- Forma simétrica de la dualidad, 473
- Forma canónica, de las condiciones de un programa lineal, 392
- Forma estándar, de un programa lineal, 367
- Formato,
 - entero, 699
 - coma flotante, o punto flotante, 699
- Fórmula de Broyden, 321
- Fórmula de Sherman-Morrison-Woodbury, 330
- Fórmulas de Cramer, 22
- FRECUENCIA REINVERSION, 772
- Frentes, método, matrices dispersas, 263
- Frontera, de un conjunto, 691
- FTRAN, operación del *simplex*, 446
- Fuertemente conexo, digrafo, 249
 - componente, 249
- Función, 672
 - barrera logarítmica, métodos para programación lineal, 586–595
 - continua, 693
 - matriz Hessiana de, 694
 - matriz Jacobiana de, 694
 - no decreciente, 631
 - objetivo, de un programa lineal, 366
 - superaditiva, 631
 - vector gradiente de, 694
- G**
- Gauss, Carl Friedrich, 7
- Gauss-Jordan, método de, 23
 - matriz de transformación, 23
 - número de operaciones, 23
- Gauss-Newton, método de, 345–352
 - convergencia, 350
- Gauss-Seidel, método iterativo,
 - para mínimos cuadrados, 195
 - para solución de $A\mathbf{x} = \mathbf{b}$, 149–163
 - esquema iterativo, 150
 - matriz, 150
 - relación de recurrencia, 150
 - variante de Newton-Raphson para solución de sistemas de ecuaciones no lineales, 318
- Generación de desigualdades, en programación entera, 627–632
 - desigualdades disyuntivas, 630
 - desigualdades superaditivas, 631
 - redondeo entero, 627
- George y Heath, método de transformaciones ortogonales en matrices dispersas, 269–272
- Gerschgorin, teorema, 691

- Gestión de un servicio hospitalario, problema, 615
 $G^T G$, 41
Gilmore y Gomory, y el problema del corte de materiales, 546
Givens, matriz de, 105
Givens, transformaciones de, 105
 y factorización QR , 105–110
 número de operaciones, 109
 y resolución numérica de mínimos cuadrados lineales, 105–110
 número de operaciones, 109
 y su utilización con matrices dispersas, 109
transformaciones rápidas de, 110–115
 resolución numérica de mínimos cuadrados lineales, 110–115
 número de operaciones, 112
Golub-Kahan, método de, 121
Golub-Reinsch, método de, 115–132
 número de operaciones, 127
Gomory,
 algoritmo de los planos cortantes, 643
 corte fraccionario, 641
Gradientes conjugados, método iterativo para solución de $A\mathbf{x} = \mathbf{b}$, 179–192
 con preconditionamiento, 190–192
 convergencia, 183
 implementación práctica, 188
 relación de recurrencia, 180
 y mínimos cuadrados, 196
Grado, de un nudo de un grafo, 229, 500
 de entrada de un nudo de un digrafo, 248
 de salida de un nudo de un digrafo, 248
Grado mínimo, algoritmo para la reducción del número de elementos de relleno en una matriz dispersa, 234
Grafo, 227, 248, 500
 árbol, 230, 500
 cociente, 230
 enraizado, 230
 maximal, 231
 partición, 230
 asociado a una matriz de estructura simétrica, 227
 componentes conexos, 230
 conexo, 230
 conjunto accesible, 230
 conjunto separador, 230
 de eliminación, 231, 250
 diámetro, 229
 dirigido, 227, 248, 500
 estructura de niveles, 230
 inconexo, 230
 numerado, 227, 248
 partición, 230
 separador de un grafo, 230
 mínimo, 230
 teoría de y eliminación de Gauss, 227, 248
Gram-Schmidt, método para la ortonormalización de los vectores columna de una matriz, 83–88
 clásico, 84
 modificado, 86
 número de operaciones, 88
 y solución del problema lineal de mínimos cuadrados, 87
- H**
 h , parámetro para el cálculo de derivadas mediante diferencias finitas, 313
Hall,
 algoritmo de, 251
 propiedad fuerte de, 273
Hanson y Lawson, método de, 117
Harwell Subroutine Library, 225, 262
Hebra, *depth*, estructura de datos para programación lineal para flujos en redes, 512
Hermítica, forma, 678
Hessemberg, matriz, 54
Hessiana, matriz, de una función, 694
Hijo, nudo, 230, 645
Hilbert, espacio vectorial, 678
 matriz, 882
Hiperplano, 389
 separador, 471, 697
 teorema de existencia, 696
 soporte, 391, 697
 teorema de existencia, 697
 vector característico, 389
Hölder, normas p , 675
Homomorfismo, 678
Householder, matriz, 90
Householder, transformaciones, 90
 factorización QR , 90–105
 número de operaciones, 97
 resolución numérica de mínimos cuadrados lineales, 90–105
 número de operaciones, 97
- I**
Imagen,
 de una aplicación, 672

de una matriz, 681
 subespacio, 679
 Incidencia nudo-arco, matriz de, 500
 Independencia lineal, vectores en espacio vectorial, 674
 Infimo, o cota inferior máxima, 672
 INT, 768
 Interior, de un conjunto, 691
 punto, 691
 Interpretación geométrica
 de la factibilidad y de la búsqueda de óptimo de un programa lineal, 379–388
 de la factibilidad de un programa lineal con condiciones de desigualdad, 387
 de la factibilidad de un programa lineal con condiciones de igualdad, 385
 en el subespacio de bienes, 383
 sistemas lineales de ecuaciones, 7
 en $\mathbb{R}^2$, 7
 en $\mathbb{R}^n$, 7
 en $\text{Im}(A)$, 8
 Interpretación grafo-teórica de la eliminación de Gauss de matrices dispersas, simétricas, 231
 no simétricas, 250
 Intersección, de conjuntos, 671
 INVERT, operación del *simplex*, 447
 Inyectiva, aplicación, 672
 Isométrico, operador, 679

J

Jacobi, método iterativo
 para mínimos cuadrados, 194
 para sistemas de ecuaciones no lineales, 316
 para solución de $A\mathbf{x} = \mathbf{b}$, 145–149
 esquema iterativo matricial, 146
 matriz, 146
 relación de recurrencia, 146
 variante de Newton-Raphson para solución de sistemas de ecuaciones no lineales, 316
 Jacobiana, matriz, de una función vectorial, 694

K

Karmarkar, método de escalado proyectivo para programación lineal, 561
 Karush-Kuhn-Tucker, condiciones en programación lineal, 476
 Kuhn-Tucker, condiciones en programación lineal, 476

L

L , límite *underflow* de un ordenador, 700
 Lagrangiana, relajación, 632
 Lagrange,
 multiplicadores, 415, 476
 LAPACK, 138
 LBL^T , 58
 LDL^T , 40
 Lexicográfica, regla para el método *simplex*, 429
 Levenberg-Marquardt, método de, 352–357
 Libre, familia, en espacio vectorial, 674
 LIFO, regla en algoritmos *branch and bound*, 652
 Límite, de una sucesión, 677
 LIMITE ITERACIONES, 772
 Lineal, forma, 678
Linesearch, 171
 LINPACK, 138
 Lipschitz,
 condición de, 693
 función continua, 693
 Listas encadenadas, forma de almacenar matrices dispersas en un ordenador, 207
 Llegada,
 de un arco de un digrafo, 248
 de un arco de un grafo, 229
 Localización de almacenes, problema, 618
 Longitud, de un camino de un grafo, 229
 LTL^T , 50
 LU , 24
 LU_1 , 29
 L_1U , 34

M

M, gran, método para llegar a solución básica factible inicial en *simplex*, 441
 MA17, rutina del paquete de software Harwell Subroutine Library, 225
 MA28, rutina del paquete de software Harwell Subroutine Library, 225
 MA32, rutina del paquete de software Harwell Subroutine Library, 225
 MA37, rutina del paquete de software Harwell Subroutine Library, 225
 Maestro, problema, en descomposición de Dantzig-Wolfe, 532
 Markowitz, criterio, 261
 Matriz, 679
 ancho de banda, 205
 matriz simétrica, 206
 aumentada, 9
 B , o básica de un programa lineal, 392
 básica, de un programa lineal, 392

- birreducible, 247
 - congruente ortogonal, 686
 - definida negativa, 691
 - definida positiva, 41, 689
 - de coeficientes de un sistemas de ecuaciones lineales, 4
 - de covarianzas, 341
 - de curvatura, 345
 - de diagonal dominante, 156, 691
 - de Gauss, 10
 - de Givens, 105
 - de Hessemberg, 54
 - de Hilbert, 882
 - de Householder, 90
 - de incidencia nudo-arco de un digrafo, 500
 - de Jacobi, 146
 - de permutación, 684
 - de proyección, 684
 - de proyección ortogonal, 684
 - de transformación de Gauss, 10
 - de transformación de Gauss-Jordan, 23
 - de Vandermonde, 74
 - dispersa, 144, 202
 - almacenamiento en ordenador, 202–208
 - por coordenadas, 202
 - por filas/columnas, 203
 - por listas encadenadas, 207
 - por perfil o envolvente, 204
 - operaciones algebraicas, 208–219
 - multiplicación AB , 215
 - multiplicación $A^T A$, 217
 - multiplicación por vector, 210
 - suma, 211
 - numérica, 215
 - simbólica, 211
 - paquetes software,
 - Harwell Surrountine Library, 225, 262 268
 - NAG, 225, 268
 - SMMS, 225, 268
 - SPARSKIT, 274
 - SPARSPAK, 225, 263, 268
 - YSMP, 225, 263, 268
 - ecuación característica, 685
 - en banda, 204
 - ancho de banda, 205
 - matriz simétrica, 206
 - envolvente de, 205
 - matriz simétrica, 206
 - espectro, 152, 685
 - eta, en método *simplex*, 445
 - Hessiana, de una función, 694
 - imagen, 681
 - indefinida, 691
 - inversa generalizada Moore-Penrose, 77
 - Jacobiana, de una función vectorial, 694
 - N , o no básica de un programa lineal, 392
 - no básica de un programa lineal, 392
 - núcleo, 681
 - número de condición, 69
 - matriz invertible, 69
 - matriz cualquiera, 69
 - ortogonal, 683
 - pseudoinversa, 69, 687
 - radio espectral, 152, 685
 - rango, 681
 - completo, 681
 - reducible, 247
 - regular, 681
 - semejante a otra, 686
 - semidefinida negativa, 689
 - semidefinida positiva, 41, 689
 - simbólicamente singular, 251
 - simétrica, 683
 - ancho de banda, 206
 - envolvente, 206
 - singular, 681
 - totalmente unimodular, 503
 - triangular inferior unitaria, 10
- Máxima pendiente, método de 174
- MAXIMIZAR, 772
- MCDQ, mínimos cuadrados con restricciones cuadráticas de desigualdad, 131
- MCI, mínimos cuadrados con restricciones lineales de igualdad, 131
- resolución numérica, 132–138
- MCLD, mínimos cuadrados con restricciones lineales de desigualdad, 131
- Menores, números de una matriz, 687
- Método(s), ver también Algoritmo
- Aasen, 53–58
- Broyden, para solución de sistemas de ecuaciones no lineales, 321
- convergencia, 326
- implementación práctica, 330
- Bunch y Kaufman, 60
- Bunch y Parlett, 59
- Crout, 29–36
- versión LU_1 , 29–34
- con pivotación parcial, 32
- versión L_1U , 34–36
- directos, para solución de sistemas lineales de ecuaciones, por medio de factorización,

- 24
- Aasen, 53–58
- basados en transformaciones ortogonales,
Givens, 105–110
Householder, 90–105
rápidas de Givens, 110–115
- Bunch y Kaufman, 60
- Bunch y Parlett, 59
- Crout, 29–36
- Doolittle, 36–39
- eliminación de Gauss, 7–23
- Gauss-Jordan, 23
- Gram-Schmidt, 83–88
- Disección,
anidada, 244
en un sentido, 245
- Doolittle, versión L_1U , 36–39
- eliminación de Gauss, 7–23
con pivotación parcial, 14–20
- escalado afín para programación lineal, 571
- escalado proyectivo de Karmarkar para
programación lineal, 561
- falsa posición, *regula falsi*, para solución de
sistemas de ecuaciones en una variable,
303
- frentes, 263
- función barrera logarítmica, para programación
lineal, 586–595
- Gauss-Jordan, 23
- Gauss-Newton, método de, 345–352
convergencia, 350
- George y Heath, 269–272
- Golub-Kahan, 121
- Golub-Reinsch, 115–132
- Gradientes conjugados, 179–192
con preconditionamiento, 190–192
implementación práctica, 188
y mínimos cuadrados, 196
- Gram-Schmidt, 83–88
clásico, 84
modificado, 86
- gran M, para solución básica factible inicial en
simplex, 441
- iterativos,
de descenso para solución de $Ax = b$, 170
dirección, 170
factor de avance o amplitud de paso, 170
- Gauss-Seidel,
en mínimos cuadrados, 195
en solución de $Ax = b$, 149–163
esquema iterativo, 150
- matriz, 150
relación de recurrencia, 150
variante de Newton-Raphson para solución
de sistemas de ecuaciones no lineales,
318
- Jacobi,
en mínimos cuadrados, 194
en sistemas de ecuaciones no lineales, 316
en solución de $Ax = b$, 145–149
esquema iterativo matricial, 146
matriz, 146
relación de recurrencia, 146
variante de Newton-Raphson para solución
de sistemas de ecuaciones no lineales,
316
- Jacobi y Gauss-Seidel, convergencia, 152–163
matriz de diagonal dominante, 156
matriz generales, 152
matriz simétrica definida positiva, 159
- Relajación, SOR,
para mínimos cuadrados, 195
para sistemas de ecuaciones no lineales,
319
para solución de $Ax = b$, 163–168
esquema iterativo matricial, 164
matriz, 164
relación de recurrencia, 164
variante de Newton-Raphson para solución
de sistemas de ecuaciones no lineales,
319
- Relajación, SSOR,
para solución de $Ax = b$, 168
esquema iterativo matricial, 169
relación de recurrencia, 169
- SGS (Symmetric Gauss-Seidel), 169
esquema iterativo matricial, 169
- Richardson, 144
- solución de sistemas de ecuaciones lineales,
esquema iterativo tipo, 144
de minimización, 170
direcciones conjugadas, 177
gradientes conjugados, 179–192
con preconditionamiento, 190–192
dirección de descenso, 170
máxima pendiente, 173
relajación univariante, 171
relajación SOR, 173
- iterativos estacionarios, 144
- iterativos y mínimos cuadrados, 194
Gauss-Seidel, 195
Jacobi, 194

- Relajación SOR, 195
- Gradientes conjugados, 196
- multifrentes, 266
- Newton, ver Newton-Raphson
- Newton-Raphson, para solución de ecuaciones
 - no lineales en una variable, 286–300
 - convergencia, 291
 - relación de recurrencia, 289
 - variantes, 294–300
 - de la secante, 300
 - diferencias finitas, 295
 - Newton modificado, 299
 - regula falsi*, 303
 - y mecanismo de salvaguarda, 294
- Newton-Raphson, para solución de sistemas de ecuaciones no lineales, 306–320
 - convergencia, 310
 - relación de recurrencia, 306
 - variantes, 313–320
 - diferencias finitas, 313
 - Gauss-Seidel, 318
 - Jacobi, 316
 - Newton modificado, 316
 - relajación SOR, 318
- Parlett y Reid, 50
- primal–dual, en programación lineal, 487
- punto interior, métodos en programación lineal, 557
- Richardson, 144
- simplex*, 411
 - factorización *LU* de la base, *B*, 447
 - fase I y II, 441
 - forma de tableau, 441
 - forma producto de la inversa de la base, 445
 - para problemas de flujos en redes, 505
 - para variables acotadas, 450
 - solución básica factible inicial, 429
 - variables artificiales, 431
- secante, para solución de sistemas de ecuaciones no lineales en una variable, 300
- Métrico, espacio vectorial, 677
- Minimización, métodos iterativos para solución de sistemas de ecuaciones lineales, 170–194
 - direcciones conjugadas, 177
 - gradientes conjugados, 179–192
 - con preconditionamiento, 190–192
 - dirección de descenso, 170
 - máxima pendiente, 171
 - relajación univariante, 171
 - relajación SOR, 173
- Mínimos cuadrados lineales, 73
 - comparación número de operaciones diversos métodos, 129
 - con matrices dispersas, 266
 - ecuaciones normales, 266
 - transformaciones ortogonales Givens de George y Heath, 269–272
 - con restricciones lineales, 131
 - resolución numérica, 132
- resolución numérica, 81
 - descomposición en valores singulares, 115
 - Gram-Schmidt, 83–88
 - transformaciones de Givens, 105–110
 - transformaciones de Householder, 90–105
 - transformaciones rápidas de Givens, 110–115
- sistemas incompatibles, ecuaciones normales, 79
- sistemas indeterminados, 81
- y ajuste de curvas, 73
- Mínimos cuadrados no lineales, 335–358
 - resolución numérica, métodos, 345
 - de Gauss-Newton, 345–352
 - convergencia, 350
 - de región de confianza, 353
 - Levenberg-Marquardt, 352–357
 - Newton, 358
- Mínimos cuadrados ponderado, 129
- Mínimo relativo, de una función, 695
- Mínimo relativo estricto, de una función, 695
- Mochila, problema, 547, 616
- Modelización con variables binarias, en programación entera, 622
- Modelo matemático, 3
- Monótona, numeración de un árbol, 230
- Moore-Penrose, matriz inversa generalizada, 77
- MPS, formato, 765
- Multifrentes, método, 266
- Multiplicaciones/divisiones, de un método o lagoritmo, 20
- Multiplicadores, en eliminación de Gauss, 9
- Multiplicadores *simplex*, 415
 - algoritmo para obtención, en programación lineal para flujos en redes, 514
 - y multiplicadores de Lagrange, 415, 476
- N**
- N**, conjunto de los números naturales, 671
- N*, matriz no básica de un programa lineal, 392
- NAG, paquete de *software*, 225, 268
- NAME, 766
- Nested Dissection*, ver Disección anidada

-
- Newton, Isaac 288
 - teorema, 291
 - ver también Newton-Raphson
 - Newton modificado,
 - para solución de ecuaciones no lineales en una variable, 295
 - para solución de ecuaciones no lineales en más de una variable, 316
 - Newton-Raphson, para solución de ecuaciones no lineales en una variable, 286–300
 - convergencia, 291
 - relación de recurrencia, 289
 - variantes, 294–300
 - de la secante, 300
 - diferencias finitas, 295
 - Newton modificado, 299
 - regula falsi*, 303
 - y mecanismo de salvaguarda, 294
 - Newton-Raphson, para solución de sistemas de ecuaciones no lineales, 306–320
 - convergencia, 310
 - relación de recurrencia, 306
 - variantes, 313–320
 - diferencias finitas, 313
 - Gauss-Seidel, 318
 - Jacobi, 316
 - Newton modificado, 316
 - relajación SOR, 318
 - Niveles, estructura de, en un grafo, 230
 - Norma,
 - matricial, 681
 - consistente, 681
 - de Frobenius, 681
 - espectral, 682
 - inducida, 682
 - vectorial, 675
 - euclídea, 675
 - p de Hölder, 675
 - Núcleo,
 - de una aplicación, 679
 - de una matriz, 681
 - Nudo(s), de un grafo, de un digrafo, 227, 248, 500
 - accesibilidad, 230
 - adyacentes, 229, 248
 - compuesto, 256
 - de demanda, en un digrafo, 248, 501
 - de demanda, en flujos en redes, 501
 - de holgura, 279
 - ver también Estudio de Cargas
 - de oferta en un digrafo, 248, 501
 - de oferta, en flujos en redes, 501
 - de llegada de un camino, 229
 - de llegada de un arco en un digrafo, 248
 - de partida de un camino, 229
 - de transbordo, en flujos en redes, 501
 - destino de un arco, 227, 248, 500
 - distancia entre, 229, 248
 - excentricidad, 229
 - fuelle, en flujos en redes, 501
 - grado, 229
 - hijo, 230, 645
 - origen de un arco, 227, 248, 500
 - padre, 230, 645
 - periférico, 229
 - raíz, de un árbol, 230, 506
 - sumidero, en flujos en redes, 501
 - Numerable, conjunto, 672
 - Numeración, monótona de un árbol, 230
 - Número de condición de una matriz, 69
 - invertible, 69
 - cualquiera, 69
 - Número de dígitos significativos, de una máquina, 705
 - Número de operaciones del método o algoritmo,
 - Aasen, 53
 - Bunch y Parlett, 59
 - Bunch y Kaufman, 60
 - Cholesky, 47
 - Crout, 34
 - Doolittle, 36
 - eliminación de Gauss, 20–23
 - fórmulas de Cramer, 22
 - factorización LDL^T de una matriz simétrica, 40
 - factorización QR por transformaciones de Householder, 97
 - factorización QR por transformaciones de Givens, 109
 - factorización QR por transformaciones rápidas de Givens, 112
 - Gauss-Jordan, 23
 - Gram-Schmidt,
 - clásico, 84
 - modificado, 86
 - Parlett y Reid, 50
 - resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$,
 - mediante transformaciones ortogonales de Householder, 90
 - mediante transformaciones ortogonales de Givens, 109
 - mediante transformaciones ortogonales rápidas de Givens, 112

Números, representación en ordenador, 700
exponente, 700
mantisa, 700

O

Operaciones aritméticas de un método, 20
ver también multiplicaciones/divisiones,
sumas/restas

Operador,
adjunto, 679
autoadjunto, 679
hermítico, 679
isométrico, 679
lineal 678
simétrico, 679
unitario, 679

Optimización, 365
en redes, problemas lineales, 499

Orden de convergencia, 281

Ordenación de las ecuaciones, matrices dispersas,
219

Ordenador,
dígitos significativos, 705
epsilon, 704
errores, 703
de cancelación, 267, 707
de redondeo, 703
de truncamiento, 703
precisión, 704
representación de números, 700

Origen, nudo origen de un arco, 227, 248, 500

Ortogonal(es),
matriz, 683
subespacio, 678
vectores, 678

Ortonormales, vectores, 678

Overflow, límite de, en un ordenador, 700

P

p^3 y p^4 , en método *simplex*, 447

Padre, nudo, 230, 645

Paige, método de, 129
número de operaciones, 130

Parte generadora, de un espacio vectorial, 674

Partición, de la región factible de un programa
entero, 645

Partición, de un grafo, 230
árbol, 230

Parlett y Reid, método de, 50
número de operaciones, 52

Penalización, o gran M, método para llegar a
solución básica factible inicial en
simplex, 441

Penalizaciones, en programación entera, 654

Perfil, de una matriz, 204
monótono, 238
perfil o envolvente, forma de almacenar
matrices dispersas en un ordenador, 204

Periférico, nudo de un grafo, 229

Permutación, matriz de, 683

Permutación simétrica, 49, 229

Pivote, 9

Pivotación, 15
operación en método *simplex* revisado, 420, 454
en el algoritmo de descomposición de
Dantzig-Wolfe, 538
en método *simplex* para programación lineal de
flujos en redes, 516
parcial, 15
total, 15

Plan de fabricación, problema, 369

Planificación de la generación de energía en una
empresa eléctrica, problema, 370, 616

Polihedro, 390

Politopo(s), 390
cónico, 390
racional, 625
y región factible de un programa lineal, 390

Ponderación, método de la, 138

Ponderado, mínimos cuadrados, 129

Post-optimización, análisis en programación
lineal, 492

Potencial, de nudos en programación lineal para
flujos en redes, 514

Precios,
de equilibrio, 477
sombra, 477

Precisión de la máquina, u ordenador, 704
ver *epsilon*

Precondicionador, 191
gradientes conjugados con
precondicionamiento, 190–192

Predecesor, en estructura de datos para
programación lineal de flujos en redes,
512

Prehilbertiano, espacio vectorial, 678

Preorden, en implementación del método *simplex*
para flujos en redes, 512, 518

Prestaciones de algoritmos, criterios, 20
y errores de redondeo, 20
y tiempo de cálculo, 20

- Price-out*, ver Asignación de precio
- Principio de descomposición de Dantzig-Wolfe, 527–545
- Problema,
- del corte de materiales, 546
 - del plan de fabricación, 369
 - del representante de comercio, 620
 - del transporte, 370
 - de la asignación de tráfico, 375
 - de la dieta alimenticia, 368
 - de la dicotomía, 617
 - de la emisión de deuda, 369
 - de la gestión de un servicio hospitalario, 615
 - de la localización de almacenes, 618
 - de la mochila, 547, 616
 - de la planificación de la generación de energía en una empresa eléctrica, 370, 616
 - de los mínimos cuadrados ponderado, 129
- Problema generalizado de mínimos cuadrados, 129
- Problema lineal de mínimos cuadrados, 73
- descomposición en valores singulares y solución, 75
 - formulación, 73
 - resolución numérica, 81
- Problema maestro, en descomposición de Dantzig-Wolfe, 532
- Problema no lineal de mínimos cuadrados, 335
- formulación, 335
 - resolución numérica, 345
 - método de Gauss-Newton, 345–352
 - convergencia, 350
 - métodos de región de confianza, 353
 - Levenberg-Marquardt, 352–357
 - Newton, 358
- Problema primal restringido, 487
- ver también método primal–dual
- Problemas con estructura en escalera, 545
- Problemas de flujos en redes, 499
- asignación, 505
 - camino más corto, 503
 - coste mínimo, 502
 - flujo máximo, 504
 - transbordo, 502
- PROCESO ITERATIVO, 772
- Producto de la inversa de la base, forma del algoritmo *simplex* para programas lineales, 446
- Producto escalar, 678
- Profundidad, *depth*, estructura de datos para programación lineal para flujos en redes, 512
- Programa combinatorio, 614
- Programa dual, de uno lineal, 468
- Programa entero, 614
- combinatorio, 614
 - de la gestión de un servicio hospitalario, 615
 - de la localización de almacenes, 618
 - de la mochila, 547, 616
 - del representante de comercio, 620
 - dicotomías, 617
 - mixto, 614
 - puro, 614
 - región factible, 625
 - set covering*, 622
 - set partitioning*, 622
 - simple matching*, 622
- Programa(s) en FORTRAN 77, C y FORTRAN 90,
- Aasen**, factorización LTL^T de una matriz simétrica indefinida, 57, 834, 860
 - Bbmi**, solución de problemas de programación lineal, entera pura y mixta, 791
 - Beta-y-t**, cálculo de los parámetros β y t de una máquina, 701, 856
 - Bisec**, solución de ecuaciones no lineales en una variable, 285, 847, 868
 - Broyden**, solución de sistemas de ecuaciones no lineales mediante método cuasi Newton, 323, 852, 872
 - Condest**, estimación del número de condición 1 de una matriz cuadrada mediante el algoritmo de Cline, Moler, Stewart y Wilkinson, 882, 856, 876
 - CondHag**, estimación del número de condición 1 de una matriz cuadrada mediante el algoritmo de Hager, 886
 - Bunchkauf**, factorización LBL^T de una matriz simétrica indefinida, 66, 835, 860
 - Ccnet**, solución de problemas de optimización de flujos en redes, 703
 - Cg**, gradientes conjugados para solución iterativa de $Ax = b$, 188, 841, 867
 - Cgp**, gradientes conjugados con preconditionamiento para solución iterativa de $Ax = b$, 192, 841, 867
 - Chol**, factorización de Cholesky, 45, 834, 859
 - dcmsvd**, descomposición en valores singulares de una matriz cualquiera mediante Golub-Reinsch, rutina de **Svdre**.
 - eliminación de Gauss con pivotación parcial para solución de $Ax = b$,
 - Gauss**, un solo término independiente, 15, 832, 858

- Gaussc, cualquier término independiente, 19, 832, 858
- Epsmaq, cálculo de la precisión de una máquina, 705, 856
- Fastgivens, resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$ mediante transformaciones ortogonales rápidas de Givens, 112, 838, 863
- GaussSeidel, para solución de $\mathbf{Ax} = \mathbf{b}$ iterativamente, 151, 840, 866
- Gausnewt, para resolver problemas no lineales de mínimos cuadrados, 347, 854, 874
- factorización LU_1 ,
 - Crout, método de Crout, 30, 833, 858
 - Croutp, método de Crout con pivotación parcial, 33, 833, 858
- factorización L_1U ,
 - Crout11u, método de Crout, 36, 833, 859
 - Dool, método de Doolittle, 38, 834, 859
- Givens, resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$ mediante transformaciones ortogonales de Givens, 107, 837, 863
- Grmsch, resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$ mediante Gram-Schmidt modificado, 87, 836, 861
- Jacobi, para solución de $\mathbf{Ax} = \mathbf{b}$ iterativamente, 147, 840, 866
- Levmar, para resolver problemas no lineales de mínimos cuadrados, 353, 855, 874
- Mci, resolución del problema MCI (mínimos cuadrados con restricciones lineales de igualdad), 134, 839, 865
- Mincuat, resolución de $\mathbf{Ax} = \mathbf{b}$, $A^{m \times n}$, $m > n$ ó $m < n$ y rango incompleto, 102, 836, 862
- Muller, resolución de $x^3 - \text{sen } x = 0$ mediante el método de Muller, 305, 848, 869
- Newt, solución de $x^3 - \text{sen } x = 0$ mediante Newton-Raphson, 290, 847, 868
- Newtarmijo, solución de sistemas de ecuaciones no lineales mediante Newton-Raphson con el mecanismo de salvaguarda de Armijo, 333
- Newjac, solución de sistemas de ecuaciones no lineales por Newton-Raphson, variante de Jacobi, 317, 850, 871
- Newton, solución de $x^2 - 1 = 0$ mediante Newton-Raphson, 298, 847, 868
- Newtondf, solución de $x^2 - 1 = 0$ mediante Newton-Raphson por diferencias finitas, 298, 847, 868
- Newtonmod, solución de $x^3 - \text{sen } x = 0$ mediante el método de Newton modificado, 299, 848, 869
- Newtonsecante, solución de $x^3 - \text{sen } x = 0$ mediante el método de la secante, 301, 848, 869
- Newtrp, solución de sistemas de ecuaciones no lineales por Newton-Raphson, 307, 848, 869
- Newtrpdf, solución de sistemas de ecuaciones no lineales por Newton-Raphson con diferencias finitas, 314, 849, 870
- Newsor, solución de sistemas de ecuaciones no lineales por Newton-Raphson, variante de relajación, 319, 851, 871
- Qrdes, resolución de $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$ mediante transformaciones ortogonales de Householder, 96, 836, 862
- Sor, para solución de $\mathbf{Ax} = \mathbf{b}$ iterativamente, 165, 841, 866
- Steep, para solución de $\mathbf{Ax} = \mathbf{b}$ iterativamente mediante el método de la máxima pendiente, 176, 867
- Suma-de-serie, para sumar una serie de infinitos sumandos, 710, 856
- Svdre, resolución de $\mathbf{Ax} = \mathbf{b}$, $A^{m \times n}$, $m > n$ ó $m < n$ y rango incompleto mediante descomposición de A en valores singulares, 128, 838, 864
- Programa lineal, 366
 - coeficientes de coste, 366
 - dual, 468
 - forma estándar, 367
 - formulación de Karmarkar, 561
 - función objetivo, 366
 - inconsistente, o no factible, 382
 - punto factible, 366
 - primal restringido, 487
 - región factible, 366
 - restricciones o condiciones, 366
 - solución óptima, 381
 - alternativas, 381
 - no acotada, 381
 - única, 381
 - término de la derecha, 366
 - variables de decisión, 366
 - variables de holgura, 367
- Programación matemática, 365
 - dinámica, 547
 - entera, 613
 - lineal, 365
 - paramétrica, 492

Propiedad fuerte de Hall, 273

Proyección,

matriz de, 684

ortogonal de un vector, 77, 684

matriz de, 78, 684

sobre un subespacio, 78

teorema de la, 692

Proyector suplementario, 684

Punto,

de acumulación, 691

de adherencia, 691

de equilibrio, 529

de silla, 345

extremo, de región factible de programa lineal, 384

de conjunto convexo, 384

y solución básica, 393

factible, de un programa lineal, 366

interior, 691

y programación lineal, 557

Q

Q, conjunto de los números racionales, 671

QR, 88

ver Factorización *QR*

R

$\mathbb{R}$, cuerpo de los números reales, 671, 672

Radio espectral de una matriz, 152, 685

Raíz, nudo de un árbol, 230, 506

RANGES, 769

Rango,

de una forma cuadrática, 687

de una matriz, 681

completo, 681

simbólico, de una matriz, 251

Rápidas de Givens, transformaciones, 110

Rápidez de convergencia, 281

Razón áurea, 301

Red, eléctrica de 30 nudos de IEEE, 337

Redes, flujos en, 501

optimización en, 499

Redondeo, en ordenador, 703

Reflexiones de Householder, 90

ver también Factorización *QR*,

transformaciones de Householder

Región de confianza, métodos, 353

Levenberg-Marquardt, 352–357

Región factible,

de un programa entero, 625

de un programa lineal, 366

vacía, 382

punto extremo, o vértice, 380

y politopo, 390

Regla,

de Bland, 429

del triángulo, 675

lexicográfica, 429

LIFO, 652

Regula falsi, 303

Relajación, SOR,

para mínimos cuadrados, 195

para sistemas de ecuaciones no lineales, 319

para solución de $Ax = b$, 163–168

esquema iterativo matricial, 164

matriz, 164

relación de recurrencia, 164

variante de Newton-Raphson para solución de sistemas de ecuaciones no lineales, 319

Relajación, SSOR,

para solución de $Ax = b$, 169

esquema iterativo matricial, 169

relación de recurrencia, 169

SGS (Symmetric Gauss-Seidel), 169

esquema iterativo matricial, 169

Relajaciones, de un programa entero, 626

lineal, 626

Lagrangiana, 632

descomposición y separación de costes, 632

descomposición de Benders, 633

Relleno, elementos de, 219

Representación de números en ordenador, 700

exponente, 700

mantisa, 700

Representante de comercio, problema, 620

Restricciones, de un programa lineal, 366

RHS, 769

Richardson, esquema iterativo para resolución de sistemas de ecuaciones lineales, 144

ROWS, 767

E, 767

G, 767

L, 767

N, 767

S

Salida, grado de un nudo de un digrafo, 248

Sargent y Westerberg, algoritmo, 256

Schwarz, desigualdad de Cauchy-Schwarz, 678

Secante, método para solución de ecuaciones no lineales en una variable, 300

Semidefinida negativa,

- matriz, 689
- Semidefinida positiva,
 - matriz, 689
- Semiespacio,
 - abierto, 389
 - cerrado, 389
- Sensibilidad, análisis en programación lineal, 492–493
- Separación y acotación, algoritmos enumerativos o *branch and bound*, 649
- Separador,
 - conjunto, de un grafo, 230
 - hiperplano, 471, 697
 - mínimo, de un grafo, 230
- Serie geométrica, 154
- Sesquilineal, forma, 678
- Set covering*, 622
- Set partitioning*, 622
- Sherman-Morrison-Woodbury, fórmula de, 330
- Simplex*, método, 411
 - factorización *LU* de la base, *B*, 447
 - fase I y II, 432
 - forma de tableau, 441
 - forma producto de la inversa de la base, 445
 - para problemas de flujos en redes, 505
 - para variables acotadas, 450
 - solución básica factible inicial, 429
 - variables artificiales, 431
- Signatura, de una forma cuadrática, 687
- Simple matching*, 622
- Sistema de ecuaciones lineales, 4
 - coeficientes, 4
 - con matriz dispersa, 201
 - de estructura simétrica, 226–246
 - de estructura no simétrica, 246–261
 - interpretación geométrica,
 - en $\mathbb{R}^2$, 7
 - en $\mathbb{R}^n$, 7
 - en $\text{Im}(A)$, 8
 - matriz de coeficientes, 4
 - teorema de la compatibilidad, 4
 - términos independientes, 4
- Sistema de ecuaciones no lineales, 465
- Sistema de numeración en ordenador, 700
- Sistema lineal triangular superior, 7
- Software,
 - Harwell Subroutine Library, 225, 262
 - para matrices dispersas, 225
- Solución básica, de un programa lineal, 392
 - degenerada, 395
 - factible, 395
 - factible, 395
 - inicial para el método *simplex*, 429
 - óptima, condición, 418
 - y punto extremo, 393
- Solución de $A\mathbf{x} = \mathbf{b}$, y mínimo de forma cuadrática, 170
 - ver métodos concretos
- Solución óptima, de programa lineal, 418
 - alternativas, 381
 - no acotada, 381
 - única, 381
- SPARSKIT, 274
- SPARSPAK, 225, 263, 268
- SMMS, Sparse Matrix Manipulation System, 263
- Subconjunto, 671
 - abierto, 691
 - cerrado, 691
 - compacto, 691
- Subespacio(s),
 - de actividad, en programación lineal, 383
 - de Krylov, 184
 - de productos, en programación lineal, 383
 - fundammatriz *A*, 78
 - imagen, de una aplicación, 679
 - ortogonal, 678
 - propio, 685
 - requisito, en programación lineal, 383
 - suplementarios, 674
 - vectorial, 674
- Subgrafo,
 - de un digrafo, 500
 - maximal, 500
 - de un grafo, 229
 - sección,
 - de un digrafo 249
 - de un grafo, 229
- Sucesión, 672
 - convergencia en un espacio vectorial normado, 677
 - de Cauchy, 677
 - de elementos de un conjunto, aplicación, 672
 - límite de, 677
- Sucesor, estructura de datos de programación lineal para flujos en redes, 512
- Suma directa, de dos subespacios vectoriales, 674
- Sumas/restas, de un método o lagoritmo, 20
- Sustitución inversa, 9
- Suprayectiva, aplicación, 672
- Supremo, o cota superior mínima, 672

T

t , precisión de un ordenador, 700
 Tableau, método *simplex* en forma de, 441
 Tasa, o relación de convergencia, 281
 Taylor, teorema de, 695
 desarrollo en serie de, 291
 Teorema
 de Gerschgorin, 691
 de la compatibilidad de un sistema de ecuaciones lineales, 4
 de la dualidad, en programación lineal, 470
 de las direcciones conjugadas, 178
 de la proyección, 692
 de Newton, 291
 de Taylor, 695
 de Weierstrass, 691
 fundamental de la programación lineal, 402
 Término de la derecha, de un programa lineal, 366
 Thread, estructura de datos para programación lineal para flujos en redes, 512
 Tiempo,
 de cálculo de un algoritmo, 20
 de multiplicación/división, 20
 de suma/resta, 20
 TOLERANCIA CERO, 772
 TOLERANCIA COSTES, 772
 TOLERANCIA PIVOTE, 772
 Transformaciones de Gauss, 10
 Transformaciones ortogonales, 88
 de Givens, 105
 de Householder, 90
 rápidas de Givens, 110
 Transformación proyectiva, Karmarkar, 562
 Transporte, problema, 370
 Transversal de una matriz dispersa, 247
 completo, 247
 búsqueda, 251–254
 Triangularización, de una base de un programa de flujos en redes, 510
 Triangularización en bloques de una matriz dispersa, 247
 algoritmo de Hall, 251
 algoritmo de Sargent y Westerberg, 256
 algoritmo de Tarjan, 260
 fases del proceso, 247
 Triangularización ortogonal, ver Factorización *QR*
 Triángulo, regla, 675
 Tronco de pivotación, en implementación del método *simplex* para flujos en redes, 518
 Truncamiento, en un ordenador, 703

U

U , límite de *overflow* de un ordenador, 700
Underflow, límite de, en un ordenador, 700
 Unimodular, matriz, 503
 Unión, de conjuntos, 671

V

Valores singulares, de una matriz, 75, 686
 descomposición en, 75
 numérica, 115
 Valor propio, 685
 Vandermonde, matriz de, 74
 Variables,
 artificiales, en el método *simplex*, 431
 básicas, 392
 binarias, 620
 de decisión, de un programa lineal, 366
 de holgura, de un programa lineal, 367
 de ramificación, en programación entera, 654
 penalizaciones, 654
 no básicas, 392
 Variedad lineal, 679
 ver también Hiperplano
 Vector(es), 672
 característico, de un hiperplano o variedad lineal, 389
 conjugados, 179
 de Householder, 90
 gradiente, 694
 linealmente dependientes, 674
 linealmente independientes, 674
 ortogonales, 678
 A ortogonales, 177
 ortonormales, 678
 propio, 685
 Velocidad de convergencia, 281
 Verosimilitud, 341
 estimador de máxima verosimilitud, 341
 Vértice,
 de un grafo, ver Nudo de un grafo
 de una región factible, ver Punto extremo

W

Weierstrass, teorema de, 691
 WRETA, operación del *simplex*, 446

Y

YSMP, Yale Sparse Matrix Package, 225, 263, 268

Z

$\mathbb{Z}$, conjunto de los números enteros, 671, 672